XCPC Template Library

Morning Flower
and Evening Oath

Tongji University

vector 算法模板与数学速查

建设稿 · 2026-09-12

状态说明

本稿包含首批已进行本地验证的模块与完整来源覆盖表的入口。尚未完成 kuangbin、WIDA 全部条目的迁移，尚未完成近三年中国赛站逐题覆盖审计，最大流、平衡树、双连通分量、可持久化等部分接口已有在线 AC 记录，具体题号、源码摘要及验证范围见 `verification/oj.json`。其余接口须分别核验，不能由同文件中某算法通过推断全部通过。本地对拍通过不代表对所有输入的绝对正确性证明；适用范围、数值上限与退化约定属于接口的一部分。

目录

使用与验证 xviii

第一部分　搜索 1

第一章　BFS（搜索） 2
1.1　八数码：反向搜索与最短方案 . . . 2

第二章　Dancing Links 5
2.1　精确覆盖（Dancing Links） . . . 5
2.2　最少行重复覆盖（Dancing Links） . . . 9

第二部分　动态规划 14

第三章　数位 DP 15
3.1　数位 DP：十进制数码计数 . . . 15

第四章　插头 DP 17
4.1　插头 DP：网格单闭环计数 . . . 17
4.2　插头 DP：恰好 K 个不嵌套回路 . . . 21

第五章　DP 优化 25
5.1　斜率优化 . . . 25
5.1.1　单调斜率凸包：最小值与决策 . . . 25
5.2　四边形不等式优化 . . . 27
5.2.1　分治决策单调性：一层分组 DP . . . 27
5.3　WQS 二分 . . . 30
5.3.1　WQS 二分：至多 k 个不相邻位置 . . . 30

第三部分　字符串 32

第六章　字符串哈希 33
6.1　子串与反串双哈希 . . . 33

第七章　前缀函数与 KMP 算法 37
7.1　前缀函数 . . . 37
7.2　KMP 匹配 . . . 39
7.3　顺序同构匹配 . . . 41

第八章　Z 函数（扩展 KMP） 43
8.1　Z 函数 . . . 43
8.2　两串扩展 KMP . . . 44

第九章 AC 自动机 **45**
9.1 AC 自动机出现次数 . . . 45
9.2 AC 自动机最短后缀匹配长度 . . . 51
9.3 AC 自动机带权匹配 . . . 53
9.4 动态 AC：二进制分组重建 . . . 55

第十章 后缀数组 (SA) **57**
10.1 后缀数组简介 . . . 57
10.1.1 线性后缀数组（SA-IS） . . . 57
10.1.2 倍增后缀数组与 LCP . . . 61
10.1.3 后缀 LCP 与子串比较 . . . 64
10.1.4 两个前缀的最长公共后缀 . . . 67
10.1.5 重复子串 AA 的起止位置计数 . . . 67
10.1.6 后缀数组：DC3 线性构造 . . . 69

第十一章 后缀自动机 (SAM) **74**
11.1 后缀自动机 . . . 74
11.2 SAM 字典序子串查询 . . . 79
11.3 SAM 最长公共子串 . . . 82

第十二章 广义后缀自动机 **84**
12.1 广义后缀自动机：Trie BFS 构建 . . . 84
12.2 指定状态在线扩展 SAM . . . 91
12.3 SAM 文档频率与阈值后缀 . . . 94

第十三章 Manacher **99**
13.1 Manacher 回文半径 . . . 99

第十四章 回文树 **102**
14.1 回文树 PAM：回文后缀与出现次数 . . . 102

第十五章 最小表示法 **105**
15.1 最小循环表示 . . . 105

第十六章 Lyndon 分解 **107**
16.1 Lyndon 分解：Duval . . . 107

第四部分 数学 **109**

第十七章 高精度计算 **110**
17.1 精确分数四则运算 . . . 110
17.2 高精度整数：有符号加减乘与小整数除法 . . . 115

第十八章 快速幂 **124**
18.1 无符号 64 位模乘与模幂 . . . 124
18.2 模幂与几何级数同时求值 . . . 125

第十九章 数论 **127**
19.1 数论基础 . . . 127
19.1.1 幂的约数和 . . . 127
19.2 模算术简介 . . . 129

19.2.1 动态模整数与合数模逆元 . . . 129
19.2.2 静态模整数 . . . 133
19.2.3 完整 64 位固定模整数 . . . 136
19.2.4 Montgomery 固定模整数 . . . 140
19.2.5 32 位 Barrett 快速模乘 . . . 144
19.2.6 模运算的适用条件 . . . 146
19.2.6.1 除法与解同余式不是一回事 . . . 146
19.2.6.2 知道整数可整除时，先升模数再整除 . . . 147
19.2.6.3 组合数的整数性不等于分母可逆 . . . 147
19.2.6.4 大指数要同时保留余数和阈值信息 . . . 148
19.3 素数 . . . 149
19.3.1 确定性 64 位 Miller–Rabin 判素 . . . 149
19.4 欧拉函数 . . . 150
19.4.1 单个数的欧拉函数 . . . 150
19.5 筛法 . . . 151
19.5.1 线性筛、欧拉函数与 Möbius 函数 . . . 151
19.5.2 区间筛素数 . . . 152
19.6 分解质因数 . . . 154
19.6.1 Pollard–Rho 因数分解 . . . 154
19.7 裴蜀定理 & 一次不定方程 . . . 156
19.7.1 扩展欧几里得与 Bézout 系数 . . . 156
19.7.2 二元一次不定方程 . . . 157
19.7.3 一次不定方程：特解与正整数解范围 . . . 159
19.7.3.1 有符号矩形范围：把两维约束化成一个参数区间 . . . 160
19.8 费马小定理 & 欧拉定理 . . . 161
19.8.1 十进制大指数：扩展欧拉降幂 . . . 161
19.8.2 Carmichael：模乘法群的公共指数 . . . 162
19.9 模逆元 . . . 163
19.9.1 模整数类型的批量逆元 . . . 163
19.9.2 任意正模数的单个逆元 . . . 164
19.9.3 任意数组的批量逆元 . . . 165
19.9.4 连续整数的线性逆元表 . . . 167
19.10 线性同余方程 . . . 168
19.10.1 线性同余方程 . . . 168
19.11 中国剩余定理 . . . 169
19.11.1 广义 CRT：逐项合并同余 . . . 169
19.11.2 Garner：大周期 CRT 与混合进制 . . . 171
19.12 升幂引理 . . . 173
19.12.1 素因子估值与升幂：先约去公共因子 . . . 173
19.12.1.1 基本运算与公共因子 . . . 173
19.12.1.2 奇素数：差式与和式的条件不同 . . . 173
19.12.1.3 素数 2：先分指数奇偶 . . . 173
19.12.1.4 把大整数整除转成指数条件 . . . 174
19.13 阶乘取模 . . . 175
19.13.1 Legendre 赋值与 Kummer 进位判据 . . . 175
19.13.1.1 组合数：逐位相加时进了几次位 . . . 175
19.13.1.2 素数幂为零与 Lucas 的边界 . . . 176

19.14 卢卡斯定理 . . . 177
19.14.1 Lucas：小素数模的大下标组合数 . . . 177
19.14.2 合数模数组合数（扩展 Lucas） . . . 178
19.14.3 有界上标的任意模数组合数 . . . 181
19.14.4 素数模 Gaussian 二项式 . . . 185
19.15 二次剩余 . . . 189
19.15.1 素数模平方根（Cipolla） . . . 189
19.16 阶 & 原根 . . . 190
19.16.1 乘法阶与原根 . . . 190
19.17 离散对数 . . . 194
19.17.1 扩展 BSGS . . . 194
19.18 高次剩余 & 单位根 . . . 195
19.18.1 素数模高次剩余：全部根的紧凑表示 . . . 195
19.18.2 素数幂模高次剩余：含非互素被开方数 . . . 196
19.18.3 合数求根准备：分解模数与素数幂原根 . . . 199
19.18.4 一般合数模高次剩余：CRT 组合与按索引取根 . . . 200
19.19 数论分块 . . . 203
19.19.1 整除分块：前缀和来源决定算法 . . . 203
19.19.1.1 按数据与资源选前缀和 . . . 203
19.20 莫比乌斯反演 . . . 205
19.20.1 矩形 GCD 计数 . . . 205
19.20.2 整除与反演：先辨认求和方向 . . . 206
19.20.2.1 约数和与倍数和不能混用 . . . 206
19.20.2.2 GCD 权值先反演，再交换求和 . . . 207
19.20.3 GCD 计数：缩放、边界与对称性 . . . 207
19.20.3.1 恰好等于 k：先缩放整除对象，再做矩形容斥 . . . 207
19.20.3.2 去掉顺序之前，先检查交换是否仍在定义域内 . . . 207
19.21 狄利克雷双曲线法 & 杜教筛 . . . 208
19.21.1 杜教筛 . . . 208
19.22 素数计数 . . . 210
19.22.1 素数计数（整除分块筛） . . . 210
19.23 Min_25 筛 . . . 212
19.23.1 Min_25 积性函数求和 . . . 212
19.24 类欧几里德算法 . . . 215
19.24.1 有符号整除和：类欧几里德 . . . 215
19.24.2 整除和、带权和与平方和 . . . 217
19.24.3 类欧几里德：有符号整除求和 . . . 219
19.24.4 类欧几里德的三个求和量 . . . 220

第二十章 多项式与生成函数 221
20.1 快速傅里叶变换 . . . 221
20.1.1 复数 FFT（可复用单位根） . . . 221
20.1.2 浮点 FFT 整数卷积 . . . 225
20.1.3 拆系数 FFT 任意模卷积 . . . 227
20.1.4 CDQ 分治卷积递推 . . . 229
20.2 快速数论变换 . . . 231
20.2.1 可指定素数与原根的 NTT 卷积 . . . 231
20.2.2 精确有符号整数卷积 . . . 234

20.2.3 三模 NTT：精确任意模数卷积 . . . 236
20.3 快速沃尔什变换 . . . 242
20.3.1 按位 OR、AND、XOR 卷积 . . . 242
20.3.2 不相交子集卷积 . . . 244
20.4 Chirp Z 变换 . . . 246
20.4.1 Chirp Z：等比点求值 . . . 246
20.5 多项式牛顿迭代 . . . 248
20.5.1 形式幂级数求逆 . . . 248
20.5.2 多项式带余除法 . . . 249
20.6 多项式多点求值 | 快速插值 . . . 252
20.6.1 任意点多项式多点求值 . . . 252
20.6.2 任意点多项式快速插值 . . . 255
20.7 多项式初等函数 . . . 257
20.7.1 形式幂级数开根（含前导零与无解） . . . 257
20.7.2 形式幂级数微积分、对数与指数 . . . 261
20.7.3 形式幂级数快速幂（十进制大指数） . . . 264
20.8 常系数齐次线性递推 . . . 266
20.8.1 特征多项式求线性递推第 k 项 . . . 266
20.8.2 快速递推与有理生成函数单项 . . . 268
20.9 多项式平移 | 连续点值平移 . . . 270
20.9.1 多项式系数平移 . . . 270
20.10 符号化方法 . . . 271
20.10.1 生成函数建模到模板：三个可核验例子 . . . 271
20.10.1.1 例一：不同盒子、相同球、每盒有上下界（OGF 乘积） . . . 271
20.10.1.2 例二：步长 1、2 的有序序列（OGF 求逆） . . . 272
20.10.1.3 例三：不同元素分成单点与无序对（EGF 指数） . . . 272
20.10.1.4 抄代码前的四项检查与本地证据 . . . 273
20.11 Lagrange 反演 . . . 274
20.11.1 隐式生成函数与 Lagrange 反演 . . . 274
20.11.1.1 适用条件与取系数公式 . . . 274
20.11.1.2 形式留数证明 . . . 274
20.11.1.3 模型一按内部结点计数的满有序多叉树 . . . 274
20.11.1.4 模型二带标号有根树的 EGF . . . 275
20.11.1.5 落到现有接口及模数边界 . . . 275

第二十一章 组合数学 276
21.1 排列组合 . . . 276
21.1.1 阶乘组合数 . . . 276
21.1.2 计数建模：先确认哪些对象被区分 . . . 277
21.1.2.1 五个判别问题与两个最小反例 . . . 277
21.1.2.2 OGF、EGF 的选择来自乘法，而非题目用词 . . . 278
21.1.3 组合数选型：先看模数，再看下标与预算 . . . 278
21.2 容斥原理 . . . 279
21.2.1 容斥与二项式反演：恰好与至少 . . . 279
21.2.1.1 从交集和恢复恰好数量 . . . 279
21.2.1.2 至少数量与变换方向 . . . 279
21.2.1.3 例：有标号球放入有标号盒 . . . 280
21.2.1.4 直接使用现有组合数与模整数 . . . 280

21.3 斐波那契数列 . . . 281
21.3.1 斐波那契最小循环节（Pisano 周期） . . . 281
21.4 卡特兰数 . . . 286
21.4.1 Catalan 与投票问题：把前缀限制变成反射双射 . . . 286
21.4.1.1 弱前缀条件：减去首次走到负一的坏路径 . . . 286
21.4.1.2 严格领先：先取走必需的第一步 . . . 286
21.4.1.3 初始有余额：反射起点，而非只改终点 . . . 287
21.4.1.4 取模时优先使用差式，除法条件不能省 . . . 287
21.5 斯特林数与斯特林反演 . . . 288
21.5.1 定义与性质 . . . 288
21.5.1.1 行列算法的选择与推导 . . . 288
21.5.2 无符号第一类斯特林数：同一行 . . . 289
21.5.3 无符号第一类斯特林数：同一列 . . . 291
21.5.4 第二类斯特林数：同一行前缀 . . . 293
21.5.5 第二类斯特林数：同一列 . . . 294
21.5.6 斯特林反演与应用 . . . 296
21.5.6.1 手算检查与建模用法 . . . 296
21.5.6.2 题目协议与验证范围 . . . 296
21.6 分拆数 . . . 297
21.6.1 整数分拆与重数上限 . . . 297
21.6.2 整数分拆：重数限制的差一问题 . . . 299
21.6.2.1 三个容易混淆的限制 . . . 300
21.6.2.2 重数限制与禁止倍数部分 . . . 300
21.7 Pólya 计数 . . . 301
21.7.1 Burnside 平均：任意模数下的精确除法 . . . 301
21.7.2 置换的循环长度分解 . . . 304
21.7.3 Pólya 环染色：旋转与翻转 . . . 305
21.7.4 群作用计数：先确定什么算同一个方案 . . . 307
21.7.5 项链与手链：从置换循环推公式 . . . 307
21.7.6 指定颜色库存：给每个循环分配整份颜色 . . . 308
21.7.7 群阶不可逆：先提升模数，再作整数除法 . . . 308

第二十二章 线性代数 **310**
22.1 矩阵 . . . 310
22.1.1 模矩阵乘法与幂 . . . 310
22.1.2 素数模矩阵求逆 . . . 312
22.1.3 模 2 矩阵求逆（按位消元） . . . 315
22.1.4 Max-plus 矩阵幂 . . . 317
22.2 行列式 . . . 319
22.2.1 素数模行列式 . . . 319
22.2.2 任意正模数行列式：欧几里得消元 . . . 322
22.2.3 精确整数行列式（Bareiss） . . . 324
22.3 线性基 . . . 326
22.3.1 64 位异或线性基 . . . 326
22.3.2 带位置限制的异或线性基 . . . 328
22.3.3 异或线性空间求交 . . . 332
22.3.4 Zassenhaus：同时求和空间与交空间 . . . 334
22.3.5 无向图最大异或行走 . . . 336

22.4 特征多项式 . . . 339
22.4.1 有限状态计数到线性递推：先证明阶数，再用 BM . . . 339
22.4.1.1 先把状态、方向和输出写清楚 . . . 339
22.4.1.2 Cayley–Hamilton 给出阶数至多 d . . . 339
22.4.1.3 为什么前 2d 项足以恢复这个标量序列 . . . 339
22.4.1.4 模型：不含连续子串 101 的二进制串 . . . 340
22.4.1.5 按目标选算法，并把采样成本算进去 . . . 340

第二十三章 线性规划 341
23.1 单纯形法 . . . 341
23.1.1 线性规划：两阶段单纯形 . . . 341

第二十四章 概率论 346
24.1 随机变量的数字特征 . . . 346
24.1.1 期望选型：计数拆指标，等待拆尾和 . . . 346
24.1.2 条件期望：先定义状态，再解自环 . . . 346
24.1.3 模概率：可逆的不只是输入分母 . . . 347

第二十五章 博弈论 348
25.1 公平组合游戏 . . . 348
25.1.1 SG 的前提、递推与异或策略 . . . 348
25.1.2 减法游戏小例与三类失效边界 . . . 348

第二十六章 数值算法 350
26.1 插值 . . . 350
26.1.1 拉格朗日插值与连续点优化 . . . 350
26.2 数值积分 . . . 353
26.2.1 有工作上限的自适应 Simpson 积分 . . . 353
26.2.2 数值积分：局部估计、误差预算与采样盲区 . . . 356
26.3 高斯消元 . . . 357
26.3.1 压位模 2 高斯消元与全部解 . . . 357
26.3.2 素数模高斯消元与全部解 . . . 361
26.3.3 实数高斯消元与数值解空间 . . . 364

第二十七章 Berlekamp–Massey 算法 367
27.1 Berlekamp–Massey：最短线性递推 . . . 367

第五部分 数据结构 369

第二十八章 哈希表 370
28.1 GNU gp_hash_table 与 cc_hash_table . . . 370

第二十九章 并查集 373
29.1 并查集 . . . 373
29.1.1 并查集 . . . 373
29.1.2 可撤销并查集 . . . 375
29.1.3 带势并查集与差值约束 . . . 377
29.1.4 时间线段树分治：离线动态连通性 . . . 380

第三十章 堆 **386**
30.1 配对堆 . . . 386
30.1.1 PBDS 可修改、可合并优先队列 . . . 386
30.2 左偏树 . . . 390
30.2.1 左偏树可并小根堆 . . . 390

第三十一章 块状数据结构 **392**
31.1 树分块 . . . 392
31.1.1 树分块与在线路径去重 . . . 392

第三十二章 ST 表 **399**
32.1 位掩码静态区间最值与位置 . . . 399
32.2 二维静态矩形最值与位置 . . . 403

第三十三章 树状数组 **407**
33.1 稀疏二维树状数组：点加与矩形和 . . . 407
33.2 四矩二维树状数组：矩形加与矩形和 . . . 410
33.3 树状数组第 k 小与组合依赖 . . . 412
33.4 离线静态矩形权值和 . . . 414

第三十四章 线段树 **417**
34.1 线段树基础 . . . 417
34.1.1 通用懒标记线段树与边界搜索 . . . 417
34.1.2 通用线段树与左右边界搜索 . . . 422
34.1.3 区间加与区间和 . . . 427
34.1.4 线段树：区间乘、加、赋值与模意义求和 . . . 428
34.1.5 线段树维护单调栈：区间加与接雨水 . . . 432
34.2 线段树合并 & 分裂 . . . 437
34.2.1 线段树合并与分裂 . . . 437
34.3 李超线段树 . . . 448
34.3.1 精确线段李超树：最大值编号 . . . 448
34.3.2 离散李超树与最优直线编号 . . . 454
34.4 区间最值操作 & 区间历史最值 . . . 457
34.4.1 Segment Tree Beats：区间截断、加法与和 . . . 457

第三十五章 划分树 **463**
35.1 划分树：静态区间第 k 小 . . . 463
35.2 压位小波矩阵：静态区间顺序统计 . . . 466

第三十六章 二叉搜索树 & 平衡树 **471**
36.1 二叉搜索树 & 平衡树 . . . 471
36.1.1 GNU rope 与版本共享 . . . 471
36.1.2 PBDS 有序树与顺序统计 . . . 473
36.2 Treap . . . 475
36.2.1 FHQ Treap：可重集合与顺序统计 . . . 475
36.2.2 隐式 Treap：顺序反转、位翻转、加值与区间和 . . . 479
36.2.3 动态仿射序列：插删、翻转与区间和 . . . 484
36.2.4 状态序列 Treap：分组区间 GCD . . . 489
36.2.5 序列最大值 Treap . . . 494
36.3 Splay 树 . . . 498

36.3.1 Splay：可重集合与顺序统计 . . . 498
36.3.2 可合并 Splay：固定编号与集合顺序统计 . . . 504
36.3.3 按值分裂与整体平移 Splay . . . 512
36.3.4 区间翻转 Splay 与固定编号定位 . . . 516
36.4 替罪羊树 . . . 523
36.4.1 替罪羊树：可重集合与顺序统计 . . . 523
36.4.2 在线序列：插入、修改与区间第 k 小 . . . 527
36.4.3 序列替罪羊树：按位置插删与区间等值计数 . . . 532
36.5 笛卡尔树 . . . 538
36.5.1 笛卡尔树与相等值优先级 . . . 538

第三十七章 可持久化数据结构 540
37.1 可持久化线段树 . . . 540
37.1.1 主席树：静态区间第 k 小 . . . 540
37.1.2 可持久化数组：历史版本分叉与单点修改 . . . 542
37.1.3 可持久化区间加与版本拼接 . . . 544
37.1.4 树上路径点权第 k 小 . . . 552
37.1.5 静态区间不同数计数（前驱位置主席树） . . . 555
37.1.6 持久化计数与权值索引 . . . 558
37.1.7 持久化奇偶计数与降序选择 . . . 562
37.1.8 可持久化并查集 . . . 567
37.2 可持久化平衡树 . . . 570
37.2.1 可持久化有序 Treap：分叉版本与顺序统计 . . . 570
37.3 可持久化字典树 . . . 576
37.3.1 前缀可持久化 01 字典树 . . . 576

第三十八章 树套树 579
38.1 树状数组套权值线段树 . . . 579
38.1.1 动态区间第 k 小（树状数组套权值线段树） . . . 579

第三十九章 K-D Tree 583
39.1 动态二维 K-D Tree：单点累加与矩形和 . . . 583
39.2 静态 K-D Tree 最近与最远点 . . . 588
39.3 动态 K-D Tree 矩形取点与删除 . . . 593
39.4 静态二维矩形最小权与删点 . . . 599

第四十章 动态树 604
40.1 Link Cut Tree . . . 604
40.1.1 Link-cut tree：动态森林与路径异或 . . . 604
40.1.2 动态换根路径乘积总和 . . . 609
40.1.3 动态树上生长与采摘可行性 . . . 613
40.1.4 点权修改与全树路径异或和 . . . 618
40.1.5 动态森林路径加与最大值 . . . 624

第六部分 图论 629

第四十一章 DFS（图论） 630
41.1 顶点划分：各诱导子图全为奇度 . . . 630

第四十二章 树上问题 **633**
42.1 树的直径 . . . 633
42.1.1 边权修改动态树直径 . . . 633
42.1.2 树上点集直径摘要 . . . 638
42.2 最近公共祖先 . . . 641
42.2.1 离线 Tarjan LCA . . . 641
42.2.2 欧拉序与 ST 表 LCA . . . 644
42.2.3 倍增 LCA、祖先与最大边权 . . . 647
42.2.4 两条树路径的交集 . . . 652
42.3 树链剖分 . . . 654
42.3.1 长链剖分与常数时间 K 级祖先 . . . 654
42.3.2 重链剖分与 LCA . . . 659
42.3.3 树上翻色、改权与同色连通块最大值 . . . 671
42.3.4 树上翻色与同色连通块 . . . 675
42.4 树上启发式合并 . . . 680
42.4.1 树上启发式合并：子树众数颜色和 . . . 680
42.5 虚树 . . . 683
42.5.1 虚树：关键点压缩与路径信息 . . . 683
42.6 树分治 . . . 687
42.6.1 点分治：树上距离点对计数 . . . 687
42.6.2 树上新增市场的支配范围 . . . 691
42.7 动态树分治 . . . 695
42.7.1 动态点分树：点权赋值与距离球求和 . . . 695
42.7.2 动态点分树：最近活动点与点号方案 . . . 700
42.7.3 动态点分树：有符号边权活动点直径与端点 . . . 704
42.8 树哈希 . . . 710
42.8.1 确定性有根与无根树同构 . . . 710

第四十三章 拓扑排序 **713**
43.1 DAG 有符号权最长路与路径方案 . . . 713

第四十四章 最短路问题 **719**
44.1 最短路 . . . 719
44.1.1 Johnson：负边全源最短路 . . . 719
44.1.2 非负权最短路 . . . 722
44.1.3 顶点开放时间的单位边最短路 . . . 724
44.1.4 Floyd 全源最短路与原边方案 . . . 726
44.1.5 最小总边权最短路树（含零权边） . . . 733
44.1.6 稠密图最短路：矩阵 Dijkstra . . . 739
44.1.7 SPFA：队列松弛、负环判定与有限路径 . . . 743
44.1.8 Bellman–Ford：负环影响与路径方案 . . . 750
44.2 k 短路 . . . 757
44.2.1 非负权严格次短行走 . . . 757
44.2.2 正权前 K 短行走 . . . 760

第四十五章 生成树问题 **763**
45.1 最小生成树 . . . 763
45.1.1 Kruskal 重构森林与阈值连通块 . . . 763
45.1.2 曼哈顿最小生成树 . . . 768
45.1.3 严格与非严格次小生成树、换边方案 . . . 771
45.1.4 稠密图 Prim 最小生成森林 . . . 776
45.1.5 Kruskal 最小生成树与生成森林 . . . 780
45.1.6 曼哈顿最小生成树：闭锥扫描与平局割证明 . . . 784
45.2 最小树形图 . . . 785
45.2.1 有向最小树形图 . . . 785

第四十六章 斯坦纳树 **788**
46.1 最小斯坦纳树与原边方案 . . . 788

第四十七章 连通性相关 **794**
47.1 强连通分量 . . . 794
47.1.1 Tarjan 强连通分量与缩点 . . . 794
47.1.2 函数图分解、跳转与到达 . . . 799
47.1.3 强连通分量 . . . 804
47.2 双连通分量 . . . 807
47.2.1 点双与边双：共享 lowlink 核心 . . . 807
47.2.2 边双缩点森林（保留桥编号） . . . 811
47.2.3 属于简单奇环的全部顶点 . . . 813
47.2.4 边双缩点的大小与内部边数 . . . 816
47.2.5 无向边定向与强连通定向 . . . 819
47.3 割点和桥 . . . 820
47.3.1 由点双分解计算删点连通块数 . . . 820
47.3.2 连通无向图最少加边方案 . . . 823
47.3.3 割点与桥 . . . 824
47.4 圆方树 . . . 827
47.4.1 静态仙人掌森林最短路 . . . 827
47.4.2 由点双构造圆方森林 . . . 832

第四十八章 环计数问题 **834**
48.1 简单无向图四元环计数 . . . 834
48.2 无向简单图三元环枚举 . . . 836

第四十九章 最小环 **838**
49.1 有向图最小平均权环：线性空间 Karp . . . 838
49.2 非负权无向图最小环与原边方案 . . . 842

第五十章 2-SAT **847**
50.1 2-SAT 与任意可行方案 . . . 847
50.2 递归染色与回滚求最小赋值 . . . 850

第五十一章 欧拉图 **853**
51.1 有向图字典序最小欧拉路 . . . 853
51.2 字典序最小单词链 . . . 856
51.3 无向图欧拉路与边方向 . . . 858
51.4 欧拉回路与指定端点定向 . . . 861

51.5 任意起终点的混合欧拉路径 . . . 864

第五十二章 网络流 865

52.1 最大流 . . . 865
52.1.1 HLPP 最高标号预流推进与流方案 . . . 865
52.1.2 最大流与方案 . . . 871
52.1.3 最大流中单位容量边的可行性与必选性 . . . 877
52.1.4 递归 ISAP 最大流 . . . 881
52.1.5 最大流方案唯一性 . . . 885
52.2 最小割 . . . 887
52.2.1 正容量最小割的可行边与必选边 . . . 887
52.2.2 最大权闭合子图与选点方案 . . . 889
52.2.3 Gomory–Hu 最小割树 . . . 891
52.2.4 最小割树全点对查询 . . . 893
52.3 费用流 . . . 894
52.3.1 势能最小费用最大流 . . . 894
52.3.2 SPFA 最小费用最大流 . . . 898
52.3.3 允许负环的最小费用最大流 . . . 902
52.3.4 zkw 最小费用最大流 . . . 905
52.4 上下界网络流 . . . 909
52.4.1 上下界可行循环流 . . . 909
52.4.2 有源汇上下界最值流与边流量 . . . 911
52.5 Stoer–Wagner 算法 . . . 915
52.5.1 无向全局最小割 . . . 915

第五十三章 图的匹配 918

53.1 二分图最大匹配 . . . 918
53.1.1 正则二分图完美匹配 . . . 918
53.1.2 Hopcroft–Karp 最大匹配与最小点覆盖 . . . 922
53.1.3 二分图最大匹配的可行边与必选边 . . . 926
53.1.4 二分图最大独立集方案 . . . 927
53.2 二分图最大权匹配 . . . 929
53.2.1 矩形二分图最小权匹配 . . . 929
53.2.2 二分图最大权匹配（缺边与可选匹配） . . . 931
53.2.3 按匹配基数求最大权与顶标证书 . . . 934
53.3 一般图最大匹配 . . . 940
53.3.1 一般图最大基数匹配：带花树 . . . 940
53.4 一般图最大权匹配 . . . 943
53.4.1 一般图最大权匹配：带权带花树 . . . 943

第五十四章 Prüfer 序列 954

54.1 Prüfer 双射与有标号树计数 . . . 954
54.1.1 双射为何成立 . . . 954
54.1.2 Cayley、指定度数与叶子数 . . . 954
54.1.3 加权 Cayley 与连接已有连通块 . . . 955

第五十五章 矩阵树定理 **956**
55.1 Tutte 多项式求值 . . . 956
55.2 带权矩阵树：无向树与内外向树形图 . . . 961
55.3 随机图恰为生成树的概率 . . . 964
55.4 任意正模数带权矩阵树 . . . 967
55.5 精确整数带权生成树计数 . . . 970
55.6 带权矩阵树定理与方向约定 . . . 973
55.6.1 为什么这个余子式恰好计数 . . . 973
55.6.2 既有接口与验证记录 . . . 973
55.7 任意正模数的矩阵树计数 . . . 974

第五十六章 LGV 引理 **975**
56.1 LGV：DAG 路径权值行列式 . . . 975

第五十七章 支配树 **977**
57.1 支配树：Lengauer–Tarjan . . . 977
57.2 DAG 支配树：拓扑序与在线倍增 LCA . . . 981

第七部分 计算几何 **986**

第五十八章 二维计算几何基础 **987**
58.1 浮点点向量、圆与交点结果 . . . 987
58.2 有限二进制浮点极角精确排序 . . . 989
58.3 点在直线上的投影 . . . 991
58.4 浮点两直线交点 . . . 992
58.5 浮点直线与圆交点 . . . 994
58.6 浮点两圆交点 . . . 996
58.7 两圆交面积 . . . 999
58.8 整数点向量与线段判定 . . . 1000
58.9 整数输入直线与圆交点 . . . 1002
58.10 整数输入两圆交点 . . . 1004
58.11 整数多边形有向面积 . . . 1005
58.12 点在整数简单多边形内 . . . 1008
58.13 圆与简单多边形相交面积 . . . 1009
58.14 两圆公切线与过点切线 . . . 1011
58.15 整数圆公切线与精确切点排序 . . . 1013
58.16 一般多边形面积并：整数精确判定 . . . 1016
58.17 圆面积并：未覆盖圆弧积分 . . . 1022
58.18 圆覆盖重数面积：恰好覆盖 k 次 . . . 1026
58.19 凸多边形裁剪：保留有向直线左侧 . . . 1030
58.20 简单多边形面积重心 . . . 1032
58.21 三角形内切圆：边长加权内心 . . . 1034

第五十九章 三维计算几何基础 **1036**
59.1 整数三维共线、共面与方向 . . . 1036
59.2 三维凸包：精确方向与退化处理 . . . 1038
59.3 整数凸多面体：体积重心与表面距离 . . . 1048
59.4 实数三维向量与夹角 . . . 1053

59.5 三维直线投影、线段距离与绕轴旋转 . . . 1056
59.6 三维平面投影与分类求交 . . . 1059

第六十章 距离 1063
60.1 点到闭线段的距离 . . . 1063

第六十一章 Pick 定理 1065
61.1 Pick 定理：格点、剖分与孔洞 . . . 1065

第六十二章 凸包 1067
62.1 凸多边形外点的两条切线 . . . 1067
62.2 整数凸包：单调链 . . . 1071
62.3 点在整数凸包内（二分） . . . 1074
62.4 凸多边形 Minkowski 和 . . . 1075
62.5 支撑点查询恢复下凸包 . . . 1077

第六十三章 扫描线 1081
63.1 整数轴对齐矩形面积并 . . . 1081

第六十四章 旋转卡壳 1083
64.1 整数凸包直径：旋转卡壳 . . . 1083
64.2 最小面积包围矩形：旋转卡壳与四角 . . . 1086

第六十五章 半平面交 1089
65.1 整数半平面的有界多边形交 . . . 1089

第六十六章 平面最近点对 1093
66.1 整数最近点对（精确平方距离） . . . 1093
66.2 浮点平面最近点对 . . . 1096

第六十七章 随机增量法 1098
67.1 随机增量最小覆盖圆 . . . 1098

第六十八章 计算几何：精确判定与适用范围 1100
68.1 定点凸包、直径和极角排序 . . . 1100
68.2 整数系数的有界半平面交 . . . 1101
68.3 最近点对、Minkowski 和与三维整数判定 . . . 1102
68.4 浮点最近点对 . . . 1103
68.5 公切线与过点切线 . . . 1104
68.6 随机增量最小覆盖圆 . . . 1105
68.7 圆与简单多边形相交面积 . . . 1106
68.8 最近点对编号与多边形闭集查询 . . . 1106
68.9 整数圆交点：精确判定与浮点构造 . . . 1107
68.10 公切线：切点次序也需要精确判定 . . . 1107
68.11 圆交点：容差分类与坐标误差 . . . 1108
68.12 三维投影、旋转与平面求交的数值约定 . . . 1109
68.12.1 原三维片段中需要避开的错误 . . . 1110

第八部分 杂项 1111

第六十九章 离线算法 1112

69.1 CDQ 分治 . . . 1112
69.1.1 四维偏序：双层 CDQ . . . 1112
69.1.2 静态三维偏序计数 . . . 1117
69.1.3 三维最长链、计数与方案 . . . 1119
69.2 整体二分 . . . 1124
69.2.1 整体二分：动态区间第 k 小 . . . 1124
69.3 莫队算法 . . . 1128
69.3.1 普通莫队算法 . . . 1128
69.3.1.1 普通莫队：静态区间统计 . . . 1128
69.3.1.2 曼哈顿 MST 的递归 DFS 查询顺序 . . . 1132
69.3.2 带修改莫队 . . . 1135
69.3.2.1 带修改的莫队 . . . 1135
69.3.3 树上莫队 . . . 1138
69.3.3.1 树上莫队：点权与边权 . . . 1138
69.3.4 回滚莫队 . . . 1142
69.3.4.1 回滚莫队 . . . 1142
69.3.5 莫队二次离线 . . . 1146
69.3.5.1 莫队二次离线：异或位数配对 . . . 1146

第九部分 数学速查与维护 1149

第七十章 数学速查：条件先于公式 1150

70.1 杜教筛：预筛与缓存都不可省略 . . . 1150
70.2 64 位判素与质因数分解 . . . 1150
70.3 单个数的欧拉函数 . . . 1151
70.4 线性筛：三张表的约定 . . . 1151
70.5 区间筛：只保存目标区间 . . . 1151
70.6 矩形 GCD 计数：缩放后容斥 . . . 1152
70.7 幂的约数和：不除以 $p-1$ . . . 1152
70.8 任意数组的批量逆元 . . . 1152
70.9 连续整数的线性逆元表 . . . 1153
70.10 线性同余：最小解与最小周期 . . . 1153
70.11 扩展 CRT：逐式合并 . . . 1153
70.12 Garner：总周期可以超过整数范围 . . . 1154
70.13 素数模平方根 . . . 1154
70.14 乘法阶与原根 . . . 1155
70.15 Carmichael：最小公共指数 . . . 1155
70.16 Lucas：按素数进制逐位计算 . . . 1155
70.17 合数模数组合数与插值 . . . 1156
70.18 阶乘表：组合数与排列数 . . . 1156
70.19 组合计数 . . . 1156
70.20 生成函数与递推 . . . 1156
70.21 线性代数与生成树 . . . 1157
70.22 模线性方程组：工厂工期 . . . 1157
70.23 数值算法与几何 . . . 1157

70.24 线性递推：系数方向与外推条件 . . . 1157
70.25 素数模高次剩余与单位根 . . . 1159
70.26 素数幂模高次剩余与根的提升 . . . 1160
70.27 一般合数模高次剩余：CRT 组合 . . . 1161
70.28 自动准备模数分解与原根 . . . 1162
70.29 任意正模数行列式 . . . 1163
70.30 参数化 NTT 与模数适配 . . . 1164
70.31 Bostan–Mori 与快速递推单项 . . . 1165
70.32 集合变换与不相交子集卷积 . . . 1166
70.33 Taylor 平移与等比点求值 . . . 1167
70.34 可持久化区间拼接的懒标记 . . . 1168
70.35 杭州 2023 K：跨版本拼接应用 . . . 1169
70.36 树路径交：投影与单点交 . . . 1170
70.37 非负权树的点集直径摘要 . . . 1171
70.38 函数图的入环链与到达查询 . . . 1172
70.39 杭州 2023 H：随机事件顺序与函数图 . . . 1173
70.40 杭州 2023 G：开放时间的单位边最短路 . . . 1174
70.41 带修莫队：时间轴与区间状态 . . . 1174
70.42 SCC 模板题与速度榜口径 . . . 1175
70.43 并查集模板题的接口范围 . . . 1175
70.44 ACL 的组合约定 . . . 1175
70.45 动态树路径乘积：虚子树与双向信息 . . . 1175
70.46 隐式 Treap 位翻转与顺序反转 . . . 1176
70.47 替罪羊树：重复计数与墓碑清理 . . . 1176
70.48 莫队二次离线：前缀事件与自身扣除 . . . 1176
70.49 线段树维护单调栈：双向扫描与接雨水 . . . 1176
70.50 动态 K-D Tree：矩形和与重建边界 . . . 1177
70.51 CDQ 卷积递推：贡献只跨越一次 . . . 1177
70.52 多项式带余除法：先反转再求逆 . . . 1177
70.53 级数开根：前导零和自由尾项 . . . 1178
70.54 ln 与 exp：只处理所需前缀 . . . 1178
70.55 幂函数的大指数：保留三份指数信息 . . . 1178
70.56 集合变换：逆变换对模数的要求 . . . 1178
70.57 浮点 FFT：舍入前先约束最坏误差 . . . 1178
70.58 SPFA 费用流与残量方案 . . . 1179
70.59 2-SAT 编号方向与双连通定义 . . . 1179
70.60 回文半径与在线回文计数 . . . 1180
70.61 整数后缀数组的边界与 SAM 次数 . . . 1181
70.62 广义 SAM 点数与后缀语言 . . . 1181
70.63 排列约束与最大流单位边分类 . . . 1181
70.64 点分治、启发式合并与虚树用法 . . . 1182
70.65 压位小波矩阵与区间顺序统计 . . . 1182
70.66 模 2 压位消元与完整解空间 . . . 1182
70.67 严格次小生成树与换边 . . . 1183
70.68 模 2 消元后的最少开关用法 . . . 1183
70.69 闭合集、树形图与全局割的完整用法 . . . 1183

第七十一章 来源与覆盖维护 **1184**
71.1 参考基线 1184
71.2 后续覆盖项 1184

使用与验证

英文署名已于 QOJ 队伍账号 `ucup-team7502` 核对。按最新要求只保留 vector 实现，风格依据训练提交 2932575 提取，不擅自将共享队号的提交归属到个人。

代码采用 C++20。以赛时快速抄写为先，短语句可适度合并，复杂控制流与函数体保持清晰。无状态的简单算法使用独立函数，有状态算法使用轻量 struct。默认需要 `bits/stdc++.h` 和 `cassert`；各头文件列明依赖。抄写单个函数或结构体时需同时抄写其依赖结构。历史静态版仅留在 Git 历史与评测档案中，不属于当前模板库。点号除特别注明外为 1-based，字符串和多项式为 0-based；所有区间约定应以条目说明为准。

测试源码位于 `tests/`，固定种子便于复现。验证包含独立暴力参考、边界案例、静态与动态实现、AddressSanitizer 和 UndefinedBehaviorSanitizer。`verification/` 保存实际运行记录与源码摘要；OJ 状态只在获得可查提交结果后登记。最大流记录为 https://www.luogu.com.cn/record/297501480 与 https://www.luogu.com.cn/record/297502364。

原始来源目录共登记 713 项，见 `docs/coverage.csv`。它包含不同版本、例题及公式，不应将其数目视为独立算法数，也不能将“已登记”视为“已完成”。

第一部分

搜索

第一章　BFS（搜索）

1.1　八数码：反向搜索与最短方案

构造一次 EightPuzzle，solve(a) 可多次查询；a 按行存 0..8 的一个排列，0 为空格，固定目标 123456780。返回 pair<bool,string>，false 表示不可达；true 的字符串按空格移动方向 u/d/l/r 给出最短方案，目标状态返回空串。rank(a) 是 0..9!−1 的康托编号；parent 和 step 仅供实现，不应改写。

反向 BFS 首次访问记录朝目标的前驱及反向动作，不给每个状态保存整条字符串。最多 181440 个可达状态，最远 31 步。对固定 9 格，预处理 $O(9! \cdot 9^2)$ 时间、$O(9! \cdot 9)$ 空间（含队列），持久表 O(9!)；查询 $O(9^2+$ 路径长度)。仅适用于 3×3 和上述固定目标，不适用于十五数码；输入须合法，不检查重复数。

```
struct EightPuzzle
{
    vector<int> parent;
    vector<char> step;

    int rank(const array<int, 9> &a) const
    {
        int id = 0;
        for (int i = 0; i < 9; i++)
        {
            int smaller = 0;
            for (int j = i + 1; j < 9; j++) smaller += a[j] < a[i];
            id = id * (9 - i) + smaller;
        }
        return id;
    }
```

反向预处理与最短方案（接上页同一结构体）：

```
    EightPuzzle() : parent(362880, -1), step(362880)
    {
        array<int, 9> goal = {1, 2, 3, 4, 5, 6, 7, 8, 0};
        int root = rank(goal);
        parent[root] = root;
        queue<array<int, 9>> q;
        q.push(goal);
        int dr[] = {-1, 1, 0, 0};
        int dc[] = {0, 0, -1, 1};
        string inverse = "durl";
        while (!q.empty())
        {
            auto a = q.front();
            q.pop();
            int id = rank(a);
            int zero = find(a.begin(), a.end(), 0) - a.begin();
            for (int d = 0; d < 4; d++)
            {
                int r = zero / 3 + dr[d];
                int c = zero % 3 + dc[d];
                if (r < 0 || r >= 3 || c < 0 || c >= 3) continue;
                auto b = a;
                swap(b[zero], b[r * 3 + c]);
                int next = rank(b);
                if (parent[next] != -1) continue;
                parent[next] = id;
                step[next] = inverse[d];
                q.push(b);
            }
        }
    }

    pair<bool, string> solve(const array<int, 9> &a) const
    {
        int id = rank(a);
        if (parent[id] == -1) return {false, ""};
        string path;
        while (parent[id] != id)
        {
            path += step[id];
            id = parent[id];
        }
        return {true, path};
    }
};
```

应用题补充：HDU1043 八数码路径

题意：EOF 读若干 3×3 排列，x 为空格；输出空格移动方向 u/d/l/r 或 unsolvable，目标状态输出空行。题面只要求任意合法方案，本实现保证最短，不按官方样例字符串逐字比较。

所需模板：EightPuzzle（第 2 页，1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://acm.hdu.edu.cn/showproblem.php?pid=1043

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    EightPuzzle solver;
    char ch;
    while (cin >> ch)
    {
        array<int, 9> a;
        a[0] = ch == 'x' ? 0 : ch - '0';
        for (int i = 1; i < 9; i++)
        {
            cin >> ch;
            a[i] = ch == 'x' ? 0 : ch - '0';
        }
        auto result = solver.solve(a);
        cout << (result.first ? result.second : "unsolvable") << '\n';
    }
}
```

第二章　Dancing Links

2.1　精确覆盖（Dancing Links）

ExactCover(m,rows)：列 0..m-1，各稀疏行内列号互异、无需排序；行可重复或为空，保留原 0-based 行号。solve() 返回 optional<vector<int»，无解为 nullopt，m=0 为有值的空解。只求任意首解，不保证最少行数。成功/失败均恢复链与列计数，可重复调用及复制；实现字段不可外改，移后源对象仅销毁或重新赋值。m 和行数<INT_MAX，1+m+ 非零数 ≤INT_MAX。构造 O(行数 +m+ 非零数)，空间 O(m+ 非零数)，递归深度 ≤m，搜索最坏指数。

```
struct ExactCover
{
    struct Node { int l, r, u, d, c, row; };
    int m;
    vector<Node> a;
    vector<int> s, ans;

    ExactCover(int columns, const vector<vector<int>> &rows) : m(columns)
    {
        assert(0 <= m && m < INT_MAX && rows.size() < INT_MAX);
        size_t z = size_t(m) + 1;
        for (const auto &r : rows)
        {
            assert(r.size() <= size_t(INT_MAX) - z);
            z += r.size();
        }
        a.reserve(z);
        s.assign(m + 1, 0);
        for (int c = 0; c <= m; c++)
            a.push_back({c ? c - 1 : m, c == m ? 0 : c + 1, c, c, c, -1});
```

稀疏行建链与删除（接上页同一结构体）：

```
        vector<int> seen(m, -1);
        for (int i = 0; i < int(rows.size()); i++)
        {
            int h = -1;
            for (int col : rows[i])
            {
                assert(0 <= col && col < m && seen[col] != i);
                seen[col] = i;
                int c = col + 1, x = a.size(), u = a[c].u;
                a.push_back({x, x, u, c, c, i});
                a[u].d = a[c].u = x;
                s[c]++;
                if (h == -1) h = x;
                else
                {
                    int l = a[h].l;
                    a[x].l = l; a[x].r = h;
                    a[l].r = a[h].l = x;
                }
            }
        }
    }

    void cover(int c)
    {
        a[a[c].l].r = a[c].r;
        a[a[c].r].l = a[c].l;
        for (int i = a[c].d; i != c; i = a[i].d)
            for (int j = a[i].r; j != i; j = a[j].r)
            {
                a[a[j].u].d = a[j].d;
                a[a[j].d].u = a[j].u;
                s[a[j].c]--;
            }
    }
```

逆序恢复与搜索（接上页同一结构体）：

```
    void uncover(int c)
    {
        for (int i = a[c].u; i != c; i = a[i].u)
            for (int j = a[i].l; j != i; j = a[j].l)
            {
                s[a[j].c]++;
                a[a[j].u].d = a[a[j].d].u = j;
            }
        a[a[c].l].r = a[a[c].r].l = c;
    }

    bool dfs()
    {
        int c = a[0].r;
        if (!c) return true;
        for (int j = a[c].r; j; j = a[j].r)
            if (s[j] < s[c]) c = j;
        cover(c);
        for (int i = a[c].d; i != c; i = a[i].d)
        {
            ans.push_back(a[i].row);
            for (int j = a[i].r; j != i; j = a[j].r) cover(a[j].c);
            bool ok = dfs();
            for (int j = a[i].l; j != i; j = a[j].l) uncover(a[j].c);
            if (ok)
            {
                uncover(c);
                return true;
            }
            ans.pop_back();
        }
        uncover(c);
        return false;
    }

    optional<vector<int>> solve()
    {
        ans.clear();
        ans.reserve(m);
        if (!dfs()) return nullopt;
        return ans;
    }
};
```

模板题使用：Luogu P4929 / 精确覆盖

题意：读入 N×M 的 01 矩阵，N,M≤500 且非零数 ≤5000；每列恰被选择行覆盖一次。稀疏建模后输出原 1-based 行号，任一可行解均可；无解输出 No Solution!。

所需模板：ExactCover（第 5 页，2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4929

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<int>> rows(n);
    for (int i = 0; i < n; i++)
        for (int j = 0, x; j < m; j++)
        {
            cin >> x;
            if (x) rows[i].push_back(j);
        }
    ExactCover dlx(m, rows);
    auto ans = dlx.solve();
    if (!ans) cout << "No Solution!\n";
    else
    {
        for (int i : *ans) cout << i + 1 << ' ';
        cout << '\n';
    }
}
```

2.2 最少行重复覆盖（Dancing Links）

MinimumCover(m, rows)，列号 0..m-1，每行内部互异，允许空行/重复行。solve() 返回最少行数的一组原 0-based 行号；nullopt 无解，有值空向量表示 m=0。每列至少一次，允许重叠；不含权值、覆盖次数或全解枚举。0≤m<INT_MAX，行数 <INT_MAX，m+1+ 非零数 ≤INT_MAX；前提由调用方保证。构造 O(行数 +m+ 非零数)，空间 O(m+ 非零数)；递归搜索最坏指数、深度 ≤m，每状态 O(m+ 非零数)。成功失败均恢复链，可重复求解或复制；重建用重新赋值。字段及辅助方法不作为可修改接口，移动后源对象只销毁或赋值。下界选取不能被同一行共同覆盖的列，至少还需同样多行；按最少候选列搜索，逆序恢复只改左右链。

```
struct MinimumCover
{
    struct Node
    {
        int l, r, u, d, c, row;
    };
    int m, best = 0;
    vector<Node> a;
    vector<int> s, path, ans;
    vector<unsigned char> mark;

    MinimumCover(int columns, const vector<vector<int>> &rows) : m(columns)
    {
        assert(0 <= m && m < INT_MAX && rows.size() < INT_MAX);
        size_t z = size_t(m) + 1;
        for (const auto &r : rows)
        {
            assert(r.size() <= size_t(INT_MAX) - z);
            z += r.size();
        }
        a.reserve(z);
        s.assign(m + 1, 0);
        mark.resize(m + 1);
        path.reserve(m);
        ans.reserve(m);
        for (int c = 0; c <= m; c++)
            a.push_back({c ? c - 1 : m, c == m ? 0 : c + 1, c, c, -1});
```

稀疏建链与删列（接上页同一结构体）：

```
        vector<int> seen(m, -1);
        for (int i = 0; i < int(rows.size()); i++)
        {
            int h = -1;
            for (int col : rows[i])
            {
                assert(0 <= col && col < m && seen[col] != i);
                seen[col] = i;
                int c = col + 1, x = a.size(), u = a[c].u;
                a.push_back({x, x, u, c, c, i});
                a[u].d = x;
                a[c].u = x;
                s[c]++;
                if (h == -1) h = x;
                else
                {
                    int l = a[h].l;
                    a[x].l = l;
                    a[x].r = h;
                    a[l].r = x;
                    a[h].l = x;
                }
            }
        }
    }

    // x is a selected row node, not a column header.
    void remove(int x)
    {
        for (int i = a[x].d; i != x; i = a[i].d)
        {
            a[a[i].l].r = a[i].r;
            a[a[i].r].l = a[i].l;
        }
    }
```

逆序恢复与可采纳下界（接上页同一结构体）：

```
    void restore(int x)
    {
        for (int i = a[x].u; i != x; i = a[i].u)
        {
            a[a[i].l].r = i;
            a[a[i].r].l = i;
        }
    }

    int lower_bound()
    {
        fill(mark.begin(), mark.end(), 0);
        for (int c = a[0].r; c; c = a[c].r) mark[c] = 1;
        int bound = 0;
        for (int c = a[0].r; c; c = a[c].r)
        {
            if (!mark[c]) continue;
            bound++;
            mark[c] = 0;
            for (int i = a[c].d; i != c; i = a[i].d)
                for (int j = a[i].r; j != i; j = a[j].r)
                    mark[a[j].c] = 0;
        }
        return bound;
    }
```

最优搜索与求解入口（接上页同一结构体）：

```
    void dfs()
    {
        int depth = path.size(), c = a[0].r;
        if (!c)
        {
            if (depth < best)
            {
                best = depth;
                ans = path;
            }
            return;
        }
        if (lower_bound() >= best - depth) return;
        for (int j = a[c].r; j; j = a[j].r)
            if (s[j] < s[c]) c = j;
        for (int i = a[c].d; i != c; i = a[i].d)
        {
            path.push_back(a[i].row);
            remove(i);
            for (int j = a[i].r; j != i; j = a[j].r) remove(j);
            dfs();
            for (int j = a[i].l; j != i; j = a[j].l) restore(j);
            restore(i);
            path.pop_back();
        }
    }

    optional<vector<int>> solve()
    {
        path.clear();
        ans.clear();
        best = m + 1;
        dfs();
        if (best > m) return nullopt;
        return ans;
    }
};
```

接口用法演示：最少行重复覆盖（本地接口示例）

演示约定：读入 n 个稀疏集合和 m 列；每行先给元素数，再给互异的 0-based 列号。输出最少选行数及原 0-based 行号，使每列至少出现一次；无解输出-1，m=0 输出 0 与空方案。此为接口演示，不冒充线上模板题。

所需模板：MinimumCover（第 9 页，2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/search/dlx/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<int>> rows(n);
    for (auto &row : rows)
    {
        int k;
        cin >> k;
        row.resize(k);
        for (int &c : row) cin >> c;
    }
    MinimumCover cover(m, rows);
    auto ans = cover.solve();
    if (!ans) cout << -1 << '\n';
    else
    {
        cout << ans->size() << '\n';
        for (int row : *ans) cout << row << ' ';
        cout << '\n';
    }
}
```

第二部分

动态规划

第三章　数位 DP

3.1　数位 DP：十进制数码计数

```cpp
array<__int128, 10> digit_count(long long n)
{
    struct State
    {
        __int128 ways = 0;
        array<__int128, 10> count{};
    };
    if (n <= 0) return {};
    string s = to_string(n);
    vector<array<State, 2>> memo(s.size());
    vector<array<bool, 2>> seen(s.size());
    auto dfs = [&](auto &&self, int pos, bool tight, bool started) -> State
    {
        if (pos == (int)s.size())
        {
            State result;
            result.ways = started;
            return result;
        }
        if (!tight && seen[pos][started]) return memo[pos][started];
        State result;
        int limit = tight ? s[pos] - '0' : 9;
        for (int d = 0; d <= limit; d++)
        {
            bool next = started || d != 0;
            State child = self(self, pos + 1, tight && d == limit, next);
            result.ways += child.ways;
            for (int x = 0; x < 10; x++)
                result.count[x] += child.count[x];
            if (next) result.count[d] += child.ways;
        }
        if (!tight)
        {
            seen[pos][started] = true;
            memo[pos][started] = result;
        }
        return result;
    };
    return dfs(dfs, 0, true, false).count;
}
```

digit_count(n) 返回正整数 1..n 中十种十进制数码的出现次数；n≤0 返回全零，整数 0 本身不计入。支持 signed long long 的正数全范围，返回 array<__int128,10>，不可在完整范围下擅自转成 64 位。区间 [a,b]（1≤a≤b）用两次前缀结果相减。

递归状态 pos、tight、started：tight 表示前缀仍贴上界；started 为假时前导零不计数，终点全前导零不算一个正整数。State 同时累计方案数 ways 和数码次数 count；选定实际数码 d 后增加 child.ways。只记忆化非 tight 状态，每次调用独立创建 memo/seen，可反复处理不同上界。L 为位数，时间 O(100L)、空间 O(10L)、递归深度 O(L)，L≤19。P2602 的 b≤10^12，答案小于 13×10^12，可转 long long 输出；更大上界需 128 位输出。P2602 当前代码已 AC，10 个测试点，单点 4ms；排名未核查。

应用题补充：P2602 [ZJOI2010] 数字计数

题意：输入 1≤a≤b≤10^12，输出区间中 0..9 的出现次数；前缀相减，前导零不计，两个上界独立记忆化。该题范围内每个计数可转 long long。

所需模板：digit_count（第 15 页，3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2602

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    long long a, b;
    cin >> a >> b;
    auto left = digit_count(a - 1);
    auto right = digit_count(b);
    for (int d = 0; d < 10; d++)
        cout << (long long)(right[d] - left[d]) << " \n"[d == 9];
}
```

第四章　插头 DP

4.1　插头 DP：网格单闭环计数

grid_cycle<Count>(a) 统计覆盖所有可用格子的单个无向简单环；矩形网格中. 必须经过、* 不能经过，仅上下左右相邻。空网格/全障碍返回 0。自动转置到较窄方向，宽度 m≤15，尺寸须可用 int 表示。状态每槽 4 位，同编号表示同一路径的两个端点，按首次出现规范化；闭环只允许发生在最后一个可用格子且无其他插头时。

Count 支持 0/1 构造和 +=；固定整数必须保证所有中间计数不溢出，可改模整数或 cpp_int。S 为最多可达状态数，固定宽度计数的期望时间 O(n*m^2*S)、状态表 O(S)，另有网格 O(nm)。S 随宽度指数增长；cpp_int 还需计入位复杂度。

```
template <class Count>
Count grid_cycle(vector<string> a)
{
    int n = a.size();
    if (n == 0) return Count(0);
    int m = a[0].size();
    for (auto &row : a)
    {
        assert((int)row.size() == m);
        for (char c : row) assert(c == '.' || c == '*');
    }
    if (m == 0) return Count(0);
    if (n < m)
    {
        vector<string> b(m, string(n, '*'));
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++) b[j][i] = a[i][j];
        a.swap(b);
        swap(n, m);
    }
    assert(m <= 15);
    pair<int, int> last = {-1, -1};
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (a[i][j] == '.') last = {i, j};
    using Key = unsigned long long;
    unordered_map<Key, Count> dp, next;
    dp[0] = Count(1);
    Count answer = 0;
    for (int i = 0; i < n; i++)
    {
        next.clear();
        for (auto &[s, ways] : dp) next[s << 4] += ways;
        dp.swap(next);
```

轮廓转移（接上页同一函数）：

```
    for (int j = 0; j < m; j++)
    {
        next.clear();
        bool down = i + 1 < n && a[i + 1][j] == '.';
        bool right = j + 1 < m && a[i][j + 1] == '.';
        for (auto &[s, ways] : dp)
        {
            vector<int> p(m + 1);
            for (int k = 0; k <= m; k++) p[k] = s >> (4 * k) & 15;
            int x = p[j];
            int y = p[j + 1];
            p[j] = 0;
            p[j + 1] = 0;
            auto add = [&]()
            {
                array<int, 16> id{};
                int count = 0;
                Key key = 0;
                for (int k = 0; k <= m; k++)
                {
                    int v = p[k];
                    if (v && !id[v]) id[v] = ++count;
                    key |= Key(id[v]) << (4 * k);
                }
                next[key] += ways;
            };
            if (a[i][j] == '*')
            {
                if (!x && !y) add();
            }
            else if (!x && !y)
            {
                if (down && right)
                {
                    p[j] = 15;
                    p[j + 1] = 15;
                    add();
                }
            }
```

延伸、合并与最终闭环（接上页同一函数）：

```
                else if (!x || !y)
                {
                    if (down)
                    {
                        p[j] = x + y;
                        add();
                        p[j] = 0;
                    }
                    if (right)
                    {
                        p[j + 1] = x + y;
                        add();
                    }
                }
                else if (x != y)
                {
                    for (int &v : p)
                        if (v == y) v = x;
                    add();
                }
                else if (make_pair(i, j) == last)
                {
                    if (all_of(p.begin(), p.end(), [](int v)
                    {
                        return v == 0;
                    })) answer += ways;
                }
            }
            dp.swap(next);
        }
    }
    return answer;
}
```

轮廓与不变量 槽 j、j+1 是当前格的左、上插头，转移后成为下、右插头。处理过的可用点度数固定为 2，每个未闭合连通块是一条路径，在轮廓上恰有两个同编号端点。行首左移 4 位；右边界插头恒为 0，所以移位不丢有效信息。最多 8 条路径，临时编号 15 不会与规范编号冲突。

转移与去重 无入边时必须创建下、右两条出边；单入边选择延伸方向；不同编号的双入边合并路径并统一编号；相同编号表示闭环，只能在最后一个可用点且其他槽全零时接收。按首次出现重编号使相同连通关系唯一。每个边集唯一对应逐格选择，不按起点或方向重复计数。

数值与验证 固定整数要保证所有中间计数可表示，不能只检查最后答案。P5056 仅内部 (n-1)(m-1) 个格子可能二选一，其余转移至多一个。所有中间计数不超过 2 的 121 次方，因此下面用 128 位整数仍能保证精确计数。已穷举 74954 个小障碍网格并与独立递归环搜索核对；完整 2 至 14 阶偶数方格与 OEIS A003763 核对。输入超过题目范围时状态数仍可能很大。

模板题使用：P5056 插头 DP

题意：2≤n,m≤12，所有点格必须形成唯一闭环。仅同时存在下、右邻格时可能二选一，至多 $(12\text{-}1)^2$=121 次，因此全部中间计数 ≤2^121，用 ___int128 安全；十进制输出。

所需模板：grid_cycle（第 17 页，4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5056

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<string> a(n);
    for (auto &row : a) cin >> row;
    __int128 answer = grid_cycle<__int128>(a);
    string out;
    do
    {
        out += char('0' + answer % 10);
        answer /= 10;
    } while (answer);
    reverse(out.begin(), out.end());
    cout << out << '\n';
}
```

4.2 插头 DP：恰好 K 个不嵌套回路

grid_cycles<Count>(a,k,nested=false) 统计覆盖全部. 格子的恰好 k 个无向简单环；* 为障碍，只有上下左右相邻。默认禁止任意环包含另一个环；nested=true 允许嵌套。无可用格且 k=0 时返回 1，其余无法覆盖时返回 0。k 为非负 int 且 k<INT_MAX；矩形尺寸可用 int 表示，自动转置后宽度 m≤15。

同编号的两个插头表示同一路径端点，规范化编号。闭环时环数加 1；禁止嵌套时检查当前格左侧插头数为偶数。每个环在其最后扫描顶点闭合，该奇偶性是环的嵌套深度奇偶；若存在嵌套，必有深度 1 的环被拒绝。只在扫完全部格子、无剩余插头时取答案。

Count 需支持 0/1 构造、默认构造为 0、复制和 +=；所有中间计数均须可表示，可用模整数或 cpp_int。S 为固定环数下最多轮廓状态数，固定宽度计数下期望时间 $O(n*m^2*(k+1)*(S+1))$，两层状态表 $O((k+1)*(S+1))$，网格 $O(nm)$。S 随宽度指数增长；高精度计数另计位复杂度。

```
template <class Count>
Count grid_cycles(vector<string> a, int k, bool nested = false)
{
    assert(k >= 0);
    int n = a.size();
    if (n == 0) return Count(k == 0);
    int m = a[0].size();
    long long cells = 0;
    for (auto &row : a)
    {
        assert((int)row.size() == m);
        for (char c : row)
        {
            assert(c == '.' || c == '*');
            cells += c == '.';
        }
    }
    if (cells == 0) return Count(k == 0);
    if (k == 0 || k > cells / 4) return Count(0);
    if (n < m)
    {
        vector<string> b(m, string(n, '*'));
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++) b[j][i] = a[i][j];
        a.swap(b);
        swap(n, m);
    }
    assert(m <= 15);
    using Key = unsigned long long;
    vector<unordered_map<Key, Count>> dp(k + 1), next(k + 1);
    dp[0][0] = Count(1);
```

行移位、规范化与创建路径（接上页同一函数）：

```
    for (int i = 0; i < n; i++)
    {
        for (int c = 0; c <= k; c++)
        {
            next[c].clear();
            for (auto &[s, ways] : dp[c]) next[c][s << 4] += ways;
        }
        dp.swap(next);
        for (int j = 0; j < m; j++)
        {
            for (auto &t : next) t.clear();
            bool down = i + 1 < n && a[i + 1][j] == '.';
            bool right = j + 1 < m && a[i][j + 1] == '.';
            for (int c = 0; c <= k; c++)
            {
                for (auto &[s, ways] : dp[c])
                {
                    array<int, 16> p{};
                    for (int t = 0; t <= m; t++) p[t] = s >> (4 * t) & 15;
                    int x = p[j];
                    int y = p[j + 1];
                    p[j] = 0;
                    p[j + 1] = 0;
                    auto add = [&](int count)
                    {
                        array<int, 16> id{};
                        int top = 0;
                        Key key = 0;
                        for (int t = 0; t <= m; t++)
                        {
                            int v = p[t];
                            if (v && !id[v]) id[v] = ++top;
                            key |= Key(id[v]) << (4 * t);
                        }
                        next[count][key] += ways;
                    };
                    if (a[i][j] == '*')
                    {
                        if (!x && !y) add(c);
                    }
                    else if (!x && !y)
                    {
                        if (down && right)
                        {
                            p[j] = 15;
                            p[j + 1] = 15;
                            add(c);
                        }
                    }
```

延伸、合并与闭环计数（接上页同一函数）：

```
                    else if (!x || !y)
                    {
                        if (down)
                        {
                            p[j] = x + y;
                            add(c);
                            p[j] = 0;
                        }
                        if (right)
                        {
                            p[j + 1] = x + y;
                            add(c);
                        }
                    }
                    else if (x != y)
                    {
                        for (int &v : p)
                            if (v == y) v = x;
                        add(c);
                    }
                    else if (c < k)
                    {
                        int parity = 0;
                        for (int t = 0; t < j; t++) parity ^= p[t] != 0;
                        if (nested || parity == 0) add(c + 1);
                    }
                }
            }
            dp.swap(next);
        }
    }
    auto it = dp[k].find(0);
    return it == dp[k].end() ? Count(0) : it->second;
}
```

应用题补充：HDU4285 circuits

题意：官方竞赛应用：多组 2≤n,m≤12 网格，恰好 K 个互不嵌套回路覆盖所有可用格，答案模 10^9+7。官方样例为 2、6；不是单环模板的旧 AC。当前最小抄写版 41022698 已 AC，11216MS、27236K。

所需模板：grid_cycles（第 21 页，4.2 节）、ModInt（第 133 页，19.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://acm.hdu.edu.cn/showproblem.php?pid=4285

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        int n, m, k;
        cin >> n >> m >> k;
        vector<string> a(n);
        for (auto &row : a) cin >> row;
        cout << grid_cycles<ModInt<1000000007>>(a, k).v << '\n';
    }
}
```

接口用法演示：多环覆盖与嵌套开关（精确计数）

演示约定：输入 n,m,k,nested 与网格，nested=0 禁止嵌套，非 0 允许嵌套；使用 cpp_int 避免中间计数溢出，需要 Boost 头文件。空网格 k=0 有一个空方案。此驱动为接口演示。

所需模板：grid_cycles（第 21 页，4.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

额外依赖（不属于标准头文件）：`boost/multiprecision/cpp_int.hpp`，须由编译环境提供并显式包含。

来源：参考文档

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k, nested;
    cin >> n >> m >> k >> nested;
    vector<string> a(n);
    for (auto &row : a) cin >> row;
    using Count = boost::multiprecision::cpp_int;
    cout << grid_cycles<Count>(a, k, nested != 0) << '\n';
}
```

第五章　DP 优化

5.1　斜率优化

5.1.1　单调斜率凸包：最小值与决策

默认空；add(m,b,id) 按斜率非增加入直线 mx+b，query(x) 按 x 非减查询，返回 (pair<int128,int>)(最小值, 编号)。首次查询前至少加一条线；可交替加线与查询。并列保留最早加入的直线，不是最小 id；同斜率只留最小截距，截距也同则留旧线。q/head 仅供只读；查询会永久淘汰旧候选，不支持回退 x 或任意斜率。要求 |m|,|b|,|x|<=10^18，总加线数 <INT_MAX。先提升 int128 再做差与交叉乘法，范围内精确无浮点；返回值勿未经证明窄化为 long long。总 M 次加线、Q 次查询 O(M+Q)，空间 O(M)，被查询淘汰的前缀仍留在 vector 中。

```
struct MonotoneHull
{
    using ll = long long;
    using I = __int128_t;
    struct Line
    {
        ll m, b;
        int id;

        I value(ll x) const
        {
            return I(m) * x + b;
        }
    };
    vector<Line> q;
    int head = 0;
    ll last_x = 0;
    bool queried = false;

    void add(ll m, ll b, int id)
    {
        assert(q.empty() || m <= q.back().m);
        if (int(q.size()) > head && q.back().m == m)
        {
            if (q.back().b <= b) return;
            q.pop_back();
        }
        Line c{m, b, id};
        while (int(q.size()) - head >= 2)
        {
            auto a = q[q.size() - 2];
            auto b = q.back();
            if ((I(b.b) - a.b) * (b.m - I(c.m)) <
                (I(c.b) - b.b) * (a.m - I(b.m))) break;
            q.pop_back();
        }
        q.push_back(c);
    }
```

查询（接上页同一结构体）：

```
    pair<I, int> query(ll x)
    {
        assert(int(q.size()) > head);
        assert(!queried || last_x <= x);
        queried = true;
        last_x = x;
        while (head + 1 < int(q.size()) && q[head + 1].value(x) < q[head].value(x))
        {
            head++;
        }
        return {q[head].value(x), q[head].id};
    }
};
```

应用题补充：P2365 任务安排

题意：IOI2002 任务安排：保持任务顺序分批，最小化完成时刻乘费用系数的总和。n<=5000，s<=50，t 和 f 在 1..100。前缀 T/C，dp[i]=T[i]*C[i]+s*C[n]+min(dp[j]-(T[i]+s)*C[j])；斜率和查询方向均满足单调，答案及截距在 long long 范围内。

所需模板：MonotoneHull（第 25 页，5.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2365

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long s;
    cin >> n >> s;
    vector<long long> t(n + 1), c(n + 1);
    for (int i = 1; i <= n; i++)
    {
        cin >> t[i] >> c[i];
        t[i] += t[i - 1];
        c[i] += c[i - 1];
    }
    MonotoneHull hull;
    hull.add(0, 0, 0);
    long long dp = 0;
    for (int i = 1; i <= n; i++)
    {
        dp = hull.query(t[i] + s).first + t[i] * c[i] + s * c[n];
        hull.add(-c[i], dp, i);
    }
    cout << dp << '\n';
}
```

5.2 四边形不等式优化

5.2.1 分治决策单调性：一层分组 DP

monotone_dp_layer(prev,first,cost,opt) 返回 cur，prev 下标 0..n，1<=first<=n<INT_MAX。对 i=first..n 求 min(prev[j]+cost(j+1,i))，j 在 first-1..i-1，cost 是闭区间代价；用于第 g 组时 first=g。inf=LLONG_MAX/4；prev[first-1] 必须可达，其他 inf 跳过，cost 对全部合法区间有限。所有有限 prev、cost 及转移和须严格在 (-inf,inf)，保证 long long 加法安全。cur[0..first-1]=inf，opt 对应为-1；其余 opt 返回最小最优 j。调用方须证明该并列规则下 opt[i] 非减，本函数不会检查，任意 DP 不能套用。cost 须无副作用且 O(1) 才能得 O(n log(n+1)) 单层时间；递归深度 O(log n)，输出与辅助 O(n)。

```
template<class Cost>
vector<long long> monotone_dp_layer(const vector<long long> &prev, int first,
                                    Cost cost, vector<int> &opt)
{
    using ll = long long;
    constexpr ll inf = LLONG_MAX / 4;
    int n = int(prev.size()) - 1;
    assert(1 <= first && first <= n && prev[first - 1] != inf);
    vector<ll> cur(n + 1, inf);
    opt.assign(n + 1, -1);
    auto solve = [&](auto &&self, int l, int r, int lo, int hi) -> void
    {
        if (l > r) return;
        int mid = l + (r - l) / 2;
        for (int j = lo; j <= min(hi, mid - 1); j++)
        {
            if (prev[j] == inf) continue;
            ll value = prev[j] + cost(j + 1, mid);
            if (value < cur[mid])
            {
                cur[mid] = value;
                opt[mid] = j;
            }
        }
        assert(opt[mid] != -1);
        self(self, l, mid - 1, lo, opt[mid]);
        self(self, mid + 1, r, opt[mid], hi);
    };
    solve(solve, first, n, first - 1, n - 1);
    return cur;
}
```

应用题补充：CF321E Ciel and Gondolas

题意：Ciel and Gondolas：n<=4000，k<=800，非负对称矩阵对角 0；恰分 k 个非空连续组，最小化组内无序对权和。二维前缀和 O(1) 求闭区间代价；四边形不等式保证最小最优决策非减。

所需模板：monotone_dp_layer（第 27 页，5.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/CF321E

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    vector<vector<int>> sum(n + 1, vector<int>(n + 1));
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= n; j++)
        {
            char c;
            cin >> c;
            sum[i][j] = c - '0' + sum[i - 1][j] + sum[i][j - 1] - sum[i - 1][j - 1];
        }
    }
    auto cost = [&](int l, int r) -> long long
    {
        return (sum[r][r] - sum[l - 1][r] - sum[r][l - 1] + sum[l - 1][l - 1]) / 2;
    };
    vector<long long> dp(n + 1, LLONG_MAX / 4);
    vector<int> opt;
    dp[0] = 0;
    for (int g = 1; g <= k; g++)
    {
        dp = monotone_dp_layer(dp, g, cost, opt);
    }
    cout << dp[n] << '\n';
}
```

应用题补充：P4767 [IOI 2000] 邮局加强版

题意：n<=3000 个村庄，k<=300 个邮局，坐标 1..10000；最小化到最近邮局距离之和。先排序；每个连续组在中位数处建邮局，前缀和 O(1) 求组代价。该代价满足四边形不等式，使用恰分 k 组 DP。

所需模板：monotone_dp_layer（第 27 页，5.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4767

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    vector<long long> x(n + 1), sum(n + 1);
    for (int i = 1; i <= n; i++) cin >> x[i];
    sort(x.begin() + 1, x.end());
    for (int i = 1; i <= n; i++) sum[i] = sum[i - 1] + x[i];
    auto cost = [&](int l, int r) -> long long
    {
        int mid = (l + r) / 2;
        return x[mid] * (mid - l + 1) - (sum[mid] - sum[l - 1]) +
               (sum[r] - sum[mid]) - x[mid] * (r - mid);
    };
    vector<long long> dp(n + 1, LLONG_MAX / 4);
    vector<int> opt;
    dp[0] = 0;
    for (int g = 1; g <= k; g++)
    {
        dp = monotone_dp_layer(dp, g, cost, opt);
    }
    cout << dp[n] << '\n';
}
```

5.3 WQS 二分

5.3.1 WQS 二分：至多 k 个不相邻位置

wqs_independent_set(a,k) 返回 0-based 数组中至多选 k 个两两不相邻位置的最大权和；允许空选、空数组、负权，k>=0。n=a.size()<INT_MAX，W=max|a[i]|，要求 n*W<=LLONG_MAX/4（以足够宽类型核验），因此不含 LLONG_MIN。只返回最优值，不返回方案。路径独立集等价于路径边匹配，定基数最优值离散凹；每次选择扣 penalty，pair 比较先权和、并列取更多位置。若罚 0 的最优数量 <=k 直接返回；否则二分仍能选至少 k 个的最大非负整数罚值，答案为惩罚后最优值 +penalty*k。不是任意 DP 的 WQS 通用封装。O(n log(W+2)) 时间、O(1) 额外空间。

```
long long wqs_independent_set(const vector<long long> &a, int k)
{
    using ll = long long;
    assert(k >= 0);
    if (k == 0 || a.empty()) return 0;
    auto solve = [&](ll penalty) -> pair<ll, int>
    {
        pair<ll, int> prev{0, 0}, cur{0, 0};
        for (ll w : a)
        {
            pair<ll, int> take{prev.first + w - penalty, prev.second + 1};
            prev = cur;
            cur = max(cur, take);
        }
        return cur;
    };
    auto free = solve(0);
    if (free.second <= k) return free.first;
    ll lo = 0, hi = max(0LL, *max_element(a.begin(), a.end()));
    while (lo < hi)
    {
        ll mid = lo + (hi - lo + 1) / 2;
        if (solve(mid).second >= k) lo = mid;
        else hi = mid - 1;
    }
    return solve(lo).first + lo * k;
}
```

应用题补充：P1484 种树（WQS）

题意：种树：至多选 k 个不相邻坑，最大获利；允许负数。n<=300000，k<=n/2，|a[i]|<=10^6。WQS 的凹性证明依赖路径匹配模型。

所需模板：wqs_independent_set（第 30 页，5.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1484

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    cout << wqs_independent_set(a, k) << '\n';
}
```

第三部分

字符串

第六章　字符串哈希

6.1　子串与反串双哈希

StringHash(s, base)，n<INT_MAX，两个底数分别在 [257,mod-2]，所有比较对象共享同组底数。get(l, r, rev=false) 取 0-based 半开区间；rev=true 反转原区间，空区间返回 {0,0}。join(a, b, len_b) 拼接，要求规范余数、同底数及 0≤len_b≤n，可拼出长度超过 n 的哈希。构造 O(n)，查询/拼接 O(1)，约 24(n+1) 字节；按 unsigned char 加 1 编码。判等必须先比长度；双哈希仍可能碰撞。理想独立均匀底数下，固定同长不同串的碰撞上界为两个 min(1,(L-1)/(mod[j]-258)) 之积，多次比较用联合界；可预测种子和适应性输入不享有该保证。需确定性时用直接比较或 SuffixArray + SuffixLCP；回文可用 Manacher。成员只读。

```
struct StringHash
{
    using H = array<int, 2>;
    static constexpr H mod = {1000000007, 1000000009};
    int n;
    H base;
    vector<H> pw, pre, rev;

    StringHash(const string &s, H b)
        : n(s.size()), base(b), pw(n + 1), pre(n + 1), rev(n + 1)
    {
        pw[0] = {1, 1};
        for (int j = 0; j < 2; j++)
        {
            assert(257 <= base[j] && base[j] <= mod[j] - 2);
            for (int i = 0; i < n; i++)
            {
                int x = (unsigned char)s[i] + 1;
                int y = (unsigned char)s[n - 1 - i] + 1;
                pw[i + 1][j] = 1LL * pw[i][j] * base[j] % mod[j];
                pre[i + 1][j] = (1LL * pre[i][j] * base[j] + x) % mod[j];
                rev[i + 1][j] = (1LL * rev[i][j] * base[j] + y) % mod[j];
            }
        }
    }
```

子串、反串与拼接（接上页同一结构体）：

```
    H get(int l, int r, bool reversed = false) const
    {
        assert(0 <= l && l <= r && r <= n);
        if (reversed)
        {
            int x = n - r;
            r = n - l;
            l = x;
        }
        const auto &h = reversed ? rev : pre;
        H ans;
        for (int j = 0; j < 2; j++)
        {
            long long x = h[r][j] - 1LL * h[l][j] * pw[r - l][j] % mod[j];
            ans[j] = (x + mod[j]) % mod[j];
        }
        return ans;
    }

    H join(H a, H b, int len_b) const
    {
        assert(0 <= len_b && len_b <= n);
        for (int j = 0; j < 2; j++)
        {
            a[j] = (1LL * a[j] * pw[len_b][j] + b[j]) % mod[j];
        }
        return a;
    }
};
```

模板题使用：P3370 【模板】字符串哈希

题意：输入 N 个非空字母数字串，输出不同串数。随机底数全局共享，按长度与双哈希去重；概率判等，AC 不证明零碰撞。

所需模板：StringHash（第 33 页，6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3370

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    mt19937 rng(random_device{}());
    StringHash::H base;
    for (int j = 0; j < 2; j++)
    {
        base[j] = uniform_int_distribution<int>(257, StringHash::mod[j] - 2)(rng);
    }
    int n;
    cin >> n;
    set<pair<int, StringHash::H>> seen;
    while (n--)
    {
        string s;
        cin >> s;
        StringHash h(s, base);
        seen.insert({int(s.size()), h.get(0, s.size())});
    }
    cout << seen.size() << '\n';
}
```

接口用法演示：API 子串、反串与拼接

演示约定：首行字符串可空，再输入两分量底数、q；每次 l r a b 为原串半开区间，输出 [l,r) 正向/反向、[l,r)+[a,b) 的双哈希。底数显式输入便于重现，不是固定底数推荐。

所需模板：StringHash（第 33 页，6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/string/hash/

```
int main()
{
    string s;
    getline(cin, s);
    StringHash::H base;
    cin >> base[0] >> base[1];
    StringHash h(s, base);
    int q;
    cin >> q;
    while (q--)
    {
        int l, r, a, b;
        cin >> l >> r >> a >> b;
        auto x = h.get(l, r);
        auto y = h.get(l, r, true);
        auto z = h.join(x, h.get(a, b), b - a);
        cout << x[0] << ' ' << x[1] << '\n';
        cout << y[0] << ' ' << y[1] << '\n';
        cout << z[0] << ' ' << z[1] << '\n';
    }
}
```

第七章　前缀函数与 KMP 算法

7.1　前缀函数

prefix_function(s) 返回每个前缀的最长真 border 长度，0-based；空串返回空数组。O(n)，无状态，直接抄写函数。

```
vector<int> prefix_function(const string &s)
{
    int n = (int)s.size();
    vector<int> p(n);
    for (int i = 1; i < n; i++)
    {
        int j = p[i - 1];
        while (j && s[i] != s[j]) j = p[j - 1];
        p[i] = j + (s[i] == s[j]);
    }
    return p;
}
```

联合使用示例见第 40 页（Luogu P3375），其中直接调用本模板。

接口用法演示：优化 KMP 失败表 nextval 与重叠匹配

演示约定：多组直到 EOF：输入文本长度 n≥0、非空模式长度 1≤m<INT_MAX，随后两串各自的 0..255 字节整数。每组输出 nextval[0..m]、匹配数量、0-based 起点三行。对 i<m 跳过下一个模式字符仍相同的失败状态；nextval[m] 必须取普通最长 border，保留重叠。无需尾零，支持零字节。O(n+m) 时间，O(m+ 答案数) 额外空间；来源接口演示，不是线上模板题。

所需模板：prefix_function（第 37 页，7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/kuangbin/ACM-ICPC

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    while (cin >> n >> m)
    {
        assert(0 <= n && 1 <= m && m < INT_MAX);
        string s(n, 0), t(m, 0);
        for (char &c : s)
        {
            int x;
            cin >> x;
            assert(0 <= x && x < 256);
            c = x;
        }
        for (char &c : t)
        {
            int x;
            cin >> x;
            assert(0 <= x && x < 256);
            c = x;
        }
        auto p = prefix_function(t);
        vector<int> nextval(m + 1, -1);
        for (int i = 1; i < m; i++)
        {
            int j = p[i - 1];
            nextval[i] = t[i] == t[j] ? nextval[j] : j;
        }
        nextval[m] = p[m - 1];
        for (int x : nextval)
            cout << x << ' ';
        cout << '\n';
```

继续处理操作（接上页同一 main）：

```
        vector<int> positions;
        int j = 0;
        for (int i = 0; i < n; i++)
        {
            while (j != -1 && s[i] != t[j])
                j = nextval[j];
            j++;
            if (j == m)
            {
                positions.push_back(i - m + 1);
                j = nextval[m];
            }
        }
        cout << positions.size() << '\n';
        for (int x : positions)
            cout << x << ' ';
        cout << '\n';
    }
    return 0;
}
```

7.2 KMP 匹配

代码依赖：prefix_function（第 37 页，7.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

kmp_match(s,t) 返回非空模式串 t 在 s 中的所有起点（0-based），允许重叠。需要同章前缀函数。O(|s|+|t|)。

```
vector<int> kmp_match(const string &s, const string &t)
{
    assert(!t.empty());
    auto p = prefix_function(t);
    vector<int> ans;
    for (int i = 0, j = 0; i < (int)s.size(); i++)
    {
        while (j && s[i] != t[j]) j = p[j - 1];
        j += (s[i] == t[j]);
        if (j == (int)t.size())
        {
            ans.push_back(i - j + 1);
            j = p[j - 1];
        }
    }
    return ans;
}
```

模板题使用：Luogu P3375

题意：求模式串 t 在文本 s 中所有出现的位置，包含重叠匹配；模板返回 0-based 位置，输出加 1。最后一行直接输出 prefix_function(t)，它是模式串各前缀的最长真 border 长度。题目输入为非空大写串，函数按字符相等比较，不限于小写。

所需模板：prefix_function（第 37 页，7.1 节）、kmp_match（第 39 页，7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3375

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s, t;
    cin >> s >> t;
    for (int p : kmp_match(s, t)) cout << p + 1 << '\n';
    for (int x : prefix_function(t)) cout << x << ' ';
    cout << '\n';
}
```

相关应用：大小和相等关系的顺序同构匹配；KMP 推广。

7.3 顺序同构匹配

order_match(s,t) 接收 vector<long long>，t 非空；返回所有 0-based 起点，含重叠。匹配要求任意两位置的大小及相等关系一致，数值本身可不同。支持完整 signed64，仅比较不作差；输入不变，长度均 <INT_MAX。模式预处理记录之前的相等、前驱和后继位置，再用相对下标检查 KMP 扩展。O(|t|log(|t|+1)+|s|) 时间，O(|t|+答案数) 空间。P6080 为应用题，不作为正式模板题覆盖。

```
// Match all pairwise <, =, > relations; t must be nonempty.
vector<int> order_match(const vector<long long> &s, const vector<long long> &t)
{
    assert(!t.empty());
    int n = s.size(), m = t.size();
    vector<int> eq(m, -1), lo(m, -1), hi(m, -1), p(m), ans;
    map<long long, int> last;
    for (int j = 0; j < m; j++)
    {
        auto it = last.lower_bound(t[j]);
        if (it != last.end() && it->first == t[j])
            eq[j] = it->second;
        else
        {
            if (it != last.end()) hi[j] = it->second;
            if (it != last.begin()) lo[j] = prev(it)->second;
        }
        last[t[j]] = j;
    }
    auto fits = [&](const vector<long long> &a, int i, int j)
    {
        int start = i - j;
        if (eq[j] != -1) return a[i] == a[start + eq[j]];
        if (lo[j] != -1 && a[i] <= a[start + lo[j]]) return false;
        if (hi[j] != -1 && a[i] >= a[start + hi[j]]) return false;
        return true;
    };
    for (int i = 1, j = 0; i < m; i++)
    {
        while (j && !fits(t, i, j)) j = p[j - 1];
        if (fits(t, i, j)) j++;
        p[i] = j;
    }
    for (int i = 0, j = 0; i < n; i++)
    {
        while (j && !fits(s, i, j)) j = p[j - 1];
        if (fits(s, i, j)) j++;
        if (j == m)
        {
            ans.push_back(i - m + 1);
            j = p[j - 1];
        }
    }
    return ans;
}
```

应用题补充：Luogu P6080 / Cow Patterns

题意：N≤100000、K≤25000、S≤25，值在 1..S；输出匹配的个数与升序 1-based 起点，逐行输出。保持相等和严格大小关系，允许重叠；S 无需用于算法。应用补充，未在线 AC。

所需模板：order_match（第 41 页，7.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6080

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, alphabet;
    cin >> n >> m >> alphabet;
    vector<long long> s(n), t(m);
    for (auto &x : s) cin >> x;
    for (auto &x : t) cin >> x;
    auto ans = order_match(s, t);
    cout << ans.size() << '\n';
    for (int x : ans) cout << x + 1 << '\n';
}
```

第八章　Z 函数（扩展 KMP）

8.1　Z 函数

z_function(s) 返回后缀 s[i..] 与原串的 LCP，约定 z[0]=n，空串返回空数组。O(n)。

```
vector<int> z_function(const string &s)
{
    int n = (int)s.size();
    vector<int> a(n);
    if (n) a[0] = n;
    for (int i = 1, l = 0, r = 0; i < n; i++)
    {
        if (i < r) a[i] = min(r - i, a[i - l]);
        while (i + a[i] < n && s[a[i]] == s[i + a[i]]) ++a[i];
        if (i + a[i] > r)
        {
            l = i;
            r = i + a[i];
        }
    }
    return a;
}
```

模板题使用：Library Checker zalgorithm

题意：给非空字符串，对每个起点求其后缀与原串的最长公共前缀长度。直接输出 Z 数组，z[0] 等于串长。

所需模板：z_function（第 43 页，8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/zalgorithm

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    auto z = z_function(s);
    for (int i = 0; i < (int)z.size(); i++)
        cout << z[i] << (i + 1 == (int)z.size() ? '\n' : ' ');
}
```

8.2 两串扩展 KMP

代码依赖：z_function（第 43 页，8.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

exkmp(s,t) 返回 {z,p}：z 为模式 t 的 Z 数组，z[0]=|t|；p[i] 是文本后缀 s[i..] 与 t 的 LCP 长度，均 0-based。空串合法，t 空时 p 全 0，s 空时 p 为空。任意字节相等比较，含零字节与高位字节；不拼接分隔符、不拷贝输入。需要 z_function。两串长度均须 <INT_MAX；O(|s|+|t|) 时间，两个输出 vector 共 O(|s|+|t|) 空间，返回时移动缓冲区。维护右端最远的匹配区间 [l,r)，区间内利用模式自身 Z 值再向右扩展。P5410 的 1-based 权值乘法应先提升到 long long。

```
// Return {Z(t), LCP(s[i..], t) for every i}; empty strings are allowed.
pair<vector<int>, vector<int>> exkmp(const string &s, const string &t)
{
    int n = s.size(), m = t.size();
    auto z = z_function(t);
    vector<int> p(n);
    for (int i = 0, l = 0, r = 0; i < n; i++)
    {
        if (i < r) p[i] = min(r - i, z[i - l]);
        while (p[i] < m && p[i] < n - i && s[i + p[i]] == t[p[i]])
            p[i]++;
        if (i + p[i] > r)
        {
            l = i;
            r = i + p[i];
        }
    }
    return {move(z), move(p)};
}
```

模板题使用：Luogu P5410 / 扩展 KMP

题意：输入小写串 a、b，长度均 1..20000000。求 b 的 Z 数组与 b 对 a 各后缀的 LCP 数组，各输出 xor((i+1)*(长度 +1))。z[0]=|b|；乘法先提升 long long。直接求两串 LCP，不拼接整串。

所需模板：z_function（第 43 页，8.1 节）、exkmp（第 44 页，8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5410

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string a, b;
    cin >> a >> b;
    auto [z, p] = exkmp(a, b);
    long long x = 0, y = 0;
    for (int i = 0; i < (int)z.size(); i++)
        x ^= 1LL * (i + 1) * (z[i] + 1);
    for (int i = 0; i < (int)p.size(); i++)
        y ^= 1LL * (i + 1) * (p[i] + 1);
    cout << x << '\n' << y << '\n';
}
```

第九章　AC 自动机

9.1　AC 自动机出现次数

add(s)/count(s) 默认小写 string；add(s,offset)/count(s,offset) 接受 string 或 vector<int>，每个元素减 offset 须在 0..25，同一自动机使用一致编码。字符串字面量请先转 string 再传双参数入口。add 返回终态，支持空模式返回根 0；count 返回逐状态重叠出现次数，根恒为文本长度 +1。全部插入后 build 一次，随后只读且可重复查询；重复模式共享状态。a[u].len 为 Trie 深度，go/fail 与 order 可读，order 不含根。size 用 a.size()，重置赋新对象。构建 O(26V+L)，查询 O(|text|+V)，空间 O(26V)。状态数、深度须在 int 内。非空旧用法不变；旧根计数语义需减 1。出现过的模式编号数仍按每个终点分别判断 cnt>0。

```
struct AhoCorasick
{
    struct Node
    {
        array<int, 26> go{};
        int fail = 0, len = 0;
    };

    vector<Node> a{Node{}};
    vector<int> order;
    bool built = false;

    template <class S>
    int add(const S &s, int offset)
    {
        assert(!built);
        int u = 0;
        for (auto ch : s)
        {
            long long c = (long long)ch - offset;
            assert(0 <= c && c < 26);
            if (!a[u].go[c])
            {
                int v = (int)a.size();
                a[u].go[c] = v;
                a.push_back(Node{});
                a[v].len = a[u].len + 1;
            }
            u = a[u].go[c];
        }
        return u;
    }

    int add(const string &s)
    {
        return add(s, 'a');
    }
```

构建失败链接与统计出现次数（接上页同一结构体）：

```
    void build()
    {
        assert(!built);
        built = true;
        queue<int> q;
        for (int v : a[0].go)
            if (v) q.push(v);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            order.push_back(u);
            for (int c = 0; c < 26; c++)
            {
                int v = a[u].go[c];
                if (v)
                {
                    a[v].fail = a[a[u].fail].go[c];
                    q.push(v);
                }
                else
                    a[u].go[c] = a[a[u].fail].go[c];
            }
        }
    }

    template <class S>
    vector<long long> count(const S &s, int offset) const
    {
        assert(built);
        vector<long long> ans(a.size());
        ans[0] = 1;
        int u = 0;
        for (auto ch : s)
        {
            long long c = (long long)ch - offset;
            assert(0 <= c && c < 26);
            u = a[u].go[c];
            ++ans[u];
        }
        for (int i = (int)order.size() - 1; i >= 0; i--)
            ans[a[order[i]].fail] += ans[order[i]];
        return ans;
    }

    vector<long long> count(const string &s) const
    {
        return count(s, 'a');
    }
};
```

模板题使用：Luogu P5357

题意：给多个小写字母模式和文本，输出每个模式的出现次数，允许重叠。先保存 add 返回的终止状态，全部插入后 build，再 count 文本并按状态取值；重复模式共享状态，仍按输入顺序输出。

所需模板：AhoCorasick（第 45 页，9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5357

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    AhoCorasick ac;
    vector<int> terminal(n);
    for (int &id : terminal)
    {
        string pattern;
        cin >> pattern;
        id = ac.add(pattern);
    }
    ac.build();
    string text;
    cin >> text;
    auto count = ac.count(text);
    for (int id : terminal) cout << count[id] << '\n';
}
```

模板题使用：Luogu P3808 / AC 自动机简单版

题意：给非空小写模式串和文本，求出现过的模式编号数；两个相同字符串但编号不同要分别计数，同一编号多次出现只算一次。模式数、模式总长和文本长度均至 1000000。保存每次 add 返回的终点，count 后逐编号判断是否为正；不清空终态，因此可重复查询。

所需模板：AhoCorasick（第 45 页，9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3808

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    AhoCorasick ac;
    vector<int> end(n);
    for (int &u : end)
    {
        string s;
        cin >> s;
        u = ac.add(s);
    }
    ac.build();
    string t;
    cin >> t;
    auto cnt = ac.count(t);
    int ans = 0;
    for (int u : end) ans += cnt[u] > 0;
    cout << ans << '\n';
}
```

接口用法演示：整数序列 AC 与深度（接口演示）

演示约定：输入 n,q,offset，每条模式/查询给出长度和整数序列。各元素减 offset 必须为 0..25，允许空序列。先输出各模式 Trie 深度，再逐查询输出各模式出现次数；空模式贡献文本长度 +1。

所需模板：AhoCorasick（第 45 页，9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/string/ac-automaton/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q, offset;
    cin >> n >> q >> offset;
    AhoCorasick ac;
    vector<int> end(n);
    for (int &u : end)
    {
        int m;
        cin >> m;
        vector<int> s(m);
        for (int &x : s) cin >> x;
        u = ac.add(s, offset);
    }
    ac.build();
    for (int u : end) cout << ac.a[u].len << ' ';
    cout << '\n';
    while (q--)
    {
        int m;
        cin >> m;
        vector<int> s(m);
        for (int &x : s) cin >> x;
        auto cnt = ac.count(s, offset);
        for (int u : end) cout << cnt[u] << ' ';
        cout << '\n';
    }
}
```

接口用法演示：自定义字符偏移 AC（接口演示）

演示约定：输入 n,q 及字母表起始字符 offset，再输入 n 个模式串和 q 个文本，均用双引号包围以支持空串。字符减 offset 为 0..25。逐查询输出所有模式的重叠出现次数；例如 offset 为 A 可处理大写字母。

所需模板：AhoCorasick（第 45 页，9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/string/ac-automaton/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    char offset;
    cin >> n >> q >> offset;
    AhoCorasick ac;
    vector<int> end(n);
    for (int &u : end)
    {
        string s;
        cin >> quoted(s);
        u = ac.add(s, offset);
    }
    ac.build();
    while (q--)
    {
        string s;
        cin >> quoted(s);
        auto cnt = ac.count(s, offset);
        for (int u : end) cout << cnt[u] << ' ';
        cout << '\n';
    }
}
```

9.2 AC 自动机最短后缀匹配长度

代码依赖：AhoCorasick（第 45 页，9.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

先用 AhoCorasick.add(s) 得到终止节点，记录 (节点, 模式长度)，再 build 后调用 ac_shortest(ac,ends)。返回每个状态沿 fail 链对应的最短已登记模式长度，无模式为 INT_MAX。登记长度须真实且在 1..INT_MAX-1；重复模式、部分模式集合、空集合均可，未登记的模式不会参与最小值。按 ac.order 的 BFS 顺序传播，父 fail 状态先完成；O(状态数 + 登记数) 时间，O(状态数) 额外空间。扫描文本时 u=ac.a[u].go[c]，len[u] 就是当前位置结尾的最短匹配长度；文本仍须为小写字母。辅助函数不修改自动机，也不影响 count(text)，可对同一自动机的不同登记集合重复调用。

```
vector<int> ac_shortest(const AhoCorasick &ac, const vector<pair<int, int>> &ends)
{
    assert(ac.built);
    vector<int> len(ac.a.size(), INT_MAX);
    for (auto [u, n] : ends)
    {
        assert(0 < u && u < (int)len.size() && 0 < n && n < INT_MAX);
        len[u] = min(len[u], n);
    }
    for (int u : ac.order) len[u] = min(len[u], len[ac.a[u].fail]);
    return len;
}
```

应用题补充：AC 最短后缀匹配（本地接口示例）

题意：输入模式数、各个非空小写模式串及文本，逐位置输出以该位置结尾的最短模式长度，无匹配输出 0。先登记终止节点与长度，再 build 和传播。示例对应附件 min_len 变体，只作本地接口演示，不是 OJ 提交记录。

所需模板：AhoCorasick（第 45 页，9.1 节）、ac_shortest（第 51 页，9.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://oi-wiki.org/string/ac-automaton/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    AhoCorasick ac;
    vector<pair<int, int>> ends;
    while (n--)
    {
        string s;
        cin >> s;
        ends.push_back({ac.add(s), (int)s.size()});
    }
    ac.build();
    auto len = ac_shortest(ac, ends);
    string s;
    cin >> s;
    int u = 0;
    for (char c : s)
    {
        u = ac.a[u].go[c - 'a'];
        cout << (len[u] == INT_MAX ? 0 : len[u]) << ' ';
    }
    cout << '\n';
}
```

9.3 AC 自动机带权匹配

代码依赖：AhoCorasick（第 45 页，9.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

构造接收 (小写模式串, long long 权重) 列表；重复串权重相加，可负、可零。sum[u] 为当前状态所有后缀模式的权重和，query(text) 累加全部重叠匹配，O(|text|)。允许空串，在每个边界匹配，共 |text|+1 次；空模式集合返回 0。构造 O(26V+L+n)，空间 O(26V+n)，L 为总模式长。对象自持有 ac 和 sum，可复制、移动；构造后成员只读，重置赋新对象。所有中间和及答案须在 long long 内。

```
struct ACWeighted
{
    AhoCorasick ac;
    vector<long long> sum;

    ACWeighted(const vector<pair<string, long long>> &patterns = {})
    {
        vector<pair<int, long long>> ends;
        for (const auto &[s, w] : patterns)
        {
            int u = s.empty() ? 0 : ac.add(s);
            ends.push_back({u, w});
        }
        ac.build();
        sum.resize(ac.a.size());
        for (auto [u, w] : ends) sum[u] += w;
        for (int u : ac.order) sum[u] += sum[ac.a[u].fail];
    }

    long long query(const string &s) const
    {
        long long ans = sum[0];
        int u = 0;
        for (char c : s)
        {
            assert('a' <= c && c <= 'z');
            u = ac.a[u].go[c - 'a'];
            ans += sum[u];
        }
        return ans;
    }
};
```

接口用法演示：静态带权多模式查询（接口演示）

演示约定：输入 n,q，随后 n 组带引号模式串及有符号权重，再读 q 个带引号文本。双引号允许表示空串。输出全部重叠匹配的权重和；空模式在每个边界匹配。

所需模板：AhoCorasick（第 45 页，9.1 节）、ACWeighted（第 53 页，9.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/string/ac-automaton/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<pair<string, long long>> patterns(n);
    for (auto &[s, w] : patterns) cin >> quoted(s) >> w;
    ACWeighted ac(patterns);
    while (q--)
    {
        string s;
        cin >> quoted(s);
        cout << ac.query(s) << '\n';
    }
}
```

9.4 动态 AC：二进制分组重建

代码依赖：ACWeighted（第 53 页，9.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

add(s,w) 累加模式权重；插入计数用 +1，删除用-1，合法删除由调用者保证。query(s) 求所有模式出现次数乘当前权重之和，支持重复、负权和空串，边界同 ACWeighted。第 k 组为空或含 2^k 次更新；进位合并后重建。M 次更新、总长度 L，累计构建 O(26(L+M)log(M+2))，查询 O((|s|+1)log(M+2))，空间 O(26(L+M))；一次更新可线性重建，非最坏对数。历史删除不会回收更新记录，空间按全部历史更新计。block/ac 只读；重置赋新对象。所有组内及跨组中间和须在 long long 内。

```
struct DynamicAC
{
    vector<vector<pair<string, long long>>> block;
    vector<ACWeighted> ac;

    void add(string s, long long w)
    {
        vector<pair<string, long long>> cur;
        cur.push_back({move(s), w});
        int k = 0;
        while (k < (int)block.size() && !block[k].empty())
        {
            for (auto &p : block[k]) cur.push_back(move(p));
            block[k].clear();
            ac[k] = ACWeighted();
            k++;
        }
        if (k == (int)block.size())
        {
            block.emplace_back();
            ac.emplace_back();
        }
        block[k] = move(cur);
        ac[k] = ACWeighted(block[k]);
    }

    long long query(const string &s) const
    {
        long long ans = 0;
        for (int k = 0; k < (int)block.size(); k++)
        {
            if (!block[k].empty()) ans += ac[k].query(s);
        }
        return ans;
    }
};
```

应用题补充：Codeforces 710F / String Set Queries

题意：在线维护字符串集合。1 插入，2 删除，3 询问文本中所有集合模式的总出现次数。题目保证新增串从未加入、删除串存在；每次回答必须 endl 刷新后再读下一条，不能预读全部操作。用 +1/-1 更新权重。

所需模板：AhoCorasick（第 45 页，9.1 节）、ACWeighted（第 53 页，9.3 节）、DynamicAC（第 55 页，9.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/710/F

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    cin >> m;
    DynamicAC dict;
    while (m--)
    {
        int t;
        string s;
        cin >> t >> s;
        if (t == 3)
        {
            cout << dict.query(s) << endl;
        }
        else
        {
            dict.add(move(s), t == 1 ? 1 : -1);
        }
    }
}
```

第十章　后缀数组 (SA)

10.1　后缀数组简介

10.1.1　线性后缀数组（SA-IS）

SAIS(s,alphabet)：整数符号 0..alphabet-1，1≤alphabet<INT_MAX，长度 n<INT_MAX；SAIS(string) 按 unsigned byte 处理，支持零字节。sa/rk/lcp 均长 n，0-based；lcp[0]=0，其余为相邻后缀 LCP。无需哨兵，空串结果为空。诱导排序时间/空间 O(n+alphabet)，递归深度 O(log n)；任意值离散化另需 O(n log n)。保留倍增 SuffixArray；SuffixLCP 不直接接受本类型。

```
struct SAIS
{
    vector<int> sa, rk, lcp;

    SAIS(const string &s)
    {
        assert(s.size() < INT_MAX);
        vector<int> a;
        a.reserve(s.size());
        for (unsigned char c : s) a.push_back(c);
        *this = SAIS(a, 256);
    }

    SAIS(const vector<int> &s, int alphabet)
    {
        assert(s.size() < INT_MAX && 1 <= alphabet && alphabet < INT_MAX);
        for (int c : s) assert(0 <= c && c < alphabet);
        int n = s.size();
        sa = sort_suffixes(s, alphabet);
        rk.resize(n);
        lcp.assign(n, 0);
        for (int i = 0; i < n; i++) rk[sa[i]] = i;
        for (int i = 0, k = 0; i < n; i++)
        {
            if (!rk[i])
            {
                k = 0;
                continue;
            }
            int j = sa[rk[i] - 1];
            while (k < n - i && k < n - j && s[i + k] == s[j + k]) k++;
            lcp[rk[i]] = k;
            if (k) k--;
        }
    }
```

类型与桶内诱导（接上页同一结构体）：

```
private:
    static vector<int> sort_suffixes(const vector<int> &s, int alphabet)
    {
        int n = s.size();
        if (!n) return {};
        if (n == 1) return {0};
        vector<int> sa(n), lo(alphabet + 1), hi(alphabet + 1);
        vector<bool> type(n);
        for (int i = n - 2; i >= 0; i--)
            type[i] = s[i] == s[i + 1] ? type[i + 1] : s[i] < s[i + 1];
        for (int i = 0; i < n; i++)
            if (type[i]) lo[s[i] + 1]++;
            else hi[s[i]]++;
        for (int c = 0; c < alphabet; c++)
        {
            hi[c] += lo[c];
            lo[c + 1] += hi[c];
        }
        auto induce = [&](const vector<int> &lms)
        {
            fill(sa.begin(), sa.end(), -1);
            vector<int> pos = hi;
            for (int i : lms) sa[pos[s[i]]++] = i;
            pos = lo;
            sa[pos[s[n - 1]]++] = n - 1;
            for (int i = 0; i < n; i++)
            {
                int v = sa[i];
                if (v > 0 && !type[v - 1]) sa[pos[s[v - 1]]++] = v - 1;
            }
            pos = lo;
            for (int i = n - 1; i >= 0; i--)
            {
                int v = sa[i];
                if (v > 0 && type[v - 1]) sa[--pos[s[v - 1] + 1]] = v - 1;
            }
        };
```

LMS 命名与递归（接上页同一结构体）：

```
        vector<int> id(n, -1), lms;
        for (int i = 1; i < n; i++)
            if (!type[i - 1] && type[i])
            {
                id[i] = lms.size();
                lms.push_back(i);
            }
        induce(lms);
        int m = lms.size();
        if (m)
        {
            vector<int> order, reduced(m);
            for (int v : sa)
                if (id[v] >= 0) order.push_back(v);
            int names = 1;
            for (int i = 1; i < m; i++)
            {
                int a = order[i - 1], b = order[i];
                int x = id[a] + 1 < m ? lms[id[a] + 1] : n;
                int y = id[b] + 1 < m ? lms[id[b] + 1] : n;
                bool same = x - a == y - b;
                if (same)
                {
                    while (a < x && s[a] == s[b]) a++, b++;
                    same = a < n && b < n && s[a] == s[b];
                }
                if (!same) names++;
                reduced[id[order[i]]] = names - 1;
            }
            order = sort_suffixes(reduced, names);
            for (int &i : order) i = lms[i];
            induce(order);
        }
        return sa;
    }
};
```

模板题使用：Library Checker suffixarray

题意：输入长度 1..500000 的小写字符串，按字典序输出所有非空后缀的 0-based 起点。独立 SA-IS 实现；无需附加哨兵，不继承倍增版本的线上记录。

所需模板：SAIS（第 57 页，10.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/suffixarray

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    SAIS suffix(s);
    for (int i = 0; i < (int)s.size(); i++)
        cout << suffix.sa[i] << (i + 1 == (int)s.size() ? '\n' : ' ');
}
```

10.1.2 倍增后缀数组与 LCP

构造 SuffixArray(s) 支持任意字节串，按 unsigned char 的 0..255 数值比较，包含内嵌零字节；也可构造 SuffixArray(a,alphabet)，整数序列要求 0≤a[i]<alphabet、1≤alphabet<INT_MAX。重新赋值 SuffixArray(a,alphabet) 可重建。长度须小于 INT_MAX。sa[k] 为第 k 个非空后缀起点，rk[i] 为排名，全部 0-based；lcp[k] 为 sa[k-1] 与 sa[k] 的最长公共前缀，lcp[0]=0。空序列合法，三数组均为空。无需手动追加哨兵，也不输出空后缀。每轮按第二关键字已有顺序、第一关键字稳定计数排序，倍增 O(n log n+alphabet)，空间 O(n+alphabet)；字节串 alphabet=256。大整数先自行保序离散化，避免按最大原值开桶。Kasai 求 lcp 为 O(n)。不同起点 x,y 的 LCP 为 lcp[min(rk[x],rk[y])+1..max(rk[x],rk[y])] 的最小值；本结构不内建 RMQ，需要频繁查询时另接静态 RMQ。同一起点的 LCP 为 n-x。覆盖倍增构造，不替代 DC3/SA-IS。

```
struct SuffixArray
{
    vector<int> sa, rk, lcp;

    SuffixArray(const string &s)
    {
        vector<int> a;
        a.reserve(s.size());
        for (unsigned char c : s) a.push_back(c);
        *this = SuffixArray(a, 256);
    }

    SuffixArray(const vector<int> &s, int alphabet)
    {
        assert(s.size() < INT_MAX);
        assert(1 <= alphabet && alphabet < INT_MAX);
        int n = (int)s.size();
        sa.resize(n);
        rk.resize(n);
        lcp.assign(n, 0);
        if (!n) return;
        vector<int> order(n), old(n), cnt(max(n + 1, alphabet + 1));
        int m = alphabet;
        for (int i = 0; i < n; i++)
        {
            assert(0 <= s[i] && s[i] < alphabet);
            rk[i] = s[i] + 1;
            cnt[rk[i]]++;
        }
        for (int i = 1; i <= m; i++) cnt[i] += cnt[i - 1];
        for (int i = n - 1; i >= 0; i--) sa[--cnt[rk[i]]] = i;
```

```
        for (int k = 1; k < n; k *= 2)
        {
            int p = 0;
            for (int i = n - k; i < n; i++) order[p++] = i;
            for (int i : sa)
                if (i >= k) order[p++] = i - k;
            fill(cnt.begin(), cnt.begin() + m + 1, 0);
            for (int x : rk) cnt[x]++;
            for (int i = 1; i <= m; i++) cnt[i] += cnt[i - 1];
            for (int i = n - 1; i >= 0; i--) sa[--cnt[rk[order[i]]]] = order[i];
            old.swap(rk);
            auto second = [&](int i)
            {
                return i < n - k ? old[i + k] : 0;
            };
            m = 1;
            rk[sa[0]] = m;
            for (int i = 1; i < n; i++)
            {
                int a = sa[i - 1], b = sa[i];
                if (old[a] != old[b] || second(a) != second(b)) m++;
                rk[b] = m;
            }
            if (m == n) break;
        }
        for (int i = 0; i < n; i++) rk[sa[i]] = i;
        for (int i = 0, k = 0; i < n; i++)
        {
            if (!rk[i])
            {
                k = 0;
                continue;
            }
            int j = sa[rk[i] - 1];
            while (k < n - i && k < n - j && s[i + k] == s[j + k]) ++k;
            lcp[rk[i]] = k;
            if (k) --k;
        }
    }
};
```

模板题使用：Luogu P3809

题意：把所有非空后缀按字典序排序，输出各后缀在原串中的起点。模板 sa 为 0-based，题目输出为 1-based，因此加 1。

所需模板：SuffixArray（第 61 页，10.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3809

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::string s;
    std::cin >> s;
    SuffixArray suffix(s);
    for (int i = 0; i < (int)s.size(); i++)
        std::cout << suffix.sa[i] + 1 << (i + 1 == (int)s.size() ? '\n' : ' ');
}
```

模板题使用：Library Checker longest_common_substring

题意：求两个小写字符串的一段最长公共子串，输出两串各自 0-based 半开区间，各串长度至 50 万。整数拼接以 26 为分隔符、alphabet=27；扫描异来源相邻后缀，并将 LCP 截断至原串边界。

所需模板：SuffixArray（第 61 页，10.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/longest_common_substring

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s, t;
    cin >> s >> t;
    int n = s.size(), m = t.size();
    vector<int> a;
    for (char c : s) a.push_back(c - 'a');
    a.push_back(26);
    for (char c : t) a.push_back(c - 'a');
    SuffixArray suffix(a, 27);
    int best = 0, left = 0, right = 0;
    for (int k = 1; k < (int)a.size(); k++)
    {
        int x = suffix.sa[k - 1], y = suffix.sa[k];
        if (x > y) swap(x, y);
        if (x >= n || y <= n) continue;
        int len = min({suffix.lcp[k], n - x, n + 1 + m - y});
        if (len > best)
        {
            best = len;
            left = x;
            right = y - n - 1;
        }
    }
    cout << left << ' ' << left + best << ' ' << right << ' ' << right + best << '\n';
}
```

10.1.3 后缀 LCP 与子串比较

代码依赖：SuffixArray（第 61 页，10.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

先构造 SuffixArray，再构造 SuffixLCP(sa)。重新赋值 SuffixLCP(sa) 可建新表。复制所需排名与 LCP 数据，不保存原字符串或源对象引用，源对象可销毁或重建。query(x,y) 返回 s[x..n) 与 s[y..n) 的最长公共前缀，0≤x,y≤n，允许空后缀；同起点返回 n-x。compare(l1,r1,l2,r2) 比较两个 0-based 半开子串，允许空串，返回 -1/0/1。若公共前缀覆盖较短子串，按长度比较，否则原后缀排名给出首个不同字符的顺序。字节排序及整数字母表约束继承 SuffixArray。预处理稀疏表 O(n log n) 时间、空间，每次查询 O(1)，无需哈希和原串。n、rk、lg、st 可读，禁止调用方破坏。

```
struct SuffixLCP
{
    int n;
    vector<int> rk, lg;
    vector<vector<int>> st;

    SuffixLCP(const SuffixArray &suffix)
    {
        n = suffix.sa.size();
        rk = suffix.rk;
        lg.assign(n + 1, 0);
        for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
        st.assign(lg[n] + 1, {});
        st[0] = suffix.lcp;
        for (int k = 1; k <= lg[n]; k++)
        {
            int len = 1 << k;
            st[k].resize(n - len + 1);
            for (int i = 0; i <= n - len; i++)
                st[k][i] = min(st[k - 1][i], st[k - 1][i + len / 2]);
        }
    }
```

```
    int query(int x, int y) const
    {
        assert(0 <= x && x <= n && 0 <= y && y <= n);
        if (x == y || x == n || y == n) return n - max(x, y);
        int a = rk[x], b = rk[y];
        if (a > b) swap(a, b);
        int k = lg[b - a];
        return min(st[k][a + 1], st[k][b - (1 << k) + 1]);
    }

    // Compare [l1,r1) and [l2,r2); return -1, 0 or 1.
    int compare(int l1, int r1, int l2, int r2) const
    {
        assert(0 <= l1 && l1 <= r1 && r1 <= n);
        assert(0 <= l2 && l2 <= r2 && r2 <= n);
        int a = r1 - l1, b = r2 - l2;
        if (query(l1, l2) >= min(a, b)) return (a > b) - (a < b);
        return rk[l1] < rk[l2] ? -1 : 1;
    }
};
```

接口用法演示：静态后缀查询（自定义协议）

演示约定：自定义接口演示，非官方题目或 OJ 驱动。首行 n q，次行恰为 n 个小写字母（n=0 时保留空行）；0≤n,q≤200000 仅为演示上限。操作 0 x y 输出 s[x..n)、s[y..n) 的 LCP；1 l1 r1 l2 r2 比较两个半开子串，输出-1/0/1；2 x y 输出前缀 s[0..x)、s[0..y) 的最长公共后缀长度，LCS 在此不是子序列。所有端点 0..n，允许空区间；同一起点 LCP 为 n-x，同一前缀 LCS 为 x。反向结构必须从原串倒序构造。子串 LCP 取 min(query(l1,l2),r1-l1,r2-l2)，子串公共后缀取 min(prefix_lcs(reversed,r1,r2),r1-l1,r2-l2)。预处理 O(n log n) 时间/空间，各查询 O(1)；字符串修改后重建。

所需模板：SuffixArray（第 61 页，10.1.2 节）、SuffixLCP（第 64 页，10.1.3 节）、prefix_lcs（第 67 页，10.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/src/compact/suffix_lcp.hpp

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    assert(0 <= n && n <= 200000 && 0 <= q && q <= 200000);
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    string s;
    getline(cin, s); // Keep the empty line when n = 0; do not use cin >> ws.
    assert((int)s.size() == n);
    for (char c : s) assert('a' <= c && c <= 'z');
    SuffixLCP forward{SuffixArray(s)};
    string r(s.rbegin(), s.rend());
    SuffixLCP reversed{SuffixArray(r)};
    while (q--)
    {
        int op;
        cin >> op;
        if (op == 0)
        {
            int x, y;
            cin >> x >> y;
            cout << forward.query(x, y) << '\n';
        }
        else if (op == 1)
        {
            int l1, r1, l2, r2;
            cin >> l1 >> r1 >> l2 >> r2;
            cout << forward.compare(l1, r1, l2, r2) << '\n';
        }
        else
        {
            assert(op == 2);
            int x, y;
            cin >> x >> y;
            cout << prefix_lcs(reversed, x, y) << '\n';
        }
    }
}
```

接口用法演示：kuangbin DA 来源模型：反串与 LCP 求最长回文

演示约定：来源程序协议：读入非空字符串直到 EOF，每组输出最靠左的最长回文子串。组合原串、独立分隔符与反串，通过奇偶中心的 LCP 求半径；i=0 的偶中心查询空后缀，返回 0。新实现按 unsigned char 加 2，分隔符为 1；长度须 ≤(INT_MAX−2)/2 且满足内存限制。O(n log n) 时间空间。此为来源应用协议演示，不是独立线上模板题或 AC；仅需最长回文时优先用 Manacher。

所需模板：SuffixArray（第 61 页，10.1.2 节）、SuffixLCP（第 64 页，10.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/kuangbin/ACM-ICPC

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    while (cin >> s)
    {
        assert(s.size() <= (INT_MAX - 2) / 2);
        int len = s.size();
        int n = 2 * len + 1;
        vector<int> a(n);
        for (int i = 0; i < len; i++)
        {
            a[i] = (unsigned char)s[i] + 2;
            a[len + 1 + i] = (unsigned char)s[len - 1 - i] + 2;
        }
        a[len] = 1;
        SuffixArray suffix(a, 258);
        SuffixLCP lcp(suffix);
        int best = 0, start = 0;
        for (int i = 0; i < len; i++)
        {
            int radius = lcp.query(i, n - i);
            if (2 * radius > best)
            {
                best = 2 * radius;
                start = i - radius;
            }
            radius = lcp.query(i, n - i - 1);
            if (2 * radius - 1 > best)
            {
                best = 2 * radius - 1;
                start = i - radius + 1;
            }
        }
        cout << s.substr(start, best) << '\n';
    }
    return 0;
}
```

10.1.4 两个前缀的最长公共后缀

代码依赖：SuffixLCP（第 64 页，10.1.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

prefix_lcs(reversed,x,y) 返回原序列 s[0..x) 与 s[0..y) 的最长公共后缀，端点满足 0≤x,y≤n。reversed 必须在原序列的反转序列上构造 SuffixLCP，不能传正向对象；转换为 reversed.query(n-x,n-y)，单次 O(1)。只需前向 LCP 时不必构造反向对象；该接口中的 LCS 是最长公共后缀，不是最长公共子序列。对于两个任意子串，以右端点查询后还需截断到两个子串长度的最小值。

```
// reversed must be built on the reversed original sequence.
int prefix_lcs(const SuffixLCP &reversed, int x, int y)
{
    assert(0 <= x && x <= reversed.n && 0 <= y && y <= reversed.n);
    return reversed.query(reversed.n - x, reversed.n - y);
}
```

联合使用示例见第 65 页（静态后缀查询（自定义协议）），其中直接调用本模板。

10.1.5 重复子串 AA 的起止位置计数

代码依赖：SuffixLCP（第 64 页，10.1.3 节）、prefix_lcs（第 67 页，10.1.4 节）。抄写时一并准备；标准库类型不列入算法依赖。

square_counts(forward,reversed) 返回 (start,finish) 两个长度 n 的 long long 数组，start[i]/finish[i] 为开始/结束于位置 i 的非空 AA 型子串个数；结束位置是最后一个字符下标，均 0-based。两对象必须分别对应同一原序列及其反序列，长度相同，允许空序列。枚举半长 len，并以其整数倍 i 为锚点；向左匹配截断为 len-1，向右截断为 len，由 LCP/LCS 得出合法起点连续区间，差分累计。左侧不能放宽到 len，否则相邻锚点会重复计数。预处理查询器后，此函数 O(n log n) 时间、O(n) 额外空间。不同出现位置、不同半长分别计数；不要求 A 为最小周期。P1117 的 AABB 拆分数为所有分界点 finish[i]*start[i+1] 之和，累加类型需按题目范围选择，函数本身只返回起止计数。

```
// Count nonempty squares AA by their starting and ending positions.
pair<vector<long long>, vector<long long>> square_counts(const SuffixLCP &forward,
                                                         const SuffixLCP &reversed)
{
    int n = forward.n;
    assert(reversed.n == n);
    vector<long long> start(n + 1), finish(n + 1);
    for (int len = 1; len <= n / 2; len++)
        for (int i = 0; i < n - len; i += len)
        {
            int j = i + len;
            int left = min(len - 1, prefix_lcs(reversed, i, j));
            int right = min(len, forward.query(i, j));
            if (left + right < len) continue;
            int l = i - left, r = i + right - len;
            start[l]++;
            start[r + 1]--;
            finish[l + 2 * len - 1]++;
            finish[r + 2 * len]--;
        }
```

```
    for (int i = 1; i < n; i++)
    {
        start[i] += start[i - 1];
        finish[i] += finish[i - 1];
    }
    start.resize(n);
    finish.resize(n);
    return {start, finish};
}
```

应用题补充：Luogu P1117 [NOI2016] 优秀的拆分

题意：统计所有子串的非空 AABB 拆分，不同出现位置、不同 A/B 长度分别计数，允许 A=B。T 为 1 至 10，每组小写串长度不超过 30000。正向与反向 SuffixLCP 必须来自同一字符串；start[i] 和 finish[i] 分别计开始/结束于字符 i 的 AA。每个分界点贡献 finish[i]*start[i+1]，以 long long 累加。此为 NOI 比赛应用题，尚未计入独立模板题覆盖。

所需模板：SuffixArray（第 61 页，10.1.2 节）、SuffixLCP（第 64 页，10.1.3 节）、prefix_lcs（第 67 页，10.1.4 节）、square_counts（第 67 页，10.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1117

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int tests;
    std::cin >> tests;
    while (tests--)
    {
        std::string s;
        std::cin >> s;
        int n = s.size();
        SuffixLCP forward{SuffixArray(s)};
        std::reverse(s.begin(), s.end());
        SuffixLCP backward{SuffixArray(s)};
        auto [start, finish] = square_counts(forward, backward);
        long long answer = 0;
        for (int i = 0; i + 1 < n; i++) answer += finish[i] * start[i + 1];
        std::cout << answer << '\n';
    }
}
```

10.1.6 后缀数组：DC3 线性构造

DC3(string) 按 unsigned char 比较全部字节（含 0/255）；DC3(vector<int>,alphabet) 要求每项在 [0,alphabet)，1≤alphabet<INT_MAX−3，长度 n<INT_MAX−3。空输入合法。sa 为所有非空后缀的 0-based 起点，rk[sa[i]]=i；lcp[0]=0，lcp[i] 为 sa[i−1] 与 sa[i] 的公共前缀长度。三数组均长 n，空输入均为空，不含空后缀；输入不被改写，无需调用者补零或三倍数组。内部把符号加一，0 只作哨兵；n mod3=1 时补一个虚拟采样点，归并时跳过。稳定基数排序三元组、递归命名、排序余 0 类并合并；时间空间均 O(n+alphabet)，仅 alphabet=O(n) 或固定字节集时称线性。稀疏大整数先保序离散化，其比较排序代价另计。递归深度 O(log(n+1))，私有 sort_suffixes 要求正符号，用户只用构造及结果数组。与 SuffixArray/SAIS 接口约定一致，无线上性能优越性声明。

```
struct DC3
{
    vector<int> sa, rk, lcp;

    DC3(const string &s)
    {
        vector<int> a;
        for (unsigned char c : s)
            a.push_back(c);
        *this = DC3(a, 256);
    }

    DC3(const vector<int> &s, int alphabet)
    {
        assert(s.size() < INT_MAX - 3 && 1 <= alphabet && alphabet < INT_MAX - 3);
        int n = s.size();
        vector<int> a(n);
        for (int i = 0; i < n; i++)
        {
            assert(0 <= s[i] && s[i] < alphabet);
            a[i] = s[i] + 1;
        }
        sa = sort_suffixes(move(a), alphabet);
        rk.resize(n);
        lcp.assign(n, 0);
        for (int i = 0; i < n; i++)
            rk[sa[i]] = i;
        for (int i = 0, k = 0; i < n; i++)
        {
            if (!rk[i])
            {
                k = 0;
                continue;
            }
            int j = sa[rk[i] - 1];
            while (k < n - i && k < n - j && s[i + k] == s[j + k])
                k++;
            lcp[rk[i]] = k;
            if (k)
                k--;
        }
    }
```

采样三元组排序与递归命名（接上页同一结构体）：

```
private:
    static vector<int> sort_suffixes(vector<int> s, int alphabet)
    {
        int n = s.size();
        if (!n)
            return {};
        if (n == 1)
            return {0};
        s.resize(n + 3);
        int n0 = (n + 2) / 3;
        int extra = n % 3 == 1;
        vector<int> sample, zero;
        for (int i = 0; i < n + extra; i++)
            if (i % 3)
                sample.push_back(i);
        auto radix = [&](vector<int> &a, int shift)
        {
            vector<int> count(alphabet + 1), b(a.size());
            for (int i : a)
                count[s[i + shift]]++;
            for (int i = 1; i <= alphabet; i++)
                count[i] += count[i - 1];
            for (int i = (int)a.size() - 1; i >= 0; i--)
                b[--count[s[a[i] + shift]]] = a[i];
            a.swap(b);
        };
        for (int shift = 2; shift >= 0; shift--)
            radix(sample, shift);
        int names = 0;
        vector<int> reduced(sample.size()), order(sample.size());
        array<int, 3> last{-1, -1, -1};
        for (int i : sample)
        {
            array<int, 3> key{s[i], s[i + 1], s[i + 2]};
            if (key != last)
                names++;
            last = key;
            reduced[i / 3 + (i % 3 == 2 ? n0 : 0)] = names;
        }
        if (names < (int)sample.size())
            order = sort_suffixes(move(reduced), names);
        else
            for (int i = 0; i < (int)reduced.size(); i++)
                order[reduced[i] - 1] = i;
```

恢复采样排名并合并（接上页同一结构体）：

```
        vector<int> rank(n + 3);
        for (int k = 0; k < (int)order.size(); k++)
        {
            int i = order[k];
            int p = i < n0 ? 3 * i + 1 : 3 * (i - n0) + 2;
            sample[k] = p;
            rank[p] = k + 1;
            if (p % 3 == 1)
                zero.push_back(p - 1);
        }
        radix(zero, 0);
        vector<int> result;
        int a = extra, b = 0;
        while (a < (int)sample.size() && b < (int)zero.size())
        {
            int i = sample[a], j = zero[b];
            bool first;
            if (i % 3 == 1)
                first = pair{s[i], rank[i + 1]} < pair{s[j], rank[j + 1]};
            else
                first = tuple{s[i], s[i + 1], rank[i + 2]}
                      < tuple{s[j], s[j + 1], rank[j + 2]};
            if (first)
                result.push_back(sample[a++]);
            else
                result.push_back(zero[b++]);
        }
        while (a < (int)sample.size())
            result.push_back(sample[a++]);
        while (b < (int)zero.size())
            result.push_back(zero[b++]);
        return result;
    }
};
```

模板题使用：Library Checker suffixarray（DC3）

题意：输入长度 1..500000 的小写字符串，输出所有非空后缀按字典序排列的 0-based 起点。DC3 独立程序，无需附加哨兵；原倍增或 SA-IS 用法的线上记录不转移到本程序。

所需模板：DC3（第 69 页，10.1.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/suffixarray

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    DC3 d(s);
    for (int i = 0; i < (int)d.sa.size(); i++)
        cout << d.sa[i] << (i + 1 == (int)d.sa.size() ? '\n' : ' ');
    return 0;
}
```

接口用法演示：稠密整数后缀数组、逆排名及 LCP

演示约定：输入 n,alphabet 与 n 个 [0,alphabet) 整数，分别输出 sa、rk、lcp 三行。允许 n=0，此时仍输出三个空行。lcp[0]=0，其余对当前排名与前一名后缀比较；输入符号 0 可用，不应手工插入空后缀哨兵。API 演示，不是线上题目或 AC。

所需模板：DC3（第 69 页，10.1.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/string/sa/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, alphabet;
    cin >> n >> alphabet;
    vector<int> s(n);
    for (int &c : s)
        cin >> c;
    DC3 d(s, alphabet);
    for (int x : d.sa)
        cout << x << ' ';
    cout << '\n';
    for (int x : d.rk)
        cout << x << ' ';
    cout << '\n';
    for (int x : d.lcp)
        cout << x << ' ';
    cout << '\n';
    return 0;
}
```

第十一章　后缀自动机 (SAM)

11.1　后缀自动机

extend(c) 接收整数 0..25，每次追加一个小写字符；last 为整串状态，a[0] 为根，根 link=-1。a[u].len 为状态最大长度，非根代表 (len[link[u]],len[u]] 且具有同一 endpos 集合。occ 是原始种子：真实扩展为 1、克隆为 0，不能直接当出现次数。counts() 按长度倒序汇总后返回副本，不改结构，重复查询及查询后继续追加安全；根计数为 n，非空子串只看 u>0。distinct() 为不同非空子串数。重赋值 SuffixAutomaton() 清空；固定 26 字母表构建 O(n)，counts/distinct 为 O(n)，空间 O(26n)。如需前缀位置对应状态，在每次 extend 后保存 last；本结构不输出广义 SAM 或最小 DFA 状态数。

```
struct SuffixAutomaton
{
    struct Node
    {
        array<int, 26> go{};
        int link = -1, len = 0;
        long long occ = 0;
    };

    vector<Node> a{Node{}};
    int last = 0;
```

接上页同一结构体：

```
    void extend(int c)
    {
        assert(0 <= c && c < 26);
        int cur = (int)a.size();
        a.push_back(Node{});
        a[cur].len = a[last].len + 1;
        a[cur].occ = 1;
        int p = last;
        while (p != -1 && !a[p].go[c])
        {
            a[p].go[c] = cur;
            p = a[p].link;
        }
        if (p == -1)
            a[cur].link = 0;
        else
        {
            int q = a[p].go[c];
            if (a[p].len + 1 == a[q].len)
                a[cur].link = q;
            else
            {
                int clone = (int)a.size();
                Node copy = a[q];
                a.push_back(copy);
                a[clone].len = a[p].len + 1;
                a[clone].occ = 0;
                while (p != -1 && a[p].go[c] == q)
                {
                    a[p].go[c] = clone;
                    p = a[p].link;
                }
                a[q].link = a[cur].link = clone;
            }
        }
        last = cur;
    }
```

接上页同一结构体：

```
    vector<long long> counts() const
    {
        vector<int> bucket(a[last].len + 1), order(a.size());
        vector<long long> ans(a.size());
        for (auto &v : a) ++bucket[v.len];
        partial_sum(bucket.begin(), bucket.end(), bucket.begin());
        for (int i = 0; i < (int)a.size(); i++)
        {
            order[--bucket[a[i].len]] = i;
            ans[i] = a[i].occ;
        }
        for (int i = (int)order.size() - 1; i > 0; i--)
            ans[a[order[i]].link] += ans[order[i]];
        return ans;
    }

    long long distinct() const
    {
        long long ans = 0;
        for (int i = 1; i < (int)a.size(); i++) ans += a[i].len - a[a[i].link].len;
        return ans;
    }
};
```

模板题使用：Library Checker number_of_substrings

题意：给小写字母串，求不同非空子串的个数。extend 接收 0..25 的整数，故字符减去 a；所有字符追加后调用 distinct，结果使用 long long。

所需模板：SuffixAutomaton（第 74 页，11.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/number_of_substrings

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    SuffixAutomaton sam;
    for (char c : s) sam.extend(c - 'a');
    cout << sam.distinct() << '\n';
}
```

模板题使用：Luogu P3804

题意：长度至 100 万的小写串，只考虑出现次数大于 1 的子串，最大化长度乘次数。counts 按长度汇总，取非根状态 len*count；无重复子串输出 0，乘积使用 long long。

所需模板：SuffixAutomaton（第 74 页，11.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3804

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    SuffixAutomaton sam;
    for (char c : s) sam.extend(c - 'a');
    auto count = sam.counts();
    long long answer = 0;
    for (int u = 1; u < (int)sam.a.size(); u++)
    {
        if (count[u] > 1) answer = max(answer, count[u] * sam.a[u].len);
    }
    cout << answer << '\n';
}
```

接口用法演示：区间不同子串（kuangbin HDU4622 模型）

演示约定：来源协议：T 组，每组非空小写字母串 s、q 及 q 个 1-based 闭区间 [l,r]，输出该区间内不同非空子串数量。每个左端点新建 SAM，每次 extend 后累加新 last 的 len 减 link.len；克隆只重分配旧子串，不另增贡献。固定 26 字母表，预处理 $O(n^2)$，查询 O(1)，答案表 $O(n^2)$、临时 SAM O(n) 空间。不要在内层循环调用扫描所有状态的 distinct()，也不能跨左端点复用旧 SAM。当前为来源模型/API 用法，官方题面、资源限制及线上 AC 未核验。

所需模板：SuffixAutomaton（第 74 页，11.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://acm.hdu.edu.cn/showproblem.php?pid=4622

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        string s;
        cin >> s;
        int n = (int)s.size();
        vector<vector<long long>> ans(n, vector<long long>(n));
        for (int l = 0; l < n; l++)
        {
            SuffixAutomaton sam;
            long long sum = 0;
            for (int r = l; r < n; r++)
            {
                sam.extend(s[r] - 'a');
                int u = sam.last;
                sum += sam.a[u].len - sam.a[sam.a[u].link].len;
                ans[l][r] = sum;
            }
        }
        int q;
        cin >> q;
        while (q--)
        {
            int l, r;
            cin >> l >> r;
            cout << ans[l - 1][r - 1] << '\n';
        }
    }
}
```

11.2 SAM 字典序子串查询

SAMLex(sam,w={}) 引用已建好的 SuffixAutomaton 或已 build 的 GeneralSAM；索引使用期间 sam 必须存活且保持不变。默认每个非根状态权重 1，得到不同子串；传单串 sam.counts() 按出现次数计数。自定义 w 长度为状态数、非负 long long，每条到达该状态的串有 w[u] 份，根权重强制 0。sum[u] 为本状态及后续路径权重和，超过 LLONG_MAX 截断，仍正确支持所有 1≤k≤LLONG_MAX。kth(k,alphabet) 返回 optional<string>，不足或 k≤0 为 nullopt；默认 a..z，alphabet 必须是 26 字母的排列。短串先于其扩展，支持重复查询与自定义字母序。构建 O(26V+L)，空间 O(V+L)，L 为最长串长；单次查询 O(26× 答案长度)，额外空间为答案。weight/sum 只读，不支持扩展 SAM 后沿用旧索引。

```
// Keep sam alive and unchanged while this index is used.
template <class S>
struct SAMLex
{
    const S &sam;
    vector<long long> weight, sum;

    SAMLex(const S &sam, vector<long long> w = {}) : sam(sam), weight(move(w))
    {
        int n = sam.a.size(), length = 0;
        if (weight.empty()) weight.assign(n, 1);
        assert((int)weight.size() == n);
        weight[0] = 0;
        sum = weight;
        for (auto &v : sam.a) length = max(length, v.len);
        vector<int> bucket(length + 1), order(n);
        for (auto &v : sam.a) bucket[v.len]++;
        for (int i = 1; i <= length; i++) bucket[i] += bucket[i - 1];
        for (int i = 0; i < n; i++) order[--bucket[sam.a[i].len]] = i;
        for (int i = n - 1; i >= 0; i--)
        {
            int u = order[i];
            for (int v : sam.a[u].go)
                if (v) sum[u] += min(sum[v], LLONG_MAX - sum[u]);
        }
    }
```

字典序查询（接上页同一结构体）：

```
    // alphabet must be a permutation of a..z; k is 1-based.
    optional<string> kth(long long k, const string &alphabet = "abcdefghijklmnopqrstuvwxyz") const
    {
        if (k <= 0 || k > sum[0]) return nullopt;
        int u = 0;
        string ans;
        while (k > weight[u])
        {
            k -= weight[u];
            for (char c : alphabet)
            {
                int v = sam.a[u].go[c - 'a'];
                if (!v) continue;
                if (k > sum[v]) k -= sum[v];
                else
                {
                    ans += c;
                    u = v;
                    break;
                }
            }
        }
        return ans;
    }
};
```

应用题补充：Luogu P3975 / 弦论

题意：小写串长至 500000，t=0 时相同子串只计一次，t=1 按出现次数计；1≤k≤10^9。用 counts 作为状态权重，根权重由索引清零；不足输出-1。竞赛应用，不计正式模板题。

所需模板：SuffixAutomaton（第 74 页，11.1 节）、SAMLex（第 79 页，11.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3975

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    int multiple;
    long long k;
    cin >> s >> multiple >> k;
    SuffixAutomaton sam;
    for (char c : s) sam.extend(c - 'a');
    SAMLex index(sam, multiple ? sam.counts() : vector<long long>{});
    auto ans = index.kth(k);
    cout << (ans ? *ans : "-1") << '\n';
}
```

接口用法演示：广义 SAM 查询 / API 示例

演示约定：读 n 个非空小写文档，再读 26 字母排列、1-based k 及非空查询文本 t；输出文档子串并集的第 k 小串（不足-1），然后输出 t 与任一文档 LCS 在 t 内的起点和长度。不计文档重复次数，不求所有文档交集；示例 n=0 合法。

所需模板：GeneralSAM（第 84 页，12.1 节）、SAMLex（第 79 页，11.2 节）、sam_lcs（第 82 页，11.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    GeneralSAM sam;
    for (int i = 0; i < n; i++)
    {
        string s;
        cin >> s;
        sam.add(s);
    }
    sam.build();
    string alphabet, t;
    long long k;
    cin >> alphabet >> k >> t;
    SAMLex index(sam);
    auto ans = index.kth(k, alphabet);
    cout << (ans ? *ans : "-1") << '\n';
    auto [start, length] = sam_lcs(sam, t);
    cout << start << ' ' << length << '\n';
}
```

11.3 SAM 最长公共子串

sam_lcs(sam,t) 接收已有单串 SuffixAutomaton 或已 build 的 GeneralSAM 及小写文本 t，返回 t 中的 {0-based 起点, 长度}；多解取 t 中最早起点，无公共非空串返回 {0,0}。广义 SAM 时求 t 与任一已加入文档的最长公共子串，不是所有文档公共串。t 可空，长度 <INT_MAX；构建 SAM 的成本另计，查询 O(|t|)、额外空间 O(1)，输入与 SAM 不变。沿后缀链接回退并同步匹配长度，不返回源文档坐标；需实际串时用 t.substr(start,length)。

```
// Return {start, length} in t; ties choose the earliest start.
template <class S>
pair<int, int> sam_lcs(const S &sam, const string &t)
{
    int p = 0, length = 0;
    pair<int, int> ans{0, 0};
    for (int i = 0; i < (int)t.size(); i++)
    {
        int c = t[i] - 'a';
        assert(0 <= c && c < 26);
        while (p && !sam.a[p].go[c])
        {
            p = sam.a[p].link;
            length = sam.a[p].len;
        }
        if (sam.a[p].go[c])
        {
            p = sam.a[p].go[c];
            length++;
        }
        else length = 0;
        if (length > ans.second) ans = {i - length + 1, length};
    }
    return ans;
}
```

应用题补充：SPOJ LCS / 最长公共子串

题意：两行小写字符串，各长 ≤250000，输出最长公共子串长度，无公共串输出 0。getline 保留空行，并兼容 CRLF。返回位置属于第二个串，本用法只输出长度；线上 AC 和时间/内存限制待核验。

所需模板：SuffixAutomaton（第 74 页，11.1 节）、sam_lcs（第 82 页，11.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/SP1811

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s, t;
    getline(cin, s);
    getline(cin, t);
    if (!s.empty() && s.back() == '\r') s.pop_back();
    if (!t.empty() && t.back() == '\r') t.pop_back();
    SuffixAutomaton sam;
    for (char c : s) sam.extend(c - 'a');
    cout << sam_lcs(sam, t).second << '\n';
}
```

联合使用示例见第 81 页（广义 SAM 查询 / API 示例），其中直接调用本模板。

第十二章　广义后缀自动机

12.1　广义后缀自动机：Trie BFS 构建

先 add(s) 登记若干小写字符串，再 build() 原地将 Trie 变为广义 SAM；build 后禁止继续 add，可重复 build，重新收集串需新建对象。空串与重复串允许，distinct() 返回所有输入串中不同非空子串的总数，不包含跨串拼接。根为 0，a[v].go[c] 为转移、0 表示无边，link 为后缀链接、len 为最长代表串长度；从根沿字符转移可判定子串是否存在。a.size() 为按文档 endpos 等价类构造的状态数，不应直接称为后缀并集语言的最小 DFA 点数；ab、b 是两者不同的例子。P6139 官方样例一要求 endpos 点数 10，后缀语言最小化仅 7 点；该题用 a.size()，勿再最小化。固定 26 字符，设累计输入长度 L、Trie 点数 K，登记总时间 O(L)，build 时间与空间 O(26K)，状态少于 2K；累计输入长度及 2K 须可由 int 表示。克隆只复制已处理转移，不能带入尚未 BFS 处理的 Trie 子边。此版不提供出现次数或各串所属标记。新增 add(p,c) 在 build 前按 Trie 边登记，0<=c<26，p 为已有 Trie 节点；返回子节点号，重复边复用。单次均摊 O(1)，避免把输入树展开成多条长串；add(s) 复用此入口。原 Trie 编号在 build 后仍代表原根路径，但禁止继续 add。

```
struct GeneralSAM
{
    struct Node
    {
        array<int, 26> go{};
        int link = -1, len = 0;
    };

    vector<Node> a{Node{}};
    bool ready = false;

    // Trie step before build; repeated edges return the same node.
    int add(int p, int c)
    {
        assert(!ready && 0 <= p && p < (int)a.size());
        assert(0 <= c && c < 26);
        if (!a[p].go[c])
        {
            int v = a.size();
            a.push_back(Node{});
            a[p].go[c] = v;
        }
        return a[p].go[c];
    }

    void add(const string &s)
    {
        assert(!ready);
        int p = 0;
        for (char ch : s) p = add(p, ch - 'a');
    }
```

BFS 构建（接上页同一结构体）：

```
    void build()
    {
        if (ready) return;
        a.reserve(2 * a.size());
        queue<pair<int, int>> todo;
        for (int c = 0; c < 26; c++)
            if (a[0].go[c]) todo.push({0, c});
        while (!todo.empty())
        {
            auto [parent, c] = todo.front();
            todo.pop();
            int cur = a[parent].go[c], p = a[parent].link;
            a[cur].len = a[parent].len + 1;
            while (p != -1 && !a[p].go[c])
            {
                a[p].go[c] = cur;
                p = a[p].link;
            }
            if (p == -1)
                a[cur].link = 0;
            else
            {
                int q = a[p].go[c];
                if (a[p].len + 1 == a[q].len)
                    a[cur].link = q;
                else
                {
                    int clone = a.size();
                    Node copy = a[q];
                    copy.len = a[p].len + 1;
                    for (int &v : copy.go)
                        if (!a[v].len) v = 0;
                    a.push_back(copy);
                    while (p != -1 && a[p].go[c] == q)
                    {
                        a[p].go[c] = clone;
                        p = a[p].link;
                    }
                    a[q].link = a[cur].link = clone;
                }
            }
            for (int ch = 0; ch < 26; ch++)
                if (int v = a[cur].go[ch]; v && !a[v].len) todo.push({cur, ch});
        }
        ready = true;
    }
```

不同子串计数（接上页同一结构体）：

```
    long long distinct() const
    {
        assert(ready);
        long long ans = 0;
        for (int v = 1; v < (int)a.size(); v++) ans += a[v].len - a[a[v].link].len;
        return ans;
    }
};
```

模板题使用：Library Checker number_of_substrings

题意：本题只有一个字符串，输出不同非空子串数。广义模板可在 build 前多次 add；本例验证其单串特例，多串行为另有独立对拍。若只处理单串，原 SuffixAutomaton 更合适。此题不验证广义 SAM 状态数或最小 DFA 状态数。

所需模板：GeneralSAM（第 84 页，12.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/number_of_substrings

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    GeneralSAM sam;
    sam.add(s);
    sam.build();
    cout << sam.distinct() << '\n';
}
```

模板题使用：Luogu P6139

题意：读入所有小写字符串，求不同非空子串数及含根的广义 SAM 点数。先全部 add，再 build，输出 distinct 和 a.size；总长度至 100 万、文档数至 40 万。点数按官方样例的 endpos 等价类口径：aa、ab、bac、caa 输出 10 和 10，不另做后缀语言 DFA 最小化。本例通过完整驱动本地验证，线上待补。

所需模板：GeneralSAM（第 84 页，12.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6139

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    GeneralSAM sam;
    while (n--)
    {
        string s;
        cin >> s;
        sam.add(s);
    }
    sam.build();
    cout << sam.distinct() << '\n' << sam.a.size() << '\n';
}
```

接口用法演示：直接 Trie 建图与含空串排名 / API 示例

演示约定：输入 n,q，再给 n 条 parent,c（0≤parent<i），节点 0 为空根；重复同标签边可合并。一次 BFS 构建后输出含空串的不同子串数。每次输入字母序排列和 1-based k，k=1 输出空行，其余调用 kth(k-1)，不足输出-1。自行约定的 API 演示，不冒充 TSUBSTR 官方驱动。

所需模板：GeneralSAM（第 84 页，12.1 节）、SAMLex（第 79 页，11.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    GeneralSAM sam;
    vector<int> state(n + 1);
    for (int i = 1; i <= n; i++)
    {
        int parent;
        char c;
        cin >> parent >> c;
        state[i] = sam.add(state[parent], c - 'a');
    }
    sam.build();
    cout << sam.distinct() + 1 << '\n';
    SAMLex index(sam);
    while (q--)
    {
        string alphabet;
        long long k;
        cin >> alphabet >> k;
        if (k == 1) cout << '\n';
        else
        {
            auto ans = index.kth(k - 1, alphabet);
            cout << (ans ? *ans : "-1") << '\n';
        }
    }
}
```

应用题补充：CodeChef TSUBSTR / Substrings on a Tree

题意：给以 1 为根的无向树及每个顶点的小写字母，求所有祖先到后代路径串中不同串的数量，包含空串；每次按给定 26 字母排列求第 k 小，不足输出-1，k=1 输出空行。N 至 250000、Q 至 50000，k 至 2^63-1，总输出不超过 800KB，原题字母随机。根顶点字母也必须登记；重复标签路径由 Trie 合并。先按树做 BFS 登记，再构建 SAM，不展开根叶字符串。

所需模板：GeneralSAM（第 84 页，12.1 节）、SAMLex（第 79 页，11.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.codechef.com/problems/TSUBSTR

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    string s;
    cin >> n >> q >> s;
    vector<vector<int>> g(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        u--;
        v--;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    GeneralSAM sam;
    vector<int> par(n, -1), state(n);
    state[0] = sam.add(0, s[0] - 'a');
    queue<int> todo;
    todo.push(0);
    while (!todo.empty())
    {
        int u = todo.front();
        todo.pop();
        for (int v : g[u])
        {
            if (v == par[u]) continue;
            par[v] = u;
            state[v] = sam.add(state[u], s[v] - 'a');
            todo.push(v);
        }
    }
```

继续处理操作（接上页同一 main）：

```
    sam.build();
    cout << sam.distinct() + 1 << '\n';
    SAMLex index(sam);
    while (q--)
    {
        string alphabet;
        long long k;
        cin >> alphabet >> k;
        if (k == 1) cout << '\n';
        else
        {
            auto ans = index.kth(k - 1, alphabet);
            cout << (ans ? *ans : "-1") << '\n';
        }
    }
}
```

相关应用：指定状态在线扩展；任意插入顺序不保证 Trie 节点数线性复杂度。

12.2 指定状态在线扩展 SAM

OnlineSAM() 以 0 为根；extend(p,c) 给状态 p 代表的最长串末尾接字符 0<=c<26，返回该串的状态号，调用后立即可沿 go 判断已有子串。旧状态的 len 与编号保持，但 link/go 可变化；不可用短于 len[p] 的匹配串冒充该最长串。逐串输入时每串从 p=0 开始；重复路径允许。total 为当前不同非空子串数，a 含根，公开字段只读；不自动统计出现次数、文档频率或终态。一次至多增加 2 状态，需状态数和长度可用 int 索引，数组可分配。单次 O(当前最大串长 D+26)，Q 次保守 O(QD+26Q)，任意顺序可能平方级；已知整棵 Trie 优先 GeneralSAM 的 BFS 构造。空间 O(26Q)，重置赋新对象。内部递归只进入已有转移分支，深度至多 2；构建完成后可用 SAMLex/sam_lcs，继续扩展前先弃用旧查询索引。

```
struct OnlineSAM
{
    struct Node
    {
        array<int, 26> go{};
        int link = -1, len = 0;
    };

    vector<Node> a{Node{}};
    long long total = 0;
```

指定状态在线扩展（接上页同一结构体）：

```
    // Append c to the longest string represented by state p.
    int extend(int p, int c)
    {
        assert(0 <= p && p < (int)a.size() && 0 <= c && c < 26);
        if (int q = a[p].go[c])
        {
            if (a[q].len == a[p].len + 1) return q;
            int clone = a.size();
            Node copy = a[q];
            copy.len = a[p].len + 1;
            a.push_back(copy);
            a[q].link = clone;
            while (p != -1 && a[p].go[c] == q)
            {
                a[p].go[c] = clone;
                p = a[p].link;
            }
            return clone;
        }
        int cur = a.size();
        Node node;
        node.len = a[p].len + 1;
        a.push_back(node);
        while (p != -1 && !a[p].go[c])
        {
            a[p].go[c] = cur;
            p = a[p].link;
        }
        int parent = p == -1 ? 0 : extend(p, c);
        a[cur].link = parent;
        total += a[cur].len - a[parent].len;
        return cur;
    }
};
```

接口用法演示：在线分支扩展 / API 示例

演示约定：q 次操作，每次给历史版本 parent（0≤parent<i）和小写字符 c；将该版本的完整串接 c 作为新版本，版本 0 为空。保存返回状态号，每次输出当前全部版本的不同非空子串并集大小。这是添加路径，不是持久化 SAM：所有查询读取累计语言。

所需模板：OnlineSAM（第 91 页，12.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    OnlineSAM sam;
    vector<int> state(q + 1);
    for (int i = 1; i <= q; i++)
    {
        int parent;
        char c;
        cin >> parent >> c;
        state[i] = sam.extend(state[parent], c - 'a');
        cout << sam.total << '\n';
    }
}
```

模板题使用：Luogu P6139 / 在线扩展版本

题意：每个文档从根重新扩展，输出不同非空子串数和含根 endpos 状态数；题面最多 40 万串、总长至 100 万。与已有 GeneralSAM 相同的状态口径，不做后缀语言 DFA 最小化；本地验证，线上待补。任意父状态 API 的最坏复杂度不能据此称线性。

所需模板：OnlineSAM（第 91 页，12.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6139

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    OnlineSAM sam;
    for (int i = 0; i < n; i++)
    {
        string s;
        cin >> s;
        int p = 0;
        for (char c : s) p = sam.extend(p, c - 'a');
    }
    cout << sam.total << '\n';
    cout << sam.a.size() << '\n';
}
```

12.3 SAM 文档频率与阈值后缀

代码依赖：Fenwick（第 412 页，33.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

SAMDocuments(sam,s) 接收已完成构建的 GeneralSAM 及同一批小写文档 s；也兼容按该文档集完成的 OnlineSAM 或单文档 SuffixAutomaton。允许空串和重复串，重复输入按不同文档计数。索引期间 sam 须存活且不变。cnt[u] 是该状态任一非空代表串出现的文档数，cnt[0]=s.size() 约定空串出现在每篇文档；不能把单串出现次数当文档数。lengths(k) 要求 k>=1，返回每个状态最长代表串中、出现于至少 k 篇文档的最长后缀长度，无非空解为 0；k 大于文档数全 0。查询任意非空模式串时，从根转移成功后读取 cnt，缺边返回 0。L 为文档总长，V 为状态数，D 为文档数：构造 O((L+V)log(L+2)+D)，空间 O(L+V+D)，每次 lengths 为 O(V)。递归后缀链接树深度不超过最长文档长度加 1；保留递归，不模拟开栈。L、D、V 须小于 INT_MAX，数组可分配；a/g/cnt 视为只读。

```
template <class S>
struct SAMDocuments
{
    const S &sam;
    vector<vector<int>> g;
    vector<int> cnt;
```

文档标记与子树去重（接上页同一结构体）：

```
    SAMDocuments(const S &sam, const vector<string> &s)
        : sam(sam), g(sam.a.size()), cnt(sam.a.size())
    {
        int n = sam.a.size(), size = 0;
        vector<vector<int>> tag(n);
        for (int u = 1; u < n; u++) g[sam.a[u].link].push_back(u);
        for (int id = 0; id < (int)s.size(); id++)
        {
            int p = 0, len = 0;
            for (char c : s[id])
            {
                assert('a' <= c && c <= 'z');
                p = sam.a[p].go[c - 'a'];
                len++;
                assert(p && sam.a[p].len == len);
                tag[p].push_back(id);
                size++;
            }
        }
        Fenwick<int> bit(size);
        vector<int> last(s.size());
        int timer = 0;
        auto dfs = [&](auto &&self, int u) -> void
        {
            int l = timer + 1;
            for (int id : tag[u])
            {
                if (last[id]) bit.add(last[id], -1);
                last[id] = ++timer;
                bit.add(timer, 1);
            }
            for (int v : g[u]) self(self, v);
            cnt[u] = bit.query(l, timer);
        };
        dfs(dfs, 0);
        cnt[0] = s.size();
    }
```

阈值后缀长度（接上页同一结构体）：

```
    // Longest suffix length occurring in at least k documents, for each state.
    vector<int> lengths(int k) const
    {
        assert(k >= 1);
        vector<int> best(cnt.size());
        auto dfs = [&](auto &&self, int u) -> void
        {
            if (cnt[u] >= k) best[u] = sam.a[u].len;
            for (int v : g[u])
            {
                best[v] = best[u];
                self(self, v);
            }
        };
        dfs(dfs, 0);
        return best;
    }
};
```

应用题补充：Codeforces 204E / Little Elephant and Strings

题意：给 n 篇非空小写文档和 k，分别求每篇中出现于至少 k 篇文档的子串区间个数；重复文档分别计数，区间重复出现也分别计数。n、k 及总长度至 100000，k 允许大于 n。先统计文档频率，再按每个前缀累加合格最长后缀长度；答案用 long long。比赛应用，不登记为非比赛模板题。

所需模板：GeneralSAM（第 84 页，12.1 节）、Fenwick（第 412 页，33.3 节）、SAMDocuments（第 94 页，12.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/204/E

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    vector<string> s(n);
    GeneralSAM sam;
    for (auto &t : s)
    {
        cin >> t;
        sam.add(t);
    }
    sam.build();
    SAMDocuments index(sam, s);
    auto best = index.lengths(k);
    for (int id = 0; id < n; id++)
    {
        int p = 0;
        long long ans = 0;
        for (char c : s[id])
        {
            p = sam.a[p].go[c - 'a'];
            ans += best[p];
        }
        cout << ans << (id + 1 == n ? '\n' : ' ');
    }
}
```

接口用法演示：子串文档频率 / API 示例

演示约定：输入 n,q、n 篇非空小写文档和 q 个非空小写模式串，输出每个模式出现于多少篇输入文档；同一文档内重复出现只算一次，相同输入文档分别计数。走转移失败立即返回 0。本例 n 可为 0；核心另支持空文档，空串频率为 cnt[0]。

所需模板：GeneralSAM（第 84 页，12.1 节）、Fenwick（第 412 页，33.3 节）、SAMDocuments（第 94 页，12.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<string> s(n);
    GeneralSAM sam;
    for (auto &t : s)
    {
        cin >> t;
        sam.add(t);
    }
    sam.build();
    SAMDocuments index(sam, s);
    while (q--)
    {
        string t;
        cin >> t;
        int p = 0;
        for (char c : t)
        {
            p = sam.a[p].go[c - 'a'];
            if (!p) break;
        }
        cout << (p ? index.cnt[p] : 0) << '\n';
    }
}
```

第十三章　Manacher

13.1　Manacher 回文半径

manacher(s) 返回 odd/even，下标从 0 开始。odd[i] 包含中心，闭区间 [i-odd[i]+1,i+odd[i]-1]，长度 2*odd[i]-1；even[i] 的中心在 i 前，区间 [i-even[i],i+even[i]-1]，长度 2*even[i]，零半径为空。空串合法，支持任意字节（含零），按字节而非 Unicode 字素比较；int 索引要求 n<2^30。最长回文取上述长度最大值；全部 odd/even 以 long long 求和得到回文出现总数。O(n) 时间和空间，边界显式检查，无需选择哨兵字符。输入为 string、vector<int> 或 vector<long long> 对象，支持空序列和任意整数；泛型 S 需 size()、随机下标及 O(1) 相等比较。字符串字面量请显式写 string("abc")。输入不拷贝，输出移动返回。

```
// odd[i] includes the center; even[i] is centered before i.
template <class S>
pair<vector<int>, vector<int>> manacher(const S &s)
{
    int n = (int)s.size();
    vector<int> odd(n), even(n);
    for (int i = 0, l = 0, r = -1; i < n; i++)
    {
        int k = i > r ? 1 : min(odd[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) ++k;
        odd[i] = k--;
        if (i + k > r)
        {
            l = i - k;
            r = i + k;
        }
    }
    for (int i = 0, l = 0, r = -1; i < n; i++)
    {
        int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) ++k;
        even[i] = k--;
        if (i + k > r)
        {
            l = i - k - 1;
            r = i + k;
        }
    }
    return {move(odd), move(even)};
}
```

模板题使用：Library Checker enumerate_palindromes

题意：给非空字符串，从左到右输出每个字符及相邻字符间隙处的最长回文长度，共 2n-1 项。odd 含中心，even[i] 的中心在 i 前面，因此分别转成 2*odd[i]-1 和 2*even[i]。

所需模板：manacher（第 99 页，13.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/enumerate_palindromes

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    int n = s.size();
    auto [odd, even] = manacher(s);
    vector<int> answer(2 * n - 1);
    for (int i = 0; i < n; i++)
    {
        answer[2 * i] = 2 * odd[i] - 1;
        if (i) answer[2 * i - 1] = 2 * even[i];
    }
    for (int i = 0; i < (int)answer.size(); i++)
        cout << answer[i] << (i + 1 == (int)answer.size() ? '\n' : ' ');
}
```

模板题使用：Luogu P3805

题意：给定长度不超过 11000000 的小写字符串，输出最长回文子串长度。odd 含中心，even[i] 以 i 前间隙为中心，分别用 2*odd[i]-1 和 2*even[i] 取最大值。

所需模板：manacher（第 99 页，13.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3805

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    auto [odd, even] = manacher(s);
    int answer = 0;
    for (int i = 0; i < (int)s.size(); i++)
    {
        answer = max(answer, 2 * odd[i] - 1);
        answer = max(answer, 2 * even[i]);
    }
    cout << answer << '\n';
}
```

接口用法演示：整数序列回文半径 / API 示例

演示约定：输入 n 及 n 个 signed64 整数，输出两行 odd/even 半径，均 0-based；odd 含中心，even 中心在 i 之前。n=0 输出两空行。接口示例，不对应正式模板题或线上 AC。

所需模板：manacher（第 99 页，13.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    auto [odd, even] = manacher(a);
    for (int x : odd) cout << x << ' ';
    cout << '\n';
    for (int x : even) cout << x << ' ';
    cout << '\n';
}
```

第十四章　回文树

14.1　回文树 PAM：回文后缀与出现次数

add(ch) 追加小写英文字母，返回最长回文后缀节点，亦保存于 last。节点 0 是长度 0 根，1 是长度 -1 根，实际回文节点从 2 开始；distinct() 为不同非空回文串数。t[u].len 为长度，fail 为最长真回文后缀链接，suffixes 为该节点的非空回文后缀数量；追加后的 t[last].suffixes 就是本位置结尾的回文串数。total 为当前全部回文子串的出现次数之和。end 为首次出现时的 0-based 右端点，可恢复原串子段；hits 仅统计作为最长后缀的次数，不能当出现次数。occurrences() 返回按节点编号的完整出现次数副本，根置零，O(n)；重复调用以及查询后继续追加均安全。重新赋值 PalindromicTree() 清空重建。固定 26 字符集下建树 O(n)，空间 O(26n)。

```
struct PalindromicTree
{
    struct Node
    {
        int next[26]{};
        int len = 0, fail = 0, suffixes = 0, end = -1;
        long long hits = 0;
    };

    vector<Node> t;
    vector<int> s;
    int last = 0;
    long long total = 0;

    PalindromicTree() : t(2), s(1, -1)
    {
        t[0].fail = 1;
        t[1].len = -1;
    }

    int get_fail(int u) const
    {
        int pos = s.size() - 1;
        while (s[pos - t[u].len - 1] != s[pos]) u = t[u].fail;
        return u;
    }
```

接上页同一结构体：

```
    // Append a lowercase letter; return the longest palindromic suffix node.
    int add(char ch)
    {
        int c = ch - 'a';
        assert(0 <= c && c < 26);
        s.push_back(c);
        int u = get_fail(last);
        if (!t[u].next[c])
        {
            int v = t.size();
            t.push_back(Node{});
            t[v].len = t[u].len + 2;
            t[v].fail = t[get_fail(t[u].fail)].next[c];
            t[v].suffixes = t[t[v].fail].suffixes + 1;
            t[v].end = (int)s.size() - 2;
            t[u].next[c] = v;
        }
        last = t[u].next[c];
        t[last].hits++;
        total += t[last].suffixes;
        return last;
    }

    int distinct() const { return (int)t.size() - 2; }

    // Copy-and-propagate: repeated calls and later appends remain valid.
    vector<long long> occurrences() const
    {
        vector<long long> count(t.size());
        for (int u = 2; u < (int)t.size(); u++) count[u] = t[u].hits;
        for (int u = (int)t.size() - 1; u >= 2; u--) count[t[u].fail] += count[u];
        count[0] = 0;
        count[1] = 0;
        return count;
    }
};
```

模板题使用：Luogu P5496

题意：按上一次答案把当前字符循环偏移后追加，输出以当前位置结尾的回文串个数。add 返回最长回文后缀节点，读取其 suffixes。

所需模板：PalindromicTree（第 102 页，14.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5496

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::string s;
    std::cin >> s;
    PalindromicTree pam;
    int answer = 0;
    for (int i = 0; i < (int)s.size(); i++)
    {
        char ch = 'a' + (s[i] - 'a' + answer) % 26;
        int u = pam.add(ch);
        answer = pam.t[u].suffixes;
        std::cout << answer << (i + 1 == (int)s.size() ? '\n' : ' ');
    }
}
```

应用题补充：Luogu P3649

题意：求所有回文子串中“长度乘出现次数”的最大值。整串追加完后调用 occurrences 汇总次数，不能直接把 hits 当完整出现次数。

所需模板：PalindromicTree（第 102 页，14.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3649

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::string s;
    std::cin >> s;
    PalindromicTree pam;
    for (char ch : s) pam.add(ch);
    auto count = pam.occurrences();
    long long answer = 0;
    for (int u = 2; u < (int)count.size(); u++)
        answer = std::max(answer, count[u] * pam.t[u].len);
    std::cout << answer << '\n';
}
```

第十五章　最小表示法

15.1　最小循环表示

minimum_rotation(s) 返回字典序最小循环表示的起点 (0-based)，空串返回 0；按无符号字节比较，与 string 字典序一致。O(n)。

```
int minimum_rotation(const string &s)
{
    int n = (int)s.size(), i = 0, j = 1, k = 0;
    if (!n) return 0;
    while (i < n && j < n && k < n)
    {
        unsigned char a = s[(i + k) % n], b = s[(j + k) % n];
        if (a == b)
        {
            ++k;
            continue;
        }
        if (a > b)
        {
            i += k + 1;
            if (i == j) ++i;
        }
        else
        {
            j += k + 1;
            if (i == j) ++j;
        }
        k = 0;
    }
    return min(i, j);
}
```

模板题使用：Luogu P13270【模板】最小表示法

题意：给定长度 1..10^7 的小写字母字符串，输出所有循环位移中字典序最小的字符串。minimum_rotation 返回 0-based 起点，使用 rotate 原地重排后输出；O(n) 时间，算法本体 O(1) 额外空间。

所需模板：minimum_rotation（第 105 页，15.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P13270

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    string s;
    if (!(cin >> n >> s)) return 0;
    int p = minimum_rotation(s);
    rotate(s.begin(), s.begin() + p, s.end());
    cout << s << '\n';
}
```

第十六章　Lyndon 分解

16.1　Lyndon 分解：Duval

lyndon(s) 返回严格递增的因子右端点 ends；按 0-based 半开区间 [0,ends[0])、[ends[i-1],ends[i]) 取因子，数值恰好也是 1-based 闭区间右端点。各因子严格小于自己的所有非空真后缀，因子序列按字典序非递增，分解唯一。输入按 unsigned char 的 0..255 字节序比较，支持空串、零字节与高位字节；不是 Unicode 字符顺序。空串返回空 vector，s 不变，长度须 ≤INT_MAX。Duval 时间 O(n)，预留 n 个 int 的输出容量，空间 O(n)，除返回数组外只需 O(1) 工作空间；不复制子串。P6114 直接异或 ends，无需再加 1；全部相同字符会拆成 n 个单字符，不能合并相等因子。

```
vector<int> lyndon(const string &s)
{
    assert(s.size() <= INT_MAX);
    int n = s.size();
    vector<int> ends;
    ends.reserve(n);
    int i = 0;
    while (i < n)
    {
        int j = i + 1, k = i;
        while (j < n && (unsigned char)s[k] <= (unsigned char)s[j])
        {
            if (s[k] == s[j])
                k++;
            else
                k = i;
            j++;
        }
        int len = j - k;
        while (i <= k)
        {
            i += len;
            ends.push_back(i);
        }
    }
    return ends;
}
```

模板题使用：P6114 【模板】Lyndon 分解

题意：输入小写字符串，输出非递增 Lyndon 分解的全部 1-based 右端点异或和；n≤5000001。

所需模板：lyndon（第 107 页，16.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6114

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    int ans = 0;
    for (int r : lyndon(s))
        ans ^= r;
    cout << ans << '\n';
}
```

接口用法演示：API 字节串完整分解

演示约定：输入 n 和 n 个 0..255 整数作为字节串，可为空；输出因子个数及全部右端点，按 unsigned char 字节序。

所需模板：lyndon（第 107 页，16.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/string/lyndon/

```
int main()
{
    int n;
    cin >> n;
    string s(n, '\0');
    for (int i = 0; i < n; i++)
    {
        int x;
        cin >> x;
        s[i] = (char)x;
    }
    auto ends = lyndon(s);
    cout << ends.size() << '\n';
    for (int r : ends)
        cout << r << ' ';
    cout << '\n';
}
```

第四部分

数学

第十七章　高精度计算

相关应用：约分有理数：signed64 值与 signed128 中间值，或 cpp_int 任意精度；不是实数浮点模板

17.1　精确分数四则运算

Rational<> 默认保存 long long 分子 num 与正分母 den，所有构造和运算后立即约分，零固定为 0/1。构造 Rational<>(n,d)，默认 0/1；支持四则、复合赋值、负号、六种比较和输出，整值输出整数，其余输出 num/den。x+=x、x-=x、x*=x、非零 x 的 x/=x 均有效。num/den 可读，不要直接修改以破坏约分和正分母不变量。

默认以 signed128 计算中间值，合法 64 位规范分数的交叉乘积及加减均可表示；要求每次构造及运算后的最简分子在 signed64 内、正分母 ≤LLONG_MAX，除数非零。不要求约分前的积能放入 64 位。比如 LLONG_MIN 除以 LLONG_MIN 可约为 1，而 1 除以 LLONG_MIN 的正分母无法表示，后者不属于接口域；LLONG_MIN 取负也超域。assert 仅辅助检查前置条件，NDEBUG 下仍必须遵守。

任意精度用 Rational<cpp_int,cpp_int>，额外包含 Boost 多精度整数头文件，详见完整用法；按位数承担大整数乘除及 gcd 成本，不能把它当常数时间。其他 T/W 组合须自行证明中间值与符号处理范围。assign/make 仅内部辅助，外部调用构造和运算符。double 显式转换只是近似；大整数分子分母各自转 double 可能溢出，不用于精确判定。比较始终采用整数。

```
template<class T = long long, class W = __int128_t>
struct Rational
{
    T num = 0, den = 1;

    void assign(W n, W d)
    {
        assert(d != 0);
        if (d < 0)
        {
            n = -n;
            d = -d;
        }
        W a = n < 0 ? -n : n, b = d;
        while (b != 0)
        {
            W r = a % b;
            a = b;
            b = r;
        }
        n /= a;
        d /= a;
        if constexpr (numeric_limits<T>::is_bounded)
        {
            assert(n >= W(numeric_limits<T>::lowest()));
            assert(n <= W(numeric_limits<T>::max()));
            assert(d <= W(numeric_limits<T>::max()));
        }
        num = T(n);
        den = T(d);
    }
```

精确分数（接上页同一结构体）：

```
    Rational(T n = 0, T d = 1)
    {
        assign(W(n), W(d));
    }

    static Rational make(W n, W d)
    {
        Rational r;
        r.assign(n, d);
        return r;
    }

    friend Rational operator+(const Rational &a, const Rational &b)
    {
        return make(W(a.num) * b.den + W(b.num) * a.den, W(a.den) * b.den);
    }

    friend Rational operator-(const Rational &a, const Rational &b)
    {
        return make(W(a.num) * b.den - W(b.num) * a.den, W(a.den) * b.den);
    }

    friend Rational operator*(const Rational &a, const Rational &b)
    {
        return make(W(a.num) * b.num, W(a.den) * b.den);
    }

    friend Rational operator/(const Rational &a, const Rational &b)
    {
        assert(b.num != 0);
        return make(W(a.num) * b.den, W(a.den) * b.num);
    }

    Rational operator-() const
    {
        return make(-W(num), W(den));
    }

    Rational &operator+=(const Rational &b)
    {
        return *this = *this + b;
    }

    Rational &operator-=(const Rational &b)
    {
        return *this = *this - b;
    }
```

精确分数（接上页同一结构体）：

```
    Rational &operator*=(const Rational &b)
    {
        return *this = *this * b;
    }

    Rational &operator/=(const Rational &b)
    {
        return *this = *this / b;
    }

    friend bool operator==(const Rational &a, const Rational &b)
    {
        return a.num == b.num && a.den == b.den;
    }

    friend bool operator!=(const Rational &a, const Rational &b)
    {
        return !(a == b);
    }

    friend bool operator<(const Rational &a, const Rational &b)
    {
        return W(a.num) * b.den < W(b.num) * a.den;
    }

    friend bool operator>(const Rational &a, const Rational &b)
    {
        return b < a;
    }

    friend bool operator<=(const Rational &a, const Rational &b)
    {
        return !(b < a);
    }

    friend bool operator>=(const Rational &a, const Rational &b)
    {
        return !(a < b);
    }

    explicit operator double() const
    {
        return double(num) / double(den);
    }

    friend ostream &operator<<(ostream &out, const Rational &a)
    {
        out << a.num;
        if (a.den != 1)
            out << '/' << a.den;
        return out;
    }
};
```

模板题使用：P1572 计算分数

题意：读取只含分数、加减号的表达式，输出最简分数或整数；用输入流直接读带符号分子。原题保证逐项约分后的中间结果在 int 范围内，故默认 64 位版足够。本地已验证，尚无本次线上 AC。

所需模板：Rational（第 110 页，17.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1572

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    Rational<> sum;
    long long num, den;
    char slash;
    while (cin >> num >> slash >> den)
        sum += Rational<>(num, den);
    cout << sum << '\n';
}
```

接口用法演示：任意精度分数 API

演示约定：先读 q，随后每行 an ad op bn bd；分母非零，op 为 +、-、*、/或 <，除法要求 bn 非零。前四种输出最简结果，< 输出比较值-1/0/1。需要额外抄入 Boost 头文件，cpp_int 不依赖本书 BigInteger。整数长度不受 64 位限制，时间与内存随位数增长。

所需模板：Rational（第 110 页，17.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

额外依赖（不属于标准头文件）：`boost/multiprecision/cpp_int.hpp`，须由编译环境提供并显式包含。

来源：参考文档

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using boost::multiprecision::cpp_int;
    using F = Rational<cpp_int, cpp_int>;
    int q;
    cin >> q;
    while (q--)
    {
        cpp_int an, ad, bn, bd;
        char op;
        cin >> an >> ad >> op >> bn >> bd;
        F a(an, ad), b(bn, bd);
        if (op == '+')
            cout << a + b << '\n';
        else if (op == '-')
            cout << a - b << '\n';
        else if (op == '*')
            cout << a * b << '\n';
        else if (op == '/')
            cout << a / b << '\n';
        else
            cout << (a < b ? -1 : a == b ? 0 : 1) << '\n';
    }
}
```

17.2 高精度整数：有符号加减乘与小整数除法

BigInteger(x) 接收 long long，BigInteger(s) 接收可带正负号及前导零的非空十进制串；str() 返回规范化十进制，cmp(b) 返回 −1/0/1。支持一元负号、加减乘，divmod(int b) 返回商和余数，b≠0；商向零截断，余数与被除数同号或为 0。pow(unsigned e) 计算非负整数幂，约定 0 的 0 次方为 1。

d 是低位在前的 10^9 进制绝对值，sign 为 ±1；零为空 d 且 sign=1，勿直接修改。n,m 为分节数：加减比较 O(n+m)，乘法 O(nm+n+m)，小整数除法 O(n)，结果空间 O(n+m)；幂执行 O(log(e+1)) 次乘法，位数增长另计。字符串长度、所有中间分节数及两操作数长度之和须小于 INT_MAX 并留进位空间。不含大整数除大整数；不用浮点或 Boost。

```
struct BigInteger
{
    static constexpr int base = 1000000000;
    vector<int> d;
    int sign = 1;
    void trim()
    {
        while (!d.empty() && d.back() == 0)
            d.pop_back();
        if (d.empty())
            sign = 1;
    }
    BigInteger(long long x = 0)
    {
        sign = x < 0 ? -1 : 1;
        auto v = x < 0 ? 0ULL - (unsigned long long)x : (unsigned long long)x;
        while (v)
        {
            d.push_back(v % base);
            v /= base;
        }
        trim();
    }
```

十进制构造、输出与比较（接上页结构体）：

```
    explicit BigInteger(const string &s)
    {
        assert(!s.empty());
        int first = s[0] == '-' || s[0] == '+';
        sign = s[0] == '-' ? -1 : 1;
        assert(first < (int)s.size());
        for (int r = int(s.size()); r > first; r -= 9)
        {
            int v = 0;
            for (int i = max(first, r - 9); i < r; i++)
            {
                assert('0' <= s[i] && s[i] <= '9');
                v = v * 10 + s[i] - '0';
            }
            d.push_back(v);
        }
        trim();
    }
    string str() const
    {
        if (d.empty())
            return "0";
        string s = sign < 0 ? "-" : "";
        s += to_string(d.back());
        for (int i = int(d.size()) - 2; i >= 0; i--)
        {
            string t = to_string(d[i]);
            s += string(9 - t.size(), '0') + t;
        }
        return s;
    }
    int abs_cmp(const BigInteger &b) const
    {
        if (d.size() != b.d.size())
            return d.size() < b.d.size() ? -1 : 1;
        for (int i = int(d.size()) - 1; i >= 0; i--)
            if (d[i] != b.d[i])
                return d[i] < b.d[i] ? -1 : 1;
        return 0;
    }
    int cmp(const BigInteger &b) const
    {
        if (sign != b.sign)
            return sign < b.sign ? -1 : 1;
        return sign * abs_cmp(b);
    }
```

取负与加减法（接上页结构体）：

```
    BigInteger operator-() const
    {
        BigInteger c = *this;
        if (!c.d.empty())
            c.sign = -c.sign;
        return c;
    }
    BigInteger operator+(const BigInteger &b) const
    {
        if (sign != b.sign)
        {
            if (abs_cmp(b) < 0)
                return b + *this;
            BigInteger c = *this;
            int borrow = 0;
            for (int i = 0; i < (int)d.size(); i++)
            {
                int v = d[i] - borrow;
                if (i < (int)b.d.size())
                    v -= b.d[i];
                borrow = v < 0;
                c.d[i] = v + borrow * base;
            }
            c.trim();
            return c;
        }
        BigInteger c;
        c.sign = sign;
        int carry = 0;
        int n = max(d.size(), b.d.size());
        for (int i = 0; i < n || carry; i++)
        {
            int v = carry;
            if (i < (int)d.size())
                v += d[i];
            if (i < (int)b.d.size())
                v += b.d[i];
            c.d.push_back(v % base);
            carry = v / base;
        }
        return c;
    }
    BigInteger operator-(const BigInteger &b) const
    {
        return *this + (-b);
    }
```

乘法、小整数除法与整数幂（接上页结构体）：

```
    BigInteger operator*(const BigInteger &b) const
    {
        BigInteger c;
        c.sign = sign * b.sign;
        c.d.resize(d.size() + b.d.size());
        for (int i = 0; i < (int)d.size(); i++)
        {
            long long carry = 0;
            for (int j = 0; j < (int)b.d.size(); j++)
            {
                long long v = 1LL * d[i] * b.d[j] + c.d[i + j] + carry;
                c.d[i + j] = v % base;
                carry = v / base;
            }
            c.d[i + b.d.size()] = carry;
        }
        c.trim();
        return c;
    }
    pair<BigInteger, int> divmod(int b) const
    {
        assert(b != 0);
        long long v = b < 0 ? -1LL * b : b;
        long long rem = 0;
        BigInteger q;
        q.sign = sign * (b < 0 ? -1 : 1);
        q.d.resize(d.size());
        for (int i = int(d.size()) - 1; i >= 0; i--)
        {
            rem = rem * base + d[i];
            q.d[i] = rem / v;
            rem %= v;
        }
        q.trim();
        return {q, int(sign * rem)};
    }
    BigInteger pow(unsigned int e) const
    {
        BigInteger a = *this;
        BigInteger r = 1;
        while (e)
        {
            if (e & 1)
                r = r * a;
            e >>= 1;
            if (e)
                a = a * a;
        }
        return r;
    }
};
```

模板题使用：P1480 高精度除法 I

题意：0≤a≤10^5000，1≤b≤10^9；输出整数商。divmod 同时返回余数，这里只打印 first。完整协议本地通过，在线状态见验证记录。

所需模板：BigInteger（第 115 页，17.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1480

```
int main()
{
    string s;
    int b;
    cin >> s >> b;
    BigInteger a(s);
    cout << a.divmod(b).first.str() << '\n';
}
```

模板题使用：P2142 高精度减法

题意：输入两个正整数，输出 a−b，结果可能为负；字符串构造后直接减法。完整协议本地通过，尚无本实现在线 AC。

所需模板：BigInteger（第 115 页，17.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2142

```
int main()
{
    string s, t;
    cin >> s >> t;
    BigInteger a(s), b(t);
    cout << (a - b).str() << '\n';
}
```

模板题使用：P1303 高精度乘法

题意：读取两个非负十进制整数，输出乘积。竖式压位乘法，不适用于要求 NTT 复杂度的超长输入。完整协议本地通过，尚无本实现在线 AC。

所需模板：BigInteger（第 115 页，17.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1303

```
int main()
{
    string s, t;
    cin >> s >> t;
    BigInteger a(s), b(t);
    cout << (a * b).str() << '\n';
}
```

接口用法演示：HDU1134 来源模型：卡特兰数

演示约定：按照 kuangbin 完整来源程序，预处理 C0..C100，反复读 n，−1 结束。递推 Ci=Ci−1(4i−2)/(i+1)，每步整除。官方题面暂未读取成功，当前仅来源模型，不计正式题目或 AC。

所需模板：BigInteger（第 115 页，17.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/BIG-INTEGER.md

```
int main()
{
    vector<BigInteger> f(101);
    f[0] = 1;
    for (int i = 1; i <= 100; i++)
        f[i] = (f[i - 1] * BigInteger(4 * i - 2)).divmod(i + 1).first;
    int n;
    while (cin >> n && n != -1)
        cout << f[n].str() << '\n';
}
```

接口用法演示：有符号高精度运算接口

演示约定：输入 a,b、非零 int 除数 d 和非负 unsigned 指数 e。依次输出和、差、积、比较结果、a/d 的商与余数、a 的 e 次方。负数除法向零截断；控制指数和位数以免结果过大。这是接口演示，不计 OJ。

所需模板：BigInteger（第 115 页，17.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/BIG-INTEGER.md

```
int main()
{
    string s, t;
    int d;
    unsigned int e;
    cin >> s >> t >> d >> e;
    BigInteger a(s), b(t);
    cout << (a + b).str() << '\n';
    cout << (a - b).str() << '\n';
    cout << (a * b).str() << '\n';
    cout << a.cmp(b) << '\n';
    auto [q, r] = a.divmod(d);
    cout << q.str() << ' ' << r << '\n';
    cout << a.pow(e).str() << '\n';
}
```

第十八章　快速幂

18.1　无符号 64 位模乘与模幂

Mod64::mul(a,b,m) 与 power(a,e,m)，a、b、e 支持完整 unsigned long long，m 为非零无符号 64 位模数。乘积用 uint128 后取模，不通过浮点近似；m=1 返回 0，e=0 返回 1 mod m。mul 为 O(1)，power 为 O(log(e+1)) 次模乘、O(1) 空间。它也是 64 位判素的底层组件。

```
struct Mod64
{
    using ull = unsigned long long;
    using u128 = __uint128_t;

    static ull mul(ull a, ull b, ull m) { return u128(a) * b % m; }

    static ull power(ull a, ull b, ull m)
    {
        assert(m);
        ull r = 1 % m;
        a %= m;
        for (; b; b >>= 1, a = mul(a, a, m))
            if (b & 1) r = mul(r, a, m);
        return r;
    }
};
```

接口用法演示：完整无符号 64 位模运算（接口演示）

演示约定：自定义协议，非 OJ 题：0≤q≤200000，随后 q 行 a b m，无符号十进制 0≤a,b≤2^64−1、1≤m≤2^64−1；每行输出 a*b mod m 及 a^b mod m。用 unsigned long long 直接读写，不先转有符号类型；乘法前提升 uint128。模 1 两项均 0，零指数返回 1 mod m（含 0^0）；不要求素数或互质。读入失败、q 越界或模 0 返回 1，不作完整词法或 EOF 校验。须 64 位 unsigned long long 及 uint128 支持；不恢复已排除的 P1226。

所需模板：Mod64（第 124 页，18.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/src/compact/mod64.hpp

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    if (!(cin >> q) || q < 0 || q > 200000) return 1;
    while (q--)
    {
        unsigned long long a, b, m;
        if (!(cin >> a >> b >> m) || !m) return 1;
        cout << Mod64::mul(a, b, m) << ' ' << Mod64::power(a, b, m) << '\n';
    }
}
```

18.2 模幂与几何级数同时求值

power_sum(a,n,mod) 返回 {a^n mod mod, ∑(i=0..n-1) a^i mod mod}。a 为完整 unsigned long long，n 为 uint128 项数，mod 为正 unsigned long long。n=0 返回 {1%mod,0}；a=0 且 n>0 时和为 1%mod。无需逆元，合数模数也可用。时间与递归空间 O(log(n+1))，至多 128 次折半。只抄此函数需 utility、cassert。

```
pair<unsigned long long, unsigned long long>
power_sum(unsigned long long a, __uint128_t n, unsigned long long mod)
{
    assert(mod >= 1);
    if (!n) return {1 % mod, 0};
    auto [p, s] = power_sum(a, n / 2, mod);
    unsigned long long square = __uint128_t(p) * p % mod;
    unsigned long long sum = __uint128_t(s) * (p + 1) % mod;
    if (n % 2)
        return {__uint128_t(square) * a % mod, (__uint128_t(sum) + square) % mod};
    return {square, sum};
}
```

应用题补充：AtCoder ABC293 E Geometric Progression

题意：给定 1≤A,M≤10^9、1≤X≤10^12，求 ∑(i=0..X-1) A^i mod M。直接读取 power_sum(A,X,M).second；M 可能是合数，A-1 不一定可逆，不能无条件使用几何级数除法。M=1 输出 0，A=1 时结果 X mod M。第一分量为 A^X，本题不输出。此为比赛应用题，不计独立模板题覆盖。

所需模板：power_sum（第 125 页，18.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://atcoder.jp/contests/abc293/tasks/abc293_e

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    unsigned long long a, x, mod;
    cin >> a >> x >> mod;
    cout << power_sum(a, x, mod).second << '\n';
    return 0;
}
```

第十九章　数论

19.1　数论基础

相关应用：约数和公式应用；需核对正文定位，导航无独立幂约数和页。

19.1.1　幂的约数和

代码依赖：PollardRho（第 154 页，19.6.1 节）、power_sum（第 125 页，18.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

传入 a、b、mod 和可复用的 PollardRho 分解器 solver，返回 a^b 的正约数和取模。a、b、mod 均为 unsigned long long，a≥1、mod≥1。a=1 或 b=0 时结果为 1%mod。依赖 power_sum 与分解器；重数 e 对应 e*b+1 项，长度用 uint128。时间由分解及每个不同质因子的 O(log(e*b+2)) 求和组成。

```
unsigned long long divisor_sum_power(unsigned long long a,
                                     unsigned long long b,
                                     unsigned long long mod,
                                     PollardRho &solver)
{
    assert(a >= 1 && mod >= 1);
    if (!b) return 1 % mod;
    auto factors = solver.factor(a);
    unsigned long long answer = 1 % mod;
    for (int i = 0, j; i < (int)factors.size(); i = j)
    {
        j = i + 1;
        while (j < (int)factors.size() && factors[j] == factors[i]) j++;
        __uint128_t length = __uint128_t(j - i) * b + 1;
        auto sum = power_sum(factors[i], length, mod).second;
        answer = __uint128_t(answer) * sum % mod;
    }
    return answer;
}
```

模板题使用：Luogu P1593 因子和

题意：给定 1≤a≤50000000、0≤b≤50000000，求 a^b 的所有正约数之和模 9901。只分解 a，质因子重数 e 对应 e*b+1 项几何级数，无需构造 a^b 或求逆元。a=1 或 b=0 输出 1。PollardRho 可复用；本题模数固定，但核心允许任意正 uint64 模数，长度以 uint128 计算。

所需模板：Mod64（第 124 页，18.1 节）、Prime64（第 149 页，19.3.1 节）、PollardRho（第 154 页，19.6.1 节）、power_sum（第 125 页，18.2 节）、divisor_sum_power（第 127 页，19.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1593

```
int main()
{
    unsigned long long a, b;
    scanf("%llu%llu", &a, &b);
    PollardRho solver;
    printf("%llu\n", divisor_sum_power(a, b, 9901, solver));
    return 0;
}
```

19.2 模算术简介

19.2.1 动态模整数与合数模逆元

代码依赖：Barrett32（第 144 页，19.2.5 节）。抄写时一并准备；标准库类型不列入算法依赖。

mint<tag> 按 tag 独立保存运行时模数，默认 998244353。先 set_mod(p)，1≤p≤INT_MAX，再构造该模数下的所有对象；更换同 tag 的模数后，旧对象、容器元素和缓存全部失效，必须重建，不能跨模混算。构造支持 long long 全范围，v 为规范余数；加减乘及复合赋值、相等比较为 O(1)，加减只作一次区间校正，乘法使用 Barrett32，不依赖素数。pow(k) 的 k 为 uint64_t 非负指数，0 次幂为 1 mod p。try_inv() 以扩展欧几里得求逆，gcd(v,p)≠1 返回 nullopt；inv() 与除法要求可逆，否则违反断言前提。p=1 时唯一余数为 0，乘法单位元也为 0，try_inv 返回 0。幂和求逆为 O(log p) 或 O(log k)，除法包含求逆。可与 Fenwick 区间和组合，不能用于模数和的 kth。mod、reducer 只读，必须通过 set_mod 同步切换；Barrett32 本身支持完整 uint32，但此包装仍受 int 上界保护；不支持超过 INT_MAX 的模数，也不提供线程间安全切换模数。

```
template <int tag = 0> struct mint
{
    static inline int mod = 998244353;
    static inline Barrett32 reducer{998244353};
    int v;

    static void set_mod(int p)
    {
        assert(p >= 1);
        mod = p;
        reducer = Barrett32(p);
    }

    mint(long long x = 0) : v(x % mod)
    {
        if (v < 0)
            v += mod;
    }

    mint operator+(mint b) const
    {
        mint r;
        long long sum = (long long)v + b.v;
        r.v = sum >= mod ? sum - mod : sum;
        return r;
    }
```

模整数（接上页同一结构体）：

```
    mint operator-(mint b) const
    {
        mint r;
        r.v = v - b.v;
        if (r.v < 0)
            r.v += mod;
        return r;
    }

    mint operator*(mint b) const
    {
        mint r;
        r.v = reducer.mul(v, b.v);
        return r;
    }

    mint &operator+=(mint b)
    {
        return *this = *this + b;
    }

    mint &operator-=(mint b)
    {
        return *this = *this - b;
    }

    mint &operator*=(mint b)
    {
        return *this = *this * b;
    }

    mint pow(uint64_t k) const
    {
        mint a = *this, r = 1;
        while (k)
        {
            if (k & 1)
                r *= a;
            a *= a;
            k >>= 1;
        }
        return r;
    }
```

模整数（接上页同一结构体）：

```
    optional<mint> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            tie(a, b) = pair{b, a - q * b};
            tie(x, y) = pair{y, x - q * y};
        }
        if (a != 1)
            return nullopt;
        return mint(x);
    }

    mint inv() const
    {
        auto x = try_inv();
        assert(x.has_value());
        return *x;
    }

    mint operator/(mint b) const
    {
        return *this * b.inv();
    }

    mint &operator/=(mint b)
    {
        return *this = *this / b;
    }

    bool operator==(const mint &) const = default;
};
```

模板题使用：Luogu P5431 【模板】模意义下的乘法逆元 2

题意：输入 1≤n≤5000000、2≤k<p≤10^9、1≤a_i<p 且 p 为素数，求 ∑k^i/a_i 模 p。先设置 mint 的模数 p，再构造数组；传 move(a) 调用 batch_units，只求一次总积逆元。权重从 1 开始，每项先乘 k 再累加。read 与 main 一起抄；move 后不再读取 a。更换同 tag 模数后旧对象失效，计算期间不能切换。核心支持合数单位，本题只覆盖素数；动态模数、构造、乘法及累加、v 输出在此直接使用，合数求逆接口另由核心测试核验。

所需模板：Barrett32（第 144 页，19.2.5 节）、mint（第 129 页，19.2.1 节）、batch_units（第 163 页，19.9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5431

```
int read()
{
    int c = getchar_unlocked();
    while (c < '0' || c > '9')
    {
        if (c == EOF) return 0;
        c = getchar_unlocked();
    }
    int value = 0;
    while ('0' <= c && c <= '9')
    {
        value = value * 10 + c - '0';
        c = getchar_unlocked();
    }
    return value;
}

int main()
{
    int n = read(), p = read(), k = read();
    mint<>::set_mod(p);
    vector<mint<>> a(n);
    for (auto &x : a) x = read();
    auto inverse = batch_units(move(a));
    if (!inverse) return 1;
    mint<> answer = 0, weight = 1;
    for (auto x : *inverse)
    {
        weight *= mint<>(k);
        answer += weight * x;
    }
    printf("%d\n", answer.v);
    return 0;
}
```

19.2.2 静态模整数

固定 int 模数 1≤mod≤INT_MAX，可为合数。构造接受 signed64 并正规化，v 保持 0..mod-1；不要手动破坏正规化。加减乘 O(1)，pow 非负 signed64 指数、时间 O(log(e+1))。try_inv 用欧几里得求逆：gcd(v,mod)=1 时返回 optional 值，否则 nullopt；mod=1 返回代表 0。inv、/、/= 要求分母可逆；断言开启时诊断违反前提，关闭断言不承诺拒绝非法除法。逆元 O(log(mod+1))、额外空间 O(1)，只用 long long，不新增辅助模板依赖。+=、-=、*=、/= 返回自身引用，支持自赋值与链式运算，可作为 Fenwick 区间和值类型。类型的扩展不放宽 Binomial、NTT、高斯或 FPS 自身的素数等前提。

```
template <int mod> struct ModInt
{
    static_assert(mod >= 1);
    int v;

    ModInt(long long x = 0)
    {
        v = x % mod;
        if (v < 0)
            v += mod;
    }

    ModInt operator+(ModInt b) const
    {
        long long x = (long long)v + b.v;
        if (x >= mod)
            x -= mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator-(ModInt b) const
    {
        int x = v - b.v;
        if (x < 0)
            x += mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator*(ModInt b) const
    {
        ModInt r;
        r.v = 1LL * v * b.v % mod;
        return r;
    }
```

模整数（接上页同一结构体）：

```
    ModInt pow(long long e) const
    {
        assert(e >= 0);
        ModInt a = *this, r = 1;
        while (e)
        {
            if (e & 1)
                r = r * a;
            a = a * a;
            e >>= 1;
        }
        return r;
    }

    optional<ModInt> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            a -= q * b;
            swap(a, b);
            x -= q * y;
            swap(x, y);
        }
        if (a != 1)
            return nullopt;
        return ModInt(x);
    }

    ModInt inv() const
    {
        auto r = try_inv();
        assert(r);
        return *r;
    } // requires an invertible residue
```

模整数（接上页同一结构体）：

```
    ModInt operator/(ModInt b) const
    {
        return *this * b.inv();
    }

    ModInt &operator+=(ModInt b)
    {
        return *this = *this + b;
    }

    ModInt &operator-=(ModInt b)
    {
        return *this = *this - b;
    }

    ModInt &operator*=(ModInt b)
    {
        return *this = *this * b;
    }

    ModInt &operator/=(ModInt b)
    {
        return *this = *this / b;
    }
};
```

联合使用示例见第 277 页（Luogu P1313 计算系数），其中直接调用本模板。

相关应用：完整 uint64 固定模整数与合数单位逆元，精确整数模乘替代依赖 long double 精度的实现

19.2.3 完整 64 位固定模整数

代码依赖：Mod64（第 124 页，18.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

StaticMod64<P> 支持 1≤P≤UINT64_MAX 的固定模数，可为合数。构造接受不超过 64 位的标准有符号/无符号整数，默认零；公开 v 为规范 uint64 余数，只读，不要转为 long long 输出大余数。加减用先比较后运算避免溢出；乘法依赖 Mod64 的 uint128 精确取余，不用浮点。加减乘 O(1)，支持一元负号、相等比较与复合赋值。

pow(k) 接受完整 uint64 非负指数，0^0=1 mod P，O(log(k+1))。try_inv() 用 int128 扩展欧几里得，gcd(v,P)≠1 时返回 nullopt；inv 及除法要求分母可逆，O(log P)。模 1 的唯一余数为 0，逆元也返回 0。构造用 int128 先求余再校正，支持 LLONG_MIN 与 UINT64_MAX；求逆系数用 int128，最终规范到 [0,P)。依赖 GNU/Clang 的 128 位整数；不支持把更宽整数直接传给构造函数。不提供字符串流重载：按题目读 signed64 或 unsigned64 后构造，输出 v。

```
template <uint64_t P> struct StaticMod64
{
    static_assert(P > 0);
    uint64_t v = 0;

    StaticMod64() = default;

    template <integral T> StaticMod64(T x)
    {
        static_assert(sizeof(T) <= 8);
        __int128_t r = __int128_t(x) % __int128_t(P);
        if (r < 0)
            r += P;
        v = r;
    }

    StaticMod64 operator+(StaticMod64 b) const
    {
        StaticMod64 r;
        r.v = v >= P - b.v ? v - (P - b.v) : v + b.v;
        return r;
    }

    StaticMod64 operator-(StaticMod64 b) const
    {
        StaticMod64 r;
        r.v = v >= b.v ? v - b.v : P - (b.v - v);
        return r;
    }
```

64 位模整数（接上页同一结构体）：

```
    StaticMod64 operator-() const
    {
        StaticMod64 r;
        r.v = v == 0 ? 0 : P - v;
        return r;
    }

    StaticMod64 operator*(StaticMod64 b) const
    {
        StaticMod64 r;
        r.v = Mod64::mul(v, b.v, P);
        return r;
    }

    StaticMod64 &operator+=(StaticMod64 b)
    {
        return *this = *this + b;
    }

    StaticMod64 &operator-=(StaticMod64 b)
    {
        return *this = *this - b;
    }

    StaticMod64 &operator*=(StaticMod64 b)
    {
        return *this = *this * b;
    }

    StaticMod64 pow(uint64_t k) const
    {
        StaticMod64 r;
        r.v = Mod64::power(v, k, P);
        return r;
    }
```

64 位模整数（接上页同一结构体）：

```
    optional<StaticMod64> try_inv() const
    {
        __int128_t a = v, b = P, x = 1, y = 0;
        while (b)
        {
            __int128_t q = a / b;
            tie(a, b) = pair{b, a - q * b};
            tie(x, y) = pair{y, x - q * y};
        }
        if (a != 1)
            return nullopt;
        if (x < 0)
            x += P;
        return StaticMod64(uint64_t(x));
    }

    StaticMod64 inv() const
    {
        auto r = try_inv();
        assert(r.has_value());
        return *r;
    }

    StaticMod64 operator/(StaticMod64 b) const
    {
        return *this * b.inv();
    }

    StaticMod64 &operator/=(StaticMod64 b)
    {
        return *this = *this / b;
    }

    bool operator==(const StaticMod64 &) const = default;
};
```

接口用法演示：完整 64 位模整数与合数逆元 API

演示约定：固定合数模 UINT64_MAX。每组输入 signed64 整数 a、uint64 整数 b 和非负指数 k，输出 a、b 规范余数及 a+b、a-b、a*b、a^k、b 逆元；b 不可逆输出-1。大余数直接输出 uint64 成员 v，不能先转 signed64。

所需模板：Mod64（第 124 页，18.1 节）、StaticMod64（第 136 页，19.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/STATIC-MOD64.md

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Z = StaticMod64<UINT64_MAX>;
    int q;
    cin >> q;
    while (q--)
    {
        long long a;
        uint64_t b, k;
        cin >> a >> b >> k;
        Z x(a), y(b);
        auto inv = y.try_inv();
        cout << x.v << ' ' << y.v << ' ';
        cout << (x + y).v << ' ' << (x - y).v << ' ';
        cout << (x * y).v << ' ' << x.pow(k).v << ' ';
        if (inv)
            cout << inv->v << '\n';
        else
            cout << -1 << '\n';
    }
}
```

相关应用：奇数固定模数的 Montgomery 表示及整数约减；辅助原根/平方根另列

19.2.4 Montgomery 固定模整数

Montgomery<P> 为可选固定模整数，要求奇数 1<P<2^30，允许合数。构造接受完整 long long 并正规化；get() 返回普通余数。v 是 [0,P) 内的 Montgomery 表示，只读，不能作为普通余数使用或直接赋值。加减乘与一元负号 O(1)，乘法不用除法；比较直接比较规范内部表示。pow(k) 保留完整 uint64 非负指数，约定 0^0=1，不把指数对 P-1 取模。try_inv() 用扩展欧几里得，非单位返回 nullopt；inv 与除法要求除数可逆，复杂度 O(log P)。

内部取 R=2^32, neg_inv=-P^{-1} mod R, r2=R^2 mod P; reduce(x) 要求 x<P*R, 以 q=(x mod R)*neg_inv mod R 使 x+qP 能被 R 整除，再取高 32 位并最多减 P。x+qP<2PR<2^63；核心只用 uint64。乘积和构造均满足输入界。inverse32/reduce 为内部辅助；原根、模平方根另用已有模板。本类型不替换普通 ModInt 或动态 mint，不保证所有平台都比编译期常模取余快。

```
// Odd modulus, including composites. v is a Montgomery residue; use get().
template <uint32_t P> struct Montgomery
{
    static_assert(P > 1 && P < (1U << 30) && (P & 1));

    static constexpr uint32_t inverse32()
    {
        uint32_t x = P;
        for (int i = 0; i < 4; i++)
            x *= 2U - P * x;
        return -x;
    }

    static constexpr uint32_t neg_inv = inverse32();
    static constexpr uint32_t r2 = -uint64_t(P) % P;
    uint32_t v = 0;

    // Internal: x < P * 2^32. The returned residue lies in [0, P).
    static uint32_t reduce(uint64_t x)
    {
        uint32_t q = uint32_t(x) * neg_inv;
        uint32_t y = (x + uint64_t(q) * P) >> 32;
        return y >= P ? y - P : y;
    }

    Montgomery(long long x = 0)
    {
        long long y = x % P;
        if (y < 0)
            y += P;
        v = reduce(uint64_t(y) * r2);
    }

    uint32_t get() const
    {
        return reduce(v);
    }
```

模整数（接上页同一结构体）：

```
    Montgomery operator+(Montgomery b) const
    {
        Montgomery r;
        r.v = v + b.v;
        if (r.v >= P)
            r.v -= P;
        return r;
    }

    Montgomery operator-(Montgomery b) const
    {
        Montgomery r;
        r.v = v >= b.v ? v - b.v : v + P - b.v;
        return r;
    }

    Montgomery operator-() const
    {
        Montgomery r;
        r.v = v == 0 ? 0 : P - v;
        return r;
    }

    Montgomery operator*(Montgomery b) const
    {
        Montgomery r;
        r.v = reduce(uint64_t(v) * b.v);
        return r;
    }

    Montgomery &operator+=(Montgomery b)
    {
        return *this = *this + b;
    }

    Montgomery &operator-=(Montgomery b)
    {
        return *this = *this - b;
    }

    Montgomery &operator*=(Montgomery b)
    {
        return *this = *this * b;
    }
```

模整数（接上页同一结构体）：

```
    Montgomery pow(uint64_t k) const
    {
        Montgomery a = *this, r = 1;
        while (k)
        {
            if (k & 1)
                r *= a;
            a *= a;
            k >>= 1;
        }
        return r;
    }

    optional<Montgomery> try_inv() const
    {
        long long a = get(), b = P, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            tie(a, b) = pair{b, a - q * b};
            tie(x, y) = pair{y, x - q * y};
        }
        if (a != 1)
            return nullopt;
        return Montgomery(x);
    }

    Montgomery inv() const
    {
        auto r = try_inv();
        assert(r.has_value());
        return *r;
    }

    Montgomery operator/(Montgomery b) const
    {
        return *this * b.inv();
    }

    Montgomery &operator/=(Montgomery b)
    {
        return *this = *this / b;
    }

    bool operator==(const Montgomery &) const = default;
};
```

接口用法演示：Montgomery 固定模整数 API

演示约定：固定模 998244353。每组输入 signed64 整数 a、b 与 uint64 非负指数 k，依次输出 a+b、a-b、a*b、a^k、b 逆元，均正规化；b 不可逆输出-1。内部 v 不能直接输出，须 get()。

所需模板：Montgomery（第 140 页，19.2.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/MONTGOMERY.md

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Z = Montgomery<998244353>;
    int q;
    cin >> q;
    while (q--)
    {
        long long a, b;
        uint64_t k;
        cin >> a >> b >> k;
        Z x(a), y(b);
        auto inv = y.try_inv();
        cout << (x + y).get() << ' ' << (x - y).get() << ' ';
        cout << (x * y).get() << ' ' << x.pow(k).get() << ' ';
        cout << (inv ? (long long)inv->get() : -1) << '\n';
    }
}
```

相关应用：预计算倒数的 32 位 Barrett 模乘；不代表 64 位浮点快速乘或 Montgomery

19.2.5 32 位 Barrett 快速模乘

Barrett32(m) 预计算运行时模数的 64 位倒数，1≤m≤UINT32_MAX；mul(a,b) 要求 0≤a,b<m，返回 a*b mod m 的规范 uint32 余数。构造 O(1) 且含一次 64 位除法，此后的每次乘法只用整数乘法、取高位和一次校正，不使用浮点数或整数除法。对象可复制，多个对象的模数互不影响；mod 和 reciprocal 只读，需要换模时重新构造。

令 R=2^64，m>1 时 reciprocal=ceil(R/m)，估计商只可能等于真实商或多 1。用 64 位 z<q*m 判断是否多减一次，再补回 m；不要先截成 32 位后比较，这在大于 INT_MAX 的模数下会错误。m=1 时规范输入只能为 0，倒数按 unsigned64 回绕为 0，返回 0。依赖 GNU/Clang 的 uint128 乘法；不适用于 64 位模数，后者仍用 Mod64。

```
struct Barrett32
{
    uint32_t mod;
    uint64_t reciprocal;

    Barrett32(uint32_t m) : mod(m)
    {
        assert(m != 0);
        reciprocal = UINT64_MAX / m + 1;
    }

    // Requires a < mod and b < mod.
    uint32_t mul(uint32_t a, uint32_t b) const
    {
        uint64_t z = uint64_t(a) * b;
        uint64_t q = (__uint128_t(z) * reciprocal) >> 64;
        uint64_t y = q * mod;
        return uint32_t(z - y + (z < y ? mod : 0));
    }
};
```

接口用法演示：Barrett 规范余数模乘 API

演示约定：输入非零 uint32 模数 m 与询问数，每组 a b 满足 0≤a,b<m，输出 a*b 模 m。支持 m=1 和大于 INT_MAX 的模数；输入本身不正规化。

所需模板：Barrett32（第 144 页，19.2.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/BARRETT32.md

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    uint32_t mod;
    int tests;
    cin >> mod >> tests;
    Barrett32 reducer(mod);
    while (tests--)
    {
        uint32_t a, b;
        cin >> a >> b;
        cout << reducer.mul(a, b) << '\n';
    }
}
```

19.2.6 模运算的适用条件

模逆元存在当且仅当 $\gcd(a,m)=1$。费马逆元 a^{p-2} 仅适用于素数模数及 $a\neq 0$。对正模数 m,n，同余式 $x\equiv a \pmod m$、$x\equiv b \pmod n$ 可合并当且仅当 $\gcd(m,n)\mid(b-a)$；新模数为 $\operatorname{lcm}(m,n)$。

$$\binom{n}{k}\equiv\prod_i\binom{n_i}{k_i}\pmod p \quad (p \text{ 为素数，} n_i,k_i \text{ 是}p \text{ 进制位}).$$

Lucas 不能直接套在合数模数。素数模数下，阶乘与逆阶乘预处理要求上限小于模数；达到模数会出现阶乘为零。扩展欧拉降幂中，非互质情况不能直接将指数对 $\varphi(m)$ 取模；需要判断原指数是否达到足够阈值。大整数指数可以在读入时同时维护余数与是否超过阈值。

19.2.6.1 除法与解同余式不是一回事

以下先设 $m\geq 2$。把除以 a 写成乘逆元，要求 a 是单位元，即 $\gcd(a,m)=1$。若 $g=\gcd(a,m)>1$，并非所有含 a 的同余式都无解：

$$ax+b\equiv 0 \pmod m \iff \frac{a}{g}x+\frac{b}{g}\equiv 0 \pmod{m/g} \qquad (g\mid b).$$

$g\nmid b$ 时无解；否则约掉 g 后系数与模数互质，在模 m/g 下恰有一个解 x_0。原模 m 的全部不同解为 $x_0+t(m/g)$，$0\leq t<g$。约去公因子时模数也要除以 g，不能从 $2x\equiv 2 \pmod 6$ 错误推出 $x\equiv 1 \pmod 6$；实际是 $x\equiv 1 \pmod 3$，在 $[0,6)$ 内有 1、4 两个解。而 $2x\equiv 1 \pmod 6$ 无解，二者都不能用“除以 2”统一处理。

本库 `linear_congruence(a,b,m)` 的符号约定就是 $ax+b\equiv 0$，返回最小非负解和最小正周期；无解返回 $(-1,-1)$。例如 `linear_congruence(2,-2,6)` 返回 $(1,3)$。若题面是 $ax\equiv c$，转换常数时不能在 signed64 中直接对最小负数取负；应先把右端规范化到 $[0,m)$ 再变号传入；只在 128 位中取负后直接窄化回 signed64 仍不安全。实现见第 19.10.1 节（第 168 页）。

单逆元 `mod_inverse` 不可逆返回 -1，`ModInt::try_inv` 不可逆返回 `nullopt`，不是同一种失败表示。`inv` 和除法仍以可逆为前提；关闭断言不意味着非法除法会被安全拒绝。分别见第 19.9.2 节（第 164 页）和第 19.2.2 节（第 133 页）。$m=1$ 在这些整数模接口中另按唯一剩余类处理：代表为 0，逆元也返回 0，同余式周期为 1；不能因此把只适用于域的高斯、NTT、FPS 或组合数条件推广到模 1。

19.2.6.2 知道整数可整除时，先升模数再整除

若已知整数 X 被正整数 d 整除，目标是 $(X/d) \bmod m$（$m \geq 1$），即使 $\gcd(d, m) > 1$，仍可先计算规范剩余 $r = X \bmod (md)$，使 $0 \leq r < md$。写成 $X = qmd + r$，可知 $d \mid r$，且

$$\frac{X}{d} = qm + \frac{r}{d} \equiv \frac{r}{d} \pmod{m}.$$

这是整数整除，不是模逆元。负 X 也成立，但须先规范化剩余，不能直接依赖 C++ 负余数。例如 $X = 6, d = 2, m = 4$，正确答案是 3；先取 $X \bmod m = 2$ 再除以 2 会错误得到 1。只知道 $X \bmod m$ 不足以恢复商：$X = 2$ 与 6 同余模 4，其除以 2 后的余数却不同。

必须保证能计算 md 并能在这个新模数下求出 X；先扩宽乘法、检查范围，乘模的中间乘积同样要安全，必要时用多精度。提升后的模数可能是合数，不能简单给素数模 Binomial、NTT 或费马逆元更换参数。若所有生成 X 的运算本来只有加法与乘法，可以在新模数下直接进行；含除法时要重新验证。轨道计数中的群阶整除应用见第 21.7.7 节（第 308 页），不重复抄一套专门模板。

19.2.6.3 组合数的整数性不等于分母可逆

p 为素数且预处理 $N < p$ 时，$N!$ 才能保证非零可逆，适用现有 `Binomial` 契约。仅有 $N < m$ 在合数模下不够，例如 $2! \bmod 6 = 2$ 没有逆元。二项式系数作为整数始终存在，不代表阶乘比值可以逐项在任意模数下相除。大下标、素数模考虑 Lucas；合数模考虑素数幂分解、去除质因子后的单位部分和 CRT，使用现有 exLucas 的条件与资源限制，不把素数模版直接套过去。接口和边界见第 21.1.1 节（第 276 页）。同理，插值中的节点差、级数积分的分母以及模概率的分母，都须各自检查可逆性。

19.2.6.4 大指数要同时保留余数和阈值信息

对于整数 $a \geq 0$、非负整数指数 b 与正模数 m，令 $\phi = \varphi(m)$。一个足够但不一定最小的安全降幂规则是

$$a^b \equiv a^e \pmod{m}, \qquad e = \begin{cases} b, & b < \phi, \\ b \bmod \phi + \phi, & b \geq \phi. \end{cases}$$

约定零次幂为 1，结果再对 m 取模；$m = 1$ 返回 0。若 $\gcd(a, m) = 1$，直接对 ϕ 取模也可；不互质时则需阈值。例如 $2^4 \bmod 8 = 0$，只将 4 对 $\varphi(8) = 4$ 取模会得到 $2^0 \bmod 8 = 1$。反过来，对小指数无条件加 ϕ 也错：$2^1 \bmod 8 = 2$，而 $2^5 \bmod 8 = 0$。

为什么这个阈值足够。 分解 $m = \prod p^k$。若 $p \nmid a$，欧拉定理给出周期 $\varphi(p^k)$，它整除 ϕ。若 $p \mid a$，只要指数至少 k，幂就为 0 模 p^k；而 $\phi \geq \varphi(p^k) = p^{k-1}(p-1) \geq k$。因此 $b \geq \phi$ 时原指数和降幂后的指数在每个素数幂分量都一致，再由 CRT 合并。$b < \phi$ 时保留原指数，不需要讨论何时进入周期。

十进制字符串可逐位维护 $b \bmod \phi$ 和饱和值 $\min(b, \phi)$，只需知道是否达到阈值，不必保存完整大整数；前导零不改变它们。现有 `euler_power(a,b,m,phi)` 要求调用者传入*真实的* $\varphi(m)$，其断言不能证明传入值就是欧拉函数。模数分解/求 ϕ 的成本需另算。降幂后的指数可能需要 65 位，不能用 unsigned64 暂存相加结果；本实现内部用 128 位。接口见第 19.8.1 节（第 161 页）。本库基数/模数为 unsigned64，不能把负数直接窄化后当成数学上的规范取模。费马、欧拉与线性同余的基础背景可参见 MIT 18.781 第 4 讲；这里的非互质阈值和升模整除附有独立推导，不借用素数模 API 扩大适用范围。

19.3 素数

19.3.1 确定性 64 位 Miller–Rabin 判素

代码依赖：Mod64（第 124 页，18.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

Prime64::prime(n) 接收完整 unsigned long long，n<2 返回 false。先试除小素数，再使用 2、325、9375、28178、450775、9780504、1795265022 七个底数，结果在 64 位输入范围内确定。继承 Mod64，模乘用 uint128；O(log n) 次模乘，O(1) 空间，不包含因数分解。

```
struct Prime64 : Mod64
{
    static bool prime(ull n)
    {
        if (n < 2) return false;
        for (ull p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
            if (n % p == 0) return n == p;
        ull d = n - 1;
        int s = 0;
        while (!(d & 1))
        {
            d >>= 1;
            ++s;
        }
        for (ull a :
             {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL})
        {
            if (a % n == 0) continue;
            ull x = power(a, d, n);
            if (x == 1 || x == n - 1) continue;
            bool ok = false;
            for (int j = 1; j < s; j++)
            {
                x = mul(x, x, n);
                if (x == n - 1)
                {
                    ok = true;
                    break;
                }
            }
            if (!ok) return false;
        }
        return true;
    }
};
```

模板题使用：Library Checker primality_test

题意：逐个判断给定正整数是否为素数，输出 Yes 或 No。此题输入不超过 10^{18}；模板支持的完整范围另见接口约定。

所需模板：Mod64（第 124 页，18.1 节）、Prime64（第 149 页，19.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/primality_test

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    while (q--)
    {
        unsigned long long n;
        cin >> n;
        cout << (Prime64::prime(n) ? "Yes" : "No") << '\n';
    }
}
```

19.4 欧拉函数

19.4.1 单个数的欧拉函数

代码依赖：PollardRho（第 154 页，19.6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

euler_phi(n) 使用试除，$O(\sqrt{n})$ 时间、O(1) 额外空间；euler_phi(n,solver) 复用 PollardRho，时间由随机分解主导，分解结果与递归占 O(log n) 空间。$1 \le n \le$ ULLONG_MAX，phi(1)=1。大素数或平衡大半素数应选分解器入口，不要用试除穷扫平方根；批量求 1..N 用 LinearSieve。先除质因子再乘 p-1，结果及中间乘积均不超过原 n。只抄试除函数时不需要 Prime64/PollardRho。

```
unsigned long long euler_phi(unsigned long long n)
{
    assert(n >= 1);
    unsigned long long answer = n;
    for (unsigned long long p = 2; p <= n / p; p++)
        if (n % p == 0)
        {
            answer = answer / p * (p - 1);
            while (n % p == 0) n /= p;
        }
    if (n > 1) answer = answer / n * (n - 1);
    return answer;
}

unsigned long long euler_phi(unsigned long long n, PollardRho &solver)
{
    assert(n >= 1);
    auto factors = solver.factor(n);
    unsigned long long answer = n;
```

```
    unsigned long long previous = 0;
    for (auto p : factors)
        if (p != previous)
        {
            answer = answer / p * (p - 1);
            previous = p;
        }
    return answer;
}
```

联合使用示例见第 161 页（Luogu P5091），其中直接调用本模板。

19.5 筛法

19.5.1 线性筛、欧拉函数与 Möbius 函数

构造参数 0≤n<INT_MAX，且数组须能分配内存。prime 为递增质数表；lp/phi/mu 为 0..n 数组。x≥2 时 lp[x] 是最小质因子，lp[x]==x 等价于 x 为素数；反复除以 lp[x] 可分解质因数。lp[0]=lp[1]=0，不能直接用 lp[x]==x 判断 0。n≥1 时 phi[1]=mu[1]=1；第 0 项均为占位 0。O(n) 时间和空间，分解单个表内正整数耗时 O(log x)。n=0 不可访问下标 1。

```
struct LinearSieve
{
    vector<int> prime, lp, phi, mu;

    LinearSieve(int n) : lp(n + 1), phi(n + 1), mu(n + 1)
    {
        if (n) phi[1] = mu[1] = 1;
        for (int i = 2; i <= n; i++)
        {
            if (!lp[i])
            {
                lp[i] = i;
                prime.push_back(i);
                phi[i] = i - 1;
                mu[i] = -1;
            }
            for (int p : prime)
            {
                if (p > n / i) break;
                int j = i * p;
                lp[j] = p;
                if (i % p == 0)
                {
                    phi[j] = phi[i] * p;
                    mu[j] = 0;
                    break;
                }
                phi[j] = phi[i] * (p - 1);
                mu[j] = -mu[i];
            }
        }
    }
};
```

应用题补充：Luogu P2158 [SDOI2008] 仪仗队

题意：从方阵一角看可见格点，观察者所在原点不计；1≤N≤40000。坐标范围 0..N−1，可见当且仅当两坐标互素。N=1 输出 0；其余用 1+2 倍 phi[1..N−1] 之和，注意 phi(1)=1。先筛到 N−1，再累加 phi；时间、空间均 O(N)，答案用 long long。

所需模板：LinearSieve（第 151 页，19.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2158

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    if (n == 1)
    {
        cout << 0 << '\n';
        return 0;
    }
    LinearSieve sieve(n - 1);
    long long ans = 1;
    for (int i = 1; i < n; i++) ans += 2LL * sieve.phi[i];
    cout << ans << '\n';
}
```

19.5.2 区间筛素数

segmented_primes(l,r) 返回闭区间内递增的全部素数，0≤l≤r≤INT_MAX；区间长度须能分配内存。独立函数，每次重建底筛；不依赖预存素数表。令 W=r-l+1，时间 O((√r+W)log log(r+3))，空间 O(√r+W)。倍数起点、递增变量均用 long long，避免 INT_MAX 端点溢出；0 和 1 不进入结果。

```
vector<int> segmented_primes(int l, int r)
{
    assert(0 <= l && l <= r);
    int limit = 0;
    while (1LL * (limit + 1) * (limit + 1) <= r) limit++;
    vector<char> composite(limit + 1);
    vector<char> removed(1LL * r - l + 1);
    for (int p = 2; p <= limit; p++)
        if (!composite[p])
        {
            for (long long j = 1LL * p * p; j <= limit; j += p) composite[j] = true;
            long long start = max(1LL * p * p, (l + 1LL * p - 1) / p * p);
            for (long long j = start; j <= r; j += p) removed[j - l] = true;
        }
    vector<int> primes;
    for (long long x = max(l, 2); x <= r; x++)
        if (!removed[x - l]) primes.push_back(x);
    return primes;
}
```

应用题补充：POJ 2689 / UVA 10140 Prime Distance

题意：读闭区间 [L,U] 至 EOF，筛出递增素数，再比较相邻素数差；最近/最远均取最先出现的并列对，更新必须用严格 < 和 >。不足两个素数输出 There are no adjacent primes.，否则保留驱动的逗号、空格和句点格式。无 0 0 结束标记，每次调用重建筛。UVA 官方同题限制 1≤L<U≤2147483647、U-L≤1000000，所以闭区间容量是 U-L+1≤1000001；核心另支持 0 和单点，0/1 不为素数。倍数及末端递增用 long long，避免 INT_MAX 溢出。POJ 原站本次不可达；限制和样例仅据 UVA 官方 PDF 复核，不声称 POJ 限制或线上 AC 已复核。

所需模板：segmented_primes（第 152 页，19.5.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.org/external/101/10140.pdf

```
int main()
{
    int l, r;
    while (scanf("%d%d", &l, &r) == 2)
    {
        auto primes = segmented_primes(l, r);
        if (primes.size() < 2)
        {
            puts("There are no adjacent primes.");
            continue;
        }
        int closest = 1;
        int distant = 1;
        for (int i = 2; i < (int)primes.size(); i++)
        {
            int gap = primes[i] - primes[i - 1];
            if (gap < primes[closest] - primes[closest - 1]) closest = i;
            if (gap > primes[distant] - primes[distant - 1]) distant = i;
        }
        printf("%d,%d are closest, %d,%d are most distant.\n",
               primes[closest - 1],
               primes[closest],
               primes[distant - 1],
               primes[distant]);
    }
    return 0;
}
```

19.6 分解质因数

19.6.1 Pollard–Rho 因数分解

代码依赖：Mod64（第 124 页，18.1 节）、Prime64（第 149 页，19.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

factor(n) 返回升序质因子（含重数），1≤n≤ULLONG_MAX。n=1 返回空数组；n≥2 时 size()==1 等价于 n 为素数，否则 front()/back() 分别取最小/最大质因子。依赖 Mod64 与 Prime64。公开使用 factor；rho/split 为内部辅助，rho 只接受大于 1 的合数，不能直接传素数。Las Vegas 随机运行时间，固定默认种子方便复现，可由构造参数更换；无确定性最坏时间保证。每次 rho 尝试最多 200000 轮，失败后换随机参数重试，不将失败当作素数。递归仅沿真因子拆分，深度 O(log n)。

```
struct PollardRho
{
    using ull = unsigned long long;
    using u128 = __uint128_t;
    mt19937_64 rng;

    PollardRho(ull seed = 712367821) : rng(seed) {}

    ull rho(ull n)
    {
        if (n % 2 == 0) return 2;
        for (;;)
        {
            ull c = rng() % (n - 1) + 1, x = rng() % n, y = x, d = 1;
            auto f = [&](ull v)
            {
                return (Mod64::mul(v, v, n) + u128(c)) % n;
            };
            // Retry bounded attempts; Las Vegas: only exact divisors returned.
            for (int i = 0; i < 200000 && d == 1; i++)
            {
                x = f(x);
                y = f(f(y));
                d = gcd(x > y ? x - y : y - x, n);
            }
            if (1 < d && d < n) return d;
        }
    }

    void split(ull n, vector<ull> &a)
    {
        if (n == 1) return;
        if (Prime64::prime(n))
        {
            a.push_back(n);
            return;
        }
        ull d = rho(n);
        split(d, a);
        split(n / d, a);
    }
```

```
42
43      vector<ull> factor(ull n)
44      {
45          assert(n >= 1);
46          vector<ull> a;
47          split(n, a);
48          sort(a.begin(), a.end());
49          return a;
50      }
51  };
```

模板题使用：Luogu P4718

题意：逐个判断 n 是否为素数，是则输出 Prime，否则输出最大质因子。题目保证 n≥2，factor 返回升序含重数的质因子，因此可取 back；若用于 n=1，须先处理空结果。

所需模板：Mod64（第 124 页，18.1 节）、Prime64（第 149 页，19.3.1 节）、PollardRho（第 154 页，19.6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4718

```
1   int main()
2   {
3       int t;
4       scanf("%d", &t);
5       PollardRho solver;
6       while (t--)
7       {
8           unsigned long long n;
9           scanf("%llu", &n);
10          auto factors = solver.factor(n);
11          if (factors.size() == 1)
12              puts("Prime");
13          else
14              printf("%llu\n", factors.back());
15      }
16      return 0;
17  }
```

19.7 裴蜀定理 & 一次不定方程

19.7.1 扩展欧几里得与 Bézout 系数

extended_gcd(a,b,x,y) 约定 0≤a,b≤2^63，参数和输出系数为 int128，返回非负 gcd 并满足 ax+by=gcd(a,b)。a=b=0 返回 0、x=1、y=0；有符号线性方程用 linear_equation。递归实现，中间乘加为 int128；O(log(min(a,b)+1)) 时间与递归空间。

```
__int128_t extended_gcd(__int128_t a, __int128_t b, __int128_t &x, __int128_t &y)
{
    using i128 = __int128_t;
    if (!b)
    {
        x = 1;
        y = 0;
        return a;
    }
    i128 d = extended_gcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}
```

模板题使用：AOJ NTL_1_E Extended Euclid Algorithm

题意：给出正整数 1≤a,b≤10^9，求 ax+by=gcd(a,b)，先最小化 |x|+|y|，仅在并列最小时取 x≤y。直接调用 extended_gcd；该正整数范围内递归所得解已满足最小性，a=b 时返回 (0,1)。系数用 ___int128_t 接收，输出转 long long 安全；不应对所有答案强行要求 x≤y。时间、递归空间均 O(log(min(a,b)+1))。

所需模板：extended_gcd（第 156 页，19.7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=NTL_1_E

```
int main()
{
    long long a, b;
    scanf("%lld%lld", &a, &b);
    __int128_t x, y;
    extended_gcd(a, b, x, y);
    printf("%lld %lld\n", (long long)x, (long long)y);
    return 0;
}
```

19.7.2 二元一次不定方程

代码依赖：extended_gcd（第 156 页，19.7.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

linear_equation(a,b,c,x,y) 求 ax+by=c 的任意整数特解。a、b、c 支持完整 long long，x、y 必须为 int128。无解返回 false 且 x=y=0；a=b=c=0 返回 true、x=y=0，此时所有整数对都是解。其他有解情况，令 g=gcd(|a|,|b|)，通解为 (x+(b/g)t,y-(a/g)t)。依赖 extended_gcd；只需抄递归 exgcd 与本函数。时间、递归空间 O(log M)，其中 M=max(|a|,|b|)+2；不承诺特解最小或为正。

```
bool linear_equation(
    long long a, long long b, long long c, __int128_t &x, __int128_t &y)
{
    using i128 = __int128_t;
    x = 0;
    y = 0;
    if (!a && !b) return c == 0;
    i128 aa = a < 0 ? -i128(a) : i128(a);
    i128 bb = b < 0 ? -i128(b) : i128(b);
    i128 u, v;
    i128 g = extended_gcd(aa, bb, u, v);
    if (c % g) return false;
    x = u * (c / g);
    y = v * (c / g);
    if (a < 0) x = -x;
    if (b < 0) y = -y;
    return true;
}
```

模板题使用：Luogu P5656

题意：解 ax+by=c，题目 a、b、c 均为正数。无整数解输出 -1；有正整数解则输出数量及 x、y 的最小/最大值，否则输出 x、y 各自的最小正值，它们不一定构成同一组解。正整数不含 0。

所需模板：extended_gcd（第 156 页，19.7.1 节）、linear_equation（第 157 页，19.7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5656

```
int main()
{
    int t;
    scanf("%d", &t);
    while (t--)
    {
        long long a, b, c;
        scanf("%lld%lld%lld", &a, &b, &c);
        __int128_t x, y;
        if (!linear_equation(a, b, c, x, y))
        {
            puts("-1");
            continue;
        }
        long long g = gcd(a, b);
        long long dx = b / g;
        long long dy = a / g;
        long long xmin = (x % dx + dx - 1) % dx + 1;
        long long ymin = (y % dy + dy - 1) % dy + 1;
        long long ymax = (c - a * xmin) / b;
        if (ymax <= 0)
        {
            printf("%lld %lld\n", xmin, ymin);
            continue;
        }
        long long count = (ymax - 1) / dy + 1;
        long long xmax = xmin + (count - 1) * dx;
        printf("%lld %lld %lld %lld %lld\n", count, xmin, ymin, xmax, ymax);
    }
    return 0;
}
```

19.7.3 一次不定方程：特解与正整数解范围

实现见第 19.7.2 节（第 157 页）。对不全为零的 a,b，令 $g=\gcd(|a|,|b|)>0$。递归扩展欧几里得先求 $|a|u+|b|v=g$；当且仅当 $g\mid c$ 时有整数解。把系数乘以 c/g，再恢复 a,b 的符号，就得到 $ax_0+by_0=c$。两个特解相减，再利用 $a/g,b/g$ 互素，可知所有解恰为

$$x=x_0+\frac{b}{g}t,\qquad y=y_0-\frac{a}{g}t,\qquad t\in\mathbb{Z}.$$

若 $a=b=0$，则 $c=0$ 时所有整数对都是解，否则无解；此时不能使用除以 g 的通解式。

底层 `exgcd` 的本库调用约定为非负系数，上限 2^{63}；返回非负的 g，$(0,0)$ 约定返回 0。包装函数先转为 128 位再取相反数，因而可以处理 `LLONG_MIN`。缩放后的特解也用 128 位保存；不要提前转回 `long long`。即使特解可表示，直接验证 $ax+by$ 的两个乘积仍可能超过 128 位，本地验证使用任意精度整数。

对 P5656 的正系数 a,b,c，设 $d_x=b/g,d_y=a/g$，先将 x_0 调整到 $x_{\min}\in[1,d_x]$，并计算与它配对的 $y_{\max}=(c-ax_{\min})/b$。若 $y_{\max}>0$，正整数解个数为

$$N=\left\lfloor\frac{y_{\max}-1}{d_y}\right\rfloor+1,\quad x_{\max}=x_{\min}+(N-1)d_x,\quad y_{\min}=y_{\max}-(N-1)d_y.$$

否则不存在同时为正的解，但 x 和 y 仍各自有最小正值；分别按周期归一化即可，它们通常不是同一组解。正值归一化使用 `(v % d + d - 1) % d + 1`，不要把余数 0 当作正整数。驱动中 $a,b,c\le 10^9$，归一化后的乘积不超过 10^{18}，故该部分可用 64 位。

双版本以小范围有符号输入穷举、任意精度等式证书、64 位端点和斐波那契递归链验证。完整 P5656 驱动另验证 200000 组输入，包含正解数量与四个极值、无解和无正解分支；普通与 ASan/UBSan 测试均通过，在线 AC 待补。

来源：kuangbin 2.3；WIDA 数论打印稿的 exgcd 与不定方程例题。

https://oi-wiki.org/math/number-theory/bezouts/。

底层接口见第 19.7.1 节（第 156 页）。

19.7.3.1 有符号矩形范围：把两维约束化成一个参数区间

现在允许 a,b,c 为负或零，要求

$$L_x \le x \le R_x, \qquad L_y \le y \le R_y.$$

先处理空区间；任意一维下界大于上界，答案为 0。若 $a=b=0$，$c\neq 0$ 时无解；$c=0$ 时答案为 $(R_x-L_x+1)(R_y-L_y+1)$，不能套用一维参数族。否则先检查 $g \mid c$，再取得上面的特解与步长 $d_x=b/g$、$d_y=-a/g$。不要把负系数或负步长直接取绝对值而保留旧特解。

对任一坐标 $z=z_0+dt$，条件 $L\le z\le R$ 给出：

$$\begin{array}{r|l} d>0 & \lceil (L-z_0)/d \rceil \le t \le \lfloor (R-z_0)/d \rfloor \\ d<0,\ k=-d>0 & \lceil (z_0-R)/k \rceil \le t \le \lfloor (z_0-L)/k \rfloor \\ d=0 & L\le z_0\le R \text{ 时不限制}t\text{，否则无解} \end{array}$$

例如 $6x-9y=3$，$x\in[-5,4], y\in[-1,5]$ 时，解恰为 $(2,1)$ 和 $(-1,-1)$。分别计算 x,y 允许的区间，取交集 $[T_l,T_r]$；解数是 $\max(0,T_r-T_l+1)$。不全为零的系数保证至少一个步长非零，有限矩形因此会给出有限参数区间；实现时用“尚无上下界”标记，不必猜一个足够大的哨兵。通解对整数 t 是单射，且已包含全部整数解，所以交集计数没有遗漏或重复。若要实际列举答案，再依次代入这些整数 t，输出成本为 $O(\text{解数})$。

这里的 floor/ceil 是数学取整，不能用 C++ 的负数向零截断替代。对正分母 D，令 C++ 商余为 $q=n/D, r=n\%D$，则

$$\lfloor n/D \rfloor = q-[r<0], \qquad \lceil n/D \rceil = q+[r>0].$$

例如 $-5/2$ 的 floor 是 -3、ceil 是 -2。不要使用只对非负数成立的 `(n+D-1)/D`，也不要在未证明安全时先计算 `-n`。参数平移 $x_0'=x_0+d_xs, y_0'=y_0+d_ys$ 只会把允许的 t 整体平移 $-s$，答案不变；这是检查实现符号的一个独立不变量。

范围运算比求一个特解要求更强的整数宽度：即使输入端点都是 64 位，差值和区间长度也未必能用 64 位保存。当两系数全零、矩形覆盖全部有符号 64 位坐标时，答案为 2^{128}，连无符号 128 位也放不下。本节通式按精确整数理解；实现前证明每一步范围，不能证明时使用任意精度整数。尤其不要用窄类型计算长度后才转宽，也不能在等式证书中先让 ax, by 溢出再相消。求特解之后只需常数次整数运算；大整数版本的位复杂度取决于操作数长度，不能据此承诺固定字长常数耗时。

本节的有界小整数矩形用逐点枚举核对，并检查方程同乘负一、交换变量、特解平移、零系数、无解与 64 位端点构造。这验证本地公式和建模，不新增模板、正式题覆盖或线上 AC。

19.8 费马小定理 & 欧拉定理

19.8.1 十进制大指数：扩展欧拉降幂

代码依赖：Mod64（第 124 页，18.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

euler_power(a,b,m,phi) 求 a^b mod m，a、m 为 unsigned long long，m>0；b 为非空十进制数字串，可为 0、可含前导零，约定 0^0=1。调用者必须传入正确的 phi=φ(m)，包括 $\varphi(1)=1$；函数只检查数值范围，不验证 phi 正确性。指数小于 phi 时保持原值，否则使用 b mod phi + phi，非互质底数也适用。相加和十进制更新都用 int128，支持完整 unsigned64 模数。O(|b|+log m) 时间、O(1) 额外空间，不含求 phi 的成本；依赖 Mod64::mul。

```
unsigned long long euler_power(unsigned long long a,
                               string_view b,
                               unsigned long long m,
                               unsigned long long phi)
{
    using U = unsigned long long;
    using I = __uint128_t;
    assert(m && phi && phi <= m && !b.empty());
    U rem = 0, cap = 0;
    for (char c : b)
    {
        assert('0' <= c && c <= '9');
        int d = c - '0';
        rem = (I(rem) * 10 + d) % phi;
        cap = min(I(phi), I(cap) * 10 + d);
    }
    I e = I(rem) + (cap == phi ? I(phi) : 0);
    U ans = 1 % m;
    a %= m;
    for (; e; e >>= 1, a = Mod64::mul(a, a, m))
        if (e & 1) ans = Mod64::mul(ans, a, m);
    return ans;
}
```

模板题使用：Luogu P5091

题意：输入顺序为底数 a、模数 m、指数 b；b 最多 20000001 位，以字符串读入。先用试除法计算 φ(m)，再做扩展欧拉降幂；题目 m≤10^8，不能把输入顺序写成 a,b,m。

所需模板：Mod64（第 124 页，18.1 节）、Prime64（第 149 页，19.3.1 节）、PollardRho（第 154 页，19.6.1 节）、euler_phi（第 150 页，19.4.1 节）、euler_power（第 161 页，19.8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5091

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    unsigned long long a, m;
    string b;
    cin >> a >> m >> b;
    auto phi = euler_phi(m);
    cout << euler_power(a, b, m, phi) << '\n';
}
```

相关应用：公共指数与欧拉定理相关；导航无独立 Carmichael 页，不伪造叶节点。

19.8.2 Carmichael：模乘法群的公共指数

carmichael(n) 返回 $\lambda(n)$，即使所有与 n 互素的 a 都满足 a 的 $\lambda(n)$ 次幂模 n 为 1 的最小正指数；约定 $\lambda(1)=1$。要求 $1\le n\le$INT_MAX。试除分解后将各素数幂的指数取 lcm，特别处理 2 的三次及以上幂。时间 $O(\sqrt{n})$，额外空间 O(1)；所有中间值不超过有符号 64 位范围，结果不超过 n。该指数只适用于可逆底数，不能据此直接对非互素底数的幂降指数。函数不依赖原根模块。

```
int carmichael(int n)
{
    assert(n >= 1);
    long long answer = 1;
    for (int p = 2; p <= n / p; p++)
        if (n % p == 0)
        {
            int power = 1;
            while (n % p == 0)
            {
                n /= p;
                power *= p;
            }
            int value = power / p * (p - 1);
            if (p == 2 && power >= 8) value /= 2;
            answer = answer / gcd(answer, (long long)value) * value;
        }
    if (n > 1) answer = answer / gcd(answer, (long long)(n - 1)) * (n - 1);
    return answer;
}
```

应用题补充：UVa 10006 Carmichael Numbers

题意：多组 3≤n<65000，输入 0 结束，判定 n 是否为 Carmichael 数：必须为合数，并且对所有整数 a 满足 a^n≡a(mod n)。这里先排除素数，再判断 $\lambda(n)$ 是否整除 n-1；等价性包含非互素底数，不要直接拿 $\lambda(n)$ 给任意底数降幂。输出必须完全符合题面两种句式。该题只检查判定，不能替代 $\lambda(n)$ 数值及最小性的接口测试，亦不计独立模板题覆盖。

所需模板：Mod64（第 124 页，18.1 节）、Prime64（第 149 页，19.3.1 节）、carmichael（第 162 页，19.8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.org/external/100/10006.pdf

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    while (cin >> n && n)
    {
        bool special = !Prime64::prime(n) && (n - 1) % carmichael(n) == 0;
        if (special)
            cout << "The number " << n << " is a Carmichael number.\n";
        else
            cout << n << " is normal.\n";
    }
    return 0;
}
```

19.9 模逆元

19.9.1 模整数类型的批量逆元

batch_units(a) 返回 optional<vector<Z»，保持输入顺序；任一元素不可逆返回 nullopt，空输入返回空数组。Z 须可从 1 构造，支持乘法、复合乘法及 try_inv，且元素来自同一交换模整数环；目前可与 mint<tag> 直接组合。先累计前缀乘积，只对总乘积求一次逆，再倒序求各项逆元。总乘积可逆当且仅当所有因子可逆，因此合数模数也可使用。O(n) 次乘法加一次求逆，空间 O(n)，要求大小可表示为 int；建议传 move(a) 避免保留额外输入副本。返回前不能切换动态模数。现有 long long 版 batch_inverse 保留独立接口与历史证据，不由本函数替换。

```
template <class Z> optional<vector<Z>> batch_units(vector<Z> a)
{
    int n = a.size();
    vector<Z> pre(n + 1, Z(1));
    for (int i = 0; i < n; i++) pre[i + 1] = pre[i] * a[i];
    auto x = pre[n].try_inv();
    if (!x) return nullopt;
    Z inv = *x;
    for (int i = n - 1; i >= 0; i--)
    {
        Z old = a[i];
        a[i] = inv * pre[i];
        inv *= old;
    }
    return a;
}
```

联合使用示例见第 132 页（Luogu P5431 【模板】模意义下的乘法逆元 2），其中直接调用本模板。

19.9.2 任意正模数的单个逆元

代码依赖：extended_gcd（第 156 页，19.7.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

mod_inverse(a,m) 支持完整 long long a 和正 long long m，将 a 规范化后用 extended_gcd 求逆。不可逆返回-1，可逆返回 0..m-1 内的代表；模数 1 返回 0。允许合数模数，不使用费马小定理。O(log m) 时间与递归空间。

```
long long mod_inverse(long long a, long long m)
{
    using i128 = __int128_t;
    assert(m > 0);
    a %= m;
    if (a < 0) a += m;
    i128 x, y;
    if (extended_gcd(a, m, x, y) != 1) return -1;
    return (x % m + m) % m;
}
```

应用题补充：Luogu P1082 [NOIP 2012 提高组] 同余方程

题意：给出 2≤a,b≤2000000000，保证 ax≡1 (mod b) 有解，求最小正整数 x。直接调用 mod_inverse(a,b)；保证有解等价于 gcd(a,b)=1，且 b≥2，因此归一化结果位于 [1,b−1]，正好是最小正解。b 不必为素数，a 可以大于 b，不能改用费马小定理的幂公式。

所需模板：extended_gcd（第 156 页，19.7.1 节）、mod_inverse（第 164 页，19.9.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1082

```
int main()
{
    long long a, b;
    scanf("%lld%lld", &a, &b);
    printf("%lld\n", mod_inverse(a, b));
    return 0;
}
```

19.9.3 任意数组的批量逆元

代码依赖：mod_inverse（第 164 页，19.9.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

batch_inverse(a,mod) 返回与 a 同序的逆元向量。要求 mod 为正 long long，每个输入可为完整 long long；只要某一项与 mod 不互素，整个结果返回 nullopt，不输出部分逆元。空输入返回有效的空向量；模数 1 约定返回全零，与单个逆元接口一致。模数可以是合数。函数按值接收数组并归一化；不再需要原数组时传 move(a) 可避免输入副本。依赖 mod_inverse 和 extended_gcd。仅求一次逆元，时间 O(n+log mod)，空间 O(n)；128 位中间乘法防止溢出。

```
optional<vector<long long>> batch_inverse(vector<long long> a, long long mod)
{
    assert(mod > 0);
    int n = a.size();
    vector<long long> prefix(n + 1, 1 % mod);
    for (int i = 0; i < n; i++)
    {
        a[i] %= mod;
        if (a[i] < 0) a[i] += mod;
        prefix[i + 1] = __int128_t(prefix[i]) * a[i] % mod;
    }
    long long inverse = mod_inverse(prefix[n], mod);
    if (inverse == -1) return nullopt;
    for (int i = n - 1; i >= 0; i--)
    {
        long long value = a[i];
        a[i] = __int128_t(inverse) * prefix[i] % mod;
        inverse = __int128_t(inverse) * value % mod;
    }
    return a;
}
```

模板题使用：Luogu P5431 【模板】模意义下的乘法逆元 2

题意：给定 1≤n≤5000000、2≤k<p≤10^9、1≤a_i<p，p 为素数，输出 ∑(i=1..n)k^i/a_i 模 p。先批量求全部逆元再累乘权重，注意首项权重是 k。完整抄写 read 与 main；read 仅适用于本题的非负十进制输入。move(a) 避免输入副本，此后不再读取 a。本题所有元素可逆；一般模数或零元素须检查 nullopt。乘积在 long long 范围内，批量核心内部用 int128 保护一般模数。

所需模板：extended_gcd（第 156 页，19.7.1 节）、mod_inverse（第 164 页，19.9.2 节）、batch_inverse（第 165 页，19.9.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5431

```
int read()
{
    int c = getchar_unlocked();
    while (c < '0' || c > '9')
    {
        if (c == EOF) return 0;
        c = getchar_unlocked();
    }
    int value = 0;
    while ('0' <= c && c <= '9')
    {
        value = value * 10 + c - '0';
        c = getchar_unlocked();
    }
    return value;
}

int main()
{
    int n = read(), p = read(), k = read();
    vector<long long> a(n);
    for (auto &x : a) x = read();
    auto inverse = batch_inverse(move(a), p);
    if (!inverse) return 1;
    long long answer = 0, weight = 1;
    for (auto x : *inverse)
    {
        weight = weight * k % p;
        answer = (answer + weight * x) % p;
    }
    printf("%lld\n", answer);
    return 0;
}
```

19.9.4 连续整数的线性逆元表

inverse_table(n,p) 返回长度 n+1 的 int 向量，位置 i 保存 i 模 p 的逆元，1≤i≤n。位置 0 固定为 0，仅为占位，不表示零有逆元；n=0 返回单个占位项。要求 p 为素数、2≤p≤INT_MAX 且 0≤n<p，不检测素性。时间和空间均 O(n)，中间乘法用 long long；独立可抄，不依赖其他模板。不能将同一递推直接用于合数模数，即使当前 i 与模数互素；一般模数使用扩展欧几里得或任意数组批量逆元。

```
vector<int> inverse_table(int n, int p)
{
    assert(p >= 2 && 0 <= n && n < p);
    vector<int> inverse(n + 1);
    if (n) inverse[1] = 1;
    for (int i = 2; i <= n; i++) inverse[i] = p - 1LL * (p / i) * inverse[p % i] % p;
    return inverse;
}
```

模板题使用：Luogu P3811 【模板】模意义下的乘法逆元

题意：输入 1≤n≤3000000、n<p<20000528，p 保证为素数，输出 1..n 各自的逆元，每项一行。inverse_table 返回 n+1 项，0 是占位且不输出。该递推要求素数，不能照搬到合数；一般数组使用 batch_inverse。

所需模板：inverse_table（第 167 页，19.9.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3811

```
int main()
{
    int n, p;
    scanf("%d%d", &n, &p);
    auto inverse = inverse_table(n, p);
    for (int i = 1; i <= n; i++) printf("%d\n", inverse[i]);
    return 0;
}
```

19.10 线性同余方程

19.10.1 线性同余方程

代码依赖：extended_gcd（第 156 页，19.7.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

linear_congruence(a,b,m) 求 a*x+b≡0 (mod m)，返回 {最小非负解, 最小正周期}，无解返回 {-1,-1}。m 为正 long long，a、b 支持完整 long long。若 g=gcd(a,m) 整除 b，周期为 m/g；包括 a=0 和 m=1，所有整数均满足时返回 {0,1}。先归一化 a，并在 int128 中对 b 取相反数和乘法。依赖 extended_gcd；递归时间、空间 O(log(m+1))。注意本接口的常数项是 +b，若题目写 a*x≡c，应传 -c；c 为 LLONG_MIN 时须避免在 64 位中取负。

```
pair<long long, long long> linear_congruence(long long a, long long b, long long m)
{
    assert(m > 0);
    using i128 = __int128_t;
    a %= m;
    if (a < 0) a += m;
    i128 x, y;
    i128 g = extended_gcd(a, m, x, y);
    if (i128(b) % g) return {-1, -1};
    i128 period = m / g;
    x = x * (-i128(b) / g) % period;
    if (x < 0) x += period;
    return {(long long)x, (long long)period};
}
```

应用题补充：Luogu P1516 [ZJOI2002] 青蛙的约会

题意：两只青蛙从 x,y 同向跳，每次前进 m,n，圆周长 L。相遇式为 (m-n)*t+(x-y)≡0 (mod L)，故传 linear_congruence(m-n,x-y,L)，不能把常数项误写为 y-x。负速度差无需交换或另取负。返回 {最小非负次数, 最小正周期}；只输出第一项，无解输出 Impossible，不能把周期当答案。x,y,m,n 在 1..2×10^9 且 x≠y，L 在 1..2.1×10^9；差值用 long long。位置按 L 取模，初始重合可返回 0，等速不重合则无解。单组输入，无多测或结束标记。

所需模板：extended_gcd（第 156 页，19.7.1 节）、linear_congruence（第 168 页，19.10.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1516

```
int main()
{
    long long x, y, m, n, length;
    scanf("%lld%lld%lld%lld%lld", &x, &y, &m, &n, &length);
    auto [time, period] = linear_congruence(m - n, x - y, length);
    if (time < 0)
        puts("Impossible");
    else
        printf("%lld\n", time);
    return 0;
}
```

19.11 中国剩余定理

19.11.1 广义 CRT：逐项合并同余

代码依赖：extended_gcd（第 156 页，19.7.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

crt_merge(r,m,b,n) 合并 x≡r (mod m) 与 x≡b (mod n)，m,n>0，r、m 须为不同变量。从 r=0、m=1 开始逐项合并；成功后 r 为最小非负解、m 为最小正周期。余数支持完整 long long。无解返回 false；有解但新周期超过 LLONG_MAX 抛 overflow_error。两种失败都会先规范化 r, m 不变, 失败后不能视为完整解。依赖 extended_gcd，中间量用 int128；O(log min(m,n)) 时间与递归空间。

```
bool crt_merge(long long &r, long long &m, long long b, long long n)
{
    using ll = long long;
    using i128 = __int128_t;
    assert(m > 0 && n > 0);
    r %= m;
    if (r < 0) r += m;
    b %= n;
    if (b < 0) b += n;
    i128 x, y;
    ll g = (ll)extended_gcd(m, n, x, y);
    i128 diff = i128(b) - r;
    if (diff % g) return false;
    i128 q = n / g, k = (diff / g * x % q + q) % q, mod = i128(m) * q;
    if (mod > LLONG_MAX) throw overflow_error("CRT modulus");
    r = (r + i128(m) * k) % mod;
    m = (ll)mod;
    return true;
}
```

模板题使用：Luogu P4777

题意：求同时满足多个同余式的最小非负整数，模数可以不互质。每行按模数、余数输入，逐式合并；合并后模数须在 long long 范围内。

所需模板：extended_gcd（第 156 页，19.7.1 节）、crt_merge（第 169 页，19.11.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4777

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int count;
    cin >> count;
    long long r = 0, modulus = 1;
    while (count--)
    {
        long long m, b;
        cin >> m >> b;
        if (!crt_merge(r, modulus, b, m))
        {
            cout << -1 << '\n';
            return 0;
        }
    }
    cout << r << '\n';
    return 0;
}
```

19.11.2 Garner：大周期 CRT 与混合进制

代码依赖：mod_inverse（第 164 页，19.9.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

garner_digits(b,moduli) 返回按输入顺序排列的混合进制各位；garner(b,moduli,target) 返回最小非负 CRT 解对 target 的余数。两个输入向量等长，长度可为 0；模数为正且两两互素，允许模数 1，余数支持完整 long long 范围。target 为正 long long，与输入模数无需互素。输入不互素时抛 invalid_argument，即使方程有解也不适用；此时用扩展 CRT。空输入的规范解取 0。算法从不构造模数乘积，中间运算使用 int128。k 个模数耗时 $O(k^2+k \log M)$，空间 O(k)；M 为最大输入模数。依赖 mod_inverse 和 extended_gcd。

```
// Pairwise coprime positive moduli. Digits use the input order.
vector<long long> garner_digits(const vector<long long> &b,
                                const vector<long long> &moduli)
{
    using ll = long long;
    using i128 = __int128_t;
    int n = b.size();
    assert(moduli.size() == b.size());
    vector<ll> value(n), coefficient(n, 1), digit(n);
    for (ll m : moduli) assert(m > 0);
    for (int i = 0; i < n; i++)
    {
        ll m = moduli[i];
        ll inverse = mod_inverse(coefficient[i], m);
        if (inverse == -1) throw invalid_argument("Garner moduli are not coprime");
        i128 delta = (i128(b[i]) - value[i]) % m;
        if (delta < 0) delta += m;
        digit[i] = delta * inverse % m;
        for (int j = i + 1; j < n; j++)
        {
            value[j] = (value[j] + i128(coefficient[j]) * digit[i]) % moduli[j];
            coefficient[j] = i128(coefficient[j]) * m % moduli[j];
        }
    }
    return digit;
}
```

规范解对目标数取模

```
// Return the least nonnegative CRT solution modulo target, not all solutions.
long long
garner(const vector<long long> &b, const vector<long long> &moduli, long long target)
{
    assert(target > 0);
    auto digit = garner_digits(b, moduli);
    long long answer = 0;
    for (int i = (int)digit.size() - 1; i >= 0; i--)
        answer = (__int128_t(answer) * moduli[i] + digit[i]) % target;
    return answer;
}
```

模板题使用：Library Checker Convolution (Mod 1,000,000,007)

题意：给出长度 n、m 的升幂系数，输出模 1000000007 的卷积，共 n+m−1 项。两数组长度均为 1 到 524288；输入系数非负且小于目标模数。分别在 167772161、469762049、1224736769 下做 NTT，再逐项调用既有 garner。真实非负系数不超过 524288 乘以 1000000006 的平方，严格小于三素数乘积。因此 CRT 最小非负代表就是原整数系数，最后对目标模数取余；仅有模数互素并不足够。输入通过 ModInt 构造器归一化。三模数公共变换容量为 2^24，本题至多需 2^20；无系数界时不能直接套用。

所需模板：extended_gcd（第 156 页，19.7.1 节）、mod_inverse（第 164 页，19.9.2 节）、garner（第 171 页，19.11.2 节）、ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/convolution_mod_1000000007

```
template <int p>
auto conv(const vector<int> &a, const vector<int> &b)
{
    using N = NttConvolution<p>;
    typename N::Poly x(a.begin(), a.end()), y(b.begin(), b.end());
    return N::multiply(move(x), move(y));
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<int> a(n), b(m);
    for (auto &x : a) cin >> x;
    for (auto &x : b) cin >> x;
    auto x = conv<167772161>(a, b);
    auto y = conv<469762049>(a, b);
    auto z = conv<1224736769>(a, b);
    vector<long long> r(3), p = {167772161, 469762049, 1224736769};
    for (int i = 0; i < n + m - 1; i++)
    {
        r[0] = x[i].v;
        r[1] = y[i].v;
        r[2] = z[i].v;
        cout << garner(r, p, 1000000007) << (i + 1 == n + m - 1 ? '\n' : ' ');
    }
}
```

19.12 升幂引理

19.12.1 素因子估值与升幂：先约去公共因子

本节只处理整数的素因子重数。以下 p 为素数，a,b,n 为正整数；对非零整数 x，定义 $v_p(x)=\max\{e\ge 0: p^e \mid x\}$，负数按绝对值计算。**不定义有限的** $v_p(0)$：求差时先判断 $a=b$，此时 $a^n-b^n=0$，被每个正整数整除，却没有最大的有限素因子重数，不能进入下面的估值公式或反复除 p 的循环。

19.12.1.1 基本运算与公共因子

对非零整数 x,y，有

$$v_p(xy)=v_p(x)+v_p(y),\qquad v_p(x^n)=nv_p(x).$$

若 $y\mid x$，整数商满足 $v_p(x/y)=v_p(x)-v_p(y)$。当 $x+y\neq 0$ 时，$v_p(x+y)\ge \min(v_p(x),v_p(y))$；两者估值不同才必定取等。证明只需提出较小的 p 次幂：剩下两项中恰有一项不被 p 整除；估值相同时可能抵消，例如 $v_3(1)=v_3(2)=0$，但 $v_3(1+2)=1$。

计算 $a^n\pm b^n$ 时先记 $\alpha=v_p(a)$、$\beta=v_p(b)$。若 $\alpha\neq\beta$，则直接得到

$$v_p(a^n\pm b^n)=n\min(\alpha,\beta).$$

若 $\alpha=\beta=t$，写成 $a=p^tA, b=p^tB$，其中 $p\nmid AB$，则

$$v_p(a^n\pm b^n)=nt+v_p(A^n\pm B^n).$$

差式仍要求 $a\neq b$。约去公共因子后，才能对 A,B 检查下面的 LTE 条件；不能把 nt 误写为 t，也不能只凭 $p\mid a-b$ 就直接套公式。

19.12.1.2 奇素数：差式与和式的条件不同

以下两式均要求 p 为**奇素数**且 $p\nmid ab$：

$$p\mid a-b,\ a\neq b \quad\Longrightarrow\quad v_p(a^n-b^n)=v_p(a-b)+v_p(n),$$
$$p\mid a+b,\ n\text{ 为奇数} \quad\Longrightarrow\quad v_p(a^n+b^n)=v_p(a+b)+v_p(n).$$

差式不要求 n 为奇数；和式不能漏掉这一条件。这里 $p\nmid ab$ 只要求两数都不含 p，并不要求 $\gcd(a,b)=1$。

为什么指数中每个 p 恰好贡献一次。设 $n=mp^s$，$p\nmid m$。当 $p\mid a-b$ 时，$(a^m-b^m)/(a-b)$ 模 p 等于 mb^{m-1}，故指数的 m 部分不增加估值。再设 $x=y+h$，$p\nmid y$、$v_p(h)=r\ge 1$。二项式展开 x^p-y^p：一次项 $py^{p-1}h$ 的估值为 $r+1$，其余项均至少为 $r+2$（末项用 $pr\ge r+2$）。所以每次把指数乘 p 都使估值增加 1，重复 s 次即得差式。和式可对 a^2,b^2 使用差式，再用 $a^{2n}-b^{2n}=(a^n-b^n)(a^n+b^n)$：当 n 奇且 $a\equiv -b\pmod p$ 时，$p\nmid a-b$、$p\nmid a^n-b^n$，两边相减只剩所求和式的估值。

19.12.1.3 素数 2：先分指数奇偶

若 a,b 均为奇数且 $a\neq b$，则

$$v_2(a^n-b^n)=\begin{cases} v_2(a-b), & n\text{ 为奇数},\\ v_2(a-b)+v_2(a+b)+v_2(n)-1, & n\text{ 为偶数}.\end{cases}$$

对奇数 a,b 的和式（允许 $a=b$），有

$$v_2(a^n+b^n)=\begin{cases} v_2(a+b), & n\text{ 为奇数},\\ 1, & n\text{ 为偶数}.\end{cases}$$

奇指数时，差式与和式各自除以 $a-b$、$a+b$ 后的多项式均是奇数个奇数的和或交错和，商为奇数。偶指数差式写 $n=2^sm$、m 奇：先消去奇指数 m，再反复平方差分解；首两因子是 $a-b, a+b$，以后每个 $a^{2^j}+b^{2^j}$（$j\ge 1$）均模 4 余 2，恰好再贡献 $s-1$。偶指数和式也因奇数的偶次幂模 4 余 1，估值恰为 1。特别地，若 $4\mid a-b$，则 $v_2(a+b)=1$，差式可统一写成 $v_2(a^n-b^n)=v_2(a-b)+v_2(n)$；没有这个附加条件不能照搬奇素数公式。若 a,b 奇偶不同，和差均为奇数；若都偶，先按上一小节约去公共 2 因子。

19.12.1.4 把大整数整除转成指数条件

例：求使 $2^6 \cdot 3^2 \mid 7^n - 1$ 的最小正整数 n。奇数 n 时 $v_2(7^n-1) = v_2(6) = 1$，必不满足。偶数 n 时两种素数分别给出

$$v_2(7^n-1) = 1 + 3 + v_2(n) - 1 = 3 + v_2(n), \qquad v_3(7^n-1) = 1 + v_3(n).$$

所以须且仅须 $v_2(n) \geq 3$、$v_3(n) \geq 1$，即 $24 \mid n$，最小值为 24。一般对非零整数 X 和 $M = \prod p_i^{e_i}$，$M \mid X$ 等价于逐项检查 $v_{p_i}(X) \geq e_i$；无需真的构造可能远超机器字长的幂。LTE 只能处理满足条件的局部因子，不能代替任意底数的乘法阶或离散对数问题。实现时非零整数反复除 p 即可求估值；$a+b$、差的绝对值及 nt 的计算仍须防溢出。

条件漏一项，答案就可能错。

- 不检查 $p \nmid ab$：$p = 3, a = 6, b = 3, n = 2$，实际 $v_3(27) = 3$，硬套差式只得 $v_3(3) + v_3(2) = 1$。
- 不检查 $p \mid a - b$：$p = 3, a = 2, b = 1, n = 2$，实际 $v_3(3) = 1$，右侧 $v_3(1) + v_3(2) = 0$。
- 将奇素数差式用于 2：$a = 3, b = 1, n = 2$，实际 $v_2(8) = 3$，错式只得 $v_2(2) + v_2(2) = 2$。
- 和式允许偶指数：$p = 3, a = 2, b = 1, n = 2$，实际 $v_3(5) = 0$，错式却给 $v_3(3) + v_3(2) = 1$。

定理条件参见 OI Wiki：升幂引理。本节限于整数估值、LTE 及整除建模，不代表完整的 p 进数理论，也不新增实现或在线评测记录。

与已有模板分工。扩展 Lucas 的阶乘赋值用 $v_p(N!) = \sum_{j \geq 1} \lfloor N/p^j \rfloor$，不是把整个阶乘硬套 LTE；见 ExLucas （第 178 页，19.14.2 节）。素数幂开根还要检查单位部分是否有根，单有估值条件并不充分；见 PrimePowerRoots （第 196 页，19.18.2 节）。

19.13 阶乘取模

19.13.1 Legendre 赋值与 Kummer 进位判据

设 p 为素数，$v_p(a)$ 表示正整数 a 中质因子 p 的指数。$n \geq 0$ 时，Legendre 公式为

$$v_p(n!) = \sum_{j \geq 1} \left\lfloor \frac{n}{p^j} \right\rfloor = \frac{n - s_p(n)}{p-1},$$

其中 $s_p(n)$ 是 n 的 p 进制数位和，$0! = 1$ 给出赋值 0。第一式把每个数的质因子逐层数：p 的倍数贡献至少 1 个，p^2 的倍数再贡献 1 个，依此类推。若 $n = \sum_i a_i p^i$，则交换有限求和得到

$$\sum_{j \geq 1} \lfloor n/p^j \rfloor = \sum_{i \geq 1} a_i (1 + p + \cdots + p^{i-1}) = \sum_i a_i \frac{p^i - 1}{p-1}.$$

素数条件不可省略：若把 p 错设为 4，$6! = 720$ 能被 4^2 整除，但 $\lfloor 6/4 \rfloor + \lfloor 6/16 \rfloor = 1$，不是最高 4 次幂的指数。

实现时反复执行 $n \leftarrow \lfloor n/p \rfloor$ 并累加，直到 $n = 0$，时间 $O(1 + \log_p(n+1))$。避免逐次计算 p^j，后者可能在判断终止前就溢出。已有 `ExLucas::valuation(n,p)` 使用这种方式，见第 19.14.2 节（第 178 页）；n 是 unsigned64、p 是正的 int 素数。该内部辅助函数不验证素数，尤其不能传入 0 或 1。由于 $v_p(n!) \leq n$，其 unsigned64 累加不会超出合法 n 的范围。

19.13.1.1 组合数：逐位相加时进了几次位

对于 $0 \leq k \leq n$，令 $h = n - k$，由阶乘比值的赋值相减可得

$$v_p\binom{n}{k} = v_p(n!) - v_p(k!) - v_p(h!) = \frac{s_p(k) + s_p(h) - s_p(n)}{p-1}.$$

把 $k + h = n$ 按 p 进制相加，记第 i 位的输入进位为 c_i，$c_0 = 0$，则

$$k_i + h_i + c_i = n_i + p c_{i+1}.$$

两数相加的 c_i 只可能是 0 或 1。对所有位求和，包含最终进位并在更高位补 0，得到 $s_p(k) + s_p(h) - s_p(n) = (p-1)\sum_{i \geq 1} c_i$。所以赋值恰好是加法产生的进位次数，这就是 Kummer 定理。传播进位也要计数：$1 + 7 = 8$ 的二进制加法有 3 次进位，对应 $\binom{8}{1} = 8$ 中的 2^3；不能只数原始两位都为 1 的位置。$k = 0$ 或 $k = n$ 均无进位，对应组合数 1。

19.13.1.2 素数幂为零与 Lucas 的边界

对整数 $a \geq 1$，

$$\binom{n}{k} \equiv 0 \pmod{p^a} \iff v_p\binom{n}{k} \geq a.$$

这是“至少 a 次进位”，不是严格大于 a；例如 $\binom{4}{2} = 6$ 模 2 为 0，模 4 不为 0。若模数 $m = \prod_j p_j^{a_j} > 1$，模 m 为零须每个素数幂分量都满足阈值，不能只命中一个。模 1 的结果恒为 0，另行处理。

零进位等价于每个数位都满足 $k_i \leq n_i$：逐位相减无需借位，恰好能无进位相加回 n。这也解释了 Lucas 公式在素数模下何时非零，见第 19.14.1 节（第 177 页）。但知道赋值只够判定可整除性，不足以恢复非零余数。例如 $\binom{4}{1} = 4$ 与 $\binom{5}{1} = 5$ 的 v_3 都是 0，模 3 余数却分别为 1 和 2。`ExLucas::choose` 因此还需计算去除 p 因子的单位部分、乘回 p^v，再通过 CRT 合并。这里不扩大其模数、预处理空间或参数类型的现有契约，也不能把不可逆阶乘直接相除。

$k > n$ 时按组合数约定答案为 0，必须先处理；0 没有有限的上述赋值，不能做无符号 $n - k$ 或套用有限进位公式。负参数不在本节范围。阶乘赋值是逐项质因子计数，不是对指数差使用 LTE；相关选型见第 19.12.1 节（第 173 页）。证明背景见 Jorza 数论讲义中的 Legendre 与 Kummer 定理；已有赋值算法的导航见 固定版本 OI Wiki 阶乘取模。

19.14 卢卡斯定理

19.14.1 Lucas：小素数模的大下标组合数

Lucas(p) 预处理素数模数 p，choose(n,k) 返回 C(n,k) 模 p；n、k 为完整 unsigned long long 范围，k>n 返回 0，C(0,0)=1。重新赋值 Lucas(p) 可更换模数并重新预处理；不检测素性，要求调用者保证 p 为素数且 p≥2。预处理时间 O(p+log p)，动态版空间 O(p)；单次查询 O(1+log_p(n+1))。适合小素数模数，通常 $p \le 10^6$；大素数不能因查询下标小就忽略这里整段 O(p) 的预处理。fac、ifac 下标为 0..p-1，模数和表只读；中间乘法用 long long。合数模数请使用 exLucas。

```
struct Lucas
{
    int mod;
    vector<int> fac, ifac;

    Lucas(int p)
    {
        assert(p >= 2);
        mod = p;
        fac.resize(p);
        ifac.resize(p);
        fac[0] = 1;
        for (int i = 1; i < p; i++) fac[i] = 1LL * fac[i - 1] * i % p;
        long long a = fac[p - 1], r = 1;
        for (int e = p - 2; e; e >>= 1)
        {
            if (e & 1) r = r * a % p;
            a = a * a % p;
        }
        ifac[p - 1] = r;
        for (int i = p - 1; i; i--) ifac[i - 1] = 1LL * ifac[i] * i % p;
    }
```

```
    int choose(unsigned long long n, unsigned long long k) const
    {
        if (k > n) return 0;
        long long answer = 1;
        while (n || k)
        {
            int a = n % mod, b = k % mod;
            if (b > a) return 0;
            answer = answer * fac[a] % mod;
            answer = answer * ifac[b] % mod;
            answer = answer * ifac[a - b] % mod;
            n /= mod;
            k /= mod;
        }
        return answer;
    }
};
```

模板题使用：Luogu P3807

题意：多组查询 C(n+m,n) 模 p，题目保证 p 为素数且不超过 10^5。每组构造 Lucas(p)，choose 的参数为 n+m、n；此题范围内 n+m 不会溢出。

所需模板：Lucas（第 177 页，19.14.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3807

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        unsigned long long n, m;
        int p;
        cin >> n >> m >> p;
        Lucas solver(p);
        cout << solver.choose(n + m, n) << '\n';
    }
    return 0;
}
```

19.14.2 合数模数组合数（扩展 Lucas）

ExLucas(mod) 预处理固定模数，choose(n,k) 返回 C(n,k) 模 mod；n、k 为 unsigned long long，k>n 返回 0，mod=1 返回 0。适合 mod≤10^6 的合数或素数模数，不能用于 10^9 级素数的整段预处理。分解 mod 为互素素数幂 q=p^e，预处理每段内不含 p 因子的前缀积；unit(n,f) 去除 n! 中全部 p 因子，valuation 统计指数。分子分母指数之差不少于 e 时该项为零，否则只对互素单位部分求逆，最后 CRT 合并。预处理 O($\sqrt{\text{mod}}+\sum$q)，空间 O($\sum$(q+1))；每次查询 O($\sum$(log_p(n+1) · log(n+1)+log q))，求和遍历不同素因子。所有乘法用 long long；同一对象可重复查询，factors、fac 与辅助函数视为内部数据。

```
struct ExLucas
{
    using ull = unsigned long long;
    using ll = long long;

    struct Factor
    {
        int p, q, exponent, start, crt;
    };

    int mod;
    vector<int> fac;
    vector<Factor> factors;

    static ll power(ll a, ull b, int m)
    {
        ll result = 1;
        while (b)
        {
            if (b & 1) result = result * a % m;
            a = a * a % m;
```

```
            b >>= 1;
        }
        return result;
    }

    ExLucas(int m) : mod(m)
    {
        assert(m >= 1);
        int x = m;
        for (int p = 2; p <= x / p; p++)
        {
            if (x % p) continue;
            int q = 1, e = 0;
            do
            {
                x /= p;
                q *= p;
                e++;
            } while (x % p == 0);
            add_factor(p, q, e);
        }
        if (x > 1) add_factor(x, x, 1);
    }

    void add_factor(int p, int q, int e)
    {
        int start = fac.size();
        fac.push_back(1);
        for (int i = 1; i < q; i++) fac.push_back(ll(fac.back()) * (i % p ? i : 1) % q);
        fac.push_back(fac.back());
        int phi = q - q / p;
        int crt = ll(mod / q) * power(mod / q % q, phi - 1, q) % mod;
        factors.push_back({p, q, e, start, crt});
    }

    static ull valuation(ull n, int p)
    {
        ull result = 0;
        while (n)
        {
            n /= p;
            result += n;
        }
        return result;
    }

    ll unit(ull n, const Factor &f) const
    {
```

```
        ll result = 1;
        while (n)
        {
            result = result * power(fac[f.start + f.q], n / f.q, f.q) % f.q;
            result = result * fac[f.start + n % f.q] % f.q;
            n /= f.p;
        }
        return result;
    }
```

```
    int choose(ull n, ull k) const
    {
        if (k > n || mod == 1) return 0;
        ll answer = 0;
        for (const auto &f : factors)
        {
            ull e = valuation(n, f.p) - valuation(k, f.p) - valuation(n - k, f.p);
            if (e >= ull(f.exponent)) continue;
            ll denominator = unit(k, f) * unit(n - k, f) % f.q;
            ll residue =
                unit(n, f) * power(denominator, f.q - f.q / f.p - 1, f.q) % f.q;
            residue = residue * power(f.p, e, f.q) % f.q;
            answer = (answer + residue * f.crt) % mod;
        }
        return answer;
    }
};
```

模板题使用：Luogu P4720

题意：求 C(n,m) 模 p，p 可以为合数且不超过 10^6。构造 ExLucas(p) 后 choose(n,m)；预处理与模数规模有关，不能直接用于 10^9 级模数。

所需模板：ExLucas（第 178 页，19.14.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4720

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    unsigned long long n, m;
    int p;
    std::cin >> n >> m >> p;
    ExLucas comb(p);
    std::cout << comb.choose(n, m) << '\n';
}
```

相关应用：按上标范围预处理的合数模数组合数，与小模数大上标 ExLucas 互补

19.14.3 有界上标的任意模数组合数

CompositeBinomial(N,mod) 预处理上标 0..N 的组合数，0≤N<INT_MAX、1≤mod≤INT_MAX；mod 可为合数。choose(n,k) 返回 C(n,k) 模 mod；负 n、负 k、k>n 返回 0，其余要求 n≤N。mod=1 时所有结果为 0。每个素数幂分别保存去除对应素因子的阶乘、逆阶乘和指数前缀，再用 CRT 合并。

设 s 为 mod 的不同素因子个数。预处理 O($\sqrt{\text{mod}}$+s(N+log mod)) 时间、O(s(N+1)) 空间，单次查询 O(s)。各素数幂的 p 次幂表很短；指数达到该素数幂的次数即直接贡献 0。表长取决于 N，不按 mod 或素数幂大小开数组，适合 N≤10^5、mod 为 10^9 级数的多次查询。n 为完整 unsigned64 且模数小的场景仍用 ExLucas。

limit、mod、factors 及辅助函数视为内部数据；不支持自动扩表或修改模数。所有模乘提升为 long long，CRT 每乘一次即取模。已通过独立 Pascal 递推、Python 精确组合数、原稿安全小域对照和 ASan/UBSan；CF896D 为已验证的本地应用驱动，尚无本次线上 AC。

```
struct CompositeBinomial
{
    struct Factor
    {
        int p, q, crt;
        vector<int> fac, ifac, exponent, powers;
    };

    int limit, mod;
    vector<Factor> factors;

    static long long power(long long a, int e, int m)
    {
        long long result = 1;
        while (e)
        {
            if (e & 1)
                result = result * a % m;
            a = a * a % m;
            e >>= 1;
        }
        return result;
    }
```

有界上标组合数（接上页同一结构体）：

```
    CompositeBinomial(int n, int m) : limit(n), mod(m)
    {
        assert(n >= 0 && n < INT_MAX && m >= 1);
        int rest = m;
        for (int p = 2; p <= rest / p; p++)
        {
            if (rest % p != 0)
                continue;
            int q = 1;
            do
            {
                rest /= p;
                q *= p;
            } while (rest % p == 0);
            add_factor(p, q);
        }
        if (rest > 1)
            add_factor(rest, rest);
    }

    void add_factor(int p, int q)
    {
        int phi = q - q / p;
        int crt = (mod / q) * power(mod / q % q, phi - 1, q) % mod;
        Factor f{p, q, crt, vector<int>(limit + 1, 1),
                 vector<int>(limit + 1, 1), vector<int>(limit + 1), {}};
        for (long long x = 1; x < q; x *= p)
            f.powers.push_back(x);
        for (int i = 1; i <= limit; i++)
        {
            int x = i;
            f.exponent[i] = f.exponent[i - 1];
            while (x % p == 0)
            {
                x /= p;
                f.exponent[i]++;
            }
            f.fac[i] = 1LL * f.fac[i - 1] * x % q;
        }
        f.ifac[limit] = power(f.fac[limit], phi - 1, q);
        for (int i = limit; i >= 1; i--)
        {
            int x = i;
            while (x % p == 0)
                x /= p;
            f.ifac[i - 1] = 1LL * f.ifac[i] * x % q;
        }
        factors.push_back(move(f));
    }
```

有界上标组合数（接上页同一结构体）：

```
    int choose(int n, int k) const
    {
        if (n < 0 || k < 0 || k > n)
            return 0;
        assert(n <= limit);
        long long answer = 0;
        for (const auto &f : factors)
        {
            int e = f.exponent[n] - f.exponent[k] - f.exponent[n - k];
            if (e >= int(f.powers.size()))
                continue;
            long long residue = 1LL * f.fac[n] * f.ifac[k] % f.q;
            residue = residue * f.ifac[n - k] % f.q;
            residue = residue * f.powers[e] % f.q;
            answer = (answer + residue * f.crt) % mod;
        }
        return answer;
    }
};
```

接口用法演示：有界上标合数模数组合数 API

演示约定：输入预处理上界 N、正 int 模数 mod 和询问数；每组 n k 输出 C(n,k) 模 mod。负 n、负 k 或 k>n 为 0，否则必须 n≤N。数组按 N 开，支持大合数或大素数模数。

所需模板：CompositeBinomial（第 181 页，19.14.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/COMPOSITE-BINOMIAL.md

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, mod, tests;
    cin >> n >> mod >> tests;
    CompositeBinomial comb(n, mod);
    while (tests--)
    {
        int a, b;
        cin >> a >> b;
        cout << comb.choose(a, b) << '\n';
    }
}
```

应用题补充：CF 896D Nephren Runs a Cinema

题意：输入 n、mod、l、r，统计长度 n、步长为-1/0/1、所有前缀和非负且末尾高度在 [l,r] 的序列数。n≤10^5，1≤mod≤2×10^9。设 S(h) 为不限制前缀的末尾高度 h 计数，反射原理给出合法数 S(h)-S(h+2)，区间和只需四项 S；按向下步数枚举两个组合数乘积求 S。总时间 O(√mod+s(n+log mod))、空间 O(s(n+1))，s 为不同素因子数。

所需模板：CompositeBinomial（第 181 页，19.14.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/896/D

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, mod, l, r;
    cin >> n >> mod >> l >> r;
    CompositeBinomial comb(n, mod);
    auto coefficient = [&](int h)
    {
        if (h > n)
            return 0LL;
        long long answer = 0;
        for (int down = 0; down <= (n - h) / 2; down++)
        {
            long long ways = comb.choose(n, down);
            ways = ways * comb.choose(n - down, down + h) % mod;
            answer = (answer + ways) % mod;
        }
        return answer;
    };
    long long answer = coefficient(l) + coefficient(l + 1);
    answer -= coefficient(r + 1) + coefficient(r + 2);
    answer = (answer % mod + mod) % mod;
    cout << answer << '\n';
}
```

相关应用：Gaussian 二项式的 q-Lucas 取值；普通组合数 Lucas 只是其高位部分

19.14.4 素数模 Gaussian 二项式

q_binomial(n,k,q,p) 返回 Gaussian 二项式在 q 处的值模素数 p，规范为 [0,p)。n、k 为 unsigned long long；k>n 返回 0。2≤p≤INT_MAX 且必须为素数，函数不做素性检验；q 为 int，可负，先归一化。q=0 时合法 k 返回 1，q=1 退化为普通组合数。

非零 q 先由 p-1 的因子求乘法阶 r。q-Lucas 把答案分成 C(n/r,k/r) 与低位 Gaussian 二项式 [n%r,k%r]；低位 k>n 直接为 0，其余只对非零因子求逆。普通组合数再按 p 进制 Lucas 计算，避免在单位根处直接计算 0/0。

空间 O(1)。试除分解 p-1 花 $O(\sqrt{p})$，缩阶至多 O(log p) 次模幂，各 O(log p)；设 a=n%r、b=k%r，则低位乘积 O(min(b,a-b))，普通 Lucas 逐位成本为 $\sum$min(ki,ni-ki)，另有 O(log p+log_p(n+1)) 开销。因此大素数且某位 k 接近 p/2 仍可能很慢，不是任意大模数的快速组合数算法。所有乘积提升为 long long。与 WIDA13B 原稿及独立递推本地对照，QOJ4281 子空间计数已 AC。WIDA13A 原调用存在零因子约分错误，使用本函数替代其 Gaussian 二项式任务；不提供通用 p 进阶数/单位部分接口。

```
// Gaussian binomial [n choose k]_q modulo a prime p, 2 <= p <= INT_MAX.
int q_binomial(unsigned long long n, unsigned long long k, int q, int p)
{
    assert(p >= 2);
    if (k > n)
        return 0;
    q %= p;
    if (q < 0)
        q += p;
    if (q == 0)
        return 1;
    auto power = [&](long long a, unsigned long long e)
    {
        long long r = 1;
        while (e)
        {
            if (e & 1)
                r = r * a % p;
            a = a * a % p;
            e >>= 1;
        }
        return r;
    };

    int order = p - 1, rest = p - 1;
    for (int d = 2; d <= rest / d; d++)
        if (rest % d == 0)
        {
            while (rest % d == 0)
                rest /= d;
            while (order % d == 0 && power(q, order / d) == 1)
                order /= d;
        }
    if (rest > 1)
        while (order % rest == 0 && power(q, order / rest) == 1)
            order /= rest;
```

Gaussian 二项式（接上页同一函数）：

```
    int a = n % order, b = k % order;
    if (b > a)
        return 0;
    b = min(b, a - b);
    long long num = 1, den = 1;
    long long upper = power(q, a), lower = q, invq = power(q, p - 2);
    for (int i = 0; i < b; i++)
    {
        num = num * (upper + p - 1) % p;
        den = den * (lower + p - 1) % p;
        upper = upper * invq % p;
        lower = lower * q % p;
    }

    n /= order;
    k /= order;
    while (n || k)
    {
        int x = n % p, y = k % p;
        if (y > x)
            return 0;
        y = min(y, x - y);
        for (int i = 1; i <= y; i++)
        {
            num = num * (x - i + 1) % p;
            den = den * i % p;
        }
        n /= p;
        k /= p;
    }
    return num * power(den, p - 2) % p;
}
```

接口用法演示：Gaussian 二项式 API

演示约定：读测试组数，每组 n k q p，n/k 为非负 64 位无符号整数，q 为 int，p 是 [2,INT_MAX] 内的素数；输出 Gaussian 二项式模 p。k>n 输出 0。注意乘法阶及 Lucas 各位的乘积循环成本，详见接口说明。

所需模板：q_binomial（第 185 页，19.14.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/Q-BINOMIAL.md

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        unsigned long long n, k;
        int q, p;
        cin >> n >> k >> q >> p;
        cout << q_binomial(n, k, q, p) << '\n';
    }
}
```

应用题补充：QOJ 4281 Subspace

题意：输入素数 q、维数 n、子空间维数 k、素数 p，求 F_q^n 中 k 维子空间个数模 p。原题 q,n≤10^9，1≤k≤n，p≤200000。按有序线性无关向量组数除以每个子空间的有序基数，得到 Gaussian 二项式；单位根处用 q-Lucas，不能直接除零。QOJ 提交 3166447 已 AC（1ms）。

所需模板：q_binomial（第 185 页，19.14.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://qoj.ac/contest/1341/problem/4281

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q, p;
    unsigned long long n, k;
    cin >> q >> n >> k >> p;
    cout << q_binomial(n, k, q, p) << '\n';
}
```

19.15 二次剩余

19.15.1 素数模平方根（Cipolla）

mod_sqrt(n,p) 返回 vector<long long>，按升序列出全部不同根；无解返回空向量，零或模数 2 返回单根。要求 p 为素数，2≤p≤INT_MAX，n 支持完整 long long 范围并在入口归一化。先用 Euler 判别；p≡3 (mod 4) 时直接快速幂，其余使用 Cipolla 二次扩域。函数内保存固定种子的伪随机生成器；随机性影响寻找非剩余的尝试次数，不影响结果，通常期望 O(log p) 时间、O(1) 辅助空间。乘法用 long long，扩域实部的三因子乘积中间先取模；指数中的 p+1 先转 long long，避免 INT_MAX 溢出。此版本不处理合数模数及 64 位大素数，也不用于浮点开方。

```
vector<long long> mod_sqrt(long long n, int p)
{
    using ll = long long;
    assert(p >= 2);
    n = (n % p + p) % p;
    if (n == 0 || p == 2) return {n};
    auto power = [p](ll a, ll b)
    {
        ll result = 1;
        while (b)
        {
            if (b & 1) result = result * a % p;
            a = a * a % p;
            b >>= 1;
        }
        return result;
    };
    if (power(n, (p - 1) / 2) != 1) return {};
    ll answer;
    if (p % 4 == 3)
        answer = power(n, (ll(p) + 1) / 4);
    else
    {
        static mt19937_64 rng(712367);
        ll a, w;
        do
        {
            a = rng() % p;
            w = (a * a % p - n + p) % p;
        } while (power(w, (p - 1) / 2) != p - 1);
        using P = pair<ll, ll>;
        auto mul = [p, w](P x, P y)
        {
            ll real = (x.first * y.first + x.second * y.second % p * w) % p;
            ll imag = (x.first * y.second + x.second * y.first) % p;
            return P{real, imag};
        };
        P result{1, 0}, base{a, 1};
        ll exponent = (ll(p) + 1) / 2;
        while (exponent)
        {
            if (exponent & 1) result = mul(result, base);
```

```
            base = mul(base, base);
            exponent >>= 1;
        }
        answer = result.first;
    }
    ll other = p - answer;
    if (answer > other) swap(answer, other);
    return {answer, other};
}
```

模板题使用：Luogu P5491

题意：多组求 x*x 与 N 模 p 同余的全部解，题目保证 p 为奇素数。mod_sqrt 返回升序且去重的根，空数组表示无解并输出 Hola!。

所需模板：mod_sqrt（第 189 页，19.15.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5491

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int tests;
    std::cin >> tests;
    while (tests--)
    {
        long long n;
        int p;
        std::cin >> n >> p;
        auto roots = mod_sqrt(n, p);
        if (roots.empty())
            std::cout << "Hola!\n";
        else
        {
            for (int i = 0; i < (int)roots.size(); i++)
                std::cout << roots[i] << (i + 1 == (int)roots.size() ? '\n' : ' ');
        }
    }
}
```

19.16 阶 & 原根

19.16.1 乘法阶与原根

代码依赖：Mod64（第 124 页，18.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

构造 PrimitiveRoot(n)，要求 2≤n≤INT_MAX；重新赋值 PrimitiveRoot(n) 可重建。order(a) 返回 a 模 n 的乘法阶，a 不可逆时返回 0；is_root(a) 判断该剩余类是否为原根，a 可为完整 long long。minimum() 返回 1..n-1 内最小原根，无原根返回 -1；all() 返回升序的全部原根，无原根返回空向量。mod、phi、exists 只读；factors 为 phi 的不同素因子，辅助 factor 为试除分解。依赖 Mod64::power。初始化 $O(\sqrt{n})$，求阶 $O(\log^2 n)$，原根判定 $O((1+\omega(phi))\log n)$。若最小原根为 G，顺序枚举最坏检查 G 个候选，不能假定 G 总是常数。枚举全部另需 O(phi log phi+c log c) 时间、O(c) 空间，c 为原根数；大模数只求最小原根或阶时，不应调用 all。

```
struct PrimitiveRoot
```

```
{
    int mod, phi;
    bool exists;
    vector<int> factors;

    static vector<int> factor(int n)
    {
        vector<int> result;
        for (int p = 2; p <= n / p; p++)
            if (n % p == 0)
            {
                result.push_back(p);
                while (n % p == 0) n /= p;
            }
        if (n > 1) result.push_back(n);
        return result;
    }
```

```
    PrimitiveRoot(int n)
    {
        assert(n >= 2);
        mod = n;
        phi = n;
        for (int p : factor(n)) phi -= phi / p;
        int odd = n;
        if (odd % 2 == 0) odd /= 2;
        exists = n == 2 || n == 4 || (odd % 2 && factor(odd).size() == 1);
        factors = factor(phi);
    }

    int order(long long a) const
    {
        a %= mod;
        if (a < 0) a += mod;
        if (gcd(a, (long long)mod) != 1) return 0;
        int result = phi;
        for (int p : factors)
            while (result % p == 0 && Mod64::power(a, result / p, mod) == 1)
                result /= p;
        return result;
    }
```

```
    bool is_root(long long a) const
    {
        if (!exists) return false;
        a %= mod;
        if (a < 0) a += mod;
        if (gcd(a, (long long)mod) != 1) return false;
        for (int p : factors)
            if (Mod64::power(a, phi / p, mod) == 1) return false;
        return true;
    }

    int minimum() const
    {
        if (!exists) return -1;
        for (int g = 1; g < mod; g++)
            if (is_root(g)) return g;
        return -1;
    }
```

```
    vector<int> all() const
    {
        int g = minimum();
        vector<int> roots;
        if (g == -1) return roots;
        long long value = 1;
        for (int k = 1; k <= phi; k++)
        {
            value = value * g % mod;
            if (gcd(k, phi) == 1) roots.push_back(value);
        }
        sort(roots.begin(), roots.end());
        return roots;
    }
};
```

模板题使用：Luogu P6091

题意：多组枚举模 n 的全部原根，先输出个数，再输出升序原根的第 d、2d、3d 等项。all 返回升序数组，下标从 d-1 开始；即使没有取到任何原根，第二行也必须输出空行。

所需模板：Mod64（第 124 页，18.1 节）、PrimitiveRoot（第 190 页，19.16.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6091

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        int n, d;
        cin >> n >> d;
        auto roots = PrimitiveRoot(n).all();
        cout << roots.size() << '\n';
        bool first = true;
        for (int i = d - 1; i < (int)roots.size(); i += d)
        {
            if (!first) cout << ' ';
            cout << roots[i];
            first = false;
        }
        cout << '\n';
    }
    return 0;
}
```

19.17 离散对数

19.17.1 扩展 BSGS

代码依赖：Mod64（第 124 页，18.1 节）、mod_inverse（第 164 页，19.9.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

依赖 number_theory.hpp 的求逆与快速幂。solve(a,b,m) 返回最小 x≥0 使 a^x≡b (mod m)，无解返回 -1；1≤m≤10^{12}，a、b 可为完整 long long 范围，入口归一化，约定 0^0=1。先剥离 gcd 并逐次检查较小指数，再在互质剩余问题上 BSGS。baby 表对同余幂保留最小指数 j，按递增大步 i 检查区间，因此首个命中即最小解。单次期望时间 O($\sqrt{m}$+log m)，空间 O($\sqrt{m}$)，哈希表先 reserve；基于 unordered_map 的界为期望界。每次调用独立建表；底数 1 及零指数解直接返回。

```
struct DiscreteLog
{
    using ll = long long;

    // 1<=m<=1e12; Expected O(sqrt(m)) time, O(sqrt(m)) storage. Smallest x>=0, or -1.
    static ll solve(ll a, ll b, ll m)
    {
        assert(m >= 1 && m <= 1000000000000LL);
        a = (a % m + m) % m;
        b = (b % m + m) % m;
        if (m == 1 || b == 1) return 0;
        if (a == 1) return -1;
        ll offset = 0, k = 1;
        for (ll g; (g = gcd(a, m)) > 1;)
        {
            if (b == k) return offset;
            if (b % g) return -1;
            b /= g;
            m /= g;
            k = (__int128)k * (a / g) % m;
            ++offset;
        }
        ll target = (__int128)b * mod_inverse(k, m) % m;
        ll step = sqrtl(m) + 1;
        unordered_map<ll, ll> baby;
        baby.reserve(step);
        ll cur = 1 % m;
        for (ll j = 0; j < step; j++)
        {
            if (!baby.count(cur)) baby[cur] = j;
            cur = (__int128)cur * a % m;
        }
        ll inv = mod_inverse(Mod64::power(a, step, m), m);
        cur = target;
        for (ll i = 0; i <= step; i++)
        {
            auto it = baby.find(cur);
            if (it != baby.end()) return offset + i * step + it->second;
            cur = (__int128)cur * inv % m;
        }
        return -1;
    }
};
```

模板题使用：Luogu P4195

题意：多组求 a^x 与 b 模 p 同余的最小非负整数 x，a 与 p 不必互质。输入顺序为 a、p、b，调用顺序为 solve(a,b,p)；返回 -1 时输出 No Solution，三个 0 结束输入。

所需模板：Mod64（第 124 页，18.1 节）、extended_gcd（第 156 页，19.7.1 节）、mod_inverse（第 164 页，19.9.2 节）、DiscreteLog（第 194 页，19.17.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4195

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    long long a, p, b;
    while (std::cin >> a >> p >> b)
    {
        if (a == 0 && p == 0 && b == 0) break;
        long long answer = DiscreteLog::solve(a, b, p);
        if (answer == -1)
            std::cout << "No Solution\n";
        else
            std::cout << answer << '\n';
    }
}
```

19.18 高次剩余 & 单位根

19.18.1 素数模高次剩余：全部根的紧凑表示

代码依赖：DiscreteLog（第 194 页，19.17.1 节）、Mod64（第 124 页，18.1 节）、linear_congruence（第 168 页，19.10.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

solve(a,k,p,g) 求 x^k≡a (mod p)；p 必须是 2 到 1e12 的素数，g 是模 p 原根，k 为正 unsigned long long，a 支持完整 signed long long。无解返回 nullopt；否则得到 first、ratio、count，全部解为 first*ratio^i mod p，0≤i<count，无重复；first 不保证最小。a≡0 时仅有根 0，返回 {0,1,1}。枚举时每次乘 ratio，乘法先转 int128；无需展开全部根即可读取解数。给定原根后，期望 $O(\sqrt{p})$ 时间与空间，依赖 DiscreteLog、linear_congruence 及模快速幂。不验证 p 的素性或 g 的阶；合数模数不在此接口范围内。

```
struct KthResidue
{
    using ll = long long;
    ll first, ratio, count;

    // Prime p <= 1e12; g is a primitive root of p; k > 0.
    // All roots: first * ratio^i mod p, 0 <= i < count.
    static optional<KthResidue> solve(ll a, unsigned long long k, ll p, ll g)
    {
        assert(p >= 2 && p <= 1000000000000LL && g >= 1 && g < p && k > 0);
        a %= p;
        if (a < 0) a += p;
```

```
        if (a == 0) return KthResidue{0, 1, 1};
        ll logarithm = DiscreteLog::solve(g, a, p);
        if (logarithm < 0) return nullopt;
        auto [exponent, period] = linear_congruence(k % (p - 1), -logarithm, p - 1);
        if (exponent < 0) return nullopt;
        ll first = Mod64::power(g, exponent, p);
        ll ratio = Mod64::power(g, period, p);
        return KthResidue{first, ratio, (p - 1) / period};
    }
};
```

联合使用示例见第 202 页（Luogu P5668 【模板】N 次剩余），其中直接调用本模板。

19.18.2 素数幂模高次剩余：含非互素被开方数

代码依赖：DiscreteLog（第 194 页，19.17.1 节）、Mod64（第 124 页，18.1 节）、linear_congruence（第 168 页，19.10.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

solve(a,k,p,e,g) 求 x^k≡a (mod p^e)。p 是素数，e≥1，p^e≤1e12，k 为正 unsigned long long；a 为 signed long long。奇素数须提供模 p^e 的原根 g；p=2 时不使用 g。无解返回 nullopt，否则 size() 返回解数，get(i,j,lift) 返回指定根，范围是 count[0]、count[1]、lifts，全部组合无重复且无遗漏。字段 first、ratio[2]、step 描述乘法根族与加法提升；step*lifts=p^e。原根与素性由调用者保证。期望 $O(\sqrt{p^e})$ 时间和空间；单次 get 为 $O(\log(p^e))$，不强制展开全部根。依赖离散对数、线性同余及模幂。

```
struct PrimePowerRoots
{
    using ll = long long;
    ll first = 0, step = 1, lifts = 1;
    array<ll, 2> ratio{1, 1}, count{1, 1};

    ll size() const { return count[0] * count[1] * lifts; }

    ll get(ll i, ll j, ll lift) const
    {
        assert(i >= 0 && i < count[0] && j >= 0 && j < count[1]);
        assert(lift >= 0 && lift < lifts);
        ll x = (__int128)first * Mod64::power(ratio[0], i, step) % step;
        x = (__int128)x * Mod64::power(ratio[1], j, step) % step;
        return x + lift * step;
    }

    // p prime, 1 <= e, p^e <= 1e12, k > 0.
    // For odd p, g must be a primitive root modulo p^e. For p=2, g is unused.
```

接上页同一结构体（第三页继续同一函数）：

```
    static optional<PrimePowerRoots>
    solve(ll a, unsigned long long k, ll p, int e, ll g = 0)
    {
        assert(p >= 2 && p <= 1000000000000LL && e >= 1 && e <= 39 && k > 0);
        array<ll, 40> power{1};
        for (int i = 1; i <= e; i++)
        {
            assert(power[i - 1] <= 1000000000000LL / p);
            power[i] = power[i - 1] * p;
        }
        ll mod = power[e];
        a %= mod;
        if (a < 0) a += mod;
        PrimePowerRoots answer;
        if (a == 0)
        {
            answer.step = power[(e - 1) / k + 1];
            answer.lifts = mod / answer.step;
            return answer;
        }
        int t = 0;
        while (a % p == 0)
        {
            a /= p;
            ++t;
        }
        if (t % k) return nullopt;
```

接上页同一结构体（第三页继续同一函数）：

```
        ll scale = power[t / k], q = power[e - t];
        answer.first = 1;
        answer.step = scale * q;
        answer.lifts = mod / answer.step;
        auto cyclic = [&](ll base, ll target, ll order, int slot)
        {
            ll logarithm = DiscreteLog::solve(base, target, q);
            if (logarithm < 0) return false;
            auto [x, period] = linear_congruence(k % order, -logarithm, order);
            if (x < 0) return false;
            answer.first = (__int128)answer.first * Mod64::power(base, x, q) % q;
            answer.ratio[slot] = Mod64::power(base, period, q);
            answer.count[slot] = order / period;
            return true;
        };
        if (p != 2)
        {
            assert(g >= 1 && g < mod);
            if (!cyclic(g % q, a, q / p * (p - 1), 0)) return nullopt;
        }
        else if (q >= 4)
        {
            int sign = a % 4 == 3;
            auto [x, period] = linear_congruence(k % 2, -sign, 2);
            if (x < 0) return nullopt;
            answer.first = x ? q - 1 : 1;
            answer.ratio[0] = period == 1 ? q - 1 : 1;
            answer.count[0] = 2 / period;
            if (q >= 8 && !cyclic(5, sign ? q - a : a, q / 4, 1)) return nullopt;
        }
        answer.first *= scale;
        return answer;
    }
};
```

联合使用示例见第 202 页（Luogu P5668 【模板】N 次剩余），其中直接调用本模板。

19.18.3 合数求根准备：分解模数与素数幂原根

代码依赖：Mod64（第 124 页，18.1 节）、PollardRho（第 154 页，19.6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

root_factors(M,rho) 为 1≤M≤1e12 生成完整的 (p,e,g) 列表，可直接传入 CompositeRoots。rho 是可复用的 PollardRho 对象。按素数递增返回；p=2 时 g=0，其余 g 为模 p^e 的最小正原根。先分解 M，再分解各个欧拉函数值，逐个检查原根候选的阶。M=1 返回空列表。分解使用随机重试；原根搜索耗时取决于候选数，不包含 BSGS。

```
vector<tuple<long long, int, long long>> root_factors(long long mod, PollardRho &rho)
{
    assert(mod >= 1 && mod <= 1000000000000LL);
    auto primes = rho.factor(mod);
    vector<tuple<long long, int, long long>> answer;
    for (int i = 0; i < (int)primes.size();)
    {
        long long p = primes[i], power = 1;
        int e = 0;
        while (i < (int)primes.size() && primes[i] == (unsigned long long)p)
        {
            power *= p;
            ++e;
            ++i;
        }
        long long g = 0;
        if (p != 2)
        {
            long long phi = power / p * (p - 1);
            auto factors = rho.factor(phi);
            factors.erase(unique(factors.begin(), factors.end()), factors.end());
            auto primitive = [&](long long candidate)
            {
                if (candidate % p == 0) return false;
                for (auto q : factors)
                    if (Mod64::power(candidate, phi / q, power) == 1) return false;
                return true;
            };
            g = 2;
            while (!primitive(g)) ++g;
        }
        answer.push_back({p, e, g});
    }
    return answer;
}
```

联合使用示例见第 202 页（Luogu P5668 【模板】N 次剩余），其中直接调用本模板。

19.18.4 一般合数模高次剩余：CRT 组合与按索引取根

代码依赖：PollardRho（第 154 页，19.6.1 节）、PrimePowerRoots（第 196 页，19.18.2 节）、mod_inverse（第 164 页，19.9.2 节）、root_factors（第 199 页，19.18.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

solve(a,k,factors) 求 x^k≡a (mod M)，factors 是完整且素数互异的 (p,e,g) 列表，M 为所有 p^e 的乘积，M≤1e12，k 为正 unsigned long long，a 为 signed long long。奇素数的 g 是模 p^e 原根；p=2 时填 0。无解返回 nullopt，否则 mod=M、total 为解数，get(index) 在 0≤index<total 上取出每个根恰好一次，不按数值排序。空因子列表表示模 1，唯一标准代表为 0。依赖 PrimePowerRoots 和模逆元；列表接口不做分解；重载 solve(a,k,M,rho) 自动分解并寻找原根，rho 为 PollardRho。重复同模数宜复用因子列表。预处理的主要成本是各素数幂的 BSGS；get 为 O(F log M)，F 为不同素因子数。无需展开全部根，parts 和 weight 保存局部根族及 CRT 系数。

```
struct CompositeRoots
{
    using ll = long long;
    ll mod = 1, total = 1;
    vector<PrimePowerRoots> parts;
    vector<ll> weight;

    ll get(ll index) const
    {
        assert(index >= 0 && index < total);
        ll answer = 0;
        for (int t = 0; t < (int)parts.size(); t++)
        {
            const auto &p = parts[t];
            ll local = index % p.size();
            index /= p.size();
            ll lift = local % p.lifts;
            local /= p.lifts;
            ll j = local % p.count[1], i = local / p.count[1];
            ll root = p.get(i, j, lift);
            answer = (answer + (__int128)root * weight[t]) % mod;
        }
        return answer;
    }

    // Factors are distinct primes with positive exponents: (p, e, g).
    // Odd-prime g is a primitive root modulo p^e; for p=2 use g=0.
```

局部求根与 CRT 系数（接上页同一结构体）：

```
    static optional<CompositeRoots>
    solve(ll a, unsigned long long k, const vector<tuple<ll, int, ll>> &factors)
    {
        assert(k > 0);
        CompositeRoots answer;
        for (auto [p, e, g] : factors)
        {
            auto roots = PrimePowerRoots::solve(a, k, p, e, g);
            if (!roots) return nullopt;
            ll m = roots->step * roots->lifts;
            assert(gcd(answer.mod, m) == 1);
            assert(answer.mod <= 1000000000000LL / m);
            answer.mod *= m;
            answer.total *= roots->size();
            answer.parts.push_back(*roots);
        }
        for (const auto &p : answer.parts)
        {
            ll m = p.step * p.lifts, rest = answer.mod / m;
            ll inverse = mod_inverse(rest % m, m);
            answer.weight.push_back((__int128)rest * inverse % answer.mod);
        }
        return answer;
    }

    static optional<CompositeRoots>
    solve(ll a, unsigned long long k, ll mod, PollardRho &rho)
    {
        return solve(a, k, root_factors(mod, rho));
    }
};
```

模板题使用：Luogu P5668 【模板】N 次剩余

题意：T≤100，每组按 n,m,k 读入，求 x^n≡k(mod m) 在 0..m-1 的所有解，先输出解数，非零时下一行升序输出全部根。1≤n≤10^9、0≤k<m≤10^9；所有组总解数 ≤10^6，每个素数幂上的根数亦 ≤10^6。root_factors 准备分解和原根：素数走 KthResidue（p=2 用原根 1），素数幂走 PrimePowerRoots，一般合数走 CompositeRoots。m=1 对应空因子列表，唯一根 0；无解只输出 0 一行。各入口原始顺序不保证升序，最后统一排序，不去重来掩盖重复根。枚举会占 O(c) 空间；大根族应仅在题目输出界内展开。

所需模板：Mod64（第 124 页，18.1 节）、Prime64（第 149 页，19.3.1 节）、PollardRho（第 154 页，19.6.1 节）、extended_gcd（第 156 页，19.7.1 节）、mod_inverse（第 164 页，19.9.2 节）、DiscreteLog（第 194 页，19.17.1 节）、linear_congruence（第 168 页，19.10.1 节）、KthResidue（第 195 页，19.18.1 节）、PrimePowerRoots（第 196 页，19.18.2 节）、root_factors（第 199 页，19.18.3 节）、CompositeRoots（第 200 页，19.18.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5668

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    PollardRho rho;
    while (t--)
    {
        unsigned long long n;
        long long m, k;
        cin >> n >> m >> k;
        auto f = root_factors(m, rho);
        vector<long long> a;
        if (f.size() == 1)
        {
            auto [p, e, g] = f[0];
            if (e == 1)
            {
                auto r = KthResidue::solve(k, n, p, p == 2 ? 1 : g);
                if (r)
                {
                    long long x = r->first;
                    for (long long i = 0; i < r->count; i++)
                    {
                        a.push_back(x);
                        x = (__int128)x * r->ratio % p;
                    }
                }
            }
            else
            {
                auto r = PrimePowerRoots::solve(k, n, p, e, g);
                if (r)
                    for (long long i = 0; i < r->count[0]; i++)
                        for (long long j = 0; j < r->count[1]; j++)
                            for (long long z = 0; z < r->lifts; z++)
                                a.push_back(r->get(i, j, z));
            }
        }
```

继续处理操作（接上页同一 main）：

```
        else
        {
            auto r = CompositeRoots::solve(k, n, f);
            if (r)
                for (long long i = 0; i < r->total; i++) a.push_back(r->get(i));
        }
        sort(a.begin(), a.end());
        cout << a.size() << '\n';
        if (!a.empty())
        {
            for (int i = 0; i < (int)a.size(); i++)
                cout << a[i] << (i + 1 == (int)a.size() ? '\n' : ' ');
        }
    }
    return 0;
}
```

19.19 数论分块

19.19.1 整除分块：前缀和来源决定算法

对 $H(t)=\sum_{i=1}^{t}h(i)$、$H(0)=0$，设 $q_A=\lfloor A/l\rfloor, q_B=\lfloor B/l\rfloor$。两个商同时不变的最大右端点为

$$r=\min(\lfloor A/q_A\rfloor, \lfloor B/q_B\rfloor), \quad \text{整块贡献}(H(r)-H(l-1))q_Aq_B.$$

循环只到 $l\le\min(A,B)$，所以进入循环时两个商均非零；零边长直接得到空和。例如 $A=10,B=8,l=3$，只按 B 得到 $r=4$，而 $\lfloor 10/3\rfloor=3,\lfloor 10/4\rfloor=2$，必须取 $r=3$。不应只按一边分块再假定另一边的商相同。

单个 n 的商至多 $O(\sqrt{n})$ 种；双商分块的边界来自两组边界的并集，段数为 $O(\sqrt{A}+\sqrt{B})$，同时不超过 $\min(A,B)$。**不能统一写成** $O(\sqrt{\min(A,B)})$**：**当 $A=m,B=m^2$ 时，B/d 在 $d=1,\dots,m$ 的整数商两两不同，产生 m 段。分块只减少求和段数；若每段都临时线性计算 $H(r)$，仍然可能很慢。

19.19.1.1 按数据与资源选前缀和

上界可预筛，询问多： 第 19.5.1 节（第 151 页）的 `LinearSieve s(L)` 提供 `s.mu`、`s.phi`；再建前缀和为 $O(L)$ 时间、空间，分块中每次查表 $O(1)$。互质矩形直接使用 `CoprimePairs`；GCD 总和则用 φ 的前缀和。筛上限需覆盖所查询的前缀下标，矩形只需短边。μ 的前缀可为负，不能用无符号类型；φ 前缀不能沿用单项的 `int`。

只求少量巨大前缀： 第 19.21.1 节（第 208 页）的 `DuJiao s(B)` 支持 `s.mertens(n)` 与 `s.totient_sum(n)`，分别返回有符号 64 位与 128 位精确整数，当前约定 $0\le n\le 10^{11}$。$1\le B<$ `INT_MAX` 且内存可用；构造参数是预筛上限，不是最大查询值。对单次最大参数 N，预筛加递归的常用期望时间界为 $O(B+N/\sqrt{B})$，选 B 约 $N^{2/3}$ 得 $O(N^{2/3})$；哈希按期望常数计。这不是随便取一个很小 B 就能保证快速。

内存先于理论最优值。当前 `DuJiao` 两张长期前缀表各用 64 位，仅这部分就约 $16(B+1)$ 字节；构造时还同时存在 `LinearSieve` 的三张 `int` 表与素数表，缓存另计（按常见 4 字节 `int`、8 字节 `long long`）。$N=10^{11}$ 时 $N^{2/3}$ 约为 2.15×10^7，两张前缀表已约 345 MB，不能不看内存限制就照此开表。多组不同查询会新增缓存，不能直接承诺单次查询的空间界。重复参数可复用缓存；不要每查一次前缀就重新构造一个筛对象。

优先利用问题本身的化简。例如巨大正方形的互质有序对可以直接取 `2 * s.totient_sum(n) - 1`（$n\ge1$），全程保持 128 位。一般矩形可把 H 换为 `mertens`，但各段前缀查询也会触发递归，不能把整个嵌套过程仅按外层 $O(\sqrt{A}+\sqrt{B})$ 计时；没有额外分析时也不能直接套一个单前缀的 $O(N^{2/3})$ 总界。当前 `CoprimePairs` 不接受杜教筛对象，需自行写相应分块调用。

递推从哪来。对 $n \ge 1$，再用 $\mu * \mathbf{1} = \varepsilon$ 与 $\varphi * \mathbf{1} = \mathrm{id}$ 对 $1, \dots, n$ 求和并交换次序，得到

$$M(n) = 1 - \sum_{i=2}^{n} M(\lfloor n/i \rfloor), \qquad \Phi(n) = \frac{n(n+1)}{2} - \sum_{i=2}^{n} \Phi(\lfloor n/i \rfloor).$$

其中 $M(n) = \sum_{i=1}^{n} \mu(i)$，两式均按等商段乘上段长，第二式没有额外的 i 权重。递归只作用于小于 n 的参数；前缀 0 为 0。这些结论及分块背景参见 OI Wiki：数论分块、OI Wiki：杜教筛。涉及乘法时应先提升类型再乘；模数下遇到负莫比乌斯和要归一化。这里只讲精确计数，若最后除以 2，先完成整数除法再取模，合数模数下不能擅自求 2 的逆元。

19.20 莫比乌斯反演

19.20.1 矩形 GCD 计数

代码依赖：LinearSieve（第 151 页，19.5.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

构造 CoprimePairs(n) 预处理 mu 前缀和，0≤n<INT_MAX，O(n) 时间与空间；依赖 LinearSieve。count(a,b) 求 [1,a]×[1,b] 的互质数对，a,b 为非负 int，要求 min(a,b)≤n，任一为 0 返回 0。rectangle(a,b,c,d,k) 求 [a,b]×[c,d] 中 gcd(x,y)=k 的数对，要求 1≤a≤b、1≤c≤d、k≥1，min(b/k,d/k)≤n；闭区间容斥。返回 long long，中间分块累加用 int128。单次前缀查询 $O(\sqrt{a}+\sqrt{b})$，额外空间 O(1)。sum 为 mu 前缀和，只读。

```
struct CoprimePairs
{
    vector<int> sum;

    CoprimePairs(int n) : sum(LinearSieve(n).mu)
    {
        for (int i = 1; i <= n; i++) sum[i] += sum[i - 1];
    }

    long long count(int a, int b) const
    {
        assert(a >= 0 && b >= 0 && min(a, b) < (int)sum.size());
        __int128_t answer = 0;
        for (int l = 1, r; l <= min(a, b); l = r + 1)
        {
            int x = a / l;
            int y = b / l;
            r = min(a / x, b / y);
            answer += __int128_t(sum[r] - sum[l - 1]) * x * y;
        }
        return (long long)answer;
    }

    long long rectangle(int a, int b, int c, int d, int k) const
    {
        assert(1 <= a && a <= b && 1 <= c && c <= d && k >= 1);
        return count(b / k, d / k) - count((a - 1) / k, d / k) -
               count(b / k, (c - 1) / k) + count((a - 1) / k, (c - 1) / k);
    }
};
```

应用题补充：Luogu P2522 [HAOI2011] Problem b

题意：给出最多 50000 组闭区间 [a,b]、[c,d] 和 k，统计 gcd(x,y)=k 的数对，所有端点及 k 在 1..50000。只初始化一次 CoprimePairs(50000)，每组调用 rectangle(a,b,c,d,k)，返回 long long。它先除以 k，再用四个互质前缀容斥；k 超过区间端点时零前缀自动处理。一般调用 count(A,B) 须预处理到 min(A,B)，本题固定 50000 即可。此为比赛应用题，不计独立模板题覆盖。

所需模板：LinearSieve（第 151 页，19.5.1 节）、CoprimePairs（第 205 页，19.20.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2522

```
int main()
{
    CoprimePairs solver(50000);
    int t;
    scanf("%d", &t);
    while (t--)
    {
        int a, b, c, d, k;
        scanf("%d%d%d%d%d", &a, &b, &c, &d, &k);
        printf("%lld\n", solver.rectangle(a, b, c, d, k));
    }
    return 0;
}
```

19.20.2 整除与反演：先辨认求和方向

以下数论函数定义在**正整数**上；数组下标 0 只是前缀和的占位。狄利克雷卷积、单位元与两条常用恒等式为

$$(f*g)(n)=\sum_{d\mid n} f(d)g(n/d),\qquad \varepsilon(n)=[n=1],\quad \mathbf{1}(n)=1,\quad \mathrm{id}(n)=n.$$

$$\mu*\mathbf{1}=\varepsilon,\qquad \varphi*\mathbf{1}=\mathrm{id}\,.$$

本节按“难以直接统计的量、容易统计的整除条件、换序后的权重”推导，不要看到 GCD 就直接在原式前乘一个 μ。

19.20.2.1 约数和与倍数和不能混用

约数方向。若 $F(n)=\sum_{d\mid n} f(d)$，则 $f(n)=\sum_{d\mid n}\mu(d)F(n/d)$。这不要求 f 积性；用卷积写成 $F=f*\mathbf{1}$，再卷积 μ 就消掉了 $\mathbf{1}$。积性只在后续选用筛法或拆质数幂时才需要。

倍数方向。若 $E(k)$ 仅在 $1\le k\le V$ 非零，且 $G(d)=\sum_{j\ge 1,\,dj\le V} E(dj)$，则

$$E(k)=\sum_{t=1}^{\lfloor V/k\rfloor}\mu(t)G(kt).$$

代入 G 后，每个 $E(ks)$ 的系数是 $\sum_{t\mid s}\mu(t)=[s=1]$。这里求的是 k 的倍数，不能改成 k 的约数；有限支持保证换序合法，无限级数还须另外证明收敛，不能把竞赛里的有限证明直接外推。

例一：数组中互质的不同下标对。正整数数组 $a_1,\dots,a_n\le V$，统计 $i<j$ 且 $\gcd(a_i,a_j)=1$。令 $c_d=\#\{i: d\mid a_i\}$，则 $G(d)=\binom{c_d}{2}$ 统计 GCD 被 d 整除的下标对，答案是 $E(1)=\sum_{d=1}^{V}\mu(d)\binom{c_d}{2}$。例如数组 $(2,3,4,6)$，非零相关项给出 $6-3-1=2$，恰为下标对应的 $(2,3),(3,4)$。重复值仍按下标区分：$(1,1,2)$ 的三个下标对都互质，答案为 3。既然已用 $\binom{c_d}{2}$ 排除了相同下标，就不能再减去一个“自配对数”。

从频率表求所有 c_d，枚举 d 的倍数，总时间 $\sum_{d\le V} O(V/d)=O(V\log V)$、空间 $O(V)$；只求互质对，再筛出 μ 并线性累加即可。若要全部 $E(k)$，也可从大到小做 $E(k)=G(k)-\sum_{j\ge 2,\,kj\le V}E(kj)$，同为 $O(V\log V)$。数值 V 太大时不能因为 n 小就开长度 V 的数组，须改为分解、枚举实际约数等其他方案。

19.20.2.2 GCD 权值先反演，再交换求和

设所求为 $S=\sum_{x=1}^{A}\sum_{y=1}^{B} w(\gcd(x,y))$。先定义 $h=\mu*w$，于是 $w=h*\mathbf{1}$；再将公共约数提到外层：

$$S=\sum_{x=1}^{A}\sum_{y=1}^{B}\sum_{d\mid x,\,d\mid y} h(d)=\sum_{d=1}^{\min(A,B)} h(d)\left\lfloor\frac{A}{d}\right\rfloor\left\lfloor\frac{B}{d}\right\rfloor.$$

这同时涵盖互质计数 $w(t)=[t=1]$、$h=\mu$，以及 GCD 总和 $w(t)=t$、$h=\varphi$。乘出来的是**卷积后的** $h(d)$，一般不是 $\mu(d)w(d)$。

例二：$[1,3]\times[1,4]$ 的 GCD 总和。按三行直接求得 $4+6+6=16$；按上式得到

$$\varphi(1)\cdot 3\cdot 4+\varphi(2)\cdot 1\cdot 2+\varphi(3)\cdot 1\cdot 1=12+2+2=16.$$

若误用 $\mu(d)d$ 作系数，结果变成 $12-4-3=5$。已知 h 的前缀和后可按第 19.19.1 节分块；不知道前缀和如何高效取得时，“反演完了”还不等于算法已经足够快。基本反演恒等式参见 OI Wiki：莫比乌斯反演。

19.20.3 GCD 计数：缩放、边界与对称性

记 $C(A,B)$ 为 $[1,A]\times[1,B]$ 中的**有序**互质数对数，任一边为 0 时为 0。第 19.20.1 节（第 205 页）的 `CoprimePairs c(L)` 预筛至 L，`c.count(A,B)` 就是 $C(A,B)$；要求 A,B 为非负 `int` 且 $\min(A,B)\le L$。它不包括坐标轴，不会自动去掉 $x=y$，也不会替题目把有序对变成无序对。

19.20.3.1 恰好等于 k：先缩放整除对象，再做矩形容斥

当 $k\ge 1$ 且 $x,y>0$，$\gcd(x,y)=k$ 等价于 $x=ku, y=kv$ 且 $\gcd(u,v)=1$。对闭区间 $a\le x\le b$、$c\le y\le d$，其中 $a,c\ge 1$，令

$$A=\lfloor b/k\rfloor,\quad A_0=\lfloor (a-1)/k\rfloor,\qquad B=\lfloor d/k\rfloor,\quad B_0=\lfloor (c-1)/k\rfloor.$$

答案为 $C(A,B)-C(A_0,B)-C(A,B_0)+C(A_0,B_0)$。$A_0$ 统计严格小于 a 的 k 的倍数；应**先减 1 再除**，一般不等于 $\lfloor a/k\rfloor-1$。当 $k>\min(b,d)$ 时答案自动为 0。

例三：$[3,8]\times[5,10]$ 内 GCD 恰为 2。缩放后为 $[2,4]\times[3,5]$，前缀四项为 $C(4,5)-C(1,5)-C(4,2)+C(1,2)=15-5-6+2=6$。原坐标中的六对是 $(4,6),(4,10),(6,8),(6,10),(8,6),(8,10)$。当前接口的完整调用可写为：

```
int a = 3, b = 8, c = 5, d = 10, k = 2;
CoprimePairs solver(min(b / k, d / k));
long long answer = solver.rectangle(a, b, c, d, k); // 6
```

多测可一次预筛至所有查询的 $\max\min(b/k,d/k)$ 后复用。这里的 a,b,c,d,k 是正 `int`，$a\le b, c\le d$；接口本身不是 64 位大端点版本，不能把大整数截断后传入。

19.20.3.2 去掉顺序之前，先检查交换是否仍在定义域内

同一正整数区间 I。设 T 为 $I\times I$ 中 GCD 为 k 的有序对数，对角线上只有 (k,k) 可能合法。令 $\delta=[k\in I]$，则

$$\#\{x<y\}=\frac{T-\delta}{2},\qquad \#\{x\le y\}=\frac{T+\delta}{2}.$$

只有定义域在交换 x,y 后不变，才能将非对角线两两配对。例如 $C(5,5)=19$，互质的 $x<y$ 有 9 对、$x\le y$ 有 10 对。而 $C(2,3)=5$：其中 $(1,3),(2,3)$ 的交换不在矩形内，不能把 5 直接除以 2。若题目要求不同下标，即使两个值相同也可能是合法对象，应回到整除反演建模（第 19.20.2 节，第 206 页）的频率计数。

正方形可进一步化简。对 $n\ge 1$，按较大的坐标 y 分类，$x<y$ 且互质的数量为 $\sum_{y=2}^{n}\varphi(y)=\Phi(n)-1$，故

$$C(n,n)=2\Phi(n)-1,\qquad \Phi(n)=\sum_{i=1}^{n}\varphi(i).$$

例如 $\Phi(5)=10$，立即得到 19、9、10 三种计数。$n=0$ 单独返回 0，不能得到 -1。这个化简还能避免为一个正方形答案逐块查询莫比乌斯前缀和。

含零或负数时另外建模。正整数反演式不能直接包含 $\mu(0)$。按通常 $\gcd(0,y)=|y|$，若非负矩形包含坐标轴，GCD 为 $k>0$ 的轴上候选只有 $(0,k),(k,0)$；分别判断是否落在范围内，再加到正整数部分。$(0,0)$ 的 GCD 通常定义为 0，题面另有约定时从其约定；负数需要处理绝对值与重数。

19.21 狄利克雷双曲线法 & 杜教筛

19.21.1 杜教筛

代码依赖：LinearSieve（第 151 页，19.5.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

构造预筛上限 1≤B<INT_MAX，内存须可用；mertens(n) 返回 μ 的前缀和，totient_sum(n) 返回 φ 的前缀和（int128）。约定 0≤n≤10^{11}，n=0 返回 0。按整除分块递归并写入 mmu/mphi 缓存；重复查询复用结果，清空缓存可重算，不可修改 pmu/pphi/limit。单次最大参数 N 的常用均摊分析为 O(B+N/$\sqrt{B}$)（哈希按期望常数计算），B 约 N^(2/3) 时为 O(N^(2/3))；小 B 不构成大输入性能保证。多次不同 N 可能新增缓存，空间随实际缓存增长；递归深度 O(log N)。

```
struct DuJiao
{
    using ll = long long;
    using I = __int128_t;
    int limit;
    vector<ll> pmu, pphi;
    unordered_map<ll, ll> mmu;
    unordered_map<ll, I> mphi;

    DuJiao(int limit) : limit(limit), pmu(limit + 1), pphi(limit + 1)
    {
        assert(limit >= 1);
        LinearSieve s(limit);
        for (int i = 1; i <= limit; i++)
        {
            pmu[i] = pmu[i - 1] + s.mu[i];
            pphi[i] = pphi[i - 1] + s.phi[i];
        }
    }

    ll mertens(ll n)
    {
        if (n <= limit) return pmu[n];
        if (mmu.count(n)) return mmu[n];
        ll ans = 1;
        for (ll l = 2, r; l <= n; l = r + 1)
        {
            r = n / (n / l);
            ans -= (r - l + 1) * mertens(n / l);
        }
        return mmu[n] = ans;
    }

    I totient_sum(ll n)
    {
        if (n <= limit) return pphi[n];
        if (mphi.count(n)) return mphi[n];
```

```
        I ans = I(n) * (n + 1) / 2;
        for (ll l = 2, r; l <= n; l = r + 1)
        {
            r = n / (n / l);
            ans -= I(r - l + 1) * totient_sum(n / l);
        }
        return mphi[n] = ans;
    }
};
```

模板题使用：Luogu P4213

题意：多次查询 φ 与 μ 的前缀和，先输出 φ，再输出 μ。先读入全部 n，预筛到 min(最大 n,2000000)，同一结构复用缓存。题目 n<2^31，所以 φ 的总和可安全转为 long long；模板更大范围的 int128 返回值不能照搬此转换。

所需模板：LinearSieve（第 151 页，19.5.1 节）、DuJiao（第 208 页，19.21.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4213

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    vector<long long> a(t);
    for (auto &n : a) cin >> n;
    int b = min(2000000LL, *max_element(a.begin(), a.end()));
    DuJiao d(b);
    for (long long n : a)
        cout << (long long)d.totient_sum(n) << ' ' << d.mertens(n) << '\n';
}
```

模板题使用：Library Checker sum_of_totient_function

题意：求 φ 前缀和对 998244353 的余数，n≤10^10。先在 int128 中求和并取模，再转为 long long 输出；不能先把未取模的总和截成 64 位。预筛上限按输入截到 2000000。

所需模板：LinearSieve（第 151 页，19.5.1 节）、DuJiao（第 208 页，19.21.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/sum_of_totient_function

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    long long n;
    cin >> n;
    DuJiao d(min(n, 2000000LL));
    cout << (long long)(d.totient_sum(n) % 998244353) << '\n';
}
```

19.22 素数计数

19.22.1 素数计数（整除分块筛）

prime_count(n) 返回不大于 n 的素数个数，精确 long long，不取模。支持 0≤n≤10^12；n<2 返回 0。仅存不同的 floor(n/i)，由大到小筛去最小质因子为 p 的合数；平方根经整数修正。O(n^0.75 / log n) 时间、O($\sqrt{n}$) 空间，小 n 按常数处理；范围上界不表示所有平台时限保证。

```
long long prime_count(long long n)
{
    assert(0 <= n && n <= 1000000000000LL);
    if (n < 2) return 0;
    int s = sqrtl(n);
    while (1LL * (s + 1) * (s + 1) <= n) s++;
    while (1LL * s * s > n) s--;
    vector<long long> w, g;
    vector<int> small(s + 1), large(s + 1);
    for (long long l = 1, r; l <= n; l = r + 1)
    {
        long long x = n / l;
        r = n / x;
        int id = w.size();
        if (x <= s)
            small[x] = id;
        else
            large[n / x] = id;
        w.push_back(x);
        g.push_back(x - 1);
    }
    auto id = [&](long long x)
    {
        return x <= s ? small[x] : large[n / x];
    };
    for (int p = 2; p <= s; p++)
    {
        long long before = g[id(p - 1)];
        if (g[id(p)] == before) continue;
        for (int j = 0; j < (int)w.size() && 1LL * p * p <= w[j]; j++)
            g[j] -= g[id(w[j] / p)] - before;
    }
    return g[0];
}
```

模板题使用：Library Checker counting_primes

题意：输入 n，输出不大于 n 的素数个数，不取模。题目 n≤10^11。

所需模板：prime_count（第 210 页，19.22.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/counting_primes

```
int main()
{
    long long n;
    cin >> n;
    cout << prime_count(n) << '\n';
}
```

19.23 Min_25 筛

19.23.1 Min_25 积性函数求和

代码依赖：ModInt（第 133 页，19.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

构造 Min25<mod>(n)，1≤n≤10^12，mod 为大于 3 的素数。sum(c, power) 求 ∑(1≤i≤n) f(i)，要求 f 为积性函数、f(1)=1，f(p)=c[0]+c[1]p+c[2]p²；power(p,e,pe) 返回 f(p^e)，其中 pe=p^e、e≥1，回调必须与一次幂系数一致。支持同一对象多次 sum；不改筛表。预处理 O(n^0.75 / log n)，回调 O(1) 时递归求和保守 O(n) 上界；O(√n) 空间、O(log n) 递归栈。w 降序存不同整除商，g[id(x)][d] 是模意义下 ∑(p≤x) p^d；id 仅允许查询这些整除商（包含 1..√n），各字段只读。依赖 ModInt；普通递归版，未承诺全部官方时限达标。

```
template <int mod> struct Min25
{
    using ll = long long;
    using Z = ModInt<mod>;
    ll n;
    int s;
    vector<ll> w;
    vector<int> small, large, primes;
    vector<array<Z, 3>> g;

    Min25(ll n) : n(n)
    {
        static_assert(mod > 3);
        assert(1 <= n && n <= 1000000000000LL);
        s = sqrtl(n);
        while (1LL * (s + 1) * (s + 1) <= n) s++;
        while (1LL * s * s > n) s--;
        small.resize(s + 1);
        large.resize(s + 1);
        Z half = Z(2).inv(), sixth = Z(6).inv();
        for (ll l = 1, r; l <= n; l = r + 1)
        {
            ll x = n / l;
            r = n / x;
            int k = w.size();
            if (x <= s)
                small[x] = k;
            else
                large[n / x] = k;
            w.push_back(x);
            Z v = x;
            g.push_back(
                {v - 1, v * (v + 1) * half - 1, v * (v + 1) * (v * 2 + 1) * sixth - 1});
        }
```

```
        array<Z, 3> prefix{};
        for (int p = 2; p <= s; p++)
        {
            if (g[id(p)][0].v == g[id(p - 1)][0].v) continue;
            primes.push_back(p);
            array<Z, 3> weight{1, p, Z(p) * p};
            for (int j = 0; j < (int)w.size() && 1LL * p * p <= w[j]; j++)
            {
                int k = id(w[j] / p);
                for (int d = 0; d < 3; d++)
                    g[j][d] = g[j][d] - weight[d] * (g[k][d] - prefix[d]);
            }
            for (int d = 0; d < 3; d++) prefix[d] = prefix[d] + weight[d];
        }
    }

    int id(ll x) const { return x <= s ? small[x] : large[n / x]; }

    template <class F> Z sum(array<Z, 3> c, F power) const
    {
        vector<Z> prefix(primes.size() + 1);
        for (int i = 0; i < (int)primes.size(); i++)
        {
            Z p = primes[i];
            prefix[i + 1] = prefix[i] + c[0] + c[1] * p + c[2] * p * p;
        }
        auto dfs = [&](auto &&self, ll x, int k) -> Z
        {
            if (x < 2 || (k && x <= primes[k - 1])) return 0;
            auto a = g[id(x)];
            Z ans = c[0] * a[0] + c[1] * a[1] + c[2] * a[2] - prefix[k];
            for (int i = k; i < (int)primes.size() && primes[i] <= x / primes[i]; i++)
            {
                ll p = primes[i], pe = p;
                for (int e = 1; pe <= x / p; e++, pe *= p)
                    ans = ans + power(p, e, pe) * self(self, x / pe, i + 1) +
                          power(p, e + 1, pe * p);
            }
            return ans;
        };
        return dfs(dfs, n, 0) + 1;
    }
};
```

模板题使用：Luogu P5325

题意：求积性函数前缀和，f(p^e)=p^e(p^e-1)，模 1000000007。一次幂为 p^2-p，故系数传 {0,-1,1}；回调的 pe 已是整数 p^e，无需再快速幂。

所需模板：ModInt（第 133 页，19.2.2 节）、Min25（第 212 页，19.23.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5325

```
int main()
{
    using Z = ModInt<1000000007>;
    long long n;
    cin >> n;
    Min25<1000000007> sieve(n);
    auto power = [](long long p, int e, long long pe)
    {
        Z x = pe;
        return x * (x - 1);
    };
    cout << sieve.sum({0, -1, 1}, power).v << '\n';
}
```

模板题使用：Library Checker sum_of_multiplicative_function

题意：多组输入 n,a,b，求 f(p^e)=ae+bp 的积性函数前缀和，模 469762049。一次幂系数为 {a,b,0}；每组重新构造筛表。官方把普通 Min_25 参考解标为允许超时，本地正确性通过不代表在线时限通过。

所需模板：ModInt（第 133 页，19.2.2 节）、Min25（第 212 页，19.23.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/sum_of_multiplicative_function

```
int main()
{
    using Z = ModInt<469762049>;
    int t;
    cin >> t;
    while (t--)
    {
        long long n;
        int a, b;
        cin >> n >> a >> b;
        Min25<469762049> sieve(n);
        auto power = [&](long long p, int e, long long pe)
        {
            return Z(a) * e + Z(b) * p;
        };
        cout << sieve.sum({a, b, 0}, power).v << '\n';
    }
}
```

19.24 类欧几里德算法

19.24.1 有符号整除和：类欧几里德

floor_sum(n,m,a,b) 返回 i=0..n-1 的 floor((ai+b)/m) 之和，结果为 int128。要求 0≤n≤10^9、1≤m≤10^9，a、b 支持完整 long long；n=0 返回 0。先按数学向下取整规范化负系数，再迭代交换分母和余数；不能把 C++ 负数截零除法当作 floor。O(log m) 时间、O(1) 空间，不依赖完整数论模块。

```
__int128_t floor_sum(long long n, long long m, long long a, long long b)
{
    using ll = long long;
    using i128 = __int128_t;
    assert(n >= 0 && m > 0);
    i128 ans = 0;
    auto norm = [&](ll &v)
    {
        ll q = v / m;
        if (v % m < 0) --q;
        v = (ll)(i128(v) - i128(q) * m);
        return q;
    };
    ans += i128(norm(a)) * n * (n - 1) / 2;
    ans += i128(norm(b)) * n;
    while (true)
    {
        ans += i128(n) * (n - 1) / 2 * (a / m);
        a %= m;
        ans += i128(n) * (b / m);
        b %= m;
        i128 y = i128(a) * n + b;
        if (y < m) break;
        n = (ll)(y / m);
        b = (ll)(y % m);
        swap(a, m);
    }
    return ans;
}
```

模板题使用：Library Checker sum_of_floor_of_linear

题意：求 i=0..n-1 的 floor((a*i+b)/m) 之和。此题有 0≤a,b<m，因此结果可转为 long long 输出；更一般的有符号输入不能照搬该转换。

所需模板：floor_sum（第 215 页，19.24.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/sum_of_floor_of_linear

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        long long n, m, a, b;
        cin >> n >> m >> a >> b;
        // This task has 0 <= a,b < m, so the sum is at most n*(n-1)/2.
        cout << (long long)floor_sum(n, m, a, b) << '\n';
    }
}
```

19.24.2 整除和、带权和与平方和

代码依赖：ModInt（第 133 页，19.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

floor_moments(n,m,a,b) 对 i=0..n-1，令 y_i=floor((a*i+b)/m)，返回 {∑y_i, ∑i*y_i, ∑y_i^2}，元素为 ModInt<998244353>，取整数用.v。0≤n≤1000000001，1≤m≤10^9；a、b 支持完整 long long。n=0 返回全零。时间与递归空间 O(log(m+1))；模数固定，499122177 与 166374059 分别是 2、6 的逆元。P5170 输入上限为闭区间，驱动传 n+1，输出索引顺序 0、2、1。

```
array<ModInt<998244353>, 3>
floor_moments(long long n, long long m, long long a, long long b)
{
    using Z = ModInt<998244353>;
    assert(n >= 0 && n <= 1000000001 && m >= 1 && m <= 1000000000);
    if (!n) return {0, 0, 0};
    auto normalize = [&](long long &v)
    {
        long long q = v / m;
        if (v % m < 0) q--;
        v = (__int128_t(v) - __int128_t(q) * m);
        return Z(q);
    };
    Z x = normalize(a);
    Z y = normalize(b);
    Z count = n;
    Z s1 = count * Z(n - 1) * Z(499122177);
    Z s2 = count * Z(n - 1) * Z(2 * n - 1) * Z(166374059);
    Z f = 0, g = 0, h = 0;
    if (a)
    {
        long long height = (a * (n - 1) + b) / m;
        auto [u, v, w] = floor_moments(height, a, m, m + a - b - 1);
        Z t = height;
        f = count * t - u;
        g = t * s1 - (w - u) * Z(499122177);
        h = count * t * t - Z(2) * v - u;
    }
    return {f + x * s1 + y * count,
            g + x * s2 + y * s1,
            h + x * x * s2 + y * y * count + Z(2) * x * y * s1 + Z(2) * x * g +
                Z(2) * y * f};
}
```

模板题使用：Luogu P5170

题意：求 i=0..n 的整除和、整除值平方和、i 乘整除值之和，模 998244353。t 至 10^5，n/a/b/c 至 10^9 且 c>0。核心参数顺序 (n+1,c,a,b)，返回顺序是和/带权和/平方和；本题输出索引 0、2、1，不能只核对第一项。

所需模板：ModInt（第 133 页，19.2.2 节）、floor_moments（第 217 页，19.24.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5170

```
int main()
{
    int t;
    scanf("%d", &t);
    while (t--)
    {
        long long n, a, b, c;
        scanf("%lld%lld%lld%lld", &n, &a, &b, &c);
        auto result = floor_moments(n + 1, c, a, b);
        printf("%d %d %d\n", result[0].v, result[2].v, result[1].v);
    }
    return 0;
}
```

19.24.3 类欧几里德：有符号整除求和

接口见第 19.24.1 节（第 215 页）。`floor_sum(n,m,a,b)` 返回

$$F(n,m,a,b)=\sum_{i=0}^{n-1}\left\lfloor\frac{ai+b}{m}\right\rfloor.$$

范围约定 $0\le n\le 10^9$、$1\le m\le 10^9$，a,b 支持完整有符号 64 位，返回 128 位。n 是项数；若题目下标为 $0\ldots N$，需传 $N+1$ 并检查仍在约定范围内。

先令 $a=q_am+r_a$、$b=q_bm+r_b$，其中余数属于 $[0,m)$，答案先加上 $q_an(n-1)/2+q_bn$。C++ 的负整数除法向零截断，因此余数为负时需要把商减 1，再用 128 位乘法计算规范余数。这一步不能简单用两次取模替代后丢弃商的贡献。

对规范化后的 $0\le a,b<m$，设 $y=an+b$。若 $y<m$，余下贡献为 0；否则交换格点计数的求和方向可得

$$F(n,m,a,b)=F(\lfloor y/m\rfloor,a,m,y \bmod m).$$

下一轮先提取新的整商，再继续交换。进入交换分支时 $a>0$，不会产生零分母。模数按欧几里得过程减小，时间 $O(\log(m+1))$，额外空间 $O(1)$。

双版本以任意精度向下取整逐项求和对拍，覆盖负系数、零项数和整除边界；十亿项的常数、连续整数、反向连续整数及整倍数斜率使用闭式公式独立核验。同时检查模数 1 下两个系数的 64 位极值。普通与 ASan/UBSan 均通过，暂无在线 AC。本节只验证基本整除和；下节单独给出带下标权与平方和的接口及证据。

19.24.4 类欧几里德的三个求和量

实现见第 19.24.2 节（第 217 页）。令 $y_i = \lfloor (ai+b)/m \rfloor$，维护 $f = \sum y_i$、$g = \sum i y_i$、$h = \sum y_i^2$，下标均为 $0 \le i < n$。范围为 $0 \le n \le 1000000001$、$1 \le m \le 10^9$，a, b 支持完整有符号 64 位。空和 $n = 0$ 直接返回 $(0,0,0)$；下述含 $n-1$ 的高度递推只在 $n \ge 1$ 时使用。结果固定模 998244353，不要更换模数却保留 2、6 的逆元常数。

先把 $a = xm + a'$、$b = ym + b'$ 归一化；设规范化问题返回 (f_0, g_0, h_0)，并记 $S_1 = \sum i$、$S_2 = \sum i^2$，则

$$\begin{aligned} f &= f_0 + xS_1 + yn, \\ g &= g_0 + xS_2 + yS_1, \\ h &= h_0 + x^2 S_2 + y^2 n + 2xyS_1 + 2xg_0 + 2yf_0. \end{aligned}$$

负系数的商仍须向下取整。规范化后的 $a' = 0$ 时三个剩余量均为 0。

对 $0 < a' < m$，令 $t = \lfloor (a'(n-1) + b')/m \rfloor$。高度 $j = 1 \ldots t$ 首次出现的下标为 $r_j = \lceil (mj - b')/a' \rceil$。以 $q = j - 1$ 编号，递归参数为 $(t, a', m, m + a' - b' - 1)$；设返回 (u, v, w)，即 $u = \sum r_j$、$v = \sum (j-1) r_j$、$w = \sum r_j^2$，则

$$f_0 = nt - u, \qquad g_0 = tS_1 - (w-u)/2, \qquad h_0 = nt^2 - 2v - u.$$

这来自按高度计算每一层的贡献，其中 $y_i^2 = \sum_{j=1}^{y_i} (2j-1)$。分母每次严格减小，所以只有 $O(\log(m+1))$ 层递归。

P5170 的上界包含 n，驱动传入 $n+1$；题目输出顺序为 f, h, g，与本接口不同。P5170 完整输入输出见第 19.24.2 节（第 218 页）；本题多组数量至 10^5。核心函数支持十亿加一项，并对完整有符号 64 位系数做任意精度逐项对拍；大项数用一次多项式的和与平方和闭式检查。两套完整程序各验证十万组输入，检查最大闭区间端点与全部三个输出。普通与 ASan/UBSan 均通过，在线 AC 待补；任意更高次矩尚不在这个接口内。

补充的大参数测试包含 3000 组随机参数、64 位系数端点和斐波那契型欧几里得链。反转 $i \mapsto n-1-i$ 应保持 f, h 不变，并使 g 变为 $(n-1)f - g$。将每项替换为 $-y_i - 1$，三个结果应变为 $-f - n$、$-g - S_1$、$h + 2f + n$。相邻前缀还应满足增量 $(y_n, n y_n, y_n^2)$，其中新增项以任意精度计算。仅在两个接口的共同范围 $n \le 10^9$ 内，第一结果另与独立接口 `floor_sum` 交叉核对。这些恒等式测试是大参数一致性证据，与前述逐项独立答案对拍区分记录；普通与 ASan/UBSan 均通过。

来源与题目：https://www.luogu.com.cn/problem/P5170。

第二十章　多项式与生成函数

20.1　快速傅里叶变换

20.1.1　复数 FFT（可复用单位根）

ComplexFFT(n) 预计算单位根，n 为 1..2^22 的二次幂。transform(a,inverse=false) 原地正变换，正向指数为正；inverse=true 用共轭根并除以 n。a 长度须等于 n；C=complex<long double>。根直接用角度计算，不沿整层连乘。单次变换 O(n log(n+1))，根表 O(n)，可复用。浮点环境须二进制至少 53 位、最近舍入、禁用 fast-math；整数卷积还要求单位根绝对误差 ≤8×2^-53，本机全根已用百位参考核验。变换本身是近似值。

```
struct ComplexFFT
{
    using Real = long double;
    using C = complex<Real>;
    static constexpr int max_size = 1 << 22;
    int n;
    vector<C> roots;

    explicit ComplexFFT(int size) : n(size)
    {
        static_assert(numeric_limits<Real>::radix == 2);
        static_assert(numeric_limits<Real>::digits >= 53);
        assert(n > 0 && n <= max_size && (n & (n - 1)) == 0);
        roots.resize(n / 2);
        Real pi = acosl(-1);
        for (int i = 0; i < n / 2; i++)
        {
            Real angle = pi * (2.0L * i / n);
            roots[i] = C(cosl(angle), sinl(angle));
        }
    }
```

正逆变换（接上页同一结构体）：

```
    void transform(vector<C> &a, bool inverse = false) const
    {
        assert(a.size() == (unsigned)n);
        for (int i = 1, j = 0; i < n; i++)
        {
            int bit = n >> 1;
            for (; j & bit; bit >>= 1) j ^= bit;
            j ^= bit;
            if (i < j) swap(a[i], a[j]);
        }
        for (int half = 1; half < n; half *= 2)
        {
            int step = n / (2 * half);
            for (int i = 0; i < n; i += 2 * half)
            {
                for (int j = 0; j < half; j++)
                {
                    C w = roots[j * step];
                    if (inverse) w = conj(w);
                    C u = a[i + j], v = a[i + j + half] * w;
                    a[i + j] = u + v;
                    a[i + j + half] = u - v;
                }
            }
        }
        if (inverse)
            for (auto &x : a) x /= n;
    }
};
```

模板题使用：Luogu P3803 【模板】多项式乘法（FFT）—直接变换接口

题意：给出次数 0≤n,m≤1000000、系数均为 0..9 整数的两个多项式，按升幂读入并输出乘积的 n+m+1 项。补零到不小于 n+m+1 的最小二次幂 L（至多 2^21），显式构造 ComplexFFT，分别正变换、逐点相乘、逆变换后 llroundl 取整。保留本节的二进制至少 53 位、最近舍入、单位根误差界及禁用 fast-math 前提；本题范数乘积至多 81000081。时间 O(L log L)，空间 O(L)。这是底层变换接口的完整用法，卷积包装器和精确 CRT 用法另列。

所需模板：ComplexFFT（第 221 页，20.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3803

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    int size = 1;
    while (size < n + m + 1) size *= 2;
    auto fft = ComplexFFT(size);
    vector<ComplexFFT::C> a(size), b(size);
    for (int i = 0, x; i <= n; i++) cin >> x, a[i] = x;
    for (int i = 0, x; i <= m; i++) cin >> x, b[i] = x;
    fft.transform(a);
    fft.transform(b);
    for (int i = 0; i < size; i++) a[i] *= b[i];
    fft.transform(a, true);
    for (int i = 0; i <= n + m; i++) cout << llroundl(a[i].real()) << ' ';
    cout << '\n';
}
```

应用题补充：HDU4609 随机三根组成三角形的概率

题意：T≤100，3≤n≤100000，长度 1..100000。频数平方卷积去掉自选再除 2；正长度使坏三元组有唯一最长根，减去所有“两根和 ≤ 第三根”的计数。范数平方 $\le n^2$，2^18 点下误差界小于 0.001。

所需模板：ComplexFFT（第 221 页，20.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://acm.hdu.edu.cn/showproblem.php?pid=4609

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        vector<int> a(n);
        for (int &v : a) cin >> v;
        int m = *max_element(a.begin(), a.end()), size = 1;
        while (size <= 2 * m) size *= 2;
        ComplexFFT fft(size);
        vector<ComplexFFT::C> f(size);
        for (int v : a) f[v] += 1;
        fft.transform(f);
        for (auto &v : f) v *= v;
        fft.transform(f, true);
        vector<long long> count(size);
        for (int i = 0; i < size; i++) count[i] = llroundl(f[i].real());
        for (int v : a) count[2 * v]--;
        for (auto &v : count) v /= 2;
        for (int i = 1; i < size; i++) count[i] += count[i - 1];
        long long all = 1LL * n * (n - 1) * (n - 2) / 6;
        long long good = all;
        for (int v : a) good -= count[v];
        cout << fixed << setprecision(7) << (long double)good / all << '\n';
    }
}
```

20.1.2 浮点 FFT 整数卷积

代码依赖：ComplexFFT（第 221 页，20.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

convolution_fft(a,b) 接受 vector<int>，返回 vector<long long>，空输入返回空；非空结果长度 ≤2^22。令 A、B 为两输入最大绝对值，要求 A · B · sqrt(a.size()*b.size())≤2×10^12。依赖 ComplexFFT 及其浮点前提，最坏误差界小于 0.166，llroundl 对正负数正确舍入。超过此界须改用精确整数卷积，不能只看结果是否装入 long long。O(N log N) 时间、O(N) 空间；覆盖 P3803 百万次数/系数 0..9。

```
// IEEE nearest rounding, root error <= 8 * 2^-53; no fast-math.
vector<long long> convolution_fft(const vector<int> &a, const vector<int> &b)
{
    if (a.empty() || b.empty()) return {};
    assert(a.size() + b.size() - 1 <= ComplexFFT::max_size);
    long double ma = 0, mb = 0;
    for (int v : a) ma = max(ma, fabsl((long double)v));
    for (int v : b) mb = max(mb, fabsl((long double)v));
    assert(ma * mb * sqrtl((long double)a.size() * b.size()) <= 2e12L);
    int size = a.size() + b.size() - 1, n = 1;
    while (n < size) n *= 2;
    ComplexFFT fft(n);
    vector<ComplexFFT::C> x(n), y(n);
    for (int i = 0; i < (int)a.size(); i++) x[i] = a[i];
    for (int i = 0; i < (int)b.size(); i++) y[i] = b[i];
    fft.transform(x);
    fft.transform(y);
    for (int i = 0; i < n; i++) x[i] *= y[i];
    fft.transform(x, true);
    vector<long long> result(size);
    for (int i = 0; i < size; i++) result[i] = llroundl(x[i].real());
    return result;
}
```

模板题使用：Luogu P3803

题意：输入两个多项式的次数与升幂系数，输出整数乘积。次数最多一百万，系数 0..9，满足本浮点入口的范数限制；输出用 long long。此处实际调用复数 FFT，原精确整数卷积入口另行保留。

所需模板：ComplexFFT（第 221 页，20.1.1 节）、convolution_fft（第 225 页，20.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3803

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<int> a(n + 1), b(m + 1);
    for (int &x : a) cin >> x;
    for (int &x : b) cin >> x;
    auto c = convolution_fft(a, b);
    for (long long x : c) cout << x << ' ';
    cout << '\n';
}
```

应用题补充：HDU1402 十进制高精度乘法

题意：EOF 读两个至多 50000 位整数，支持可选正负号和前导零。逐位卷积后进位，零不带负号。范数乘积 ≤4050000；同 FFT 数值前提下舍入误差小于 0.000001。

所需模板：ComplexFFT（第 221 页，20.1.1 节）、convolution_fft（第 225 页，20.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://acm.hdu.edu.cn/showproblem.php?pid=1402

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s, t;
    while (cin >> s >> t)
    {
        bool negative = (s[0] == '-') != (t[0] == '-');
        int x = s[0] == '+' || s[0] == '-';
        int y = t[0] == '+' || t[0] == '-';
        vector<int> a(s.size() - x), b(t.size() - y);
        for (int i = x; i < int(s.size()); i++) a[s.size() - 1 - i] = s[i] - '0';
        for (int i = y; i < int(t.size()); i++) b[t.size() - 1 - i] = t[i] - '0';
        auto c = convolution_fft(a, b);
        c.push_back(0);
        for (int i = 0; i + 1 < int(c.size()); i++)
        {
            c[i + 1] += c[i] / 10;
            c[i] %= 10;
        }
        while (c.size() > 1 && c.back() == 0) c.pop_back();
        if (negative && c.back() != 0) cout << '-';
        for (int i = int(c.size()) - 1; i >= 0; i--) cout << c[i];
        cout << '\n';
    }
}
```

20.1.3 拆系数 FFT 任意模卷积

代码依赖：ComplexFFT（第 221 页，20.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

convolution_mod_fft(a,b,mod) 接受 vector<int>，返回 [0,mod) 系数；1≤mod≤INT_MAX，允许合数与负输入，mod=1 返回零。空输入返回空，非空结果长度 ≤2^19。依赖 ComplexFFT 及其浮点前提；归一化后拆成三个 11 位块，六次正变换和九次逆变换，分别舍入后模合并。每路误差界小于 0.079；不依赖更宽的 long double。时间 O(N log N)，空间 O(N)，常数高于打包版本以保持数值界与代码清楚；覆盖 P4245。

```
// Three 11-bit limbs; same floating-point assumptions as convolution_fft.
vector<int> convolution_mod_fft(const vector<int> &a, const vector<int> &b, int mod)
{
    assert(mod >= 1);
    if (a.empty() || b.empty()) return {};
    assert(a.size() + b.size() - 1 <= (1 << 19));
    int size = a.size() + b.size() - 1, n = 1;
    while (n < size) n *= 2;
    vector<int> result(size);
    if (mod == 1) return result;
    ComplexFFT fft(n);
    using C = ComplexFFT::C;
    const int base = 1 << 11;
    vector<vector<C>> x(3, vector<C>(n)), y = x;
    for (int i = 0; i < (int)a.size(); i++)
    {
        unsigned v = (a[i] % mod + (long long)mod) % mod;
        for (int j = 0; j < 3; j++) x[j][i] = (v >> (11 * j)) & (base - 1);
    }
    for (int i = 0; i < (int)b.size(); i++)
    {
        unsigned v = (b[i] % mod + (long long)mod) % mod;
        for (int j = 0; j < 3; j++) y[j][i] = (v >> (11 * j)) & (base - 1);
    }
    for (int j = 0; j < 3; j++)
    {
        fft.transform(x[j]);
        fft.transform(y[j]);
    }
    long long weight[5] = {1};
    for (int i = 1; i < 5; i++) weight[i] = weight[i - 1] * base % mod;
    vector<C> z(n);
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            for (int k = 0; k < n; k++) z[k] = x[i][k] * y[j][k];
            fft.transform(z, true);
            for (int k = 0; k < size; k++)
            {
                long long v = llroundl(z[k].real()) % mod;
                result[k] = (result[k] + v * weight[i + j]) % mod;
            }
        }
    }
    return result;
}
```

模板题使用：Luogu P4245

题意：输入两个多项式的次数、任意模数与升幂系数，输出模乘积。题目次数不超过十万、模数最多 1000000009；函数会先归一化再拆系数，不要求模数支持 NTT。这里使用三块浮点 FFT，并遵守其单位根与舍入前提。

所需模板：ComplexFFT（第 221 页，20.1.1 节）、convolution_mod_fft（第 227 页，20.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4245

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, mod;
    cin >> n >> m >> mod;
    vector<int> a(n + 1), b(m + 1);
    for (int &x : a) cin >> x;
    for (int &x : b) cin >> x;
    auto c = convolution_mod_fft(a, b, mod);
    for (int x : c) cout << x << ' ';
    cout << '\n';
}
```

相关应用：CDQ 分治配合 NTT 的卷积递推，附件标题为分治 FFT；算法不做浮点变换。

20.1.4 CDQ 分治卷积递推

代码依赖：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

cdq_convolution<mod=998244353,primitive=3>(kernel,initial) 求 f[i]=initial[i]+∑(j<i)f[j]kernel[i-j]，返回前 n 项。两数组为等长 vector<ModInt<mod»，kernel[0] 必须为 0；空输入返回空。模数须为 int 范围素数，primitive 是原根；依赖 NttConvolution。非空时要求 n+floor(n/2)-1≤NTT::max_size，保证每层完整卷积可变换。初始项可任意，并非只有 f[0]=1；左半区解定后，将其到右半区的卷积贡献加上，再递归右半区。O(n $\log^2$(n+1)) 时间、O(n) 额外空间，递归深度 O(log n)。本函数是静态线性递推，不提供流式追加或非线性叶子回调。P4721 使用 initial[0]=1、其他项 0。

```
// f[i] = initial[i] + sum_{j<i} f[j] * kernel[i-j]; kernel[0] must be zero.
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>> cdq_convolution(const vector<ModInt<mod>> &kernel,
                                    vector<ModInt<mod>> initial)
{
    using Ntt = NttConvolution<mod, primitive>;
    using Poly = typename Ntt::Poly;
    assert(kernel.size() == initial.size());
    if (initial.empty()) return {};
    assert(kernel[0].v == 0);
    assert(initial.size() + initial.size() / 2 - 1 <= Ntt::max_size);
    int n = initial.size();
    auto solve = [&](auto &&self, int l, int r) -> void
    {
        if (r - l == 1) return;
        int m = (l + r) / 2;
        self(self, l, m);
        Poly left(initial.begin() + l, initial.begin() + m);
        Poly right(kernel.begin(), kernel.begin() + r - l);
        Poly product = Ntt::multiply(move(left), move(right));
        for (int i = m; i < r; i++) initial[i] += product[i - l];
        self(self, m, r);
    };
    solve(solve, 0, n);
    return initial;
}
```

模板题使用：Luogu P4721 分治 FFT

题意：输入 g[1..n-1]，计算 f[0]=1、f[i]=∑(j=1..i)f[i-j]g[j]，模 998244353。kernel[0] 留零，initial 仅第零项为 1；调用的是真正 CDQ 分治卷积，依赖 NTT 与模整数。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、cdq_convolution（第 229 页，20.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4721

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    using Z = ModInt<998244353>;
    vector<Z> g(n), b(n);
    for (int i = 1; i < n; i++)
    {
        int x;
        cin >> x;
        g[i] = x;
    }
    b[0] = 1;
    auto f = cdq_convolution(g, b);
    for (int i = 0; i < n; i++) cout << f[i].v << " \n"[i + 1 == n];
}
```

20.2 快速数论变换

20.2.1 可指定素数与原根的 NTT 卷积

代码依赖：ModInt（第 133 页，19.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

声明 NttConvolution<mod, primitive>，primitive 默认 3。mod 必须为 int 范围内的素数，primitive 是模 mod 原根；不在接口内验证素性或阶。max_size 是 mod-1 的最大二次幂因子。ntt(a,invert) 原地变换，长度须为正二次幂且不超过 max_size；invert=true 为逆变换。multiply(a,b) 返回普通卷积，任一为空则为空；非空时长度和减一不超过 max_size。元素类型 Z，数组类型 Poly。O(n log n) 时间、O(n) 空间。998244353 和 167772161 均可用原根 3。

```
template <int mod, int primitive = 3> struct NttConvolution
{
    static_assert(mod >= 2 && primitive >= 1 && primitive < mod);
    static constexpr int max_size = (mod - 1) & -(mod - 1);
    using Z = ModInt<mod>;
    using Poly = vector<Z>;

    // mod is prime, primitive is a primitive root modulo mod.
    static void ntt(Poly &a, bool invert = false)
    {
        int n = a.size();
        assert(n > 0 && (n & (n - 1)) == 0 && n <= max_size);
        for (int i = 1, j = 0; i < n; i++)
        {
            int bit = n >> 1;
            for (; j & bit; bit >>= 1) j ^= bit;
            j ^= bit;
            if (i < j) swap(a[i], a[j]);
        }
```

蝶形变换与卷积（接上页同一结构体与函数）：

```
        for (int half = 1; half < n; half *= 2)
        {
            int len = half * 2;
            Z step = Z(primitive).pow((mod - 1) / len);
            if (invert) step = step.inv();
            for (int i = 0; i < n; i += len)
            {
                Z w = 1;
                for (int j = 0; j < half; j++)
                {
                    Z u = a[i + j], v = a[i + j + half] * w;
                    a[i + j] = u + v;
                    a[i + j + half] = u - v;
                    w = w * step;
                }
            }
        }
        if (invert)
        {
            Z inverse = Z(n).inv();
            for (auto &x : a) x = x * inverse;
        }
    }

    static Poly multiply(Poly a, Poly b)
    {
        if (a.empty() || b.empty()) return {};
        assert(a.size() + b.size() - 1 <= max_size);
        int size = a.size() + b.size() - 1, n = 1;
        while (n < size) n *= 2;
        a.resize(n);
        b.resize(n);
        ntt(a);
        ntt(b);
        for (int i = 0; i < n; i++) a[i] = a[i] * b[i];
        ntt(a, true);
        a.resize(size);
        return a;
    }
};
```

模板题使用：Library Checker convolution_mod

题意：求两个多项式的卷积，模 998244353。输入为各自系数个数和升幂系数；输出 n+m-1 个系数。题目保证输入系数已经取模。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/convolution_mod

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using C = NttConvolution<998244353>;
    int n, m;
    cin >> n >> m;
    C::Poly a(n), b(m);
    for (auto &x : a) cin >> x.v;
    for (auto &x : b) cin >> x.v;
    auto c = C::multiply(move(a), move(b));
    for (int i = 0; i < (int)c.size(); i++)
        cout << c[i].v << (i + 1 == (int)c.size() ? '\n' : ' ');
}
```

20.2.2 精确有符号整数卷积

代码依赖：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

convolution_i64(a,b) 返回 signed64 整数卷积，空输入返回空。非空结果长度 ≤2^24，所有真实系数必须在 signed64 范围内。依赖 NttConvolution，三个素数模卷积后用 int128 CRT 重构中心代表。断言不能检测任意越界结果的模数别名。O(L log L) 时间、O(L) 空间，L 为补齐的二次幂长度。负数与极值已通过 cpp_int 参考验证；P3803 最大次数驱动本地通过，线上性能待验证。

```
template <int p>
vector<int> convolution_residue(const vector<long long> &a, const vector<long long> &b)
{
    using N = NttConvolution<p>;
    typename N::Poly x(a.begin(), a.end()), y(b.begin(), b.end());
    auto c = N::multiply(move(x), move(y));
    vector<int> r(c.size());
    for (int i = 0; i < (int)c.size(); i++) r[i] = c[i].v;
    return r;
}

vector<long long> convolution_i64(const vector<long long> &a,
                                  const vector<long long> &b)
{
    if (a.empty() || b.empty()) return {};
    assert(a.size() <= (1 << 24) && b.size() <= (1 << 24));
    assert(a.size() + b.size() - 1 <= (1 << 24));
    const long long p = 167772161, q = 469762049, r = 1224736769;
    const long long pq = p * q;
    const __int128 mod = (__int128)pq * r;
    static const long long ip = ModInt<q>(p).inv().v;
    static const long long ipq = ModInt<r>(pq).inv().v;
    auto x = convolution_residue<p>(a, b);
    auto y = convolution_residue<q>(a, b);
    auto z = convolution_residue<r>(a, b);
    vector<long long> c(x.size());
    for (int i = 0; i < (int)c.size(); i++)
    {
        long long t = (y[i] - (long long)x[i] + q) % q * ip % q;
        long long v = x[i] + p * t;
        long long u = (z[i] - v % r + r) % r * ipq % r;
        __int128 value = v + (__int128)pq * u;
        if (value > mod / 2) value -= mod;
        assert(LLONG_MIN <= value && value <= LLONG_MAX);
        c[i] = (long long)value;
    }
    return c;
}
```

模板题使用：Luogu P3803

题意：求两个整数多项式的乘积，系数按升幂输入。首行 n、m 是次数，系数个数分别为 n+1、m+1；输出精确整数，不能取模。本题系数在 0..9，满足 signed64 结果范围。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、convolution_i64（第 234 页，20.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3803

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> a(n + 1), b(m + 1);
    for (auto &x : a) cin >> x;
    for (auto &x : b) cin >> x;
    auto c = convolution_i64(a, b);
    for (int i = 0; i < (int)c.size(); i++)
        cout << c[i] << (i + 1 == (int)c.size() ? '\n' : ' ');
}
```

20.2.3 三模 NTT：精确任意模数卷积

代码依赖：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

convolution_mod(a,b,mod) 接受 vector<int>，1≤mod≤INT_MAX，支持负输入、合数和模数 1。返回 [0,mod) 普通卷积；任一输入为空则返回空。非空结果长度 ≤2^24；先将输入归一化，再用三个固定 NTT 素数重构非负整数系数，int128 中间值后取目标模数，无浮点误差及 signed64 真实系数限制。直接取非负 CRT 代表，不经 signed64 窄化。O(L log L) 时间、O(L) 空间，L 为补齐二次幂；长度上限不是内存或时限承诺。参数按值传递，普通调用不改原数组，愿意移交数据时可 move。依赖 NttConvolution 及 ModInt，convolution_prime 为内部三模辅助。

```
template <int p>
vector<int> convolution_prime(const vector<int> &a, const vector<int> &b)
{
    using N = NttConvolution<p>;
    typename N::Poly x(a.begin(), a.end()), y(b.begin(), b.end());
    auto c = N::multiply(move(x), move(y));
    vector<int> result(c.size());
    for (int i = 0; i < (int)c.size(); i++)
        result[i] = c[i].v;
    return result;
}

vector<int> convolution_mod(vector<int> a, vector<int> b, int mod)
{
    assert(mod >= 1);
    if (a.empty() || b.empty())
        return {};
    assert(a.size() <= (1 << 24) && b.size() <= (1 << 24));
    assert(a.size() + b.size() - 1 <= (1 << 24));
    int n = a.size() + b.size() - 1;
    if (mod == 1)
        return vector<int>(n);
    for (int &v : a)
    {
        v %= mod;
        if (v < 0)
            v += mod;
    }
    for (int &v : b)
    {
        v %= mod;
        if (v < 0)
            v += mod;
    }
```

三模卷积与固定逆元 CRT 重构（接上页同一函数）：

```
    const long long p = 167772161, q = 469762049, r = 1224736769;
    const long long pq = p * q;
    static const long long ip = ModInt<q>(p).inv().v;
    static const long long ipq = ModInt<r>(pq).inv().v;
    auto x = convolution_prime<p>(a, b);
    auto y = convolution_prime<q>(a, b);
    auto z = convolution_prime<r>(a, b);
    for (int i = 0; i < n; i++)
    {
        long long t = (y[i] - (long long)x[i] + q) % q * ip % q;
        long long v = x[i] + p * t;
        long long u = (z[i] - v % r + r) % r * ipq % r;
        __int128 value = v + (__int128)pq * u;
        x[i] = value % mod;
    }
    return x;
}
```

模板题使用：Luogu P4245 任意模数多项式乘法（三模 NTT）

题意：输入两多项式次数 n,m≤100000 和模数 2≤p≤1000000009，系数 0..10^9，输出 n+m+1 个模 p 系数。独立精确三模版本，不继承浮点版的提交记录。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、convolution_mod（第 236 页，20.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4245

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, mod;
    cin >> n >> m >> mod;
    vector<int> a(n + 1), b(m + 1);
    for (int &x : a)
        cin >> x;
    for (int &x : b)
        cin >> x;
    auto c = convolution_mod(a, b, mod);
    for (int x : c)
        cout << x << ' ';
    cout << '\n';
    return 0;
}
```

接口用法演示：精确模卷积：空数组、负数和合数模

演示约定：多组直到 EOF，输入两个数组长度 n,m 和 1≤mod≤INT_MAX，再输入 signed int 系数。每组输出标准模数代表；任一数组为空输出空行，mod=1 输出对应数量的 0。非空结果长度 ≤2^24，实际还须满足内存限制。API 扩展不属于 P4245 正式输入范围。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、convolution_mod（第 236 页，20.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://www.luogu.com.cn/problem/P4245

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, mod;
    while (cin >> n >> m >> mod)
    {
        vector<int> a(n), b(m);
        for (int &x : a)
            cin >> x;
        for (int &x : b)
            cin >> x;
        auto c = convolution_mod(a, b, mod);
        for (int x : c)
            cout << x << ' ';
        cout << '\n';
    }
    return 0;
}
```

应用题补充：HDU4656 Evaluation：移位等比点求值

题意：官方仅一组，n 为 1..100000；b、c、d 为 1..1000000，n 个升幂系数为 0..1000000。按 k=0..n−1 输出 F(b * c^(2k) + d)，结果模 1000003。两次精确卷积，O(n log n) 时间、O(n) 空间；3 秒、65536K，当前最小抄写版已 AC。代码兼容 EOF 多组、signed int 参数及 c=0，扩展边界只由本地测试验证。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、convolution_mod（第 236 页，20.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://acm.hdu.edu.cn/showproblem.php?pid=4656

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    const int mod = 1000003;
    using Z = ModInt<mod>;
    int n, b, c, d;
    while (cin >> n >> b >> c >> d)
    {
        assert(1 <= n && n <= 100010);
        vector<Z> a(n);
        for (Z &v : a)
        {
            int x;
            cin >> x;
            v = x;
        }
        Z base = b, ratio = c, shift = d;
        if (ratio.v == 0)
        {
            Z first = 0, rest = 0;
            for (int i = n - 1; i >= 0; i--)
            {
                first = first * (base + shift) + a[i];
                rest = rest * shift + a[i];
            }
            cout << first.v << '\n';
            for (int i = 1; i < n; i++)
                cout << rest.v << '\n';
            continue;
        }
```

继续处理操作（接上页同一 main）：

```
        vector<Z> fact(n, 1), inv(n, 1);
        for (int i = 1; i < n; i++)
            fact[i] = fact[i - 1] * Z(i);
        inv[n - 1] = fact[n - 1].inv();
        for (int i = n - 1; i > 0; i--)
            inv[i - 1] = inv[i] * Z(i);
        vector<int> x(n), y(n);
        Z power = 1;
        for (int i = 0; i < n; i++)
        {
            x[n - 1 - i] = (a[i] * fact[i]).v;
            y[i] = (power * inv[i]).v;
            power = power * shift;
        }
        auto product = convolution_mod(x, y, mod);
        power = 1;
        for (int j = 0; j < n; j++)
        {
            a[j] = Z(product[n - 1 - j]) * inv[j] * power;
            power = power * base;
        }
        vector<Z> square(n, 1), inverse(n, 1);
        Z step = ratio, backward = ratio.inv();
        Z step2 = step * step, backward2 = backward * backward;
        for (int i = 1; i < n; i++)
        {
            square[i] = square[i - 1] * step;
            inverse[i] = inverse[i - 1] * backward;
            step = step * step2;
            backward = backward * backward2;
        }
        y.resize(2 * n - 1);
        for (int j = 0; j < n; j++)
        {
            x[j] = (a[j] * square[j]).v;
            y[n - 1 - j] = inverse[j].v;
            y[n - 1 + j] = inverse[j].v;
        }
        product = convolution_mod(x, y, mod);
        for (int k = 0; k < n; k++)
            cout << (square[k] * Z(product[n - 1 + k])).v << '\n';
    }
    return 0;
}
```

20.3 快速沃尔什变换

20.3.1 按位 OR、AND、XOR 卷积

代码依赖：ModInt（第 133 页，19.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

SetConvolution<mod=998244353>；Z 为 ModInt<mod>，Poly 为 vector<Z>。数组长度 N 必须是正的 2 次幂且 ≤INT_MAX，表示固定二进制位数的全集，长度 1 合法。transform(a,op,invert=false) 原地变换，op 只能为 |、&、^；| 为子集和，& 为超集和，^ 为 Walsh 变换。multiply(a,b,op) 返回 $\sum$ a[i]b[j]（满足 i op j=s），a、b 长度必须相同。mod 可为 ≥2 的 int 范围合数；XOR 要求 mod 为奇数，逆 2 用 (mod+1)/2，不使用费马逆元。O(N log N) 时间，原地变换 O(1) 额外空间，卷积 O(N) 空间。

```
template <int mod = 998244353> struct SetConvolution
{
    static_assert(mod >= 2);
    using Z = ModInt<mod>;
    using Poly = vector<Z>;

    static void transform(Poly &a, char op, bool invert = false)
    {
        assert(!a.empty() && a.size() <= INT_MAX);
        int n = a.size();
        assert((n & (n - 1)) == 0);
        assert(op == '|' || op == '&' || op == '^');
        assert(op != '^' || mod % 2 == 1);
        Z half = (mod + 1LL) / 2;
        for (int len = 1; len < n; len *= 2)
            for (int i = 0; i < n; i += len * 2)
                for (int j = 0; j < len; j++)
                {
                    Z &u = a[i + j], &v = a[i + j + len];
                    if (op == '|')
                        v = invert ? v - u : v + u;
                    else if (op == '&')
                        u = invert ? u - v : u + v;
                    else
                    {
                        Z x = u, y = v;
                        u = x + y;
                        v = x - y;
                        if (invert)
                        {
                            u = u * half;
                            v = v * half;
                        }
                    }
                }
    }
```

点乘并逆变换（接上页同一结构体）：

```
    static Poly multiply(Poly a, Poly b, char op)
    {
        assert(a.size() == b.size());
        transform(a, op);
        transform(b, op);
        for (int i = 0; i < int(a.size()); i++) a[i] = a[i] * b[i];
        transform(a, op, true);
        return a;
    }
};
```

模板题使用：Luogu P4717

题意：给两个长度为 2^m 的数组，分别求按下标 OR、AND、XOR 合并的卷积，模 998244353，按此顺序输出三行。multiply 的最后参数选择运算。

所需模板：ModInt（第 133 页，19.2.2 节）、SetConvolution（第 242 页，20.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4717

```
int main()
{
    int m;
    scanf("%d", &m);
    int n = 1 << m;
    using F = SetConvolution<998244353>;
    F::Poly a(n), b(n);
    for (auto &x : a)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    for (auto &x : b)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    for (char op : string("|&^"))
    {
        auto c = F::multiply(a, b, op);
        for (int s = 0; s < n; s++) printf("%d%c", c[s].v, s + 1 == n ? '\n' : ' ');
    }
}
```

20.3.2 不相交子集卷积

代码依赖：ModInt（第 133 页，19.2.2 节）、SetConvolution（第 242 页，20.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

subset_convolution<mod=998244353>(a,b) 返回 c[s]=$\sum$ a[t]b[s xor t]，求和枚举 t⊆s，等价于 i|j=s 且 i&j=0。依赖 SetConvolution，只使用 OR 变换；mod 为 ≥2 的 int 范围整数，允许偶数与合数。两个数组长度相同、非空，N=2^m≤INT_MAX；长度 1 的结果为 a[0]b[0]。按 popcount 分层，层间普通卷积再逆 OR 变换，保留总层数等于目标集合大小的项。O(m^{2}2^m) 时间、O(m2^m) 空间；两组分层数组约占 8(m+1)2^m 字节，另有线性辅助数组。m=20 时仅分层数组约 168 MiB，使用前核对内存。

```
// c[s] = sum_{t subset s} a[t] * b[s xor t].
template <int mod = 998244353>
vector<ModInt<mod>> subset_convolution(const vector<ModInt<mod>> &a,
                                       const vector<ModInt<mod>> &b)
{
    using F = SetConvolution<mod>;
    using Poly = typename F::Poly;
    assert(!a.empty() && a.size() == b.size() && a.size() <= INT_MAX);
    int n = a.size(), m = 0;
    assert((n & (n - 1)) == 0);
    while ((1 << m) < n) m++;
    vector<Poly> f(m + 1, Poly(n)), g(m + 1, Poly(n));
    vector<int> rank(n);
    for (int s = 0; s < n; s++)
    {
        rank[s] = __builtin_popcount(unsigned(s));
        f[rank[s]][s] = a[s];
        g[rank[s]][s] = b[s];
    }
    for (int k = 0; k <= m; k++)
    {
        F::transform(f[k], '|');
        F::transform(g[k], '|');
    }
    Poly answer(n), h(n);
    for (int k = 0; k <= m; k++)
    {
        fill(h.begin(), h.end(), 0);
        for (int i = 0; i <= k; i++)
            for (int s = 0; s < n; s++) h[s] = h[s] + f[i][s] * g[k - i][s];
        F::transform(h, '|', true);
        for (int s = 0; s < n; s++)
            if (rank[s] == k) answer[s] = h[s];
    }
    return answer;
}
```

模板题使用：Luogu P6097

题意：把下标看作集合，求 c[s]=∑ a[t]*b[s⁀], t 遍历 s 的子集，两部分不相交。输入首项为集合大小 m，数组长度 2^m；本题模数为 1000000009。

所需模板：ModInt（第 133 页，19.2.2 节）、SetConvolution（第 242 页，20.3.1 节）、subset_convolution（第 244 页，20.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6097

```
int main()
{
    int m;
    scanf("%d", &m);
    int n = 1 << m;
    using F = SetConvolution<1000000009>;
    F::Poly a(n), b(n);
    for (auto &x : a)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    for (auto &x : b)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    auto c = subset_convolution<1000000009>(a, b);
    for (int s = 0; s < n; s++) printf("%d%c", c[s].v, s + 1 == n ? '\n' : ' ');
}
```

20.4 Chirp Z 变换

20.4.1 Chirp Z：等比点求值

代码依赖：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

chirp_z(f,a,r,m) 返回 f(a*r^i)，i=0..m-1；f 为升幂系数数组，a、r 为 ModInt<mod>。m≥0、f.size()≤INT_MAX；空 f 表示零多项式，m=0 返回空。约定 r^0=1，因此 r=0 时首项为 f(a)、其余为 f(0)；a=0 或 r=1 也单独处理。普通分支依赖 NTT 素数及原根，非空长度 n 要求 n+m-1≤NTT::max_size。利用三角数指数与循环卷积，避免平方指数的整数溢出；O((n+m) log(n+m)) 时间、O(n+m) 空间，特殊分支 O(n+m)。默认模板参数 mod=998244353、primitive=3。输出顺序允许重复点。

```
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>>
chirp_z(const vector<ModInt<mod>> &f, ModInt<mod> a, ModInt<mod> r, int m)
{
    using N = NttConvolution<mod, primitive>;
    using Z = typename N::Z;
    using Poly = typename N::Poly;
    assert(m >= 0 && f.size() <= INT_MAX);
    if (m == 0 || f.empty()) return Poly(m);
    if (a.v == 0 || r.v == 0 || r.v == 1)
    {
        Poly answer(m, f[0]);
        Z value = 0;
        for (int i = int(f.size()) - 1; i >= 0; i--) value = value * a + f[i];
        answer[0] = value;
        if (r.v == 1) fill(answer.begin(), answer.end(), value);
        return answer;
    }
    assert(f.size() + m - 1 <= N::max_size);
    int n = f.size(), size = 1;
    while (size < n + m - 1) size *= 2;
    Poly x(size), y(size), inverse(max(n, m));
    Z power = 1, triangle = 1, ri = r.inv();
    for (auto &v : inverse)
    {
        v = triangle;
        triangle = triangle * power;
        power = power * ri;
    }
```

构造循环卷积并取点值（接上页同一函数）：

```
    power = 1;
    for (int j = 0; j < n; j++)
    {
        x[n - 1 - j] = f[j] * power * inverse[j];
        power = power * a;
    }
    power = 1;
    triangle = 1;
    for (int j = 0; j < n + m - 1; j++)
    {
        y[j] = triangle;
        triangle = triangle * power;
        power = power * r;
    }
    N::ntt(x);
    N::ntt(y);
    for (int j = 0; j < size; j++) x[j] = x[j] * y[j];
    N::ntt(x, true);
    Poly answer(m);
    for (int i = 0; i < m; i++) answer[i] = x[n - 1 + i] * inverse[i];
    return answer;
}
```

模板题使用：Library Checker multipoint_evaluation_on_geometric_sequence

题意：给多项式 f 的升幂系数及 a、r，依次输出 f(a*r^i)，i=0..m-1，模 998244353。r=0 时首项仍为 f(a)，后续各项为 f(0)。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、chirp_z（第 246 页，20.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/multipoint_evaluation_on_geometric_sequence

```
int main()
{
    int n, m, a, r;
    scanf("%d%d%d%d", &n, &m, &a, &r);
    using Z = ModInt<998244353>;
    vector<Z> f(n);
    for (auto &x : f)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    auto answer = chirp_z(f, Z(a), Z(r), m);
    for (int i = 0; i < m; i++) printf("%d%c", answer[i].v, i + 1 == m ? '\n' : ' ');
}
```

20.5 多项式牛顿迭代

20.5.1 形式幂级数求逆

代码依赖：NttConvolution（第 231 页，20.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

FpsInverse::inverse(a,n) 返回 a 的逆级数前 n 项，模数固定 998244353。要求 n>0、a 非空且 a[0]≠0；不足项视为零，多余项只取需要的前缀。继承 NttConvolution<998244353> 的 Poly、Z、multiply，按 Newton 公式 b←b(2-ab) 倍增。所有内部卷积结果长度须 $\le 2^{23}$，n≤2^{22} 是统一充分界。O(n log n) 时间、O(n) 辅助空间。

```
struct FpsInverse : NttConvolution<998244353>
{
    static constexpr int mod = 998244353;

    static Poly inverse(const Poly &a, int n)
    {
        assert(n > 0 && !a.empty() && a[0].v);
        Poly b{a[0].inv()};
        while ((int)b.size() < n)
        {
            int m = min(n, 2 * (int)b.size());
            Poly f(a.begin(), a.begin() + min(m, (int)a.size()));
            auto t = multiply(f, b);
            t.resize(m);
            for (auto &x : t) x = Z(0) - x;
            t[0] = t[0] + 2;
            b = multiply(b, t);
            b.resize(m);
        }
        return b;
    }
};
```

模板题使用：Library Checker inv_of_formal_power_series

题意：给出常数项非零的形式幂级数，求其乘法逆的前 n 项，模 998244353。系数按升幂输入，题目保证已经取模。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、FpsInverse（第 248 页，20.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/inv_of_formal_power_series

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using F = FpsInverse;
    int n;
    cin >> n;
    F::Poly a(n);
    for (auto &x : a) cin >> x.v;
    auto b = F::inverse(a, n);
    for (int i = 0; i < n; i++) cout << b[i].v << (i + 1 == n ? '\n' : ' ');
}
```

20.5.2 多项式带余除法

代码依赖：FpsInverse（第 248 页，20.5.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

PolynomialDivision::divide(a,b) 返回 pair<Poly,Poly>{商, 余式}，模数固定 998244353，系数从低次到高次。输入按值传入；先删除高次端零系数，零多项式统一返回空 vector；b 不可为零。支持零被除式、常数除数及 deg(a)<deg(b)。返回 a=q*b+r，deg(r)<deg(b)，商和余式都去掉高次零。依赖 FpsInverse 与 NttConvolution；反转后求逆得到商，再计算余式。输入各长度 $\leq 2^{22}$；令 N=max(a.size(),b.size())，时间 O(N log(N+1))，额外空间 O(N)。trim(a) 是可直接使用的去高次零工具。P4512 的固定长度输出需在使用处 resize 补零。

```
struct PolynomialDivision : FpsInverse
{
    static void trim(Poly &a)
    {
        while (!a.empty() && a.back().v == 0) a.pop_back();
    }

    static pair<Poly, Poly> divide(Poly a, Poly b)
    {
        assert(a.size() <= max_size / 2 && b.size() <= max_size / 2);
        trim(a);
        trim(b);
        assert(!b.empty());
        if (a.size() < b.size()) return {{}, a};
        int k = a.size() - b.size() + 1;
        Poly ar(a.rbegin(), a.rbegin() + k);
        Poly br(b.rbegin(), b.rend());
        br.resize(min(k, (int)br.size()));
        auto q = multiply(ar, inverse(br, k));
        q.resize(k);
        reverse(q.begin(), q.end());
        auto product = multiply(q, b);
        for (int i = 0; i < (int)a.size(); i++) a[i] -= product[i];
        a.resize(b.size() - 1);
        trim(a);
        return {q, a};
    }
};
```

模板题使用：Luogu P4512 多项式除法

题意：输入两个多项式的次数与系数，输出商的 n-m+1 项和余式的 m 项；模板返回的商、余式均已去高次零，打印前按题目要求补零。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、FpsInverse（第 248 页，20.5.1 节）、PolynomialDivision（第 249 页，20.5.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4512

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using P = PolynomialDivision;
    P::Poly a(n + 1), b(m + 1);
    for (auto &x : a)
    {
        int value;
        cin >> value;
        x = value;
    }
    for (auto &x : b)
    {
        int value;
        cin >> value;
        x = value;
    }
    auto [q, r] = P::divide(a, b);
    q.resize(n - m + 1);
    r.resize(m);
    for (auto x : q) cout << x.v << ' ';
    cout << '\n';
    for (auto x : r) cout << x.v << ' ';
    cout << '\n';
}
```

模板题使用：Library Checker Division of Polynomials

题意：输入系数个数 N、M；输出去高次零后的商和余式长度，再输出两行系数。零商或零余式长度为 0，对应空行；允许被除式次数更小。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、FpsInverse（第 248 页，20.5.1 节）、PolynomialDivision（第 249 页，20.5.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/division_of_polynomials

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using P = PolynomialDivision;
    P::Poly a(n), b(m);
    for (auto &x : a)
    {
        int value;
        cin >> value;
        x = value;
    }
    for (auto &x : b)
    {
        int value;
        cin >> value;
        x = value;
    }
    auto [q, r] = P::divide(a, b);
    cout << q.size() << ' ' << r.size() << '\n';
    for (auto x : q) cout << x.v << ' ';
    cout << '\n';
    for (auto x : r) cout << x.v << ' ';
    cout << '\n';
}
```

20.6 多项式多点求值 | 快速插值

20.6.1 任意点多项式多点求值

代码依赖：PolynomialDivision（第 249 页，20.5.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

固定模数 998244353，Poly 为升幂系数 vector<Z>。MultipointEvaluation(x) 建立点集乘积树，evaluate(f) 按输入点顺序返回 f(x[i])；允许重复点、零点、空点集、空或尾零多项式，返回长度始终等于点数。构造复制点值，查询按值接收 f，不改变树或调用者的左值，可多次复用；不再使用 f 时可 move 传入。系数个数 n≤2^22，点数 m<2^22，且内存可用；本地最大规模证据按报告列出，不表示已实测此绝对上限。每结点存首一多项式，查询将余式下传，叶值就是答案；仅求值，不包含快速插值。建树 O(m $\log^2$(m+1))，查询 O(n log(n+1)+m $\log^2$(m+1))，常驻空间 O(m log(m+1))、含临时数组 O(n+m log(m+1))，递归深度 O(log(m+1))。依赖 PolynomialDivision/FpsInverse/NTT 及模整数。

```
struct MultipointEvaluation : PolynomialDivision
{
    int n;
    vector<Poly> p;

    MultipointEvaluation(const Poly &x)
    {
        assert(x.size() < max_size / 2);
        n = x.size();
        p.resize(4 * max(1, n));
        if (n) build(1, 0, n, x);
    }

    void build(int u, int l, int r, const Poly &x)
    {
        if (r - l == 1)
        {
            p[u] = {Z(0) - x[l], 1};
            return;
        }
        int m = (l + r) / 2;
        build(2 * u, l, m, x);
        build(2 * u + 1, m, r, x);
        p[u] = multiply(p[2 * u], p[2 * u + 1]);
    }
```

余式下传与求值入口（接上页同一结构体）：

```
    void eval(int u, int l, int r, Poly f, Poly &a) const
    {
        if (f.size() >= p[u].size()) f = divide(move(f), p[u]).second;
        if (f.empty()) return;
        if (f.size() == 1)
        {
            fill(a.begin() + l, a.begin() + r, f[0]);
            return;
        }
        int m = (l + r) / 2;
        eval(2 * u, l, m, f, a);
        eval(2 * u + 1, m, r, move(f), a);
    }

    Poly evaluate(Poly f) const
    {
        assert(f.size() <= max_size / 2);
        trim(f);
        Poly a(n);
        if (n) eval(1, 0, n, move(f), a);
        return a;
    }
};
```

模板题使用：Library Checker Multipoint Evaluation

题意：给出 n 项升幂系数和 m 个任意横坐标，按输入顺序输出全部 f(x[i])，模 998244353。1≤n,m≤131072，系数和点均已取模，最高次系数非零；横坐标可以重复。构造一次乘积树后调用 evaluate；同一树可以继续求其他多项式的点值。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、FpsInverse（第 248 页，20.5.1 节）、PolynomialDivision（第 249 页，20.5.2 节）、MultipointEvaluation（第 252 页，20.6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/multipoint_evaluation

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using P = MultipointEvaluation;
    P::Poly f(n), x(m);
    for (auto &a : f)
    {
        int v;
        cin >> v;
        a = v;
    }
    for (auto &a : x)
    {
        int v;
        cin >> v;
        a = v;
    }
    auto a = P(x).evaluate(f);
    for (int i = 0; i < m; i++) cout << a[i].v << (i + 1 == m ? '\n' : ' ');
}
```

20.6.2 任意点多项式快速插值

代码依赖：MultipointEvaluation（第 252 页，20.6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

polynomial_interpolation(x,y) 返回模 998244353 下次数 <m 的唯一插值多项式，升幂 vector 恰好 m 项并保留高次零。optional 为空表示长度不符或归一化横坐标重复；两个空数组返回有值的空 vector，不改输入。复用 MultipointEvaluation 乘积树及导数求值，按 y/P'(x) 递归合并；重复点只在独立求值接口合法。$m \le 2^{22}-1$ 是既有除法/NTT 的统一充分界，官方最大实测 131072。时间 $O(m \log^2(m+1)+m \log p)$，p=998244353；单点逆元沿用模幂，不省略 m log p 项。总空间 O(m log(m+1)+1)，递归 O(log(m+1))。

```
optional<MultipointEvaluation::Poly>
polynomial_interpolation(const MultipointEvaluation::Poly &x,
                         const MultipointEvaluation::Poly &y)
{
    using P = MultipointEvaluation;
    using Poly = P::Poly;
    if (x.size() != y.size()) return nullopt;
    assert(x.size() < P::max_size / 2);
    int n = x.size();
    if (!n) return Poly{};
    const P tree(x);
    Poly d(n);
    for (int i = 1; i <= n; i++) d[i - 1] = tree.p[1][i] * i;
    auto w = tree.evaluate(move(d));
    for (auto v : w)
        if (!v.v) return nullopt;
    for (int i = 0; i < n; i++) w[i] = y[i] * w[i].inv();
    auto combine = [&](auto &&self, int u, int l, int r) -> Poly
    {
        if (r - l == 1) return {w[l]};
        int m = (l + r) / 2;
        auto a = self(self, 2 * u, l, m);
        auto b = self(self, 2 * u + 1, m, r);
        a = P::multiply(move(a), tree.p[2 * u + 1]);
        b = P::multiply(move(b), tree.p[2 * u]);
        for (int i = 0; i < r - l; i++) a[i] += b[i];
        return a;
    };
    return combine(combine, 1, 0, n);
}
```

模板题使用：Library Checker Polynomial Interpolation

题意：给出 n 个互异横坐标及对应点值，输出唯一的次数小于 n 的多项式的 n 项升幂系数，模 998244353。1≤n≤131072，坐标与点值均已取模；先以乘积根多项式的导数求权重，再递归合并。接口返回 optional，长度不符或模意义重复横坐标返回空 optional，空输入返回有值的空向量；本题保证输入合法。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、FpsInverse（第 248 页，20.5.1 节）、PolynomialDivision（第 249 页，20.5.2 节）、MultipointEvaluation（第 252 页，20.6.1 节）、polynomial_interpolation（第 255 页，20.6.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/polynomial_interpolation

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    using Poly = MultipointEvaluation::Poly;
    Poly x(n), y(n);
    for (auto &a : x)
    {
        int v;
        cin >> v;
        a = v;
    }
    for (auto &a : y)
    {
        int v;
        cin >> v;
        a = v;
    }
    auto f = polynomial_interpolation(x, y);
    assert(f);
    for (int i = 0; i < n; i++) cout << (*f)[i].v << (i + 1 == n ? '\n' : ' ');
}
```

20.7 多项式初等函数

20.7.1 形式幂级数开根（含前导零与无解）

代码依赖：FpsInverse（第 248 页，20.5.1 节）、mod_sqrt（第 189 页，19.15.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

FpsSqrt::sqrt(a,n) 返回 optional<Poly>，模数固定 998244353。无解返回 nullopt；有解返回恰好 n 项，n=0 返回有值的空 vector。要求 $0 \le n \le 2^{22}$、a.size() 在 int 范围内；仅看输入前 n 项，不足补零。全零返回全零；首个非零项次数 k 为奇数或该系数为非二次剩余时无解。否则提出 x^k，首项选两个模平方根中较小者，Newton 迭代 b←(b+a/b)/2；最后乘 x^(k/2)，未被条件确定的末 k/2 项取零。这只是确定地返回一个解，不枚举全部根。依赖 mod_sqrt、FpsInverse 与 NttConvolution。时间 O(n log(n+1)) 加一次标量开根的期望 O(log mod)，额外空间 O(n)。P5205 保证 a[0]=1，得到常数项为 1 的根。

```
struct FpsSqrt : FpsInverse
{
    static optional<Poly> sqrt(const Poly &a, int n)
    {
        assert(n >= 0 && n <= max_size / 2);
        int k = 0;
        while (k < n && (k >= (int)a.size() || a[k].v == 0)) k++;
        if (k == n) return Poly(n);
        if (k & 1) return nullopt;
        auto roots = mod_sqrt(a[k].v, mod);
        if (roots.empty()) return nullopt;
        Poly b{Z(roots[0])};
        int target = n - k;
        while ((int)b.size() < target)
        {
            int m = min(target, 2 * (int)b.size());
            Poly f(a.begin() + k, a.begin() + min(k + m, (int)a.size()));
            auto t = multiply(f, inverse(b, m));
            t.resize(m);
            for (int i = 0; i < (int)b.size(); i++) t[i] += b[i];
            for (auto &x : t) x *= Z((mod + 1) / 2);
            b = move(t);
        }
        b.insert(b.begin(), k / 2, Z(0));
        b.resize(n);
        return b;
    }
};
```

模板题使用：Luogu P5205 多项式开根

题意：输入 n 项、保证 a[0]=1，输出平方根的前 n 项，首项取较小的 1。通用接口也可用于无解输入，须先检查 optional。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、FpsInverse（第 248 页，20.5.1 节）、mod_sqrt（第 189 页，19.15.1 节）、FpsSqrt（第 257 页，20.7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5205

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    FpsSqrt::Poly a(n);
    for (auto &x : a)
    {
        int value;
        cin >> value;
        x = value;
    }
    auto b = FpsSqrt::sqrt(a, n);
    if (!b)
    {
        cout << -1 << '\n';
        return 0;
    }
    for (auto x : *b) cout << x.v << ' ';
    cout << '\n';
}
```

模板题使用：Library Checker Sqrt of Formal Power Series

题意：输入任意常数项的 n 项级数；没有平方根输出 -1，否则输出 n 项。接口处理前导零、非二次剩余和全零级数，检查 optional 后才能读取结果。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、FpsInverse（第 248 页，20.5.1 节）、mod_sqrt（第 189 页，19.15.1 节）、FpsSqrt（第 257 页，20.7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/sqrt_of_formal_power_series

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    FpsSqrt::Poly a(n);
    for (auto &x : a)
    {
        int value;
        cin >> value;
        x = value;
    }
    auto b = FpsSqrt::sqrt(a, n);
    if (!b)
    {
        cout << -1 << '\n';
        return 0;
    }
    for (auto x : *b) cout << x.v << ' ';
    cout << '\n';
}
```

接口用法演示：形式幂级数平方根倒数与重复调用

演示约定：输入多组 n 项级数，1≤n≤2^22，模 998244353；求所选平方根的逆元模 x^n，平方根首项取较小根。无平方根或常数项为 0 时输出-1。先 FpsSqrt::sqrt，再检查首项非零，最后 FpsInverse::inverse；同一进程支持多组调用。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、FpsInverse（第 248 页，20.5.1 节）、mod_sqrt（第 189 页，19.15.1 节）、FpsSqrt（第 257 页，20.7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/WIDA-MONTGOMERY-POLY.md

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        FpsSqrt::Poly a(n);
        for (auto &x : a)
        {
            long long value;
            cin >> value;
            x = value;
        }
        auto root = FpsSqrt::sqrt(a, n);
        if (!root || (*root)[0].v == 0)
        {
            cout << -1 << '\n';
            continue;
        }
        auto inverse = FpsInverse::inverse(*root, n);
        for (int i = 0; i < n; i++)
            cout << inverse[i].v << " \n"[i + 1 == n];
    }
}
```

20.7.2 形式幂级数微积分、对数与指数

代码依赖：FpsInverse（第 248 页，20.5.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

FpsFunctions 继承 FpsInverse，复用同一 NTT 和求逆。derivative(a) 形式求导；integral(a) 返回常数项为零的积分，要求 a.size()<998244353。log(a,n) 要求 n>0、a 非空、a[0]=1；exp(a,n) 要求 n>0、a 为空或 a[0]=0。返回前 n 项，不足输入项视为零，超过 n 的尾项不参与计算。内部每次卷积结果长度须 $\le 2^{23}$；$n \le 2^{22}$ 是统一充分界，输入数组长度须在 int 范围内。微积分 O(a.size())；log/exp 为 O(n log(n+1)) 时间和 O(n) 辅助空间。

```
struct FpsFunctions : FpsInverse
{
    static Poly derivative(const Poly &a)
    {
        Poly b;
        for (int i = 1; i < (int)a.size(); i++) b.push_back(a[i] * i);
        return b;
    }

    static Poly integral(const Poly &a)
    {
        assert(a.size() < mod);
        Poly b(a.size() + 1);
        vector<Z> inv(a.size() + 1);
        if (!a.empty()) inv[1] = 1;
        for (int i = 2; i <= (int)a.size(); i++)
            inv[i] = Z(mod - mod / i) * inv[mod % i];
        for (int i = 0; i < (int)a.size(); i++) b[i + 1] = a[i] * inv[i + 1];
        return b;
    }

    static Poly log(const Poly &a, int n)
    {
        assert(n > 0 && !a.empty() && a[0].v == 1);
        Poly f(a.begin(), a.begin() + min(n, (int)a.size()));
        auto b = multiply(derivative(f), inverse(f, n));
        b.resize(n - 1);
        return integral(b);
    }
```

```
    static Poly exp(const Poly &a, int n)
    {
        assert(n > 0 && (a.empty() || a[0].v == 0));
        Poly b{1};
        while ((int)b.size() < n)
        {
            int m = min(n, 2 * (int)b.size());
            auto t = log(b, m);
            for (int i = 0; i < m; i++) t[i] = (i < (int)a.size() ? a[i] : Z(0)) - t[i];
            t[0] = t[0] + 1;
            b = multiply(b, t);
            b.resize(m);
        }
        return b;
    }
};
```

模板题使用：Library Checker log_of_formal_power_series

题意：给定常数项为 1 的形式幂级数，求其形式对数的前 n 项，模 998244353。系数按升幂输入且已取模，结果常数项为 0。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、FpsInverse（第 248 页，20.5.1 节）、FpsFunctions（第 261 页，20.7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/log_of_formal_power_series

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using F = FpsFunctions;
    int n;
    cin >> n;
    F::Poly a(n);
    for (auto &x : a) cin >> x.v;
    auto b = F::log(a, n);
    for (int i = 0; i < n; i++) cout << b[i].v << (i + 1 == n ? '\n' : ' ');
}
```

模板题使用：Library Checker exp_of_formal_power_series

题意：给定常数项为 0 的形式幂级数，求其形式指数的前 n 项，模 998244353。系数按升幂输入且已取模，结果常数项为 1。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、FpsInverse（第 248 页，20.5.1 节）、FpsFunctions（第 261 页，20.7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/exp_of_formal_power_series

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using F = FpsFunctions;
    int n;
    cin >> n;
    F::Poly a(n);
    for (auto &x : a) cin >> x.v;
    auto b = F::exp(a, n);
    for (int i = 0; i < n; i++) cout << b[i].v << (i + 1 == n ? '\n' : ' ');
}
```

20.7.3 形式幂级数快速幂（十进制大指数）

代码依赖：FpsFunctions（第 261 页，20.7.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

FpsPower::power(a,exponent,n) 返回 a 的非负整数次幂模 x^n 的前 n 项，固定模数 998244353。exponent 为非空十进制数字 string_view，允许前导零；整数指数可传 to_string(k)。$0 \le n \le 2^{22}$，a.size() 须在 int 范围内；短输入补零、长输入只看前 n 项。n=0 返回空，零次幂（含零多项式的零次幂）返回 1。对首个非零项次数 s，若真实 s*k≥n 则返回全零；否则提取常数 c，计算 x^(s*k)*c^k*exp(k*log(a/(c*x^s)))。指数分别保留模 p、模 p-1 和截到 n 的值；不可仅用 k mod p 判零或算位移。依赖 FpsFunctions、FpsInverse、NTT。指数位数 L，时间 O(L+n log(n+1))、额外空间 O(n)，不支持负指数或小数指数。

```
struct FpsPower : FpsFunctions
{
    static Poly power(const Poly &a, string_view exponent, int n)
    {
        assert(n >= 0 && n <= max_size / 2 && !exponent.empty());
        long long k = 0, phi = 0;
        int cap = 0;
        for (char c : exponent)
        {
            assert('0' <= c && c <= '9');
            int digit = c - '0';
            k = (k * 10 + digit) % mod;
            phi = (phi * 10 + digit) % (mod - 1);
            cap = min(n, cap * 10 + digit);
        }
        Poly result(n);
        if (n == 0) return result;
        if (cap == 0)
        {
            result[0] = 1;
            return result;
        }
        int first = 0;
        while (first < n && (first >= (int)a.size() || a[first].v == 0)) first++;
        if (first == n || (first && cap > (n - 1) / first)) return result;
        int shift = first * cap, m = n - shift;
        Poly b(a.begin() + first, a.begin() + min(first + m, (int)a.size()));
        Z leading = a[first], inverse_leading = leading.inv();
        for (auto &x : b) x *= inverse_leading;
        auto f = log(b, m);
        for (auto &x : f) x *= Z(k);
        auto t = exp(f, m);
        Z scale = leading.pow(phi);
        for (int i = 0; i < m; i++) result[i + shift] = t[i] * scale;
        return result;
    }
};
```

模板题使用：Luogu P5273 多项式幂函数（加强版）

题意：输入 n 和十进制大指数 k，输出 a^k 的前 n 项；k 必须用 string 读入，不能只读 k mod p。允许前导零和 k=0。本程序同样适用于 P5245（a[0]=1）及 Library Checker Pow of Formal Power Series，三个入口均已本地验证。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、FpsInverse（第 248 页，20.5.1 节）、FpsFunctions（第 261 页，20.7.2 节）、FpsPower（第 264 页，20.7.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5273

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    string k;
    cin >> n >> k;
    FpsPower::Poly a(n);
    for (auto &x : a)
    {
        int value;
        cin >> value;
        x = value;
    }
    auto b = FpsPower::power(a, k, n);
    for (auto x : b) cout << x.v << ' ';
    cout << '\n';
}
```

20.8 常系数齐次线性递推

20.8.1 特征多项式求线性递推第 k 项

recurrence_nth(init,c,n,p) 使用 0-based 下标，n 为 unsigned long long。init 与 c 长度均为阶数 k；a[i]=∑c[j-1]a[i-j]。2≤p≤INT_MAX，数值均在 [0,p)，本函数允许合数模数。空系数约定全零序列。计算 x^n 对特征多项式取余再与初值点乘，O(k^2 log(n+1)) 时间、O(k) 空间；不依赖 BM 或其他模板。较大阶数可使用本库 BostanMori::recurrence，须满足其 NTT 素数模数与容量条件；本函数允许合数模数。

```cpp
// init has exactly k entries; c[j-1] multiplies a[n-j]. Modulus need not be prime.
int recurrence_nth(const vector<int> &init,
                   const vector<int> &c,
                   unsigned long long n,
                   int p = 998244353)
{
    assert(p >= 2 && init.size() == c.size());
    int k = c.size();
    if (!k) return 0;
    for (int x : init) assert(0 <= x && x < p);
    for (int x : c) assert(0 <= x && x < p);
    if (n < (unsigned)k) return init[n];
    auto multiply = [&](const vector<int> &a, const vector<int> &b)
    {
        vector<int> t(2 * k - 1);
        for (int i = 0; i < k; i++)
        {
            for (int j = 0; j < k; j++) t[i + j] = (t[i + j] + 1LL * a[i] * b[j]) % p;
        }
        for (int i = 2 * k - 2; i >= k; i--)
        {
            for (int j = 1; j <= k; j++)
                t[i - j] = (t[i - j] + 1LL * t[i] * c[j - 1]) % p;
        }
        t.resize(k);
        return t;
    };
    vector<int> answer(k), x(k);
    answer[0] = 1;
    if (k == 1)
        x[0] = c[0];
    else
        x[1] = 1;
    while (n)
    {
        if (n & 1) answer = multiply(answer, x);
        x = multiply(x, x);
        n >>= 1;
    }
    long long value = 0;
    for (int i = 0; i < k; i++) value = (value + 1LL * answer[i] * init[i]) % p;
    return value;
}
```

模板题使用：Luogu P5487

题意：给出 n 个初值，恢复唯一的最短递推并求第 m 项，n≤10000、n<m≤10^9、阶数 ≤5000，模 998244353。BM 返回 c[j-1] 乘 a[i-j]，首行仅输出系数、不带阶数；裁取恰好 c.size() 个初值后调用 recurrence_nth。全零序列空递推仍输出空首行和第二行 0。复杂度 O(nk+k^2log m)，O(n+k) 空间。

所需模板：berlekamp_massey（第 367 页，27.1 节）、recurrence_nth（第 266 页，20.8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5487

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    unsigned long long m;
    cin >> n >> m;
    vector<int> s(n);
    for (auto &x : s)
    {
        cin >> x;
        x %= 998244353;
        if (x < 0) x += 998244353;
    }
    auto c = berlekamp_massey(s);
    for (int i = 0; i < (int)c.size(); i++)
    {
        if (i) cout << ' ';
        cout << c[i];
    }
    cout << '\n';
    vector<int> init(s.begin(), s.begin() + c.size());
    cout << recurrence_nth(init, c, m) << '\n';
}
```

20.8.2 快速递推与有理生成函数单项

代码依赖：NttConvolution（第 231 页，20.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

BostanMori<mod=998244353,primitive=3>，依赖参数化 NTT；素数模数与原根条件同 NttConvolution。Poly 为升幂系数 vector<Z>。coefficient(p,q,n) 返回 [x^n]p(x)/q(x)，q 非空且 q[0]≠0，p 可空或次数不小于 q；令 d=max(p.size(),q.size())，要求 2d-1≤NTT::max_size。recurrence(init,c,n) 返回 a[n]，从下标 k 起 a[i]=c[0]a[i-1]+⋯+c[k-1]a[i-k]，init、c 长度均为 k；k=0 约定全零。非初值查询要求 2k+1≤NTT::max_size。n 支持完整 unsigned long long。每轮两次卷积后抽取奇偶项，O(d log d log(n+1)) 时间、O(d) 空间；递推接口 d=k+1。

```
template <int mod = 998244353, int primitive = 3> struct BostanMori
{
    using N = NttConvolution<mod, primitive>;
    using Z = typename N::Z;
    using Poly = typename N::Poly;
    using U = unsigned long long;

    // Coefficient of x^n in p(x) / q(x), with q[0] != 0.
    static Z coefficient(Poly p, Poly q, U n)
    {
        assert(!q.empty() && q[0].v != 0);
        assert(max(p.size(), q.size()) <= (N::max_size + 1ULL) / 2);
        while (n && !p.empty())
        {
            Poly r = q;
            for (int i = 1; i < int(r.size()); i += 2) r[i] = Z(0) - r[i];
            Poly a = N::multiply(p, r);
            Poly b = N::multiply(q, r);
            p.clear();
            q.clear();
            for (int i = n & 1; i < int(a.size()); i += 2) p.push_back(a[i]);
            for (int i = 0; i < int(b.size()); i += 2) q.push_back(b[i]);
            n >>= 1;
        }
        return p.empty() ? Z(0) : p[0] / q[0];
    }
```

从初值构造递推生成函数（接上页同一结构体）：

```
    // a[i] = c[0]*a[i-1] + ... + c[k-1]*a[i-k], for i >= k.
    static Z recurrence(const Poly &init, const Poly &c, U n)
    {
        assert(init.size() == c.size());
        int k = c.size();
        if (k == 0) return 0;
        if (n < init.size()) return init[n];
        assert(k + 1ULL <= (N::max_size + 1ULL) / 2);
        Poly q(k + 1);
        q[0] = 1;
        for (int i = 0; i < k; i++) q[i + 1] = Z(0) - c[i];
        Poly p = N::multiply(init, q);
        p.resize(k);
        return coefficient(p, q, n);
    }
};
```

模板题使用：Luogu P4723

题意：给 k 个递推系数和 a[0..k-1]，求 a[n] 模 998244353。c[0] 乘前一项，c[k-1] 乘前 k 项；输入先系数后初值，recurrence 参数先初值后系数。通过 ModInt 赋值归一化负数。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、BostanMori（第 268 页，20.8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4723

```
int main()
{
    unsigned long long n;
    int k;
    scanf("%llu%d", &n, &k);
    using B = BostanMori<>;
    B::Poly c(k), init(k);
    for (auto &x : c)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    for (auto &x : init)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    printf("%d\n", B::recurrence(init, c, n).v);
}
```

20.9 多项式平移 | 连续点值平移

20.9.1 多项式系数平移

代码依赖：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

polynomial_shift<mod=998244353,primitive=3>(f,c) 返回 f(x+c) 的升幂系数，结果长度始终与 f 相同，保留末尾零；空输入返回空。参数 c 使用 ModInt<mod>。依赖参数化 NTT，mod 为素数且 primitive 为原根；非空长度 n 要求 2n-1≤NTT::max_size，因此阶乘均可逆。逆序的 f[i]i! 与 c^i/i! 卷积后提取对应项，O(n log n) 时间、O(n) 空间。c=0 合法。

```
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>> polynomial_shift(const vector<ModInt<mod>> &f, ModInt<mod> c)
{
    using N = NttConvolution<mod, primitive>;
    using Z = typename N::Z;
    using Poly = typename N::Poly;
    if (f.empty()) return {};
    assert(f.size() <= (N::max_size + 1ULL) / 2);
    int n = f.size();
    Poly fact(n), inv(n), a(n), b(n);
    fact[0] = 1;
    for (int i = 1; i < n; i++) fact[i] = fact[i - 1] * i;
    inv[n - 1] = fact[n - 1].inv();
    for (int i = n - 1; i > 0; i--) inv[i - 1] = inv[i] * i;
    Z power = 1;
    for (int i = 0; i < n; i++)
    {
        a[n - 1 - i] = f[i] * fact[i];
        b[i] = power * inv[i];
        power = power * c;
    }
    auto product = N::multiply(a, b);
    for (int i = 0; i < n; i++) a[i] = product[n - 1 - i] * inv[i];
    return a;
}
```

模板题使用：Library Checker polynomial_taylor_shift

题意：给多项式 f 的 n 个升幂系数和常数 c，输出 f(x+c) 的 n 个升幂系数，模 998244353。平移方向为加 c，c=0 时原样返回。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、polynomial_shift（第 270 页，20.9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/polynomial_taylor_shift

```
int main()
{
    int n, c;
    scanf("%d%d", &n, &c);
    using Z = ModInt<998244353>;
    vector<Z> f(n);
    for (auto &x : f)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    auto answer = polynomial_shift(f, Z(c));
    for (int i = 0; i < n; i++) printf("%d%c", answer[i].v, i + 1 == n ? '\n' : ' ');
}
```

20.10 符号化方法

20.10.1 生成函数建模到模板：三个可核验例子

下面按“约束、因子、取系数、截断、算法”走完建模。记 N 为所求最大次数、$L = N + 1$ 为数组长度，$M(L) = O(L \log L)$ 为 NTT 乘法复杂度。第 20.2.1 节（第 231 页）的 `NttConvolution<p,g>` 可指定素数与原根；`FpsInverse`（第 20.5.1 节，第 248 页）、`FpsFunctions`（第 20.7.2 节，第 261 页）、`FpsPower`（第 20.7.3 节，第 264 页）当前固定为 $p = 998244353$。这些 FPS 操作的长度参数 n 表示项数，求 x^N 必须传 $N + 1$。

20.10.1.1 例一：不同盒子、相同球、每盒有上下界（OGF 乘积）

盒子 i 要求 $l_i \le a_i \le u_i$，总数 $\sum a_i = N$。令 $b_i = a_i - l_i$、$c_i = u_i - l_i$、$R = N - \sum l_i$。若有 $u_i < l_i$ 或 $R < 0$，答案为 0；否则所求为

$$[x^R] \prod_{i=1}^{k} (1 + x + \cdots + x^{c_i}).$$

各因子选一个次数，唯一对应一个盒子的数量，所以系数不乘球的排列数。例如容量为 $(2, 3, 1)$、下界均为 0、总数为 4，方案恰为

$$(1, 3, 0),\ (2, 2, 0),\ (0, 3, 1),\ (1, 2, 1),\ (2, 1, 1),$$

答案是 5。零容量的因子为 1，不应当删除这个盒子所携带的其他条件。

截断。只保留 $0, \ldots, R$ 次，因为因子没有负次数，高次不会回流到低次。先把 c_i 截至 R，每次乘法后仅当长度超过 $R + 1$ 才截短；不要把较短中间多项式补零至 $R + 1$，否则会增加分治卷积的时间。最终若数组尚无第 R 项，该系数就是 0。小规模用有界背包的滑动窗口：

$$d_i(s) = d_i(s - 1) + d_{i-1}(s) - [s > c_i] d_{i-1}(s - c_i - 1),$$

以 $d_0(0) = 1$、负下标为 0 初始化，时间 $O(k(R + 1))$、空间 $O(R + 1)$，只用加减，任意模数可用。不要为了出现乘积就立即写 NTT。

卷积与特殊结构。因子较多且总次数适中时，分治调用 `NttConvolution<998244353>::multiply`；设截短后的总次数为 $D=\sum\min(c_i,R)$，常用上界 $O(k+M(D+1)\log(k+1))$，每层截断通常还能减少工作。若所有 $c_i=c$，令 $P(x)=1+\cdots+x^{\min(c,R)}$，可用 `FpsPower::power(P,to_string(k),R+1)`，时间 $O(M(R+1)+\log(k+1))$。当 $k\geq 1$，另一条路线是容斥：

$$\sum_{j=0}^{\min(k,\lfloor R/(c+1)\rfloor)}(-1)^j\binom{k}{j}\binom{R-j(c+1)+k-1}{k-1}.$$

按组合数选型（第 21.1.3 节，第 278 页）选择组合数接口，再比较项数与卷积成本。$k=0$ 单独返回 $[N=0]$，不向公式塞入 $k-1=-1$。

20.10.1.2 例二：步长 1、2 的有序序列（OGF 求逆）

一次操作占用长度 1 或 2，序列总长度为 N，允许空序列。一个操作的 OGF 为 $B(x)=x+x^2$；因为操作有先后，

$$F(x)=\sum_{j\geq 0}B(x)^j=\frac{1}{1-x-x^2},\qquad f_N=[x^N]F(x).$$

$N=3$ 时有 $111,12,21$，$N=5$ 时为 8。求前 $N+1$ 项可调用 `FpsInverse::inverse({1,-1,-1},N+1)`（初始化为其 `Poly` 类型），常数项为 1，可求逆。若只处理这个二阶递推，$f_n=f_{n-1}+f_{n-2}$ 的 $O(N)$ 动态规划更快；这里只是演示可推广到一般步长计数的建模，FPS 不必优于特化算法。

一般每个长度 $s\geq 1$ 有 b_s 种操作，则 $B(x)=\sum b_sx^s$，仍取 $(1-B)^{-1}\bmod x^{N+1}$，时间 $O(M(N+1))$、空间 $O(N+1)$。若 N 巨大且允许步长有固定最大值，可证明有界阶递推后改用远项算法，不应先分配长达 $N+1$ 的数组。

反例。若每种长度只记录使用多少次而不区分先后，应改为 $(1-x)^{-1}(1-x^2)^{-1}$，长度 3 的答案为 2。若允许一种长度为 0 的操作，任意插入它便有无限多序列，$B(0)=0$ 的条件被破坏；不能把一个本来不有限的计数问题交给求逆模板。

20.10.1.3 例三：不同元素分成单点与无序对（EGF 指数）

n 个不同元素，每块大小为 1 或 2，块内与块间均无序。固定的一个单点集合只有 1 种结构，一个二元集合也只有 1 种，故块的 EGF 为

$$B(x)=x+\frac{x^2}{2!},\qquad F(x)=\exp B(x),\qquad a_n=n![x^n]F(x).$$

指数展开的 $B(x)^k/k!$ 先将标签分到 k 个非空有序块，再去掉块的次序；这里各块不空且标签集不交，除以 $k!$ 有严格的计数依据。$n=4$ 时：全单点 1 种，一对加两单点有 $\binom{4}{2}=6$ 种，两对有 3 种，总计 10；不是系数 $[x^4]F=5/12$ 本身。

落到当前接口。在 $p=998244353$ 且 $N<p$ 下：

```
using F = FpsFunctions;
using Z = F::Z;
F::Poly b(N + 1);
if (N >= 1) b[1] = 1;
if (N >= 2) b[2] = Z(2).inv();
auto f = F::exp(b, N + 1);
Binomial<998244353> comb(N);
Z answer = f[N] * comb.fac[N];
```

数组存的是除过阶乘的系数，最后还原；本例建模与 FPS 共耗时 $O(M(N+1))$。若只求这一个模型，选出元素 1 的去向，还可独立得到 $a_0=a_1=1$、$a_n=a_{n-1}+(n-1)a_{n-2}$，给出 $O(N)$ 的简单算法与核验。

增加块数条件。恰好 k 块改为 $n![x^n]B(x)^k/k!$。可调用 `FpsPower::power(b,to_string(k),n+1)`，取第 n 项后乘 `comb.fac[n]` 与 `comb.ifac[k]`，表预处理至 $\max(n,k)<p$。先判 $k\leq n\leq 2k$，不满足便为 0；空对象 $(n,k)=(0,0)$ 为 1。例如 $(n,k)=(4,2)$ 只有三种两两配对。若盒子本身带标号，就没有除以 $k!$ 的步骤；若盒子允许空，单盒因子还应添 1。

20.10.1.4 抄代码前的四项检查与本地证据

1. **目标系数与长度：** 求 $[x^N]$ 使用长度 $N+1$；常数项是否代表空方案；EGF 输入是否已乘逆阶乘，输出是否已乘阶乘。
2. **代数前提：** 逆元分母非零；FPS 的 log 要求首项 1，exp 要求首项 0；涉及阶乘和积分时不能跨过特征。整数有意义的计数，不保证其 EGF 在任意模数下可直接计算。
3. **实际容量：** NTT 接口检查的是完整乘积长度 $a.size()+b.size()-1$，不是最后保留的 $N+1$；“乘后截断” 不会免掉中间数组。固定模数的最大 NTT 长度为 2^{23}，上述 FPS 取 $L \le 2^{22}$ 是安全的保守长度条件。`FpsPower` 还显式检查这个半容量限制；不同 FPS 的局部乘积不能只凭最终次数估算。
4. **先用极小样例数对象：** 至少检查空对象、一个元素、两个元素，以及能区分 “有序/无序” 的第一个例子。错误模型也能被正确的卷积模板稳定算错。

配套 `tests/math_knowledge_combinatorics.py` 用笛卡尔积枚举有界数量向量、显式枚举步长序列、枚举集合划分，独立核对三个例子的系数与边界；同时编译当前 C++ 接口并对照精确整数组合数。运行命令为 `python3 tests/math_knowledge_combinatorics.py`。这是本地建模与接口集成验证，不是新增在线评测 AC，也不取代各模板的完整回归。

20.11 Lagrange 反演

20.11.1 隐式生成函数与 Lagrange 反演

递归对象常给出 $T = x\Phi(T)$，而题目只问 T 或 $H(T)$ 的某一项。Lagrange 反演把这个未知级数的系数，改写为已知级数 Φ 的整数幂系数；不一定要先求出整个 T。本节的“反演”是**复合逆**，与第 26.1.1 节（第 350 页）的已知点值插值不同。

20.11.1.1 适用条件与取系数公式

先在 $\mathbb{Q}$ 或特征零域 K 上讨论形式幂级数。设 $\Phi(t), H(t) \in K[[t]]$，$\Phi(0) \neq 0$；若改用一般的 $\mathbb{Q}$-代数，要求 $\Phi(0)$ 是单位，即有乘法逆元。则

$$T(x) = x\Phi(T(x)), \qquad T(0) = 0$$

唯一确定 $T \in xK[[x]]$：右侧的 x^n 项只依赖 T 的前 $n-1$ 次项，可从常数项起递推；其一次项为 $\Phi(0)$。令 $f(t) = t/\Phi(t)$，则 $f(T(x)) = x$，且 $T(f(t)) = t$。对 $n \geq 1$，有

$$\boxed{[x^n]H(T(x)) = \frac{1}{n}[t^{n-1}]H'(t)\Phi(t)^n.}$$

特别地，取 $H(t) = t^k$，在 $n \geq k \geq 1$ 时得到

$$\boxed{[x^n]T(x)^k = \frac{k}{n}[t^{n-k}]\Phi(t)^n.}$$

边界先分支： $[x^0]H(T) = H(0)$；$T^0 = 1$，故 $[x^n]T^0 = [n=0]$；$k \geq 1$ 且 $0 \leq n < k$ 时为 0。不要把 $n = 0$ 代进带 $1/n$ 的公式。这里每个固定次数只涉及有限次代数运算，**不要求级数具有解析收敛半径**。

20.11.1.2 形式留数证明

对只有有限个负次项的形式 Laurent 级数，记 $\operatorname{Res}_t F = [t^{-1}]F$。形式导数没有 t^{-1} 项，故 $\operatorname{Res}_t F' = 0$。若 $g(0) = 0$ 且 $g'(0)$ 可逆，换元公式为

$$\operatorname{Res}_x F(x) = \operatorname{Res}_t F(g(t))g'(t).$$

只需检查单项式 $F(x) = x^j$：$j \neq -1$ 时右边是 $(g^{j+1})'/(j+1)$ 的留数，为 0；$j = -1$ 时，写 $g = tu$、$u(0) \neq 0$，便有 $g'/g = t^{-1} + u'/u$，留数为 1。取 $g = f = t/\Phi(t)$，再用乘积求导，得到

$$\begin{aligned}[x^n]H(T(x)) &= \operatorname{Res}_t \frac{H(t)f'(t)}{f(t)^{n+1}} \\ &= \frac{1}{n}\operatorname{Res}_t \frac{H'(t)}{f(t)^n} = \frac{1}{n}[t^{n-1}]H'(t)\Phi(t)^n.\end{aligned}$$

中间等号来自 $\operatorname{Res}_t(H(t)f(t)^{-n})' = 0$；它也清楚标出除以 n 的位置。

20.11.1.3 模型一按内部结点计数的满有序多叉树

固定整数 $d \geq 2$，每个结点要么是叶子，要么恰有 d 个**有序**孩子。用 x 标记内部结点，叶子本身是大小 0 的一种树，故 OGF 满足

$$A(x) = 1 + xA(x)^d.$$

令 $T = A - 1$，化为 $T = x(1+T)^d$。因此 $a_0 = 1$，而 $n \geq 1$ 时

$$a_n = [x^n]A = \frac{1}{n}\binom{dn}{n-1} = \frac{1}{(d-1)n+1}\binom{dn}{n}.$$

$d = 2$ 得到 $1, 1, 2, 5, 14, \ldots$；$d = 3$ 得到 $1, 1, 3, 12, 55, \ldots$。有 n 个内部结点时，共有 $dn+1$ 个结点、$(d-1)n+1$ 个叶子，所以这不是按所有结点计数。孩子位置有序；若把孩子改成无序集合，不能仍用 A^d。若先用 `Binomial<p> c(d*n)` 再求组合数，须确认 $dn < p$、表能放下，并在宽类型中计算 dn，最后检查选用的分母是否可逆。即使 $n < p$，也可能有 $dn \geq p$；此时回到第 21.1.3 节（第 278 页）的组合数选型，不能越界调用阶乘表。

20.11.1.4 模型二带标号有根树的 EGF

结点标签为 $1,\dots,n$，根也是结构的一部分，根的孩子**没有先后顺序**。删除根后，剩下的是一组标签互不相交的非空有根树，因此

$$T(x)=x\exp T(x),\qquad T(x)=\sum_{n\ge1}r_n\frac{x^n}{n!}.$$

取 $\Phi(t)=e^t$，对 $n\ge1$ 有

$$[x^n]T=\frac{1}{n}[t^{n-1}]e^{nt}=\frac{n^{n-1}}{n!},\qquad r_n=n!\,[x^n]T=n^{n-1}.$$

$n=0$ 没有有根树，$n=1$ 有 1 棵；$r_2=2,r_3=9,r_4=64$。数组里的 $[x^3]T=3/2$ 不是树的数目 9，必须还原阶乘。这里计算的是“选定根也计入方案”的数目；若指定某个标签为根，$n\ge2$ 时为 n^{n-2}。也不能把隐式的 $\exp T$ 替换成已知的 $\exp x$。

20.11.1.5 落到现有接口及模数边界

只求一个幂系数。第 20.7.3 节（第 264 页）的 `FpsPower::power` 当前固定模数 $p=998244353$，只支持非负整数指数。下面采用简单充分范围：$1\le k\le n<p$、$L=n-k+1\le 2^{22}$，且内存足够。把 Φ 的前 L 项存入 `FpsPower::Poly phi`，可写

```
using F = FpsPower;
using Z = F::Z;
int l = n - k + 1;
auto q = F::power(phi, to_string(n), l);
Z answer = Z(k) * Z(n).inv() * q[n - k];
```

长度参数是**项数**，故取 t^{n-k} 要传 $n-k+1$；不能只传 $n-k$。短输入缺项按零处理，若真有非零项则必须提供。指数是完整非负十进制字符串，不要自行只传 $n \bmod p$。单次耗时 $O(L\log(L+1)+\log(n+1))$、额外空间 $O(L)$；对 $n=1,\dots,N$ 分别取系数不能据此声称整体 $O(M(N))$，按此做法的通用上界为 $O(N^2\log(N+1))$，还未计输入构造。

辅助接口。第 20.7.2 节（第 261 页）提供 `derivative(a)`、`integral(a)`、`log(a,l)`、`exp(a,l)`；log 要求首项 1，exp 要求首项 0，积分分母须可逆。这些操作仍固定模数 998244353，$1\le l\le 2^{22}$ 是保守安全长度界。它们可用于构造 H' 或已知表达式的指数，**不是求解任意隐式方程的接口**。对一般 H，若长度 n 满足上述容量界，先算 $q=\Phi^n \bmod t^n$，再取 $n^{-1}\sum_{i=1}^{n} iH_iq_{n-i}$，其中 $H_i=[t^i]H$。已知 q 后只需 $O(n)$ 点积，不必计算 $H'q$ 的整段卷积。第 21.1.1 节（第 276 页）的 `Binomial<p> c(N)` 在素数 p、$N<p$ 下提供 `c.choose(n,k)`、`c.fac[n]` 和 `c.ifac[n]`；先显式建表至所有查询的最大下标，再读取或还原 EGF 阶乘。

不要混淆两种逆。$f^{\langle-1\rangle}$ 满足 $f(T)=x$，而第 20.5.1 节（第 248 页）的 `inverse` 求 $fB=1 \pmod{x^L}$，要求 $f(0)\ne0$。这里 $f=t/\Phi(t)$ 的常数项恰为 0，不能交给乘法求逆模板。本节不新增通用 FPS 复合、复合逆或整段隐式解的实现覆盖。

整数除法不能随意改成模逆元。在 $\mathbb{F}_p$ 中使用带 $1/n$ 的公式，至少要求 $p\nmid n$，且 Φ,H 的所需系数本来就在该域中有定义；EGF 的 $1/j!$ 也不能跨过 $j=p$。例如 $T=x(1+T)^2$、$n=p=3$，整数答案为

$$[x^3]T=\frac{1}{3}\binom{6}{2}=5\equiv2 \pmod 3,$$

但分子 15 与分母 3 先取模都为 0，不能用 0^{-1}；只保留分子模 3 也已经丢掉了除以 3 所需的信息。可改用无除法递推，或在能保证整除时保留足够整数/提升模数精度再除。这例也可改用等价式 $\binom{6}{3}/4$，但必须重新检查分母 4 在模 3 下可逆。

来源。定理、形式留数换元及两类树的公式核对 Ira M. Gessel, *Lagrange Inversion* 的定理 2.1.1、第 4.1 节、第 3.2–3.3 节；中文概念导航参见 OI Wiki：Lagrange 反演。以上模型说明、接口对应和模 3 反例为本手册重新组织的推导，不将知识补充记作在线评测通过。

第二十一章　组合数学

21.1　排列组合

21.1.1　阶乘组合数

代码依赖：ModInt（第 133 页，19.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

Binomial<prime>(n=0) 预处理 0..n，要求 prime 为素数且 0≤n<prime。init(n) 只扩大当前上限，较小或相同的 n 不改变表；choose(n,k) 返回 C(n,k)，permute(n,k) 返回 n!/(n-k)!。查询前显式预处理到 n，查询本身不扩表。n<0、k<0 或 k>n 返回 0，C(0,0)=P(0,0)=1。fac、ifac 为可直接读取的阶乘与逆阶乘表，增长可能使旧引用失效。单次查询 O(1)；从 old 扩到 n 新增 O(n-old+log prime) 次域运算，重分配还可能复制旧表。首次预处理 O(n+log prime)，空间 O(n)。超过模数不能使用阶乘除法：小素数用 Lucas，小合数用 exLucas。

```
template <int mod> struct Binomial
{
    using Z = ModInt<mod>;
    vector<Z> fac{Z(1)}, ifac{Z(1)};

    Binomial(int n = 0) { init(n); }

    void init(int n)
    {
        assert(0 <= n && n < mod);
        int old = (int)fac.size() - 1;
        if (n <= old) return;
        fac.resize(n + 1);
        ifac.resize(n + 1);
        for (int i = old + 1; i <= n; i++) fac[i] = fac[i - 1] * i;
        ifac[n] = fac[n].inv();
        for (int i = n; i > old; i--) ifac[i - 1] = ifac[i] * i;
    }
```

```
    Z choose(int n, int k) const
    {
        if (k < 0 || k > n) return 0;
        assert(n < (int)fac.size());
        return fac[n] * ifac[k] * ifac[n - k];
    }

    Z permute(int n, int k) const
    {
        if (k < 0 || k > n) return 0;
        assert(n < (int)fac.size());
        return fac[n] * ifac[n - k];
    }
};
```

应用题补充：Luogu P1313 计算系数

题意：求 (by+ax) 的 k 次方中指定项的系数，模 10007。x、y 的指数分别为 n、m，均非负且 n+m=k。k 至多 1000，a、b 均为不超过 1000000 的非负数。组合 C(k,n) 与两个幂即可。模数为素数且 k 小于模数，满足阶乘逆元前提；零次幂按空乘积为 1。

所需模板：ModInt（第 133 页，19.2.2 节）、Binomial（第 276 页，21.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1313

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int a, b, k, n, m;
    cin >> a >> b >> k >> n >> m;
    Binomial<10007> c(k);
    auto x = ModInt<10007>(a).pow(n);
    auto y = ModInt<10007>(b).pow(m);
    cout << (c.choose(k, n) * x * y).v << '\n';
}
```

21.1.2 计数建模：先确认哪些对象被区分

写公式前，先用一句话说明：**一个方案是什么，什么变化会产生新方案？**“把 n 个物品分成 k 组”不足以选公式；物品是否带标号、组是否带标号、组内是否有序、组间是否有序、空组是否允许，分别决定不同问题。

21.1.2.1 五个判别问题与两个最小反例

1. **物品可区分吗？**给定标签 $1,\dots,n$，交换两个物品可能改变方案，属于带标号计数。相同球的方案只记录各类或各盒数量，不能再乘一个 $n!$。

2. **容器可区分吗？**盒子 A、B 是不同位置，即使容量相同也不同。无标号盒子通常记录一个集合划分；空盒不形成新的块。

3. **次序在哪里？**一组元素组成集合、线性排列、圆排列，分别是不同结构。例如固定 s 个标签组成非空块时，集合只有 1 种，线性排列有 $s!$ 种，不区分起点的圆排列有 $(s-1)!$ 种。不能只看到“无序分组”就忽略组内次序。

4. **哪些数量固定？**“恰好 k 个非空组”与“至多 k 个组”不同；如果块数也要统计，可用额外变量 y 标记，每个块贡献一个 y。

5. **空对象是否是一种方案？**通常空序列、空集合各有 1 种，所以常数项为 1。“至少一个块”则要减去 1。无物品且无盒时通常计 1，非空物品放入零盒计 0；题目另有定义时以题面为准。

不能机械除以盒子数的阶乘。两个不同球放入两个有标号且允许空的盒子，有 $2^2=4$ 种；商去盒子标号后只有 $\{\{1,2\}\}$ 与 $\{\{1\},\{2\}\}$ 两种。这个例子恰巧等于 $4/2!$，却不是通用理由：三个盒子时有 $3^2=9$ 种，无标号且至多三块仍只有 2 种，而 $9/3!$ 甚至不是整数。原因是含空盒时不同方案的稳定子大小可能不同。若恰好 k 个非空块且只区分盒子标号，每个划分有恰好 $k!$ 个标号方式，才可除以 $k!$。

不能把“顺序不同”当成“类型数量不同”。用长度 1、2 的砖铺长度 3 的一行，方案是 $111, 12, 21$，共 3 种；若只问凑出总重量 3 的多重集合，则 12 与 21 是同一种，共 2 种。前者是序列，后者是每种类型独立选择重数。

21.1.2.2 OGF、EGF 的选择来自乘法，而非题目用词

当独立部分只需把大小相加，且大小为 i,j 的两个对象合并后恰有 a_ib_j 种，使用 OGF 的普通卷积。若还须从 $i+j$ 个不同标签中选 i 个分给第一部分，多出 $\binom{i+j}{i}$，使用 EGF 更自然。

$$\text{OGF 数组：}A_i=a_i,\qquad \text{EGF 数组：}A_i=a_i/i!,\qquad a_n=n!A_n.$$

卷积模板始终计算普通数组卷积；它不会识别并补上 EGF 的阶乘。题目出现标签不代表只能用 EGF，出现无序也不代表只能用 OGF；判断依据是当前分解方式如何分配标签与次序。一般非空带标号块的 EGF 记为 $B(x)$，则有序块序列是 $1/(1-B(x))$，无序块集合是 $\exp B(x)$；二者都要求 $B(0)=0$，使每个固定次数只有有限项贡献。本段组合解释参见 OI Wiki：指数生成函数。

21.1.3 组合数选型：先看模数，再看下标与预算

组合恒等式负责推导；本节负责把 $\binom{n}{k}$ 落到可运行的接口。先用有符号或更宽类型处理负下标、$k>n$ 与 $n+k$ 溢出，再调用模板。特别是 `Lucas::choose` 与 `ExLucas::choose` 接收无符号参数，不要把容斥里的负数直接转换后传入。

素数 p，所有查询 $n\le N<p$： 用第 21.1.1 节（第 276 页）的 `Binomial<p> c(N)`，查询 `c.choose(n,k)`；排列数用 `c.permute(n,k)`。一次建表时间 $O(N+\log p)$、空间 $O(N)$，每次查询 $O(1)$。后续用 `c.init(M)` 显式扩表；每次真正扩表都做一次扩展欧几里得求逆，所以连续逐项扩表不能简单视作总共 $O(N)$。尽量一次扩至本批最大上界。此处 p 是编译期模数，调用者保证素数；模板的断言不等于判素。

素数 p，n 很大且 p 足够小： 用第 19.14.1 节（第 177 页）的 `Lucas c(p)`，再调用 `c.choose(n,k)`。完整预处理时间 $O(p+\log p)$、空间 $O(p)$，单次 $O(1+\log_p n)$。“下标大”不自动意味着适合此实现：$p=10^9+7$ 时，两张长度为 p 的 `int` 表约需 8 GB，不能靠 Lucas 定理消除这笔开销。

合数 m，所有素数幂因子的表可以放下： 用第 19.14.2 节（第 178 页）的 `ExLucas c(m)`，查询 `c.choose(n,k)`。它按 $m=\prod q_i$、$q_i=p_i^{e_i}$ 建表，不是只预处理至最大查询的 n。空间 $O(\sum q_i)$，建表时间可保守记为 $O(\sqrt{m}+\sum q_i+\sum\log q_i)$。当前 `unit` 的每一层还有一次快速幂，故单次可保守记为 $O(\sum_i(1+\log_{p_i}(n+1))\log(n+2)+\sum_i\log q_i)$，不能误写成纯粹 $O(\log n)$。模数 1 返回 0。若某个 q_i 很大，表就很大；ExLucas 不是任意 32 位合数的低内存黑盒。

三者都不合适时不要硬套。若 n 小、模数任意，Pascal 递推只用加法，算到第 n 行耗时 $O(n^2)$、滚动空间 $O(n)$；若只需 $k\le K$ 的列，可降至 $O(nK)$。若素数 p 很大、$k'=\min(k,n-k)<p$ 很小，可将连续 k' 个分子相乘，再乘 $(k'!)^{-1}$，耗时 $O(k'+\log p)$，即使 $n\ge p$ 也可用，因为分母仍是单位；这需要自行写短循环。合数模数下即便整数商存在，也不代表分母可逆，例如 $\binom{4}{2}=6\equiv 2\pmod 4$，而 $2!$ 在模 4 下没有逆元。不能把整数公式中的“除法”直接换成模逆元。

21.2 容斥原理

21.2.1 容斥与二项式反演：恰好与至少

容斥不只求并集。题目若能算出若干性质的交集，往往可以先求交集和，再恢复“恰好满足几个性质”的分布。关键是**交集和带有重数**，不能把它直接当成“至少”的数量；性质之间也不要求独立。

21.2.1.1 从交集和恢复恰好数量

设有限全集为 Ω，性质对应集合为 $A_1,\dots,A_m$。记 $q(x)=|\{i:x\in A_i\}|$，以及

$$N_t=|\{x\in\Omega:q(x)=t\}|,\qquad S_j=\sum_{\substack{J\subseteq\{1,\dots,m\}\\|J|=j}}\left|\bigcap_{i\in J}A_i\right|,\quad S_0=|\Omega|.$$

空指标集的交集是全集。满足 t 个性质的对象在 S_j 中出现 $\binom{t}{j}$ 次，故

$$\boxed{S_j=\sum_{t=j}^{m}\binom{t}{j}N_t,\qquad N_r=\sum_{j=r}^{m}(-1)^{j-r}\binom{j}{r}S_j.}$$

第二式适用于 $0\le r\le m$；范围外的恰好数量为 0。**证明**。代入第一式，N_t 的系数为

$$\sum_{j=r}^{t}(-1)^{j-r}\binom{j}{r}\binom{t}{j}=\binom{t}{r}\sum_{h=0}^{t-r}(-1)^h\binom{t-r}{h}=\begin{cases}1,&t=r,\\0,&t>r.\end{cases}$$

因此其他重数全部抵消。取 $r=0$ 即没有任何性质的数量；普通并集容斥是 $|\bigcup_i A_i|=S_0-N_0=\sum_{j=1}^{m}(-1)^{j-1}S_j$。

最小反例。全集只有一个对象且它同时满足两个性质，则 $(S_0,S_1,S_2)=(1,2,1)$，但 $(N_0,N_1,N_2)=(0,0,1)$。$S_1=2$ 不是并集大小，$S_1-S_2=1$ 也不是恰好满足一个性质的数量；正确式为 $N_1=S_1-2S_2=0$。

21.2.1.2 至少数量与变换方向

对 $1\le r\le m$，将上述 N_t 从 $t=r$ 到 m 求和，得到

$$\boxed{|\{x:q(x)\ge r\}|=\sum_{j=r}^{m}(-1)^{j-r}\binom{j-1}{r-1}S_j.}$$

其中 S_j 的系数是 $\sum_{t=r}^{j}(-1)^{j-t}\binom{j}{t}=(-1)^{j-r}\binom{j-1}{r-1}$，可用 Pascal 恒等式逐项相消证明。注意系数不再是 $\binom{j}{r}$。$r=0$ 时直接返回 S_0，$r>m$ 时返回 0，不能把负下标交给组合数公式。

不要抄反方向。上面的 $S_j=\sum_{t\ge j}\binom{t}{j}N_t$ 是有限上三角变换。常见的另一种二项式反演是

$$b_n=\sum_{k=0}^{n}\binom{n}{k}a_k\quad\Longleftrightarrow\quad a_n=\sum_{k=0}^{n}(-1)^{n-k}\binom{n}{k}b_k.$$

它是下三角变换：待求第 n 项只需 $b_0,\dots,b_n$；前者恢复 N_r 却需要 $S_r,\dots,S_m$。可用同一系数消去证明，但下标、符号和求和范围必须随对象含义重新确定。

适用范围。这些等式只有整数乘加减，没有除法，先在整数中成立，再对任意正模数取模都成立；不要求模数为素数。给每个对象一个权重也可用同样的逐对象线性证明。但模意义下的答案为 0 不表示实际对象不存在，且选用阶乘逆元实现组合数时另有条件。计算全部一般 S_j 可能需要枚举 2^m 个子集及各自交集，反演公式本身不会自动消除这个成本。只有证明交集大小仅依赖子集大小 j 后，才能用 $\binom{m}{j}$ 乘一个代表值。例如 $A_1=A_2=\{1,2\},A_3=\{3,4\}$ 的各集合都等大，但三个两两交集大小为 $2,0,0$，不能仅凭单集等大作此压缩。

21.2.1.3 例：有标号球放入有标号盒

n 个互异球各选择 m 个互异盒之一，计恰好 r 个非空盒的方案数；盒内不计顺序。不等同于无标号球的插板模型，也不额外乘 $n!$ 或除以 $m!$。先选出非空盒，再对其中的空盒做容斥，得到

$$\boxed{\binom{m}{r}\sum_{j=0}^{r}(-1)^j\binom{r}{j}(r-j)^n=\binom{m}{r}r!\left\{{n\atop r}\right\}.}$$

也可把原来 m 个盒的“为空”当作性质，此时 $S_j=\binom{m}{j}(m-j)^n$，恰好 r 个**非空**盒对应 N_{m-r}，不是 N_r。将它代入前式，并用 $\binom{j}{m-r}\binom{m}{j}=\binom{m}{r}\binom{r}{j-(m-r)}$ 即得到同一答案。右侧 Stirling 写法只是同一模型的交叉解释，不新增算法。

例如 $n=3,m=4$，恰好占用 1、2、3 个盒的数量分别为 $4,36,24$，合计 $4^3=64$。空盒性质的交集和为 $(64,108,48,4,0)$；恰好两个空盒是 $48-3\cdot4=36$，至少两个空盒却是 $48-2\cdot4=40$。这同时展示了“恰好”和“至少”的系数区别。

边界。$r<0$、$r>m$ 或 $r>n$ 时答案为 0。$n=0$ 时只有一个空映射，恰好 $r=0$ 才为 1，包括 $m=0$；$n>0,m=0$ 时无解。统一公式中的 $0^0=1$ 是空映射约定，不是浮点极限。

21.2.1.4 直接使用现有组合数与模整数

已给非负 `long long n` 和非负 `int m`、整数 `r`，固定 $p=998244353$，要求 $m<p$ 且能容纳长度 $m+1$ 的阶乘表。第 19.2.2 节（第 133 页）和第 21.1.1 节（第 276 页）可直接组合：

```
using Z = ModInt<998244353>;
Z answer = 0;
if (0 <= r && r <= m && r <= n)
{
    Binomial<998244353> c(m);
    for (int j = 0; j <= r; j++)
    {
        Z term = c.choose(r, j) * Z(r - j).pow(n);
        if (j & 1) answer -= term;
        else answer += term;
    }
    answer *= c.choose(m, r);
}
```

只需建到 m，不需要 $n!$；n 可以远大于 p，但必须在非负 signed-long-long 范围内，因为当前 `pow` 的指数类型不是 `unsigned long long`。单次建表与计算总耗时 $O(m+\log p+(r+1)\log(n+2))$、空间 $O(m)$；其中 $\log p$ 计入一次阶乘求逆，$\log(n+2)$ 包括 $n=0$ 的常数工作。不合法 r 的分支不建表。多询问可复用已建表，但要另行说明统一上界。

这里的素数限制来自 `Binomial` 用费马逆元，不来自容斥。合数模数不可直接替换该模板参数；$m\ge p$ 也不能继续使用此阶乘逆元表。应根据实际规模改用已有 Pascal、Lucas 或扩展 Lucas 路线，并重新检查各自条件。取模前后都应使用带符号的模运算处理交错和，不依赖无符号下溢。

来源与范围。恰好/至少性质数的两式核对 Richard P. Stanley, *Enumerative Combinatorics*, Volume 1 第 2 章习题 3，式 (2.39)–(2.40)；本节系数推导、反例、盒模型和接口衔接重新组织。基础容斥与概念导航参见 OI Wiki：容斥原理。本节不声称覆盖该来源页面的所有计数主题，也不计作新模板或在线评测通过。

21.3 斐波那契数列

21.3.1 斐波那契最小循环节（Pisano 周期）

代码依赖：Mod64（第 124 页，18.1 节）、PollardRho（第 154 页，19.6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

period(m) 返回精确最小正周期，1≤m≤2^64−1，m=1 返回 1；结果为 unsigned int128，可能超过 64 位。fib(k,m) 递归倍增返回 (F_k,F_{k+1}) 模 m，k 为 unsigned int128；is_period(k,m) 检查 k>0 及完整状态 (0,1%m)。prime_period(p) 仅供素数 p≥2。复用 PollardRho 分解 m 和素数候选的 p±1；逐个除因子求素数最小周期，再逐层检查素数幂，不能无条件每层乘 p。互素分量周期取 lcm。总复杂度含随机整数分解，无确定的 O(log m) 保证；每次状态检查 O(log k) 次 128 位模运算，递归空间 O(log k)，k≤128 位。对象可重复使用，factorizer 仅控制分解随机过程；$\pi(m)\le 6m$，所有周期算术使用 128 位。

```
struct FibonacciPeriod
{
    using ull = unsigned long long;
    using u128 = __uint128_t;
    PollardRho factorizer;

    static pair<ull, ull> fib(u128 k, ull m)
    {
        assert(m >= 1);
        if (k == 0)
            return {0, 1 % m};
        auto [a, b] = fib(k / 2, m);
        ull c = Mod64::mul(a, (u128(2) * b + m - a) % m, m);
        ull d = (u128(Mod64::mul(a, a, m)) + Mod64::mul(b, b, m)) % m;
        if (k & 1)
            return {d, (u128(c) + d) % m};
        return {c, d};
    }

    static bool is_period(u128 k, ull m)
    {
        return k > 0 && fib(k, m) == make_pair(0ULL, 1 % m);
    }
```

素数候选缩减、素数幂逐层检查与互素 lcm（接上页同一结构体）：

```
    u128 prime_period(ull p)
    {
        if (p == 2)
            return 3;
        if (p == 5)
            return 20;
        bool square = p % 5 == 1 || p % 5 == 4;
        ull base = square ? p - 1 : p + 1;
        u128 t = u128(base) * (square ? 1 : 2);
        auto factors = factorizer.factor(base);
        if (!square)
            factors.push_back(2);
        for (ull q : factors)
            while (t % q == 0 && is_period(t / q, p))
                t /= q;
        return t;
    }

    static u128 gcd128(u128 a, u128 b)
    {
        while (b)
        {
            u128 r = a % b;
            a = b;
            b = r;
        }
        return a;
    }

    u128 period(ull m)
    {
        assert(m >= 1);
        auto factors = factorizer.factor(m);
        u128 answer = 1;
        for (int i = 0; i < (int)factors.size();)
        {
            ull p = factors[i];
            ull power = p;
            u128 t = prime_period(p);
            i++;
            while (i < (int)factors.size() && factors[i] == p)
            {
                power *= p;
                if (!is_period(t, power))
                    t *= p;
                i++;
            }
            answer = answer / gcd128(answer, t) * t;
        }
        return answer;
    }
};
```

模板题使用：Luogu P4994 / 终于结束的起点

题意：输入单个模数 2≤M≤706150，输出最小 n>0 使 F_n=0、F_(n+1)=1（模 M）。π(M)≤6M，题目范围内可安全转 unsigned long long。独立模板问题；题面与样例已核对，无本项目线上 AC 或排名。

所需模板：Mod64（第 124 页，18.1 节）、Prime64（第 149 页，19.3.1 节）、PollardRho（第 154 页，19.6.1 节）、FibonacciPeriod（第 281 页，21.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4994

```
int main()
{
    unsigned long long m;
    cin >> m;
    FibonacciPeriod solver;
    cout << (unsigned long long)solver.period(m) << '\n';
    return 0;
}
```

接口用法演示：完整 64 位模数与 128 位最小周期

演示约定：输入 T 与 T 个正 unsigned64 模数，逐行输出精确最小周期，使用 128 位十进制转换。包含 m=1 的约定；m=2*5^27 时答案 89406967163085937500 超过 64 位。API 演示，不是线上通过记录。

所需模板：Mod64（第 124 页，18.1 节）、Prime64（第 149 页，19.3.1 节）、PollardRho（第 154 页，19.6.1 节）、FibonacciPeriod（第 281 页，21.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/math/combinatorics/fibonacci/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    FibonacciPeriod solver;
    while (tests--)
    {
        unsigned long long m;
        cin >> m;
        auto answer = solver.period(m);
        string output;
        do
        {
            output.push_back('0' + answer % 10);
            answer /= 10;
        }
        while (answer);
        reverse(output.begin(), output.end());
        cout << output << '\n';
    }
    return 0;
}
```

接口用法演示：kuangbin HDU3977 来源程序协议

演示约定：按完整来源程序输入 T 及正 int 模数，输出 Case #k: 周期。来源初值 (1,1) 与标准 (0,1) 仅相差一次可逆转移，周期相同。原 PDF 的注释吞掉左括号、旧 MSVC 输出格式须显式兼容修改；官方题面不可访问，故仅归为来源协议 API，不声称正式题目或 AC。

所需模板：Mod64（第 124 页，18.1 节）、Prime64（第 149 页，19.3.1 节）、PollardRho（第 154 页，19.6.1 节）、FibonacciPeriod（第 281 页，21.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://acm.hdu.edu.cn/showproblem.php?pid=3977

```
int main()
{
    int tests;
    cin >> tests;
    FibonacciPeriod solver;
    for (int i = 1; i <= tests; i++)
    {
        int m;
        cin >> m;
        cout << "Case #" << i << ": ";
        cout << (unsigned long long)solver.period(m) << '\n';
    }
    return 0;
}
```

21.4 卡特兰数

21.4.1 Catalan 与投票问题：把前缀限制变成反射双射

先问限制的是最终数量，还是每个前缀。把一种操作记为 $+1$，另一种记为 -1，高度就是两种操作数量之差。固定 p 个正步、q 个负步，共有 $\binom{p+q}{q}$ 个次序；这只固定终点，未限制中途高度。合法括号串要求高度始终非负且最终为零；从空栈开始的入栈/出栈操作序列也有同样条件。这只计操作类型序列，不自动计带标签元素的任意排列、操作耗时或额外栈约束。

21.4.1.1 弱前缀条件：减去首次走到负一的坏路径

假设 $p \ge q \ge 0$，从高度 0 出发，每个非空前缀高度都要求 ≥ 0。坏路径必然有第一次到达 -1 的时刻；只把从起点到该时刻的所有正负步交换，后缀不变。该前缀原来负步比正步多 1，交换后总正步变成 $p+1$、负步变成 $q-1$。反过来，任意这样的新路径终点为 $p-q+2 \ge 2$，必然第一次到达 $+1$；把到此处的前缀再交换，就唯一恢复一条坏路径。因此这是双射，而不只是“看起来对称”，合法数为

$$W(p,q) = \binom{p+q}{q} - \binom{p+q}{q-1}.$$

约定 $\binom{N}{k} = 0$ 当 $k < 0$ 或 $k > N$。若 $p < q$，终点已非法，答案直接为 0，不能继续使用可能给负数的差式。$p = q = 0$ 的空序列计 1。反射必须截止于第一次越界；翻转整条路径或任意一个越界前缀都不构成上述逆映射。

取 $p = q = n$，得到括号串、Dyck 路径的 Catalan 数

$$C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1}\binom{2n}{n}, \qquad C_0 = 1.$$

这里 $\binom{2n}{n-1} = \binom{2n}{n+1}$，与前文差分公式相同。这些经典计数对象可参见 Stanley 的 Catalan 习题 19(h),(i)；使用时仍需先证明题目对象与路径一一对应。

21.4.1.2 严格领先：先取走必需的第一步

若要求每个非空前缀高度 > 0，且 $p > q$，第一步必须为正步。删去这一步，并将剩余路径的高度整体减 1，得到 $p-1$ 个正步、q 个负步的弱条件，故

$$S(p,q) = \binom{p+q-1}{q} - \binom{p+q-1}{q-1} = \frac{p-q}{p+q}\binom{p+q}{q}.$$

当 $p+q > 0$ 且 $p \le q$ 时答案为 0；空路径若按“所有非空前缀”定义，则真空成立、单独计 1，不能把它代入含 $p+q$ 分母的式子。例如 $p = 3, q = 2$：弱条件有 5 种，严格条件只有 $+++--$ 和 $++-+-$ 两种。“过程中可以打平”和“始终领先”不是同一条件，不能只检查最终 $p > q$。

21.4.1.3 初始有余额：反射起点，而非只改终点

从整数高度 $h \geq 0$ 出发，走 $L \geq 0$ 个 ± 1 步，到达 $e \geq 0$，全程不低于 0。先检查 $|e-h| \leq L$ 且 $L+e-h$ 为偶数，否则答案为 0。令 $k=(L+e-h)/2$ 为正步数，总路径数为 $\binom{L}{k}$。坏路径第一次到达 -1 之前，按高度变换 $y \mapsto -2-y$ 反射前缀，把起点 h 变成 $-h-2$，越界点 -1 不动，终点仍为 e。由于新起点在 -1 下方、终点非负，每条新路径都必经 -1，在第一次到达处反射回来就是逆映射。新路径正步数为 $k+h+1$，故

$$B(L,h,e)=\binom{L}{k}-\binom{L}{k+h+1}.$$

例：$L=4,h=2,e=0$，四种无约束次序中只有 $---+$ 越界，答案为 $4-1=3$。$L=0$ 时只有 $h=e$ 的空路径计 1。若下界为整数 b，先把起终点减去 b；若要求整数高度严格大于 b，则下界应取 $b+1$。

适用边界。单个常数下界和单位 ± 1 步使“第一次碰到边界”可逆。再加上界、禁止位置、加权路径、非单位步长或斜率边界时，不能照搬两项差式；可能需要状态 DP、重复反射或另一种专门计数。单步能跳过 -1 时，上述首次命中论证已经失效。

21.4.1.4 取模时优先使用差式，除法条件不能省

差式只要求能正确计算两个组合数，不要求 $n+1$、$p+q$ 可逆。例如 $C_2=2$，模 6 仍为 2；把 $\binom{4}{2}$ 先取模得到 0，再除以 3 无法恢复答案，因为 3 没有模 6 逆元。即使模数是素数，分母也可能被模数整除。

选择见第 278 页（21.1.3 节）：素数模数且所需最大上标小于模数时可用 Binomial（第 276 页，21.1.1 节）；跨过素数模数时考虑 Lucas（第 177 页，19.14.1 节），合数模数可按资源预算考虑 ExLucas（第 178 页，19.14.2 节）。后两者都有预处理空间/时间限制，不意味着任意大模数都可直接建立表。先以有符号或更宽类型检查负下标、不可达性及 $p+q, 2n, L+e-h, k+h+1$ 的溢出，再调用接收无符号参数的接口；最后将两个模值之差规范化。

21.5 斯特林数与斯特林反演

21.5.1 定义与性质

设 $n,k\ge 0$。无符号第一类 $c(n,k)$ 计数 n 个带标号元素的排列中，恰有 k 个不区分顺序的循环的方案数；循环内只忽略起点，方向不等同。第二类 $S(n,k)$ 计数把 n 个带标号元素划分为 k 个无标号非空集合的方案数。有符号第一类定义为 $s(n,k)=(-1)^{n-k}c(n,k)$。例如 $c(3,1)=2$、$c(3,2)=3$，而 $S(3,1)=1$、$S(3,2)=3$。

加入第 n 个元素：第一类可以自成一圈，或者插在已有 $n-1$ 个元素后；第二类可以自成一个块，或者加入已有 k 个块。因此

$$c(n,k)=c(n-1,k-1)+(n-1)c(n-1,k),\qquad S(n,k)=S(n-1,k-1)+kS(n-1,k),$$

$$s(n,k)=s(n-1,k-1)-(n-1)s(n-1,k).$$

三者的 $(0,0)$ 项均为 1；$n>0$ 的第零列为 0；$k>n$ 或任一下标为负时为 0。特别地 $c(n,n)=S(n,n)=1$，$n\ge 1$ 时 $c(n,1)=(n-1)!$、$S(n,1)=1$。

记 $x^{\underline{k}}=x(x-1)\cdots(x-k+1)$，$x^{\overline{k}}=x(x+1)\cdots(x+k-1)$，零次幂均为 1。三种基底转换是

$$x^{\overline{n}}=\sum_{k=0}^{n}c(n,k)x^k,\qquad x^{\underline{n}}=\sum_{k=0}^{n}s(n,k)x^k,\qquad x^n=\sum_{k=0}^{n}S(n,k)x^{\underline{k}}=\sum_{k=0}^{n}(-1)^{n-k}S(n,k)x^{\overline{k}}.$$

固定列的指数生成函数为

$$\sum_{n\ge k}c(n,k)\frac{x^n}{n!}=\frac{(-\log(1-x))^k}{k!},\quad \sum_{n\ge k}s(n,k)\frac{x^n}{n!}=\frac{(\log(1+x))^k}{k!},\quad \sum_{n\ge k}S(n,k)\frac{x^n}{n!}=\frac{(e^x-1)^k}{k!}.$$

按循环/非空集合的标号集合构造可分别得到前后两式。第二类还有普通生成函数 $\sum_{n\ge k}S(n,k)x^n=x^k/\prod_{j=1}^{k}(1-jx)$。双变量指数生成函数分别为 $\sum c(n,k)y^kx^n/n!=(1-x)^{-y}$、$\sum S(n,k)y^kx^n/n!=\exp(y(e^x-1))$。这些等式先在有理数形式幂级数中成立；模素数 p 计算到 N 次时需 $N<p$，确保出现的整数和阶乘可逆。以下快速实现还须满足 NTT 长度限制。

21.5.1.1 行列算法的选择与推导

第一类整行用 $F_n(x)=x^{\overline{n}}$，递归一次求 F_m，再用 $F_{2m}(x)=F_m(x)F_m(x+m)$；奇数长度额外乘 $x+2m$。泰勒平移用逆阶乘卷积，递推 $T(n)=T(\lfloor n/2\rfloor)+O(n\log n)$，所以时间 $O(n\log n)$、空间 $O(n)$。第二类整行使用容斥 $S(n,k)=\sum_{i=0}^{k}(i^n/i!)((-1)^{k-i}/(k-i)!)$，一次卷积获得前缀。$0^0=1$ 单独处理，正底数的指数才对 $p-1$ 取模。

整列先取出最低次 x^k，只算 $m=N-k+1$ 项：第一类的单位常数项级数 $A(x)=\sum_{j\ge 0}x^j/(j+1)$，第二类为 $A(x)=\sum_{j\ge 0}x^j/(j+1)!$。用 $A^k=\exp(k\log A)$ 求幂，再令 $c(n,k)$ 或 $S(n,k)=[x^{n-k}]A^k\cdot n!/k!$。从 $n=k$ 的倍率 1 逐次乘 $n+1$，无需另求 $k!$。时间 $O(N+m\log m)$、空间 $O(N)$；$k=0$ 和 $k>N$ 直接按边界返回。

21.5.2 无符号第一类斯特林数：同一行

代码依赖：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

stirling_first_row<mod,primitive>(n) 返回无符号 c(n,0..n)，默认模 998244353、原根 3。c(n,k) 为 n 元排列恰有 k 个循环的个数。n≥0，n+1≤NTT 的 max_size；n=0 返回 {1}。倍增配合多项式平移计算 x(x+1)⋯(x+n-1)，O(n log n) 时间、O(n) 空间。带符号版本再乘 (-1)^(n-k)。依赖 NttConvolution；模 167772161 可用 <167772161>。

```
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>> stirling_first_row(int n)
{
    using N = NttConvolution<mod, primitive>;
    assert(n >= 0 && n < N::max_size);
    using Z = typename N::Z;
    using Poly = typename N::Poly;
    Poly fact(n + 1), inv(n + 1);
    fact[0] = 1;
    for (int i = 1; i <= n; i++) fact[i] = fact[i - 1] * i;
    inv[n] = fact[n].inv();
    for (int i = n; i > 0; i--) inv[i - 1] = inv[i] * i;
    function<Poly(int)> solve = [&](int len) -> Poly
    {
        if (len == 0) return {1};
        int m = len / 2;
        Poly f = solve(m);
        Poly a(m + 1), b(m + 1);
        Z power = 1;
        for (int i = 0; i <= m; i++)
        {
            a[m - i] = f[i] * fact[i];
            b[i] = power * inv[i];
            power = power * m;
        }
        auto c = N::multiply(a, b);
        for (int i = 0; i <= m; i++) a[i] = c[m - i] * inv[i];
        f = N::multiply(f, a);
        if (len % 2)
        {
            f.push_back(0);
            for (int i = len; i > 0; i--) f[i] = f[i - 1] + f[i] * (len - 1);
            f[0] = f[0] * (len - 1);
        }
        return f;
    };
    return solve(n);
}
```

模板题使用：Luogu P5408

题意：输出无符号第一类斯特林数 c(n,0..n)，即 n 元排列按循环个数计数。本题模数为 167772161，须显式传模板参数；无需乘交替符号。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、stirling_first_row（第 289 页，21.5.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5408

```
int main()
{
    int n;
    scanf("%d", &n);
    auto row = stirling_first_row<167772161>(n);
    for (int i = 0; i <= n; i++) printf("%d%c", row[i].v, i == n ? '\n' : ' ');
}
```

21.5.3 无符号第一类斯特林数：同一列

代码依赖：FpsPower（第 264 页，20.7.3 节）、ModInt（第 133 页，19.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

stirling_first_column(N,k) 返回无符号 c(0..N,k)，固定模 998244353。N>=0、N<2^22、k>=0；k>N 返回全零，k=0 仅第零项为 1。取生成函数的最低 x^k 后，用 FpsPower 求剩余单位级数的 k 次幂，逐项乘 n!/k!；第一类若需有符号数，另乘 (-1)^(n-k)。令 m=N-k+1，合法非零列时间 O(N+m log m)，空间 O(N)。所有分母小于模数；NTT 工作长度由 N 限制保证。依赖 FpsPower 及其逆、对数、指数和 NTT 链，不能直接改用 167772161。

```
vector<ModInt<998244353>> stirling_first_column(int n, int k)
{
    using F = FpsPower;
    using Z = F::Z;
    assert(n >= 0 && n < F::max_size / 2 && k >= 0);
    vector<Z> answer(n + 1);
    if (k > n)
        return answer;
    if (k == 0)
    {
        answer[0] = 1;
        return answer;
    }
    vector<Z> a(n - k + 1);
    vector<Z> inv(n - k + 2);
    inv[1] = 1;
    for (int i = 2; i < (int)inv.size(); i++)
        inv[i] = Z(F::mod - F::mod / i) * inv[F::mod % i];
    for (int i = 0; i < (int)a.size(); i++)
        a[i] = inv[i + 1];
    auto power = F::power(a, to_string(k), a.size());
    Z scale = 1;
    for (int i = k; i <= n; i++)
    {
        answer[i] = power[i - k] * scale;
        scale *= i + 1;
    }
    return answer;
}
```

模板题使用：Library Checker Signed First Stirling Fixed K

题意：输入 N K，0<=K<=N<=500000，模 998244353；只输出 n=K..N 的有符号第一类 s(n,K)。库函数返回无符号 c(n,K)，逐项乘 (-1)^(n-K)，勿遗漏符号。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、FpsInverse（第 248 页，20.5.1 节）、FpsFunctions（第 261 页，20.7.2 节）、FpsPower（第 264 页，20.7.3 节）、stirling_first_column（第 291 页，21.5.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/stirling_number_of_the_first_kind_fixed_k

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    auto column = stirling_first_column(n, k);
    for (int i = k; i <= n; i++)
    {
        auto value = column[i];
        if ((i - k) % 2)
            value = decltype(value)(0) - value;
        cout << value.v << (i == n ? '\n' : ' ');
    }
}
```

21.5.4 第二类斯特林数：同一行前缀

代码依赖：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

stirling_second_row<mod,primitive>(n,m) 返回 S(n,0..m)，默认 mod=998244353、primitive=3。模数必须是素数，primitive 是原根。n 支持完整 unsigned long long；2m+1 不能超过 NTT 的 max_size。S(n,k) 为无标号非空集合划分数；n=0 时仅第零项为 1，k>n 为零。一次卷积配合逆阶乘，O(m log m + m log mod) 时间、O(m) 空间。依赖 NttConvolution；模 167772161 可直接使用模板参数 <167772161>。

```
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>> stirling_second_row(unsigned long long n, int m)
{
    using N = NttConvolution<mod, primitive>;
    assert(m >= 0 && m <= (N::max_size - 1) / 2);
    using Z = typename N::Z;
    typename N::Poly a(m + 1), b(m + 1);
    b[0] = 1;
    for (int i = 1; i <= m; i++) b[i] = b[i - 1] * i;
    b[m] = b[m].inv();
    for (int i = m; i > 0; i--) b[i - 1] = b[i] * i;
    for (int i = 0; i <= m; i++)
    {
        Z value = i ? Z(i).pow(n % (mod - 1)) : Z(n == 0);
        a[i] = value * b[i];
        if (i % 2) b[i] = Z(0) - b[i];
    }
    auto answer = N::multiply(a, b);
    answer.resize(m + 1);
    return answer;
}
```

模板题使用：Luogu P5395

题意：输出第二类斯特林数 S(n,0..n)，即把 n 个不同元素分成各个数目的非空无标号集合的方案数。本题模数为 167772161，须显式传模板参数。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、stirling_second_row（第 293 页，21.5.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5395

```
int main()
{
    int n;
    scanf("%d", &n);
    auto row = stirling_second_row<167772161>(n, n);
    for (int i = 0; i <= n; i++) printf("%d%c", row[i].v, i == n ? '\n' : ' ');
}
```

21.5.5 第二类斯特林数：同一列

代码依赖：FpsPower（第 264 页，20.7.3 节）、ModInt（第 133 页，19.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

stirling_second_column(N,k) 返回 S(0..N,k)，固定模 998244353。N>=0、N<2^22、k>=0；k>N 返回全零，k=0 仅第零项为 1。取生成函数的最低 x^k 后，用 FpsPower 求剩余单位级数的 k 次幂，逐项乘 n!/k!；令 m=N-k+1，合法非零列时间 O(N+m log m)，空间 O(N)。所有分母小于模数；NTT 工作长度由 N 限制保证。依赖 FpsPower 及其逆、对数、指数和 NTT 链，不能直接改用 167772161。

```
vector<ModInt<998244353>> stirling_second_column(int n, int k)
{
    using F = FpsPower;
    using Z = F::Z;
    assert(n >= 0 && n < F::max_size / 2 && k >= 0);
    vector<Z> answer(n + 1);
    if (k > n)
        return answer;
    if (k == 0)
    {
        answer[0] = 1;
        return answer;
    }
    int m = n - k + 1;
    vector<Z> inverse_fact(m + 1, 1), a(m);
    for (int i = 1; i <= m; i++)
        inverse_fact[i] = inverse_fact[i - 1] * i;
    inverse_fact[m] = inverse_fact[m].inv();
    for (int i = m; i > 0; i--)
        inverse_fact[i - 1] = inverse_fact[i] * i;
    for (int i = 0; i < m; i++)
        a[i] = inverse_fact[i + 1];
    auto power = F::power(a, to_string(k), m);
    Z scale = 1;
    for (int i = k; i <= n; i++)
    {
        answer[i] = power[i - k] * scale;
        scale *= i + 1;
    }
    return answer;
}
```

模板题使用：Library Checker Second Stirling Fixed K

题意：输入 N K，0<=K<=N<=500000，模 998244353；只输出 n=K..N 的第二类 S(n,K)。K=0 时输出首项 1、其余 0；不输出 n<K 的前导零。

所需模板：ModInt（第 133 页，19.2.2 节）、NttConvolution（第 231 页，20.2.1 节）、FpsInverse（第 248 页，20.5.1 节）、FpsFunctions（第 261 页，20.7.2 节）、FpsPower（第 264 页，20.7.3 节）、stirling_second_column（第 294 页，21.5.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/stirling_number_of_the_second_kind_fixed_k

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    auto column = stirling_second_column(n, k);
    for (int i = k; i <= n; i++)
    {
        auto value = column[i];
        cout << value.v << (i == n ? '\n' : ' ');
    }
}
```

21.5.6 斯特林反演与应用

对下标从 0 开始的任意序列，

$$b_n=\sum_{k=0}^{n}S(n,k)a_k \quad\Longleftrightarrow\quad a_n=\sum_{k=0}^{n}s(n,k)b_k=\sum_{k=0}^{n}(-1)^{n-k}c(n,k)b_k.$$

若正变换使用无符号第一类，则相应逆变换为

$$b_n=\sum_{k=0}^{n}c(n,k)a_k \quad\Longleftrightarrow\quad a_n=\sum_{k=0}^{n}(-1)^{n-k}S(n,k)b_k.$$

符号不能省略，也不能把正逆公式中的 n,k 对调。将 $x^n=\sum_k S(n,k)x^{\underline{k}}$ 再展开下降阶乘并比较 x^j 系数，得到 $\sum_{k=j}^{n}S(n,k)s(k,j)=\delta_{n,j}$。两矩阵均为单位下三角矩阵，故这个右逆也是左逆，即 $\sum_{k=j}^{n}s(n,k)S(k,j)=\delta_{n,j}$，证明上述第一对公式。用对角矩阵 $D_{n,n}=(-1)^n$，有 $s=DcD$，得到第二对公式。这些逆关系在整数上成立，取任意模数仍成立；只有快速行列算法的实现要求素数和逆元。

21.5.6.1 手算检查与建模用法

取 $a_0,a_1,a_2,a_3=(1,2,3,4)$，第二类变换为 $b=(1,2,5,15)$。逆变换检查 $a_2=b_2-b_1=3$，$a_3=b_3-3b_2+2b_1=4$。以 k 为块数的无标号集合划分，若给一个含 k 块的划分赋权 a_k，总权即 $b_n=\sum_k S(n,k)a_k$；已知这些总权时，可以用带符号第一类恢复 a_n。例如 $a_k=1$ 得到 Bell 数 b_n，逆变换应恢复全 1 序列。若每个块还要独立涂一种颜色，则取 $a_k=q^k$。

在指数生成函数层面，令 $A(x)=\sum a_nx^n/n!$、$B(x)=\sum b_nx^n/n!$，第一对公式等价于 $B(x)=A(e^x-1)$ 与 $A(x)=B(\log(1+x))$。这是形式复合，不能把它误写成逐项乘积或一次普通卷积。四种快速行列模板不自动给出任意整个序列的快速斯特林变换；逐行点积适合少量行或小规模示例，不能冒称 $O(N\log N)$ 的完整变换。

21.5.6.2 题目协议与验证范围

整行的洛谷 P5408/P5395 使用素数 167772161；本节整列函数固定 998244353，不能只改调用处的类型就用于 P5409/P5396。Library Checker 第一类 fixed-k 题要求**有符号** $s(n,k)$，第二类要求 $S(n,k)$；两题只输出 $n=k,\dots,N$。紧随各函数的完整用法分别展示这些差异。原整行实现的洛谷历史 AC：第一类 P5408与第二类 P5395。新增整列已在 Library Checker 以 C++20 实际 AC：第一类 409999（4457ms）、第二类 410000（4407ms），均 14 点全过，时限 10 秒。递推对拍 69158 项、反演矩阵恒等式与 50 万规模边界通过，头文件及精确抄写版本均通过普通与 ASan/UBSan 检查。本轮整行也以当前源码通过 Library Checker：第一类 410002（589ms）、第二类 410001（392ms）。性能提醒：本地同编译器 C++17/O2、最大规模小 k 输入交替各 3 次，整列中位耗时约为官方参考的 6.06 倍（第一类）与 4.92 倍（第二类），超过 80% 慢速提醒线。复用通用幂级数便于抄写，但常数较大；此比较不是 OJ 排名。算法及公式参考OI Wiki 斯特林数，本库完整状态见 issue 18 与 verification/stirling-columns*.json。

21.6 分拆数

21.6.1 整数分拆与重数上限

Partitions(N,mod) 求无序正整数分拆，0≤N<INT_MAX、1≤mod≤INT_MAX，p[n] 为不限制重数的分拆数；p[0]=1%mod。预处理 O(N$\sqrt{N}$) 时间、O(N) 空间；模数不要求素数。limited(n,k) 限制每个正整数最多出现 k 次，0≤n≤N、0≤k≤INT_MAX；k=0 时仅空分拆存在，k=1 为互异分拆。单次 O(1+$\sqrt{(n/(k+1))}$)，额外空间 O(1)。p/mod 只读；中间五边形数与累加采用 long long。

```
struct Partitions
{
    int mod;
    vector<int> p;

    Partitions(int n, int mod) : mod(mod), p(n + 1)
    {
        assert(n >= 0 && mod >= 1);
        p[0] = 1 % mod;
        for (int i = 1; i <= n; i++)
        {
            long long value = 0;
            for (long long j = 1; j * (3 * j - 1) / 2 <= i; j++)
            {
                long long a = j * (3 * j - 1) / 2;
                long long b = j * (3 * j + 1) / 2;
                int sign = j % 2 ? 1 : -1;
                value += 1LL * sign * p[i - a];
                if (b <= i) value += 1LL * sign * p[i - b];
            }
            p[i] = (value % mod + mod) % mod;
        }
    }

    int limited(int n, int k) const
    {
        assert(0 <= n && n < (int)p.size() && k >= 0);
        long long step = 1LL + k;
        long long value = p[n];
        for (long long j = 1; j * (3 * j - 1) / 2 <= n / step; j++)
        {
            long long a = step * (j * (3 * j - 1) / 2);
            long long b = step * (j * (3 * j + 1) / 2);
            int sign = j % 2 ? -1 : 1;
            value += 1LL * sign * p[n - a];
            if (b <= n) value += 1LL * sign * p[n - b];
        }
        return (value % mod + mod) % mod;
    }
};
```

模板题使用：Library Checker partition_function

题意：输出 0..N 的全部整数分拆数，模 998244353，N≤500000。构造 Partitions 后直接读 p；p[0]=1 表示空分拆。五边形数递推 O(N$\sqrt{N}$) 时间、O(N) 空间，不是有序拆分。此用法不调用 limited。

所需模板：Partitions（第 297 页，21.6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/partition_function

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    Partitions part(n, 998244353);
    for (int i = 0; i <= n; i++) cout << part.p[i] << (i == n ? '\n' : ' ');
}
```

应用题补充：Luogu P6189 [NOI Online #1 入门组] 跑步

题意：正整数非增序列的总和为 n，等价于 n 的无序分拆。n≤10^5，1≤p<2^{30} 且不保证素数；直接构造 Partitions(n,p)，输出 p[n]。这是比赛应用，单独记录，不替代正式模板题。

所需模板：Partitions（第 297 页，21.6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6189

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, p;
    cin >> n >> p;
    Partitions part(n, p);
    cout << part.p[n] << '\n';
}
```

21.6.2 整数分拆：重数限制的差一问题

实现见第 21.6.1 节（第 297 页）。分拆不区分加数顺序，部分均为正整数。无约束生成函数 $P(z)=\prod_{i\ge1}(1-z^i)^{-1}$，空分拆给出 $p(0)=1$。五边形数定理为

$$\prod_{i\ge1}(1-z^i)=1+\sum_{j\ge1}(-1)^j\left(z^{j(3j-1)/2}+z^{j(3j+1)/2}\right).$$

与 $P(z)$ 相乘后比较系数，得到 $p(n)$ 的递推，符号按 $+,+,-,-,+,+,\dots$ 成对出现。每项只依赖更小的下标，每个 n 有 $O(\sqrt{n})$ 个非零偏移。实现先用 64 位累加，再归一化到模数范围；无需逆元，合数模数与模数 1 均可使用。

每个正整数最多出现 k 次的生成函数是

$$\prod_{i\ge1}(1+z^i+\cdots+z^{ki})=P(z)\prod_{i\ge1}(1-z^{(k+1)i}).$$

因此用已有 p 表与放大 $k+1$ 倍的五边形偏移相乘，符号为 $-,-,+,+,\dots$。本库 `limited(n,k)` 的上限包含 k；$k=0$ 时非零 n 无解，$k=1$ 为互异分拆。kuangbin 的注释写“不超过 k 个”，但其 `solve(n,k)` 使用放大 k 倍的公式，实际表示每种数少于 k 次，对应本库 `limited(n,k-1)`（$k\ge1$）。HDU 原题页面本次无法读取，不能由这条注释推断原题措辞。

两套实现以任意精度完全背包验证到 1000，以逐种部分、逐个重数枚举验证有界情况到 100，覆盖素数与合数模数、0/1 重数限制和大于总和的限制；另检查 $N=100000$ 的边界。普通与 ASan/UBSan 均通过，暂无在线 AC。

Library Checker 的分拆函数要求输出 $p(0),\dots,p(N)$，$N\le500000$，完整用法见第 21.6.1 节（第 298 页）；固定官方 11 组数据的本地 checker 已通过。P6189 将非增跑步距离序列对应到无序分拆，模数不保证素数，用法见第 21.6.1 节（第 298 页）。它是比赛应用，不替代正式模板题。额外独立参考将部分分成不超过 B 与大于 B 两组：前者完全背包，后者按恰好 j 个部分维护 $g_j(s)=g_j(s-j)+g_{j-1}(s-B-1)$，再卷积。取 $B=\lfloor\sqrt{N}\rfloor$，时间 $O(N\sqrt{N})$、滚动数组空间 $O(N)$，不复用被测的五边形数递推；覆盖 $N=10^5$ 的合数模数与模数 1。这两份用法只调用构造和 p 表，不代表 limited 接口已有正式题的线上验证。

来源：kuangbin 2.11；https://oi-wiki.org/math/combinatorics/partition/。

21.6.2.1 三个容易混淆的限制

令 $A_k(n)$ 为 n 的分拆中每个部分不超过 k 的个数。Ferrers 图每行长度为一个部分；交换行列是自身的逆变换，交换最大部分与部分总数。因此 $A_k(n)$ 也等于至多 k 个部分的分拆数，但两类分拆集合通常不同。其生成函数为 $\prod_{i=1}^{k}(1-z^i)^{-1}$，可用部分大小 $1,\dots,k$ 的完全背包计算。

恰好 k 个正部分时，从每个部分减去 1，删除零，得到 $n-k$ 的至多 k 部分分拆；反向用零补足到 k 项，再每项加 1，故答案是 $A_k(n-k)$。若还要求这 k 个部分互异，按非增序写为 $\lambda_1 > \cdots > \lambda_k$，减去阶梯 $(k, k-1, \dots, 1)$，得到弱降非负序列，答案为

$$A_k\left(n - \frac{k(k+1)}{2}\right).$$

以上默认 $n, k \geq 0$，负目标和的答案为零；$A_0(0) = 1$，$A_0(n) = 0$ $(n > 0)$。计算阶梯和时须先扩宽乘法，int 范围的 k 可写 `1LL*k*(k+1LL)/2`。空分拆是恰好零个部分的唯一对象。

这不同于本库 `limited(n,k)`：它限制每个值的出现次数，不限制部分总数。例如 $n = 5, k = 2$：至多两部分有 $(5), (4,1), (3,2)$ 共 3 种；每部分不超过 2 也有 3 种，恰好两部分只有 2 种；重数限制的整数计数为 5，`limited(5,2)` 返回它对构造模数的余数。对于 $n = 6, k = 2$，恰好两部分有 3 种，其中互异的只有 2 种。不能只看题面出现“不超过 k”就调用同一个接口。

21.6.2.2 重数限制与禁止倍数部分

设 $d = k + 1 \geq 1$，则

$$\prod_{i\geq 1} \frac{1-z^{di}}{1-z^i} = \prod_{\substack{i\geq 1 \\ d\nmid i}} \frac{1}{1-z^i}.$$

左边限制每个值至多出现 k 次，右边只允许不被 d 整除的部分，次数不限。等式来自消去分母中下标为 d 倍数的因子。取 z^n 系数时只涉及有限多个因子，故这是形式级数恒等式，不要求数值收敛。因此可用排除 d 倍数面值的完全背包独立核对 `limited`。$k = 1$ 给出互异分拆数等于奇数部分分拆数；$k = 0$ 则所有正部分被禁止，只剩空分拆。若 $k \geq n$，重数限制不再生效。以上整数计数等式可对任意正模数取模，不需模逆元。生成函数与奇数/互异特例可参考 MIT 18.212 第 20 讲；这里将消因子推导写成一般 d。原五边形数定理保留为引用定理，本节不展开 Franklin 对合证明。

本节的 A_k、恰好部分数及阶梯变换是建模公式，不是新增的 `Partitions` 成员函数。

21.7 Pólya 计数

21.7.1 Burnside 平均：任意模数下的精确除法

BurnsideAverage(g,p)：群阶 g>0，目标模数 p>0，要求 g*p<=LLONG_MAX/2（先用 int128 检查）。mod=g*p 供不动点计数使用；add(fixed,multiplicity=1) 累加非负 long long 计数与重数，内部 int128 乘法。power(a,k) 求 a^k 模 mod，a,k 非负，0^0=1。result() 返回 sum/g，支持 p=1 与 g 不可逆；sum 必须是对全部群元素的不动点计数之和模 mod，不是先模 p 后的计数。调用方保证确为群作用并完整枚举群元素或共轭类型，重数合计 g；不会生成闭包或验证群公理。字段只读，add 后可继续累计，只有完整合法总和才能 result。每次 add/result 为 O(1)，power 为 O(log(k+1))，空间 O(1)。

```
struct BurnsideAverage
{
    using ll = long long;
    ll order, mod, sum = 0;

    BurnsideAverage(ll g, ll p) : order(g)
    {
        assert(g > 0 && p > 0 && __int128(g) * p <= LLONG_MAX / 2);
        mod = g * p;
    }

    ll power(ll a, long long k) const
    {
        assert(a >= 0 && k >= 0);
        a %= mod;
        ll ans = 1 % mod;
        while (k)
        {
            if (k & 1) ans = __int128(ans) * a % mod;
            a = __int128(a) * a % mod;
            k >>= 1;
        }
        return ans;
    }

    void add(ll fixed, ll multiplicity = 1)
    {
        assert(fixed >= 0 && multiplicity >= 0);
        sum = (sum + __int128(fixed) * multiplicity) % mod;
    }

    ll result() const
    {
        assert(sum % order == 0);
        return sum / order;
    }
};
```

应用题补充：P1446 [HNOI2008] Cards

题意：三色各用 r/b/g 张，均 <=20；m<=60，m+1<p<100 且 p 为素数。输入置换与恒等构成完整群；补恒等并去重，绝不能把任意生成元直接平均。每循环整体染一种颜色，用二维背包计红蓝数量，绿色由已处理点数推得；中间模群阶 *p。

所需模板：BurnsideAverage（第 301 页，21.7.1 节）、permutation_cycles（第 304 页，21.7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1446

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int r, b, g, m, mod;
    cin >> r >> b >> g >> m >> mod;
    int n = r + b + g;
    vector<vector<int>> group(m, vector<int>(n));
    for (auto &p : group)
    {
        for (int &x : p)
        {
            cin >> x;
            x--;
        }
    }
    vector<int> identity(n);
    iota(identity.begin(), identity.end(), 0);
    group.push_back(identity);
    sort(group.begin(), group.end());
    group.erase(unique(group.begin(), group.end()), group.end());
```

继续处理操作（接上页同一 main）：

```
    BurnsideAverage avg(group.size(), mod);
    for (auto &p : group)
    {
        vector<vector<long long>> dp(r + 1, vector<long long>(b + 1));
        dp[0][0] = 1;
        int used = 0;
        for (int len : permutation_cycles(p))
        {
            vector<vector<long long>> next(r + 1, vector<long long>(b + 1));
            for (int i = 0; i <= r; i++)
            {
                for (int j = 0; j <= b; j++)
                {
                    if (i + len <= r) next[i + len][j] += dp[i][j];
                    if (j + len <= b) next[i][j + len] += dp[i][j];
                    if (used - i - j + len <= g) next[i][j] += dp[i][j];
                }
            }
            for (auto &row : next)
            {
                for (auto &value : row) value %= avg.mod;
            }
            dp.swap(next);
            used += len;
        }
        avg.add(dp[r][b]);
    }
    cout << avg.result() << '\n';
}
```

21.7.2 置换的循环长度分解

permutation_cycles(p) 返回各不交循环长度, 按循环最小位置升序。p 是 0-based 的合法置换, n<INT_MAX; 空置换返回空 vector。本函数不校验输入，非置换不允许。O(n) 时间与空间；环上循环迭代不是 DFS。固定染色在每个循环内同色，命名颜色不限数量时为 colors^ 循环数；限定各色用量时以循环长度做背包。

```
vector<int> permutation_cycles(const vector<int> &p)
{
    int n = p.size();
    vector<int> seen(n), lengths;
    for (int i = 0; i < n; i++)
    {
        if (seen[i]) continue;
        int u = i, len = 0;
        do
        {
            seen[u] = 1;
            len++;
            u = p[u];
        } while (u != i);
        lengths.push_back(len);
    }
    return lengths;
}
```

接口用法演示：API 显式置换群染色

演示约定：输入 n size colors mod，然后 size 个 0-based 置换，必须完整列出群作用（包括恒等），不是生成元。统计循环数，累加 colors^ 循环数并平均；允许 n=0，空集合染色数为 1。

所需模板：BurnsideAverage（第 301 页，21.7.1 节）、permutation_cycles（第 304 页，21.7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/math/combinatorics/polya/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, size;
    long long colors, mod;
    cin >> n >> size >> colors >> mod;
    BurnsideAverage avg(size, mod);
    for (int i = 0; i < size; i++)
    {
        vector<int> p(n);
        for (int &x : p) cin >> x;
        avg.add(avg.power(colors, permutation_cycles(p).size()));
    }
    cout << avg.result() << '\n';
}
```

21.7.3 Pólya 环染色：旋转与翻转

代码依赖：BurnsideAverage（第 301 页，21.7.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

necklace_colorings(n,colors,mod,reflection=false)：n>=1 的环，每位置可选 colors>=0 种命名颜色，返回本质不同染色数模 mod。n 为 int；mod>0，群阶 g 为 n 或 2n，须 g*mod<=LLONG_MAX/2。默认只视旋转相同；reflection=true 也视翻转相同。允许颜色不用完，不能用于相邻异色、固定颜色用量或颜色重命名模型。n=1/2 仍适用，二面体抽象群作用可不忠实。用 BurnsideAverage 模 g*mod 后整除，无需模逆元；因子分解与枚举约数 O(sqrt(n))，每约数 O(omega(n)+log(n))，额外 O(omega(n))。

```
long long necklace_colorings(int n, long long colors, long long mod,
                             bool reflection = false)
{
    assert(n >= 1 && colors >= 0);
    BurnsideAverage avg((reflection ? 2LL : 1LL) * n, mod);
    vector<int> primes;
    int x = n;
    for (int p = 2; p <= x / p; p++)
    {
        if (x % p != 0) continue;
        primes.push_back(p);
        while (x % p == 0) x /= p;
    }
    if (x > 1) primes.push_back(x);
    auto add = [&](int d)
    {
        long long phi = d;
        for (int p : primes)
        {
            if (d % p == 0) phi = phi / p * (p - 1);
        }
        avg.add(avg.power(colors, n / d), phi);
    };
    for (int d = 1; d <= n / d; d++)
    {
        if (n % d != 0) continue;
        add(d);
        if (d != n / d) add(n / d);
    }
    if (reflection)
    {
        if (n & 1) avg.add(avg.power(colors, n / 2 + 1), n);
        else
        {
            avg.add(avg.power(colors, n / 2), n / 2);
            avg.add(avg.power(colors, n / 2 + 1), n / 2);
        }
    }
    return avg.result();
}
```

模板题使用：P4980 【模板】Pólya 定理

题意：t<=1000，1<=n<=10^9；n 个点用 n 种命名颜色，仅旋转等价，模 10^9+7。直接调用旋转环计数。

所需模板：BurnsideAverage（第 301 页，21.7.1 节）、necklace_colorings（第 305 页，21.7.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4980

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        int n;
        cin >> n;
        cout << necklace_colorings(n, n, 1000000007) << '\n';
    }
}
```

接口用法演示：API 环与手链命名颜色计数

演示约定：输入 n colors mod，输出仅旋转、旋转加翻转两种计数。遵守核心乘积界；特别可测试 mod 与群阶不互质和 n=1/2。

所需模板：BurnsideAverage（第 301 页，21.7.1 节）、necklace_colorings（第 305 页，21.7.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/math/combinatorics/polya/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long colors, mod;
    cin >> n >> colors >> mod;
    cout << necklace_colorings(n, colors, mod) << '\n';
    cout << necklace_colorings(n, colors, mod, true) << '\n';
}
```

21.7.4 群作用计数：先确定什么算同一个方案

把有标号的合法对象组成有限集合 X，把题目允许的等价变换组成有限群 G。作用必须把 X 映回 X，恒等变换不改变对象，复合变换与逐次操作一致。一个轨道 $Gx = \{gx : g \in G\}$ 就是一个最终方案。先确定对象与变换，再计算；不能因为题目出现“圆环”就除以长度。

- **只允许改变起点：**长度 $n \geq 1$ 的项链商去 n 个旋转，群为 C_n。
- **还允许翻面：**手链商去旋转与反射；$n \geq 3$ 时是阶 $2n$ 的二面体群。反射是否等价必须由题意决定，不能把项链答案再机械除以 2。
- **颜色仍有名字：**红、蓝互换通常是不同方案；若颜色也可重命名，作用已经改变，不能沿用本节的 k 种具名颜色公式。
- **标记与约束也要保留：**指定位置、位置权值或障碍可能破坏旋转对称。只有保持整个合法对象集合的变换，才能直接在这个集合上使用 Burnside。

记稳定子 $G_x = \{g : gx = x\}$，不动点集 $\mathrm{Fix}_X(g) = \{x : gx = x\}$。轨道–稳定子公式与 Burnside 引理为

$$|Gx| = \frac{|G|}{|G_x|}, \qquad |X/G| = \frac{1}{|G|} \sum_{g \in G} |\mathrm{Fix}_X(g)|.$$

证明只需双计数 (g, x) 且 $gx = x$ 的配对：一个轨道贡献 $\sum_{x \in Gx} |G_x| = |G|$，于是所有不动点数之和等于轨道数乘群阶。只有每个对象的稳定子都平凡时，才可直接用 $|X|/|G|$。例如 4 位二进制项链有 6 类，代表为 0000、1111、0001、0111、0011、0101；轨道大小依次为 $1, 1, 4, 4, 4, 2$，所以 $2^4/4 = 4$ 并非答案。理论说明可参见 Ben Lynn：Burnside's Lemma。

21.7.5 项链与手链：从置换循环推公式

位置暂用 $0, \ldots, n-1$，所有下标模 n，$n \geq 1$、$k \geq 1$。若位置置换 g 有 $c(g)$ 个循环，不动染色在每个循环上必须同色，不同循环独立选色，故不动染色数为 $k^{c(g)}$。

旋转。旋转 t 步为 $i \mapsto i + t$，有 $\gcd(n, t)$ 个循环，每个循环长度为 $n/\gcd(n, t)$；约定 $\gcd(n, 0) = n$。按循环长度 $d \mid n$ 归类，有 $\varphi(d)$ 个旋转，故旋转不动点总和与项链数为

$$R(n, k) = \sum_{t=0}^{n-1} k^{\gcd(n,t)} = \sum_{d \mid n} \varphi(d) k^{n/d}, \qquad N(n, k) = R(n, k)/n.$$

规模不大时逐个枚举 t 最容易抄；规模大时分解 n，枚举约数并计算各自的 $\varphi(d)$，减少求幂次数，别把 $O(n)$ 循环误当成约数枚举。

反射。写成 $i \mapsto t - i$；单点循环满足 $2i \equiv t \pmod{n}$，其余位置组成二元循环。这直接给出奇偶分类：

长度与反射类型	个数	单点、二元循环数	每个反射的不动染色数
n 奇数	n	$1,\ (n-1)/2$	$k^{(n+1)/2}$
n 偶数，穿过对顶点	$n/2$	$2,\ (n-2)/2$	$k^{n/2+1}$
n 偶数，穿过对边中点	$n/2$	$0,\ n/2$	$k^{n/2}$

于是手链数为

$$B(n, k) = \frac{R(n, k) + F(n, k)}{2n}, \qquad F(n, k) = \begin{cases} nk^{(n+1)/2}, & n \text{ 奇数}, \\ \dfrac{n}{2}\left(k^{n/2+1} + k^{n/2}\right), & n \text{ 偶数}. \end{cases}$$

循环分类亦可参见 Ben Lynn：The Cycle Index Polynomial。

偶数例：6 个位置、2 色。 $R = 2^6 + 2^3 + 2 \cdot 2^2 + 2 \cdot 2 = 84$，项链数为 14。反射贡献为 $3 \cdot 2^4 + 3 \cdot 2^3 = 72$，手链数为 $(84 + 72)/12 = 13$。**奇数例：5 个位置、3 色。** $R = 3^5 + 4 \cdot 3 = 255$，项链数为 51；5 个反射各贡献 3^3，手链数为 $(255 + 135)/10 = 39$。

退化长度不能偷换分母。 $n=1$ 时答案均为 k；$n=2$ 时均为 $k(k+1)/2$，即选一个允许重复的无序颜色对。上述公式仍成立：把旋转与反射保留为 $2n$ 个形式群元素时，不同元素可能诱导同一位置置换，这是非忠实作用，重复置换必须连同权重保留。也可以把置换去重后对像群求平均，但分母必须同步改为不同置换的个数；不能分子去重而仍除以 $2n$。本节不定义空环 $n=0$；若 $n>0, k=0$，没有染色，答案为 0。

21.7.6 指定颜色库存：给每个循环分配整份颜色

设颜色 $1,\dots,k$ 的数量为 $a_1,\dots,a_k\ge 0$，总和为 n。固定库存对位置置换不变，可以在这个合法子集上取轨道。若 g 的循环长度依次为 $\ell_1,\dots,\ell_c$，则

$$|\operatorname{Fix}_{\boldsymbol{a}}(g)| = [x_1^{a_1}\cdots x_k^{a_k}]\prod_{j=1}^{c}\left(x_1^{\ell_j}+\cdots+x_k^{\ell_j}\right).$$

每个因子给一个完整循环选颜色；各循环仍是不同的位置集合，即使长度相同也不能再除阶乘。把各个 g 的系数相加，最后除以 $|G|$。这也是循环指标代换 $z_\ell \leftarrow \sum_i x_i^\ell$ 的含义，不必为了使用 Pólya 再写通用群类。加权推导参见 Ben Lynn：Pólya's Inventory Theorem。

等长旋转循环可直接化成多项式系数。循环长度为 d 时，若存在 $d\nmid a_i$，不动点数为 0；否则为

$$\frac{(n/d)!}{\prod_i (a_i/d)!}.$$

实际计算可依次选每种颜色所占的循环：从 $r=n/d$ 开始，累乘 $\binom{r}{a_i/d}$ 后令 $r\leftarrow r-a_i/d$。小于素数模数的预处理范围内可用 `Binomial<p> c(n)` 与 `c.choose(r,a[i]/d)`，见第 21.1.1 节（第 276 页）。若随后需要在 $M|G|$ 下算总和，这个模数通常是合数，不能继续用素数阶乘逆元；按第 21.1.3 节（第 278 页）选组合数算法，或使用以下只加法的 DP。

二色恰有 r 个红色：取 $[x^r]\prod_j(1+x^{\ell_j})$。初始化 $f_0(0)=1$，其他项为 0；加入一个长度 ℓ 的循环时

$$f_{j+1}(s) = f_j(s) + [s\ge \ell] f_j(s-\ell), \qquad 0\le s\le r.$$

可用一维数组从 r 降序更新到 ℓ，时间 $O(c(r+1))$、空间 $O(r+1)$；加法后取模，对任意正模数都适用。$r\notin[0,n]$ 时答案为 0。多色可维护前 $k-1$ 色的库存，最后一色由已处理总长度确定；任何一色超出库存就丢弃状态；状态数至多 $\prod_{i<k}(a_i+1)$，每个循环最多 k 个转移，先估内存再选它。

例：6 个位置，3 红 3 蓝。恒等旋转贡献 $\binom{6}{3}=20$；旋转 2、4 步各有两个长度 3 的循环，各贡献 2；其他非恒等旋转贡献 0。项链数为 $(20+2+2)/6=4$。穿顶点反射的多项式为 $(1+x)^2(1+x^2)^2$，x^3 系数为 4；穿边反射为 $(1+x^2)^3$，x^3 系数为 0。手链数为 $(24+3\cdot 4)/12=3$。

额外约束须重新数不动点。"相邻位置异色"对旋转反射不变，Burnside 仍成立，但循环不能独立选色：应把每个循环缩成一个点，原相邻关系变成约束边；出现自环就无解，再数该约束图的染色。"位置 0 必须红色"则通常不对旋转封闭，不能在该子集上直接平均。若问题要求统计哪些完整轨道存在满足条件的代表，须另建模，不能把存在性条件当成不变库存。

21.7.7 群阶不可逆：先提升模数，再作整数除法

设不动点数总和为整数 S，群阶为 h，已知 $h\mid S$，目标是 $(S/h)\bmod M$，$M\ge 1$。只有 $\gcd(h,M)=1$ 时，才能乘模逆元；仅检查 $M\nmid h$ 不够。通用办法是在模 $L=Mh$ 下算出标准余数 $s=S \bmod L$，然后做普通整数除法：

$$S = qMh+s \quad\Longrightarrow\quad h\mid s, \qquad S/h\equiv s/h \pmod M, \qquad 0\le s/h<M.$$

必须先合计所有项，再除以 h；每个不动点数通常不单独被 h 整除。例如 4 位二进制项链，$S=24, h=4, M=8$：先模 8 得到 0 已丢失信息；先模 32 得到 24，再除以 4 才得到正确的 6。

可直接编译的无库存限制示例。输入每行 `n k M flip`，`flip=0` 只计旋转，`flip=1` 还计反射，输出轨道数模 M。使用第 18.1 节（第 124 页）的 `Mod64::mul` 与 `Mod64::power`；赛时把下面的仓库 include 换为所抄的 `Mod64`。

```cpp
#include "src/compact/mod64.hpp"

using ull = unsigned long long;
using u128 = __uint128_t;

ull color_orbits(int n, ull k, ull m, bool flip)
{
    assert(n >= 1 && m >= 1);
    ull h = (flip ? 2ULL : 1ULL) * n;
    u128 wide = u128(m) * h;
    assert(wide <= numeric_limits<ull>::max());
    ull mod = (ull)wide, sum = 0;
    auto add = [&](ull count, ull cycles)
    {
        ull term = Mod64::mul(count, Mod64::power(k, cycles, mod), mod);
        sum = (ull)((u128(sum) + term) % mod);
    };
    for (int t = 0; t < n; t++) add(1, gcd(n, t));
    if (flip)
    {
        if (n & 1) add(n, n / 2 + 1);
        else
        {
            add(n / 2, n / 2 + 1);
            add(n / 2, n / 2);
        }
    }
    assert(sum % h == 0);
    return sum / h;
}

int main()
{
    int n, flip;
    ull k, m;
    while (cin >> n >> k >> m >> flip)
        cout << color_orbits(n, k, m, flip != 0) << '\n';
}
```

该示例要求 $1 \le n \le \mathrm{INT_MAX}, k$ 为无符号 64 位整数，$1 \le M$ 且 $Mh \le 2^{64}-1$；运行预算还须容纳 $O(n \log(n+1))$ 时间，额外空间 $O(1)$。没有跨测试状态，支持多测与 $M = 1$。先转宽类型再算 Mh，加法也在 128 位中进行；单有安全模乘并不能防止模数乘积或求和溢出。若 $Mh > 2^{64} - 1$，本函数不适用；不可截断后再传给 `Mod64`。可改用多精度整数在 Mh 下计算，或在预算允许时精确算出 S 后整除；仅把模数类型改成 128 位，仍不能保证两个 128 位数的乘积不溢出。

配套 `tests/math_knowledge_orbits.py` 从本节直接抽取 C++ 示例编译，对照小染色的规范轨道枚举；另用循环库存 DP 对照按库存分类的轨道，检查奇偶反射、$n = 1, 2$、零色、复合模数与 64 位边界。运行 `python3 tests/math_knowledge_orbits.py`，加 `--sanitize` 可在 AddressSanitizer/UndefinedBehaviorSanitizer 下运行同一批 C++ 案例。这是本地知识示例验证，不是新增 OJ AC，也不替代底层模板的完整回归。

第二十二章　线性代数

22.1　矩阵

22.1.1　模矩阵乘法与幂

代码依赖：ModInt（第 133 页，19.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

ModMatrix<p>::multiply(a,b) 计算 n×k 与 k×m 的矩阵积，要求非空且每行尺寸一致。power(a,e) 接收非空 n×n 方阵和 unsigned long long 指数；e=0 返回单位阵。只使用加乘，p 为 ≥2 的 int 模数，允许合数。矩阵乘法 O(nkm)，矩阵幂 O(n^3 log(e+1))，额外空间 O(n^2)；复用 ModInt。

```
template <int mod> struct ModMatrix
{
    using Z = ModInt<mod>;
    using Matrix = vector<vector<Z>>;

    static Matrix multiply(const Matrix &a, const Matrix &b)
    {
        assert(!a.empty() && !b.empty());
        int n = a.size(), m = b[0].size(), k = b.size();
        assert((int)a[0].size() == k);
        Matrix c(n, vector<Z>(m));
        for (int i = 0; i < n; i++)
            for (int t = 0; t < k; t++)
                for (int j = 0; j < m; j++) c[i][j] = c[i][j] + a[i][t] * b[t][j];
        return c;
    }

    static Matrix power(Matrix a, unsigned long long e)
    {
        int n = a.size();
        assert(n > 0 && (int)a[0].size() == n);
        Matrix r(n, vector<Z>(n));
        for (int i = 0; i < n; i++) r[i][i] = 1;
        for (; e; e >>= 1, a = multiply(a, a))
            if (e & 1) r = multiply(r, a);
        return r;
    }
};
```

模板题使用：Library Checker matrix_product

题意：计算 n×m 矩阵与 m×k 矩阵的乘积，模 998244353。输入元素已取模，按行输出 n×k 个结果。

所需模板：ModInt（第 133 页，19.2.2 节）、ModMatrix（第 310 页，22.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/matrix_product

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using M = ModMatrix<998244353>;
    int n, m, k;
    cin >> n >> m >> k;
    M::Matrix a(n, vector<M::Z>(m)), b(m, vector<M::Z>(k));
    for (auto &row : a)
        for (auto &x : row) cin >> x.v;
    for (auto &row : b)
        for (auto &x : row) cin >> x.v;
    auto c = M::multiply(a, b);
    for (auto &row : c)
        for (int j = 0; j < k; j++) cout << row[j].v << (j + 1 == k ? '\n' : ' ');
}
```

22.1.2 素数模矩阵求逆

代码依赖：ModInt（第 133 页，19.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

matrix_inverse<mod>(a) 返回 optional 方阵；a 为 n×n 的 ModInt<mod> 矩阵，mod 必须是素数。可逆时返回逆矩阵，奇异时返回 nullopt；按值传入不改原矩阵，空方阵返回有值的空矩阵。每次同时对原矩阵和单位阵进行换行、倍乘、消元；必须同步交换两边的行。依赖 ModInt；$O(n^3 + n \log mod)$ 时间、$O(n^2)$ 额外空间。

```
template <int mod>
optional<vector<vector<ModInt<mod>>>> matrix_inverse(vector<vector<ModInt<mod>>> a)
{
    using Z = ModInt<mod>;
    int n = a.size();
    vector<vector<Z>> b(n, vector<Z>(n));
    for (int i = 0; i < n; i++)
    {
        assert((int)a[i].size() == n);
        b[i][i] = 1;
    }
    for (int i = 0; i < n; i++)
    {
        int p = i;
        while (p < n && !a[p][i].v) p++;
        if (p == n) return nullopt;
        swap(a[p], a[i]);
        swap(b[p], b[i]);
        Z inv = a[i][i].inv();
        for (int k = i; k < n; k++) a[i][k] = a[i][k] * inv;
        for (int k = 0; k < n; k++) b[i][k] = b[i][k] * inv;
        for (int j = 0; j < n; j++)
            if (j != i && a[j][i].v)
            {
                Z f = a[j][i];
                for (int k = i; k < n; k++) a[j][k] = a[j][k] - f * a[i][k];
                for (int k = 0; k < n; k++) b[j][k] = b[j][k] - f * b[i][k];
            }
    }
    return b;
}
```

模板题使用：Luogu P4783

题意：给定 n 阶方阵，求模 1000000007 的逆矩阵；不可逆输出 No Solution。输入按行读取，输出按行给出逆矩阵，不对原矩阵进行覆盖。

所需模板：ModInt（第 133 页，19.2.2 节）、matrix_inverse（第 312 页，22.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4783

```
int main()
{
    using Z = ModInt<1000000007>;
    int n;
    scanf("%d", &n);
    vector<vector<Z>> a(n, vector<Z>(n));
    for (auto &row : a)
        for (auto &x : row)
        {
            int v;
            scanf("%d", &v);
            x = v;
        }
    auto b = matrix_inverse<1000000007>(a);
    if (!b)
        puts("No Solution");
    else
        for (auto &row : *b)
        {
            for (auto x : row) printf("%d ", x.v);
            puts("");
        }
}
```

模板题使用：Library Checker inverse_matrix

题意：给定 n 阶方阵，求模 998244353 的逆矩阵；不可逆输出 -1。输入按行读取，输出按行给出逆矩阵，不对原矩阵进行覆盖。

所需模板：ModInt（第 133 页，19.2.2 节）、matrix_inverse（第 312 页，22.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/inverse_matrix

```
int main()
{
    using Z = ModInt<998244353>;
    int n;
    scanf("%d", &n);
    vector<vector<Z>> a(n, vector<Z>(n));
    for (auto &row : a)
        for (auto &x : row)
        {
            int v;
            scanf("%d", &v);
            x = v;
        }
    auto b = matrix_inverse<998244353>(a);
    if (!b)
        puts("-1");
    else
        for (auto &row : *b)
        {
            for (auto x : row) printf("%d ", x.v);
            puts("");
        }
}
```

22.1.3 模 2 矩阵求逆（按位消元）

matrix_inverse_mod2(a) 接收 n 行长度 n 的 01 字符串，返回 optional<vector<string»。可逆时返回同样格式的逆矩阵，奇异返回 nullopt；空矩阵返回有值的空矩阵，不改输入。将原矩阵与单位阵拼成 2n 位一行，按 uint64_t 存储，换行后用异或消元；无需固定 bitset 上限，移位量始终在 0..63。O(n^2 ceil(n/64)) 时间，工作数组 O(n ceil(n/64)) 个机器字，输入输出 O(n^2) 字节。仅用于 F2，不处理一般模数。

```
optional<vector<string>> matrix_inverse_mod2(const vector<string> &a)
{
    int n = a.size(), words = (2LL * n + 63) / 64;
    vector<vector<uint64_t>> b(n, vector<uint64_t>(words));
    for (int i = 0; i < n; i++)
    {
        assert((int)a[i].size() == n);
        for (int j = 0; j < n; j++)
        {
            assert(a[i][j] == '0' || a[i][j] == '1');
            if (a[i][j] == '1') b[i][j / 64] |= 1ULL << (j % 64);
        }
        b[i][(n + i) / 64] |= 1ULL << ((n + i) % 64);
    }
    for (int i = 0; i < n; i++)
    {
        int p = i;
        uint64_t mask = 1ULL << (i % 64);
        while (p < n && !(b[p][i / 64] & mask)) p++;
        if (p == n) return nullopt;
        swap(b[p], b[i]);
        for (int j = 0; j < n; j++)
            if (j != i && (b[j][i / 64] & mask))
                for (int k = i / 64; k < words; k++) b[j][k] ^= b[i][k];
    }
    vector<string> ans(n, string(n, '0'));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if ((b[i][(n + j) / 64] >> ((n + j) % 64)) & 1) ans[i][j] = '1';
    return ans;
}
```

模板题使用：Library Checker inverse_matrix_mod_2

题意：输入 n 阶模 2 方阵，每行是无空格的 01 字符串。求逆矩阵，按同样形式逐行输出；奇异则输出 -1。

所需模板：matrix_inverse_mod2（第 315 页，22.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/inverse_matrix_mod_2

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<string> a(n);
    for (auto &row : a) cin >> row;
    auto b = matrix_inverse_mod2(a);
    if (!b)
        cout << -1 << '\n';
    else
        for (auto &row : *b) cout << row << '\n';
}
```

22.1.4 Max-plus 矩阵幂

a[i][j] 为转移收益，不存在的边为 neg；power(k) 求恰好 k 步的最优转移，单位矩阵对角为 0。所有有限中间值绝对值小于 2^119。O(n^3 log k)；2025 成都 B 的转移类型，具体题目的建模仍需调用者完成。

```
struct MaxPlusMatrix
{
    using I = __int128_t;
    static constexpr I neg = -(I(1) << 120);
    int n;
    vector<vector<I>> a;

    MaxPlusMatrix(int n, bool identity = false) : n(n), a(n, vector<I>(n, neg))
    {
        if (identity)
            for (int i = 0; i < n; i++) a[i][i] = 0;
    }

    MaxPlusMatrix operator*(const MaxPlusMatrix &b) const
    {
        assert(n == b.n);
        MaxPlusMatrix c(n);
        for (int i = 0; i < n; i++)
            for (int k = 0; k < n; k++)
                if (a[i][k] != neg)
                    for (int j = 0; j < n; j++)
                        if (b.a[k][j] != neg)
                            c.a[i][j] = max(c.a[i][j], a[i][k] + b.a[k][j]);
        return c;
    }

    MaxPlusMatrix power(unsigned long long e) const
    {
        MaxPlusMatrix b = *this, r(n, true);
        for (; e; e >>= 1, b = b * b)
            if (e & 1) r = r * b;
        return r;
    }
};
```

应用题补充：CSES 1724 Graph Paths II

题意：有向图求从 1 到 n 恰好经过 k 条边的最小费用，不可达输出-1。1≤n≤100，1≤m≤n(n−1)，1≤k,c≤10^9。邻接矩阵初始全为不可达，不能设成单位阵；边 u→v 存-c，重边取最大值，再调用 power(k)。先判不可达哨兵，再把结果取负输出；有限答案至多 10^18。中间值含最后一次平方也满足本模板 128 位范围。时间 O(n^3 log(k+1))、空间 O(n^2)；这是负权转换应用，不是 max-plus 正式模板题。

所需模板：MaxPlusMatrix（第 317 页，22.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://cses.fi/problemset/task/1724

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    unsigned long long k;
    if (!(cin >> n >> m >> k)) return 0;
    MaxPlusMatrix a(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        --u, --v;
        a.a[u][v] = max(a.a[u][v], -MaxPlusMatrix::I(c));
    }
    auto r = a.power(k);
    auto x = r.a[0][n - 1];
    // Official bounds imply each finite answer is in [1, 10^18].
    if (x == MaxPlusMatrix::neg) cout << -1 << '\n';
    else cout << (long long)(-x) << '\n';
}
```

22.2 行列式

22.2.1 素数模行列式

代码依赖：ModInt（第 133 页，19.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

det_prime<p>(a) 输入 n×n 方阵，返回行列式模 p；p 必须是素数。允许奇异矩阵和零维矩阵，det(empty)=1。按行交换改变符号，主元求逆消元。$O(n^3)$ 时间、传值副本 $O(n^2)$ 空间，可传 move(a)。合数模数改用 determinant_mod。

```
template <int mod> ModInt<mod> det_prime(vector<vector<ModInt<mod>>> a)
{
    using Z = ModInt<mod>;
    int n = (int)a.size();
    Z ans = 1;
    for (int i = 0; i < n; i++)
    {
        assert((int)a[i].size() == n);
        int p = i;
        while (p < n && !a[p][i].v) ++p;
        if (p == n) return 0;
        if (p != i)
        {
            swap(a[p], a[i]);
            ans = Z(0) - ans;
        }
        ans = ans * a[i][i];
        Z inv = a[i][i].inv();
        for (int j = i + 1; j < n; j++)
        {
            Z f = a[j][i] * inv;
            for (int k = i; k < n; k++) a[j][k] = a[j][k] - f * a[i][k];
        }
    }
    return ans;
}
```

模板题使用：Library Checker matrix_det

题意：求方阵行列式，模 998244353。输入元素已取模；函数按值接收矩阵，示例用 move 交出矩阵避免额外复制。

所需模板：ModInt（第 133 页，19.2.2 节）、det_prime（第 319 页，22.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/matrix_det

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Z = ModInt<998244353>;
    int n;
    cin >> n;
    vector<vector<Z>> a(n, vector<Z>(n));
    for (auto &row : a)
        for (auto &x : row) cin >> x.v;
    cout << det_prime<998244353>(move(a)).v << '\n';
}
```

应用题补充：Luogu P4336 黑暗前的幻想乡：公司集合容斥

题意：n 点、n-1 家公司，每家恰选一边成树；不同公司的同端点边分别计数。公司集合容斥配合矩阵树；Gray 码每步增删一家。2<=n<=17，模 1000000007，时间 O(2^(n-1)n^3)，空间 O(n^3)。删去末点；lap 后续仍需使用，勿 move。空集贡献为 0。本地验证通过，线上待验证。

所需模板：ModInt（第 133 页，19.2.2 节）、det_prime（第 319 页，22.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4336

```
int main()
{
    using Z = ModInt<1000000007>;
    int n;
    cin >> n;
    int k = n - 1;
    vector<vector<pair<int, int>>> edges(k);
    for (auto &group : edges)
    {
        int m;
        cin >> m;
        while (m--)
        {
            int u, v;
            cin >> u >> v;
            group.push_back({u - 1, v - 1});
        }
    }
    vector<vector<Z>> lap(k, vector<Z>(k));
    Z ans = 0;
    int prev = 0;
    for (int mask = 1; mask < (1 << k); mask++)
    {
        int gray = mask ^ (mask >> 1);
        int bit = __builtin_ctz(unsigned(gray ^ prev));
        Z delta = (gray >> bit & 1) ? 1 : -1;
        for (auto [u, v] : edges[bit])
        {
            if (u < k)
                lap[u][u] = lap[u][u] + delta;
            if (v < k)
                lap[v][v] = lap[v][v] + delta;
            if (u < k && v < k)
            {
                lap[u][v] = lap[u][v] - delta;
                lap[v][u] = lap[v][u] - delta;
            }
        }
        Z ways = det_prime<1000000007>(lap);
        if ((k - __builtin_popcount(unsigned(gray))) & 1)
            ans = ans - ways;
        else
            ans = ans + ways;
        prev = gray;
    }
    cout << ans.v << '\n';
}
```

22.2.2 任意正模数行列式：欧几里得消元

determinant_mod(a,mod) 返回 det(a) 的最小非负余数。a 为 long long 方阵，元素支持完整 signed 64 位；1≤mod≤LLONG_MAX，可以是合数。不使用模逆元，允许不可逆主元。矩阵按值传入，调用后原矩阵不变；空矩阵返回 1%mod，mod=1 时返回 0。换行改变符号，最终乘上三角矩阵的对角元素。所有乘法先转 int128。O(n^3 log mod) 时间、O(n^2) 空间；素数模通常优先用已有高斯行列式。

```
long long determinant_mod(vector<vector<long long>> a, long long mod)
{
    assert(mod >= 1);
    int n = a.size();
    for (auto &row : a)
    {
        assert((int)row.size() == n);
        for (auto &x : row)
        {
            x %= mod;
            if (x < 0) x += mod;
        }
    }
    long long answer = 1 % mod;
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
            while (a[j][i])
            {
                long long quotient = a[i][i] / a[j][i];
                for (int k = i; k < n; k++)
                {
                    __int128 value = (__int128)a[i][k] - (__int128)quotient * a[j][k];
                    value %= mod;
                    if (value < 0) value += mod;
                    a[i][k] = (long long)value;
                }
                swap(a[i], a[j]);
                answer = answer ? mod - answer : 0;
            }
        if (a[i][i] == 0) return 0;
        answer = (__int128)answer * a[i][i] % mod;
    }
    return answer;
}
```

模板题使用：Library Checker matrix_det_arbitrary_mod

题意：给方阵和模数，求行列式的最小非负余数，模数可以为合数。函数无需模逆元，按值接收矩阵，示例用 move 交出数据。0 阶方阵的行列式为 1，再对模数取余。

所需模板：determinant_mod（第 322 页，22.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/matrix_det_arbitrary_mod

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long mod;
    cin >> n >> mod;
    vector<vector<long long>> a(n, vector<long long>(n));
    for (auto &row : a)
        for (auto &x : row) cin >> x;
    cout << determinant_mod(move(a), mod) << '\n';
}
```

22.2.3 精确整数行列式（Bareiss）

determinant_exact(a) 按值接收 n 阶方阵，0≤n<INT_MAX，返回带符号行列式，空矩阵为 1，不修改调用者矩阵。Z 必须支持精确有符号整数四则运算，推荐 Boost cpp_int；不是模整数或浮点类型。固定宽度仅在所有中间乘积、差和商均不溢出时可用。每轮选择非零主元并换行，分子必须先完整计算，再除以上一主元；由 Sylvester 恒等式保证整除。当前列无主元则返回 0。$O(n^3)$ 次大整数算术操作、$O(n^2)$ 个整数对象；实际时间和位存储取决于中间数位数，不能视为机器字常数成本。

```
template<class Z>
Z determinant_exact(vector<vector<Z>> a)
{
    int n = a.size();
    for (const auto &row : a)
        assert((int)row.size() == n);
    if (!n)
        return Z(1);
    Z prev = 1;
    int sign = 1;
    for (int k = 0; k + 1 < n; k++)
    {
        int p = k;
        while (p < n && a[p][k] == 0)
            p++;
        if (p == n)
            return Z(0);
        if (p != k)
        {
            swap(a[p], a[k]);
            sign = -sign;
        }
        Z pivot = a[k][k];
        for (int i = k + 1; i < n; i++)
        {
            for (int j = k + 1; j < n; j++)
            {
                Z value = a[i][j] * pivot - a[i][k] * a[k][j];
                a[i][j] = value / prev;
            }
            a[i][k] = 0;
        }
        prev = pivot;
    }
    return sign * a[n - 1][n - 1];
}
```

接口用法演示：精确整数行列式

演示约定：T 组；每组 n 及 n^2 个有符号十进制整数，输出精确行列式。n=0 输出 1。cpp_int 为 Boost 任意精度有符号整数，输入与中间结果受可用内存和时间限制；此处为 API 用法，无线上 AC 声明。

所需模板：determinant_exact（第 324 页，22.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

额外依赖（不属于标准头文件）：`boost/multiprecision/cpp_int.hpp`，须由编译环境提供并显式包含。

来源：参考文档

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Z = boost::multiprecision::cpp_int;
    int t;
    cin >> t;
    while (t--)
    {
        int n;
        cin >> n;
        vector<vector<Z>> a(n, vector<Z>(n));
        for (auto &row : a)
            for (auto &x : row)
                cin >> x;
        cout << determinant_exact(move(a)) << '\n';
    }
    return 0;
}
```

22.3 线性基

22.3.1 64 位异或线性基

值类型 U 为 unsigned long long，完整支持 64 位。默认空基；重新赋值 XorBasis() 清空。insert/Insert 返回是否增秩；contains/Contains 判断可表示性（允许空子集，所以 0 总可表示）。query/Query(x=0) 返回 x 与所张成空间异或后的最大无符号值。kth/Kth(k,nonempty=true) 返回第 k 小的不同异或值，1-based，参数 k 为 __uint128_t，非法序号返回 nullopt。默认只允许非空输入子集，0 仅在 dependent 为真时出现；传 false 则包括空子集所得的 0。minimum/Minimum 同此约定，空输入的非空最小值不存在。merge/Merge 将另一基所代表的输入多重集并入；重复元素和自合并可新增异或为 0 的非空方案。rank 是维数，dependent 表示存在非空零异或子集。插入、成员判断、最大值为 O(64)；合并 $O(64^2)$。kth 会在需要时 rebuild/Rebuild 规范化缓存，首次有效查询 $O(64^2)$，后续 O(64)；仅新增独立向量使缓存失效。所有成员仅供只读，不要直接改数组或标志。

```
struct XorBasis
{
    using U = unsigned long long;
    using Wide = __uint128_t;
    U a[64]{};
    vector<U> p;
    int rank = 0;
    bool dependent = false, dirty = false;

    bool insert(U x)
    {
        for (int i = 63; i >= 0; i--)
            if (x >> i & 1)
            {
                if (!a[i])
                {
                    a[i] = x;
                    rank++;
                    dirty = true;
                    return true;
                }
                x ^= a[i];
            }
        dependent = true;
        return false;
    }
```

```
28      bool contains(U x) const
29      {
30          for (int i = 63; i >= 0; i--)
31              if (x >> i & 1) x ^= a[i];
32          return x == 0;
33      }
34
35      U query(U x = 0) const
36      {
37          for (int i = 63; i >= 0; i--) x = max(x, x ^ a[i]);
38          return x;
39      }
40
41      void rebuild()
42      {
43          vector<U> b(a, a + 64);
44          for (int i = 0; i < 64; i++)
45              if (b[i])
46                  for (int j = i + 1; j < 64; j++)
47                      if (b[j] >> i & 1) b[j] ^= b[i];
48          p.clear();
49          for (U x : b)
50              if (x) p.push_back(x);
51          dirty = false;
52      }
```

```
    // k is 1-based; nonempty excludes the empty input subset, not value zero.
    optional<U> kth(Wide k, bool nonempty = true)
    {
        if (!k) return nullopt;
        Wide index = k;
        if (!nonempty || dependent) index--;
        if (index >= (Wide(1) << rank)) return nullopt;
        if (dirty) rebuild();
        U answer = 0;
        for (int i = 0; i < rank; i++)
            if (index >> i & 1) answer ^= p[i];
        return answer;
    }

    optional<U> minimum(bool nonempty = true) { return kth(1, nonempty); }

    void merge(const XorBasis &other)
    {
        dependent |= other.dependent;
        for (U x : other.a)
            if (x) insert(x);
    }
};
```

模板题使用：Luogu P3812

题意：给定一组可能重复的非负整数，选任意子集使异或和最大。逐个 insert 后 query()；使用 unsigned long long 读取和输出。

所需模板：XorBasis（第 326 页，22.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3812

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n;
    std::cin >> n;
    XorBasis basis;
    for (int i = 0; i < n; i++)
    {
        unsigned long long x;
        std::cin >> x;
        basis.insert(x);
    }
    std::cout << basis.query() << '\n';
}
```

22.3.2 带位置限制的异或线性基

默认构造为空，重新赋值 PositionBasis() 可清空。insert/Insert(x,at) 插入 unsigned long long 值及非负 int 标签，支持完整 64 位，标签可重复、乱序。contains/Contains(x,lower=0) 判断 x 能否由标签至少为 lower 的输入异或得到；query/Query(lower=0,x=0) 返回 x 与这些输入子集异或后的最大值；rank/Rank(lower=0) 返回对

应空间维数。lower 为非负 long long，可大于所有 int 标签，空集仍可产生 0。全部操作 O(64)，空间 O(64)。消元时同一主元优先保留标签较大的向量，较旧向量继续消元；按阈值筛选后仍是正确线性基。标签是组合可用范围的界，不是选点方案；数组仅供只读。区间 [l,r] 应按右端点排序询问，扫描并仅插入到 r，再查询 lower=l；接口本身没有上界，插入超过 r 后不能只用 lower 恢复旧区间。若需保留历史，可复制每个前缀的对象，但空间为 O(64n)。不提供非空零异或判定或第 k 小；普通线性基的 dependent 不能直接移植到所有阈值。

```
struct PositionBasis
{
    using U = unsigned long long;
    U a[64]{};
    int pos[64];

    PositionBasis() { fill(pos, pos + 64, -1); }
```

```
    void insert(U x, int at)
    {
        assert(at >= 0);
        for (int i = 63; i >= 0; i--)
            if (x >> i & 1)
            {
                if (!a[i])
                {
                    a[i] = x;
                    pos[i] = at;
                    return;
                }
                if (at > pos[i])
                {
                    swap(a[i], x);
                    swap(pos[i], at);
                }
                x ^= a[i];
            }
    }

    bool contains(U x, long long lower = 0) const
    {
        assert(lower >= 0);
        for (int i = 63; i >= 0; i--)
            if ((x >> i & 1) && pos[i] >= lower) x ^= a[i];
        return x == 0;
    }

    U query(long long lower = 0, U x = 0) const
    {
        assert(lower >= 0);
        for (int i = 63; i >= 0; i--)
            if (pos[i] >= lower) x = max(x, x ^ a[i]);
        return x;
    }

    int rank(long long lower = 0) const
    {
        assert(lower >= 0);
        int answer = 0;
        for (int i = 0; i < 64; i++) answer += pos[i] >= lower;
        return answer;
    }
};
```

应用题补充：Luogu CF1100F

题意：给数组及多个闭区间，求区间内任意子集的最大异或和。把查询按右端点排序，依次加入前缀元素及其 1-based 位置；query(l) 限制只使用位置不小于 l 的元素，最后按原查询顺序输出。

所需模板：PositionBasis（第 328 页，22.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/CF1100F

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, q;
    std::cin >> n;
    std::vector<unsigned long long> a(n + 1);
    for (int i = 1; i <= n; i++) std::cin >> a[i];
    std::cin >> q;
    std::vector<std::array<int, 3>> queries(q);
    for (int i = 0; i < q; i++)
    {
        int l, r;
        std::cin >> l >> r;
        queries[i] = {r, l, i};
    }
    std::sort(queries.begin(), queries.end());
    std::vector<unsigned long long> answer(q);
    PositionBasis basis;
    int right = 0;
    for (auto [r, l, id] : queries)
    {
        while (right < r)
        {
            right++;
            basis.insert(a[right], right);
        }
        answer[id] = basis.query(l);
    }
    for (auto x : answer) std::cout << x << '\n';
}
```

22.3.3 异或线性空间求交

代码依赖：XorBasis（第 326 页，22.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

basis_intersection(a,b) 返回 span(a) 与 span(b) 的交空间的一组独立生成元，结果类型 XorBasis。支持完整 64 位无符号值，不修改输入，也不要求先 rebuild。使用第一个空间的主元，消去第二个空间的向量，同时记录来自第二个空间的部分；消元为零时，该部分属于交空间。时间 $O(64^2)$，额外空间 O(64)。输出按独立生成元解释：dependent 为 false，输入的重复次数及非空零异或关系不传播；表示的是子空间，不是两组原始输入元素的集合交。枚举整个交空间应调用 result.kth(k,false)，包含零；零空间返回空基，rank=0、contains(0)=true。

```
XorBasis basis_intersection(const XorBasis &a, const XorBasis &b)
{
    using U = unsigned long long;
    U pivot[64], from_b[64]{};
    copy(a.a, a.a + 64, pivot);
    XorBasis result;
    for (U value : b.a)
    {
        U x = value, part = value;
        for (int i = 63; i >= 0 && x; i--)
            if (x >> i & 1)
            {
                if (!pivot[i])
                {
                    pivot[i] = x;
                    from_b[i] = part;
                    break;
                }
                x ^= pivot[i];
                part ^= from_b[i];
            }
        if (!x && part) result.insert(part);
    }
    return result;
}
```

模板题使用：LC Intersection of F2 Vector Spaces

题意：多组给两个 F2 向量空间的基，输出交空间的一组基。T≤100000，n,m≤30，输入独立且小于 2^30。把输入插入两个 XorBasis，调用求交，输出 rank 和 a 中的非零主元；答案不唯一，不能逐字比较不同实现输出。模板本身支持 64 位和相关生成元，属于题外扩展。零维输出 0。

所需模板：XorBasis（第 326 页，22.3.1 节）、basis_intersection（第 332 页，22.3.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/intersection_of_f2_vector_spaces

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int tests;
    std::cin >> tests;
    while (tests--)
    {
        XorBasis a, b;
        int n, m;
        std::cin >> n;
        for (int i = 0; i < n; i++)
        {
            unsigned long long x;
            std::cin >> x;
            a.insert(x);
        }
        std::cin >> m;
        for (int i = 0; i < m; i++)
        {
            unsigned long long x;
            std::cin >> x;
            b.insert(x);
        }
        auto result = basis_intersection(a, b);
        std::cout << result.rank;
        for (auto x : result.a)
            if (x) std::cout << ' ' << x;
        std::cout << '\n';
    }
}
```

22.3.4 Zassenhaus：同时求和空间与交空间

代码依赖：XorBasis（第 326 页，22.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

basis_sum_intersection(a,b) 返回和空间基、交空间基这一对结果。和空间由两组基共同张成，一般不是两个空间的集合并；两结果都是独立生成元，其 dependent 均为 false。将 a 的向量 x 编成 (x,x)，b 的向量 y 编成 (y,0)，用 ___uint128_t 储存高低各 64 位并按高位到低位消元。高半部非零的行投影构成和空间的基，高半部为零的行取低半部构成交空间的基。必须先提升到 128 位再左移 64；不能用 64 位变量移位 64。时间 $O(128^2)$，额外空间 O(128)。结果满足 dim(A+B)+dim(A 交 B)=dim(A)+dim(B)，但维数恒等式只是检查条件，不能单独证明返回向量正确。包含空空间、相同空间和输入别名。

```
// Return independent generators of (sum space, intersection space).
pair<XorBasis, XorBasis> basis_sum_intersection(const XorBasis &a, const XorBasis &b)
{
    using W = __uint128_t;
    W pivot[128]{};
    auto insert = [&](W x)
    {
        for (int i = 127; i >= 0 && x; i--)
            if (x >> i & 1)
            {
                if (!pivot[i])
                {
                    pivot[i] = x;
                    return;
                }
                x ^= pivot[i];
            }
    };
    for (auto x : a.a) insert((W(x) << 64) | x);
    for (auto x : b.a) insert(W(x) << 64);
    XorBasis sum, intersection;
    for (int i = 0; i < 128; i++)
        if (pivot[i])
        {
            if (i < 64)
                intersection.insert((unsigned long long)pivot[i]);
            else
                sum.insert((unsigned long long)(pivot[i] >> 64));
        }
    return {sum, intersection};
}
```

模板题使用：LC Intersection / Zassenhaus

题意：同一线性空间求交题，约束与前例相同。Zassenhaus 返回 {和空间, 交空间}，本题仅输出 second 的 rank 和非零主元，不能据此声称 first 也获线上验证。和空间由独立本地测试检查。两部分均为独立生成元；无需对输入枚举全部异或值。

所需模板：XorBasis（第 326 页，22.3.1 节）、basis_sum_intersection（第 334 页，22.3.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/intersection_of_f2_vector_spaces

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int tests;
    std::cin >> tests;
    while (tests--)
    {
        XorBasis a, b;
        int n, m;
        std::cin >> n;
        for (int i = 0; i < n; i++)
        {
            unsigned long long x;
            std::cin >> x;
            a.insert(x);
        }
        std::cin >> m;
        for (int i = 0; i < m; i++)
        {
            unsigned long long x;
            std::cin >> x;
            b.insert(x);
        }
        auto result = basis_sum_intersection(a, b).second;
        std::cout << result.rank;
        for (auto x : result.a)
            if (x) std::cout << ' ' << x;
        std::cout << '\n';
    }
}
```

22.3.5 无向图最大异或行走

代码依赖：XorBasis（第 326 页，22.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

XorWalk(n) 建图，点号 1..n，边权为 unsigned long long，支持完整 64 位。add(u,v,w) 加无向边，允许重边、自环和零权；重新赋值 XorWalk(n) 清空图。build(root=1) 用递归 DFS 预处理 root 的连通块，query(u,v) 返回该块内 u 到 v 的最大边权异或和；任一点未被本次 build 覆盖则返回 nullopt，即使这两点在另一连通块相连。加入新边后必须重新 build；可更换根重复构建。空图可初始化，但不能 build。这里的行走允许重复点、重复边；不是最大异或简单路径。u=v 也可能通过绕环得到正值。DFS 距离 dist 给出一条树路径的异或值，每条已访问边形成 dist[u] xor dist[v] xor w 的闭合行走值，加入线性基；两点答案为 cycles.query(dist[u] xor dist[v])。build 后 vis、dist 可只读；cycles 表示环异或空间，不保留具体环或行走方案，不能将其非空子集标志解释为行走非空性。建图 O(n+m)，build 为 O(n+64m)，每次 query 为 O(64)，动态空间 O(n+m+64)。

```
struct XorWalk
{
    using U = unsigned long long;
    int n;
    bool built = false;
    vector<vector<pair<int, U>>> g;
    vector<int> vis;
    vector<U> dist;
    XorBasis cycles;

    XorWalk(int size)
    {
        assert(size >= 0);
        n = size;
        g.resize(size_t(n) + 1);
        vis.resize(size_t(n) + 1);
        dist.resize(size_t(n) + 1);
    }
```

```
    void add(int u, int v, U w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
        built = false;
    }

    void dfs(int u)
    {
        vis[u] = 1;
        for (auto [v, w] : g[u])
        {
            if (!vis[v])
            {
                dist[v] = dist[u] ^ w;
                dfs(v);
            }
            else
                cycles.insert(dist[u] ^ dist[v] ^ w);
        }
    }

    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        fill(vis.begin(), vis.end(), 0);
        fill(dist.begin(), dist.end(), 0);
        cycles = XorBasis();
        dfs(root);
        built = true;
    }

    optional<U> query(int u, int v) const
    {
        assert(built && 1 <= u && u <= n && 1 <= v && v <= n);
        if (!vis[u] || !vis[v]) return nullopt;
        return cycles.query(dist[u] ^ dist[v]);
    }
};
```

应用题补充：Luogu P4151

题意：无向连通带权图，求 1 到 n 可重复经过点边的最大异或和；n≤50000，m≤100000，权值 ≤10^18，允许重边、自环。点号保留 1-based，build(1) 后 query(1,n)。题目保证连通才可直接解引用；通用图先检查 optional。此题验证行走，不能改成简单路径。O((n+m) · 64)，DFS 递归。

所需模板：XorBasis（第 326 页，22.3.1 节）、XorWalk（第 336 页，22.3.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4151

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    XorWalk graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        unsigned long long w;
        std::cin >> u >> v >> w;
        graph.add(u, v, w);
    }
    graph.build(1);
    std::cout << *graph.query(1, n) << '\n';
}
```

22.4 特征多项式

22.4.1 有限状态计数到线性递推：先证明阶数，再用 BM

当题目只问极大长度 N 的一个计数，有限状态 DP 可以先生成少量项，再用标量递推求远项。关键不是“样本看起来有规律”，而是先证明无限序列的递推阶数上界。本节给出从固定转移矩阵到该上界、再到可靠外推的完整链条。

22.4.1.1 先把状态、方向和输出写清楚

设状态数 $d \ge 1$ 固定，$v_n \in R^d$ 是长度 n 时各状态的计数**列向量**。令 A_{ij} 为一步从状态 j 到状态 i 的方案数，重边或不同字符走向同一状态时须累加，不是只填一个 1。若每步规则相同，就有

$$v_{n+1} = Av_n, \qquad v_n = A^n v_0, \qquad a_n = u^{\mathsf{T}} v_n.$$

u 可以是终态的 0/1 指示向量，也可以是固定权重；$a_0 = u^{\mathsf{T}} v_0$ 必须保留。这里 R 可取整数环或模 m 的交换环。若按行向量建模，矩阵方向须整体转置。

三个前提缺一不可：状态维数有限且不随 n 增长，转移矩阵固定，更新与输出均为线性。最短路的 min-plus 更新不是这里的环上线性运算；随位置任意变化的 A_n 也不能直接套一个 A。固定周期转移可加“相位”状态，固定仿射项可加一个恒为 1 的坐标，但阶数上界须按扩充后的维数重算。

22.4.1.2 Cayley–Hamilton 给出阶数至多 d

写特征多项式为

$$\chi_A(t) = \det(tI - A) = t^d + q_{d-1}t^{d-1} + \cdots + q_0.$$

Cayley–Hamilton 定理说 $\chi_A(A) = 0$。给出一个不依赖特征值或可对角化性的证明：由伴随矩阵恒等式，令 $B(t) = \operatorname{adj}(tI - A) = \sum_{i=0}^{d-1} B_i t^i$，有 $(tI - A)B(t) = \chi_A(t)I$。取 $B_{-1} = B_d = 0$、$q_d = 1$，比较 t^i 的系数得 $B_{i-1} - AB_i = q_i I$ $(0 \le i \le d)$。分别左乘 A^i 后求和，得到

$$\sum_{i=0}^{d} q_i A^i = \sum_{i=0}^{d} \left(A^i B_{i-1} - A^{i+1} B_i\right) = 0.$$

中间项逐个抵消，两端为零。这才是矩阵多项式求值的证明；不能把标量行列式里的 t 直接换成矩阵，写成“$\det(AI - A) = 0$”。整个证明没有除法，因此对交换环同样成立。

把 $\chi_A(A) = 0$ 左乘 $u^{\mathsf{T}} A^n$、右乘 v_0，便有

$$\boxed{a_{n+d} = -\sum_{i=0}^{d-1} q_i a_{n+i} \quad (n \ge 0).}$$

故从下标 d 起就有一个至多 d 阶的递推，不必等到状态分布“稳定”。若接口约定 $a_n = \sum_{j=1}^{k} c_{j-1} a_{n-j}$，直接用特征多项式时取 $k = d$、$c_j = -q_{d-1-j}$。这里只需存在性和上界，**不必先算出整个特征多项式**才能调用 BM。

22.4.1.3 为什么前 2d 项足以恢复这个标量序列

现在限定在素数域 $\mathbb{F}_p$ 中。取精确的前 $2d$ 项 $a_0, \ldots, a_{2d-1}$，BM 返回该前缀的最短递推，阶数记为 k。已知 d 阶递推存在，故 $k \le d$。按返回系数及前 k 个初值延伸出无限序列 b，它与 a 的前 $2d$ 项相同。

令前移算子 E 满足 $(Ea)_n = a_{n+1}$，并令 $g(t) = t^k - c_0 t^{k-1} - \cdots - c_{k-1}$。则 $\chi_A(E)a = 0$、$g(E)b = 0$ 都从 $n = 0$ 成立。多项式算子可交换，因而差序列 $h = a - b$ 满足

$$\chi_A(E)g(E)h = 0, \qquad \deg(\chi_A g) = d + k \le 2d.$$

这是一个**首**一递推；h 的前 $2d$ 项全零，依次递推便知所有项全零。所以外推正确。不能无依据地说“两个至多 d 阶序列之差仍至多 d 阶”；此处用乘积得到的上界是 $2d$。$k = 0$ 时约定 b 为全零序列、$g = 1$，同一论证仍成立。

恢复的是指定输入与输出的标量递推。例如 $A = \operatorname{diag}(1, 2)$、$u = v_0 = (1, 0)^{\mathsf{T}}$，则 $a_n = 1$，BM 只需返回 [1]；但 $A - I \ne 0$，矩阵最小多项式仍是 $(t-1)(t-2)$。在域上，标量最小消去多项式整除矩阵最小多项式，后者整除 χ_A；不能凭一次投影把标量递推当成对所有向量都有效的矩阵恒等式。

22.4.1.4 模型：不含连续子串 101 的二进制串

状态保存末尾与禁串前缀的最长匹配，依次为 $\varepsilon, 1, 10$。三者都是合法终态；到达完整 101 的转移丢弃。追加字符的转移为

当前状态	0	1
ε	ε	1
1	10	1
10	ε	丢弃

$$A=\begin{pmatrix}1&0&1\\1&1&0\\0&1&0\end{pmatrix},\quad v_0=\begin{pmatrix}1\\0\\0\end{pmatrix},\quad u=\begin{pmatrix}1\\1\\1\end{pmatrix}.$$

这里 10 后接 0 变为状态 ε，后接 1 才违法；1 后接 1 仍在状态 1。直接算得

$$\chi_A(t)=t(t-1)^2-1=t^3-2t^2+t-1,\qquad (a_0,\dots,a_5)=(1,2,4,7,12,21),$$

于是 $a_n=2a_{n-1}-a_{n-2}+a_{n-3}$ $(n\ge 3)$。模 $p=998244353$ 的现有接口可按下列方向衔接；调用者用 `unsigned long long` 声明目标零基下标 N：

```
constexpr int p = 998244353;
vector<int> s{1, 2, 4, 7, 12, 21}; // a[0..2*d-1], d = 3
vector<int> c = berlekamp_massey(s, p); // {2, p-1, 1}
vector<int> init(s.begin(), s.begin() + c.size());
int answer = recurrence_nth(init, c, N, p);
```

若模数改变，应先对样本及系数正规化到 $[0,p)$，并在**目标素数域**内恢复递推；模一个素数得到的短递推，不保证在整数或另一个模数下有效。

22.4.1.5 按目标选算法，并把采样成本算进去

设 e 是存储的非零转移数，采样用稀疏边表更新、清零长度 d 的向量并做一次点积。前 $2d$ 项需要 $O(d(e+d))$ 时间、$O(e+d)$ 空间；稠密矩阵采样为 $O(d^3)$。本库 BM 在每个非零偏差处用二进制幂求逆，因此这段采样的 BM 后处理为 $O(d^2+d\log p)$ 时间、$O(d)$ 额外空间，不能把求逆无条件写成常数时间。

直接 DP。N 不大或要整个前缀时，稀疏实现为 $O(N(e+d))$。

矩阵幂。要完整 v_N，或复用多种初值/输出时，考虑第 22.1.1 节（第 310 页）的 `ModMatrix<p>::power(A,N)`。当前稠密实现为 $O(d^3\log(N+1))$ 时间、$O(d^2)$ 空间；求出矩阵幂后，可复用于不同初值。标量 BM 的系数一般不能如此复用。

小阶标量递推。只要固定 u, v_0 的远项时，先采样与 BM，再调用第 20.8.1 节（第 266 页）的 `recurrence_nth(init,c,N,p)`，单次 $O(k^2\log(N+1))$ 时间、$O(k)$ 额外空间。

大阶快速求项。k 较大时，第 20.8.2 节（第 268 页）提供 `BostanMori<p,g>` 的 `recurrence(init,c,N)`，在固定模数下为 $O(k\log(k+1)\log(N+1))$ 时间、$O(k)$ 空间。参数须转换为其 `Poly`，p 为素数、g 为原根；非初值查询要求 $2k+1\le$ `NttConvolution<p,g>::max_size`，不能对任意素数不加检查地使用。

BM 接口见第 27.1 节（第 367 页）。两种远项接口都要求初值**恰好**有 k 项，使用 `unsigned long long` 下标；$k=0$ 返回 0，$N<k$ 返回初值。Bostan–Mori 的快速求项不会消掉前面的采样和 BM 成本。

合数模数分清存在性与恢复算法。模 m 时，Cayley–Hamilton 仍保证递推存在，但本库 BM 用 x^{p-2} 求逆，只支持素数模数。若已独立得知正确递推，`recurrence_nth` 只用加乘及首一多项式约减，支持 $2\le m\le$ `INT_MAX` 的合数模数；初值和系数仍须在 $[0,m)$。`ModMatrix` 的乘法和非负次幂也不需要逆元。不要把这个结论扩展到求逆矩阵或当前 NTT 版 Bostan–Mori。

少取一项与删除末尾零的反例。取 $A_\lambda=\begin{pmatrix}0&\lambda\\1&0\end{pmatrix}$、$v_0=(1,0)^{\mathsf T}$、$u=(0,1)^{\mathsf T}$。$\lambda=0,1$ 的两种二状态模型都有前三项 $0,1,0$，第四项却分别为 0、1；即使知道 $d=2$，$2d-1$ 项也不能普遍保证唯一外推。$\lambda=0$ 时矩阵幂零，序列为 $0,1,0,0,\dots$，BM 的长度 2 系数 $[0,0]$ 不能删成空数组：它保留了从下标 2 才开始适用的递推，空数组会错误抹掉初始的 1。额外保留若干未参与 BM 的项做交叉检查很有用，但**检查通过不代替阶数证明**。

来源。定理与中文导航核对 OI Wiki：特征多项式；伴随矩阵系数证明参照 IIT Kanpur：Eigenvalues and Eigenvectors 的 Cayley–Hamilton 第二证明。算法概念另核对 OI Wiki 的 BM、常系数齐次线性递推及 矩阵。模型、外推论证与接口成本按本库重新整理；本节不新增矩阵最小多项式实现或在线 AC 覆盖。

第二十三章　线性规划

23.1　单纯形法

23.1.1　线性规划：两阶段单纯形

Simplex(A,b,c) 求 max c · x，约束 Ax≤b、x≥0；A 为 m×n，b 长 m，c 长 n，变量/约束均 0 基。构造后只调用一次 solve()，返回 status、value、x；optimal 时 value 与 x 有效，infeasible/unbounded 时只读 status。允许负 b、冗余约束及零维；内部表与 pivot/improve 仅供实现使用。

long double、绝对 eps=1e-12。两阶段人工变量法；按变量编号选入基、最小比值并按基变量编号解平局。精确运算下 Bland 规则避免循环，浮点容差下不保证任意病态输入的正确分类或终止；勿把 AC 视为精确 LP 保证。先缩放数量级，求解后复核原约束和目标值。每次换基 O((m+1)(n+1))，空间同阶；总换基次数最坏指数级。

```
struct Simplex
{
    using Real = long double;
    static constexpr Real eps = 1e-12L;
    enum Status { optimal, infeasible, unbounded };
    struct Result
    {
        Status status;
        Real value;
        vector<Real> x;
    };
    int n, m;
    vector<int> basic, nonbasic;
    vector<vector<Real>> d;

    Simplex(const vector<vector<Real>> &a, const vector<Real> &b,
            const vector<Real> &c) : n(c.size()), m(b.size()),
            basic(m), nonbasic(n + 1), d(m + 2, vector<Real>(n + 2))
    {
        assert((int)a.size() == m);
        for (int i = 0; i < m; i++)
        {
            assert((int)a[i].size() == n);
            for (int j = 0; j < n; j++) d[i][j] = a[i][j];
            basic[i] = n + i;
            d[i][n] = -1;
            d[i][n + 1] = b[i];
        }
        iota(nonbasic.begin(), nonbasic.end(), 0);
        nonbasic[n] = n + m;
        for (int j = 0; j < n; j++) d[m][j] = -c[j];
        d[m + 1][n] = 1;
    }
```

换基（接上页结构体）：

```
    void pivot(int r, int s)
    {
        Real inv = 1 / d[r][s];
        for (int i = 0; i < m + 2; i++)
        {
            if (i == r) continue;
            Real scale = d[i][s] * inv;
            for (int j = 0; j < n + 2; j++)
                if (j != s) d[i][j] -= scale * d[r][j];
        }
        for (int j = 0; j < n + 2; j++)
            if (j != s) d[r][j] *= inv;
        for (int i = 0; i < m + 2; i++)
            if (i != r) d[i][s] *= -inv;
        d[r][s] = inv;
        swap(basic[r], nonbasic[s]);
    }
```

表的含义 第 i 行表示基本变量等于右端值减去非基本变量的线性组合。目标行同样表示当前目标常数减去约化系数乘变量。负约化系数表示增加该变量可改善目标；仅系数为正的约束限制增加量，故用右端除系数的最小值选择离基。若没有限制行则沿该方向无界。

人工变量与两阶段 对每条约束引入同一个人工变量 t，系数为-1。从最负的右端行让 t 入基即得到可行起点，第一阶段最大化-t。最优值小于零说明原问题无解；为零时把仍在基内的人工变量换出，零冗余行可以保留。原目标行始终参加换基，第二阶段禁止人工变量入基，直接优化原目标。

选择入基与出基变量（接上页结构体）：

```
    bool improve(int objective)
    {
        while (true)
        {
            int s = -1;
            for (int j = 0; j <= n; j++)
            {
                if (nonbasic[j] == n + m) continue;
                if (d[objective][j] >= -eps) continue;
                if (s == -1 || nonbasic[j] < nonbasic[s]) s = j;
            }
            if (s == -1) return true;
            int r = -1;
            for (int i = 0; i < m; i++)
            {
                if (d[i][s] <= eps) continue;
                if (r == -1)
                {
                    r = i;
                    continue;
                }
                Real x = d[i][n + 1] / d[i][s];
                Real y = d[r][n + 1] / d[r][s];
                if (x < y - eps || (abs(x - y) <= eps && basic[i] < basic[r]))
                    r = i;
            }
            if (r == -1) return false;
            pivot(r, s);
        }
    }
```

两阶段求解与方案恢复（接上页结构体）：

```
    Result solve()
    {
        int r = -1;
        for (int i = 0; i < m; i++)
            if (r == -1 || d[i][n + 1] < d[r][n + 1]) r = i;
        if (r != -1 && d[r][n + 1] < -eps)
        {
            pivot(r, n);
            improve(m + 1);
            if (d[m + 1][n + 1] < -eps) return {infeasible, 0, {}};
            for (int i = 0; i < m; i++)
            {
                if (basic[i] != n + m) continue;
                int s = -1;
                for (int j = 0; j <= n; j++)
                    if (abs(d[i][j]) > eps &&
                        (s == -1 || nonbasic[j] < nonbasic[s])) s = j;
                if (s != -1) pivot(i, s);
            }
        }
        if (!improve(m)) return {unbounded, 0, {}};
        vector<Real> x(n);
        for (int i = 0; i < m; i++)
            if (basic[i] < n) x[basic[i]] = d[i][n + 1];
        return {optimal, d[m][n + 1], x};
    }
};
```

退化与数值 在精确运算下，入基选最小变量编号，最小比值并列时选最小基本变量编号，是 Bland 防循环规则。这里按 eps 判断约化系数和比值并列，所以不承诺浮点下严格防循环。极小系数、严重消去或尺度悬殊都可能改变分类；先对模型缩放，并用原输入复核返回方案。需要精确判定时应使用有理数算法，不能仅缩小 eps。

模板题使用：P13337 线性规划

题意：n,m≤100，整数系数绝对值 ≤100，题目保证随机数据。最大化 c · x，Ax≤b、x≥0。输出最优值及任一最优解，或 Infeasible/Unbounded；允许绝对或相对误差 1e-6。

所需模板：Simplex（第 341 页，23.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P13337

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long double> c(n), b(m);
    vector<vector<long double>> a(m, vector<long double>(n));
    for (auto &v : c) cin >> v;
    for (int i = 0; i < m; i++)
    {
        for (auto &v : a[i]) cin >> v;
        cin >> b[i];
    }
    Simplex lp(a, b, c);
    auto result = lp.solve();
    if (result.status == Simplex::infeasible) cout << "Infeasible\n";
    else if (result.status == Simplex::unbounded) cout << "Unbounded\n";
    else
    {
        cout << fixed << setprecision(12) << result.value << '\n';
        for (auto x : result.x) cout << x << ' ';
        cout << '\n';
    }
}
```

第二十四章　概率论

24.1　随机变量的数字特征

24.1.1　期望选型：计数拆指标，等待拆尾和

先确定随机对象。均匀随机排列、逐步等概率转移和等概率选择最终方案是三种不同模型。例如从起点等概率走向 A 或 B，A 只有一个终点，B 有两个等概率终点：三个终点概率为 $1/2, 1/4, 1/4$，不是各 $1/3$。不能直接用路径条数代替路径概率。

计数型答案：给每个对象一个指示变量。若 $X = \sum_{i=1}^{m} I_i$，其中 $I_i = [\text{事件} A_i \text{ 发生}]$，则

$$\mathbb{E}X = \sum_{i=1}^{m} \Pr(A_i).$$

有限和的期望线性性不要求独立；一般实值变量应保证期望存在，避免相减两个无穷大。随机排列的不动点数为 $X = \sum_i [\pi(i) = i]$，每个位置固定的概率为 $1/n$，所以 $n \geq 1$ 时 $\mathbb{E}X = 1$。这比先求全部不动点个数的分布容易。$n = 2$ 时两个指示变量完全相同，恰好说明它们并不独立。

线性性不能推广成 $\mathbb{E}(XY) = \mathbb{E}X\mathbb{E}Y$。令 $X = Y$ 为一次公平硬币正面的指示变量，则左边为 $1/2$、右边为 $1/4$。方差应写成

$$\operatorname{Var}\left(\sum_i X_i\right) = \sum_i \operatorname{Var}(X_i) + 2\sum_{i<j} \operatorname{Cov}(X_i, X_j).$$

两两独立足以消去协方差；仅仅知道各自成功概率不足以计算方差。

等待时间或最大值：先问是否超过阈值。非负整数随机变量逐点满足 $X = \sum_{k\geq 1} [X \geq k]$，因此

$$\mathbb{E}X = \sum_{k\geq 1} \Pr(X \geq k) = \sum_{k\geq 0} \Pr(X > k).$$

非负项允许交换求和，结果也可能是 $+\infty$。两种写法的起点相差一。例如两个独立公平六面骰子的最大值 M 满足 $\Pr(M < k) = ((k-1)/6)^2$，故

$$\mathbb{E}M = \sum_{k=1}^{6} \left(1 - \frac{(k-1)^2}{36}\right) = 6 - \frac{55}{36} = \frac{161}{36}.$$

这里乘出平方用到了两个骰子独立；若两个骰子总是同点，期望就变成 $7/2$。

覆盖过程：按完成一件新事的阶段拆分。每次独立均匀抽取 n 种券，已有 i 种时，下一张为新种类的概率为 $(n-i)/n$。含成功那次在内的等待时间期望是 $n/(n-i)$，收齐的期望为 $n\sum_{j=1}^{n} 1/j$。每次抽样方式必须保持不变；非均匀抽券时，“已见种类数”通常不是充分状态，因为尚缺哪几种会改变转移概率，不能直接套调和数。

24.1.2　条件期望：先定义状态，再解自环

状态必须保存决定未来分布所需的信息。若到达终点后不再花费，设终点 t 的 $E_t = 0$。一次转移从 i 到 j 的概率为 p_{ij}，本次费用为 c_{ij}，则第一步分析给出

$$E_i = \sum_j p_{ij}(c_{ij} + E_j) = b_i + \sum_j p_{ij}E_j, \qquad b_i = \sum_j p_{ij}c_{ij}.$$

每步统一花费 1 时才有 $b_i = 1$；已经是终点时不能再多加一次。若目标是命中某个好终点的概率，则好终点边界为 1、坏终点为 0，非终点方程为 $h_i = \sum_j p_{ij}h_j$，没有常数 1。若过程可能永不终止，命中概率是边界问题的最小非负解，不能把任意代数解当成概率。

只有自环时可以局部移项。若 $p_{ii} < 1$，则

$$E_i = \frac{b_i + \sum_{j \neq i} p_{ij} E_j}{1 - p_{ii}}.$$

其余边若指向已算出的状态，就按依赖顺序做 DP。不能在存在 $i \to j \to i$ 时只去掉自环后仍当 DAG。每次独立以概率 $q > 0$ 成功，失败回到原状，每次花费 1，有 $E = 1 + (1-q)E$，故 $E = 1/q$。当 $q = 0$ 时永不成功，不能继续做除法。若统计“成功前失败次数”，当前成功不计费用，结果才是 $(1-q)/q$。

小例：等到连续两个正面。公平硬币独立抛掷，状态 0 表示当前没有末尾正面，状态 1 表示末尾恰有一个正面，状态 2 表示已完成。第一步方程为

$$E_0 = 1 + \tfrac{1}{2}E_0 + \tfrac{1}{2}E_1, \qquad E_1 = 1 + \tfrac{1}{2}E_0, \qquad E_2 = 0.$$

解得 $E_0 = 6, E_1 = 4$。错误地把每两次抛掷作为成功率 $1/4$ 的独立尝试会得到 8，它只检查不重叠的分组，漏掉跨组的连续正面。反过来把所有相邻两次都当独立试验也不成立。

有环时检查吸收，再选择消元。只保留起点可达、尚未吸收的状态，记它们之间的转移矩阵为 Q。有限期望向量满足 $(I - Q)E = b$；稠密直接消元的时间为 $O(s^3)$、空间为 $O(s^2)$，其中 s 是非终点状态数。DAG、链、带状矩阵或可按强连通分量分块的问题应先利用结构，不必一律构建全局稠密矩阵。概率极小时实数消元可能病态；残差小也不保证答案相对误差小。实数接口见GaussReal（第 364 页，26.3.3 节）；其秩与一致性按显式阈值判定，不能代替精确可解性或下述吸收条件的证明。

对有限状态、固定转移概率的链，若每个可达非终点都有一条正概率路径通向吸收态，则存在统一的 L 和 $\varepsilon > 0$，使任何非终点在未来 L 步内吸收的概率至少为 ε。由此 $\Pr(T > kL) \le (1-\varepsilon)^k$，所以几乎必然吸收且 $\mathbb{E}T \le L/\varepsilon < \infty$。等价的图判据是：可达区域中没有封闭的非吸收强连通分量。这里的边只取正概率转移，且每步费用有界时累计费用也具有有限绝对期望。

两类反例不能混淆。以概率 $1/2$ 立即终止、以概率 $1/2$ 进入永久自环，有正概率永不吸收，停止时间期望为无穷大。另取正整数随机变量 T，令 $\Pr(T = k) = 1/(k(k+1))$。概率和为 1，所以 T 几乎必然有限；但 $\Pr(T \ge k) = 1/k$，故 $\mathbb{E}T = \infty$。第二例说明无限状态或无限支撑下，“几乎必然终止”不足以推出有限期望，不能丢掉上述有限状态条件。

24.1.3 模概率：可逆的不只是输入分母

竞赛中的“概率对素数 p 取模”不是一个介于 0 与 1 的数，而是有理数的代数表示。既约分数 a/b 只有在 $p \nmid b$ 时，才能映射为 $a\,b^{-1} \pmod p$。模值不能比较大小、判定真实概率是否很小，也不能由模值为零断言真实概率为零。例如概率 $p/(p+1) > 0$，其模值为零。

直接转模方程的充分条件：每个转移概率和费用的分母均可逆，且 $I - Q$ 在模 p 下可逆。实数下可逆不保证模下可逆；自环移项中的 $1 - p_{ii}$ 也必须非零。例如每次以 $p/(p+1)$ 成功，期望为 $(p+1)/p$；原始转移概率的分母 $p+1$ 可逆，但期望的分母 p 不可逆。模方程变成 $0 \cdot E = 1$，无解不是概率模型矛盾，而是所求期望没有该模表示。

未约分表达式的分母不可逆，不代表最终答案一定不可表示。表达式 $p/p = 1$ 应先在整数或有理数层面约分，不能调用逆元求 $0/0$。类似地，模线性系统奇异时，可能存在有理数层面的消去或约分，但不能从任意模特解恢复唯一的真实期望。合数模数 m 下，分母须满足 $\gcd(b, m) = 1$，且不能用费马小定理求逆。

本库接口见第 19.2.2 节（第 133 页）的 `ModInt<p>` 与第 26.3.2 节（第 361 页）的 `GaussMod<p>::solve(a,n)`。前者的 `try_inv()` 用欧几里得，允许合数模数的单位元，不可逆时返回空值；`inv()` 与除法仍要求分母可逆。后者仍要求 p 为素数，不能随基础类型扩展而放宽；接收 s 行 $s+1$ 列增广矩阵，可填为 $[I - Q \mid b]$，显式传入变量数 s。先检查 `consistent`，再检查 `rank==s`，唯一解才读取 `particular`。这些接口不会替你证明吸收、期望有限或有理答案可取模。

第二十五章　博弈论

25.1　公平组合游戏

25.1.1　SG 的前提、递推与异或策略

先核对模型：双人轮流操作；双方在同一状态的合法操作完全相同（无偏游戏）；没有随机性与隐藏信息；不能行动者输（正常玩法）；状态图为有限 DAG。本节只在这些条件下使用普通 Sprague–Grundy 理论。“公平”在这里不表示胜率各半。

令 $g(u)$ 是状态的 SG 值，mex 表示集合中未出现的最小非负整数，则

$$g(u)=\operatorname{mex}\{g(v):u\to v\},\qquad \operatorname{mex}\varnothing=0.$$

按逆拓扑顺序计算；若递归记忆化，栈深度仍可能等于最长路径长度。若有 d 个后继，则 $g(u)\le d$，只需标记 $0,\dots,d$，超过 d 的后继值可以忽略。重复出现同一个后继值不影响 mex；用时间戳避免每个点都清空一张全局大表，可做到 $O(|V|+|E|)$ 时间和 $O(|V|+|E|)$ 空间（包含存图）。

$g(u)=0$ 时所有后继非零；$g(u)>0$ 时一定存在后继值为零。由 DAG 上归纳，零状态恰为先手必败态。不过 SG 数值不是胜率，也不是距离终点的步数。若题目只问一个不可分解游戏的胜负，布尔递推“存在后继必败则必胜”已足够；需要组合多个游戏时才必须保留比输赢更多的信息。

异或只针对不交和。一步恰好选择一个子游戏走一步，其他子游戏的状态及合法操作不变，且总游戏不能走时才输。此时组合值为 $G=g_1\oplus\cdots\oplus g_k$。$G=0$ 时任意一步都把一个 g_i 改为不同值，因此异或不再为零。$G\neq 0$ 时，取 G 最高为一位上也为一的某个 g_i，令 $h=g_i\oplus G<g_i$。mex 定义保证存在一步从该子状态走到 SG 值 h 的后继，操作后整体异或为零。实际输出方案时枚举这个子游戏的合法后继寻找 h，不能只输出一个目标数字。

如果一次允许同时改变两堆，或者一堆的操作会禁止另一堆的操作，就不是上述不交和。例如两堆各有一颗石子，允许取走任意一堆或同时取走两堆：把两堆当普通 Nim 会算出 $1\oplus 1=0$，但先手同时取完立刻获胜。

25.1.2　减法游戏小例与三类失效边界

一堆 n 颗石子，每次可取 $S=\{1,3,4\}$ 中一种数量且不能取成负数。空堆无合法操作，故 $g(0)=0$，对 $n>0$ 有

$$g(n)=\operatorname{mex}\{g(n-s):s\in S,\ s\le n\}.$$

从 $n=0$ 到 13 逐项得到

n	0	1	2	3	4	5	6	7	8	9	10	11	12	13
$g(n)$	0	1	0	1	2	3	2	0	1	0	1	2	3	2

例如 $g(4)=\operatorname{mex}\{g(3),g(1),g(0)\}=\operatorname{mex}\{1,0\}=2$，不是合法操作数 3。两堆 $(4,5)$ 的异或为 $2\oplus 3=1$，从第二堆取一颗得到 $(4,4)$ 即可归零。两堆 $(1,1)$ 各自都是必胜态，但组合为必败态，不能把多个布尔“必胜”作逻辑或。

表中前两段相等本身不是周期证明。本例先验证 $g(n+7)=g(n)$ 对 $0\le n\le 3$ 成立，随后对 $n\ge 4$，两边的后继都完整包含减 $1,3,4$；由更小下标的归纳假设，mex 集合相同，故周期 7 对所有 $n\ge 0$ 成立。对一般有限减法集合，SG 值不超过 $|S|$，且从足够大的 n 起只依赖前 $\max S$ 项，因此整个长度为 $\max S$ 的窗口重复后才保证后续周期。它只保证最终周期，不能仅凭几项重复猜出从零开始的周期或短周期。

- **反常玩法 (misère)。**最后行动者输时，普通 SG 的终点规则与异或结论不可直接搬用。只有一颗石子的 Nim 在正常玩法下先手赢，在反常玩法下先手输。反常 Nim 有专门结论，不意味着所有减法游戏都能“把最终答案取反”。

- **有环游戏（loopy）**。单点自环将要求 $g = \mathrm{mex}\{g\}$，不存在这样的非负整数。允许无限行走时还可能有和局；必须按题目规定处理重复局面、步数上限和无限对局。若加入剩余步数后形成 DAG，可以在扩展状态上分析，不能把环当成已访问后直接返回零。
- **有偏游戏（partizan）**。双方在相同局面可走步不同，不满足无偏性。仅一条“左方可走、右方不可走”的边，就不能用不带行动方信息的 mex 描述。可按完整状态做胜负搜索，但一般不能将各部分的普通 SG 值异或。

本地知识例证。 `tests/math_knowledge_probability.py` 使用有理数精确运算检查指示变量、骰子尾和、连续正面方程及模分母反例，并以直接胜负递归核对小规模减法游戏不交和。它验证本节例子，不是通用概率求解器、SG 模板或在线评测 AC。

第二十六章　数值算法

26.1　插值

26.1.1　拉格朗日插值与连续点优化

Lagrange(xs,ys,p) 使用任意横坐标，xs、ys 为等长 0-based vector<long long>；Lagrange(ys,p) 专用于横坐标 0..n-1。重新构造赋值可更换数据。要求 p 为素数，2≤p≤INT_MAX，点数 n≤p，任意横坐标取模后两两不同；不是仅要求原整数不同。query(k) 返回次数小于 n 的插值多项式在 k 处的值，允许 k、xs、ys 为完整 long long 范围，内部先归一化；空数据约定零多项式。通用预处理 $O(n^2+n \log p)$，连续点预处理 O(n+log p)，每次查询 O(n)，空间 O(n)。查询利用前缀积与滚动后缀积，无需求逆，因此 k 与样本点模 p 相等时也正确。若求 1^d+…+k^d，要求 0≤d≤p-2，先计算 0..d+1 的前缀和作为 ys，再构造 Lagrange(ys,p)；用快速幂生成这些样本另花 O(d log(d+1))。仅处理素数域中的精确插值，不适用于浮点曲线拟合或重复节点的导数约束。

```
struct Lagrange
{
    using ll = long long;
    int n, mod;
    vector<ll> x, w;

    ll norm(ll a) const
    {
        a %= mod;
        if (a < 0) a += mod;
        return a;
    }

    ll power(ll a, int b) const
    {
        ll result = 1;
        while (b)
        {
            if (b & 1) result = result * a % mod;
            a = a * a % mod;
            b >>= 1;
        }
        return result;
    }

    Lagrange(const vector<ll> &xs, const vector<ll> &ys, int p)
    {
        n = ys.size();
        mod = p;
        assert(p >= 2 && n <= p && xs.size() == ys.size());
        x.resize(n);
        w.resize(n);
        for (int i = 0; i < n; i++) x[i] = norm(xs[i]);
        for (int i = 0; i < n; i++)
        {
            ll denominator = 1;
            for (int j = 0; j < n; j++)
                if (i != j) denominator = denominator * norm(x[i] - x[j]) % mod;
```

```
            assert(denominator != 0);
            w[i] = norm(ys[i]) * power(denominator, mod - 2) % mod;
        }
    }

    Lagrange(const vector<ll> &ys, int p)
    {
        n = ys.size();
        mod = p;
        assert(p >= 2 && n <= p);
        x.resize(n);
        w.resize(n);
        if (!n) return;
        vector<ll> invfac(n, 1);
        ll factorial = 1;
        for (int i = 1; i < n; i++) factorial = factorial * i % mod;
        invfac[n - 1] = power(factorial, mod - 2);
        for (int i = n - 1; i > 0; i--) invfac[i - 1] = invfac[i] * i % mod;
        for (int i = 0; i < n; i++)
        {
            x[i] = i;
            w[i] = norm(ys[i]) * invfac[i] % mod * invfac[n - 1 - i] % mod;
            if ((n - 1 - i) & 1) w[i] = norm(-w[i]);
        }
    }

    ll query(ll k) const
    {
        k = norm(k);
        vector<ll> pre(n + 1, 1);
        for (int i = 0; i < n; i++) pre[i + 1] = pre[i] * norm(k - x[i]) % mod;
        ll answer = 0, suffix = 1;
        for (int i = n - 1; i >= 0; i--)
        {
            answer = (answer + w[i] * pre[i] % mod * suffix) % mod;
            suffix = suffix * norm(k - x[i]) % mod;
        }
        return answer;
    }
};
```

模板题使用：Luogu P4781

题意：给出 n 个横坐标模素数后互不相同的点，求次数小于 n 的插值多项式在 k 处的值，模 998244353。使用任意点构造函数。

所需模板：Lagrange（第 350 页，26.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4781

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n;
    long long k;
    std::cin >> n >> k;
    std::vector<long long> x(n), y(n);
    for (int i = 0; i < n; i++) std::cin >> x[i] >> y[i];
    Lagrange poly(x, y, 998244353);
    std::cout << poly.query(k) << '\n';
}
```

26.2 数值积分

26.2.1 有工作上限的自适应 Simpson 积分

integrate(f,a,b,eps,depth=20,limit=100000)，long double 绝对容差 eps>0，有限端点，0≤depth≤60、limit≥0。返回 value、error 估计、evaluations 与 met；met 仅指局部估计条件满足，不是实际精度保证。反向区间变号，零长度不调用 f；预算/深度/中点停滞/非有限计算会停止且 met=false。正常有限样本下保留粗或修正估计；无初始估计时 value 为 NaN；不可把失败值当答案。f 须返回确定值，异常原样传播，不能启用 fast-math。复用样本，O(求值数) 次固定字长运算、O(depth+1) 栈；需 numeric 的 midpoint、cmath、limits、utility。

```
struct AdaptiveSimpson
{
    using ld = long double;
    struct Result
    {
        ld value, error;
        int evaluations;
        bool met;
    };

    template<class F>
    static Result integrate(F &&f, ld a, ld b, ld eps,
                            int depth = 20, int limit = 100000)
    {
        const ld inf = numeric_limits<ld>::infinity();
        const ld nan = numeric_limits<ld>::quiet_NaN();
        if (!isfinite(a) || !isfinite(b) || !isfinite(eps) || eps <= 0 ||
            depth < 0 || depth > 60 || limit < 0)
            return {nan, inf, 0, false};
        if (a == b) return {0, 0, 0, true};
        if (limit < 3) return {nan, inf, 0, false};
        bool reverse = a > b;
        if (reverse) swap(a, b);
        auto area = [](ld l, ld r, ld u, ld v, ld w)
        {
            return (r - l) / 6 * (u + 4 * v + w);
        };
        ld fa = f(a), fm = f(midpoint(a, b)), fb = f(b);
        int used = 3;
        ld whole = area(a, b, fa, fm, fb);
        if (!isfinite(fa) || !isfinite(fm) || !isfinite(fb) || !isfinite(whole))
            return {nan, inf, used, false};
```

采样细分与预算合并（接上页同一 integrate 函数）：

```
        auto rec = [&](auto &&self, ld l, ld r, ld u, ld v, ld w,
                       ld s, ld tol, int left) -> Result
        {
            ld m = midpoint(l, r), x = midpoint(l, m), y = midpoint(m, r);
            if (m == l || m == r || x == l || x == m || y == m || y == r ||
                used > limit - 2)
                return {s, inf, 0, false};
            ld fx = f(x), fy = f(y);
            used += 2;
            ld sl = area(l, m, u, fx, v), sr = area(m, r, v, fy, w);
            ld sum = sl + sr, delta = sum - s;
            ld value = sum + delta / 15, error = abs(delta) / 15;
            if (!isfinite(fx) || !isfinite(fy) || !isfinite(sl) ||
                !isfinite(sr) || !isfinite(value) || !isfinite(error))
                return {s, inf, 0, false};
            if (error <= tol) return {value, error, 0, true};
            if (!left || tol / 2 == 0) return {value, error, 0, false};
            auto p = self(self, l, m, u, fx, v, sl, tol / 2, left - 1);
            auto q = self(self, m, r, v, fy, w, sr, tol / 2, left - 1);
            value = p.value + q.value;
            error = p.error + q.error;
            if (!isfinite(value)) return {s, inf, 0, false};
            return {value, error, 0, p.met && q.met && error <= tol};
        };
        auto answer = rec(rec, a, b, fa, fm, fb, whole, eps, depth);
        if (reverse) answer.value = -answer.value;
        answer.evaluations = used;
        return answer;
    }
};
```

接口用法演示：有限区间积分（接口演示）

演示约定：自定义协议，非 OJ 题：输入 a b eps depth limit，对 f(x)=1/(1+x*x) 积分；a,b 有限且绝对值≤100、有限 eps>0、0≤depth≤60、0≤limit≤200000。估计条件满足时打印 value，否则 FAILED；这不是严格误差证书。反向区间变号；零长度返回 0 且不求值。depth=0 仍允许根的 5 点比较，但不递归细分。示例避免奇点，任意函数仍可能发生采样混叠，须审查正则性并按已知间断分段。

所需模板：AdaptiveSimpson（第 353 页，26.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/src/compact/adaptive_simpson.hpp

```
int main()
{
    long double a, b, eps;
    int depth, limit;
    if (!(cin >> a >> b >> eps >> depth >> limit) ||
        !isfinite(a) || !isfinite(b) || abs(a) > 100 || abs(b) > 100 ||
        !isfinite(eps) || eps <= 0 || depth < 0 || depth > 60 ||
        limit < 0 || limit > 200000)
        return 1;
    auto f = [](long double x) { return 1 / (1 + x * x); };
    auto r = AdaptiveSimpson::integrate(f, a, b, eps, depth, limit);
    if (!r.met) cout << "FAILED\n";
    else cout << setprecision(20) << r.value << '\n';
}
```

26.2.2 数值积分：局部估计、误差预算与采样盲区

接口见第 26.2.1 节（第 353 页），用法见第 26.2.1 节（第 355 页）。先确认定积分存在，函数在闭区间采样点有定义，并分析间断、尖峰与振荡。端点奇异问题需要有依据的变量变换或其他适用算法，不能只靠不断减小 eps。

令 $m = (a+b)/2$，单段 Simpson 取

$$S(a,b) = \frac{b-a}{6}\big(f(a) + 4f(m) + f(b)\big).$$

它在精确算术下对至多三次多项式精确。若 $f \in C^4[a,b]$，有某个 $\xi \in (a,b)$ 满足

$$\int_a^b f(x)\,dx - S(a,b) = -\frac{(b-a)^5}{2880} f^{(4)}(\xi).$$

这是单段误差；等距复合 Simpson 的小步长为 h、小区间数为偶数时，整体误差形式是 $-(b-a)h^4 f^{(4)}(\xi)/180$，不能混用两个长度参数。

设 $S_2 = S(a,m) + S(m,b)$、$\delta = S_2 - S(a,b)$。若已进入主导误差按四次步长缩放的渐近区间，则二分约把误差缩为原来的 $1/16$，由此得到指标 $E = |\delta|/15$ 与修正值 $S_2 + \delta/15$。递归时把当前绝对误差预算各分一半，端点与中点采样直接传给孩子，只增加两个四分点样本。返回的 error 是叶指标之和，`met` 仅说明计算出的指标满足预算；它不是实际误差证书。浮点预算二分和求和也会舍入，本实现会再检查父区间的汇总指标不超预算。

光滑也会漏采样 在 $[0,1]$ 上取

$$f(x) = \big[x(x-\tfrac14)(x-\tfrac12)(x-\tfrac34)(x-1)\big]^2.$$

初始五个采样点都得到 0，故本接口返回 0、指标 0、`met=true`，但该非负光滑多项式在区间内部并非恒零，真实积分严格为正。因此“足够光滑”配合一次小差值仍不能给任意函数保证。已知尖峰、间断或频率应先用于分段；强制若干层只能降低某些漏采样风险，不能消灭它。

有限资源与数值合同 eps 为正的绝对容差。depth=0 仍允许根的五点比较，只有继续细分被禁止；limit 是实际回调次数上限。需要继续细分时深度耗尽或容差下溢，以及所需求值预算不足、四分点与端点重合或非有限计算，都会令 `met=false`。失败 value 可能只是覆盖该区间的粗估计，初始估计不可用时为 NaN；不要直接打印失败值。零长度不调用函数，反向区间只反转积分值的符号。函数必须给出确定的采样值，异常直接传播；有限样本也可能在中间加法或乘法溢出时失败。

参考：`https://dlmf.nist.gov/3.5#ii`；OI Wiki 固定版本 `math/numerical/integral.md` 与 WIDA jiangly 的 Simpson 文件。只补本有界自适应接口；不把目录中的等距复合实现、原应用函数或未读到的 kuangbin PDF 页视为完成。

26.3 高斯消元

26.3.1 压位模 2 高斯消元与全部解

GaussXor::solve(a,n)，a 为 m 行 n+1 列的 01 字符串增广矩阵，0≤n<INT_MAX，最末字符为右端项。返回 consistent、系数秩 rank、01 字符串特解 particular 与零空间基 kernel；所有解为特解异或任意基子集。无解时解数组为空；rank 仍为系数秩。有解时 kernel.size()=n-rank，唯一解的 kernel 为空。允许空方程组或零变量，原输入不变。自由列按原列号递增，基在各自自由列为 1、其他自由列为 0；没有最少 1 个数保证。时间 O(mn+mn min(m,n)/64+n^2)，除输入外空间 O(m ceil((n+1)/64)+n^2)；输出本身可能有 n^2 字符。全程精确异或，不用 eps。

```
struct GaussXor
{
    struct Solution
    {
        bool consistent;
        int rank;
        string particular;
        vector<string> kernel;
    };

    // m rows of n+1 characters: coefficients followed by the right-hand side.
    static Solution solve(const vector<string> &a, int n)
    {
        assert(n >= 0);
        int m = a.size(), w = n / 64 + 1, row = 0;
        vector<vector<unsigned long long>> b(m, vector<unsigned long long>(w));
        for (int i = 0; i < m; i++)
        {
            assert(a[i].size() == (size_t)n + 1);
            for (int j = 0; j <= n; j++)
            {
                assert(a[i][j] == '0' || a[i][j] == '1');
                if (a[i][j] == '1') b[i][j / 64] |= 1ULL << (j % 64);
            }
        }
        auto get = [&](int i, int j)
        {
            return b[i][j / 64] >> (j % 64) & 1;
        };
        vector<int> pos(n, -1);
        for (int col = 0; col < n && row < m; col++)
        {
            int p = row;
            while (p < m && !get(p, col)) p++;
            if (p == m) continue;
            swap(b[p], b[row]);
            for (int i = 0; i < m; i++)
                if (i != row && get(i, col))
                    for (int j = col / 64; j < w; j++) b[i][j] ^= b[row][j];
            pos[col] = row++;
        }
        for (int i = row; i < m; i++)
            if (get(i, n)) return {false, row, {}, {}};
```

构造特解与零空间基（接上页 solve）（接上页同一结构体）：

```
        Solution ans{true, row, string(n, '0'), {}};
        for (int j = 0; j < n; j++)
            if (pos[j] != -1) ans.particular[j] += get(pos[j], n);
        for (int j = 0; j < n; j++)
            if (pos[j] == -1)
            {
                string v(n, '0');
                v[j] = '1';
                for (int k = 0; k < n; k++)
                    if (pos[k] != -1) v[k] += get(pos[k], j);
                ans.kernel.push_back(v);
            }
        return ans;
    }
};
```

模板题使用：Library Checker system_of_linear_equations_mod_2

题意：给定模 2 矩阵 A 和右端 b，输出无解-1，或解空间维数、一个特解和全部零空间基。输出维数是变量数减系数秩，不是 rank。输入每行是连续 01 字符，先把 b 追加到对应行；行列数各至 4096。

所需模板：GaussXor（第 357 页，26.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/system_of_linear_equations_mod_2

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m, n;
    cin >> m >> n;
    vector<string> a(m);
    for (auto &s : a) cin >> s;
    string b;
    cin >> b;
    for (int i = 0; i < m; i++) a[i] += b[i];
    auto sol = GaussXor::solve(a, n);
    if (!sol.consistent)
    {
        cout << -1 << '\n';
        return 0;
    }
    cout << sol.kernel.size() << '\n';
    cout << sol.particular << '\n';
    for (auto &v : sol.kernel) cout << v << '\n';
}
```

应用题补充：POJ 1681 Painter's Problem

题意：方格 w 为白、y 为黄，每次翻转自身及四邻，求全部变黄的最少操作数，无解输出 inf。按 kuangbin2.10.1 建模，枚举特解异或零空间基的所有子集；不是直接数特解的 1。本地验证到 15×15，原 OJ 题面当前不可达，线上尚待验证。

所需模板：GaussXor（第 357 页，26.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：http://poj.org/problem?id=1681

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        int n;
        cin >> n;
        int m = n * n;
        vector<string> a(m, string(m + 1, '0'));
        vector<pair<int, int>> dir{{0, 0}, {0, 1}, {0, -1}, {1, 0}, {-1, 0}};
        for (int i = 0; i < n; i++)
        {
            string s;
            cin >> s;
            for (int j = 0; j < n; j++)
            {
                int id = i * n + j;
                a[id][m] = s[j] == 'w' ? '1' : '0';
                for (auto [di, dj] : dir)
                {
                    int u = i + di, v = j + dj;
                    if (0 <= u && u < n && 0 <= v && v < n) a[id][u * n + v] = '1';
                }
            }
        }
        auto sol = GaussXor::solve(a, m);
        if (!sol.consistent)
        {
            cout << "inf\n";
            continue;
        }
```

继续处理操作（接上页同一 main）：

```
        string x = sol.particular;
        int ans = m;
        function<void(int)> dfs = [&](int k)
        {
            if (k == (int)sol.kernel.size())
            {
                ans = min(ans, (int)count(x.begin(), x.end(), '1'));
                return;
            }
            dfs(k + 1);
            for (int j = 0; j < m; j++) x[j] ^= sol.kernel[k][j] - '0';
            dfs(k + 1);
            for (int j = 0; j < m; j++) x[j] ^= sol.kernel[k][j] - '0';
        };
        dfs(0);
        cout << ans << '\n';
    }
}
```

26.3.2 素数模高斯消元与全部解

代码依赖：ModInt（第 133 页，19.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

GaussMod<p>::solve(a,n) 接收 m 行 n+1 列增广矩阵，p 必须是素数，n≥0。返回 consistent、系数矩阵秩 rank、特解 particular 和零空间基 kernel；解为特解加基向量的任意线性组合。无解时解数组为空；空方程组需显式给出变量数。O(m n min(m,n)+n^2) 时间，矩阵及解空间输出 O(mn+n^2) 空间。只依赖 ModInt，不含行列式或矩阵树。

```
template <int mod> struct GaussMod
{
    using Z = ModInt<mod>;
    using Matrix = vector<vector<Z>>;

    struct Solution
    {
        bool consistent;
        int rank;
        vector<Z> particular;
        Matrix kernel;
    };

    // Augmented m x (n+1), prime modulus. Empty system needs explicit n.
    static Solution solve(Matrix a, int n)
    {
        int m = (int)a.size(), row = 0;
        vector<int> where(n, -1);
        for (const auto &v : a) assert((int)v.size() == n + 1);
        for (int col = 0; col < n && row < m; col++)
        {
            int p = row;
            while (p < m && !a[p][col].v) ++p;
            if (p == m) continue;
            swap(a[p], a[row]);
            Z inv = a[row][col].inv();
            for (int j = col; j <= n; j++) a[row][j] = a[row][j] * inv;
            for (int i = 0; i < m; i++)
                if (i != row && a[i][col].v)
                {
                    Z f = a[i][col];
                    for (int j = col; j <= n; j++) a[i][j] = a[i][j] - f * a[row][j];
                }
            where[col] = row++;
        }
        for (int i = row; i < m; i++)
            if (a[i][n].v) return {false, row, {}, {}};
```

```
        Solution ans{true, row, vector<Z>(n), {}};
        for (int j = 0; j < n; j++)
            if (where[j] != -1) ans.particular[j] = a[where[j]][n];
        for (int j = 0; j < n; j++)
            if (where[j] == -1)
            {
                vector<Z> v(n);
                v[j] = 1;
                for (int k = 0; k < n; k++)
                    if (where[k] != -1) v[k] = Z(0) - a[where[k]][j];
                ans.kernel.push_back(v);
            }
        return ans;
    }
};
```

模板题使用：Library Checker system_of_linear_equations

题意：模 998244353 下解 n 个方程、m 个未知数的线性方程组。输入先给系数矩阵，再给常数列。无解输出 -1；否则输出解空间维数、一个特解和齐次解基。

所需模板：ModInt（第 133 页，19.2.2 节）、GaussMod（第 361 页，26.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/system_of_linear_equations

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using G = GaussMod<998244353>;
    int n, m;
    cin >> n >> m;
    G::Matrix a(n, vector<G::Z>(m + 1));
    for (auto &row : a)
        for (int j = 0; j < m; j++) cin >> row[j].v;
    for (auto &row : a) cin >> row[m].v;
    auto s = G::solve(move(a), m);
    if (!s.consistent)
    {
        cout << -1 << '\n';
        return 0;
    }
    cout << s.kernel.size() << '\n';
    auto print = [&](const vector<G::Z> &row)
    {
        for (int j = 0; j < m; j++) cout << row[j].v << (j + 1 == m ? '\n' : ' ');
    };
    print(s.particular);
    for (auto &row : s.kernel) print(row);
}
```

应用题补充：SP2883 Widget Factory（POJ2947 同题）

题意：EOF 读 n,m，0 0 结束；每条记录累加重复编号，星期差包含首尾两天。模 7 消元后区分无解、多解；唯一解的余数 0、1、2 加 7，映射到 3..9 天。n,m≤300，每条记录至多 10000 个编号。

所需模板：ModInt（第 133 页，19.2.2 节）、GaussMod（第 361 页，26.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/SP2883

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using A = GaussMod<7>;
    vector<string> days = {"MON", "TUE", "WED", "THU", "FRI", "SAT", "SUN"};
    int n, m;
    while (cin >> n >> m && n)
    {
        A::Matrix a(m, vector<A::Z>(n + 1));
        for (int i = 0; i < m; i++)
        {
            int k;
            string start, finish;
            cin >> k >> start >> finish;
            int l = find(days.begin(), days.end(), start) - days.begin();
            int r = find(days.begin(), days.end(), finish) - days.begin();
            a[i][n] = r - l + 1;
            while (k--)
            {
                int x;
                cin >> x;
                a[i][x - 1] = a[i][x - 1] + 1;
            }
        }
        auto answer = A::solve(a, n);
        if (!answer.consistent)
            cout << "Inconsistent data.\n";
        else if (answer.rank < n)
            cout << "Multiple solutions.\n";
        else
        {
            for (int i = 0; i < n; i++)
            {
                int value = answer.particular[i].v;
                if (value < 3) value += 7;
                if (i) cout << ' ';
                cout << value;
            }
            cout << '\n';
        }
    }
    return 0;
}
```

26.3.3 实数高斯消元与数值解空间

solve(a, n, eps) 接收 m×(n+1) 增广矩阵，0≤n<INT_MAX、m≤INT_MAX，输入/中间值须有限可表示，0<eps<1。long double 精度依平台；先按每行系数最大绝对值归一化（不含右端项），再部分选主元；绝对值 ≤eps 的候选列在活动行中明确置零。对所有非零消元因子更新，不以 eps 跳过。返回 consistent、rank、particular 和 kernel：均描述最终数值消元模型，不是原系统精确可解性/秩的证书；剩余右端项绝对值 ≤eps 按零处理。原全零系数行保留原始右端尺度，n=0 也用此约定。相容时自由变量取零给特解，kernel 列出自由变量方向；任意大线性组合的原方程误差不受 eps 统一保证。非相容时两个解数组为空；输入按值传入不变。主消元通常 O(mn min(m,n)) 时间，另有初始化；O(m(n+1)+n) 工作空间，另有 O(n(n−rank)) 解基输出。近阈值、病态或非有限过程不能当成数学无解；须结合原方程残差与问题条件判断。正常浮点语义，禁 fast-math；assert 仅调试诊断，非发布模式运行时校验。

```
struct GaussReal
{
    using R = long double;
    using Matrix = vector<vector<R>>;

    struct Solution
    {
        bool consistent;
        int rank;
        vector<R> particular;
        Matrix kernel;
    };

    // Numerical classification after coefficient-row scaling, not exact rank.
    static Solution solve(Matrix a, int n, R eps)
    {
        assert(0 <= n && n < INT_MAX && a.size() <= (size_t)INT_MAX);
        assert(isfinite(eps) && 0 < eps && eps < 1);
        int m = a.size(), row = 0;
        vector<int> where(n, -1);
        for (auto &v : a)
        {
            assert(v.size() == (size_t)n + 1);
            R s = 0;
            for (int j = 0; j <= n; j++)
            {
                assert(isfinite(v[j]));
                if (j < n) s = max(s, abs(v[j]));
            }
            if (s > 0)
                for (R &x : v)
                {
                    x /= s;
                    assert(isfinite(x));
                }
        }
```

主元消元与数值解空间（接上页同一 solve 函数）：

```
        for (int col = 0; col < n && row < m; col++)
        {
            int p = row;
            for (int i = row + 1; i < m; i++)
                if (abs(a[i][col]) > abs(a[p][col])) p = i;
            if (abs(a[p][col]) <= eps)
            {
                for (int i = row; i < m; i++) a[i][col] = 0;
                continue;
            }
            swap(a[p], a[row]);
            R pivot = a[row][col];
            for (int j = col + 1; j <= n; j++)
            {
                a[row][j] /= pivot;
                assert(isfinite(a[row][j]));
            }
            a[row][col] = 1;
            for (int i = 0; i < m; i++)
                if (i != row && a[i][col] != 0)
                {
                    R f = a[i][col];
                    for (int j = col + 1; j <= n; j++)
                    {
                        a[i][j] -= f * a[row][j];
                        assert(isfinite(a[i][j]));
                    }
                    a[i][col] = 0;
                }
            where[col] = row++;
        }
        for (int i = row; i < m; i++)
            if (abs(a[i][n]) > eps) return {false, row, {}, {}};
        Solution ans{true, row, vector<R>(n), {}};
        for (int j = 0; j < n; j++)
            if (where[j] != -1) ans.particular[j] = a[where[j]][n];
        for (int j = 0; j < n; j++)
            if (where[j] == -1)
            {
                vector<R> v(n);
                v[j] = 1;
                for (int k = 0; k < n; k++)
                    if (where[k] != -1) v[k] = -a[where[k]][j];
                ans.kernel.push_back(v);
            }
        return ans;
    }
};
```

接口用法演示：实数线性系统（自定义数值协议）

演示约定：接口演示，非官方题目或线上 AC。输入 m n eps 及 m 行各 n+1 个实数；0≤m,n≤100 只是演示规模，输入和全部中间值有限，0<eps<1，需结合尺度/条件数选取。输出数值 consistent 与 rank；相容时再输出一个特解、自由方向个数和逐行方向向量。行系数最大值归一化后进行阈值消元，原全零系数行的右端项保持原尺度。不可把结果当作精确无解/唯一解证书。例中的 0.5 E0−0.5 E1=1、−0.5 E0+E1=1 来自连续两个正面等待时间，得到 E0=6,E1=4；概率模型有限性另行证明。

所需模板：GaussReal（第 364 页，26.3.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/src/compact/gauss_real.hpp

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m, n;
    long double eps;
    cin >> m >> n >> eps;
    GaussReal::Matrix a(m, vector<long double>(n + 1));
    for (auto &row : a)
        for (auto &x : row) cin >> x;
    auto r = GaussReal::solve(a, n, eps);
    cout << r.consistent << ' ' << r.rank << '\n';
    if (!r.consistent) return 0;
    cout << setprecision(20);
    for (auto x : r.particular) cout << x << ' ';
    cout << '\n' << r.kernel.size() << '\n';
    for (const auto &v : r.kernel)
    {
        for (auto x : v) cout << x << ' ';
        cout << '\n';
    }
}
```

第二十七章　Berlekamp–Massey 算法

27.1　Berlekamp–Massey：最短线性递推

berlekamp_massey(s,p) 要求 p 为素数，2≤p≤INT_MAX，输入均在 [0,p)；返回 c，使 s[i]=∑ c[j-1]s[i-j] (j=1..k，i≥k)。空序列或全零序列返回空系数。有限前缀可能对应多种未来，不能无条件外推；若已知原序列满足阶数至多 d 的齐次线性递推，提供至少 2d 项才能恢复可外推的递推。模运算次数 O(Nk+N log p)，后项来自逐次求逆；空间 O(N)。不适用于合数模数或浮点拟合。

```
// Prime p; normalized input. s[i] = sum(c[j-1] * s[i-j]), i >= c.size().
vector<int> berlekamp_massey(const vector<int> &s, int p = 998244353)
{
    assert(p >= 2);
    vector<int> c{1}, b{1};
    int len = 0, shift = 1, last = 1;
    for (int i = 0; i < (int)s.size(); i++)
    {
        assert(0 <= s[i] && s[i] < p);
        long long d = s[i];
        for (int j = 1; j <= len; j++) d = (d + 1LL * c[j] * s[i - j]) % p;
        if (d == 0)
        {
            shift++;
            continue;
        }
        long long inv = 1, x = last;
        for (int e = p - 2; e; e >>= 1)
        {
            if (e & 1) inv = inv * x % p;
            x = x * x % p;
        }
        long long factor = d * inv % p;
        auto old = c;
        if (c.size() < b.size() + shift) c.resize(b.size() + shift);
        for (int j = 0; j < (int)b.size(); j++)
            c[j + shift] = (c[j + shift] - factor * b[j] % p + p) % p;
        if (2 * len <= i)
        {
            len = i + 1 - len;
            b = old;
            last = d;
            shift = 1;
        }
        else
            shift++;
    }
    c.resize(len + 1);
    c.erase(c.begin());
    for (int &x : c) x = x == 0 ? 0 : p - x;
    return c;
}
```

模板题使用：Library Checker find_linear_recurrence

题意：给模 998244353 下的有限序列，求最短线性递推。c[0] 乘前一项，输出阶数及系数；空序列或全零序列可以是 0 阶。递推只保证对已给序列成立，不能仅凭前缀推定未知后续。

所需模板：berlekamp_massey（第 367 页，27.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/find_linear_recurrence

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    auto c = berlekamp_massey(a);
    cout << c.size() << '\n';
    for (int x : c) cout << x << ' ';
    cout << '\n';
}
```

联合使用示例见第 267 页（Luogu P5487），其中直接调用本模板。

第五部分

数据结构

第二十八章　哈希表

28.1　GNU gp_hash_table 与 cc_hash_table

GNU libstdc++ 扩展，包含 ext/pb_ds/assoc_container.hpp。gp_map<K,V> 为探测哈希表，cc_map<K,V> 为碰撞链接哈希表；K 为不超过 64 位的整数，复用下方 hash64。两者支持 operator[]、find、insert、erase(key)、clear、size、empty；insert 不覆盖已有键，operator[] 对不存在的键插入默认值。只查询时用 find 判断 end。通常查改摊还 O(1)，最坏 O(n)，空间 O(n)，不支持排名或有序前驱。纯键值查改可先试 gp_map；需要在插入/删除其他键后保留找到的元素引用时可考虑 cc_map，其节点分配和链接有成本，速度仍须按题实测。gp_map 修改后重新 find；cc_map 的点迭代器不等于可递增的遍历迭代器，删除该元素、clear 或销毁后不能保留引用。默认策略不直接调用 resize，也不照搬 unordered_map::reserve。hash64 使用进程内固定随机种子，无符号回绕；只针对整数键，不保证已知种子下的最坏碰撞。历史 associative_array AC 仅覆盖 gp_map，cc_map 另行验证。

需要排名与有序前驱时，PBDS 平衡树见第 473 页。

```
struct hash64
{
    static uint64_t mix(uint64_t x)
    {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }

    size_t operator()(uint64_t x) const
    {
        static const uint64_t seed =
            chrono::steady_clock::now().time_since_epoch().count();
        return mix(x + seed);
    }
};

template <class K, class V> using gp_map = __gnu_pbds::gp_hash_table<K, V, hash64>;
template <class K, class V> using cc_map = __gnu_pbds::cc_hash_table<K, V, hash64>;
```

模板题使用：Library Checker associative_array

题意：维护整数键值映射，修改为覆盖赋值，查询不存在的键输出 0。gp_map 用 operator[] 写入，用 find 查询，避免查询插入新键。所需条目中包含 hash64。

所需模板：gp_map（第 370 页，28.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/associative_array

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    gp_map<uint64_t, uint64_t> a;
    while (q--)
    {
        int op;
        uint64_t k, v;
        cin >> op >> k;
        if (op == 0)
        {
            cin >> v;
            a[k] = v;
        }
        else
        {
            auto it = a.find(k);
            cout << (it == a.end() ? 0 : it->second) << '\n';
        }
    }
}
```

模板题使用：Library Checker associative_array

题意：同一关联数组题的 cc_map 用法：覆盖赋值，查询不存在的键输出 0。cc_map 与 hash64 均在 gp_map 条目中；本例独立执行验证，不借用 gp_map 的运行结果。

所需模板：gp_map（第 370 页，28.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/associative_array

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    cc_map<uint64_t, uint64_t> a;
    while (q--)
    {
        int op;
        uint64_t k, v;
        cin >> op >> k;
        if (op == 0)
        {
            cin >> v;
            a[k] = v;
        }
        else
        {
            auto it = a.find(k);
            cout << (it == a.end() ? 0 : it->second) << '\n';
        }
    }
}
```

第二十九章　并查集

29.1　并查集

29.1.1　并查集

dsu(n)，点号 0..n-1，n≥0。find(x) 路径减半；same(x,y) 判断同集合；merge(x,y) 把 y 的根接到 x 的根，已同集合返回 false。size(x) 返回所在集合大小。groups() 返回所有非空分组，组内点号递增、组间按当前根编号排列；可能压缩路径，但不改变分组。fa、sz 可读，sz 仅根处有效；不要外部改写。本版只做路径减半，不按大小或秩合并；单次 find 最坏 O(n)。空间 O(n)，groups 另需 O(n) 输出与工作空间，并执行 n 次 find。空结构可调用 groups，不能查询顶点。

```
struct dsu
{
    vector<int> fa, sz;

    dsu(int n) : fa(n), sz(n, 1) { iota(fa.begin(), fa.end(), 0); }

    int find(int x)
    {
        while (x != fa[x]) x = fa[x] = fa[fa[x]];
        return x;
    }

    bool same(int x, int y) { return find(x) == find(y); }

    bool merge(int x, int y)
    {
        x = find(x), y = find(y);
        if (x == y) return false;
        sz[x] += sz[y], fa[y] = x;
        return true;
    }

    int size(int x) { return sz[find(x)]; }

    vector<vector<int>> groups()
    {
        vector<vector<int>> res(fa.size());
        for (int i = 0; i < int(fa.size()); i++) res[find(i)].push_back(i);
        res.erase(
            remove_if(res.begin(), res.end(), [&](const auto &v) { return v.empty(); }),
            res.end());
        return res;
    }
};
```

模板题使用：Luogu P3367

题意：维护集合合并与两点是否同集合。题目点号 1..n，传给 dsu 前减 1；查询输出 Y 或 N。

所需模板：dsu（第 373 页，29.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3367

```
int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    dsu d(n);
    while (m--)
    {
        int op, x, y;
        scanf("%d%d%d", &op, &x, &y);
        --x, --y;
        if (op == 1)
            d.merge(x, y);
        else
            puts(d.same(x, y) ? "Y" : "N");
    }
}
```

29.1.2 可撤销并查集

RollbackDSU(n) 可使用 0..n（若题目 0-based 则不用 n）。find 不压缩路径，merge 按集合大小合并，已同集合返回 false 且不记历史；siz 仅根处有效。snapshot() 返回历史长度，退出递归分支时 rollback(t) 撤销此后成功合并。t 必须是当前历史的祖先快照，不能用已回滚分支的旧快照恢复版本。find/merge 为 O(log(n+1))，每次撤销成功合并 O(1)，空间 O(n)；适合版本树或时间分治。它按大小合并以保证回滚复杂度，接口与普通 dsu 的根连接方向不同。

```
struct RollbackDSU
{
    vector<int> fa, siz;
    vector<pair<int, int>> his;

    RollbackDSU(int n) : fa(n + 1), siz(n + 1, 1) { iota(fa.begin(), fa.end(), 0); }

    int find(int x) const
    {
        while (x != fa[x]) x = fa[x];
        return x;
    }

    int snapshot() const { return (int)his.size(); }

    bool merge(int x, int y)
    {
        x = find(x);
        y = find(y);
        if (x == y) return false;
        if (siz[x] < siz[y]) swap(x, y);
        his.push_back({y, siz[x]});
        fa[y] = x;
        siz[x] += siz[y];
        return true;
    }

    void rollback(int t)
    {
        assert(0 <= t && t <= snapshot());
        while (snapshot() > t)
        {
            auto [y, s] = his.back();
            his.pop_back();
            siz[fa[y]] = s;
            fa[y] = y;
        }
    }
};
```

模板题使用：Library Checker persistent_unionfind

题意：离线读入所有操作，按所依赖的版本建立树。版本 -1 映射到树根 0，操作 i 映射到 i+1；顶点仍是 0-based。递归进入版本前保存 snapshot，退出时 rollback，答案按原查询顺序输出。只有合并产生可引用版本；本例是离线解法，不提供在线任意版本访问。总时间 O(n+q log(n+1))，空间 O(n+q)，版本链递归深度可为 q。

所需模板：RollbackDSU（第 375 页，29.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/persistent_unionfind

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<int> type(q + 1), u(q + 1), v(q + 1), ans(q + 1);
    vector<vector<int>> g(q + 1);
    for (int i = 1; i <= q; i++)
    {
        int k;
        cin >> type[i] >> k >> u[i] >> v[i];
        g[k + 1].push_back(i);
    }
    RollbackDSU d(n);
    auto dfs = [&](auto &&self, int x) -> void
    {
        int t = d.snapshot();
        if (x)
        {
            if (type[x] == 0)
                d.merge(u[x], v[x]);
            else
                ans[x] = d.find(u[x]) == d.find(v[x]);
        }
        for (int y : g[x]) self(self, y);
        d.rollback(t);
    };
    dfs(dfs, 0);
    for (int i = 1; i <= q; i++)
        if (type[i] == 1) cout << ans[i] << '\n';
}
```

29.1.3 带势并查集与差值约束

PotentialDSU<T>(n)，n≥0，点号 0..n−1。merge(u,v,w) 尝试添加 potential[u]−potential[v]=w，返回是否与已接受信息一致；重复一致约束也返回 true，矛盾返回 false，已有逻辑关系不变（仍可能路径压缩）。diff(u,v) 返回 optional<T>，未连通为 nullopt，连通时为 u−v；合法零/负值不是失败标志。find 求根并压缩路径，size 返回连通块大小。T 须提供精确加法交换群的 +、−、== 及零 T(0)，不是非交换群或浮点近似比较；整数须保证所有中间运算不溢出，动态模数不可在实例存活时改变。n 点、每条输入差值绝对值 ≤W 时，(2n+1)W≤LLONG_MAX 是 long long 的一个保守充分条件。按大小合并，摊还 $O(\alpha(n))$ 次 T 运算、O(n) 空间；递归 find 未压缩前深度 O(log n)。fa/sz/d 为内部状态，不直接改写。原普通并查集的指定根合并方向与 bool 语义不变。

```
template <class T = long long> struct PotentialDSU
{
    vector<int> fa, sz;
    vector<T> d;

    PotentialDSU(int n) : fa(n), sz(n, 1), d(n, T(0))
    {
        iota(fa.begin(), fa.end(), 0);
    }

    int find(int u)
    {
        assert(0 <= u && u < (int)fa.size());
        int p = fa[u];
        if (p != u)
        {
            fa[u] = find(p);
            d[u] = d[u] + d[p];
        }
        return fa[u];
    }
```

约束合并、差值查询与分量大小（接上页同一结构体）：

```
    // Accept potential[u] - potential[v] = w iff it is consistent.
    bool merge(int u, int v, T w)
    {
        int a = find(u), b = find(v);
        if (a == b) return d[u] - d[v] == w;
        T t = d[u] - d[v] - w;
        if (sz[a] < sz[b])
        {
            swap(a, b);
            t = T(0) - t;
        }
        fa[b] = a;
        d[b] = t;
        sz[a] += sz[b];
        return true;
    }

    optional<T> diff(int u, int v)
    {
        int a = find(u), b = find(v);
        if (a != b) return nullopt;
        return d[u] - d[v];
    }

    int size(int u) { return sz[find(u)]; }
};
```

模板题使用：Library Checker Unionfind with Potential

题意：未知序列 a，1≤N,Q≤200000，模 998244353。0 u v x 尝试加入 a[u]−a[v]=x；与此前已接受信息一致输出 1，否则输出 0 并拒绝，不把矛盾边加入状态。1 u v 查询 a[u]−a[v] 的规范模值，未连通输出 −1；点号 0-based，0≤x<mod。重复一致、自约束及零差值均有效。先设置模整数的模数再构造结构，存活期间不改变它；复用已有加减与相等比较，不需要乘法逆元。

所需模板：PotentialDSU（第 377 页，29.1.3 节）、Barrett32（第 144 页，19.2.5 节）、mint（第 129 页，19.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/unionfind_with_potential

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    using Z = mint<>;
    Z::set_mod(998244353);
    PotentialDSU<Z> d(n);
    while (q--)
    {
        int t, u, v;
        cin >> t >> u >> v;
        if (t == 0)
        {
            int x;
            cin >> x;
            cout << d.merge(u, v, Z(x)) << '\n';
        }
        else
        {
            auto x = d.diff(u, v);
            cout << (x ? x->v : -1) << '\n';
        }
    }
}
```

29.1.4 时间线段树分治：离线动态连通性

代码依赖：RollbackDSU（第 375 页，29.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

TimeConnectivity(n,q)，点号 0..n（1-based 图不使用 0），时间 0..q-1。add(l,r,u,v) 登记边在半开区间 [l,r) 有效，允许空区间、重叠区间、重边和自环。run(visit) 按时间递增调用 visit(t,const RollbackDSU &d)，可用 find 判断连通、siz[find(u)] 取分量大小；只在回调内读取，不保留引用，不在遍历中改图。每次 run 重新建并查集，可重复调用，也可在两次 run 间加区间。q=0 不调用回调。n>=0 且 n<INT_MAX，0<=q<=INT_MAX/4，并须满足内存限制。K 个非空区间：存储 O(n+q+K log(q+1))；建区间 O(K log(q+1))，遍历 O(n+q+K log(q+1) log(n+2))，另加回调代价。依赖 RollbackDSU，不做路径压缩；DFS 保持递归。

```
struct TimeConnectivity
{
    int n, q;
    vector<vector<pair<int, int>>> edges;

    TimeConnectivity(int n, int q) : n(n), q(q), edges(4 * size_t(q) + 1)
    {
        assert(n >= 0 && n < INT_MAX && q >= 0 && q <= INT_MAX / 4);
    }

    // Edge (u,v) exists at times l <= t < r. Empty intervals are allowed.
    void add(int l, int r, int u, int v)
    {
        assert(0 <= l && l <= r && r <= q);
        assert(0 <= u && u <= n && 0 <= v && v <= n);
        if (l < r)
            insert(1, 0, q, l, r, {u, v});
    }

    template <class F> void run(F visit) const
    {
        RollbackDSU d(n);
        if (q > 0)
            dfs(1, 0, q, d, visit);
    }
```

时间区间插入与回滚递归（接上页同一结构体）：

```
private:
    void insert(int p, int l, int r, int a, int b, pair<int, int> e)
    {
        if (a <= l && r <= b)
        {
            edges[p].push_back(e);
            return;
        }
        int m = l + (r - l) / 2;
        if (a < m)
            insert(p * 2, l, m, a, b, e);
        if (m < b)
            insert(p * 2 + 1, m, r, a, b, e);
    }

    template <class F>
    void dfs(int p, int l, int r, RollbackDSU &d, F &visit) const
    {
        int saved = d.snapshot();
        for (auto [u, v] : edges[p])
            d.merge(u, v);
        if (r - l == 1)
            visit(l, static_cast<const RollbackDSU &>(d));
        else
        {
            int m = l + (r - l) / 2;
            dfs(p * 2, l, m, d, visit);
            dfs(p * 2 + 1, m, r, d, visit);
        }
        d.rollback(saved);
    }
};
```

应用题补充：P2147 洞穴勘测（离线解法）

题意：初始无边，读 Connect/Destroy/Query；合法增删且始终为森林。点号 1..n；加边时刻 a、删边时刻 b 配成 [a,b)，结束未删边配到 q。输出 Yes/No。竞赛应用，不是任意多重图完整接口的正式模板题。

所需模板：RollbackDSU（第 375 页，29.1.2 节）、TimeConnectivity（第 380 页，29.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2147

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    TimeConnectivity graph(n, q);
    map<pair<int, int>, int> start;
    vector<pair<int, int>> ask(q, {-1, -1});
    for (int t = 0; t < q; ++t)
    {
        string op;
        int u, v;
        cin >> op >> u >> v;
        if (u > v)
            swap(u, v);
        pair<int, int> e = {u, v};
        if (op == "Connect")
            start[e] = t;
        else if (op == "Destroy")
        {
            auto it = start.find(e);
            graph.add(it->second, t, u, v);
            start.erase(it);
        }
        else
            ask[t] = e;
    }
    for (auto [e, t] : start)
        graph.add(t, q, e.first, e.second);
    graph.run([&](int t, const RollbackDSU &d)
    {
        auto [u, v] = ask[t];
        if (u != -1)
            cout << (d.find(u) == d.find(v) ? "Yes" : "No") << '\n';
    });
    return 0;
}
```

接口用法演示：带初始边的多重图增删连通性（接口演示）

演示约定：EOF 多组 n,m,q，初始 m 条无向边，再读 q 行 + u v / - u v / ? u v。点号 1..n，允许重边和自环，删除须存在且只删一份；每次? 输出连通性 0/1 及 u 的分量大小。计数从 0 到 1 开启生存区间，从 1 到 0 关闭；初始边从 0 开始。n>=1，m+q<=INT_MAX。

所需模板：RollbackDSU（第 375 页，29.1.2 节）、TimeConnectivity（第 380 页，29.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    while (cin >> n >> m >> q)
    {
        TimeConnectivity graph(n, q);
        // value = {active multiplicity, start of positive multiplicity}
        map<pair<int, int>, pair<int, int>> active;
        for (int i = 0; i < m; ++i)
        {
            int u, v;
            cin >> u >> v;
            if (u > v)
                swap(u, v);
            ++active[{u, v}].first;
        }
```

继续处理操作（接上页同一 main）：

```
        vector<pair<int, int>> ask(q, {-1, -1});
        for (int t = 0; t < q; ++t)
        {
            char op;
            int u, v;
            cin >> op >> u >> v;
            if (op == '?')
            {
                ask[t] = {u, v};
                continue;
            }
            if (u > v)
                swap(u, v);
            pair<int, int> e = {u, v};
            auto &state = active[e];
            if (op == '+')
            {
                if (state.first == 0)
                    state.second = t;
                ++state.first;
            }
            else
            {
                assert(state.first > 0);
                --state.first;
                if (state.first == 0)
                    graph.add(state.second, t, u, v);
            }
        }
        for (auto [e, state] : active)
            if (state.first > 0)
                graph.add(state.second, q, e.first, e.second);
        graph.run([&](int t, const RollbackDSU &d)
        {
            auto [u, v] = ask[t];
            if (u != -1)
                cout << (d.find(u) == d.find(v)) << ' '
                     << d.siz[d.find(u)] << '\n';
        });
    }
    return 0;
}
```

模板题使用：LOJ121 动态图连通性

题意：初始空的无向简单图，点号 1..n；0 加不存在的边、1 删存在的边、2 查连通性，输出 Y/N。n≤5000，q≤500000；端点统一排序，加删配成半开生存区间，剩余边持续到 q。

所需模板：RollbackDSU（第 375 页，29.1.2 节）、TimeConnectivity（第 380 页，29.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://loj.ac/p/121

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    TimeConnectivity tree(n, q);
    map<pair<int, int>, int> start;
    vector<pair<int, int>> query(q, {-1, -1});
    for (int t = 0; t < q; t++)
    {
        int op, u, v;
        cin >> op >> u >> v;
        if (u > v) swap(u, v);
        pair<int, int> edge{u, v};
        if (op == 0)
            start[edge] = t;
        else if (op == 1)
        {
            auto it = start.find(edge);
            tree.add(it->second, t, u, v);
            start.erase(it);
        }
        else
            query[t] = edge;
    }
    for (auto [edge, l] : start)
        tree.add(l, q, edge.first, edge.second);
    tree.run([&](int t, const RollbackDSU &d)
    {
        auto [u, v] = query[t];
        if (u != -1)
            cout << (d.find(u) == d.find(v) ? 'Y' : 'N') << '\n';
    });
}
```

第三十章　堆

30.1　配对堆

30.1.1　PBDS 可修改、可合并优先队列

pheap<T,C> 默认比较器为 less<T>，固定 pairing_heap_tag，形成大根堆；小根堆用 greater<T>。

包含 ext/pb_ds/priority_queue.hpp。push(x) 返回 point_iterator 句柄；modify(h,x) 改值，erase(h) 删除指定元素。top/pop 要求非空，size/empty/clear 沿用容器接口。a.join(b) 把 b 全部移入 a，b 变空；原 b 元素句柄仍指向同一元素，此后应交给 a。a.swap(b) 后句柄随元素交换归属。split(pred,b) 将满足谓词的元素移入 b，本模板约定 b 原先为空。join/split 两个堆须不同，比较规则一致。删除元素、pop 对应堆顶、clear 或销毁后，相应句柄失效；不要把原堆句柄用于深复制的副本。top、push、join 为 O(1)；pop 摊还 O(log n)，modify/erase 的单次最坏可达 O(n)，不当作最坏常数。split/复制/clear 可能扫描整个堆。适合需要修改已入堆元素或合并的任务；只要普通入队出队时可用 std::priority_queue。复杂度按比较、复制标量或编号对为常数计，T 须满足容器的复制要求。

```
template <class T, class C = less<T>>
using pheap = __gnu_pbds::priority_queue<T, C, __gnu_pbds::pairing_heap_tag>;
```

```
pheap<int> a, b;
auto h = a.push(7);
a.modify(h, 2);
b.join(a);
b.erase(h);
```

模板题使用：Library Checker Double-Ended Priority Queue

题意：多重集合支持插入和删除一个最小/最大值；0≤n≤500000、1≤q≤500000，值在 $[-10^9,10^9]$，删除时保证非空。用两个大根配对堆分别保存 (-值,id)/(值,id)，同次插入保存两份句柄；删除时先 erase 另一堆的句柄，再 pop 当前堆。id 每次新插入都递增，重复值也可区分。删除后两份句柄失效，禁止再次使用。本题取负在 int 范围内；全 int64 值不能直接照搬取负。

所需模板：pheap（第 386 页，30.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/double_ended_priority_queue

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using H = pheap<pair<int, int>>;
    array<H, 2> q;
    vector<array<H::point_iterator, 2>> h;
    h.reserve(n + m);
    auto add = [&](int x)
    {
        int id = h.size();
        h.push_back({q[0].push({-x, id}), q[1].push({x, id})});
    };
    for (int i = 0; i < n; i++)
    {
        int x;
        cin >> x;
        add(x);
    }
    while (m--)
    {
        int t;
        cin >> t;
        if (t == 0)
        {
            int x;
            cin >> x;
            add(x);
        }
        else
        {
            int k = t - 1;
            auto [x, id] = q[k].top();
            cout << (k ? x : -x) << '\n';
            q[k ^ 1].erase(h[id][k ^ 1]);
            q[k].pop();
        }
    }
}
```

模板题使用：Library Checker Shortest Path

题意：非负权简单有向图，0 起点号，n 和 m 不超过 500000，权 $\leq 10^9$，s≠t。输出最短距离和一条有序路径，无解输出-1。小根配对堆存 (距离, 点)，每个堆内点保存句柄；松弛时已入堆则 modify 降键，否则 push。pop 后置 in=false，不再访问旧句柄。距离用 long long，pre 还原原边；零权边允许。这个用法展示 push/top/pop/modify，不覆盖 join/split。

所需模板：pheap（第 386 页，30.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/shortest_path

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v, w;
        cin >> u >> v >> w;
        g[u].push_back({v, w});
    }
    using P = pair<long long, int>;
    pheap<P, greater<P>> q;
    vector<decltype(q)::point_iterator> h(n);
    const long long inf = LLONG_MAX;
    vector<long long> d(n, inf);
    vector<int> pre(n, -1);
    vector<bool> in(n);
    d[s] = 0;
    h[s] = q.push({0, s});
    in[s] = true;
    while (!q.empty())
    {
        auto [du, u] = q.top();
        q.pop();
        in[u] = false;
        for (auto [v, w] : g[u])
        {
            if (du + w >= d[v]) continue;
            d[v] = du + w;
            pre[v] = u;
            if (in[v])
                q.modify(h[v], {d[v], v});
            else
            {
                h[v] = q.push({d[v], v});
                in[v] = true;
            }
        }
    }
```

继续处理操作（接上页同一 main）：

```
    if (d[t] == inf)
    {
        cout << -1 << '\n';
        return 0;
    }
    vector<pair<int, int>> path;
    for (int v = t; v != s; v = pre[v]) path.push_back({pre[v], v});
    reverse(path.begin(), path.end());
    cout << d[t] << ' ' << path.size() << '\n';
    for (auto [u, v] : path) cout << u << ' ' << v << '\n';
}
```

30.2 左偏树

30.2.1 左偏树可并小根堆

默认构造为空节点池；singleton(key) 创建单点堆并返回节点编号，编号从 1 开始，0 表示空堆。meld(x,y) 原地合并两个不相交堆并返回新根，调用后旧根不能再当独立堆使用；pop(root) 删除非空堆的最小节点并返回新根，必须保存返回值。相同 key 按节点编号小者优先，t[root].key 为最小值，root 本身就是最小节点编号。key 支持完整 long long；仅比较而不做减法。dist[空]=0，叶子 dist=1，每点左子树 dist 不小于右子树；递归沿右链合并，meld/pop 为 O(log(n+1))，其中 n 为参与堆节点数。singleton 均摊 O(1)，节点池空间 O(累计创建数)，节点不回收，不保留版本；pop 后删除节点已脱离堆，不应继续作为根操作。只保证根操作，不提供任意点所属堆查询，P3377 驱动另用并查集和删除标记维护。不要改写内部 l/r/dist，也不要合并有交集的堆；assert(x!=y) 不能检测所有非法重叠。

```
struct LeftistHeap
{
    struct Node
    {
        long long key;
        int l, r, dist;
    };

    vector<Node> t{{0, 0, 0, 0}};

    int singleton(long long key)
    {
        t.push_back({key, 0, 0, 1});
        return int(t.size()) - 1;
    }

    int meld(int x, int y)
    {
        if (!x || !y) return x ? x : y;
        assert(x != y);
        if (pair(t[x].key, x) > pair(t[y].key, y)) swap(x, y);
        t[x].r = meld(t[x].r, y);
        if (t[t[x].l].dist < t[t[x].r].dist) swap(t[x].l, t[x].r);
        t[x].dist = t[t[x].r].dist + 1;
        return x;
    }

    int pop(int x)
    {
        assert(x > 0);
        int root = meld(t[x].l, t[x].r);
        t[x].l = t[x].r = 0;
        t[x].dist = 1;
        return root;
    }
};
```

模板题使用：Luogu P3377

题意：合并指定元素所在的堆，或弹出其堆最小值；已删除元素的操作无效。同值先删先输入者。并查集代表不一定是堆根，root 单独保存；节点编号比输入数组下标大 1。

所需模板：LeftistHeap（第 390 页，30.2.1 节）、dsu（第 373 页，29.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3377

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    LeftistHeap heap;
    dsu sets(n);
    vector<int> root(n);
    vector<bool> removed(n);
    for (int i = 0; i < n; i++)
    {
        long long key;
        cin >> key;
        root[i] = heap.singleton(key);
    }
    while (m--)
    {
        int op, x;
        cin >> op >> x;
        x--;
        if (op == 1)
        {
            int y;
            cin >> y;
            y--;
            if (removed[x] || removed[y]) continue;
            int a = sets.find(x), b = sets.find(y);
            if (a == b) continue;
            root[a] = heap.meld(root[a], root[b]);
            sets.merge(a, b);
        }
        else if (removed[x])
            cout << -1 << '\n';
        else
        {
            int a = sets.find(x), id = root[a];
            cout << heap.t[id].key << '\n';
            removed[id - 1] = true;
            root[a] = heap.pop(id);
        }
    }
}
```

第三十一章　块状数据结构

31.1　树分块

31.1.1　树分块与在线路径去重

TreeBlockDistinct(n)，1≤n<INT_MAX，点 1..n；add(u,v) 加入无向边，调用方保证恰为一棵树。build(values, b=512, root=1) 离散化 signed64 点颜色，values 长度 n+1 且忽略 0 号；b≥1，root 为合法点。query(u,v) 返回闭路径不同颜色数，可强制在线；无修改操作，改颜色或加边后必须重新 build 且仍保证树结构。支持换根、换块长、重建和值复制。

递归按残余子树大小选关键点并补根，关键点数 K≤floor(n/B)+1，每点到最近关键祖先距离 <B。预存每对关键点路径颜色位集，重复颜色用计数回溯。令 C 为颜色种数、W=ceil(C/64)，预处理 O(n log n+Kn+K^2W)，空间 O(n log n+K^2W+n)，查询 O(B+W+log n)。n=40000、B=512 时 K≤79，位集表最多约 31.2MB；B 过小会导致平方级关键点表，不保证任意块长都能装入内存。DFS 保持递归，按题目内存提供足够调用栈。

查询在 LCA 分成两条祖先链；长链选 LCA 下方 B-1 层的 x，以 near[x] 为上端关键点，保证它不会越过 LCA。不能直接使用 near[LCA]。最后显式加入 LCA 颜色。g、up、c、dep、id、near、keys、paths 等构建后只读；partition、mark、collect、leg 为内部过程，jump 要求 0≤steps≤dep[u]。

结构与关键点选择:

```
struct TreeBlockDistinct
{
    using ull = unsigned long long;
    int n, block, lg, words, k;
    vector<vector<int>> g, up;
    vector<int> c, dep, id, near, keys, count;
    vector<vector<ull>> paths;
    vector<ull> bits;

    TreeBlockDistinct(int n) : n(n), g(n + 1)
    {
        assert(n >= 1 && n < INT_MAX);
    }

    void add(int u, int v)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    int partition(int u, int p)
    {
        up[0][u] = p;
        for (int j = 1; j < lg; j++)
            up[j][u] = up[j - 1][up[j - 1][u]];
        int size = 1;
        for (int v : g[u])
        {
            if (v == p) continue;
            dep[v] = dep[u] + 1;
            size += partition(v, u);
        }
        if (size < block) return size;
        id[u] = keys.size();
        keys.push_back(u);
        return 0;
    }
```

最近关键祖先与路径位集（接上页同一结构体）：

```
    void mark(int u, int p, int last)
    {
        if (id[u] >= 0) last = u;
        near[u] = last;
        for (int v : g[u])
            if (v != p) mark(v, u, last);
    }

    void collect(int u, int p, int source)
    {
        int x = c[u];
        if (count[x]++ == 0) bits[x >> 6] |= 1ULL << (x & 63);
        if (id[u] >= 0) paths[size_t(source) * k + id[u]] = bits;
        for (int v : g[u])
            if (v != p) collect(v, u, source);
        if (--count[x] == 0) bits[x >> 6] &= ~(1ULL << (x & 63));
    }
```

分块界 每个按残余大小选出的关键点消耗至少 B 个尚未归入更深关键点的点，故数量不超过 floor(n/B)，补根后至多再加 1。连续 B 个非关键祖先会使其中最高点的残余大小至少 B，矛盾；因此 near 距离严格小于 B。

重复颜色 collect 沿当前简单路径维护颜色出现次数，回溯时只有计数归零才能清除对应位。每个关键点作为源递归遍历整棵树，遇到另一个关键点就保存当前位集。

离散化与从零重建（接上页同一结构体）：

```
    void build(const vector<long long> &values, int b = 512, int root = 1)
    {
        assert(values.size() == size_t(n + 1) && b >= 1);
        assert(1 <= root && root <= n);
        block = b;
        vector<long long> all(values.begin() + 1, values.end());
        sort(all.begin(), all.end());
        all.erase(unique(all.begin(), all.end()), all.end());
        c.resize(n + 1);
        for (int u = 1; u <= n; u++)
            c[u] = lower_bound(all.begin(), all.end(), values[u]) - all.begin();
        words = (all.size() + 63) / 64;
        count.assign(all.size(), 0);
        bits.assign(words, 0);
        dep.assign(n + 1, 0);
        near.assign(n + 1, 0);
        id.assign(n + 1, -1);
        keys.clear();
        lg = 1;
        while ((1LL << lg) <= n) lg++;
        up.assign(lg, vector<int>(n + 1));
        partition(root, 0);
        if (id[root] < 0)
        {
            id[root] = keys.size();
            keys.push_back(root);
        }
        mark(root, 0, root);
        k = keys.size();
        paths = vector<vector<ull>>(size_t(k) * k);
        for (int i = 0; i < k; i++) collect(keys[i], 0, i);
    }
```

倍增祖先与 LCA（接上页同一结构体）：

```
    int jump(int u, int steps) const
    {
        for (int j = 0; j < lg; j++)
            if (steps >> j & 1) u = up[j][u];
        return u;
    }

    int lca(int u, int v) const
    {
        if (dep[u] < dep[v]) swap(u, v);
        u = jump(u, dep[u] - dep[v]);
        if (u == v) return u;
        for (int j = lg - 1; j >= 0; j--)
        {
            if (up[j][u] != up[j][v])
            {
                u = up[j][u];
                v = up[j][v];
            }
        }
        return up[0][u];
    }
```

上端关键点不能越过 LCA 设当前祖先链终点为 p。长链中 x 在 p 下方 B-1 层，near[x] 距 x 至多 B-1，所以 b=near[x] 仍在 p 的子树内；a=near[u] 位于 b 下方或等于 b。关键路径 a 至 b 之外，两端各不足 B 步，枚举这些短段即可。短链直接枚举，最后补上 p 的颜色。

空间选型 令 W=ceil(C/64)。K 个源的遍历需 O(Kn)，保存 K 平方份位集需 $O(K^2W)$ 时间和空间；另有倍增表。默认 B=512 针对四万点量级，不能把 B 任意缩小后仍假定线性内存。

拼接路径与在线查询（接上页同一结构体）：

```
    void leg(int u, int ancestor, vector<ull> &ans) const
    {
        if (dep[u] - dep[ancestor] >= block)
        {
            int a = near[u];
            int x = jump(u, dep[u] - dep[ancestor] - block + 1);
            int b = near[x];
            const auto &path = paths[size_t(id[a]) * k + id[b]];
            for (int i = 0; i < words; i++) ans[i] |= path[i];
            while (u != a)
            {
                ans[c[u] >> 6] |= 1ULL << (c[u] & 63);
                u = up[0][u];
            }
            u = b;
        }
        while (u != ancestor)
        {
            ans[c[u] >> 6] |= 1ULL << (c[u] & 63);
            u = up[0][u];
        }
    }

    int query(int u, int v) const
    {
        int p = lca(u, v);
        vector<ull> ans(words);
        leg(u, p, ans);
        leg(v, p, ans);
        ans[c[p] >> 6] |= 1ULL << (c[p] & 63);
        int result = 0;
        for (ull x : ans) result += __builtin_popcountll(x);
        return result;
    }
};
```

模板题使用：P6177 【模板】树分块 / Count on a tree II

题意：静态点颜色，u 与上次答案异或解码；在线输出闭路径颜色种数。

所需模板：TreeBlockDistinct（第 392 页，31.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6177

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n + 1);
    for (int u = 1; u <= n; u++) cin >> a[u];
    TreeBlockDistinct g(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        g.add(u, v);
    }
    g.build(a);
    int last = 0;
    while (q--)
    {
        int u, v;
        cin >> u >> v;
        u ^= last;
        last = g.query(u, v);
        cout << last << '\n';
    }
}
```

第三十二章　ST 表

32.1　位掩码静态区间最值与位置

StaticRMQ<T,Cmp>(a,cmp) 复制数组；默认 T=long long、Cmp=less<T> 求最小，greater<T> 求最大，也可传有状态比较器。query(l,r) 返回 0-based 半开区间 [l,r) 内最优元素的原编号，并列总取最左；值读 rmq.a[编号]。要求 0≤l<r≤n；空数组可构造但不可查询；重新赋值新对象重建，不支持修改。比较器必须是稳定的严格弱序、const 可调用，元素类型可复制；浮点数据不能含 NaN。

单调栈位掩码处理块内区间，块间用 ST 表；块长为 n 的二进制位数（空数组取 1），n≤INT_MAX/2 且内存可分配，32 位掩码足够。在复制/比较/位操作 O(1) 的模型下，预处理 O(n)、查询 O(1)、空间 O(n)。所有成员约定只读，better/small 是内部辅助；small 仅供非空同块区间。只比较元素，不做元素算术，signed64 极值合法。

```
template <class T = long long, class Cmp = less<T>>
struct StaticRMQ
{
    int n, b;
    vector<T> a;
    vector<uint32_t> mask;
    vector<vector<int>> st;
    Cmp cmp;

    StaticRMQ(const vector<T> &v, Cmp cmp = Cmp()) : a(v), cmp(cmp)
    {
        assert(v.size() <= INT_MAX / 2);
        n = v.size();
        b = max(1, (int)bit_width((uint32_t)n));
        mask.resize(n);
        if (!n) return;
        st.push_back({});
        uint32_t s = 0;
        for (int i = 0; i < n; i++)
        {
            int start = i - i % b;
            if (i % b == 0) s = 0;
            while (s && cmp(a[i], a[start + bit_width(s) - 1]))
            {
                s ^= uint32_t(1) << (bit_width(s) - 1);
            }
            s |= uint32_t(1) << (i % b);
            mask[i] = s;
            if (i % b == b - 1 || i == n - 1)
            {
                st[0].push_back(start + countr_zero(s));
            }
        }
```

块间表构建、最优位置合并与查询（接上页同一结构体）：

```
        int m = st[0].size();
        for (int k = 1; (1 << k) <= m; k++)
        {
            st.push_back(vector<int>(m - (1 << k) + 1));
            for (int i = 0; i + (1 << k) <= m; i++)
            {
                st[k][i] = better(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
            }
        }
    }

    int better(int x, int y) const
    {
        if (cmp(a[x], a[y])) return x;
        if (cmp(a[y], a[x])) return y;
        return min(x, y);
    }

    int small(int l, int r) const
    {
        return l + countr_zero(mask[r - 1] >> (l % b));
    }

    int query(int l, int r) const
    {
        assert(0 <= l && l < r && r <= n);
        int x = l / b, y = (r - 1) / b;
        if (x == y) return small(l, r);
        int ans = better(small(l, (x + 1) * b), small(y * b, r));
        x++;
        if (x < y)
        {
            int k = bit_width((uint32_t)(y - x)) - 1;
            ans = better(ans, better(st[k][x], st[k][y - (1 << k)]));
        }
        return ans;
    }
};
```

模板题使用：Library Checker Static RMQ

题意：静态数组上查询 0-based 半开区间 [l,r) 最小值；n,q≤500000，值为 0..10^9。模板返回最左最小值位置，通过 rmq.a[位置] 输出值。

所需模板：StaticRMQ（第 399 页，32.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/staticrmq

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    StaticRMQ<int> rmq(a);
    while (q--)
    {
        int l, r;
        cin >> l >> r;
        cout << rmq.a[rmq.query(l, r)] << '\n';
    }
}
```

模板题使用：Luogu P3865 【模板】ST 表 && RMQ 问题

题意：静态区间最大值；输入 1-based 闭区间 [l,r]，转换为 query(l-1,r)。用 greater<int> 选择最大值，再由返回位置读取值。

所需模板：StaticRMQ（第 399 页，32.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3865

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    StaticRMQ<int, greater<int>> rmq(a);
    while (q--)
    {
        int l, r;
        cin >> l >> r;
        cout << rmq.a[rmq.query(l - 1, r)] << '\n';
    }
}
```

接口用法演示：静态区间最左最小/最大位置（接口演示）

演示约定：输入 n,q、signed64 数组和 q 个非空 0-based 半开区间，分别输出最左最小与最左最大位置。允许重复值；n=0 时要求 q=0。

所需模板：StaticRMQ（第 399 页，32.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/ds/sparse-table/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    StaticRMQ<long long> mn(a);
    StaticRMQ<long long, greater<long long>> mx(a);
    while (q--)
    {
        int l, r;
        cin >> l >> r;
        cout << mn.query(l, r) << ' ' << mx.query(l, r) << '\n';
    }
}
```

32.2 二维静态矩形最值与位置

StaticRMQ2D<T,Cmp>(a,cmp) 复制矩形矩阵；默认 T=long long、less 求最小，greater 求最大。

query(x1,y1,x2,y2) 查询 0-based 半开矩形 [x1,x2)×[y1,y2)，返回最优元素的 (row,col)；并列按行号再列号取最小，值读 rmq.a[row][col]。要求 0≤x1<x2≤n、0≤y1<y2≤m；空矩阵或零列可构造但不可查询，所有行必须等长。重新赋值新对象重建，不支持修改。

比较器须稳定严格弱序、const 可调用，支持有状态比较器和可复制元素，不接受 NaN；不做元素算术，signed64 极值合法。n,m 及 n*m≤INT_MAX/2 且内存可分配；所有成员只读，better 是内部辅助。四块 ST 查询 O(1)，非空矩阵的预处理和空间 O(nm log(n+1) log(m+1))（元素比较/复制 O(1) 时）。每个层对保存完整 n*m 个 int 位置，空间较大：305×305 时仅 st 的 int 数据约 28.7MiB，构造前需计算内存。

```
template <class T = long long, class Cmp = less<T>>
struct StaticRMQ2D
{
    int n, m;
    vector<vector<T>> a;
    vector<vector<vector<int>>> st;
    Cmp cmp;

    StaticRMQ2D(const vector<vector<T>> &v, Cmp cmp = Cmp()) : a(v), cmp(cmp)
    {
        assert(v.size() <= INT_MAX / 2);
        n = v.size();
        assert(n == 0 || v[0].size() <= INT_MAX / 2);
        m = n ? v[0].size() : 0;
        for (const auto &row : v) assert(row.size() == (size_t)m);
        assert(1LL * n * m <= INT_MAX / 2);
        if (!n || !m) return;
        int h = bit_width((unsigned)n), w = bit_width((unsigned)m);
        st.assign(h, vector<vector<int>>(w, vector<int>(n * m)));
        iota(st[0][0].begin(), st[0][0].end(), 0);
```

层表递推、位置比较与四块查询（接上页同一结构体）：

```
        for (int k = 0; k < h; k++)
        {
            for (int l = 0; l < w; l++)
            {
                if (k == 0 && l == 0) continue;
                for (int x = 0; x + (1 << k) <= n; x++)
                {
                    for (int y = 0; y + (1 << l) <= m; y++)
                    {
                        int id = x * m + y;
                        if (k)
                        {
                            st[k][l][id] = better(st[k - 1][l][id],
                                st[k - 1][l][id + (1 << (k - 1)) * m]);
                        }
                        else
                        {
                            st[k][l][id] = better(st[k][l - 1][id],
                                st[k][l - 1][id + (1 << (l - 1))]);
                        }
                    }
                }
            }
        }
    }

    int better(int x, int y) const
    {
        const auto &u = a[x / m][x % m], &v = a[y / m][y % m];
        if (cmp(u, v)) return x;
        if (cmp(v, u)) return y;
        return min(x, y);
    }

    pair<int, int> query(int x1, int y1, int x2, int y2) const
    {
        assert(0 <= x1 && x1 < x2 && x2 <= n);
        assert(0 <= y1 && y1 < y2 && y2 <= m);
        int k = bit_width((unsigned)(x2 - x1)) - 1;
        int l = bit_width((unsigned)(y2 - y1)) - 1;
        x2 -= 1 << k;
        y2 -= 1 << l;
        const auto &t = st[k][l];
        int top = better(t[x1 * m + y1], t[x1 * m + y2]);
        int bottom = better(t[x2 * m + y1], t[x2 * m + y2]);
        int id = better(top, bottom);
        return {id / m, id % m};
    }
};
```

接口用法演示：二维矩形最小/最大位置（接口演示）

演示约定：输入 n,m,q、矩阵与 q 个 0-based 非空半开矩形，依次输出最小值的行列和最大值的行列；并列按行再列取最小。值支持 signed64；空矩阵或零列时 q 必须为 0。

所需模板：StaticRMQ2D（第 403 页，32.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/ds/sparse-table/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    vector<vector<long long>> a(n, vector<long long>(m));
    for (auto &row : a)
    {
        for (auto &x : row) cin >> x;
    }
    StaticRMQ2D<long long> mn(a);
    StaticRMQ2D<long long, greater<long long>> mx(a);
    while (q--)
    {
        int x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;
        auto [u, v] = mn.query(x1, y1, x2, y2);
        auto [s, t] = mx.query(x1, y1, x2, y2);
        cout << u << ' ' << v << ' ' << s << ' ' << t << '\n';
    }
}
```

接口用法演示：矩形最大值是否出现在四角（kuangbin 源码模型）

演示约定：读到 EOF，每组 n,m、矩阵、q 和 q 个 1-based 闭矩形端点，分别交换逆序的行/列端点。输出最大值与 yes/no，表示四个角是否至少有一个等于最大值。演示 n,m 为 1..305，值为 signed64；不是已核验的原 OJ 协议、资源或正式模板题。

所需模板：StaticRMQ2D（第 403 页，32.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/kuangbin/ACM-ICPC

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    while (cin >> n >> m)
    {
        vector<vector<long long>> a(n, vector<long long>(m));
        for (auto &row : a)
        {
            for (auto &x : row) cin >> x;
        }
        StaticRMQ2D<long long, greater<long long>> rmq(a);
        int q;
        cin >> q;
        while (q--)
        {
            int x1, y1, x2, y2;
            cin >> x1 >> y1 >> x2 >> y2;
            if (x1 > x2) swap(x1, x2);
            if (y1 > y2) swap(y1, y2);
            auto [x, y] = rmq.query(x1 - 1, y1 - 1, x2, y2);
            long long value = rmq.a[x][y];
            bool corner = value == a[x1 - 1][y1 - 1] || value == a[x1 - 1][y2 - 1] ||
                          value == a[x2 - 1][y1 - 1] || value == a[x2 - 1][y2 - 1];
            cout << value << ' ' << (corner ? "yes" : "no") << '\n';
        }
    }
}
```

第三十三章　树状数组

33.1　稀疏二维树状数组：点加与矩形和

Fenwick2D<T>(points)，T 默认 long long；预登记所有将来更新的 (x,y)，重复点自动去重，初始权值全 0。add(x,y,v) 在已登记点累加，删除可加负值；不是不预知坐标的在线数据结构。prefix(x,y) 求横坐标 <x 且纵坐标 <y 的权和，sum(l,d,r,u) 求 [l,r)×[d,u)，允许空矩形；坐标、端点均为 long long，不做端点 ±1。上界须可表示，因此不能用有限半开区间包含 LLONG_MAX 坐标。init(points) 重新登记并清空权值，空点集合法。K 为登记项数（含重复），P 为不同点数且 P<2^30：构造 O(K log K+P log P)，构造峰值空间 O(K+P log P)，更新与查询 $O(\log^2 P)$。按 y 有序分发到各横轴 Fenwick 节点，使内层坐标有序且去重。T 支持零值、加减，所有节点和与容斥中间值均须可表示。内部字段只读，不提供未经登记的动态插入。

```
template <class T = long long> struct Fenwick2D
{
    using ll = long long;
    int n = 0;
    vector<pair<ll, ll>> pt;
    vector<ll> xs;
    vector<vector<ll>> ys;
    vector<vector<T>> bit;

    Fenwick2D(vector<pair<ll, ll>> p = {})
    {
        init(move(p));
    }

    // Register every future update coordinate; all weights start at zero.
    void init(vector<pair<ll, ll>> p)
    {
        sort(p.begin(), p.end());
        p.erase(unique(p.begin(), p.end()), p.end());
        assert(p.size() < (1U << 30));
        pt = p;
        xs.clear();
        for (auto [x, y] : p)
            if (xs.empty() || xs.back() != x) xs.push_back(x);
        n = xs.size();
        ys.assign(n + 1, {});
        bit.assign(n + 1, {});
        sort(p.begin(), p.end(), [](auto a, auto b)
        {
            return a.second < b.second;
        });
        for (auto [x, y] : p)
        {
            int k = lower_bound(xs.begin(), xs.end(), x) - xs.begin() + 1;
            for (int i = k; i <= n; i += i & -i)
                if (ys[i].empty() || ys[i].back() != y) ys[i].push_back(y);
        }
        for (int i = 1; i <= n; i++) bit[i].assign(ys[i].size() + 1, T{});
    }
```

更新与半开矩形求和（接上页同一结构体）：

```
    void add(ll x, ll y, T v)
    {
        assert(binary_search(pt.begin(), pt.end(), pair<ll, ll>{x, y}));
        int k = lower_bound(xs.begin(), xs.end(), x) - xs.begin() + 1;
        for (int i = k; i <= n; i += i & -i)
        {
            int j = lower_bound(ys[i].begin(), ys[i].end(), y) - ys[i].begin() + 1;
            for (; j < (int)bit[i].size(); j += j & -j) bit[i][j] += v;
        }
    }

    // Sum over coordinates strictly less than x and y; no endpoint +/-1.
    T prefix(ll x, ll y) const
    {
        T ans{};
        int i = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
        for (; i; i -= i & -i)
        {
            int j = lower_bound(ys[i].begin(), ys[i].end(), y) - ys[i].begin();
            for (; j; j -= j & -j) ans += bit[i][j];
        }
        return ans;
    }

    T sum(ll l, ll d, ll r, ll u) const
    {
        assert(l <= r && d <= u);
        if (l == r || d == u) return T{};
        return prefix(r, u) - prefix(l, u) - prefix(r, d) + prefix(l, d);
    }
};
```

模板题使用：Library Checker point_add_rectangle_sum

题意：N,Q≤100000，初始 N 个带权平面点。0 x y w 加入点权，重合点累加；1 l d r u 查询 [l,r)×[d,u) 内总权。坐标和权值 0..10^9，查询边界严格递增。先读完所有操作，预登记初始与未来增加点的坐标，再按原顺序执行；这不改变原题操作语义。

所需模板：Fenwick2D（第 407 页，33.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/point_add_rectangle_sum

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, q;
    std::cin >> n >> q;
    using ll = long long;
    std::vector<std::array<ll, 3>> a(n);
    std::vector<std::pair<ll, ll>> p;
    for (auto &[x, y, w] : a)
    {
        std::cin >> x >> y >> w;
        p.push_back({x, y});
    }
    std::vector<std::array<ll, 5>> ops(q);
    for (auto &[op, x, y, r, u] : ops)
    {
        std::cin >> op >> x >> y >> r;
        if (op == 0) p.push_back({x, y});
        else std::cin >> u;
    }
    Fenwick2D tree(p);
    for (auto [x, y, w] : a) tree.add(x, y, w);
    for (auto [op, x, y, r, u] : ops)
    {
        if (op == 0) tree.add(x, y, r);
        else std::cout << tree.sum(x, y, r, u) << '\n';
    }
}
```

33.2 四矩二维树状数组：矩形加与矩形和

RectangleFenwick<T>(n,m)，T 默认 long long，0≤n,m<2^30，初始化全 0；稠密空间与构造 O((n+1)(m+1))，需先确认可分配。add(l,d,r,u,v) 给 0-based 半开矩形 [l,r)×[d,u) 加 v；sum(l,d,r,u) 查询同一区间，prefix(x,y) 查询 [0,x)×[0,y)。端点须在 [0,n]×[0,m]，允许空矩形及零维矩阵。差分四角维护 D、xD、yD、xyD 四个矩，前缀和为 xyS0-ySx-xSy+Sxy；每次操作 O(log n log m)，实际存储每格 4 个 T。单点操作令右上界各加 1。T 支持零值、加减、取负和乘 int；包括-v、各坐标乘积及容斥在内的中间值均须可表示，仅最终答案可表示不够。内部 bit 与 corner 不供直接修改。重新赋值 RectangleFenwick(n,m) 清空；不支持稀疏大坐标自动压缩，因为区间加仍取决于真实格子数量。

```
template <class T = long long> struct RectangleFenwick
{
    int n, m;
    vector<vector<array<T, 4>>> bit;

    RectangleFenwick(int n, int m) : n(n), m(m)
    {
        assert(0 <= n && n < (1 << 30) && 0 <= m && m < (1 << 30));
        bit.assign(n + 1, vector<array<T, 4>>(m + 1));
    }

    // Difference impulse at (x,y); boundary impulses do not affect the grid.
    void corner(int x, int y, T v)
    {
        if (x == n || y == m) return;
        array<T, 4> a{v, v * x, v * y, v * x * y};
        for (int i = x + 1; i <= n; i += i & -i)
            for (int j = y + 1; j <= m; j += j & -j)
                for (int k = 0; k < 4; k++) bit[i][j][k] += a[k];
    }

    // Add to [l,r) x [d,u), 0-based. Empty rectangles are allowed.
    void add(int l, int d, int r, int u, T v)
    {
        assert(0 <= l && l <= r && r <= n && 0 <= d && d <= u && u <= m);
        if (l == r || d == u) return;
        corner(l, d, v);
        corner(r, d, -v);
        corner(l, u, -v);
        corner(r, u, v);
    }
```

更新与半开矩形求和（接上页同一结构体）：

```
    T prefix(int x, int y) const
    {
        assert(0 <= x && x <= n && 0 <= y && y <= m);
        array<T, 4> a{};
        for (int i = x; i; i -= i & -i)
            for (int j = y; j; j -= j & -j)
                for (int k = 0; k < 4; k++) a[k] += bit[i][j][k];
        return a[0] * x * y - a[1] * y - a[2] * x + a[3];
    }

    T sum(int l, int d, int r, int u) const
    {
        assert(0 <= l && l <= r && r <= n && 0 <= d && d <= u && u <= m);
        if (l == r || d == u) return T{};
        return prefix(r, u) - prefix(l, u) - prefix(r, d) + prefix(l, d);
    }
};
```

模板题使用：Luogu P4514 / 上帝造题的七分钟

题意：输入 X n m 初始化零矩阵，n,m≤2048；读到 EOF，L a b c d delta 给 1-based 闭矩形加 delta，k a b c d 查询区域和（小写 k）。至多 200000 操作，|delta|≤500；转换为 [a-1,c)×[b-1,d)。最终答案在 32 位内，中间值未保证，使用 long long。

所需模板：RectangleFenwick（第 410 页，33.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4514

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    char op;
    int n, m;
    std::cin >> op >> n >> m;
    RectangleFenwick tree(n, m);
    int a, b, c, d;
    while (std::cin >> op >> a >> b >> c >> d)
    {
        if (op == 'L')
        {
            long long v;
            std::cin >> v;
            tree.add(a - 1, b - 1, c, d, v);
        }
        else
            std::cout << tree.sum(a - 1, b - 1, c, d) << '\n';
    }
}
```

33.3 树状数组第 k 小与组合依赖

保留频数第 k 小和泛型组合用途；P3374 基础单点加/区间和题不再独立收录或安排提交。add(x,v)、sum(x)、query(l,r)，下标 1..n，闭区间；kth(k) 要求所有频率非负且 k>0，超过总和返回 n+1。单次 O(log n)。T 须能从 0 构造并支持 += 与减法；可直接使用 ModInt 维护模数下的区间和。kth 只适用于有序累计频数，不适用于取模后的区间和。

```
template <class T = long long> struct Fenwick
{
    int n;
    vector<T> a;

    Fenwick(int n = 0) : n(n), a(n + 1) {}

    void add(int x, T v)
    {
        assert(x >= 1);
        for (; x <= n; x += x & -x) a[x] += v;
    }

    T sum(int x) const
    {
        T ans = 0;
        for (; x; x -= x & -x) ans += a[x];
        return ans;
    }

    T query(int l, int r) const { return sum(r) - sum(l - 1); }

    // Nonnegative frequencies; returns n+1 if k exceeds total.
    int kth(T k) const
    {
        assert(k > 0);
        int x = 0, b = 1;
        while (b <= n / 2) b *= 2;
        for (; b; b >>= 1)
            if (x + b <= n && a[x + b] < k)
            {
                k -= a[x + b];
                x += b;
            }
        return x + 1;
    }
};
```

模板题使用：Luogu P3369

题意：维护可重集合的插入、删除一个、排名、第 k 小和严格前驱/后继。预读全部操作并离散化非第 k 小查询的值域；频数下标为 1-based，kth 的返回值减 1 后映射回原值。排名查询允许 x 不在集合中；第 k 小与前驱/后继按题目保证存在。n≤100000，|x|≤10000000；总复杂度 O(n log n)，空间 O(n)。

所需模板：Fenwick（第 412 页，33.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<pair<int, int>> q(n);
    vector<int> a;
    for (auto &[op, x] : q)
    {
        cin >> op >> x;
        if (op != 4) a.push_back(x);
    }
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    Fenwick<int> f(a.size());
    for (auto [op, x] : q)
    {
        int p = lower_bound(a.begin(), a.end(), x) - a.begin();
        if (op == 1)
            f.add(p + 1, 1);
        else if (op == 2)
        {
            if (f.query(p + 1, p + 1) > 0) f.add(p + 1, -1);
        }
        else if (op == 3)
            cout << f.sum(p) + 1 << '\n';
        else if (op == 4)
            cout << a[f.kth(x) - 1] << '\n';
        else if (op == 5)
            cout << a[f.kth(f.sum(p)) - 1] << '\n';
        else
        {
            p = upper_bound(a.begin(), a.end(), x) - a.begin();
            cout << a[f.kth(f.sum(p) + 1) - 1] << '\n';
        }
    }
}
```

相关应用：离线二维扫描线应用，不代表整页覆盖

33.4 离线静态矩形权值和

代码依赖：Fenwick（第 412 页，33.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

offline_rectangle_sum(points,queries)：points 每项为 {x,y,w}，queries 每项为 {l,d,r,u}，返回按询问原顺序排列的 [l,r)×[d,u) 内权值和。坐标、端点与权值均为 long long；要求 l<=r、d<=u，允许空矩形、空点集、空询问、重合点和负权。传入点按值复制，两个输入均不改变，多次调用互不影响。坐标仅比较，不用端点 ±1，支持 LLONG_MIN/MAX；半开矩形无法包含位于 LLONG_MAX 的上边界坐标。

一次读完所有询问，按横坐标扫描，在纵轴离散化树状数组上加点。同横坐标事件只包含 x< 事件横坐标的点，保证左右开闭正确。时间 O((N+Q)log(N+Q+1))，空间 O(N+Q)，N,Q 均小于 2^30 且内存可分配。要求所有权值绝对值之和 <=LLONG_MAX，保证树状数组、容斥与符号乘法不溢出。依赖 Fenwick；若需逐条在线回答，改用持久化前缀版本。

```
vector<long long> offline_rectangle_sum(vector<array<long long, 3>> points,
                                        const vector<array<long long, 4>> &queries)
{
    using ll = long long;
    assert(points.size() < (1U << 30) && queries.size() < (1U << 30));
    vector<ll> ys;
    for (auto [x, y, w] : points)
        ys.push_back(y);
    sort(ys.begin(), ys.end());
    ys.erase(unique(ys.begin(), ys.end()), ys.end());
    auto rank = [&](ll y)
    {
        return int(lower_bound(ys.begin(), ys.end(), y) - ys.begin());
    };
    struct Event
    {
        ll x;
        int d, u, id, sign;
    };
    vector<Event> events;
    events.reserve(2 * queries.size());
    for (int i = 0; i < (int)queries.size(); i++)
    {
        auto [l, d, r, u] = queries[i];
        assert(l <= r && d <= u);
        if (l == r || d == u)
            continue;
        events.push_back({l, rank(d), rank(u), i, -1});
        events.push_back({r, rank(d), rank(u), i, 1});
    }
    sort(points.begin(), points.end());
    sort(events.begin(), events.end(), [](const Event &a, const Event &b)
    {
        return a.x < b.x;
    });
    Fenwick<ll> bit(ys.size());
    vector<ll> answer(queries.size());
    int p = 0;
    for (auto e : events)
    {
```

```
        while (p < (int)points.size() && points[p][0] < e.x)
        {
            auto [x, y, w] = points[p];
            bit.add(rank(y) + 1, w);
            p++;
        }
        ll sum = bit.sum(e.u) - bit.sum(e.d);
        answer[e.id] += e.sign * sum;
    }
    return answer;
}
```

模板题使用：Library Checker Rectangle Sum（离线）

题意：N,Q<=200000，点坐标与权值在 [0,1e9]；读取全部询问后用扫描线返回半开矩形权值和。此例可离线处理，节省持久化节点的内存；需要在线查询时使用持久化版本（用法 432）。Library Checker C++20 线上 AC 410016，258ms、29.98MiB。

所需模板：Fenwick（第 412 页，33.3 节）、offline_rectangle_sum（第 414 页，33.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/rectangle_sum

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<array<long long, 3>> points(n);
    vector<array<long long, 4>> queries(q);
    for (auto &p : points)
        cin >> p[0] >> p[1] >> p[2];
    for (auto &r : queries)
        cin >> r[0] >> r[1] >> r[2] >> r[3];
    for (auto answer : offline_rectangle_sum(points, queries))
        cout << answer << '\n';
}
```

第三十四章　线段树

34.1　线段树基础

34.1.1　通用懒标记线段树与边界搜索

模板参数依次为 S、F、op、e、mapping、composition、id；以数组 a 构造，0-based 半开区间；n 构造将 n 个元素置为 e。0≤n≤2^28-1，受内存限制。op 满足结合律、e 为双侧单位元；单位标记不改变节点，mapping 必须保持 op 的合并结果；composition(f,g) 表示先 g 后 f，须满足作用复合一致性，id 为标记单位元。无需比较标记是否等于 id，每次下传都应用标记。set/get 单点，apply(l,r,f) 区间作用，prod(l,r) 保序合并，all() 读取根；空区间不更新且查询返回 e。区间长度若参与修改，必须存入 S，不能从 e 推断长度：区间和节点为 {sum,len}，真实叶 len=1，e={0,0}。max_right/min_left 的谓词须满足 ok(e())、无副作用、扩展时真变假后不再变真；两者查询会下传标记，非 const。递归深度 O(log n)，构造 O(n)、操作/搜索 O(log n)、all O(1)，空间 O(n)，假设运算和类型复制 O(1)。d/tag 及其他递归辅助方法仅供内部使用。

```
template <class S, class F, auto op, auto e, auto mapping, auto composition, auto id>
struct lazy_segtree
{
    int n;
    vector<S> d;
    vector<F> tag;

    lazy_segtree(int n = 0) : lazy_segtree(vector<S>(n, e())) {}

    lazy_segtree(const vector<S> &a)
        : n(a.size()), d(4 * n + 4, e()), tag(4 * n + 4, id())
    {
        if (n) build(1, 0, n, a);
    }

    void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }

    void build(int p, int l, int r, const vector<S> &a)
    {
        if (r - l == 1)
        {
            d[p] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m, r, a);
        pull(p);
    }
```

```
    void act(int p, F f)
    {
        d[p] = mapping(f, d[p]);
        tag[p] = composition(f, tag[p]);
    }

    void push(int p)
    {
        act(p * 2, tag[p]);
        act(p * 2 + 1, tag[p]);
        tag[p] = id();
    }

    void set(int x, S v)
    {
        assert(0 <= x && x < n);
        put(1, 0, n, x, v);
    }

    void put(int p, int l, int r, int x, S v)
    {
        if (r - l == 1)
        {
            d[p] = v;
            tag[p] = id();
            return;
        }
        push(p);
        int m = (l + r) / 2;
        if (x < m)
            put(p * 2, l, m, x, v);
        else
            put(p * 2 + 1, m, r, x, v);
        pull(p);
    }

    void apply(int l, int r, F f)
    {
        assert(0 <= l && l <= r && r <= n);
        if (l < r) update(1, 0, n, l, r, f);
    }

    void update(int p, int l, int r, int a, int b, F f)
    {
        if (a <= l && r <= b)
        {
            act(p, f);
            return;
        }
        push(p);
        int m = (l + r) / 2;
        if (a < m) update(p * 2, l, m, a, b, f);
        if (m < b) update(p * 2 + 1, m, r, a, b, f);
        pull(p);
    }
```

```
87      S all() const { return d[1]; }
88
89      S get(int x)
90      {
91          assert(0 <= x && x < n);
92          return prod(x, x + 1);
93      }
94
95      S prod(int l, int r)
96      {
97          assert(0 <= l && l <= r && r <= n);
98          return l == r ? e() : query(1, 0, n, l, r);
99      }
100
101     S query(int p, int l, int r, int a, int b)
102     {
103         if (a <= l && r <= b) return d[p];
104         push(p);
105         int m = (l + r) / 2;
106         if (b <= m) return query(p * 2, l, m, a, b);
107         if (m <= a) return query(p * 2 + 1, m, r, a, b);
108         return op(query(p * 2, l, m, a, b), query(p * 2 + 1, m, r, a, b));
109     }
110
111     template <class G> int max_right(int l, G ok)
112     {
113         assert(0 <= l && l <= n && ok(e()));
114         S s = e();
115         return l == n ? n : right(1, 0, n, l, s, ok);
116     }
117
118     template <class G> int right(int p, int l, int r, int a, S &s, G &ok)
119     {
120         if (r <= a) return n;
121         if (a <= l && ok(op(s, d[p])))
122         {
123             s = op(s, d[p]);
124             return n;
125         }
126         if (r - l == 1) return l;
127         push(p);
128         int m = (l + r) / 2;
129         int x = right(p * 2, l, m, a, s, ok);
130         return x != n ? x : right(p * 2 + 1, m, r, a, s, ok);
131     }
```

```
    template <class G> int min_left(int r, G ok)
    {
        assert(0 <= r && r <= n && ok(e()));
        S s = e();
        return r == 0 ? 0 : left(1, 0, n, r, s, ok);
    }

    template <class G> int left(int p, int l, int r, int b, S &s, G &ok)
    {
        if (b <= l) return 0;
        if (r <= b && ok(op(d[p], s)))
        {
            s = op(d[p], s);
            return 0;
        }
        if (r - l == 1) return r;
        push(p);
        int m = (l + r) / 2;
        int x = left(p * 2 + 1, m, r, b, s, ok);
        return x != 0 ? x : left(p * 2, l, m, b, s, ok);
    }
};
```

模板题使用：Library Checker range_affine_range_sum

题意：对数组区间施加 x→b*x+c，并查询区间和。S 保存和与长度；空单位元长度 0，单个元素长度 1。composition(f,g) 表示先 g 后 f，mapping 按长度累加常数项。区间为 0-based 半开，模 998244353。

所需模板：lazy_segtree（第 417 页，34.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/range_affine_range_sum

```
using S = pair<long long, int>;
using F = pair<long long, long long>;
const int mod = 998244353;

S op(S a, S b)
{
    return {(a.first + b.first) % mod, a.second + b.second};
}

S e()
{
    return {0, 0};
}

S mapping(F f, S a)
{
    return {(f.first * a.first + f.second * a.second) % mod, a.second};
}

F composition(F f, F g)
{
    return {f.first * g.first % mod, (f.first * g.second + f.second) % mod};
}

F id()
{
    return {1, 0};
}
```

使用入口（接上页配置函数）：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n);
    for (auto &[x, len] : a)
    {
        cin >> x;
        len = 1;
    }
    lazy_segtree<S, F, op, e, mapping, composition, id> s(a);
    while (q--)
    {
        int t, l, r;
        cin >> t >> l >> r;
        if (t == 0)
        {
            long long b, c;
            cin >> b >> c;
            s.apply(l, r, {b, c});
        }
        else
            cout << s.prod(l, r).first << '\n';
    }
}
```

34.1.2 通用线段树与左右边界搜索

segtree<S,op,e>(a) 从 0-based vector 构建，或 segtree(n) 初始化为单位元。0≤n≤2^29，实际受内存限制。op(S,S) 须满足结合律，e() 是双侧单位元；无需交换律。set(p,x)、get(p) 操作单点，prod(l,r) 返回半开区间 [l,r) 从左到右的合并，空区间返回 e；all() 返回全区间结果。max_right(l,ok) 返回最大的 r 使 ok(prod(l,r)) 为真；min_left(r,ok) 返回最小的 l。要求 ok(e()) 为真，ok 无副作用，并随对应方向扩展从真变假后不再变真；例如非负和不超过阈值，不能对任意负数和这样搜索。l=n、r=0 支持空端点。构建 O(n)，单点/区间/边界搜索 O(log n)，all O(1)，空间 O(n)，假设 op、e、ok 及 S 复制为 O(1)。d、sz、pull 为内部实现，不直接修改。函数复合时 op(f,g) 表示 g(f(x))，对应先执行左区间再执行右区间；min_left 不能照抄 max_right 的合并顺序。

```
template <class S, auto op, auto e> struct segtree
{
    int n, sz = 1;
    vector<S> d;

    segtree(int n = 0) : segtree(vector<S>(n, e())) {}

    segtree(const vector<S> &a) : n(a.size())
    {
        while (sz < n) sz <<= 1;
        d.assign(2 * sz, e());
        copy(a.begin(), a.end(), d.begin() + sz);
        for (int i = sz - 1; i; i--) pull(i);
    }

```

```
16      void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }
17
18      void set(int p, S x)
19      {
20          assert(0 <= p && p < n);
21          d[p += sz] = x;
22          while (p >>= 1) pull(p);
23      }
24
25      S get(int p) const
26      {
27          assert(0 <= p && p < n);
28          return d[p + sz];
29      }
30
31      S all() const { return d[1]; }
```

```
    S prod(int l, int r) const
    {
        assert(0 <= l && l <= r && r <= n);
        S a = e(), b = e();
        for (l += sz, r += sz; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1) a = op(a, d[l++]);
            if (r & 1) b = op(d[--r], b);
        }
        return op(a, b);
    }

    template <class F> int max_right(int l, F ok) const
    {
        assert(0 <= l && l <= n && ok(e()));
        if (l == n) return n;
        S a = e();
        l += sz;
        do
        {
            while (!(l & 1)) l >>= 1;
            if (!ok(op(a, d[l])))
            {
                while (l < sz)
                {
                    l *= 2;
                    if (ok(op(a, d[l]))) a = op(a, d[l++]);
                }
                return l - sz;
            }
            a = op(a, d[l++]);
        } while ((l & -l) != l);
        return n;
    }
```

```
    template <class F> int min_left(int r, F ok) const
    {
        assert(0 <= r && r <= n && ok(e()));
        if (r == 0) return 0;
        S a = e();
        r += sz;
        do
        {
            --r;
            while (r > 1 && (r & 1)) r >>= 1;
            if (!ok(op(d[r], a)))
            {
                while (r < sz)
                {
                    r = r * 2 + 1;
                    if (ok(op(d[r], a))) a = op(d[r--], a);
                }
                return r + 1 - sz;
            }
            a = op(d[r], a);
        } while ((r & -r) != r);
        return 0;
    }
};
```

模板题使用：Library Checker point_set_range_composite

题意：单点修改一次函数，查询区间内从左到右依次作用于 x 的结果。op(f,g) 表示先 f 后 g，单位元为 x；prod(l,r) 使用 0-based 半开区间，空区间返回单位元。配置函数须和 main 一起抄写。

所需模板：segtree（第 422 页，34.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/point_set_range_composite

```
using S = pair<long long, long long>;
const int mod = 998244353;

S op(S f, S g)
{
    return {f.first * g.first % mod, (f.second * g.first + g.second) % mod};
}

S e()
{
    return {1, 0};
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n);
    for (auto &[x, y] : a) cin >> x >> y;
    segtree<S, op, e> s(a);
    while (q--)
    {
        int t;
        cin >> t;
        if (t == 0)
        {
            int p;
            long long c, d;
            cin >> p >> c >> d;
            s.set(p, {c, d});
        }
        else
        {
            int l, r;
            long long x;
            cin >> l >> r >> x;
            auto [c, d] = s.prod(l, r);
            cout << (c * x + d) % mod << '\n';
        }
    }
}
```

34.1.3 区间加与区间和

初值全零；add(l,r,v)、query(l,r)，1≤l≤r≤n。维护 sum 与加法 tag；所有中间和与长度乘积须在 long long 范围内。单次 O(log n)。

```
struct LazySeg
{
    int n;
    vector<long long> sum, tag;

    LazySeg(int n) : n(n), sum(4 * n + 4), tag(4 * n + 4) { assert(n > 0); }

    void apply(int p, int l, int r, long long v)
    {
        sum[p] += (r - l + 1) * v;
        tag[p] += v;
    }

    void push(int p, int l, int r)
    {
        int m = (l + r) / 2;
        apply(p * 2, l, m, tag[p]);
        apply(p * 2 + 1, m + 1, r, tag[p]);
        tag[p] = 0;
    }

    void add(int l, int r, long long v) { add(1, 1, n, l, r, v); }

    void add(int p, int l, int r, int ql, int qr, long long v)
    {
        if (ql <= l && r <= qr)
        {
            apply(p, l, r, v);
            return;
        }
        push(p, l, r);
        int m = (l + r) / 2;
        if (ql <= m) add(p * 2, l, m, ql, qr, v);
        if (qr > m) add(p * 2 + 1, m + 1, r, ql, qr, v);
        sum[p] = sum[p * 2] + sum[p * 2 + 1];
    }

    long long query(int l, int r) { return query(1, 1, n, l, r); }

    long long query(int p, int l, int r, int ql, int qr)
    {
        if (ql <= l && r <= qr) return sum[p];
        push(p, l, r);
        int m = (l + r) / 2;
        long long ans = 0;
        if (ql <= m) ans += query(p * 2, l, m, ql, qr);
        if (qr > m) ans += query(p * 2 + 1, m + 1, r, ql, qr);
        return ans;
    }
};
```

模板题使用：Luogu P3372 【模板】线段树 1

题意：给出长度 n 的正整数序列与 m 次操作，1≤n,m≤100000，支持闭区间加正整数 k 与闭区间求和；任意时刻全序列之和不超过 2×10^18。LazySeg(n) 零初始化后用 n 次单点 add(i,i,a) 装入初值；修改调用 add(l,r,k)，询问调用 query(l,r)。初始化 O(n log n)，每次操作 O(log n)，空间 O(n)；初值、增量、区间和均用 long long。

所需模板：LazySeg（第 427 页，34.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3372

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    LazySeg seg(n);
    for (int i = 1; i <= n; i++)
    {
        long long a;
        cin >> a;
        seg.add(i, i, a);
    }
    while (m--)
    {
        int op, l, r;
        cin >> op >> l >> r;
        if (op == 1)
        {
            long long k;
            cin >> k;
            seg.add(l, r, k);
        }
        else
            cout << seg.query(l, r) << '\n';
    }
}
```

34.1.4 线段树：区间乘、加、赋值与模意义求和

AffineSegTree(a,mod) 传入非空 0-based 初值数组；查询与修改采用 1-based 非空闭区间。update(l,r,mul,add) 将区间元素 x 改为 mul*x+add；乘法用 (mul,0)，加法用 (1,add)，赋值用 (0,value)。query(l,r) 返回区间和。1≤mod≤INT_MAX，模数无需为素数，初值和修改参数允许 long long 全范围，入口统一归一化。旧标记 (m,b) 后应用新标记 (a,c)，复合为 (m*a,b*a+c)，乘法取模后再相加。建树 O(n)，单次 O(log n)，空间 O(n)。内部 build、apply 等函数不作为公共接口；动态版重建新对象。

```
struct AffineSegTree
{
    using ll = long long;

    struct Node
    {
        ll sum = 0, mul = 1, add = 0;
    };

    int n, mod;
```

```
    vector<Node> t;

    AffineSegTree(const vector<ll> &a, int mod) : n(a.size()), mod(mod), t(4 * n + 4)
    {
        assert(n > 0 && mod > 0);
        build(1, 1, n, a);
    }

    ll norm(ll x) const
    {
        x %= mod;
        if (x < 0) x += mod;
        return x;
    }

    void pull(int p) { t[p].sum = (t[p * 2].sum + t[p * 2 + 1].sum) % mod; }

    void build(int p, int l, int r, const vector<ll> &a)
    {
        t[p] = Node{};
        if (l == r)
        {
            t[p].sum = norm(a[l - 1]);
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m + 1, r, a);
        pull(p);
    }

    // New transform follows the old pending transform: x -> x*a+b.
    void apply(int p, int len, ll a, ll b)
    {
        t[p].sum = (t[p].sum * a % mod + len * b % mod) % mod;
        t[p].mul = t[p].mul * a % mod;
        t[p].add = (t[p].add * a % mod + b) % mod;
    }

    void push(int p, int l, int r)
    {
        if (t[p].mul == 1 && t[p].add == 0) return;
        int m = (l + r) / 2;
        apply(p * 2, m - l + 1, t[p].mul, t[p].add);
        apply(p * 2 + 1, r - m, t[p].mul, t[p].add);
        t[p].mul = 1;
        t[p].add = 0;
    }

    void update(int l, int r, ll a, ll b)
    {
        assert(1 <= l && l <= r && r <= n);
        update(1, 1, n, l, r, norm(a), norm(b));
    }

    void update(int p, int l, int r, int ql, int qr, ll a, ll b)
```

```
    {
        if (ql <= l && r <= qr)
        {
            apply(p, r - l + 1, a, b);
            return;
        }
        push(p, l, r);
        int m = (l + r) / 2;
        if (ql <= m) update(p * 2, l, m, ql, qr, a, b);
        if (qr > m) update(p * 2 + 1, m + 1, r, ql, qr, a, b);
        pull(p);
    }

    ll query(int l, int r)
    {
        assert(1 <= l && l <= r && r <= n);
        return query(1, 1, n, l, r);
    }

    ll query(int p, int l, int r, int ql, int qr)
    {
        if (ql <= l && r <= qr) return t[p].sum;
        push(p, l, r);
        int m = (l + r) / 2;
        ll answer = 0;
        if (ql <= m) answer += query(p * 2, l, m, ql, qr);
        if (qr > m) answer += query(p * 2 + 1, m + 1, r, ql, qr);
        return answer % mod;
    }
};
```

模板题使用：Luogu P3373

题意：对数组执行区间乘、区间加和区间求和，结果对给定模数取模。区间为 1-based 闭区间；update 的两个参数表示先乘再加。

所需模板：AffineSegTree（第 428 页，34.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3373

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, q, mod;
    std::cin >> n >> q >> mod;
    std::vector<long long> a(n);
    for (auto &x : a) std::cin >> x;
    AffineSegTree tree(a, mod);
    while (q--)
    {
        int op, l, r;
        std::cin >> op >> l >> r;
        if (op == 3)
            std::cout << tree.query(l, r) << '\n';
        else
        {
            long long k;
            std::cin >> k;
            if (op == 1)
                tree.update(l, r, k, 0);
            else
                tree.update(l, r, 1, k);
        }
    }
}
```

相关应用：线段树维护单调栈的双向极大值贡献；区间加接雨水应用，不将普通单调栈直接视为覆盖。

34.1.5 线段树维护单调栈：区间加与接雨水

MonotoneStackSeg(a) 要求非空 long long 数组；add(l,r,delta) 对 0-based 半开区间加值，空区间不变。water() 返回整个数组的接雨水体积。scan(l,r,height,reverse=false) 按左到右（reverse=true 则右到左）扫描，将 height 与当前元素取 max 并累加 height-a[i]，返回 (累加值, 最后的 height)；空区间返回 (0,height)。支持负高度与负增量，无固定零边界。节点缓存双向前缀极大值减原值的和；区间平移不改变两者差。建树 O(n)，add/scan 最坏 $O(\log^2(n+1))$，water 为 O(1)，空间 O(n)；递归深度 O(log n)。4n 须可由 int 表示；高度、和、懒标记、外部 height 与长度的乘积以及所有中间运算均须在 long long 范围内。t 与 extra/pull/push 及带节点编号的函数属内部实现。

```
struct MonotoneStackSeg
{
    using ll = long long;

    struct Node
    {
        int len = 0;
        ll mx = 0, sum = 0, lazy = 0;
        array<ll, 2> fill{};
    };

    int n;
    vector<Node> t;

    MonotoneStackSeg(const vector<ll> &a) : n(a.size()), t(4 * n)
    {
        assert(n > 0);
        build(1, 0, n, a);
    }

    void apply(int p, ll delta)
    {
        t[p].mx += delta;
        t[p].sum += delta * t[p].len;
        t[p].lazy += delta;
    }

    void push(int p)
    {
        if (!t[p].lazy) return;
        apply(p * 2, t[p].lazy);
        apply(p * 2 + 1, t[p].lazy);
        t[p].lazy = 0;
    }
```

单路扫描、合并与建树（接上页同一结构体）：

```
    ll extra(int p, ll height, int dir)
    {
        if (height >= t[p].mx) return height * t[p].len - t[p].sum;
        if (t[p].len == 1) return 0;
        push(p);
        int first = p * 2 + dir, second = p * 2 + 1 - dir;
        if (height >= t[first].mx)
            return height * t[first].len - t[first].sum + extra(second, height, dir);
        return extra(first, height, dir) + (t[p].fill[dir] - t[first].fill[dir]);
    }

    void pull(int p)
    {
        int l = p * 2, r = l + 1;
        t[p].len = t[l].len + t[r].len;
        t[p].mx = max(t[l].mx, t[r].mx);
        t[p].sum = t[l].sum + t[r].sum;
        t[p].fill[0] = t[l].fill[0] + extra(r, t[l].mx, 0);
        t[p].fill[1] = t[r].fill[1] + extra(l, t[r].mx, 1);
    }

    void build(int p, int l, int r, const vector<ll> &a)
    {
        if (r - l == 1)
        {
            t[p].len = 1;
            t[p].mx = a[l];
            t[p].sum = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m, r, a);
        pull(p);
    }
```

区间加、双向查询与雨水体积（接上页同一结构体）：

```
    void add(int p, int l, int r, int ql, int qr, ll delta)
    {
        if (ql <= l && r <= qr) return apply(p, delta);
        push(p);
        int m = (l + r) / 2;
        if (ql < m) add(p * 2, l, m, ql, qr, delta);
        if (m < qr) add(p * 2 + 1, m, r, ql, qr, delta);
        pull(p);
    }

    void add(int l, int r, ll delta)
    {
        assert(0 <= l && l <= r && r <= n);
        if (l < r) add(1, 0, n, l, r, delta);
    }

    ll scan(int p, int l, int r, int ql, int qr, ll &height, int dir)
    {
        if (ql <= l && r <= qr)
        {
            ll answer = extra(p, height, dir);
            height = max(height, t[p].mx);
            return answer;
        }
        push(p);
        int m = (l + r) / 2;
        ll answer = 0;
        if (dir && m < qr) answer += scan(p * 2 + 1, m, r, ql, qr, height, dir);
        if (ql < m) answer += scan(p * 2, l, m, ql, qr, height, dir);
        if (!dir && m < qr) answer += scan(p * 2 + 1, m, r, ql, qr, height, dir);
        return answer;
    }

    // Return (sum of running maximum - a[i], final running maximum).
    pair<ll, ll> scan(int l, int r, ll height, bool reverse = false)
    {
        assert(0 <= l && l <= r && r <= n);
        ll answer = 0;
        if (l < r) answer = scan(1, 0, n, l, r, height, reverse);
        return {answer, height};
    }

    ll water() const
    {
        ll gap = t[1].mx * n - t[1].sum;
        return t[1].fill[0] - (gap - t[1].fill[1]);
    }
};
```

应用题补充：NERC 2023 G / Luogu P12438

题意：Great City Saint Petersburg：先输出初始雨水量，每次输入 1-based 闭区间使高度加一，再输出全局雨水量。调用时只减 l。该题为比赛应用，不计正式模板题覆盖；一般负增量和 scan 接口另有本地验证。

所需模板：MonotoneStackSeg（第 432 页，34.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P12438

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    MonotoneStackSeg s(a);
    cout << s.water() << '\n';
    while (q--)
    {
        int l, r;
        cin >> l >> r;
        s.add(l - 1, r, 1);
        cout << s.water() << '\n';
    }
}
```

应用题补充：带外部高度的双向扫描（本地接口示例）

题意：输入 n,q 和高度数组，再输入 op,l,r,x；区间均为 0-based 半开。op=0 对区间加 x；op=1 从左向右、op=2 从右向左，初始最高值为 x，输出逐步最高值与当前元素之差的和，以及最终最高值。空区间保持 x，支持负数。

所需模板：MonotoneStackSeg（第 432 页，34.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://github.com/gsh20040816/CPCtemplate/issues/11

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    MonotoneStackSeg s(a);
    while (q--)
    {
        int op, l, r;
        long long x;
        cin >> op >> l >> r >> x;
        if (op == 0)
            s.add(l, r, x);
        else
        {
            auto [cost, height] = s.scan(l, r, x, op == 2);
            cout << cost << ' ' << height << '\n';
        }
    }
}
```

34.2 线段树合并 & 分裂

34.2.1 线段树合并与分裂

MergeSplitTree(n) 建空集合 0，或传 vector<long long> 建初始重数；n>0，值域 [0,n)。add(s,x,delta) 点增减，结果重数非负；sum(s,l,r) 查询半开值域；kth(s,k) 按重数返回第 k 小的 0-based 值，不存在返回-1。split(s,l,r) 移走区间并返回新集合号，空区间也创建集合；split_rank(s,k,reverse=false) 移走最小 k 个元素到新集合；reverse=true 移走最大 k 个，k 在 [0, 集合总重数] 内，同值的重数可拆开，k=0 也创建空集合。merge(dst,src) 消耗不同的 src，src 变空后可重用。所有集合总量分别须 ≤LLONG_MAX，节点/集合索引须适合 int。仅通过集合号操作，root/t/free 只读，其余递归函数内部使用，禁止浅拷贝根。初始化 O(n)，增减/分裂/查询 O(log(n+1))；单次合并非 O(log n)，遍历重叠节点并回收，总合并开销可摊到节点创建。完整对象值复制独立。

```
struct MergeSplitTree
{
    using ll = long long;
    struct Node
    {
        int l = 0, r = 0;
        ll sum = 0;
    };

    int n;
    vector<Node> t{Node{}};
    vector<int> root{0}, free;

    MergeSplitTree(int n) : n(n)
    {
        assert(n > 0);
    }

    MergeSplitTree(const vector<ll> &a) : MergeSplitTree((int)a.size())
    {
        root[0] = build(0, n, a);
    }

    int make()
    {
        if (free.empty())
        {
            t.push_back(Node{});
            return (int)t.size() - 1;
        }
        int p = free.back();
        free.pop_back();
        return p;
    }
```

线段树合并与分裂（接上页同一结构体）：

```
    void release(int p)
    {
        t[p] = Node{};
        free.push_back(p);
    }

    void pull(int p)
    {
        assert(t[t[p].l].sum <= LLONG_MAX - t[t[p].r].sum);
        t[p].sum = t[t[p].l].sum + t[t[p].r].sum;
    }

    int build(int l, int r, const vector<ll> &a)
    {
        if (r - l == 1)
        {
            assert(a[l] >= 0);
            if (!a[l]) return 0;
            int p = make();
            t[p].sum = a[l];
            return p;
        }
        int m = l + (r - l) / 2;
        int x = build(l, m, a);
        int y = build(m, r, a);
        if (!x && !y) return 0;
        int p = make();
        t[p].l = x;
        t[p].r = y;
        pull(p);
        return p;
    }
```

线段树合并与分裂（接上页同一结构体）：

```
    int add_node(int p, int l, int r, int x, ll v)
    {
        if (!p) p = make();
        if (r - l == 1)
        {
            assert(v >= -t[p].sum);
            t[p].sum += v;
        }
        else
        {
            int m = l + (r - l) / 2;
            if (x < m)
            {
                int q = add_node(t[p].l, l, m, x, v);
                t[p].l = q;
            }
            else
            {
                int q = add_node(t[p].r, m, r, x, v);
                t[p].r = q;
            }
            pull(p);
        }
        if (t[p].sum) return p;
        release(p);
        return 0;
    }

    // Sets and coordinates are 0-based; resulting multiplicities are nonnegative.
    void add(int s, int x, ll v)
    {
        assert(0 <= s && s < (int)root.size());
        assert(0 <= x && x < n);
        assert(v <= 0 || t[root[s]].sum <= LLONG_MAX - v);
        if (!v) return;
        root[s] = add_node(root[s], 0, n, x, v);
    }
```

线段树合并与分裂（接上页同一结构体）：

```
    int meld(int p, int q, int l, int r)
    {
        if (!p || !q) return p ? p : q;
        if (r - l == 1)
            t[p].sum += t[q].sum;
        else
        {
            int m = l + (r - l) / 2;
            t[p].l = meld(t[p].l, t[q].l, l, m);
            t[p].r = meld(t[p].r, t[q].r, m, r);
            pull(p);
        }
        release(q);
        return p;
    }

    // Consume src; its set ID remains valid and becomes empty.
    void merge(int dst, int src)
    {
        assert(0 <= dst && dst < (int)root.size());
        assert(0 <= src && src < (int)root.size() && dst != src);
        assert(t[root[dst]].sum <= LLONG_MAX - t[root[src]].sum);
        root[dst] = meld(root[dst], root[src], 0, n);
        root[src] = 0;
    }
```

线段树合并与分裂（接上页同一结构体）：

```
    pair<int, int> cut(int p, int l, int r, int a, int b)
    {
        if (!p || b <= l || r <= a) return {p, 0};
        if (a <= l && r <= b) return {0, p};
        int m = l + (r - l) / 2;
        auto [x, u] = cut(t[p].l, l, m, a, b);
        auto [y, v] = cut(t[p].r, m, r, a, b);
        t[p].l = x;
        t[p].r = y;
        pull(p);
        if (!t[p].sum)
        {
            release(p);
            p = 0;
        }
        int q = 0;
        if (u || v)
        {
            q = make();
            t[q].l = u;
            t[q].r = v;
            pull(q);
        }
        return {p, q};
    }

    // Move [l,r) into a new set, even when the extracted set is empty.
    int split(int s, int l, int r)
    {
        assert(0 <= s && s < (int)root.size());
        assert(0 <= l && l <= r && r <= n);
        auto [p, q] = cut(root[s], 0, n, l, r);
        root[s] = p;
        root.push_back(q);
        return (int)root.size() - 1;
    }

    pair<int, int> cut_rank(int p, int l, int r, ll k, bool reverse)
    {
        if (!p || k == 0)
            return {p, 0};
        if (k == t[p].sum)
            return {0, p};
        int q = make();
        if (r - l == 1)
        {
            t[p].sum -= k;
            t[q].sum = k;
            return {p, q};
        }
        int m = l + (r - l) / 2;
        ll left = t[t[p].l].sum;
        ll right = t[t[p].r].sum;
        ll take_left = k < left ? k : left;
```

```
        if (reverse)
            take_left = k > right ? k - right : 0;
        auto [a, b] = cut_rank(t[p].l, l, m, take_left, reverse);
        auto [c, d] = cut_rank(t[p].r, m, r, k - take_left, reverse);
        t[p].l = a;
        t[p].r = c;
        t[q].l = b;
        t[q].r = d;
        pull(p);
        pull(q);
        return {p, q};
    }

    // Move the k smallest (or largest) occurrences into a new set.
    int split_rank(int s, ll k, bool reverse = false)
    {
        assert(0 <= s && s < (int)root.size());
        assert(0 <= k && k <= t[root[s]].sum);
        auto [p, q] = cut_rank(root[s], 0, n, k, reverse);
        root[s] = p;
        root.push_back(q);
        return (int)root.size() - 1;
    }
```

线段树合并与分裂（接上页同一结构体）：

```
    ll query(int p, int l, int r, int a, int b) const
    {
        if (!p || b <= l || r <= a) return 0;
        if (a <= l && r <= b) return t[p].sum;
        int m = l + (r - l) / 2;
        return query(t[p].l, l, m, a, b) + query(t[p].r, m, r, a, b);
    }

    ll sum(int s, int l, int r) const
    {
        assert(0 <= s && s < (int)root.size());
        assert(0 <= l && l <= r && r <= n);
        return query(root[s], 0, n, l, r);
    }

    int select(int p, int l, int r, ll k) const
    {
        if (r - l == 1) return l;
        int m = l + (r - l) / 2;
        ll left = t[t[p].l].sum;
        if (k <= left) return select(t[p].l, l, m, k);
        return select(t[p].r, m, r, k - left);
    }

    int kth(int s, ll k) const
    {
        assert(0 <= s && s < (int)root.size());
        if (k <= 0 || k > t[root[s]].sum) return -1;
        return select(root[s], 0, n, k);
    }
};
```

模板题使用：Luogu P5494 / 线段树分裂

题意：n,m≤200000，初始集合 1 含每种值 1..n 的重数 ai≤m。0 p x y 移走闭区间到新集合；1 p t 合并并清空 t，之后不再使用 t；2 p x q 增加 x 个值 q；3 p x y 查区间重数；4 p k 查第 k 小，不存在-1。集合和值均转 0-based，闭区间转 [x-1,y)，重数用 long long。

所需模板：MergeSplitTree（第 437 页，34.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5494

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    MergeSplitTree seg(a);
    for (int i = 0; i < m; i++)
    {
        int op, p;
        cin >> op >> p;
        p--;
        if (op == 0 || op == 3)
        {
            int x, y;
            cin >> x >> y;
            if (op == 0)
                seg.split(p, x - 1, y);
            else
                cout << seg.sum(p, x - 1, y) << '\n';
        }
        else if (op == 1)
        {
            int t;
            cin >> t;
            seg.merge(p, t - 1);
        }
        else if (op == 2)
        {
            long long x;
            int q;
            cin >> x >> q;
            seg.add(p, q - 1, x);
        }
        else
        {
            long long k;
            cin >> k;
            int x = seg.kth(p, k);
            cout << (x == -1 ? -1 : x + 1) << '\n';
        }
    }
}
```

应用题补充：UVA1479 / 删边改权与连通块第 k 大

题意：多组无向图；D e 删边（每边至多一次），C x v 改点权，Q x k 查所在连通块第 k 大，非法 k 贡献 0；E 结束当前组，0 0 结束输入，输出查询均值六位小数。先读完操作保存每次修改前的旧值，倒序处理；删边变合并，改权还原旧值。所有初值/新值离散化，重复权值按频数累计。split(0,0,0) 连续创建空集合，编号对应顶点；合并根 u,v 时 t.merge(u,v) 与 d.merge(u,v) 顺序一致。k 的换算先转 long long，避免 INT_MIN 溢出。

所需模板：MergeSplitTree（第 437 页，34.2.1 节）、dsu（第 373 页，29.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.org/external/14/1479.pdf

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    struct Op
    {
        char op;
        int x, y;
    };
    int n, m, tc = 0;
    while (cin >> n >> m && n)
    {
        vector<int> a(n), values;
        for (auto &x : a) cin >> x;
        values = a;
        vector<pair<int, int>> edges(m);
        for (auto &[u, v] : edges)
        {
            cin >> u >> v;
            u--;
            v--;
        }
        vector<int> deleted(m);
        vector<Op> ops;
        char op;
        while (cin >> op && op != 'E')
        {
            int x, y = 0;
            cin >> x;
            x--;
            if (op == 'D') deleted[x] = 1;
            else cin >> y;
            if (op == 'C')
            {
                values.push_back(y);
                swap(a[x], y);
            }
            ops.push_back({op, x, y});
        }
```

继续处理操作（接上页同一 main）：

```
        sort(values.begin(), values.end());
        values.erase(unique(values.begin(), values.end()), values.end());
        auto index = [&](int x)
        {
            return lower_bound(values.begin(), values.end(), x) - values.begin();
        };
        MergeSplitTree t(values.size());
        for (int u = 0; u < n; u++)
        {
            if (u) t.split(0, 0, 0);
            t.add(u, index(a[u]), 1);
        }
        dsu d(n);
        auto join = [&](int e)
        {
            auto [u, v] = edges[e];
            u = d.find(u);
            v = d.find(v);
            if (u == v) return;
            t.merge(u, v);
            d.merge(u, v);
        };
        for (int e = 0; e < m; e++)
            if (!deleted[e]) join(e);
        long long sum = 0;
        int count = 0;
        for (int i = (int)ops.size() - 1; i >= 0; i--)
        {
            auto [op, x, y] = ops[i];
            if (op == 'D') join(x);
            else if (op == 'C')
            {
                int r = d.find(x);
                t.add(r, index(a[x]), -1);
                a[x] = y;
                t.add(r, index(a[x]), 1);
            }
            else
            {
                int r = d.find(x);
                long long k = (long long)d.size(r) - y + 1;
                int p = t.kth(r, k);
                if (p != -1) sum += values[p];
                count++;
            }
        }
        cout << "Case " << ++tc << ": " << fixed << setprecision(6)
             << (long double)sum / count << '\n';
    }
}
```

接口用法演示：按重数升序/降序分裂 API

演示约定：读 n q，初始集合 0 为空，值域 [0,n)。1 s x delta 增减重数且结果非负；2 s k reverse 移走 k 个最小值（reverse=0）或最大值（1）并输出新集合号；3 s l r 输出 [l,r) 重数和；4 s other 把不同的 other 合并到 s，other 清空可复用；5 s k 输出 1-based 第 k 小的值，无结果-1。k 可等于 0 或集合总重数，重复值可分到两边。自定义 API 协议，非 OJ 判定。

所需模板：MergeSplitTree（第 437 页，34.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    MergeSplitTree tree(n);
    while (q--)
    {
        int op, s;
        cin >> op >> s;
        if (op == 1)
        {
            int x;
            long long delta;
            cin >> x >> delta;
            tree.add(s, x, delta);
        }
        else if (op == 2)
        {
            long long k;
            bool reverse;
            cin >> k >> reverse;
            cout << tree.split_rank(s, k, reverse) << '\n';
        }
        else if (op == 3)
        {
            int l, r;
            cin >> l >> r;
            cout << tree.sum(s, l, r) << '\n';
        }
        else if (op == 4)
        {
            int other;
            cin >> other;
            tree.merge(s, other);
        }
        else
        {
            long long k;
            cin >> k;
            cout << tree.kth(s, k) << '\n';
        }
    }
}
```

34.3 李超线段树

34.3.1 精确线段李超树：最大值编号

构造传入非空的所有查询横坐标，排序去重后数量 M。add(x1,y1,x2,y2) 插入闭线段并返回从 1 开始的编号；可交换端点，竖直线段在该横坐标取较高端点。query(x) 返回交点最高的线段编号，同值取较早插入者；无交点返回 0，x 必须在预登记坐标内。value(id,x) 返回未约分的 (分子, 正分母)，要求 id>0 且 x 在线段内；不直接相除即可保留精度。横坐标绝对值 ≤10^9，端点纵坐标绝对值 ≤2×10^18，int128 交叉乘积在这些界内安全；不使用浮点或 epsilon。最小值问题可先把纵坐标取负，得到编号后再还原。整条直线用原 LiChao 更短；这里插入仅作用于端点范围。构造 O(M log M)，插入 O($\log^2$(M+1))，查询 O(log(M+1))，空间 O(M+ 线段数)；要求 M<INT_MAX/4，累计插入线段数 <INT_MAX。

```
struct SegmentLiChao
{
    using ll = long long;
    using I = __int128_t;

    struct Line
    {
        ll x1, y1, x2, y2;
    };

    vector<ll> xs;
    vector<Line> lines{{0, 0, 0, 0}};
    vector<int> t;

    SegmentLiChao(vector<ll> x) : xs(move(x))
    {
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()), xs.end());
        assert(!xs.empty() && xs.front() >= -1000000000 && xs.back() <= 1000000000);
        t.resize(4 * xs.size());
    }

    pair<I, I> value(int id, ll x) const
    {
        auto [a, b, c, d] = lines[id];
        if (a == c) return {max(b, d), 1};
        return {I(b) * (c - x) + I(d) * (x - a), c - a};
    }
```

精确比较与线段插入（接上页同一结构体）：

```
    bool better(int u, int v, ll x) const
    {
        if (!u || !v) return u != 0;
        auto [a, b] = value(u, x);
        auto [c, d] = value(v, x);
        return a * d == c * b ? u < v : a * d > c * b;
    }

    void insert(int p, int l, int r, int ql, int qr, int id)
    {
        int m = (l + r) / 2;
        if (ql <= l && r <= qr)
        {
            if (better(id, t[p], xs[m])) swap(id, t[p]);
            if (!id || l == r) return;
            if (better(id, t[p], xs[l]))
                insert(p * 2, l, m, l, m, id);
            else if (better(id, t[p], xs[r]))
                insert(p * 2 + 1, m + 1, r, m + 1, r, id);
            return;
        }
        if (ql <= m) insert(p * 2, l, m, ql, qr, id);
        if (qr > m) insert(p * 2 + 1, m + 1, r, ql, qr, id);
    }

    int add(ll x1, ll y1, ll x2, ll y2)
    {
        assert(-1000000000 <= x1 && x1 <= 1000000000);
        assert(-1000000000 <= x2 && x2 <= 1000000000);
        assert(-2000000000000000000LL <= y1 && y1 <= 2000000000000000000LL);
        assert(-2000000000000000000LL <= y2 && y2 <= 2000000000000000000LL);
        if (x1 > x2)
        {
            swap(x1, x2);
            swap(y1, y2);
        }
        int id = lines.size();
        lines.push_back({x1, y1, x2, y2});
        int l = lower_bound(xs.begin(), xs.end(), x1) - xs.begin();
        int r = upper_bound(xs.begin(), xs.end(), x2) - xs.begin() - 1;
        if (l <= r) insert(1, 0, (int)xs.size() - 1, l, r, id);
        return id;
    }
```

查询最优线段编号（接上页同一结构体）：

```
    int query(ll x) const
    {
        int i = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
        assert(i < (int)xs.size() && xs[i] == x);
        int p = 1, l = 0, r = (int)xs.size() - 1, ans = 0;
        while (true)
        {
            if (better(t[p], ans, x)) ans = t[p];
            if (l == r) return ans;
            int m = (l + r) / 2;
            if (i <= m)
            {
                p *= 2;
                r = m;
            }
            else
            {
                p = p * 2 + 1;
                l = m + 1;
            }
        }
    }
};
```

模板题使用：Library Checker segment_add_get_min

题意：在线段有效区间内查询最小值。预读所有查询横坐标以离散化，再按原序处理；没有查询时直接退出。题目横坐标都是整数，因此半开区间 [l,r) 换成闭区间 [l,r-1]。端点值取负，把最小值转为最大值；无结果输出 INFINITY，返回分数在本题必为整数。原线性式的端点计算在题目范围内可用 long long。

所需模板：SegmentLiChao（第 448 页，34.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/segment_add_get_min

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;

    struct Query
    {
        int type;
        long long l, r, a, b;
    };

    vector<Query> ops(n + q);
    vector<long long> xs;
    for (int i = 0; i < n + q; i++)
    {
        auto &o = ops[i];
        o.type = 0;
        if (i >= n) cin >> o.type;
        if (o.type == 0)
            cin >> o.l >> o.r >> o.a >> o.b;
        else
        {
            cin >> o.l;
            xs.push_back(o.l);
        }
    }
```

继续处理操作（接上页同一 main）：

```
    if (xs.empty()) return 0;
    SegmentLiChao t(xs);
    for (auto o : ops)
        if (o.type == 0)
            t.add(o.l, -o.a * o.l - o.b, o.r - 1, -o.a * (o.r - 1) - o.b);
        else
        {
            int id = t.query(o.l);
            if (!id)
                cout << "INFINITY\n";
            else
            {
                auto [num, den] = t.value(id, o.l);
                cout << (long long)(-num / den) << '\n';
            }
        }
}
```

应用题补充：Luogu P4097 / HEOI2013 Segment

题意：强制在线插入整数端点线段，查询最高交点的最早线段编号。预登记全部 1..39989 横坐标；每个操作用上次查询答案解码，插入不会修改 last。竖直线段取较高端点，无交点输出并保存 0。此题源自 HEOI，作为应用补充单列。

所需模板：SegmentLiChao（第 448 页，34.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4097

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    vector<long long> xs(39989);
    iota(xs.begin(), xs.end(), 1);
    SegmentLiChao t(xs);
    int q, last = 0;
    cin >> q;
    while (q--)
    {
        int op, x1, x2;
        long long y1, y2;
        cin >> op >> x1;
        x1 = (x1 + last - 1) % 39989 + 1;
        if (op == 0)
        {
            last = t.query(x1);
            cout << last << '\n';
        }
        else
        {
            cin >> y1 >> x2 >> y2;
            x2 = (x2 + last - 1) % 39989 + 1;
            y1 = (y1 + last - 1) % 1000000000 + 1;
            y2 = (y2 + last - 1) % 1000000000 + 1;
            t.add(x1, y1, x2, y2);
        }
    }
}
```

34.3.2 离散李超树与最优直线编号

构造传入所有可能查询的横坐标，add({k,b,id}) 插入直线，query(x) 返回最小值与最小编号，要求 x 在预登记坐标内且至少有一条线。整数求值用 int128，k、b、x 为有符号 64 位且中间值可表示。单次 O(log n)。

```
struct LiChao
{
    using ll = long long;
    using I = __int128_t;

    struct Line
    {
        ll k = 0, b = 0;
        int id = -1;

        I value(ll x) const { return I(k) * x + b; }
    };

    struct Node
    {
        Line line;
        int left = -1, right = -1;
    };

    vector<ll> xs;
    vector<Node> a{Node{}};

    LiChao(vector<ll> x) : xs(x)
    {
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()), xs.end());
        assert(!xs.empty());
    }

    bool better(Line u, Line v, ll x) const
    {
        return v.id == -1 ||
               (u.id != -1 && pair{u.value(x), u.id} < pair{v.value(x), v.id});
    }

    void add(Line line)
    {
        assert(line.id >= 0);
        add(0, 0, (int)xs.size() - 1, line);
    }
```

插入与查询（接上页同一结构体）：

```
    void add(int p, int l, int r, Line line)
    {
        int m = (l + r) / 2;
        if (better(line, a[p].line, xs[m])) swap(line, a[p].line);
        if (l == r || line.id == -1) return;
        bool left = better(line, a[p].line, xs[l]);
        bool right = better(line, a[p].line, xs[r]);
        if (!left && !right) return;
        int next = left ? a[p].left : a[p].right;
        if (next == -1)
        {
            next = (int)a.size();
            a.push_back(Node{});
            if (left)
                a[p].left = next;
            else
                a[p].right = next;
        }
        if (left)
            add(next, l, m, line);
        else
            add(next, m + 1, r, line);
    }

    pair<I, int> query(ll x) const
    {
        int idx = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
        assert(idx < (int)xs.size() && xs[idx] == x);
        int p = 0, l = 0, r = (int)xs.size() - 1;
        Line best;
        while (p != -1)
        {
            if (better(a[p].line, best, x)) best = a[p].line;
            if (l == r) break;
            int m = (l + r) / 2;
            if (idx <= m)
            {
                p = a[p].left;
                r = m;
            }
            else
            {
                p = a[p].right;
                l = m + 1;
            }
        }
        assert(best.id != -1);
        return {best.value(x), best.id};
    }
};
```

模板题使用：Library Checker line_add_get_min

题意：插入整条直线并查询最小值。先收集查询横坐标，再按输入顺序处理；初始至少一条线，查询必有结果。编号由调用者递增指定，本题只输出值。题目系数与横坐标范围保证最终值可转为 long long；模板内部用 int128 比较。

所需模板：LiChao（第 454 页，34.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/line_add_get_min

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;

    struct Query
    {
        int type;
        long long a, b;
    };

    vector<Query> ops(n + q);
    vector<long long> xs;
    for (int i = 0; i < n + q; i++)
    {
        auto &o = ops[i];
        o.type = 0;
        if (i >= n) cin >> o.type;
        cin >> o.a;
        if (o.type == 0)
            cin >> o.b;
        else
            xs.push_back(o.a);
    }
    if (xs.empty()) return 0;
    LiChao t(xs);
    int id = 0;
    for (auto o : ops)
        if (o.type == 0)
            t.add({o.a, o.b, id++});
        else
            cout << (long long)t.query(o.a).first << '\n';
}
```

34.4 区间最值操作 & 区间历史最值

34.4.1 Segment Tree Beats：区间截断、加法与和

SegmentBeats(a) 接受 0-based 数组，chmin/chmax/add(l,r,x)、sum(l,r) 均用半开区间。空数组/空区间允许，空和为 0，重新构造用于多测。维护最大/严格第二大/次数及最小对称项；阈值等于第二极值必须递归，先下传加法再按父极值截断。n≤(INT_MAX-4)/4 且内存可用；真实值、阈值、有限延迟极值、加法标记及先加后截断的中间值须严格在 ±2^60 内，返回和须可用 long long 表示。内部和与差值乘次数用 int128，避免尚未同步的孩子临时总和溢出；不能由最终值有界推断中间和安全。建树/空间 O(n)，sum/add 为 O(log n)，单次 chmin/chmax 最坏 O(n)，混合操作总摊还 $O((n+q)\log^2(n+1))$。sum 会下传，不是 const。内部辅助函数与 t 不得直接调用/改写；不维护历史最值。

```
struct SegmentBeats
{
    using ll = long long;
    static constexpr ll inf = 1LL << 60;

    struct Node
    {
        __int128 sum = 0;
        ll mx = -inf, mx2 = -inf, mn = inf, mn2 = inf, tag = 0;
        int cmx = 0, cmn = 0;
    };

    int n;
    vector<Node> t;

    SegmentBeats(const vector<ll> &a) : n(a.size()), t(4 * a.size() + 4)
    {
        assert(a.size() <= (INT_MAX - 4) / 4);
        if (n) build(1, 0, n, a);
    }
```

极值合并与递归建树（接上页同一结构体）：

```
    void pull(int p)
    {
        auto &u = t[p];
        const auto &a = t[p * 2], &b = t[p * 2 + 1];
        u.sum = a.sum + b.sum;
        u.mx = max(a.mx, b.mx);
        u.mx2 = max(a.mx == u.mx ? a.mx2 : a.mx,
                    b.mx == u.mx ? b.mx2 : b.mx);
        u.cmx = (a.mx == u.mx ? a.cmx : 0) + (b.mx == u.mx ? b.cmx : 0);
        u.mn = min(a.mn, b.mn);
        u.mn2 = min(a.mn == u.mn ? a.mn2 : a.mn,
                    b.mn == u.mn ? b.mn2 : b.mn);
        u.cmn = (a.mn == u.mn ? a.cmn : 0) + (b.mn == u.mn ? b.cmn : 0);
        u.tag = 0;
    }

    void build(int p, int l, int r, const vector<ll> &a)
    {
        if (r - l == 1)
        {
            assert(-inf < a[l] && a[l] < inf);
            t[p].sum = t[p].mx = t[p].mn = a[l];
            t[p].cmx = t[p].cmn = 1;
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m, r, a);
        pull(p);
    }
```

整段加法和单极值组截断（接上页同一结构体）：

```
    void apply_add(int p, int len, ll x)
    {
        auto &u = t[p];
        assert(-(__int128)inf < (__int128)u.mn + x &&
               (__int128)u.mx + x < inf);
        assert(-(__int128)inf < (__int128)u.tag + x &&
               (__int128)u.tag + x < inf);
        u.sum += (__int128)x * len;
        u.mx += x;
        u.mn += x;
        if (u.mx2 != -inf) u.mx2 += x;
        if (u.mn2 != inf) u.mn2 += x;
        u.tag += x;
    }

    // Only the maximum changes; equality with mx2 must descend instead.
    void apply_min(int p, ll x)
    {
        auto &u = t[p];
        if (u.mx <= x) return;
        assert(u.mx2 < x);
        u.sum += ((__int128)x - u.mx) * u.cmx;
        if (u.mn == u.mx) u.mn = x;
        else if (u.mn2 == u.mx) u.mn2 = x;
        u.mx = x;
    }

    void apply_max(int p, ll x)
    {
        auto &u = t[p];
        if (u.mn >= x) return;
        assert(x < u.mn2);
        u.sum += ((__int128)x - u.mn) * u.cmn;
        if (u.mx == u.mn) u.mx = x;
        else if (u.mx2 == u.mn) u.mx2 = x;
        u.mn = x;
    }
```

标记下传与有条件递归更新（接上页同一结构体）：

```
    void push(int p, int l, int r)
    {
        int m = (l + r) / 2;
        if (t[p].tag)
        {
            apply_add(p * 2, m - l, t[p].tag);
            apply_add(p * 2 + 1, r - m, t[p].tag);
            t[p].tag = 0;
        }
        for (int c : {p * 2, p * 2 + 1})
        {
            apply_min(c, t[p].mx);
            apply_max(c, t[p].mn);
        }
    }

    // type: 0 = chmin, 1 = chmax, 2 = add.
    void update(int p, int l, int r, int ql, int qr, ll x, int type)
    {
        if (qr <= l || r <= ql) return;
        if (type == 0 && t[p].mx <= x) return;
        if (type == 1 && t[p].mn >= x) return;
        if (ql <= l && r <= qr)
        {
            if (type == 0 && t[p].mx2 < x)
            {
                apply_min(p, x);
                return;
            }
            if (type == 1 && x < t[p].mn2)
            {
                apply_max(p, x);
                return;
            }
            if (type == 2)
            {
                apply_add(p, r - l, x);
                return;
            }
        }
        push(p, l, r);
        int m = (l + r) / 2;
        update(p * 2, l, m, ql, qr, x, type);
        update(p * 2 + 1, m, r, ql, qr, x, type);
        pull(p);
    }
```

公开修改与区间和查询（接上页同一结构体）：

```
    void chmin(int l, int r, ll x)
    {
        assert(0 <= l && l <= r && r <= n && -inf < x && x < inf);
        if (l < r) update(1, 0, n, l, r, x, 0);
    }

    void chmax(int l, int r, ll x)
    {
        assert(0 <= l && l <= r && r <= n && -inf < x && x < inf);
        if (l < r) update(1, 0, n, l, r, x, 1);
    }

    void add(int l, int r, ll x)
    {
        assert(0 <= l && l <= r && r <= n && -inf < x && x < inf);
        if (l < r) update(1, 0, n, l, r, x, 2);
    }

    __int128 query(int p, int l, int r, int ql, int qr)
    {
        if (qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return t[p].sum;
        push(p, l, r);
        int m = (l + r) / 2;
        return query(p * 2, l, m, ql, qr) + query(p * 2 + 1, m, r, ql, qr);
    }

    ll sum(int l, int r)
    {
        assert(0 <= l && l <= r && r <= n);
        __int128 ans = l < r ? query(1, 0, n, l, r) : 0;
        assert(LLONG_MIN <= ans && ans <= LLONG_MAX);
        return (ll)ans;
    }
};
```

模板题使用：Library Checker Range Chmin Chmax Add Range Sum

题意：维护数组，操作 0/1/2/3 分别为区间 chmin、chmax、加法、求和。全部使用 0-based 非空半开区间。N,Q≤200000；所有时刻元素绝对值 ≤10^12，修改参数绝对值 ≤2×10^12。因此延迟值满足本模板的范围要求，公开区间和也可用 long long 表示。输入区间不需要加减 1，按操作号调用对应方法。

所需模板：SegmentBeats（第 457 页，34.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/range_chmin_chmax_add_range_sum

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    SegmentBeats t(a);
    while (q--)
    {
        int type, l, r;
        cin >> type >> l >> r;
        if (type == 3)
        {
            cout << t.sum(l, r) << '\n';
            continue;
        }
        long long x;
        cin >> x;
        if (type == 0) t.chmin(l, r, x);
        else if (type == 1) t.chmax(l, r, x);
        else t.add(l, r, x);
    }
}
// END USAGE
```

第三十五章　划分树

35.1　划分树：静态区间第 k 小

DivisionTree(v)，v 为 0-based vector<long long>，完整 signed64 位、可重复；0≤n<INT_MAX。kth(l,r,k) 返回半开区间 [l,r) 内 0-based 第 k 小，要求 0≤l<r≤n、0≤k<r-l；第 k 大改传 r-l-1-k。允许构造空数组但不能查询；输入不改变，建后无修改接口。按中位数稳定划分，重复中位值只给左边所需配额。构造和空间 O(n log(n+1))，单次 O(log(n+1))；build/query 为递归内部函数，深度 O(log(n+1))。sorted/a/pre 只读。与压位小波矩阵同功能但存储更多、代码更短；原书第 k 大注释与实现不一致，本接口明确为第 k 小。

```
struct DivisionTree
{
    int n;
    vector<long long> sorted;
    vector<vector<long long>> a;
    vector<vector<int>> pre;

    DivisionTree(const vector<long long> &v) : n(v.size()), sorted(v)
    {
        sort(sorted.begin(), sorted.end());
        int h = bit_width((unsigned)max(1, n) - 1) + 1;
        a.assign(h, vector<long long>(n));
        pre.assign(h, vector<int>(n + 1));
        a[0] = v;
        if (n) build(0, n, 0);
    }
```

稳定划分与区间查询（接上页同一结构体）：

```
    void build(int l, int r, int d)
    {
        if (r - l == 1) return;
        int m = l + (r - l) / 2, same = m - l;
        for (int i = l; i < r; i++) same -= a[d][i] < sorted[m - 1];
        int x = l, y = m;
        for (int i = l; i < r; i++)
        {
            bool left = a[d][i] < sorted[m - 1];
            if (a[d][i] == sorted[m - 1] && same > 0)
            {
                left = true;
                same--;
            }
            if (left)
                a[d + 1][x++] = a[d][i];
            else
                a[d + 1][y++] = a[d][i];
            pre[d][i + 1] = pre[d][i] + left;
        }
        build(l, m, d + 1);
        build(m, r, d + 1);
    }

    long long query(int L, int R, int l, int r, int d, int k) const
    {
        if (r - l == 1) return a[d][l];
        int m = L + (R - L) / 2;
        int before = pre[d][l] - pre[d][L], cnt = pre[d][r] - pre[d][l];
        if (k < cnt) return query(L, m, L + before, L + before + cnt, d + 1, k);
        int x = m + (l - L - before);
        return query(m, R, x, x + (r - l - cnt), d + 1, k - cnt);
    }

    long long kth(int l, int r, int k) const
    {
        assert(0 <= l && l < r && r <= n && 0 <= k && k < r - l);
        return query(0, n, l, r, 0, k);
    }
};
```

模板题使用：Luogu P3834

题意：静态区间第 k 小；n、q 至 20 万，非负值至 10^9。输入为 1-based 闭区间与排名，调用 kth(l-1,r,k-1)。本程序使用中位数划分树，重复数仍按出现次数排名。

所需模板：DivisionTree（第 463 页，35.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3834

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    DivisionTree tree(a);
    while (q--)
    {
        int l, r, k;
        cin >> l >> r >> k;
        cout << tree.kth(l - 1, r, k - 1) << '\n';
    }
}
```

模板题使用：Library Checker range_kth_smallest

题意：半开区间 [l,r) 内第 k 小，位置和 k 均为 0-based，直接调用 kth；n、q 至 20 万，值在 $0..10^9$。

所需模板：DivisionTree（第 463 页，35.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/range_kth_smallest

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    DivisionTree tree(a);
    while (q--)
    {
        int l, r, k;
        cin >> l >> r >> k;
        cout << tree.kth(l, r, k) << '\n';
    }
}
```

相关应用：小波矩阵使用逐位稳定划分和压位 rank，解决静态区间顺序统计；在固定导航的划分树下按相关结构收录，不等同于原文划分树实现。

35.2 压位小波矩阵：静态区间顺序统计

WaveletMatrix(a)，a 为 0-based vector<long long>，支持完整有符号 64 位和重复值，建后不可修改；n 小于 INT_MAX。接口都用半开区间 [l,r)。kth(l,r,k) 取 0-based 第 k 小，要求 0≤k<r-l；第 k 大改传 r-l-1-k。less(l,r,x) 统计严格小于 x，freq 统计等于 x；两计数允许空数组和空区间，kth 不允许。构造 O(n log(n+1))，查询 O(log(s+1))，s 为不同值数。常驻空间 O(n+ceil(n/64)log(s+1)) 机器字，构造另用 O(n) 空间。rank 为内部方法，成员只读。字边界、区间值域计数及证明见本册补充说明。

```
struct WaveletMatrix
{
    int n, h;
    vector<long long> val;
    vector<int> mid;
    vector<vector<unsigned long long>> bit;
    vector<vector<int>> sum;

    WaveletMatrix(const vector<long long> &a) : n(a.size()), val(a)
    {
        sort(val.begin(), val.end());
        val.erase(unique(val.begin(), val.end()), val.end());
        h = bit_width((unsigned)max(1, (int)val.size()) - 1);
        mid.resize(h);
        bit.assign(h, vector<unsigned long long>(n / 64 + 1));
        sum.assign(h, vector<int>(n / 64 + 1));
        vector<int> v(n), w(n);
        for (int i = 0; i < n; i++)
            v[i] = lower_bound(val.begin(), val.end(), a[i]) - val.begin();
```

逐层稳定划分（接上页构造函数）、rank 与第 k 小（接上页同一结构体）：

```
        for (int d = h - 1; d >= 0; d--)
        {
            for (int i = 0; i < n; i++)
            {
                if (v[i] >> d & 1)
                    bit[d][i / 64] |= 1ULL << (i % 64);
                else
                    mid[d]++;
            }
            for (int i = 1; i < (int)sum[d].size(); i++)
                sum[d][i] = sum[d][i - 1] + popcount(bit[d][i - 1]);
            int l = 0, r = mid[d];
            for (int x : v)
            {
                if (x >> d & 1)
                    w[r++] = x;
                else
                    w[l++] = x;
            }
            v.swap(w);
        }
    }

    // Internal: number of one bits before position i at level d.
    int rank(int d, int i) const
    {
        return sum[d][i / 64] + popcount(bit[d][i / 64] & ((1ULL << (i % 64)) - 1));
    }

    // [l, r), k is zero-based; requires 0 <= k < r-l.
    long long kth(int l, int r, int k) const
    {
        assert(0 <= l && l <= r && r <= n && 0 <= k && k < r - l);
        int x = 0;
        for (int d = h - 1; d >= 0; d--)
        {
            int a = rank(d, l), b = rank(d, r);
            int z = r - l - b + a;
            if (k < z)
            {
                l -= a;
                r -= b;
            }
            else
            {
                k -= z;
                x |= 1 << d;
                l = mid[d] + a;
                r = mid[d] + b;
            }
        }
        return val[x];
    }
```

严格小于计数与频次（接上页同一结构体）：

```
    // Count values < x; empty intervals and absent values are allowed.
    int less(int l, int r, long long x) const
    {
        assert(0 <= l && l <= r && r <= n);
        int q = lower_bound(val.begin(), val.end(), x) - val.begin();
        if (q == (int)val.size()) return r - l;
        int ans = 0;
        for (int d = h - 1; d >= 0; d--)
        {
            int a = rank(d, l), b = rank(d, r);
            if (q >> d & 1)
            {
                ans += r - l - b + a;
                l = mid[d] + a;
                r = mid[d] + b;
            }
            else
            {
                l -= a;
                r -= b;
            }
        }
        return ans;
    }

    int freq(int l, int r, long long x) const
    {
        assert(0 <= l && l <= r && r <= n);
        int q = lower_bound(val.begin(), val.end(), x) - val.begin();
        if (q == (int)val.size() || val[q] != x) return 0;
        for (int d = h - 1; d >= 0; d--)
        {
            int a = rank(d, l), b = rank(d, r);
            if (q >> d & 1)
            {
                l = mid[d] + a;
                r = mid[d] + b;
            }
            else
            {
                l -= a;
                r -= b;
            }
        }
        return r - l;
    }
};
```

模板题使用：Library Checker range_kth_smallest

题意：静态数组，查询半开区间 [l,r) 内第 k 小，l/r/k 均按题面 0-based；直接调用 kth。n、q 至 20 万，值在 $0..10^9$。

所需模板：WaveletMatrix（第 466 页，35.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/range_kth_smallest

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    WaveletMatrix wm(a);
    while (q--)
    {
        int l, r, k;
        cin >> l >> r >> k;
        cout << wm.kth(l, r, k) << '\n';
    }
}
```

模板题使用：Library Checker static_range_frequency

题意：静态数组，统计半开区间 [l,r) 中 x 的出现次数；允许空数组、空查询区间、q=0 和未出现的 x。n、q 至 50 万，值在 $0..10^9$。

所需模板：WaveletMatrix（第 466 页，35.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/static_range_frequency

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    WaveletMatrix wm(a);
    while (q--)
    {
        int l, r;
        long long x;
        cin >> l >> r >> x;
        cout << wm.freq(l, r, x) << '\n';
    }
}
```

模板题使用：Luogu P3834

题意：静态区间第 k 小；题目为 1-based 闭区间和 1-based k，调用时转换成 kth(l-1,r,k-1)。n、q 至 20 万，题面值在 $0..10^9$；本程序采用小波矩阵，不建立历史版本。

所需模板：WaveletMatrix（第 466 页，35.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3834

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    WaveletMatrix wm(a);
    while (q--)
    {
        int l, r, k;
        cin >> l >> r >> k;
        cout << wm.kth(l - 1, r, k - 1) << '\n';
    }
}
```

第三十六章　二叉搜索树 & 平衡树

36.1　二叉搜索树 & 平衡树

36.1.1　GNU rope 与版本共享

实际类型为 __gnu_cxx::rope<T>，不是 std::rope；它是 GNU 扩展序列，包含 ext/rope。本工具链随后重新包含 cassert，保留断言接口。rp<int>、rp<char> 表示序列，默认空。size()、empty()、push_back(x)、insert(p,x)、erase(p,len)、replace(p,x)、substr(p,len) 直接沿用 rope；位置为 0-based，len 是长度而非右端点。约定 0≤p≤size，删除/截取要求 len≤size-p，读写单点要求 p<size。r[i] 读取第 i 项；substr 返回新序列。rp<T> b=a 共享底层树，随后通过容器接口修改 b 不影响 a，可用 vector<rp<int» 保存分支版本。复制构造共享根为 O(1)；随机访问沿树路径，正常平衡时 O(log n)。插入/删除/截取涉及路径、叶块复制和可能的再平衡，不能把每次操作都写成严格最坏 O(log n)；释放最后引用可能递归回收整棵子树。长期保留所有版本的内存不能按单个序列 O(n) 估计。不可缓存修改前的迭代器或使用可变代理跨版本写入。仅使用默认内存分配器及普通整数/字符值；函数生成型 rope 不在本模板范围。可持久化队列已通过定长构造加头尾下标版本：Q 个零单元，复制根与 l/r，入队 replace(r++,x)，出队读取 [l++]；直接 push_back/erase 版本在针对数据上耗尽内存。线上 942 ms、591.26 MiB，具体失败与通过范围见 docs/ROPE-NOTES.md。适合需要序列中间插删、截取和少量分支快照的场景；只按下标读写通常使用 vector 更直接。需要区间和、翻转等懒标记聚合时使用隐式 Treap，rope 本身不提供这些接口。下方示例通过修改 b 演示 a 保持原版本。

只需键值查改而不需要有序排名时，gp_hash_table / cc_hash_table 及选型建议见第 370 页。

```
template <class T> using rp = __gnu_cxx::rope<T>;
```

```
rp<int> a;
a.push_back(3);
a.insert(0, 7);
rp<int> b = a;
b.replace(1, 9);
b.erase(0, 1);
auto c = a.substr(0, 1);
int x = a[1];
```

模板题使用：Library Checker persistent_queue

题意：维护可持久化队列，每次从指定版本入队或出队并建立新版本。题目的 -1 号空版本映射为下标 0；rope 副本和头尾指针一起复制，replace 写入不会改变历史版本，出队只推进该版本队头。

所需模板：rp（第 471 页，36.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/persistent_queue

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    vector<rp<int>> s(q + 1);
    vector<int> l(q + 1), r(q + 1);
    s[0] = rp<int>(q, 0);
    for (int i = 1; i <= q; i++)
    {
        int op, t, x;
        cin >> op >> t;
        ++t;
        s[i] = s[t];
        l[i] = l[t];
        r[i] = r[t];
        if (op == 0)
        {
            cin >> x;
            s[i].replace(r[i]++, x);
        }
        else
        {
            cout << s[i][l[i]++] << '\n';
        }
    }
}
```

36.1.2 PBDS 有序树与顺序统计

GNU libstdc++ 扩展，包含 ext/pb_ds/assoc_container.hpp 和 ext/pb_ds/tree_policy.hpp。ost<T> 为去重有序集合；insert(x)、erase(x)、find(x)、lower_bound(x)、upper_bound(x) 沿用集合接口。order_of_key(x) 为严格小于 x 的元素数，find_by_order(k) 返回 0-based 第 k 个的迭代器，越界返回 end。默认 less<T>；改用其他严格弱序比较器时，排名和边界均按该比较器解释。单次 O(log n)，空间 O(n)，假设比较为 O(1)。重复值使用 ost<pair<T,int»，插入 {x,id} 且 id 全局唯一、1≤id<INT_MAX。order_of_key({x,0}) 统计小于 x，order_of_key({x,INT_MAX}) 统计不大于 x；find_by_order(k)->first 取值，须先检查 end。删除一个 x：it=lower_bound({x,0})，检查 it!=end 且 it->first==x 后 erase(it)。不可用 less_equal 模拟多重集合。前驱先检查迭代器是否为 begin，再递减；空集合和无解不得解引用 end。

只需键值查改而不需要有序排名时，gp_hash_table / cc_hash_table 及选型建议见第 370 页。

序列中间插删、截取与版本共享的 GNU rope 见第 471 页。

```
template <class T, class C = less<T>>
using ost = __gnu_pbds::tree<T,
                             __gnu_pbds::null_type,
                             C,
                             __gnu_pbds::rb_tree_tag,
                             __gnu_pbds::tree_order_statistics_node_update>;
```

模板题使用：Library Checker ordered_set

题意：维护无重复整数集合，支持第 k 小、计数及含等号的前驱/后继。find_by_order 使用 0-based 排名；order_of_key 只计严格小于 x，故不大于 x 的数量需补上相等元素。越界查询按题意输出 -1。

所需模板：ost（第 473 页，36.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/ordered_set

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q, x;
    cin >> n >> q;
    ost<int> s;
    while (n--)
    {
        cin >> x;
        s.insert(x);
    }
    while (q--)
    {
        int op;
        cin >> op >> x;
        if (op == 0) s.insert(x);
        if (op == 1) s.erase(x);
        if (op == 2)
        {
            auto it = s.find_by_order(x - 1);
            cout << (it == s.end() ? -1 : *it) << '\n';
        }
        if (op == 3) cout << s.order_of_key(x) + (s.find(x) != s.end()) << '\n';
        if (op == 4)
        {
            auto it = s.upper_bound(x);
            cout << (it == s.begin() ? -1 : *--it) << '\n';
        }
        if (op == 5)
        {
            auto it = s.lower_bound(x);
            cout << (it == s.end() ? -1 : *it) << '\n';
        }
    }
}
```

36.2 Treap

36.2.1 FHQ Treap：可重集合与顺序统计

insert(x) 插入一个副本，erase(x) 删除一个副本并返回是否存在；rank(x) 为严格小于 x 的元素数加一，x 不必存在；kth(k) 要求 1≤k≤size()。prev/next 为严格前驱/后继，不存在时返回 nullopt。支持重复值、LLONG_MIN 和 LLONG_MAX，分裂不使用 x-1。随机优先级下单次期望 O(log n)，并非确定性最坏界。节点池不回收已删除节点，空间按累计插入次数计算；可在构造时指定随机种子。

```
struct OrderedTreap
{
    using ll = long long;

    struct Node
    {
        int l = 0, r = 0, siz = 0;
        ll val = 0;
        unsigned long long pri = 0;
    };

    vector<Node> a{Node{}};
    mt19937_64 rng;
    int root = 0;

    OrderedTreap(unsigned long long seed = 712367821) : rng(seed) {}

    void pull(int p) { a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1; }

    int node(ll x)
    {
        int p = a.size();
        a.push_back({0, 0, 1, x, rng()});
        return p;
    }

    // left: val < x, or val <= x when equal is true. No x-1 overflow.
    void split(int p, ll x, int &l, int &r, bool equal = false)
    {
        if (!p)
        {
            l = r = 0;
            return;
        }
        if (a[p].val < x || (equal && a[p].val == x))
        {
            l = p;
            split(a[p].r, x, a[p].r, r, equal);
        }
        else
        {
            r = p;
            split(a[p].l, x, l, a[p].l, equal);
        }
        pull(p);
    }

```

```
    int merge(int l, int r)
    {
        if (!l || !r) return l ? l : r;
        if (a[l].pri > a[r].pri)
        {
            a[l].r = merge(a[l].r, r);
            pull(l);
            return l;
        }
        a[r].l = merge(l, a[r].l);
        pull(r);
        return r;
    }

    void insert(ll x)
    {
        int p = node(x), l, r;
        split(root, x, l, r);
        root = merge(merge(l, p), r);
    }

    bool erase(ll x)
    {
        int l, mid, r;
        split(root, x, l, mid);
        split(mid, x, mid, r, true);
        bool found = mid != 0;
        if (mid) mid = merge(a[mid].l, a[mid].r);
        root = merge(l, merge(mid, r));
        return found;
    }

    int size() const { return a[root].siz; }

    int less(ll x, bool equal = false) const
    {
        int p = root, ans = 0;
        while (p)
        {
            if (a[p].val < x || (equal && a[p].val == x))
            {
                ans += a[a[p].l].siz + 1;
                p = a[p].r;
            }
            else
                p = a[p].l;
        }
        return ans;
    }

    int rank(ll x) const { return less(x) + 1; }

    ll kth(int k) const
    {
        assert(1 <= k && k <= size());
        int p = root;
```

```
        while (true)
        {
            int left = a[a[p].l].siz;
            if (k == left + 1) return a[p].val;
            if (k <= left)
                p = a[p].l;
            else
            {
                k -= left + 1;
                p = a[p].r;
            }
        }
    }

    optional<ll> prev(ll x) const
    {
        int k = less(x);
        if (!k) return nullopt;
        return kth(k);
    }

    optional<ll> next(ll x) const
    {
        int k = less(x, true);
        if (k == size()) return nullopt;
        return kth(k + 1);
    }
};
```

模板题使用：Luogu P3369

题意：维护可重复整数集合，支持插入、删除一个值、排名、第 k 小、严格前驱和严格后继。题目保证前驱/后继存在，因此可解引用返回值。

所需模板：OrderedTreap（第 475 页，36.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    OrderedTreap g;
    while (n--)
    {
        int op;
        long long x;
        cin >> op >> x;
        if (op == 1)
            g.insert(x);
        else if (op == 2)
            g.erase(x);
        else if (op == 3)
            cout << g.rank(x) << '\n';
        else if (op == 4)
            cout << g.kth(x) << '\n';
        else if (op == 5)
            cout << *g.prev(x) << '\n';
        else
            cout << *g.next(x) << '\n';
    }
}
```

36.2.2 隐式 Treap：顺序反转、位翻转、加值与区间和

insert(k,value) 在前 k 个元素后插入，0≤k≤size()；区间接口 erase、reverse、flip_bits、add、query 均使用 1-based 非空闭区间。reverse 反转元素顺序；flip_bits 要求区间内全为 0/1，逐位取反并保持顺序，不检查这一前提。query 返回和，二进制区间即 1 的个数。values() 递归中序返回完整序列，O(n)。其余操作期望 O(log n)，不是最坏界。符号与加法标记表示 s*x+b，位翻转通过取负后加 1 实现，下传必须先符号再加法；顺序反转另行维护。所有值、和、标记、乘法及取负中间结果须在 long long 范围内。删除不回收节点，空间按累计插入次数计算；辅助函数和节点数组只供内部使用。

```
struct SequenceTreap
{
    using ll = long long;

    struct Node
    {
        int l = 0, r = 0, siz = 0;
        ll val = 0, sum = 0, tag = 0;
        unsigned long long pri = 0;
        bool rev = false, neg = false;
    };

    vector<Node> a{Node{}};
    mt19937_64 rng;
    int root = 0;

    SequenceTreap(unsigned long long seed = 712367821) : rng(seed) {}

    int size() const { return a[root].siz; }

    void pull(int p)
    {
        a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1;
        a[p].sum = a[a[p].l].sum + a[p].val + a[a[p].r].sum;
    }

    void apply_add(int p, ll x)
    {
        if (!p) return;
        a[p].val += x;
        a[p].sum += x * a[p].siz;
        a[p].tag += x;
    }

    void apply_negate(int p)
    {
        if (!p) return;
        a[p].val = -a[p].val;
        a[p].sum = -a[p].sum;
        a[p].tag = -a[p].tag;
        a[p].neg = !a[p].neg;
    }
```

顺序反转、懒标记下传与分裂（接上页同一结构体）：

```
    void apply_reverse(int p)
    {
        if (!p) return;
        swap(a[p].l, a[p].r);
        a[p].rev = !a[p].rev;
    }

    void push(int p)
    {
        if (a[p].rev)
        {
            apply_reverse(a[p].l);
            apply_reverse(a[p].r);
            a[p].rev = false;
        }
        if (a[p].neg)
        {
            apply_negate(a[p].l);
            apply_negate(a[p].r);
            a[p].neg = false;
        }
        if (a[p].tag)
        {
            apply_add(a[p].l, a[p].tag);
            apply_add(a[p].r, a[p].tag);
            a[p].tag = 0;
        }
    }

    // Split into the first k elements and the rest.
    void split(int p, int k, int &l, int &r)
    {
        if (!p)
        {
            l = r = 0;
            return;
        }
        push(p);
        int left = a[a[p].l].siz;
        if (k > left)
        {
            l = p;
            split(a[p].r, k - left - 1, a[p].r, r);
        }
        else
        {
            r = p;
            split(a[p].l, k, l, a[p].l);
        }
        pull(p);
    }
```

合并、插入、删除与顺序反转（接上页同一结构体）：

```
    int merge(int l, int r)
    {
        if (!l || !r) return l ? l : r;
        if (a[l].pri > a[r].pri)
        {
            push(l);
            a[l].r = merge(a[l].r, r);
            pull(l);
            return l;
        }
        push(r);
        a[r].l = merge(l, a[r].l);
        pull(r);
        return r;
    }

    // Insert after the first k elements, 0 <= k <= size().
    void insert(int k, ll value)
    {
        assert(0 <= k && k <= size());
        int p = a.size(), l, r;
        a.push_back({0, 0, 1, value, value, 0, rng(), false, false});
        split(root, k, l, r);
        root = merge(merge(l, p), r);
    }

    // All range operations use 1-based inclusive intervals.
    void erase(int l, int r)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        root = merge(x, z);
    }

    void reverse(int l, int r)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        apply_reverse(y);
        root = merge(merge(x, y), z);
    }
```

位翻转、加法、查询与输出（接上页同一结构体）：

```
    // Requires every value in [l,r] to be 0 or 1; preserves their order.
    void flip_bits(int l, int r)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        apply_negate(y);
        apply_add(y, 1);
        root = merge(merge(x, y), z);
    }

    void add(int l, int r, ll value)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        apply_add(y, value);
        root = merge(merge(x, y), z);
    }

    ll query(int l, int r)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        ll ans = a[y].sum;
        root = merge(merge(x, y), z);
        return ans;
    }

    void collect(int p, vector<ll> &out)
    {
        if (!p) return;
        push(p);
        collect(a[p].l, out);
        out.push_back(a[p].val);
        collect(a[p].r, out);
    }

    vector<ll> values()
    {
        vector<ll> answer;
        collect(root, answer);
        return answer;
    }
};
```

模板题使用：Luogu P3391

题意：初始序列为 1..n，多次翻转闭区间 [l,r]，最后输出序列。插入参数为前面已有元素数，翻转区间为 1-based 闭区间。

所需模板：SequenceTreap（第 479 页，36.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3391

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    SequenceTreap g;
    for (int i = 1; i <= n; i++) g.insert(i - 1, i);
    while (m--)
    {
        int l, r;
        cin >> l >> r;
        g.reverse(l, r);
    }
    for (long long x : g.values()) cout << x << ' ';
    cout << '\n';
}
```

应用题补充：二进制序列区间取反（本地接口示例）

题意：输入 n,q、长度 n 的 01 字符串，再输入 q 个 1-based 闭区间。每次逐位取反，最后输出字符串。保持位置顺序；要反转顺序应调用 reverse。按插入构造，保证随机优先级堆序；附件未提供完整牛客题号，仅登记本地应用。

所需模板：SequenceTreap（第 479 页，36.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://github.com/gsh20040816/CPCtemplate/issues/11

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    string s;
    cin >> n >> q >> s;
    SequenceTreap tree;
    for (int i = 0; i < n; i++) tree.insert(i, s[i] - '0');
    while (q--)
    {
        int l, r;
        cin >> l >> r;
        tree.flip_bits(l, r);
    }
    for (auto bit : tree.values()) cout << bit;
    cout << '\n';
}
```

36.2.3 动态仿射序列：插删、翻转与区间和

代码依赖：ModInt（第 133 页，19.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

AffineSequenceTreap<mod>，默认模数 998244353，正 int 模数也可为 1 或合数，不做除法。insert(k,x) 在前 k 项后插入；erase、reverse、affine、query 均用 1-based 非空闭区间，affine(l,r,b,c) 令值变为 b*x+c，乘数 0 合法。query 返回 ModInt，values 递归导出序列；二者会改变内部标记或树形，不保证并发读。新标记复合为 (b*mul,b*add+c)，不能颠倒；顺序反转与统一仿射可交换。沿用递归 split/merge，逐项构造期望 O(N log(N+1))，操作期望 O(log(n+1))、最坏 O(n)，values 为 O(n)。固定种子可复现但无对抗最坏保证。删除不回收，空间按累计插入计；节点索引、大小须在 int 内。可 reserve 但不影响正确性；辅助函数及节点数组仅供内部使用。

```
template <int mod = 998244353> struct AffineSequenceTreap
{
    static_assert(mod > 0);
    using Z = ModInt<mod>;

    struct Node
    {
        int l = 0, r = 0, siz = 0;
        Z val = 0, sum = 0, mul = 1, add = 0;
        unsigned long long pri = 0;
        bool rev = false;
    };

    vector<Node> a{Node{}};
    mt19937_64 rng;
    int root = 0;

    AffineSequenceTreap(unsigned long long seed = 712367821) : rng(seed) {}

    int size() const { return a[root].siz; }

    void pull(int p)
    {
        a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1;
        a[p].sum = a[a[p].l].sum + a[p].val + a[a[p].r].sum;
    }

    void apply(int p, Z b, Z c)
    {
        if (!p) return;
        a[p].val = b * a[p].val + c;
        a[p].sum = b * a[p].sum + c * Z(a[p].siz);
        a[p].mul = b * a[p].mul;
        a[p].add = b * a[p].add + c;
    }
```

反转、仿射下传与递归分裂（接上页同一结构体）：

```
    void flip(int p)
    {
        if (!p) return;
        swap(a[p].l, a[p].r);
        a[p].rev = !a[p].rev;
    }

    void push(int p)
    {
        if (a[p].rev)
        {
            flip(a[p].l);
            flip(a[p].r);
            a[p].rev = false;
        }
        if (a[p].mul.v != Z(1).v || a[p].add.v)
        {
            apply(a[p].l, a[p].mul, a[p].add);
            apply(a[p].r, a[p].mul, a[p].add);
            a[p].mul = 1;
            a[p].add = 0;
        }
    }

    // Split into the first k elements and the rest.
    void split(int p, int k, int &l, int &r)
    {
        if (!p)
        {
            l = r = 0;
            return;
        }
        push(p);
        int left = a[a[p].l].siz;
        if (k > left)
        {
            l = p;
            split(a[p].r, k - left - 1, a[p].r, r);
        }
        else
        {
            r = p;
            split(a[p].l, k, l, a[p].l);
        }
        pull(p);
    }
```

合并、插入与删除（接上页同一结构体）：

```
    int merge(int l, int r)
    {
        if (!l || !r) return l ? l : r;
        if (a[l].pri > a[r].pri)
        {
            push(l);
            a[l].r = merge(a[l].r, r);
            pull(l);
            return l;
        }
        push(r);
        a[r].l = merge(l, a[r].l);
        pull(r);
        return r;
    }

    // Insert after k elements; deleted nodes are not recycled.
    void insert(int k, Z value)
    {
        assert(0 <= k && k <= size());
        int p = a.size(), l, r;
        a.push_back({0, 0, 1, value, value, 1, 0, rng(), false});
        split(root, k, l, r);
        root = merge(merge(l, p), r);
    }

    // All range operations use nonempty 1-based inclusive intervals.
    void cut(int l, int r, int &x, int &y, int &z)
    {
        assert(1 <= l && l <= r && r <= size());
        split(root, r, y, z);
        split(y, l - 1, x, y);
    }

    void erase(int l, int r)
    {
        int x, y, z;
        cut(l, r, x, y, z);
        root = merge(x, z);
    }
```

区间操作与递归输出（接上页同一结构体）：

```
    void reverse(int l, int r)
    {
        int x, y, z;
        cut(l, r, x, y, z);
        flip(y);
        root = merge(merge(x, y), z);
    }

    void affine(int l, int r, Z b, Z c)
    {
        int x, y, z;
        cut(l, r, x, y, z);
        apply(y, b, c);
        root = merge(merge(x, y), z);
    }

    Z query(int l, int r)
    {
        int x, y, z;
        cut(l, r, x, y, z);
        Z ans = a[y].sum;
        root = merge(merge(x, y), z);
        return ans;
    }

    void collect(int p, vector<Z> &out)
    {
        if (!p) return;
        push(p);
        collect(a[p].l, out);
        out.push_back(a[p].val);
        collect(a[p].r, out);
    }

    vector<Z> values()
    {
        vector<Z> out;
        collect(root, out);
        return out;
    }
};
```

模板题使用：Library Checker Dynamic Sequence Range Affine Range Sum

题意：维护模 998244353 的动态序列，支持按位置插入、删除单项、顺序翻转、仿射修改和区间和。初始长度与操作数各至多 500000，乘数可以为 0。题目区间为 0-based 半开，转成模板的闭区间 [l+1,r]；插入位置不偏移，删除位置加 1。预留 N+Q+1 个节点，删除不回收。

所需模板：ModInt（第 133 页，19.2.2 节）、AffineSequenceTreap（第 484 页，36.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/dynamic_sequence_range_affine_range_sum

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    AffineSequenceTreap<> t;
    t.a.reserve(n + q + 1);
    for (int i = 0, x; i < n; i++)
    {
        cin >> x;
        t.insert(i, x);
    }
    while (q--)
    {
        int op, l, r, b, c;
        cin >> op >> l;
        if (op == 0)
        {
            cin >> b;
            t.insert(l, b);
        }
        else if (op == 1)
            t.erase(l + 1, l + 1);
        else
        {
            cin >> r;
            if (op == 2)
                t.reverse(l + 1, r);
            else if (op == 3)
            {
                cin >> b >> c;
                t.affine(l + 1, r, b, c);
            }
            else
                cout << t.query(l + 1, r).v << '\n';
        }
    }
}
// END USAGE
```

36.2.4 状态序列 Treap：分组区间 GCD

build(v) 用数值与状态组成的序列替换当前内容。数值为无符号 64 位，状态为 0 或 1。insert(k,value,tag) 在前 k 项后插入。erase(l,r)、query(l,r,tag) 使用 1-based 非空闭区间；set(k,value) 修改值，toggle(k) 翻转状态。query 返回选中状态的 GCD，无对应元素为 nullopt，实际全零则返回 0。回收删除节点，空间按峰值长度计算。建树 O(n) 次 GCD，随机优先级下单次期望 O(log n) 次 GCD；删除另需 O(删除长度) 回收。一次 GCD 为 O(log U)。覆盖 kuangbin 3.7 的序列操作，尚无该原题 OJ 记录。建树后的聚合与删除节点回收均使用递归遍历；笛卡尔树的构造单调栈保留。

```
struct GcdSequenceTreap
{
    using U = unsigned long long;

    struct Node
    {
        int l = 0, r = 0, siz = 0, tag = 0;
        int cnt[2]{};
        U val = 0, g[2]{}, pri = 0;
    };

    vector<Node> a{Node{}};
    vector<int> recycled;
    mt19937_64 rng;
    int root = 0;

    GcdSequenceTreap(U seed = 712367821) : rng(seed) {}

    int size() const { return a[root].siz; }

    int node(U value, int tag)
    {
        assert(tag == 0 || tag == 1);
        int p;
        if (recycled.empty())
        {
            p = a.size();
            a.push_back(Node{});
        }
        else
        {
            p = recycled.back();
            recycled.pop_back();
            a[p] = Node{};
        }
        a[p].siz = 1;
        a[p].val = value;
        a[p].tag = tag;
        a[p].cnt[tag] = 1;
        a[p].g[tag] = value;
        a[p].pri = rng();
        return p;
    }
```

聚合与递归分裂合并（接上页同一结构体）：

```
    void pull(int p)
    {
        int l = a[p].l, r = a[p].r;
        a[p].siz = a[l].siz + a[r].siz + 1;
        for (int t = 0; t < 2; t++)
        {
            a[p].cnt[t] = a[l].cnt[t] + a[r].cnt[t] + (a[p].tag == t);
            a[p].g[t] = gcd(a[l].g[t], a[r].g[t]);
            if (a[p].tag == t) a[p].g[t] = gcd(a[p].g[t], a[p].val);
        }
    }

    void split(int p, int k, int &l, int &r)
    {
        if (!p)
        {
            l = 0;
            r = 0;
            return;
        }
        int left = a[a[p].l].siz;
        if (k > left)
        {
            l = p;
            split(a[p].r, k - left - 1, a[p].r, r);
        }
        else
        {
            r = p;
            split(a[p].l, k, l, a[p].l);
        }
        pull(p);
    }

    int merge(int l, int r)
    {
        if (!l || !r) return l ? l : r;
        if (a[l].pri > a[r].pri)
        {
            a[l].r = merge(a[l].r, r);
            pull(l);
            return l;
        }
        a[r].l = merge(l, a[r].l);
        pull(r);
        return r;
    }
```

构建、重算与节点回收（接上页同一结构体）：

```
    void rebuild(int p)
    {
        if (!p) return;
        rebuild(a[p].l);
        rebuild(a[p].r);
        pull(p);
    }

    void release(int p)
    {
        if (!p) return;
        release(a[p].l);
        release(a[p].r);
        recycled.push_back(p);
    }

    // O(n) GCD calls; replaces the current sequence.
    void build(const vector<pair<U, int>> &v)
    {
        a.assign(1, Node{});
        recycled.clear();
        root = 0;
        vector<int> st;
        for (auto [value, tag] : v)
        {
            int p = node(value, tag), last = 0;
            while (!st.empty() && a[st.back()].pri < a[p].pri)
            {
                last = st.back();
                st.pop_back();
            }
            a[p].l = last;
            if (!st.empty()) a[st.back()].r = p;
            st.push_back(p);
        }
        if (st.empty()) return;
        root = st.front();
        rebuild(root);
    }
```

插删、单点修改与分组查询（接上页同一结构体）：

```
    // Insert after the first k entries. Range operations below are 1-based.
    void insert(int k, U value, int tag)
    {
        assert(0 <= k && k <= size());
        int p = node(value, tag), l, r;
        split(root, k, l, r);
        root = merge(merge(l, p), r);
    }

    void erase(int l, int r)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        root = merge(x, z);
        release(y);
    }

    void set(int k, U value)
    {
        assert(1 <= k && k <= size());
        int x, y, z;
        split(root, k, y, z);
        split(y, k - 1, x, y);
        a[y].val = value;
        pull(y);
        root = merge(merge(x, y), z);
    }

    void toggle(int k)
    {
        assert(1 <= k && k <= size());
        int x, y, z;
        split(root, k, y, z);
        split(y, k - 1, x, y);
        a[y].tag ^= 1;
        pull(y);
        root = merge(merge(x, y), z);
    }

    optional<U> query(int l, int r, int tag)
    {
        assert(1 <= l && l <= r && r <= size());
        assert(tag == 0 || tag == 1);
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        optional<U> answer;
        if (a[y].cnt[tag]) answer = a[y].g[tag];
        root = merge(merge(x, y), z);
        return answer;
    }
};
```

接口用法演示：自定义带标签 GCD 序列操作

演示约定：自定义协议，非 OJ 题；n≤200000、q≤100000 均可为 0，是本演示约定。先读 n 组 (value,tag)，值为非负 unsigned long long，标签 0 或 1。I k v t 在前 k 项后插入；E l r 删 1-based 闭区间；S k v 改单值；T k 翻单标签；Q l r t 查询该标签 GCD，无该标签输出 NONE，全零输出 0。位置必须合法，可删空再插入，不支持翻转或负值。

所需模板：GcdSequenceTreap（第 489 页，36.2.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/src/compact/gcd_sequence.hpp

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    vector<pair<GcdSequenceTreap::U, int>> a(n);
    for (auto &[v, t] : a) cin >> v >> t;
    GcdSequenceTreap tr;
    tr.build(a);
    while (q--)
    {
        char op;
        int k, l, r, t;
        GcdSequenceTreap::U v;
        cin >> op;
        if (op == 'I')
        {
            cin >> k >> v >> t;
            tr.insert(k, v, t);
        }
        else if (op == 'E')
        {
            cin >> l >> r;
            tr.erase(l, r);
        }
        else if (op == 'S')
        {
            cin >> k >> v;
            tr.set(k, v);
        }
        else if (op == 'T')
        {
            cin >> k;
            tr.toggle(k);
        }
        else if (op == 'Q')
        {
            cin >> l >> r >> t;
            auto ans = tr.query(l, r, t);
            if (ans) cout << *ans << '\n';
            else cout << "NONE\n";
        }
    }
}
```

相关应用：按位置分裂拼接与最大值剪枝，不代表整页覆盖

36.2.5 序列最大值 Treap

空构造，外部根变量 0 为空；同一对象可管理多棵互不相交的序列。insert(root,pos,value) 在 0-based 位置 pos 前插入，pos 可等于长度，通过引用更新根，返回按插入顺序递增的稳定节点 ID。split(root,k) 消耗旧根，返回前 k 项及其余项；merge(left,right) 消耗两根并按顺序拼接，无值域大小限制，但两根必须互不相交且属于同一对象。last_greater(root,value) 返回最右严格大于 value 的 0-based 位置，无结果为-1；后缀中 <=value 的个数等于 size(root)-1-last_greater(root,value)。ids(root) 按序返回稳定 ID。

插入、分裂、拼接和查询期望 O(log(N+1))，随机优先级下单次最坏 O(N)；输出 O(N)。空间 O(N)，节点不回收，递归最坏需 O(N) 栈。value 允许任意 signed64 及重复值，不作值加减。节点总数须小于 INT_MAX。消耗的根不可再作为独立树使用；root 引用必须为外部变量，不能引用 vector 内字段。t、rng 及辅助函数仅供内部实现。CF38G 使用互异正整数，重复值扩展由独立测试覆盖；当前无线上判定。

```
struct SequenceMaxTreap
{
    using ll = long long;
    struct Node
    {
        int left = 0, right = 0, size = 0;
        ll value = LLONG_MIN, maximum = LLONG_MIN;
        unsigned long long priority = 0;
    };
    vector<Node> t{Node{}};
    mt19937_64 rng;

    SequenceMaxTreap(unsigned long long seed = 712367821) : rng(seed)
    {
    }

    int size(int root) const
    {
        return t[root].size;
    }

    void pull(int p)
    {
        t[p].size = size(t[p].left) + size(t[p].right) + 1;
        t[p].maximum = max({t[p].value, t[t[p].left].maximum,
                            t[t[p].right].maximum});
    }

    pair<int, int> split(int p, int k)
    {
        assert(0 <= k && k <= size(p));
        if (!p)
            return {0, 0};
        if (k <= size(t[p].left))
        {
            auto [left, right] = split(t[p].left, k);
            t[p].left = right;
            pull(p);
            return {left, p};
        }
        auto [left, right] = split(t[p].right, k - size(t[p].left) - 1);
```

```
        t[p].right = left;
        pull(p);
        return {p, right};
    }

    int merge(int left, int right)
    {
        if (!left || !right)
            return left ? left : right;
        if (t[left].priority > t[right].priority)
        {
            t[left].right = merge(t[left].right, right);
            pull(left);
            return left;
        }
        t[right].left = merge(left, t[right].left);
        pull(right);
        return right;
    }

    int insert(int &root, int pos, ll value)
    {
        assert(0 <= pos && pos <= size(root) && t.size() < INT_MAX);
        int id = t.size();
        t.push_back({0, 0, 1, value, value, rng()});
        auto [left, right] = split(root, pos);
        root = merge(merge(left, id), right);
        return id;
    }

    int last_greater(int p, ll value) const
    {
        if (!p || t[p].maximum <= value)
            return -1;
        int right = last_greater(t[p].right, value);
        if (right != -1)
            return size(t[p].left) + 1 + right;
        if (t[p].value > value)
            return size(t[p].left);
        return last_greater(t[p].left, value);
    }

    void collect(int p, vector<int> &order) const
    {
        if (!p)
            return;
        collect(t[p].left, order);
        order.push_back(p);
```

```
        collect(t[p].right, order);
    }

    vector<int> ids(int root) const
    {
        vector<int> order;
        collect(root, order);
        return order;
    }
};
```

模板题使用：Codeforces 38G Queue

题意：n<=100000；重要度是 1..n 的排列，0<=c_i<=n。每人先入队尾，最多 c_i 次与重要度更低的前一个人交换，输出最终输入编号。最右重要度更大的位置是不可越过的屏障，插入位置 max(i-c_i,barrier+1)。期望 O(nlogn)、空间 O(n)。三组题面样例、独立逐次交换及最大规模边界通过；尚无线上 AC。

所需模板：SequenceMaxTreap（第 494 页，36.2.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/38/G

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    SequenceMaxTreap tree;
    int root = 0;
    for (int i = 0; i < n; i++)
    {
        int importance, limit;
        cin >> importance >> limit;
        int barrier = tree.last_greater(root, importance);
        int pos = max(i - limit, barrier + 1);
        tree.insert(root, pos, importance);
    }
    auto order = tree.ids(root);
    for (int i = 0; i < n; i++)
        cout << order[i] << (i + 1 == n ? '\n' : ' ');
}
```

36.3 Splay 树

36.3.1 Splay：可重集合与顺序统计

insert/erase/rank/kth/prev/next 与有序 Treap 同义，严格前驱后继不存在返回 nullopt。重复键计数存储；失败查询也伸展最后访问节点，单次均摊 O(log n)。不回收节点，空间按累计新建节点数计算。支持 long long 全范围；不包含序列 Splay、虚子树信息或矩阵动态 DP。

```
struct OrderedSplay
{
    using ll = long long;

    struct Node
    {
        int ch[2]{}, fa = 0, cnt = 0, siz = 0;
        ll val = 0;
    };

    vector<Node> a{Node{}};
    int root = 0;

    int size() const { return a[root].siz; }

    void pull(int x) { a[x].siz = a[a[x].ch[0]].siz + a[x].cnt + a[a[x].ch[1]].siz; }

    int node(ll value, int parent)
    {
        int x = a.size();
        a.push_back({{0, 0}, parent, 1, 1, value});
        return x;
    }

    void rotate(int x)
    {
        int y = a[x].fa, z = a[y].fa;
        int k = a[y].ch[1] == x;
        int w = a[x].ch[k ^ 1];
        if (z)
            a[z].ch[a[z].ch[1] == y] = x;
        else
            root = x;
        a[x].fa = z;
        a[y].ch[k] = w;
        if (w) a[w].fa = y;
        a[x].ch[k ^ 1] = y;
        a[y].fa = x;
        pull(y);
        pull(x);
    }
```

伸展与查找（接上页同一结构体）：

```
    void splay(int x)
    {
        while (a[x].fa)
        {
            int y = a[x].fa, z = a[y].fa;
            if (z)
            {
                bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
                rotate(same ? y : x);
            }
            rotate(x);
        }
    }

    // Splay the last visited node even when the value does not exist.
    int find(ll value)
    {
        int x = root, last = 0;
        while (x && a[x].val != value)
        {
            last = x;
            x = a[x].ch[value > a[x].val];
        }
        if (x)
            splay(x);
        else if (last)
            splay(last);
        return x;
    }
```

插入与删除（接上页同一结构体）：

```
    void insert(ll value)
    {
        int x = root, parent = 0;
        while (x && a[x].val != value)
        {
            parent = x;
            x = a[x].ch[value > a[x].val];
        }
        if (x)
        {
            ++a[x].cnt;
            pull(x);
        }
        else
        {
            x = node(value, parent);
            if (parent)
                a[parent].ch[value > a[parent].val] = x;
            else
                root = x;
        }
        splay(x);
    }

    bool erase(ll value)
    {
        int x = find(value);
        if (!x) return false;
        if (a[x].cnt > 1)
        {
            --a[x].cnt;
            pull(x);
            return true;
        }
        int l = a[x].ch[0], r = a[x].ch[1];
        if (l) a[l].fa = 0;
        if (r) a[r].fa = 0;
        if (!l)
        {
            root = r;
            return true;
        }
        root = l;
        x = l;
        while (a[x].ch[1]) x = a[x].ch[1];
        splay(x);
        a[x].ch[1] = r;
        if (r) a[r].fa = x;
        pull(x);
        return true;
    }
```

排名与第 k 小（接上页同一结构体）：

```
    int rank(ll value)
    {
        int x = root, last = 0, ans = 1;
        while (x)
        {
            last = x;
            if (value <= a[x].val)
                x = a[x].ch[0];
            else
            {
                ans += a[a[x].ch[0]].siz + a[x].cnt;
                x = a[x].ch[1];
            }
        }
        if (last) splay(last);
        return ans;
    }

    ll kth(int k)
    {
        assert(1 <= k && k <= size());
        int x = root;
        while (true)
        {
            int left = a[a[x].ch[0]].siz;
            if (k <= left)
                x = a[x].ch[0];
            else if (k <= left + a[x].cnt)
            {
                splay(x);
                return a[x].val;
            }
            else
            {
                k -= left + a[x].cnt;
                x = a[x].ch[1];
            }
        }
    }
```

严格前驱与后继（接上页同一结构体）：

```
    optional<ll> prev(ll value)
    {
        int x = root, last = 0, answer = 0;
        while (x)
        {
            last = x;
            if (a[x].val < value)
            {
                answer = x;
                x = a[x].ch[1];
            }
            else
                x = a[x].ch[0];
        }
        if (last) splay(last);
        if (!answer) return nullopt;
        splay(answer);
        return a[answer].val;
    }

    optional<ll> next(ll value)
    {
        int x = root, last = 0, answer = 0;
        while (x)
        {
            last = x;
            if (a[x].val > value)
            {
                answer = x;
                x = a[x].ch[0];
            }
            else
                x = a[x].ch[1];
        }
        if (last) splay(last);
        if (!answer) return nullopt;
        splay(answer);
        return a[answer].val;
    }
};
```

模板题使用：Luogu P3369

题意：维护可重复整数集合，支持插入、删除一个值、排名、第 k 小、严格前驱和严格后继。题目保证前驱/后继存在，因此可解引用返回值。

所需模板：OrderedSplay（第 498 页，36.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    OrderedSplay g;
    while (n--)
    {
        int op;
        long long x;
        cin >> op >> x;
        if (op == 1)
            g.insert(x);
        else if (op == 2)
            g.erase(x);
        else if (op == 3)
            cout << g.rank(x) << '\n';
        else if (op == 4)
            cout << g.kth(x) << '\n';
        else if (op == 5)
            cout << *g.prev(x) << '\n';
        else
            cout << *g.next(x) << '\n';
    }
}
```

36.3.2 可合并 Splay：固定编号与集合顺序统计

MergeSplay(v) 为 0-based 元素建立独立单点集合，v 为 vector<long long>，允许空数组，元素数 <INT_MAX 且内存足够。merge(u,v) 合并两个元素所在集合，已经同集合返回 false，否则 true；same(u,v) 判同集合。set(id,val) 改该元素的值而不改编号或集合；value(id) 读值，size(id) 查集合大小。rank(id) 是集合内按 (值, 原编号) 排序的 1-based 名次；kth(id,k) 返回所在集合第 k 小的原编号 optional<int>，k 为 long long，非法时 nullopt，读值用 value(*ans)。重复值保留独立元素，signed64 极值只比较不加减。

初建 O(n)、空间 O(n)，始终仅 n+1 个节点槽；无独立插删、分裂或删除元素接口。合并递归遍历较小集合，将原节点逐个插入较大集合，不用显式栈；递归深度最坏 O(n)。从 n 个单点集合出发，全部成功合并的总成本 $O(n \log^2(n+1))$，其余操作除 value 外均摊 O(log(n+1))，value 为 O(1)，单次非合并操作最坏 O(n)；合并复杂度按前述总界计。节点池与辅助函数内部使用；公开查询会伸展树，根不稳定但原编号稳定。支持独立值拷贝，重建用新对象赋值。

```
struct MergeSplay
{
    using ll = long long;
    struct Node
    {
        int ch[2]{}, fa = 0, siz = 1;
        ll val = 0;
    };
    int n;
    vector<Node> a;

    // Each original ID starts in its own singleton set.
    MergeSplay(const vector<ll> &v)
    {
        assert(v.size() < INT_MAX);
        n = v.size();
        a.resize(n + 1);
        a[0].siz = 0;
        for (int i = 0; i < n; i++) a[i + 1].val = v[i];
    }

    void pull(int x)
    {
        a[x].siz = a[a[x].ch[0]].siz + a[a[x].ch[1]].siz + 1;
    }

    bool less(int x, int y) const
    {
        return pair{a[x].val, x} < pair{a[y].val, y};
    }
```

旋转、伸展与固定编号（接上页同一结构体）：

```
    void rotate(int x)
    {
        int y = a[x].fa, z = a[y].fa;
        int k = a[y].ch[1] == x, w = a[x].ch[k ^ 1];
        if (z) a[z].ch[a[z].ch[1] == y] = x;
        a[x].fa = z;
        a[y].ch[k] = w;
        if (w) a[w].fa = y;
        a[x].ch[k ^ 1] = y;
        a[y].fa = x;
        pull(y);
        pull(x);
    }

    void splay(int x)
    {
        while (a[x].fa)
        {
            int y = a[x].fa, z = a[y].fa;
            if (z)
            {
                bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
                rotate(same ? y : x);
            }
            rotate(x);
        }
    }

    int expose(int id)
    {
        assert(0 <= id && id < n);
        int x = id + 1;
        splay(x);
        return x;
    }
```

节点插入与小集合合并（接上页同一结构体）：

```
    int insert(int root, int x)
    {
        if (!root) return x;
        int u = root;
        while (true)
        {
            int k = less(u, x);
            if (!a[u].ch[k])
            {
                a[u].ch[k] = x;
                a[x].fa = u;
                break;
            }
            u = a[u].ch[k];
        }
        splay(x);
        return x;
    }

    void transfer(int u, int &root)
    {
        if (!u) return;
        int l = a[u].ch[0], r = a[u].ch[1];
        a[u] = Node{{0, 0}, 0, 1, a[u].val};
        root = insert(root, u);
        transfer(l, root);
        transfer(r, root);
    }

    bool same(int u, int v)
    {
        int x = expose(u), y = expose(v);
        return x == y || a[x].fa != 0;
    }

    bool merge(int u, int v)
    {
        int x = expose(u), y = expose(v);
        if (x == y || a[x].fa) return false;
        if (a[x].siz < a[y].siz) swap(x, y);
        transfer(y, x);
        return true;
    }

    int size(int id)
    {
        return a[expose(id)].siz;
    }

    ll value(int id) const
    {
        assert(0 <= id && id < n);
        return a[id + 1].val;
    }
```

保持编号的改值、名次与第 k 小（接上页同一结构体）：

```
    // Change this element's value; its ID and set membership do not change.
    void set(int id, ll val)
    {
        int x = expose(id);
        int l = a[x].ch[0], r = a[x].ch[1];
        if (l) a[l].fa = 0;
        if (r) a[r].fa = 0;
        int root = r;
        if (l)
        {
            root = l;
            while (a[root].ch[1]) root = a[root].ch[1];
            splay(root);
            a[root].ch[1] = r;
            if (r) a[r].fa = root;
            pull(root);
        }
        a[x] = Node{{0, 0}, 0, 1, val};
        insert(root, x);
    }

    // 1-based rank among (value, original ID) in this element's set.
    int rank(int id)
    {
        int x = expose(id);
        return a[a[x].ch[0]].siz + 1;
    }

    // Return an original ID, not its value. Invalid k returns nullopt.
    optional<int> kth(int id, ll k)
    {
        int x = expose(id);
        if (k <= 0 || k > a[x].siz) return nullopt;
        while (true)
        {
            int l = a[a[x].ch[0]].siz;
            if (k == l + 1)
            {
                splay(x);
                return x - 1;
            }
            if (k <= l) x = a[x].ch[0];
            else
            {
                k -= l + 1;
                x = a[x].ch[1];
            }
        }
    }
};
```

应用题补充：UVA1479 / 可合并 Splay 与离线图查询

题意：D e 删边、C x v 改点权、Q x k 查连通块第 k 大，非法 k 贡献 0 且计入均值；E 结束组，0 0 结束输入。读入 C 时保存旧值，最终图按未删除边合并，倒序将 D 还原为 merge、C 还原为 set。kth 返回元素编号，value 取权值；第 k 大换算先转 long long，避免 INT_MIN 溢出。无需额外 dsu 和离散化；输出各组查询均值六位小数。

所需模板：MergeSplay（第 504 页，36.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.org/external/14/1479.pdf

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    struct Op
    {
        char op;
        int x, y;
    };
    int n, m, tc = 0;
    while (cin >> n >> m && n)
    {
        vector<long long> a(n);
        for (auto &x : a) cin >> x;
        vector<pair<int, int>> edges(m);
        for (auto &[u, v] : edges)
        {
            cin >> u >> v;
            u--;
            v--;
        }
        vector<int> deleted(m);
        vector<Op> ops;
        char op;
        while (cin >> op && op != 'E')
        {
            int x, y = 0;
            cin >> x;
            x--;
            if (op == 'D') deleted[x] = 1;
            else cin >> y;
            if (op == 'C')
            {
                int old = a[x];
                a[x] = y;
                y = old;
            }
            ops.push_back({op, x, y});
        }
```

继续处理操作（接上页同一 main）：

```
        MergeSplay t(a);
        auto join = [&](int e)
        {
            auto [u, v] = edges[e];
            t.merge(u, v);
        };
        for (int e = 0; e < m; e++)
            if (!deleted[e]) join(e);
        long long sum = 0;
        int count = 0;
        for (int i = (int)ops.size() - 1; i >= 0; i--)
        {
            auto [op, x, y] = ops[i];
            if (op == 'D') join(x);
            else if (op == 'C')
            {
                t.set(x, y);
            }
            else
            {
                long long k = (long long)t.size(x) - y + 1;
                auto id = t.kth(x, k);
                if (id) sum += t.value(*id);
                count++;
            }
        }
        cout << "Case " << ++tc << ": " << fixed << setprecision(6)
             << (long double)sum / count << '\n';
    }
}
```

应用题补充：Luogu P3224 / 永无乡

题意：n 个岛的重要度为 1..n 排列，初始 m 条桥；B x y 连桥，Q x k 输出连通块中第 k 小重要度的岛编号，不存在输出 −1。merge 以元素原编号为入口，kth 结果加 1 转换为题目编号；不能误输出重要度。n,m≤100000，q≤300000。

所需模板：MergeSplay（第 504 页，36.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3224

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    MergeSplay t(a);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        t.merge(u - 1, v - 1);
    }
    int q;
    cin >> q;
    while (q--)
    {
        char op;
        int x, y;
        cin >> op >> x >> y;
        if (op == 'B') t.merge(x - 1, y - 1);
        else
        {
            auto id = t.kth(x - 1, y);
            cout << (id ? *id + 1 : -1) << '\n';
        }
    }
}
```

接口用法演示：可合并有序集合与元素改值（接口演示）

演示约定：输入 n,q 及 n 个 signed64 值，初始各为单点集合，编号 0-based。0 x y 合并集合并输出是否改变；1 x v 改值；2 x k 输出第 k 小元素的原编号和值，非法 k 输出 −1；3 x 输出集合大小与 x 的 1-based 名次。并列按原编号升序；合并与修改之后编号仍有效。

所需模板：MergeSplay（第 504 页，36.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/ds/splay/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    MergeSplay t(a);
    while (q--)
    {
        int op, x;
        long long y;
        cin >> op >> x;
        if (op <= 2) cin >> y;
        if (op == 0) cout << t.merge(x, y) << '\n';
        else if (op == 1) t.set(x, y);
        else if (op == 2)
        {
            auto id = t.kth(x, y);
            if (id) cout << *id << ' ' << t.value(*id) << '\n';
            else cout << -1 << '\n';
        }
        else cout << t.size(x) << ' ' << t.rank(x) << '\n';
    }
}
```

相关应用：来源专用值域平移/历史计数权值索引，不代表整页已覆盖

36.3.3 按值分裂与整体平移 Splay

空构造，根 0 为空；同一对象内可管理多棵互不相交的树。insert(root,value) 通过引用更新根，返回稳定的 1-based 节点 ID；重复值允许。split(root,key) 消耗旧根，返回值 <key 与 >=key 两棵根。add(root,delta) 全树平移。merge(left,right) 消耗两根，要求左树所有值 <= 右树所有值；不是任意集合合并。value(root,id) 读取本树节点并通过引用更新根，节点 ID 稳定，根会变化。所有根和 ID 必须属于同一对象，id 必须属于给定树；消耗后的旧根不能再当独立树使用。

insert/split/merge/value 均摊 O(log(N+1))，单次最坏 O(N)，add 为 O(1)，空间 O(N)，不回收节点。所有实际值及 val/lazy 每次加法中间结果须适配 signed64；最终值合法不能代替懒标记不溢出的要求。祖先下传递归，需 O(N) 栈空间。对象复制独立，辅助函数与 a 仅供内部实现。

```
struct ShiftSplay
{
    using ll = long long;
    struct Node
    {
        int ch[2]{}, fa = 0;
        ll val = 0, lazy = 0;
    };
    vector<Node> a{Node{}};

    void add(int root, ll delta)
    {
        if (!root)
            return;
        a[root].val += delta;
        a[root].lazy += delta;
    }

    void push(int x)
    {
        add(a[x].ch[0], a[x].lazy);
        add(a[x].ch[1], a[x].lazy);
        a[x].lazy = 0;
    }

    void down(int x)
    {
        if (a[x].fa)
            down(a[x].fa);
        push(x);
    }
```

接上页同一结构体：

```
    void rotate(int x)
    {
        int y = a[x].fa, z = a[y].fa;
        int k = a[y].ch[1] == x;
        int w = a[x].ch[k ^ 1];
        if (z)
            a[z].ch[a[z].ch[1] == y] = x;
        a[x].fa = z;
        a[y].ch[k] = w;
        if (w)
            a[w].fa = y;
        a[x].ch[k ^ 1] = y;
        a[y].fa = x;
    }

    void splay(int x)
    {
        down(x);
        while (a[x].fa)
        {
            int y = a[x].fa, z = a[y].fa;
            if (z)
            {
                bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
                rotate(same ? y : x);
            }
            rotate(x);
        }
    }

    int insert(int &root, ll value)
    {
        assert(a.size() < INT_MAX);
        int x = a.size();
        a.push_back(Node{});
        a[x].val = value;
        int p = root, parent = 0, side = 0;
        while (p)
        {
            push(p);
            parent = p;
            side = value >= a[p].val;
            p = a[p].ch[side];
        }
        a[x].fa = parent;
        if (parent)
            a[parent].ch[side] = x;
        splay(x);
        root = x;
        return x;
    }
```

接上页同一结构体：

```
    pair<int, int> split(int root, ll key)
    {
        if (!root)
            return {0, 0};
        int p = root, last = root, next = 0;
        while (p)
        {
            push(p);
            last = p;
            if (a[p].val >= key)
            {
                next = p;
                p = a[p].ch[0];
            }
            else
                p = a[p].ch[1];
        }
        splay(last);
        if (!next)
            return {last, 0};
        splay(next);
        int left = a[next].ch[0];
        a[next].ch[0] = 0;
        if (left)
            a[left].fa = 0;
        return {left, next};
    }

    int merge(int left, int right)
    {
        if (!left || !right)
            return left ? left : right;
        int x = left;
        while (a[x].ch[1])
            x = a[x].ch[1];
        splay(x);
        a[x].ch[1] = right;
        a[right].fa = x;
        return x;
    }

    ll value(int &root, int id)
    {
        assert(0 < id && id < (int)a.size());
        splay(id);
        root = id;
        return a[id].val;
    }
};
```

接口用法演示：按值分裂与整体平移 Splay API

演示约定：输入 q，初始树槽 0 为空。1 r v 插入并输出节点 ID；2 r key 把槽 r 保留 <key 部分，新建槽存 >=key 部分并输出槽号；3 r delta 全树平移；4 r s 有序合并到 r 并清空 s（r!=s，且左侧所有值 <= 右侧）；5 r id 输出该槽节点当前值。槽号 0-based，ID 从 1 递增，空槽可复用。仅合法操作，所有中间加法需适配 signed64。本地 API 例子，不是 OJ 判定。

所需模板：ShiftSplay（第 512 页，36.3.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/WIDA-SHIFT-SPLAY.md

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    ShiftSplay tree;
    vector<int> roots(1);
    while (q--)
    {
        int op, r;
        long long x;
        cin >> op >> r >> x;
        if (op == 1)
        {
            cout << tree.insert(roots[r], x) << '\n';
        }
        else if (op == 2)
        {
            auto [left, right] = tree.split(roots[r], x);
            roots[r] = left;
            roots.push_back(right);
            cout << roots.size() - 1 << '\n';
        }
        else if (op == 3)
        {
            tree.add(roots[r], x);
        }
        else if (op == 4)
        {
            int s = x;
            roots[r] = tree.merge(roots[r], roots[s]);
            roots[s] = 0;
        }
        else
        {
            int id = x;
            cout << tree.value(roots[r], id) << '\n';
        }
    }
}
```

36.3.4 区间翻转 Splay 与固定编号定位

SequenceSplay(n) 建立原编号 0..n−1 的序列，允许 n=0，要求 n≤INT_MAX−3 且内存足够。reverse(l,r) 翻转当前 0-based 半开区间 [l,r)，允许空区间；pos(id) 返回原编号 id 的当前位置；at(k) 返回当前位置 k 的原编号；order() 按当前顺序返回全部原编号。值由调用方按原编号存储，重复值不会合并。

初建 O(n)，pos/at/reverse 均摊 O(log(n+1))、单次最坏 O(n)，order 为 O(n)，空间 O(n)；不含插入、删除和数值聚合。所有辅助函数与 a/root/n 约定内部只读，pos/at 会伸展并改变树形。两哨兵不暴露给用户；节点编号不随旋转变化。build/祖先下传/遍历均递归，退化时深度 O(n)，调用方保证栈空间。支持独立值拷贝；重置用新建对象赋值。

```
struct SequenceSplay
{
    struct Node
    {
        int ch[2]{}, fa = 0, siz = 0;
        bool rev = false;
    };
    int n, root;
    vector<Node> a;

    // Initial order: original IDs 0, 1, ..., n-1.
    SequenceSplay(int n) : n(n)
    {
        assert(0 <= n && n <= INT_MAX - 3);
        a.resize(n + 3);
        root = build(1, n + 3, 0);
    }

    void pull(int x)
    {
        a[x].siz = a[a[x].ch[0]].siz + a[a[x].ch[1]].siz + 1;
    }

    int build(int l, int r, int p)
    {
        if (l == r) return 0;
        int x = l + (r - l) / 2;
        a[x].fa = p;
        a[x].ch[0] = build(l, x, x);
        a[x].ch[1] = build(x + 1, r, x);
        pull(x);
        return x;
    }
```

翻转下传与伸展旋转（接上页同一结构体）：

```
    void flip(int x)
    {
        if (!x) return;
        swap(a[x].ch[0], a[x].ch[1]);
        a[x].rev = !a[x].rev;
    }

    void push(int x)
    {
        if (!a[x].rev) return;
        flip(a[x].ch[0]);
        flip(a[x].ch[1]);
        a[x].rev = false;
    }

    void push_path(int x)
    {
        if (a[x].fa) push_path(a[x].fa);
        push(x);
    }

    void rotate(int x)
    {
        int y = a[x].fa, z = a[y].fa;
        int k = a[y].ch[1] == x, w = a[x].ch[k ^ 1];
        if (z) a[z].ch[a[z].ch[1] == y] = x;
        else root = x;
        a[x].fa = z;
        a[y].ch[k] = w;
        if (w) a[w].fa = y;
        a[x].ch[k ^ 1] = y;
        a[y].fa = x;
        pull(y);
        pull(x);
    }

    void splay(int x, int goal = 0)
    {
        push_path(x);
        while (a[x].fa != goal)
        {
            int y = a[x].fa, z = a[y].fa;
            if (z != goal)
            {
                bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
                rotate(same ? y : x);
            }
            rotate(x);
        }
    }
```

位置和原编号互查（接上页同一结构体）：

```
    // Internal rank includes the two sentinels; does not splay.
    int select(int k)
    {
        int x = root;
        while (true)
        {
            push(x);
            int l = a[a[x].ch[0]].siz;
            if (k == l) return x;
            if (k < l) x = a[x].ch[0];
            else
            {
                k -= l + 1;
                x = a[x].ch[1];
            }
        }
    }

    // Current 0-based position of an original ID.
    int pos(int id)
    {
        assert(0 <= id && id < n);
        int x = id + 2;
        splay(x);
        return a[a[x].ch[0]].siz - 1;
    }

    // Original ID at the current 0-based position.
    int at(int k)
    {
        assert(0 <= k && k < n);
        int x = select(k + 1);
        splay(x);
        return x - 2;
    }
```

区间翻转与完整顺序（接上页同一结构体）：

```
    void reverse(int l, int r)
    {
        assert(0 <= l && l <= r && r <= n);
        if (l == r) return;
        int x = select(l);
        splay(x);
        int y = select(r + 1);
        splay(y, x);
        flip(a[y].ch[0]);
    }

    void collect(int x, vector<int> &v)
    {
        if (!x) return;
        push(x);
        collect(a[x].ch[0], v);
        if (2 <= x && x <= n + 1) v.push_back(x - 2);
        collect(a[x].ch[1], v);
    }

    vector<int> order()
    {
        vector<int> v;
        v.reserve(n);
        collect(root, v);
        return v;
    }
};
```

模板题使用：Luogu P3391 / Splay 区间翻转

题意：初始序列 1..n，m 次翻转 1-based 闭区间 [l,r]，输出最终序列；转为 reverse(l−1,r)，order() 中的原编号加 1。n,m≤100000。

所需模板：SequenceSplay（第 516 页，36.3.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3391

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    SequenceSplay t(n);
    while (m--)
    {
        int l, r;
        cin >> l >> r;
        t.reverse(l - 1, r);
    }
    auto a = t.order();
    for (int i = 0; i < n; i++)
        cout << a[i] + 1 << (i + 1 == n ? '\n' : ' ');
}
```

应用题补充：Luogu P3165 / 排序机械臂

题意：按 (值, 原编号) 排序确定稳定目标次序，第 i 步定位该元素当前 0-based 位置 p，输出 p+1，再翻转 [i,p+1)。重复高度按原编号打破并列，不能按翻转后的次序重新判定。n≤100000；与 kuangbin3.4.1 的固定编号定位、反转操作模型对应。

所需模板：SequenceSplay（第 516 页，36.3.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3165

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> a(n), id(n);
    for (auto &x : a) cin >> x;
    iota(id.begin(), id.end(), 0);
    sort(id.begin(), id.end(), [&](int u, int v)
    {
        return pair{a[u], u} < pair{a[v], v};
    });
    SequenceSplay t(n);
    for (int i = 0; i < n; i++)
    {
        int p = t.pos(id[i]);
        cout << p + 1 << (i + 1 == n ? '\n' : ' ');
        t.reverse(i, p + 1);
    }
}
```

接口用法演示：原编号与当前位置互查（接口演示）

演示约定：输入 n,q，初始编号 0..n−1；0 l r 翻转半开区间；1 id 查原编号的当前位置；2 k 查当前位置的原编号；3 输出完整顺序。允许空序列与空翻转；空序列不允许 pos/at。

所需模板：SequenceSplay（第 516 页，36.3.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/ds/splay/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    SequenceSplay t(n);
    while (q--)
    {
        int op, x, y;
        cin >> op;
        if (op == 0)
        {
            cin >> x >> y;
            t.reverse(x, y);
        }
        else if (op == 1)
        {
            cin >> x;
            cout << t.pos(x) << '\n';
        }
        else if (op == 2)
        {
            cin >> x;
            cout << t.at(x) << '\n';
        }
        else
        {
            auto a = t.order();
            for (int i = 0; i < n; i++)
                cout << a[i] << (i + 1 == n ? '\n' : ' ');
            if (n == 0) cout << '\n';
        }
    }
}
```

36.4 替罪羊树

36.4.1 替罪羊树：可重集合与顺序统计

默认空树；insert(x) 插入一个副本，erase(x) 删除一个副本并返回是否存在。less(x) 计严格小于 x 的元素，less(x,true) 计小于等于；rank(x)=less(x)+1，x 不必存在。kth(k) 要求 1≤k≤size()。prev/next 返回严格前驱/后继，不存在为 nullopt。键支持完整 long long，不使用数值哨兵。节点数过度偏向单侧或有效不同值不足一半时递归重建，复用活节点编号。若 d 为有效不同值数，查询 O(log(d+2))，更新摊 O(log(d+2))、单次最坏 O(d+1)。元素数与累计分配节点数均须小于 INT_MAX。内存 O(K)，K 为累计分配节点数；墓碑从树中清除后不回收到节点池，重建本身不增分配量。a、root 和辅助接口只供内部使用；重新赋值 ScapegoatTree() 可清空。

```
struct ScapegoatTree
{
    using ll = long long;

    struct Node
    {
        int l = 0, r = 0, count = 0, size = 0, nodes = 0, live = 0;
        ll value = 0;
    };

    vector<Node> a{Node{}};
    int root = 0;

    int size() const { return a[root].size; }

    void pull(int p)
    {
        int l = a[p].l, r = a[p].r;
        a[p].size = a[l].size + a[r].size + a[p].count;
        a[p].nodes = a[l].nodes + a[r].nodes + 1;
        a[p].live = a[l].live + a[r].live + (a[p].count > 0);
    }

    void collect(int p, vector<int> &v)
    {
        if (!p) return;
        collect(a[p].l, v);
        if (a[p].count) v.push_back(p);
        collect(a[p].r, v);
    }

    int build(const vector<int> &v, int l, int r)
    {
        if (l == r) return 0;
        int m = l + (r - l) / 2, p = v[m];
        a[p].l = build(v, l, m);
        a[p].r = build(v, m + 1, r);
        pull(p);
        return p;
    }
```

重建与插入删除（接上页同一结构体）：

```
    int balance(int p)
    {
        pull(p);
        int heavy = max(a[a[p].l].nodes, a[a[p].r].nodes);
        if (4LL * heavy <= 3LL * a[p].nodes && 2LL * a[p].live >= a[p].nodes) return p;
        vector<int> v;
        v.reserve(a[p].live);
        collect(p, v);
        return build(v, 0, v.size());
    }

    int insert(int p, ll x)
    {
        if (!p)
        {
            a.push_back({0, 0, 1, 1, 1, 1, x});
            return a.size() - 1;
        }
        if (x < a[p].value)
            a[p].l = insert(a[p].l, x);
        else if (x > a[p].value)
            a[p].r = insert(a[p].r, x);
        else
            a[p].count++;
        return balance(p);
    }

    int erase(int p, ll x, bool &found)
    {
        if (!p) return 0;
        if (x < a[p].value)
            a[p].l = erase(a[p].l, x, found);
        else if (x > a[p].value)
            a[p].r = erase(a[p].r, x, found);
        else if (a[p].count)
        {
            a[p].count--;
            found = true;
        }
        return balance(p);
    }

    void insert(ll x) { root = insert(root, x); }

    bool erase(ll x)
    {
        bool found = false;
        root = erase(root, x, found);
        return found;
    }
```

排名、第 k 小与严格前驱后继（接上页同一结构体）：

```
    int less(ll x, bool equal = false) const
    {
        int p = root, answer = 0;
        while (p)
        {
            if (a[p].value < x || (equal && a[p].value == x))
            {
                answer += a[a[p].l].size + a[p].count;
                p = a[p].r;
            }
            else
                p = a[p].l;
        }
        return answer;
    }

    int rank(ll x) const { return less(x) + 1; }

    ll kth(int k) const
    {
        assert(1 <= k && k <= size());
        int p = root;
        while (true)
        {
            int left = a[a[p].l].size;
            if (k <= left)
                p = a[p].l;
            else if (k <= left + a[p].count)
                return a[p].value;
            else
            {
                k -= left + a[p].count;
                p = a[p].r;
            }
        }
    }

    optional<ll> prev(ll x) const
    {
        int k = less(x);
        if (!k) return nullopt;
        return kth(k);
    }

    optional<ll> next(ll x) const
    {
        int k = less(x, true);
        if (k == size()) return nullopt;
        return kth(k + 1);
    }
};
```

模板题使用：Luogu P3369（替罪羊树）

题意：维护可重集合，操作 1..6 分别为插入、删除一个副本、排名、第 k 小、严格前驱、严格后继。排名查询键不一定存在；题目保证第 k 小和前驱后继的答案存在，因此可解引用 optional。该驱动单独调用替罪羊树，不沿用其他平衡树的 AC。

所需模板：ScapegoatTree（第 523 页，36.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    ScapegoatTree g;
    while (n--)
    {
        int op;
        long long x;
        cin >> op >> x;
        if (op == 1)
            g.insert(x);
        else if (op == 2)
            g.erase(x);
        else if (op == 3)
            cout << g.rank(x) << '\n';
        else if (op == 4)
            cout << g.kth(x) << '\n';
        else if (op == 5)
            cout << *g.prev(x) << '\n';
        else
            cout << *g.next(x) << '\n';
    }
}
```

36.4.2 在线序列：插入、修改与区间第 k 小

代码依赖：ost（第 473 页，36.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

依赖 ost（GNU PBDS，非 ISO 标准库），C++20。以 vector<long long> 构造，默认空序列。insert(k,x) 在前 k 项后插入，0≤k≤size()；set(k,x) 修改第 k 项。less(l,r,x) 计严格小于 x，末参 true 计 ≤x；kth(l,r,k) 取第 k 小，区间和 k 均 1-based 且非空合法。支持重复值、完整 signed64，midpoint 避免二分溢出；不提供删除。稳定节点编号区分重复值，递归重量重建。秩/修改 $O(\log^2(n+2))$，kth 最多 64 轮秩查询；构建 $O(n \log^2(n+2))$，插入保守均摊 $O(\log^3(n+2))$、单次最坏 $O(n \log^2(n+2))$。空间 $O(n \log(n+2))$，vector 扩容可能复制内树并增大峰值；节点数小于 INT_MAX。a/root 及带节点编号辅助方法内部使用，不外部改写。

```
struct SequenceKth
{
    using ll = long long;
    using Key = pair<ll, int>;

    struct Node
    {
        int l = 0, r = 0, sz = 0;
        ll val = 0;
        ost<Key> keys;
    };

    vector<Node> a;
    int root = 0;

    SequenceKth(const vector<ll> &v = {}) : a(v.size() + 1)
    {
        vector<int> ids;
        for (int i = 0; i < (int)v.size(); i++)
        {
            a[i + 1].val = v[i];
            ids.push_back(i + 1);
        }
        root = build(ids, 0, (int)ids.size());
    }

    int size() const
    {
        return a[root].sz;
    }
```

内树合并与递归重建（接上页同一结构体）：

```
    void pull(int u)
    {
        int l = a[u].l;
        int r = a[u].r;
        a[u].sz = a[l].sz + a[r].sz + 1;
        a[u].keys.clear();
        a[u].keys.insert({a[u].val, u});
        for (auto x : a[l].keys)
            a[u].keys.insert(x);
        for (auto x : a[r].keys)
            a[u].keys.insert(x);
    }

    int build(const vector<int> &v, int l, int r)
    {
        if (l == r)
            return 0;
        int m = l + (r - l) / 2;
        int u = v[m];
        a[u].l = build(v, l, m);
        a[u].r = build(v, m + 1, r);
        pull(u);
        return u;
    }

    void collect(int u, vector<int> &v) const
    {
        if (!u)
            return;
        collect(a[u].l, v);
        v.push_back(u);
        collect(a[u].r, v);
    }

    int balance(int u)
    {
        a[u].sz = a[a[u].l].sz + a[a[u].r].sz + 1;
        if (4LL * max(a[a[u].l].sz, a[a[u].r].sz) <= 3LL * a[u].sz)
            return u;
        vector<int> v;
        v.reserve(a[u].sz);
        collect(u, v);
        return build(v, 0, (int)v.size());
    }
```

按位置插入与修改（接上页同一结构体）：

```
    int insert(int u, int k, int id)
    {
        if (!u)
            return id;
        a[u].keys.insert({a[id].val, id});
        int left = a[a[u].l].sz;
        if (k <= left)
            a[u].l = insert(a[u].l, k, id);
        else
            a[u].r = insert(a[u].r, k - left - 1, id);
        return balance(u);
    }

    void insert(int k, ll x)
    {
        assert(0 <= k && k <= size());
        int id = (int)a.size();
        a.emplace_back();
        a[id].val = x;
        a[id].sz = 1;
        a[id].keys.insert({x, id});
        root = insert(root, k, id);
    }

    Key set(int u, int k, ll x)
    {
        int left = a[a[u].l].sz;
        Key old;
        if (k <= left)
            old = set(a[u].l, k, x);
        else if (k == left + 1)
        {
            old = {a[u].val, u};
            a[u].val = x;
        }
        else
            old = set(a[u].r, k - left - 1, x);
        a[u].keys.erase(old);
        a[u].keys.insert({x, old.second});
        return old;
    }

    void set(int k, ll x)
    {
        assert(1 <= k && k <= size());
        set(root, k, x);
    }
```

区间秩与第 k 小查询（接上页同一结构体）：

```
    int less(int u, int l, int r, ll x, bool equal) const
    {
        if (!u)
            return 0;
        if (l <= 1 && a[u].sz <= r)
            return a[u].keys.order_of_key({x, equal ? (int)a.size() : 0});
        int left = a[a[u].l].sz;
        int ans = 0;
        if (l <= left)
            ans += less(a[u].l, l, r, x, equal);
        if (l <= left + 1 && left + 1 <= r)
            ans += a[u].val < x || (equal && a[u].val == x);
        if (r > left + 1)
            ans += less(a[u].r, l - left - 1, r - left - 1, x, equal);
        return ans;
    }

    int less(int l, int r, ll x, bool equal = false) const
    {
        assert(1 <= l && l <= r && r <= size());
        return less(root, l, r, x, equal);
    }

    ll kth(int l, int r, int k) const
    {
        assert(1 <= l && l <= r && r <= size());
        assert(1 <= k && k <= r - l + 1);
        ll lo = a[root].keys.begin()->first;
        ll hi = a[root].keys.rbegin()->first;
        while (lo < hi)
        {
            ll mid = midpoint(lo, hi);
            if (less(l, r, mid, true) >= k)
                hi = mid;
            else
                lo = mid + 1;
        }
        return lo;
    }
};
```

接口用法演示：带插入区间第 k 小（kuangbin BZOJ3065 模型）

演示约定：来源协议：EOF 多组，每组 n、初值、q 及 Q/M/I 操作，值域 0..100000。所有整数参数与上次查询答案异或；Q x y k 查闭区间第 k 小，M x val 修改第 x 项，I x val 在第 x 项前插入（调用 insert(x-1,val)）。仅 Q 更新 lastans，解码后端点有序且合法。核心可用 signed64，协议答案限定来源值域后转 int。官方题面与资源限制未核验，只登记来源模型/API 用法。

所需模板：ost（第 473 页，36.1.2 节）、SequenceKth（第 527 页，36.4.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://www.lydsy.com/JudgeOnline/problem.php?id=3065

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    while (cin >> n)
    {
        vector<long long> a(n);
        for (auto &x : a)
            cin >> x;
        SequenceKth tree(a);
        int q;
        cin >> q;
        int ans = 0;
        while (q--)
        {
            char op;
            int x, y;
            cin >> op >> x >> y;
            x ^= ans;
            y ^= ans;
            if (op == 'Q')
            {
                int k;
                cin >> k;
                k ^= ans;
                ans = (int)tree.kth(x, y, k);
                cout << ans << '\n';
            }
            else if (op == 'M')
                tree.set(x, y);
            else
                tree.insert(x - 1, y);
        }
    }
}
```

36.4.3 序列替罪羊树：按位置插删与区间等值计数

以 vector<long long> 构造，默认空序列。insert(k,x) 在前 k 项后插入，0≤k≤size()；erase(k) 删除第 k 项并返回值；count(l,r,x) 计 1-based 非空闭区间内等于 x 的项数；rotate(l,r) 将区间末项移至开头。键支持 long long 全范围。递归按位置导航，墓碑过多或节点数失衡时重建，回收死节点并删除零频次。map 频次表令查询 $O(\log^2(n+2))$，重建 s 点 $O(s\ \log^2(s+2))$，更新保守均摊 $O(\log^3(N+2))$、单次最坏 $O(N\ \log^2(N+2))$，N 为历史最大长度；初建 $O(n\ \log^2(n+2))$。频次表总空间 $O(n\ \log(n+2))$，vector 池 O(N)，节点数须小于 INT_MAX。a/root/spare 及带节点编号辅助函数均内部使用，不能外部改动。

```
struct SequenceScapegoat
{
    using ll = long long;

    struct Node
    {
        int l = 0, r = 0, sz = 0, cnt = 0;
        ll val = 0;
        bool live = false;
        map<ll, int> freq;
    };

    vector<Node> a{Node{}};
    vector<int> spare;
    int root = 0;

    SequenceScapegoat(const vector<ll> &v = {})
    {
        vector<int> ids;
        for (ll x : v)
            ids.push_back(make(x));
        root = build(ids, 0, (int)ids.size());
    }

    int size() const
    {
        return a[root].sz;
    }
```

节点复用与频次维护（接上页同一结构体）：

```
    int make(ll x)
    {
        int u;
        if (spare.empty())
        {
            u = (int)a.size();
            a.push_back(Node{});
        }
        else
        {
            u = spare.back();
            spare.pop_back();
        }
        a[u].sz = 1;
        a[u].cnt = 1;
        a[u].val = x;
        a[u].live = true;
        a[u].freq[x] = 1;
        return u;
    }

    void change(int u, ll x, int d)
    {
        int &v = a[u].freq[x];
        v += d;
        if (v == 0)
            a[u].freq.erase(x);
    }

    void pull(int u)
    {
        int l = a[u].l;
        int r = a[u].r;
        a[u].sz = a[l].sz + a[r].sz + a[u].live;
        a[u].cnt = a[l].cnt + a[r].cnt + 1;
        a[u].freq.clear();
        if (a[u].live)
            a[u].freq[a[u].val] = 1;
        for (auto [x, c] : a[l].freq)
            a[u].freq[x] += c;
        for (auto [x, c] : a[r].freq)
            a[u].freq[x] += c;
    }
```

递归收集、重建与平衡（接上页同一结构体）：

```
    void collect(int u, vector<int> &v)
    {
        if (!u)
            return;
        collect(a[u].l, v);
        if (a[u].live)
            v.push_back(u);
        collect(a[u].r, v);
        if (!a[u].live)
        {
            a[u] = Node{};
            spare.push_back(u);
        }
    }

    int build(const vector<int> &v, int l, int r)
    {
        if (l == r)
            return 0;
        int m = l + (r - l) / 2;
        int u = v[m];
        a[u].l = build(v, l, m);
        a[u].r = build(v, m + 1, r);
        pull(u);
        return u;
    }

    int balance(int u)
    {
        int heavy = max(a[a[u].l].cnt, a[a[u].r].cnt);
        if (4LL * heavy <= 3LL * a[u].cnt && 2LL * a[u].sz >= a[u].cnt)
            return u;
        vector<int> v;
        v.reserve(a[u].sz);
        collect(u, v);
        return build(v, 0, (int)v.size());
    }
```

按位置插入与删除（接上页同一结构体）：

```
    int insert(int u, int k, ll x)
    {
        if (!u)
            return make(x);
        int left = a[a[u].l].sz;
        if (k <= left)
            a[u].l = insert(a[u].l, k, x);
        else
            a[u].r = insert(a[u].r, k - left - a[u].live, x);
        a[u].sz++;
        a[u].cnt = a[a[u].l].cnt + a[a[u].r].cnt + 1;
        change(u, x, 1);
        return balance(u);
    }

    void insert(int k, ll x)
    {
        assert(0 <= k && k <= size());
        root = insert(root, k, x);
    }

    int erase(int u, int k, ll &x)
    {
        int left = a[a[u].l].sz;
        if (k <= left)
            a[u].l = erase(a[u].l, k, x);
        else if (a[u].live && k == left + 1)
        {
            x = a[u].val;
            a[u].live = false;
        }
        else
            a[u].r = erase(a[u].r, k - left - a[u].live, x);
        a[u].sz--;
        a[u].cnt = a[a[u].l].cnt + a[a[u].r].cnt + 1;
        change(u, x, -1);
        return balance(u);
    }

    ll erase(int k)
    {
        assert(1 <= k && k <= size());
        ll x = 0;
        root = erase(root, k, x);
        return x;
    }
```

区间等值计数与右移（接上页同一结构体）：

```
    int count(int u, int l, int r, ll x) const
    {
        if (!u || l > r)
            return 0;
        if (l <= 1 && a[u].sz <= r)
        {
            auto it = a[u].freq.find(x);
            return it == a[u].freq.end() ? 0 : it->second;
        }
        int left = a[a[u].l].sz;
        int ans = 0;
        if (l <= left)
            ans += count(a[u].l, l, r, x);
        if (a[u].live && l <= left + 1 && left + 1 <= r && a[u].val == x)
            ans++;
        int offset = left + a[u].live;
        if (r > offset)
            ans += count(a[u].r, l - offset, r - offset, x);
        return ans;
    }

    int count(int l, int r, ll x) const
    {
        assert(1 <= l && l <= r && r <= size());
        return count(root, l, r, x);
    }

    void rotate(int l, int r)
    {
        assert(1 <= l && l <= r && r <= size());
        ll x = erase(r);
        insert(l - 1, x);
    }
};
```

应用题补充：Codeforces 455D / Serega and Fun

题意：n,q≤100000，初值和原始参数均 1..n。操作 1 把解码区间的末项移至开头，操作 2 计区间中等于解码 k 的项数。每个参数先用 (v+lastans-1)%n+1 解码，再交换逆序端点；lastans 只在查询后更新。比赛应用，不代替独立模板题；插入/删除等额外接口另有独立测试，尚无线上 AC 或资源通过声明。

所需模板：SequenceScapegoat（第 532 页，36.4.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/455/D

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<long long> a(n);
    for (auto &x : a)
        cin >> x;
    SequenceScapegoat tree(a);
    int q;
    cin >> q;
    int ans = 0;
    while (q--)
    {
        int op, l, r;
        cin >> op >> l >> r;
        l = (l + ans - 1) % n + 1;
        r = (r + ans - 1) % n + 1;
        if (l > r)
            swap(l, r);
        if (op == 1)
            tree.rotate(l, r);
        else
        {
            int k;
            cin >> k;
            k = (k + ans - 1) % n + 1;
            ans = tree.count(l, r, k);
            cout << ans << '\n';
        }
    }
}
```

36.5 笛卡尔树

36.5.1 笛卡尔树与相等值优先级

cartesian_tree(a) 返回 CartesianTree {root,left,right}，下标 0..n-1，缺失孩子及空树根为-1。中序遍历为原下标顺序，按值为小根堆；相等值保留较小下标在上方，即区间最左最小值为根。输入不修改，n 须能用 int 表示，元素比较须构成严格弱序，浮点 NaN 不适用。

比较 O(1) 时，构造 O(n)，结果数组及辅助单调栈均 O(n)。每点入栈一次、出栈至多一次；单调栈维护树的最右路径，不模拟 DFS。弹出节点中最后一个成为新节点的左孩子，剩余栈顶的右孩子改成新节点。P5854 用 1..n 排列，n≤10^7；P5854 已用 C++14 GCC9 满分通过，排名未核验。

```
struct CartesianTree
{
    int root;
    vector<int> left, right;
};

template <class T>
CartesianTree cartesian_tree(const vector<T> &a)
{
    int n = a.size();
    CartesianTree tree{-1, vector<int>(n, -1), vector<int>(n, -1)};
    vector<int> st;
    st.reserve(n);
    for (int i = 0; i < n; i++)
    {
        int last = -1;
        while (!st.empty() && a[i] < a[st.back()])
        {
            last = st.back();
            st.pop_back();
        }
        tree.left[i] = last;
        if (!st.empty()) tree.right[st.back()] = i;
        st.push_back(i);
    }
    if (n) tree.root = st.front();
    return tree;
}
```

模板题使用：P5854 笛卡尔树

题意：输入 1..n 的排列，构造小根笛卡尔树并输出左右孩子的异或校验和。孩子由 0-based 转为 1-based，缺失孩子为 0；乘法使用 long long。

所需模板：CartesianTree（第 538 页，36.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5854

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    auto tree = cartesian_tree(a);
    long long left = 0;
    long long right = 0;
    for (int i = 0; i < n; i++)
    {
        left ^= 1LL * (i + 1) * (tree.left[i] + 2);
        right ^= 1LL * (i + 1) * (tree.right[i] + 2);
    }
    cout << left << ' ' << right << '\n';
}
```

第三十七章　可持久化数据结构

37.1　可持久化线段树

37.1.1　主席树：静态区间第 k 小

构造传入 0-based 数组，kth(l,r,k) 的区间使用 1-based 闭区间；要求 1≤k≤r-l+1。离散化后每个前缀一个版本，单次 O(log n)，总空间 O(n log n)。

```
struct PersistentKth
{
    struct Node
    {
        int l = 0, r = 0, sum = 0;
    };

    vector<Node> t{Node{}};
    vector<int> root{0};
    vector<long long> vals;

    PersistentKth(const vector<long long> &a) : vals(a)
    {
        sort(vals.begin(), vals.end());
        vals.erase(unique(vals.begin(), vals.end()), vals.end());
        for (long long x : a)
        {
            int p = lower_bound(vals.begin(), vals.end(), x) - vals.begin();
            root.push_back(add(root.back(), 0, (int)vals.size() - 1, p));
        }
    }

    int add(int old, int l, int r, int x)
    {
        int p = t.size();
        Node copy = t[old];
        t.push_back(copy);
        ++t[p].sum;
        if (l < r)
        {
            int m = (l + r) / 2;
            if (x <= m)
            {
                int v = add(t[old].l, l, m, x);
                t[p].l = v;
            }
            else
            {
                int v = add(t[old].r, m + 1, r, x);
                t[p].r = v;
            }
        }
        return p;
```

```
    }

    long long kth(int l, int r, int k) const
    {
        assert(1 <= l && l <= r && r < (int)root.size() && 1 <= k && k <= r - l + 1);
        int u = root[l - 1], v = root[r], L = 0, R = (int)vals.size() - 1;
        while (L < R)
        {
            int m = (L + R) / 2, c = t[t[v].l].sum - t[t[u].l].sum;
            if (k <= c)
            {
                u = t[u].l;
                v = t[v].l;
                R = m;
            }
            else
            {
                k -= c;
                u = t[u].r;
                v = t[v].r;
                L = m + 1;
            }
        }
        return vals[L];
    }
};
```

模板题使用：Luogu P3834

题意：给定不再修改的数组，查询闭区间 [l,r] 内第 k 小的值，重复值按出现次数计数。构造传入 0-based vector，查询 l、r、k 均从 1 开始。

所需模板：PersistentKth（第 540 页，37.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3834

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    std::vector<long long> a(n);
    for (auto &x : a) std::cin >> x;
    PersistentKth tree(a);
    while (m--)
    {
        int l, r, k;
        std::cin >> l >> r >> k;
        std::cout << tree.kth(l, r, k) << '\n';
    }
}
```

37.1.2 可持久化数组：历史版本分叉与单点修改

PersistentArray(a,changes=0)，a 是非空 0-based 初值数组；第 0 版为初始状态。set(version,pos,value) 基于任意已有版本修改 1-based 位置，创建并返回新版本编号；copy(version) 以 O(1) 时间复制版本；query(version,pos) 只读且不自动创建版本。values 支持 long long 全范围。每次修改复制根到叶的节点，旧版本不变；build 为内部函数，不可覆盖历史节点。建树 O(n)，修改及查询 O(log n)，初始 2n-1 个节点，每次修改至多 ceil(log2(n))+1 个；总空间 O(n+ 修改数 *log n+ 版本数)。changes 仅用于预留节点池，低估后可扩容，所有外部引用应使用下标，不保存 vector 元素引用。节点下标用 int，节点总量须可表示。

```
struct PersistentArray
{
    using ll = long long;

    struct Node
    {
        int l = 0, r = 0;
        ll value = 0;
    };

    int n;
    vector<Node> t{Node{}};
    vector<int> root;

    PersistentArray(const vector<ll> &a, int changes = 0) : n(a.size())
    {
        assert(n > 0 && changes >= 0);
        int levels = 1;
        for (int x = n - 1; x; x >>= 1) levels++;
        t.reserve(2ULL * n + 1ULL * changes * levels + 1);
        root.push_back(build(1, n, a));
    }

    int build(int l, int r, const vector<ll> &a)
    {
        int p = t.size();
        t.push_back(Node{});
        if (l == r)
            t[p].value = a[l - 1];
        else
        {
            int m = (l + r) / 2;
            int left = build(l, m, a);
            int right = build(m + 1, r, a);
            t[p].l = left;
            t[p].r = right;
        }
        return p;
    }

    int modify(int old, int l, int r, int pos, ll value)
    {
        int p = t.size();
        Node copy = t[old];
        t.push_back(copy);
        if (l == r)
            t[p].value = value;
```

```
        else
        {
            int m = (l + r) / 2;
            if (pos <= m)
            {
                int child = modify(t[old].l, l, m, pos, value);
                t[p].l = child;
            }
            else
            {
                int child = modify(t[old].r, m + 1, r, pos, value);
                t[p].r = child;
            }
        }
        return p;
    }

    // Create a new version from any existing version. Positions are 1-based.
    int set(int version, int pos, ll value)
    {
        assert(0 <= version && version < (int)root.size());
        assert(1 <= pos && pos <= n);
        int p = modify(root[version], 1, n, pos, value);
        root.push_back(p);
        return (int)root.size() - 1;
    }

    int copy(int version)
    {
        assert(0 <= version && version < (int)root.size());
        root.push_back(root[version]);
        return (int)root.size() - 1;
    }

    ll query(int version, int pos) const
    {
        assert(0 <= version && version < (int)root.size());
        assert(1 <= pos && pos <= n);
        int p = root[version], l = 1, r = n;
        while (l < r)
        {
            int m = (l + r) / 2;
            if (pos <= m)
            {
                p = t[p].l;
                r = m;
```

```
            }
            else
            {
                p = t[p].r;
                l = m + 1;
            }
        }
        return t[p].value;
    }
};
```

模板题使用：Luogu P3919

题意：从指定历史版本修改单点或查询单点，每次操作都创建新版本。查询后也要 copy 原版本；版本号从 0 开始，位置为 1-based。

所需模板：PersistentArray（第 542 页，37.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3919

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    std::vector<long long> a(n);
    for (auto &x : a) std::cin >> x;
    PersistentArray tree(a, m);
    while (m--)
    {
        int version, op, pos;
        std::cin >> version >> op >> pos;
        if (op == 1)
        {
            long long value;
            std::cin >> value;
            tree.set(version, pos, value);
        }
        else
        {
            std::cout << tree.query(version, pos) << '\n';
            tree.copy(version);
        }
    }
}
```

37.1.3 可持久化区间加与版本拼接

PersistentRange(n) 建立长度 n 的全零版本 0，n≥1；使用隐式节点，不预建整棵树。add(v,l,r,x) 在版本 v 的 [l,r) 加 x；splice(dst,src,l,r) 将 dst 的 [l,r) 替换为 src 同一位置的值，其他位置保留 dst；二者均返回新版本编号。query(v,l,r) 返回区间和，const 且不分配节点。下标 0-based、区间半开，允许空区间；空修改及零增量仍创建版本但复用根，整段拼接直接共享源根。全体版本属于同一对象、同一坐标域。懒标记保留在祖先，用 offset 修正跨版本拼接；每次修改/拼接/查询 O(log n)，版本别名 O(1)，总节点 O(q log n)。sum、lazy、delta、carry、offset

及其中间和差和长度乘积均须在 long long 内；节点总数和版本数须小于 INT_MAX。t、root 可读以统计空间或查看根，不要外部改写。

```
struct PersistentRange
{
    using ll = long long;

    struct Node
    {
        int left = 0, right = 0;
        ll sum = 0, lazy = 0;
    };

    int n;
    vector<Node> t{Node{}};
    vector<int> root{0};

    explicit PersistentRange(int size) : n(size) { assert(n > 0); }

    int clone(int p)
    {
        Node node = t[p];
        t.push_back(node);
        return int(t.size()) - 1;
    }

    void pull(int p, int len)
    {
        t[p].sum = t[t[p].left].sum + t[t[p].right].sum + t[p].lazy * len;
    }
```

区间加与版本拼接（接上页同一结构体）：

```
    int change(int p, int l, int r, int ql, int qr, ll value)
    {
        if (qr <= l || r <= ql) return p;
        int u = clone(p);
        if (ql <= l && r <= qr)
        {
            t[u].lazy += value;
            t[u].sum += value * (r - l);
            return u;
        }
        int m = l + (r - l) / 2;
        int a = change(t[p].left, l, m, ql, qr, value);
        int b = change(t[p].right, m, r, ql, qr, value);
        t[u].left = a;
        t[u].right = b;
        pull(u, r - l);
        return u;
    }

    int replace(int p, int q, int l, int r, int ql, int qr, ll offset)
    {
        if (qr <= l || r <= ql) return p;
        if (ql <= l && r <= qr)
        {
            if (offset == 0) return q;
            int u = clone(q);
            t[u].lazy += offset;
            t[u].sum += offset * (r - l);
            return u;
        }
        int u = clone(p), m = l + (r - l) / 2;
        offset += t[q].lazy - t[p].lazy;
        int a = replace(t[p].left, t[q].left, l, m, ql, qr, offset);
        int b = replace(t[p].right, t[q].right, m, r, ql, qr, offset);
        t[u].left = a;
        t[u].right = b;
        pull(u, r - l);
        return u;
    }
```

只读查询与对外接口（接上页同一结构体）：

```
    ll get(int p, int l, int r, int ql, int qr, ll carry) const
    {
        if (qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return t[p].sum + carry * (r - l);
        carry += t[p].lazy;
        int m = l + (r - l) / 2;
        return get(t[p].left, l, m, ql, qr, carry) +
               get(t[p].right, m, r, ql, qr, carry);
    }

    void check(int version, int l, int r) const
    {
        assert(0 <= version && version < int(root.size()));
        assert(0 <= l && l <= r && r <= n);
    }

    int add(int version, int l, int r, ll value)
    {
        check(version, l, r);
        int p = l == r || value == 0 ? root[version]
                                     : change(root[version], 0, n, l, r, value);
        root.push_back(p);
        return int(root.size()) - 1;
    }

    int splice(int destination, int source, int l, int r)
    {
        check(destination, l, r);
        check(source, l, r);
        int p = l == r ? root[destination]
                       : replace(root[destination], root[source], 0, n, l, r, 0);
        root.push_back(p);
        return int(root.size()) - 1;
    }

    ll query(int version, int l, int r) const
    {
        check(version, l, r);
        return l == r ? 0 : get(root[version], 0, n, l, r, 0);
    }
};
```

应用题补充：SPOJ TTM / Luogu SP11470

题意：C 对当前版本区间加并使时间戳加 1；Q 查当前和，H 查指定时间和，B 回退时间；n,m≤100000，初值绝对值 ≤10^9，增量绝对值 ≤10^4。模板存相对初值的增量，初值用前缀和单独累加。ver[t] 映射逻辑时间到追加版本 ID，回退后 C 覆盖 ver[t+1] 而非使用 t 作为模板版本。闭区间转 [l-1,r]，O(n+m log n)。

所需模板：PersistentRange（第 544 页，37.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/SP11470

SPOJ TTM / Luogu SP11470：使用代码

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    while (cin >> n >> m)
    {
        vector<long long> pre(n + 1);
        for (int i = 1; i <= n; i++)
        {
            cin >> pre[i];
            pre[i] += pre[i - 1];
        }
        PersistentRange tree(n);
        vector<int> ver(m + 1);
        int t = 0;
        while (m--)
        {
            char op;
            cin >> op;
            if (op == 'B')
            {
                cin >> t;
                continue;
            }
            int l, r;
            cin >> l >> r;
            l--;
            if (op == 'C')
            {
                long long d;
                cin >> d;
                int v = tree.add(ver[t], l, r, d);
                ver[++t] = v;
            }
            else
            {
                int h = t;
                if (op == 'H') cin >> h;
                cout << pre[r] - pre[l] + tree.query(ver[h], l, r) << '\n';
            }
        }
    }
    return 0;
}
```

应用题补充：QOJ 8240 Card Game

题意：ICPC 杭州 2023K：给区间牌列，顺序放牌，遇到已有同号牌就连同其后所有牌及新牌一起删除，问最终剩余数。n,q≤300000，牌号 1..n；端点须与上次答案异或解码。version[i] 的坐标 r 表示 [i,r] 答案；下一同号位置 p 前区间加 1，p 起后缀从 version[p+1] 拼接。nodes 仅估算分配上界，最后按右端点单点查。O((n+q)log n)。

所需模板：PersistentRange（第 544 页，37.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://qoj.ac/problem/8240

QOJ 8240 Card Game：使用代码

```
int main()
{
    int n, q;
    scanf("%d%d", &n, &q);
    vector<int> a(n), next(n), last(n + 1, n), version(n + 1);
    for (auto &x : a) scanf("%d", &x);
    for (int i = n - 1; i >= 0; i--)
    {
        next[i] = last[a[i]];
        last[a[i]] = i;
    }
    PersistentRange tree(n);
    function<size_t(int, int, int, int)> nodes =
        [&](int l, int r, int ql, int qr) -> size_t
    {
        if (qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return 1;
        int m = l + (r - l) / 2;
        return 1 + nodes(l, m, ql, qr) + nodes(m, r, ql, qr);
    };
    size_t capacity = 1;
    for (int i = 0; i < n; i++)
    {
        capacity += nodes(0, n, i, next[i]);
        capacity += nodes(0, n, next[i], n);
    }
    tree.t.reserve(capacity);
    tree.root.reserve(2 * n + 1);
    for (int i = n - 1; i >= 0; i--)
    {
        int p = next[i];
        int v = tree.add(version[i + 1], i, p, 1);
        version[i] = p == n ? v : tree.splice(v, version[p + 1], p, n);
    }
    int answer = 0;
    while (q--)
    {
        int l, r;
        scanf("%d%d", &l, &r);
        l ^= answer;
        r ^= answer;
        answer = tree.query(version[l - 1], r - 1, r);
        printf("%d\n", answer);
    }
}
```

37.1.4 树上路径点权第 k 小

TreePathKth(a) 传入非空 0-based 权值数组，节点编号 1..n；add(u,v) 加无向边，要求恰好构成一棵树。build(root=1) 递归 DFS 建立根到各节点的频数版本与倍增表，可换根重建，旧版本自动清空。kth(u,v,k) 返回简单路径上第 k 小的点权，包含两个端点，每个节点计一次；1≤k≤ 路径点数。lca(u,v) 返回当前预处理根下的最近公共祖先。使用 root[u]+root[v]-root[lca]-root[parent(lca)]，编号 0 为全零版本，避免少算或重复算 LCA。支持重复值与完整 long long 权值，内部只统计频数；权值和树结构在查询阶段保持不变。预处理 O(n log n)，每次查询 O(log n)，空间 O(n log n)。递归深度最坏 O(n)，保持递归写法。

```
struct TreePathKth
{
    using ll = long long;

    struct Node
    {
        int l = 0, r = 0, sum = 0;
    };

    int n, lg;
    vector<vector<int>> g, up;
    vector<int> root, dep;
    vector<ll> value, vals;
    vector<Node> t;

    TreePathKth(const vector<ll> &a)
        : n(a.size()), g(n + 1), root(n + 1), dep(n + 1), value(a)
    {
        assert(n > 0);
        lg = 1;
        while ((1LL << lg) <= n) lg++;
        up.assign(lg, vector<int>(n + 1));
    }

    void add(int u, int v)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }
```

版本插入与递归建树（接上页同一结构体）：

```
    int insert(int old, int l, int r, int x)
    {
        int p = t.size();
        Node copy = t[old];
        t.push_back(copy);
        t[p].sum++;
        if (l == r) return p;
        int m = (l + r) / 2;
        if (x <= m)
        {
            int child = insert(t[old].l, l, m, x);
            t[p].l = child;
        }
        else
        {
            int child = insert(t[old].r, m + 1, r, x);
            t[p].r = child;
        }
        return p;
    }

    void dfs(int u, int p)
    {
        up[0][u] = p;
        for (int j = 1; j < lg; j++) up[j][u] = up[j - 1][up[j - 1][u]];
        int x = lower_bound(vals.begin(), vals.end(), value[u - 1]) - vals.begin();
        root[u] = insert(root[p], 0, (int)vals.size() - 1, x);
        for (int v : g[u])
        {
            if (v == p) continue;
            dep[v] = dep[u] + 1;
            dfs(v, u);
        }
    }

    void build(int r = 1)
    {
        assert(1 <= r && r <= n);
        vals = value;
        sort(vals.begin(), vals.end());
        vals.erase(unique(vals.begin(), vals.end()), vals.end());
        t.assign(1, Node{});
        dep[r] = 0;
        dfs(r, 0);
    }
```

最近公共祖先与路径第 k 小（接上页同一结构体）：

```
    int lca(int u, int v) const
    {
        if (dep[u] < dep[v]) swap(u, v);
        int diff = dep[u] - dep[v];
        for (int j = 0; j < lg; j++)
            if (diff >> j & 1) u = up[j][u];
        if (u == v) return u;
        for (int j = lg - 1; j >= 0; j--)
            if (up[j][u] != up[j][v])
            {
                u = up[j][u];
                v = up[j][v];
            }
        return up[0][u];
    }

    ll kth(int u, int v, int k) const
    {
        int z = lca(u, v);
        assert(1 <= k && k <= dep[u] + dep[v] - 2 * dep[z] + 1);
        int a = root[u], b = root[v], c = root[z], d = root[up[0][z]];
        int l = 0, r = (int)vals.size() - 1;
        while (l < r)
        {
            int count = t[t[a].l].sum + t[t[b].l].sum - t[t[c].l].sum - t[t[d].l].sum;
            int m = (l + r) / 2;
            if (k <= count)
            {
                a = t[a].l;
                b = t[b].l;
                c = t[c].l;
                d = t[d].l;
                r = m;
            }
            else
            {
                k -= count;
                a = t[a].r;
                b = t[b].r;
                c = t[c].r;
                d = t[d].r;
                l = m + 1;
            }
        }
        return vals[l];
    }
};
```

模板题使用：Luogu P2633

题意：给点权树，查询路径上第 k 小的点权，包含两个端点。构造传入 0-based 权值数组，点号为 1..n；每次先将输入的第一个端点与上次答案异或解码，初始 last=0。

所需模板：TreePathKth（第 552 页，37.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2633

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    std::vector<long long> a(n);
    for (auto &x : a) std::cin >> x;
    TreePathKth tree(a);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        std::cin >> u >> v;
        tree.add(u, v);
    }
    tree.build();
    long long last = 0;
    while (m--)
    {
        long long encoded;
        int v, k;
        std::cin >> encoded >> v >> k;
        int u = encoded ^ last;
        last = tree.kth(u, v, k);
        std::cout << last << '\n';
    }
}
```

37.1.5 静态区间不同数计数（前驱位置主席树）

PersistentDistinct(a) 传入非空 0-based 数组，query(l,r) 返回闭区间内不同值的个数，1≤l≤r≤n。支持重复值与完整 long long 值域；建成后不支持改值，查询顺序任意。令 prev[i] 为同值上次出现的位置，未出现为 0；root[i] 在值域 0..n-1 中统计前 i 个 prev。区间内每种值恰有一次 prev[i]<l，因此答案为 root[r]-root[l-1] 中位置 0..l-1 的频数和。每个元素只复制一条节点路径，预处理 O(n log n)，查询 O(log n)，空间 O(n log n)。动态版预留 n · (ceil(log2(n))+1)+1 个节点，避免扩容峰值；root 和节点池为内部数据。

```
struct PersistentDistinct
{
    struct Node
    {
        int l = 0, r = 0, sum = 0;
    };

    int n;
    vector<Node> t{Node{}};
    vector<int> root;

```

```
    PersistentDistinct(const vector<long long> &a) : n(a.size()), root(n + 1)
    {
        assert(n > 0);
        int height = 1;
        while ((1LL << (height - 1)) < n) height++;
        t.reserve(size_t(n) * height + 1);
        vector<long long> vals = a;
        sort(vals.begin(), vals.end());
        vals.erase(unique(vals.begin(), vals.end()), vals.end());
        vector<int> last(vals.size());
        for (int i = 1; i <= n; i++)
        {
            int x = lower_bound(vals.begin(), vals.end(), a[i - 1]) - vals.begin();
            root[i] = insert(root[i - 1], 0, n - 1, last[x]);
            last[x] = i;
        }
    }

    int insert(int old, int l, int r, int x)
    {
        int p = t.size();
        Node copy = t[old];
        t.push_back(copy);
        t[p].sum++;
        if (l < r)
        {
            int m = (l + r) / 2;
            if (x <= m)
            {
                int child = insert(t[old].l, l, m, x);
                t[p].l = child;
            }
            else
            {
                int child = insert(t[old].r, m + 1, r, x);
                t[p].r = child;
            }
        }
        return p;
    }

    int query(int l, int r) const
    {
        assert(1 <= l && l <= r && r <= n);
        int u = root[l - 1], v = root[r];
        int L = 0, R = n - 1, answer = 0;
        while (L < R)
        {
            int m = (L + R) / 2;
            if (l - 1 <= m)
            {
                u = t[u].l;
                v = t[v].l;
                R = m;
            }
            else
```

```
            {
                answer += t[t[v].l].sum - t[t[u].l].sum;
                u = t[u].r;
                v = t[v].r;
                L = m + 1;
            }
        }
        return answer + t[v].sum - t[u].sum;
    }
};
```

应用题补充：Luogu P1972

题意：给定固定数组，查询闭区间 [l,r] 内不同值的个数。构造传入 0-based vector，查询使用 1-based 闭区间；重复出现的同一值只计一次。

所需模板：PersistentDistinct（第 555 页，37.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1972

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n;
    std::cin >> n;
    std::vector<long long> a(n);
    for (auto &x : a) std::cin >> x;
    PersistentDistinct tree(a);
    int m;
    std::cin >> m;
    while (m--)
    {
        int l, r;
        std::cin >> l >> r;
        std::cout << tree.query(l, r) << '\n';
    }
}
```

相关应用：来源专用值域平移/历史计数权值索引，不代表整页已覆盖

37.1.6 持久化计数与权值索引

PersistentWeightedIndex(n)，位置域 [0,n)，n 为正 int，版本 0 为空。add(version,pos,weight) 从任意已有版本分叉，新增一个元素，count 加 1、sum 加 weight，返回新版本号；weight 允许负数。query(version,l,r) 返回半开区间的 {count,sum}，空区间合法。first(before,after,key) 返回计数版本差中 >=key 的最小位置，无结果-1；last(before,after,key) 返回严格 <key 的最大位置，无结果-1；二者 key 均可越界。必须保证 after-before 逐位置计数非负，祖先版本与后代版本满足此前提，任意两个分支不一定满足。后继按 count 判断，不按 sum。

每次插入新增 O(log(n+1)) 节点，query/first/last 时间 O(log(n+1))，查询不分配节点。元素计数、版本数和节点数需可用 int 编号；所有节点权值和及查询累加中间结果须适配 signed64。两次 query 相减可得区间版本差，但减法也需防溢出。内部根与辅助函数不可外部修改，对象复制独立。

```
struct PersistentWeightedIndex
{
    using ll = long long;
    struct Node
    {
        int left = 0, right = 0, count = 0;
        ll sum = 0;
    };
    int n;
    vector<Node> t{Node{}};
    vector<int> roots{0};

    PersistentWeightedIndex(int n) : n(n)
    {
        assert(n > 0);
    }

    int insert(int old, int l, int r, int pos, ll weight)
    {
        assert(t.size() < INT_MAX && t[old].count < INT_MAX);
        int u = t.size();
        Node copy = t[old];
        copy.count++;
        copy.sum += weight;
        t.push_back(copy);
        if (r - l == 1)
            return u;
        int m = l + (r - l) / 2;
        if (pos < m)
            t[u].left = insert(t[u].left, l, m, pos, weight);
        else
            t[u].right = insert(t[u].right, m, r, pos, weight);
        return u;
    }
```

接上页同一结构体：

```
    int add(int version, int pos, ll weight)
    {
        assert(0 <= version && version < (int)roots.size());
        assert(0 <= pos && pos < n && roots.size() < INT_MAX);
        int root = insert(roots[version], 0, n, pos, weight);
        roots.push_back(root);
        return (int)roots.size() - 1;
    }

    pair<int, ll> query_node(int u, int l, int r, int ql, int qr) const
    {
        if (!u || qr <= l || r <= ql)
            return {0, 0};
        if (ql <= l && r <= qr)
            return {t[u].count, t[u].sum};
        int m = l + (r - l) / 2;
        auto a = query_node(t[u].left, l, m, ql, qr);
        auto b = query_node(t[u].right, m, r, ql, qr);
        return {a.first + b.first, a.second + b.second};
    }

    pair<int, ll> query(int version, int l, int r) const
    {
        assert(0 <= version && version < (int)roots.size());
        assert(0 <= l && l <= r && r <= n);
        return query_node(roots[version], 0, n, l, r);
    }

    int find(int before, int after, int l, int r, int key) const
    {
        if (r <= key || t[after].count == t[before].count)
            return -1;
        if (r - l == 1)
            return l;
        int m = l + (r - l) / 2;
        int answer = find(t[before].left, t[after].left, l, m, key);
        if (answer == -1)
            answer = find(t[before].right, t[after].right, m, r, key);
        return answer;
    }

    // The after-before count at every position must be nonnegative.
    int first(int before, int after, int key) const
    {
        assert(0 <= before && before < (int)roots.size());
        assert(0 <= after && after < (int)roots.size());
        return find(roots[before], roots[after], 0, n, key);
    }

    int find_last(int before, int after, int l, int r, int key) const
    {
        if (l >= key || t[after].count == t[before].count)
            return -1;
        if (r - l == 1)
```

```
            return l;
        int m = l + (r - l) / 2;
        int answer = find_last(t[before].right, t[after].right, m, r, key);
        if (answer == -1)
            answer = find_last(t[before].left, t[after].left, l, m, key);
        return answer;
    }

    // Strict predecessor; the same nonnegative count-difference contract applies.
    int last(int before, int after, int key) const
    {
        assert(0 <= before && before < (int)roots.size());
        assert(0 <= after && after < (int)roots.size());
        return find_last(roots[before], roots[after], 0, n, key);
    }
};
```

接口用法演示：持久化计数与权值索引 API

演示约定：输入 n q，位置域 [0,n)、初始版本 0 为空。1 version pos weight 从该版本新增元素并输出新版本号；2 version l r 输出 [l,r) 内 count 和 sum；3 before after key 输出版本差中 >=key 的首位置或-1，4 before after key 输出版本差中严格 <key 的末位置或-1；操作 3/4 要求逐位置计数差非负。旧版本不变，允许分叉和负权。本地 API 例子，不是 OJ 判定。

所需模板：PersistentWeightedIndex（第 558 页，37.1.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/WIDA-SHIFT-SPLAY.md

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    PersistentWeightedIndex tree(n);
    while (q--)
    {
        int op, version;
        cin >> op >> version;
        if (op == 1)
        {
            int pos;
            long long weight;
            cin >> pos >> weight;
            cout << tree.add(version, pos, weight) << '\n';
        }
        else if (op == 2)
        {
            int l, r;
            cin >> l >> r;
            auto [count, sum] = tree.query(version, l, r);
            cout << count << ' ' << sum << '\n';
        }
        else
        {
            int after, key;
            cin >> after >> key;
            if (op == 3)
                cout << tree.first(version, after, key) << '\n';
            else
                cout << tree.last(version, after, key) << '\n';
        }
    }
}
```

模板题使用：Library Checker Rectangle Sum

题意：N 个带权点，Q 次查询 [l,r)×[d,u) 内权值和；1<=N,Q<=200000，坐标及权值在 [0,1e9]。按 x 排序建立前缀版本，y 离散化；两个版本相减，重复点与零权合法，答案用 signed64。预处理 O(NlogN)，每次查询 O(logN)，空间 O(NlogN)。Library Checker C++20 线上 AC 410009，1072ms、101.38MiB。

所需模板：PersistentWeightedIndex（第 558 页，37.1.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/rectangle_sum

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<array<int, 3>> a(n);
    vector<int> xs, ys;
    for (auto &[x, y, w] : a)
    {
        cin >> x >> y >> w;
        ys.push_back(y);
    }
    sort(a.begin(), a.end());
    sort(ys.begin(), ys.end());
    ys.erase(unique(ys.begin(), ys.end()), ys.end());
    auto rank = [&](int y)
    {
        return lower_bound(ys.begin(), ys.end(), y) - ys.begin();
    };
    PersistentWeightedIndex tree(ys.size());
    for (int i = 0; i < n; i++)
    {
        auto [x, y, w] = a[i];
        xs.push_back(x);
        tree.add(i, rank(y), w);
    }
    while (q--)
    {
        int l, d, r, u;
        cin >> l >> d >> r >> u;
        int before = lower_bound(xs.begin(), xs.end(), l) - xs.begin();
        int after = lower_bound(xs.begin(), xs.end(), r) - xs.begin();
        long long answer = tree.query(after, rank(d), rank(u)).second;
        answer -= tree.query(before, rank(d), rank(u)).second;
        cout << answer << '\n';
    }
}
```

相关应用：按原始整数奇偶维护历史重数及专用降序选择，不代表整页覆盖

37.1.7 持久化奇偶计数与降序选择

PersistentParityIndex(n)，原始整数位置域 [0,n)，n 为正 int，版本 0 为空。add(version,x,delta) 从任意版本分叉，位置 x 的重数加 signed64 增量 delta，返回新版本号；要求结果重数非负、版本总重数 <=LLONG_MAX。delta=0 也创建版本但不分配节点。奇偶性按原始 x 判断，不能直接把数值离散化后的编号当原值。

difference(before,after) 返回 {总重数差, 奇数位置重数差}，允许有符号差。以下两个查询要求每个位置 after-before 重数非负；有删除操作时，即使祖先与后代版本也不自动满足。stop(before,after,k) 从大到小删除至多 k 个偶数元素，遇到奇数立即停止，返回剩余最大位置，无元素返回-1，查询不改版本。odd_kth(before,after,k) 只按奇数位置元素降序选择 0-based 第 k 项，返回 {位置, 该位置总重数, 在该位置内的 0-based 偏移}；不存在返回 {-1,0,k-奇数总重数}。k 为非负 signed64，位置 0 合法。

每次非零更新 O(log(n+1)) 时间和新增节点；difference 为 O(1)，其余查询 O(log(n+1)) 且不分配节点。总空间 O(版本数 + 更新数 log(n+1))，节点不回收，节点和版本编号须适合 int；递归深度 O(log(n+1))。t/roots 及递归辅助函数仅供内部实现；完整对象复制独立。原稿无结果的 0 哨兵由驱动按需映射，不能把 0 当本接口的空结果。

```
struct PersistentParityIndex
{
    using ll = long long;
    struct Node
    {
        int left = 0, right = 0;
        ll count = 0, odd = 0;
    };
    int n;
    vector<Node> t{Node{}};
    vector<int> roots{0};

    PersistentParityIndex(int n) : n(n)
    {
        assert(n > 0);
    }

    int update(int p, int l, int r, int x, ll delta)
    {
        assert(t.size() < INT_MAX);
        int u = t.size();
        Node node = t[p];
        t.push_back(node);
        if (r - l == 1)
        {
            assert(delta >= -node.count);
            t[u].count += delta;
            t[u].odd = (x % 2) * t[u].count;
            return u;
        }
        int m = l + (r - l) / 2;
        if (x < m)
            t[u].left = update(node.left, l, m, x, delta);
        else
            t[u].right = update(node.right, m, r, x, delta);
        t[u].count = t[t[u].left].count + t[t[u].right].count;
        t[u].odd = t[t[u].left].odd + t[t[u].right].odd;
        return u;
    }

    int add(int version, int x, ll delta)
    {
        assert(0 <= version && version < (int)roots.size());
        assert(0 <= x && x < n && roots.size() < INT_MAX);
        int p = roots[version];
```

```
        assert(delta <= 0 || t[p].count <= LLONG_MAX - delta);
        int root = delta == 0 ? p : update(p, 0, n, x, delta);
        roots.push_back(root);
        return (int)roots.size() - 1;
    }

    array<ll, 2> difference(int before, int after) const
    {
        assert(0 <= before && before < (int)roots.size());
        assert(0 <= after && after < (int)roots.size());
        auto a = t[roots[before]], b = t[roots[after]];
        return {b.count - a.count, b.odd - a.odd};
    }

    int stop_node(int a, int b, int l, int r, ll k) const
    {
        if (r - l == 1)
            return l;
        int m = l + (r - l) / 2;
        int x = t[a].right, y = t[b].right;
        ll count = t[y].count - t[x].count;
        ll odd = t[y].odd - t[x].odd;
        if (odd > 0 || count > k)
            return stop_node(x, y, m, r, k);
        return stop_node(t[a].left, t[b].left, l, m, k - count);
    }

    // Queries require nonnegative after-before counts at every position.
    // Remove up to k largest even elements, stopping at the first odd element.
    int stop(int before, int after, ll k) const
    {
        assert(k >= 0);
        auto [count, odd] = difference(before, after);
        if (odd == 0 && count <= k)
            return -1;
        return stop_node(roots[before], roots[after], 0, n, k);
    }

    array<ll, 3> odd_node(int a, int b, int l, int r, ll k) const
    {
        if (r - l == 1)
            return {l, t[b].odd - t[a].odd, k};
        int m = l + (r - l) / 2;
        int x = t[a].right, y = t[b].right;
        ll count = t[y].odd - t[x].odd;
        if (count > k)
            return odd_node(x, y, m, r, k);
        return odd_node(t[a].left, t[b].left, l, m, k - count);
    }
```

```
95
96      // Descending zero-based rank; result is {position, multiplicity, offset}.
97      array<ll, 3> odd_kth(int before, int after, ll k) const
98      {
99          assert(k >= 0);
100         auto [count, odd] = difference(before, after);
101         if (k >= odd)
102             return {-1, 0, k - odd};
103         return odd_node(roots[before], roots[after], 0, n, k);
104     }
105 };
```

接口用法演示：持久化奇偶计数与降序选择 API

演示约定：读 n q，初始版本 0。1 v x delta 在 v 上增减位置 x 重数，输出新版本号；2 before after k 输出 stop；3 before after k 输出 odd_kth 三元组；4 before after 输出总重数差与奇数重数差。操作 2/3 要求逐位置版本差非负，k>=0；原值域 [0,n)，下标 0-based，重数与 k 使用 signed64。自定义 API 协议，非 OJ 判定。

所需模板：PersistentParityIndex（第 562 页，37.1.7 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    PersistentParityIndex tree(n);
    while (q--)
    {
        int op, a, b;
        cin >> op >> a >> b;
        if (op == 1)
        {
            long long delta;
            cin >> delta;
            cout << tree.add(a, b, delta) << '\n';
        }
        else if (op == 2)
        {
            long long k;
            cin >> k;
            cout << tree.stop(a, b, k) << '\n';
        }
        else if (op == 3)
        {
            long long k;
            cin >> k;
            auto result = tree.odd_kth(a, b, k);
            cout << result[0] << ' ' << result[1] << ' ' << result[2] << '\n';
        }
        else
        {
            auto result = tree.difference(a, b);
            cout << result[0] << ' ' << result[1] << '\n';
        }
    }
}
```

37.1.8 可持久化并查集

PersistentDSU(n)，n>0，点号 0..n-1，版本 0 为单点集合。merge(ver,x,y) 从任意版本合并并返回新版本号，已连通也创建版本；copy(ver) 复制版本。find、same、size 只读，不压缩路径；按大小合并，等大时第一根挂到第二根。区别于普通 dsu 的固定方向合并。稀疏可持久化线段树的叶子存父亲或负大小，空节点表示-1。初始化 O(1)，copy 均摊 O(1)，其余 $O(\log^2(n+1))$；成功合并增加 O(log(n+1)) 节点。节点数/版本数须可用 int 索引。root、t 只读，get/set 内部函数勿直接调用；旧版本永不被改写。

```
struct PersistentDSU
{
    struct Node
    {
        int l = 0, r = 0, v = -1;
    };

    int n;
    vector<Node> t{Node{}};
    vector<int> root{0};

    PersistentDSU(int n) : n(n)
    {
        assert(n > 0);
    }

    int get(int p, int l, int r, int x) const
    {
        if (!p) return -1;
        if (r - l == 1) return t[p].v;
        int m = l + (r - l) / 2;
        if (x < m) return get(t[p].l, l, m, x);
        return get(t[p].r, m, r, x);
    }

    int set(int old, int l, int r, int x, int v)
    {
        Node node = t[old];
        int p = t.size();
        t.push_back(node);
        if (r - l == 1)
            t[p].v = v;
        else
        {
            int m = l + (r - l) / 2;
            if (x < m)
            {
                int q = set(node.l, l, m, x, v);
                t[p].l = q;
            }
            else
            {
                int q = set(node.r, m, r, x, v);
                t[p].r = q;
            }
        }
        return p;
    }
```

可持久化并查集（接上页同一结构体）：

```
    // Queries never compress paths or allocate nodes. Vertices are 0-based.
    int find(int ver, int x) const
    {
        assert(0 <= ver && ver < (int)root.size());
        assert(0 <= x && x < n);
        int p = get(root[ver], 0, n, x);
        while (p >= 0)
        {
            x = p;
            p = get(root[ver], 0, n, x);
        }
        return x;
    }

    int size(int ver, int x) const
    {
        return -get(root[ver], 0, n, find(ver, x));
    }

    bool same(int ver, int x, int y) const
    {
        return find(ver, x) == find(ver, y);
    }

    int copy(int ver)
    {
        assert(0 <= ver && ver < (int)root.size());
        root.push_back(root[ver]);
        return (int)root.size() - 1;
    }

    // Always create one version; ties attach the first root to the second.
    int merge(int ver, int x, int y)
    {
        x = find(ver, x);
        y = find(ver, y);
        if (x == y) return copy(ver);
        int a = get(root[ver], 0, n, x);
        int b = get(root[ver], 0, n, y);
        if (a < b)
        {
            swap(x, y);
            swap(a, b);
        }
        int p = set(root[ver], 0, n, x, y);
        p = set(p, 0, n, y, a + b);
        root.push_back(p);
        return (int)root.size() - 1;
    }
};
```

模板题使用：Luogu P3402 / 可持久化并查集

题意：n≤100000，m≤200000。1 a b 合并；2 k 回到第 k 次操作后的状态，可回到 0；3 a b 输出是否连通。每种操作均产生版本，操作号等于版本号，点号转 0-based，无异或。

所需模板：PersistentDSU（第 567 页，37.1.8 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3402

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    PersistentDSU d(n);
    for (int i = 1; i <= m; i++)
    {
        int op, x, y;
        cin >> op >> x;
        if (op == 2)
            d.copy(x);
        else
        {
            cin >> y;
            if (op == 1)
                d.merge(i - 1, x - 1, y - 1);
            else
            {
                cout << d.same(i - 1, x - 1, y - 1) << '\n';
                d.copy(i - 1);
            }
        }
    }
}
```

37.2 可持久化平衡树

37.2.1 可持久化有序 Treap：分叉版本与顺序统计

版本 0 为空。insert(v,x)、erase(v,x)、copy(v) 追加并返回新版本；删除不存在的值也追加相同集合的版本。size/less/rank/kth/prev/next 均显式传版本且只读，不创建版本或树节点。重复值计数，rank 为严格小于 x 的副本数加一；kth 为 1-based 且要求合法，严格前后继缺失返回 nullopt。支持 long long 全范围，不用 x±1。随机优先级下期望 O(log(d+1))，d 为版本不同键数；copy 摊还 O(1)，没有最坏对数保证。递归复制更新路径，节点不回收，空间按全部历史分配节点与版本数计。节点/版本/各版副本数须小于 INT_MAX；t、root 及带节点编号的辅助函数属内部状态。当前常见 64 位布局每节点 32 字节；500000 次随机不同键插入实测约 1200 万节点，不预留的 vector 扩容可能超过 512MiB 峰值。预留 26*n+1 仅是明确预算而非上界，先核对内存限制；重新构造清空全部版本。

```
struct PersistentOrderedTreap
{
    using ll = long long;

    struct Node
    {
        ll val = 0;
        unsigned long long pri = 0;
        int l = 0, r = 0, siz = 0, cnt = 0;
    };

    vector<Node> t{Node{}};
    vector<int> root{0};
    mt19937_64 rng;

    PersistentOrderedTreap(unsigned long long seed = 712367821) : rng(seed) {}

    void check(int v) const { assert(0 <= v && v < (int)root.size()); }

    int clone(int p)
    {
        Node node = t[p];
        t.push_back(node);
        return (int)t.size() - 1;
    }

    void pull(int p) { t[p].siz = t[t[p].l].siz + t[t[p].r].siz + t[p].cnt; }
```

复制路径插入与旋转（接上页同一结构体）：

```
    // Only rotate freshly copied nodes; the promoted child was copied by add.
    int add(int p, ll x)
    {
        if (!p)
        {
            t.push_back({x, rng(), 0, 0, 1, 1});
            return (int)t.size() - 1;
        }
        int u = clone(p);
        if (x == t[u].val)
            t[u].cnt++;
        else if (x < t[u].val)
        {
            int q = add(t[u].l, x);
            t[u].l = q;
            if (t[q].pri > t[u].pri)
            {
                t[u].l = t[q].r;
                t[q].r = u;
                pull(u);
                u = q;
            }
        }
        else
        {
            int q = add(t[u].r, x);
            t[u].r = q;
            if (t[q].pri > t[u].pri)
            {
                t[u].r = t[q].l;
                t[q].l = u;
                pull(u);
                u = q;
            }
        }
        pull(u);
        return u;
    }
```

复制路径合并与删除（接上页同一结构体）：

```
    int merge(int l, int r)
    {
        if (!l || !r) return l ? l : r;
        if (t[l].pri > t[r].pri)
        {
            int u = clone(l);
            int q = merge(t[u].r, r);
            t[u].r = q;
            pull(u);
            return u;
        }
        int u = clone(r);
        int q = merge(l, t[u].l);
        t[u].l = q;
        pull(u);
        return u;
    }

    int remove(int p, ll x)
    {
        if (!p) return 0;
        if (x == t[p].val)
        {
            if (t[p].cnt == 1) return merge(t[p].l, t[p].r);
            int u = clone(p);
            t[u].cnt--;
            pull(u);
            return u;
        }
        bool left = x < t[p].val;
        int old = left ? t[p].l : t[p].r;
        int q = remove(old, x);
        if (q == old) return p;
        int u = clone(p);
        if (left)
            t[u].l = q;
        else
            t[u].r = q;
        pull(u);
        return u;
    }
```

版本接口、大小与排名（接上页同一结构体）：

```
    // Updates append versions; erase removes one occurrence, or changes nothing.
    int insert(int v, ll x)
    {
        check(v);
        int p = add(root[v], x);
        root.push_back(p);
        return (int)root.size() - 1;
    }

    int erase(int v, ll x)
    {
        check(v);
        int p = remove(root[v], x);
        root.push_back(p);
        return (int)root.size() - 1;
    }

    int copy(int v)
    {
        check(v);
        root.push_back(root[v]);
        return (int)root.size() - 1;
    }

    int size(int v) const
    {
        check(v);
        return t[root[v]].siz;
    }

    int less(int v, ll x, bool equal = false) const
    {
        check(v);
        int p = root[v], ans = 0;
        while (p)
        {
            if (t[p].val < x || (equal && t[p].val == x))
            {
                ans += t[t[p].l].siz + t[p].cnt;
                p = t[p].r;
            }
            else
                p = t[p].l;
        }
        return ans;
    }

    int rank(int v, ll x) const { return less(v, x) + 1; }
```

第 k 小与严格前驱后继（接上页同一结构体）：

```
    // Ranks and k are 1-based. No sentinel values are reserved.
    ll kth(int v, int k) const
    {
        assert(1 <= k && k <= size(v));
        int p = root[v];
        while (true)
        {
            int left = t[t[p].l].siz;
            if (k <= left)
                p = t[p].l;
            else if (k <= left + t[p].cnt)
                return t[p].val;
            else
            {
                k -= left + t[p].cnt;
                p = t[p].r;
            }
        }
    }

    optional<ll> prev(int v, ll x) const
    {
        int k = less(v, x);
        if (!k) return nullopt;
        return kth(v, k);
    }

    optional<ll> next(int v, ll x) const
    {
        int k = less(v, x, true);
        if (k == size(v)) return nullopt;
        return kth(v, k + 1);
    }
};
```

模板题使用：Luogu P3835 【模板】可持久化平衡树

题意：维护可重整数集合的历史版本；每条插入、删除或排名/第 k 小/严格前后继查询都从指定旧版本生成版本 i。删除缺失值忽略但仍产生版本；查询输出后 copy(v)，避免版本编号错位。第 k 小保证合法；缺失前后继分别输出-2147483647、2147483647，仅由驱动映射 optional。n≤500000、键绝对值 ≤10^9。reserve(26*n+1) 是约 32 字节节点的预留预算，不是分配上界；最大 n 预留约 397MiB，需先核对内存限制，不能将本地通过当作线上 AC。

所需模板：PersistentOrderedTreap（第 570 页，37.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3835

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    PersistentOrderedTreap tree;
    tree.root.reserve(n + 1);
    tree.t.reserve(26ULL * n + 1);
    for (int i = 1; i <= n; i++)
    {
        int v, op;
        long long x;
        cin >> v >> op >> x;
        if (op == 1)
            tree.insert(v, x);
        else if (op == 2)
            tree.erase(v, x);
        else
        {
            if (op == 3) cout << tree.rank(v, x) << '\n';
            if (op == 4) cout << tree.kth(v, (int)x) << '\n';
            if (op == 5) cout << tree.prev(v, x).value_or(-2147483647LL) << '\n';
            if (op == 6) cout << tree.next(v, x).value_or(2147483647LL) << '\n';
            tree.copy(v);
        }
    }
}
```

37.3 可持久化字典树

37.3.1 前缀可持久化 01 字典树

PersistentXorTrie(bits,cap)，1≤bits≤64，0≤cap≤(INT_MAX−1)/(bits+1)，构造时预留最多 cap 个键。append(x) 追加一个 uint64_t 原值，重复值按出现次数保留；满容量或 x 不满足位宽时返回 false 且逻辑状态不变。size() 为键数；max_xor(l,r,x) 从 0-based 半开区间 [l,r) 选择一个键 y，返回最大 x xor y，空区间 nullopt。查询 x 允许完整 uint64；不是子集异或、任意分支版本或字符串 Trie。构造 O(cap*bits) 容量空间，追加/查询 O(bits)；复制后首次追加为恢复预留空间可花 O(size*bits)。append 分配失败传播异常，逻辑内容不变；构造参数非法抛异常，查询下标是调用者前置条件，assert 只作调试检查。内部成员只读；不要复用已移动对象。

```
struct PersistentXorTrie
{
    using U = uint64_t;
    struct Node
    {
        int ch[2] = {}, cnt = 0;
    };
    int bits, cap;
    vector<Node> t{Node{}};
    vector<int> root{0};

    PersistentXorTrie(int bits, int cap) : bits(bits), cap(cap)
    {
        if (bits < 1 || bits > 64 || cap < 0)
            throw invalid_argument("PersistentXorTrie domain");
        if (cap > (INT_MAX - 1) / (bits + 1))
            throw length_error("PersistentXorTrie capacity");
        reserve();
    }

    void reserve()
    {
        t.reserve(1 + size_t(cap) * (bits + 1));
        root.reserve(size_t(cap) + 1);
    }

    int size() const { return (int)root.size() - 1; }
```

前缀追加与区间单值最大异或（接上页同一结构体）：

```
    bool append(U x)
    {
        if (size() == cap || (bits < 64 && (x >> bits))) return false;
        reserve(); // A copied vector need not preserve its original capacity.
        int old = root.back(), p = t.size();
        Node copy = t[old];
        t.push_back(copy);
        root.push_back(p);
        t[p].cnt++;
        for (int d = bits - 1; d >= 0; d--)
        {
            int b = x >> d & 1, q = t.size();
            old = t[old].ch[b];
            copy = t[old];
            t.push_back(copy);
            t[p].ch[b] = q;
            p = q;
            t[p].cnt++;
        }
        return true;
    }

    optional<U> max_xor(int l, int r, U x) const
    {
        assert(0 <= l && l <= r && r <= size());
        if (l == r) return nullopt;
        int a = root[l], b = root[r];
        U y = 0;
        for (int d = bits - 1; d >= 0; d--)
        {
            int k = (x >> d & 1) ^ 1;
            if (t[t[b].ch[k]].cnt == t[t[a].ch[k]].cnt) k ^= 1;
            y |= U(k) << d;
            a = t[a].ch[k];
            b = t[b].ch[k];
        }
        return x ^ y;
    }
};
```

应用题补充：Luogu P4735 最大异或和

题意：N,M≤300000，序列元素 0..10^7；追加 A x 或查询 Q l r x，1≤l≤r≤ 当前 N。令 S 为前缀异或，答案为 (S_N xor x) 与 S_[l−1,r−1] 中一个数异或的最大值。先插 S0，再插后续前缀，调用 max_xor(l−1,r,S_N xor x)。前缀可能超过 10^7 但小于 2^24；库储存 24 位，查询 x 按 uint64_t 读取并保留高位，题面未单列查询 x 上界。此为组合应用；最多 600001 个键，12 字节节点平台数组负载约 174MiB，不含进程开销，不声称通过 OJ 时间限制。

所需模板：PersistentXorTrie（第 576 页，37.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4735

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    PersistentXorTrie trie(24, n + m + 1);
    uint64_t sum = 0, x;
    trie.append(0);
    for (int i = 0; i < n; i++)
    {
        cin >> x;
        sum ^= x;
        trie.append(sum);
    }
    while (m--)
    {
        char op;
        cin >> op;
        if (op == 'A')
        {
            cin >> x;
            sum ^= x;
            trie.append(sum);
        }
        else
        {
            int l, r;
            cin >> l >> r >> x;
            cout << *trie.max_xor(l - 1, r, sum ^ x) << '\n';
        }
    }
}
```

第三十八章　树套树

38.1　树状数组套权值线段树

38.1.1　动态区间第 k 小（树状数组套权值线段树）

DynamicKth(a,future) 传入非空 0-based 初值，future 包含所有将来可能赋入的新值；内部自动合并初值并离散化。set(pos,value) 单点赋值，kth(l,r,k) 查询闭区间第 k 小，下标均为 1-based；1≤k≤r-l+1。允许重复值与完整 long long 权值；新值必须已在离散化集合内，操作可按原顺序执行，不保留历史版本。每个树状数组节点对应一棵可变权值线段树；删除旧值后插入新值。空节点回收到 free_head 链，重复修改不会按操作次数累计泄漏节点。设 D 为不同值数，S=n+future.size()，预处理 O(S log S+n log n log(D+1))，单次修改及查询 O(log n log(D+1))。节点数上界 B=(ceil(log2(D))+1) · $\sum$ lowbit(i)，求和 i=1..n；空间 O(S+n+B)。update、modify、id 和节点池均为内部接口；不能手动减去不存在的值。

```
struct DynamicKth
{
    using ll = long long;

    struct Node
    {
        int l = 0, r = 0, sum = 0;
    };

    int n, free_head = 0;
    vector<int> bit, rank;
    vector<ll> vals;
    vector<Node> t{Node{}};

    DynamicKth(const vector<ll> &a, vector<ll> future = {})
        : n(a.size()), bit(n + 1), rank(n + 1)
    {
        assert(n > 0);
        vals = future;
        vals.insert(vals.end(), a.begin(), a.end());
        sort(vals.begin(), vals.end());
        vals.erase(unique(vals.begin(), vals.end()), vals.end());
        for (int i = 1; i <= n; i++)
        {
            rank[i] = id(a[i - 1]);
            modify(i, rank[i], 1);
        }
    }

    int id(ll value) const
    {
        int p = lower_bound(vals.begin(), vals.end(), value) - vals.begin();
        assert(p < (int)vals.size() && vals[p] == value);
        return p;
    }

    int new_node()
    {
```

```
        if (!free_head)
        {
            t.push_back(Node{});
            return (int)t.size() - 1;
        }
        int p = free_head;
        free_head = t[p].l;
        t[p] = Node{};
        return p;
    }

    int update(int p, int l, int r, int x, int delta)
    {
        if (!p) p = new_node();
        t[p].sum += delta;
        if (l < r)
        {
            int m = (l + r) / 2;
            if (x <= m)
            {
                int child = update(t[p].l, l, m, x, delta);
                t[p].l = child;
            }
            else
            {
                int child = update(t[p].r, m + 1, r, x, delta);
                t[p].r = child;
            }
        }
        if (!t[p].sum)
        {
            t[p].l = free_head;
            free_head = p;
            return 0;
        }
        return p;
    }

    void modify(int pos, int x, int delta)
    {
        for (int i = pos; i <= n; i += i & -i)
            bit[i] = update(bit[i], 0, (int)vals.size() - 1, x, delta);
    }
```

```
    void set(int pos, ll value)
    {
        assert(1 <= pos && pos <= n);
        int x = id(value);
        if (rank[pos] == x) return;
        modify(pos, rank[pos], -1);
        modify(pos, x, 1);
        rank[pos] = x;
    }

    ll kth(int l, int r, int k) const
    {
        assert(1 <= l && l <= r && r <= n && 1 <= k && k <= r - l + 1);
        vector<int> a, b;
        for (int i = r; i; i -= i & -i) a.push_back(bit[i]);
        for (int i = l - 1; i; i -= i & -i) b.push_back(bit[i]);
        int L = 0, R = (int)vals.size() - 1;
        while (L < R)
        {
            int count = 0;
            for (int p : a) count += t[t[p].l].sum;
            for (int p : b) count -= t[t[p].l].sum;
            bool left = k <= count;
            if (!left) k -= count;
            for (int &p : a) p = left ? t[p].l : t[p].r;
            for (int &p : b) p = left ? t[p].l : t[p].r;
            int m = (L + R) / 2;
            if (left)
                R = m;
            else
                L = m + 1;
        }
        return vals[L];
    }
};
```

模板题使用：Luogu P2617

题意：支持单点赋值及闭区间第 k 小。先读完操作，把所有修改的新值收集到 future，再构造并按原顺序执行；初值 vector 为 0-based，位置、查询区间和 k 为 1-based。Query 仅保存输入字段。

所需模板：DynamicKth（第 579 页，38.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2617

```
int main()
{
    struct Query
    {
        char op;
        int x, y, k;
        long long value;
    };

    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    std::vector<long long> a(n), future;
    for (auto &x : a) std::cin >> x;
    std::vector<Query> queries(m);
    for (auto &q : queries)
    {
        std::cin >> q.op;
        if (q.op == 'Q')
            std::cin >> q.x >> q.y >> q.k;
        else
        {
            std::cin >> q.x >> q.value;
            future.push_back(q.value);
        }
    }
    DynamicKth tree(a, future);
    for (const auto &q : queries)
    {
        if (q.op == 'Q')
            std::cout << tree.kth(q.x, q.y, q.k) << '\n';
        else
            tree.set(q.x, q.value);
    }
}
```

第三十九章　K-D Tree

39.1　动态二维 K-D Tree：单点累加与矩形和

KDTreeSum<Coord=int>(reserve_points=0) 初始为空；Coord 取 int 或 long long。add(x,y,delta) 累加有符号 long long 权值，相同坐标只占一个节点；零权节点仍保留，不支持删除坐标。query(x1,y1,x2,y2) 查询闭矩形和，左下端超过右上端则返回 0。坐标只比较，不相减，不用有限哨兵；可用所选整数类型极值。当前各点绝对权值总和及更新中间量须在 long long 内。不同点数 d、预留容量和节点编号须小于 INT_MAX。按轴交替分割，子树过重时递归收集编号并原位重建；树高 O(log(d+1))。nth_element 按平均线性选择计，插入摊还 $O(\log^2(d+1))$，单次大重建可 O(d log(d+1))；矩形查询最坏 O(d)，不能保证对数。空间 O(d)，预留容量另计，默认 int 节点常见为 56 字节；紧内存题预留最大不同点数，避免扩容峰值。a/root 及带节点编号的函数均为内部状态；可重新赋值空对象重置。

```
template <class Coord = int> struct KDTreeSum
{
    using ll = long long;
    using Point = array<Coord, 2>;

    struct Node
    {
        Point point{}, low{}, high{};
        array<int, 2> child{};
        int size = 1;
        ll value = 0, sum = 0;
    };

    vector<Node> a;
    int root = 0;

    KDTreeSum(int reserve_points = 0)
    {
        a.reserve(reserve_points + 1);
        a.push_back(Node{});
        a[0].size = 0;
    }

    bool less(Point x, Point y, int axis) const
    {
        if (x[axis] != y[axis]) return x[axis] < y[axis];
        return x[axis ^ 1] < y[axis ^ 1];
    }
```

包围盒维护、递归收集与重建（接上页同一结构体）：

```
    void pull(int p)
    {
        a[p].low = a[p].high = a[p].point;
        a[p].size = 1;
        a[p].sum = a[p].value;
        for (int q : a[p].child)
        {
            if (!q) continue;
            a[p].size += a[q].size;
            a[p].sum += a[q].sum;
            for (int d = 0; d < 2; d++)
            {
                a[p].low[d] = min(a[p].low[d], a[q].low[d]);
                a[p].high[d] = max(a[p].high[d], a[q].high[d]);
            }
        }
    }

    void collect(int p, vector<int> &v)
    {
        if (!p) return;
        collect(a[p].child[0], v);
        v.push_back(p);
        collect(a[p].child[1], v);
    }

    int build(vector<int> &v, int l, int r, int axis)
    {
        if (l == r) return 0;
        int m = l + (r - l) / 2;
        nth_element(v.begin() + l,
                    v.begin() + m,
                    v.begin() + r,
                    [&](int x, int y)
                    {
                        return less(a[x].point, a[y].point, axis);
                    });
        int p = v[m];
        a[p].child[0] = build(v, l, m, axis ^ 1);
        a[p].child[1] = build(v, m + 1, r, axis ^ 1);
        pull(p);
        return p;
    }
```

点权累加与闭矩形查询（接上页同一结构体）：

```
    int add(int p, Point point, ll delta, int axis)
    {
        if (!p)
        {
            p = a.size();
            a.push_back(Node{});
            a[p].point = a[p].low = a[p].high = point;
            a[p].value = a[p].sum = delta;
            return p;
        }
        if (a[p].point == point)
            a[p].value += delta;
        else
        {
            int side = !less(point, a[p].point, axis);
            a[p].child[side] = add(a[p].child[side], point, delta, axis ^ 1);
        }
        pull(p);
        int heavy = max(a[a[p].child[0]].size, a[a[p].child[1]].size);
        if (4LL * heavy <= 3LL * a[p].size) return p;
        vector<int> v;
        v.reserve(a[p].size);
        collect(p, v);
        return build(v, 0, v.size(), axis);
    }

    void add(Coord x, Coord y, ll delta)
    {
        root = add(root, {x, y}, delta, 0);
    }

    ll query(int p, Point low, Point high) const
    {
        if (!p) return 0;
        bool covered = true, inside = true;
        for (int d = 0; d < 2; d++)
        {
            if (a[p].high[d] < low[d] || high[d] < a[p].low[d]) return 0;
            covered &= low[d] <= a[p].low[d] && a[p].high[d] <= high[d];
            inside &= low[d] <= a[p].point[d] && a[p].point[d] <= high[d];
        }
        if (covered) return a[p].sum;
        ll answer = inside ? a[p].value : 0;
        for (int q : a[p].child) answer += query(q, low, high);
        return answer;
    }

    ll query(Coord x1, Coord y1, Coord x2, Coord y2) const
    {
        if (x1 > x2 || y1 > y2) return 0;
        return query(root, {x1, y1}, {x2, y2});
    }
};
```

应用题补充：Luogu P4148 简单题

题意：在线单点累加与闭矩形求和，每条命令除操作码外都异或上次答案，3 结束。答案保证在 int 范围内，因此 last 使用 int。预留 200000 个不同点，减少 vector 扩容峰值；本地验证不等于通过该题 20 MB 平台内存限制。

所需模板：KDTreeSum（第 583 页，39.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4148

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, op, last = 0;
    cin >> n;
    KDTreeSum<> s(200000);
    while (cin >> op && op != 3)
    {
        int x, y;
        cin >> x >> y;
        x ^= last;
        y ^= last;
        if (op == 1)
        {
            int value;
            cin >> value;
            value ^= last;
            s.add(x, y, value);
        }
        else
        {
            int x2, y2;
            cin >> x2 >> y2;
            x2 ^= last;
            y2 ^= last;
            last = s.query(x, y, x2, y2);
            cout << last << '\n';
        }
    }
}
```

接口用法演示：动态二维偏序计数（WIDA 接口适配）

演示约定：输入操作数 m。操作 1 x y 插入一个二维整数点，允许重复；操作 2 x y 统计已经插入且两坐标分别 ≤x、≤y 的点数。每次插入权值 1，查询从 INT_MIN 到 (x,y) 的闭矩形，边界坐标也计入，不对极小值减 1。

所需模板：KDTreeSum（第 583 页，39.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/ds/kdt/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    cin >> m;
    KDTreeSum<> tree(m);
    while (m--)
    {
        int op, x, y;
        cin >> op >> x >> y;
        if (op == 1) tree.add(x, y, 1);
        else cout << tree.query(INT_MIN, INT_MIN, x, y) << '\n';
    }
}
```

39.2 静态 K-D Tree 最近与最远点

KDNearest<D>(points)，D>0，Point=array<int,D>，坐标为有符号 32 位整数，输入编号 0..n-1 且允许重合点。query(q,k,far=false) 返回最近 k 个原编号；far=true 改为最远 k 个。结果按距离优先、等距编号小者优先排列，0≤k≤n；k=0 返回空，包含与查询重合的点，不自动排除自身。平方距离用 128 位整数精确计算。p 保留输入顺序；p/a/root 仅供读取，重建请重新赋值。递归中位数建树，平均 O(Dn log(n+1))，空间 O(Dn)；包围盒剪枝没有对任意数据的次线性保证，单次最坏 O(Dn+n log(k+1))，额外 O(k+log(n+1)) 空间。search/build/bound 为内部辅助，不直接调用。

```
template <int D = 2> struct KDNearest
{
    static_assert(D > 0);
    using Point = array<int, D>;
    using Wide = __int128_t;
    using Key = pair<Wide, int>;

    struct Node
    {
        Point low, high;
        array<int, 2> child{-1, -1};
        int first;
    };

    vector<Point> p;
    vector<Node> a;
    int root;

    KDNearest(const vector<Point> &points = {}) : p(points), a(p.size())
    {
        vector<int> order(p.size());
        iota(order.begin(), order.end(), 0);
        root = build(order, 0, order.size(), 0);
    }
```

中位数建树与距离界（接上页同一结构体）：

```
    int build(vector<int> &order, int l, int r, int axis)
    {
        if (l == r) return -1;
        int m = (l + r) / 2;
        nth_element(order.begin() + l, order.begin() + m, order.begin() + r,
            [&](int x, int y)
            {
                return pair(p[x][axis], x) < pair(p[y][axis], y);
            });
        int u = order[m];
        a[u].low = a[u].high = p[u];
        a[u].first = u;
        a[u].child[0] = build(order, l, m, (axis + 1) % D);
        a[u].child[1] = build(order, m + 1, r, (axis + 1) % D);
        for (int v : a[u].child)
        {
            if (v == -1) continue;
            a[u].first = min(a[u].first, a[v].first);
            for (int d = 0; d < D; d++)
            {
                a[u].low[d] = min(a[u].low[d], a[v].low[d]);
                a[u].high[d] = max(a[u].high[d], a[v].high[d]);
            }
        }
        return u;
    }

    Wide distance(const Point &x, const Point &y) const
    {
        Wide ans = 0;
        for (int d = 0; d < D; d++)
        {
            Wide t = Wide(x[d]) - y[d];
            ans += t * t;
        }
        return ans;
    }

    Key bound(int u, const Point &q, bool far) const
    {
        Wide ans = 0;
        for (int d = 0; d < D; d++)
        {
            Wide l = Wide(a[u].low[d]) - q[d];
            Wide r = Wide(a[u].high[d]) - q[d];
            if (far) ans -= max(l * l, r * r);
            else if (l > 0) ans += l * l;
            else if (r < 0) ans += r * r;
        }
        return {ans, a[u].first};
    }
```

剪枝搜索与查询入口（接上页同一结构体）：

```
    void search(int u, const Point &q, int k, bool far,
                priority_queue<Key> &heap) const
    {
        if (u == -1) return;
        if ((int)heap.size() == k && bound(u, q, far) >= heap.top()) return;
        Wide dist = distance(p[u], q);
        Key cur{far ? -dist : dist, u};
        if ((int)heap.size() < k || cur < heap.top())
        {
            heap.push(cur);
            if ((int)heap.size() > k) heap.pop();
        }
        int x = a[u].child[0];
        int y = a[u].child[1];
        if (x == -1 || (y != -1 && bound(y, q, far) < bound(x, q, far)))
        {
            swap(x, y);
        }
        search(x, q, k, far, heap);
        search(y, q, k, far, heap);
    }

    vector<int> query(const Point &q, int k, bool far = false) const
    {
        assert(0 <= k && k <= (int)p.size());
        if (k == 0) return {};
        priority_queue<Key> heap;
        search(root, q, k, far, heap);
        vector<int> ans;
        while (!heap.empty())
        {
            ans.push_back(heap.top().second);
            heap.pop();
        }
        reverse(ans.begin(), ans.end());
        return ans;
    }
};
```

接口用法演示：多维最近 / 最远前 k 点（接口演示）

演示约定：输入 n,d,m（1≤d≤10），n 个 d 维整数点，随后 m 组查询坐标、k、far。far=0 查询最近 k 点，far=1 查询最远 k 点；0≤k≤n。每行输出从优到劣的 0-based 输入编号，等距编号小者优先。其余维补 0。固定维度时直接用 KDNearest<d> 以节省常数。

所需模板：KDNearest（第 588 页，39.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/ds/kdt/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, d, m;
    cin >> n >> d >> m;
    vector<KDNearest<10>::Point> p(n);
    for (auto &x : p)
    {
        for (int j = 0; j < d; j++) cin >> x[j];
    }
    KDNearest<10> tree(p);
    while (m--)
    {
        KDNearest<10>::Point q{};
        for (int j = 0; j < d; j++) cin >> q[j];
        int k, far;
        cin >> k >> far;
        auto ids = tree.query(q, k, far);
        for (int id : ids) cout << id << ' ';
        cout << '\n';
    }
}
```

应用题补充：Luogu P2093 / JZPFAR

题意：给 n 个互异二维整数点，每次查询 (px,py,k) 的第 k 远点；等距时编号小者算更远。点和查询坐标绝对值至 10^9，n 至 10^5，m 至 10^4，k 至 20 且不超过 n。原题有随机数据条件；本算法不承诺对任意输入次线性。输出 1-based 输入编号。

所需模板：KDNearest（第 588 页，39.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2093

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<KDNearest<>::Point> p(n);
    for (auto &x : p) cin >> x[0] >> x[1];
    KDNearest<> tree(p);
    int m;
    cin >> m;
    while (m--)
    {
        KDNearest<>::Point q;
        int k;
        cin >> q[0] >> q[1] >> k;
        auto ids = tree.query(q, k, true);
        cout << ids.back() + 1 << '\n';
    }
}
```

39.3 动态 K-D Tree 矩形取点与删除

KDRange(points={}), Item 含 Point p=array<long long,2> 和 int id。add({{x,y},id}) 新增一条记录; extract(low,high) 取出并删除闭矩形内所有记录, 返回 vector<Item> 且顺序不保证, 空矩形返回空。允许坐标和 id 重复, 每条插入独立计数; 原编号由调用者提供, 需要唯一编号时自行保证。返回后可改坐标再 add; 一次 extract 完成后再插入, 避免同轮重复移动。a[root].alive 为当前记录数, a/root/free 只读; 其余方法是内部辅助。坐标仅比较, 支持完整有符号 64 位; 活跃及物理节点数须小于 INT_MAX。递归维护物理节点 3/4 平衡, 删除做标记, 存活数少于物理数一半时清理根; 重建回收槽位。树高 O(log(n+1)), 其中 n 为操作前后最大存活数; 按 nth_element 平均线性选择计, 构建 O(n log(n+1)), 单次插入或取点保守上界 O(n $\log^2$(n+1)), 查询可能扫描全部点, 不能承诺对数; 空间 O(历史最大存活数), 空对象重新赋值可释放历史容量。

```
struct KDRange
{
    using Point = array<long long, 2>;

    struct Item
    {
        Point p;
        int id;
    };

    struct Node
    {
        Item item;
        Point low, high;
        array<int, 2> child{};
        int size = 0, alive = 0;
        bool present = false;
    };

    vector<Node> a{Node{}};
    vector<int> free;
    int root = 0;

    KDRange(vector<Item> points = {})
    {
        a.reserve(points.size() + 1);
        root = build(points, 0, points.size(), 0);
    }
```

节点复用与子树维护（接上页同一结构体）：

```
    bool less(const Item &x, const Item &y, int axis) const
    {
        if (x.p[axis] != y.p[axis]) return x.p[axis] < y.p[axis];
        if (x.p[axis ^ 1] != y.p[axis ^ 1]) return x.p[axis ^ 1] < y.p[axis ^ 1];
        return x.id < y.id;
    }

    int node(const Item &item)
    {
        int u;
        if (free.empty())
        {
            u = a.size();
            a.emplace_back();
        }
        else
        {
            u = free.back();
            free.pop_back();
        }
        a[u] = Node{};
        a[u].item = item;
        a[u].low = a[u].high = item.p;
        a[u].size = a[u].alive = 1;
        a[u].present = true;
        return u;
    }

    void pull(int u)
    {
        a[u].low = a[u].high = a[u].item.p;
        a[u].size = 1;
        a[u].alive = a[u].present;
        for (int v : a[u].child)
        {
            if (!v) continue;
            a[u].size += a[v].size;
            a[u].alive += a[v].alive;
            for (int d = 0; d < 2; d++)
            {
                a[u].low[d] = min(a[u].low[d], a[v].low[d]);
                a[u].high[d] = max(a[u].high[d], a[v].high[d]);
            }
        }
    }
```

建树、回收与插入（接上页同一结构体）：

```
    int build(vector<Item> &v, int l, int r, int axis)
    {
        if (l == r) return 0;
        int m = l + (r - l) / 2;
        nth_element(v.begin() + l, v.begin() + m, v.begin() + r,
            [&](const Item &x, const Item &y)
            {
                return less(x, y, axis);
            });
        int u = node(v[m]);
        a[u].child[0] = build(v, l, m, axis ^ 1);
        a[u].child[1] = build(v, m + 1, r, axis ^ 1);
        pull(u);
        return u;
    }

    void collect(int u, vector<Item> &v)
    {
        if (!u) return;
        collect(a[u].child[0], v);
        collect(a[u].child[1], v);
        if (a[u].present) v.push_back(a[u].item);
        free.push_back(u);
    }

    int rebuild(int u, int axis)
    {
        vector<Item> v;
        v.reserve(a[u].alive);
        collect(u, v);
        return build(v, 0, v.size(), axis);
    }

    int add(int u, const Item &item, int axis)
    {
        if (!u) return node(item);
        int side = !less(item, a[u].item, axis);
        a[u].child[side] = add(a[u].child[side], item, axis ^ 1);
        pull(u);
        int heavy = max(a[a[u].child[0]].size, a[a[u].child[1]].size);
        if (4LL * heavy > 3LL * a[u].size) u = rebuild(u, axis);
        return u;
    }

    void add(Item item)
    {
        root = add(root, item, 0);
    }
```

矩形取出与删除入口（接上页同一结构体）：

```
    void take(int u, Point low, Point high, vector<Item> &out)
    {
        if (!u || !a[u].alive) return;
        bool inside = true;
        for (int d = 0; d < 2; d++)
        {
            if (a[u].high[d] < low[d] || high[d] < a[u].low[d]) return;
            inside &= low[d] <= a[u].item.p[d] && a[u].item.p[d] <= high[d];
        }
        if (a[u].present && inside)
        {
            out.push_back(a[u].item);
            a[u].present = false;
        }
        for (int v : a[u].child) take(v, low, high, out);
        pull(u);
    }

    vector<Item> extract(Point low, Point high)
    {
        vector<Item> out;
        if (low[0] > high[0] || low[1] > high[1]) return out;
        take(root, low, high, out);
        if (2LL * a[root].alive < a[root].size) root = rebuild(root, 0);
        return out;
    }
};
```

接口用法演示：动态矩形取点与删除（接口演示）

演示约定：输入 n,m 和 n 条 (x,y,id)。操作 1 x y id 新增记录；操作 2 x1 y1 x2 y2 取出并删除闭矩形中的全部记录，输出数量和递增排序后的编号。允许坐标及编号重复，重复记录分别保留；空矩形输出 0。坐标为完整有符号 64 位整数。

所需模板：KDRange（第 593 页，39.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/ds/kdt/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<KDRange::Item> points(n);
    for (auto &v : points) cin >> v.p[0] >> v.p[1] >> v.id;
    KDRange tree(points);
    while (m--)
    {
        int op;
        cin >> op;
        if (op == 1)
        {
            KDRange::Item v;
            cin >> v.p[0] >> v.p[1] >> v.id;
            tree.add(v);
        }
        else
        {
            KDRange::Point low, high;
            cin >> low[0] >> low[1] >> high[0] >> high[1];
            auto removed = tree.extract(low, high);
            vector<int> ids;
            for (auto v : removed) ids.push_back(v.id);
            sort(ids.begin(), ids.end());
            cout << ids.size();
            for (int id : ids) cout << ' ' << id;
            cout << '\n';
        }
    }
}
```

接口用法演示：曼哈顿邻域同步搬移（kuangbin 源码模型）

演示约定：输入 n,q,W,H 及 n 个点；每次给 X,Y,E,a,b,c,d,f,g，把操作开始时曼哈顿距离 ≤E 的点同步改为 ((a*x+b*y+(id+1)*c)%W,(d*x+f*y+(id+1)*g)%H)。输入点在 [0,W)×[0,H)，W,H 为 1..10^9，系数和 E 为 0..10^9，中心坐标绝对值 ≤10^9，n≤10^5；编号为输入次序 0-based。最后按原序输出坐标。使用 (x+y,x-y) 转闭矩形；128 位乘加。只核对源码模型，不冒充 UVALive6045 原题协议；原文小于与源码闭边界存在差异，待原题确认。

所需模板：KDRange（第 593 页，39.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/kuangbin/ACM-ICPC

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    long long w, h;
    cin >> n >> q >> w >> h;
    vector<KDRange::Point> p(n);
    vector<KDRange::Item> items;
    for (int i = 0; i < n; i++)
    {
        cin >> p[i][0] >> p[i][1];
        auto [x, y] = p[i];
        items.push_back({{x + y, x - y}, i});
    }
    KDRange tree(items);
    while (q--)
    {
        long long x, y, e, a, b, c, d, f, g;
        cin >> x >> y >> e >> a >> b >> c >> d >> f >> g;
        auto removed = tree.extract({x + y - e, x - y - e},
                                    {x + y + e, x - y + e});
        for (auto v : removed)
        {
            int id = v.id;
            auto [u, t] = p[id];
            long long nx = ((__int128)u * a + (__int128)t * b + (__int128)(id + 1) * c) % w;
            long long ny = ((__int128)u * d + (__int128)t * f + (__int128)(id + 1) * g) % h;
            p[id] = {nx, ny};
            tree.add({{nx + ny, nx - ny}, id});
        }
    }
    for (auto [x, y] : p) cout << x << ' ' << y << '\n';
}
```

39.4 静态二维矩形最小权与删点

KDMin(points) 批量建树；Item 含二维 signed64 坐标 p 与 signed64 权 key，原输入位置为 0-based 编号。query(low, high) 查闭矩形内尚存记录，返回 (key, 编号) 最小的原编号；空集或反向矩形返回-1。erase(id) 删除该编号，重复删除无影响；相同坐标与权的记录彼此独立。p 保留原序可只读访问，其他成员也是只读约定，内部 build/pull 不直接调用；不支持插入、改权或恢复。没有数值哨兵，坐标与权只比较，均支持完整 signed64；n≤INT_MAX 且内存可分配。按 nth_element 平均线性选择计，构建 O(n log(n+1))，erase 为 O(log(n+1))，query 保守最坏 O(n)，空间 O(n)，递归深度 O(log(n+1))。包围盒保持建树时范围，删除后可能偏松但不会漏掉活点；best 只汇总活点。

```
struct KDMin
{
    using Point = array<long long, 2>;

    struct Item
    {
        Point p;
        long long key;
    };

    struct Node
    {
        Point low, high;
        int ch[2] = {-1, -1}, fa = -1, best = -1;
        bool alive = true;
    };

    vector<Item> p;
    vector<Node> a;
    int root = -1;

    KDMin(const vector<Item> &points) : p(points), a(points.size())
    {
        assert(points.size() <= INT_MAX);
        vector<int> order(p.size());
        iota(order.begin(), order.end(), 0);
        root = build(order, 0, order.size(), 0);
    }

    int better(int x, int y) const
    {
        if (x == -1) return y;
        if (y == -1) return x;
        if (p[x].key != p[y].key) return p[x].key < p[y].key ? x : y;
        return min(x, y);
    }

    void pull(int u)
    {
        a[u].best = a[u].alive ? u : -1;
        for (int v : a[u].ch)
        {
            if (v != -1) a[u].best = better(a[u].best, a[v].best);
        }
    }
```

中位数建树与固定包围盒（接上页同一结构体）：

```
    int build(vector<int> &order, int l, int r, int d)
    {
        if (l == r) return -1;
        int mid = l + (r - l) / 2;
        nth_element(order.begin() + l, order.begin() + mid, order.begin() + r,
                    [&](int x, int y)
        {
            if (p[x].p[d] != p[y].p[d]) return p[x].p[d] < p[y].p[d];
            return x < y;
        });
        int u = order[mid];
        a[u].low = a[u].high = p[u].p;
        a[u].ch[0] = build(order, l, mid, d ^ 1);
        a[u].ch[1] = build(order, mid + 1, r, d ^ 1);
        for (int v : a[u].ch)
        {
            if (v == -1) continue;
            a[v].fa = u;
            for (int j = 0; j < 2; j++)
            {
                a[u].low[j] = min(a[u].low[j], a[v].low[j]);
                a[u].high[j] = max(a[u].high[j], a[v].high[j]);
            }
        }
        pull(u);
        return u;
    }
```

按编号删除与矩形查询（接上页同一结构体）：

```
    void erase(int id)
    {
        assert(0 <= id && id < (int)p.size());
        a[id].alive = false;
        for (int u = id; u != -1; u = a[u].fa) pull(u);
    }

    int query(int u, const Point &low, const Point &high) const
    {
        if (u == -1 || a[u].best == -1) return -1;
        bool covered = true, inside = true;
        for (int d = 0; d < 2; d++)
        {
            if (high[d] < a[u].low[d] || low[d] > a[u].high[d]) return -1;
            if (low[d] > a[u].low[d] || high[d] < a[u].high[d]) covered = false;
            if (p[u].p[d] < low[d] || p[u].p[d] > high[d]) inside = false;
        }
        if (covered) return a[u].best;
        int ans = inside && a[u].alive ? u : -1;
        for (int v : a[u].ch) ans = better(ans, query(v, low, high));
        return ans;
    }

    int query(const Point &low, const Point &high) const
    {
        if (low[0] > high[0] || low[1] > high[1]) return -1;
        return query(root, low, high);
    }
};
```

接口用法演示：矩形最小权与按编号删除（接口演示）

演示约定：输入 n,q 和 n 条 x,y,key 记录；操作 1 id 删除 0-based 原编号，操作 2 xl yl xr yr 查询闭矩形内 (key, 编号) 最小的活记录编号，无解输出-1。支持重复坐标/权、重复删除、反向矩形与空点集。

所需模板：KDMin（第 599 页，39.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/ds/kdt/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<KDMin::Item> p(n);
    for (auto &v : p) cin >> v.p[0] >> v.p[1] >> v.key;
    KDMin kd(p);
    while (q--)
    {
        int op;
        cin >> op;
        if (op == 1)
        {
            int id;
            cin >> id;
            kd.erase(id);
        }
        else
        {
            KDMin::Point low, high;
            cin >> low[0] >> low[1] >> high[0] >> high[1];
            cout << kd.query(low, high) << '\n';
        }
    }
}
```

应用题补充：Codeforces 44G Shooting Gallery

题意：按顺序发射子弹，每发击碎最近尚存靶子，边界也算命中；输出每发击中的 1-based 靶子编号或 0。n,m≤10^5，坐标 0..10^7，矩形严格非退化，z 互异且为 1..10^7。把子弹时间作为 key，按 z 从近到远处理靶子，查询靶面内最早未用子弹并删除。比赛应用，非正式模板题；本地验证不等于在线 AC。

所需模板：KDMin（第 599 页，39.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/44/G

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<array<long long, 5>> targets(n);
    for (auto &t : targets)
    {
        for (auto &x : t) cin >> x;
    }
    vector<int> order(n);
    iota(order.begin(), order.end(), 0);
    sort(order.begin(), order.end(), [&](int x, int y)
    {
        return targets[x][4] < targets[y][4];
    });
    int m;
    cin >> m;
    vector<KDMin::Item> shots(m);
    for (int i = 0; i < m; i++)
    {
        cin >> shots[i].p[0] >> shots[i].p[1];
        shots[i].key = i;
    }
    KDMin kd(shots);
    vector<int> ans(m);
    for (int id : order)
    {
        auto [xl, xr, yl, yr, z] = targets[id];
        int shot = kd.query({xl, yl}, {xr, yr});
        if (shot == -1) continue;
        ans[shot] = id + 1;
        kd.erase(shot);
    }
    for (int x : ans) cout << x << '\n';
}
```

第四十章　动态树

40.1　Link Cut Tree

40.1.1　Link-cut tree：动态森林与路径异或

构造 LinkCutTree(n)，点号 1..n，初始权值为零。set(x,value) 修改点权；link(x,y) 仅在不同连通块之间连边并返回是否成功；cut(x,y) 只删除真实存在的边，否则返回 false。connected 查询连通性；query(x,y) 返回路径点权异或，要求两点连通，支持 x=y。make_root/find_root 维护与查询当前表示树根。每次操作均摊 O(log n)，O(n) 空间，splay 下传使用显式栈，支持长链。权值为 unsigned long long。仅维护路径异或，不包含虚子树聚合或动态直径。

```
struct LinkCutTree
{
    using U = unsigned long long;

    struct Node
    {
        int ch[2]{}, fa = 0;
        U val = 0, sum = 0;
        bool rev = false;
    };

    vector<Node> a;
    vector<int> st;

    LinkCutTree(int n) : a(n + 1), st(n + 1) {}

    bool is_root(int x) const
    {
        int f = a[x].fa;
        return a[f].ch[0] != x && a[f].ch[1] != x;
    }

    void pull(int x) { a[x].sum = a[a[x].ch[0]].sum ^ a[x].val ^ a[a[x].ch[1]].sum; }

    void reverse(int x)
    {
        if (!x) return;
        swap(a[x].ch[0], a[x].ch[1]);
        a[x].rev = !a[x].rev;
    }

    void push(int x)
    {
        if (!a[x].rev) return;
        reverse(a[x].ch[0]);
        reverse(a[x].ch[1]);
        a[x].rev = false;
    }
```

```
    void rotate(int x)
    {
        int y = a[x].fa, z = a[y].fa;
        int k = a[y].ch[1] == x;
        int w = a[x].ch[k ^ 1];
        if (!is_root(y)) a[z].ch[a[z].ch[1] == y] = x;
        a[x].fa = z;
        a[y].ch[k] = w;
        if (w) a[w].fa = y;
        a[x].ch[k ^ 1] = y;
        a[y].fa = x;
        pull(y);
        pull(x);
    }

    void splay(int x)
    {
        int top = 0, y = x;
        st[top++] = y;
        while (!is_root(y))
        {
            y = a[y].fa;
            st[top++] = y;
        }
        while (top) push(st[--top]);
        while (!is_root(x))
        {
            int y = a[x].fa, z = a[y].fa;
            if (!is_root(y))
            {
                bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
                rotate(same ? y : x);
            }
            rotate(x);
        }
    }
```

```
77      void access(int x)
78      {
79          int last = 0;
80          for (int y = x; y; y = a[y].fa)
81          {
82              splay(y);
83              // The old right child's fa remains y: it becomes a path-parent.
84              a[y].ch[1] = last;
85              if (last) a[last].fa = y;
86              pull(y);
87              last = y;
88          }
89          splay(x);
90      }
91
92      void make_root(int x)
93      {
94          access(x);
95          reverse(x);
96      }
97
98      int find_root(int x)
99      {
100         access(x);
101         push(x);
102         while (a[x].ch[0])
103         {
104             x = a[x].ch[0];
105             push(x);
106         }
107         splay(x);
108         return x;
109     }
```

```
    bool connected(int x, int y) { return x == y || find_root(x) == find_root(y); }

    bool link(int x, int y)
    {
        make_root(x);
        if (find_root(y) == x) return false;
        a[x].fa = y;
        return true;
    }

    bool cut(int x, int y)
    {
        make_root(x);
        access(y);
        if (a[y].ch[0] != x || a[x].ch[1]) return false;
        a[y].ch[0] = 0;
        a[x].fa = 0;
        pull(y);
        return true;
    }

    void set(int x, U value)
    {
        access(x);
        a[x].val = value;
        pull(x);
    }

    // x and y must be connected. This is a path aggregate, not a subtree sum.
    U query(int x, int y)
    {
        make_root(x);
        access(y);
        return a[y].sum;
    }
};
```

模板题使用：Luogu P3690

题意：维护点权森林，支持路径异或、连边、删边和单点赋值。初始没有边；0 查询、1 连边、2 删边、3 赋值。题目保证查询两点连通，成环的连边和不存在的删边由接口拒绝。点号 1..n。

所需模板：LinkCutTree（第 604 页，40.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3690

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    LinkCutTree g(n);
    for (int i = 1; i <= n; i++)
    {
        unsigned long long x;
        cin >> x;
        g.set(i, x);
    }
    while (m--)
    {
        int op, x, y;
        cin >> op >> x >> y;
        if (op == 0)
            cout << g.query(x, y) << '\n';
        else if (op == 1)
            g.link(x, y);
        else if (op == 2)
            g.cut(x, y);
        else
            g.set(x, y);
    }
}
```

40.1.2 动态换根路径乘积总和

代码依赖：ModInt（第 133 页，19.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

TreePathProducts<mod>(g,weight) 要求 g 是非空无向树，点号 0..n-1、每条边双向存储，weight 为等长 vector<ModInt<mod»。set(u,w) 修改点权；query(u) 返回从 u 到所有顶点 v 的唯一路径上点权乘积之和，含 u=v 的单点路径。固定拓扑，不提供 link/cut。建树 O(n)，修改和查询摊 O(log n)，空间 O(n)，递归建树和 splay 下传最坏深度 O(n)。允许零和负权，mod>1 且不要求素数，不用除法或逆元。a 与全部辅助接口仅供内部使用，内部点号 1..n；不能把公开接口的 0-based 点号直接传给 make_root。依赖 ModInt；附件爱莲说的输入 x 由主程序显式转为权值 1-x。

```
template <int mod> struct TreePathProducts
{
    using Z = ModInt<mod>;

    struct Node
    {
        int ch[2]{}, fa = 0;
        Z weight = 0, product = 1, left = 0, right = 0, virt = 0;
        bool rev = false;
    };

    vector<Node> a;

    // g is a nonempty tree, with 0-based vertices and one weight per vertex.
    TreePathProducts(const vector<vector<int>> &g, const vector<Z> &weight)
        : a(g.size() + 1)
    {
        assert(!g.empty() && g.size() == weight.size());
        auto dfs = [&](auto &&self, int u, int parent) -> void
        {
            int x = u + 1;
            a[x].fa = parent + 1;
            a[x].weight = weight[u];
            for (int v : g[u])
                if (v != parent)
                {
                    self(self, v, u);
                    a[x].virt += a[v + 1].left;
                }
            pull(x);
        };
        dfs(dfs, 0, -1);
    }
```

聚合、翻转、递归下传与旋转（接上页同一结构体）：

```
    bool is_root(int x) const
    {
        int f = a[x].fa;
        return a[f].ch[0] != x && a[f].ch[1] != x;
    }

    void pull(int x)
    {
        auto &t = a[x];
        const auto &l = a[t.ch[0]], &r = a[t.ch[1]];
        t.product = l.product * t.weight * r.product;
        t.left = l.left + l.product * t.weight * (Z(1) + t.virt + r.left);
        t.right = r.right + r.product * t.weight * (Z(1) + t.virt + l.right);
    }

    void reverse(int x)
    {
        if (!x) return;
        swap(a[x].ch[0], a[x].ch[1]);
        swap(a[x].left, a[x].right);
        a[x].rev = !a[x].rev;
    }

    void push(int x)
    {
        if (!a[x].rev) return;
        reverse(a[x].ch[0]);
        reverse(a[x].ch[1]);
        a[x].rev = false;
    }

    void push_path(int x)
    {
        if (!is_root(x)) push_path(a[x].fa);
        push(x);
    }

    void rotate(int x)
    {
        int y = a[x].fa, z = a[y].fa;
        int k = a[y].ch[1] == x, w = a[x].ch[k ^ 1];
        if (!is_root(y)) a[z].ch[a[z].ch[1] == y] = x;
        a[x].fa = z;
        a[y].ch[k] = w;
        if (w) a[w].fa = y;
        a[x].ch[k ^ 1] = y;
        a[y].fa = x;
        pull(y);
        pull(x);
    }
```

Splay、虚实边切换与公开接口（接上页同一结构体）：

```
    void splay(int x)
    {
        push_path(x);
        while (!is_root(x))
        {
            int y = a[x].fa, z = a[y].fa;
            if (!is_root(y))
            {
                bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
                rotate(same ? y : x);
            }
            rotate(x);
        }
    }

    void make_root(int x)
    {
        int last = 0;
        for (int y = x; y; y = a[y].fa)
        {
            splay(y);
            a[y].virt += a[a[y].ch[1]].left - a[last].left;
            a[y].ch[1] = last;
            if (last) a[last].fa = y;
            pull(y);
            last = y;
        }
        splay(x);
        reverse(x);
    }

    void set(int u, Z weight)
    {
        int x = u + 1;
        make_root(x);
        a[x].weight = weight;
        pull(x);
    }

    Z query(int u)
    {
        int x = u + 1;
        make_root(x);
        return a[x].left;
    }
};
```

应用题补充：附件爱莲说接口（本地应用示例）

题意：沿用附件输入：n,q，n 个 x，n-1 条 1-based 无向边；1 u x 修改参数，2 u 查询。主程序把每个 x 转为点权 1-x，并把顶点转为 0-based。输出所有从 u 出发路径的点权乘积总和。原题外部地址和线上结果尚未核验。

所需模板：ModInt（第 133 页，19.2.2 节）、TreePathProducts（第 609 页，40.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://github.com/gsh20040816/CPCtemplate/issues/11

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Z = ModInt<998244353>;
    int n, q;
    cin >> n >> q;
    vector<Z> weight(n);
    for (auto &w : weight)
    {
        int x;
        cin >> x;
        w = Z(1) - Z(x);
    }
    vector<vector<int>> g(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        u--;
        v--;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    TreePathProducts<998244353> tree(g, weight);
    while (q--)
    {
        int op, u;
        cin >> op >> u;
        if (op == 1)
        {
            int x;
            cin >> x;
            tree.set(u - 1, Z(1) - Z(x));
        }
        else
            cout << tree.query(u - 1).v << '\n';
    }
}
```

相关应用：固定树三状态 min-plus 仿射动态 DP；生长采摘专用可行性模型

40.1.3 动态树上生长与采摘可行性

DynamicPeachTree(parent,b)：0-based 固定有根树，唯一根父亲-1，1≤n≤200000；b 为非负 long long 需求数组。初始各点果实数为 0。add(type,x,value) 累计事件并返回当前整个前缀是否可行：type=1 的每个生长事件可放到 x 的父亲或 x 子树任意一点；type=2 的采摘事件可从 x 子树任意一点取走一个。每个前缀独立，可重新安排所有事件的顺序及分配，最终每点至少 b[i] 且过程中不能为负。根的生长只能放在自身子树。healthy() 读取当前可行性；不是输出实际分配方案。

所有 b 及历次 value 之和须 ≤10^15，value 非负；原 CF1942H 约束满足。三状态 min-plus 仿射转移，第四维恒为 0；不可达项相加前跳过，不能直接把 INF 加负数。sumg/sumh/sumb 存虚子树贡献；更新必须用旧聚合扣除父亲的旧贡献。内部 t 与辅助函数不供调用方修改。初始化 O(n)，单次 add 摊还 O(logn)，空间 O(n)，矩阵常数固定；递归初建深度最坏 n，无换根或 link/cut。

对应 WIDA07B，来源原稿、静态树 DP、独立小规模最大流和完整 CF1942H 输入输出协议本地验证；不等于本次线上 AC。

```
struct DynamicPeachTree
{
    using ll = long long;
    static constexpr ll INF = 1LL << 60;
    using State = array<ll, 4>;
    struct Matrix
    {
        array<State, 4> a;
        Matrix(bool identity = false)
        {
            for (int i = 0; i < 4; i++)
            {
                a[i].fill(INF);
                if (identity)
                    a[i][i] = 0;
            }
        }
    };
    struct Node
    {
        array<int, 2> ch{};
        int p = 0;
        ll b = 0, g = 0, h = 0;
        ll sumg = 0, sumh = 0, sumb = 0;
        Matrix mat, tot;
    };
    vector<Node> t;
    State ans{};
    ll budget = 0;
```

动态树上供需判定（接上页同一结构体）：

```
    static Matrix join(const Matrix &a, const Matrix &b)
    {
        Matrix c;
        for (int i = 0; i < 3; i++)
        {
            for (int j = 0; j < 3; j++)
                for (int k = 0; k < 4; k++)
                    if (a.a[i][j] != INF && b.a[j][k] != INF)
                        c.a[i][k] = min(c.a[i][k], a.a[i][j] + b.a[j][k]);
            c.a[i][3] = min(c.a[i][3], a.a[i][3]);
        }
        c.a[3][3] = 0;
        return c;
    }

    State get(int u) const
    {
        State a{};
        for (int i = 0; i < 3; i++)
            a[i] = *min_element(t[u].tot.a[i].begin(), t[u].tot.a[i].end());
        return a;
    }

    void pull(int u)
    {
        t[u].tot = join(join(t[t[u].ch[0]].tot, t[u].mat), t[t[u].ch[1]].tot);
    }

    void branch(int u, const State &a, int sign)
    {
        t[u].sumg += sign * a[0];
        t[u].sumh += sign * min({a[1], a[2], 0LL});
        t[u].sumb += sign * *min_element(a.begin(), a.end());
    }

    void update(int u)
    {
        auto &a = t[u];
        a.mat = Matrix();
        a.mat.a[0][0] = a.b + a.h - a.g + a.sumg;
        for (int j = 1; j < 4; j++)
            a.mat.a[1][j] = a.h + a.sumh;
        for (int j = 0; j < 4; j++)
            a.mat.a[2][j] = a.b + a.h + a.sumb;
        a.mat.a[3][3] = 0;
        pull(u);
    }
```

动态树上供需判定（接上页同一结构体）：

```
    int init(int u, const vector<vector<int>> &children)
    {
        int size = 1;
        for (int v : children[u])
        {
            size += init(v, children);
            branch(u, get(v), 1);
        }
        update(u);
        return size;
    }

    // 0-based parent, one -1 root. Nonnegative b; total b + all event counts <= 1e15.
    DynamicPeachTree(const vector<int> &parent, const vector<ll> &b)
        : t(parent.size() + 1)
    {
        int n = parent.size(), root = 0;
        assert(n > 0 && n <= 200000 && (int)b.size() == n);
        t[0].tot = Matrix(true);
        vector<vector<int>> children(n + 1);
        for (int i = 0; i < n; i++)
        {
            assert(b[i] >= 0 && b[i] <= 1000000000000000LL - budget);
            budget += b[i];
            t[i + 1].b = b[i];
            assert(parent[i] >= -1 && parent[i] < n && parent[i] != i);
            t[i + 1].p = parent[i] + 1;
            if (parent[i] == -1)
            {
                assert(root == 0);
                root = i + 1;
            }
            else
                children[parent[i] + 1].push_back(i + 1);
        }
        assert(root > 0);
        int size = init(root, children);
        assert(size == n);
        (void)size;
        ans = get(root);
    }
```

动态树上供需判定（接上页同一结构体）：

```
    bool isroot(int u) const
    {
        int p = t[u].p;
        return t[p].ch[0] != u && t[p].ch[1] != u;
    }

    void rotate(int u)
    {
        int p = t[u].p, q = t[p].p;
        int k = t[p].ch[1] == u;
        int v = t[u].ch[k ^ 1];
        if (!isroot(p))
            t[q].ch[t[q].ch[1] == p] = u;
        t[u].p = q;
        t[p].ch[k] = v;
        if (v)
            t[v].p = p;
        t[u].ch[k ^ 1] = p;
        t[p].p = u;
        pull(p);
    }

    void splay(int u)
    {
        while (!isroot(u))
        {
            int p = t[u].p, q = t[p].p;
            if (!isroot(p))
                rotate((t[p].ch[1] == u) == (t[q].ch[1] == p) ? p : u);
            rotate(u);
        }
        pull(u);
    }
```

动态树上供需判定（接上页同一结构体）：

```
    // type 1: growth, type 2: harvest. Returns feasibility of the whole prefix.
    bool add(int type, int x, ll value)
    {
        assert(type == 1 || type == 2);
        assert(x >= 0 && x + 1 < (int)t.size());
        assert(value >= 0 && value <= 1000000000000000LL - budget);
        budget += value;
        int u = x + 1;
        State old{};
        for (int i = u, q = 0; i; q = i, i = t[i].p)
        {
            splay(i);
            if (t[i].ch[1])
                branch(i, get(t[i].ch[1]), 1);
            t[i].ch[1] = q;
            if (q)
                branch(i, old, -1);
            else if (type == 1)
                t[i].g += value;
            else
                t[i].h += value;
            old = get(i);
            update(i);
        }
        splay(u);
        ans = get(u);
        return healthy();
    }

    bool healthy() const
    {
        return ans[0] == *min_element(ans.begin(), ans.end());
    }
};
```

应用题补充：CF1942H Farmer John's Favorite Intern

题意：完整 CF1942H 协议：t 组，每组 n q，节点 2..n 的 1-based 父亲，n 个最低需求，q 行 type x value；输出每个独立前缀是否存在可行事件分配。1≤n,q≤200000，总 n 及总 q 各 ≤200000，b≤10^6，1≤value≤10^6。当前仅本地验证，非新线上 AC。

所需模板：DynamicPeachTree（第 613 页，40.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/1942/H

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n, q;
        cin >> n >> q;
        vector<int> parent(n, -1);
        for (int i = 1; i < n; i++)
        {
            cin >> parent[i];
            parent[i]--;
        }
        vector<long long> b(n);
        for (auto &x : b)
            cin >> x;
        DynamicPeachTree tree(parent, b);
        while (q--)
        {
            int type, x;
            long long value;
            cin >> type >> x >> value;
            cout << (tree.add(type, x - 1, value) ? "YES" : "NO") << '\n';
        }
    }
}
```

相关应用：固定拓扑的虚子树聚合与点权修改后全树路径异或和，不表示通用动态森林接口

40.1.4 点权修改与全树路径异或和

XorPathSum(parent,value)，两数组 0-based 且等长，1≤n≤200000，parent 构成一棵固定有根树，唯一根的父亲为-1；根和节点编号任意。点权为 [0,2^27) 整数。sum() 返回所有无序端点对 u≤v 的简单路径上点权异或值之和，包含 n 条单点路径；set(x,value) 修改一个点权并返回新总和。不是路径异或查询，也不支持 link/cut 或改变拓扑。

逐位维护首尾路径异或计数及虚子树之间的配对数，计数相乘先提升为 long long。27 位固定，n≤200000 时总和至多 n(n+1)/2 乘 (2^27-1)，可由 signed64 表示。初建 O(27n)，set 摊还 O(27logn)，sum 为 O(1)，空间 O(27n) 且常数较大：每个节点保存两份 Info、虚子树计数及配对数；按实际 sizeof(Node) 估算内存。初建 DFS 保持递归，最坏深度 n；splay 无翻转标记。t 和其他函数仅供内部实现，调用方只用构造、set、sum；完整对象复制后相互独立。

WIDA07C 原稿的 cnt 乘法在两个 5 万点分支时会出现 int 中间值溢出，本版显式提升；原题完整约束尚未核实，不据此断言原题实现有错。当前为独立路径枚举、原稿小规模对照及大树公式验证，非线上 AC。

```
struct XorPathSum
{
    using ll = long long;
    static constexpr int B = 27;
    struct Info
    {
        array<array<int, 2>, B> up{}, down{};
        int path = 0;
        ll sum = 0;
    };
    struct Node
    {
        array<int, 2> ch{};
        int p = 0;
        Info val, tot;
        array<array<int, 2>, B> cnt{};
        array<array<ll, 2>, B> pairs{};
        ll sum = 0;
    };
    vector<Node> t;
    ll ans = 0;

    static Info join(const Info &a, const Info &b)
    {
        Info c;
        c.path = a.path ^ b.path;
        c.sum = a.sum + b.sum;
        for (int i = 0; i < B; i++)
            for (int j = 0; j < 2; j++)
            {
                c.sum += (1LL << i) * a.down[i][j] * b.up[i][j ^ 1];
                c.up[i][j] = a.up[i][j] + b.up[i][j ^ (a.path >> i & 1)];
                c.down[i][j] = b.down[i][j] + a.down[i][j ^ (b.path >> i & 1)];
            }
        return c;
    }

    void pull(int u)
    {
        t[u].tot = join(join(t[t[u].ch[0]].tot, t[u].val), t[t[u].ch[1]].tot);
    }
```

全树路径异或和（接上页同一结构体）：

```
    // sign = +1 adds a virtual child; -1 removes its old aggregate.
    void branch(int u, const Info &s, int sign)
    {
        auto &a = t[u];
        for (int i = 0; i < B; i++)
        {
            if (sign < 0)
                for (int j = 0; j < 2; j++)
                    a.cnt[i][j] -= s.up[i][j];
            for (int x = 0; x < 2; x++)
            {
                ll ways = s.up[i][1 ^ x];
                for (int j = 0; j < 2; j++)
                    ways += 1LL * a.cnt[i][j] * s.up[i][j ^ 1 ^ x];
                a.pairs[i][x] += sign * ways;
            }
            if (sign > 0)
                for (int j = 0; j < 2; j++)
                    a.cnt[i][j] += s.up[i][j];
        }
        a.sum += sign * s.sum;
    }

    void update(int u)
    {
        auto &a = t[u];
        a.val.sum = a.val.path + a.sum;
        for (int i = 0; i < B; i++)
        {
            int bit = a.val.path >> i & 1;
            a.val.sum += (1LL << i) * a.pairs[i][bit];
            for (int j = 0; j < 2; j++)
            {
                a.val.up[i][j] = a.cnt[i][j ^ bit];
                a.val.down[i][j] = a.cnt[i][j ^ bit];
            }
            a.val.up[i][bit]++;
            a.val.down[i][bit]++;
        }
        pull(u);
    }

    int init(int u, const vector<vector<int>> &g)
    {
        int size = 1;
        for (int v : g[u])
        {
            size += init(v, g);
            branch(u, t[v].tot, 1);
        }
        update(u);
        return size;
    }
```

全树路径异或和（接上页同一结构体）：

```
    // parent/value are 0-based; exactly one parent is -1, topology stays fixed.
    XorPathSum(const vector<int> &parent, const vector<int> &value)
        : t(parent.size() + 1)
    {
        int n = parent.size();
        assert(n > 0 && n <= 200000 && (int)value.size() == n);
        vector<vector<int>> g(n + 1);
        int root = 0;
        for (int i = 0; i < n; i++)
        {
            assert(value[i] >= 0 && value[i] < (1 << B));
            assert(parent[i] >= -1 && parent[i] < n && parent[i] != i);
            t[i + 1].val.path = value[i];
            t[i + 1].p = parent[i] + 1;
            if (parent[i] == -1)
            {
                assert(root == 0);
                root = i + 1;
            }
            else
                g[parent[i] + 1].push_back(i + 1);
        }
        assert(root > 0);
        int size = init(root, g);
        assert(size == n);
        (void)size;
        ans = t[root].tot.sum;
    }

    bool isroot(int u) const
    {
        int p = t[u].p;
        return t[p].ch[0] != u && t[p].ch[1] != u;
    }

    void rotate(int u)
    {
        int p = t[u].p, q = t[p].p;
        int k = t[p].ch[1] == u;
        int v = t[u].ch[k ^ 1];
        if (!isroot(p))
            t[q].ch[t[q].ch[1] == p] = u;
        t[u].p = q;
        t[p].ch[k] = v;
        if (v)
            t[v].p = p;
        t[u].ch[k ^ 1] = p;
        t[p].p = u;
        pull(p);
    }
```

全树路径异或和（接上页同一结构体）：

```
    void splay(int u)
    {
        while (!isroot(u))
        {
            int p = t[u].p, q = t[p].p;
            if (!isroot(p))
                rotate((t[p].ch[1] == u) == (t[q].ch[1] == p) ? p : u);
            rotate(u);
        }
        pull(u);
    }

    ll set(int x, int value)
    {
        assert(x >= 0 && x + 1 < (int)t.size());
        assert(value >= 0 && value < (1 << B));
        int u = x + 1;
        Info old;
        for (int i = u, q = 0; i; q = i, i = t[i].p)
        {
            splay(i);
            if (t[i].ch[1])
                branch(i, t[t[i].ch[1]].tot, 1);
            t[i].ch[1] = q;
            if (q)
                branch(i, old, -1);
            else
                t[i].val.path = value;
            old = t[i].tot;
            update(i);
        }
        splay(u);
        ans = t[u].tot.sum;
        return ans;
    }

    ll sum() const
    {
        return ans;
    }
};
```

接口用法演示：点权修改与全树路径异或和 API

演示约定：读 n q，接着 n 个 0-based 父亲编号（唯一根为-1）与 n 个 27 位非负点权；输出初始所有无序端点简单路径的点权异或和，包含单点路径。随后 q 行 x value，每次修改点 x 并输出新总和。固定树，1≤n≤200000；输出 signed64。自定义 API 协议，非原比赛题判定。

所需模板：XorPathSum（第 618 页，40.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<int> parent(n), value(n);
    for (int &p : parent)
        cin >> p;
    for (int &x : value)
        cin >> x;
    XorPathSum tree(parent, value);
    cout << tree.sum() << '\n';
    while (q--)
    {
        int x, value;
        cin >> x >> value;
        cout << tree.set(x, value) << '\n';
    }
}
```

40.1.5 动态森林路径加与最大值

以 0-based 点权 vector 构造，点号 1..n，初始无边，n<INT_MAX。link(x, y) 只连接不同树；cut(x, y) 仅删直接边；cut_parent(r, x) 以 r 为根删除 x 的父边，r 与 x 可不相邻。三者成功返回 true，非法操作返回 false。add(x, y, w) 对路径全部点加 w，不连通返回 false；query(x, y) 返回 optional<long long> 路径最大值，不连通为空，支持同点及负权。失败保持森林和点权，可改变辅助树。所有点权、最大值及累计懒加标记的 64 位加法均须不溢出。make_root 与 find_root 改变或查询表示树根；其余方法及 a 属内部。操作均摊 O(log n)，空间 O(n)，递归下传辅助树路径，单次深度可达 O(n)，需足够栈。不维护虚子树聚合。

```
struct DynamicPathMax
{
    using ll = long long;

    struct Node
    {
        int ch[2]{}, fa = 0;
        ll val = 0, mx = 0, add = 0;
        bool rev = false;
    };

    vector<Node> a;

    DynamicPathMax(const vector<ll> &v) : a(v.size() + 1)
    {
        for (int i = 0; i < (int)v.size(); i++)
            a[i + 1].val = a[i + 1].mx = v[i];
    }

    bool is_root(int x) const
    {
        int f = a[x].fa;
        return a[f].ch[0] != x && a[f].ch[1] != x;
    }

    void pull(int x)
    {
        a[x].mx = a[x].val;
        for (int y : a[x].ch)
            if (y)
                a[x].mx = max(a[x].mx, a[y].mx);
    }

    void flip(int x)
    {
        if (!x)
            return;
        swap(a[x].ch[0], a[x].ch[1]);
        a[x].rev = !a[x].rev;
    }
```

递归下传与旋转（接上页同一结构体）：

```
    void apply(int x, ll w)
    {
        if (!x)
            return;
        a[x].val += w;
        a[x].mx += w;
        a[x].add += w;
    }

    void push(int x)
    {
        if (a[x].rev)
        {
            flip(a[x].ch[0]);
            flip(a[x].ch[1]);
            a[x].rev = false;
        }
        if (a[x].add)
        {
            apply(a[x].ch[0], a[x].add);
            apply(a[x].ch[1], a[x].add);
            a[x].add = 0;
        }
    }

    void push_path(int x)
    {
        if (!is_root(x))
            push_path(a[x].fa);
        push(x);
    }

    void rotate(int x)
    {
        int y = a[x].fa, z = a[y].fa;
        int k = a[y].ch[1] == x;
        int w = a[x].ch[k ^ 1];
        if (!is_root(y))
            a[z].ch[a[z].ch[1] == y] = x;
        a[x].fa = z;
        a[y].ch[k] = w;
        if (w)
            a[w].fa = y;
        a[x].ch[k ^ 1] = y;
        a[y].fa = x;
        pull(y);
        pull(x);
    }
```

伸展、访问与换根（接上页同一结构体）：

```
    void splay(int x)
    {
        push_path(x);
        while (!is_root(x))
        {
            int y = a[x].fa, z = a[y].fa;
            if (!is_root(y))
            {
                bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
                rotate(same ? y : x);
            }
            rotate(x);
        }
    }

    void access(int x)
    {
        int last = 0;
        for (int y = x; y; y = a[y].fa)
        {
            splay(y);
            a[y].ch[1] = last;
            if (last)
                a[last].fa = y;
            pull(y);
            last = y;
        }
        splay(x);
    }

    void make_root(int x)
    {
        access(x);
        flip(x);
    }

    int find_root(int x)
    {
        access(x);
        push(x);
        while (a[x].ch[0])
        {
            x = a[x].ch[0];
            push(x);
        }
        splay(x);
        return x;
    }
```

连边、两种删边与路径操作（接上页同一结构体）：

```
    bool expose(int x, int y)
    {
        make_root(x);
        if (find_root(y) != x)
            return false;
        access(y);
        return true;
    }

    bool link(int x, int y)
    {
        make_root(x);
        if (find_root(y) == x)
            return false;
        a[x].fa = y;
        return true;
    }

    bool cut(int x, int y)
    {
        if (!expose(x, y) || a[y].ch[0] != x || a[x].ch[1])
            return false;
        a[y].ch[0] = 0;
        a[x].fa = 0;
        pull(y);
        return true;
    }

    bool cut_parent(int root, int x)
    {
        if (root == x || !expose(root, x))
            return false;
        int left = a[x].ch[0];
        a[left].fa = 0;
        a[x].ch[0] = 0;
        pull(x);
        return true;
    }

    bool add(int x, int y, ll w)
    {
        if (!expose(x, y))
            return false;
        apply(y, w);
        return true;
    }

    optional<ll> query(int x, int y)
    {
        if (!expose(x, y))
            return nullopt;
        return a[y].mx;
    }
};
```

接口用法演示：动态森林路径加最大值（kuangbin HDU4010 模型）

演示约定：EOF 多组：读 n、n-1 条边、点权和 q。1 x y 连边；2 x y 以 x 为根删 y 的父边；3 w x y 路径加；4 x y 路径最大值。非法输出-1，成功修改不输出，组末空行。来源模型/API，官方题面、资源限制及线上 AC 未核验。

所需模板：DynamicPathMax（第 624 页，40.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://acm.hdu.edu.cn/showproblem.php?pid=4010

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    while (cin >> n)
    {
        vector<pair<int, int>> edges(n - 1);
        for (auto &[x, y] : edges)
            cin >> x >> y;
        vector<long long> v(n);
        for (auto &x : v)
            cin >> x;
        DynamicPathMax t(v);
        for (auto [x, y] : edges)
            t.link(x, y);
        int q;
        cin >> q;
        while (q--)
        {
            int op, x, y;
            long long w = 0;
            cin >> op;
            if (op == 3)
                cin >> w;
            cin >> x >> y;
            bool ok = true;
            if (op == 1)
                ok = t.link(x, y);
            else if (op == 2)
                ok = t.cut_parent(x, y);
            else if (op == 3)
                ok = t.add(x, y, w);
            else
            {
                auto ans = t.query(x, y);
                cout << (ans ? *ans : -1) << '\n';
            }
            if (!ok)
                cout << -1 << '\n';
        }
        cout << '\n';
    }
    return 0;
}
```

第六部分

图论

第四十一章　DFS（图论）

41.1　顶点划分：各诱导子图全为奇度

代码依赖：dsu（第 373 页，29.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

odd_induced_partition(n,edges) 接收 0..n-1 的无向简单图，禁止自环、重边。返回 optional<vector<int»：无解为 nullopt，否则每点一个正组号，同组顶点的诱导子图中每个点度数均为奇数。不删选组内边，不要求组数最少。充要条件是每个连通分量点数为偶数；空图返回空划分。先递归 DFS 取得原深度和后序，再合并未完成分组；内部点已为奇度、根为偶度，奇数个子组可封口，偶数时留一个可上挂的子组给祖先。level 只记录活动根的挂接深度，原 depth 不修改。祖先邻点按原深度排序，以二分寻找最近的严格祖先，避免反复扫描高出度根的全部邻居。时间 O((n+m) log(n+1))，空间 O(n+m)，递归栈 O(n)；节点数、边数须可由 int 表示。依赖 dsu 的按调用方向连接根语义。

```
optional<vector<int>> odd_induced_partition(int n, const vector<pair<int, int>> &edges)
{
    vector<vector<int>> g(n), up(n);
    for (auto [u, v] : edges)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<int> depth(n, -1), order;
    auto dfs = [&](auto &&self, int u, int h) -> int
    {
        depth[u] = h;
        int size = 1;
        for (int v : g[u])
            if (depth[v] == -1) size += self(self, v, h + 1);
        order.push_back(u);
        return size;
    };
    for (int u = 0; u < n; u++)
        if (depth[u] == -1 && dfs(dfs, u, 0) % 2) return nullopt;
    for (auto [u, v] : edges)
    {
        if (depth[u] > depth[v]) swap(u, v);
        up[v].push_back(u);
    }
    for (auto &a : up)
        sort(a.begin(), a.end(), [&](int u, int v) { return depth[u] < depth[v]; });
```

合并未完成分组（接上页同一函数）：

```
    dsu d(n);
    vector<int> level = depth, color(n);
    int count = 0;
    for (int u : order)
    {
        vector<int> children;
        for (int v : g[u])
            if (level[v] == level[u] + 1 && !color[d.find(v)]) children.push_back(v);
        if (children.empty()) continue;
        int skip = -1, best = -1;
        if (children.size() % 2 == 0)
            for (int v : children)
            {
                auto it = lower_bound(up[v].begin(),
                                      up[v].end(),
                                      depth[u],
                                      [&](int w, int h) { return depth[w] < h; });
                if (it != up[v].begin() && depth[*prev(it)] > best)
                {
                    skip = v;
                    best = depth[*prev(it)];
                }
            }
        if (skip != -1) level[skip] = best + 1;
        if (children.size() % 2 || skip != -1) color[d.find(u)] = ++count;
        for (int v : children)
            if (v != skip) d.merge(u, v);
    }
    for (int u = 0; u < n; u++) color[u] = color[d.find(u)];
    return color;
}
```

应用题补充：奇度诱导子图划分（本地接口示例）

题意：输入无向简单图，点号减 1 后调用；输出同组顶点的诱导子图全部为奇度的顶点分组，无解输出 -1。组号本身无唯一答案，验证时按原图所有同组边计算度数。本例是本地接口演示，链接为存在性定理来源，不是 OJ 记录。

所需模板：dsu（第 373 页，29.1.1 节）、odd_induced_partition（第 630 页，41.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://people.maths.ox.ac.uk/scott/Papers/odds2.pdf

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> edges(m);
    for (auto &[u, v] : edges)
    {
        cin >> u >> v;
        u--;
        v--;
    }
    auto color = odd_induced_partition(n, edges);
    if (!color)
        cout << -1 << '\n';
    else
    {
        for (int x : *color) cout << x << ' ';
        cout << '\n';
    }
}
```

第四十二章　树上问题

42.1　树的直径

相关应用：固定拓扑下修改非负边权，递归欧拉序与区间加维护动态直径。

42.1.1　边权修改动态树直径

DynamicDiameter(n)，n≥1，点号 1..n。add(u,v,w) 返回从 0 开始的原边编号；输入须恰为一棵非负权无向树。build(root=1) 预处理；update(id,w) 设置原边权值；diameter() 返回最长路径长度，单点树为 0。仅返回长度，不支持 link/cut 和负权。可换根重建、复制；所有内部成员仅供只读。全部边权之和始终 ≤LLONG_MAX/4，保证中间值安全。

递归 DFS 记录进入顶点及每次孩子返回后的根距离，得到 2n-1 项欧拉序。修改父边等价于对子树首次至最后出现的闭区间加 Δ。维护 max(d[i]-2d[j]+d[k])，i≤j≤k；非负权下区间最小根距离等于 LCA 根距离，故最大值即直径。线段树五项 high、low、left、right、best 分别表示 d、-2d、前两项、后两项、完整三项最大值。统一加 Δ 后依次加 Δ、-2Δ、-Δ、-Δ、0。建树 O(n)，修改 O(log n)，查询 O(1)，空间 O(n)。DFS 保持递归，深链须给足原生栈。LOJ3163 已满分通过，排名未核验。

字段、加边与五项合并：

```
struct DynamicDiameter
{
    using ll = long long;
    struct Node
    {
        ll high, low, left, right, best, lazy;
    };
    struct Edge
    {
        int u, v;
        ll w;
    };
    int n, size = 0;
    ll total = 0;
    vector<Edge> edges;
    vector<vector<pair<int, int>>> g;
    vector<int> first, last, child;
    vector<Node> t;

    explicit DynamicDiameter(int n) : n(n), g(n + 1), first(n + 1), last(n + 1)
    {
        assert(n >= 1);
    }
    int add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(w >= 0 && w <= LLONG_MAX / 4 - total);
        int id = edges.size();
        edges.push_back({u, v, w});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        total += w;
        size = 0;
        return id;
    }
    static Node merge(const Node &a, const Node &b)
    {
        Node c;
        c.high = max(a.high, b.high);
        c.low = max(a.low, b.low);
        c.left = max({a.left, b.left, a.high + b.low});
        c.right = max({a.right, b.right, a.low + b.high});
        c.best = max({a.best, b.best, a.left + b.high, a.high + b.right});
        c.lazy = 0;
        return c;
    }
```

递归欧拉遍历与建树（接上页结构体）：

```
    void build(int root = 1)
    {
        assert(1 <= root && root <= n && (int)edges.size() == n - 1);
        vector<ll> d;
        child.resize(edges.size());
        auto dfs = [&](auto &&self, int u, int parent, ll depth) -> void
        {
            first[u] = d.size();
            d.push_back(depth);
            for (auto [v, id] : g[u])
            {
                if (v == parent) continue;
                child[id] = v;
                self(self, v, u, depth + edges[id].w);
                d.push_back(depth);
            }
            last[u] = d.size() - 1;
        };
        dfs(dfs, root, 0, 0);
        size = d.size();
        t.resize(4 * size);
        auto init = [&](auto &&self, int x, int l, int r) -> void
        {
            if (l == r)
            {
                t[x] = {d[l], -2 * d[l], -d[l], -d[l], 0, 0};
                return;
            }
            int mid = (l + r) / 2;
            self(self, x * 2, l, mid);
            self(self, x * 2 + 1, mid + 1, r);
            t[x] = merge(t[x * 2], t[x * 2 + 1]);
        };
        init(init, 1, 0, size - 1);
    }
```

区间加与原边修改（接上页结构体）：

```
    void apply(int x, ll delta)
    {
        t[x].high += delta;
        t[x].low -= 2 * delta;
        t[x].left -= delta;
        t[x].right -= delta;
        t[x].lazy += delta;
    }
    void change(int x, int l, int r, int ql, int qr, ll delta)
    {
        if (ql <= l && r <= qr)
        {
            apply(x, delta);
            return;
        }
        apply(x * 2, t[x].lazy);
        apply(x * 2 + 1, t[x].lazy);
        t[x].lazy = 0;
        int mid = (l + r) / 2;
        if (ql <= mid) change(x * 2, l, mid, ql, qr, delta);
        if (qr > mid) change(x * 2 + 1, mid + 1, r, ql, qr, delta);
        t[x] = merge(t[x * 2], t[x * 2 + 1]);
    }
    void update(int id, ll w)
    {
        assert(size && 0 <= id && id < (int)edges.size());
        assert(w >= 0 && w <= LLONG_MAX / 4 - (total - edges[id].w));
        ll delta = w - edges[id].w;
        total += delta;
        edges[id].w = w;
        int u = child[id];
        change(1, 0, size - 1, first[u], last[u], delta);
    }
    ll diameter() const
    {
        assert(size);
        return t[1].best;
    }
};
```

模板题使用：CEOI2019 Dynamic Diameter（LOJ3163 待复核提交）

题意：每次修改一条边后输出直径；用上次答案解码边号和新权，边编号 0-based。

所需模板：DynamicDiameter（第 633 页，42.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://loj.ac/p/3163

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    long long w;
    cin >> n >> q >> w;
    DynamicDiameter tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        tree.add(u, v, c);
    }
    tree.build();
    long long last = 0;
    while (q--)
    {
        long long d, e;
        cin >> d >> e;
        int id = (d + last) % (n - 1);
        tree.update(id, (e + last) % w);
        last = tree.diameter();
        cout << last << '\n';
    }
}
```

42.1.2 树上点集直径摘要

默认空点集，a=b=-1、length=0。insert(v,distance) 插入顶点，重复插入合法；merge(other,distance) 合并两个点集摘要，允许自合并；farthest(v,distance) 返回 optional<pair<int,long long»（最远顶点、距离），空集返回 nullopt，距离并列时可返回任意最远点。distance(u,v) 回调必须精确返回同一棵固定非负权树的距离，点号为非负整数；所有距离 ≤LLONG_MAX。每次插入或最远查询最多两次距离调用，合并最多四次，摘要只占 O(1) 空间。保存 a、b 为点集中某对直径端点，不保留全部点；字段只读，不能用一般图度量或负权树代替。复制摘要即可建立前缀、后缀或线段树聚合，重置用默认构造。

```
struct TreeDiameter
{
    using ll = long long;
    int a = -1, b = -1;
    ll length = 0;

    // distance is the metric of one fixed tree with nonnegative edge weights.
    template <class Distance> void insert(int v, const Distance &distance)
    {
        assert(v >= 0);
        if (a == -1)
        {
            a = v;
            b = v;
            return;
        }
        ll x = distance(a, v), y = distance(b, v);
        if (x >= y && x > length)
        {
            b = v;
            length = x;
        }
        else if (y > length)
        {
            a = v;
            length = y;
        }
    }
```

合并与最远点查询（接上页同一结构体）：

```
    template <class Distance> void merge(TreeDiameter other, const Distance &distance)
    {
        if (other.a == -1) return;
        insert(other.a, distance);
        insert(other.b, distance);
    }

    template <class Distance>
    optional<pair<int, ll>> farthest(int v, const Distance &distance) const
    {
        if (a == -1) return nullopt;
        ll x = distance(a, v), y = distance(b, v);
        return x >= y ? make_pair(a, x) : make_pair(b, y);
    }
};
```

应用题补充：Codeforces 379F New Year Tree

题意：初始 1 连接 2、3、4；每次给指定叶子添两个新叶子，输出直径边数。q≤500000，最终点数 4+2q。先离线建最终树并做 HLD，再按编号顺序 insert；添叶不会改变旧点间距离，满足固定非负权树前提。只维护已出现顶点。O(q log q) 时间、O(q) 空间，DFS 递归；只调用 insert 及 length，merge/farthest 由接口测试验证。

所需模板：HLD（第 659 页，42.3.2 节）、TreeDiameter（第 638 页，42.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/379/F

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    int n = 4 + 2 * q;
    HLD tree(n);
    for (int v = 2; v <= 4; v++) tree.add(1, v);
    for (int i = 0; i < q; i++)
    {
        int v;
        cin >> v;
        tree.add(v, 5 + 2 * i);
        tree.add(v, 6 + 2 * i);
    }
    tree.build();
    auto dis = [&](int u, int v)
    {
        return tree.dep[u] + tree.dep[v] - 2 * tree.dep[tree.lca(u, v)];
    };
    TreeDiameter cur;
    for (int v = 1; v <= 4; v++) cur.insert(v, dis);
    for (int i = 0; i < q; i++)
    {
        cur.insert(5 + 2 * i, dis);
        cur.insert(6 + 2 * i, dis);
        cout << cur.length << '\n';
    }
    return 0;
}
```

42.2 最近公共祖先

42.2.1 离线 Tarjan LCA

构造 OfflineLCA(n)，add(u,v) 加无向树边，add_query(u,v) 登记查询并返回从 0 开始的编号，run(root=1) 后读取 answer[id]。输入必须是非空树，点号 1..n；允许同点、重复查询及零个查询。run 可重复调用或换根，不清空查询；追加查询后需重新 run，旧答案在重算前不可按新根使用。DFS 与并查集查找均递归。并查集按大小合并并压缩路径，ancestor[find(u)] 保存已合并部分对应的当前树祖先；不能直接拿并查集代表当 LCA。每个子树完成后合并到父点并更新 ancestor；查询另一端已完成时，从其集合读取祖先。总时间 O((n+q) α(n))，空间 O(n+q)。全部点对和根的父链对拍、50 万点链及 P3379、POJ1470 完整驱动本地验证，在线待补。

```
struct OfflineLCA
{
    int n;
    vector<vector<int>> g;
    vector<vector<pair<int, int>>> queries;
    vector<int> parent, size, ancestor, done, answer;

    OfflineLCA(int n)
        : n(n),
          g(n + 1),
          queries(n + 1),
          parent(n + 1),
          size(n + 1),
          ancestor(n + 1),
          done(n + 1)
    {
    }

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        g[u].push_back(v);
        g[v].push_back(u);
    }

    int add_query(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        int id = answer.size();
        answer.push_back(0);
        queries[u].push_back({v, id});
        queries[v].push_back({u, id});
        return id;
    }
```

```
    int find(int u)
    {
        if (parent[u] != u) parent[u] = find(parent[u]);
        return parent[u];
    }

    void merge(int u, int v)
    {
        u = find(u);
        v = find(v);
        if (size[u] < size[v]) swap(u, v);
        parent[v] = u;
        size[u] += size[v];
    }

    void dfs(int u, int p)
    {
        for (int v : g[u])
            if (v != p)
            {
                dfs(v, u);
                merge(u, v);
                ancestor[find(u)] = u;
            }
        done[u] = 1;
        for (auto [v, id] : queries[u])
            if (done[v]) answer[id] = ancestor[find(v)];
    }

    // Input must be a nonempty tree. Query IDs are 0-based.
    void run(int root = 1)
    {
        assert(1 <= root && root <= n);
        iota(parent.begin(), parent.end(), 0);
        iota(ancestor.begin(), ancestor.end(), 0);
        fill(size.begin(), size.end(), 1);
        fill(done.begin(), done.end(), 0);
        fill(answer.begin(), answer.end(), 0);
        dfs(root, 0);
    }
};
```

模板题使用：Luogu P3379

题意：给定以 root 为根的树，回答两点的最近公共祖先。先登记全部查询，再 run(root)，answer 按查询加入顺序输出；点号为 1..n。

所需模板：OfflineLCA（第 641 页，42.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3379

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, root;
    cin >> n >> m >> root;
    OfflineLCA graph(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add_query(u, v);
    }
    graph.run(root);
    for (int u : graph.answer) cout << u << '\n';
    return 0;
}
```

42.2.2 欧拉序与 ST 表 LCA

构造 EulerLCA(n)，add(u,v,w=1) 加非负权无向树边，build(root=1) 预处理；lca(u,v) 返回最近公共祖先，distance(u,v) 仍返回无权路径边数；weighted_distance(u,v) 返回 long long 带权距离。边权非负，所有树上路径的权和须 ≤LLONG_MAX。输入必须为非空树，点号 1..n；允许同点查询和重复 build 换根。只可在 build 后查询，加边会清除 ready 标志。递归 DFS 在进入顶点及从每个儿子返回时记录顶点，euler 长度为 2n-1，first[u] 是从 0 开始的首次出现位置；depth[root]=0。两点首次出现位置间深度最小的顶点就是 LCA，用 ST 表保存该顶点。预处理 O(n log n) 时间和空间，每次查询 O(1)。ST 内部下标从 0 开始；first、depth、euler、root_dist 可直接读取；root_dist 为当前根的带权距离。50 万点链、全部小树根与点对以及 P3379、POJ1330 双风格完整程序本地验证；上述无权题在线待补；QOJ8235 的带权应用已双 AC。

```
struct EulerLCA
{
    using ll = long long;
    int n;
    bool ready = false;
    vector<vector<int>> g, st;
    vector<vector<ll>> cost;
    vector<ll> root_dist;
    vector<int> first, depth, euler, lg;

    EulerLCA(int n)
        : n(n), g(n + 1), cost(n + 1), root_dist(n + 1), first(n + 1), depth(n + 1)
    {
    }

    void add(int u, int v, ll w = 1)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(w >= 0);
        g[u].push_back(v);
        g[v].push_back(u);
        cost[u].push_back(w);
        cost[v].push_back(w);
        ready = false;
    }

    int shallower(int u, int v) const { return depth[u] <= depth[v] ? u : v; }
```

递归欧拉序与 RMQ（接上页同一结构体）：

```
    void dfs(int u, int parent)
    {
        first[u] = euler.size();
        euler.push_back(u);
        for (int i = 0; i < int(g[u].size()); i++)
        {
            int v = g[u][i];
            if (v != parent)
            {
                depth[v] = depth[u] + 1;
                root_dist[v] = root_dist[u] + cost[u][i];
                dfs(v, u);
                euler.push_back(u);
            }
        }
    }

    // Input must be a nonempty tree. Rebuilding may change the root.
    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        euler.clear();
        depth[root] = 0;
        root_dist[root] = 0;
        dfs(root, 0);
        int m = euler.size();
        lg.assign(m + 1, 0);
        for (int i = 2; i <= m; i++) lg[i] = lg[i / 2] + 1;
        st.assign(lg[m] + 1, {});
        st[0] = euler;
        for (int k = 1; k <= lg[m]; k++)
        {
            st[k].resize(m - (1 << k) + 1);
            for (int i = 0; i + (1 << k) <= m; i++)
                st[k][i] = shallower(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
        }
        ready = true;
    }
```

LCA、边数距离与带权距离（接上页同一结构体）：

```
    int lca(int u, int v) const
    {
        assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
        int l = first[u], r = first[v];
        if (l > r) swap(l, r);
        int k = lg[r - l + 1];
        return shallower(st[k][l], st[k][r - (1 << k) + 1]);
    }

    int distance(int u, int v) const
    {
        int z = lca(u, v);
        return depth[u] + depth[v] - 2 * depth[z];
    }

    ll weighted_distance(int u, int v) const
    {
        int z = lca(u, v);
        return (root_dist[u] - root_dist[z]) + (root_dist[v] - root_dist[z]);
    }
};
```

模板题使用：Luogu P3379

题意：给定以 root 为根的树，回答两点的最近公共祖先。加完所有边后 build(root)，再逐条调用 lca；点号为 1..n。此题只使用无权 LCA 接口。

所需模板：EulerLCA（第 644 页，42.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3379

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q, root;
    cin >> n >> q >> root;
    EulerLCA graph(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    graph.build(root);
    while (q--)
    {
        int u, v;
        cin >> u >> v;
        cout << graph.lca(u, v) << '\n';
    }
    return 0;
}
```

42.2.3 倍增 LCA、祖先与最大边权

构造 LiftingLCA(n)，add(u,v,w=1) 加边，build(root=1) 预处理非空无向树，点号 1..n。jump(u,k) 返回 k 级祖先，k 非负，超过根返回 0；lca(u,v) 求最近公共祖先；distance 返回路径边数，不是权值和；max_edge 返回路径最大边权，同点返回 nullopt，其他情况返回有值的 optional<long long>。支持负权和 LLONG_MIN，LOW 只是最大值合并的单位元，不表示空路径。DFS 递归，根深度为 0；up[k][u] 与 mx[k][u] 是 2 的 k 次方长度向上段的祖先与最大边权，0 号节点为哨兵。可重复 build 换根；修改图后必须重建。预处理 O(n log n) 时间与空间，各查询 O(log n)。不需要边权最值时，可删去 mx 和 max_edge 相关部分。全小树点对/根、极值权重及 50 万点负权链本地验证；P3379、POJ1330 驱动只验证 LCA，最大边权仍是本地证据。

```
struct LiftingLCA
{
    using ll = long long;
    static constexpr ll LOW = numeric_limits<ll>::lowest();
    int n, levels;
    bool ready = false;
    vector<vector<pair<int, ll>>> g;
    vector<vector<int>> up;
    vector<vector<ll>> mx;
    vector<int> depth;

    LiftingLCA(int n) : n(n), g(n + 1), depth(n + 1)
    {
        assert(n > 0);
        levels = 1;
        while ((1LL << levels) <= n) levels++;
        up.assign(levels, vector<int>(n + 1));
        mx.assign(levels, vector<ll>(n + 1, LOW));
    }

    void add(int u, int v, ll w = 1)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
        ready = false;
    }
```

```
    void dfs(int u, int parent, ll weight)
    {
        up[0][u] = parent;
        mx[0][u] = weight;
        for (int k = 1; k < levels; k++)
        {
            int p = up[k - 1][u];
            up[k][u] = up[k - 1][p];
            mx[k][u] = max(mx[k - 1][u], mx[k - 1][p]);
        }
        for (auto [v, w] : g[u])
            if (v != parent)
            {
                depth[v] = depth[u] + 1;
                dfs(v, u, w);
            }
    }

    // Nonempty tree only. Node 0 means no ancestor.
    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        depth[root] = 0;
        dfs(root, 0, LOW);
        ready = true;
    }

    int jump(int u, long long steps) const
    {
        assert(ready && 1 <= u && u <= n && steps >= 0);
        if (steps > depth[u]) return 0;
        for (int k = 0; k < levels; k++)
            if (steps >> k & 1) u = up[k][u];
        return u;
    }
```

```
65      int lca(int u, int v) const
66      {
67          assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
68          if (depth[u] < depth[v]) swap(u, v);
69          u = jump(u, depth[u] - depth[v]);
70          if (u == v) return u;
71          for (int k = levels - 1; k >= 0; k--)
72              if (up[k][u] != up[k][v])
73              {
74                  u = up[k][u];
75                  v = up[k][v];
76              }
77          return up[0][u];
78      }
79
80      int distance(int u, int v) const
81      {
82          return depth[u] + depth[v] - 2 * depth[lca(u, v)];
83      }
84
85      optional<ll> max_edge(int u, int v) const
86      {
87          assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
88          if (u == v) return nullopt;
89          if (depth[u] < depth[v]) swap(u, v);
90          ll answer = LOW;
91          int delta = depth[u] - depth[v];
92          for (int k = 0; k < levels; k++)
93              if (delta >> k & 1)
94              {
95                  answer = max(answer, mx[k][u]);
96                  u = up[k][u];
97              }
98          if (u == v) return answer;
99          for (int k = levels - 1; k >= 0; k--)
100             if (up[k][u] != up[k][v])
101             {
102                 answer = max(answer, max(mx[k][u], mx[k][v]));
103                 u = up[k][u];
104                 v = up[k][v];
105             }
106         return max(answer, max(mx[0][u], mx[0][v]));
107     }
108 };
```

模板题使用：Luogu P3379

题意：给定以 root 为根的树，回答两点的最近公共祖先。加完所有边后 build(root)，再逐条调用 lca；点号为 1..n。此题不涉及跳祖先或路径最大边权。

所需模板：LiftingLCA（第 647 页，42.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3379

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q, root;
    cin >> n >> q >> root;
    LiftingLCA graph(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    graph.build(root);
    while (q--)
    {
        int u, v;
        cin >> u >> v;
        cout << graph.lca(u, v) << '\n';
    }
    return 0;
}
```

应用题补充：SPOJ QTREE2 / 路径距离与第 k 个点

题意：多测 t≤25、n≤10000、边长 c≤100000，DIST u v 查距离，KTH u v k 查从 u 到 v 的第 k 个点（1-based 且合法），DONE 结束，每组后空行。根距离 d 额外用递归 DFS 求出；切勿把 h.distance 的边数当权值和。令 a、b 为两端到 LCA 的边数，k≤a+1 从 u 跳 k−1，否则从 v 跳 a+b+1−k。预处理 O(n log n)、查询 O(log n)、空间 O(n log n)；本用法所有根距离、差和答案须在 long long 范围。

所需模板：LiftingLCA（第 647 页，42.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.spoj.com/problems/QTREE2/

SPOJ QTREE2 / 路径距离与第 k 个点：使用代码

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        LiftingLCA h(n);
        for (int i = 1; i < n; i++)
        {
            int u, v;
            long long w;
            cin >> u >> v >> w;
            h.add(u, v, w);
        }
        h.build();
        vector<long long> d(n + 1);
        auto dfs = [&](auto &&self, int u, int p) -> void
        {
            for (auto [v, w] : h.g[u])
            {
                if (v == p) continue;
                d[v] = d[u] + w;
                self(self, v, u);
            }
        };
        dfs(dfs, 1, 0);
        string cmd;
        while (cin >> cmd && cmd != "DONE")
        {
            int u, v;
            cin >> u >> v;
            int p = h.lca(u, v);
            if (cmd == "DIST") cout << (d[u] - d[p]) + (d[v] - d[p]) << '\n';
            else
            {
                int k;
                cin >> k;
                int a = h.depth[u] - h.depth[p];
                int b = h.depth[v] - h.depth[p];
                int ans = k <= a + 1 ? h.jump(u, k - 1) : h.jump(v, a + b + 1 - k);
                cout << ans << '\n';
            }
        }
        cout << '\n';
    }
}
```

42.2.4 两条树路径的交集

path_intersection(a,b,c,d,depth,lca) 返回路径 a-b 与 c-d 的公共路径，字段 u、v 是交集端点，vertices 是公共顶点数；空交返回 {0,0,0}，单点交的 u=v。点号从 1 开始，depth 数组与 lca 回调必须来自同一棵非空树、同一个根，深度按边数计算，不能传带权根距离。支持数组或 vector 深度表，回调接收两个点并返回 LCA。将 c、d 分别投影到 a-b 得 p、q；投影是 lca(a,b)、lca(a,x)、lca(b,x) 中最深者。若 p≠q，c-d 必经过 p-q；若 p=q，检验 p 是否在 c-d 上即可区分单点交与空交。不依赖顶点编号大小。常数次 LCA 调用、O(1) 额外空间；使用 EulerLCA 时为 O(1)，HLD/倍增时为 O(log n)。公共边数为 max(0,vertices-1)。全小树根与四端点、随机换号树用路径点集独立验证；P3398 完整程序只检查是否相交，端点及数量由本地验证覆盖。

```
struct PathIntersection
{
    int u, v, vertices;
};

// Same nonempty tree and root for depth[] and lca(). Empty result is {0,0,0}.
template <class Depth, class Lca>
PathIntersection
path_intersection(int a, int b, int c, int d, const Depth &depth, Lca lca)
{
    int z = lca(a, b);
    auto project = [&](int x)
    {
        int p = z;
        for (int v : {lca(a, x), lca(b, x)})
            if (depth[v] > depth[p]) p = v;
        return p;
    };
    auto distance = [&](int u, int v)
    {
        return 1LL * depth[u] + depth[v] - 2LL * depth[lca(u, v)];
    };
    int p = project(c), q = project(d);
    if (p == q && distance(c, p) + distance(p, d) != distance(c, d)) return {0, 0, 0};
    return {p, q, int(distance(p, q) + 1)};
}
```

应用题补充：Luogu P3398

题意：树上两条路径是否有公共顶点，n,q≤100000。点号 1..n，EulerLCA 的 depth 与 lca 必须来自同一根，depth 是边数而非带权距离。vertices 非零输出 Y，否则 N；此题只使用是否相交，交集端点和顶点数另由本地路径点集验证。EulerLCA 预处理 O(n log n)，每次查询 O(1)，DFS 递归。

所需模板：EulerLCA（第 644 页，42.2.2 节）、path_intersection（第 652 页，42.2.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3398

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    EulerLCA tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        tree.add(u, v);
    }
    tree.build();
    auto lca = [&](int u, int v)
    {
        return tree.lca(u, v);
    };
    while (q--)
    {
        int a, b, c, d;
        cin >> a >> b >> c >> d;
        auto result = path_intersection(a, b, c, d, tree.depth, lca);
        cout << (result.vertices ? 'Y' : 'N') << '\n';
    }
    return 0;
}
```

42.3 树链剖分

42.3.1 长链剖分与常数时间 K 级祖先

LongChain(n)，1≤n<INT_MAX，点号 1..n；add(u,v) 加入无向树边，调用方保证全部边恰好构成一棵树，不支持环、重边、自环或森林。build(root=1) 以指定根重建，dep[root]=1；kth(u,k) 返回 k 级祖先，k≥0，k=0 返回自身，越过根返回 0。查询前必须 build，加边后必须重建；支持重复 build、换根与值复制。son 按子树最大高度选长儿子，不能换成子树大小；len 含自身。链顶 t 的 down[t] 从 t 向下，up[t][0]=t 且向上越根补 0；成员只读，dfs1/dfs2 仅内部使用。预处理 O(n log(n+1))，空间 O(n log(n+1))，两次 DFS 均递归、栈深 O(n)。查询先跳最高 2 的幂，再在链顶上下表中查一次，32 位字长模型下 O(1)；k=0 必须先处理，不能求 0 的最高位。

```
struct LongChain
{
    int n;
    vector<int> dep, len, son, top;
    vector<vector<int>> g, fa, up, down;

    LongChain(int n) : n(n), g(n + 1)
    {
        assert(1 <= n && n < INT_MAX);
    }

    void add(int u, int v)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs1(int u, int p)
    {
        fa[0][u] = p;
        dep[u] = dep[p] + 1;
        for (int j = 1; j < (int)fa.size(); j++)
            fa[j][u] = fa[j - 1][fa[j - 1][u]];
        for (int v : g[u])
        {
            if (v == p) continue;
            dfs1(v, u);
            if (len[v] + 1 > len[u])
            {
                len[u] = len[v] + 1;
                son[u] = v;
            }
        }
    }
```

递归划分长链并填表（接上页同一结构体）：

```
    void dfs2(int u, int t)
    {
        top[u] = t;
        if (u == t)
        {
            down[t].reserve(len[u]);
            up[t].resize(len[u]);
            int x = u;
            for (int &v : up[t])
            {
                v = x;
                x = fa[0][x];
            }
        }
        down[t].push_back(u);
        if (son[u]) dfs2(son[u], t);
        for (int v : g[u])
        {
            if (v != fa[0][u] && v != son[u])
                dfs2(v, v);
        }
    }
```

重建与常数时间查询（接上页同一结构体）：

```
    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        dep.assign(n + 1, 0);
        len.assign(n + 1, 1);
        son.assign(n + 1, 0);
        top.assign(n + 1, 0);
        fa.assign(bit_width((unsigned)n), vector<int>(n + 1));
        up.assign(n + 1, {});
        down.assign(n + 1, {});
        dfs1(root, 0);
        dfs2(root, root);
    }

    int kth(int u, int k) const
    {
        assert(1 <= u && u <= n && k >= 0 && !fa.empty());
        if (k >= dep[u]) return 0;
        if (k == 0) return u;
        int j = bit_width((unsigned)k) - 1;
        u = fa[j][u];
        k -= 1 << j;
        int t = top[u];
        int d = dep[u] - dep[t];
        if (k <= d) return down[t][d - k];
        return up[t][k - d];
    }
};
```

为什么一次跳跃就够 先向上跳 p（不超过 k 的最大 2 的幂），到达 u，剩余 r 小于 p。原查询点是 u 下方距离 p 的后代，所以 u 的子树高度至少 p+1。u 所在长链顶 t 的链长至少 dep[u]-dep[t]+p+1，足以覆盖 r；若 r 不超过 u 到 t 的距离就查 down，否则查 up。选择长儿子必须按高度，不能按子树大小。

边界与空间 up[t][0] 和 down[t][0] 都等于 t；up 越过根填 0。先处理 k=0 和越根，避免对 0 求最高位。各长链长度之和为 n，上下表各存 n 个 int；倍增表为 O(n log n)。预处理两次 DFS 都保持递归，栈深可达 n。

模板题使用：P5903 【模板】树上 K 级祖先

题意：按题意用 32 位无符号 xorshift 顺序生成询问；根深度为 1，先生成 x 再生成 k，按 64 位累计 i* 答案的异或；n≤500000，q≤5000000。

所需模板：LongChain（第 654 页，42.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5903

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    uint32_t s;
    cin >> n >> q >> s;
    LongChain tree(n);
    int root = 0;
    for (int u = 1; u <= n; u++)
    {
        int p;
        cin >> p;
        if (p) tree.add(u, p);
        else root = u;
    }
    tree.build(root);
    auto get = [&]()
    {
        s ^= s << 13;
        s ^= s >> 17;
        s ^= s << 5;
        return s;
    };
    int last = 0;
    unsigned long long ans = 0;
    for (int i = 1; i <= q; i++)
    {
        int x = (get() ^ last) % n + 1;
        int k = (get() ^ last) % tree.dep[x];
        last = tree.kth(x, k);
        ans ^= 1ULL * i * last;
    }
    cout << ans << '\n';
}
```

接口用法演示：API 任意根与越根祖先

演示约定：输入 n root q、n-1 条 1-based 树边和 q 组 u k；逐行输出 k 级祖先，k=0 为自身，越根为 0。

所需模板：LongChain（第 654 页，42.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/hld/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, root, q;
    cin >> n >> root >> q;
    LongChain tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        tree.add(u, v);
    }
    tree.build(root);
    while (q--)
    {
        int u, k;
        cin >> u >> k;
        cout << tree.kth(u, k) << '\n';
    }
}
```

42.3.2 重链剖分与 LCA

add(u,v) 建无向树；build(root) 后读取 fa、dep、siz、son、top、dfn、rk。子树对应 [dfn[u],dfn[u]+siz[u]-1]。lca(u,v) 为 O(log n)，path(u,v,work,edge) 分解路径；work(l,r) 仅适用于可交换聚合，边权模式排除 LCA 位置。输入必须是连通无环树，建树 O(n)，两遍递归 DFS；支持改变根后重复 build，dfn 会重新编号。path_ordered(u, v, work, edge) 的 edge 默认为 false；按真实 u 到 v 顺序回调 work(l, r, rev)：1-based 闭区间 l≤r，rev=true 按 r 到 l 访问，否则按 l 到 r。支持非交换聚合，O(log n) 段与额外空间；结合线段树通常每次查询 $O(\log^2 n)$。edge=true 仅删除 LCA 对应位置，仍须自行处理边算子的方向；不自动求逆。u=v 时点模式一次单点回调，边模式无回调；改变根后须重建依赖 dfn 的结构。

```
struct HLD
{
    int n, timer = 0;
    vector<vector<int>> g;
    vector<int> fa, dep, siz, son, top, dfn, rk;

    HLD(int n)
        : n(n),
          g(n + 1),
          fa(n + 1),
          dep(n + 1),
          siz(n + 1),
          son(n + 1),
          top(n + 1),
          dfn(n + 1),
          rk(n + 1)
    {
    }

    void add(int u, int v)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs1(int u, int p)
    {
        fa[u] = p;
        siz[u] = 1;
        son[u] = 0;
        for (int v : g[u])
        {
            if (v == p) continue;
            dep[v] = dep[u] + 1;
            dfs1(v, u);
            siz[u] += siz[v];
            if (!son[u] || siz[v] > siz[son[u]]) son[u] = v;
        }
    }
```

重编号、LCA 与可交换路径（接上页同一结构体）：

```
    void dfs2(int u, int t)
    {
        top[u] = t;
        dfn[u] = ++timer;
        rk[timer] = u;
        if (son[u]) dfs2(son[u], t);
        for (int v : g[u])
        {
            if (v != fa[u] && v != son[u]) dfs2(v, v);
        }
    }

    // Connected tree only. Recursive DFS; subtree interval is [dfn, dfn+siz-1].
    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        timer = 0;
        dep[root] = 0;
        dfs1(root, root);
        dfs2(root, root);
        assert(timer == n);
    }

    int lca(int u, int v) const
    {
        while (top[u] != top[v])
        {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            u = fa[top[u]];
        }
        return dep[u] < dep[v] ? u : v;
    }

    // Commutative operations only; edge=true excludes the LCA's position.
    template <class F> void path(int u, int v, F work, bool edge = false) const
    {
        while (top[u] != top[v])
        {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            work(dfn[top[u]], dfn[u]);
            u = fa[top[u]];
        }
        if (dep[u] > dep[v]) swap(u, v);
        if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v]);
    }
```

按真实方向分解路径（接上页同一结构体）：

```
    // Emit u-to-v order; reverse=true traverses the closed interval from r to l.
    template <class F> void path_ordered(int u, int v, F work, bool edge = false) const
    {
        vector<pair<int, int>> down;
        while (top[u] != top[v])
        {
            if (dep[top[u]] >= dep[top[v]])
            {
                work(dfn[top[u]], dfn[u], true);
                u = fa[top[u]];
            }
            else
            {
                down.emplace_back(dfn[top[v]], dfn[v]);
                v = fa[top[v]];
            }
        }
        if (dep[u] >= dep[v])
        {
            if (dfn[v] + edge <= dfn[u]) work(dfn[v] + edge, dfn[u], true);
        }
        else if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v], false);
        for (auto it = down.rbegin(); it != down.rend(); ++it)
            work(it->first, it->second, false);
    }
};
```

模板题使用：Luogu P3384

题意：给定有根点权树，支持路径加、路径和、子树加、子树和，答案模 mod。先按 dfn 重排初值再构造线段树；path 回调给出 1-based 闭区间，子树区间为 [dfn[u],dfn[u]+siz[u]-1]。

所需模板：HLD（第 659 页，42.3.2 节）、AffineSegTree（第 428 页，34.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3384

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, root, mod;
    cin >> n >> m >> root >> mod;
    vector<long long> values(n + 1);
    for (int u = 1; u <= n; u++) cin >> values[u];
    HLD h(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        h.add(u, v);
    }
    h.build(root);
    vector<long long> ordered(n);
    for (int u = 1; u <= n; u++) ordered[h.dfn[u] - 1] = values[u];
    AffineSegTree seg(ordered, mod);
    while (m--)
    {
        int op, u;
        cin >> op >> u;
        if (op == 1)
        {
            int v;
            long long x;
            cin >> v >> x;
            h.path(u, v, [&](int l, int r) { seg.update(l, r, 1, x); });
        }
        else if (op == 2)
        {
            int v;
            cin >> v;
            long long ans = 0;
            h.path(u, v, [&](int l, int r) { ans = (ans + seg.query(l, r)) % mod; });
            cout << ans << '\n';
        }
        else if (op == 3)
        {
            long long x;
            cin >> x;
            seg.update(h.dfn[u], h.dfn[u] + h.siz[u] - 1, 1, x);
        }
        else
            cout << seg.query(h.dfn[u], h.dfn[u] + h.siz[u] - 1) << '\n';
    }
}
```

模板题使用：Library Checker Vertex Set Path Composite

题意：静态无向树，1≤N,Q≤200000。点函数 f(x)=a*x+b，对 998244353 取模；题面 1≤a,c<mod，0≤b,d,x<mod。0 p c d 改单点函数，1 u v x 按 u 到 v（含两端点）依次代入。题目点号 0-based，传 HLD 时加 1；HLD 闭区间 [l,r] 对应 segtree 半开 [l−1,r)。S 存正向与反向复合，不能把组合顺序反过来或用逆函数代替反向序列。点改 O(log N)，路径 O($\log^2$ N)，空间 O(N)；保留递归 DFS，最大链须足够大栈。

所需模板：HLD（第 659 页，42.3.2 节）、segtree（第 422 页，34.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/vertex_set_path_composite

```
using S = array<long long, 4>;
const int mod = 998244353;

S op(S a, S b)
{
    return {b[0] * a[0] % mod,
            (b[0] * a[1] + b[1]) % mod,
            a[2] * b[2] % mod,
            (a[2] * b[3] + a[3]) % mod};
}

S e()
{
    return {1, 0, 1, 0};
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n), val(n);
    for (auto &x : a)
    {
        cin >> x[0] >> x[1];
        x[2] = x[0];
        x[3] = x[1];
    }
    HLD h(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        h.add(u + 1, v + 1);
    }
    h.build();
    for (int u = 1; u <= n; u++) val[h.dfn[u] - 1] = a[u - 1];
    segtree<S, op, e> s(val);
```

继续处理操作（接上页同一 main）：

```
    while (q--)
    {
        int t;
        cin >> t;
        if (t == 0)
        {
            int p;
            long long c, d;
            cin >> p >> c >> d;
            s.set(h.dfn[p + 1] - 1, {c, d, c, d});
        }
        else
        {
            int u, v;
            long long x;
            cin >> u >> v >> x;
            h.path_ordered(u + 1, v + 1, [&](int l, int r, bool rev)
            {
                S z = s.prod(l - 1, r);
                int k = rev ? 2 : 0;
                x = (z[k] * x + z[k + 1]) % mod;
            });
            cout << x << '\n';
        }
    }
}
```

接口用法演示：路径点权增减与点查（kuangbin 3.3.1 源码模型）

演示约定：读到 EOF：n,m,q、n 个初值、m 条树边，要求 m=n−1。I u v x/D u v x 给路径所有点加/减 x，Q u 查点值。1-based，初值保留在 a，Fenwick 仅记录增量差分；HLD 闭区间 [l,r] 在 l 加 x、r+1 减 x，包含 LCA 与两端。演示 n,q≤50000、| 初值 |≤10^12、0≤x≤10^6，全部中间值用 long long。每次更新 $O(\log^2 n)$、点查 O(log n)、空间 O(n)。原 HDU3966 题面未核验。

所需模板：HLD（第 659 页，42.3.2 节）、Fenwick（第 412 页，33.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/kuangbin/ACM-ICPC

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    while (cin >> n >> m >> q)
    {
        vector<long long> a(n + 1);
        for (int u = 1; u <= n; u++) cin >> a[u];
        HLD h(n);
        for (int i = 0; i < m; i++)
        {
            int u, v;
            cin >> u >> v;
            h.add(u, v);
        }
        h.build();
        auto bit = Fenwick<long long>(n);
        while (q--)
        {
            char op;
            int u;
            cin >> op >> u;
            if (op == 'Q') cout << a[u] + bit.sum(h.dfn[u]) << '\n';
            else
            {
                int v;
                long long x;
                cin >> v >> x;
                if (op == 'D') x = -x;
                h.path(u, v, [&](int l, int r)
                {
                    bit.add(l, x);
                    bit.add(r + 1, -x);
                });
            }
        }
    }
}
```

应用题补充：SPOJ QTREE / 单边修改与路径最大边权

题意：多测 t≤20、n≤10000。输入边按 1..n−1 编号；CHANGE i x 修改第 i 条边，QUERY u v 查路径最大边权，DONE 结束本组。把边存到较深端点的 dfn−1；path 的 edge=true 排除 LCA，闭 [l,r] 转为 segtree 的半开 [l−1,r)。空路径约定输出 0，非空路径支持 signed64 负权及最小值；题面仅写 c≤10^6，未写修改值下界。预处理 O(n)、修改 O(log n)、查询 O($\log^2$ n)、空间 O(n)，递归 DFS。

所需模板：HLD（第 659 页，42.3.2 节）、segtree（第 422 页，34.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.spoj.com/problems/QTREE/

```
auto op(long long x, long long y)
{
    return max(x, y);
}

auto e()
{
    return numeric_limits<long long>::lowest();
}
```

使用入口（接上页配置函数）：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        HLD h(n);
        vector<int> a(n), b(n), child(n);
        vector<long long> w(n), base(n, e());
        for (int i = 1; i < n; i++)
        {
            cin >> a[i] >> b[i] >> w[i];
            h.add(a[i], b[i]);
        }
        h.build();
        for (int i = 1; i < n; i++)
        {
            child[i] = h.dep[a[i]] > h.dep[b[i]] ? a[i] : b[i];
            base[h.dfn[child[i]] - 1] = w[i];
        }
        auto seg = segtree<long long, op, e>(base);
        string cmd;
        while (cin >> cmd && cmd != "DONE")
        {
            int u;
            cin >> u;
            if (cmd == "CHANGE")
            {
                long long x;
                cin >> x;
                seg.set(h.dfn[child[u]] - 1, x);
            }
            else
            {
                int v;
                cin >> v;
                long long ans = e();
                h.path(u, v, [&](int l, int r)
                {
                    ans = max(ans, seg.prod(l - 1, r));
                }, true);
                cout << (u == v ? 0 : ans) << '\n';
            }
        }
    }
}
```

接口用法演示：树边翻转与奇路径计数（kuangbin HDU5039 模型）

演示约定：来源协议：T 组，每组 n、n 个不同名字、n-1 条 (名字, 名字,0/1) 边、q 条操作；Q 开头查询，其余操作接 1-based 输入边号并翻转该边。输出 Case #k: 及每次有向端点对数量，故答案 2*zero*one；无向点对则不乘 2。固定根 1，根路径异或值不同的端点间为奇路径；翻边即翻较深端点整棵子树，HLD 闭区间转 lazy_segtree 半开 [dfn-1,dfn-1+siz)。树上预处理 O(n)，名字 map 映射另计字符串比较和字典查询；改边 O(log n)，查询 O(1)，空间 O(n)，DFS 递归。官方 HDU 题面和限制未核验，只作来源模型/API 用法。

所需模板：HLD（第 659 页，42.3.2 节）、lazy_segtree（第 417 页，34.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://acm.hdu.edu.cn/showproblem.php?pid=5039

```
using S = pair<int, int>;

S op(S a, S b)
{
    return {a.first + b.first, a.second + b.second};
}

S e()
{
    return {0, 0};
}

S mapping(bool flip, S a)
{
    if (flip) swap(a.first, a.second);
    return a;
}

bool composition(bool f, bool g)
{
    return f ^ g;
}

bool id()
{
    return false;
}
```

使用入口（接上页配置函数）：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    for (int tc = 1; tc <= tests; tc++)
    {
        int n;
        cin >> n;
        HLD h(n);
        map<string, int> names;
        for (int u = 1; u <= n; u++)
        {
            string name;
            cin >> name;
            names[name] = u;
        }
        vector<int> u(n), v(n), bit(n), child(n), parity(n + 1);
        for (int i = 1; i < n; i++)
        {
            string a, b;
            cin >> a >> b >> bit[i];
            u[i] = names.at(a);
            v[i] = names.at(b);
            h.add(u[i], v[i]);
        }
        h.build();
        for (int i = 1; i < n; i++)
        {
            child[i] = h.fa[u[i]] == v[i] ? u[i] : v[i];
            parity[child[i]] = bit[i];
        }
        vector<S> values(n);
        for (int i = 1; i <= n; i++)
        {
            int x = h.rk[i];
            if (x != 1) parity[x] ^= parity[h.fa[x]];
            values[i - 1] = {parity[x] == 0, parity[x] == 1};
        }
        lazy_segtree<S, bool, op, e, mapping, composition, id> seg(values);
```

继续处理操作（接上页同一 main）：

```
        int q;
        cin >> q;
        cout << "Case #" << tc << ":\n";
        while (q--)
        {
            string command;
            cin >> command;
            if (command[0] == 'Q')
            {
                auto [zero, one] = seg.all();
                cout << 2LL * zero * one << '\n';
            }
            else
            {
                int edge;
                cin >> edge;
                int x = child[edge];
                int l = h.dfn[x] - 1;
                seg.apply(l, l + h.siz[x], true);
            }
        }
    }
}
```

42.3.3 树上翻色、改权与同色连通块最大值

代码依赖：HLD（第 659 页，42.3.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

先 build 连通树 HLD h，再构造 ColorMax(h,color,w)，color 和 w 长 n+1、点 1..n，颜色为 0/1，权值支持 signed64 全范围。flip(u) 单点翻色，set_weight(u,w) 改权，query(u) 返回所在同色连通块最大权值；col、weight 可读。h 须比对象长寿，构造后不可修改或重新 build；其余成员仅供实现使用。

固定拓扑，初始化 O(n log n)，每次操作 O($\log^2$n)，空间 O(n)，HLD 使用递归 DFS。1≤n≤INT_MAX/2。轻儿子贡献按颜色存 multiset，重链连续同色区间用线段树取最大值；删除相等贡献只删一个。LLONG_MIN 仅作 max 单位元，不做加减，不会屏蔽合法最小权。

```
struct ColorMax
{
    const HLD &h;
    vector<int> col, tail;
    vector<long long> weight, seg;
    vector<array<multiset<long long>, 2>> light;
    array<set<int>, 2> pos;

    ColorMax(const HLD &tree, vector<int> color, vector<long long> w)
        : h(tree), col(move(color)), tail(h.n + 1), weight(move(w)),
          seg(2 * h.n, LLONG_MIN), light(h.n + 1)
    {
        assert(h.n >= 1 && h.n <= INT_MAX / 2 && h.timer == h.n);
        assert(col.size() == size_t(h.n + 1));
        assert(weight.size() == size_t(h.n + 1));
        vector<long long> f = weight;
        for (int i = h.n; i >= 1; i--)
        {
            int u = h.rk[i];
            assert(col[u] == 0 || col[u] == 1);
            pos[col[u]].insert(i);
            tail[h.top[u]] = max(tail[h.top[u]], i);
            int p = h.fa[u];
            if (p == u) continue;
            if (col[p] == col[u]) f[p] = max(f[p], f[u]);
            if (h.son[p] != u) light[p][col[u]].insert(f[u]);
        }
        for (int u = 1; u <= h.n; u++) seg[h.n + h.dfn[u] - 1] = base(u);
        for (int i = h.n - 1; i; i--) seg[i] = max(seg[2 * i], seg[2 * i + 1]);
    }
```

重链区间最大值与连通块最高点（接上页结构体）：

```
    long long base(int u) const
    {
        long long ans = weight[u];
        if (!light[u][col[u]].empty()) ans = max(ans, *light[u][col[u]].rbegin());
        return ans;
    }

    void put(int u)
    {
        int x = h.n + h.dfn[u] - 1;
        seg[x] = base(u);
        for (x /= 2; x; x /= 2) seg[x] = max(seg[2 * x], seg[2 * x + 1]);
    }

    long long down(int u) const
    {
        int r = tail[h.top[u]];
        auto it = pos[col[u] ^ 1].upper_bound(h.dfn[u]);
        if (it != pos[col[u] ^ 1].end()) r = min(r, *it - 1);
        long long ans = LLONG_MIN;
        int l = h.n + h.dfn[u] - 1;
        r += h.n;
        while (l < r)
        {
            if (l & 1) ans = max(ans, seg[l++]);
            if (r & 1) ans = max(ans, seg[--r]);
            l /= 2;
            r /= 2;
        }
        return ans;
    }

    int top(int u) const
    {
        int c = col[u];
        while (true)
        {
            int v = h.top[u];
            auto it = pos[c ^ 1].upper_bound(h.dfn[u]);
            if (it != pos[c ^ 1].begin())
            {
                --it;
                if (*it >= h.dfn[v]) return h.rk[*it + 1];
            }
            if (h.fa[v] == v || col[h.fa[v]] != c) return v;
            u = h.fa[v];
        }
    }
```

修改并向轻边父亲传播（接上页结构体）：

```
    void change(int u, int color, long long w)
    {
        assert(color == 0 || color == 1);
        bool first = true;
        while (true)
        {
            int t = h.top[u];
            int old = col[t];
            long long before = down(t);
            if (first)
            {
                pos[col[u]].erase(h.dfn[u]);
                col[u] = color;
                pos[col[u]].insert(h.dfn[u]);
                weight[u] = w;
                first = false;
            }
            put(u);
            if (h.fa[t] == t) break;
            int p = h.fa[t];
            auto it = light[p][old].find(before);
            assert(it != light[p][old].end());
            light[p][old].erase(it);
            light[p][col[t]].insert(down(t));
            u = p;
        }
    }

    void flip(int u)
    {
        change(u, col[u] ^ 1, weight[u]);
    }

    void set_weight(int u, long long w)
    {
        change(u, col[u], w);
    }

    long long query(int u) const
    {
        return down(top(u));
    }
};
```

维护不变量 light[u][c] 保存每个颜色 c 的轻儿子向下同色连通块最大值，保留重复项。线段树在 u 处存本点权与同色轻儿子贡献的最大值；down 取重链上从 u 开始的连续同色区间。更新前保存旧链头颜色和贡献，更新后删旧加新传给链头父亲，每次跨轻边，最多 O(log n) 层。

应用题补充：SPOJ QTREE7 / SP16580 同色连通块最大权

题意：输入 n、n−1 条边、初始颜色数组、权值数组、q 及操作；0 u 查询、1 u 翻色、2 u w 改权。驱动兼容 EOF 多组。镜像数值约束残缺，未推断上界；当前记录 302644732 为 Waiting，本地验证通过。

所需模板：HLD（第 659 页，42.3.2 节）、ColorMax（第 671 页，42.3.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/SP16580

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    while (cin >> n)
    {
        HLD h(n);
        for (int i = 1; i < n; i++)
        {
            int u, v;
            cin >> u >> v;
            h.add(u, v);
        }
        vector<int> color(n + 1);
        vector<long long> w(n + 1);
        for (int u = 1; u <= n; u++) cin >> color[u];
        for (int u = 1; u <= n; u++) cin >> w[u];
        h.build();
        ColorMax tree(h, color, w);
        int q;
        cin >> q;
        while (q--)
        {
            int op, u;
            cin >> op >> u;
            if (op == 0) cout << tree.query(u) << '\n';
            else if (op == 1) tree.flip(u);
            else
            {
                long long value;
                cin >> value;
                tree.set_weight(u, value);
            }
        }
    }
}
```

42.3.4 树上翻色与同色连通块

代码依赖：HLD（第 659 页，42.3.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

先构造并 build 连通树 HLD h，再构造 ColorComponents(h, color={})；点 1..n，颜色 0/1，省略 color 时全 0，否则传长 n+1 的数组。flip(u) 单点翻色，query(u) 返回 u 所在同色连通块大小，col[u] 可读。h 须比本对象长寿且不能再修改或 build；内部 pos、bit、add、change 不应直接修改或调用。

固定树拓扑，不支持 link/cut。初始化 O(n log n)，翻色/查询 O($\log^2$n)，额外空间 O(n)。依赖 HLD 的递归 DFS，需足够栈空间。1≤n≤INT_MAX/2，计数用 long long 维护、结果 ≤n。两色子树贡献沿祖先链传播，包含首个异色祖先；有序集合定位同色链顶。

```
struct ColorComponents
{
    const HLD &h;
    vector<int> col;
    array<set<int>, 2> pos;
    array<vector<long long>, 2> bit;

    ColorComponents(const HLD &tree, vector<int> color = {})
        : h(tree), col(move(color))
    {
        assert(h.n >= 1 && h.n <= INT_MAX / 2 && h.timer == h.n);
        if (col.empty()) col.resize(h.n + 1);
        assert(col.size() == size_t(h.n + 1));
        vector<array<int, 2>> f(h.n + 1);
        for (int i = h.n; i >= 1; i--)
        {
            int u = h.rk[i];
            assert(col[u] == 0 || col[u] == 1);
            pos[col[u]].insert(i);
            f[u][0]++;
            f[u][1]++;
            if (h.fa[u] != u) f[h.fa[u]][col[u]] += f[u][col[u]];
        }
        for (int c = 0; c < 2; c++)
        {
            bit[c].resize(h.n + 1);
            for (int i = 1; i <= h.n; i++)
                bit[c][i] = f[h.rk[i]][c] - f[h.rk[i - 1]][c];
            for (int i = 1; i <= h.n; i++)
            {
                int j = i + (i & -i);
                if (j <= h.n) bit[c][j] += bit[c][i];
            }
        }
    }
```

差分树状数组与同色链顶（接上页结构体）：

```
    void add(int c, int x, long long v)
    {
        for (; x <= h.n; x += x & -x) bit[c][x] += v;
    }

    long long value(int u, int c) const
    {
        long long ans = 0;
        for (int x = h.dfn[u]; x; x -= x & -x) ans += bit[c][x];
        return ans;
    }

    int top(int u) const
    {
        int c = col[u];
        while (true)
        {
            int v = h.top[u];
            auto it = pos[c ^ 1].upper_bound(h.dfn[u]);
            if (it != pos[c ^ 1].begin())
            {
                --it;
                if (*it >= h.dfn[v]) return h.rk[*it + 1];
            }
            if (h.fa[v] == v || col[h.fa[v]] != c) return v;
            u = h.fa[v];
        }
    }
```

贡献传播与公开操作（接上页结构体）：

```
    void change(int u, int c, long long delta)
    {
        int p = h.fa[u];
        if (p == u) return;
        int v = p;
        if (col[p] == c)
        {
            v = top(p);
            v = h.fa[v];
        }
        h.path(p, v, [&](int l, int r)
        {
            add(c, l, delta);
            add(c, r + 1, -delta);
        });
    }

    void flip(int u)
    {
        int c = col[u];
        change(u, c, -value(u, c));
        change(u, c ^ 1, value(u, c ^ 1));
        pos[c].erase(h.dfn[u]);
        col[u] ^= 1;
        pos[c ^ 1].insert(h.dfn[u]);
    }

    int query(int u) const
    {
        return value(top(u), col[u]);
    }
};
```

维护不变量 f[u][c] 为把 u 假定为颜色 c 时，其向下同色连通块大小。翻转 u 时，减去旧色贡献、加入新色贡献，更新从父亲起的同色祖先链，并包含首个异色祖先；它自身虽阻止继续传播，其假定颜色 c 的值仍需更新。query 从同色连通块最高点读取 f。

应用题补充：SPOJ QTREE6 / SP16549 树上同色连通块

题意：n,q≤100000，初始全黑。0 u 输出同色连通块大小；1 u 翻色。单组输入；记录 302642339 当前 Waiting，本地验证通过，不推断远端 AC。

所需模板：HLD（第 659 页，42.3.2 节）、ColorComponents（第 675 页，42.3.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/SP16549

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    HLD h(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        h.add(u, v);
    }
    h.build();
    ColorComponents tree(h);
    int q;
    cin >> q;
    while (q--)
    {
        int op, u;
        cin >> op >> u;
        if (op == 0) cout << tree.query(u) << '\n';
        else tree.flip(u);
    }
}
```

接口用法演示：任意初始颜色与根的翻色接口

演示约定：输入 n、root、q，再读 n 个 0/1 初始颜色、n−1 条边、q 个操作。0 u 查询，1 u 翻色；根只影响内部表示。

所需模板：HLD（第 659 页，42.3.2 节）、ColorComponents（第 675 页，42.3.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/COLOR-COMPONENTS.md

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, root, q;
    cin >> n >> root >> q;
    vector<int> color(n + 1);
    for (int u = 1; u <= n; u++) cin >> color[u];
    HLD h(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        h.add(u, v);
    }
    h.build(root);
    ColorComponents tree(h, color);
    while (q--)
    {
        int op, u;
        cin >> op >> u;
        if (op == 0) cout << tree.query(u) << '\n';
        else tree.flip(u);
    }
}
```

42.4 树上启发式合并

42.4.1 树上启发式合并：子树众数颜色和

SubtreeColors(n)，点号 1..n，add(u,v) 建无向树；要求图连通且无环，n=0 可空运行。run(colors,root=1) 传入长度 n+1 的 int 颜色数组，colors[0] 忽略。answer[u] 为子树内所有最高频颜色的原始编号之和，max_freq[u] 为该频次；支持负颜色与稀疏编号，内部离散化，和用 long long。重复 run 可更换颜色或根。重儿子信息保留，轻儿子通过 DFS 序区间加入；清空整棵临时子树后再重置最大值，不尝试逐次删除时维护众数。O(n log n) 时间、O(n) 空间，全部 DFS 递归。insert 是内部增量统计，可按题目修改统计量，但相应清空规则也必须调整。

```
struct SubtreeColors
{
    int n, best = 0;
    long long sum = 0;
    vector<vector<int>> g;
    vector<int> siz, heavy, tin, tout, order, color, value, freq;
    vector<int> max_freq;
    vector<long long> answer;

    SubtreeColors(int n) : n(n), g(n + 1) {}

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v);
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void prepare(int u, int p)
    {
        siz[u] = 1;
        tin[u] = order.size();
        order.push_back(u);
        for (int v : g[u])
        {
            if (v == p) continue;
            prepare(v, u);
            siz[u] += siz[v];
            if (siz[v] > siz[heavy[u]]) heavy[u] = v;
        }
        tout[u] = (int)order.size() - 1;
    }

    void insert(int u)
    {
        int c = color[u];
        freq[c]++;
        if (freq[c] > best)
        {
            best = freq[c];
            sum = value[c];
        }
        else if (freq[c] == best)
            sum += value[c];
    }
```

启发式合并与运行入口（接上页同一结构体）：

```
    void solve(int u, int p, bool keep)
    {
        for (int v : g[u])
        {
            if (v != p && v != heavy[u]) solve(v, u, false);
        }
        if (heavy[u]) solve(heavy[u], u, true);
        for (int v : g[u])
        {
            if (v == p || v == heavy[u]) continue;
            for (int i = tin[v]; i <= tout[v]; i++) insert(order[i]);
        }
        insert(u);
        max_freq[u] = best;
        answer[u] = sum;
        if (!keep)
        {
            for (int i = tin[u]; i <= tout[u]; i++) freq[color[order[i]]]--;
            best = 0;
            sum = 0;
        }
    }

    // A connected tree, colors[1..n]. Sum all colors tied for maximum frequency.
    void run(const vector<int> &colors, int root = 1)
    {
        assert((int)colors.size() == n + 1);
        siz.assign(n + 1, 0);
        heavy = siz;
        tin = siz;
        tout = siz;
        color = siz;
        max_freq = siz;
        answer.assign(n + 1, 0);
        order.clear();
        value.assign(colors.begin() + 1, colors.end());
        sort(value.begin(), value.end());
        value.erase(unique(value.begin(), value.end()), value.end());
        freq.assign(value.size(), 0);
        best = 0;
        sum = 0;
        if (!n) return;
        assert(1 <= root && root <= n);
        for (int u = 1; u <= n; u++)
            color[u] =
                lower_bound(value.begin(), value.end(), colors[u]) - value.begin();
        prepare(root, 0);
        solve(root, 0, false);
    }
};
```

应用题补充：Codeforces 600E

题意：以 1 为根，对每个子树输出出现次数最多的所有颜色编号之和；并列颜色都计入。颜色数组 1-based，run 后直接读 answer，和用 long long。n 至 10 万、颜色在 1..n；比赛应用，不替代正式模板题。

所需模板：SubtreeColors（第 680 页，42.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/600/E

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n;
    std::cin >> n;
    std::vector<int> colors(n + 1);
    for (int u = 1; u <= n; u++) std::cin >> colors[u];
    SubtreeColors tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        std::cin >> u >> v;
        tree.add(u, v);
    }
    tree.run(colors);
    for (int u = 1; u <= n; u++) std::cout << tree.answer[u] << (u == n ? '\n' : ' ');
}
```

42.5 虚树

42.5.1 虚树：关键点压缩与路径信息

VirtualTree(n)，n≥1，点号 1..n；add(u,v,w=1) 建非负权连通树，无环和重边。prepare(root=1) 递归预处理，可换根重建；根到任一点距离须在 long long 范围内。build(keys) 自动去重并补齐必要 LCA，返回 vertices 和 edges；全部编号沿用原树，vertices 按 DFS 序排列，非空时首项即虚树根。edges 为 (from,to,distance,minimum)，分别是祖先到后代的虚边、压缩路径长度和路径最小边权。空关键点返回空结果，单点返回单节点；原树根不会强制加入，需要时由调用者加入 keys。边按后代 DFS 序排列，可逆序做自底向上的 DP。不分配或清空 n 个查询邻接表；每次仅返回 O(k) 结果。预处理 O(n log n) 时间空间，单次 O(k log k+k log n)。lca、ancestor 也可直接使用，要求有效点号且已经 prepare。构造用的祖先链栈属于虚树算法，原树 DFS 保持递归。

```
struct VirtualTree
{
    using ll = long long;

    struct Edge
    {
        int from, to;
        ll distance, minimum;
    };

    struct Result
    {
        vector<int> vertices;
        vector<Edge> edges;
    };

    int n, log = 1, timer = 0;
    vector<vector<pair<int, ll>>> g;
    vector<vector<int>> up;
    vector<vector<ll>> mn;
    vector<int> depth, tin, tout;
    vector<ll> dist;

    VirtualTree(int n) : n(n), g(n + 1) {}

    void add(int u, int v, ll w = 1)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
```

预处理与最近公共祖先（接上页同一结构体）：

```
    void dfs(int u, int p)
    {
        tin[u] = ++timer;
        up[0][u] = p;
        for (int j = 1; j < log; j++)
        {
            up[j][u] = up[j - 1][up[j - 1][u]];
            mn[j][u] = min(mn[j - 1][u], mn[j - 1][up[j - 1][u]]);
        }
        for (auto [v, w] : g[u])
        {
            if (v == p) continue;
            assert(dist[u] <= numeric_limits<ll>::max() - w);
            depth[v] = depth[u] + 1;
            dist[v] = dist[u] + w;
            mn[0][v] = w;
            dfs(v, u);
        }
        tout[u] = timer;
    }

    void prepare(int root = 1)
    {
        assert(1 <= root && root <= n);
        log = 1;
        while ((1LL << log) <= n) log++;
        up.assign(log, vector<int>(n + 1));
        mn.assign(log, vector<ll>(n + 1, numeric_limits<ll>::max()));
        depth.assign(n + 1, 0);
        tin = depth;
        tout = depth;
        dist.assign(n + 1, 0);
        timer = 0;
        dfs(root, root);
    }

    bool ancestor(int u, int v) const { return tin[u] <= tin[v] && tout[v] <= tout[u]; }

    int lca(int u, int v) const
    {
        if (ancestor(u, v)) return u;
        if (ancestor(v, u)) return v;
        for (int j = log - 1; j >= 0; j--)
        {
            if (!ancestor(up[j][u], v)) u = up[j][u];
        }
        return up[0][u];
    }
```

压缩路径与虚树构造（接上页同一结构体）：

```
    Edge path(int u, int v) const
    {
        assert(u != v && ancestor(u, v));
        ll minimum = numeric_limits<ll>::max();
        int x = v, steps = depth[v] - depth[u];
        for (int j = 0; j < log; j++)
        {
            if (steps >> j & 1)
            {
                minimum = min(minimum, mn[j][x]);
                x = up[j][x];
            }
        }
        return {u, v, dist[v] - dist[u], minimum};
    }

    // Uses original vertex IDs. Empty input gives an empty result.
    Result build(vector<int> keys) const
    {
        if (keys.empty()) return {};
        for (int u : keys) assert(1 <= u && u <= n);
        auto cmp = [&](int u, int v)
        {
            return tin[u] < tin[v];
        };
        sort(keys.begin(), keys.end(), cmp);
        keys.erase(unique(keys.begin(), keys.end()), keys.end());
        int k = keys.size();
        for (int i = 1; i < k; i++) keys.push_back(lca(keys[i - 1], keys[i]));
        sort(keys.begin(), keys.end(), cmp);
        keys.erase(unique(keys.begin(), keys.end()), keys.end());
        Result result{keys, {}};
        vector<int> st;
        for (int u : keys)
        {
            while (!st.empty() && !ancestor(st.back(), u)) st.pop_back();
            if (!st.empty()) result.edges.push_back(path(st.back(), u));
            st.push_back(u);
        }
        return result;
    }
};
```

应用题补充：Luogu P2495 [SDOI2011] 消耗战

题意：每次给若干非根关键点，求删边使它们都与根 1 断开的最小代价。显式加入根构造虚树，逆序遍历虚边，使用路径最小边权与子树代价取小；只清空本次顶点和标记。n 至 25 万、询问及关键点总数至 50 万。省选应用，正式非比赛模板题映射仍待补。

所需模板：VirtualTree（第 683 页，42.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2495

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n;
    std::cin >> n;
    VirtualTree tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        long long w;
        std::cin >> u >> v >> w;
        tree.add(u, v, w);
    }
    tree.prepare();
    std::vector<int> marked(n + 1);
    std::vector<long long> dp(n + 1);
    int q;
    std::cin >> q;
    while (q--)
    {
        int k;
        std::cin >> k;
        std::vector<int> keys(k);
        for (int &u : keys)
        {
            std::cin >> u;
            marked[u] = 1;
        }
        keys.push_back(1);
        auto result = tree.build(keys);
        for (int u : result.vertices) dp[u] = 0;
        for (int i = (int)result.edges.size() - 1; i >= 0; i--)
        {
            auto e = result.edges[i];
            long long cost = marked[e.to] ? e.minimum : std::min(e.minimum, dp[e.to]);
            dp[e.from] += cost;
        }
        std::cout << dp[1] << '\n';
        for (int u : keys) marked[u] = 0;
    }
}
```

42.6 树分治

42.6.1 点分治：树上距离点对计数

CentroidPairs(n)，点号 1..n；add(u,v,w) 建树，允许森林但禁止环、重边、自环，边权非负；所有简单路径长度须能用 long long 表示。build() 预处理，可重复调用；改变边后须重建。count_leq(k) 统计距离 ≤k 的无序不同顶点对，count_exact(k) 统计距离恰为 k 的点对；负 k 返回 0，零距离只统计不同顶点，跨连通块不配对。parent[c] 是点分树父节点，根为 0。all[c] 为该层分治块到重心的距离，含重心距离 0；part[c] 为去掉重心后各子块的距离。预处理 $O(n \log^2 n)$，空间 $O(n \log n)$，每次计数 $O(n \log n)$；固定树上多次查询可以复用排序结果。DFS 递归，距离和比较采用减法避免溢出。不是动态点分树更新模板。

```
struct CentroidPairs
{
    using ll = long long;
    int n;
    vector<vector<pair<int, ll>>> g;
    vector<int> siz, removed, parent;
    vector<vector<ll>> all;
    vector<vector<vector<ll>>> part;

    CentroidPairs(int n)
        : n(n),
          g(n + 1),
          siz(n + 1),
          removed(n + 1),
          parent(n + 1),
          all(n + 1),
          part(n + 1)
    {
    }

    void add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }

    void size_dfs(int u, int p)
    {
        siz[u] = 1;
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            size_dfs(v, u);
            siz[u] += siz[v];
        }
    }
```

重心与分解（接上页同一结构体）：

```
    int centroid(int u, int p, int total)
    {
        for (auto [v, w] : g[u])
        {
            if (v != p && !removed[v] && siz[v] > total / 2)
                return centroid(v, u, total);
        }
        return u;
    }

    void collect(int u, int p, ll d, vector<ll> &out)
    {
        out.push_back(d);
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            assert(d <= numeric_limits<ll>::max() - w);
            collect(v, u, d + w, out);
        }
    }

    void decompose(int entry, int p)
    {
        size_dfs(entry, 0);
        int c = centroid(entry, 0, siz[entry]);
        removed[c] = 1;
        parent[c] = p;
        all[c] = {0};
        for (auto [v, w] : g[c])
        {
            if (removed[v]) continue;
            part[c].push_back({});
            auto &d = part[c].back();
            collect(v, c, w, d);
            sort(d.begin(), d.end());
            all[c].insert(all[c].end(), d.begin(), d.end());
        }
        sort(all[c].begin(), all[c].end());
        for (auto [v, w] : g[c])
        {
            if (!removed[v]) decompose(v, c);
        }
    }
```

重建与距离计数（接上页同一结构体）：

```
    // Input must be a forest. Rebuild after changing edges.
    void build()
    {
        fill(removed.begin(), removed.end(), 0);
        for (int u = 1; u <= n; u++)
        {
            all[u].clear();
            part[u].clear();
        }
        for (int u = 1; u <= n; u++)
        {
            if (!removed[u]) decompose(u, 0);
        }
    }

    ll pairs(const vector<ll> &d, ll k) const
    {
        int l = 0, r = (int)d.size() - 1;
        ll answer = 0;
        while (l < r)
        {
            if (d[l] <= k - d[r])
            {
                answer += r - l;
                l++;
            }
            else
                r--;
        }
        return answer;
    }

    // Unordered pairs of distinct connected vertices. Nonnegative edge weights.
    ll count_leq(ll k) const
    {
        if (k < 0) return 0;
        ll answer = 0;
        for (int c = 1; c <= n; c++)
        {
            answer += pairs(all[c], k);
            for (const auto &d : part[c]) answer -= pairs(d, k);
        }
        return answer;
    }

    ll count_exact(ll k) const
    {
        if (k < 0) return 0;
        return count_leq(k) - (k ? count_leq(k - 1) : 0);
    }
};
```

模板题使用：Luogu P3806

题意：正权树上查询距离为 k 的不同顶点对是否存在。先 build 一次，再用 count_exact 的结果是否非零输出 AYE/NAY。n 至 1 万、询问至 100、边权至 1 万；该程序查询存在性，底层计数另支持零权和森林。

所需模板：CentroidPairs（第 687 页，42.6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3806

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    CentroidPairs tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        long long w;
        std::cin >> u >> v >> w;
        tree.add(u, v, w);
    }
    tree.build();
    while (m--)
    {
        long long k;
        std::cin >> k;
        std::cout << (tree.count_exact(k) ? "AYE" : "NAY") << '\n';
    }
}
```

42.6.2 树上新增市场的支配范围

固定非负边权树，n≥1，点号 0..n-1；add(u,v,w)，w 为非负 long long。solve(existing) 返回各点新建市场后支配的顶点数，existing 长度 n、0/1 表示原有市场；原有市场点返回 0，不允许重复建。归属按 (距离, 市场点号) 字典序，距离相同时编号小者胜；零边合法。无原有市场时各点答案 n；全是市场时全 0。可在同一树上换 existing 重复 solve，每次重算。int128 保存距离和比较阈值；最近市场多源 Dijkstra，递归点分治总集合加、同分支减。O(n $\log^2$ n) 时间、O(n) 辅助空间，递归深度最坏 O(n)。返回完整方案计数，取 max 即最佳新增市场；内部字段和辅助方法不外改。不支持拓扑动态更新。

```
struct TreeMarket
{
    using I = __int128_t;
    using Key = pair<I, int>;
    int n;
    vector<vector<pair<int, long long>>> g;
    vector<int> size, removed, market, answer;
    vector<Key> best;

    TreeMarket(int n) : n(n), g(n), size(n), removed(n), answer(n)
    {
    }

    void add(int u, int v, long long w)
    {
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }

    void size_dfs(int u, int p)
    {
        size[u] = 1;
        for (auto [v, w] : g[u])
            if (v != p && !removed[v])
            {
                size_dfs(v, u);
                size[u] += size[v];
            }
    }

    int centroid(int u, int p, int total)
    {
        for (auto [v, w] : g[u])
            if (v != p && !removed[v] && size[v] > total / 2)
                return centroid(v, u, total);
        return u;
    }
```

接上页同一结构体：

```
    void collect(int u, int p, I d, vector<pair<int, I>> &nodes)
    {
        nodes.push_back({u, d});
        for (auto [v, w] : g[u])
            if (v != p && !removed[v]) collect(v, u, d + w, nodes);
    }

    void count(const vector<pair<int, I>> &nodes, int sign)
    {
        vector<Key> keys;
        for (auto [u, d] : nodes) keys.push_back({best[u].first - d, best[u].second});
        sort(keys.begin(), keys.end());
        for (auto [u, d] : nodes)
            if (!market[u])
            {
                int pos = upper_bound(keys.begin(), keys.end(), Key{d, u}) - keys.begin();
                answer[u] += sign * ((int)keys.size() - pos);
            }
    }

    void decompose(int entry)
    {
        size_dfs(entry, -1);
        int c = centroid(entry, -1, size[entry]);
        vector<pair<int, I>> nodes;
        collect(c, -1, 0, nodes);
        count(nodes, 1);
        removed[c] = 1;
        for (auto [v, w] : g[c])
            if (!removed[v])
            {
                nodes.clear();
                collect(v, c, w, nodes);
                count(nodes, -1);
                decompose(v);
            }
    }
```

接上页同一结构体：

```
    // Existing markets get answer 0. No existing market: every site wins n.
    vector<int> solve(const vector<int> &existing)
    {
        market = existing;
        fill(answer.begin(), answer.end(), 0);
        fill(removed.begin(), removed.end(), 0);
        best.assign(n, {I(1) << 120, n});
        using State = tuple<I, int, int>;
        priority_queue<State, vector<State>, greater<State>> q;
        for (int u = 0; u < n; u++)
            if (market[u])
            {
                best[u] = {0, u};
                q.push({0, u, u});
            }
        if (q.empty())
        {
            fill(answer.begin(), answer.end(), n);
            return answer;
        }
        while (!q.empty())
        {
            auto [d, id, u] = q.top();
            q.pop();
            if (best[u] != Key{d, id}) continue;
            for (auto [v, w] : g[u])
                if (Key{d + w, id} < best[v])
                {
                    best[v] = {d + w, id};
                    q.push({d + w, id, v});
                }
        }
        decompose(0);
        return answer;
    }
};
```

接口用法演示：树上新增市场（来源模型接口演示）

演示约定：按 kuangbin 来源模型：每组输入 n、n-1 条 1-based 非负边权树边和 n 个 0/1 市场标记，读到 EOF。新增市场只能选原来没有市场的点，按距离、市场编号字典序分配顶点，输出最多可支配的顶点数；无候选点输出 0，无原有市场输出 n。调用时点号减一，编号相对顺序保持。官方 HDU5016 题面和资源限制尚未核验，此处只是来源模型/API 演示，不登记正式题目 AC。

所需模板：TreeMarket（第 691 页，42.6.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://acm.hdu.edu.cn/showproblem.php?pid=5016

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    while (cin >> n)
    {
        TreeMarket tree(n);
        for (int i = 1; i < n; i++)
        {
            int u, v;
            long long w;
            cin >> u >> v >> w;
            tree.add(u - 1, v - 1, w);
        }
        vector<int> existing(n);
        for (int &x : existing) cin >> x;
        auto answer = tree.solve(existing);
        cout << *max_element(answer.begin(), answer.end()) << '\n';
    }
}
```

42.7 动态树分治

42.7.1 动态点分树：点权赋值与距离球求和

代码依赖：Fenwick（第 412 页，33.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

构造 CentroidSum(n)，T 默认为 long long；n≥1，输入是固定连通单位边树。点号 0..n-1；先 add(u,v)，再 build(values)。set(u,x) 是点权赋值；query(u,k) 返回到 u 距离 ≤k 的全部点权和，含 u 自身；负半径返回 0，半径用 long long。build 可在同一拓扑上重建并替换全部权值，建树后不再 add 边，不提供 link/cut。每点保存重心祖先、距离和分支，使用总距离桶减同支桶容斥；距离 d 对应 Fenwick 下标 d+1。线性建每层 Fenwick，构造/空间 O(n log n)，修改/查询 O($\log^2$ n)。T 支持零值及加减，所有桶和、赋值差与容斥中间值均须可表示；可用负权，不使用 kth。内部图、路径、桶和辅助函数不能外部改写。原树 DFS 递归深度最坏 O(n)，不等于重心层数；最大链测试提供 256MiB 栈，仍需核对目标评测栈限制。

```
template <class T = long long> struct CentroidSum
{
    struct Record
    {
        int centroid, distance, branch;
    };
    int n;
    vector<vector<int>> g;
    vector<int> siz, removed;
    vector<T> value;
    vector<vector<Record>> path;
    vector<Fenwick<T>> all;
    vector<vector<Fenwick<T>>> part;

    CentroidSum(int n)
        : n(n), g(n), siz(n), removed(n), value(n), path(n), all(n), part(n)
    {
        assert(n > 0);
    }

    void add(int u, int v)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        g[u].push_back(v);
        g[v].push_back(u);
    }
```

子树大小、重心与距离记录（接上页同一结构体）：

```
    void size_dfs(int u, int p)
    {
        siz[u] = 1;
        for (int v : g[u])
        {
            if (v == p || removed[v]) continue;
            size_dfs(v, u);
            siz[u] += siz[v];
        }
    }

    int centroid(int u, int p, int total)
    {
        for (int v : g[u])
            if (v != p && !removed[v] && siz[v] > total / 2)
                return centroid(v, u, total);
        return u;
    }

    void collect(int u, int p, int d, int c, int b, vector<T> &bucket)
    {
        path[u].push_back({c, d, b});
        if ((int)bucket.size() <= d + 1) bucket.resize(d + 2);
        bucket[d + 1] += value[u];
        for (int v : g[u])
            if (v != p && !removed[v]) collect(v, u, d + 1, c, b, bucket);
    }
```

线性构建距离桶和递归分解（接上页同一结构体）：

```
    // Raw distance d is stored at 1-based Fenwick index d+1.
    void make_bit(Fenwick<T> &bit, vector<T> bucket)
    {
        bit.n = (int)bucket.size() - 1;
        bit.a = move(bucket);
        for (int i = 1; i <= bit.n; i++)
            if (i + (i & -i) <= bit.n) bit.a[i + (i & -i)] += bit.a[i];
    }

    void decompose(int entry)
    {
        size_dfs(entry, -1);
        int c = centroid(entry, -1, siz[entry]);
        removed[c] = 1;
        path[c].push_back({c, 0, -1});
        vector<T> total{T{}, value[c]};
        for (int v : g[c])
        {
            if (removed[v]) continue;
            vector<T> bucket(1);
            collect(v, c, 1, c, (int)part[c].size(), bucket);
            if (total.size() < bucket.size()) total.resize(bucket.size());
            for (int d = 1; d < (int)bucket.size(); d++) total[d] += bucket[d];
            part[c].emplace_back();
            make_bit(part[c].back(), move(bucket));
        }
        make_bit(all[c], move(total));
        for (int v : g[c])
            if (!removed[v]) decompose(v);
    }
```

重建、点权赋值与距离球求和（接上页同一结构体）：

```
    // Rebuild resets all values and indices; do not change the tree after building.
    void build(const vector<T> &values)
    {
        assert((int)values.size() == n);
        value = values;
        fill(removed.begin(), removed.end(), 0);
        for (int u = 0; u < n; u++)
        {
            path[u].clear();
            part[u].clear();
        }
        decompose(0);
    }

    void set(int u, T x)
    {
        assert(0 <= u && u < n);
        T delta = x - value[u];
        value[u] = x;
        for (auto [c, d, b] : path[u])
        {
            all[c].add(d + 1, delta);
            if (b != -1) part[c][b].add(d + 1, delta);
        }
    }

    T prefix(const Fenwick<T> &bit, long long radius) const
    {
        if (radius < 0) return T{};
        return bit.sum(radius >= bit.n ? bit.n : (int)radius + 1);
    }

    T query(int u, long long radius) const
    {
        assert(0 <= u && u < n);
        T answer{};
        if (radius < 0) return answer;
        for (auto [c, d, b] : path[u])
        {
            answer += prefix(all[c], radius - d);
            if (b != -1) answer -= prefix(part[c][b], radius - d);
        }
        return answer;
    }
};
```

模板题使用：Luogu P6329 【模板】点分树 / 震波

题意：固定单位边树上修改点权，查询到中心距离不超过 k 的权值和。操作 0 查询，操作 1 赋值；每次先将 x 及 y/k 与最近一次查询答案异或，再把 1-based 顶点 x 减一。更新不改变 last，操作类型不异或。n,m≤100000，点权/新值在 1..10000，解码后 k 在 0..n-1。构造后先 build；原树递归 DFS 需要足够栈空间。

所需模板：Fenwick（第 412 页，33.3 节）、CentroidSum（第 695 页，42.7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6329

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    std::vector<long long> value(n);
    for (auto &x : value) std::cin >> x;
    CentroidSum tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        std::cin >> u >> v;
        tree.add(u - 1, v - 1);
    }
    tree.build(value);
    long long last = 0;
    while (m--)
    {
        int op;
        long long x, y;
        std::cin >> op >> x >> y;
        x ^= last;
        y ^= last;
        if (op == 0)
        {
            last = tree.query((int)x - 1, y);
            std::cout << last << '\n';
        }
        else
            tree.set((int)x - 1, y);
    }
}
```

42.7.2 动态点分树：最近活动点与点号方案

CentroidNearest(n)，n≥1，点号 0..n-1；先 add(u,v,w=1) 建固定连通树，禁止环、重边、自环。边权非负，所有原树简单路径长度须能用 long long 表示。build() 清空活动集，可在同一拓扑上重建，建树后不再 add。set(u,on) 启用或停用点，重复设置无影响；query(u) 返回 optional<pair<long long,int»，即最近距离和同距离中最小点号，空集返回 nullopt，距离 LLONG_MAX 合法。active[u] 只读，可用 set(u,!active[u]) 翻转；其他成员均为内部数据，不直接修改。每个重心用 set 维护 (距离, 原点号)，删除时精确到点号。查询跳过溢出的重心绕路距离；最优点与查询点的某个公共重心在原路径上，因此仍能取得精确答案。构建 O(n log n)，单次更新 $O(\log^2 n)$，查询 O(log n)，空间 O(n log n)。所有 DFS 递归，原树最大递归深度 O(n)。不支持负边权或动态拓扑。

```
struct CentroidNearest
{
    using ll = long long;
    int n;
    bool built = false;
    vector<vector<pair<int, ll>>> g, path;
    vector<int> siz, removed, active;
    vector<std::set<pair<ll, int>>> bag;

    CentroidNearest(int n)
        : n(n), g(n), path(n), siz(n), removed(n), active(n), bag(n)
    {
        assert(n > 0);
    }

    void add(int u, int v, ll w = 1)
    {
        assert(!built && 0 <= u && u < n && 0 <= v && v < n);
        assert(u != v && w >= 0);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }

    void size_dfs(int u, int p)
    {
        siz[u] = 1;
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            size_dfs(v, u);
            siz[u] += siz[v];
        }
    }

    int centroid(int u, int p, int total)
    {
        for (auto [v, w] : g[u])
            if (v != p && !removed[v] && siz[v] > total / 2)
                return centroid(v, u, total);
        return u;
    }
```

重心距离与递归分解（接上页同一结构体）：

```
    void collect(int u, int p, ll d, int c)
    {
        path[u].push_back({c, d});
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            assert(d <= LLONG_MAX - w);
            collect(v, u, d + w, c);
        }
    }

    void decompose(int entry)
    {
        size_dfs(entry, -1);
        int c = centroid(entry, -1, siz[entry]);
        removed[c] = 1;
        collect(c, -1, 0, c);
        for (auto [v, w] : g[c])
            if (!removed[v]) decompose(v);
    }
```

清空重建、活动点增删与最近点查询（接上页同一结构体）：

```
    // Fixed connected nonnegative-weight tree; rebuild clears all active points.
    void build()
    {
        fill(removed.begin(), removed.end(), 0);
        fill(active.begin(), active.end(), 0);
        for (int u = 0; u < n; u++)
        {
            path[u].clear();
            bag[u].clear();
        }
        decompose(0);
        built = true;
    }

    void set(int u, bool on)
    {
        assert(built && 0 <= u && u < n);
        if (active[u] == on) return;
        active[u] = on;
        for (auto [c, d] : path[u])
        {
            if (on) bag[c].insert({d, u});
            else bag[c].erase({d, u});
        }
    }

    // Minimum (distance, vertex ID); nullopt means no active point.
    optional<pair<ll, int>> query(int u) const
    {
        assert(built && 0 <= u && u < n);
        optional<pair<ll, int>> ans;
        for (auto [c, d] : path[u])
        {
            if (bag[c].empty()) continue;
            auto [x, v] = *bag[c].begin();
            // A detour may overflow even when every simple path fits.
            if (x > LLONG_MAX - d) continue;
            pair<ll, int> now{d + x, v};
            if (!ans || now < *ans) ans = now;
        }
        return ans;
    }
};
```

应用题补充：SPOJ QTREE5 / Query on a tree V（Luogu SP2939）

题意：N,Q≤100000，单位边树点号 1..N，初始全黑；先 N 和 N-1 条边，再 Q 条操作。0 u 翻转黑白，1 u 输出最近白点距离（可为自身），没有白点输出-1。内部转换为 0-based，白点对应活动点。原站注明 modified from ZJOI07，按应用用法收录；非独立非比赛模板题。

所需模板：CentroidNearest（第 700 页，42.7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.spoj.com/problems/QTREE5/

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n;
    std::cin >> n;
    CentroidNearest tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        std::cin >> u >> v;
        tree.add(u - 1, v - 1);
    }
    tree.build();
    int q;
    std::cin >> q;
    while (q--)
    {
        int op, u;
        std::cin >> op >> u;
        --u;
        if (op == 0)
            tree.set(u, !tree.active[u]);
        else
        {
            auto ans = tree.query(u);
            std::cout << (ans ? ans->first : -1) << '\n';
        }
    }
}
```

42.7.3 动态点分树：有符号边权活动点直径与端点

CentroidDiameter(n)，固定连通无向树、0-based 点号，n>0 且 2n 不溢出 int。add(u,v,w) 仅在首次 build 前；允许负边权，所有简单路径权值和必须能用 long long 表示。build() 递归分解并清空活动集，可反复重建；set(u,on) 幂等启停点。query() 返回 optional<tuple<long long,int,int»，内容为最大距离与任意一对达到最优值的活动端点，允许两端相同，所以非空集答案至少 0；空集 nullopt。各重心按不同分支取最大距离，只合并跨分支真实路径，不能套用非负权直径端点摘要；并列不承诺最小编号。空集预处理 O(n log n)、set $O(\log^2 n)$、query O(1)，空间 O(n log n)；逐点激活全树 $O(n \log^2 n)$。支持值拷贝；active 只读用于反色，其余字段与 sum/size_dfs/centroid/collect/decompose/candidate 为内部状态。DFS 保持递归，树输入合法性与足够栈空间由调用方保证。

```
struct CentroidDiameter
{
    using ll = long long;
    using Item = tuple<ll, int, int>;
    using Best = tuple<ll, int, int, int>;
    struct Entry
    {
        int c, b;
        ll d;
    };
    int n;
    bool built = false;
    vector<vector<pair<int, ll>>> g;
    vector<vector<Entry>> path;
    vector<int> siz, removed, active;
    vector<std::set<pair<ll, int>>> bag;
    vector<std::set<Item>> top;
    std::set<Best> best;
    std::set<int> live;

    CentroidDiameter(int n)
        : n(n), g(n), path(n), siz(n), removed(n), active(n), top(n)
    {
        assert(0 < n && n <= INT_MAX / 2);
    }

    ll sum(ll a, ll b) const
    {
        assert(b >= 0 ? a <= LLONG_MAX - b : a >= LLONG_MIN - b);
        return a + b;
    }

    void add(int u, int v, ll w = 1)
    {
        assert(!built && 0 <= u && u < n && 0 <= v && v < n && u != v);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
```

递归分解与不同分支的距离（接上页同一结构体）：

```
    void size_dfs(int u, int p)
    {
        siz[u] = 1;
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            size_dfs(v, u);
            siz[u] += siz[v];
        }
    }

    int centroid(int u, int p, int total)
    {
        for (auto [v, w] : g[u])
            if (v != p && !removed[v] && siz[v] > total / 2)
                return centroid(v, u, total);
        return u;
    }

    void collect(int u, int p, ll d, int c, int b)
    {
        path[u].push_back({c, b, d});
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            collect(v, u, sum(d, w), c, b);
        }
    }

    void decompose(int entry)
    {
        size_dfs(entry, -1);
        int c = centroid(entry, -1, siz[entry]);
        removed[c] = 1;
        path[c].push_back({c, (int)bag.size(), 0});
        bag.emplace_back();
        for (auto [v, w] : g[c])
        {
            if (removed[v]) continue;
            int b = bag.size();
            bag.emplace_back();
            collect(v, c, w, c, b);
        }
        for (auto [v, w] : g[c])
            if (!removed[v]) decompose(v);
    }
```

清空重建与重心候选（接上页同一结构体）：

```
    // Fixed tree, signed weights. Rebuild clears all active points.
    void build()
    {
        fill(removed.begin(), removed.end(), 0);
        fill(active.begin(), active.end(), 0);
        for (int u = 0; u < n; u++)
        {
            path[u].clear();
            top[u].clear();
        }
        bag.clear();
        best.clear();
        live.clear();
        decompose(0);
        built = true;
    }

    optional<Best> candidate(int c) const
    {
        if (top[c].size() < 2) return nullopt;
        auto it = top[c].rbegin();
        auto [x, u, a] = *it++;
        auto [y, v, b] = *it;
        if (u > v) swap(u, v);
        return Best{sum(x, y), u, v, c};
    }
```

活动点设置与端点查询（接上页同一结构体）：

```
    void set(int u, bool on)
    {
        assert(built && 0 <= u && u < n);
        if (active[u] == on) return;
        active[u] = on;
        if (on) live.insert(u);
        else live.erase(u);
        for (auto [c, b, d] : path[u])
        {
            if (auto old = candidate(c)) best.erase(*old);
            if (!bag[b].empty())
            {
                auto [x, v] = *bag[b].rbegin();
                top[c].erase({x, v, b});
            }
            if (on) bag[b].insert({d, u});
            else bag[b].erase({d, u});
            if (!bag[b].empty())
            {
                auto [x, v] = *bag[b].rbegin();
                top[c].insert({x, v, b});
            }
            if (auto now = candidate(c)) best.insert(*now);
        }
    }

    // Maximum distance and any attaining pair; equal endpoints are allowed.
    optional<Item> query() const
    {
        assert(built);
        if (live.empty()) return nullopt;
        if (!best.empty())
        {
            auto [d, u, v, c] = *best.rbegin();
            if (d > 0) return Item{d, u, v};
        }
        int u = *live.begin();
        return Item{0, u, u};
    }
};
```

应用题补充：SPOJ QTREE4 / 有符号边权动态白点直径

题意：n,q≤100000，边权 −1000..1000，初始全白；C u 反色，A 查白点对的最大距离，两端允许相同。无白点输出 They have disappeared.。题目 1-based，传入模板时减 1；query() 非空时取 get<0>。固定树且允许负边，不能改用只支持非负权的 TreeDiameter 摘要。

所需模板：CentroidDiameter（第 704 页，42.7.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.spoj.com/problems/QTREE4/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    CentroidDiameter t(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u - 1, v - 1, w);
    }
    t.build();
    for (int u = 0; u < n; u++) t.set(u, true);
    int q;
    cin >> q;
    while (q--)
    {
        char op;
        cin >> op;
        if (op == 'C')
        {
            int u;
            cin >> u;
            --u;
            t.set(u, !t.active[u]);
        }
        else
        {
            auto ans = t.query();
            if (ans) cout << get<0>(*ans) << '\n';
            else cout << "They have disappeared.\n";
        }
    }
}
```

接口用法演示：活动点最大距离与端点方案（接口演示）

演示约定：输入 n,q、n−1 条 0-based 带权树边，初始全不活动。0 u on 设置状态（重复设置不变）；1 查询，空集输出 −1，否则输出最大距离 d 及任意最优活动点 u,v。允许负边和 u=v，并列不指定点号；所有简单路径和须在 signed64 内。

所需模板：CentroidDiameter（第 704 页，42.7.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/tree-divide/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    CentroidDiameter t(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u, v, w);
    }
    t.build();
    while (q--)
    {
        int op;
        cin >> op;
        if (op == 0)
        {
            int u, on;
            cin >> u >> on;
            t.set(u, on);
        }
        else
        {
            auto ans = t.query();
            if (!ans) cout << -1 << '\n';
            else
            {
                auto [d, u, v] = *ans;
                cout << d << ' ' << u << ' ' << v << '\n';
            }
        }
    }
}
```

42.8 树哈希

相关应用：Deterministic sorted-child interning and centroid rooting; exact alternative to randomized tree hashing.

42.8.1 确定性有根与无根树同构

TreeIsomorphism iso；rooted(g,root) 返回有根树编号，unrooted(g) 返回无根树 pair 编号。g 是 0-based 非空简单无向连通树，邻接表双向各存一次；root 在 0..n-1。递归 DFS，不验证非法图。所有待比较的树必须共用同一个 iso；编号依赖插入次序，不同对象或清空 codes 前后的编号不能直接比较。对子节点编号排序并用 map 精确驻留，无随机哈希碰撞。无根树取重心：单重心返回 (code,0)，双重心分别以两点为根编码并排序。支持单点；不考虑点/边标签，也不返回顶点映射。codes 只读，编号数须 <INT_MAX，n<=INT_MAX。S 为本次结束时驻留签名数，一次 n 点调用 O(n log(n+1)+n log(S+1))，临时 O(n)，跨调用累计存储至多 O(T)，T 为累计编码遍历的顶点数。

```
struct TreeIsomorphism
{
    map<vector<int>, int> codes;

    int rooted(const vector<vector<int>> &g, int root)
    {
        assert(0 <= root && root < int(g.size()));
        auto dfs = [&](auto &&self, int u, int p) -> int
        {
            vector<int> children;
            for (int v : g[u])
            {
                if (v == p) continue;
                children.push_back(self(self, v, u));
            }
            sort(children.begin(), children.end());
            int next = int(codes.size()) + 1;
            return codes.emplace(children, next).first->second;
        };
        return dfs(dfs, root, -1);
    }
```

继续求解（接上页同一结构体）：

```
    pair<int, int> unrooted(const vector<vector<int>> &g)
    {
        int n = g.size();
        assert(n > 0);
        vector<int> centers;
        auto dfs = [&](auto &&self, int u, int p) -> int
        {
            int size = 1, largest = 0;
            for (int v : g[u])
            {
                if (v == p) continue;
                int child = self(self, v, u);
                size += child;
                largest = max(largest, child);
            }
            largest = max(largest, n - size);
            if (largest <= n / 2) centers.push_back(u);
            return size;
        };
        dfs(dfs, 0, -1);
        int a = rooted(g, centers[0]);
        if (centers.size() == 1) return {a, 0};
        int b = rooted(g, centers[1]);
        if (a > b) swap(a, b);
        return {a, b};
    }
};
```

模板题使用：P5043 【模板】树同构 / [BJOI2015] 树的同构

题意：M 棵无根树，每棵 N 个点，1<=N,M<=50。输入 N 个父亲编号，0 表示根；转换无向树后忽略输入根。输出与本树同构的最早树编号；所有树共用同一驻留表。

所需模板：TreeIsomorphism（第 710 页，42.8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5043

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    cin >> m;
    TreeIsomorphism iso;
    map<pair<int, int>, int> first;
    for (int id = 1; id <= m; id++)
    {
        int n;
        cin >> n;
        vector<vector<int>> g(n);
        for (int u = 0; u < n; u++)
        {
            int p;
            cin >> p;
            if (p == 0) continue;
            p--;
            g[u].push_back(p);
            g[p].push_back(u);
        }
        auto code = iso.unrooted(g);
        auto result = first.emplace(code, id);
        cout << result.first->second << '\n';
    }
}
```

第四十三章　拓扑排序

43.1　DAG 有符号权最长路与路径方案

DagLongest(n)，点号 1..n，0<n<INT_MAX；add(u,v,w) 返回 0-based 原边号，支持重边和完整 signed64 权值，边数 ≤INT_MAX。run(s) 求指定起点的最长路；run(0) 允许任意起点及零边路径。每次运行重建入度，返回 false 表示整张图存在环（包括与 s 不连通的环），此时全部 dis/pre/ord 结果无效；它不是一般图最长路求解器。

成功时 valid=true，ord 为全图拓扑序，dis[v]==−inf 为不可达，其余为 int128 最长距离；path(v) 返回正向原边号，不可达和零边路径都为空，结合 dis 区分。并列任选一条，不承诺字典序。add 使旧结果失效；支持重复改源点和独立对象复制。

run 时间和空间 O(n+m)，path 为 O(路径长)。拓扑处理必须包含不可达点，并正常消减其出边入度，但禁止用不可达哨兵参与加法。有限路径至多 n−1 边，绝对值 <(n · 2^63)<2^94，int128 和 inf=2^126 足够，不能无条件转 long long。e/g/dis/pre/ord/valid 只读。拓扑序用 vector 作 FIFO 队列，与 DFS 改写为显式栈无关。

```
struct DagLongest
{
    using I = __int128_t;
    static constexpr I inf = I(1) << 126;
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n;
    bool valid = false;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<I> dis;
    vector<int> pre, ord;

    DagLongest(int n) : n(n), g(n + 1)
    {
        assert(0 < n && n < INT_MAX);
    }

    int add(int u, int v, long long w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(e.size() < INT_MAX);
        int id = e.size();
        e.push_back({u, v, w});
        g[u].push_back(id);
        valid = false;
        return id;
    }
```

全图拓扑序与最长路（接上页同一结构体）：

```
    // s=0 allows any starting vertex, including a zero-edge path.
    // A cycle anywhere returns false and invalidates all results.
    bool run(int s)
    {
        assert(0 <= s && s <= n);
        valid = false;
        dis.assign(n + 1, -inf);
        pre.assign(n + 1, -1);
        ord.clear();
        vector<int> deg(n + 1);
        for (auto [u, v, w] : e) deg[v]++;
        for (int u = 1; u <= n; u++)
        {
            if (!deg[u]) ord.push_back(u);
            if (!s || u == s) dis[u] = 0;
        }
        for (int i = 0; i < (int)ord.size(); i++)
        {
            int u = ord[i];
            for (int id : g[u])
            {
                auto [from, v, w] = e[id];
                if (dis[u] != -inf && dis[v] < dis[u] + w)
                {
                    dis[v] = dis[u] + w;
                    pre[v] = id;
                }
                if (!--deg[v]) ord.push_back(v);
            }
        }
        valid = (int)ord.size() == n;
        return valid;
    }
```

成功运行后的原边路径（接上页同一结构体）：

```
    // Forward original-edge IDs; inspect dis to distinguish unreachable.
    vector<int> path(int t) const
    {
        assert(valid && 1 <= t && t <= n);
        vector<int> ids;
        if (dis[t] == -inf) return ids;
        while (pre[t] != -1)
        {
            ids.push_back(pre[t]);
            t = e[pre[t]].u;
        }
        reverse(ids.begin(), ids.end());
        return ids;
    }
};
```

应用题补充：Luogu P1807 / 最长路

题意：有符号权 DAG 中求 1 到 n 的最长路，不可达输出 −1。n≤1500，m≤50000，|w|≤100000，有限结果可转 long long；负数也可能是合法答案，不能以 dis<0 判不可达。

所需模板：DagLongest（第 713 页，43.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1807

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    DagLongest t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u, v, w);
    }
    t.run(1);
    if (t.dis[n] == -DagLongest::inf) cout << -1 << '\n';
    else cout << (long long)t.dis[n] << '\n';
}
```

应用题补充：CSES 1680 / Longest Flight Route

题意：DAG 中求从 1 到 n 经过城市数最多的路线；每条航线加权 1，path 返回原边号，依次输出边终点。不可达输出 IMPOSSIBLE；城市数为边数 +1。n≤100000，m≤200000。

所需模板：DagLongest（第 713 页，43.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://cses.fi/problemset/task/1680/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    DagLongest t(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        t.add(u, v, 1);
    }
    t.run(1);
    if (t.dis[n] == -DagLongest::inf)
    {
        cout << "IMPOSSIBLE\n";
        return 0;
    }
    auto p = t.path(n);
    cout << p.size() + 1 << '\n';
    cout << 1;
    for (int id : p) cout << ' ' << t.e[id].v;
    cout << '\n';
}
```

接口用法演示：DAG 最长距离、全图判环与原边路径（接口演示）

演示约定：输入 n,m,s 及 signed64 有向边；s=0 允许任意起点。整图有环输出 CYCLIC，否则逐点输出 INF 或 “int128 最长距离边数正向原边号”。INF 表示不可达，负数距离有效，边号从 0 计。

所需模板：DagLongest（第 713 页，43.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/topo/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    DagLongest t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u, v, w);
    }
    if (!t.run(s))
    {
        cout << "CYCLIC\n";
        return 0;
    }
    auto print = [](__int128 x)
    {
        if (x < 0)
        {
            cout << '-';
            x = -x;
        }
        string a;
        do
        {
            a += char('0' + x % 10);
            x /= 10;
        } while (x);
        reverse(a.begin(), a.end());
        cout << a;
    };
```

继续处理操作（接上页同一 main）：

```
    for (int u = 1; u <= n; u++)
    {
        if (t.dis[u] == -DagLongest::inf) cout << "INF\n";
        else
        {
            auto path = t.path(u);
            print(t.dis[u]);
            cout << ' ' << path.size();
            for (int id : path) cout << ' ' << id;
            cout << '\n';
        }
    }
}
```

第四十四章　最短路问题

44.1　最短路

44.1.1　Johnson：负边全源最短路

Johnson(n)，点号 1..n，1≤n<INT_MAX，总边数不超过 INT_MAX；add(u,v,w) 加有向边，允许重边、自环和完整 long long 边权。build() 用初值全零的 Bellman–Ford 求势能，等价于超级源点连向所有点；全图任意位置有负环时返回 false，否则返回 true。仅成功后可 distances(s)，返回 vector<int128> 的原图距离，d[s]=0，不可达为 Johnson::inf；0 号不用。可按需逐源调用，不强制存 n^2 矩阵。加边会使预处理失效，再 build 后查询；可重复 build。重赋权为 w+h[u]-h[v]，非负；查询后加 h[v]-h[s] 还原。势能与距离全程 int128，不能无条件转为 long long。预处理 O(nm+n) 时间，单源用惰性优先队列，O(n+m log(m+1)) 时间、O(n+m) 辅助空间；整个结构 O(n+m)。g、edges、h 仅供读取，勿手动修改。

```
struct Johnson
{
    using I = __int128_t;
    static constexpr I inf = I(1) << 120;
    int n;
    vector<tuple<int, int, long long>> edges;
    vector<vector<pair<int, long long>>> g;
    vector<I> h;
    bool ready = false;

    Johnson(int n) : n(n), g(n + 1), h(n + 1) { assert(n > 0); }

    void add(int u, int v, long long w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        edges.push_back({u, v, w});
        g[u].push_back({v, w});
        ready = false;
    }
```

势能预处理、单源查询与距离还原（接上页同一结构体）：

```
    bool build()
    {
        ready = false;
        fill(h.begin(), h.end(), 0);
        for (int step = 0; step < n; step++)
        {
            bool changed = false;
            for (auto [u, v, w] : edges)
                if (h[v] > h[u] + w)
                {
                    h[v] = h[u] + w;
                    changed = true;
                }
            if (!changed)
            {
                ready = true;
                return true;
            }
        }
        return false;
    }

    vector<I> distances(int s) const
    {
        assert(ready && 1 <= s && s <= n);
        vector<I> d(n + 1, inf);
        priority_queue<pair<I, int>, vector<pair<I, int>>, greater<pair<I, int>>> q;
        d[s] = 0;
        q.push({0, s});
        while (!q.empty())
        {
            auto [value, u] = q.top();
            q.pop();
            if (value != d[u]) continue;
            for (auto [v, w] : g[u])
            {
                I next = value + w + h[u] - h[v];
                if (next < d[v])
                {
                    d[v] = next;
                    q.push({next, v});
                }
            }
        }
        for (int v = 1; v <= n; v++)
            if (d[v] != inf) d[v] += h[v] - h[s];
        return d;
    }
};
```

模板题使用：Luogu P5905

题意：有向图可能有负边，若任意位置存在负环输出 -1。否则逐源求距离，按题意把不可达距离换为 10^9，再求编号加权和。题目规模和边权界保证最终和可转为 long long。

所需模板：Johnson（第 719 页，44.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5905

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Johnson graph(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        graph.add(u, v, w);
    }
    if (!graph.build())
    {
        cout << -1 << '\n';
        return 0;
    }
    for (int s = 1; s <= n; s++)
    {
        auto d = graph.distances(s);
        Johnson::I answer = 0;
        for (int v = 1; v <= n; v++)
            answer += v * (d[v] == Johnson::inf ? 1000000000 : d[v]);
        cout << (long long)answer << '\n';
    }
}
```

44.1.2 非负权最短路

add(u,v,w)、run(s)，dis 为距离，pre 为前驱，path(t) 返回点序列。不可达用 LLONG_MAX；要求所有有限最短距离严格小于该哨兵，w≥0。懒删除堆在多重边图中按 O(n+m log(m+1)) 计，空间 O(n+m)；简单图常写 O((n+m)log(n+1))。

```
struct Dijkstra
{
    using ll = long long;
    static constexpr ll inf = LLONG_MAX;
    int n;
    vector<vector<pair<int, ll>>> g;
    vector<ll> dis;
    vector<int> pre;

    Dijkstra(int n) : n(n), g(n + 1) {}

    void add(int u, int v, ll w)
    {
        assert(w >= 0);
        g[u].push_back({v, w});
    }

    void run(int s)
    {
        dis.assign(n + 1, inf);
        pre.assign(n + 1, -1);
        dis[s] = 0;
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
        q.push({0, s});
        while (!q.empty())
        {
            auto [d, u] = q.top();
            q.pop();
            if (d != dis[u]) continue;
            for (auto [v, w] : g[u])
                if (w < inf - d && d + w < dis[v])
                {
                    dis[v] = d + w;
                    pre[v] = u;
                    q.push({dis[v], v});
                }
        }
    }

    vector<int> path(int t) const
    {
        if (dis[t] == inf) return {};
        vector<int> ans;
        for (; t != -1; t = pre[t]) ans.push_back(t);
        reverse(ans.begin(), ans.end());
        return ans;
    }
};
```

模板题使用：Luogu P4779

题意：给非负权有向图和源点 s，输出 s 到各点的最短距离。加完边后 run(s)，读取 dis[1..n]；本题保证源点可达所有点，一般输入的不可达值须按目标题目格式另行转换。

所需模板：Dijkstra（第 722 页，44.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4779

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m, s;
    std::cin >> n >> m >> s;
    Dijkstra graph(n);
    while (m--)
    {
        int u, v;
        long long w;
        std::cin >> u >> v >> w;
        graph.add(u, v, w);
    }
    graph.run(s);
    for (int i = 1; i <= n; i++) std::cout << graph.dis[i] << (i == n ? '\n' : ' ');
}
```

44.1.3 顶点开放时间的单位边最短路

release_bfs(open,source,neighbors) 返回各点最早到达时间，不可达为 -1；点号 0..n-1，n>0，open[v] 为非负整数，源点初始时间为 open[source]。允许原地等待，每条边耗时恰为 1，neighbors(u,visit) 枚举所有出邻点并调用 visit(v)，可直接枚举网格而不建边。令 R=max(open)，要求 R+n<INT_MAX 且内存可承受；时间 O(n+m+R)，空间 O(n+R)。第一次发现即可定距，只适用于单位边和固定开放时间；不能用于一般带权边、关闭时间或周期可用边。

```
template <class Neighbors>
vector<int> release_bfs(const vector<int> &open, int source, Neighbors neighbors)
{
    int n = open.size();
    assert(0 <= source && source < n);
    int last = *max_element(open.begin(), open.end());
    assert(last >= 0 && last < INT_MAX - n);
    vector<vector<int>> pending(last + 1);
    vector<int> dist(n, -1), queue;
    queue.reserve(n);
    dist[source] = open[source];
    queue.push_back(source);
    int head = 0, time = 0, waiting = 0;
    while (head < int(queue.size()) || waiting)
    {
        int limit = head < int(queue.size()) ? dist[queue[head]] : INT_MAX;
        while (time <= last && time <= limit && pending[time].empty()) time++;
        int u;
        if (time <= last && time <= limit)
        {
            u = pending[time].back();
            pending[time].pop_back();
            waiting--;
        }
        else
            u = queue[head++];
        neighbors(u,
                  [&](int v)
                  {
                      if (dist[v] != -1) return;
                      dist[v] = max(dist[u] + 1, open[v]);
                      if (dist[v] == dist[u] + 1)
                          queue.push_back(v);
                      else
                      {
                          pending[open[v]].push_back(v);
                          waiting++;
                      }
                  });
    }
    return dist;
}
```

应用题补充：QOJ 8236 / ICPC 2023 Hangzhou G —Snake Move

题意：蛇可移动或缩尾，求蛇头到各格最少步数的平方和，模 2^64。初始身体按 0-based i（不含头）设 open=k-i，尾格为 1；源点时间 0，障碍不连边。n,m 至多 3000，k 至多 min(nm,100000)。不可达不计，使用 unsigned long long 自然回绕；这是比赛应用，不是独立模板题。

所需模板：release_bfs（第 724 页，44.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/gym/104976/attachments/download/24253/hangzhou(en).pdf

```
int main()
{
    int n, m, k;
    scanf("%d%d%d", &n, &m, &k);
    vector<int> open(n * m);
    int source = 0;
    for (int i = 0; i < k; i++)
    {
        int x, y;
        scanf("%d%d", &x, &y);
        int u = (x - 1) * m + y - 1;
        if (i == 0)
            source = u;
        else
            open[u] = k - i;
    }
    vector<string> grid(n);
    vector<char> row(m + 1);
    for (auto &s : grid)
    {
        scanf("%s", row.data());
        s = row.data();
    }
    auto dist =
        release_bfs(open,
                    source,
                    [&](int u, auto visit)
                    {
                        int x = u / m, y = u % m;
                        const int dx[] = {-1, 1, 0, 0};
                        const int dy[] = {0, 0, -1, 1};
                        for (int i = 0; i < 4; i++)
                        {
                            int a = x + dx[i], b = y + dy[i];
                            if (a >= 0 && a < n && b >= 0 && b < m && grid[a][b] == '.')
                                visit(a * m + b);
                        }
                    });
    unsigned long long answer = 0;
    for (int d : dist)
        if (d >= 0) answer += 1ULL * d * d;
    printf("%llu\n", answer);
}
```

44.1.4 Floyd 全源最短路与原边方案

Floyd(n)，点号 1..n，0<n<INT_MAX；add(u,v,w) 返回 0-based 原边号，有向边，允许重边、自环及完整 signed64 权值，边数 ≤INT_MAX。无向图需分别加入双向边。run() 重新建立矩阵；存在任何负环返回 false，valid=false，全部结果不可读，不能只取未受负环影响的点对。

成功时 dis[u][v]==inf 为不可达，其他为 int128 最短距离，dis[u][u]=0。path(u,v) 返回正向原边号，并列任选；不可达和 u=v 均为空，结合 dis 区分。nxt 为恢复路径的首条原边号矩阵，通常只需调用 path；e/dis/nxt/valid 只读。add 清除 valid，支持重复运行、加边后重跑与独立值拷贝。

run 时间 $O(n^3+m)$，空间 $O(n^2+m)$，path 时间与输出空间 O(路径长)。每次处理 k 之前检查 dis[k][k]<0，立即拒绝，不能延后到所有松弛结束，否则负环下中间量可指数增长。通过的枢轴不含负环；有限标签取简单路径或闭合路径，拼接候选绝对值 <2n · 2^63<2^95，int128 与 inf=2^126 足够。两段都有限才相加；不要无条件转 long long。矩阵占用约 $20(n+1)^2$ 字节，另计 vector 和原边表，n 应按时间/内存选择。

```
struct Floyd
{
    using I = __int128_t;
    static constexpr I inf = I(1) << 126;
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n;
    bool valid = false;
    vector<Edge> e;
    vector<vector<I>> dis;
    vector<vector<int>> nxt;

    Floyd(int n) : n(n)
    {
        assert(0 < n && n < INT_MAX);
    }

    int add(int u, int v, long long w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(e.size() < INT_MAX);
        int id = e.size();
        e.push_back({u, v, w});
        valid = false;
        return id;
    }
```

矩阵初始化与 Floyd 松弛（接上页同一结构体）：

```
    // Any negative cycle invalidates all results, including finite pairs.
    bool run()
    {
        valid = false;
        dis.assign(n + 1, vector<I>(n + 1, inf));
        nxt.assign(n + 1, vector<int>(n + 1, -1));
        for (int u = 1; u <= n; u++) dis[u][u] = 0;
        for (int id = 0; id < (int)e.size(); id++)
        {
            auto [u, v, w] = e[id];
            if (w < dis[u][v])
            {
                dis[u][v] = w;
                nxt[u][v] = id;
            }
        }
        for (int k = 1; k <= n; k++)
        {
            // Stop before using a negative diagonal as a pivot.
            if (dis[k][k] < 0) return false;
            for (int u = 1; u <= n; u++)
            {
                if (dis[u][k] == inf) continue;
                for (int v = 1; v <= n; v++)
                    if (dis[k][v] != inf && dis[u][v] > dis[u][k] + dis[k][v])
                    {
                        dis[u][v] = dis[u][k] + dis[k][v];
                        nxt[u][v] = nxt[u][k];
                    }
            }
        }
        valid = true;
        return true;
    }
```

成功运行后的原边路径（接上页同一结构体）：

```
    // Forward original-edge IDs; empty for unreachable or s=t.
    vector<int> path(int s, int t) const
    {
        assert(valid && 1 <= s && s <= n && 1 <= t && t <= n);
        vector<int> ids;
        if (dis[s][t] == inf) return ids;
        while (s != t)
        {
            int id = nxt[s][t];
            ids.push_back(id);
            s = e[id].v;
        }
        return ids;
    }
};
```

模板题使用：AOJ GRL_1_C / All Pairs Shortest Path

题意：0-based 有向图输入转换为 1-based；整图有负环输出 NEGATIVE CYCLE，否则逐行输出距离矩阵，不可达为 INF。n≤100、m≤9900、|w|≤20000000，有限结果可转 long long。题面无自环重边，模板额外支持。

所需模板：Floyd（第 726 页，44.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=GRL_1_C&lang=en

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Floyd t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u + 1, v + 1, w);
    }
    if (!t.run())
    {
        cout << "NEGATIVE CYCLE\n";
        return 0;
    }
    for (int u = 1; u <= n; u++)
    {
        for (int v = 1; v <= n; v++)
        {
            if (v > 1) cout << ' ';
            if (t.dis[u][v] == Floyd::inf) cout << "INF";
            else cout << (long long)t.dis[u][v];
        }
        cout << '\n';
    }
}
```

应用题补充：CSES 1672 / Shortest Routes II

题意：无向正权图多次查询最短距离；每条道路加入双向边，重边保留最小值，不可达输出 -1。$n\le500$、$m\le n^2$、$q\le100000$、$w\le10^9$，有限结果用 long long。

所需模板：Floyd（第 726 页，44.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://cses.fi/problemset/task/1672/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    Floyd t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u, v, w);
        t.add(v, u, w);
    }
    t.run();
    while (q--)
    {
        int u, v;
        cin >> u >> v;
        if (t.dis[u][v] == Floyd::inf) cout << -1 << '\n';
        else cout << (long long)t.dis[u][v] << '\n';
    }
}
```

接口用法演示：Floyd 全源距离与指定点对路径（接口演示）

演示约定：输入 n,m,q 及 signed64 有向边，再输入 q 个点对。整图有负环只输出 NEGATIVE CYCLE；否则每问输出 INF 或 “int128 距离边数正向原边号”，原边号从 0 计。同点零边路径与不可达分别处理。

所需模板：Floyd（第 726 页，44.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/shortest-path/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    Floyd t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u, v, w);
    }
    if (!t.run())
    {
        cout << "NEGATIVE CYCLE\n";
        return 0;
    }
    auto print = [](__int128 x)
    {
        if (x < 0)
        {
            cout << '-';
            x = -x;
        }
        string a;
        do
        {
            a += char('0' + x % 10);
            x /= 10;
        } while (x);
        reverse(a.begin(), a.end());
        cout << a;
    };
```

继续处理操作（接上页同一 main）：

```
    while (q--)
    {
        int u, v;
        cin >> u >> v;
        if (t.dis[u][v] == Floyd::inf) cout << "INF\n";
        else
        {
            auto path = t.path(u, v);
            print(t.dis[u][v]);
            cout << ' ' << path.size();
            for (int id : path) cout << ' ' << id;
            cout << '\n';
        }
    }
}
```

44.1.5 最小总边权最短路树（含零权边）

ShortestPathTree(n)，无向非负权图，1-based 点号，0<n<INT_MAX；add(u,v,w) 自动加入双向邻接并返回 0-based 无向原边号，支持自环重边，0≤w≤LLONG_MAX，边数 ≤INT_MAX。run(s) 返回可达点数（含 s），构造所有可达点上的最短路树，并在这些树中最小化所选原边权值总和 weight。断开图只覆盖 s 所在连通块；不是最小生成树。

运行后 valid=true，dis 为 int128 最短距离，inf 为不可达；ids 为树边原编号，weight 为 int128 总边权。path(v) 返回从 s 到 v 的正向无向原边号，按当前点决定边方向；s=v 与不可达均为空，结合 dis 区分。pre[v] 为树中父边号，根与不可达点为 −1。并列任选，不保证字典序。add 使旧结果失效；支持反复换源点及独立复制。

Dijkstra 求距离，递归划分可达零权连通块；每块选零权生成树，非根块选择最轻的紧入边，最后递归定向整棵树。时间 O(n+m log(m+1))、空间 O(n+m)，path 为 O(路径长)。zero/dfs 与 bel/tree 为内部辅助，成员只读；递归深度最坏 O(n)，调用方保证栈空间。有限距离和总树权绝对值 <2^94，int128 和 inf=2^126 足够，不能无条件转 long long。

```
struct ShortestPathTree
{
    using I = __int128_t;
    static constexpr I inf = I(1) << 126;
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n, root = 0;
    bool valid = false;
    I weight = 0;
    vector<Edge> e;
    vector<vector<pair<int, int>>> g, tree;
    vector<I> dis;
    vector<int> bel, ids, pre;

    ShortestPathTree(int n) : n(n), g(n + 1)
    {
        assert(0 < n && n < INT_MAX);
    }

    int add(int u, int v, long long w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && w >= 0);
        assert(e.size() < INT_MAX);
        int id = e.size();
        e.push_back({u, v, w});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        valid = false;
        return id;
    }
```

递归辅助与 run 中的 Dijkstra（接上页同一结构体）：

```
    // Internal: span one reachable zero-weight component recursively.
    void zero(int u, int c)
    {
        bel[u] = c;
        for (auto [v, id] : g[u])
            if (!e[id].w && !bel[v])
            {
                ids.push_back(id);
                zero(v, c);
            }
    }

    // Internal: orient the selected undirected tree from the query root.
    void dfs(int u, int p)
    {
        for (auto [v, id] : tree[u])
            if (v != p)
            {
                pre[v] = id;
                dfs(v, u);
            }
    }

    // Minimum total edge weight among shortest-path trees of reachable vertices.
    // Returns the number of reachable vertices, including s.
    int run(int s)
    {
        assert(1 <= s && s <= n);
        valid = false;
        root = s;
        dis.assign(n + 1, inf);
        priority_queue<pair<I, int>, vector<pair<I, int>>, greater<pair<I, int>>> q;
        dis[s] = 0;
        q.push({0, s});
        while (!q.empty())
        {
            auto [d, u] = q.top();
            q.pop();
            if (d != dis[u]) continue;
            for (auto [v, id] : g[u])
                if (dis[v] > d + e[id].w)
                {
                    dis[v] = d + e[id].w;
                    q.push({dis[v], v});
                }
        }
```

继续 run：零权块、最轻入口与定向（接上页同一结构体）：

```
        bel.assign(n + 1, 0);
        ids.clear();
        int cnt = 0, reached = 0;
        for (int u = 1; u <= n; u++)
            if (dis[u] != inf)
            {
                reached++;
                if (!bel[u]) zero(u, ++cnt);
            }
        vector<int> best(cnt + 1, -1);
        for (int u = 1; u <= n; u++)
            if (dis[u] != inf)
                for (auto [v, id] : g[u])
                    if (bel[u] != bel[v] && dis[v] == dis[u] + e[id].w)
                    {
                        int &b = best[bel[v]];
                        if (b == -1 || e[id].w < e[b].w) b = id;
                    }
        for (int c = 1; c <= cnt; c++)
            if (c != bel[s])
            {
                assert(best[c] != -1);
                ids.push_back(best[c]);
            }
        tree.assign(n + 1, {});
        weight = 0;
        for (int id : ids)
        {
            auto [u, v, w] = e[id];
            tree[u].push_back({v, id});
            tree[v].push_back({u, id});
            weight += w;
        }
        pre.assign(n + 1, -1);
        dfs(s, 0);
        valid = true;
        return reached;
    }
```

读取最短路树中的路径（接上页同一结构体）：

```
    // Forward undirected edge IDs; follow each edge from the current vertex.
    vector<int> path(int t) const
    {
        assert(valid && 1 <= t && t <= n);
        vector<int> p;
        if (dis[t] == inf) return p;
        while (t != root)
        {
            int id = pre[t];
            p.push_back(id);
            auto [u, v, w] = e[id];
            t = u == t ? v : u;
        }
        reverse(p.begin(), p.end());
        return p;
    }
};
```

应用题补充：Codeforces 545E / Paths and Trees

题意：连通无向正权图，给定源点，输出总边权最小的最短路树：先输出 weight，再输出 ids 加 1。n,m≤300000，w≤10^9，总树权可转 long long。题面不含零权、自环和重边，额外接口范围另由 API 验证。

所需模板：ShortestPathTree（第 733 页，44.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/545/E

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    ShortestPathTree t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u, v, w);
    }
    int s;
    cin >> s;
    t.run(s);
    cout << (long long)t.weight << '\n';
    for (int id : t.ids) cout << id + 1 << ' ';
    cout << '\n';
}
```

接口用法演示：零权最短路树、断开图与原边路径（接口演示）

演示约定：输入 n,m,s 及无向非负 signed64 边，每条仅 add 一次。首行输出“可达点数最小总边权树边数原边号”；随后逐点输出 INF 或“最短距离路径边数正向无向原边号”。所有权和用 int128 打印，原边号从 0 计。

所需模板：ShortestPathTree（第 733 页，44.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/shortest-path/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    ShortestPathTree t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u, v, w);
    }
    int reached = t.run(s);
    auto print = [](__int128 x)
    {
        if (x < 0)
        {
            cout << '-';
            x = -x;
        }
        string a;
        do
        {
            a += char('0' + x % 10);
            x /= 10;
        } while (x);
        reverse(a.begin(), a.end());
        cout << a;
    };
```

继续处理操作（接上页同一 main）：

```
    cout << reached << ' ';
    print(t.weight);
    cout << ' ' << t.ids.size();
    for (int id : t.ids) cout << ' ' << id;
    cout << '\n';
    for (int u = 1; u <= n; u++)
    {
        if (t.dis[u] == ShortestPathTree::inf) cout << "INF\n";
        else
        {
            auto path = t.path(u);
            print(t.dis[u]);
            cout << ' ' << path.size();
            for (int id : path) cout << ' ' << id;
            cout << '\n';
        }
    }
}
```

44.1.6 稠密图最短路：矩阵 Dijkstra

DenseDijkstra(n)，1-based 点号，0<n<INT_MAX；add(u,v,w) 加入有向非负 signed64 边，矩阵合并重边取最小值，允许零权、自环。无向图分别加双向；不保存原边编号。cost[u][v]=−1 表示无边，LLONG_MAX 仍是合法边权。

run(s) 每次从头求指定源点的最短距离，成功后 valid=true；dis 为 int128，inf 表示不可达。path(t) 返回正向顶点列表，不可达为空，源点自身为 {s}；pre 为父点，根及不可达为 −1。相同距离任选方案，不承诺字典序。add 使旧结果失效，支持重复换源与独立值拷贝；cost/dis/pre/root/valid 只读。

构造时间和空间 $O(n^2)$，add 为 O(1)，run 为 $O(n^2)$，不使用堆；path 为 O(路径长)。矩阵约 $8(n+1)^2$ 字节，另加 O(n) 的距离和辅助数组，按内存选择 n；稀疏图优先堆 Dijkstra。有限最短路径至多 n−1 条边，距离 <2^94，int128 及 inf=2^126 足够。每步选择未定点中距离最小者，只松弛未定点并严格改善，零权环不会形成父点环。

```
struct DenseDijkstra
{
    using I = __int128_t;
    static constexpr I inf = I(1) << 126;
    int n, root = 0;
    bool valid = false;
    vector<vector<long long>> cost;
    vector<I> dis;
    vector<int> pre;

    DenseDijkstra(int n) : n(n), cost(n + 1, vector<long long>(n + 1, -1))
    {
        assert(0 < n && n < INT_MAX);
    }

    void add(int u, int v, long long w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && w >= 0);
        auto &x = cost[u][v];
        if (x == -1 || w < x) x = w;
        valid = false;
    }
```

矩阵扫描、松弛与路径输出（接上页同一结构体）：

```
    void run(int s)
    {
        assert(1 <= s && s <= n);
        root = s;
        dis.assign(n + 1, inf);
        pre.assign(n + 1, -1);
        vector<bool> vis(n + 1);
        dis[s] = 0;
        for (int step = 0; step < n; step++)
        {
            int u = 0;
            for (int v = 1; v <= n; v++)
            {
                if (!vis[v] && (!u || dis[v] < dis[u])) u = v;
            }
            if (!u || dis[u] == inf) break;
            vis[u] = true;
            for (int v = 1; v <= n; v++)
                if (!vis[v] && cost[u][v] >= 0 && dis[v] > dis[u] + cost[u][v])
                {
                    dis[v] = dis[u] + cost[u][v];
                    pre[v] = u;
                }
        }
        valid = true;
    }

    // Forward vertex IDs; empty iff unreachable, {root} for the root.
    vector<int> path(int t) const
    {
        assert(valid && 1 <= t && t <= n);
        if (dis[t] == inf) return {};
        vector<int> p;
        for (; t != -1; t = pre[t]) p.push_back(t);
        reverse(p.begin(), p.end());
        return p;
    }
};
```

模板题使用：AOJ ALDS1_12_B / Single Source Shortest Path

题意：输入 n 和 n 行 “0-based 点号出度邻点权值…”，求 0 到各点距离，按原点号顺序逐行输出 “点号距离”。n≤100，m≤10000，0≤w≤100000，题面保证从 0 全部可达，距离可转 long long。

所需模板：DenseDijkstra（第 739 页，44.1.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=ALDS1_12_B&lang=en

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    DenseDijkstra g(n);
    for (int i = 0; i < n; i++)
    {
        int u, k;
        cin >> u >> k;
        while (k--)
        {
            int v;
            long long w;
            cin >> v >> w;
            g.add(u + 1, v + 1, w);
        }
    }
    g.run(1);
    for (int u = 1; u <= n; u++)
    {
        cout << u - 1 << ' ' << (long long)g.dis[u] << '\n';
    }
}
```

接口用法演示：矩阵 Dijkstra 距离与顶点路径（接口演示）

演示约定：输入 n,m,s 及 1-based 有向非负 signed64 边，逐点输出 INF 或 “int128 距离路径顶点数正向顶点列表”。重边合并为最小权；源点路径有一个点，与不可达空路径不同。

所需模板：DenseDijkstra（第 739 页，44.1.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/shortest-path/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    DenseDijkstra g(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add(u, v, w);
    }
    g.run(s);
    for (int v = 1; v <= n; v++)
        if (g.dis[v] == DenseDijkstra::inf)
            cout << "INF\n";
        else
        {
            __int128 x = g.dis[v];
            string a;
            do
            {
                a += char('0' + x % 10);
                x /= 10;
            } while (x);
            reverse(a.begin(), a.end());
            cout << a;
            auto p = g.path(v);
            cout << ' ' << p.size();
            for (int u : p) cout << ' ' << u;
            cout << '\n';
        }
}
```

44.1.7 SPFA：队列松弛、负环判定与有限路径

Spfa(n)，点号 1..n，0<n<INT_MAX。add(u,v,w) 返回 0-based 原边号，允许重边、自环、完整 signed64 权值，边数 ≤INT_MAX；加边令 valid=false。run(s) 查从 s 可达的负环，s=0 同时初始化所有点为 0 以查全图。返回 false 时 valid=false，全部 dis/pre 均为未完成状态，不能读取为最短路或恢复路径；需要负环方案或逐点负无穷分类用 BellmanFord。

run 返回 true 时 valid=true，dis[v]==inf 为不可达，其余为 int128 最短距离；run(0) 距离为任意起点出发的最小值，且本身就是全部约束 x_v−x_u≤w 的一个可行解。path(v) 返回正向原边号；不可达与零边路径均为空，结合 dis 区分。每次 run 重置临时队列/标记/路径边数，支持反复改起点与对象复制。

FIFO 队列，最坏 O(nm+n+m) 时间，O(n+m) 空间，不保证近线性；队列去重不等于永久访问标记。严格松弛后维护标签所代表游走的边数，达到 n 立即判负环。停止前标签最多 n 条边，绝对值 ≤n · 2^63<2^94，int128 与 inf=2^126 足够。不要无条件转 long long；e/g/dis/pre/valid 只读，其余按接口使用。

```
struct Spfa
{
    using I = __int128_t;
    static constexpr I inf = I(1) << 126;
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n;
    bool valid = false;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<I> dis;
    vector<int> pre;

    Spfa(int n) : n(n), g(n + 1)
    {
        assert(0 < n && n < INT_MAX);
    }

    int add(int u, int v, long long w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(e.size() < INT_MAX);
        int id = e.size();
        e.push_back({u, v, w});
        g[u].push_back(id);
        valid = false;
        return id;
    }
```

FIFO 队列与负环判定（接上页同一结构体）：

```
    // s=0 checks the whole graph. A false result invalidates all distances.
    bool run(int s)
    {
        assert(0 <= s && s <= n);
        valid = false;
        dis.assign(n + 1, inf);
        pre.assign(n + 1, -1);
        vector<int> in(n + 1), len(n + 1);
        queue<int> q;
        for (int u = 1; u <= n; u++)
            if (!s || u == s)
            {
                dis[u] = 0;
                in[u] = 1;
                q.push(u);
            }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            in[u] = 0;
            for (int id : g[u])
            {
                auto [from, v, w] = e[id];
                if (dis[v] <= dis[u] + w) continue;
                dis[v] = dis[u] + w;
                pre[v] = id;
                len[v] = len[u] + 1;
                if (len[v] >= n) return false;
                if (!in[v])
                {
                    in[v] = 1;
                    q.push(v);
                }
            }
        }
        valid = true;
        return true;
    }
```

成功运行后的有限路径（接上页同一结构体）：

```
    // Forward original-edge IDs; empty for unreachable or a zero-edge path.
    vector<int> path(int t) const
    {
        assert(valid && 1 <= t && t <= n);
        vector<int> ids;
        if (dis[t] == inf) return ids;
        while (pre[t] != -1)
        {
            ids.push_back(pre[t]);
            t = e[pre[t]].u;
        }
        reverse(ids.begin(), ids.end());
        return ids;
    }
};
```

模板题使用：Luogu P3385 / 从 1 可达的负环

题意：T 组图，每条输入边权非负时加双向边，负数仅加 u 到 v；run(1) 只查从 1 可达的负环，有则 YES，否则 NO。m 是输入记录数，不是最终边数；不可达部分的负环不影响本题。

所需模板：Spfa（第 743 页，44.1.7 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3385

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T;
    cin >> T;
    while (T--)
    {
        int n, m;
        cin >> n >> m;
        Spfa t(n);
        while (m--)
        {
            int u, v, w;
            cin >> u >> v >> w;
            t.add(u, v, w);
            if (w >= 0) t.add(v, u, w);
        }
        cout << (t.run(1) ? "NO" : "YES") << '\n';
    }
}
```

模板题使用：Luogu P5960 / 差分约束

题意：输入 x_u−x_v≤w，加入 v 到 u 权 w。run(0) 覆盖全部约束；无解输出 NO，否则直接输出 dis[1..n] 为任意可行解。n,m≤5000、|w|≤10000，有限解在 int 范围内；不必另建超级源点。

所需模板：Spfa（第 743 页，44.1.7 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5960

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Spfa t(n);
    while (m--)
    {
        int u, v, w;
        cin >> u >> v >> w;
        t.add(v, u, w);
    }
    if (!t.run(0)) cout << "NO\n";
    else
        for (int u = 1; u <= n; u++)
            cout << (long long)t.dis[u] << " \n"[u == n];
}
```

应用题补充：Luogu P1993 / 小 K 的农场

题意：三类关系：1 a b c 表示 x_a−x_b≥c，加入 a 到 b 权 −c；2 表示 ≤c，加入 b 到 a 权 c；3 表示相等，加入双向 0。run(0) 成功输出 Yes，否则 No。解可整体平移到非负，不要额外加入上界。

所需模板：Spfa（第 743 页，44.1.7 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1993

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Spfa t(n);
    while (m--)
    {
        int op, a, b, c;
        cin >> op >> a >> b;
        if (op != 3) cin >> c;
        if (op == 1) t.add(a, b, -c);
        else if (op == 2) t.add(b, a, c);
        else
        {
            t.add(a, b, 0);
            t.add(b, a, 0);
        }
    }
    cout << (t.run(0) ? "Yes" : "No") << '\n';
}
```

接口用法演示：SPFA 有限最短距离与路径（接口演示）

演示约定：输入 n,m,s 和 1-based 有向边，权值 signed64；s=0 查全图。发现相关负环只输出 NEGATIVE CYCLE；否则逐点输出 INF 或“int128 距离边数正向原边号”。全图负环与指定起点可达负环必须区分。

所需模板：Spfa（第 743 页，44.1.7 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/shortest-path/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    Spfa t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u, v, w);
    }
    if (!t.run(s))
    {
        cout << "NEGATIVE CYCLE\n";
        return 0;
    }
    auto print = [](__int128 x)
    {
        if (x < 0)
        {
            cout << '-';
            x = -x;
        }
        string a;
        do
        {
            a += char('0' + x % 10);
            x /= 10;
        } while (x);
        reverse(a.begin(), a.end());
        cout << a;
    };
```

继续处理操作（接上页同一 main）：

```
    for (int u = 1; u <= n; u++)
    {
        if (t.dis[u] == Spfa::inf) cout << "INF\n";
        else
        {
            auto path = t.path(u);
            print(t.dis[u]);
            cout << ' ' << path.size();
            for (int id : path) cout << ' ' << id;
            cout << '\n';
        }
    }
}
```

44.1.8 Bellman–Ford：负环影响与路径方案

BellmanFord(n)，点号 1..n，0<n<INT_MAX。add(u,v,w) 加有向边并返回 0-based 原边号，允许重边、自环和完整 signed64 边权，总边数 ≤INT_MAX。run(s) 返回是否不存在从 s 可达的负环；run(0) 把全体点初值设 0，等价于超级源点，用于查全图负环。每次 run 重置状态；add 后旧结果失效，须重新 run。

运行后 dis[v]==inf 表示不可达；neg[v] 为真表示从起点可达的负环能到达 v，距离为负无穷（这时不可读取 dis 作为答案）；其他点的 dis 为有限 int128 最短距离。path(v) 返回有限最短路的正向原边号；不可达、负无穷或空路径都返回空 vector，须结合 dis/neg 判定。cycle 为空表示无相关负环，否则是一个相关负环的正向原边号，可通过 e[id].u/v/w 还原顶点和权值。run(0) 的有限距离是从任意起点出发的最小值，允许零边路径，不是某个固定单源距离。

O(nm+n+m) 时间、O(n+m) 空间；path 输出 O(路径长)。inf=2^126，全过程 int128 累加，不能无条件转为 long long。在给定 n,m 的 int 边界内，n 轮最多 nm 次更新，标签对应游走至多 nm 条边，绝对值 <2^125，不碰 inf 也不溢出。e/g/dis/pre/neg/cycle 和 ready 只读，不手改。

```
struct BellmanFord
{
    using I = __int128_t;
    static constexpr I inf = I(1) << 126;
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n;
    bool ready = false;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<I> dis;
    vector<int> pre, neg, cycle;

    BellmanFord(int n) : n(n)
    {
        assert(0 < n && n < INT_MAX);
        g.resize(n + 1);
    }

    int add(int u, int v, long long w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(e.size() < INT_MAX);
        int id = e.size();
        e.push_back({u, v, w});
        g[u].push_back(v);
        ready = false;
        return id;
    }
```

run：逐轮松弛与负环种子（接上页同一结构体）：

```
    // s=0 initializes every vertex to zero, for detecting any negative cycle.
    bool run(int s)
    {
        assert(0 <= s && s <= n);
        dis.assign(n + 1, s ? inf : 0);
        pre.assign(n + 1, -1);
        neg.assign(n + 1, 0);
        cycle.clear();
        dis[s] = 0;
        queue<int> q;
        int last = -1;
        for (int step = 0; step < n; step++)
        {
            last = -1;
            for (int id = 0; id < (int)e.size(); id++)
            {
                auto [u, v, w] = e[id];
                if (dis[u] == inf || dis[v] <= dis[u] + w) continue;
                dis[v] = dis[u] + w;
                pre[v] = id;
                last = v;
                if (step == n - 1 && !neg[v])
                {
                    neg[v] = 1;
                    q.push(v);
                }
            }
            if (last == -1) break;
        }
```

继续 run：恢复负环并传播；随后为 path（接上页同一结构体）：

```
        if (last != -1)
        {
            int x = last;
            for (int i = 0; i < n; i++) x = e[pre[x]].u;
            int start = x;
            do
            {
                cycle.push_back(pre[x]);
                x = e[pre[x]].u;
            } while (x != start);
            reverse(cycle.begin(), cycle.end());
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int v : g[u])
                if (!neg[v])
                {
                    neg[v] = 1;
                    q.push(v);
                }
        }
        ready = true;
        return cycle.empty();
    }

    // Finite path in forward original-edge IDs; empty also for unreachable/-inf.
    vector<int> path(int t) const
    {
        assert(ready && 1 <= t && t <= n);
        vector<int> ids;
        if (dis[t] == inf || neg[t]) return ids;
        while (pre[t] != -1)
        {
            ids.push_back(pre[t]);
            t = e[pre[t]].u;
        }
        reverse(ids.begin(), ids.end());
        return ids;
    }
};
```

模板题使用：AOJ GRL_1_B / 含负边的单源最短路

题意：0-based 有向图，n≤1000、m≤2000、边权绝对值 ≤10000。从 r 可达负环时输出 NEGATIVE CYCLE，否则逐点输出距离，不可达输出 INF。转成模板 1-based 编号；本题有限距离在 signed64 内，故可显式转 long long 输出。不可达位置的负环不影响此题判断。

所需模板：BellmanFord（第 750 页，44.1.8 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=GRL_1_B

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    BellmanFord t(n);
    while (m--)
    {
        int u, v, w;
        cin >> u >> v >> w;
        t.add(u + 1, v + 1, w);
    }
    if (!t.run(s + 1)) cout << "NEGATIVE CYCLE\n";
    else
        for (int u = 1; u <= n; u++)
        {
            if (t.dis[u] == BellmanFord::inf) cout << "INF\n";
            else cout << (long long)t.dis[u] << '\n';
        }
}
```

应用题补充：CSES1197 / 输出全图负环

题意：n≤2500、m≤5000，查任意位置负环并输出一条闭合顶点序列。调用 run(0) 而不是 run(1)；cycle 存正向边号，先输出第一条边的起点，再逐边输出终点。无负环 NO，有负环 YES。原题输出顶点，模板保留边号以区分平行边。

所需模板：BellmanFord（第 750 页，44.1.8 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://cses.fi/problemset/task/1197/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    BellmanFord t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u, v, w);
    }
    if (t.run(0)) cout << "NO\n";
    else
    {
        cout << "YES\n" << t.e[t.cycle[0]].u;
        for (int id : t.cycle) cout << ' ' << t.e[id].v;
        cout << '\n';
    }
}
```

接口用法演示：有限路径、不可达与负无穷（接口演示）

演示约定：输入 n,m,s 和 1-based 边，边权 signed64；s=0 代表全体点为起点。逐点输出 INF、−INF，或“int128 距离边数正向原边号列表”；末行 C 边数负环边号列表。空有限路径与不可达不同，负无穷点不用当前 dis 作答案。边号按输入顺序从 0 计。

所需模板：BellmanFord（第 750 页，44.1.8 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/shortest-path/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    BellmanFord t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u, v, w);
    }
    t.run(s);
    auto print = [](__int128 x)
    {
        if (x < 0)
        {
            cout << '-';
            x = -x;
        }
        string a;
        do
        {
            a += char('0' + x % 10);
            x /= 10;
        } while (x);
        reverse(a.begin(), a.end());
        cout << a;
    };
```

继续处理操作（接上页同一 main）：

```
    for (int u = 1; u <= n; u++)
    {
        if (t.dis[u] == BellmanFord::inf) cout << "INF\n";
        else if (t.neg[u]) cout << "-INF\n";
        else
        {
            auto path = t.path(u);
            print(t.dis[u]);
            cout << ' ' << path.size();
            for (int id : path) cout << ' ' << id;
            cout << '\n';
        }
    }
    cout << "C " << t.cycle.size();
    for (int id : t.cycle) cout << ' ' << id;
    cout << '\n';
}
```

44.2 k 短路

44.2.1 非负权严格次短行走

StrictSecondShortest(n)，点号 1..n，n<INT_MAX；add(u,v,w) 添加有向边，无向边添加两次。允许重边、自环及零权，0<=w<LLONG_MAX。run(s) 返回 dis[v][0/1]，分别为最短距离与严格大于它的最小距离；同长路线不重复计数，允许重复点边，s 到自身包含空路。不存在或长度 >=LLONG_MAX 返回 inf=LLONG_MAX；只保证此阈值以下的精确答案，超界扩展跳过。只读 g/n；可重复求解、加边后重算。O((n+m) log(n+m+1)) 时间，O(n+m) 空间，不返回行走方案。

```
struct StrictSecondShortest
{
    using ll = long long;
    static constexpr ll inf = LLONG_MAX;
    int n;
    vector<vector<pair<int, ll>>> g;

    StrictSecondShortest(int n) : n(n), g(n + 1)
    {
        assert(n > 0 && n < INT_MAX);
    }

    void add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(0 <= w && w < inf);
        g[u].push_back({v, w});
    }
```

继续求解（接上页同一结构体）：

```
    vector<array<ll, 2>> run(int s) const
    {
        assert(1 <= s && s <= n);
        vector<array<ll, 2>> dis(n + 1, {inf, inf});
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
        dis[s][0] = 0;
        q.push({0, s});
        while (!q.empty())
        {
            auto [d, u] = q.top();
            q.pop();
            if (d > dis[u][1]) continue;
            for (auto [v, w] : g[u])
            {
                if (w >= inf - d) continue;
                ll next = d + w;
                if (next < dis[v][0])
                {
                    swap(next, dis[v][0]);
                    q.push({dis[v][0], v});
                }
                if (dis[v][0] < next && next < dis[v][1])
                {
                    dis[v][1] = next;
                    q.push({next, v});
                }
            }
        }
        return dis;
    }
};
```

应用题补充：P2865 [USACO06NOV] Roadblocks G

题意：无向正权图，从 1 到 n 求严格次短距离，允许回溯；多条等长最短路不算次短。n<=5000，m<=100000，边权 1..5000，题目保证次短存在。

所需模板：StrictSecondShortest（第 757 页，44.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2865

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    StrictSecondShortest solver(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        solver.add(u, v, w);
        solver.add(v, u, w);
    }
    cout << solver.run(1)[n][1] << '\n';
}
```

44.2.2 正权前 K 短行走

KShortestWalks(n)，点号 1..n，n<INT_MAX；add(u,v,w) 添加有向边，要求 0<w<LLONG_MAX，零权边/环不在接口范围内。允许自环与重边，每次 add 视为不同边。run(s,t,k) 按非降序返回至多 k 个行走长度，k>=0；不同边序列分别计数，即使等长。允许重复点边，不是简单路径算法。s=t 时第一个答案为空路 0；若排除空路，调用 k+1 后删除首项，需 k<INT_MAX。终点仍可继续走再回来。长度 >=LLONG_MAX 不返回；只有阈值以下答案有保证。每个顶点至多扩展 k 次，O(n+km log(km+2)) 时间、O(n+km) 空间；k 大时应改用更高效算法。只读 g/n，可重复求解和加边后重算，不返回方案。

```
struct KShortestWalks
{
    using ll = long long;
    static constexpr ll inf = LLONG_MAX;
    int n;
    vector<vector<pair<int, ll>>> g;

    KShortestWalks(int n) : n(n), g(n + 1)
    {
        assert(n > 0 && n < INT_MAX);
    }

    void add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(0 < w && w < inf);
        g[u].push_back({v, w});
    }
```

继续求解（接上页同一结构体）：

```
    vector<ll> run(int s, int t, int k) const
    {
        assert(1 <= s && s <= n && 1 <= t && t <= n && k >= 0);
        vector<ll> answer;
        if (k == 0) return answer;
        vector<int> count(n + 1);
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
        q.push({0, s});
        while (!q.empty())
        {
            auto [d, u] = q.top();
            q.pop();
            if (count[u] == k) continue;
            count[u]++;
            if (u == t)
            {
                answer.push_back(d);
                if (int(answer.size()) == k) break;
            }
            for (auto [v, w] : g[u])
            {
                if (count[v] < k && w < inf - d)
                {
                    q.push({d + w, v});
                }
            }
        }
        return answer;
    }
};
```

应用题补充：P2901 [USACO08MAR] Cow Jogging G

题意：从 n 到 1 求前 k 条路线长度，等长按边序列分别计数，不足输出-1。n<=1000，m<=10000，k<=100，边权 1..10^6；边均从大编号指向小编号。此题为 DAG 应用，不验证有环接口域。

所需模板：KShortestWalks（第 760 页，44.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2901

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k;
    cin >> n >> m >> k;
    KShortestWalks solver(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        solver.add(u, v, w);
    }
    auto answer = solver.run(n, 1, k);
    for (int i = 0; i < k; i++)
    {
        cout << (i < int(answer.size()) ? answer[i] : -1) << '\n';
    }
}
```

第四十五章　生成树问题

45.1　最小生成树

45.1.1　Kruskal 重构森林与阈值连通块

代码依赖：dsu（第 373 页，29.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

KruskalTree(n)，0≤n≤INT_MAX/2，原点 0..n-1；add(u,v,w) 加入无向边，返回插入编号，边数 ≤INT_MAX，支持完整 signed64 权、重边、自环、负权和断连。build(descending=false) 重建并返回连通块数；默认升序，true 降序。每次成功合并创建父点：ch 为左右儿子，val 为合并边权；叶点 ch={-1,-1} 且 val 无意义。总点数 k=2n-连通块数，父编号大于子编号；可按 n..k-1 聚合叶点信息，不需要模拟 DFS。up[0] 为父点，根为-1。

component(u,w,strict=false) 返回阈值子图中 u 所属分量的最高重构节点；默认升序保留边权 ≤w，降序保留 ≥w，strict 改成 < 或 >。其子树叶点恰为分量，可从聚合数组查大小、最小值等。同权边必须连续爬完所有合法祖先；不以叶点 val 充当无穷哨兵。构建 O(m log(m+1)+n log(n+1))，空间 O(m+n log(n+1))，查询 O(log(n+1))。依赖 dsu，合并前按大小交换根。支持重复 build、切换方向与值复制；加边后必须重建再查询，构建后成员只读。空图只允许 build、不允许查询。

```
struct KruskalTree
{
    struct Edge
    {
        int u, v;
        long long w;
    };

    int n;
    bool down = false;
    vector<Edge> e;
    vector<long long> val;
    vector<array<int, 2>> ch;
    vector<vector<int>> up;

    KruskalTree(int n) : n(n)
    {
        assert(0 <= n && n <= INT_MAX / 2);
    }

    int add(int u, int v, long long w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        e.push_back({u, v, w});
        return (int)e.size() - 1;
    }
```

重建森林与倍增表（接上页同一结构体）：

```
    int build(bool descending = false)
    {
        down = descending;
        ch.assign(n, {-1, -1});
        val.assign(n, 0);
        dsu d(n);
        vector<int> top(n), ids(e.size());
        iota(top.begin(), top.end(), 0);
        iota(ids.begin(), ids.end(), 0);
        sort(ids.begin(), ids.end(), [&](int a, int b)
        {
            if (e[a].w != e[b].w)
                return down ? e[a].w > e[b].w : e[a].w < e[b].w;
            return a < b;
        });
        for (int id : ids)
        {
            auto [u, v, w] = e[id];
            u = d.find(u);
            v = d.find(v);
            if (u == v) continue;
            if (d.sz[u] < d.sz[v]) swap(u, v);
            int x = ch.size();
            ch.push_back({top[u], top[v]});
            val.push_back(w);
            d.merge(u, v);
            top[u] = x;
        }
        int k = ch.size();
        int h = max(1, (int)bit_width((unsigned)k));
        up.assign(h, vector<int>(k, -1));
        for (int u = n; u < k; u++)
        {
            for (int v : ch[u]) up[0][v] = u;
        }
        for (int j = 1; j < h; j++)
        {
            for (int u = 0; u < k; u++)
            {
                int v = up[j - 1][u];
                if (v != -1) up[j][u] = up[j - 1][v];
            }
        }
        return 2 * n - k;
    }
```

阈值分量查询（接上页同一结构体）：

```
    int component(int u, long long w, bool strict = false) const
    {
        assert(0 <= u && u < n && !up.empty());
        for (int j = (int)up.size() - 1; j >= 0; j--)
        {
            int v = up[j][u];
            if (v == -1) continue;
            bool ok = down ? val[v] >= w : val[v] <= w;
            if (strict && val[v] == w) ok = false;
            if (ok) u = v;
        }
        return u;
    }
};
```

为何阈值查询正确 升序处理完所有权不超过 w 的边后，并查集恰好是该阈值子图的连通分量。每次成功合并的两棵子树叶点集合不交，新父点记录其并集；沿叶点到根的内部点权单调不减。因此最高合法祖先的叶点集合就是答案。降序情形反向比较；严格阈值排除等权合并。同权内部点可能连续出现，倍增必须爬到最高合法点。

如何聚合分量 叶点先存原数据，随后按编号 n 到 k-1 合并两个儿子的结果，可求大小、最小值、最大值等。不必为聚合另跑 DFS；这利用的是建点时已经确定的父子拓扑顺序。断连图的各棵树独立，无须添加超级根。

归程建模 先用长度求每个点到 1 号点的最短步行距离。汽车只能走海拔严格大于水位的边，故选降序构建和 strict=true；可在分量内任意点下车，答案为该重构子树的叶点距离最小值。普通最小生成树的总权不能代替这个分量查询。

应用题补充：P4768 [NOI2018] 归程

题意：汽车只能经过海拔严格大于水位的边；可在可达分量任意点下车，再步行到 1 号点。Dijkstra 预处理步行距离，降序重构树聚合最小值，支持依赖上次答案的在线解码。

所需模板：dsu（第 373 页，29.1.1 节）、KruskalTree（第 763 页，45.1.1 节）、Dijkstra（第 722 页，44.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4768

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        int n, m;
        cin >> n >> m;
        KruskalTree tr(n);
        Dijkstra g(n);
        for (int i = 0; i < m; i++)
        {
            int u, v;
            long long l, a;
            cin >> u >> v >> l >> a;
            g.add(u, v, l);
            g.add(v, u, l);
            tr.add(u - 1, v - 1, a);
        }
        g.run(1);
        tr.build(true);
        vector<long long> best(tr.ch.size());
        for (int u = 0; u < n; u++) best[u] = g.dis[u + 1];
        for (int u = n; u < (int)tr.ch.size(); u++)
        {
            auto [l, r] = tr.ch[u];
            best[u] = min(best[l], best[r]);
        }
        int q, k;
        long long s, last = 0;
        cin >> q >> k >> s;
        while (q--)
        {
            long long v, p;
            cin >> v >> p;
            v = (v + k * last - 1) % n;
            p = (p + k * last) % (s + 1);
            last = best[tr.component(v, p, true)];
            cout << last << '\n';
        }
    }
}
```

接口用法演示：API 阈值连通块聚合

演示约定：输入 n m q down，点 0-based；m 条 u v w，无向可断连。先输出全图连通块数。每次 u w strict，输出对应阈值连通块的点数与最小原点号；down 和 strict 取 0/1。

所需模板：dsu（第 373 页，29.1.1 节）、KruskalTree（第 763 页，45.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/mst/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q, down;
    cin >> n >> m >> q >> down;
    KruskalTree tr(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        tr.add(u, v, w);
    }
    cout << tr.build(down) << '\n';
    int k = tr.ch.size();
    vector<int> cnt(k, 1), low(k);
    iota(low.begin(), low.end(), 0);
    for (int u = n; u < k; u++)
    {
        auto [l, r] = tr.ch[u];
        cnt[u] = cnt[l] + cnt[r];
        low[u] = min(low[l], low[r]);
    }
    while (q--)
    {
        int u, strict;
        long long w;
        cin >> u >> w >> strict;
        int v = tr.component(u, w, strict);
        cout << cnt[v] << ' ' << low[v] << '\n';
    }
}
```

45.1.2 曼哈顿最小生成树

代码依赖：dsu（第 373 页，29.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

Point 为 pair<long long,long long>，坐标支持完整有符号 64 位，点数须 ≤INT_MAX 且内存可分配。candidates(points) 给出最多 4n 条 (weight,u,v) 候选，允许重复边；solve(points) 返回精确 128 位 weight 与原下标边对 edges。空/单点返回 0 且无边，重合点仍为不同顶点；同权最优树不唯一。先转 128 位再变换/加减，输入不变。四方向扫描与 Kruskal 为 O(n log(n+1))、O(n) 空间；调用处选较大 dsu 根在前，不改变普通 dsu 语义。候选保证只针对无约束完整 L1 图，不能直接删禁边后声称仍含受限 MST。

```
struct ManhattanMST
{
    using I = __int128;
    using Point = pair<long long, long long>;
    using Edge = tuple<I, int, int>;
    struct Result
    {
        I weight = 0;
        vector<pair<int, int>> edges;
    };

    static vector<Edge> candidates(const vector<Point> &points)
    {
        assert(points.size() <= INT_MAX);
        int n = int(points.size());
        vector<pair<I, I>> p(points.begin(), points.end());
        vector<int> id(n);
        iota(id.begin(), id.end(), 0);
        vector<Edge> e;
        e.reserve(size_t(n) * 4);
        for (int dir = 0; dir < 4; dir++)
        {
            sort(id.begin(), id.end(), [&](int i, int j)
            {
                return pair{p[i].first + p[i].second, i} <
                       pair{p[j].first + p[j].second, j};
            });
```

活动点扫描与 Kruskal（接上页同一结构体，仍在四轮循环中）：

```
            map<I, int> sweep;
            for (int i : id)
            {
                auto it = sweep.lower_bound(-p[i].second);
                while (it != sweep.end())
                {
                    int j = it->second;
                    I dx = p[i].first - p[j].first;
                    I dy = p[i].second - p[j].second;
                    if (dx < dy) break;
                    e.emplace_back(dx + dy, i, j);
                    it = sweep.erase(it);
                }
                sweep[-p[i].second] = i;
            }
            for (auto &[x, y] : p)
                if (dir & 1) x = -x;
                else swap(x, y);
        }
        return e;
    }

    static Result solve(const vector<Point> &points)
    {
        auto e = candidates(points);
        int n = int(points.size());
        Result r;
        if (n < 2) return r;
        sort(e.begin(), e.end());
        dsu uf(n);
        for (auto [w, u, v] : e)
        {
            int a = uf.find(u), b = uf.find(v);
            if (a == b) continue;
            if (uf.size(a) < uf.size(b)) swap(a, b);
            uf.merge(a, b);
            r.weight += w;
            r.edges.emplace_back(u, v);
            if (r.edges.size() == points.size() - 1) break;
        }
        assert(r.edges.size() == points.size() - 1);
        return r;
    }
};
```

模板题使用：Library Checker Manhattan MST

题意：1≤N≤200000，N 个点的坐标 0..10^9，可重合且每个输入位置仍是不同顶点。完整图边权为曼哈顿距离；输出最小总权及 N−1 条 0-based 原编号边，任意最优树均可。结果 weight 为 128 位；本题粗上界 2e9 乘以 N−1，严格小于 4e14，所以此驱动才可转 long long 打印。N=1 输出 0、不输出边；库的空输入和有符号 64 位坐标另属 API 扩展。

所需模板：dsu（第 373 页，29.1.1 节）、ManhattanMST（第 768 页，45.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/manhattanmst

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<ManhattanMST::Point> p(n);
    for (auto &[x, y] : p) cin >> x >> y;
    auto r = ManhattanMST::solve(p);
    cout << (long long)r.weight << '\n';
    for (auto [u, v] : r.edges) cout << u << ' ' << v << '\n';
}
```

45.1.3 严格与非严格次小生成树、换边方案

代码依赖：dsu（第 373 页，29.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

SecondMST(n)，n≥1，点号 0..n-1。add(u,v,w) 返回 0-based 边号；允许自环、重边、完整 signed64 位权。solve(strict=true) 返回 connected、128 位 MST 总权 weight、所选边号 tree，以及可选 128 位次小权 next 和换入/换出边号 in/out。strict=true 要求权值严格增大；false 要求边集与返回的 MST 不同，可同权。不连通返回默认空结果；无次小树时 next 为空且 in/out=-1。存在时将 tree 中的 out 删去并加入 in 即得方案。可加边后重算；每次 solve 重建。依赖 dsu，沿用路径减半及 y 根挂 x 根；不用按秩合并。时间 O((n+m)log(n+m))，空间 O(m+n log n)，DFS 递归深度 O(n)。merge/dfs/path 与工作数组为内部实现；原始 e 只读。

```
struct SecondMST
{
    using I = __int128_t;
    using Pair = array<int, 2>;

    struct Edge
    {
        int u, v;
        long long w;
    };

    struct Result
    {
        bool connected = false;
        I weight = 0;
        vector<int> tree;
        optional<I> next;
        int in = -1, out = -1;
    };

    int n, h;
    vector<Edge> e;
    vector<vector<pair<int, int>>> g;
    vector<int> dep;
    vector<vector<int>> up;
    vector<vector<Pair>> mx;

    SecondMST(int n) : n(n), h(bit_width((unsigned)n)) { assert(n > 0); }

    int add(int u, int v, long long w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        e.push_back({u, v, w});
        return (int)e.size() - 1;
    }
```

不同权摘要与递归预处理（接上页同一结构体）：

```
    // Keep edge IDs of the two largest DISTINCT weights; -1 means absent.
    Pair merge(Pair a, Pair b) const
    {
        for (int x : b)
        {
            if (x == -1) continue;
            if (a[0] == -1 || e[x].w > e[a[0]].w)
            {
                a[1] = a[0];
                a[0] = x;
            }
            else if (e[x].w != e[a[0]].w && (a[1] == -1 || e[x].w > e[a[1]].w))
                a[1] = x;
        }
        return a;
    }

    void dfs(int u, int p)
    {
        for (int j = 1; j < h; j++)
        {
            int v = up[j - 1][u];
            up[j][u] = up[j - 1][v];
            mx[j][u] = merge(mx[j - 1][u], mx[j - 1][v]);
        }
        for (auto [v, id] : g[u])
            if (v != p)
            {
                dep[v] = dep[u] + 1;
                up[0][v] = u;
                mx[0][v] = {id, -1};
                dfs(v, u);
            }
    }
```

路径查询（接上页同一结构体）：

```
    Pair path(int u, int v) const
    {
        Pair ans{-1, -1};
        if (dep[u] < dep[v]) swap(u, v);
        for (int j = h - 1; j >= 0; j--)
            if ((dep[u] - dep[v]) >> j & 1)
            {
                ans = merge(ans, mx[j][u]);
                u = up[j][u];
            }
        if (u == v) return ans;
        for (int j = h - 1; j >= 0; j--)
            if (up[j][u] != up[j][v])
            {
                ans = merge(ans, merge(mx[j][u], mx[j][v]));
                u = up[j][u];
                v = up[j][v];
            }
        return merge(ans, merge(mx[0][u], mx[0][v]));
    }
```

Kruskal 与逐边换入（接上页同一结构体）：

```
    Result solve(bool strict = true)
    {
        vector<int> ids(e.size());
        iota(ids.begin(), ids.end(), 0);
        sort(ids.begin(),
             ids.end(),
             [&](int a, int b) { return tie(e[a].w, a) < tie(e[b].w, b); });
        dsu ds(n);
        vector<bool> used(e.size());
        g.assign(n, {});
        Result ans;
        for (int id : ids)
        {
            auto [u, v, w] = e[id];
            if (!ds.merge(u, v)) continue;
            used[id] = true;
            ans.weight += (I)w;
            ans.tree.push_back(id);
            g[u].push_back({v, id});
            g[v].push_back({u, id});
        }
        if ((int)ans.tree.size() != n - 1) return {};
        ans.connected = true;
        dep.assign(n, 0);
        up.assign(h, vector<int>(n));
        mx.assign(h, vector<Pair>(n, Pair{-1, -1}));
        dfs(0, -1);
        for (int id : ids)
        {
            auto [u, v, w] = e[id];
            if (used[id] || u == v) continue;
            auto p = path(u, v);
            int out = p[0];
            if (strict && e[out].w == w) out = p[1];
            if (out == -1) continue;
            I cost = ans.weight + (I)w - (I)e[out].w;
            if (!ans.next || cost < *ans.next)
            {
                ans.next = cost;
                ans.in = id;
                ans.out = out;
            }
        }
        return ans;
    }
};
```

应用题补充：Luogu P4180 [BJWC2010] 严格次小生成树

题意：无向图求总权严格大于 MST 的最小生成树权值；n 至 10 万、m 至 30 万、边权 $0..10^9$，允许自环并保证答案存在。1-based 顶点读入后减一，solve 默认严格；题目答案可安全转 long long。比赛应用，不计非比赛模板覆盖。

所需模板：dsu（第 373 页，29.1.1 节）、SecondMST（第 771 页，45.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4180

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    SecondMST mst(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        mst.add(u - 1, v - 1, w);
    }
    auto ans = mst.solve();
    // The problem guarantees a strict second tree and a 64-bit answer.
    assert(ans.connected && ans.next);
    cout << (long long)*ans.next << '\n';
}
```

45.1.4 稠密图 Prim 最小生成森林

Prim(n)，0≤n≤INT_MAX，点号 0..n-1；add(u,v,w) 自动加无向边，允许完整 signed64 权、零权、负权、自环，重边只保留最小权。不保存原边编号；cost 为 int128 矩阵，inf=2^126 表示缺边，−1 和 LLONG_MAX 均为合法边权。

run() 每次重建最小生成森林，返回原图连通块数。n>0 时返回 1 表示 MST，空图返回 0。weight 为 int128 总权；pre[v] 为父点，根为 −1，非根选边权为 cost[pre[v]][v]。父点森林无环，支持断连和孤立点；同权任选，不承诺字典序。加边后须重新 run，支持重复运行与独立复制；成员只读。

构造和存储 $O(n^2)$，add 为 $O(1)$，run 为 $O(n^2)$，矩阵约 $16n^2$ 字节，按实际内存选择 n；稀疏图优先 Kruskal。每步选到已选点集的最轻边；所有剩余点均无跨边时开启新树，不把 inf 计入总权。只向未选点严格更新父点，负自环不入树。总权绝对值 <2^94，可为 −1 或超出 long long，不能用权值判断断连。

```
struct Prim
{
    using I = __int128_t;
    static constexpr I inf = I(1) << 126;
    int n;
    vector<vector<I>> cost;
    vector<int> pre;
    I weight = 0;

    Prim(int n) : n(n), cost(n, vector<I>(n, inf))
    {
        assert(n >= 0);
    }

    void add(int u, int v, long long w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        cost[u][v] = cost[v][u] = min(cost[u][v], I(w));
    }
```

矩阵扫描、割边松弛与父点森林（接上页同一结构体）：

```
    // Rebuild a minimum spanning forest; return its component count.
    int run()
    {
        vector<I> d(n, inf);
        vector<bool> vis(n);
        pre.assign(n, -1);
        weight = 0;
        int cnt = 0;
        for (int step = 0; step < n; step++)
        {
            int u = -1;
            for (int v = 0; v < n; v++)
            {
                if (!vis[v] && (u == -1 || d[v] < d[u])) u = v;
            }
            if (d[u] == inf)
            {
                cnt++;
            }
            else
            {
                weight += d[u];
            }
            vis[u] = true;
            for (int v = 0; v < n; v++)
                if (!vis[v] && cost[u][v] < d[v])
                {
                    d[v] = cost[u][v];
                    pre[v] = u;
                }
        }
        return cnt;
    }
};
```

模板题使用：AOJ ALDS1_12_A / Minimum Spanning Tree

题意：输入 n 及 n×n 对称邻接矩阵，−1 表示缺边，其余为 0..2000；n≤100 且保证连通。只读取上三角的有效边，输出 MST 总权；本题可安全转 long long。注意输入 −1 缺边是题目约定，Prim.add 的 −1 是合法负权。

所需模板：Prim（第 776 页，45.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=ALDS1_12_A&lang=en

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    Prim g(n);
    for (int u = 0; u < n; u++)
        for (int v = 0; v < n; v++)
        {
            int w;
            cin >> w;
            if (u < v && w != -1) g.add(u, v, w);
        }
    g.run();
    cout << (long long)g.weight << '\n';
}
```

接口用法演示：矩阵 Prim 总权与父点森林（接口演示）

演示约定：输入 n,m 及 0-based 无向 signed64 边，输出连通块数、int128 最小生成森林总权，以及 pre[0..n-1]；根为 −1，其余为父点。允许空图、断连、负权、自环和重边，重边合并为最小权；不输出原边编号。

所需模板：Prim（第 776 页，45.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/mst/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Prim g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add(u, v, w);
    }
    cout << g.run() << ' ';
    __int128 x = g.weight;
    if (x < 0)
    {
        cout << '-';
        x = -x;
    }
    string a;
    do
    {
        a += char('0' + x % 10);
        x /= 10;
    } while (x);
    reverse(a.begin(), a.end());
    cout << a;
    for (int p : g.pre) cout << ' ' << p;
    cout << '\n';
}
```

45.1.5 Kruskal 最小生成树与生成森林

代码依赖：dsu（第 373 页，29.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

Kruskal(n)，0≤n≤INT_MAX，点号 0..n-1；add(u,v,w) 加入无向边并返回 0-based 原边号，边权为完整 signed64，支持重边、自环和负权，边数 ≤INT_MAX。run() 每次从全部原边重建最小生成森林，返回连通块数；n>0 时返回 1 表示存在 MST，空图返回 0。weight 为 int128 总权，ids 为所选原边号，按 (权值, 原编号) 排序；e 保持插入顺序。加边后必须重新 run，旧结果不代表新图。支持重复运行和独立值复制。

依赖普通 dsu，调用方先把较大根放在前，保留 dsu 的 fa[y]=x 语义。时间 O(n+m log(m+1)+m α(n+1))，空间 O(n+m)；add 均摊 O(1)。总权绝对值 <2^94，不能无条件转 long long。自环不入树，断开图仍返回各块最小树；不使用 −1 总权判断失败，合法 MST 总权也可为 −1。成员只读，n 和 e 不直接修改。

```
struct Kruskal
{
    using I = __int128_t;
    struct Edge
    {
        int u, v;
        long long w;
    };

    int n;
    vector<Edge> e;
    vector<int> ids;
    I weight = 0;

    Kruskal(int n) : n(n)
    {
        assert(n >= 0);
    }

    int add(int u, int v, long long w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        assert(e.size() < INT_MAX);
        e.push_back({u, v, w});
        return int(e.size()) - 1;
    }
```

边排序、合并与原边方案（接上页同一结构体）：

```
    // Rebuild a minimum spanning forest; return its component count.
    int run()
    {
        vector<int> order(e.size());
        iota(order.begin(), order.end(), 0);
        sort(order.begin(), order.end(), [&](int a, int b)
        {
            return tie(e[a].w, a) < tie(e[b].w, b);
        });
        dsu d(n);
        ids.clear();
        weight = 0;
        int cnt = n;
        for (int id : order)
        {
            auto [u, v, w] = e[id];
            u = d.find(u);
            v = d.find(v);
            if (u == v) continue;
            if (d.sz[u] < d.sz[v]) swap(u, v);
            d.merge(u, v);
            ids.push_back(id);
            weight += I(w);
            cnt--;
        }
        return cnt;
    }
};
```

模板题使用：Luogu P3366 / 最小生成树

题意：无向图求最小生成树总权，不连通输出 orz；n≤5000、m≤200000、边权 1..10000，允许自环重边。输入 1-based 点号减一，run()!=1 表示断连；本题总权可安全转 long long。

所需模板：dsu（第 373 页，29.1.1 节）、Kruskal（第 780 页，45.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3366

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Kruskal g(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add(u - 1, v - 1, w);
    }
    if (g.run() != 1)
    {
        cout << "orz\n";
    }
    else
    {
        cout << (long long)g.weight << '\n';
    }
}
```

接口用法演示：最小生成森林与原边方案（接口演示）

演示约定：输入 n,m 及 0-based 无向 signed64 边，输出连通块数、int128 最小生成森林总权、选边数和 0-based 原边编号。支持空图、负权、自环重边与断连；方案按 (权值, 原编号) 排序。接口演示，不计在线模板题。

所需模板：dsu（第 373 页，29.1.1 节）、Kruskal（第 780 页，45.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/mst/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Kruskal g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add(u, v, w);
    }
    cout << g.run() << ' ';
    __int128 x = g.weight;
    if (x < 0)
    {
        cout << '-';
        x = -x;
    }
    string a;
    do
    {
        a += char('0' + x % 10);
        x /= 10;
    } while (x);
    reverse(a.begin(), a.end());
    cout << a << ' ' << g.ids.size();
    for (int id : g.ids) cout << ' ' << id;
    cout << '\n';
}
```

45.1.6 曼哈顿最小生成树：闭锥扫描与平局割证明

接口见第 45.1.2 节（第 768 页），正式题用法见第 45.1.2 节（第 770 页）。完整无向图的边权是 $d(p,q) = |p_x - q_x| + |p_y - q_y|$。不枚举二次边集：每个点只保留四个向右闭锥中的最近邻，再对候选边作 Kruskal。候选图含有至少一棵最小生成树，不保证含所有最优边；禁边、颜色等额外约束不能直接套用。

一个方向的扫描 变换后的锥取 $q_x - p_x \geq q_y - p_y \geq 0$。按 $(x+y,$ 原编号$)$ 排序，活动点用 $-y$ 作 map 键。依次从 $y_p \leq y_q$ 的点开始检查，直到 $x_q - x_p < y_q - y_p$。活动点的 $x-y$ 随 y 递减而严格递增，因此首次失败后可停止。第一次满足条件的后继具有最小 $x+y$，其距离正是 $(x_q+y_q)-(x_p+y_p)$。删掉已找到邻居的活动点，每点每轮最多删除一次，故每轮至多 $n-1$ 条候选边。四轮坐标分别为 $(x,y),(y,x),(-y,x),(x,-y)$，覆盖右侧四个闭锥。任意无向边都能从左端点指向其中一个锥；竖直边任选一个方向。总时间 $O(n \log n)$，额外空间 $O(n)$。

重复坐标与闭边界 重复坐标是不同顶点。排序平局按原编号，连续代表之间生成零权链；同一 $x+y$ 的其他位置不能以非零距离删掉该代表，最后一个代表继续寻找非重合邻居。可先在证明中收缩这些零链。若一个割拆开重合点类，零链已有最小的过割边；否则割可直接下降到收缩图。闭锥内只能使用 $d(r,q) \leq \max(d(p,r), d(p,q))$，不能写严格小于：$p=(0,0), q=(2,0), r=(1,1)$ 的三条距离都是 2。

平局也成立的割证明 在非重合点图的任意割中，选距离最小为 δ 的过割点对 p,q，并在平局中令 p_x+q_x 最大；令 p 为左端点。选择包含 q 的右向锥，若边恰在对角线上，选靠近水平轴的浅锥。该锥的最近邻 r 满足 $d(p,r) \leq \delta$。若 r 和 q 在割的同侧，则 pr 已是候选最小过割边。否则 rq 过割且 $d(r,q) \leq \delta$：严格小于违反最小性，等于也会违反上述平局选择。

后一个结论可在局部锥坐标 $X \geq Y \geq 0$ 中检查。满足 $d(p,r) \leq \delta$ 的点位于顶点 $(0,0),(\delta,0),(\delta/2,\delta/2)$ 的三角形。当 q 严格位于锥内时，距离达到 δ 只能取已排除的原点；q 在主轴时，非零等号点在 $X=Y$ 的射线上；q 在对角线时，非零等号点只能是 $(\delta,0)$。后两种情况在所选右向锥中都使 $r_x > p_x$（对角线选浅锥正是为了保证这一点），于是过割点对 r,q 的横坐标和更大，矛盾。所以每个割都保留最小边；在候选图上应用 Prim 的割性质，得到的树也是完整图的 MST。

整数范围与抄写合同 输入为 signed 64 位整数对；先整体转换为 signed 128 位，再进行和、差、变号及距离求和。单边最大距离为 $2^{65}-2$；在通常 32 位 int 编号上限内，总权远小于 signed 128 位上限。返回原始 0 基编号和 128 位总权；空集与单点返回 0 和空边集。正式题限制 $1 \leq n \leq 200000$、$0 \leq x,y \leq 10^9$，仅该驱动可安全转回 long long 输出。并查集只在本调用点按大小选择根的合并顺序，不改普通 dsu 的接口语义。

参考：Zhou–Shenoy–Nicholls (2002)，DOI `10.1016/S0020-0190(01)00232-0`；四轮 map 实现参考 KACTL `ManhattanMST.h`（CC0，固定版本与字节哈希见来源记录）。原论文半开锥的严格不等式不能直接代入这里的闭锥代码。

45.2 最小树形图

45.2.1 有向最小树形图

solve(n,root,edges)，点号 0..n-1，n≥1、0≤root<n；返回以 root 为根向外可达的最小树形图费用，无解 nullopt。允许负权、重边和自环（自环不选），n=1 的费用为 0；费用 -1 与无解可区分。输入、缩环后边权及累计费用绝对值须小于 2^120，类型为 int128。每个非根点选最小入边；若成环则缩点，跨环入边减去原终点已选入边费用，以计算替换代价。O(VE)，edges 按值传入，原图保持不变。仅返回费用，不恢复边方案。双版本已做所有根的入边枚举与 800 点连续缩环核验，暂无专门 OJ AC。

```
struct Arborescence
{
    using I = __int128_t;

    struct Edge
    {
        int u, v;
        I w;
    };

    // Directed minimum rooted OUT-arborescence cost, 0-based, or nullopt.
    static optional<I> solve(int n, int root, vector<Edge> e)
    {
        const I inf = I(1) << 120;
        I ans = 0;
        while (true)
        {
            vector<I> in(n, inf);
            vector<int> pre(n, -1), id(n, -1), vis(n, -1);
            for (auto a : e)
                if (a.u != a.v && a.w < in[a.v])
                {
                    in[a.v] = a.w;
                    pre[a.v] = a.u;
                }
            in[root] = 0;
            for (I x : in)
            {
                if (x == inf) return nullopt;
                ans += x;
            }
```

判环与缩点（接上页同一函数）：

```
            int cnt = 0;
            for (int i = 0; i < n; i++)
            {
                int u = i;
                while (vis[u] != i && id[u] == -1 && u != root)
                {
                    vis[u] = i;
                    u = pre[u];
                }
                if (u != root && id[u] == -1)
                {
                    id[u] = cnt;
                    for (int v = pre[u]; v != u; v = pre[v]) id[v] = cnt;
                    ++cnt;
                }
            }
            if (!cnt) return ans;
            for (int i = 0; i < n; i++)
                if (id[i] == -1) id[i] = cnt++;
            for (auto &a : e)
            {
                int v = a.v;
                a.u = id[a.u];
                a.v = id[a.v];
                if (a.u != a.v) a.w -= in[v];
            }
            root = id[root];
            n = cnt;
        }
    }
};
```

模板题使用：Luogu P4716

题意：给定根的有向最小树形图费用，无解输出-1。n 至 100、m 至 10^4、正权至 10^6；顶点和根从 1-based 转 0-based。optional 有值时才解引用，本题总费用可转 long long；核心只返回费用，不恢复选边。

所需模板：Arborescence（第 785 页，45.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4716

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, r;
    cin >> n >> m >> r;
    vector<Arborescence::Edge> e;
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        e.push_back({u - 1, v - 1, w});
    }
    auto ans = Arborescence::solve(n, r - 1, e);
    // This problem's answer fits in long long.
    cout << (ans ? (long long)*ans : -1) << '\n';
}
```

第四十六章　斯坦纳树

46.1　最小斯坦纳树与原边方案

SteinerTree(n)，点号 0..n-1；add(u,v,w) 添加无向边并返回 0-based 原边编号，允许重边、自环、零权。n≥1，w≥0，所有边权总和严格小于 LLONG_MAX/4。solve(terminals) 自动去重，返回 {cost,edges}，edges 为不重复、升序原边编号；空终端返回 0 及空方案，无法连通返回-1 及空方案。输入图不修改，可重复求解；恢复最优连通子图，不额外保证零权环树化，递归回溯。

令 k 为去重终端数。dp[mask][v] 先枚举互补非空子集在 v 合并，再多源 Dijkstra 移动根。时间 O(3^k n+2^k(m+n)log n)，空间 O(2^k n+n+m)。位掩码约束 k≤20，不代表此规模必然能在时限内完成；P6192 为 n≤100、m≤500、k≤10。负权不适用。当前已通过独立边子集枚举和方案校验，P6192 已 100 分通过，排名未核验。

正确性要点　非负权最优连通子图可删环成树。以 v 为根，分叉处将关键点分成两个非空集合，在同根合并；单分支路径由最短路扩展得到。真子集先处理，逐 mask 闭包即可。回溯合并严格缩小 mask，同 mask 前驱遵循 Dijkstra 的确定次序；严格改善避免零权前驱环。原边去重后的连通方案费用不大于 dp，最优性又排除严格更小，故边权和恰等于 cost。

图存储与加边：

```
struct SteinerTree
{
    using ll = long long;
    static constexpr ll inf = LLONG_MAX / 4;
    struct Edge
    {
        int u, v;
        ll w;
    };
    struct Result
    {
        ll cost;
        vector<int> edges;
    };
    int n;
    ll total = 0;
    vector<Edge> edges;
    vector<vector<pair<int, int>>> g;

    explicit SteinerTree(int n) : n(n), g(n)
    {
        assert(n >= 1);
    }
    int add(int u, int v, ll w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        assert(0 <= w && w < inf - total);
        int id = edges.size();
        edges.push_back({u, v, w});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        total += w;
        return id;
    }
```

状态初始化与同根合并（接上页结构体）：

```
    Result solve(vector<int> terminals) const
    {
        sort(terminals.begin(), terminals.end());
        terminals.erase(unique(terminals.begin(), terminals.end()), terminals.end());
        for (int v : terminals) assert(0 <= v && v < n);
        int k = terminals.size();
        assert(k <= 20);
        if (!k) return {0, {}};
        int count = 1 << k;
        vector<vector<ll>> dp(count, vector<ll>(n, inf));
        vector<vector<int>> split(count, vector<int>(n));
        vector<vector<int>> last(count, vector<int>(n, -1));
        for (int i = 0; i < k; i++) dp[1 << i][terminals[i]] = 0;
        for (int mask = 1; mask < count; mask++)
        {
            for (int sub = (mask - 1) & mask; sub; sub = (sub - 1) & mask)
            {
                int other = mask ^ sub;
                if (sub > other) continue;
                for (int v = 0; v < n; v++)
                {
                    ll value = dp[sub][v] + dp[other][v];
                    if (value < dp[mask][v])
                    {
                        dp[mask][v] = value;
                        split[mask][v] = sub;
                    }
                }
            }
```

多源最短路（接上页 mask 循环）：

```
            priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
            for (int v = 0; v < n; v++)
                if (dp[mask][v] < inf) pq.push({dp[mask][v], v});
            while (!pq.empty())
            {
                auto [distance, u] = pq.top();
                pq.pop();
                if (distance != dp[mask][u]) continue;
                for (auto [v, id] : g[u])
                {
                    ll value = distance + edges[id].w;
                    if (value < dp[mask][v])
                    {
                        dp[mask][v] = value;
                        split[mask][v] = 0;
                        last[mask][v] = id;
                        pq.push({value, v});
                    }
                }
            }
        }
```

最优根与递归方案恢复（接上页 solve 函数）：

```
        int root = min_element(dp.back().begin(), dp.back().end()) - dp.back().begin();
        if (dp.back()[root] == inf) return {-1, {}};
        vector<vector<char>> seen(count, vector<char>(n));
        vector<char> used(edges.size());
        auto trace = [&](auto &&self, int mask, int u) -> void
        {
            if (seen[mask][u]) return;
            seen[mask][u] = 1;
            int sub = split[mask][u];
            if (sub)
            {
                self(self, sub, u);
                self(self, mask ^ sub, u);
            }
            else if (last[mask][u] != -1)
            {
                int id = last[mask][u];
                used[id] = 1;
                self(self, mask, edges[id].u ^ edges[id].v ^ u);
            }
        };
        trace(trace, count - 1, root);
        Result result{dp.back()[root], {}};
        for (int id = 0; id < (int)edges.size(); id++)
            if (used[id]) result.edges.push_back(id);
        return result;
    }
};
```

模板题使用：P6192 最小斯坦纳树

题意：输入无向正权图及关键点，输出连接所有关键点的最小边权和。方案边编号可直接读取 solve 返回值的 edges。

所需模板：SteinerTree（第 788 页，46.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6192

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k;
    cin >> n >> m >> k;
    SteinerTree tree(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        tree.add(u - 1, v - 1, w);
    }
    vector<int> terminals(k);
    for (int &v : terminals)
    {
        cin >> v;
        v--;
    }
    cout << tree.solve(terminals).cost << '\n';
}
```

第四十七章　连通性相关

47.1　强连通分量

47.1.1　Tarjan 强连通分量与缩点

TarjanSCC(n)，点号 1..n；add(u,v) 加有向边，run() 从头计算。bel[u] 为 1..cnt 的强连通分量编号，跨分量边总从大编号指向小编号，可按 cnt..1 做拓扑 DP。dag() 返回缩点邻接表，保留重边；若 DP 需要对不同邻点去重，调用者应排序去重。支持自环、重边、不连通图和重复 run。时间及图存储 O(n+m)，递归 DFS 另需 O(n) 调用栈；本地验证应配置足够栈内存。注意已有 SCC 模板的编号方向与本模板相反。

```
struct TarjanSCC
{
    int n, timer = 0, cnt = 0;
    vector<vector<int>> g;
    vector<int> dfn, low, bel, st;

    TarjanSCC(int n) : n(n), g(n + 1) {}

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        g[u].push_back(v);
    }

    void dfs(int u)
    {
        dfn[u] = low[u] = ++timer;
        st.push_back(u);
        for (int v : g[u])
        {
            if (!dfn[v])
            {
                dfs(v);
                low[u] = min(low[u], low[v]);
            }
            else if (!bel[v])
                low[u] = min(low[u], dfn[v]);
        }
        if (low[u] == dfn[u])
        {
            ++cnt;
            while (true)
            {
                int v = st.back();
                st.pop_back();
                bel[v] = cnt;
                if (v == u) break;
            }
        }
    }

```

```
    // IDs 1..cnt are in reverse topological order. Recursive DFS needs O(n) stack.
    void run()
    {
        timer = cnt = 0;
        dfn.assign(n + 1, 0);
        low = dfn;
        bel = dfn;
        st.clear();
        for (int u = 1; u <= n; u++)
        {
            if (!dfn[u]) dfs(u);
        }
    }

    // Retains parallel edges. Use after run().
    vector<vector<int>> dag() const
    {
        vector<vector<int>> result(cnt + 1);
        for (int u = 1; u <= n; u++)
        {
            for (int v : g[u])
            {
                if (bel[u] != bel[v]) result[bel[u]].push_back(bel[v]);
            }
        }
        return result;
    }
};
```

模板题使用：Library Checker scc

题意：把有向图分成强连通分量，并按缩点图的拓扑顺序输出各分量。题目点号为 0..n-1，调用时加 1，输出时减 1。

所需模板：TarjanSCC（第 794 页，47.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/scc

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    TarjanSCC g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        g.add(u + 1, v + 1);
    }
    g.run();
    vector<vector<int>> c(g.cnt + 1);
    for (int u = 1; u <= n; u++) c[g.bel[u]].push_back(u - 1);
    cout << g.cnt << '\n';
    for (int i = g.cnt; i >= 1; i--)
    {
        cout << c[i].size();
        for (int u : c[i]) cout << ' ' << u;
        cout << '\n';
    }
}
```

应用题补充：Luogu P3275 / 糖果

题意：每人至少 1 颗，五类关系转成 x_v≥x_u+w，w 只取 0/1。强连通分量内出现权 1 边即无解 −1；其余按 Tarjan 分量编号从大到小递推最长路，每分量初值 1，按原顶点累加最小糖果总量。n,k≤100000，O(n+k)，总量用 long long；不能把每分量只计一次。

所需模板：TarjanSCC（第 794 页，47.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3275

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    TarjanSCC t(n);
    vector<array<int, 3>> e;
    auto add = [&](int u, int v, int w)
    {
        t.add(u, v);
        e.push_back({u, v, w});
    };
    while (m--)
    {
        int op, a, b;
        cin >> op >> a >> b;
        if (op == 1)
        {
            add(a, b, 0);
            add(b, a, 0);
        }
        else if (op == 2) add(a, b, 1);
        else if (op == 3) add(b, a, 0);
        else if (op == 4) add(b, a, 1);
        else add(a, b, 0);
    }
```

继续处理操作（接上页同一 main）：

```
    t.run();
    vector<vector<pair<int, int>>> g(t.cnt + 1);
    for (auto [u, v, w] : e)
    {
        int a = t.bel[u], b = t.bel[v];
        if (a == b)
        {
            if (w)
            {
                cout << -1 << '\n';
                return 0;
            }
        }
        else g[a].push_back({b, w});
    }
    vector<int> dis(t.cnt + 1, 1);
    for (int u = t.cnt; u >= 1; u--)
        for (auto [v, w] : g[u])
            dis[v] = max(dis[v], dis[u] + w);
    long long sum = 0;
    for (int u = 1; u <= n; u++) sum += dis[t.bel[u]];
    cout << sum << '\n';
    return 0;
}
```

相关应用：函数图包含尾链与环分解，导航无独立页；需正文定位，不应等同一般 SCC 算法。

47.1.2 函数图分解、跳转与到达

FunctionalGraph(to) 或 build(to) 重建，每个点恰有一条出边，to[u] 为后继，点号 0..n-1；空图允许，但不能查询。advance(u,k) 返回第 k 步到达的点，k 为完整 unsigned long long；steps(u,v) 返回首次到达 v 的最少步数，无法到达返回 -1，自身为 0。component[u] 为所在环的编号；cycles[id] 按后继方向保存环；entry[u] 为首次入环点；depth[u] 为入环步数；position 仅在环点上有效，非环点为 -1。order 为入度剥离顺序，逆序可用于从后继向入环树传播状态；up 是倍增表。所有字段只读。O(n log n) 预处理及空间，入环前查询 O(log n)，入环后取模 O(1)，不为 64 位步数分配 64 层表。默认构造为空图，支持自环、多个分量及重建。n≤INT_MAX，实际规模须满足内存。

```
struct FunctionalGraph
{
    using U = unsigned long long;
    int n;
    vector<int> component, position, entry, depth, order;
    vector<vector<int>> cycles, up;

    explicit FunctionalGraph(vector<int> to = {}) { build(move(to)); }

    void build(vector<int> to)
    {
        assert(to.size() <= INT_MAX);
        n = to.size();
        vector<int> degree(n);
        for (int v : to)
        {
            assert(0 <= v && v < n);
            degree[v]++;
        }
        order.clear();
        for (int u = 0; u < n; u++)
            if (degree[u] == 0) order.push_back(u);
        for (int i = 0; i < int(order.size()); i++)
        {
            int v = to[order[i]];
            if (--degree[v] == 0) order.push_back(v);
        }
```

环、入环树与倍增表（接上页同一结构体与函数）：

```
        component.assign(n, -1);
        position.assign(n, -1);
        entry.resize(n);
        depth.assign(n, 0);
        cycles.clear();
        for (int u = 0; u < n; u++)
            if (degree[u] && component[u] == -1)
            {
                vector<int> cycle;
                int v = u, id = cycles.size();
                do
                {
                    component[v] = id;
                    position[v] = cycle.size();
                    entry[v] = v;
                    cycle.push_back(v);
                    v = to[v];
                } while (v != u);
                cycles.push_back(move(cycle));
            }
        for (int i = int(order.size()) - 1; i >= 0; i--)
        {
            int u = order[i], v = to[u];
            component[u] = component[v];
            entry[u] = entry[v];
            depth[u] = depth[v] + 1;
        }
        up.assign(1, to);
        for (int k = 1; (1ULL << k) <= U(n); k++)
        {
            vector<int> row(n);
            for (int u = 0; u < n; u++) row[u] = up[k - 1][up[k - 1][u]];
            up.push_back(move(row));
        }
    }
```

跳转与最少步数接口（接上页同一结构体）：

```
    int advance(int u, U k) const
    {
        assert(0 <= u && u < n);
        if (k >= U(depth[u]))
        {
            k -= depth[u];
            u = entry[u];
            const auto &cycle = cycles[component[u]];
            return cycle[(U(position[u]) + k % cycle.size()) % cycle.size()];
        }
        for (int bit = 0; k; bit++, k >>= 1)
            if (k & 1) u = up[bit][u];
        return u;
    }

    int steps(int u, int v) const
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        if (component[u] != component[v] || depth[u] < depth[v]) return -1;
        if (depth[v] > 0)
        {
            int d = depth[u] - depth[v];
            return advance(u, d) == v ? d : -1;
        }
        int offset = position[v] - position[entry[u]];
        if (offset < 0) offset += int(cycles[component[u]].size());
        return depth[u] + offset;
    }
};
```

模板题使用：CSES 1750 Planets Queries I

题意：给每点唯一后继，查询从 x 走 k 步后的点，n,q≤200000，0≤k≤10^9。题目 1-based 点号读入减 1、输出加 1；k 用 unsigned long long，模板也支持完整 64 位步数。advance 先处理入环链再对环长取模，不能只建 30 层就宣称支持任意 64 位。预处理 O(n log n)，查询 O(log n)，入环后 O(1)。

所需模板：FunctionalGraph（第 799 页，47.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://cses.fi/problemset/task/1750

```
int main()
{
    int n, q;
    scanf("%d%d", &n, &q);
    vector<int> to(n);
    for (auto &v : to)
    {
        scanf("%d", &v);
        v--;
    }
    FunctionalGraph graph(to);
    while (q--)
    {
        int u;
        unsigned long long k;
        scanf("%d%llu", &u, &k);
        printf("%d\n", graph.advance(u - 1, k) + 1);
    }
}
```

模板题使用：CSES 1160 Planets Queries II

题意：同样每点一个后继，查询 u 到 v 的最少步数，不可达输出-1；n,q≤200000。1-based 读入转 0-based，steps 返回步数，不能加 1；同一点答案 0。处于同一弱连通分量也可能不可达，例如不同入环分支或从环走回链。预处理 O(n log n)，单次 O(log n)。

所需模板：FunctionalGraph（第 799 页，47.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://cses.fi/problemset/task/1160

```
int main()
{
    int n, q;
    scanf("%d%d", &n, &q);
    vector<int> to(n);
    for (auto &v : to)
    {
        scanf("%d", &v);
        v--;
    }
    FunctionalGraph graph(to);
    while (q--)
    {
        int u, v;
        scanf("%d%d", &u, &v);
        printf("%d\n", graph.steps(u - 1, v - 1));
    }
}
```

应用题补充：Luogu P2921

题意：每点一个后继，输出从各点出发直到首次重复前访问的不同点数，包含起点，n≤100000。结果为入环链长 depth[u] 加所在环的长度；component 是内部 0-based 环编号，不能当成环顶点。自环答案 1。这份用法只读取分解字段，不调用 advance/steps。当前模板含倍增表，建图 O(n log n)，输出 O(n)。

所需模板：FunctionalGraph（第 799 页，47.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2921

```
int main()
{
    int n;
    scanf("%d", &n);
    vector<int> to(n);
    for (auto &v : to)
    {
        scanf("%d", &v);
        v--;
    }
    FunctionalGraph graph(to);
    for (int u = 0; u < n; u++)
        printf("%d\n", graph.depth[u] + int(graph.cycles[graph.component[u]].size()));
}
```

47.1.3 强连通分量

add(u,v)、run()，bel[1..n] 为分量编号，cnt 为数量；分量编号按缩点图拓扑序递增。递归 Kosaraju，按用户要求保留 DFS 写法；O(V+E)，同时存正反图。

```
struct SCC
{
    int n, cnt = 0;
    vector<vector<int>> g, rg;
    vector<int> bel;

    SCC(int n) : n(n), g(n + 1), rg(n + 1) {}

    void add(int u, int v)
    {
        g[u].push_back(v);
        rg[v].push_back(u);
    }

    void dfs1(int u, vector<int> &vis, vector<int> &order)
    {
        vis[u] = 1;
        for (int v : g[u])
        {
            if (!vis[v]) dfs1(v, vis, order);
        }
        order.push_back(u);
    }

    void dfs2(int u)
    {
        bel[u] = cnt;
        for (int v : rg[u])
        {
            if (!bel[v]) dfs2(v);
        }
    }
```

重算入口与拓扑编号（接上页同一结构体）：

```
    // Recursive Kosaraju; component IDs are in topological order, 1..cnt.
    void run()
    {
        vector<int> vis(n + 1), order;
        cnt = 0;
        bel.assign(n + 1, 0);
        for (int s = 1; s <= n; s++)
        {
            if (!vis[s]) dfs1(s, vis, order);
        }
        reverse(order.begin(), order.end());
        for (int s : order)
        {
            if (!bel[s])
            {
                ++cnt;
                dfs2(s);
            }
        }
    }
};
```

模板题使用：Library Checker scc (Kosaraju)

题意：给有向图，输出强连通分量并按缩点拓扑序排列；N、M≤500000，可有重边和自环。题面零基点号先加 1；SCC.bel 为 1..cnt 且沿跨分量边递增，按 1..cnt 输出并将点号减 1。不要照搬 TarjanSCC 的逆序循环。时间、空间 O(N+M)，DFS 递归。

所需模板：SCC（第 804 页，47.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/scc

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    SCC g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        g.add(u + 1, v + 1);
    }
    g.run();
    vector<vector<int>> c(g.cnt + 1);
    for (int u = 1; u <= n; u++) c[g.bel[u]].push_back(u - 1);
    cout << g.cnt << '\n';
    for (int i = 1; i <= g.cnt; i++)
    {
        cout << c[i].size();
        for (int u : c[i]) cout << ' ' << u;
        cout << '\n';
    }
}
```

47.2 双连通分量

47.2.1 点双与边双：共享 lowlink 核心

BiconnectedCore(n)，点号 1..n；add(u,v) 返回原边的 0-based 编号，run() 从头计算，允许重复运行。

点双结果：blocks 为点双的顶点列表，桥两端成块，孤立点单独成块，割点可出现在多个块；cut[u] 标记割点。

边双结果：bridge[id] 标记桥，bel[u] 是 1..cnt 的边双编号；cnt 是全图边双个数。重边按边号处理；自环保留边号但不进 DFS，不是桥，不额外产生点双。一次共享 DFS 后递归染色，O(n+m) 时间和空间，DFS 保持递归。n=0 合法。run 后才可读取结果或构造森林；g、edges 为原图，不在 DFS 期间修改。

```
struct BiconnectedCore
{
    int n, timer = 0, cnt = 0;
    vector<vector<pair<int, int>>> g;
    vector<pair<int, int>> edges;
    vector<vector<int>> blocks;
    vector<int> dfn, low, cut, bridge, bel, st;

    BiconnectedCore(int n) : n(n), g(n + 1) {}

    int add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        int id = edges.size();
        edges.push_back({u, v});
        if (u != v)
        {
            g[u].push_back({v, id});
            g[v].push_back({u, id});
        }
        return id;
    }
```

```
    void dfs(int u, int parent_edge)
    {
        dfn[u] = low[u] = ++timer;
        st.push_back(u);
        int children = 0;
        for (auto [v, id] : g[u])
        {
            if (id == parent_edge) continue;
            if (!dfn[v])
            {
                children++;
                dfs(v, id);
                low[u] = min(low[u], low[v]);
                if (low[v] > dfn[u]) bridge[id] = 1;
                if (low[v] >= dfn[u])
                {
                    if (parent_edge != -1 || children > 1) cut[u] = 1;
                    blocks.push_back({u});
                    while (true)
                    {
                        int x = st.back();
                        st.pop_back();
                        blocks.back().push_back(x);
                        if (x == v) break;
                    }
                }
            }
            else
                low[u] = min(low[u], dfn[v]);
        }
        if (parent_edge == -1 && children == 0) blocks.push_back({u});
    }
```

```
    void paint(int u)
    {
        bel[u] = cnt;
        for (auto [v, id] : g[u])
        {
            if (!bridge[id] && !bel[v]) paint(v);
        }
    }
```

```
    void run()
    {
        timer = cnt = 0;
        dfn.assign(n + 1, 0);
        low = dfn;
        cut = dfn;
        bel = dfn;
        bridge.assign(edges.size(), 0);
        blocks.clear();
        st.clear();
        for (int u = 1; u <= n; u++)
        {
            if (!dfn[u])
            {
                dfs(u, -1);
                st.pop_back();
            }
        }
        for (int u = 1; u <= n; u++)
        {
            if (bel[u]) continue;
            ++cnt;
            paint(u);
        }
    }
};
```

模板题使用：Luogu P8435

题意：给定无向图，输出所有点双连通分量的顶点集合；割点可出现在多个集合中，孤立点单独成块。点号 1..n。

所需模板：BiconnectedCore（第 807 页，47.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P8435

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    BiconnectedCore graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        std::cin >> u >> v;
        graph.add(u, v);
    }
    graph.run();
    const auto &blocks = graph.blocks;
    std::cout << blocks.size() << '\n';
    for (const auto &block : blocks)
    {
        std::cout << block.size();
        for (int u : block) std::cout << ' ' << u;
        std::cout << '\n';
    }
}
```

模板题使用：Luogu P8436

题意：给定无向图，输出所有边双连通分量的顶点集合；利用 bel 按边双编号分组。点号 1..n。

所需模板：BiconnectedCore（第 807 页，47.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P8436

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    BiconnectedCore graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        std::cin >> u >> v;
        graph.add(u, v);
    }
    graph.run();
    std::vector<std::vector<int>> blocks(graph.cnt);
    for (int u = 1; u <= n; u++) blocks[graph.bel[u] - 1].push_back(u);
    std::cout << blocks.size() << '\n';
    for (const auto &block : blocks)
    {
        std::cout << block.size();
        for (int u : block) std::cout << ' ' << u;
        std::cout << '\n';
    }
}
```

47.2.2 边双缩点森林（保留桥编号）

代码依赖：BiconnectedCore（第 807 页，47.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

bridge_component_forest(graph) 接收已 run 的 BiconnectedCore，返回按 1..cnt 编号的边双森林，邻接项为 (相邻分量, 原桥编号)。每条桥变成一条森林边，重边和自环不会误作桥。0 号空置，不连通输入可产生多棵树，孤立边双保留空邻接表。O(n+m) 时间、O(n) 输出空间，不修改分解结果。

```
vector<vector<pair<int, int>>> bridge_component_forest(const BiconnectedCore &graph)
{
    int cnt = graph.cnt;
    const auto &edges = graph.edges;
    const auto &bridge = graph.bridge;
    const auto &bel = graph.bel;
    vector<vector<pair<int, int>>> tree(cnt + 1);
    for (int id = 0; id < (int)edges.size(); id++)
    {
```

```
        if (!bridge[id]) continue;
        auto [u, v] = edges[id];
        u = bel[u];
        v = bel[v];
        tree[u].push_back({v, id});
        tree[v].push_back({u, id});
    }
    return tree;
}
```

应用题补充：Luogu P2860 (bridge forest)

题意：连通无向图，n≤5000、m≤10000；允许已有重边及新增重边，求最少新增边使任意两点间有两条边不相交路径。run 后边双缩点，编号 1..cnt，邻接项为 (边双号, 原桥号)。度 1 点数为 L，答案 (L+1)/2；只剩一个边双时为 0。该公式要求原图连通，不可对任意森林直接套用。O(n+m) 时间、空间。

所需模板：BiconnectedCore（第 807 页，47.2.1 节）、bridge_component_forest（第 811 页，47.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2860

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    BiconnectedCore graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    graph.run();
    auto tree = bridge_component_forest(graph);
    int leaves = 0;
    for (int u = 1; u <= graph.cnt; u++)
        if (tree[u].size() == 1) leaves++;
    cout << (leaves + 1) / 2 << '\n';
    return 0;
}
```

47.2.3 属于简单奇环的全部顶点

代码依赖：BiconnectedCore（第 807 页，47.2.1 节）、block_cut_forest（第 832 页，47.4.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

先对 BiconnectedCore 调用 run，再调用 odd_cycle_vertices(graph)，返回 answer[1..n]，值为 1 表示该点属于某个长度至少为 3 的简单奇环，answer[0]=0。允许重边、断开图及空图；自环不计入这里的奇环。某点双非二分当且仅当其中存在奇环，此时该点双每个顶点都属于某个奇环；答案为所有非二分点双的顶点并集。通过圆方森林把每条非自环边分到唯一点双：两端中森林深度较大者的父方点即该块，同深时二者父方点相同。随后只对块内边染色，避免在每个块中重复扫描割点的全部邻边。两次 DFS 均递归，分解后额外 O(n+m) 时间和空间。可重排 blocks，不能改变分解内容。六点以内所有简单图枚举奇环对拍，另有重边/自环、长链、星形和大奇环测试；POJ2942 补图驱动本地验证，在线待补。依赖 biconnected.hpp。

```
// After run(). Simple odd cycles have length >= 3; self-loops are ignored.
vector<int> odd_cycle_vertices(const BiconnectedCore &graph)
{
    int n = graph.n;
    auto forest = block_cut_forest(graph);
    vector<int> parent(forest.size()), depth(forest.size());
    auto root_tree = [&](auto &&self, int u, int p) -> void
    {
        parent[u] = p;
        for (int v : forest[u])
            if (v != p)
            {
                depth[v] = depth[u] + 1;
                self(self, v, u);
            }
    };
    for (int u = 1; u <= n; u++)
        if (!depth[u])
        {
            depth[u] = 1;
            root_tree(root_tree, u, 0);
        }
```

```
    vector<vector<pair<int, int>>> edges(graph.blocks.size());
    for (auto [u, v] : graph.edges)
        if (u != v)
        {
            int block = parent[depth[u] >= depth[v] ? u : v] - n - 1;
            assert(0 <= block && block < (int)edges.size());
            edges[block].push_back({u, v});
        }
    vector<vector<int>> g(n + 1);
    vector<int> color(n + 1), answer(n + 1);
    for (int block = 0; block < (int)edges.size(); block++)
    {
        for (int u : graph.blocks[block])
        {
            g[u].clear();
            color[u] = 0;
        }
        for (auto [u, v] : edges[block])
        {
            g[u].push_back(v);
            g[v].push_back(u);
        }
        bool bipartite = true;
        auto paint = [&](auto &&self, int u) -> void
        {
            for (int v : g[u])
                if (!color[v])
                {
                    color[v] = -color[u];
                    self(self, v);
                }
                else if (color[v] == color[u])
                    bipartite = false;
        };
        int root = graph.blocks[block][0];
        color[root] = 1;
        paint(paint, root);
        if (!bipartite)
            for (int u : graph.blocks[block]) answer[u] = 1;
    }
    return answer;
}
```

应用题补充：POJ 2942 Knights of the Round Table

题意：互相憎恨的骑士不能相邻，合法圆桌至少 3 人且人数为奇数；求无法参加任何合法圆桌的人数。输入憎恨关系，顶点 1..n；对 u<v 且不憎恨的二人建补图边，不能把憎恨边直接加入。每组新建矩阵和 BiconnectedCore，先 run 再调用 odd_cycle_vertices，统计 odd[1..n] 中的 0，每组输出一行；0 0 终止且不输出。函数只返回各点能否出现在某个简单奇环，不给圆桌座次；无合法圆桌时输出 n，不是失败。POJ 原站本次不可达；输入格式和样例另与同题 UVA1364 官方 PDF 核对，未据此声称 POJ 限制或线上 AC 已复核。

所需模板：BiconnectedCore（第 807 页，47.2.1 节）、block_cut_forest（第 832 页，47.4.2 节）、odd_cycle_vertices（第 813 页，47.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://poj.org/problem?id=2942

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    while (cin >> n >> m && (n || m))
    {
        vector<vector<bool>> hate(n + 1, vector<bool>(n + 1));
        while (m--)
        {
            int u, v;
            cin >> u >> v;
            hate[u][v] = true;
            hate[v][u] = true;
        }
        BiconnectedCore graph(n);
        for (int u = 1; u <= n; u++)
            for (int v = u + 1; v <= n; v++)
                if (!hate[u][v]) graph.add(u, v);
        graph.run();
        auto odd = odd_cycle_vertices(graph);
        int answer = 0;
        for (int u = 1; u <= n; u++) answer += !odd[u];
        cout << answer << '\n';
    }
    return 0;
}
```

47.2.4 边双缩点的大小与内部边数

代码依赖：BiconnectedCore（第 807 页，47.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

在 BiconnectedCore.run() 后构造 EdgeCompression(graph)。n 为边双数量；size[c] 为第 c 块的原顶点数，inside[c] 为块内逻辑无向边数，分量号 1..n，0 号槽忽略。edges 每项为 {块 u, 块 v, 原边 id}，只含跨块的桥，每条桥出现一次，按原边号递增；是无向森林，不承诺端点为父子方向。重边逐条计数，自环在所属块内部计一次，不能用邻接表度数直接计边而漏掉这些约定。空图返回 n=0、长度 1 的零计数数组和空边表。时间 O(N+M)，输出空间 O(N+M)，N/M 指原图规模。

```
struct EdgeCompression
{
    int n;
    vector<int> size, inside;
    vector<array<int, 3>> edges;

    EdgeCompression(const BiconnectedCore &graph)
        : n(graph.cnt), size(n + 1), inside(n + 1)
    {
        for (int u = 1; u <= graph.n; u++) size[graph.bel[u]]++;
        for (int id = 0; id < (int)graph.edges.size(); id++)
        {
            auto [u, v] = graph.edges[id];
            u = graph.bel[u];
            v = graph.bel[v];
            if (u == v)
                inside[u]++;
            else
                edges.push_back({u, v, id});
        }
    }
};
```

应用题补充：Codeforces 732F Tourist Reform

题意：连通无向简单图，2≤n≤400000、1≤m≤400000，给每条边定向，使各点可达点数的最小值最大（含自身）。输出最优值和按原输入顺序的 m 条有向边。先求边双缩点，选 size 最大的块为根；orient_edges 定向块内边，再按 edges 中的原边编号把桥全部朝根定向。答案是最大块大小，不把缩点边端点顺序当成父子关系。O(n+m) 时间/空间；保留递归核心，需足够大栈，局部 8 MiB 栈在最大链上溢出，评测栈未知。本题是应用，不是正式缩点模板题。

所需模板：BiconnectedCore（第 807 页，47.2.1 节）、EdgeCompression（第 816 页，47.2.4 节）、Lowlink（第 824 页，47.3.3 节）、orient_edges（第 819 页，47.2.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/732/F

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    if (!(std::cin >> n >> m)) return 0;
    BiconnectedCore core(n);
    Lowlink graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        std::cin >> u >> v;
        core.add(u, v);
        graph.add(u, v); // Same insertion order preserves logical edge IDs.
    }
    // Existing recursive cores require a sufficiently large process stack.
    // An 8 MiB stack fails at n=400000; judge stack availability is not assumed.
    core.run();
    EdgeCompression compressed(core);
    int root = 1;
    for (int c = 2; c <= compressed.n; c++)
        if (compressed.size[c] > compressed.size[root]) root = c;
    auto direction = orient_edges(graph); // Does not need graph.run().
    vector<vector<pair<int, int>>> tree(compressed.n + 1);
    for (auto [u, v, id] : compressed.edges)
    {
        tree[u].push_back({v, id});
        tree[v].push_back({u, id}); // Compressed endpoints are not parent/child.
    }
```

继续处理操作（接上页同一 main）：

```
    vector<int> parent(compressed.n + 1), order{root};
    parent[root] = -1;
    for (int i = 0; i < (int)order.size(); i++)
    {
        int u = order[i];
        for (auto [v, id] : tree[u])
        {
            if (parent[v]) continue;
            parent[v] = u;
            order.push_back(v);
            auto [a, b] = core.edges[id];
            if (core.bel[a] != v) swap(a, b);
            direction[id] = {a, b}; // Original bridge points child -> parent.
        }
    }
    std::cout << compressed.size[root] << '\n';
    for (auto [u, v] : direction) std::cout << u << ' ' << v << '\n';
    return 0;
}
```

47.2.5 无向边定向与强连通定向

代码依赖：Lowlink（第 824 页，47.3.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

orient_edges(graph) 接受 Lowlink 对象。返回长度等于逻辑边数的方向数组，第 id 项为该边定向后的 {起点, 终点}，顶点 1-based、边 id 为 0-based。每条无向边仅处理一次：DFS 树边向下，非树边指向祖先，自环保留 {u,u}，重边不合并。支持非连通图；每个边双块内定向后强连通，桥只给出某个方向，不保证跨桥互相可达。若要求整个非空图强连通，需另外 run/Run 检查 components==1 且无桥；不满足则不存在这样的方向。该函数只读取邻接表与边数，不依赖已有 DFS 状态，不修改原图，调用时无须先 run。复杂度 O(N+M) 时间、O(N+M) 额外空间，DFS 保持递归。

```
// Tree edges go down; other edges go to ancestors. Handles each logical edge once.
vector<pair<int, int>> orient_edges(const Lowlink &graph)
{
    vector<pair<int, int>> direction(graph.edges);
    vector<int> seen(graph.n + 1);
    auto dfs = [&](auto &&self, int u) -> void
    {
        seen[u] = 1;
        for (auto [v, id] : graph.g[u])
        {
            if (direction[id].first) continue;
            direction[id] = {u, v};
            if (!seen[v]) self(self, v);
        }
    };
    for (int u = 1; u <= graph.n; u++)
        if (!seen[u]) dfs(dfs, u);
    return direction;
}
```

应用题补充：Codeforces 118E Bertown roads

题意：给连通无向简单图，把每条边定向使整图强连通，无解输出 0。n≤100000、m≤300000。先用 Lowlink.run 确认连通且无桥，再调用 orient_edges；返回数组与 add 的原边编号逐项对应，每条逻辑边只加一次。任意可行方向均可。O(n+m) 时间、空间，DFS 递归；比赛应用不替代正式模板题。

所需模板：Lowlink（第 824 页，47.3.3 节）、orient_edges（第 819 页，47.2.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/118/E

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    Lowlink graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        std::cin >> u >> v;
        graph.add(u, v);
    }
    graph.run();
    if (graph.components != 1 ||
        std::count(graph.bridge.begin(), graph.bridge.end(), 1))
    {
        std::cout << 0 << '\n';
        return 0;
    }
    for (auto [u, v] : orient_edges(graph)) std::cout << u << ' ' << v << '\n';
    return 0;
}
```

47.3 割点和桥

47.3.1 由点双分解计算删点连通块数

代码依赖：BiconnectedCore（第 807 页，47.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

在 BiconnectedCore.run() 后调用 removal_components(graph)，返回 (原连通块数 before, 删点结果 after)。after[u] 为独立删除原顶点 u 后剩余连通块数，点号 1..n；after[0]=0 不参与计算。before 等于边双数量减去桥数量；after[u] 等于 before-1 加包含 u 的非单点点双数量。单点块必须跳过，因此孤立点删去后会少一个连通块。空图返回 (0,{0})。不修改原分解，O(n+m) 时间、O(n) 额外空间。若只需要割点、桥和删点增量，直接用更短的 Lowlink/Low_Link；已经需要点双分解时可用本函数，避免再次 DFS。

```
// Call after run(). Each vertex is deleted independently from the original graph.
pair<int, vector<int>> removal_components(const BiconnectedCore &graph)
```

```
{
    int before = graph.cnt - count(graph.bridge.begin(), graph.bridge.end(), 1);
    vector<int> after(graph.n + 1, before - 1);
    after[0] = 0;
    for (const auto &block : graph.blocks)
        if (block.size() > 1)
            for (int u : block) after[u]++;
    return {before, after};
}
```

模板题使用：AOJ GRL_3_A

题意：连通无向简单图，按编号升序输出所有割点，N、M≤100000。输入零基转一基；BiconnectedCore.run 后取 (before,after)，仅当 after[u]>before 才输出 u-1。本例展示已有点双结果的复用；只求割点优先用更短的 Lowlink。时间、空间 O(N+M)，递归 DFS。该题只核验割点集合，完整删点数量另见 UVA10765 应用。

所需模板：BiconnectedCore（第 807 页，47.2.1 节）、removal_components（第 820 页，47.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=GRL_3_A&lang=en

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    BiconnectedCore g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        g.add(u + 1, v + 1);
    }
    g.run();
    auto [before, after] = removal_components(g);
    for (int u = 1; u <= n; u++)
        if (after[u] > before) cout << u - 1 << '\n';
}
```

应用题补充：UVA 10765 Doves and Bombs

题意：分别删去每个站点，按剩余连通块数降序、原编号升序输出前 m 名；n≤10000，原图连通。m 是输出名额而非边数，边表以-1 -1 结束，多测以 0 0 结束，每组末尾空行。after 是剩余总块数而非增量，不重复加 before。O(n log n+E) 时间、O(n+E) 空间。应用题不计正式模板覆盖。

所需模板：BiconnectedCore（第 807 页，47.2.1 节）、removal_components（第 820 页，47.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.org/external/107/10765.pdf

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    while (cin >> n >> m && n)
    {
        BiconnectedCore g(n);
        int u, v;
        while (cin >> u >> v && u != -1) g.add(u + 1, v + 1);
        g.run();
        auto [before, after] = removal_components(g);
        vector<pair<int, int>> a;
        for (int u = 1; u <= n; u++) a.push_back({-after[u], u - 1});
        sort(a.begin(), a.end());
        for (int i = 0; i < m; i++) cout << a[i].second << ' ' << -a[i].first << '\n';
        cout << '\n';
    }
}
```

47.3.2 连通无向图最少加边方案

代码依赖：BiconnectedCore（第 807 页，47.2.1 节）、bridge_component_forest（第 811 页，47.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

先对非空连通无向图调用 BiconnectedCore.run()，再调用 bridge_augmentation(graph)。返回原图点号构成的新增边列表，不修改原图。允许新增重边，不产生自环；不适用于非连通图。已无桥时返回空列表。桥树有 L 个叶子时最少加 (L+1)/2 条边（整数除法）。递归 DFS 顺序收集叶子，奇数时复制首叶，前后半段配对。每条桥一侧的叶子是循环连续区间，半周配对必跨过该割，故所有桥被消除；每个叶子至少需要一个新边端点，得到同样的下界。分解后本函数 O(n+m) 时间与空间。依赖 bridge_component_forest.hpp。P2860 的 vector 用法已通过普通及 ASan/UBSan 本地验证；新增方案以逐边删除连通性及小图最优解枚举独立核验，该应用尚无本库在线 AC。

```
// Connected, nonempty graph after run(); new parallel edges are allowed.
vector<pair<int, int>> bridge_augmentation(const BiconnectedCore &graph)
{
    assert(graph.n > 0 && graph.cnt > 0);
    int bridges = count(graph.bridge.begin(), graph.bridge.end(), 1);
    assert(graph.cnt - bridges == 1);
    if (graph.cnt == 1) return {};
    auto tree = bridge_component_forest(graph);
    vector<int> representative(graph.cnt + 1);
    for (int u = 1; u <= graph.n; u++) representative[graph.bel[u]] = u;
    vector<int> leaves;
    auto dfs = [&](auto &&self, int u, int parent) -> void
    {
        if (tree[u].size() == 1) leaves.push_back(u);
        for (auto [v, id] : tree[u])
            if (v != parent) self(self, v, u);
    };
    dfs(dfs, 1, 0);
    if (leaves.size() % 2) leaves.push_back(leaves[0]);
    int half = leaves.size() / 2;
    vector<pair<int, int>> added;
    for (int i = 0; i < half; i++)
        added.push_back({representative[leaves[i]], representative[leaves[i + half]]});
    return added;
}
```

应用题补充：Luogu P2860 (construct augmentation)

题意：同题的构造接口：连通非空图 run 后调用 bridge_augmentation，返回最少补边方案的原点端点对，点号 1..n，可直接逐对使用；题目只输出方案长度。已有边和新增边都允许平行边。接口不修改 graph，若需更新原图须自行 add 并重新 run。O(n+m) 时间、空间；DFS 顺序收集桥树叶子后对半配对，奇数叶子补首叶。

所需模板：BiconnectedCore（第 807 页，47.2.1 节）、bridge_component_forest（第 811 页，47.2.2 节）、bridge_augmentation（第 823 页，47.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2860

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    BiconnectedCore graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    graph.run();
    cout << bridge_augmentation(graph).size() << '\n';
    return 0;
}
```

47.3.3 割点与桥

无向多重图，点号 1..n；add/Insert(u,v) 返回逻辑无向边的 0-based 编号，自环和重边均可输入。run/Run 后 cut[u] 为割点标记，bridge[id] 为桥标记，components 为原图连通块数。delta[u] 是独立删除顶点 u 及其关联边后增加的连通块数，剩余块数为 components+delta[u]；孤立点 delta=-1，普通非割点为 0，割点为正。根的增量为 DFS 子树数减 1；非根统计满足 low[child]>=dfn[u] 的子树数。n=1 的点删除后剩余 0 块。DFS 按父边号跳过，支持重边、非连通图和重复运行。复杂度 O(n+m)，递归 DFS；新增边后需重新 run。每个删点结果都针对原图，不表示逐次删除。

```
struct Lowlink
{
    int n, timer = 0, components = 0;
    vector<vector<pair<int, int>>> g;
    vector<int> dfn, low, cut, bridge, delta;
    int edges = 0;

    Lowlink(int n) : n(n), g(n + 1) {}

    int add(int u, int v)
    {
        g[u].push_back({v, edges});
        g[v].push_back({u, edges});
        return edges++;
    }
```

```
    void dfs(int u, int pe)
    {
        dfn[u] = low[u] = ++timer;
        int children = 0;
        for (auto [v, id] : g[u])
        {
            if (id == pe) continue;
            if (!dfn[v])
            {
                ++children;
                dfs(v, id);
                low[u] = min(low[u], low[v]);
                if (low[v] > dfn[u]) bridge[id] = 1;
                if (pe != -1 && low[v] >= dfn[u])
                {
                    cut[u] = 1;
                    delta[u]++;
                }
            }
            else
                low[u] = min(low[u], dfn[v]);
        }
        if (pe == -1 && children > 1) cut[u] = 1;
        if (pe == -1) delta[u] = children - 1;
    }

    void run()
    {
        timer = 0;
        components = 0;
        dfn.assign(n + 1, 0);
        low = dfn;
        cut = dfn;
        delta = dfn;
        bridge.assign(edges, 0);
        for (int u = 1; u <= n; u++)
            if (!dfn[u])
            {
                components++;
                dfs(u, -1);
            }
    }
};
```

模板题使用：Luogu P3388

题意：给无向图，按编号递增输出全部割点。图可以不连通；run 后读取 cut[u] 标记。不要把 bridge 的边编号当作顶点标记。点号 1..n。

所需模板：Lowlink（第 824 页，47.3.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3388

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Lowlink graph(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    graph.run();
    vector<int> answer;
    for (int u = 1; u <= n; u++)
        if (graph.cut[u]) answer.push_back(u);
    cout << answer.size() << '\n';
    for (int u : answer) cout << u << ' ';
    cout << '\n';
}
```

47.4 圆方树

47.4.1 静态仙人掌森林最短路

Cactus(n)，0≤n<INT_MAX/2，原点 1..n。add(u,v,w) 加入无向边；输入须为无自环无重边的仙人掌森林（每条边最多属于一个简单环），不检查此结构前提。权非负，所有边权之和 ≤LLONG_MAX/4。build() 递归构造桥/环虚点森林并倍增；distance(u,v) 仅接受原点，断连返回-1，同点返回 0。查询前必须 build；加边后必须重建，支持重复 build 和复制，无动态修改或路径方案接口。编号 n+1..cnt 为环虚点；t 是从根向下的带权邻接表，成员只读。环的顶点到最上方割点距离取较短弧，虚点到该割点权 0；LCA 是虚点时，须用两个儿子在环上的较短弧替换根距离贡献。dfs 与 prepare 均递归，环上沿 par 回走仅枚举边，不模拟 DFS 栈。原图 DFS 及环枚举 O(n+m)，总预处理/空间 O((n+m)log(n+1))，查询 O(log(n+1))，递归深度 O(n)。n=0 允许 build 但无合法查询；不支持负权、非法仙人掌或修改内部数组。

```
struct Cactus
{
    using ll = long long;
    int n, m = 0, cnt, timer = 0, lg;
    vector<vector<tuple<int, int, ll>>> g;
    vector<vector<pair<int, ll>>> t;
    vector<vector<int>> up;
    vector<int> dfn, par, bel, dep, root;
    vector<ll> pref, pos, len, dis;

    Cactus(int n) : n(n), g(n + 1)
    {
        assert(0 <= n && n < INT_MAX / 2);
    }

    void add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(u != v && w >= 0);
        g[u].push_back({v, m, w});
        g[v].push_back({u, m, w});
        m++;
    }
```

递归构造环虚点与根距离（接上页同一结构体）：

```
    void dfs(int u, int pe)
    {
        dfn[u] = ++timer;
        for (auto [v, id, w] : g[u])
        {
            if (id == pe) continue;
            if (!dfn[v])
            {
                par[v] = u;
                pref[v] = pref[u] + w;
                dfs(v, id);
                if (!bel[v]) t[u].push_back({v, w});
            }
            else if (dfn[v] < dfn[u])
            {
                int b = ++cnt;
                len[b] = pref[u] - pref[v] + w;
                t[v].push_back({b, 0});
                for (int x = u; x != v; x = par[x])
                {
                    bel[x] = b;
                    pos[x] = pref[x] - pref[v];
                    t[b].push_back({x, min(pos[x], len[b] - pos[x])});
                }
            }
        }
    }

    void prepare(int u)
    {
        for (int k = 1; k < lg; k++)
            up[k][u] = up[k - 1][up[k - 1][u]];
        for (auto [v, w] : t[u])
        {
            up[0][v] = u;
            dep[v] = dep[u] + 1;
            root[v] = root[u];
            dis[v] = dis[u] + w;
            prepare(v);
        }
    }
```

从零重建（接上页同一结构体）：

```
    void build()
    {
        cnt = n;
        timer = 0;
        int size = 2 * n + 1;
        t.assign(size, {});
        dfn.assign(size, 0);
        par = bel = dep = root = dfn;
        pref.assign(size, 0);
        pos = len = dis = pref;
        for (int u = 1; u <= n; u++)
            if (!dfn[u]) dfs(u, -1);
        lg = 1;
        while ((1LL << lg) <= cnt) lg++;
        up.assign(lg, vector<int>(size));
        for (int u = 1; u <= n; u++)
        {
            if (dfn[u] && !par[u])
            {
                root[u] = u;
                prepare(u);
            }
        }
    }
```

查询与环上距离修正（接上页同一结构体）：

```
    // Original vertices only; disconnected pairs return -1.
    ll distance(int u, int v) const
    {
        if (root[u] != root[v]) return -1;
        ll ans = dis[u] + dis[v];
        if (dep[u] < dep[v]) swap(u, v);
        int delta = dep[u] - dep[v];
        for (int k = 0; k < lg; k++)
            if (delta >> k & 1) u = up[k][u];
        if (u == v) return ans - 2 * dis[u];
        for (int k = lg - 1; k >= 0; k--)
        {
            if (up[k][u] != up[k][v])
            {
                u = up[k][u];
                v = up[k][v];
            }
        }
        int p = up[0][u];
        if (p <= n) return ans - 2 * dis[p];
        ll arc = abs(pos[u] - pos[v]);
        return ans - dis[u] - dis[v] + min(arc, len[p] - arc);
    }
};
```

为什么需要环上修正 桥和环块构成森林，块间路径唯一。若最近公共祖先是原点，两条根距离减去公共前缀即可；若是环虚点，u、v 现为该虚点的两个原点儿子，先扣掉这两点到根的距离，再加环上两段弧长的较小值。不能直接把虚点当普通 LCA，否则会强迫路径绕经环顶割点。

复杂度与前提 每条边只属于至多一个环，因此沿 par 回走的总次数为 O(m)，不是每个返祖边各走 O(n)。两次 DFS 均为递归。输入须为非负权简单仙人掌森林，全部边权和不超过 LLONG_MAX/4；不自动校验结构，非法图可能越界。加边后必须重建。

模板题使用：P5236 【模板】静态仙人掌

题意：正权无重边仙人掌，查询两点最短距离；n,q≤10000、m≤20000、w≤100000。

所需模板：Cactus（第 827 页，47.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5236

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    Cactus solver(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        solver.add(u, v, w);
    }
    solver.build();
    while (q--)
    {
        int u, v;
        cin >> u >> v;
        cout << solver.distance(u, v) << '\n';
    }
}
```

47.4.2 由点双构造圆方森林

代码依赖：BiconnectedCore（第 807 页，47.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

block_cut_forest(graph) 接收已经 run 的 BiconnectedCore，返回 1-based 邻接表。原点为 1..n，blocks[i] 的方点为 n+i+1；每个点双内的所有原点连向该方点。孤立点也连向自己的单点方块。不连通原图得到森林；0 号空置。O(n+m) 时间与输出空间，不修改分解结果。

```
vector<vector<int>> block_cut_forest(const BiconnectedCore &graph)
{
    int n = graph.n;
    const auto &blocks = graph.blocks;
    vector<vector<int>> tree(n + blocks.size() + 1);
    for (int i = 0; i < (int)blocks.size(); i++)
    {
        int b = n + i + 1;
        for (int u : blocks[i])
        {
            tree[u].push_back(b);
            tree[b].push_back(u);
        }
    }
    return tree;
}
```

应用题补充：Luogu P4630

题意：无向简单图，可不连通，n≤100000、m≤200000；统计存在经过 c 的简单 s-f 路径的有序三元组 (s,c,f)，三点互异。原点 1..n 权为-1，方点 n+i+1 权为 blocks[i].size()，树路径权和就是可选 c 数。sz 仅计原点，每棵树独立累计有序端点对，答案用 long long。O(n+m) 时间、空间，递归 DFS。

所需模板：BiconnectedCore（第 807 页，47.2.1 节）、block_cut_forest（第 832 页，47.4.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4630

Luogu P4630：使用代码

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    BiconnectedCore graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    graph.run();
    auto tree = block_cut_forest(graph);
    vector<int> sz(tree.size());
    auto dfs = [&](auto &&self, int u, int p) -> void
    {
        sz[u] = (u <= n);
        for (int v : tree[u])
        {
            if (v == p) continue;
            self(self, v, u);
            sz[u] += sz[v];
        }
    };
    long long ans = 0;
    auto calc = [&](auto &&self, int u, int p, int total) -> void
    {
        long long sum = total - sz[u];
        long long cnt = u <= n ? 2LL * (total - 1) : 0;
        for (int v : tree[u])
        {
            if (v == p) continue;
            cnt += 2 * sum * sz[v];
            sum += sz[v];
            self(self, v, u, total);
        }
        long long w = u <= n ? -1 : (int)tree[u].size();
        ans += w * cnt;
    };
    for (int u = 1; u <= n; u++)
    {
        if (sz[u]) continue;
        dfs(dfs, u, 0);
        calc(calc, u, 0, sz[u]);
    }
    cout << ans << '\n';
    return 0;
}
```

第四十八章　环计数问题

48.1　简单无向图四元环计数

count_four_cycles(n,edges) 返回 long long 计数，n≥0、点 0..n-1、m=edges.size()≤INT_MAX。输入必须是简单无向图，每条边恰好给一次，不允许重边和自环；不自动去重。允许空图、孤立点、断连，输入不变，可重复调用。计四个不同点组成的四边环，不要求诱导子图无弦；K4 有 3 个四元环，旋转和反向不重复计算。按 (度数, 编号) 严格定序，以唯一最大点 u 为起点，统计 u-v-w 中 v,w 均小于 u 的二步路径；第二条路径与前面路径配对。每轮清空实际访问的 cnt。时间 O(n+m$\sqrt{m}$)，空间 O(n+m)，无需排序或递归；不可仅按编号定序后仍声称此复杂度。cnt 只需 int，答案必须 long long；在 m≤INT_MAX 约定下，环数 ≤m(m-1)/2<2^61，不溢出。

```
long long count_four_cycles(int n, const vector<pair<int, int>> &edges)
{
    assert(n >= 0 && edges.size() <= INT_MAX);
    vector<vector<int>> g(n);
    for (auto [u, v] : edges)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        g[u].push_back(v);
        g[v].push_back(u);
    }
    auto less = [&](int u, int v)
    {
        return pair{g[u].size(), u} < pair{g[v].size(), v};
    };
    vector<int> cnt(n), touched;
    long long ans = 0;
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u])
        {
            if (!less(v, u)) continue;
            for (int w : g[v])
            {
                if (!less(w, u)) continue;
                if (cnt[w] == 0) touched.push_back(w);
                ans += cnt[w];
                cnt[w]++;
            }
        }
        for (int w : touched) cnt[w] = 0;
        touched.clear();
    }
    return ans;
}
```

模板题使用：U367189 无向图四元环计数

题意：简单无向图，n≤100000，m≤200000；允许断连，不含重边和自环。输入 1-based 端点，输出非诱导四元环数。

所需模板：count_four_cycles（第 834 页，48.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/U367189

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> edges(m);
    for (auto &[u, v] : edges)
    {
        cin >> u >> v;
        u--;
        v--;
    }
    cout << count_four_cycles(n, edges) << '\n';
}
```

接口用法演示：API 零基四元环计数

演示约定：输入 n m 和 m 条 0-based 简单无向边；输出四元环数，允许 n=m=0。

所需模板：count_four_cycles（第 834 页，48.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/rings-count/

```
int main()
{
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> edges(m);
    for (auto &[u, v] : edges)
        cin >> u >> v;
    cout << count_four_cycles(n, edges) << '\n';
}
```

48.2 无向简单图三元环枚举

enumerate_triangles(n,edges,visit) 使用 0..n-1 点号；edges 须为无向简单图，无自环、无重边，每条边只给一次，端点顺序任意。每个三角形恰调用一次 visit(u,v,w)，参数按度数和编号组成的次序排列，不保证编号递增。可在回调中计数、累计权值或收集方案；计数用 long long。按 (度数, 编号) 从小到大定向，标记 u 的出邻点后枚举 u 到 v 到 w 的有向路径。每个出度为 $O(\sqrt{m})$，总时间 $O(n+m\sqrt{m})$，含常数时间回调，空间 O(n+m)；空图合法。调用期间不修改输入边集。

```cpp
template <class F>
void enumerate_triangles(int n, const vector<pair<int, int>> &edges, F visit)
{
    assert(n >= 0);
    vector<int> deg(n), mark(n, -1);
    vector<vector<int>> g(n);
    for (auto [u, v] : edges)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        deg[u]++, deg[v]++;
    }
    for (auto [u, v] : edges)
    {
        if (pair{deg[u], u} > pair{deg[v], v}) swap(u, v);
        g[u].push_back(v);
    }
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u]) mark[v] = u;
        for (int v : g[u])
            for (int w : g[v])
                if (mark[w] == u) visit(u, v, w);
    }
}
```

模板题使用：Library Checker enumerate_triangles

题意：给简单无向图的每个点一个权值，求所有三角形三个点权乘积之和，模 998244353。点号 0..n-1。

所需模板：enumerate_triangles（第 836 页，48.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/enumerate_triangles

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> x(n);
    for (auto &v : x) cin >> v;
    vector<pair<int, int>> edges(m);
    for (auto &[u, v] : edges) cin >> u >> v;
    const long long mod = 998244353;
    long long ans = 0;
    enumerate_triangles(n,
                        edges,
                        [&](int u, int v, int w)
                        { ans = (ans + x[u] * x[v] % mod * x[w]) % mod; });
    cout << ans << '\n';
}
```

第四十九章　最小环

相关应用：Karp 最小平均权有向环，目标不同于最小总权环；两遍 DP 线性空间。

49.1　有向图最小平均权环：线性空间 Karp

minimum_mean_cycle(n,edges)，0≤n<INT_MAX，点 0..n-1，edges 为 tuple<int,int,T> 有向边；支持断连、负权、自环和重边，输入不变。返回 optional<pair<W,int»，无环 nullopt；有环返回分子/正分母，未约分，不返回环方案。T 为有符号至多 64 位整数时 W=int128，全部比较精确，支持 LLONG_MIN/MAX；n<2^31 使交叉乘法绝对值 <2^126。实数 T 须支持值语义，W=T；禁止 NaN/Inf，要求 $2n^2 \max|w| < 2^{118}$ 避免溢出，仍有舍入误差，不能泛称精确。P3199 驱动用 GCC ___float128 进行 113 位核心运算，long double 仅用于输入输出；需 GCC 支持此扩展。F0 全部为 0，Fk 为恰好 k 条边的最短行走；先滚动到 Fn，再重算 F0..Fn-1，取 min_v max_k (Fn(v)-Fk(v))/(n-k)。时间 O(n(n+m))，额外 O(n)，输入 O(m)；非强连通也成立，无需 DFS、二分或负环启发式。k 为 0..n-1，不能只比较最后一层或允许至多 k 条边。

```
template <class T>
auto minimum_mean_cycle(int n, const vector<tuple<int, int, T>> &edges)
{
    static_assert(!is_integral_v<T> || (is_signed_v<T> && sizeof(T) <= 8));
    assert(0 <= n && n < INT_MAX);
    using W = conditional_t<is_integral_v<T>, __int128_t, T>;
    using Ratio = pair<W, int>;
    const W inf = W(1LL << 60) * W(1LL << 60);
    vector<W> d(n), next(n);
    auto step = [&]()
    {
        fill(next.begin(), next.end(), inf);
        for (auto [u, v, w] : edges)
        {
            if (d[u] == inf) continue;
            W value = d[u] + w;
            if (value < next[v]) next[v] = value;
        }
        d.swap(next);
    };
    for (int k = 0; k < n; k++) step();
    auto last = d;
    fill(d.begin(), d.end(), 0);
    auto less = [&](const Ratio &a, const Ratio &b)
    {
        return a.first * b.second < b.first * a.second;
    };
```

第二遍重算并取 min-max（接上页同一函数）：

```
    vector<Ratio> best(n, {0, 0});
    for (int k = 0; k < n; k++)
    {
        for (int v = 0; v < n; v++)
        {
            if (last[v] == inf || d[v] == inf) continue;
            Ratio now{last[v] - d[v], n - k};
            if (best[v].second == 0 || less(best[v], now))
                best[v] = now;
        }
        if (k + 1 < n) step();
    }
    optional<Ratio> ans;
    for (auto r : best)
    {
        if (r.second && (!ans || less(r, *ans))) ans = r;
    }
    return ans;
}
```

为什么全点初始化成立 把各边减去最小平均值后，所有环非负且存在零环。任意起点最短行走都能删环变成少于 n 条边的路径，所以每点的 max 比值非负。到零环的最短行走接无限重复零环，取长度 n 的最短前缀，其终点使 max 为 0；因此全局 min 为 0。恢复原权后所有比值增加同一常数，公式仍成立。无环图没有长度 n 行走，返回 nullopt。

返回分数与数值类型 分母为 1..n；不保证约分，也不保证分子本身就是某个环的权和。整数类型提升到 int128 进行运算和交叉比较；小数用调用者选择的 T，普通浮点不是精确计算。P3199 采用 GCC 的 113 位浮点进行核心运算，long double 仅用于输入输出。无需保存全部 DP 层，但第二遍必须重新从全 0 开始。

模板题使用：P3199 [HNOI2009] 最小圈

题意：实数有向边权，n≤3000、m≤10000、|w|≤10^7，保证有圈；GCC ___float128 核心运算，long double 读入/输出 8 位小数。

所需模板：minimum_mean_cycle（第 838 页，49.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3199

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Real = __float128;
    int n, m;
    cin >> n >> m;
    vector<tuple<int, int, Real>> edges;
    while (m--)
    {
        int u, v;
        long double w;
        cin >> u >> v >> w;
        edges.emplace_back(u - 1, v - 1, w);
    }
    auto ans = minimum_mean_cycle(n, edges);
    assert(ans);
    cout << fixed << setprecision(8) << (long double)(ans->first / ans->second) << '\n';
}
```

接口用法演示：API 整数最小平均环精确分数

演示约定：输入 n m 和 0-based 有向边 u v w，w 为 signed64；无环输出 NONE，否则输出 128 位分子和正分母，未约分。

所需模板：minimum_mean_cycle（第 838 页，49.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://www2.eecs.berkeley.edu/Pubs/TechRpts/1977/29121.html

```
int main()
{
    int n, m;
    cin >> n >> m;
    vector<tuple<int, int, long long>> edges(m);
    for (auto &[u, v, w] : edges)
        cin >> u >> v >> w;
    auto ans = minimum_mean_cycle(n, edges);
    if (!ans)
    {
        cout << "NONE\n";
        return 0;
    }
    auto num = ans->first;
    if (num < 0)
    {
        cout << '-';
        num = -num;
    }
    string out;
    do
    {
        out += char('0' + num % 10);
        num /= 10;
    } while (num);
    reverse(out.begin(), out.end());
    cout << out << ' ' << ans->second << '\n';
}
```

49.2 非负权无向图最小环与原边方案

MinCycle(n), n≥0, 点 0..n-1。add(u,v,w) 返回从 0 开始的原边号, 边数 ≤INT_MAX, 0≤w≤LLONG_MAX; 允许自环和重边。solve() 从零求至少 3 个不同顶点的简单环, 无解 nullopt; 故自环忽略, 平行边只保留最轻的一条（同权留最早）, 不计二元环。Result.weight 是 int128, vertices 不重复首点, edges[i] 连接 vertices[i] 与 vertices[(i+1)%size], 边号对应 e; 并列返回任意一个, 不保证字典序。支持空图、断连、零权、重复 solve 及加边后重算, 旧结果独立; e/id 只读。Floyd 更新中间点 k 之前枚举 i<j<k, 用 d[i][j]+ 边 (i,k)+ 边 (j,k) 更新最优值, 每轮至多恢复一次路径; 严格松弛避免零权环导致方案循环。solve 为 $O(n^3+n^2)$, 构造 $O(n^2)$, add 摊还 O(1), 总空间 $O(n^2+m)$, 恢复通过 next 逐边走路径, 不是模拟 DFS。负边不支持; 无穷大 2^120 只作内部哨兵, 合法简单路径与候选和均小于 2^95。

```
struct MinCycle
{
    using I = __int128_t;
    struct Edge
    {
        int u, v;
        long long w;
    };
    struct Result
    {
        I weight;
        vector<int> vertices, edges;
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> id;

    MinCycle(int n) : n(n), id(n, vector<int>(n, -1))
    {
        assert(n >= 0);
    }

    int add(int u, int v, long long w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && w >= 0);
        int k = e.size();
        e.push_back({u, v, w});
        if (u != v && (id[u][v] == -1 || w < e[id[u][v]].w))
            id[u][v] = id[v][u] = k;
        return k;
    }
```

初始化与本轮候选检查（接上页同一结构体）：

```
    optional<Result> solve() const
    {
        const I inf = I(1) << 120;
        vector<vector<I>> d(n, vector<I>(n, inf));
        vector<vector<int>> next(n, vector<int>(n, -1));
        for (int i = 0; i < n; i++)
        {
            d[i][i] = 0;
            for (int j = 0; j < n; j++)
            {
                if (id[i][j] == -1) continue;
                d[i][j] = e[id[i][j]].w;
                next[i][j] = j;
            }
        }
        Result ans{inf, {}, {}};
        for (int k = 0; k < n; k++)
        {
            int u = -1, v = -1;
            for (int i = 0; i < k; i++)
            {
                if (id[i][k] == -1) continue;
                for (int j = i + 1; j < k; j++)
                {
                    if (id[j][k] == -1 || d[i][j] == inf) continue;
                    I w = d[i][j] + e[id[i][k]].w + e[id[j][k]].w;
                    if (w >= ans.weight) continue;
                    ans.weight = w;
                    u = i;
                    v = j;
                }
            }
```

方案恢复与 Floyd 更新（接上页 solve 的 k 循环）：

```
            if (u != -1)
            {
                ans.vertices = {k, u};
                ans.edges = {id[k][u]};
                while (u != v)
                {
                    int x = next[u][v];
                    ans.edges.push_back(id[u][x]);
                    ans.vertices.push_back(x);
                    u = x;
                }
                ans.edges.push_back(id[v][k]);
            }
            for (int i = 0; i < n; i++)
            {
                for (int j = 0; j < n; j++)
                {
                    if (d[i][k] == inf || d[k][j] == inf) continue;
                    I w = d[i][k] + d[k][j];
                    if (w >= d[i][j]) continue;
                    d[i][j] = w;
                    next[i][j] = next[i][k];
                }
            }
        }
        if (ans.weight == inf) return nullopt;
        return ans;
    }
};
```

检查必须在松弛之前 固定环中最大编号点 k，其两侧邻点 i、j 都小于 k。此时 d[i][j] 只允许小于 k 的中间点，与两条原边共同构成合法简单环；先做本轮 Floyd 会使路径经过 k，错误接受重复点。

恢复不能每次改进都做 每轮只记最优 i、j，结束枚举后恢复一次，所有恢复共 $O(n^2)$。next 沿真实路径逐边走，不是模拟 DFS；更新必须严格变小，尤其不能把零权等长路径改成自指。

模板题使用：P6175 无向图的最小环问题

题意：求至少 3 个不同顶点的最小权环；n≤100，m≤5000，边权 1..100000，答案 ≤10000000，可转 long long 输出；无解精确输出 No solution.。

所需模板：MinCycle（第 842 页，49.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6175

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    MinCycle g(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add(u - 1, v - 1, w);
    }
    auto ans = g.solve();
    if (ans)
        cout << (long long)ans->weight << '\n';
    else
        cout << "No solution.\n";
}
```

接口用法演示：API 无向最小环与原边编号

演示约定：输入 n m 及 0-based 无向边 u v w，w 非负 long long。无解输出 NONE；否则依次输出 128 位权值、环长、顶点序列、原边编号序列，不重复首点。

所需模板：MinCycle（第 842 页，49.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/min-cycle/

```
int main()
{
    int n, m;
    cin >> n >> m;
    MinCycle g(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add(u, v, w);
    }
    auto ans = g.solve();
    if (!ans)
    {
        cout << "NONE\n";
        return 0;
    }
    auto w = ans->weight;
    string out;
    do
    {
        out += char('0' + w % 10);
        w /= 10;
    } while (w);
    reverse(out.begin(), out.end());
    cout << out << '\n';
    cout << ans->vertices.size() << '\n';
    for (int u : ans->vertices)
        cout << u << ' ';
    cout << '\n';
    for (int id : ans->edges)
        cout << id << ' ';
    cout << '\n';
}
```

第五十章　2-SAT

50.1　2-SAT 与任意可行方案

代码依赖：SCC（第 804 页，47.1.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

TwoSAT(n)，变量 1..n；add(x,a,y,b) 添加 (x=a) 或 (y=b)，强制值写 add(x,a,x,a)，蕴含 (x=a)=>(y=b) 写 add(x,!a,y,b)。solve() 返回可满足性，仅成功时读取 ans[1..n]；可重复求解和追加子句后重算，空公式可满足。不保证字典序最小。依赖同文件 SCC：Kosaraju 编号为拓扑序，故真值分量编号较大时取真；不能直接套用 Tarjan 逆拓扑编号的不等号。O(n+m) 时间和空间，DFS 递归。

```
struct TwoSAT
{
    int n;
    SCC g;
    vector<int> ans;

    TwoSAT(int n) : n(n), g(2 * n) {}

    int id(int x, bool value) const { return 2 * x - 1 + value; }

    void add(int x, bool a, int y, bool b)
    {
        g.add(id(x, !a), id(y, b));
        g.add(id(y, !b), id(x, a));
    }

    bool solve()
    {
        g.run();
        ans.assign(n + 1, 0);
        for (int x = 1; x <= n; x++)
        {
            if (g.bel[id(x, 0)] == g.bel[id(x, 1)]) return false;
            ans[x] = g.bel[id(x, 1)] > g.bel[id(x, 0)];
        }
        return true;
    }
};
```

模板题使用：Library Checker two_sat

题意：每个子句是两个文字的或，判断是否可满足并输出一组赋值。输入为 DIMACS：首行 p cnf n m，正数表示变量，负数表示取反，每个子句末尾为 0。

所需模板：SCC（第 804 页，47.1.3 节）、TwoSAT（第 847 页，50.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/two_sat

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string p, cnf;
    int n, m;
    cin >> p >> cnf >> n >> m;
    TwoSAT g(n);
    for (int i = 0; i < m; i++)
    {
        int a, b, zero;
        cin >> a >> b >> zero;
        g.add(abs(a), a > 0, abs(b), b > 0);
    }
    if (!g.solve())
    {
        cout << "s UNSATISFIABLE\n";
        return 0;
    }
    cout << "s SATISFIABLE\nv";
    for (int i = 1; i <= n; i++) cout << ' ' << (g.ans[i] ? i : -i);
    cout << " 0\n";
}
```

模板题使用：Luogu P4782

题意：n 个布尔变量，m 个 (x=a) 或 (y=b) 子句，输出任意满足全部子句的赋值或 IMPOSSIBLE。变量号 1..n；n,m≤10^6。同一变量的重复文字照常 add，成功后才读取 ans。

所需模板：SCC（第 804 页，47.1.3 节）、TwoSAT（第 847 页，50.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4782

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    TwoSAT sat(n);
    while (m--)
    {
        int x, a, y, b;
        std::cin >> x >> a >> y >> b;
        sat.add(x, a, y, b);
    }
    if (!sat.solve())
    {
        std::cout << "IMPOSSIBLE\n";
        return 0;
    }
    std::cout << "POSSIBLE\n";
    for (int i = 1; i <= n; i++) std::cout << sat.ans[i] << (i == n ? '\n' : ' ');
    return 0;
}
```

50.2 递归染色与回滚求最小赋值

构造 LexTwoSAT(n)，变量号 1..n。add(x,a,y,b) 加入子句 (x=a) OR (y=b)，force(x,v) 强制变量值；solve() 成功后 answer[1..n] 是按变量编号、0 优先于 1 的字典序最小赋值。失败时 answer 为空，n=0 时可满足，answer[0] 始终仅为占位。每个未确定变量先尝试 0，递归传播所有蕴含；冲突时仅撤销本次新增标记，再试 1。history 是撤销记录，不替代递归调用栈。可重复求解，追加子句会清空旧答案；marked/history 仅为内部工作区，失败时不可当作方案。内部 id(x,v)=2(x-1)+v，从 0 开始，可用异或 1 取反；不要混用另一份 SCC 2-SAT 的 1-based 字面量编号。最坏 O(n(n+m)) 时间、O(n+m) 空间，递归栈 O(n)；100000 变量测试是特定蕴含链，不代表一般大规模输入的性能保证。HDU1814 完整程序本地对拍，在线待补。

```
struct LexTwoSAT
{
    int n;
    vector<vector<int>> g;
    vector<int> marked, history, answer;

    LexTwoSAT(int n) : n(n), g(2 * n), marked(2 * n) { assert(n >= 0); }

    int id(int x, bool value) const
    {
        assert(1 <= x && x <= n);
        return 2 * (x - 1) + value;
    }

    // (x == a) OR (y == b). Internal literal IDs are 0-based.
    void add(int x, bool a, int y, bool b)
    {
        int u = id(x, a), v = id(y, b);
        g[u ^ 1].push_back(v);
        g[v ^ 1].push_back(u);
        answer.clear();
    }

    void force(int x, bool value) { add(x, value, x, value); }
```

```
    bool paint(int u)
    {
        if (marked[u ^ 1]) return false;
        if (marked[u]) return true;
        marked[u] = 1;
        history.push_back(u);
        for (int v : g[u])
            if (!paint(v)) return false;
        return true;
    }

    bool solve()
    {
        fill(marked.begin(), marked.end(), 0);
        history.clear();
        answer.clear();
        for (int x = 1; x <= n; x++)
        {
            int u = id(x, false);
            if (marked[u] || marked[u ^ 1]) continue;
            int checkpoint = history.size();
            if (!paint(u))
            {
                while ((int)history.size() > checkpoint)
                {
                    marked[history.back()] = 0;
                    history.pop_back();
                }
                if (!paint(u ^ 1)) return false;
            }
        }
        answer.assign(n + 1, 0);
        for (int x = 1; x <= n; x++) answer[x] = marked[id(x, true)];
        return true;
    }
};
```

应用题补充：HDU 1814 Peaceful Commission

题意：每党两名候选人编号 2x-1、2x，恰选一人且冲突双方不能同时入选，输出字典序最小代表序列。候选人 u 转为变量 x=(u+1)/2，偶号对应值 1；冲突 (u,v) 转为 (x≠a) OR (y≠b)，即 add(x,!a,y,!b)。变量与 answer 均为 1-based；按党号输出 2*x-1+answer[x]，每人一行。无解只输出 NIE，不能读取失败后的 answer。多组读至 EOF，每组重新构造，无 0 0 结束标记。原站题面本次不可达，题意与格式依据现有驱动及独立枚举测试，不声称已复核官方限制或线上 AC。

所需模板：LexTwoSAT（第 850 页，50.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://acm.hdu.edu.cn/showproblem.php?pid=1814

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    while (cin >> n >> m)
    {
        LexTwoSAT graph(n);
        while (m--)
        {
            int u, v;
            cin >> u >> v;
            int x = (u + 1) / 2;
            int y = (v + 1) / 2;
            bool a = u % 2 == 0;
            bool b = v % 2 == 0;
            graph.add(x, !a, y, !b);
        }
        if (!graph.solve())
            cout << "NIE\n";
        else
            for (int x = 1; x <= n; x++) cout << 2 * x - 1 + graph.answer[x] << '\n';
    }
    return 0;
}
```

第五十一章　欧拉图

51.1　有向图字典序最小欧拉路

构造 DirectedEuler(n)，n≥1，点号 1..n；add(u,v) 返回从 0 开始的逻辑边编号，支持自环与重边。run() 返回是否存在欧拉路；参数 start 默认 0，lexicographic 默认 true。start=0 自动选择最小可行起点，否则固定起点；孤立点不影响有边部分的连通性。成功后 vertices 长度为 m+1，edge_ids 长度为 m，第 i 条原边从 vertices[i] 指向 vertices[i+1]；失败清空两数组。无边时返回单点路径，自动起点为 1。可重复 run，add 会清空旧方案。默认求顶点序列字典序最小方案；关闭排序后只保证任意方案，邻接顺序可能保留上次排序。开启排序总时间 O(n+m log(m+1))，否则 O(n+m)，空间 O(n+m)。DFS 递归深度可能达到 m+1，不是 n。度数检查后，以消费边数判定是否遍历全部非孤立部分，逆后序形成连续走法。小图全部走法对拍同时验证存在性、字典序及边编号，P7771 双风格完整程序本地验证，在线待补。

```
struct DirectedEuler
{
    int n;
    vector<vector<pair<int, int>>> g;
    vector<pair<int, int>> edges;
    vector<int> in, out, next, vertices, edge_ids;

    DirectedEuler(int n) : n(n), g(n + 1), in(n + 1), out(n + 1), next(n + 1)
    {
        assert(n > 0);
    }

    int add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        int id = edges.size();
        edges.push_back({u, v});
        g[u].push_back({v, id});
        out[u]++;
        in[v]++;
        vertices.clear();
        edge_ids.clear();
        return id;
    }

    void dfs(int u, int incoming)
    {
        while (next[u] < (int)g[u].size())
        {
            auto [v, id] = g[u][next[u]++];
            dfs(v, id);
        }
        vertices.push_back(u);
        if (incoming != -1) edge_ids.push_back(incoming);
    }
```

```
    // start=0 chooses the smallest feasible start. Isolated vertices are ignored.
    bool run(int start = 0, bool lexicographic = true)
    {
        assert(0 <= start && start <= n);
        vertices.clear();
        edge_ids.clear();
        int source = 0, sink = 0;
        for (int u = 1; u <= n; u++)
        {
            int delta = out[u] - in[u];
            if (delta == 1 && !source)
                source = u;
            else if (delta == -1 && !sink)
                sink = u;
            else if (delta != 0)
                return false;
        }
        if (bool(source) != bool(sink)) return false;
        if (source)
        {
            if (start && start != source) return false;
            start = source;
        }
        if (!start)
        {
            start = 1;
            for (int u = 1; u <= n; u++)
                if (out[u])
                {
                    start = u;
                    break;
                }
        }
        if (lexicographic)
            for (int u = 1; u <= n; u++) sort(g[u].begin(), g[u].end());
        fill(next.begin(), next.end(), 0);
        dfs(start, -1);
        if (edge_ids.size() != edges.size())
        {
            vertices.clear();
            edge_ids.clear();
            return false;
        }
        reverse(vertices.begin(), vertices.end());
        reverse(edge_ids.begin(), edge_ids.end());
        return true;
    }
};
```

模板题使用：Luogu P7771

题意：给有向图，求字典序最小的欧拉路径。run 默认选择可行起点并按字典序遍历；成功后 vertices 含 m+1 个顶点，失败输出 No。点号 1..n。

所需模板：DirectedEuler（第 853 页，51.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P7771

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    DirectedEuler graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    if (!graph.run())
        cout << "No\n";
    else
    {
        for (int u : graph.vertices) cout << u << ' ';
        cout << '\n';
    }
    return 0;
}
```

51.2 字典序最小单词链

代码依赖：DirectedEuler（第 853 页，51.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

word_chain(words) 返回可空的编号序列 (optional), 有解时为原输入的 0-based 单词编号顺序, 每个单词恰用一次；相邻单词首尾字母相接，单词序列字典序最小。无解返回 nullopt，空输入返回有值的空序列。单词必须非空且仅含小写 a-z，重复词视为不同编号。先对原编号按单词排序，按此顺序加边，再调用 DirectedEuler.run(0,false)；不可让底层重新按终点排序。欧拉回路从最小出现首字母开始，非回路起点由度数差确定。O(m log m) 次字符串比较及 O(L+m) 其他工作，L 为总字符数；空间 O(m)，递归栈 O(m)。依赖 directed_euler.hpp。全排列对拍、重复词/前缀词、20 万相同词及 POJ2337 完整输出本地验证，在线待补。

```
// Lowercase nonempty words. Returns original 0-based IDs; duplicates stay distinct.
optional<vector<int>> word_chain(const vector<string> &words)
{
    int n = words.size();
    vector<int> order(n);
    iota(order.begin(), order.end(), 0);
    for (const auto &word : words)
    {
        assert(!word.empty());
        for (char c : word) assert('a' <= c && c <= 'z');
    }
    sort(order.begin(),
         order.end(),
         [&](int a, int b)
         {
             if (words[a] != words[b]) return words[a] < words[b];
             return a < b;
         });
    DirectedEuler graph(26);
    for (int id : order)
        graph.add(words[id].front() - 'a' + 1, words[id].back() - 'a' + 1);
    // Preserve word order in each adjacency list; do not sort by destination.
    if (!graph.run(0, false)) return nullopt;
    vector<int> result;
    for (int id : graph.edge_ids) result.push_back(order[id]);
    return result;
}
```

应用题补充：UVA 10441 Catenyms

题意：多组给出 3..1000 个互不相同的小写单词，长度 1..20。每词恰用一次，后词首字母等于前词尾字母，输出字典序最小的点分隔词链；无解输出 ***。返回原输入的 0-based 编号，按编号取词输出，末尾不加点。单词排序不可换成按终点排序。重复词和空输入仅属更宽接口范围，不在本题范围内。

所需模板：DirectedEuler（第 853 页，51.1 节）、word_chain（第 856 页，51.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.org/external/104/10441.pdf

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        vector<string> words(n);
        for (auto &word : words) cin >> word;
        auto order = word_chain(words);
        if (!order)
        {
            cout << "***\n";
            continue;
        }
        for (int i = 0; i < n; i++)
        {
            if (i) cout << '.';
            cout << words[(*order)[i]];
        }
        cout << '\n';
    }
    return 0;
}
```

51.3 无向图欧拉路与边方向

构造 UndirectedEuler(n)，n≥1，点号 1..n；add(u,v) 返回从 0 开始的逻辑边号，自环和重边均保留。run() 返回是否存在欧拉路，参数 start 默认 0（自动），lexicographic 默认 true。自动选最小可行起点；指定起点须满足奇度条件且位于有边部分。成功得到 m+1 个 vertices 和 m 个 edge_ids，失败清空两数组。第 i 条边方向是 vertices[i] 到 vertices[i+1]，与 edges[edge_ids[i]] 比较可判断是否反向，自环任选方向。used 按逻辑边号标记，使两条半边共同只消费一次；自环贡献度数 2。孤立点不影响有边部分；无边图返回指定点或点 1 的单点序列。默认求顶点序列字典序最小方案；关闭排序只保证任意方案，可能保留上次排序。可重复 run，加边清空旧方案。排序开启时 O(n+m log(m+1))，否则 O(n+m)，空间 O(n+m)，DFS 递归深度可能达到 m+1。全小图走法、原边号与方向独立验证，P2731 字典序顶点序列已线上 AC；边编号、失败输入及其他模式仍由本地验证。

```
struct UndirectedEuler
{
    int n;
    vector<vector<pair<int, int>>> g;
    vector<pair<int, int>> edges;
    vector<int> next, used, vertices, edge_ids;

    UndirectedEuler(int n) : n(n), g(n + 1), next(n + 1) { assert(n > 0); }

    int add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        int id = edges.size();
        edges.push_back({u, v});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        vertices.clear();
        edge_ids.clear();
        return id;
    }

    void dfs(int u, int incoming)
    {
        while (next[u] < (int)g[u].size())
        {
            auto [v, id] = g[u][next[u]++];
            if (used[id]) continue;
            used[id] = 1;
            dfs(v, id);
        }
        vertices.push_back(u);
        if (incoming != -1) edge_ids.push_back(incoming);
    }
```

```
    // start=0 chooses the smallest feasible start. Isolated vertices are ignored.
    bool run(int start = 0, bool lexicographic = true)
    {
        assert(0 <= start && start <= n);
        vertices.clear();
        edge_ids.clear();
        vector<int> odd;
        for (int u = 1; u <= n; u++)
            if (g[u].size() % 2) odd.push_back(u);
        if (!odd.empty() && odd.size() != 2) return false;
        if (!odd.empty())
        {
            if (start && start != odd[0] && start != odd[1]) return false;
            if (!start) start = odd[0];
        }
        if (!start)
        {
            start = 1;
            for (int u = 1; u <= n; u++)
                if (!g[u].empty())
                {
                    start = u;
                    break;
                }
        }
        if (lexicographic)
            for (int u = 1; u <= n; u++) sort(g[u].begin(), g[u].end());
        fill(next.begin(), next.end(), 0);
        used.assign(edges.size(), 0);
        dfs(start, -1);
        if (edge_ids.size() != edges.size())
        {
            vertices.clear();
            edge_ids.clear();
            return false;
        }
        reverse(vertices.begin(), vertices.end());
        reverse(edge_ids.begin(), edge_ids.end());
        return true;
    }
};
```

模板题使用：Luogu P2731

题意：USACO Training：无向多重图保证有欧拉路，输出字典序最小顶点序列。m≤1024，点号 1..500；直接构造 500 个点，不把孤立点 1 强设为起点。run() 自动选较小奇点，若全偶选最小非孤立点，并按邻点排序。输出 vertices，共 m+1 项；本题不输出 edge_ids，边编号与方向另验。DFS 递归，最深可达 m+1。

所需模板：UndirectedEuler（第 858 页，51.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2731

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    cin >> m;
    UndirectedEuler graph(500);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    if (!graph.run()) return 0;
    for (int u : graph.vertices) cout << u << '\n';
    return 0;
}
```

51.4 欧拉回路与指定端点定向

代码依赖：Dinic（第 871 页，52.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

mixed_euler_orientation(n,edges,start,finish) 返回可空的定向边序列。每条输入为 {u,v,type}，type=1 固定 u 到 v，type=0 可反转；点号 1..n，n≥1。省略两个端点或均传 0 时请求任意起点的欧拉回路；两个正端点请求指定起终点的欧拉路径，相等时为指定起点回路。无解返回 nullopt，有解返回与原边顺序对应的端点对；无边图可返回有值空序列。忽略无关孤立点，但正端点须在有边部分。先任意定向可反转边，扣除目标出入度差；每条反转将两端差值分别改变 -2 和 +2，奇数差无解，否则用容量 1 的反转边流调整。检查底层无向连通性后，满流即得到所需方向。最大流图 n+2 点、至多 m+n 条正向边，总时间可按 Dinic 的 $O((n+2)^2(m+n))$ 上界计，空间 O(n+m)。依赖 flow.hpp；返回边交给 DirectedEuler 可生成连续走法，指定端点时保持指定起点。定向不保证字典序最小。

```
// Edge = {u,v,type}: type 1 is fixed u->v; type 0 may be reversed.
// start=finish=0 requests a circuit; positive endpoints request a specified trail.
optional<vector<pair<int, int>>> mixed_euler_orientation(
    int n, const vector<array<int, 3>> &edges, int start = 0, int finish = 0)
{
    assert(n > 0 && 0 <= start && start <= n && 0 <= finish && finish <= n);
    assert(bool(start) == bool(finish));
    vector<int> parent(n + 1), id;
    vector<long long> degree(n + 1), balance(n + 1);
    iota(parent.begin(), parent.end(), 0);
    auto find = [&](auto &&self, int u) -> int
    {
        if (parent[u] != u) parent[u] = self(self, parent[u]);
        return parent[u];
    };
    Dinic flow(n + 2);
    vector<pair<int, int>> direction;
    for (auto [u, v, type] : edges)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(type == 0 || type == 1);
        parent[find(find, u)] = find(find, v);
        degree[u]++;
        degree[v]++;
        balance[u]++;
        balance[v]--;
        id.push_back(type == 0 ? flow.add(u, v, 1) : -1);
        direction.push_back({u, v});
    }
```

```
    if (!edges.empty())
    {
        int root = find(find, edges[0][0]);
        for (int u = 1; u <= n; u++)
            if (degree[u] && find(find, u) != root) return nullopt;
        if (start && (!degree[start] || !degree[finish])) return nullopt;
    }
    if (start)
    {
        balance[start]--;
        balance[finish]++;
    }
    int source = n + 1, sink = n + 2;
    long long need = 0;
    for (int u = 1; u <= n; u++)
    {
        if (balance[u] % 2) return nullopt;
        if (balance[u] > 0)
        {
            flow.add(source, u, balance[u] / 2);
            need += balance[u] / 2;
        }
        else if (balance[u] < 0)
            flow.add(u, sink, -balance[u] / 2);
    }
    if (flow.flow(source, sink) != need) return nullopt;
    for (int i = 0; i < (int)edges.size(); i++)
        if (id[i] != -1 && flow.used(id[i]))
            swap(direction[i].first, direction[i].second);
    return direction;
}
```

接口用法演示：混合欧拉路径：定向与走法（自定义协议）

演示约定：自定义接口演示，非 OJ 题目。首行 T；每组 mode n m start finish，再读 m 条 u v type。点号 1..n，type=1 固定方向、0 可反转。C 输入端点 0 0 求回路；A 也输入 0 0，由 mixed_euler_trail 选择端点；P 指定两个正端点，相等时是指定起点回路。optional 有值空数组是合法空方案。返回方向须按原边顺序加入 DirectedEuler，再 run(start,false) 恢复连续走法。成功输出 YES、VERTICES 及 EDGES 证书，原边号由内部 0-based 加一；无解输出 NO。零边 C/A 输出点 1，P s s 输出点 s，不同正端点无解。非空图忽略无关孤立点，指定端点须在有边部分。1≤T≤10000、1≤n≤200、0≤m≤800 仅为演示边界。定向采用 Dinic 一般上界，任意端点最多求两次；欧拉恢复 O(n+m)，DFS 深度可达 m+1。不保证所有定向中的全局字典序最小。

所需模板：Dinic（第 871 页，52.1.2 节）、mixed_euler_orientation（第 861 页，51.4 节）、mixed_euler_trail（第 864 页，51.5 节）、DirectedEuler（第 853 页，51.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/src/compact/mixed_euler.hpp

混合欧拉路径：定向与走法（自定义协议）：使用代码

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    assert(1 <= tests && tests <= 10000);
    while (tests--)
    {
        char mode;
        int n, m, start, finish;
        cin >> mode >> n >> m >> start >> finish;
        assert(1 <= n && n <= 200 && 0 <= m && m <= 800);
        assert(mode == 'C' || mode == 'A' || mode == 'P');
        assert(mode == 'P' ? 1 <= start && start <= n && 1 <= finish && finish <= n
                           : start == 0 && finish == 0);
        vector<array<int, 3>> edges(m);
        for (auto &[u, v, type] : edges) cin >> u >> v >> type;
        optional<vector<pair<int, int>>> direction;
        if (mode == 'A')
            direction = mixed_euler_trail(n, edges);
        else if (mode == 'C')
            direction = mixed_euler_orientation(n, edges);
        else
            direction = mixed_euler_orientation(n, edges, start, finish);
        if (!direction)
        {
            cout << "NO\n";
            continue;
        }
        // A present empty vector is a valid zero-edge circuit, not failure.
        DirectedEuler trail(n);
        for (auto [u, v] : *direction) trail.add(u, v);
        // Preserve a requested start. The orientation is not globally lexicographic.
        if (!trail.run(start, false)) return 1;
        cout << "YES\nVERTICES " << trail.vertices.size();
        for (int u : trail.vertices) cout << ' ' << u;
        cout << "\nEDGES " << trail.edge_ids.size();
        // add() used original input order; its 0-based IDs become 1-based here.
        for (int id : trail.edge_ids) cout << ' ' << id + 1;
        cout << '\n';
    }
    return 0;
}
```

51.5 任意起终点的混合欧拉路径

代码依赖：mixed_euler_orientation（第 861 页，51.4 节）。抄写时一并准备；标准库类型不列入算法依赖。

mixed_euler_trail(n,edges) 返回任意可行欧拉路径的定向边序列，格式同上一节。忽略边原方向统计度数，自环计 2；若奇度点为 0 个，请求回路；若恰有 2 个，尝试这两个端点的两种顺序；否则无解。最多两次定向求解。返回方向可交给 DirectedEuler 自动选择起点，恢复顶点与原边编号序列。枚举所有可反转边方向、全部固定端点及自由端点对拍；逐边检查固定边不变和连续走法，另有 10 万点长流量路径测试。POJ1637 双驱动本地验证，仅检查回路存在性；完整方向和开放路径依靠本地证据，在线待补。

```
// Any endpoints: zero odd degrees requests a circuit; two odd degrees tries both
// orders.
optional<vector<pair<int, int>>> mixed_euler_trail(int n,
                                                   const vector<array<int, 3>> &edges)
{
    assert(n > 0);
    vector<long long> degree(n + 1);
    vector<int> odd;
    for (auto [u, v, type] : edges)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        degree[u]++;
        degree[v]++;
    }
    for (int u = 1; u <= n; u++)
        if (degree[u] % 2) odd.push_back(u);
    if (odd.empty()) return mixed_euler_orientation(n, edges);
    if (odd.size() != 2) return nullopt;
    auto answer = mixed_euler_orientation(n, edges, odd[0], odd[1]);
    if (answer) return answer;
    return mixed_euler_orientation(n, edges, odd[1], odd[0]);
}
```

联合使用示例见第 863 页（混合欧拉路径：定向与走法（自定义协议）），其中直接调用本模板。

第五十二章　网络流

52.1　最大流

52.1.1　HLPP 最高标号预流推进与流方案

HLPP(n)，2≤n<INT_MAX/2，点 1..n。add(u,v,cap) 返回偶数残量边号 id，允许自环、重边、反向边和零容量；原边数 <INT_MAX/2，全部原容量之和 ≤LLONG_MAX。flow(s,t) 要求 s≠t，每次从零重算并返回总最大流，支持重复调用、换源汇和加边后重算；与 Dinic 返回增量的约定不同。used(id) 读原边净流，min_cut() 返回源侧 0/1 数组（下标 0 不用），只在完整 flow 后有效，加边后须重算。保证非源汇流守恒，多余预流确实退回源点；不止计算最大预流值。e/g 只读，不支持外部改容量或限流。初始反向 BFS 标号，最高标号单链桶、当前弧和 gap；没有周期全局重标号。不可达点初高 n+1，仍须放流。head/next 是活跃点桶链表，不是 DFS 栈；本算法无需 DFS。给出保守 $O(n^2(n+m))$ 时间、O(n+m) 空间界；不据经验或 AC 声称线性。

```
struct HLPP
{
    using ll = long long;
    struct Edge
    {
        int from, to;
        ll cap, initial;
    };
    int n, s, t, top;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<int> h, count, cur, head, next;
    vector<ll> ex;

    HLPP(int n) : n(n), g(n + 1)
    {
        assert(n >= 2 && n < INT_MAX / 2);
    }

    int add(int u, int v, ll cap)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && cap >= 0);
        int id = e.size();
        e.push_back({u, v, cap, cap});
        e.push_back({v, u, 0, 0});
        g[u].push_back(id);
        g[v].push_back(id ^ 1);
        return id;
    }
```

活跃点、推流与 gap 重标号（接上页同一结构体）：

```
    void activate(int u)
    {
        if (u == s || u == t || !ex[u]) return;
        next[u] = head[h[u]];
        head[h[u]] = u;
        top = max(top, h[u]);
    }

    void push(int id, ll f)
    {
        auto &a = e[id];
        e[id ^ 1].cap += f;
        a.cap -= f;
        ex[a.from] -= f;
        bool empty = ex[a.to] == 0;
        ex[a.to] += f;
        if (empty) activate(a.to);
    }

    void relabel(int u)
    {
        int old = h[u];
        count[old]--;
        h[u] = 2 * n;
        for (int id : g[u])
            if (e[id].cap) h[u] = min(h[u], h[e[id].to] + 1);
        count[h[u]]++;
        cur[u] = 0;
        if (old >= n || count[old]) return;
        for (int v = 1; v <= n; v++)
        {
            if (v != s && h[v] > old && h[v] < n)
            {
                count[h[v]]--;
                h[v] = n + 1;
                count[h[v]]++;
                cur[v] = 0;
            }
        }
        fill(head.begin(), head.end(), -1);
        top = -1;
        for (int v = 1; v <= n; v++)
            if (v != u) activate(v);
    }
```

从零初始化与反向 BFS（接上页同一结构体）：

```
    // Recompute from zero, including after adding edges or changing terminals.
    ll flow(int source, int sink)
    {
        assert(1 <= source && source <= n && 1 <= sink && sink <= n);
        assert(source != sink);
        s = source;
        t = sink;
        for (auto &a : e) a.cap = a.initial;
        h.assign(n + 1, n + 1);
        cur.assign(n + 1, 0);
        ex.assign(n + 1, 0);
        next.assign(n + 1, -1);
        head.assign(2 * n + 1, -1);
        count.assign(2 * n + 1, 0);
        queue<int> q;
        h[t] = 0;
        q.push(t);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int id : g[u])
            {
                int v = e[id].to;
                if (e[id ^ 1].cap && h[v] == n + 1)
                {
                    h[v] = h[u] + 1;
                    q.push(v);
                }
            }
        }
        h[s] = n;
        for (int u = 1; u <= n; u++) count[h[u]]++;
        top = -1;
        for (int id : g[s])
            if (e[id].to != s && e[id].cap) push(id, e[id].cap);
```

最高标号放流（接上页同一结构体）：

```
        while (top >= 0)
        {
            if (head[top] == -1)
            {
                top--;
                continue;
            }
            int u = head[top];
            head[top] = next[u];
            while (ex[u])
            {
                if (cur[u] == int(g[u].size()))
                {
                    relabel(u);
                    continue;
                }
                int id = g[u][cur[u]];
                if (e[id].cap && h[u] == h[e[id].to] + 1)
                    push(id, min(ex[u], e[id].cap));
                else
                    cur[u]++;
            }
        }
        return ex[t];
    }
```

原边流量与最小割（接上页同一结构体）：

```
    ll used(int id) const
    {
        return e[id].initial - e[id].cap;
    }

    vector<int> min_cut() const
    {
        vector<int> seen(n + 1);
        queue<int> q;
        seen[s] = 1;
        q.push(s);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int id : g[u])
            {
                int v = e[id].to;
                if (e[id].cap && !seen[v])
                {
                    seen[v] = 1;
                    q.push(v);
                }
            }
        }
        return seen;
    }
};
```

预流不能冒充流方案 不可达汇点的活跃点仍须继续放流，直到多余流退回源点；否则虽然 ex[t] 可能已是最大流值，used 仍可能违反中间点流守恒。完成后，每对残量容量之和等于原容量，各原边流量合法；源侧残量可达集给出同值最小割。

桶与 gap head[h] 和 next[u] 是活跃点单链桶，不是 DFS 栈。点仅在多余流从 0 变正时加入，取出后放流至 0；gap 改变多个点高度后重建桶，并排除当前正在处理的点，避免重复入桶或旧高度记录。初始 BFS 只做一次，反向遍历必须看反边残量；没有周期全局重标号。

调用约定 flow 每次清空旧流并重算，因此允许换源汇；不返回增量，也不支持限流。used 只接收 add 返回的偶数边号，min_cut 只在完整求流后有效。容量总和不超过 LLONG_MAX，不能只保证最大流值未溢出而忽略初始化预流。

模板题使用：P4722 【模板】最大流加强版 / 预流推进

题意：n≤1200、m≤120000，有向容量，输出最大流；允许重边/反向边。

所需模板：HLPP（第 865 页，52.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4722

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    HLPP g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        g.add(u, v, c);
    }
    cout << g.flow(s, t) << '\n';
}
```

接口用法演示：API 最大流方案与最小割

演示约定：输入 n m s t 和 m 条边；输出总流量、每条原边流量，最后输出 n 位源侧标记。

所需模板：HLPP（第 865 页，52.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://www.luogu.com.cn/problem/P4722

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    HLPP g(n);
    vector<int> ids;
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long cap;
        cin >> u >> v >> cap;
        ids.push_back(g.add(u, v, cap));
    }
    cout << g.flow(s, t) << '\n';
    for (int id : ids) cout << g.used(id) << '\n';
    auto cut = g.min_cut();
    for (int u = 1; u <= n; u++) cout << cut[u];
    cout << '\n';
}
```

52.1.2 最大流与方案

Dinic(n)，点号 1..n；add(u,v,c) 返回偶数原边编号；flow(s,t,limit) 返回本次新增流；used(id) 查询原边净流，cut(s) 返回残量可达点集。允许重边、自环，容量非负，s≠t；重复求流须使用相同源汇。只有求尽最大流后 cut 才是最小割。一般 $O(V^2E)$，递归增广深度 O(V)。总流不得超过 long long 上限。get_edge(id) 返回 tuple(起点, 终点, 总容量, 当前流)，change_edge(id,cap,f) 修改总容量与流，要求 id 为 add 返回的偶数原边编号且 0≤f≤cap；正向残量变为 cap-f，反向为 f。仅保证此残量对一致，不自动维持所有顶点的流量守恒；继续 flow 前必须由调用方保证全局可行流及同源汇。增容且保留 f 合法，缩容低于已有流时不能只截断该边。要从零重算须将所有原边一起重置为零流。两个新接口为 O(1)，尚只有本地验证。可选容量缩放：按源码各处 Scaling 注释同步修改 BFS、DFS、递归调用和外层循环，不能只取消一处注释。阈值从不超过当前最大残量 mx 的最大二次幂降到 1，mx=0 直接返回；用 lim≤mx/2 再翻倍避免溢出。bfs/dfs 的 lim 默认 1，cut 仍检查全部正残量；flow 仍返回新增流且支持限流续流。缩放一般 O(VE log(U+1))，U 为本次初始最大残量；不保证比普通版快，大 INF 可能增加空阶段。

```
struct Dinic
{
    using ll = long long;

    struct Edge
    {
        int from, to;
        ll cap, initial;
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<int> dep, cur;

    Dinic(int n) : n(n), g(n + 1), dep(n + 1), cur(n + 1) {}

    int add(int u, int v, ll c)
    {
        assert(c >= 0);
        int id = (int)e.size();
        e.push_back({u, v, c, c});
        e.push_back({v, u, 0, 0});
        g[u].push_back(id);
        g[v].push_back(id + 1);
        return id;
    }
```

```
29  bool bfs(int s, int t) // Scaling: add parameter ll lim = 1.
30  {
31      fill(dep.begin(), dep.end(), -1);
32      queue<int> q;
33      dep[s] = 0;
34      q.push(s);
35      while (!q.empty())
36      {
37          int u = q.front();
38          q.pop();
39          for (int id : g[u])
40              // Scaling: replace e[id].cap with e[id].cap >= lim.
41              if (e[id].cap && dep[e[id].to] < 0)
42              {
43                  dep[e[id].to] = dep[u] + 1;
44                  q.push(e[id].to);
45              }
46      }
47      return dep[t] >= 0;
48  }
49
50  ll dfs(int u, int t, ll f) // Scaling: add parameter ll lim = 1.
51  {
52      if (u == t) return f;
53      for (int &i = cur[u]; i < (int)g[u].size(); i++)
54      {
55          int id = g[u][i], v = e[id].to;
56          // Scaling: replace !e[id].cap with e[id].cap < lim.
57          if (!e[id].cap || dep[v] != dep[u] + 1) continue;
58          ll d = dfs(v, t, min(f, e[id].cap)); // Scaling: pass lim last.
59          if (d)
60          {
61              e[id].cap -= d;
62              e[id ^ 1].cap += d;
63              return d;
64          }
65      }
66      return 0;
67  }
```

```
    // Returns additional flow; reuse only with the same source/sink.
    ll flow(int s, int t, ll limit = LLONG_MAX)
    {
        assert(s != t && limit >= 0);
        ll ans = 0;
        // Scaling: mx = maximum current residual cap over all e (both directions).
        // Scaling: if mx == 0, return 0.
        // Scaling: start ll lim = 1, double while lim <= mx / 2.
        // Scaling: wrap the whole while below in for (; lim; lim >>= 1).
        while (ans < limit && bfs(s, t)) // Scaling: pass lim last.
        {
            fill(cur.begin(), cur.end(), 0);
            while (ans < limit)
            {
                ll d = dfs(s, t, limit - ans); // Scaling: pass lim last.
                if (!d) break;
                ans += d;
            }
        }
        return ans;
    }

    ll used(int id) const { return e[id].initial - e[id].cap; }

    tuple<int, int, ll, ll> get_edge(int id) const
    {
        assert(0 <= id && id < (int)e.size() && !(id & 1));
        return {e[id].from, e[id].to, e[id].initial, used(id)};
    }

    // Changes one residual pair; caller must preserve global flow conservation.
    void change_edge(int id, ll cap, ll f)
    {
        assert(0 <= id && id < (int)e.size() && !(id & 1));
        assert(0 <= f && f <= cap);
        e[id].initial = cap;
        e[id].cap = cap - f;
        e[id ^ 1].initial = 0;
        e[id ^ 1].cap = f;
    }

    vector<int> cut(int s)
    {
        bfs(s, s);
        vector<int> ans;
        for (int u = 1; u <= n; u++)
            if (dep[u] >= 0) ans.push_back(u);
        return ans;
    }
};
```

模板题使用：Luogu P3376

题意：给定有向容量网络与源点、汇点，求最大流。输入点号为 1..n，平行边逐条加入。

所需模板：Dinic（第 871 页，52.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3376

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    Dinic g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        g.add(u, v, c);
    }
    cout << g.flow(s, t) << '\n';
}
```

接口用法演示：右侧容量二分图匹配与方案（接口演示）

演示约定：输入 n,m,e，m 个非负 long long 右容量，再输入 e 条左右各自 0-based 的可配对边。每个左点最多使用一次；输出最大配对数和对应 (u,v)。支持空侧、零容量、重边，单容量可达 LLONG_MAX；总流至多 n，无需累加容量。保存 add 边号，用 used 还原方案。一次性建图求解，不把 flow 的新增量误作重跑总量。

所需模板：Dinic（第 871 页，52.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：容量匹配来源与接口核验

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, e;
    cin >> n >> m >> e;
    int s = n + m + 1;
    int t = s + 1;
    Dinic g(t);
    for (int u = 0; u < n; u++)
        g.add(s, u + 1, 1);
    for (int v = 0; v < m; v++)
    {
        long long c;
        cin >> c;
        g.add(n + v + 1, t, c);
    }
    vector<array<int, 3>> edges;
    for (int i = 0; i < e; i++)
    {
        int u, v;
        cin >> u >> v;
        int id = g.add(u + 1, n + v + 1, 1);
        edges.push_back({u, v, id});
    }
    cout << g.flow(s, t) << '\n';
    for (auto [u, v, id] : edges)
        if (g.used(id))
            cout << u << ' ' << v << '\n';
}
```

应用题补充：Luogu P2763 / 试题库问题

题意：k 种题型、n 道题，给出每类需求和各题可属类别；每题最多选一次，输出恰好满足各类需求的方案，或 No Solution!。2≤k≤20，k≤n≤1000；不是独立匹配模板题。题号/类别均 1-based；源到题容量 1，题到类容量 1，类到汇为需求，满流即各类满足。need 逐项截到 n+1 再累加，超过 n 的需求必无解，可避免超大容量求和溢出。

所需模板：Dinic（第 871 页，52.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2763

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int k, n;
    cin >> k >> n;
    int s = n + k + 1;
    int t = s + 1;
    Dinic g(t);
    long long need = 0;
    for (int v = 1; v <= k; v++)
    {
        long long c;
        cin >> c;
        need += min(c, 1LL * n + 1);
        g.add(n + v, t, c);
    }
    vector<vector<pair<int, int>>> ids(k + 1);
    for (int u = 1; u <= n; u++)
    {
        g.add(s, u, 1);
        int p;
        cin >> p;
        while (p--)
        {
            int v;
            cin >> v;
            int id = g.add(u, n + v, 1);
            ids[v].push_back({u, id});
        }
    }
    if (g.flow(s, t) != need)
    {
        cout << "No Solution!\n";
        return 0;
    }
    for (int v = 1; v <= k; v++)
    {
        cout << v << ':';
        for (auto [u, id] : ids[v])
            if (g.used(id))
                cout << ' ' << u;
        cout << '\n';
    }
}
```

52.1.3 最大流中单位容量边的可行性与必选性

代码依赖：Dinic（第 871 页，52.1.2 节）、TarjanSCC（第 794 页，47.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

unit_flow_edges(g,ids) 在完整普通最大流的残量图上，查询每条指定边的流量是否在至少一个/每一个整数最大流中等于 1，返回 {possible,forced}。ids 必须是 Dinic.add 返回的原始偶数边号，且每条被查询边初始容量恰为 1；其他边允许任意非负 long long 容量，最大流值须可表示。结果保持查询顺序，支持重复编号。已使用或两端残量 SCC 相同则可行，已使用且不同 SCC 则必选。这里比较的是完整流量方案，允许不改变源汇净流量的环流；单位自环可以取 0 或 1，不应解释成必经的简单 s-t 路径。不是最小割边分类，也不把容量大于 1 的边强行按布尔流处理。O(n+m+ 查询数) 时间与空间，Tarjan 保留递归；分析不修改 g。

```
vector<array<bool, 2>> unit_flow_edges(const Dinic &g, const vector<int> &ids)
{
    TarjanSCC d(g.n);
    for (auto e : g.e)
        if (e.cap) d.add(e.from, e.to);
    d.run();
    vector<array<bool, 2>> ans;
    for (int id : ids)
    {
        assert(0 <= id && id < (int)g.e.size() && id % 2 == 0);
        auto e = g.e[id];
        assert(e.initial == 1);
        bool used = g.used(id) == 1, same = d.bel[e.from] == d.bel[e.to];
        ans.push_back({used || same, used && !same});
    }
    return ans;
}
```

应用题补充：最大流单位边 Y/N/?（本地接口示例）

题意：输入 n、m、s、t、q，随后 m 条有向容量边和 q 个原边序号；被询问边容量须为 1。先输出最大流值，再输出 Y（所有方案取 1）、N（没有方案取 1）、?（两种都可能）。用 add 返回值保存残量边号。本地示例对应附件 2.14 的分类内核，不代替原比赛题的排列约束建图。

所需模板：Dinic（第 871 页，52.1.2 节）、TarjanSCC（第 794 页，47.1.1 节）、unit_flow_edges（第 877 页，52.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://github.com/gsh20040816/CPCtemplate/issues/11

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t, q;
    cin >> n >> m >> s >> t >> q;
    Dinic g(n);
    vector<int> edge(m), ids(q);
    for (int &id : edge)
    {
        int u, v;
        long long cap;
        cin >> u >> v >> cap;
        id = g.add(u, v, cap);
    }
    for (int &id : ids)
    {
        int i;
        cin >> i;
        id = edge[i - 1];
    }
    cout << g.flow(s, t) << '\n';
    for (auto [possible, forced] : unit_flow_edges(g, ids))
        cout << (forced ? 'Y' : possible ? '?' : 'N');
    cout << '\n';
}
```

应用题补充：QOJ 10424 / NERC 2024 K

题意：两套排列各给出同一 k 元素集合的部分子序列，-1 表示未知；输出每个元素必选 Y、必不选 N 或可选?，无解输出 Inconsistent。已知元素切分两侧容量组，每个元素对应一条单位边；最大流必须等于 k，再作残量 SCC 分类。n 及所有测试的 n 之和至 2000。比赛应用，不计作独立模板题覆盖。

所需模板：Dinic（第 871 页，52.1.2 节）、TarjanSCC（第 794 页，47.1.1 节）、unit_flow_edges（第 877 页，52.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://qoj.ac/problem/10424

```
bool blocks(const vector<int> &p,
            const vector<int> &s,
            vector<int> &id,
            vector<int> &cap)
{
    int n = p.size(), k = s.size(), last = -1, prev = -1;
    vector<int> pos(n + 1);
    for (int i = 0; i < n; i++) pos[p[i]] = i;
    id.assign(n + 1, -1);
    cap.clear();
    for (int i = 0; i <= k; i++)
    {
        if (i < k && s[i] == -1) continue;
        int at = i == k ? n : pos[s[i]];
        if (at <= last || at - last - 1 < i - prev - 1) return false;
        int g = cap.size();
        cap.push_back(i - prev - 1);
        for (int j = last + 1; j < at; j++) id[p[j]] = g;
        if (i < k)
        {
            id[s[i]] = cap.size();
            cap.push_back(1);
        }
        last = at;
        prev = i;
    }
    return true;
}
```

使用入口（接上页配置函数）：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n, k;
        cin >> n;
        vector<int> p(n), q(n);
        for (int &x : p) cin >> x;
        for (int &x : q) cin >> x;
        cin >> k;
        vector<int> a(k), b(k), l, r, x, y;
        for (int &v : a) cin >> v;
        for (int &v : b) cin >> v;
        if (!blocks(p, a, l, x) || !blocks(q, b, r, y))
        {
            cout << "Inconsistent\n";
            continue;
        }
        int m = x.size(), source = m + y.size() + 1, sink = source + 1;
        Dinic g(sink);
        for (int i = 0; i < m; i++) g.add(source, i + 1, x[i]);
        for (int i = 0; i < (int)y.size(); i++) g.add(m + i + 1, sink, y[i]);
        vector<int> ids;
        for (int v = 1; v <= n; v++) ids.push_back(g.add(l[v] + 1, m + r[v] + 1, 1));
        if (g.flow(source, sink) != k)
        {
            cout << "Inconsistent\n";
            continue;
        }
        for (auto [possible, forced] : unit_flow_edges(g, ids))
            cout << (forced ? 'Y' : possible ? '?' : 'N');
        cout << '\n';
    }
}
```

52.1.4 递归 ISAP 最大流

Isap(n)，点号 1..n；add(u,v,cap) 返回偶数原边编号，used(id) 读取流量；flow(s,t,limit) 返回本次新增流，默认 LLONG_MAX，重复调用保持相同源汇。cut(s) 返回残量可达点集，只有求尽后才是最小割。容量非负，总最大流 ≤LLONG_MAX，允许自环、重边与反向原边；重建对象从零重算。e[id] 含 from/to/cap/initial，其中 cap 是残量容量；e/g 可读，任意改动须维护全局可行流。反向残量 BFS 初始化距离，当前弧、重标号和 gap 剪枝，DFS 保持递归，最坏 $O(V^2E)$，空间 O(V+E)、递归深度 O(V)。bfs/dfs 为内部方法，不能用其临时标签代替 cut。与 Dinic 为独立备选，不承诺更快；两个方向有容量时分别 add，逐边方案不压成有符号净流。

```
struct Isap
{
    using ll = long long;

    struct Edge
    {
        int from, to;
        ll cap, initial;
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<int> dep, cur, gap;

    Isap(int n) : n(n), g(n + 1), dep(n + 1), cur(n + 1), gap(n + 1)
    {
    }

    int add(int u, int v, ll cap)
    {
        assert(cap >= 0);
        int id = (int)e.size();
        e.push_back({u, v, cap, cap});
        e.push_back({v, u, 0, 0});
        g[u].push_back(id);
        g[v].push_back(id ^ 1);
        return id;
    }

    ll used(int id) const
    {
        return e[id].initial - e[id].cap;
    }
```

接上页同一结构体：

```
    void bfs(int t)
    {
        fill(dep.begin(), dep.end(), n);
        fill(gap.begin(), gap.end(), 0);
        queue<int> q;
        dep[t] = 0;
        q.push(t);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int id : g[u])
            {
                int v = e[id].to;
                if (!e[id ^ 1].cap || dep[v] != n) continue;
                dep[v] = dep[u] + 1;
                q.push(v);
            }
        }
        for (int u = 1; u <= n; u++) gap[dep[u]]++;
    }

    ll dfs(int u, int s, int t, ll f)
    {
        if (u == t) return f;
        for (int &i = cur[u]; i < (int)g[u].size(); i++)
        {
            int id = g[u][i], v = e[id].to;
            if (!e[id].cap || dep[u] != dep[v] + 1) continue;
            ll take = dfs(v, s, t, min(f, e[id].cap));
            if (take)
            {
                e[id].cap -= take;
                e[id ^ 1].cap += take;
                return take;
            }
            if (dep[s] == n) return 0;
        }
        int old = dep[u], best = n;
        for (int id : g[u])
            if (e[id].cap) best = min(best, dep[e[id].to] + 1);
        dep[u] = best;
        cur[u] = 0;
        gap[best]++;
        if (--gap[old] == 0) dep[s] = n;
        return 0;
    }
```

接上页同一结构体：

```
    // Returns additional flow; reuse only with the same source/sink.
    ll flow(int s, int t, ll limit = LLONG_MAX)
    {
        assert(s != t && limit >= 0);
        bfs(t);
        fill(cur.begin(), cur.end(), 0);
        ll ans = 0;
        while (ans < limit && dep[s] < n)
            ans += dfs(s, s, t, limit - ans);
        return ans;
    }

    vector<int> cut(int s) const
    {
        vector<int> vis(n + 1), ans;
        queue<int> q;
        vis[s] = 1;
        q.push(s);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            ans.push_back(u);
            for (int id : g[u])
            {
                int v = e[id].to;
                if (!e[id].cap || vis[v]) continue;
                vis[v] = 1;
                q.push(v);
            }
        }
        return ans;
    }
};
```

模板题使用：Luogu P3376

题意：给定 n,m,s,t 和 1-based 有向容量边，输出最大流。n≤200、m≤5000、0≤ 容量 <2^31；s≠t，重边逐条加入。总流可能超 int，使用 long long。这里只验证正式题目 add/flow 入口，限流、方案及 cut 另有独立本地核验。

所需模板：Isap（第 881 页，52.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3376

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    Isap g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        g.add(u, v, c);
    }
    cout << g.flow(s, t) << '\n';
}
```

52.1.5 最大流方案唯一性

代码依赖：Dinic（第 871 页，52.1.2 节）、TarjanSCC（第 794 页，47.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

flow_unique(g) 只读检查 Dinic 已求尽的可行最大流，其原边流量向量是否唯一；源汇不同，容量非负整数，最大流可由 long long 表示。允许重边、自环、零容量和不连通图，包含独立环流；与最小割唯一性不同。残量 SCC 内按原边对各计一次，正容量自环亦计一条；某分量内部原边数不少于顶点数即多解，否则唯一。正反残量同时存在的同一边不算两条，不把无效的往返抵消视为新方案。复杂度 O(n+m)，空间 O(n+m)，依赖 Dinic、TarjanSCC，DFS 保留递归。不修改 g，代表方案仍由 used 读取；不返回第二个方案。须先求完流，不能拿尚未求完的网络当最大流结论。

```
bool flow_unique(const Dinic &g)
{
    TarjanSCC d(g.n);
    for (auto e : g.e)
        if (e.cap) d.add(e.from, e.to);
    d.run();
    vector<int> left(d.cnt + 1);
    for (int u = 1; u <= g.n; u++)
        left[d.bel[u]]++;
    for (int id = 0; id < (int)g.e.size(); id += 2)
    {
        auto e = g.e[id];
        if (!e.cap && !g.e[id ^ 1].cap) continue;
        if (d.bel[e.from] != d.bel[e.to]) continue;
        if (--left[d.bel[e.from]] == 0) return false;
    }
    return true;
}
```

接口用法演示：最大流方案唯一性（接口演示）

演示约定：输入 n,m,s,t 及 1-based 有向边 (u,v, 非负 long long 容量)，先求最大流，再输出流值、UNIQUE 或 MULTIPLE，以及按输入顺序的各原边流量。包括所有独立环流，重边分别计量；不是最小割唯一性。s≠t，总最大流 ≤LLONG_MAX。仅接口示例，尚无在线判题记录。

所需模板：Dinic（第 871 页，52.1.2 节）、TarjanSCC（第 794 页，47.1.1 节）、flow_unique（第 885 页，52.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：最大流唯一性的合同与证明

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    Dinic g(n);
    vector<int> ids;
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        ids.push_back(g.add(u, v, c));
    }
    cout << g.flow(s, t) << '\n';
    cout << (flow_unique(g) ? "UNIQUE" : "MULTIPLE") << '\n';
    for (int id : ids)
        cout << g.used(id) << '\n';
}
```

52.2 最小割

52.2.1 正容量最小割的可行边与必选边

代码依赖：Dinic（第 871 页，52.1.2 节）、TarjanSCC（第 794 页，47.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

mincut_edges(g,s,t) 只读分析 Dinic 已经完整求出的普通 s→t 最大流；不得传入限流提前停止或人为修改过的残量网络。所有原边初始容量必须严格为正，最大流值须可由 long long 表示。返回逐条原边的 {possible,forced}，对应 Dinic 原始偶数边号 id 的结果是 ans[id/2]。只有饱和边可能属于最小割：两端残量 SCC 不同则可行；起点与 s 同 SCC、终点与 t 同 SCC 则必选。支持平行边和自环，源汇不同，点号 1..g.n。O(n+m) 时间与空间，Tarjan 递归栈 O(n)。零容量边没有正流对应的反向残量边，不能套用本函数的同 SCC 简化条件。每次修改图或重新增广后都需重新分析。

```
vector<array<bool, 2>> mincut_edges(const Dinic &g, int s, int t)
{
    TarjanSCC d(g.n);
    for (auto e : g.e)
        if (e.cap) d.add(e.from, e.to);
    d.run();
    vector<array<bool, 2>> ans(g.e.size() / 2);
    for (int id = 0; id < (int)g.e.size(); id += 2)
    {
        auto e = g.e[id];
        assert(e.initial > 0);
        if (e.cap) continue;
        ans[id / 2][0] = d.bel[e.from] != d.bel[e.to];
        ans[id / 2][1] = d.bel[e.from] == d.bel[s] && d.bel[e.to] == d.bel[t];
    }
    return ans;
}
```

应用题补充：Luogu P4126 / AHOI2009 最小割

题意：求每条正容量有向边是否出现在至少一个/每一个最小割中。先不设额外限制地跑完最大流，再按原边输入顺序输出两项布尔值。容量用 long long；平行道路分别加边。此题源自 AHOI，作为应用补充单列。

所需模板：Dinic（第 871 页，52.1.2 节）、TarjanSCC（第 794 页，47.1.1 节）、mincut_edges（第 887 页，52.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4126

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    Dinic g(n);
    while (m--)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        g.add(u, v, c);
    }
    g.flow(s, t);
    for (auto [possible, forced] : mincut_edges(g, s, t))
        cout << possible << ' ' << forced << '\n';
}
```

52.2.2 最大权闭合子图与选点方案

代码依赖：Dinic（第 871 页，52.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

maximum_closure(weight,dependencies) 返回 (最大权值, 选中点列表), weight 为 0-based vector<long long>, 图中点号 1..n。依赖边 (u,v) 表示选 u 必须选 v, 允许环、自环与重边; 允许空选集, 因此答案不小于 0。依赖 flow.hpp 中的 Dinic。正权连源、负权连汇，依赖边容量取正权总和加 1；要求所有正权之和严格小于 LLONG_MAX。负权先提升到 int128 再取负，并截断到依赖边容量，因此 LLONG_MIN 也可处理。返回残量图源点可达的原顶点，权值为正权总和减最小割；超大负权边不会出现在最优割中，截断不改变原问题最优值。每次调用重建网络。复杂度沿用 Dinic：一般 $O(V^2E)$，空间 O(V+E)，增广保持递归。返回一种最优闭合集，不承诺最大或最小选点个数。

```
pair<long long, vector<int>> maximum_closure(const vector<long long> &weight,
                                             const vector<pair<int, int>> &dependencies)
{
    using I = __int128_t;
    int n = weight.size(), s = n + 1, t = n + 2;
    I positive = 0;
    for (long long w : weight)
        if (w > 0) positive += w;
    assert(positive < LLONG_MAX);
    long long inf = (long long)positive + 1;
    Dinic graph(n + 2);
    for (int u = 1; u <= n; u++)
    {
        long long w = weight[u - 1];
        if (w > 0)
            graph.add(s, u, w);
        else if (w < 0)
            graph.add(u, t, (long long)min(-I(w), I(inf)));
    }
    for (auto [u, v] : dependencies)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        graph.add(u, v, inf);
    }
    long long answer = (long long)positive - graph.flow(s, t);
    vector<int> selected;
    for (int u : graph.cut(s))
        if (u <= n) selected.push_back(u);
    return {answer, selected};
}
```

应用题补充：Luogu P2762 太空飞行计划问题

题意：选实验获得收益，配置仪器支付费用，输出最优实验/仪器编号和净收益。m、n 至 50，单项费用为正且小于 2^{31}，累计用 long long。逐行读取变长依赖，istringstream 兼容 CRLF；实验 i 依赖仪器 m+j。应用用法，不计非比赛模板覆盖。

所需模板：Dinic（第 871 页，52.1.2 节）、maximum_closure（第 889 页，52.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2762

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m, n;
    cin >> m >> n;
    vector<long long> w(m + n);
    vector<pair<int, int>> dep;
    string line;
    getline(cin, line);
    for (int i = 1; i <= m; i++)
    {
        getline(cin, line);
        istringstream in(line);
        in >> w[i - 1];
        int j;
        while (in >> j) dep.push_back({i, m + j});
    }
    for (int j = 0; j < n; j++)
    {
        long long cost;
        cin >> cost;
        w[m + j] = -cost;
    }
    auto [best, selected] = maximum_closure(w, dep);
    for (int u : selected)
        if (u <= m) cout << u << ' ';
    cout << '\n';
    for (int u : selected)
        if (u > m) cout << u - m << ' ';
    cout << '\n' << best << '\n';
}
```

52.2.3 Gomory–Hu 最小割树

代码依赖：Dinic（第 871 页，52.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

gomory_hu(n,edges) 返回 n-1 条 (u,v, 容量) 树边；n≥1，原图点号 0..n-1，输入为无向边 tuple 列表，重边保留、自环忽略、容量非负。非自环边总容量须严格小于 LLONG_MAX。两点最小割是树路径最小边权；删去树边得到的两侧也是原图该边端点的最小割。允许非连通图和零权树边，n=1 返回空树。运行 n-1 次 Dinic，一般上界 $O(n^3m+n^2)$，额外空间 O(n+m)，DFS 增广保持递归。P4897 输入 n 表示 n+1 个点，驱动传 n+1。双版本 P4897 已 AC，最长单点分别 252、185 ms。

```
vector<tuple<int, int, long long>>
gomory_hu(int n, const vector<tuple<int, int, long long>> &edges)
{
    assert(n >= 1);
    vector<int> parent(n);
    vector<long long> value(n);
    for (int s = 1; s < n; s++)
    {
        int t = parent[s];
        Dinic flow(n);
        for (auto [u, v, w] : edges)
            if (u != v)
            {
                flow.add(u + 1, v + 1, w);
                flow.add(v + 1, u + 1, w);
            }
        long long cut = flow.flow(s + 1, t + 1);
        vector<bool> side(n);
        for (int u : flow.cut(s + 1)) side[u - 1] = true;
        for (int v = 1; v < n; v++)
            if (v != s && parent[v] == t && side[v]) parent[v] = s;
        if (side[parent[t]])
        {
            parent[s] = parent[t];
            parent[t] = s;
            value[s] = value[t];
            value[t] = cut;
        }
        else
            value[s] = cut;
    }
    vector<tuple<int, int, long long>> tree;
    for (int v = 1; v < n; v++) tree.push_back({v, parent[v], value[v]});
    return tree;
}
```

模板题使用：Luogu P4897

题意：给无向图，多次询问不同两点的最小割。点号为 0..n，因此建树传 n+1。再预处理所有点对的答案，查询矩阵额外占 O(n^2) 时间和空间。

所需模板：Dinic（第 871 页，52.1.2 节）、gomory_hu（第 891 页，52.2.3 节）、cut_tree_values（第 893 页，52.2.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4897

```
int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    vector<tuple<int, int, long long>> edges;
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        scanf("%d%d%lld", &u, &v, &w);
        edges.push_back({u, v, w});
    }
    auto tree = gomory_hu(n + 1, edges);
    auto answer = cut_tree_values(n + 1, tree);
    int q;
    scanf("%d", &q);
    while (q--)
    {
        int u, v;
        scanf("%d%d", &u, &v);
        printf("%lld\n", answer[u][v]);
    }
    return 0;
}
```

52.2.4 最小割树全点对查询

cut_tree_values(n,tree) 对合法的连通无环加权树计算全点对路径最小边权；点号 0..n-1、n≥1、恰有 n-1 条边，边权为非负 long long。输入可直接使用 gomory_hu 的返回值。结果 ans[u][v] 在 u≠v 时为两点最小割；对角线为 LLONG_MAX 哨兵，不能解释成同点最小割。n=1 返回单个哨兵。$O(n^2)$ 时间与空间，递归 DFS 深度 O(n)，查询 O(1)。只需要标准库，不依赖流模板；不是任意图的查询函数。

```
vector<vector<long long>>
cut_tree_values(int n, const vector<tuple<int, int, long long>> &tree)
{
    assert(n >= 1 && tree.size() == (size_t)n - 1);
    vector<vector<pair<int, long long>>> g(n);
    for (auto [u, v, w] : tree)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && w >= 0);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
    vector<vector<long long>> answer(n, vector<long long>(n, LLONG_MAX));
    for (int s = 0; s < n; s++)
    {
        auto dfs = [&](auto &&self, int u, int parent, long long value) -> void
        {
            answer[s][u] = value;
            for (auto [v, w] : g[u])
                if (v != parent) self(self, v, u, min(value, w));
        };
        dfs(dfs, s, -1, LLONG_MAX);
    }
    return answer;
}
```

联合使用示例见第 892 页（Luogu P4897），其中直接调用本模板。

52.3 费用流

52.3.1 势能最小费用最大流

MinCostFlow(n),点号 1..n;add(u,v,cap,cost) 返回边编号,flow(s,t,limit) 返回新增流量及 128 位费用,used(id) 返回原边流量。允许负费用，初始残量网络必须没有负环；检测到负环抛异常。Bellman-Ford 初始化 O(VE)，每次增广 O(E log V)；增广次数不能简单假设为 O(V)。另要求 $\sum(\text{cap}\times|\text{cost}|)<2^{120}$，防止总费用超过 int128。slope(s,t,limit) 返回从 (0,0) 开始的本次新增流量/费用折点，相邻同斜率段合并，费用为 int128。无增广时只返回起点；每个整数流量的费用可在线性段内插值。重复调用仍用同源汇，返回值不是历史累计值。flow 不存储曲线；slope 额外空间为折点数。内部 run 不是用户接口。新曲线接口仅本地验证，P3381 驱动只输出终点，不能据此验证全部中间折点。

```
struct MinCostFlow
{
    using ll = long long;
    using i128 = __int128_t;

    struct Edge
    {
        int from, to;
        ll cap, cost, initial;
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> g;

    MinCostFlow(int n) : n(n), g(n + 1) {}

    int add(int u, int v, ll cap, ll cost)
    {
        assert(cap >= 0 && cost != LLONG_MIN);
        int id = (int)e.size();
        e.push_back({u, v, cap, cost, cap});
        e.push_back({v, u, 0, -cost, 0});
        g[u].push_back(id);
        g[v].push_back(id + 1);
        return id;
    }

    ll used(int id) const { return e[id].initial - e[id].cap; }
```

```
    // Negative costs allowed; initial residual graph must have no negative cycle.
    pair<ll, i128> run(int s, int t, ll limit, vector<pair<ll, i128>> *curve)
    {
        assert(s != t && limit >= 0);
        const i128 inf = i128(1) << 120;
        vector<i128> h(n + 1, 0), d(n + 1);
        vector<int> pre(n + 1);
        // Super-source Bellman-Ford: feasible potentials for every vertex.
        for (int it = 0; it < n; it++)
        {
            bool changed = false;
            for (auto &a : e)
                if (a.cap && h[a.to] > h[a.from] + a.cost)
                {
                    h[a.to] = h[a.from] + a.cost;
                    changed = true;
                }
            if (!changed) break;
            if (it == n - 1) throw invalid_argument("negative cycle");
        }
        ll f = 0;
        i128 cost = 0;
        optional<i128> last;
        while (f < limit)
        {
            fill(d.begin(), d.end(), inf);
            d[s] = 0;
            priority_queue<pair<i128, int>,
                           vector<pair<i128, int>>,
                           greater<pair<i128, int>>>
                q;
            q.push({0, s});
            while (!q.empty())
            {
                auto [du, u] = q.top();
                q.pop();
                if (du != d[u]) continue;
                for (int id : g[u])
                {
                    auto &a = e[id];
                    i128 nd = du + a.cost + h[u] - h[a.to];
                    if (a.cap && nd < d[a.to])
                    {
                        d[a.to] = nd;
                        pre[a.to] = id;
                        q.push({nd, a.to});
                    }
                }
            }
```

```
            if (d[t] == inf) break;
            for (int u = 1; u <= n; u++)
                if (d[u] != inf) h[u] += d[u];
            ll take = limit - f;
            for (int u = t; u != s; u = e[pre[u]].from) take = min(take, e[pre[u]].cap);
            for (int u = t; u != s; u = e[pre[u]].from)
            {
                int id = pre[u];
                e[id].cap -= take;
                e[id ^ 1].cap += take;
                cost += i128(take) * e[id].cost;
            }
            f += take;
            if (curve)
            {
                i128 unit = h[t] - h[s];
                if (last && *last == unit) curve->pop_back();
                curve->push_back({f, cost});
                last = unit;
            }
        }
        return {f, cost};
    }

    pair<ll, i128> flow(int s, int t, ll limit = LLONG_MAX)
    {
        return run(s, t, limit, nullptr);
    }

    vector<pair<ll, i128>> slope(int s, int t, ll limit = LLONG_MAX)
    {
        vector<pair<ll, i128>> curve{{0, 0}};
        run(s, t, limit, &curve);
        return curve;
    }
};
```

模板题使用：Luogu P3381

题意：给有向容量费用网络，求最大流及对应最小费用。点号 1..n，add 参数依次为起点、终点、容量、单位费用。题目保证费用答案不超过 2^31-1，因此这里可将返回的 int128 转为 long long；一般题目不能照搬转换。

所需模板：MinCostFlow（第 894 页，52.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3381

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    MinCostFlow graph(n);
    while (m--)
    {
        int u, v;
        long long capacity, cost;
        cin >> u >> v >> capacity >> cost;
        graph.add(u, v, capacity, cost);
    }
    auto [flow, cost] = graph.flow(s, t);
    cout << flow << ' ' << (long long)cost << '\n';
}
```

52.3.2 SPFA 最小费用最大流

SpfaFlow(n)，点号 1..n；add(u,v,cap,cost) 返回偶数原边编号，used(id) 查询原边流量。flow(s,t,limit) 返回本次新增流量及 int128 费用；默认 limit=LLONG_MAX，s≠t。允许重边、自环、零容量和负费用，调用前残量网络不得有负环；先用全点入队 SPFA 检查全图，含不可达负环也抛 invalid_argument，检查阶段不修改流量。重复调用须保持同源汇，返回值不累计历史。容量非负、费用不能是 LLONG_MIN，总流 ≤LLONG_MAX，$\sum(\text{cap}\times|\text{cost}|)<2^{120}$。每次增广用 SPFA 最短路，复杂度 O((A+1)VE)，A 为实际增广次数，不能假设等于点数；空间 O(V+E)。通常优先势能 Dijkstra，本版作为更直接的 SPFA 备选。e/g 可读，不能任意修改残量；shortest 为内部方法，0 仅是全点初始化标记，不是合法顶点。

```
struct SpfaFlow
{
    using ll = long long;
    using I = __int128_t;

    struct Edge
    {
        int from, to;
        ll cap, cost, initial;
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> g;

    SpfaFlow(int n) : n(n), g(n + 1) {}

    int add(int u, int v, ll cap, ll cost)
    {
        assert(cap >= 0 && cost != LLONG_MIN);
        int id = (int)e.size();
        e.push_back({u, v, cap, cost, cap});
        e.push_back({v, u, 0, -cost, 0});
        g[u].push_back(id);
        g[v].push_back(id + 1);
        return id;
    }

    ll used(int id) const { return e[id].initial - e[id].cap; }
```

接上页同一结构体：

```
    // Internal: s=0 seeds every vertex to detect any residual negative cycle.
    void shortest(int s, vector<I> &d, vector<int> &pre)
    {
        const I inf = I(1) << 120;
        d.assign(n + 1, inf);
        pre.assign(n + 1, -1);
        vector<int> len(n + 1), in(n + 1);
        queue<int> q;
        for (int u = 1; u <= n; u++)
            if (!s || u == s)
            {
                d[u] = 0;
                in[u] = 1;
                q.push(u);
            }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            in[u] = 0;
            for (int id : g[u])
            {
                auto &a = e[id];
                if (!a.cap || d[a.to] <= d[u] + a.cost) continue;
                d[a.to] = d[u] + a.cost;
                pre[a.to] = id;
                len[a.to] = len[u] + 1;
                if (len[a.to] >= n) throw invalid_argument("negative cycle");
                if (!in[a.to])
                {
                    in[a.to] = 1;
                    q.push(a.to);
                }
            }
        }
    }
```

接上页同一结构体：

```
    // Returns additional flow/cost; reuse only with the same source/sink.
    pair<ll, I> flow(int s, int t, ll limit = LLONG_MAX)
    {
        assert(s != t && limit >= 0);
        vector<I> d;
        vector<int> pre;
        shortest(0, d, pre);
        ll f = 0;
        I cost = 0;
        while (f < limit)
        {
            shortest(s, d, pre);
            if (pre[t] < 0) break;
            ll take = limit - f;
            for (int u = t; u != s; u = e[pre[u]].from) take = min(take, e[pre[u]].cap);
            for (int u = t; u != s; u = e[pre[u]].from)
            {
                int id = pre[u];
                e[id].cap -= take;
                e[id ^ 1].cap += take;
            }
            f += take;
            cost += I(take) * d[t];
        }
        return {f, cost};
    }
};
```

模板题使用：Luogu P3381

题意：求最小费用最大流。点号 1..n，输入容量与费用均为 0..1000，最多 5000 点、50000 边；本题保证最大流及最小费用 $\leq 2^{31}$-1，可把返回费用转 long long 输出。这里实际使用 SPFA 增广，通用接口允许负边但禁止负环；有负环的费用流须用另一套算法。

所需模板：SpfaFlow（第 898 页，52.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3381

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    SpfaFlow g(n);
    while (m--)
    {
        int u, v;
        long long cap, cost;
        cin >> u >> v >> cap >> cost;
        g.add(u, v, cap, cost);
    }
    auto [flow, cost] = g.flow(s, t);
    cout << flow << ' ' << (long long)cost << '\n';
}
```

52.3.3 允许负环的最小费用最大流

代码依赖：MinCostFlow（第 894 页，52.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

NegativeCostFlow(n)，点号 1..n；add(u,v,cap,cost) 返回从 0 开始的原边编号。容量非负，所有原边容量总和 ≤LLONG_MAX；费用为 long long 且不等于 LLONG_MIN。允许重边、自环及任意位置的负环。solve(s, t, limit) 只调用一次，limit 默认 LLONG_MAX，s≠t、limit≥0，返回 (不超过 limit 的最大流量, 该流量下的最小费用)，费用为 int128。limit=0 可求最小费用循环流；不足 limit 时返回实际最大流。成功后 used(id) 返回原边流量，自动还原负边退流。先满流负费用边并建立反向正费用退流边，再用超级源汇修复平衡；初始残量费用非负，修复后无负环。删除辅助边后继续最短增广，包含不经过源汇的负环收益。依赖 MinCostFlow，使用两次势能预处理；若总增广次数为 K，时间 O(VE + K(E log(E+1) + V))，空间 O(V+E)，整数容量下为伪多项式，不是费用缩放算法。节点和残量边数须可由 int 表示。

```
struct NegativeCostFlow
{
    using ll = long long;
    using I = __int128_t;
    int n;
    MinCostFlow g;
    vector<ll> balance, cap;
    vector<int> ids;
    vector<bool> reversed;
    ll total = 0;
    I cost = 0;
    bool solved = false;

    NegativeCostFlow(int n) : n(n), g(n + 2), balance(n + 1) {}

    int add(int u, int v, ll c, ll w)
    {
        assert(!solved && 1 <= u && u <= n && 1 <= v && v <= n);
        assert(0 <= c && c <= LLONG_MAX - total && w != LLONG_MIN);
        total += c;
        int id = ids.size();
        cap.push_back(c);
        reversed.push_back(w < 0);
        if (w < 0)
        {
            balance[u] -= c;
            balance[v] += c;
            cost += I(c) * w;
            ids.push_back(g.add(v, u, c, -w));
        }
        else
            ids.push_back(g.add(u, v, c, w));
        return id;
    }
```

需求修复、增广与原边流量还原（接上页同一结构体）：

```
    pair<ll, I> solve(int s, int t, ll limit = LLONG_MAX)
    {
        assert(!solved && 1 <= s && s <= n && 1 <= t && t <= n && s != t);
        assert(limit >= 0);
        solved = true;
        int first = g.e.size(), back = g.add(t, s, min(total, limit), 0);
        ll need = 0;
        for (int u = 1; u <= n; u++)
            if (balance[u] > 0)
            {
                g.add(n + 1, u, balance[u], 0);
                need += balance[u];
            }
            else if (balance[u] < 0)
                g.add(u, n + 2, -balance[u], 0);
        auto [sent, correction] = g.flow(n + 1, n + 2, need);
        assert(sent == need);
        ll base = g.used(back);
        cost += correction;
        for (int id = first; id < (int)g.e.size(); id++) g.e[id].cap = 0;
        auto [more, extra] = g.flow(s, t, limit - base);
        cost += extra;
        return {base + more, cost};
    }

    ll used(int id) const
    {
        assert(solved && 0 <= id && id < (int)ids.size());
        ll f = g.used(ids[id]);
        return reversed[id] ? cap[id] - f : f;
    }
};
```

模板题使用：Luogu P7173

题意：求允许负费用环的最小费用最大流，独立于源汇的负环也计入费用。无额外流量上限时调用 solve(s,t)。题目容量和费用界保证最终费用可转为 long long；一般模板返回 int128，不能任意照搬此转换。

所需模板：MinCostFlow（第 894 页，52.3.1 节）、NegativeCostFlow（第 902 页，52.3.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P7173

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    NegativeCostFlow g(n);
    while (m--)
    {
        int u, v;
        long long cap, cost;
        cin >> u >> v >> cap >> cost;
        g.add(u, v, cap, cost);
    }
    auto [flow, cost] = g.solve(s, t);
    cout << flow << ' ' << (long long)cost << '\n';
}
```

52.3.4 zkw 最小费用最大流

ZkwFlow(n)，点号 1..n，s≠t。add(u,v,cap,cost) 返回偶数原边编号，used(id) 查询原边流量；flow(s,t,limit) 返回本次新增流量及 int128 费用，默认 limit=LLONG_MAX，重复调用保持同源汇。允许重边、自环、零容量和负费用，但调用前残量图不得含负环；全图 Bellman-Ford 建立可行势并检查不可达负环，失败抛 invalid_argument 且不改流量，limit=0 仍检查。容量非负、cost≠LLONG_MIN，总流 ≤LLONG_MAX，$\sum(\text{cap}\times|\text{cost}|)<2^{120}$。递归沿零约化费用边增广，失败后按可达集最小跨边松弛量调势。实际增广 A 次，O((A+1)V(V+E))，空间 O(V+E)、递归深度 O(V)，不保证快于势能 Dijkstra 或 SPFA。init/dfs/relabel 为内部方法；e/g 可读，任意改动须维持成对残量与全局最小费用可行流。无负环前提下加边后续流也须重新满足此条件。返回顺序为流量、费用，与 kuangbin 原稿相反。

```
struct ZkwFlow
{
    using ll = long long;
    using I = __int128_t;

    struct Edge
    {
        int from, to;
        ll cap, cost, initial;
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<I> h;
    vector<int> vis;

    ZkwFlow(int n) : n(n), g(n + 1), h(n + 1), vis(n + 1)
    {
    }

    int add(int u, int v, ll cap, ll cost)
    {
        assert(cap >= 0 && cost != LLONG_MIN);
        int id = (int)e.size();
        e.push_back({u, v, cap, cost, cap});
        e.push_back({v, u, 0, -cost, 0});
        g[u].push_back(id);
        g[v].push_back(id ^ 1);
        return id;
    }

    ll used(int id) const
    {
        return e[id].initial - e[id].cap;
    }
```

接上页同一结构体：

```
    // Internal: a feasible potential on every residual component.
    void init()
    {
        fill(h.begin(), h.end(), 0);
        for (int it = 0; it < n; it++)
        {
            bool changed = false;
            for (auto a : e)
                if (a.cap && h[a.to] > h[a.from] + a.cost)
                {
                    h[a.to] = h[a.from] + a.cost;
                    changed = true;
                }
            if (!changed) return;
            if (it == n - 1) throw invalid_argument("negative cycle");
        }
    }

    ll dfs(int u, int t, ll f)
    {
        if (u == t) return f;
        vis[u] = 1;
        for (int id : g[u])
        {
            int v = e[id].to;
            if (!e[id].cap || vis[v]) continue;
            if (I(e[id].cost) + h[u] != h[v]) continue;
            ll take = dfs(v, t, min(f, e[id].cap));
            if (!take) continue;
            e[id].cap -= take;
            e[id ^ 1].cap += take;
            return take;
        }
        return 0;
    }
```

接上页同一结构体：

```
    bool relabel()
    {
        const I inf = I(1) << 120;
        I d = inf;
        for (auto a : e)
            if (a.cap && vis[a.from] && !vis[a.to])
                d = min(d, I(a.cost) + h[a.from] - h[a.to]);
        if (d == inf) return false;
        for (int u = 1; u <= n; u++)
            if (vis[u]) h[u] -= d;
        return true;
    }

    // Returns additional flow/cost; reuse only with the same source/sink.
    pair<ll, I> flow(int s, int t, ll limit = LLONG_MAX)
    {
        assert(s != t && limit >= 0);
        init();
        ll f = 0;
        I cost = 0;
        while (f < limit)
        {
            fill(vis.begin(), vis.end(), 0);
            ll take = dfs(s, t, limit - f);
            if (take)
            {
                f += take;
                cost += I(take) * (h[t] - h[s]);
            }
            else if (!relabel()) break;
        }
        return {f, cost};
    }
};
```

模板题使用：Luogu P3381

题意：有向网络，输入 n,m,s,t 和每条边的起终点、容量、单位费用，输出最大流及其最小费用。点号 1..n，s≠t；n≤5000、m≤50000、容量/费用均 0..1000 且无自环，题面保证流量和费用 ≤2^31−1，因此此驱动可将 int128 费用转换输出。负边、续流与 used 由独立本地测试覆盖，不把它们算作本题验证范围。

所需模板：ZkwFlow（第 905 页，52.3.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3381

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    ZkwFlow g(n);
    while (m--)
    {
        int u, v;
        long long cap, cost;
        cin >> u >> v >> cap >> cost;
        g.add(u, v, cap, cost);
    }
    auto [flow, cost] = g.flow(s, t);
    cout << flow << ' ' << (long long)cost << '\n';
}
```

52.4 上下界网络流

52.4.1 上下界可行循环流

代码依赖：Dinic（第 871 页，52.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

BoundedCirculation(n)，点号 1..n。add(u,v,lo,hi) 返回原边编号，0≤lo≤hi≤LLONG_MAX，所有下界总和须 ≤LLONG_MAX。solve() 仅调用一次，返回可行性；成功后 used(id) 读取边流量。每点要求流量守恒，属于循环流；带源汇最大流用 BoundedMaxFlow。hi-lo 是残量容量，下界引起 balance；各边上界不必求和，只要求下界总和保证 balance 和超级源总需求不溢出。依赖 Dinic。

```
struct BoundedCirculation
{
    using ll = long long;
    int n;
    Dinic g;
    vector<ll> balance, lower;
    vector<int> ids;
    bool solved = false, feasible = false;

    BoundedCirculation(int n) : n(n), g(n + 2), balance(n + 1) {}

    int add(int u, int v, ll lo, ll hi)
    {
        assert(!solved && 0 <= lo && lo <= hi);
        int id = ids.size();
        ids.push_back(g.add(u, v, hi - lo));
        lower.push_back(lo);
        balance[u] -= lo;
        balance[v] += lo;
        return id;
    }

    bool solve()
    {
        assert(!solved);
        solved = true;
        ll need = 0;
        for (int u = 1; u <= n; u++)
            if (balance[u] > 0)
            {
                g.add(n + 1, u, balance[u]);
                need += balance[u];
            }
            else if (balance[u] < 0)
                g.add(u, n + 2, -balance[u]);
        return feasible = (g.flow(n + 1, n + 2) == need);
    }

    ll used(int id) const
    {
        assert(feasible);
        return lower[id] + g.used(ids[id]);
    }
};
```

模板题使用：LibreOJ 115

题意：无源汇有上下界可行流：每个点流入等于流出，按输入顺序输出每条边的合法流量，无解输出 NO。点号 1..n，n 不超过 200、m 不超过 10200，0≤lo≤hi<3000。add 返回 0 起原始边编号，used(i) 不是 Dinic 残量边号；solve 只能调用一次，且只有可行时才可读取 used。自环、重边逐条保留，不添加源汇回边。

所需模板：Dinic（第 871 页，52.1.2 节）、BoundedCirculation（第 909 页，52.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://loj.ac/p/115

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    BoundedCirculation graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long lo, hi;
        cin >> u >> v >> lo >> hi;
        graph.add(u, v, lo, hi);
    }
    if (!graph.solve())
    {
        cout << "NO\n";
        return 0;
    }
    cout << "YES\n";
    for (int i = 0; i < m; i++) cout << graph.used(i) << '\n';
}
```

52.4.2 有源汇上下界最值流与边流量

代码依赖：BoundedCirculation（第 909 页，52.4.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

BoundedMaxFlow(n)，点号 1..n；add(u,v,lo,hi) 返回从 0 开始的原边编号，0≤lo≤hi，所有原边上界总和≤LLONG_MAX。solve(s,t) 求最大净流，minimum(s,t) 求最小净流；同一对象只能任选其一调用一次，s≠t。minimum 交换源汇求最大值后取反，原边方向及 used 不变；返回 optional<long long>，nullopt 表示无可行流。目标为源点流出减流入的净流量，两种查询都允许负答案，不能把 -1 直接当无解；minimum 不额外强制净流量非负。只在成功后读取 used(id) 的原边方案。先加 t→s 与 s→t 两条辅助边兼容负净流，再求可行循环流；保存净值后清空全部辅助残量边，最后在原图从 s 到 t 增广。最优值与增加量均受原边上界总和控制。允许自环、重边；构建和清理 O(n+m)，主体为两次 Dinic，额外空间 O(n+m)。依赖 BoundedCirculation 与 Dinic，二者内部字段只读，不应在求解后再加边或改流。

```
struct BoundedMaxFlow
{
    using ll = long long;
    BoundedCirculation b;
    ll total = 0;
    int m = 0;

    BoundedMaxFlow(int n) : b(n) {}

    int add(int u, int v, ll lo, ll hi)
    {
        assert(0 <= lo && lo <= hi && hi <= LLONG_MAX - total);
        total += hi;
        b.add(u, v, lo, hi);
        return m++;
    }

    optional<ll> solve(int s, int t)
    {
        assert(!b.solved && 1 <= s && s <= b.n && 1 <= t && t <= b.n && s != t);
        int first = b.g.e.size();
        int back = b.add(t, s, 0, total);
        int forward = b.add(s, t, 0, total);
        if (!b.solve()) return nullopt;
        __int128 ans = (__int128)b.used(back) - b.used(forward);
        for (int id = first; id < (int)b.g.e.size(); id += 2) b.g.change_edge(id, 0, 0);
        ans += b.g.flow(s, t);
        assert(-(__int128)total <= ans && ans <= total);
        return (ll)ans;
    }
```

最小净流与原边流量（接上页同一结构体）：

```
    optional<ll> minimum(int s, int t)
    {
        auto ans = solve(t, s);
        if (ans) *ans = -*ans;
        return ans;
    }

    ll used(int id) const
    {
        assert(0 <= id && id < m);
        return b.used(id);
    }
};
```

应用题补充：Luogu P5192 Shoot the Bullet

题意：每天至多拍 D 张，每次独立拍摄有 [L,R] 限制，每位目标至少拍 G 张，求最多拍摄数。源连天、天连目标、目标连汇；重复目标记录分别加边。目标原编号加 1 后再偏移 n。题目总拍摄数至多 365×30000，用作目标到汇的容量上界。读至 EOF，每组输出后空一行。本题净流非负，因此无解可输出 -1；一般接口允许负净流。作为附件建图应用单列。

所需模板：Dinic（第 871 页，52.1.2 节）、BoundedCirculation（第 909 页，52.4.1 节）、BoundedMaxFlow（第 911 页，52.4.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5192

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    while (cin >> n >> m)
    {
        int s = n + m + 1, t = s + 1;
        BoundedMaxFlow f(t);
        for (int j = 1; j <= m; j++)
        {
            long long g;
            cin >> g;
            f.add(n + j, t, g, 365LL * 30000);
        }
        for (int i = 1; i <= n; i++)
        {
            int c, d;
            cin >> c >> d;
            f.add(s, i, 0, d);
            while (c--)
            {
                int j, lo, hi;
                cin >> j >> lo >> hi;
                f.add(i, n + j + 1, lo, hi);
            }
        }
        auto ans = f.solve(s, t);
        cout << (ans ? *ans : -1) << "\n\n";
    }
}
```

应用题补充：有源汇上下界最小净流（本地接口示例）

题意：输入带上下界的有向网络，求源点流出减流入的最小净流量，并输出每条原边流量。无解输出 NO；有解输出 YES、最小值及方案。净值可以为负，因此不能用 -1 作为无解哨兵；本地接口未附加“净流非负”约束。

所需模板：Dinic（第 871 页，52.1.2 节）、BoundedCirculation（第 909 页，52.4.1 节）、BoundedMaxFlow（第 911 页，52.4.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://github.com/gsh20040816/CPCtemplate/issues/11

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    BoundedMaxFlow g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long lo, hi;
        cin >> u >> v >> lo >> hi;
        g.add(u, v, lo, hi);
    }
    auto ans = g.minimum(s, t);
    if (!ans)
        cout << "NO\n";
    else
    {
        cout << "YES " << *ans << '\n';
        for (int i = 0; i < m; i++) cout << g.used(i) << '\n';
    }
}
```

52.5 Stoer–Wagner 算法

52.5.1 无向全局最小割

solve(w)，n≥2；w 为 n×n 对称非负权矩阵，重边先累加，主对角置零。返回 {割权, 一侧顶点列表}，编号 0..n-1；两侧非空，不要求原图连通，不连通时割权为 0。并列最优割返回其中一种，不承诺点数或字典序。矩阵按值传入，原图不变。O(n^3) 时间、O(n^2) 空间；无向边总权（每条只计一次）严格小于 2^120，内部 int128。负权不适用。已逐条核验返回点集在原图上的跨割费用，暂无专门 OJ AC。

```
struct StoerWagner
{
    using I = __int128_t;

    // Symmetric nonnegative adjacency matrix; returns weight and one cut side.
    static pair<I, vector<int>> solve(vector<vector<I>> w)
    {
        int n = w.size();
        assert(n >= 2);
        vector<int> active(n);
        iota(active.begin(), active.end(), 0);
        vector<vector<int>> group(n);
        for (int i = 0; i < n; i++) group[i] = {i};
        I best = I(1) << 120;
        vector<int> side;
        while (active.size() > 1)
        {
            vector<I> dis(n);
            vector<bool> used(n);
            int prev = -1;
            for (int step = 0; step < (int)active.size(); step++)
            {
                int sel = -1;
                for (int v : active)
                    if (!used[v] && (sel == -1 || dis[v] > dis[sel])) sel = v;
```

本轮割、合并与继续扩张（接上页同一函数）：

```
                if (step + 1 == (int)active.size())
                {
                    if (dis[sel] < best)
                    {
                        best = dis[sel];
                        side = group[sel];
                    }
                    for (int v : active)
                        if (v != prev && v != sel)
                            w[prev][v] = w[v][prev] = w[prev][v] + w[sel][v];
                    group[prev].insert(
                        group[prev].end(), group[sel].begin(), group[sel].end());
                    active.erase(find(active.begin(), active.end(), sel));
                    break;
                }
                used[sel] = true;
                prev = sel;
                for (int v : active)
                    if (!used[v]) dis[v] += w[sel][v];
            }
        }
        return {best, side};
    }
};
```

模板题使用：Luogu P5632

题意：无向连通正权图的全局最小割，n 至 600、边权总和至 10^9；矩阵按无向边双向累加、自环忽略。核心返回费用与 0-based 割侧 side，本题只输出费用。核心要求 n≥2；单点没有非平凡割，驱动额外约定输出 0。

所需模板：StoerWagner（第 915 页，52.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5632

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using I = StoerWagner::I;
    vector<vector<I>> w(n, vector<I>(n));
    while (m--)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        u--;
        v--;
        if (u == v) continue;
        w[u][v] += c;
        w[v][u] += c;
    }
    // A singleton has no nontrivial cut; use zero by convention.
    if (n == 1)
    {
        cout << 0 << '\n';
        return 0;
    }
    auto [ans, side] = StoerWagner::solve(w);
    // The statement bounds the sum of edge weights by 1e9.
    cout << (long long)ans << '\n';
}
```

第五十三章　图的匹配

53.1　二分图最大匹配

相关应用：正则多重二分图的随机交替游走；概率只影响耗时，不影响返回方案正确性。

53.1.1　正则二分图完美匹配

regular_matching(g, seed) 返回左侧各点的右侧匹配点，均为 0..n-1；g 为左侧邻接表，两侧每点度数均须等于同一正整数 d。重边按重数重复存储，不可去重；空图返回空。输入不修改。仅求一个完美匹配，不适用于一般非正则二分图或非空零度图。

从均匀随机未匹配左点开始，按邻接项均匀抽边，沿匹配边继续；path 和 pos 维护无环交替游走，遇旧左点时删去环。命中未匹配右点后倒序翻转；只在匹配全部建立后返回，不存在概率性错误答案。未匹配左点数为 k 时，每次尝试最多 4(floor(n/k)+2) 步，超限清空路径重试，不改变匹配。包含当前匹配边的采样项只形成自环，可正常消除。

理想独立均匀随机模型下，游走期望 O(n log(n+1))；加上全量度数检查，总期望 O(nd+n log(n+1))，额外空间 O(n)。固定 PRNG 种子没有确定性最坏耗时保证。seed 显式传入，测试用固定种子，赛时用时钟。随机性仅影响耗时与选择哪份正确方案；QOJ265 已以 CCF_NOI 满分通过，排名未核验。

期望界　将匹配点对收缩、原边定向左到右，加入源到未匹配左点和未匹配右点到汇各 d 条边，再识别源汇。所有点入出度平衡，源出度 kd，总出度 $(n+2k)d$；随机游走返回源的期望步数不超过 $n/k+2$。截断后重试的单次成功概率至少 $1/2$，故增广期望 $O(1+n/k)$，求和得 $O(n\log(n+1))$。路径数组只保存游走并消环，不是模拟 DFS 调用栈。

度数检查、未匹配集合与随机起步：

```cpp
vector<int> regular_matching(const vector<vector<int>> &g, uint64_t seed)
{
    int n = g.size();
    if (!n) return {};
    int d = g[0].size();
    assert(d > 0);
    vector<int> degree(n);
    for (const auto &row : g)
    {
        assert((int)row.size() == d);
        for (int v : row)
        {
            assert(0 <= v && v < n);
            degree[v]++;
        }
    }
    for (int value : degree) assert(value == d);
    vector<int> left(n, -1), right(n, -1), free(n), pos(n, -1), path;
    iota(free.begin(), free.end(), 0);
    mt19937_64 rng(seed);
    uniform_int_distribution<int> edge(0, d - 1);
    while (!free.empty())
    {
        int index = uniform_int_distribution<int>(0, free.size() - 1)(rng);
        int u = free[index];
        path.push_back(u);
        pos[u] = 0;
        long long limit = 4LL * (n / free.size() + 2);
        while (limit--)
        {
            int v = g[u][edge(rng)];
```

交替游走消环、增广与重试（接上页函数）：

```
            int next = right[v];
            if (next == -1)
            {
                for (int i = (int)path.size() - 1; i >= 0; i--)
                {
                    int x = path[i];
                    int old = left[x];
                    left[x] = v;
                    right[v] = x;
                    v = old;
                }
                free[index] = free.back();
                free.pop_back();
                break;
            }
            if (pos[next] == -1)
            {
                pos[next] = path.size();
                path.push_back(next);
            }
            else
            {
                while ((int)path.size() > pos[next] + 1)
                {
                    pos[path.back()] = -1;
                    path.pop_back();
                }
            }
            u = next;
        }
        for (int x : path) pos[x] = -1;
        path.clear();
    }
    return left;
}
```

模板题使用：QOJ265 正则二分图匹配

题意：两侧各 n 点、每点度数 d，允许重边；输出任一完美匹配，输入输出 1-based，函数内部 0-based。

所需模板：regular_matching（第 918 页，53.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://qoj.ac/problem/265

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, d;
    cin >> n >> d;
    vector<vector<int>> g(n, vector<int>(d));
    for (auto &row : g)
        for (int &v : row)
        {
            cin >> v;
            v--;
        }
    uint64_t seed = chrono::steady_clock::now().time_since_epoch().count();
    auto match = regular_matching(g, seed);
    for (int v : match) cout << v + 1 << ' ';
    cout << '\n';
}
```

53.1.2 Hopcroft–Karp 最大匹配与最小点覆盖

左右两侧分别编号 1..n、1..m，可有空侧及重边；add(u,v) 加边，solve() 返回最大匹配总数，l[u] 与 r[v] 为匹配端点，0 表示未匹配。保留已有合法匹配，可加边后再次 solve；不支持删边或手动更改匹配数组。BFS 用 dep[0] 记录最短终点层，递归 DFS 只沿最短层增广，并使用当前弧。V=n+m，单次 solve 最坏 $O((V+E)\sqrt{V})$，空间 O(V+E)。cover() 在 solve 后、下一次改图前调用，返回 (左侧点列表, 右侧点列表)，为最小点覆盖；其补集是最大独立集。覆盖与匹配方案均可直接输出。

```
struct BipartiteMatching
{
    int n, m;
    vector<vector<int>> g;
    vector<int> l, r, dep, cur;

    BipartiteMatching(int n, int m)
        : n(n), m(m), g(n + 1), l(n + 1), r(m + 1), dep(n + 1), cur(n + 1)
    {
    }

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= m);
        g[u].push_back(v);
    }

    bool bfs()
    {
        queue<int> q;
        fill(dep.begin(), dep.end(), n + 1);
        fill(cur.begin(), cur.end(), 0);
        for (int u = 1; u <= n; u++)
        {
            if (!l[u])
            {
                dep[u] = 0;
                q.push(u);
            }
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            if (dep[u] >= dep[0]) continue;
            for (int v : g[u])
            {
                int next = r[v];
                if (dep[next] != n + 1) continue;
                dep[next] = dep[u] + 1;
                if (next) q.push(next);
            }
        }
        return dep[0] != n + 1;
    }
```

增广、求解与最小点覆盖（接上页结构体）：

```
    bool dfs(int u)
    {
        if (!u) return true;
        for (int &i = cur[u]; i < (int)g[u].size(); i++)
        {
            int v = g[u][i];
            if (dep[r[v]] == dep[u] + 1 && dfs(r[v]))
            {
                l[u] = v;
                r[v] = u;
                return true;
            }
        }
        dep[u] = n + 1;
        return false;
    }

    // Retains an existing matching; edges may be added between calls.
    int solve()
    {
        int answer = 0;
        for (int u = 1; u <= n; u++) answer += (l[u] != 0);
        while (bfs())
        {
            for (int u = 1; u <= n; u++)
            {
                if (!l[u] && dfs(u)) answer++;
            }
        }
        return answer;
    }

    // Call after solve(), before adding more edges. Returns left/right vertex IDs.
    pair<vector<int>, vector<int>> cover() const
    {
        vector<int> left, right;
        for (int u = 1; u <= n; u++)
        {
            if (dep[u] == n + 1) left.push_back(u);
        }
        for (int v = 1; v <= m; v++)
        {
            if (r[v] && dep[r[v]] != n + 1) right.push_back(v);
        }
        return {left, right};
    }
};
```

模板题使用：Luogu P3386

题意：给定二分图，求最大匹配数。左右两侧分别编号 1..n、1..m，add(u,v) 的 u 属于左侧，v 属于右侧；solve 返回匹配数。此题不输出匹配或最小点覆盖方案。

所需模板：BipartiteMatching（第 922 页，53.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3386

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m, e;
    std::cin >> n >> m >> e;
    BipartiteMatching graph(n, m);
    while (e--)
    {
        int u, v;
        std::cin >> u >> v;
        graph.add(u, v);
    }
    std::cout << graph.solve() << '\n';
}
```

应用题补充：CSES 1709 / Coin Grid

题意：n≤100 的方阵中 o 为硬币，每次清空一行或一列，输出最少操作数及方案。行列作为二分图两侧，硬币为边；solve 后 cover 给出最小点覆盖，左点输出 1 行号，右点输出 2 列号。

所需模板：BipartiteMatching（第 922 页，53.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://cses.fi/problemset/task/1709/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    BipartiteMatching g(n, n);
    for (int i = 1; i <= n; i++)
    {
        string row;
        cin >> row;
        for (int j = 1; j <= n; j++)
        {
            if (row[j - 1] == 'o') g.add(i, j);
        }
    }
    cout << g.solve() << '\n';
    auto [a, b] = g.cover();
    for (int u : a) cout << 1 << ' ' << u << '\n';
    for (int v : b) cout << 2 << ' ' << v << '\n';
}
```

53.1.3 二分图最大匹配的可行边与必选边

代码依赖：BipartiteMatching（第 922 页，53.1.2 节）、TarjanSCC（第 794 页，47.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

matching_edges(n,m,edges) 接收左右两侧点号各为 1..n、1..m 的边表，返回 (最大匹配大小, 逐边分类)。每项为 {possible,forced}：是否属于至少一个/每一个最大匹配；分类保持输入顺序，平行边作为不同边处理。允许空侧、空图和非完备匹配。先用 BipartiteMatching 求解，再建含源汇的残量图跑 TarjanSCC；已匹配或两端同 SCC 为可行，已匹配且两端不同 SCC 为必选。不能在非完备匹配时省略源汇，也不能把可行条件写成已匹配且同 SCC。时间 $O(E\sqrt{}(n+m)+n+m)$，空间 O(n+m+E)，节点与残量边数须可用 int 表示。HK 与 SCC 都保留递归，栈深可为 O(n+m)。

```
pair<int, vector<array<bool, 2>>>
matching_edges(int n, int m, const vector<pair<int, int>> &edges)
{
    BipartiteMatching b(n, m);
    for (auto [u, v] : edges)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= m);
        b.add(u, v);
    }
    int size = b.solve(), s = n + m + 1, t = s + 1;
    TarjanSCC d(t);
    for (int u = 1; u <= n; u++)
        if (b.l[u])
            d.add(u, s);
        else
            d.add(s, u);
    for (int v = 1; v <= m; v++)
        if (b.r[v])
            d.add(t, n + v);
        else
            d.add(n + v, t);
    vector<bool> chosen(n + 1), used(edges.size());
    for (int i = 0; i < (int)edges.size(); i++)
    {
        auto [u, v] = edges[i];
        used[i] = b.l[u] == v && !chosen[u];
        if (used[i])
        {
            chosen[u] = true;
            d.add(n + v, u);
        }
        else
            d.add(u, n + v);
    }
    d.run();
    vector<array<bool, 2>> ans(edges.size());
    for (int i = 0; i < (int)edges.size(); i++)
    {
        auto [u, v] = edges[i];
        bool same = d.bel[u] == d.bel[n + v];
        ans[i] = {used[i] || same, used[i] && !same};
    }
    return {size, ans};
}
```

应用题补充：二分图最优匹配边分类（本地接口示例）

题意：输入左右点数、边数及 1-based 边表，输出最大匹配大小，再按原边顺序输出可行/必选标记。非完备匹配和平行边都允许；本例演示接口，不是 OJ 提交记录。

所需模板：BipartiteMatching（第 922 页，53.1.2 节）、TarjanSCC（第 794 页，47.1.1 节）、matching_edges（第 926 页，53.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://oi-wiki.org/graph/graph-matching/bigraph-match/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k;
    cin >> n >> m >> k;
    vector<pair<int, int>> edges(k);
    for (auto &[u, v] : edges) cin >> u >> v;
    auto [size, labels] = matching_edges(n, m, edges);
    cout << size << '\n';
    for (auto [possible, forced] : labels) cout << possible << ' ' << forced << '\n';
}
```

53.1.4 二分图最大独立集方案

代码依赖：BipartiteMatching（第 922 页，53.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

independent_set(g) 接收已建好的 BipartiteMatching，内部调用 solve 再取最小点覆盖的补集，返回左右两侧的原点号列表，分别从 1 计、升序排列。允许空侧、孤立点、重边与断开图；返回一种最大基数独立集，不保证字典序或最大权。只适用于已知左右划分的二分图，不能直接输入一般图或同侧边。

调用会更新 g 的匹配与搜索状态，保留合法旧匹配并继续增广；调用后可立即读 g.cover()。允许继续 add 后再次调用，不可删除边或手改匹配。n,m 非负且小于 INT_MAX，内存足够。最大独立集大小等于 n+m 减最大匹配数；只求数量可直接使用该式，求和时先转 long long。

依赖 BipartiteMatching；总时间 $O((V+E)\sqrt{(V+1)})$，V=n+m，额外时间与空间 O(V)。HK 增广保持递归，深度最坏 O(V)。骑士攻击使格子黑白染色反转，故可应用此函数；障碍格不要建成孤立点。

```
// Maximum independent set; left/right IDs are separate and 1-based.
pair<vector<int>, vector<int>> independent_set(BipartiteMatching &g)
{
    g.solve();
    auto [a, b] = g.cover();
    vector<bool> x(g.n + 1, true), y(g.m + 1, true);
    for (int u : a) x[u] = false;
    for (int v : b) y[v] = false;
    vector<int> left, right;
    for (int u = 1; u <= g.n; u++)
    {
        if (x[u]) left.push_back(u);
    }
    for (int v = 1; v <= g.m; v++)
    {
        if (y[v]) right.push_back(v);
    }
    return {left, right};
}
```

应用题补充：Luogu P3355 / 骑士共存问题

题意：n×n 棋盘有障碍，求互不攻击骑士的最大数量。n≤200；可用格按 (x+y) 奇偶分成两侧并单独编号，只从奇色连向八个合法骑士落点。障碍不参与编号，输出两侧独立集大小之和。

所需模板：BipartiteMatching（第 922 页，53.1.2 节）、independent_set（第 927 页，53.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3355

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<int>> id(n + 1, vector<int>(n + 1));
    while (m--)
    {
        int x, y;
        cin >> x >> y;
        id[x][y] = -1;
    }
    int l = 0, r = 0;
    for (int x = 1; x <= n; x++)
        for (int y = 1; y <= n; y++)
            if (id[x][y] != -1)
                id[x][y] = (x + y) % 2 ? ++l : ++r;
    BipartiteMatching g(l, r);
    vector<pair<int, int>> moves = {
        {1, 2}, {1, -2}, {-1, 2}, {-1, -2},
        {2, 1}, {2, -1}, {-2, 1}, {-2, -1}
    };
    for (int x = 1; x <= n; x++)
        for (int y = 1; y <= n; y++)
            if ((x + y) % 2 && id[x][y] != -1)
                for (auto [dx, dy] : moves)
                {
                    int u = x + dx, v = y + dy;
                    if (1 <= u && u <= n && 1 <= v && v <= n && id[u][v] != -1)
                        g.add(id[x][y], id[u][v]);
                }
    auto [a, b] = independent_set(g);
    cout << a.size() + b.size() << '\n';
}
```

接口用法演示：二分图最大独立集原点方案（接口演示）

演示约定：输入左点数 n、右点数 m、边数 e 及左右各自 1-based 边端点，允许空侧、孤立点和重边。首行输出最优大小，后两行分别输出“左侧数量左点号列表”和“右侧数量右点号列表”。两侧同号是不同点。

所需模板：BipartiteMatching（第 922 页，53.1.2 节）、independent_set（第 927 页，53.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/graph-matching/bigraph-match/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, e;
    cin >> n >> m >> e;
    BipartiteMatching g(n, m);
    while (e--)
    {
        int u, v;
        cin >> u >> v;
        g.add(u, v);
    }
    auto [a, b] = independent_set(g);
    cout << a.size() + b.size() << '\n';
    cout << a.size();
    for (int u : a) cout << ' ' << u;
    cout << '\n';
    cout << b.size();
    for (int v : b) cout << ' ' << v;
    cout << '\n';
}
```

53.2 二分图最大权匹配

53.2.1 矩形二分图最小权匹配

solve(cost) 输入 n×m 完整权值矩阵，0≤n≤m；每行匹配不同列，返回最小费用及列号（0-based）。允许负费用，使用 int128 势能；$O(n^2m)$。禁止边不能用未经证明的有限大常数随意替代，当前接口仅表示完整二分图。

```
struct Hungarian
{
    using I = __int128_t;

    // Minimum cost injection from n rows to m columns, n<=m. Zero-based matrix.
    static pair<I, vector<int>> solve(const vector<vector<long long>> &cost)
    {
        int n = cost.size();
        if (!n) return {0, {}};
        int m = cost[0].size();
        assert(n <= m);
        vector<I> u(n + 1), v(m + 1);
        vector<int> p(m + 1), way(m + 1);
        const I inf = I(1) << 120;
        for (int i = 1; i <= n; i++)
        {
            assert((int)cost[i - 1].size() == m);
```

```
            p[0] = i;
            int j0 = 0;
            vector<I> dis(m + 1, inf);
            vector<bool> used(m + 1);
            do
            {
                used[j0] = true;
                int i0 = p[j0], j1 = 0;
                I delta = inf;
                for (int j = 1; j <= m; j++)
                    if (!used[j])
                    {
                        I cur = I(cost[i0 - 1][j - 1]) - u[i0] - v[j];
                        if (cur < dis[j])
                        {
                            dis[j] = cur;
                            way[j] = j0;
                        }
                        if (dis[j] < delta)
                        {
                            delta = dis[j];
                            j1 = j;
                        }
                    }
                for (int j = 0; j <= m; j++)
                    if (used[j])
                    {
                        u[p[j]] += delta;
                        v[j] -= delta;
                    }
                    else
                        dis[j] -= delta;
                j0 = j1;
            } while (p[j0]);
            do
            {
                int j1 = way[j0];
                p[j0] = p[j1];
                j0 = j1;
            } while (j0);
        }
        vector<int> ans(n);
        for (int j = 1; j <= m; j++)
            if (p[j]) ans[p[j] - 1] = j - 1;
        return {-v[0], ans};
    }
};
```

模板题使用：Library Checker assignment

题意：给完整方阵，为每一行分配一个不同的列，使费用和最小。返回列号是 0-based，按行顺序输出。题目 n≤500、单项绝对值 ≤10^9，总费用可由 int128 转为 long long；缺边问题不能直接套本例。

所需模板：Hungarian（第 929 页，53.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/assignment

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<vector<long long>> cost(n, vector<long long>(n));
    for (auto &row : cost)
        for (auto &x : row) cin >> x;
    auto [value, columns] = Hungarian::solve(cost);
    cout << (long long)value << '\n';
    for (int j : columns) cout << j << ' ';
    cout << '\n';
}
```

53.2.2 二分图最大权匹配（缺边与可选匹配）

WeightedMatching(n,m)，两侧编号 1..n、1..m。add(x,y,w) 加边，重边保留最大权；边权支持完整 long long 范围，内部势能与答案为 int128。solve() 要求全部左点匹配不同右点，无解返回 nullopt；n=m 时即最大权完美匹配。solve(true) 允许点不匹配，只最大化总权，不优先最大化边数。两种模式均从空匹配重算；l[x]、r[y] 为方案，0 表示未匹配，无解时全部清零。缺边用独立于合法边权的 -2^120 哨兵，仅用 add 修改；不把缺边当零权边。允许空侧。全匹配 $O(n^2m)$，可选模式增加 n 个虚拟零权右点，为 $O(n^2(m+n))$；空间 O(nm+n+m)。按最短松弛量扩展交错树：每次至少访问一个新右点，最多 n 次后增广；无有限松弛量即无解。

```
struct WeightedMatching
{
    using I = __int128_t;
    static constexpr I inf = I(1) << 120;
    int n, m;
    vector<vector<I>> w;
    vector<int> l, r;

    WeightedMatching(int n, int m)
        : n(n), m(m), w(n + 1, vector<I>(m + 1, -inf)), l(n + 1), r(m + 1)
    {
    }

    void add(int x, int y, long long weight)
    {
        assert(1 <= x && x <= n && 1 <= y && y <= m);
        w[x][y] = max(w[x][y], I(weight));
    }

    optional<I> solve(bool allow_unmatched = false)
    {
        fill(l.begin(), l.end(), 0);
        fill(r.begin(), r.end(), 0);
```

```
        if (!allow_unmatched && n > m) return nullopt;
        int cols = m + (allow_unmatched ? n : 0);
        vector<I> u(n + 1), v(cols + 1);
        vector<int> p(cols + 1), way(cols + 1);
        for (int i = 1; i <= n; i++)
        {
            p[0] = i;
            int j0 = 0;
            vector<I> dis(cols + 1, inf);
            vector<bool> used(cols + 1);
            do
            {
                used[j0] = true;
                int x = p[j0], j1 = 0;
                I delta = inf;
                for (int j = 1; j <= cols; j++)
                {
                    if (used[j]) continue;
                    I weight = j <= m ? w[x][j] : 0;
                    if (weight != -inf)
                    {
                        I cur = -weight - u[x] - v[j];
                        if (cur < dis[j])
                        {
                            dis[j] = cur;
                            way[j] = j0;
                        }
                    }
                    if (dis[j] < delta)
                    {
                        delta = dis[j];
                        j1 = j;
                    }
                }
                if (delta == inf) return nullopt;
                for (int j = 0; j <= cols; j++)
                {
                    if (used[j])
                    {
                        u[p[j]] += delta;
                        v[j] -= delta;
                    }
                    else if (dis[j] != inf)
                        dis[j] -= delta;
                }
                j0 = j1;
            } while (p[j0]);
            do
            {
                int j1 = way[j0];
                p[j0] = p[j1];
                j0 = j1;
            } while (j0);
        }
        I answer = 0;
```

```
        for (int j = 1; j <= m; j++)
        {
            if (!p[j]) continue;
            l[p[j]] = j;
            r[j] = p[j];
            answer += w[p[j]][j];
        }
        return answer;
    }
};
```

模板题使用：Luogu P6577

题意：求二分图最大权完美匹配，两侧均为 n 点，题目保证存在完美匹配。输出 r[j]，即右部 j 对应的左部编号，均为 1-based；题目权值与规模保证总权值可转为 long long。

所需模板：WeightedMatching（第 931 页，53.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6577

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    WeightedMatching graph(n, n);
    while (m--)
    {
        int x, y;
        long long w;
        std::cin >> x >> y >> w;
        graph.add(x, y, w);
    }
    auto answer = graph.solve();
    assert(answer);
    std::cout << (long long)*answer << '\n';
    for (int j = 1; j <= n; j++) std::cout << graph.r[j] << (j == n ? '\n' : ' ');
}
```

53.2.3 按匹配基数求最大权与顶标证书

AssignmentSpectrum(n,m)，两侧 0-based，n,m≥0。add(x,y,w) 接受完整 signed64 权，重边保留最大权，缺边不当零权。solve(limit=INT_MAX) 从空重算，返回 k=min(limit, 最大匹配数)，limit≥0；best[j] 为全图恰好 j 条边的最大权，0≤j≤k，best[0]=0。负增益也继续，best 不必单调。l/r 为最后 k 条的互逆方案，-1 未匹配；要 j 条方案重新 solve(j)。顶标 lx,ly 和 level 仅对最后基数有效：令 α=lx-level、β=ly，则 α,β 非负，每条边 $\alpha+\beta+\text{level}\geq w$，匹配边取等，未匹配点对应 α 或 β 为 0；$\text{sum}(\alpha)+\text{sum}(\beta)+k*\text{level}$ 等于 best[k]。答案与顶标为 int128；加边后须重新 solve，其他成员只读。多源松弛始终从所有未匹配左点出发，故每个基数均全局最优。时间 O((k+1)nm+n+m)、空间 O(nm+n+m)，适合小侧或稠密图。

```
struct AssignmentSpectrum
{
    using I = __int128_t;
    static constexpr I inf = I(1) << 120;
    int n, m;
    vector<vector<I>> w;
    vector<I> lx, ly, best;
    vector<int> l, r;
    I level = 0;

    AssignmentSpectrum(int n, int m)
        : n(n), m(m), w(n, vector<I>(m, -inf))
    {
    }

    void add(int x, int y, long long value)
    {
        assert(0 <= x && x < n && 0 <= y && y < m);
        w[x][y] = max(w[x][y], I(value));
    }
```

重置状态与多源松弛（接上页同一结构体）：

```
    int solve(int limit = INT_MAX)
    {
        assert(limit >= 0);
        level = 0;
        for (const auto &row : w)
            for (I value : row)
                level = max(level, value);
        lx.assign(n, level);
        ly.assign(m, 0);
        l.assign(n, -1);
        r.assign(m, -1);
        best.assign(1, 0);
        int bound = min({n, m, limit});
        for (int k = 0; k < bound; k++)
        {
            vector<bool> sx(n), sy(m);
            vector<I> slack(m, inf);
            vector<int> from(m, -1);
            auto relax = [&](int x)
            {
                sx[x] = true;
                for (int y = 0; y < m; y++)
                {
                    if (sy[y] || w[x][y] == -inf)
                        continue;
                    I value = lx[x] + ly[y] - w[x][y];
                    if (value < slack[y])
                    {
                        slack[y] = value;
                        from[y] = x;
                    }
                }
            };
            for (int x = 0; x < n; x++)
                if (l[x] == -1)
                    relax(x);
```

继续 solve：调整顶标与翻转增广路径（接上页同一结构体）：

```
            int y;
            while (true)
            {
                y = -1;
                for (int j = 0; j < m; j++)
                    if (!sy[j] && (y == -1 || slack[j] < slack[y]))
                        y = j;
                if (y == -1 || slack[y] == inf)
                    return k;
                I delta = slack[y];
                level -= delta;
                for (int x = 0; x < n; x++)
                    if (sx[x])
                        lx[x] -= delta;
                for (int j = 0; j < m; j++)
                {
                    if (sy[j])
                        ly[j] += delta;
                    else if (slack[j] != inf)
                        slack[j] -= delta;
                }
                sy[y] = true;
                if (r[y] == -1)
                    break;
                relax(r[y]);
            }
            I answer = best.back();
            while (y != -1)
            {
                int x = from[y];
                int old = l[x];
                answer += w[x][y];
                if (old != -1)
                    answer -= w[x][old];
                l[x] = y;
                r[y] = x;
                y = old;
            }
            best.push_back(answer);
        }
        return bound;
    }
};
```

接口用法演示：按基数匹配与对偶证书

演示约定：输入 n,m,e,limit 与 e 条 0-based 边 x,y,w，输出实际基数 k、best[0..k]、左侧配对 l（-1 未匹配）、level、lx、ly。本演示限定两侧 ≤1000 且 |w|≤10^9，输出可安全转 long long；核心支持全 signed64 输入及 int128 结果。证书仅对应最后基数；没有正式题目或线上 AC 声明。

所需模板：AssignmentSpectrum（第 934 页，53.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/hh2048/XCPC

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, e, limit;
    cin >> n >> m >> e >> limit;
    AssignmentSpectrum matching(n, m);
    while (e--)
    {
        int x, y;
        long long w;
        cin >> x >> y >> w;
        matching.add(x, y, w);
    }
    int k = matching.solve(limit);
    // 本演示限定 n,m<=1000，边权绝对值<=10^9，可安全转 long long。
    cout << k << '\n';
    for (auto value : matching.best)
        cout << (long long)value << ' ';
    cout << '\n';
    for (int y : matching.l)
        cout << y << ' ';
    cout << '\n';
    cout << (long long)matching.level << '\n';
    for (auto value : matching.lx)
        cout << (long long)value << ' ';
    cout << '\n';
    for (auto value : matching.ly)
        cout << (long long)value << ' ';
    cout << '\n';
    return 0;
}
```

应用题补充：AtCoder ABC247 G / Dream Team

题意：N≤30000，每人有 1..150 的大学 A 和学科 B、1..10^9 的强度 C。选人须使 A、B 各不重复，输出最大人数 k 与每种人数 1..k 的最大总强度。同属性对保留最大权；答案 ≤150×10^9，可转 long long。缺边不能填 0，各基数答案可能下降。比赛应用，已核对官方题面与样例，尚无本项目线上 AC 或排名。

所需模板：AssignmentSpectrum（第 934 页，53.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://atcoder.jp/contests/abc247/tasks/abc247_g

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    AssignmentSpectrum matching(150, 150);
    while (n--)
    {
        int a, b;
        long long c;
        cin >> a >> b >> c;
        matching.add(a - 1, b - 1, c);
    }
    int k = matching.solve();
    cout << k << '\n';
    for (int i = 1; i <= k; i++)
        cout << (long long)matching.best[i] << '\n';
    return 0;
}
```

模板题使用：Luogu P6577 / 二分图最大权完美匹配

题意：左右均 n≤500 点，m≤n^2，无重边，权值范围 [-19980731,19980731]，保证完美匹配。输出最大总权及每个右点对应的左点编号；调用前点号减一，输出 r 加一。答案绝对值 ≤500×19980731，可转 long long。该题只验证最终完美匹配，不能代替中间基数和顶标检查；新驱动尚无线上 AC。

所需模板：AssignmentSpectrum（第 934 页，53.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6577

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    AssignmentSpectrum matching(n, n);
    while (m--)
    {
        int x, y;
        long long w;
        cin >> x >> y >> w;
        matching.add(x - 1, y - 1, w);
    }
    matching.solve();
    cout << (long long)matching.best[n] << '\n';
    for (int x : matching.r)
        cout << x + 1 << ' ';
    cout << '\n';
    return 0;
}
```

53.3 一般图最大匹配

53.3.1 一般图最大基数匹配：带花树

Blossom(n)，点号 0..n-1；add(u,v) 加无向边，忽略自环，允许重边。solve() 从空匹配重新计算，返回匹配边数；mate[u] 为配对点，未匹配为 -1。结果支持直接输出方案。缩花维护交错树，单次增广 $O(n^2+m)$，总计 $O(n^3+nm)$，简单图为 $O(n^3)$；空间 O(n+m)。重复调用 solve 或加边后重算均可。不处理边权，不能代替最大权匹配。

```
struct Blossom
{
    int n;
    vector<vector<int>> g;
    vector<int> mate, pre, base, q;
    vector<bool> outer, flower, seen;

    Blossom(int n)
        : n(n), g(n), mate(n, -1), pre(n), base(n), outer(n), flower(n), seen(n)
    {
    }

    void add(int u, int v)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        if (u == v) return;
        g[u].push_back(v);
        g[v].push_back(u);
    }

    int lca(int u, int v)
    {
        fill(seen.begin(), seen.end(), false);
        while (true)
        {
            u = base[u];
            seen[u] = true;
            if (mate[u] == -1) break;
            u = pre[mate[u]];
        }
        while (!seen[base[v]]) v = pre[mate[base[v]]];
        return base[v];
    }

    void mark(int u, int b, int v)
    {
        while (base[u] != b)
        {
            flower[base[u]] = true;
            flower[base[mate[u]]] = true;
            pre[u] = v;
            v = mate[u];
            u = pre[v];
        }
    }

    bool augment(int root)
    {
```

```
        fill(outer.begin(), outer.end(), false);
        fill(pre.begin(), pre.end(), -1);
        iota(base.begin(), base.end(), 0);
        q.assign(1, root);
        outer[root] = true;
        for (int head = 0; head < (int)q.size(); head++)
        {
            int u = q[head];
            for (int v : g[u])
            {
                if (base[u] == base[v] || mate[u] == v) continue;
                if (v == root || (mate[v] != -1 && pre[mate[v]] != -1))
                {
                    int b = lca(u, v);
                    fill(flower.begin(), flower.end(), false);
                    mark(u, b, v);
                    mark(v, b, u);
                    for (int x = 0; x < n; x++)
                    {
                        if (!flower[base[x]]) continue;
                        base[x] = b;
                        if (!outer[x])
                        {
                            outer[x] = true;
                            q.push_back(x);
                        }
                    }
                }
                else if (pre[v] == -1)
                {
                    pre[v] = u;
                    if (mate[v] == -1)
                    {
                        while (v != -1)
                        {
                            int p = pre[v];
                            int next = mate[p];
                            mate[v] = p;
                            mate[p] = v;
                            v = next;
                        }
                        return true;
                    }
                    int x = mate[v];
                    outer[x] = true;
                    q.push_back(x);
                }
            }
        }
        return false;
    }

    // Rebuild the maximum matching; mate[u] is -1 for an unmatched vertex.
    int solve()
```

```
    {
        fill(mate.begin(), mate.end(), -1);
        int answer = 0;
        for (int u = 0; u < n; u++)
        {
            if (mate[u] == -1 && augment(u)) answer++;
        }
        return answer;
    }
};
```

模板题使用：Library Checker general_matching

题意：给简单无向图，输出最大匹配的边数及匹配边。点号 0..n-1；solve 后 mate[u]=-1 表示未匹配，只在 u<mate[u] 时输出，避免同一条边打印两次。

所需模板：Blossom（第 940 页，53.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/general_matching

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Blossom graph(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    cout << graph.solve() << '\n';
    for (int u = 0; u < n; u++)
        if (u < graph.mate[u]) cout << u << ' ' << graph.mate[u] << '\n';
}
```

53.4 一般图最大权匹配

53.4.1 一般图最大权匹配：带权带花树

WeightedBlossom(n)，点 1..n，0≤n<INT_MAX/2。add_edge(u,v,w) 添加 signed64 边权，自环忽略，重边取最大。solve() 只最大化总权，允许空匹配，不保证同权时边数最多；solve(true) 先最大化匹配边数，再最大化原权。返回 weight（__int128）、pairs、mate（未匹配为 0）。求最大权完美匹配用 solve(true)，并检查 2*pairs==n。

每次 solve 从头计算，支持重复、切换模式、加边后重算。$O(n^3)$ 时间、$O(n^2)$ 空间，稠密矩阵适合数百点；不因接口允许大 n 而保证能装入内存。内部成员仅供核验，不应直接修改。缩花、展开与增广使用递归；全范围 signed64 权值以 128 位对偶计算。

改编自 Shahjalal Shohag 的 MIT 实现，完整许可见后页。P6699 已 100 分通过；负权与基数优先模式由独立本地验证覆盖。

字段与加边（同一结构体内连续抄写）：

```
struct WeightedBlossom
{
    using I = __int128;
    using ll = long long;
    static constexpr I INF = I(1) << 126;
    struct Edge
    {
        int u = 0, v = 0;
        I w = 0;
    };
    struct Result
    {
        I weight = 0;
        int pairs = 0;
        vector<int> mate;
    };
    int n, top = 0;
    vector<vector<I>> weight;
    vector<vector<Edge>> g;
    vector<vector<int>> flower, from;
    vector<int> match, slack, base, parent, state, seen;
    vector<I> label;
    queue<int> q;

    explicit WeightedBlossom(int n) : n(n), weight(n + 1, vector<I>(n + 1, -INF))
    {
        assert(0 <= n && n < INT_MAX / 2);
    }
    void add_edge(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        if (u == v) return;
        weight[u][v] = max(weight[u][v], I(w));
        weight[v][u] = weight[u][v];
    }
```

松弛与递归代表元（同一结构体内连续抄写）：

```
    I gap(const Edge &e) const
    {
        return label[e.u] + label[e.v] - 2 * e.w;
    }
    void relax(int u, int x)
    {
        if (!slack[x] || gap(g[u][x]) < gap(g[slack[x]][x])) slack[x] = u;
    }
    void reset_slack(int x)
    {
        slack[x] = 0;
        for (int u = 1; u <= n; u++)
            if (g[u][x].w > 0 && base[u] != x && state[base[u]] == 0)
                relax(u, x);
    }
    void enqueue(int x)
    {
        if (x <= n) q.push(x);
        else for (int v : flower[x]) enqueue(v);
    }
    void set_base(int x, int b)
    {
        base[x] = b;
        if (x > n)
            for (int v : flower[x]) set_base(v, b);
    }
    int position(int b, int x)
    {
        int k = find(flower[b].begin(), flower[b].end(), x) - flower[b].begin();
        if (k % 2 == 0) return k;
        reverse(flower[b].begin() + 1, flower[b].end());
        return flower[b].size() - k;
    }
```

花内配对、增广与公共祖先（同一结构体内连续抄写）：

```
    void pair_with(int u, int v)
    {
        Edge e = g[u][v];
        match[u] = e.v;
        if (u <= n) return;
        int x = from[u][e.u];
        int k = position(u, x);
        for (int i = 0; i < k; i++) pair_with(flower[u][i], flower[u][i ^ 1]);
        pair_with(x, v);
        rotate(flower[u].begin(), flower[u].begin() + k, flower[u].end());
    }
    void augment(int u, int v)
    {
        int next = base[match[u]];
        pair_with(u, v);
        if (!next) return;
        pair_with(next, base[parent[next]]);
        augment(base[parent[next]], next);
    }
    int lca(int u, int v)
    {
        fill(seen.begin(), seen.end(), 0);
        while (u || v)
        {
            if (u)
            {
                if (seen[u]) return u;
                seen[u] = 1;
                u = base[match[u]];
                if (u) u = base[parent[u]];
            }
            swap(u, v);
        }
        return 0;
    }
```

缩花及原始端点映射（同一结构体内连续抄写）：

```
104 void shrink(int u, int root, int v)
105 {
106     int b = n + 1;
107     while (b <= top && base[b]) b++;
108     if (b > top) top++;
109     label[b] = 0;
110     state[b] = 0;
111     match[b] = match[root];
112     flower[b] = {root};
113     for (int end : {u, v})
114     {
115         for (int x = end; x != root; )
116         {
117             int y = base[match[x]];
118             flower[b].push_back(x);
119             flower[b].push_back(y);
120             enqueue(y);
121             x = base[parent[y]];
122         }
123         if (end == u) reverse(flower[b].begin() + 1, flower[b].end());
124     }
125     set_base(b, b);
126     for (int x = 1; x <= top; x++)
127     {
128         g[b][x] = {};
129         g[x][b] = {};
130     }
131     fill(from[b].begin(), from[b].end(), 0);
132     for (int child : flower[b])
133     {
134         for (int x = 1; x <= top; x++)
135             if (g[child][x].w > 0 &&
136                 (!g[b][x].w || gap(g[child][x]) < gap(g[b][x])))
137             {
138                 g[b][x] = g[child][x];
139                 g[x][b] = g[x][child];
140             }
141         for (int x = 1; x <= n; x++)
142             if (from[child][x]) from[b][x] = child;
143     }
144     reset_slack(b);
145 }
```

展开与紧边处理（同一结构体内连续抄写）：

```
    void expand(int b)
    {
        for (int x : flower[b]) set_base(x, x);
        int entry = from[b][g[b][parent[b]].u];
        int k = position(b, entry);
        for (int i = 0; i < k; i += 2)
        {
            int x = flower[b][i];
            int y = flower[b][i + 1];
            parent[x] = g[y][x].u;
            state[x] = 1;
            state[y] = 0;
            slack[x] = 0;
            reset_slack(y);
            enqueue(y);
        }
        state[entry] = 1;
        parent[entry] = parent[b];
        for (int i = k + 1; i < (int)flower[b].size(); i++)
        {
            int x = flower[b][i];
            state[x] = -1;
            reset_slack(x);
        }
        base[b] = 0;
    }
    bool use_edge(Edge e)
    {
        int u = base[e.u];
        int v = base[e.v];
        if (state[v] == -1)
        {
            parent[v] = e.u;
            state[v] = 1;
            int next = base[match[v]];
            slack[v] = 0;
            slack[next] = 0;
            state[next] = 0;
            enqueue(next);
        }
        else if (state[v] == 0)
        {
            int root = lca(u, v);
            if (root) shrink(u, root, v);
            else
            {
                augment(u, v);
                augment(v, u);
                return true;
            }
        }
        return false;
    }
```

交替森林搜索（同一结构体内连续抄写）：

```
bool search()
{
    fill(state.begin(), state.end(), -1);
    fill(slack.begin(), slack.end(), 0);
    q = {};
    for (int x = 1; x <= top; x++)
        if (base[x] == x && !match[x])
        {
            parent[x] = 0;
            state[x] = 0;
            enqueue(x);
        }
    if (q.empty()) return false;
    while (true)
    {
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            if (state[base[u]] == 1) continue;
            for (int v = 1; v <= n; v++)
                if (g[u][v].w > 0 && base[u] != base[v])
                {
                    if (gap(g[u][v]) == 0)
                    {
                        if (use_edge(g[u][v])) return true;
                    }
                    else relax(u, base[v]);
                }
        }
```

对偶更新与继续搜索（同一结构体内连续抄写）：

```
            I delta = INF;
            for (int b = n + 1; b <= top; b++)
                if (base[b] == b && state[b] == 1)
                    delta = min(delta, label[b] / 2);
            for (int x = 1; x <= top; x++)
                if (base[x] == x && slack[x])
                {
                    I d = gap(g[slack[x]][x]);
                    if (state[x] == -1) delta = min(delta, d);
                    if (state[x] == 0) delta = min(delta, d / 2);
                }
            for (int u = 1; u <= n; u++)
                if (state[base[u]] == 0) delta = min(delta, label[u]);
            bool finished = false;
            for (int u = 1; u <= n; u++)
            {
                if (state[base[u]] == 0)
                {
                    label[u] -= delta;
                    if (label[u] == 0) finished = true;
                }
                else if (state[base[u]] == 1) label[u] += delta;
            }
            for (int b = n + 1; b <= top; b++)
                if (base[b] == b)
                {
                    if (state[b] == 0) label[b] += 2 * delta;
                    if (state[b] == 1) label[b] -= 2 * delta;
                }
            if (finished) return false;
            q = {};
            for (int x = 1; x <= top; x++)
                if (base[x] == x && slack[x] && base[slack[x]] != x &&
                    gap(g[slack[x]][x]) == 0)
                    if (use_edge(g[slack[x]][x])) return true;
            for (int b = n + 1; b <= top; b++)
                if (base[b] == b && state[b] == 1 && label[b] == 0) expand(b);
        }
    }
```

两种目标、初始化与方案返回（同一结构体内连续抄写）：

```
    Result solve(bool cardinality_first = false)
    {
        int size = 2 * n + 1;
        top = n;
        g.assign(size, vector<Edge>(size));
        flower.assign(size, {});
        from.assign(size, vector<int>(n + 1));
        match.assign(size, 0);
        slack.assign(size, 0);
        base.assign(size, 0);
        parent.assign(size, 0);
        state.assign(size, -1);
        seen.assign(size, 0);
        label.assign(size, 0);
        I bound = 0;
        for (int u = 1; u <= n; u++)
            for (int v = u + 1; v <= n; v++)
                if (weight[u][v] != -INF) bound = max(bound, max(weight[u][v], -weight[u][v]));
        I bonus = cardinality_first ? n * bound + 1 : 0;
        I maximum = 0;
        for (int u = 1; u <= n; u++)
        {
            base[u] = u;
            from[u][u] = u;
            for (int v = 1; v <= n; v++)
                if (weight[u][v] != -INF && weight[u][v] + bonus > 0)
                {
                    g[u][v] = {u, v, weight[u][v] + bonus};
                    maximum = max(maximum, g[u][v].w);
                }
        }
        fill(label.begin() + 1, label.begin() + n + 1, maximum);
        Result result;
        for (int u = 1; u <= n; u++)
            if (!match[u] && maximum > 0)
                for (int v = u + 1; v <= n; v++)
                    if (!match[v] && g[u][v].w == maximum)
                    {
                        match[u] = v;
                        match[v] = u;
                        result.pairs++;
                        break;
                    }
        while (search()) result.pairs++;
        result.mate.assign(match.begin(), match.begin() + n + 1);
        for (int u = 1; u <= n; u++)
            if (result.mate[u] > u) result.weight += weight[u][result.mate[u]];
        return result;
    }
};
```

目标与证书 基数优先模式给每条存在边加 $B = nW + 1$，W 为最大绝对原权；任意两个匹配原权差不超过 nW，所以多一条边严格占优。求完美匹配必须选择此模式再检查 $2 \cdot pairs = n$。顶点标签与奇集标签给出匹配 LP 的对偶上界；边约束可行、匹配边紧、未匹配顶点标签零以及正标签花饱和使上下界相等。所有标签应完成更新后再终止，不能留下部分更新。

验证范围 六种构建各 28546 次子集 DP、方案和对偶检查；400 点 16 次大图证书检查通过 O2 与 ASan/UBSan。实际触发嵌套缩花与展开。P6699 记录 302632540，20 点全 AC，最慢 209ms、最高 24.02MB；负权、基数优先及完美判断为本地证据，排名未核验。

来源及 MIT 许可 改编自 Shahjalal Shohag 的带权带花树，固定提交 0884be6ab81f（完整提交号见仓库来源记录）。以下许可随本实现保留：

Copyright (c) 2021 Shahjalal Shohag License: MIT

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the ”Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED ”AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

模板题使用：P6699 一般图最大权匹配

题意：n≤400、m≤79800、1≤w≤5×10^8。输出最大总权及每点的匹配点，未匹配为 0。答案 ≤10^{11}，可转 long long；方案不唯一。当前源码记录 302632540，20 点全 AC。

所需模板：WeightedBlossom（第 943 页，53.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6699

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    WeightedBlossom g(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add_edge(u, v, w);
    }
    auto result = g.solve();
    cout << (long long)result.weight << '\n';
    for (int u = 1; u <= n; u++) cout << result.mate[u] << ' ';
    cout << '\n';
}
```

接口用法演示：基数优先与最大权完美匹配

演示约定：输入 n、m 和 signed64 无向边。先输出最大匹配边数及其中的最大原权，再输出 mate，最后判断是否完美；不是另一条线上 AC 记录。负权保留，原权用 128 位十进制输出。

所需模板：WeightedBlossom（第 943 页，53.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/WEIGHTED-BLOSSOM.md

```
int main()
{
    auto print128 = [](__int128 x)
    {
        if (x < 0)
        {
            cout << '-';
            x = -x;
        }
        string s;
        do
        {
            s.push_back('0' + x % 10);
            x /= 10;
        } while (x);
        reverse(s.begin(), s.end());
        cout << s;
    };
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    WeightedBlossom g(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add_edge(u, v, w);
    }
    auto result = g.solve(true);
    cout << result.pairs << ' ';
    print128(result.weight);
    cout << '\n';
    for (int u = 1; u <= n; u++) cout << result.mate[u] << ' ';
    cout << '\n';
    cout << (2 * result.pairs == n ? "PERFECT" : "NOT_PERFECT") << '\n';
}
```

第五十四章　Prüfer 序列

54.1　Prüfer 双射与有标号树计数

本节的树是顶点标号固定为 $1,\dots,n$ 的简单无向树，不把同构的不同标号树合并。先设 $n\ge 2$。反复删除当前**最小标号叶子**，记录它唯一的邻点，直到剩两个点；得到长度 $n-2$、每项属于 $[n]$ 的 Prüfer 序列。$n=2$ 对应唯一空序列；$n=1$ 另有唯一空边树，度数为 0，不代入含 $n-2$ 的公式。

54.1.1　双射为何成立

解码时保留所有尚未删去的标号。读下一项 u，取当前未删除且没有出现在**整个剩余序列**中的最小标号 v，连边 uv，删除 v 并消耗这一项 u；最后连接剩下的两个标号。当前有 t 个点、$t-2$ 个剩余序列项，所以至少两个标号没有出现，可选点总存在。u 自己仍在序列中，故不会选出自环。每个被删点只连接到更晚删去的点，最终得到一棵连通的 $n-1$ 边树。

关键不变式是：在当前剩余树中，标号 i 在剩余序列里的出现次数等于 d_i-1。每个点最后一条关联边在它作为叶子被删除时移除，或在最后两点之间保留到结束；这一条不记录该点。此前每删除一个邻点，就记录一次它。每一步删除一个叶子，会同时使邻点的度数和出现次数减少 1，因此不变式继续成立。反过来看解码得到的剩余树，每个活跃点有一条最终连接边，再加上剩余序列每次出现它时接来的边；其度数同样是剩余出现次数加 1。所以未出现的活跃标号恰好是叶子；两边都取最小者，编码和解码逐步互为逆操作。特别地，原树满足

$$d_i = 1 + \#\{\text{序列中等于}i\text{ 的项}\}.$$

本节给出计数依据，不新增可调用的编码/解码 API。

54.1.2　Cayley、指定度数与叶子数

任意长度 $n-2$ 的序列都对应唯一树，故树总数为 n^{n-2}。给定整数度数 $d_1,\dots,d_n$，若某个 $d_i<1$ 或 $\sum_i d_i\neq 2n-2$，答案为 0；否则各标号出现次数已确定，答案是多项式系数

$$\frac{(n-2)!}{\prod_{i=1}^{n}(d_i-1)!}.$$

例如 $n=4$，度数 $(3,1,1,1)$ 给出唯一指定中心的星形树，度数 $(2,2,1,1)$ 则有 2 棵，而不是把两名度为 2 的顶点视作不可区分。

叶子恰是没有出现的标号。恰有 ℓ 个叶子的树数为

$$\binom{n}{\ell}\,(n-\ell)!\,S(n-2,n-\ell),$$

其中 S 为第二类 Stirling 数：先选叶子，再把序列位置满射到其余标号。$n\ge 3$ 时合法范围为 $2\le\ell\le n-1$；其他叶子数为 0。$n=2,\ell=2$ 时用 $S(0,0)=1$ 得 1。若把叶子定义为度数 1，单点树没有叶子，仍单独处理。这里是整数计数恒等式；取模时不能无条件把阶乘分母换成逆元。素数模的预处理与下标限制见第 21.1.1 节（第 276 页），一般模数条件见数学分册“模运算的适用条件”。有根标号树及其生成函数见数学分册“隐式生成函数与拉格朗日反演”，不重复另造树计数模板。

54.1.3 加权 Cayley 与连接已有连通块

若完整图的边权有特殊乘积形式 $w_{ij} = x_i x_j$，则

$$\sum_T \prod_{ij \in T} w_{ij} = \sum_T \prod_i x_i^{d(i)} = \left(\prod_i x_i\right)\left(\sum_i x_i\right)^{n-2}.$$

这里 $d(i)$ 表示点 i 的度数。第二步利用出现次数恒等式，把每棵树变成公共因子 $\prod_i x_i$ 乘以其序列各项的权；对所有序列求和就是展开右侧的幂。证明没有除以任何 x_i，所以零权、负权以及任意正模数都可按多项式恒等式使用。$n = 2$ 的零次幂是常数 1，即使 $\sum_i x_i = 0$ 也如此。这只适用于上述乘积边权；缺边图或一般边权应使用矩阵树定理，见第 55.6 节（第 973 页）及第 55.4 节（第 967 页）。代数权值和为零也可能来自抵消，不能据此断言没有树。

设一个固定图有 $k \geq 2$ 个连通块，大小为正整数 $s_1, \ldots, s_k$，总顶点数 $N = \sum_i s_i$。允许任选不同连通块的顶点对加边，使全图连通，且恰加 $k-1$ 条边。缩点后新增边必须组成树；一条块间边 ij 有 $s_i s_j$ 种端点选择。因此不同**无序新增边集**的数量是

$$N^{k-2} \prod_{i=1}^{k} s_i.$$

$k = 1$ 时无需加边，答案为 1，不计算负指数。原图的内部边固定，不计内部生成树选择。若禁止部分端点对、给每条新增边限制，或数加边的执行顺序，就不能直接使用此式。有标号树也不能简单除以 $n!$ 得到无标号树数，因为不同树的自同构数量不同。

双射背景见 Stanley, Topics in Algebraic Combinatorics，导航与基础公式见 固定版本 OI Wiki Prüfer 序列。以上用最小叶子的一致约定推导；不将本节计数说明当作整个上游页面的实现覆盖。

第五十五章　矩阵树定理

相关应用：生成树计数的相关扩展：Tutte 多项式，T(1,1) 对应连通图生成树数；不将其冒称为矩阵树定理实现。

55.1　Tutte 多项式求值

tutte(adj,x,y) 返回模 998244353 的 Tutte 多项式值。简单无向图，n≤21，点号 0..n-1；adj[u] 为邻居位掩码，须对称、无自环、无越界位。0≤x,y<998244353；支持断连、孤立点及空图（值 1），不支持重边或自环；输入不修改。连通分量 DFS 保持递归。

每个连通分量独立计算并相乘。c[S] 为 S 上连通生成子图按 (y-1) 的环秩计权之和；删去 S 的最小顶点 r 后，每个剩余连通块 T 的连接权为 1+y+...+y^(h-1)，h 为 r 连向 T 的边数，h=0 时为 0。用快速子集指数求集合划分。最后固定顶点 0 所在块，其余各块乘 (x-1)。不除 x-1 或 y-1，x=1、y=1 均直接支持；普通形式指数只需 1..20 的模逆元。时间 O(n^{2}2^n)，峰值空间 O(n2^n)。21 点 Linux 九次连续求值进程峰值约 312MiB；QOJ45 已以 CCF_NOI 获得满分；排名尚未核验。

集合指数为何正确　按集合大小分次数层，zeta 后做普通形式指数，再 Mobius 并取集合大小次项。逆变换只保留并集恰为 S 的项，次数等于 $|S|$ 排除重叠；$1/k!$ 抵消 k 个块的排列。最小顶点从大到小处理时，余下块的 c 值已知。最终固定 0 所在块，其他块各乘 $x-1$，恰好给出定义中的连通分量指数。

输入约定、等比和与分层 zeta：

```
int tutte(const vector<unsigned> &adj, int x, int y)
{
    constexpr int mod = 998244353;
    int n = adj.size();
    assert(n <= 21 && 0 <= x && x < mod && 0 <= y && y < mod);
    for (int u = 0; u < n; u++)
    {
        assert((adj[u] >> n) == 0 && !(adj[u] >> u & 1));
        for (int v = 0; v < n; v++)
            assert((adj[u] >> v & 1) == (adj[v] >> u & 1));
    }
    vector<int> inv(n + 1, 1), geometric(n + 1);
    for (int i = 2; i <= n; i++)
        inv[i] = mod - (long long)(mod / i) * inv[mod % i] % mod;
    for (int i = 1; i <= n; i++)
        geometric[i] = ((long long)geometric[i - 1] * y + 1) % mod;
    auto exponential = [&](const vector<int> &a)
    {
        int size = a.size();
        int k = countr_zero((unsigned)size), width = k + 1;
        vector<int> f((size_t)size * width), g(width), answer(size);
        for (int s = 1; s < size; s++) f[(size_t)s * width + popcount((unsigned)s)] = a[s];
        for (int bit = 1; bit < size; bit <<= 1)
            for (int s = 0; s < size; s++)
                if (s & bit)
                    for (int j = 1; j <= k; j++)
                    {
                        int &value = f[(size_t)s * width + j];
                        value += f[(size_t)(s ^ bit) * width + j];
                        if (value >= mod) value -= mod;
                    }
```

形式指数与 Mobius（接上页函数）：

```
        for (int s = 0; s < size; s++)
        {
            int *row = f.data() + (size_t)s * width;
            g[0] = 1;
            for (int i = 1; i <= k; i++) row[i] = (long long)row[i] * i % mod;
            for (int i = 1; i <= k; i++)
            {
                long long sum = 0;
                for (int j = 1; j <= i; j++)
                    sum = (sum + (long long)row[j] * g[i - j]) % mod;
                g[i] = sum * inv[i] % mod;
            }
            copy(g.begin(), g.end(), row);
        }
        for (int bit = 1; bit < size; bit <<= 1)
            for (int s = 0; s < size; s++)
                if (s & bit)
                    for (int j = 0; j <= k; j++)
                    {
                        int &value = f[(size_t)s * width + j];
                        value -= f[(size_t)(s ^ bit) * width + j];
                        if (value < 0) value += mod;
                    }
        for (int s = 0; s < size; s++) answer[s] = f[(size_t)s * width + popcount((unsigned)s)];
        return answer;
    };
```

连通子集递推及分量乘积（接上页函数）：

```
    auto connected = [&](const vector<unsigned> &g)
    {
        int m = g.size();
        int size = 1 << m;
        vector<int> c(size);
        for (int root = m - 1; root >= 0; root--)
        {
            int count = 1 << (m - root - 1);
            vector<int> a(count);
            for (int s = 1; s < count; s++)
            {
                unsigned t = (unsigned)s << (root + 1);
                a[s] = (long long)c[t] * geometric[popcount(g[root] & t)] % mod;
            }
            auto e = exponential(a);
            for (int s = 0; s < count; s++) c[(s << (root + 1)) | (1 << root)] = e[s];
        }
        if (x == 1) return c.back();
        vector<int> a(size / 2);
        int z = x ? x - 1 : mod - 1;
        for (int s = 1; s < size / 2; s++) a[s] = (long long)c[s << 1] * z % mod;
        auto e = exponential(a);
        long long answer = 0;
        for (int s = 0; s < size / 2; s++)
            answer = (answer + (long long)c[(size - 1) ^ (s << 1)] * e[s]) % mod;
        return (int)answer;
    };
    vector<int> seen(n), vertices;
    auto dfs = [&](auto &&self, int u) -> void
    {
        seen[u] = 1;
        vertices.push_back(u);
        for (int v = 0; v < n; v++)
            if ((adj[u] >> v & 1) && !seen[v]) self(self, v);
    };
    long long answer = 1;
    for (int u = 0; u < n; u++)
        if (!seen[u])
        {
            vertices.clear();
            dfs(dfs, u);
            vector<unsigned> g(vertices.size());
            for (int i = 0; i < (int)vertices.size(); i++)
                for (int j = 0; j < (int)vertices.size(); j++)
                    if (adj[vertices[i]] >> vertices[j] & 1) g[i] |= 1U << j;
            answer = answer * connected(g) % mod;
        }
    return answer;
}
```

模板题使用：QOJ45 Tutte 多项式

题意：输入简单无向图邻接矩阵和 x,y，输出模 998244353 的 Tutte 多项式值；n≤21。

所需模板：tutte（第 956 页，55.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://qoj.ac/problem/45

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<unsigned> adj(n);
    for (int u = 0; u < n; u++)
        for (int v = 0; v < n; v++)
        {
            int edge;
            cin >> edge;
            if (edge) adj[u] |= 1U << v;
        }
    int x, y;
    cin >> x >> y;
    cout << tutte(adj, x, y) << '\n';
}
```

55.2 带权矩阵树：无向树与内外向树形图

代码依赖：ModInt（第 133 页，19.2.2 节）、det_prime（第 319 页，22.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

声明 MatrixTree<mod>，调用 count(n, edges, root, kind) 返回所有合法树的边权乘积之和，mod 必须是素数。Edge=(u,v,Z)，点号 0..n-1，n≥1；Kind::undirected 为无向生成树，toward_root 为每点沿边能到根，away_from_root 为根沿边能到每点。后两种使用输入边方向，不能混淆。重边按不同边计数，自环忽略；单点答案 1。支持模意义下的零权和负权，结果 0 不一定表示不存在树。无向图删除哪个根行列均可；有向图 root 决定答案。依赖 det_prime 的素数模行列式，$O(n^3+m)$ 时间、$O(n^2)$ 额外空间。

```
template <int mod> struct MatrixTree
{
    using Z = ModInt<mod>;
    using Edge = tuple<int, int, Z>;
    enum class Kind
    {
        undirected,
        toward_root,
        away_from_root
    };

    // Sum of products of edge weights; vertices 0..n-1, prime modulus.
    static Z count(int n, const vector<Edge> &edges, int root, Kind kind)
    {
        assert(n >= 1 && root >= 0 && root < n);
        vector<vector<Z>> lap(n - 1, vector<Z>(n - 1));
        auto add = [&](int u, int v, Z w)
        {
            if (u == root) return;
            int i = u - (u > root), j = v - (v > root);
            lap[i][i] = lap[i][i] + w;
            if (v != root) lap[i][j] = lap[i][j] - w;
        };
        for (auto [u, v, w] : edges)
        {
            assert(u >= 0 && u < n && v >= 0 && v < n);
            if (u == v) continue;
            if (kind == Kind::away_from_root) swap(u, v);
            add(u, v, w);
            if (kind == Kind::undirected) add(v, u, w);
        }
        return det_prime<mod>(move(lap));
    }
};
```

模板题使用：Luogu P6178

题意：求所有生成树边权乘积之和，模 1000000007。type=0 为无向图，type=1 为以输入点 1 为根的外向树；输入点号减 1 后根为 0。重边分别计数，自环不贡献。

所需模板：ModInt（第 133 页，19.2.2 节）、det_prime（第 319 页，22.2.1 节）、MatrixTree（第 961 页，55.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6178

```
int main()
{
    using M = MatrixTree<1000000007>;
    int n, m, type;
    scanf("%d%d%d", &n, &m, &type);
    vector<M::Edge> edges;
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        scanf("%d%d%lld", &u, &v, &w);
        edges.push_back({u - 1, v - 1, w});
    }
    auto kind = type ? M::Kind::away_from_root : M::Kind::undirected;
    printf("%d\n", M::count(n, edges, 0, kind).v);
}
```

应用题补充：Luogu P4455 社交网络：根 1 内向树

题意：输入 a b 表示 a 能看见 b；选择 b 为 a 的消息来源，对应 a 指向 b 的父边，所有非根点沿父边最终到达 1。因此按原输入方向调用 toward_root，不能选 away_from_root；点号减 1，根为 0，模 10007。n 不超过 250，时间 O(n^3+m)，空间 O(n^2)。按朋友关系去重，自环忽略；重复关系不视为不同传播方案。不统计转发时间顺序。单点约定空树 1。官方题面已核对，完整协议本地验证；尚无本实现在线记录。

所需模板：ModInt（第 133 页，19.2.2 节）、det_prime（第 319 页，22.2.1 节）、MatrixTree（第 961 页，55.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4455

```
int main()
{
    using M = MatrixTree<10007>;
    int n, m;
    cin >> n >> m;
    vector<vector<int>> seen(n, vector<int>(n));
    vector<M::Edge> edges;
    while (m--)
    {
        int a, b;
        cin >> a >> b;
        a--;
        b--;
        if (a != b && !seen[a][b])
        {
            seen[a][b] = 1;
            edges.push_back({a, b, 1});
        }
    }
    cout << M::count(n, edges, 0, M::Kind::toward_root).v << '\n';
}
```

相关应用：独立边随机图恰为生成树的概率应用；不代表额外来源整页覆盖

55.3 随机图恰为生成树的概率

tree_probability(p) 返回独立无向随机图恰有 n-1 条边且连通的概率。p 为对称整数百分数矩阵，元素 0..100、对角 0，1<=n<=50；输入不变，单点为空树，返回 1。概率 100 的边必须出现，成环直接返回 0，否则先缩成森林分量。其余边赋权 p/(100-p)，矩阵树结果乘所有非必选边的缺边概率；缩点后自环也必须缺失。

先以正概率边检查连通性，严格判定不可行；连通余子式正定，消元主元应为正。不用 eps 把必选边替换成接近 1，也不拿浮点行列式阈值判断连通性。用对数累加概率与主元，最后 exp，避免中间乘积下溢。时间 $O(n^3)$，空间 $O(n^2)$。这是百分数输入域的浮点近似接口，不是任意实数概率的精确算法；超出题面下界的极小结果最终仍可能下溢。long double 在部分平台只有 double 精度。

必选森林与分量编号（同一函数连续抄写）：

```cpp
// Symmetric percentages in [0,100], zero diagonal; 1 <= n <= 50.
long double tree_probability(const vector<vector<int>> &p)
{
    int n = p.size();
    assert(1 <= n && n <= 50);
    vector<int> fa(n);
    iota(fa.begin(), fa.end(), 0);
    auto find = [&](auto &&self, int u) -> int
    {
        if (fa[u] == u)
            return u;
        return fa[u] = self(self, fa[u]);
    };
    for (int u = 0; u < n; u++)
        for (int v = u + 1; v < n; v++)
            if (p[u][v] == 100)
            {
                int a = find(find, u);
                int b = find(find, v);
                if (a == b)
                    return 0;
                fa[a] = b;
            }
    vector<int> id(n, -1);
    int k = 0;
    for (int u = 0; u < n; u++)
    {
        int r = find(find, u);
        if (id[r] == -1)
            id[r] = k++;
        id[u] = id[r];
    }
```

缩点拉普拉斯、连通性与对数行列式（续）：

```
    vector<vector<long double>> a(k - 1, vector<long double>(k - 1));
    long double ans = 0;
    for (int u = 0; u < n; u++)
        for (int v = u + 1; v < n; v++)
        {
            int w = p[u][v];
            if (w > 0)
                fa[find(find, u)] = find(find, v);
            if (w == 100)
                continue;
            ans += logl((100 - w) / 100.0L);
            int x = id[u];
            int y = id[v];
            if (x == y)
                continue;
            long double t = (long double)w / (100 - w);
            if (x < k - 1)
                a[x][x] += t;
            if (y < k - 1)
                a[y][y] += t;
            if (x < k - 1 && y < k - 1)
            {
                a[x][y] -= t;
                a[y][x] -= t;
            }
        }
    for (int u = 0; u < n; u++)
        if (find(find, u) != find(find, 0))
            return 0;
    for (int i = 0; i < k - 1; i++)
    {
        // The connected graph cofactor is positive definite.
        assert(a[i][i] > 0);
        ans += logl(a[i][i]);
        for (int j = i + 1; j < k - 1; j++)
        {
            long double t = a[j][i] / a[i][i];
            for (int z = i + 1; z < k - 1; z++)
                a[j][z] -= t * a[i][z];
        }
    }
    return expl(ans);
}
```

应用题补充：Luogu P3317 重建

题意：输入对称概率矩阵，最多两位小数，转为整数百分数；2<=n<=50。求独立保留边恰好构成树的概率，非零答案保证 >=1e-4，相对误差限 1e-4。固定 12 位输出足够覆盖该下界，不能把相同格式直接用于任意小概率。完整协议本地验证，线上待验证。

所需模板：tree_probability（第 964 页，55.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3317

```
int main()
{
    int n;
    cin >> n;
    vector<vector<int>> p(n, vector<int>(n));
    for (auto &row : p)
        for (int &v : row)
        {
            long double x;
            cin >> x;
            v = lroundl(100 * x);
        }
    cout << fixed << setprecision(12) << tree_probability(p) << '\n';
}
```

55.4 任意正模数带权矩阵树

代码依赖：determinant_mod（第 322 页，22.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

count(n, edges, root, kind, mod) 返回树的边权乘积之和对 mod 的最小非负余数。mod 为任意正 signed 64 位数，边权支持完整 signed 64 位。Kind 的 undirected、toward_root、away_from_root 分别表示无向、内向和外向树；点号 0..n-1。自环忽略，重边按不同边计数，单点返回 1%mod，mod=1 返回零。先在 128 位中构造模拉普拉斯余子式，再调用 determinant_mod；$O(n^3 \log \mathrm{mod} + m)$ 时间、$O(n^2)$ 额外空间。不使用模逆元。已知模数是素数时一般优先使用 MatrixTree<mod>。

```
struct MatrixTreeMod
{
    using ll = long long;
    using Edge = tuple<int, int, ll>;
    enum class Kind
    {
        undirected,
        toward_root,
        away_from_root
    };

    static ll count(int n, const vector<Edge> &edges, int root, Kind kind, ll mod)
    {
        assert(n >= 1 && root >= 0 && root < n && mod >= 1);
        vector<vector<ll>> lap(n - 1, vector<ll>(n - 1));
        auto add = [&](int u, int v, ll w)
        {
            if (u == root) return;
            int i = u - (u > root), j = v - (v > root);
            lap[i][i] = ((__int128)lap[i][i] + w) % mod;
            if (v != root) lap[i][j] = ((__int128)lap[i][j] + mod - w) % mod;
        };
        for (auto [u, v, w] : edges)
        {
            assert(u >= 0 && u < n && v >= 0 && v < n);
            if (u == v) continue;
            w %= mod;
            if (w < 0) w += mod;
            if (kind == Kind::away_from_root) swap(u, v);
            add(u, v, w);
            if (kind == Kind::undirected) add(v, u, w);
        }
        return determinant_mod(move(lap), mod);
    }
};
```

模板题使用：Library Checker Counting Spanning Trees (Undirected)

题意：无向多重图生成树计数，模 998244353；点号 0..n-1，1≤n≤500、0≤m≤500000，允许自环和重边。每条原边权重设 1，自环不参与树，重边是不同选择。删去任意根的拉普拉斯行列，这里取根 0；n=1 的空树计 1，断连计 0。此接口也支持合数模数，复杂度 $O(n^3 \log \mathrm{mod})$，本题固定质数亦可选更快的 MatrixTree 版本。

所需模板：determinant_mod（第 322 页，22.2.2 节）、MatrixTreeMod（第 967 页，55.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/counting_spanning_tree_undirected

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<MatrixTreeMod::Edge> e;
    e.reserve(m);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        e.emplace_back(u, v, 1);
    }
    cout << MatrixTreeMod::count(n, e, 0, MatrixTreeMod::Kind::undirected, 998244353)
         << '\n';
}
```

模板题使用：Library Checker Counting Spanning Trees (Directed)

题意：有向多重图生成树计数，模 998244353，要求指定根 r 能到达所有点，使用 away_from_root。n 不超过 500、m 不超过 500000，点号及根为 0 起；每条原边权 1，自环忽略，重边分别计数。每个非根点恰有一个入边，根无入边；toward_root 表示所有点到达根，不能替代。n=1 计空树 1，无外向生成树计 0。

所需模板：determinant_mod（第 322 页，22.2.2 节）、MatrixTreeMod（第 967 页，55.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/counting_spanning_tree_directed

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, r;
    cin >> n >> m >> r;
    vector<MatrixTreeMod::Edge> e;
    e.reserve(m);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        e.emplace_back(u, v, 1);
    }
    cout << MatrixTreeMod::count(
                n, e, r, MatrixTreeMod::Kind::away_from_root, 998244353)
         << '\n';
}
```

55.5 精确整数带权生成树计数

代码依赖：determinant_exact（第 324 页，22.2.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

MatrixTreeExact<Z>::count(n,edges,root,kind)，1≤n<INT_MAX、点号 0..n-1；Edge 为 tuple<int,int,Z>。kind 为 undirected、toward_root 或 away_from_root，分别计无向生成树、朝根汇聚树、从根发散树。返回各树边权乘积之和；负权和零权合法，答案 0 可能是抵消。重边按独立选择计数，自环忽略；单点返回空树权 1。无向结果与根无关，有向根必须指定。构造出度拉普拉斯，发散模式先反边，删根行列后调用 determinant_exact。$O(m+n^3)$ 次整数算术、$O(n^2)$ 个整数工作对象，不含调用者边表；大整数位成本另计。Z 要求同精确行列式。

```
template<class Z>
struct MatrixTreeExact
{
    using Edge = tuple<int, int, Z>;
    enum class Kind
    {
        undirected,
        toward_root,
        away_from_root
    };

    static Z count(int n, const vector<Edge> &edges, int root, Kind kind)
    {
        assert(n >= 1 && root >= 0 && root < n);
        vector<vector<Z>> lap(n - 1, vector<Z>(n - 1));
        auto add = [&](int u, int v, const Z &w)
        {
            if (u == root)
                return;
            int i = u - (u > root);
            int j = v - (v > root);
            lap[i][i] += w;
            if (v != root)
                lap[i][j] -= w;
        };
        for (auto [u, v, w] : edges)
        {
            assert(0 <= u && u < n && 0 <= v && v < n);
            if (u == v)
                continue;
            if (kind == Kind::away_from_root)
                swap(u, v);
            add(u, v, w);
            if (kind == Kind::undirected)
                add(v, u, w);
        }
        return determinant_exact(move(lap));
    }
};
```

接口用法演示：带权矩阵树的精确整数答案

演示约定：T 组；每组 n,m,root,type 及 m 条 0-based 边 u,v,w。type=0 无向、1 朝根、2 离根。权值为有符号十进制整数；输出各树权积之和。重边独立、自环忽略、单点为 1；负权可抵消成 0。仅 API 用法。

所需模板：determinant_exact（第 324 页，22.2.3 节）、MatrixTreeExact（第 970 页，55.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

额外依赖（不属于标准头文件）：`boost/multiprecision/cpp_int.hpp`，须由编译环境提供并显式包含。

来源：参考文档

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Z = boost::multiprecision::cpp_int;
    using Tree = MatrixTreeExact<Z>;
    int t;
    cin >> t;
    while (t--)
    {
        int n, m, root, type;
        cin >> n >> m >> root >> type;
        vector<Tree::Edge> edges(m);
        for (auto &[u, v, w] : edges)
            cin >> u >> v >> w;
        auto kind = Tree::Kind::undirected;
        if (type == 1)
            kind = Tree::Kind::toward_root;
        if (type == 2)
            kind = Tree::Kind::away_from_root;
        cout << Tree::count(n, edges, root, kind) << '\n';
    }
    return 0;
}
```

接口用法演示：不取模生成树计数（kuangbin 来源模型）

演示约定：T 组；每组 n,m 及 m 条 1-based 无向边。按原稿布尔邻接语义去掉自环、合并重复和反向边，再输出精确树数。核心接口本身不合并重边。原稿标注 SPOJ104，但完整官方题面及资源约束未核验，故仅登记来源模型/API，不声称正式题目或线上 AC。

所需模板：determinant_exact（第 324 页，22.2.3 节）、MatrixTreeExact（第 970 页，55.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

额外依赖（不属于标准头文件）：`boost/multiprecision/cpp_int.hpp`，须由编译环境提供并显式包含。

来源：参考文档

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Tree = MatrixTreeExact<boost::multiprecision::cpp_int>;
    int t;
    cin >> t;
    while (t--)
    {
        int n, m;
        cin >> n >> m;
        set<pair<int, int>> seen;
        while (m--)
        {
            int u, v;
            cin >> u >> v;
            u--;
            v--;
            if (u > v)
                swap(u, v);
            if (u != v)
                seen.insert({u, v});
        }
        vector<Tree::Edge> edges;
        for (auto [u, v] : seen)
            edges.emplace_back(u, v, 1);
        cout << Tree::count(n, edges, n - 1, Tree::Kind::undirected) << '\n';
    }
    return 0;
}
```

55.6 带权矩阵树定理与方向约定

实现见第 55.2 节（第 961 页）。返回的是每棵树的边权乘积，再对所有树求和，并对素数取模。无权计数只需把所有边权设为 1。重边是不同选择，自环不属于任何生成树；单点图的空树乘积为 1。零权和负权都按域上的运算处理，因此结果为零可能来自抵消，不能作为连通性判定。

无向边 uv 权值 w 对拉普拉斯矩阵贡献为 L_{uu}, L_{vv} 各加 w，L_{uv}, L_{vu} 各减 w。删除 root 行和列后求行列式，与 root 的选择无关。对于每个点都能沿有向边走到根的树形图，边 $u \to v$ 贡献 $L_{uu} \mathrel{+}= w$、$L_{uv} \mathrel{-}= w$；删除根行列后得到内向树形图的权值和。外向树形图先反转所有边，再使用同一构造。

例如只有一条边 $0 \to 1$ 时，内向根 1 的答案为该边权，内向根 0 为零；外向根 0 的答案为该边权。P6178 的有向模式（type=1）指定以 1 为根的外向树；type=0 为无向图，完整用法见第 55.2 节（第 962 页）；驱动把点号减一，有向模式选择 `away_from_root`。只直接构造 $(n-1)\times(n-1)$ 余子式，时间 $O(n^3+m)$、额外空间 $O(n^2)$。行列式复用本书的素数模高斯消元，不支持将 mod 随意改成合数。

55.6.1 为什么这个余子式恰好计数

先证内向情形。删去根 r 的坐标，令 $e_r = 0$，其他 e_i 为单位行向量。余子式第 i 行是 $\sum_{i\to j} w_{ij}(e_i - e_j)$。按行的多线性展开，每项从每个非根点恰选一条出边，权为所选边权之积。若这些边含有有向环，环上向量 $e_i - e_j$ 相加为零，该项行列式为零。若无环，有限图中每条路径只能在没有出边的根终止。按到根距离从小到大，同时重排行列，得到对角全为 1 的三角矩阵，行列式为 1。因此只有内向生成树保留下来，每棵树的权恰计一次。

自环的向量为零；重边在展开时是不同选择；单点对应空行列式与空乘积 1。无向边替换成权相同的两条反向弧，每棵无向生成树恰有一种朝根定向；有向边全部反转则把外向树变成内向树。整个论证是整数系数多项式恒等式，没有除法，故允许负权、零权及任意正模数。素数限制来自消元实现，不来自矩阵树定理本身。

证明依据行列式多线性与有向树的出度刻画，参见 MIT 18.212 第 29 讲；这里直接按选边后的环与根路径给出主余子式的证明，不扩展为任意非主余子式公式。

55.6.2 既有接口与验证记录

测试独立枚举 $n-1$ 条边的所有选择，检查连通性或根向约束，用任意精度整数计算边权乘积之和，再分别对多个素数取模。两套实现覆盖所有根、三种方向、自环重边、负权及 64 位权值边界，并通过 220 点完全图的加权 Cayley 公式检查。P6178 完整驱动另测 300 点、十万条边以及输入输出约定。普通与 ASan/UBSan 均通过。两套代码已在 P6178 通过全部 11 个测试点：动态版记录 297619886、传统版记录 297620263。单点最大用时分别为 64 ms、62 ms，内存峰值为 2.85 MB、2.40 MB。在线测试覆盖无向与根为 1 的外向树；其他根、内向树和负权仍使用本地证据。定理背景可参考 OI Wiki 矩阵树定理。

55.7 任意正模数的矩阵树计数

实现见第 55.4 节（第 967 页）。拉普拉斯余子式表示生成树权值和的恒等式本身不要求模数为素数；需要换掉的是依赖模逆元的行列式算法。本节沿用第 55.6 节（第 973 页）的三种构造，但使用欧几里得行列式消元。输入边权和正模数均支持 signed 64 位，接口名为 `MatrixTreeMod::count`。行列式实现见第 22.2.2 节（第 322 页），时间 $O(n^3 \log(m+1) + |E|)$、空间 $O(n^2)$；此处 m 为模数。

先把边权正规化到 $[0, m)$，每次矩阵元素加减仍先扩展至 128 位，再取模存回 64 位。重边的权值和可能超过 64 位范围，不能先累加到普通整数中。只构造删除根后的余子式，并将其存储直接交给行列式函数。单点返回 $1 \bmod m$，模 1 返回零。负权与零权仍按代数权值处理，返回零不能直接解释为没有生成树。

独立参考程序逐个枚举边集，用任意精度整数计算权值和。已检查多个合数模数、所有根与三种方向、断图、重边自环及 64 位极值权重；另以大权值完全图的 Cayley 公式验证 160 点规模。普通与 ASan/UBSan 均通过，尚无该接口的在线 AC。此前 P6178 的 AC 属于素数模版本，不自动转移到本接口。

第五十六章　LGV 引理

56.1　LGV：DAG 路径权值行列式

代码依赖：ModInt（第 133 页，19.2.2 节）、det_prime（第 319 页，22.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

dag_path_determinant<mod>(g,sources,sinks) 返回路径权值矩阵的行列式，mod 必须为素数。g[u] 的项为 (v,ModInt<mod> 权值)，点号 0..n-1；两个端点序列等长 k，各自不能重复，但可相互重叠。允许重边、零权、负权；长度为 0 的路径权为 1。整张图有环则返回 nullopt，空端点族在 DAG 上返回 1。由 LGV，它是顶点不交路径族按汇点排列带符号的权值和；只有能证明非恒等排列均不可能时，才是对应配对的普通计数。顺序不能随意排序，模结果为 0 也不能据此断言不存在路径族。时间 $O((k+1)(n+m)+k^3+k \log mod)$，额外空间 $O(n+k^2)$，依赖 det_prime 和 ModInt。

```
template <int mod>
optional<ModInt<mod>>
dag_path_determinant(const vector<vector<pair<int, ModInt<mod>>>> &g,
                     const vector<int> &sources,
                     const vector<int> &sinks)
{
    using Z = ModInt<mod>;
    int n = g.size(), k = sources.size();
    assert((int)sinks.size() == k);
    vector<int> in(n), order;
    vector<bool> seen_source(n), seen_sink(n);
    for (int i = 0; i < k; i++)
    {
        assert(0 <= sources[i] && sources[i] < n);
        assert(0 <= sinks[i] && sinks[i] < n);
        assert(!seen_source[sources[i]] && !seen_sink[sinks[i]]);
        seen_source[sources[i]] = seen_sink[sinks[i]] = true;
    }
    for (const auto &edges : g)
        for (auto [v, w] : edges)
        {
            assert(0 <= v && v < n);
            in[v]++;
        }
    for (int u = 0; u < n; u++)
        if (!in[u]) order.push_back(u);
    for (int i = 0; i < (int)order.size(); i++)
        for (auto [v, w] : g[order[i]])
            if (!--in[v]) order.push_back(v);
    if ((int)order.size() != n) return nullopt;
```

```
    vector<vector<Z>> a(k, vector<Z>(k));
    for (int i = 0; i < k; i++)
    {
        vector<Z> dp(n);
        dp[sources[i]] = 1;
        for (int u : order)
            for (auto [v, w] : g[u]) dp[v] = dp[v] + dp[u] * w;
        for (int j = 0; j < k; j++) a[i][j] = dp[sinks[j]];
    }
    return det_prime<mod>(move(a));
}
```

应用题补充：DAG 路径行列式（本地接口示例）

题意：输入 n,m,k、m 条有向带权边、k 个起点和 k 个终点，均为 0-based。输出模 998244353 的带符号路径族权值和，整图有环输出 CYCLE。端点顺序原样保留，不能把负号当作错误去掉。

所需模板：ModInt（第 133 页，19.2.2 节）、det_prime（第 319 页，22.2.1 节）、dag_path_determinant（第 975 页，56.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://github.com/gsh20040816/CPCtemplate/issues/11

```
int main()
{
    using Z = ModInt<998244353>;
    int n, m, k;
    cin >> n >> m >> k;
    vector<vector<pair<int, Z>>> g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g[u].push_back({v, Z(w)});
    }
    vector<int> sources(k), sinks(k);
    for (int &u : sources) cin >> u;
    for (int &u : sinks) cin >> u;
    auto ans = dag_path_determinant<998244353>(g, sources, sinks);
    if (!ans)
        cout << "CYCLE\n";
    else
        cout << ans->v << '\n';
}
```

第五十七章　支配树

57.1　支配树：Lengauer–Tarjan

DominatorTree(n)，点号 1..n；add(u,v) 加有向边，允许重边、自环。build(root) 后 idom 为原点号直接支配点，根指向自身，不可达为 0。仅对 root 可达点定义支配关系：所有 root 到 v 的路径均经过 u，等价于 u 在支配树上是 v 的祖先。timer 是可达点数，vertex[1..timer] 为 DFS 顺序，dfn 为逆映射；逆序向 idom 累加可得支配点数，根不向自身累加。g 保留原点号；pred、bucket、semi 等辅助数组使用 DFS 编号。可更换根重建；加边使预处理失效，需重新 build。简单 LT 时间 O((n+m) log(n+1))，空间及递归栈 O(n+m) 与 O(n)，n+1、边数须可由 int 表示。一般图与 DAG 均适用；可由 idom 建树组合 LCA。

```
struct DominatorTree
{
    int n, timer = 0;
    vector<vector<int>> g, pred, bucket;
    vector<int> dfn, vertex, parent, semi, label, ancestor, dom, idom;

    DominatorTree(int n) : n(n), g(n + 1) { assert(n > 0); }

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        g[u].push_back(v);
        idom.clear();
    }

    void dfs(int u)
    {
        dfn[u] = ++timer;
        vertex[timer] = u;
        for (int v : g[u])
            if (!dfn[v])
            {
                dfs(v);
                parent[dfn[v]] = dfn[u];
            }
    }

    void compress(int v)
    {
        int a = ancestor[v];
        if (ancestor[a])
        {
            compress(a);
            if (semi[label[a]] < semi[label[v]]) label[v] = label[a];
            ancestor[v] = ancestor[a];
        }
    }
```

路径查询、半支配点与直接支配点计算（接上页同一结构体）：

```
    int eval(int v)
    {
        compress(v);
        return label[v];
    }

    void build(int root)
    {
        assert(1 <= root && root <= n);
        timer = 0;
        dfn.assign(n + 1, 0);
        vertex.assign(n + 1, 0);
        parent.assign(n + 1, 0);
        ancestor.assign(n + 1, 0);
        dom.assign(n + 1, 0);
        idom.assign(n + 1, 0);
        pred.assign(n + 1, {});
        bucket.assign(n + 1, {});
        semi.resize(n + 1);
        iota(semi.begin(), semi.end(), 0);
        label = semi;
        dfs(root);
        for (int u = 1; u <= n; u++)
            if (dfn[u])
                for (int v : g[u]) pred[dfn[v]].push_back(dfn[u]);
        for (int i = timer; i >= 2; i--)
        {
            for (int p : pred[i]) semi[i] = min(semi[i], semi[eval(p)]);
            bucket[semi[i]].push_back(i);
            ancestor[i] = parent[i];
            for (int v : bucket[parent[i]])
            {
                int u = eval(v);
                dom[v] = semi[u] < semi[v] ? u : parent[i];
            }
            bucket[parent[i]].clear();
        }
        for (int i = 2; i <= timer; i++)
        {
            if (dom[i] != semi[i]) dom[i] = dom[dom[i]];
            idom[vertex[i]] = vertex[dom[i]];
        }
        idom[root] = root;
    }
};
```

模板题使用：Library Checker dominatortree

题意：从指定起点求每个点的直接支配点。输入输出是 0-based，调用模板时加 1；输出减 1 后，不可达点为 -1，根仍指向自身。

所需模板：DominatorTree（第 977 页，57.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/dominatortree

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    DominatorTree t(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        t.add(u + 1, v + 1);
    }
    t.build(s + 1);
    for (int u = 1; u <= n; u++) cout << t.idom[u] - 1 << (u == n ? '\n' : ' ');
}
```

模板题使用：Luogu P5180

题意：从点 1 出发，输出各点支配的可达点数，包含自己。按 DFS 序逆序向直接支配点累加，根不向自身累加；不可达点输出 0。

所需模板：DominatorTree（第 977 页，57.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5180

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    DominatorTree t(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        t.add(u, v);
    }
    t.build(1);
    vector<int> ans(n + 1);
    for (int i = t.timer; i >= 1; i--)
    {
        int u = t.vertex[i];
        ans[u]++;
        if (u != 1) ans[t.idom[u]] += ans[u];
    }
    for (int u = 1; u <= n; u++) cout << ans[u] << (u == n ? '\n' : ' ');
```

```
}
```

57.2 DAG 支配树：拓扑序与在线倍增 LCA

DagDominator(n)，1≤n<INT_MAX，点号 1..n；add(u,v)，允许重边。build(root) 检查全图 DAG 并返回 bool，任意分量有环（含自环）都返回 false，同时清空 idom/dep/up/order，不能使用半成品。成功后 idom[root]=root，不可达点 idom=dep=0；order 为全图拓扑序。仅从可达前驱累积 LCA，不可达前驱不能影响结果。lca(u,v) 仅在成功 build 后对可达点使用，0 可作空累积值，lca(0,v)=v。idom 是直接支配点，可按逆 order 向父亲累加子树大小，跳过不可达点和根的自累加。可更换 root 重建；加边后旧结果失效，须重新 build。g 只通过 add 改动，结果数组只读。时间 O((n+m)log(n+1))，空间 O(nlog(n+1)+m)，入度须可由 int 表示。一般图请使用 DominatorTree；多入口按题意添加虚拟根，不可擅自把所有无入度点当同一个入口。

```
struct DagDominator
{
    int n, lg = 1;
    vector<vector<int>> g, up;
    vector<int> idom, dep, order;

    DagDominator(int n) : n(n), g(n + 1)
    {
        assert(n > 0);
        while ((1LL << lg) <= n)
            lg++;
    }

    void add(int u, int v)
    {
        g[u].push_back(v);
    }

    int lca(int u, int v) const
    {
        if (!u || !v)
            return u + v;
        if (dep[u] < dep[v])
            swap(u, v);
        for (int k = lg - 1; k >= 0; k--)
            if (dep[up[k][u]] >= dep[v])
                u = up[k][u];
        if (u == v)
            return u;
        for (int k = lg - 1; k >= 0; k--)
            if (up[k][u] != up[k][v])
            {
                u = up[k][u];
                v = up[k][v];
            }
        return up[0][u];
    }
```

全图拓扑检查与逐点构建支配树（接上页同一结构体）：

```
    bool build(int root)
    {
        assert(1 <= root && root <= n);
        idom.clear();
        dep.clear();
        up.clear();
        order.clear();
        vector<int> deg(n + 1);
        for (int u = 1; u <= n; u++)
            for (int v : g[u])
                deg[v]++;
        for (int u = 1; u <= n; u++)
            if (!deg[u])
                order.push_back(u);
        for (int i = 0; i < (int)order.size(); i++)
            for (int v : g[order[i]])
                if (--deg[v] == 0)
                    order.push_back(v);
        if ((int)order.size() != n)
        {
            order.clear();
            return false;
        }
        idom.assign(n + 1, 0);
        dep.assign(n + 1, 0);
        up.assign(lg, vector<int>(n + 1));
        idom[root] = root;
        for (int u : order)
        {
            if (!idom[u])
                continue;
            dep[u] = u == root ? 1 : dep[idom[u]] + 1;
            up[0][u] = idom[u];
            for (int k = 1; k < lg; k++)
                up[k][u] = up[k - 1][up[k - 1][u]];
            for (int v : g[u])
                idom[v] = lca(idom[v], u);
        }
        return true;
    }
};
```

接口用法演示：DAG 直接支配点与全图环检查

演示约定：输入 n,m,root 与 m 条 1-based 有向边。若任意分量有环，输出-1；否则输出每点 idom，根指向自身，不可达为 0。本例演示纯 DAG 接口，成功仅表示全图无环，不能说明每点都从 root 可达。

所需模板：DagDominator（第 981 页，57.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/dominator-tree/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, root;
    cin >> n >> m >> root;
    DagDominator d(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        d.add(u, v);
    }
    if (!d.build(root))
    {
        cout << -1 << '\n';
        return 0;
    }
    for (int u = 1; u <= n; u++)
        cout << d.idom[u] << (u == n ? '\n' : ' ');
    return 0;
}
```

模板题使用：Library Checker dominatortree（DAG/LT 组合）

题意：一般有向图指定起点求直接支配点，n,m≤200000，输入输出 0-based；根指向自身，不可达输出-1。先调用 DagDominator，若非 DAG 则把图移交 DominatorTree 求解。本用法验证两种算法组合，不能声称 DAG 算法单独支持有环图，既有 LT 线上记录不转移到新程序。

所需模板：DagDominator（第 981 页，57.2 节）、DominatorTree（第 977 页，57.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/dominatortree

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, root;
    cin >> n >> m >> root;
    root++;
    DagDominator d(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        d.add(u + 1, v + 1);
    }
    vector<int> ans;
    if (d.build(root))
        ans = move(d.idom);
    else
    {
        DominatorTree t(n);
        t.g = move(d.g);
        t.build(root);
        ans = move(t.idom);
    }
    for (int u = 1; u <= n; u++)
        cout << ans[u] - 1 << (u == n ? '\n' : ' ');
    return 0;
}
```

应用题补充：Luogu P2597 / ZJOI2012 灾难

题意：输入无环食物网，n≤65534，每行列出该生物的食物，0 结束；食物向捕食者连边。为所有生产者添加虚拟根 n+1，build 后逆拓扑累加支配树子树大小。输出每种生物灭绝后额外灭绝的数量，不含自身。竞赛应用，不能替代独立模板题；尚无本项目线上 AC。

所需模板：DagDominator（第 981 页，57.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2597

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    int root = n + 1;
    DagDominator d(root);
    for (int v = 1; v <= n; v++)
    {
        int u;
        cin >> u;
        if (!u)
            d.add(root, v);
        while (u)
        {
            d.add(u, v);
            cin >> u;
        }
    }
    if (!d.build(root))
        return 0;
    vector<int> size(root + 1, 1);
    for (int i = (int)d.order.size() - 1; i >= 0; i--)
    {
        int u = d.order[i];
        if (u != root)
            size[d.idom[u]] += size[u];
    }
    for (int u = 1; u <= n; u++)
        cout << size[u] - 1 << '\n';
    return 0;
}
```

第七部分

计算几何

第五十八章　二维计算几何基础

58.1　浮点点向量、圆与交点结果

R 为 long double；Point 支持向量加减、数乘和数除（除数非零）。dot/cross/norm/perp 为基本运算，norm 使用 hypotl；Circle 保存圆心和非负半径。Kind 为 none、one、two、infinite、degenerate；Result 同时保存类别与有限交点。输入须有限，尺度须使后续平方、乘积不溢出或下溢；排序不使用 epsilon。具体容差与退化约定见各函数。

```
struct RealPlane
{
    using R = long double;

    struct Point
    {
        R x = 0, y = 0;

        Point operator+(Point b) const { return {x + b.x, y + b.y}; }

        Point operator-(Point b) const { return {x - b.x, y - b.y}; }

        Point operator*(R k) const { return {x * k, y * k}; }

        Point operator/(R k) const { return {x / k, y / k}; }
    };

    struct Circle
    {
        Point o;
        R r;
    };
```

```
    enum class Kind
    {
        none,
        one,
        two,
        infinite,
        degenerate
    };

    struct Result
    {
        Kind kind;
        vector<Point> p;
    };

    static R dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }

    static R cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }

    static R norm(Point a) { return hypotl(a.x, a.y); }

    static Point perp(Point a) { return {-a.y, a.x}; }
};
```

模板题使用：AOJ CGL_2_A

题意：每组给两条非退化直线，平行输出 2，垂直输出 1，否则输出 0。输入为绝对值不超过 10000 的整数，点积与叉积在此范围可由浮点精确表示，所以直接比较 0；这不适用于一般实数或更大整数。只演示点向量与 dot/cross，不涵盖圆及 Result 分类。

所需模板：RealPlane（第 987 页，58.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_2_A

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    while (q--)
    {
        RealPlane::Point a, b, c, d;
        cin >> a.x >> a.y >> b.x >> b.y >> c.x >> c.y >> d.x >> d.y;
        auto u = b - a, v = d - c;
        if (RealPlane::cross(u, v) == 0)
            cout << 2 << '\n';
        else if (RealPlane::dot(u, v) == 0)
            cout << 1 << '\n';
        else
            cout << 0 << '\n';
    }
}
```

58.2 有限二进制浮点极角精确排序

代码依赖：RealPlane（第 987 页，58.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

RealPolarLess{}(a,b) 沿用 RealPlane::Point 存储，输入为实际已存入的有限二进制 long double，尾数 ≤64 位且 uint128 可用；禁 fast-math 及 FTZ/DAZ。原点最前，正 x 轴起逆时针 $[0,2\pi)$，同向按长度递增，同点及 ±0 等价，满足严格弱序。用 frexp/ldexp 拆尾数并以 uint128 精确比较乘积，不做浮点叉乘、不用 epsilon。每次 O(1)，无分配。十进制输入先舍入；不修复调用前的坐标差，不替代其他浮点几何谓词。product/cross_sign 是有限输入下的内部辅助。

```
struct RealPolarLess
{
    using R = RealPlane::R;
    using P = RealPlane::Point;
    using U = __uint128_t;
    static_assert(numeric_limits<R>::radix == 2 && numeric_limits<R>::digits <= 64);

    struct Product
    {
        int sign;
        long long exponent;
        U value;
    };

    static Product product(R a, R b)
    {
        if (a == 0 || b == 0) return {0, 0, 0};
        int x, y;
        constexpr int p = numeric_limits<R>::digits;
        uint64_t u = ldexp(frexp(abs(a), &x), p);
        uint64_t v = ldexp(frexp(abs(b), &y), p);
        U m = U(u) * v;
        uint64_t hi = m >> 64;
        int bits = hi ? 64 + bit_width(hi) : bit_width(uint64_t(m));
        return {signbit(a) == signbit(b) ? 1 : -1,
                (long long)x + y - 2 * p + bits, m << (128 - bits)};
    }

    static int cross_sign(P a, P b)
    {
        Product u = product(a.x, b.y), v = product(a.y, b.x);
        if (u.sign != v.sign) return u.sign < v.sign ? -1 : 1;
        if (!u.sign) return 0;
        int c = u.exponent != v.exponent ? (u.exponent < v.exponent ? -1 : 1)
              : u.value != v.value ? (u.value < v.value ? -1 : 1) : 0;
        return u.sign * c;
    }
```

接上页同一结构体：极角比较接口。

```
    bool operator()(P a, P b) const
    {
        assert(isfinite(a.x) && isfinite(a.y) && isfinite(b.x) && isfinite(b.y));
        bool az = a.x == 0 && a.y == 0, bz = b.x == 0 && b.y == 0;
        if (az || bz) return az && !bz;
        auto half = [](P p) { return p.y < 0 || (p.y == 0 && p.x < 0); };
        if (half(a) != half(b)) return half(a) < half(b);
        int c = cross_sign(a, b);
        if (c) return c > 0;
        return a.x != 0 ? abs(a.x) < abs(b.x) : abs(a.y) < abs(b.y);
    }
};
```

接口用法演示：自定义有限浮点向量极角排序

演示约定：自定义协议，非 OJ 题：0≤n≤200000，接着读 n 对坐标，须成功转换为有限 long double；读入失败、n 越界或非有限值返回 1，不作额外词法或 EOF 校验。输出 1-based 原编号：原点最前，正 x 轴起逆时针 $[0,2\pi)$，同向按长度，完全相同的数值再按原编号。±0 视为相同。比较的是转换后的二进制值，不是舍入前十进制实数；沿用本节尾数 ≤64 位及正常浮点环境条件。std::sort 共 O(n log(n+1)) 次比较，向量存储 O(n)。

所需模板：RealPlane（第 987 页，58.1 节）、RealPolarLess（第 989 页，58.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/src/compact/real_polar.hpp

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n) || n < 0 || n > 200000) return 1;
    using P = RealPlane::Point;
    vector<pair<P, int>> points(n);
    for (int i = 0; i < n; i++)
    {
        auto &p = points[i].first;
        if (!(cin >> p.x >> p.y) || !isfinite(p.x) || !isfinite(p.y)) return 1;
        points[i].second = i + 1;
    }
    RealPolarLess less;
    sort(points.begin(), points.end(), [&](const auto &a, const auto &b)
    {
        return less(a.first, b.first) ||
               (!less(b.first, a.first) && a.second < b.second);
    });
    for (const auto &p : points) cout << p.second << ' ';
    cout << '\n';
}
```

58.3 点在直线上的投影

代码依赖：RealPlane（第 987 页，58.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

line_projection(p,a,b) 返回 p 在过 a、b 直线上的正交投影；若 a=b，则约定返回 a。类型为 RealPlane::Point。输入有限且尺度适当，平方和与点积须能可靠表示；不使用 epsilon 把短线段自动视为零。O(1) 时间和空间。

```
RealPlane::Point
line_projection(RealPlane::Point p, RealPlane::Point a, RealPlane::Point b)
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    Point v = b - a;
    R d = G::dot(v, v);
    if (d == 0) return a;
    return a + v * (G::dot(p - a, v) / d);
}
```

模板题使用：AOJ CGL_1_A

题意：固定直线两端点，对每个点输出其在无限直线上的投影。题目保证两端点不同，整数坐标绝对值不超过 10000，q 不超过 1000；投影可在线段外。接口参数为 (p,a,b)，不截断到线段；a=b 时模板返回 a 属于额外边界。

所需模板：RealPlane（第 987 页，58.1 节）、line_projection（第 991 页，58.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_1_A

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    RealPlane::Point a, b;
    int q;
    cin >> a.x >> a.y >> b.x >> b.y >> q;
    cout << fixed << setprecision(12);
    while (q--)
    {
        RealPlane::Point p;
        cin >> p.x >> p.y;
        auto h = line_projection(p, a, b);
        cout << h.x << ' ' << h.y << '\n';
    }
}
```

58.4 浮点两直线交点

代码依赖：RealPlane（第 987 页，58.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

line_intersection_real(a,b,c,d,eps=1e-12L) 返回 RealPlane::Result；eps 必须为正。端点重合使任一直线退化时返回 degenerate。按方向叉积与方向长度乘积的相对阈值判断平行，再按位置阈值区分 none 与 infinite；唯一交点返回 one。输入有限且尺度适当；近似分类不等同于精确共线判定。O(1)。

```
RealPlane::Result line_intersection_real(RealPlane::Point a,
                                         RealPlane::Point b,
                                         RealPlane::Point c,
                                         RealPlane::Point d,
                                         long double eps = 1e-12L)
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Kind = G::Kind;
    assert(eps > 0);
    Point u = b - a, v = d - c;
    R nu = G::norm(u), nv = G::norm(v);
    if (nu == 0 || nv == 0) return {Kind::degenerate, {}};
    R det = G::cross(u, v);
    if (fabsl(det) <= eps * nu * nv)
        return {fabsl(G::cross(c - a, u)) <= eps * nu * max(R(1), G::norm(c - a))
                    ? Kind::infinite
                    : Kind::none,
                {}};
    return {Kind::one, {a + u * (G::cross(c - a, v) / det)}};
}
```

模板题使用：AOJ CGL_2_C

题意：输出两条线段的唯一交点；题目保证非平行且相交，q 不超过 1000，整数坐标绝对值不超过 10000，因此可直接取 one 结果。模板计算支撑直线交点，不自动检查交点在线段内；一般输入须先检查 kind。整数方向叉积非零时绝对值至少 1，大于本题范围下默认相对阈值。

所需模板：RealPlane（第 987 页，58.1 节）、line_intersection_real（第 992 页，58.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_2_C

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    cout << fixed << setprecision(12);
    while (q--)
    {
        RealPlane::Point a, b, c, d;
        cin >> a.x >> a.y >> b.x >> b.y >> c.x >> c.y >> d.x >> d.y;
        auto ans = line_intersection_real(a, b, c, d);
        assert(ans.kind == RealPlane::Kind::one);
        cout << ans.p[0].x << ' ' << ans.p[0].y << '\n';
    }
}
```

58.5 浮点直线与圆交点

代码依赖：RealPlane（第 987 页，58.1 节）、line_projection（第 991 页，58.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

line_circle_intersections(a,b,c,eps=1e-12L) 返回 Result；c 为 RealPlane::Circle，半径非负，eps 非负；以下相对阈值描述适用于 eps>0。a=b 返回 degenerate；按投影点到圆心的距离和半径的相对阈值区分 none、one、two。点圆允许；只有正半径同心圆才涉及无限交点，本函数不会返回 infinite。依赖 line_projection，O(1)。输入有限且尺度适当，接近相切时按容差归类。默认容差不能直接满足 AOJ CGL_7_D 的所有合法输入；近切时归并交点可能超出坐标误差限。eps=0 为显式无容差模式：直接计算平方判别式，不主动合并近切点；一般实数仍可能发生舍入误判、溢出或下溢。整数坐标绝对值 ≤10000、整数半径 0..10000，二进制 long double 尾数 ≥64 位且正常最近舍入、禁用 fast-math 时，中间整数精确。交点坐标仍为浮点近似；其他输入不继承这一保证。

```
RealPlane::Result line_circle_intersections(RealPlane::Point a,
                                            RealPlane::Point b,
                                            RealPlane::Circle c,
                                            long double eps = 1e-12L)
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Kind = G::Kind;
    assert(eps >= 0);
    assert(c.r >= 0);
    Point v = b - a;
    if (eps == 0)
    {
        R q = G::dot(v, v);
        if (q == 0) return {Kind::degenerate, {}};
        R z = G::cross(v, a - c.o);
        R d = c.r * c.r * q - z * z;
        if (d < 0) return {Kind::none, {}};
        Point h = c.o + G::perp(v) * (z / q);
        if (d == 0) return {Kind::one, {h}};
        Point w = v * (sqrtl(d) / q);
        return {Kind::two, {h - w, h + w}};
    }
    R len = G::norm(v);
    if (len == 0) return {Kind::degenerate, {}};
    Point h = line_projection(c.o, a, b);
    R d = G::norm(h - c.o), tol = eps * max(d, c.r);
    if (d > c.r + tol) return {Kind::none, {}};
    if (fabsl(d - c.r) <= tol) return {Kind::one, {h}};
    R t = sqrtl(max(R(0), (c.r - d) * (c.r + d)));
    Point w = v * (t / len);
    return {Kind::two, {h - w, h + w}};
}
```

模板题使用：AOJ CGL_7_D —浮点接口无容差模式

题意：整数坐标绝对值 ≤10000、整数半径 1..10000，保证交点存在；每条直线两端点不同，1≤q≤1000。直接调用浮点接口并显式传 eps=0，不合并近切交点；仅在二进制 long double 尾数至少 64 位、正常最近舍入且禁用 fast-math 的平台采用此用法。此整数域的平方判别式精确，坐标仍为浮点构造；一般实数不继承精确分类保证。相切重复同一点，按普通 (x,y) 字典序输出，不能用 epsilon 排序；默认 eps>0 接口保持原近似语义。

所需模板：RealPlane（第 987 页，58.1 节）、line_projection（第 991 页，58.3 节）、line_circle_intersections（第 994 页，58.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_D

```
int main()
{
    static_assert(numeric_limits<long double>::radix == 2 &&
                  numeric_limits<long double>::digits >= 64);
#ifdef __FAST_MATH__
#error This example requires normal floating arithmetic
#endif
    if (fegetround() != FE_TONEAREST) return 1;
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    RealPlane::Circle c;
    int q;
    cin >> c.o.x >> c.o.y >> c.r >> q;
    cout << fixed << setprecision(12);
    while (q--)
    {
        RealPlane::Point a, b;
        cin >> a.x >> a.y >> b.x >> b.y;
        auto ans = line_circle_intersections(a, b, c, 0);
        assert(ans.p.size() == 1 || ans.p.size() == 2);
        if (ans.p.size() == 1) ans.p.push_back(ans.p[0]);
        if (tie(ans.p[1].x, ans.p[1].y) < tie(ans.p[0].x, ans.p[0].y))
            swap(ans.p[0], ans.p[1]);
        cout << ans.p[0].x << ' ' << ans.p[0].y << ' ' << ans.p[1].x << ' '
             << ans.p[1].y << '\n';
    }
}
```

58.6 浮点两圆交点

代码依赖：RealPlane（第 987 页，58.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

circle_intersections(a,b,eps=1e-12L) 返回 Result；半径非负，eps 非负；以下相对阈值描述适用于 eps>0。两个完全相同的正半径圆返回 infinite；同位置两个点圆返回 one；同心不同半径返回 none。其余用距离和半径的相对阈值分类，点圆单独处理，接近相切可返回 one。输入有限且尺度适当，O(1) 时间空间；返回的有限交点仍受浮点构造误差影响。默认容差不能直接满足 AOJ CGL_7_E 的所有合法输入；近切时归并交点可能超出坐标误差限。eps=0 为显式无容差模式：直接计算平方判别式，不主动合并近切点；一般实数仍可能发生舍入误判、溢出或下溢。整数坐标绝对值 ≤10000、整数半径 0..10000，二进制 long double 尾数 ≥64 位且正常最近舍入、禁用 fast-math 时，中间整数精确。交点坐标仍为浮点近似；其他输入不继承这一保证。

```
RealPlane::Result
circle_intersections(RealPlane::Circle a, RealPlane::Circle b, long double eps = 1e-12L)
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Kind = G::Kind;
    assert(eps >= 0);
    assert(a.r >= 0 && b.r >= 0);
    Point v = b.o - a.o;
    if (eps == 0)
    {
        R q = G::dot(v, v);
        if (q == 0)
        {
            if (a.r != b.r) return {Kind::none, {}};
            if (a.r == 0) return {Kind::one, {a.o}};
            return {Kind::infinite, {}};
        }
        R k = q + a.r * a.r - b.r * b.r;
        R d = 4 * a.r * a.r * q - k * k;
        if (d < 0) return {Kind::none, {}};
        Point h = a.o + v * (k / (2 * q));
        if (d == 0) return {Kind::one, {h}};
        Point w = G::perp(v) * (sqrtl(d) / (2 * q));
        return {Kind::two, {h - w, h + w}};
    }
```

默认正容差分支（接上页同一函数）：

```
    R d = G::norm(v);
    R tol = eps * max({d, a.r, b.r});
    // Infinite intersections require identical positive-radius circles.
    if (d == 0)
    {
        if (a.r != b.r) return {Kind::none, {}};
        if (a.r == 0) return {Kind::one, {a.o}};
        return {Kind::infinite, {}};
    }
    if (a.r == 0 && b.r == 0) return {Kind::none, {}};
    if (a.r == 0 || b.r == 0)
    {
        R radius = max(a.r, b.r);
        if (fabsl(d - radius) > tol) return {Kind::none, {}};
        return {Kind::one, {a.r == 0 ? a.o : b.o}};
    }
    if (d > a.r + b.r + tol || d < fabsl(a.r - b.r) - tol) return {Kind::none, {}};
    R x = (d * d + (a.r - b.r) * (a.r + b.r)) / (2 * d);
    Point h = a.o + v * (x / d);
    // Near-concentric circles must not be mistaken for internal tangency.
    if (fabsl(x) > a.r + tol) return {Kind::none, {}};
    if (fabsl(fabsl(x) - a.r) <= tol) return {Kind::one, {h}};
    R y = sqrtl(max(R(0), (a.r - x) * (a.r + x)));
    Point w = G::perp(v) * (y / d);
    return {Kind::two, {h - w, h + w}};
}
```

模板题使用：AOJ CGL_7_E —浮点接口无容差模式

题意：整数坐标绝对值 ≤10000、整数半径 1..10000，保证交点存在；两圆圆心不同。直接调用浮点接口并显式传 eps=0，不合并近切交点；仅在二进制 long double 尾数至少 64 位、正常最近舍入且禁用 fast-math 的平台采用此用法。此整数域的平方判别式精确，坐标仍为浮点构造；一般实数不继承精确分类保证。相切重复同一点，按普通 (x,y) 字典序输出，不能用 epsilon 排序；默认 eps>0 接口保持原近似语义。

所需模板：RealPlane（第 987 页，58.1 节）、circle_intersections（第 996 页，58.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_E

```
int main()
{
    static_assert(numeric_limits<long double>::radix == 2 &&
                  numeric_limits<long double>::digits >= 64);
#ifdef __FAST_MATH__
#error This example requires normal floating arithmetic
#endif
    if (fegetround() != FE_TONEAREST) return 1;
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    RealPlane::Circle a, b;
    cin >> a.o.x >> a.o.y >> a.r >> b.o.x >> b.o.y >> b.r;
    auto ans = circle_intersections(a, b, 0);
    assert(ans.p.size() == 1 || ans.p.size() == 2);
    if (ans.p.size() == 1) ans.p.push_back(ans.p[0]);
    if (tie(ans.p[1].x, ans.p[1].y) < tie(ans.p[0].x, ans.p[0].y))
        swap(ans.p[0], ans.p[1]);
    cout << fixed << setprecision(12) << ans.p[0].x << ' ' << ans.p[0].y << ' '
         << ans.p[1].x << ' ' << ans.p[1].y << '\n';
}
```

58.7 两圆交面积

代码依赖：RealPlane（第 987 页，58.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

circle_overlap_area(a,b) 返回 long double 面积，半径非负。分离或外切返回 0；内含时返回小圆面积，含点圆和同心圆。部分相交计算两个圆弓形面积，小角度使用展开式减轻相消；acos 入参截断至 [-1,1]。不使用交点分类容差；要求有限且尺度适当的输入。O(1) 时间空间，不把浮点输出视为精确面积。

```
RealPlane::R circle_overlap_area(RealPlane::Circle a, RealPlane::Circle b)
{
    using G = RealPlane;
    using R = G::R;
    assert(a.r >= 0 && b.r >= 0);
    R d = G::norm(a.o - b.o), pi = acosl(-1.L);
    if (d >= a.r + b.r) return 0;
    if (d <= fabsl(a.r - b.r)) return pi * min(a.r, b.r) * min(a.r, b.r);
    R difference = (a.r - b.r) * (a.r + b.r);
    R x = acosl(clamp((d * d + difference) / (2 * d * a.r), R(-1), R(1)));
    R y = acosl(clamp((d * d - difference) / (2 * d * b.r), R(-1), R(1)));
    auto segment = [](R angle)
    {
        if (angle < 1e-3L)
        {
            R z = angle * angle;
            return angle * z * (R(2) / 3 + z * (-R(2) / 15 + z * R(4) / 315));
        }
        return angle - sinl(2 * angle) / 2;
    };
    return a.r * a.r * segment(x) + b.r * b.r * segment(y);
}
```

模板题使用：AOJ CGL_7_I

题意：两个圆的公共面积，整数坐标绝对值 $\leq 10^4$、半径 $1..10^4$。覆盖相离、内含、同心与部分相交；输出面积的绝对误差须小于 1e-6。

所需模板：RealPlane（第 987 页，58.1 节）、circle_overlap_area（第 999 页，58.7 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_I

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    RealPlane::Circle a, b;
    cin >> a.o.x >> a.o.y >> a.r >> b.o.x >> b.o.y >> b.r;
    cout << fixed << setprecision(12) << circle_overlap_area(a, b) << '\n';
}
```

58.8 整数点向量与线段判定

坐标绝对值不超过 10^{12}，使用 long long 存点、__int128_t 计算叉积、点积与距离平方。Point 按 x、y 字典序比较；cross(a,b,c) 为 ab 与 ac 的叉积，dot 为点积。on_segment 包含端点及退化线段；intersect 判断闭线段有交点。PolarLess 从正 x 轴按 $[0,2\pi)$ 逆时针极角排序，同向按长度平方递增；零向量排在最前。同点等价，比较器满足严格弱序。按其他起点排序应整体循环移动，LC 的 $(-\pi,\pi]$ 顺序仅将 y<0 后缀移到前面。全部单次运算 O(1)，不使用 epsilon。

```
struct IntegerPlane
{
    using ll = long long;
    using I = __int128_t;

    // Contract: |coordinate|<=1e12. Differences/products widened before arithmetic.
    struct Point
    {
        ll x = 0, y = 0;

        bool operator<(Point b) const { return tie(x, y) < tie(b.x, b.y); }

        bool operator==(Point b) const { return x == b.x && y == b.y; }
    };

    static I cross(Point a, Point b, Point c)
    {
        return (I(b.x) - a.x) * (I(c.y) - a.y) - (I(b.y) - a.y) * (I(c.x) - a.x);
    }

    static I dot(Point a, Point b, Point c)
    {
        return (I(b.x) - a.x) * (I(c.x) - a.x) + (I(b.y) - a.y) * (I(c.y) - a.y);
    }

    static I dist2(Point a, Point b) { return dot(a, b, b); }

    static int sign(I x) { return (x > 0) - (x < 0); }

    static bool on_segment(Point p, Point a, Point b)
    {
        return cross(a, b, p) == 0 && dot(p, a, b) <= 0;
    }

    static bool intersect(Point a, Point b, Point c, Point d)
    {
        int x = sign(cross(a, b, c)), y = sign(cross(a, b, d));
        int z = sign(cross(c, d, a)), w = sign(cross(c, d, b));
        if (!x && on_segment(c, a, b)) return true;
        if (!y && on_segment(d, a, b)) return true;
        if (!z && on_segment(a, c, d)) return true;
        if (!w && on_segment(b, c, d)) return true;
        return x * y < 0 && z * w < 0;
    }
```

极角比较器（接上页同一结构体）：

```
    struct PolarLess
    {
        static int half(Point p) { return p.y < 0 || (p.y == 0 && p.x < 0); }

        bool operator()(Point a, Point b) const
        {
            if (half(a) != half(b)) return half(a) < half(b);
            I c = cross({0, 0}, a, b);
            if (c) return c > 0;
            return dist2({0, 0}, a) < dist2({0, 0}, b);
        }
    };
};
```

模板题使用：Library Checker sort_points_by_argument

题意：将最多 20 万个整数点按 $(-\pi,\pi]$ 的极角排序，坐标绝对值 $\le 10^9$，约定原点角度为 0，同角顺序任意。PolarLess 从正 x 轴按 $[0,2\pi)$ 排序，再把 $y<0$ 的后缀整体移到前面；负 x 轴留在最后，不要把它和负 y 半平面一起前移。仅调用整数点与极角比较器。

所需模板：IntegerPlane（第 1000 页，58.8 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/sort_points_by_argument

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<IntegerPlane::Point> p(n);
    for (auto &v : p) cin >> v.x >> v.y;
    sort(p.begin(), p.end(), IntegerPlane::PolarLess{});
    auto first = find_if(p.begin(), p.end(), [](auto v) { return v.y < 0; });
    rotate(p.begin(), first, p.end());
    for (auto v : p) cout << v.x << ' ' << v.y << '\n';
}
```

58.9 整数输入直线与圆交点

代码依赖：IntegerPlane（第 1000 页，58.8 节）、RealPlane（第 987 页，58.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

line_circle_i64(a,b,o,r)：a、b 定直线，o 为圆心。依赖 IntegerPlane::Point 及 RealPlane::Result。所有输入坐标绝对值 $\le 10^9$，半径在 $0..10^9$；先用 int128 精确判定交点数，再用 long double 构造坐标。a=b 返回 degenerate；相离 none、相切 one、相交 two。结果 p 不排序，one 只含一个点，none/infinite/degenerate 为空。零半径按点圆处理。O(1) 时间和空间。精确的是分类，坐标仍有舍入误差；在 10^9 尺度不承诺绝对 1e-6。AOJ 整数范围 10^4 内的驱动按字典序输出，相切点重复一次。本地高精度与消毒器测试不等于线上 AC。

```
RealPlane::Result line_circle_i64(IntegerPlane::Point a,
                                  IntegerPlane::Point b,
                                  IntegerPlane::Point o,
                                  long long r)
{
    using I = __int128_t;
    using R = RealPlane::R;
    using K = RealPlane::Kind;
    for (auto p : {a, b, o})
        assert(-1000000000 <= p.x && p.x <= 1000000000 && -1000000000 <= p.y &&
               p.y <= 1000000000);
    assert(0 <= r && r <= 1000000000);
    I x = I(b.x) - a.x, y = I(b.y) - a.y;
    I q = x * x + y * y;
    if (q == 0) return {K::degenerate, {}};
    I c = x * (I(a.y) - o.y) - y * (I(a.x) - o.x);
    I d = I(r) * r * q - c * c;
    if (d < 0) return {K::none, {}};
    RealPlane::Point v{R(x), R(y)}, center{R(o.x), R(o.y)};
    auto h = center + RealPlane::perp(v) * (R(c) / R(q));
    if (d == 0) return {K::one, {h}};
    auto w = v * (sqrtl(R(d)) / R(q));
    return {K::two, {h - w, h + w}};
}
```

模板题使用：AOJ CGL_7_D

题意：整数圆心/半径及直线，坐标绝对值 $\le 10^4$、半径 $1..10^4$，q≤1000。题目保证直线非退化且至少一个交点；精确分类后构造坐标，按 x/y 字典序输出，相切点复制一次。绝对误差要求小于 1e-6。

所需模板：IntegerPlane（第 1000 页，58.8 节）、RealPlane（第 987 页，58.1 节）、line_circle_i64（第 1002 页，58.9 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_D

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    IntegerPlane::Point o;
    long long r;
    int q;
    cin >> o.x >> o.y >> r >> q;
    cout << fixed << setprecision(12);
    while (q--)
    {
        IntegerPlane::Point a, b;
        cin >> a.x >> a.y >> b.x >> b.y;
        auto ans = line_circle_i64(a, b, o, r);
        assert(ans.p.size() == 1 || ans.p.size() == 2);
        if (ans.p.size() == 1) ans.p.push_back(ans.p[0]);
        if (tie(ans.p[1].x, ans.p[1].y) < tie(ans.p[0].x, ans.p[0].y))
            swap(ans.p[0], ans.p[1]);
        cout << ans.p[0].x << ' ' << ans.p[0].y << ' ' << ans.p[1].x << ' '
             << ans.p[1].y << '\n';
    }
}
```

58.10 整数输入两圆交点

代码依赖：IntegerPlane（第 1000 页，58.8 节）、RealPlane（第 987 页，58.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

circle_intersections_i64(a,r,b,s)：a、b 为圆心。依赖 IntegerPlane::Point 及 RealPlane::Result。所有输入坐标绝对值 $\le 10^9$，半径在 $0..10^9$；先用 int128 精确判定交点数，再用 long double 构造坐标。相离或严格内含返回 none、相切 one、相交 two；同心等正半径返回 infinite，同心零半径返回 one。结果 p 不排序，one 只含一个点，none/infinite/degenerate 为空。零半径按点圆处理。O(1) 时间和空间。精确的是分类，坐标仍有舍入误差；在 10^9 尺度不承诺绝对 1e-6。AOJ 整数范围 10^4 内的驱动按字典序输出，相切点重复一次。本地高精度与消毒器测试不等于线上 AC。

```
RealPlane::Result circle_intersections_i64(IntegerPlane::Point a,
                                           long long r,
                                           IntegerPlane::Point b,
                                           long long s)
{
    using I = __int128_t;
    using R = RealPlane::R;
    using K = RealPlane::Kind;
    for (auto p : {a, b})
        assert(-1000000000 <= p.x && p.x <= 1000000000 && -1000000000 <= p.y &&
               p.y <= 1000000000);
    assert(0 <= r && r <= 1000000000 && 0 <= s && s <= 1000000000);
    I x = I(b.x) - a.x, y = I(b.y) - a.y;
    I q = x * x + y * y;
    RealPlane::Point center{R(a.x), R(a.y)}, v{R(x), R(y)};
    if (q == 0)
    {
        if (r != s) return {K::none, {}};
        if (r == 0) return {K::one, {center}};
        return {K::infinite, {}};
    }
    I k = q + I(r) * r - I(s) * s;
    I d = 4 * I(r) * r * q - k * k;
    if (d < 0) return {K::none, {}};
    auto h = center + v * (R(k) / (2 * R(q)));
    if (d == 0) return {K::one, {h}};
    auto w = RealPlane::perp(v) * (sqrtl(R(d)) / (2 * R(q)));
    return {K::two, {h - w, h + w}};
}
```

模板题使用：AOJ CGL_7_E

题意：整数圆心/半径，坐标绝对值 $\le 10^4$、半径 $1..10^4$；不同圆心且至少一个交点。按 x/y 字典序输出，相切点复制一次；不使用 eps 排序。绝对误差要求小于 1e-6。

所需模板：IntegerPlane（第 1000 页，58.8 节）、RealPlane（第 987 页，58.1 节）、circle_intersections_i64（第 1004 页，58.10 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_E

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    IntegerPlane::Point a, b;
    long long r, s;
    cin >> a.x >> a.y >> r >> b.x >> b.y >> s;
    auto ans = circle_intersections_i64(a, r, b, s);
    assert(ans.p.size() == 1 || ans.p.size() == 2);
    if (ans.p.size() == 1) ans.p.push_back(ans.p[0]);
    if (tie(ans.p[1].x, ans.p[1].y) < tie(ans.p[0].x, ans.p[0].y))
        swap(ans.p[0], ans.p[1]);
    cout << fixed << setprecision(12) << ans.p[0].x << ' ' << ans.p[0].y << ' '
         << ans.p[1].x << ' ' << ans.p[1].y << '\n';
}
```

58.11 整数多边形有向面积

代码依赖：IntegerPlane（第 1000 页，58.8 节）。抄写时一并准备；标准库类型不列入算法依赖。

polygon_area2(p) 返回 ___int128_t 的有向面积两倍；逆时针为正，反向为负。输入按边界顺序排列，隐式闭合；空集或不足三个点返回 0。单个多边形点数不超过 10^5、坐标绝对值不超过 10^{12}，确保求和在有符号 128 位内。时间 O(n)，额外空间 O(1)。

```
IntegerPlane::I polygon_area2(const vector<IntegerPlane::Point> &p)
{
    using G = IntegerPlane;
    using I = G::I;
    I ans = 0;
    int n = (int)p.size();
    for (int i = 0; i < n; i++)
        ans += I(p[i].x) * p[(i + 1) % n].y - I(p[i].y) * p[(i + 1) % n].x;
    return ans;
}
```

模板题使用：AOJ CGL_3_A

题意：给出逆时针简单多边形，3 至 100 点，整数坐标绝对值不超过 10000，输出面积并保留一位小数。无需凸包，凹多边形也适用。接口返回有向二倍面积，逆时针为正；取绝对值后按奇偶精确打印.0 或.5。本题范围允许将半面积转为 long long；一般 int128 结果不可任意缩窄。

所需模板：IntegerPlane（第 1000 页，58.8 节）、polygon_area2（第 1005 页，58.11 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_3_A

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<IntegerPlane::Point> p(n);
    for (auto &a : p) cin >> a.x >> a.y;
    auto s = polygon_area2(p);
    if (s < 0) s = -s;
    cout << (long long)(s / 2) << (s % 2 ? ".5\n" : ".0\n");
}
```

应用题补充：CSES 2193 Polygon Lattice Points

题意：简单非退化整数多边形，无孔；3≤n≤100000，坐标绝对值 ≤10^9，按边界顺序给点，方向不限。输出严格内部与边界格点数。已有有向二倍面积取绝对值，边界用逐边 gcd，Pick 分子和输出用 128 位；输入已保证简单性，此程序不验证自交。仅组合应用，不新增面积算法覆盖。

所需模板：IntegerPlane（第 1000 页，58.8 节）、polygon_area2（第 1005 页，58.11 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://cses.fi/problemset/task/2193

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<IntegerPlane::Point> p(n);
    for (auto &a : p) cin >> a.x >> a.y;
    using I = IntegerPlane::I;
    I s = polygon_area2(p), b = 0;
    if (s < 0) s = -s;
    for (int j = 0; j < n; ++j)
    {
        auto a = p[j], c = p[(j + 1) % n];
        b += gcd(abs(a.x - c.x), abs(a.y - c.y));
    }
    auto print = [](I x)
    {
        string s;
        do
        {
            s += char('0' + x % 10);
            x /= 10;
        } while (x);
        reverse(s.begin(), s.end());
        cout << s;
    };
    print((s - b + 2) / 2);
    cout << ' ';
    print(b);
    cout << '\n';
}
```

58.12 点在整数简单多边形内

代码依赖：IntegerPlane（第 1000 页，58.8 节）。抄写时一并准备；标准库类型不列入算法依赖。

polygon_contains(p,q) 返回 0 外部、1 边界、2 内部，输入简单多边形，可顺时针或逆时针，不重复首点。空多边形返回 0；单点与线段按闭集判断边界。绕数算法先判线段，再按半开纵坐标区间累计，避免顶点重复计数。坐标范围同 IntegerPlane，O(n) 时间、O(1) 空间。

```
int polygon_contains(const vector<IntegerPlane::Point> &p, IntegerPlane::Point q)
{
    using G = IntegerPlane;
    using Point = G::Point;
    using I = G::I;
    int winding = 0, n = (int)p.size();
    for (int i = 0; i < n; i++)
    {
        Point a = p[i], b = p[(i + 1) % n];
        if (G::on_segment(q, a, b)) return 1;
        I c = G::cross(a, b, q);
        if (a.y <= q.y && b.y > q.y && c > 0) ++winding;
        if (a.y > q.y && b.y <= q.y && c < 0) --winding;
    }
    return winding ? 2 : 0;
}
```

模板题使用：AOJ CGL_3_C

题意：给定简单多边形和查询点，逐个输出 0 外部、1 边界、2 内部；允许凹多边形，按题目给出的逆时针边界顺序读入，不要求重建凸包。n 至 100、q 至 1000，坐标绝对值至 10000；核心也接受反向边界。

所需模板：IntegerPlane（第 1000 页，58.8 节）、polygon_contains（第 1008 页，58.12 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_3_C

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n;
    vector<IntegerPlane::Point> p(n);
    for (auto &v : p) cin >> v.x >> v.y;
    cin >> q;
    while (q--)
    {
        IntegerPlane::Point v;
        cin >> v.x >> v.y;
        cout << polygon_contains(p, v) << '\n';
    }
}
```

58.13 圆与简单多边形相交面积

代码依赖：RealPlane（第 987 页，58.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

area(points,circle) 返回非负相交面积；多边形须简单，允许凹多边形、顺时针/逆时针和相邻重复顶点。少于三点或半径零返回零。edge(a,b,r) 返回圆心在原点时有向边 a 到 b 的面积贡献，r 必须非负。按交点分段，内部用三角形，外部用有向扇形；补偿求和，O(n) 时间、O(1) 额外空间。依赖浮点几何的 Point/Circle 和向量运算。输入须有限、尺度适当，平移到圆心后作差、乘积和面积均须可表示；极小交叠及接近相切时仍有浮点误差。

```
struct CirclePolygon
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Circle = G::Circle;

    // Signed contribution of edge a -> b; circle centered at the origin.
    static R edge(Point a, Point b, R r)
    {
        if (r == 0) return 0;
        Point v = b - a;
        R length = G::norm(v);
        if (length == 0) return 0;
        Point u = v / length;
        R h = G::cross(u, a);
        Point cuts[4] = {a};
        int count = 1;
        if (fabsl(h) <= r)
        {
            R delta = sqrtl(max(R(0), (r - fabsl(h)) * (r + fabsl(h))));
            R t = -G::dot(a, u);
            Point foot = G::perp(u) * h;
            for (R s : {-delta, delta})
                if (t + s > 0 && t + s < length) cuts[count++] = foot + u * s;
        }
        cuts[count++] = b;
```

分段面积与逐边累加（接上页同一结构体与函数）：

```
        R answer = 0;
        for (int i = 1; i < count; i++)
        {
            Point p = cuts[i - 1], q = cuts[i];
            R cross = G::cross(p, q);
            if (G::norm((p + q) / 2) <= r)
                answer += cross / 2;
            else
                answer += r * r * atan2l(cross, G::dot(p, q)) / 2;
        }
        return answer;
    }

    // Simple polygon, either orientation; duplicate consecutive vertices allowed.
    static R area(const vector<Point> &p, Circle c)
    {
        assert(c.r >= 0);
        if (p.size() < 3 || c.r == 0) return 0;
        R answer = 0, error = 0;
        for (int i = 0; i < (int)p.size(); i++)
        {
            R value = edge(p[i] - c.o, p[(i + 1) % p.size()] - c.o, c.r);
            R y = value - error;
            R sum = answer + y;
            error = (sum - answer) - y;
            answer = sum;
        }
        return fabsl(answer);
    }
};
```

模板题使用：AOJ CGL_7_H

题意：原点圆与逆时针简单多边形的公共面积，允许凹多边形。n 为 3..100，整数坐标绝对值 ≤100，半径 1..100。直接传入顶点序列，不取凸包；绝对误差小于 1e-5。

所需模板：RealPlane（第 987 页，58.1 节）、CirclePolygon（第 1009 页，58.13 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_H

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long double r;
    cin >> n >> r;
    vector<RealPlane::Point> p(n);
    for (auto &a : p) cin >> a.x >> a.y;
    cout << fixed << setprecision(12) << CirclePolygon::area(p, {{0, 0}, r}) << '\n';
}
```

58.14 两圆公切线与过点切线

代码依赖：RealPlane（第 987 页，58.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

solve(a,b,eps) 返回 Result，包含 infinite 和 lines；infinite 为真表示无穷多条，否则 lines 给出全部切线。每条 Tangent 保存切点 a、b，单位法向量 normal，以及 side。a、b 对应输入两圆，直线方程 dot(normal,x-a)=0；切点相同时仍有效。side=1 外公切线、-1 内公切线、0 两个不同点。半径零表示点，from_point(p,c,eps) 等价于第一圆取点，圆上切点读 b。输入有限，坐标及半径绝对值 ≤1e100，半径非负。eps 默认 1e-12，用于相对圆心距的近相切判断；近相切可能合并为一条，浮点坐标仍有舍入误差。严格同心等半径时 infinite=true，包括同一点；同心不等半径返回空。O(1)；依赖浮点几何的 Point/Circle、点运算及 norm/perp。

```
struct CircleTangents
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Circle = G::Circle;

    struct Tangent
    {
        Point a, b, normal;
        int side; // 1 outer, -1 inner, 0 two distinct point circles.
    };

    struct Result
    {
        bool infinite = false;
        vector<Tangent> lines;
    };

    // Line: dot(normal, x - a) = 0. Radius zero represents a point.
    static Result solve(Circle a, Circle b, R eps = 1e-12L)
    {
        assert(a.r >= 0 && b.r >= 0 && eps > 0 && eps < 1);
        Point v = b.o - a.o;
        R d = G::norm(v);
        if (d == 0) return {a.r == b.r, {}};
        Point u = v / d;
        if (a.r == 0 && b.r == 0) return {false, {{a.o, b.o, G::perp(u), 0}}};
```

内外公切线枚举（接上页同一函数）：

```
        Result answer;
        for (int side : {1, -1})
        {
            if (side == -1 && (a.r == 0 || b.r == 0)) continue;
            R gap = a.r - side * b.r;
            if (fabsl(gap) > d * (1 + eps)) continue;
            bool touch = fabsl(gap) >= d * (1 - eps);
            R c = touch ? (gap < 0 ? -1 : 1) : gap / d;
            R s = sqrtl(max(R(0), (1 - c) * (1 + c)));
            for (int sign : {-1, 1})
            {
                Point normal = u * c + G::perp(u) * (sign * s);
                Point p = a.o + normal * a.r;
                Point q = b.o + normal * (side * b.r);
                answer.lines.push_back({p, q, normal, side});
                if (touch) break;
            }
        }
        return answer;
    }

    static Result from_point(Point p, Circle c, R eps = 1e-12L)
    {
        return solve({p, 0}, c, eps);
    }
};
```

模板题使用：AOJ CGL_7_F

题意：过圆外整数点作切线，输出圆上的两个切点。坐标绝对值 ≤1000，半径 1..1000，保证点严格在圆外。from_point 把点作为第一圆，圆上切点在 line.b；按 x/y 字典序输出，每行一个点，绝对误差小于 1e-5。

所需模板：RealPlane（第 987 页，58.1 节）、CircleTangents（第 1011 页，58.14 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_F

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    RealPlane::Point p;
    RealPlane::Circle c;
    cin >> p.x >> p.y >> c.o.x >> c.o.y >> c.r;
    auto ans = CircleTangents::from_point(p, c);
    vector<RealPlane::Point> points;
    for (auto line : ans.lines) points.push_back(line.b);
    sort(points.begin(),
         points.end(),
         [](auto a, auto b) { return tie(a.x, a.y) < tie(b.x, b.y); });
    cout << fixed << setprecision(12);
    for (auto a : points) cout << a.x << ' ' << a.y << '\n';
}
```

58.15 整数圆公切线与精确切点排序

代码依赖：IntegerPlane（第 1000 页，58.8 节）、RealPlane（第 987 页，58.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

solve(a,r,b,s)：a、b 为 IntegerPlane::Point，所有坐标绝对值 $\le 10^4$，半径 $0..10^4$。依赖 RealPlane；返回 Result{infinite,lines}，每条 Tangent 含第一/第二圆切点 a、b、单位法向 normal、side（1 外切、-1 内切、0 两点圆）。直线满足 dot(normal,x-a)=0，不能用相切时重合的两个切点定直线。精确整数判断条数，并用根式符号精确比较第一圆切点的 (x,y) 次序；返回顺序已正确，不要按舍入后的坐标重排。同心同半径 infinite=true（含两个相同点圆），同心不同半径为空；异心两个点圆仅一条线。坐标和法向仍为 long double 近似构造。O(1) 时间/空间，最多 4 条。sign/root_sign/compare 仅为内部比较，不能任意扩大参数范围。此范围较圆交点版更小，因为排序包含二次平方；不是任意 64 位整数几何。

```
struct IntegerTangents
{
    using I = __int128_t;
    using R = RealPlane::R;
    using Point = IntegerPlane::Point;

    struct Tangent
    {
        RealPlane::Point a, b, normal;
        int side;
    };

    struct Result
    {
        bool infinite = false;
        vector<Tangent> lines;
    };

    static int sign(I x) { return (x > 0) - (x < 0); }

    // Internal: sign(a + b * sqrt(c)), c >= 0.
    static int root_sign(I a, I b, I c)
    {
        if (b == 0 || c == 0) return sign(a);
        if (a == 0 || sign(a) == sign(b)) return sign(b);
        return sign(a) * sign(a * a - b * b * c);
    }

    // Internal: sign(a + b * sqrt(c) + e * sqrt(f)).
    static int compare(I a, I b, I c, I e, I f)
    {
        int u = root_sign(a, b, c), v = f == 0 ? 0 : sign(e);
        if (u == 0) return v;
        if (v == 0 || u == v) return u;
        return u * root_sign(a * a + b * b * c - e * e * f, 2 * a * b, c);
    }
```

公切线枚举与精确切点排序（接上页同一结构体）：

```
    // |coordinates| <= 10000, 0 <= radii <= 10000.
    // Lines are ordered by their exact contact (x, y) on the first circle.
    static Result solve(Point a, long long r, Point b, long long s)
    {
        for (auto p : {a, b})
            assert(-10000 <= p.x && p.x <= 10000 && -10000 <= p.y && p.y <= 10000);
        assert(0 <= r && r <= 10000 && 0 <= s && s <= 10000);
        I x = I(b.x) - a.x, y = I(b.y) - a.y;
        I q = x * x + y * y;
        if (q == 0) return {r == s, {}};
        RealPlane::Point v{R(x), R(y)}, o{R(a.x), R(a.y)}, p{R(b.x), R(b.y)};
        if (r == 0 && s == 0)
            return {false, {{o, p, RealPlane::perp(v) / sqrtl(R(q)), 0}}};

        struct Item
        {
            I g, d;
            int sign, side;
        };

        vector<Item> items;
        for (int side : {1, -1})
        {
            if (side == -1 && (r == 0 || s == 0)) continue;
            I g = I(r) - side * s, d = q - g * g;
            if (d < 0) continue;
            items.push_back({g, d, -1, side});
            if (d > 0) items.push_back({g, d, 1, side});
        }
        sort(items.begin(),
             items.end(),
             [&](Item a, Item b)
             {
                 int k = compare(x * (a.g - b.g), -y * a.sign, a.d, y * b.sign, b.d);
                 if (k != 0) return k < 0;
                 return compare(y * (a.g - b.g), x * a.sign, a.d, -x * b.sign, b.d) < 0;
             });
        Result answer;
        for (auto t : items)
        {
            auto n =
                (v * R(t.g) + RealPlane::perp(v) * (t.sign * sqrtl(R(t.d)))) / R(q);
            answer.lines.push_back({o + n * R(r), p + n * R(t.side * s), n, t.side});
        }
        return answer;
    }
};
```

模板题使用：AOJ CGL_7_G

题意：不同整数圆的公切线，输出第一圆上的全部切点。坐标绝对值 ≤1000，半径 1..1000；可能 0..4 条，同心不等圆需空输出。整数版已精确排序，直接顺序输出 line.a，不再按舍入值排序；绝对误差小于 1e-5。

所需模板：IntegerPlane（第 1000 页，58.8 节）、RealPlane（第 987 页，58.1 节）、IntegerTangents（第 1013 页，58.15 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_G

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    IntegerPlane::Point a, b;
    long long r, s;
    cin >> a.x >> a.y >> r >> b.x >> b.y >> s;
    auto ans = IntegerTangents::solve(a, r, b, s);
    cout << fixed << setprecision(12);
    for (auto line : ans.lines) cout << line.a.x << ' ' << line.a.y << '\n';
}
```

相关应用：任意简单整数多边形面积并；符号化右侧极限、共线边编号去重和有理参数排序，浮点面积输出。

58.16 一般多边形面积并：整数精确判定

代码依赖：IntegerPlane（第 1000 页，58.8 节）。抄写时一并准备；标准库类型不列入算法依赖。

polygon_union_area(polygons) 求任意简单多边形的面积并，允许凹多边形、不同多边形边界交叉/重合以及并集孔洞；每个输入多边形本身不含洞、不自交。整数顶点为 IntegerPlane::Point，坐标绝对值 $\leq 10^9$（比基础点类型约定更严格）。自动转逆时针，删除连续重复点及末尾重复首点；零面积退化输入忽略，但不负责检查自交。

共 V 个顶点，时间 $O(V^2 \log V)$、空间 O(V)，下标/覆盖数须可用 int 表示。128 位叉积与有理参数排序精确处理拓扑，最终面积 long double 近似；不扰动顶点、不用 eps 排序。包围盒相交的多边形共用局部原点并补偿求和，仍须按题目精度检验最终面积，不声称浮点累加无条件精确。

```
long double polygon_union_area(vector<vector<IntegerPlane::Point>> polygons)
{
    using G = IntegerPlane;
    using P = G::Point;
    using I = G::I;
    using R = long double;
    struct Fraction
    {
        I num, den;
        Fraction(I x = 0, I y = 1) : num(x), den(y)
        {
            assert(den != 0);
            if (den < 0)
            {
                num = -num;
                den = -den;
            }
        }
        bool operator<(const Fraction &b) const
        {
            return num * b.den < b.num * den;
        }
    };
    vector<vector<P>> a;
    for (auto p : polygons)
    {
        for (auto v : p)
            assert(-1000000000 <= v.x && v.x <= 1000000000 &&
                   -1000000000 <= v.y && v.y <= 1000000000);
        p.erase(unique(p.begin(), p.end()), p.end());
        if (p.size() > 1 && p.front() == p.back()) p.pop_back();
        I area = 0;
        int m = p.size();
        for (int i = 0; i < m; i++) area += G::cross({0, 0}, p[i], p[(i + 1) % m]);
        if (area == 0) continue;
        if (area < 0) reverse(p.begin(), p.end());
        a.push_back(move(p));
    }
```

局部原点与边事件初始化（接上页函数）：

```
    int n = a.size();
    vector<array<long long, 4>> box(n);
    vector<int> parent(n);
    iota(parent.begin(), parent.end(), 0);
    auto find = [&](auto &&self, int x) -> int
    {
        if (parent[x] == x) return x;
        return parent[x] = self(self, parent[x]);
    };
    for (int i = 0; i < n; i++)
    {
        auto p = a[i][0];
        box[i] = {p.x, p.y, p.x, p.y};
        for (auto v : a[i])
        {
            box[i][0] = min(box[i][0], v.x);
            box[i][1] = min(box[i][1], v.y);
            box[i][2] = max(box[i][2], v.x);
            box[i][3] = max(box[i][3], v.y);
        }
        for (int j = 0; j < i; j++)
            if (max(box[i][0], box[j][0]) <= min(box[i][2], box[j][2]) &&
                max(box[i][1], box[j][1]) <= min(box[i][3], box[j][3]))
                parent[find(find, i)] = find(find, j);
    }
    R answer = 0;
    R error = 0;
    for (int i = 0; i < n; i++)
    {
        int m = a[i].size();
        P origin = a[find(find, i)][0];
        for (int k = 0; k < m; k++)
        {
            P p = a[i][k];
            P q = a[i][(k + 1) % m];
            vector<pair<Fraction, int>> events;
            auto add = [&](Fraction t, int delta)
            {
                t = max(Fraction(0), min(Fraction(1), t));
                events.push_back({t, delta});
            };
```

右侧覆盖与共线边去重（接上页函数）：

```
            for (int j = 0; j < n; j++)
            {
                if (i == j) continue;
                int size = a[j].size();
                for (int h = 0; h < size; h++)
                {
                    P c = a[j][h];
                    P d = a[j][(h + 1) % size];
                    I x = G::cross(p, q, c);
                    I y = G::cross(p, q, d);
                    if ((x >= 0) != (y >= 0))
                        add(Fraction(G::cross(c, d, p), y - x), x < 0 ? -1 : 1);
                    if (x == 0 && y == 0 && j < i)
                    {
                        auto project = [&](P v)
                        {
                            if (p.x != q.x) return Fraction(I(v.x) - p.x, I(q.x) - p.x);
                            return Fraction(I(v.y) - p.y, I(q.y) - p.y);
                        };
                        auto l = project(c);
                        auto r = project(d);
                        if (l < r)
                        {
                            add(l, 1);
                            add(r, -1);
                        }
                    }
                }
            }
```

有理参数排序及未覆盖段积分（接上页函数）：

```
            add(Fraction(1), 0);
            sort(events.begin(), events.end(), [](const auto &x, const auto &y)
            {
                return x.first < y.first;
            });
            int cover = 0;
            Fraction last;
            for (auto [t, delta] : events)
            {
                if (cover == 0)
                {
                    I num = t.num * last.den - last.num * t.den;
                    I den = t.den * last.den;
                    R value = R(G::cross(origin, p, q)) * (R(num) / R(den)) / 2;
                    R y = value - error;
                    R next = answer + y;
                    error = (next - answer) - y;
                    answer = next;
                }
                cover += delta;
                last = t;
            }
        }
    }
    return answer;
}
```

精确范围 令坐标界 M=10 的 9 次方。差向量分量至多 2M，叉积、交点参数分子及分母绝对值至多 8M 平方。分式比较的单个交叉乘积至多 64M 四次方，小于 128 位有符号上界；裁到 [0,1] 后的参数差也安全。不能直接沿用 IntegerPlane 允许的更大坐标界。

为何看边的右侧 每个多边形先转成逆时针，当前边的左侧已在并集中。只有其他多边形占据右侧时，该边才成为并集内部，不能贡献边界。把测试线向右侧平移无穷小量，原线上顶点视作在线上方：cross 大于等于 0 对比小于 0，等价于符号化极限，没有实际扰动坐标。向下穿越时覆盖加 1，向上穿越时减 1。凹多边形会产生多个区间，仍由相同事件累计处理。

共享边与端点 两条共线反向边的内部位于两侧，右侧测试已将其剔除；同向共线边仅编号较小的多边形保留，避免重复积分。所有事件参数精确裁到 [0,1]，同参数事件的任意顺序只影响零长度段，不影响面积。退化顶点、相切和部分重叠边均不需要随机扰动。

积分与孔洞 对边 p 到 q 的未覆盖参数段 [l,u]，贡献是 cross(p-origin,q-origin) 乘 (u-l)/2。按逆时针原边方向累加，孔洞自动带负方向，不对单边取绝对值。包围盒相交关系的连通块包含真实并集连通块，按该分组平移不改变闭合积分。最终转 long double 和补偿求和仍有误差；输入内含洞或自交轮廓不在本接口范围。

独立验证 Fraction 精确枚举所有线段交点横坐标，分竖直条带合并各多边形截面，以线性上下包络的梯形公式求真值。包含凹多边形、重复/共线/反向边、孔洞、极值坐标、刚体变换和 1000 三角形压力例。线上状态见验证记录。

模板题使用：P13338 三角形面积并加强版

题意：n≤1000 个非退化三角形，整数坐标绝对值 $\leq 10^6$；误差要求 | 答案-真值 |/max(1,| 真值 |)≤1e-6。此驱动使用一般多边形模板；凹多边形能力另由本地验证覆盖。当前实现已通过 P13338，记录 302630852。

所需模板：IntegerPlane（第 1000 页，58.8 节）、polygon_union_area（第 1016 页，58.16 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P13338

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<vector<IntegerPlane::Point>> a(n, vector<IntegerPlane::Point>(3));
    for (auto &p : a)
        for (auto &v : p) cin >> v.x >> v.y;
    cout << fixed << setprecision(12) << polygon_union_area(a) << '\n';
}
```

相关应用：圆面积并的圆弧覆盖与格林公式；支持包含、同圆、相切及孔洞，明确浮点局限。

58.17 圆面积并：未覆盖圆弧积分

代码依赖：RealPlane（第 987 页，58.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

circle_union_area(a) 求闭圆盘的面积并，输入 vector<RealPlane::Circle>，圆心及半径用 long double，半径 ≥ 0；空集/点圆贡献 0。同圆只保留最早编号的边界，内含圆无贡献；不要求连通或无孔洞。相交连通块各自平移后积分，原输入不改变。

每个圆将其他圆覆盖的角区间切到 $[0,2\pi]$，严格排序合并，只积分未覆盖的圆弧。$O(n^2 \log n)$ 时间、$O(n)$ 额外空间；并查集递归最深 $O(n)$。所有输入和中间运算须有限可表示。使用原始浮点比较，不用 eps 排序或随机扰动；近退化分类、极端尺度和最终误差仍受浮点限制。整数题域与本地对拍不构成任意实数的精确保证。

```
RealPlane::R circle_union_area(vector<RealPlane::Circle> a)
{
    using R = RealPlane::R;
    R pi = acosl(-1.L);
    R tau = 2 * pi;
    for (auto c : a)
        assert(isfinite(c.o.x) && isfinite(c.o.y) && isfinite(c.r) && c.r >= 0);
    a.erase(remove_if(a.begin(), a.end(), [](auto c)
    {
        return c.r == 0;
    }), a.end());
    int n = a.size();
    vector<int> parent(n);
    iota(parent.begin(), parent.end(), 0);
    auto find = [&](auto &&self, int x) -> int
    {
        if (parent[x] == x) return x;
        return parent[x] = self(self, parent[x]);
    };
    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            if (RealPlane::norm(a[i].o - a[j].o) < a[i].r + a[j].r)
                parent[find(find, i)] = find(find, j);
    R answer = 0;
    R error = 0;
    for (int i = 0; i < n; i++)
    {
        auto c = a[i];
        auto o = c.o - a[find(find, i)].o;
        vector<pair<R, R>> intervals;
        bool covered = false;
```

两圆关系及覆盖角区间（接上页函数）：

```
        for (int j = 0; j < n; j++)
        {
            if (i == j) continue;
            auto v = a[j].o - c.o;
            R d = RealPlane::norm(v);
            R r = c.r;
            R s = a[j].r;
            if (s >= r && d <= s - r && (s > r || j < i))
            {
                covered = true;
                break;
            }
            if (d >= r + s || d <= abs(r - s)) continue;
            R scale = max({d, r, s});
            d /= scale;
            r /= scale;
            s /= scale;
            R h = (r + s - d) * (d + r - s) * (d - r + s) * (d + r + s);
            R angle = atan2l(sqrtl(max(R(0), h)), d * d + (r - s) * (r + s));
            R middle = atan2l(v.y, v.x);
            if (middle < 0) middle += tau;
            R l = middle - angle;
            R u = middle + angle;
            if (l < 0)
            {
                intervals.push_back({0, u});
                intervals.push_back({l + tau, tau});
            }
            else if (u > tau)
            {
                intervals.push_back({l, tau});
                intervals.push_back({0, u - tau});
            }
            else intervals.push_back({l, u});
        }
        if (covered) continue;
```

未覆盖圆弧积分与补偿求和（接上页函数）：

```
        auto add = [&](R l, R r)
        {
            R h = (r - l) / 2;
            R middle = (r + l) / 2;
            R value = c.r * c.r * h;
            if (r - l != tau)
                value += c.r * sinl(h) * (o.x * cosl(middle) + o.y * sinl(middle));
            R y = value - error;
            R next = answer + y;
            error = (next - answer) - y;
            answer = next;
        };
        sort(intervals.begin(), intervals.end());
        R right = 0;
        for (auto [l, r] : intervals)
        {
            if (l > right) add(right, l);
            right = max(right, r);
        }
        if (right < tau) add(right, tau);
    }
    return answer;
}
```

为什么只算未覆盖弧 删去重合边界后，圆盘并边界恰是没有落在其他圆内部的圆弧。各圆逆时针方向的内部位于左侧，所以外边界和孔洞边界自动获得正确符号；不能对每条弧单独取绝对值。相切点面积为零，内含圆无需贡献。

积分公式 对相对连通块原点的圆心 (x,y)，半径 r，弧角从 l 到 u，令 $h=(u-l)/2$、$t=(u+l)/2$。格林公式给出贡献 $r^2h+r\sin h(x\cos t+y\sin t)$。整圆直接用 πr^2，避免 $\sin\pi$ 舍入误差；补偿求和减少大量正负弧贡献相加的损失。连通块分别平移不改变闭合边界积分。

数值与验证 角度排序使用严格原值比较，同圆按编号去重，不以 eps 合并不同圆。半角通过缩放后的 Heron 乘积与 atan2 求出，避免 acos 靠近正负 1 时的敏感性；仍不是精确几何。独立 70 位高精度验证沿竖直截面、在全部交点横坐标处分段解析积分，另有孔洞、相切、远平移和千圆压力例。线上 AC 状态见验证记录，不能由本地通过推断。

模板题使用：SP8073 CIRU 圆面积并

题意：n≤1000，整数圆心坐标绝对值及半径 ≤1000，允许零半径。输出圆盘面积并，保留三位小数；SPOJ 转交记录 302630657 仍 Waiting。当前核心另已通过用户模板题 U106005，输出精度适配见下一用法。

所需模板：RealPlane（第 987 页，58.1 节）、circle_union_area（第 1022 页，58.17 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/SP8073

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<RealPlane::Circle> circles(n);
    for (auto &c : circles) cin >> c.o.x >> c.o.y >> c.r;
    cout << fixed << setprecision(3) << circle_union_area(circles) << '\n';
}
```

模板题使用：U106005 圆面积并（用户模板题）

题意：n≤1000，整数圆心坐标绝对值及半径 ≤1000，允许零半径。误差要求 1e-3，SPJ 要求至少 4 位小数，故输出 12 位。当前源码记录 302633427，5 点全 AC；不代表 SPOJ 转交记录已通过。

所需模板：RealPlane（第 987 页，58.1 节）、circle_union_area（第 1022 页，58.17 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/U106005

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<RealPlane::Circle> circles(n);
    for (auto &c : circles) cin >> c.o.x >> c.o.y >> c.r;
    cout << fixed << setprecision(12) << circle_union_area(circles) << '\n';
}
```

相关应用：分层圆弧积分求每个精确覆盖重数的面积，重复圆按编号分层；有独立高精度截面积分验证。

58.18 圆覆盖重数面积：恰好覆盖 k 次

代码依赖：RealPlane（第 987 页，58.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

circle_coverage_area(a) 返回长 n+1 的数组，ans[k] 为恰被 k 个闭圆盘覆盖的面积（1≤k≤n）；ans[0]=0 仅为占位，不表示无界平面面积。输入 vector<RealPlane::Circle>，圆心和半径 long double，半径 ≥0，n≤INT_MAX-2；按值传参不改原数组。支持重复圆、零半径、包含、相切和孔洞。

圆弧按覆盖层数积分得到至少 k 次面积，再相邻作差；同圆按编号依次归入不同层，不能去重。$O(n^2 \log n)$ 时间、O(n) 额外空间，并查集递归最深 O(n)。输入和中间运算须有限可表示；严格原值排序、分块平移及补偿求和不能消除近退化与小面积差分的浮点误差。不对核心结果强制取绝对值或截零。

```
vector<RealPlane::R> circle_coverage_area(vector<RealPlane::Circle> a)
{
    using R = RealPlane::R;
    R pi = acosl(-1.L);
    R tau = 2 * pi;
    for (auto c : a)
        assert(isfinite(c.o.x) && isfinite(c.o.y) && isfinite(c.r) && c.r >= 0);
    assert(a.size() <= INT_MAX - 2);
    int size = int(a.size());
    a.erase(remove_if(a.begin(), a.end(), [](auto c)
    {
        return c.r == 0;
    }), a.end());
    int n = int(a.size());
    vector<int> parent(n);
    iota(parent.begin(), parent.end(), 0);
    auto find = [&](auto &&self, int x) -> int
    {
        if (parent[x] == x)
            return x;
        return parent[x] = self(self, parent[x]);
    };
    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            if (RealPlane::norm(a[i].o - a[j].o) < a[i].r + a[j].r)
                parent[find(find, i)] = find(find, j);
    vector<R> answer(size + 2);
    vector<R> error(size + 2);
    for (int i = 0; i < n; i++)
    {
        auto c = a[i];
        auto o = c.o - a[find(find, i)].o;
        vector<pair<R, int>> events = {{0, 0}, {tau, 0}};
        int count = 1;
```

两圆关系及覆盖角区间（接上页函数）：

```
        for (int j = 0; j < n; j++)
        {
            if (i == j)
                continue;
            auto v = a[j].o - c.o;
            R d = RealPlane::norm(v);
            R r = c.r;
            R s = a[j].r;
            if (s >= r && d <= s - r && (s > r || j < i))
            {
                count++;
                continue;
            }
            if (d >= r + s || d <= abs(r - s))
                continue;
            R scale = max({d, r, s});
            d /= scale;
            r /= scale;
            s /= scale;
            R h = (r + s - d) * (d + r - s) * (d - r + s) * (d + r + s);
            R angle = atan2l(sqrtl(max(R(0), h)), d * d + (r - s) * (r + s));
            R middle = atan2l(v.y, v.x);
            if (middle < 0)
                middle += tau;
            R l = middle - angle;
            R u = middle + angle;
            if (l < 0)
            {
                events.push_back({0, 1});
                events.push_back({u, -1});
                events.push_back({l + tau, 1});
                events.push_back({tau, -1});
            }
            else if (u > tau)
            {
                events.push_back({l, 1});
                events.push_back({tau, -1});
                events.push_back({0, 1});
                events.push_back({u - tau, -1});
            }
            else
            {
                events.push_back({l, 1});
                events.push_back({u, -1});
            }
        }
```

分层圆弧积分与补偿求和（接上页函数）：

```
        auto add = [&](R l, R r)
        {
            R h = (r - l) / 2;
            R middle = (r + l) / 2;
            R value = c.r * c.r * h;
            if (r - l != tau)
                value += c.r * sinl(h) * (o.x * cosl(middle) + o.y * sinl(middle));
            R y = value - error[count];
            R next = answer[count] + y;
            error[count] = (next - answer[count]) - y;
            answer[count] = next;
        };
        sort(events.begin(), events.end());
        R left = 0;
        for (auto [right, delta] : events)
        {
            if (right > left)
                add(left, right);
            count += delta;
            left = right;
        }
    }
    for (int k = 1; k <= size; k++)
        answer[k] -= answer[k + 1];
    answer.resize(size + 1);
    return answer;
}
```

覆盖分层 当前圆弧左侧包含自身，另有 count-1 个圆覆盖；它是至少 count 次覆盖区域的正向边界。分别积分到对应层后，按 k 递增作 answer[k]-answer[k+1] 得到恰好 k 次。重合的 m 个圆按编号让第 t 个额外计入前 t-1 个同圆，从而每个阈值恰贡献一次边界；不能去重。ans[0] 只是占位。

数值与验证 各连通块平移、严格角排序、补偿累加；仍有浮点分类和相邻层相减的消去误差，小面积可能有微小负误差。466 组 70 位竖条截面积分及解析压力例，在 O2、NDEBUG、抄写版与 ASan/UBSan 下通过；不据此承诺任意实数精确。

积分公式 对相对连通块原点的圆心 (x, y)，半径 r，弧角从 l 到 u，令 $h = (u - l)/2$、$t = (u + l)/2$。格林公式给出贡献 $r^2 h + r \sin h(x \cos t + y \sin t)$。整圆直接用 πr^2，避免 $\sin \pi$ 舍入误差；补偿求和减少大量正负弧贡献相加的损失。连通块分别平移不改变闭合边界积分。

模板题使用：SP8119 CIRUT 圆覆盖重数面积

题意：n≤1000，互异圆，整数圆心坐标绝对值 ≤1000、0<r≤1000；输出恰被 k 个圆覆盖的面积，保留三位小数。核心额外支持同圆及零半径，另由本地独立验证覆盖。末端 max 仅避免舍入负零，不改变核心结果。已提交记录 302660709，仍 Waiting，不记 AC。

所需模板：RealPlane（第 987 页，58.1 节）、circle_coverage_area（第 1026 页，58.18 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/SP8119

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<RealPlane::Circle> circles(n);
    for (auto &c : circles)
        cin >> c.o.x >> c.o.y >> c.r;
    auto area = circle_coverage_area(circles);
    cout << fixed << setprecision(3);
    for (int k = 1; k <= n; k++)
        cout << '[' << k << "] = " << max(0.L, area[k]) << '\n';
}
```

相关应用：凸多边形裁剪：保留有向直线左侧；几何基础上的独立补充接口。

58.19 凸多边形裁剪：保留有向直线左侧

代码依赖：RealPlane（第 987 页，58.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

convex_cut(p,a,b) 返回凸多边形与有向直线 a→b 左侧闭半平面的交。a≠b；输入按边界顺序，顺/逆时针均可，允许共线中间点；空集、点和线段也可。原输入不变，输出保留原方向，但可能有共线点或重复边界点，不能直接当作严格凸包传入 minimum_rectangle。

逐边保留左侧顶点，仅在两端严格异侧时加入交点，过顶点及沿边切割不作重复求交。O(n) 时间和输出空间。n 须能用 int 表示，输入及中间运算有限可表示。不使用 eps；任意近退化实数的符号受舍入影响，不保证精确拓扑。AOJ 整数范围内输入侧向叉积可精确表示，交点与面积仍舍入。

```
vector<RealPlane::Point> convex_cut(const vector<RealPlane::Point> &p,
    RealPlane::Point a, RealPlane::Point b)
{
    using G = RealPlane;
    assert(a.x != b.x || a.y != b.y);
    int n = int(p.size());
    vector<G::Point> q;
    auto e = b - a;
    for (int i = 0; i < n; i++)
    {
        auto u = p[i];
        auto v = p[(i + 1) % n];
        auto s = G::cross(e, u - a);
        auto t = G::cross(e, v - a);
        if (s >= 0)
            q.push_back(u);
        if ((s < 0 && t > 0) || (s > 0 && t < 0))
            q.push_back(u + (v - u) * (s / (s - t)));
    }
    return q;
}
```

模板题使用：AOJ CGL_4_C Convex Cut

题意：n,q≤100，整数坐标绝对值 ≤10000，输入逆时针凸多边形可含共线点。每条查询均裁剪原多边形，输出左侧面积 12 位小数，要求绝对误差小于 1e-5。已核对官方题面与本地精确模型；AOJ 未登录且无已保存账号，尚无在线提交。

所需模板：RealPlane（第 987 页，58.1 节）、convex_cut（第 1030 页，58.19 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_4_C&lang=en

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, queries;
    cin >> n;
    vector<RealPlane::Point> p(n);
    for (auto &v : p)
        cin >> v.x >> v.y;
    cin >> queries;
    while (queries--)
    {
        RealPlane::Point a, b;
        cin >> a.x >> a.y >> b.x >> b.y;
        auto q = convex_cut(p, a, b);
        long double area = 0;
        long double error = 0;
        for (int i = 1; i + 1 < (int)q.size(); i++)
        {
            auto value = RealPlane::cross(q[i] - q[0], q[i + 1] - q[0]);
            auto y = value - error;
            auto next = area + y;
            error = (next - area) - y;
            area = next;
        }
        cout << fixed << setprecision(12) << abs(area) / 2 << '\n';
    }
}
```

相关应用：简单多边形面积重心；几何基础上的独立补充接口。

58.20 简单多边形面积重心

代码依赖：RealPlane（第 987 页，58.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

polygon_centroid(p) 返回 optional<RealPlane::Point>，表示均匀密度简单多边形的面积重心；可凹，顺/逆时针均可。少于三点或计算有向面积恰为 0 返回 nullopt，不把原点作为失败标记。不用于自交、多环带孔多边形，也不是顶点平均或边界线重心。

相对 p[0] 作三角扇，有向面积与一阶矩补偿求和；除以总有向面积恢复坐标。O(n) 时间、O(1) 额外空间，原输入不变。n 须能用 int 表示，输入与中间运算有限可表示；不使用 eps。实数近退化时面积符号和零判定仍可能舍入错误，巨大平移也限制最终坐标精度；long double 不保证比 double 精度高。

```
optional<RealPlane::Point> polygon_centroid(const vector<RealPlane::Point> &p)
{
    using G = RealPlane;
    using R = G::R;
    int n = int(p.size());
    if (n < 3)
        return nullopt;
    array<R, 3> sum{};
    array<R, 3> error{};
    auto add = [&](int i, R v)
    {
        R y = v - error[i];
        R next = sum[i] + y;
        error[i] = (next - sum[i]) - y;
        sum[i] = next;
    };
    for (int i = 1; i + 1 < n; i++)
    {
        auto a = p[i] - p[0];
        auto b = p[i + 1] - p[0];
        R s = G::cross(a, b);
        add(0, s);
        add(1, (a.x + b.x) * s);
        add(2, (a.y + b.y) * s);
    }
    if (sum[0] == 0)
        return nullopt;
    G::Point offset{sum[1] / sum[0] / 3, sum[2] / sum[0] / 3};
    return p[0] + offset;
}
```

接口用法演示：简单多边形面积重心接口示例

演示约定：读取按边界顺序给出的简单多边形，输出面积重心 12 位小数；退化时输出 Undefined。支持凹多边形和顺时针输入。这是接口用法示例，不是在线评测题，不计作 AC。

所需模板：RealPlane（第 987 页，58.1 节）、polygon_centroid（第 1032 页，58.20 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/POLYGON-TOOLS.md

```
int main()
{
    int n;
    cin >> n;
    vector<RealPlane::Point> p(n);
    for (auto &v : p)
        cin >> v.x >> v.y;
    auto c = polygon_centroid(p);
    if (!c)
        cout << "Undefined\n";
    else
        cout << fixed << setprecision(12) << c->x << ' ' << c->y << '\n';
}
```

相关应用：三角形内心与内切圆，非完整二维几何来源覆盖

58.21 三角形内切圆：边长加权内心

代码依赖：RealPlane（第 987 页，58.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

triangle_incircle(a,b,c) 返回 optional<RealPlane::Circle>，圆心为内心、半径为面积除以半周长。顺逆时针均可，不改变输入；计算叉积恰为 0 或三点全重合返回 nullopt，不把原点作为失败标志。用对边长加权顶点，避免三角函数；O(1) 时间和空间。

先相对 a 平移，边向量按最大分量归一化再求边长；面积叉积在归一化前计算，保留 AOJ 整数域内精确零判定。要求输入、相减、叉积及输出等中间值有限可表示且不发生影响结果的下溢；任意近共线实数的零判定仍可能舍入错误，不是精确拓扑接口。AOJ 整数坐标绝对值 <=10000、三点不共线；叉积精确，圆心半径仍为浮点近似。long double 不保证比 double 精度高。

```
optional<RealPlane::Circle> triangle_incircle(RealPlane::Point a,
                                              RealPlane::Point b,
                                              RealPlane::Point c)
{
    using G = RealPlane;
    using R = G::R;
    auto u = b - a, v = c - a;
    R scale = max({fabsl(u.x), fabsl(u.y), fabsl(v.x), fabsl(v.y)});
    if (scale == 0)
        return nullopt;
    R area = fabsl(G::cross(u, v));
    if (area == 0)
        return nullopt;
    u = u / scale;
    v = v / scale;
    R x = G::norm(v - u), y = G::norm(v), z = G::norm(u);
    R sum = x + y + z;
    auto o = a + (u * (y / sum) + v * (z / sum)) * scale;
    return G::Circle{o, area / scale / sum};
}
```

模板题使用：AOJ CGL_7_B 三角形内切圆

题意：输入三个非共线整点，坐标绝对值不超过 10000，输出内心坐标与半径；要求误差小于 1e-6。官方题面已核对，完整协议本地验证，尚无本实现在线记录。

所需模板：RealPlane（第 987 页，58.1 节）、triangle_incircle（第 1034 页，58.21 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_B&lang=en

```
int main()
{
    RealPlane::Point a, b, c;
    cin >> a.x >> a.y >> b.x >> b.y >> c.x >> c.y;
    auto circle = triangle_incircle(a, b, c);
    if (circle)
        cout << fixed << setprecision(12) << circle->o.x << ' '
             << circle->o.y << ' ' << circle->r << '\n';
}
```

第五十九章　三维计算几何基础

59.1　整数三维共线、共面与方向

坐标绝对值 $\le 10^9$；orient(a,b,c,d) 为有向六倍四面体体积，0 表示共面；collinear 判三点共线，on_segment 支持退化线段。int128 精确判定，不包含三维凸包构造。

```
struct IntegerGeometry3D
{
    using I = __int128_t;

    // |coordinates|<=1e9: triple products fit signed int128.
    struct Point
    {
        long long x, y, z;
    };

    struct Vector
    {
        I x, y, z;
    };

    static Vector diff(Point a, Point b)
    {
        return {I(a.x) - b.x, I(a.y) - b.y, I(a.z) - b.z};
    }

    static Vector cross(Vector a, Vector b)
    {
        return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
    }

    static I dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y + a.z * b.z; }

    static I orient(Point a, Point b, Point c, Point d)
    {
        return dot(cross(diff(b, a), diff(c, a)), diff(d, a));
    }

    static bool collinear(Point a, Point b, Point c)
    {
        auto v = cross(diff(b, a), diff(c, a));
        return !v.x && !v.y && !v.z;
    }

    static bool on_segment(Point p, Point a, Point b)
    {
        return collinear(p, a, b) && dot(diff(a, p), diff(b, p)) <= 0;
    }
};
```

接口用法演示：自定义整数三维谓词

演示约定：自定义协议，非 OJ 题：0≤Q≤200000，每次给 a,b,c,d 四个整数点，各坐标绝对值 ≤10^9。输出 orient(a,b,c,d)、collinear(a,b,c)、on_segment(d,a,b)，后两项为 0/1。第一项是有向六倍体积，用 128 位整数精确输出，不能整除 6 后冒充体积；为 0 也不保证前三点确定唯一平面。允许重复点、退化和闭线段端点。此点域的安全界不扩展到任意 Vector 参数或反复组合运算。

所需模板：IntegerGeometry3D（第 1036 页，59.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/src/compact/geometry_extra.hpp

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    auto print = [](__int128_t x)
    {
        // Unsigned subtraction also handles the minimum signed int128 value.
        __uint128_t u = x < 0 ? __uint128_t(0) - __uint128_t(x) : __uint128_t(x);
        string s;
        do { s += char('0' + u % 10); u /= 10; } while (u);
        if (x < 0) cout << '-';
        reverse(s.begin(), s.end());
        cout << s;
    };
    int q;
    if (!(cin >> q)) return 0;
    while (q--)
    {
        IntegerGeometry3D::Point a, b, c, d;
        for (auto p : {&a, &b, &c, &d}) cin >> p->x >> p->y >> p->z;
        print(IntegerGeometry3D::orient(a, b, c, d));
        cout << ' ' << IntegerGeometry3D::collinear(a, b, c)
             << ' ' << IntegerGeometry3D::on_segment(d, a, b) << '\n';
    }
}
```

相关应用：不扰动增量凸包，精确方向符号、低维退化输出、面积及六倍体积。

59.2 三维凸包：精确方向与退化处理

Hull3D<T>(points)，T 只取 long long 或 long double（默认）。输入编号从 0 起，不修改原数组。p 为去重后字典序点，id 映射首次原编号；faces 是 p 下标组成的外向三角面。dimension 为空/点/线/平面/三维时分别为-1/0/1/2/3；低维 boundary 给出点、线端点或平面凸多边形边界。area() 在平面时只算单面，三维时算表面积；volume6() 返回六倍体积，低维为 0。

整数坐标绝对值 $\leq 10^9$，方向与六倍体积用 128 位精确运算。实数用过滤后浮点展开式确定符号，不用 eps、不扰动坐标；按已读入的二进制数求凸包，面积和实数体积仍为近似。需最近舍入、禁用 fast-math，运算及残差不得上下溢；足够安全的坐标域和具体约定见后页。

时间 $O(n^2 \log n)$、额外空间 $O(n)$。外露字段用于读取结果，不应构造后自行改写。点数与面下标须能用 int 表示。

字段与面积运算：

```
template <class T = long double>
struct Hull3D
{
    static_assert(is_same_v<T, long long> || is_same_v<T, long double>);
    using W = conditional_t<is_same_v<T, long long>, __int128, long double>;
    struct Point
    {
        T x, y, z;
    };
    vector<Point> p;
    vector<int> id, boundary;
    vector<array<int, 3>> faces;
    int dimension = -1;

    static array<W, 3> normal(Point a, Point b, Point c)
    {
        W x = W(b.x) - a.x;
        W y = W(b.y) - a.y;
        W z = W(b.z) - a.z;
        W u = W(c.x) - a.x;
        W v = W(c.y) - a.y;
        W w = W(c.z) - a.z;
        return {y * w - z * v, z * u - x * w, x * v - y * u};
    }
    static W orient(Point a, Point b, Point c, Point d)
    {
        auto v = normal(a, b, c);
        return v[0] * (W(d.x) - a.x) + v[1] * (W(d.y) - a.y) +
               v[2] * (W(d.z) - a.z);
    }
```

浮点展开式运算（接同一结构体）：

```
// Exact signs for represented floating inputs; nearest rounding, no fast-math.
// Products and residuals must neither overflow nor underflow.
using Expansion = vector<long double>;
static void grow(Expansion &a, long double q)
{
    Expansion out;
    for (long double x : a)
    {
        long double s = q + x;
        long double v = s - q;
        long double error = (q - (s - v)) + (x - v);
        if (error != 0) out.push_back(error);
        q = s;
    }
    if (q != 0) out.push_back(q);
    a.swap(out);
}
static Expansion difference(long double a, long double b)
{
    Expansion result{a};
    grow(result, -b);
    return result;
}
static Expansion product(const Expansion &a, const Expansion &b)
{
    Expansion result;
    for (long double x : a)
        for (long double y : b)
        {
            long double value = x * y;
            grow(result, fmal(x, y, -value));
            grow(result, value);
        }
    return result;
}
static int expansion_sign(const Expansion &a)
{
    if (a.empty()) return 0;
    return a.back() > 0 ? 1 : -1;
}
```

二维方向与共线检查（接同一结构体）：

```
    static int turn_sign(T x, T y, T u, T v, T s, T t)
    {
        W a = W(u) - x;
        W b = W(v) - y;
        W c = W(s) - x;
        W d = W(t) - y;
        W value = a * d - b * c;
        if constexpr (is_same_v<T, long long>)
            return (value > 0) - (value < 0);
        else
        {
            long double bound = 32 * numeric_limits<long double>::epsilon() *
                                (abs(a * d) + abs(b * c));
            if (abs(value) > bound) return value > 0 ? 1 : -1;
            auto result = product(difference(u, x), difference(t, y));
            auto subtract = product(difference(v, y), difference(s, x));
            for (long double z : subtract) grow(result, -z);
            return expansion_sign(result);
        }
    }
    static array<int, 3> normal_sign(Point a, Point b, Point c)
    {
        return {turn_sign(a.y, a.z, b.y, b.z, c.y, c.z),
                turn_sign(a.z, a.x, b.z, b.x, c.z, c.x),
                turn_sign(a.x, a.y, b.x, b.y, c.x, c.y)};
    }
```

三维方向的过滤与精确回退（接同一结构体）：

```
static int orient_sign(Point a, Point b, Point c, Point d)
{
    W value = orient(a, b, c, d);
    if constexpr (is_same_v<T, long long>)
        return (value > 0) - (value < 0);
    else
    {
        array<long double, 3> x{b.x - a.x, b.y - a.y, b.z - a.z};
        array<long double, 3> y{c.x - a.x, c.y - a.y, c.z - a.z};
        array<long double, 3> z{d.x - a.x, d.y - a.y, d.z - a.z};
        long double permanent = 0;
        for (int i = 0; i < 3; i++)
        {
            int j = (i + 1) % 3;
            int k = (i + 2) % 3;
            permanent += (abs(x[j] * y[k]) + abs(x[k] * y[j])) * abs(z[i]);
        }
        long double bound = 128 * numeric_limits<long double>::epsilon() * permanent;
        if (abs(value) > bound) return value > 0 ? 1 : -1;
        array<Expansion, 3> ex{difference(b.x, a.x), difference(b.y, a.y), difference(b.z, a.z)};
        array<Expansion, 3> ey{difference(c.x, a.x), difference(c.y, a.y), difference(c.z, a.z)};
        array<Expansion, 3> ez{difference(d.x, a.x), difference(d.y, a.y), difference(d.z, a.z)};
        Expansion result;
        for (int i = 0; i < 3; i++)
        {
            int j = (i + 1) % 3;
            int k = (i + 2) % 3;
            auto positive = product(product(ex[j], ey[k]), ez[i]);
            auto negative = product(product(ex[k], ey[j]), ez[i]);
            for (long double v : positive) grow(result, v);
            for (long double v : negative) grow(result, -v);
        }
        return expansion_sign(result);
    }
}
```

构造、排序与去重（接同一结构体）：

```
    explicit Hull3D(const vector<Point> &input)
    {
        vector<int> order(input.size());
        iota(order.begin(), order.end(), 0);
        for (auto v : input)
        {
            if constexpr (is_same_v<T, long long>)
                assert(-1000000000 <= v.x && v.x <= 1000000000 &&
                       -1000000000 <= v.y && v.y <= 1000000000 &&
                       -1000000000 <= v.z && v.z <= 1000000000);
            else assert(isfinite(v.x) && isfinite(v.y) && isfinite(v.z));
        }
        auto key = [](Point v)
        {
            return make_tuple(v.x, v.y, v.z);
        };
        sort(order.begin(), order.end(), [&](int a, int b)
        {
            return make_tuple(input[a].x, input[a].y, input[a].z, a) <
                   make_tuple(input[b].x, input[b].y, input[b].z, b);
        });
        for (int i : order)
            if (p.empty() || key(input[i]) != key(p.back()))
            {
                p.push_back(input[i]);
                id.push_back(i);
            }
```

低维识别与平面投影（接同一构造函数）：

```
    int n = p.size();
    if (n == 0) return;
    dimension = 0;
    boundary = {0};
    if (n == 1) return;
    dimension = 1;
    boundary = {0, n - 1};
    int c = 2;
    while (c < n && normal_sign(p[0], p[1], p[c]) == array<int, 3>{}) c++;
    if (c == n) return;
    auto axis = normal_sign(p[0], p[1], p[c]);
    int d = 2;
    while (d < n && orient_sign(p[0], p[1], p[c], p[d]) == 0) d++;
    if (d == n)
    {
        dimension = 2;
        int drop = 0;
        while (axis[drop] == 0) drop++;
        auto project = [&](Point v)
        {
            if (drop == 0) return pair<T, T>{v.y, v.z};
            if (drop == 1) return pair<T, T>{v.x, v.z};
            return pair<T, T>{v.x, v.y};
        };
        auto turn = [&](int a, int b, int c)
        {
            auto [x, y] = project(p[a]);
            auto [u, v] = project(p[b]);
            auto [s, t] = project(p[c]);
            return turn_sign(x, y, u, v, s, t);
        };
```

平面单调链与三角化（接同一构造函数）：

```
        order.resize(n);
        iota(order.begin(), order.end(), 0);
        sort(order.begin(), order.end(), [&](int a, int b)
        {
            return project(p[a]) < project(p[b]);
        });
        boundary.clear();
        for (int pass = 0; pass < 2; pass++)
        {
            int base = boundary.size();
            for (int v : order)
            {
                while ((int)boundary.size() >= base + 2 &&
                       turn(boundary[boundary.size() - 2], boundary.back(), v) <= 0)
                    boundary.pop_back();
                boundary.push_back(v);
            }
            boundary.pop_back();
            reverse(order.begin(), order.end());
        }
        for (int i = 1; i + 1 < (int)boundary.size(); i++)
            faces.push_back({boundary[0], boundary[i], boundary[i + 1]});
        return;
    }
```

初始四面体与增量可见面（接同一构造函数）：

```
        dimension = 3;
        boundary.clear();
        array<int, 4> seed{0, 1, c, d};
        for (int i = 0; i < 4; i++)
        {
            array<int, 3> f{seed[(i + 1) % 4], seed[(i + 2) % 4], seed[(i + 3) % 4]};
            if (orient_sign(p[f[0]], p[f[1]], p[f[2]], p[seed[i]]) > 0)
                swap(f[1], f[2]);
            faces.push_back(f);
        }
        for (int v = 0; v < n; v++)
        {
            if (find(seed.begin(), seed.end(), v) != seed.end()) continue;
            vector<array<int, 3>> keep;
            set<pair<int, int>> horizon;
            for (auto f : faces)
            {
                if (orient_sign(p[f[0]], p[f[1]], p[f[2]], p[v]) <= 0)
                {
                    keep.push_back(f);
                    continue;
                }
                for (int k = 0; k < 3; k++)
                {
                    int a = f[k];
                    int b = f[(k + 1) % 3];
                    if (!horizon.erase({b, a})) horizon.insert({a, b});
                }
            }
            for (auto [a, b] : horizon) keep.push_back({a, b, v});
            faces.swap(keep);
        }
    }
```

面积与六倍体积（接同一结构体）：

```
    long double area() const
    {
        long double answer = 0;
        for (auto f : faces)
        {
            auto v = normal(p[f[0]], p[f[1]], p[f[2]]);
            answer += hypotl(hypotl((long double)v[0], (long double)v[1]),
                             (long double)v[2]) / 2;
        }
        return answer;
    }
    W volume6() const
    {
        W answer = 0;
        if (dimension != 3) return answer;
        for (auto f : faces) answer += orient(p[0], p[f[0]], p[f[1]], p[f[2]]);
        return answer;
    }
};
```

不变量 三维初始四面体各面均背向内部。新增点只删除严格可见的面，删除面中的反向公共边两两消去，剩余有向边是轮廓。用轮廓边和新点组成新面即可恢复闭合外向凸包；零方向不算可见，不能扰动共面点。所有点先排序去重，平面情况投影到法向非零的轴，单独用二维凸包处理。

精确符号的范围 浮点展开式保存每次加法和乘法被舍弃的残差，末项符号即精确和的符号；乘法残差通过 fma 计算。先用保守舍入误差界过滤明显非零的方向，不能确定时回退。需二进制、最近舍入、禁用 fast-math 及运算重排，保证运算和残差不上下溢。对于有效位不超过 113 且指数范围至少覆盖普通 double 的实现，每个非零坐标绝对值在 2 的负 100 次方至 2 的 100 次方内是一个足够安全域。输入十进制读取误差不被恢复。

测量和退化约定 精确方向不等于精确面积。面积的平方根与求和、实数体积仍可能失去相对精度，极薄模型需额外检验。整数域内六倍体积精确且非负；平面核心只返回单面面积，点和线面积为 0。题目驱动选择平面双面包裹约定，题面本身没有明确低维语义；其他题应按题意适配。

独立验证 565 组输入分别验证整数和实数版，参考为精确支撑平面枚举及每个面的二维凸包；检查外向、无退化面、反向边配对、Euler 关系、面积和体积。另有 8000 组精确有理数方向参考，含共面、nextafter 近共面、不同指数和减法残差。原先放大整数点导致浮点行列式误判的反例保留。

模板题使用：P4724 三维凸包

题意：4≤n≤2000 个实数点，输出三维凸包表面积，保留三位小数。题面未给坐标界。驱动对纯平面输入采用正反双面包裹面积约定；核心 area() 仍返回平面单面面积。线上状态见验证记录。

所需模板：Hull3D（第 1038 页，59.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4724

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<Hull3D<>::Point> p(n);
    for (auto &v : p) cin >> v.x >> v.y >> v.z;
    Hull3D hull(p);
    long double area = hull.area();
    if (hull.dimension == 2) area *= 2;
    cout << fixed << setprecision(3) << area << '\n';
}
```

59.3 整数凸多面体：体积重心与表面距离

代码依赖：Hull3D（第 1038 页，59.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

先构造 Hull3D<long long> h，再构造 Hull3DMetrics m(h)，h 须存活且不修改。整数点和查询点坐标绝对值 $\le 10^9$。centroid() 返回 optional<array<long double,3» 体积重心；clearance() 返回重心到表面的最短距离，低维均为空。plane_distance(p,i) 为整数点到 faces[i] 所在无限平面的距离，不是到三角形的距离。planes() 给每个几何平面的代表三角面下标；二维为 1 个、点线为空。三角面数直接读 h.faces.size()。

构造 O(n+F)，重心 O(n)，距离 O(nF)，平面分组 $O(F^2)$，空间 O(n+F)。128 位精确累加非负四面体质量，距离通过非负仿射积分计算，避免薄凸包的重心点积抵消；最终浮点测量仍有舍入。mass、total 只供实现，不应改写。

字段与非负四面体质量：

```
// Integer coordinates |x|, |y|, |z| <= 1e9. Keep h alive and unchanged.
struct Hull3DMetrics
{
    using Hull = Hull3D<long long>;
    using Point = Hull::Point;
    const Hull &h;
    vector<__int128> mass;
    __int128 total = 0;

    explicit Hull3DMetrics(const Hull &h) : h(h), mass(h.p.size())
    {
        if (h.dimension != 3) return;
        for (auto f : h.faces)
        {
            __int128 v = Hull::orient(h.p[0], h.p[f[0]], h.p[f[1]], h.p[f[2]]);
            mass[0] += v;
            for (int u : f) mass[u] += v;
            total += 4 * v;
        }
    }
```

体积重心与整数点到平面距离（接同一结构体）：

```
    // Uniform volume centroid; absent for dimension < 3.
    optional<array<long double, 3>> centroid() const
    {
        if (total == 0) return nullopt;
        array<long long, 3> base{h.p[0].x, h.p[0].y, h.p[0].z};
        for (Point p : h.p)
        {
            base[0] = min(base[0], p.x);
            base[1] = min(base[1], p.y);
            base[2] = min(base[2], p.z);
        }
        array<long double, 3> result{};
        for (int i = 0; i < int(h.p.size()); i++)
        {
            array<long long, 3> p{h.p[i].x, h.p[i].y, h.p[i].z};
            for (int k = 0; k < 3; k++)
                result[k] += (long double)mass[i] * (p[k] - base[k]);
        }
        for (int k = 0; k < 3; k++) result[k] = base[k] + result[k] / total;
        return result;
    }
    long double normal_length(int face) const
    {
        auto f = h.faces[face];
        auto v = Hull::normal(h.p[f[0]], h.p[f[1]], h.p[f[2]]);
        return hypotl(hypotl((long double)v[0], (long double)v[1]),
                      (long double)v[2]);
    }
    // Distance to the infinite plane, NOT to the bounded triangle.
    long double plane_distance(Point p, int face) const
    {
        auto f = h.faces[face];
        auto v = Hull::orient(h.p[f[0]], h.p[f[1]], h.p[f[2]], p);
        return fabsl((long double)v) / normal_length(face);
    }
```

重心到表面距离及几何面分组（接同一结构体）：

```
    // Distance from the volume centroid to the surface; O(n * faces).
    optional<long double> clearance() const
    {
        if (total == 0) return nullopt;
        long double answer = numeric_limits<long double>::infinity();
        for (int i = 0; i < int(h.faces.size()); i++)
        {
            auto f = h.faces[i];
            long double sum = 0;
            for (int j = 0; j < int(h.p.size()); j++)
            {
                auto d = -Hull::orient(h.p[f[0]], h.p[f[1]], h.p[f[2]], h.p[j]);
                sum += (long double)mass[j] * (long double)d;
            }
            answer = min(answer, sum / total / normal_length(i));
        }
        return answer;
    }
    // One representative triangle per geometric plane; O(faces^2).
    vector<int> planes() const
    {
        vector<int> result;
        for (int i = 0; i < int(h.faces.size()); i++)
        {
            bool found = false;
            for (int j : result)
            {
                auto f = h.faces[j];
                bool same = true;
                for (int u : h.faces[i])
                    if (Hull::orient(h.p[f[0]], h.p[f[1]], h.p[f[2]], h.p[u]) != 0)
                        same = false;
                if (same)
                {
                    found = true;
                    break;
                }
            }
            if (!found) result.push_back(i);
        }
        return result;
    }
};
```

精度约定 取凸包点 p[0] 作公共四面体顶点，各外向面的六倍体积 v 非负，四顶点各加 v，total 为四倍六倍体积。坐标界 M 下 total 不超过 $192M^3$，128 位安全。各坐标先减最小值后非负求和；距离则对面行列式作非负质量积分，不用近似重心相减。浮点乘除和范数仍有舍入，重心接近零时不保证相对误差。

验证范围 509 组精确有理矩与支撑平面真值，六种构建通过；含行列式 1 的极薄整数四面体。原 kuangbin 完整 246 行与驱动、最小抄写版各 102 组 EOF 对拍。HDU4273 记录 41022769 已 AC，15MS、1844K；原题是整数坐标比赛应用，不据此声明任意实数测量已覆盖。

应用题补充：HDU4273 Rescue：重心到表面距离

题意：EOF 多组，4≤n≤100，整数坐标绝对值 ≤10000，输入均为非退化凸多面体顶点。输出三位小数。使用非负顶点质量直接积分各平面距离，避免薄凸包相减。

所需模板：Hull3D（第 1038 页，59.2 节）、Hull3DMetrics（第 1048 页，59.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://acm.hdu.edu.cn/showproblem.php?pid=4273

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    while (cin >> n)
    {
        vector<Hull3D<long long>::Point> p(n);
        for (auto &v : p) cin >> v.x >> v.y >> v.z;
        Hull3D<long long> h(p);
        Hull3DMetrics metrics(h);
        cout << fixed << setprecision(3) << *metrics.clearance() << '\n';
    }
}
```

接口用法演示：体积重心、代表面和原点到面平面的距离

演示约定：输入 n 及 n 个整数点。输出重心和表面距离（低维输出 NO VOLUME），再输出各几何平面的三角面编号、原输入顶点编号及原点到无限平面的距离。全部下标从 0 开始。

所需模板：Hull3D（第 1038 页，59.2 节）、Hull3DMetrics（第 1048 页，59.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/HULL3D-METRICS.md

```
int main()
{
    int n;
    cin >> n;
    vector<Hull3D<long long>::Point> p(n);
    for (auto &v : p) cin >> v.x >> v.y >> v.z;
    Hull3D<long long> h(p);
    Hull3DMetrics metrics(h);
    cout << fixed << setprecision(12);
    auto center = metrics.centroid();
    if (!center)
        cout << "NO VOLUME\n";
    else
    {
        for (auto x : *center) cout << x << ' ';
        cout << *metrics.clearance() << '\n';
    }
    auto planes = metrics.planes();
    cout << planes.size() << '\n';
    for (int i : planes)
    {
        cout << i << ' ';
        for (int v : h.faces[i]) cout << h.id[v] << ' ';
        cout << metrics.plane_distance({0, 0, 0}, i) << '\n';
    }
}
```

59.4 实数三维向量与夹角

Point 为 long double 三坐标，支持加减、数乘/除和精确字典序；排序禁用 epsilon。dot、cross、norm 为点积、叉积、长度；unit 和 angle 要求非零向量，angle 返回 $0..\pi$ 的有向向量夹角，弧度制。unit(a)*r 变换长度；点间距离为 norm(a-b)。有限输入且中间量不溢出/下溢，极端尺度须预缩放；浮点不是精确共面判定。O(1) 时间空间。

```
struct RealSpace
{
    using R = long double;

    struct Point
    {
        R x = 0, y = 0, z = 0;

        Point operator+(Point b) const
        {
            return {x + b.x, y + b.y, z + b.z};
        }

        Point operator-(Point b) const
        {
            return {x - b.x, y - b.y, z - b.z};
        }

        Point operator*(R k) const
        {
            return {x * k, y * k, z * k};
        }

        Point operator/(R k) const
        {
            return {x / k, y / k, z / k};
        }

        bool operator<(Point b) const
        {
            return tie(x, y, z) < tie(b.x, b.y, b.z);
        }
    };
```

继续三维运算（接上页同一结构体）：

```
    static R dot(Point a, Point b)
    {
        return a.x * b.x + a.y * b.y + a.z * b.z;
    }

    static Point cross(Point a, Point b)
    {
        return {a.y * b.z - a.z * b.y,
                a.z * b.x - a.x * b.z,
                a.x * b.y - a.y * b.x};
    }

    static R norm(Point a)
    {
        return hypot(a.x, a.y, a.z);
    }

    static Point unit(Point a)
    {
        R len = norm(a);
        assert(len > 0);
        return a / len;
    }

    // Directed vectors: angle in [0, pi], both must be nonzero.
    static R angle(Point a, Point b)
    {
        a = unit(a);
        b = unit(b);
        return atan2(norm(cross(a, b)), dot(a, b));
    }
};
```

接口用法演示：三维向量运算（接口演示）

演示约定：EOF 输入两个向量 a,b，输出点积、叉积、两长度和 0..π 夹角；任一向量为零时夹角输出-1。不是正式 OJ 题。

所需模板：RealSpace（第 1053 页，59.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：

```
int main()
{
    using G = RealSpace;
    G::Point a, b;
    cout << fixed << setprecision(15);
    while (cin >> a.x >> a.y >> a.z >> b.x >> b.y >> b.z)
    {
        auto c = RealSpace::cross(a, b);
        cout << G::dot(a, b) << '\n';
        cout << c.x << ' ' << c.y << ' ' << c.z << '\n';
        cout << G::norm(a) << ' ' << G::norm(b) << '\n';
        if (G::norm(a) > 0 && G::norm(b) > 0)
            cout << G::angle(a, b) << '\n';
        else
            cout << -1 << '\n';
    }
    return 0;
}
```

59.5 三维直线投影、线段距离与绕轴旋转

代码依赖：RealSpace（第 1053 页，59.4 节）。抄写时一并准备；标准库类型不列入算法依赖。

Line3(p,d) 表示 p+t*d，注意第二参数是方向；through(a,b) 从两个端点构造。线段限制 0≤t≤1。projection/segment_projection 返回最近点，distance/segment_distance 返回距离，on_segment(q,eps) 使用绝对距离容差。d=0 时这些操作按点 p 处理；rotate(q,rad) 必须 d 非零，绕 p+t*d 按右手法则旋转，角度单位弧度。公开 p,d。依赖 RealSpace；O(1)。近退化浮点运算须按输入尺度分析误差。

```
struct Line3 : RealSpace
{
    Point p, d; // p + t*d; segment uses 0 <= t <= 1.

    Line3(Point p = {}, Point d = {}) : p(p), d(d)
    {
    }

    static Line3 through(Point a, Point b)
    {
        return Line3(a, b - a);
    }

    Point projection(Point q) const
    {
        if (norm(d) == 0) return p;
        Point u = unit(d);
        return p + u * dot(q - p, u);
    }

    Point segment_projection(Point q) const
    {
        R len = norm(d);
        if (len == 0) return p;
        Point u = d / len;
        R t = clamp(dot(q - p, u), R(0), len);
        return p + u * t;
    }
```

继续三维运算（接上页同一结构体）：

```
    R distance(Point q) const
    {
        if (norm(d) == 0) return norm(q - p);
        return norm(cross(unit(d), q - p));
    }

    R segment_distance(Point q) const
    {
        return norm(q - segment_projection(q));
    }

    bool on_segment(Point q, R eps = 1e-10L) const
    {
        assert(eps >= 0);
        return segment_distance(q) <= eps;
    }

    // Right-hand rotation about p + t*d; d must be nonzero.
    Point rotate(Point q, R rad) const
    {
        Point u = unit(d), v = q - p;
        R c = cos(rad), s = sin(rad);
        return p + v * c + cross(u, v) * s + u * (dot(u, v) * (1 - c));
    }
};
```

接口用法演示：三维直线/线段投影与绕轴旋转（接口演示）

演示约定：EOF 输入端点 a,b、查询点 q 和弧度 rad；输出直线/线段垂足、两距离、线段包含判定及绕 a 到 b 方向轴的旋转结果。a=b 时按点计算距离和投影，旋转输出 no axis。

所需模板：RealSpace（第 1053 页，59.4 节）、Line3（第 1056 页，59.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：

```
int main()
{
    using G = RealSpace;
    G::Point a, b, q;
    G::R rad;
    cout << fixed << setprecision(15);
    while (cin >> a.x >> a.y >> a.z >> b.x >> b.y >> b.z
               >> q.x >> q.y >> q.z >> rad)
    {
        auto l = Line3::through(a, b);
        auto p = l.projection(q);
        auto s = l.segment_projection(q);
        cout << p.x << ' ' << p.y << ' ' << p.z << '\n';
        cout << s.x << ' ' << s.y << ' ' << s.z << '\n';
        cout << l.distance(q) << ' ' << l.segment_distance(q) << '\n';
        cout << l.on_segment(q) << '\n';
        if (G::norm(l.d) > 0)
        {
            auto r = l.rotate(q, rad);
            cout << r.x << ' ' << r.y << ' ' << r.z << '\n';
        }
        else
            cout << "no axis\n";
    }
    return 0;
}
```

59.6 三维平面投影与分类求交

代码依赖：Line3（第 1056 页，59.5 节）、RealSpace（第 1053 页，59.4 节）。抄写时一并准备；标准库类型不列入算法依赖。

Plane3(p,normal) 要求非零法向量并单位化；through(a,b,c) 要求三点不共线；equation(n,h) 表示 n 点乘 x=h。signed_distance 保留法向符号，projection 返回垂足。intersect(line,q,eps,ang) 返回 1 单点、0 无交、-1 整条非退化线在面上，退化线按点处理。intersect(plane,line,eps,ang) 返回 1 交线、0 平行不同、-1 重合。仅返回 1 时写输出参数；交线 d 为单位向量。eps 为距离容差，ang 为单位方向的正弦容差；默认 1e-10 和 1e-12。近乎平行按容差分类，非精确拓扑判定；近乎平行求交误差会放大。公开 p 和单位 n，勿破坏单位不变量。依赖 RealSpace/Line3；O(1)。

```
struct Plane3 : RealSpace
{
    Point p, n; // n is a unit normal; dot(n, x - p) = 0.

    Plane3(Point p, Point normal) : p(p), n(unit(normal))
    {
    }

    static Plane3 through(Point a, Point b, Point c)
    {
        return Plane3(a, cross(b - a, c - a));
    }

    // dot(normal, x) = h; normal must be nonzero.
    static Plane3 equation(Point normal, R h)
    {
        Point u = unit(normal);
        return Plane3(u * (h / norm(normal)), u);
    }

    R signed_distance(Point q) const
    {
        return dot(n, q - p);
    }

    Point projection(Point q) const
    {
        return q - n * signed_distance(q);
    }
```

继续三维运算（接上页同一结构体）：

```
    // 1: one point; 0: none; -1: the nondegenerate line lies in plane.
    int intersect(Line3 l, Point &q, R eps = 1e-10L, R ang = 1e-12L) const
    {
        assert(eps >= 0 && 0 <= ang && ang < 1);
        R h = signed_distance(l.p);
        if (norm(l.d) == 0)
        {
            if (abs(h) > eps) return 0;
            q = l.p;
            return 1;
        }
        Point u = unit(l.d);
        R t = dot(n, u);
        if (abs(t) <= ang) return abs(h) <= eps ? -1 : 0;
        q = l.p - u * (h / t);
        return 1;
    }

    // 1: one line; 0: distinct parallel planes; -1: coincident planes.
    int intersect(Plane3 f, Line3 &l, R eps = 1e-10L, R ang = 1e-12L) const
    {
        assert(eps >= 0 && 0 <= ang && ang < 1);
        Point d = cross(n, f.n);
        R len = norm(d);
        if (len <= ang) return abs(signed_distance(f.p)) <= eps ? -1 : 0;
        d = d / len;
        Point v = cross(d, n);
        Point q = p + v * (dot(f.n, f.p - p) / dot(f.n, v));
        l = Line3(q, d);
        return 1;
    }
};
```

接口用法演示：平面投影和线面求交（接口演示）

演示约定：EOF 输入非零法向 n、常数 h、查询点 q 及直线端点 a,b，平面为 n 点乘 x=h。输出有符号距离、垂足和交点分类；仅 type=1 时输出单点。type=-1 为整条非退化线在面上，type=0 无交。

所需模板：RealSpace（第 1053 页，59.4 节）、Line3（第 1056 页，59.5 节）、Plane3（第 1059 页，59.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：

```
int main()
{
    using G = RealSpace;
    G::Point n, q, a, b;
    G::R h;
    cout << fixed << setprecision(15);
    while (cin >> n.x >> n.y >> n.z >> h
               >> q.x >> q.y >> q.z
               >> a.x >> a.y >> a.z >> b.x >> b.y >> b.z)
    {
        auto f = Plane3::equation(n, h);
        auto p = f.projection(q);
        cout << f.signed_distance(q) << '\n';
        cout << p.x << ' ' << p.y << ' ' << p.z << '\n';
        G::Point x;
        int type = f.intersect(Line3::through(a, b), x);
        cout << type << '\n';
        if (type == 1)
            cout << x.x << ' ' << x.y << ' ' << x.z << '\n';
    }
    return 0;
}
```

接口用法演示：两平面夹角与交线（接口演示）

演示约定：EOF 输入两平面的非零法向 a,b 及常数 h,k（顺序 a,h,b,k），输出有向法向夹角及求交类型；type=1 时输出交线上一点和单位方向；-1 重合、0 平行不同。无向平面夹角可取 min(theta,π-theta)。

所需模板：RealSpace（第 1053 页，59.4 节）、Line3（第 1056 页，59.5 节）、Plane3（第 1059 页，59.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：

```
int main()
{
    using G = RealSpace;
    G::Point a, b;
    G::R h, k;
    cout << fixed << setprecision(15);
    while (cin >> a.x >> a.y >> a.z >> h >> b.x >> b.y >> b.z >> k)
    {
        auto f = Plane3::equation(a, h);
        auto g = Plane3::equation(b, k);
        cout << G::angle(f.n, g.n) << '\n';
        Line3 l;
        int type = f.intersect(g, l);
        cout << type << '\n';
        if (type == 1)
        {
            cout << l.p.x << ' ' << l.p.y << ' ' << l.p.z << '\n';
            cout << l.d.x << ' ' << l.d.y << ' ' << l.d.z << '\n';
        }
    }
    return 0;
}
```

第六十章　距离

60.1　点到闭线段的距离

代码依赖：RealPlane（第 987 页，58.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

segment_distance_real(p,a,b) 返回点 p 到闭线段 ab 的 long double 距离。a=b 时返回到端点的距离；否则把投影参数截断至 [0,1]。要求有限且尺度适当的输入；O(1) 时间和空间。

```
RealPlane::R
segment_distance_real(RealPlane::Point p, RealPlane::Point a, RealPlane::Point b)
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    Point v = b - a;
    R d = G::dot(v, v);
    if (d == 0) return G::norm(p - a);
    R t = clamp(G::dot(p - a, v) / d, R(0), R(1));
    return G::norm(p - (a + v * t));
}
```

模板题使用：AOJ CGL_2_D

题意：求两条闭线段间距离，q 不超过 1000，整数坐标绝对值不超过 10000。先用 IntegerPlane 精确判相交（含接触和共线重叠），相交输出 0；否则取四个端点到另一线段距离的最小值。单点距离接口参数为 (p,a,b)。题目线段非退化，点线段属于额外接口边界。

所需模板：IntegerPlane（第 1000 页，58.8 节）、RealPlane（第 987 页，58.1 节）、segment_distance_real（第 1063 页，60.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_2_D

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    cout << fixed << setprecision(12);
    while (q--)
    {
        vector<IntegerPlane::Point> p(4);
        vector<RealPlane::Point> a(4);
        for (int i = 0; i < 4; i++)
        {
            cin >> p[i].x >> p[i].y;
            a[i] = {(long double)p[i].x, (long double)p[i].y};
        }
        long double ans = 0;
        if (!IntegerPlane::intersect(p[0], p[1], p[2], p[3]))
            ans = min({segment_distance_real(a[0], a[2], a[3]),
                       segment_distance_real(a[1], a[2], a[3]),
                       segment_distance_real(a[2], a[0], a[1]),
                       segment_distance_real(a[3], a[0], a[1])});
        cout << ans << '\n';
    }
}
```

第六十一章　Pick 定理

61.1　Pick 定理：格点、剖分与孔洞

简单、非退化、顶点为整数点且无孔的多边形，令 A 为普通非负面积，I 为严格内部格点数，B 为边界上的不同格点数，则

$$A = I + \frac{B}{2} - 1, \qquad I = \frac{|S| - B + 2}{2}, \qquad S = \sum_i (x_i y_{i+1} - y_i x_{i+1}).$$

S 是有向二倍面积，先取绝对值再代入；不要对每一条叉积分别取绝对值。有向面积接口见第 58.11 节（第 1005 页），组合用法见第 58.11 节（第 1007 页）。

边界为什么求 gcd　一条非零整数线段的差向量为 (u, v)，令 $g = \gcd(|u|, |v|)$。

所有线段格点恰为起点加 $t(u/g, v/g)$，$t = 0, 1, \ldots, g$。这是因为 $(u/g, v/g)$ 互素，任何整数点沿这个方向的参数只能是整数。每条边只计起点、不计终点，就有 g 个格点；沿简单边界相加，每个顶点恰计一次，故

$$B = \sum_i \gcd(|x_{i+1} - x_i|, |y_{i+1} - y_i|).$$

共线的额外边界顶点不改变这个总和。重复走边或自交时不能沿用“每点恰计一次”的论证。

空格点三角形的面积　这里的“空”指除三个顶点外，内部和边上都没有格点。平移一个顶点到原点，取一条边 (u, v)；它必须是本原向量，即 $\gcd(|u|, |v|) = 1$。由裴蜀取 $ru + sv = 1$，整数矩阵

$$M = \begin{pmatrix} r & s \\ -v & u \end{pmatrix}, \qquad \det M = 1$$

将该边变成 $(1, 0)$，同时双向保留整数格点和面积。必要时把纵坐标取反，再作整数剪切 $x \leftarrow x - qy$，第三点可写成 (a, b)，$0 \le a < b$、$b > 0$。若 $b > 1$，高度 $y = 1$ 的闭截面为 $[a/b, 1 + (a-1)/b]$：$a = 0$ 时含格点 $(0, 1)$，$a \ge 1$ 时含 $(1, 1)$，都是非顶点格点，矛盾。所以 $b = 1$，面积恰为 $1/2$。仅三条边本原还不够：$(0, 0), (2, 1), (1, 2)$ 的边都本原，但内部有 $(1, 1)$。仅内部无格点也不够：$(0, 0), (k, 0), (0, 1)$ 的面积为 $k/2$，边上可能有其他格点。

相容剖分与 Euler 计数　先只使用多边形的整数顶点作三角剖分，不引入非整数辅助点，再不断用内部或边上的非顶点格点细分。点在公共边上时，必须同时细分两侧三角形，不能留下悬挂节点。每次细分都增加正面积三角形数，而每个三角形的二倍面积是正整数，数量被总二倍面积界住，因此过程终止。最终每个小三角形面积 $1/2$，所有格点都是剖分顶点，每条边界小边是本原线段。令 V, E, F 分别为顶点、边、区域内小三角形数（不计无界面），则

$$V = I + B, \qquad F = 2A, \qquad 3F = 2E - B, \qquad V - E + F = 1.$$

消去 V, E, F 即得 Pick 公式。这是定理证明；比赛计算不需要实际构造剖分。

孔洞、方向与不能套用的情况 若区域连通，外边界与 h 条孔洞边界都是简单整数多边形，被挖去的孔洞区域严格在外边界内且彼此不交、不接触、不嵌套，同样的相容剖分满足 $V-E+F=1-h$，所以

$$A = I + \frac{B}{2} - 1 + h, \qquad I = \frac{2A - B + 2}{2} - h.$$

I 不含孔洞内部及任意边界；B 包含外环与所有内环的边界点。面积为外环面积减各孔洞面积，或约定外环逆时针、内环顺时针后累加有向面积。例如外方形 $[0,4]^2$ 去掉孔洞 $(1,3)^2$，有 $A=12, B=24, I=0$；漏掉 $h=1$ 会错得 $I=1$。这个公式不是本单环用法额外实现了孔洞 API 的声明。

非整数顶点、自交、边界相接或面积退化均不在上述合同内。蝴蝶结 $(0,0),(2,2),(0,2),(2,0)$ 的有向面积为 0，却不能按 Pick 计数。检查结果非负或分子为偶数，也不能代替简单性检查；正式题是直接保证简单多边形的。

整数位宽与组合用法 现有整数几何合同为 $n \le 10^5$、$|x|, |y| \le C = 10^{12}$。差值和其绝对值可用 long long，但逐项叉积、总二倍面积以及最终分子使用 signed128。粗界 $|S| \le 2nC^2$、$B \le 2nC$，在该范围内都安全；内部点数可超过 long long，示例直接打印 128 位整数。CSES2193 的正式限制更窄：$3 \le n \le 10^5$、坐标绝对值不超过 10^9，只输入一个简单边界环。复用 `polygon_area2` 与标准库 gcd 即可，不新增一份鞋带公式算法。

参考：https://oi-wiki.org/geometry/pick/；https://cses.fi/problemset/task/2193。

第六十二章　凸包

62.1　凸多边形外点的两条切线

代码依赖：IntegerPlane（第 1000 页，58.8 节）。抄写时一并准备；标准库类型不列入算法依赖。

p 须为严格凸、逆时针的整数多边形，无重复末点或共线边界顶点；3≤n≤INT_MAX/2，起始下标任意。所有顶点及 q 坐标绝对值 ≤10^12。返回 optional<pair<int,int» 的原 0-based 下标 (right,left)：对任意顶点 v，cross(q,p[right],v)≤0，cross(q,p[left],v)≥0。若切线贴整条边，选离 q 最近的端点。q 在内部或边界时返回 nullopt。O(log n) 时间、O(1) 额外空间，无预处理且输入不变；先扇形定位异态边，再二分可见边链两端。凸性、方向及数值范围均为前置条件，assert 不验证整个多边形；n<3 不在接口范围。散点先构造 integer_hull，若需原编号须另留映射。

```
optional<pair<int, int>> convex_tangents_i64(const vector<IntegerPlane::Point> &p,
                                             IntegerPlane::Point q)
{
    using G = IntegerPlane;
    assert(p.size() >= 3 && p.size() <= INT_MAX / 2);
    int n = p.size();
    auto visible = [&](int i)
    {
        return G::cross(p[i], p[(i + 1) % n], q) < 0;
    };
    auto sector = [&](int dir)
    {
        int l = 1, r = n - 1;
        while (l + 1 < r)
        {
            int m = (l + r) / 2;
            if (dir * G::cross(p[0], p[m], q) >= 0)
                l = m;
            else
                r = m;
        }
        return l;
    };
```

选择异态种子边并二分可见边链（接上页同一函数）：

```
    int a = G::sign(G::cross(p[0], p[1], q));
    int b = G::sign(G::cross(p[0], p[n - 1], q));
    int v, h;
    if (a < 0)
    {
        v = 0;
        h = b <= 0 ? n - 1 : sector(-1);
    }
    else if (b > 0)
    {
        v = n - 1;
        h = 0;
    }
    else
    {
        v = sector(1);
        h = 0;
        if (!visible(v)) return nullopt;
    }
    auto boundary = [&](int start, int end)
    {
        int l = 0, r = (end - start + n) % n;
        bool first = visible(start);
        while (l + 1 < r)
        {
            int m = (l + r) / 2;
            if (visible((start + m) % n) == first)
                l = m;
            else
                r = m;
        }
        return (start + r) % n;
    };
    return pair{boundary(h, v), boundary(v, h)};
}
```

接口用法演示：凸多边形外点切线（接口演示）

演示约定：自定义有效输入协议，非 OJ 题：输入 n m，3≤n≤100000、0≤m≤100000；随后 n 个严格凸 CCW 顶点，再 m 个查询点，坐标绝对值均 ≤10^12。每个外点输出原 0-based 编号 right left，整边相切选最近端点；内部或边界输出-1 -1。程序不验证凸性或方向，不接受重复末点；可从任意顶点开始列举。仅演示切线接口，不包含成都 2025 I 的旋转双指针计数。

所需模板：IntegerPlane（第 1000 页，58.8 节）、convex_tangents_i64（第 1067 页，62.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/src/compact/convex_tangents_i64.hpp

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<IntegerPlane::Point> p(n);
    for (auto &v : p) cin >> v.x >> v.y;
    while (m--)
    {
        IntegerPlane::Point q;
        cin >> q.x >> q.y;
        auto ans = convex_tangents_i64(p, q);
        if (ans)
            cout << ans->first << ' ' << ans->second << '\n';
        else
            cout << "-1 -1\n";
    }
}
```

应用题补充：ICPC Chengdu 2025 I / Inside Triangle

题意：多组严格嵌套的 CCW 凸多边形 P、Q，分别 3..300000 顶点，坐标绝对值 ≤10^9；T≤1000 且两种顶点总数分别 ≤500000。选 P 的 3 个不同顶点，使三角形非严格包含 Q，贴边也计数。输入 T，各组 n、n 个 P 点、m、m 个 Q 点。复用左切线、合法弦端点的单调性和前缀和，累计每个起点贡献后除 3；实际 O(n log m+n+m)，不是官方线性旋转切线版本。

所需模板：IntegerPlane（第 1000 页，58.8 节）、convex_tangents_i64（第 1067 页，62.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/gym/106161/problem/I

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using G = IntegerPlane;
    int t;
    cin >> t;
    while (t--)
    {
        int n, m;
        cin >> n;
        vector<G::Point> p(n);
        for (auto &v : p) cin >> v.x >> v.y;
        cin >> m;
        vector<G::Point> q(m);
        for (auto &v : q) cin >> v.x >> v.y;
        vector<int> f(2 * n);
        for (int i = 0, j = 1; i < n; i++)
        {
            int k = convex_tangents_i64(q, p[i])->second;
            j = max(j, i + 1);
            while (j + 1 < i + n && G::cross(p[i], p[(j + 1) % n], q[k]) >= 0)
                j++;
            f[i] = j;
            f[i + n] = j + n;
        }
        vector<long long> sum(2 * n + 1);
        for (int i = 0; i < 2 * n; i++) sum[i + 1] = sum[i] + f[i];
        long long ans = 0;
        for (int a = 0, g = 0, h = 0; a < n; a++)
        {
            while (f[g] < a + n) g++;
            h = max(h, a + 1);
            while (h <= f[a] && f[h] < g) h++;
            if (h <= f[a])
                ans += sum[f[a] + 1] - sum[h] - 1LL * (f[a] - h + 1) * (g - 1);
        }
        cout << ans / 3 << '\n';
    }
}
```

62.2 整数凸包：单调链

代码依赖：IntegerPlane（第 1000 页，58.8 节）。抄写时一并准备；标准库类型不列入算法依赖。

integer_hull(points) 按值接收 IntegerPlane::Point 序列，输出严格逆时针凸包，不重复首点、不保留共线边内点。自动排序去重；全共线时保留两个端点，空集及单点直接返回。坐标范围同 IntegerPlane。O(n log n) 时间、O(n) 空间；供凸包包含、直径与 Minkowski 和调用。

```
vector<IntegerPlane::Point> integer_hull(vector<IntegerPlane::Point> p)
{
    using G = IntegerPlane;
    using Point = G::Point;
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    int n = (int)p.size();
    if (n < 3) return p;
    vector<Point> h;
    for (Point v : p)
    {
        while (h.size() > 1 && G::cross(h[h.size() - 2], h.back(), v) <= 0)
            h.pop_back();
        h.push_back(v);
    }
    int low = (int)h.size();
    for (int i = n - 2; i >= 0; i--)
    {
        while ((int)h.size() > low && G::cross(h[h.size() - 2], h.back(), p[i]) <= 0)
            h.pop_back();
        h.push_back(p[i]);
    }
    h.pop_back();
    return h; // CCW, no repeated endpoint, no collinear interior points.
}
```

模板题使用：Library Checker static_convex_hull

题意：多组求整数点集的凸包，去重并去掉边上非顶点。输出顶点数及逆时针顶点，不重复首点；空集输出 0，共线时仅保留两端。传 move 交出点集，避免按值传参时额外复制。

所需模板：IntegerPlane（第 1000 页，58.8 节）、integer_hull（第 1071 页，62.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/static_convex_hull

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        vector<IntegerPlane::Point> points(n);
        for (auto &p : points) cin >> p.x >> p.y;
        auto hull = integer_hull(move(points));
        cout << hull.size() << '\n';
        for (auto p : hull) cout << p.x << ' ' << p.y << '\n';
    }
}
```

应用题补充：Luogu P2742

题意：求围住所有点的最短围栏长度。坐标绝对值 $\leq 10^6$、最多两位小数；按字符串读成放大 100 倍的整数，精确求凸包，最后才开方并除以 100。全共线时周长为两端距离的两倍，全部重合为 0。USACO 来源作为应用补充。

所需模板：IntegerPlane（第 1000 页，58.8 节）、integer_hull（第 1071 页，62.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2742

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    auto read = []()
    {
        string s;
        cin >> s;
        bool negative = s[0] == '-';
        int i = negative || s[0] == '+';
        long long value = 0;
        while (i < (int)s.size() && s[i] != '.') value = value * 10 + s[i++] - '0';
        i++;
        for (int j = 0; j < 2; j++)
        {
            value *= 10;
            if (i < (int)s.size()) value += s[i++] - '0';
        }
        return negative ? -value : value;
    };
    int n;
    cin >> n;
    vector<IntegerPlane::Point> p(n);
    for (auto &v : p)
    {
        v.x = read();
        v.y = read();
    }
    auto h = integer_hull(move(p));
    long double length = 0;
    for (int i = 0; i < (int)h.size(); i++)
        length += sqrtl((long double)IntegerPlane::dist2(h[i], h[(i + 1) % h.size()]));
    cout << fixed << setprecision(2) << length / 100 << '\n';
}
```

62.3 点在整数凸包内（二分）

代码依赖：IntegerPlane（第 1000 页，58.8 节）、polygon_contains（第 1008 页，58.12 节）。抄写时一并准备；标准库类型不列入算法依赖。

convex_contains_i64(p,q) 返回 0 外部、1 边界、2 内部。要求严格逆时针凸包，无共线边内点、不重复首点，例如 integer_hull 的输出；允许循环旋转起点。空集、单点、线段转用 polygon_contains。坐标范围同 IntegerPlane，O(log n) 时间、O(1) 空间。

```
int convex_contains_i64(const vector<IntegerPlane::Point> &p, IntegerPlane::Point q)
{
    using G = IntegerPlane;
    using I = G::I;
    int n = (int)p.size();
    if (n < 3) return polygon_contains(p, q);
    I a = G::cross(p[0], p[1], q), b = G::cross(p[0], p[n - 1], q);
    if (a < 0 || b > 0) return 0;
    if (!a) return G::on_segment(q, p[0], p[1]) ? 1 : 0;
    if (!b) return G::on_segment(q, p[0], p[n - 1]) ? 1 : 0;
    int l = 1, r = n - 1;
    while (l + 1 < r)
    {
        int m = (l + r) / 2;
        if (G::cross(p[0], p[m], q) >= 0)
            l = m;
        else
            r = m;
    }
    I c = G::cross(p[l], p[r], q);
    return c < 0 ? 0 : c == 0 ? 1 : 2;
}
```

联合使用示例见第 1076 页（Luogu P4557 [JSOI2018] 战争），其中直接调用本模板。

相关应用：固定导航没有独立 Minkowski 和条目，按凸包的相关应用归档。

62.4 凸多边形 Minkowski 和

代码依赖：IntegerPlane（第 1000 页，58.8 节）、integer_hull（第 1071 页，62.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

minkowski_sum(a,b) 接受不重复首点的严格逆时针凸包；空输入返回空集，点和线段允许作为退化凸包。

点类型 IntegerPlane::Point；输入坐标及两集合坐标和的绝对值不超过 10^{12}。结果不重复首点，非退化时沿逆时针边方向合并，平行同向边同时推进；O(n+m) 时间。

退化输入枚举与另一凸包各点的和再求凸包，O((n+m) log(n+m))。按值传入，内部旋转不修改调用方；依赖 IntegerPlane 的 integer_hull/cross。

```
vector<IntegerPlane::Point> minkowski_sum(vector<IntegerPlane::Point> a,
                                          vector<IntegerPlane::Point> b)
{
    using G = IntegerPlane;
    using P = G::Point;
    using I = G::I;
    if (a.empty() || b.empty()) return {};
    auto plus = [](P u, P v)
    {
        return P{u.x + v.x, u.y + v.y};
    };
    if (a.size() < 3 || b.size() < 3)
    {
        vector<P> p;
        for (P u : a)
            for (P v : b) p.push_back(plus(u, v));
        return integer_hull(p);
    }
    auto start = [](vector<P> &p)
    {
        auto it = min_element(
            p.begin(), p.end(), [](P u, P v) { return tie(u.y, u.x) < tie(v.y, v.x); });
        rotate(p.begin(), it, p.end());
    };
    start(a);
    start(b);
    int n = a.size(), m = b.size(), i = 0, j = 0;
    vector<P> result;
```

合并凸包边（接上页同一函数）：

```
    while (i < n || j < m)
    {
        result.push_back(plus(a[i % n], b[j % m]));
        if (i == n)
        {
            ++j;
            continue;
        }
        if (j == m)
        {
            ++i;
            continue;
        }
        P u{a[(i + 1) % n].x - a[i].x, a[(i + 1) % n].y - a[i].y};
        P v{b[(j + 1) % m].x - b[j].x, b[(j + 1) % m].y - b[j].y};
        I c = G::cross({0, 0}, u, v);
        if (c >= 0) ++i;
        if (c <= 0) ++j;
    }
    return result;
}
```

应用题补充：Luogu P4557 [JSOI2018] 战争

题意：两组点各自凸包为领地；每次将第二组平移 d，问两领地是否相交，接触边界也输出 1。先构造 A+(-B)，再判断 d 是否在其闭包内。n、m、q 各至 10 万，坐标和平移量绝对值至 10^8。比赛应用，不计正式模板题覆盖。

所需模板：IntegerPlane（第 1000 页，58.8 节）、integer_hull（第 1071 页，62.2 节）、minkowski_sum（第 1075 页，62.4 节）、polygon_contains（第 1008 页，58.12 节）、convex_contains_i64（第 1074 页，62.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4557

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    vector<IntegerPlane::Point> a(n), b(m);
    for (auto &v : a) cin >> v.x >> v.y;
    for (auto &v : b)
    {
        cin >> v.x >> v.y;
        v.x = -v.x;
        v.y = -v.y;
    }
    auto p = minkowski_sum(integer_hull(a), integer_hull(b));
    while (q--)
    {
        IntegerPlane::Point v;
        cin >> v.x >> v.y;
        cout << (convex_contains_i64(p, v) != 0) << '\n';
    }
}
```

62.5 支撑点查询恢复下凸包

静态 build(left,right,oracle) 返回从左到右的严格下凸包顶点，Point 为 {x,y}。有限非空整数点集无需显式给出；回调 oracle(a,b) 必须精确返回使 a*x+b*y 最大的可行点，允许任意最优解。left/right 必须分别为最小/最大横坐标中纵坐标最低的点；全体同横坐标时二者必须相同。所有坐标绝对值不超过 10^18，方向差使用 long long，叉积先提升至 int128；回调内部也须保证数值精确且无溢出。对弦 a→b 查询方向 (b.y-a.y,a.x-b.x)，严格位于弦下方则递归分治，否则确认边；输出时删除共线中间点。若分治找到 K 个严格改善支撑点，查询次数为 2K+1（同横坐标时为 0），额外时间、空间和最坏递归深度为 O(K+1)。K 包括可能返回的共线非顶点，不将其等同于最终顶点数。不能直接传浮点近似优化器。二分图闭包组合：点为 (|S|,|N(S)|)，左点赋权 a、右点赋权 b，左 → 邻点依赖；下凸包方向满足 a≥0,b<0，最优闭包的右点恰为邻集。端点为 (0,0) 和 (左点数, 全部左点的邻点数)，有孤立右点时后者不等于全部右点数。该组合已做子集枚举对拍，尚不代表关联赛题整题通过。

```
struct SupportHull
{
    using ll = long long;
    using I = __int128_t;

    struct Point
    {
        ll x, y;
    };

    static I cross(Point a, Point b, Point c)
    {
        return (I(b.x) - a.x) * (I(c.y) - a.y) - (I(b.y) - a.y) * (I(c.x) - a.x);
    }
```

```
    // oracle(a,b) maximizes a*x+b*y exactly; |coordinates| <= 1e18.
    // left/right: lowest points at the minimum/maximum x.
    template <class Oracle>
    static vector<Point> build(Point left, Point right, Oracle &&oracle)
    {
        assert(left.x <= right.x);
        vector<Point> hull{left};
        auto dfs = [&](auto &&self, Point a, Point b) -> void
        {
            Point c = oracle(b.y - a.y, a.x - b.x);
            I turn = cross(a, b, c);
            assert(turn <= 0);
            if (turn < 0)
            {
                assert(a.x < c.x && c.x < b.x);
                self(self, a, c);
                self(self, c, b);
            }
            else
            {
                while (hull.size() >= 2 &&
                       cross(hull[hull.size() - 2], hull.back(), b) == 0)
                    hull.pop_back();
                hull.push_back(b);
            }
        };
        if (left.x < right.x)
            dfs(dfs, left, right);
        else
            assert(left.y == right.y);
        return hull;
    }
};
```

接口用法演示：自定义静态下凸链支撑查询

演示约定：自定义协议，非 OJ 题：0≤n≤2000，0≤q≤200000；点和方向坐标绝对值 ≤10^18，查询 b≤0。先输出严格下凸链长度及坐标，再输出 max(a*x+b*y) 及字典序最小的最大点；空集每次输出 EMPTY。用最左/最右横坐标上的最低点和全点集精确扫描 oracle 调用 build，再二分首个最大顶点；同 x 只留最低 y，不承诺原下标或向上查询。扫描回调使构造 O(n^2)、存储/递归 O(n)，查询 O(q log(max(2,n)))；显式点云通常用普通单调链更合适，本例演示隐式点集的 oracle 适配。生成方向 ≤2×10^18，回调点积 ≤4×10^36、叉积 ≤8×10^36，以 128 位精确运算；不扩展到完整 signed64 域。

所需模板：SupportHull（第 1077 页，62.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/src/compact/support_hull.hpp

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using P = SupportHull::Point;
    using I = __int128_t;
    int n, q;
    cin >> n >> q;
    vector<P> points(n), hull;
    for (auto &p : points) cin >> p.x >> p.y;
    auto less = [](P p, P r) { return p.x != r.x ? p.x < r.x : p.y < r.y; };
    auto dot = [](P p, long long a, long long b) { return I(a) * p.x + I(b) * p.y; };
    if (n)
    {
        P left = *min_element(points.begin(), points.end(), less), right = left;
        for (P p : points)
            if (p.x > right.x || (p.x == right.x && p.y < right.y)) right = p;
        hull = SupportHull::build(left, right, [&](long long a, long long b)
        {
            P best = points[0];
            for (P p : points)
                if (dot(p, a, b) > dot(best, a, b) ||
                    (dot(p, a, b) == dot(best, a, b) && less(p, best))) best = p;
            return best;
        });
    }
    cout << hull.size() << '\n';
    for (P p : hull) cout << p.x << ' ' << p.y << '\n';
```

继续处理操作（接上页同一 main）：

```
    while (q--)
    {
        long long a, b;
        cin >> a >> b; // Custom contract: b <= 0; upward support is excluded.
        if (hull.empty()) { cout << "EMPTY\n"; continue; }
        int l = 0, r = int(hull.size()) - 1;
        while (l < r)
        {
            int m = (l + r) / 2;
            if (dot(hull[m], a, b) < dot(hull[m + 1], a, b)) l = m + 1;
            else r = m; // First maximizing vertex, including edge/zero ties.
        }
        I value = dot(hull[l], a, b);
        if (value < 0) { cout << '-'; value = -value; }
        string digits;
        do { digits += char('0' + value % 10); value /= 10; } while (value);
        reverse(digits.begin(), digits.end());
        cout << digits << ' ' << hull[l].x << ' ' << hull[l].y << '\n';
    }
}
```

第六十三章　扫描线

63.1　整数轴对齐矩形面积并

调用 rectangle_union_area(rect)，每项按左、下、右、上存储四个 long long。端点有序，各坐标绝对值不超过 10^{18}；零面积矩形忽略，空集返回 0。返回精确 ___int128 面积；边界接触不增加面积，输入不变，多次调用独立。扫描线处理进出事件，纵向叶子代表相邻坐标间的半开区间。cover 仅记录整段覆盖次数，不下传；非零时取整段长度，为零时合并孩子。同横坐标事件间宽度为 0。时间 O(n log(n+1))、空间 O(n)，递归深度 O(log(n+1))；矩形数至多 268435454，且内存可用。题目坐标在 0 到 10^9，故驱动可将结果转 long long；一般输入不要直接窄化。不含周长、浮点或动态更新。

```
__int128 rectangle_union_area(const vector<array<long long, 4>> &rect)
{
    using ll = long long;
    assert(rect.size() <= (INT_MAX - 8) / 8);
    vector<array<ll, 4>> events;
    vector<ll> y;
    for (auto [l, d, r, u] : rect)
    {
        for (ll x : {l, d, r, u})
            assert(-1000000000000000000LL <= x && x <= 1000000000000000000LL);
        assert(l <= r && d <= u);
        if (l == r || d == u) continue;
        events.push_back({l, d, u, 1});
        events.push_back({r, d, u, -1});
        y.push_back(d);
        y.push_back(u);
    }
    if (events.empty()) return 0;
    sort(events.begin(), events.end());
    sort(y.begin(), y.end());
    y.erase(unique(y.begin(), y.end()), y.end());
    int n = int(y.size()) - 1;
    vector<int> cover(4 * n + 4);
    vector<ll> len(4 * n + 4);
```

覆盖长度维护与横向面积累加（接上页同一函数）：

```
    auto add = [&](auto &&self, int p, int l, int r, int a, int b, int v) -> void
    {
        if (a <= l && r <= b) cover[p] += v;
        else
        {
            int m = (l + r) / 2;
            if (a < m) self(self, p * 2, l, m, a, b, v);
            if (b > m) self(self, p * 2 + 1, m, r, a, b, v);
        }
        if (cover[p]) len[p] = y[r] - y[l];
        else len[p] = r - l == 1 ? 0 : len[p * 2] + len[p * 2 + 1];
    };
    __int128 area = 0;
    ll previous = events.front()[0];
    for (auto [x, d, u, v] : events)
    {
        area += (__int128)(x - previous) * len[1];
        int l = lower_bound(y.begin(), y.end(), d) - y.begin();
        int r = lower_bound(y.begin(), y.end(), u) - y.begin();
        add(add, 1, 0, n, l, r, int(v));
        previous = x;
    }
    return area;
}
```

模板题使用：Library Checker Area of Union of Rectangles

题意：输入 N 个轴对齐矩形，每行依次为 l,d,r,u。1≤N≤500000，0≤l<r≤10^9，0≤d<u≤10^9。直接按左、下、右、上存储，计算并集面积；不是面积和。所有矩形都在边长 10^9 的正方形内，最终面积 ≤10^18，可把 __int128 结果转成 long long 输出。

所需模板：rectangle_union_area（第 1081 页，63.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/area_of_union_of_rectangles

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<array<long long, 4>> rect(n);
    for (auto &a : rect) for (auto &x : a) cin >> x;
    // The statement bounds the whole union by [0,1e9]^2.
    cout << (long long)rectangle_union_area(rect) << '\n';
}
```

第六十四章　旋转卡壳

64.1　整数凸包直径：旋转卡壳

代码依赖：IntegerPlane（第 1000 页，58.8 节）。抄写时一并准备；标准库类型不列入算法依赖。

convex_diameter2(p,endpoints=nullptr) 返回最远两点距离平方，类型 __int128_t；不足两点返回 0。输入须为严格逆时针凸包，不重复首点，例如 integer_hull 的输出。坐标范围同 IntegerPlane；旋转卡壳 O(n) 时间、O(1) 额外空间。可传 pair<int,int> 的地址，接收凸包内的 0-based 端点下标；空集为 {-1,-1}，单点为 {0,0}，至少两点时端点不同。不保证并列最优端点的顺序。映射到原始输入编号由调用者完成。

```
IntegerPlane::I convex_diameter2(const vector<IntegerPlane::Point> &p,
                                 pair<int, int> *endpoints = nullptr)
{
    using G = IntegerPlane;
    using I = G::I;
    int n = (int)p.size();
    if (endpoints) *endpoints = n ? pair{0, n - 1} : pair{-1, -1};
    if (n < 2) return 0;
    if (n == 2) return G::dist2(p[0], p[1]);
    I ans = 0;
    auto update = [&](int a, int b)
    {
        I value = G::dist2(p[a], p[b]);
        if (value > ans)
        {
            ans = value;
            if (endpoints) *endpoints = {a, b};
        }
    };
    int j = 1;
    for (int i = 0; i < n; i++)
    {
        int k = (i + 1) % n;
        while (G::cross(p[i], p[k], p[(j + 1) % n]) > G::cross(p[i], p[k], p[j]))
            j = (j + 1) % n;
        update(i, j);
        update(k, j);
    }
    return ans;
}
```

模板题使用：Library Checker furthest_pair

题意：多组求距离最远的两个不同输入点编号。先建凸包，端点接口返回凸包下标，再按坐标映射回输入下标。题目保证至少两点；若凸包只剩一个点，所有坐标相同，输出 0、1。

所需模板：IntegerPlane（第 1000 页，58.8 节）、integer_hull（第 1071 页，62.2 节）、convex_diameter2（第 1083 页，64.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/furthest_pair

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        vector<IntegerPlane::Point> points(n);
        for (auto &p : points) cin >> p.x >> p.y;
        auto hull = integer_hull(points);
        if (hull.size() == 1)
        {
            cout << "0 1\n";
            continue;
        }
        pair<int, int> ends;
        convex_diameter2(hull, &ends);
        int a = -1, b = -1;
        for (int i = 0; i < n; i++)
        {
            if (points[i] == hull[ends.first]) a = i;
            if (points[i] == hull[ends.second]) b = i;
        }
        cout << a << ' ' << b << '\n';
    }
}
```

应用题补充：Luogu P1452

题意：求最多 5 万个点的最远距离平方，坐标绝对值 $\le 10^4$。先求严格凸包再旋转卡壳；本题答案 $\le 8\times 10^8$，可转 long long 输出，不能照搬到通用万亿坐标。USACO 来源作为应用补充；返回端点的正式示例见 Library Checker furthest_pair。

所需模板：IntegerPlane（第 1000 页，58.8 节）、integer_hull（第 1071 页，62.2 节）、convex_diameter2（第 1083 页，64.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1452

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<IntegerPlane::Point> p(n);
    for (auto &v : p) cin >> v.x >> v.y;
    auto h = integer_hull(move(p));
    cout << (long long)convex_diameter2(h) << '\n';
}
```

相关应用：凸包的三组支撑点旋转卡壳，最小包围矩形面积与逆时针四角。

64.2 最小面积包围矩形：旋转卡壳与四角

代码依赖：RealPlane（第 987 页，58.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

minimum_rectangle(p) 返回 pair(area,box)，box 是 array<Point,4>，非退化时按逆时针列出最优矩形四角；并列时不指定首点或方向。输入 RealPlane::Point 严格逆时针凸包，不重复首点、不保留共线中间点，不能直接传散点或顺时针多边形。空集返回 0 和四个原点；单点重复四次，线段返回 a,b,b,a；原输入不变。

沿每条凸包边维护前后投影极值与最大高度，恢复四条支撑线的交点。精确支撑比较下 O(n) 时间、O(1) 额外空间；n≤INT_MAX，输入及所有中间运算须有限可表示。浮点版不使用 eps 或扰动，近退化符号及角点精度仍受舍入影响；steps<n 是异常平坦比较的终止保护，不能修复舍入错误（异常时最坏 $O(n^2)$）。有界整数/定点题优先精确构造凸包，再传实数坐标。

```
pair<RealPlane::R, array<RealPlane::Point, 4>> minimum_rectangle(
    const vector<RealPlane::Point> &p)
{
    using G = RealPlane;
    using R = G::R;
    using P = G::Point;
    int n = int(p.size());
    if (n == 0)
        return {0, {}};
    if (n == 1)
        return {0, {p[0], p[0], p[0], p[0]}};
    if (n == 2)
        return {0, {p[0], p[1], p[1], p[0]}};
    int hi = 0;
    int lo = 0;
    int top = 0;
    P e = p[1] - p[0];
    for (int j = 1; j < n; j++)
    {
        if (G::dot(e, p[j] - p[hi]) > 0)
            hi = j;
        if (G::dot(e, p[j] - p[lo]) < 0)
            lo = j;
        if (G::cross(e, p[j] - p[top]) > 0)
            top = j;
    }
    R best = numeric_limits<R>::infinity();
    array<P, 4> box;
```

三组支撑点推进与四角恢复（接上页函数）：

```
    for (int i = 0; i < n; i++)
    {
        e = p[(i + 1) % n] - p[i];
        auto advance = [&](int &j, auto better)
        {
            int steps = 0;
            while (steps < n && better(p[(j + 1) % n] - p[j]))
            {
                j = (j + 1) % n;
                steps++;
            }
        };
        advance(hi, [&](P v)
        {
            return G::dot(e, v) >= 0;
        });
        advance(lo, [&](P v)
        {
            return G::dot(e, v) <= 0;
        });
        advance(top, [&](P v)
        {
            return G::cross(e, v) >= 0;
        });
        R d = G::dot(e, e);
        R l = G::dot(e, p[lo] - p[i]);
        R r = G::dot(e, p[hi] - p[i]);
        R h = G::cross(e, p[top] - p[i]);
        R area = (r - l) * h / d;
        if (area < best)
        {
            best = area;
            P a = p[i] + e * (l / d);
            P b = p[i] + e * (r / d);
            P v = G::perp(e) * (h / d);
            box = {a, b, b + v, a + v};
        }
    }
    return {best, box};
}
```

正确性 最小面积包围矩形至少有一条边与凸包边共线。固定此方向，左右和上方支撑线由投影最小、投影最大和叉积最大三点决定。严格凸包边方向依次旋转，三个最优支撑点只向前移动；相同投影推进到末端，以跨过平行边。第一条边完整扫描初始化，避免从错误局部极值开始。

恢复公式与边界 设底边方向 e、长度平方 d，最小/最大投影 l,r，最大叉积 h。面积为 (r-l)h/d，底边两端为 p+el/d、p+er/d，上边统一加 perp(e)h/d。不要求首角纵坐标最小。输入必须是严格逆时针凸包；任意实数判向和极薄模型不能由通过有界整数题保证。

模板题使用：P3187 最小矩形覆盖

题意：3≤n≤50000，坐标 [0,10]、至多五位小数。乘 10^5 后四舍五入还原整数，integer_hull 精确去重与判向，再将凸包转 long double 作卡壳。输出面积及逆时针四角，10 位小数；面积误差/max(1,S)<1e-4、点到矩形距离<1e-4。已 100 分 AC，记录 302662813；最慢 26ms，总计 113ms。

所需模板：IntegerPlane（第 1000 页，58.8 节）、integer_hull（第 1071 页，62.2 节）、RealPlane（第 987 页，58.1 节）、minimum_rectangle（第 1086 页，64.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3187

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<IntegerPlane::Point> points(n);
    for (auto &p : points)
    {
        long double x, y;
        cin >> x >> y;
        p = {llroundl(x * 100000), llroundl(y * 100000)};
    }
    auto hull = integer_hull(points);
    vector<RealPlane::Point> p;
    for (auto q : hull)
        p.push_back({(long double)q.x, (long double)q.y});
    auto [area, box] = minimum_rectangle(p);
    cout << fixed << setprecision(10) << area / 10000000000.L << '\n';
    for (auto q : box)
        cout << q.x / 100000 << ' ' << q.y / 100000 << '\n';
}
```

第六十五章　半平面交

65.1　整数半平面的有界多边形交

bounded_polygon(lines) 求 a*x+b*y≤c 的交集闭包顶点，Line={a,b,c}，三个整数系数绝对值均 $\leq 10^9$。要求交集有界，空集也允许；只返回正面积多边形，点或线段交集返回空向量。无界输入不在接口范围内。Point={x,y,d} 表示 (x/d,y/d)，d>0，分数不必约分；按逆时针返回，不重复首点，相邻重复点已去除。排序与所有构造、判定使用整数和 int128，不使用 epsilon。O(n log n) 时间、O(n) 空间。浮点转换与面积计算需单独处理。

```
struct IntegerHalfplanes
{
    using I = __int128_t;
    using ll = long long;

    struct Line
    {
        ll a, b, c;
    };

    struct Point
    {
        ll x, y, d;
    };

    static I cross(Line u, Line v) { return I(u.a) * v.b - I(u.b) * v.a; }

    static int half(Line u) { return u.b < 0 || (u.b == 0 && u.a < 0); }

    static Point meet(Line u, Line v)
    {
        I d = cross(u, v);
        assert(d != 0);
        I x = I(u.c) * v.b - I(u.b) * v.c;
        I y = I(u.a) * v.c - I(u.c) * v.a;
        if (d < 0)
        {
            d = -d;
            x = -x;
            y = -y;
        }
        return {(ll)x, (ll)y, (ll)d};
    }
```

接上页同一结构体与函数:

```
35      static I value(Line u, Point p)
36      {
37          return I(u.a) * p.x + I(u.b) * p.y - I(u.c) * p.d;
38      }
39
40      static bool same(Point p, Point q)
41      {
42          return I(p.x) * q.d == I(q.x) * p.d && I(p.y) * q.d == I(q.y) * p.d;
43      }
44
45      static vector<Point> bounded_polygon(vector<Line> lines)
46      {
47          vector<Line> h;
48          for (auto u : lines)
49          {
50              assert(-1000000000 <= u.a && u.a <= 1000000000);
51              assert(-1000000000 <= u.b && u.b <= 1000000000);
52              assert(-1000000000 <= u.c && u.c <= 1000000000);
53              if (!u.a && !u.b)
54              {
55                  if (u.c < 0) return {};
56              }
57              else
58                  h.push_back(u);
59          }
60          sort(h.begin(),
61               h.end(),
62               [](Line u, Line v)
63               {
64                   if (half(u) != half(v)) return half(u) < half(v);
65                   I d = cross(u, v);
66                   if (d) return d > 0;
67                   ll x = u.a ? (u.a > 0 ? u.a : -u.a) : (u.b > 0 ? u.b : -u.b);
68                   ll y = v.a ? (v.a > 0 ? v.a : -v.a) : (v.b > 0 ? v.b : -v.b);
69                   return I(u.c) * y < I(v.c) * x;
70               });
```

接上页同一结构体与函数:

```
        deque<Line> q;
        for (int i = 0; i < (int)h.size(); i++)
        {
            Line u = h[i];
            if (i && half(u) == half(h[i - 1]) && cross(u, h[i - 1]) == 0) continue;
            while (q.size() >= 2 && value(u, meet(q[q.size() - 2], q.back())) > 0)
                q.pop_back();
            while (q.size() >= 2 && value(u, meet(q[0], q[1])) > 0) q.pop_front();
            if (!q.empty() && cross(q.back(), u) == 0) return {};
            q.push_back(u);
        }
        while (q.size() >= 3 && value(q.front(), meet(q[q.size() - 2], q.back())) > 0)
            q.pop_back();
        while (q.size() >= 3 && value(q.back(), meet(q[0], q[1])) > 0) q.pop_front();
        if (q.size() < 3 || cross(q.front(), q.back()) == 0) return {};
        vector<Point> answer;
        for (int i = 0; i < (int)q.size(); i++)
        {
            Point p = meet(q[i], q[(i + 1) % q.size()]);
            if (answer.empty() || !same(answer.back(), p)) answer.push_back(p);
        }
        if (answer.size() > 1 && same(answer.front(), answer.back())) answer.pop_back();
        if (answer.size() < 3) return {};
        return answer;
    }
};
```

应用题补充：Luogu P4196

题意：逆时针输入 2..10 个凸多边形，每个 3..50 个顶点、整数坐标绝对值 ≤1000，求交面积。每条有向边的左侧转为 a*x+b*y≤c，系数满足模板范围；交集必有界，空集或退化交集面积为 0。交点与面积各项分子分母先用整数，最后转浮点累加；CQOI 来源作为应用补充。

所需模板：IntegerHalfplanes（第 1089 页，65.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4196

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using H = IntegerHalfplanes;
    int n;
    cin >> n;
    vector<H::Line> lines;
    while (n--)
    {
        int m;
        cin >> m;
        vector<pair<long long, long long>> p(m);
        for (auto &[x, y] : p) cin >> x >> y;
        for (int i = 0; i < m; i++)
        {
            auto [x, y] = p[i];
            auto [u, v] = p[(i + 1) % m];
            long long a = v - y, b = x - u;
            lines.push_back({a, b, a * x + b * y});
        }
    }
    auto p = H::bounded_polygon(move(lines));
    long double area2 = 0;
    for (int i = 0; i < (int)p.size(); i++)
    {
        auto u = p[i], v = p[(i + 1) % p.size()];
        H::I numerator = H::I(u.x) * v.y - H::I(u.y) * v.x;
        H::I denominator = H::I(u.d) * v.d;
        area2 += (long double)numerator / (long double)denominator;
    }
    cout << fixed << setprecision(3) << fabsl(area2) / 2 << '\n';
}
```

第六十六章　平面最近点对

66.1　整数最近点对（精确平方距离）

代码依赖：IntegerPlane（第 1000 页，58.8 节）。抄写时一并准备；标准库类型不列入算法依赖。

closest_pair_i64(points, endpoints=nullptr) 返回 optional<__int128_t> 的最短距离平方；传 &ids 时同时返回一对原输入的 0-based 下标，最优解不唯一时任选。少于两点返回 nullopt 且 ids={-1,-1}，重复点返回 0 和两个不同下标。坐标绝对值不超过 10^{12}。递归分治内部排序下标，不改变输入；返回时子区间按纵坐标有序。O(n log n) 时间、O(n) 辅助空间，全程精确整数。distance 更新全局端点，solve 返回局部最短平方距离；依赖 IntegerPlane。

```
optional<IntegerPlane::I> closest_pair_i64(const vector<IntegerPlane::Point> &p,
                                           pair<int, int> *endpoints = nullptr)
{
    using G = IntegerPlane;
    using I = G::I;
    int n = p.size();
    if (endpoints) *endpoints = {-1, -1};
    if (n < 2) return nullopt;
    vector<int> id(n), tmp(n);
    iota(id.begin(), id.end(), 0);
    sort(id.begin(), id.end(), [&](int a, int b) { return p[a] < p[b]; });
    for (int i = 1; i < n; i++)
        if (p[id[i]] == p[id[i - 1]])
        {
            if (endpoints) *endpoints = {id[i - 1], id[i]};
            return I(0);
        }
    I ans = I(1) << 120;
    auto distance = [&](int a, int b)
    {
        I d = G::dist2(p[a], p[b]);
        if (d < ans)
        {
            ans = d;
            if (endpoints) *endpoints = {a, b};
        }
        return d;
    };
    auto by_y = [&](int a, int b)
    {
        return tie(p[a].y, p[a].x) < tie(p[b].y, p[b].x);
    };
```

递归分治（接上页同一函数）：

```
    function<I(int, int)> solve = [&](int l, int r) -> I
    {
        if (r - l <= 3)
        {
            I best = I(1) << 120;
            for (int i = l; i < r; i++)
                for (int j = i + 1; j < r; j++)
                    best = min(best, distance(id[i], id[j]));
            sort(id.begin() + l, id.begin() + r, by_y);
            return best;
        }
        int m = (l + r) / 2;
        long long x = p[id[m]].x;
        I best = min(solve(l, m), solve(m, r));
        merge(id.begin() + l,
              id.begin() + m,
              id.begin() + m,
              id.begin() + r,
              tmp.begin() + l,
              by_y);
        copy(tmp.begin() + l, tmp.begin() + r, id.begin() + l);
        vector<int> strip;
        for (int i = l; i < r; i++)
            if ((I(p[id[i]].x) - x) * (I(p[id[i]].x) - x) < best)
            {
                for (int j = (int)strip.size() - 1; j >= 0; j--)
                {
                    I dy = I(p[id[i]].y) - p[strip[j]].y;
                    if (dy * dy >= best) break;
                    best = min(best, distance(id[i], strip[j]));
                }
                strip.push_back(id[i]);
            }
        return best;
    };
    return solve(0, n);
}
```

模板题使用：Luogu P7883

题意：求整数点集最近距离的平方，全程不取平方根。题目保证至少两点，坐标绝对值不超过 10^7，平方距离不超过 8×10^14，可将 int128 转为 long long；模板完整范围内不能无条件照搬转换。

所需模板：IntegerPlane（第 1000 页，58.8 节）、closest_pair_i64（第 1093 页，66.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P7883

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<IntegerPlane::Point> points(n);
    for (auto &p : points) cin >> p.x >> p.y;
    auto answer = closest_pair_i64(points);
    cout << (long long)*answer << '\n';
}
```

模板题使用：Library Checker closest_pair

题意：多组整数点集，输出一对不同的原输入 0-based 下标，使欧氏距离最小；最优解任选，重复坐标仍须返回不同下标。每组至少 2 点，总点数至 50 万，坐标绝对值至 10^9。直接传入端点指针，无需开方或重建坐标到编号的映射。

所需模板：IntegerPlane（第 1000 页，58.8 节）、closest_pair_i64（第 1093 页，66.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/closest_pair

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        vector<IntegerPlane::Point> p(n);
        for (auto &v : p) cin >> v.x >> v.y;
        pair<int, int> ids;
        closest_pair_i64(p, &ids);
        cout << ids.first << ' ' << ids.second << '\n';
    }
}
```

66.2 浮点平面最近点对

solve(points) 返回 long double 最短距离（不是平方），少于两个点返回 nullopt，重复点返回零。Point={x,y}；输入须有限，坐标绝对值 ≤1e100。递归分治，按纵坐标归并，O(n log n) 时间、O(n) 额外空间。用 hypotl 求距离，不先平方；排序不使用 epsilon。浮点运算仍有舍入误差，输入前已丢失的精度无法恢复。

```
struct ClosestPair
{
    using R = long double;

    struct Point
    {
        R x, y;
    };

    // Finite coordinates, |x|, |y| <= 1e100. Returns distance, not its square.
    static optional<R> solve(vector<Point> p)
    {
        int n = p.size();
        if (n < 2) return nullopt;
        sort(p.begin(),
             p.end(),
             [](Point a, Point b) { return tie(a.x, a.y) < tie(b.x, b.y); });
        for (int i = 1; i < n; i++)
            if (p[i].x == p[i - 1].x && p[i].y == p[i - 1].y) return R(0);
        auto by_y = [](Point a, Point b)
        {
            return tie(a.y, a.x) < tie(b.y, b.x);
        };
        auto distance = [](Point a, Point b)
        {
            return hypotl(a.x - b.x, a.y - b.y);
        };
        vector<Point> tmp(n);
        function<R(int, int)> dfs = [&](int l, int r) -> R
        {
            if (r - l <= 3)
            {
                R best = numeric_limits<R>::infinity();
                for (int i = l; i < r; i++)
                    for (int j = l; j < i; j++) best = min(best, distance(p[i], p[j]));
                sort(p.begin() + l, p.begin() + r, by_y);
                return best;
            }
```

分治、归并与跨分界线点对（接上页同一函数）：

```
            int m = (l + r) / 2;
            R x = p[m].x;
            R best = min(dfs(l, m), dfs(m, r));
            merge(p.begin() + l,
                  p.begin() + m,
                  p.begin() + m,
                  p.begin() + r,
                  tmp.begin() + l,
                  by_y);
            copy(tmp.begin() + l, tmp.begin() + r, p.begin() + l);
            vector<Point> strip;
            for (int i = l; i < r; i++)
                if (fabsl(p[i].x - x) <= best)
                {
                    for (int j = (int)strip.size() - 1; j >= 0; j--)
                    {
                        if (p[i].y - strip[j].y > best) break;
                        best = min(best, distance(p[i], strip[j]));
                    }
                    strip.push_back(p[i]);
                }
            return best;
        };
        return dfs(0, n);
    }
};
```

模板题使用：Luogu P1429

题意：求平面最近两点的距离，输出小数点后四位。solve 返回距离而非平方；题目保证至少两点，因此可解引用 optional。坐标按 long double 读取。

所需模板：ClosestPair（第 1096 页，66.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1429

```
int main()
{
    int n;
    scanf("%d", &n);
    vector<ClosestPair::Point> p(n);
    for (auto &v : p) scanf("%Lf%Lf", &v.x, &v.y);
    printf("%.4Lf\n", *ClosestPair::solve(p));
}
```

第六十七章　随机增量法

67.1　随机增量最小覆盖圆

代码依赖：RealPlane（第 987 页，58.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

solve(points,rng,eps) 返回 optional<Circle>，Circle={o,r}；空集返回 nullopt，单点或重复同一点返回半径零。rng 是调用者提供的 mt19937_64 随机数引擎。期望 O(n) 时间，最坏 $O(n^3)$，O(n) 空间；随机化不可省略。eps 默认 32 倍 long double 机器精度，用于相对半径的包含判断。diameter(a,b) 求直径圆；through(a,b,c) 求外接圆，共线时返回最远点对的直径圆。依赖浮点几何的 Point/Circle 与向量运算。输入及中间运算须有限且尺度适当；接近共线时圆心可能不稳定，无法恢复读入前的精度。

```
struct EnclosingCircle
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Circle = G::Circle;

    static Circle diameter(Point a, Point b)
    {
        Point o = a + (b - a) / 2;
        return {o, max(G::norm(o - a), G::norm(o - b))};
    }

    static Circle through(Point a, Point b, Point c)
    {
        Point u = b - a, v = c - a;
        R scale = max({fabsl(u.x), fabsl(u.y), fabsl(v.x), fabsl(v.y)});
        if (scale == 0) return {a, 0};
        u = u / scale;
        v = v / scale;
        R det = 2 * G::cross(u, v);
        if (det == 0)
        {
            Circle best = diameter(a, b);
            for (Circle q : {diameter(a, c), diameter(b, c)})
                if (q.r > best.r) best = q;
            return best;
        }
        R x = G::dot(u, u), y = G::dot(v, v);
        Point offset{(x * v.y - y * u.y) / det, (u.x * y - v.x * x) / det};
        Point o = a + offset * scale;
        return {o, max({G::norm(o - a), G::norm(o - b), G::norm(o - c)})};
    }
```

随机增量主过程（接上页同一结构体）：

```
    static optional<Circle>
    solve(vector<Point> p, mt19937_64 &rng, R eps = 32 * numeric_limits<R>::epsilon())
    {
        assert(eps >= 0 && eps < 1);
        if (p.empty()) return nullopt;
        shuffle(p.begin(), p.end(), rng);
        Circle c{p[0], 0};
        auto outside = [&](Point q)
        {
            return G::norm(q - c.o) > c.r * (1 + eps);
        };
        for (int i = 1; i < (int)p.size(); i++)
            if (outside(p[i]))
            {
                c = {p[i], 0};
                for (int j = 0; j < i; j++)
                    if (outside(p[j]))
                    {
                        c = diameter(p[i], p[j]);
                        for (int k = 0; k < j; k++)
                            if (outside(p[k])) c = through(p[i], p[j], p[k]);
                    }
            }
        return c;
    }
};
```

模板题使用：Luogu P1742

题意：求包含全部点的最小圆，先输出半径，再输出圆心坐标。题目保证非空，因此可解引用返回值；显式传随机引擎，不能省略随机打乱。用 long double 读取并输出十二位小数。

所需模板：RealPlane（第 987 页，58.1 节）、EnclosingCircle（第 1098 页，67.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1742

```
int main()
{
    int n;
    scanf("%d", &n);
    vector<EnclosingCircle::Point> p(n);
    for (auto &v : p) scanf("%Lf%Lf", &v.x, &v.y);
    mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    auto c = *EnclosingCircle::solve(p, rng);
    printf("%.12Lf\n%.12Lf %.12Lf\n", c.r, c.o.x, c.o.y);
}
```

第六十八章　计算几何：精确判定与适用范围

68.1　定点凸包、直径和极角排序

P2742 坐标最多两位小数，可按字符串读成放大 100 倍的整数，调用整数凸包；不要先读浮点再乘 100 截断。放大后坐标绝对值至多 10^8，满足整数几何范围。求周长时才对精确距离平方开方、最后除以 100；共线时取两端距离的两倍，全部重合为零。判定精确不等于最后的开方和小数输出没有舍入误差。

旋转卡壳输入须为严格逆时针凸包，不重复首点、不含共线边内点。用整数叉积比较三角形面积，并只在面积严格增大时推进对踵点；空集、单点和线段先单独处理。返回值是 128 位距离平方，不必开方再平方。P1452 的坐标绝对值至多 10^4，答案至多 8×10^8，其用法中可转 long long 输出；不能把这一转换推广到模板允许的万亿坐标。端点参数返回凸包下标，映射回原始编号见正式最远点对使用示例。

整数极角比较器从正 x 轴按 $[0,2\pi)$ 排序，同向按距离递增，零向量排最前。先分半圆，再比较精确叉积，最后比较长度；坐标轴归属必须明确。仅用 $y>0$ 分组会使 $(-1,0)<(0,-1)<(1,0)$ 却不满足首尾比较，破坏传递性。把 epsilon 相等用于排序也不满足严格弱序；不要把近似相等当作 sort 的等价关系。Library Checker 要求 $(-\pi,\pi]$，先按库排序，再把 $y<0$ 后缀循环移到最前；负 x 轴是 π，仍留在最后。原点约定角度为零。

凸包与卡壳用任意精度礼品包装、全点对距离作独立参考，含全部 3×3 子集、万亿坐标近共线和每个循环起点；极角比较器枚举 81 点的全部有序三元组，并与百位 atan2 逐对比较。三个应用驱动及极角正式题用法均双模式验证；极角排序全部 21 组官方本地数据双模式 checker 接受，不代表在线 AC。任意浮点输入的稳健凸包及极角谓词仍需单独补齐，不能由这些整数测试推断完成。

68.2 整数系数的有界半平面交

实现见第 1089 页。输入是不等式 $ax+by\le c$，三个系数均为绝对值不超过 10^9 的整数。本接口要求交集有界，只输出正面积多边形；无解、点交集和线段交集都返回空向量。它不是一般的可行性分类接口，不可将其空返回直接解读为闭半平面交集必为空。无界输入不在约定范围内，也不偷偷添加一个人工边框。

两条非平行边界交点用齐次坐标 (X,Y,D) 表示：

$$D=a_1b_2-b_1a_2,\quad X=c_1b_2-b_1c_2,\quad Y=a_1c_2-c_1a_2.$$

若 $D<0$，三者同时取负，使 $D>0$。测试 $aX+bY\le cD$ 即可判断交点位于半平面内，无需除法。系数界保证 $|X|,|Y|,D\le 2\times10^{18}$，可用 64 位保存；相等判断的交叉乘积至多为 4×10^{36}，使用 128 位。因此本接口内的构造、排序和位置判定都是精确整数运算。

按法向量的半圆与叉积排序，比较器不使用 epsilon。同方向平行约束按正规化后的常数项保留更紧者，避免除以法向量长度。扫描时若新约束排除队首或队尾交点，就删除相应无效边界；每条边界最多入队、出队一次，排序后为线性时间。最后闭合队列，清除重复顶点；有界的非正面积结果不会输出多边形。零法向量表示常真约束或直接矛盾，先单独处理。

测试枚举所有边界对的交点，用任意精度整数逐条验证可行性，将可行顶点集合与返回集合比较，并检查循环顺序的严格凸性。当前 vector 版本通过 5000 组有界随机输入、平行和退化案例、近乎平行的十亿系数，以及十万条冗余约束的规模测试；普通与 ASan/UBSan 均通过。P4196 的逆时针边 $(x,y)\to(u,v)$ 转换为 $a=v-y,b=x-u,c=ax+by$，则边的左侧就是 $ax'+by'\le c$。题目整数坐标绝对值至多 1000，故 $|a|,|b|\le2000$、$|c|\le2\times10^6$，满足模板范围。交集必有界；返回为空时，无论空集或零面积退化，所求面积都是零。鞋带公式每项分子 $X_iY_j-Y_iX_j$、分母 D_iD_j 先以 128 位求值，再转浮点相除累加。最多 500 个交点且坐标绝对值不超过 1000，满足本题输出精度。完整驱动另经 207 组精确有理数逐边裁剪对拍，包含近共线、点/线段交和 500 条边界。这不保证一般大坐标有理多边形面积的统一相对误差，浮点输入也未由此验证。WIDA 的一般半平面交条目保留部分完成状态，浮点版本与无界分类仍需补齐。

68.3 最近点对、Minkowski 和与三维整数判定

整数最近点对见第 1093 页；Minkowski 和见第 1075 页。最近点对返回距离平方，不开方；少于两个点时返回空 optional，重复点返回零。分治使用递归，每层按纵坐标归并，总时间 $O(n \log n)$，额外空间 $O(n)$。坐标绝对值上限为 10^{12}，作差后平方使用 128 位。

Minkowski 和输入必须是严格逆时针凸包，不重复首点，不含边上的冗余共线点；允许循环移动起点，也允许空集、单点和线段。输入坐标以及任意两个输入点的坐标和都须在 $[-10^{12}, 10^{12}]$ 内。非退化凸包按边方向归并，时间 $O(n+m)$；退化分支枚举点和后求凸包，时间 $O((n+m)\log(n+m))$。返回同样不重复首点的严格凸包。

两套代码通过任意精度整数距离枚举，以及对所有点和独立执行礼品包装法的对拍；覆盖退化、循环起点和大坐标。规模测试包含二十万点最近点对，以及 40001 顶点的抛物线凸包与自身求和，结果应为原凸包各坐标的两倍。这些属于本地验证，不表示新增在线 AC。

三维判定见第 1036 页。点的三个坐标为 `x,y,z`，绝对值均不超过 10^9。`orient(a,b,c,d)` 返回有向六倍四面体体积，零表示四点共面，交换任意两个点会改变符号。`collinear` 判断三点共线，`on_segment` 包含端点，且支持零长度线段。传统版线段接口名为 `On_Segment`。

点差每个分量至多 2×10^9，叉积分量至多 8×10^{18}，三重积绝对值至多 4.8×10^{28}，在有符号 128 位范围内。公开的叉积、点积辅助函数不对任意 128 位向量保证不溢出；上述保证针对点差及其叉积。任意精度整数按 4×4 行列式的 24 个排列独立计算方向，并用二阶子式及坐标区间验证共线与在线段上。已通过立方体八个顶点的全部有序四元组、随机十亿级坐标、换序及退化测试，普通与 ASan/UBSan 均通过。这里不包含三维凸包构造。

68.4 浮点最近点对

实现见第 1096 页。`ClosestPair::solve` 与传统版 `Closest_Pair::Solve` 接收 `Point{x,y}` 数组，返回最短距离，不是距离平方。少于两点返回空 optional；重复点返回零。输入为有限 `long double`，坐标绝对值不超过 10^{100}。

递归前按横坐标排序，每次在递归改变数组前保存分界横坐标。左右区间分别递归后按纵坐标归并，再检查分界线附近的点对。时间 $O(n \log n)$，额外空间 $O(n)$。直接使用 `hypotl`，避免先平方造成上溢或下溢。排序比较器不使用 epsilon；不能把相近坐标随意视为相同点。

浮点结果仍有舍入误差，也无法恢复读入之前已经丢失的精度。整数坐标要求精确判定时，优先使用整数最近点对，最后按题目要求开方输出。一百位十进制浮点枚举所有点对，验证两套实现，覆盖小数、相邻可表示浮点数、极小非正规数、10^{-100} 至 10^{100} 的尺度，以及二十万点规模；普通与 ASan/UBSan 均通过。本地完整 P1429 驱动测试包含 103 组输入和四位小数输出检查。在线评测的整数输入范围不能替代这里对小数与极端尺度的检查。

两套实现已在 P1429 通过全部 11 个测试点：动态版记录 297602650、传统版记录 297603238。单点最大用时均为 75 ms，内存峰值均为 12.32 MB。

68.5 公切线与过点切线

实现见第 1011 页，依赖浮点几何的点、圆和向量运算。令两圆心差为 v，$d=|v|$，$u=v/d$。外公切线取 $s=1$，内公切线取 $s=-1$，单位法向量满足

$$n\cdot u=\frac{r_1-sr_2}{d}=c,\qquad n=cu\pm\sqrt{(1-c)(1+c)}\,u^{\perp}.$$

切点为 $p=o_1+r_1n$、$q=o_2+sr_2n$。直线以 $n\cdot(x-p)=0$ 表示，避免在两圆相切、两个切点相同时丢失方向。结果中的 `side` 为 1 或 -1，表示外公切线或内公切线。每个非相切分支有两条线，相切分支只输出一次。

半径零解释为点。有一个点圆时，只枚举外公切线分支，避免内外分支重复；两个不同点直接返回连线，`side=0`。严格同心且等半径返回 `infinite=true`，包括重合的两个点圆；严格同心但半径不同没有公切线。`from_point(p,c)` 把第一圆设为点圆，圆上的切点读返回的 `b`。点在圆上时返回一条，点在圆内时为空；点与零半径圆重合时返回无限条过点直线。

输入须有限，坐标和半径绝对值不超过 10^{100}，半径非负。容差 $0<\varepsilon<1$ 默认 10^{-12}：当 $|r_1-sr_2|$ 与 d 的相对差在容差内时，合并为相切。因此极接近相切的输入可能返回一条近似切线，不承诺恢复舍入前的几何分类；使用方程时宜计算 $n\cdot(x-p)$，避免先计算大常数项造成额外消减。先比较半径差和圆心距，再做除法，避免极小圆心距造成比值溢出。

计数对拍使用任意精度整数比较圆心距平方与半径和、差的平方；一百位十进制运算验证切点到圆心的距离、圆心到切线的距离、法向量和切线一致性。两套实现通过 3000 组随机圆，在三个尺度下交换顺序测试，并覆盖重合、内外相切、点圆、极近同心与容差边界。普通与 ASan/UBSan 均通过；尚无此接口的在线 AC 记录。

68.6 随机增量最小覆盖圆

实现见第 1098 页。随机打乱点集后依次加入。新点在当前圆外时，它必须成为新覆盖圆的边界点；以该点为固定边界重扫前缀，再遇到圆外点就固定两个边界点，最后由第三个圆外点确定外接圆。随机化使期望时间为 $O(n)$，最坏时间仍为 $O(n^3)$。传入初始化好的 `mt19937_64`，不可省略打乱。本实现使用三层循环，没有递归栈需求。

空集返回空 optional，单点及重复同一点的半径为零。直径圆用两端点到计算圆心的较大距离作为半径。三点外接圆先把第一点平移到原点，再按最大坐标差缩放后解两条垂直平分线方程；共线时使用最远点对的直径圆。`through` 对非共线三点求外接圆，不是任意三点的最小覆盖圆；例如钝角三角形的最小覆盖圆可能仅由最长边支撑，须调用主过程。

包含判断使用相对半径容差，默认 32 倍机器精度。输入与中间运算须有限、尺度适当；近共线三点的外接圆仍可能数值敏感，浮点比较不能恢复读入之前的精度。P1742 的驱动已准备，输出半径与圆心，该题坐标绝对值不超过 10^4，最多两位小数。

本地参考答案枚举所有单点、点对直径圆和三点外接圆，用一百位十进制运算筛出覆盖全部点且半径最小者，检查圆心、半径及覆盖性。两套版本通过 700 组随机小数点集，每组使用 12 个随机种子，另测共线、近共线、钝角/锐角三角形、平移后的小三角形、十万点直线与重复点。普通与 ASan/UBSan 均通过。两套代码已在 P1742 通过全部 10 个测试点：动态版记录 297608403、传统版记录 297608801。单点最大用时分别为 52 ms、49 ms，内存峰值为 6.50 MB、6.49 MB。该评测验证半径与圆心，不覆盖一并展开但未调用的其他几何接口。随机增量算法可参考 Project Nayuki 的算法说明。

68.7 圆与简单多边形相交面积

实现见第 1009 页。先把圆心移到原点，再沿多边形有向边累加贡献，最后取绝对值。边与圆的交点把边分成至多三段。若小段中点在圆内，贡献是 $\operatorname{cross}(p,q)/2$；否则贡献是有向扇形面积

$$\frac{r^2}{2}\operatorname{atan2}\big(\operatorname{cross}(p,q),p\cdot q\big).$$

必须保留符号，不能先对每条边取绝对值，否则凹多边形和圆外的边无法抵消。使用补偿求和减小大量边累加时的误差，总时间 $O(n)$、额外空间 $O(1)$。

求交点时令边的单位方向为 u，有向距离 $h=\operatorname{cross}(u,a)$，垂足直接构造为 $hu^{\perp}$，交点为

$$hu^{\perp}\pm u\sqrt{(r-|h|)(r+|h|)}.$$

这避免用远端点加很长的位移来重建圆附近的交点。沿边的投影参数用于筛选位于线段内部的交点；端点本身已在分段列表内。相切交点重复不改变面积，零长度边返回零。

多边形须简单，支持凹形、两种方向和相邻重复顶点；半径非负，零半径或少于三个顶点时面积为零。输入与中间运算须有限且尺度适当，特别是平移后的乘积和面积要可表示。极小相交面积可能受到扇形贡献相消和近相切计算的影响，不作相对误差的统一保证。

测试包括完整圆、半圆、四分之一圆、内含多边形、相离、相切和高精度圆缺面积。独立参考实现沿竖直方向求多边形截线区间，再与圆的截线相交并数值积分，检查 150 组含凹形的随机多边形，另检查方向反转、重复点和平移。规模与数值测试含十万条边、极长的过圆心边和不同尺度。两套实现普通与 ASan/UBSan 均通过；尚无此模块的在线 AC。对应 kuangbin 二维几何中的圆与三角形、多边形相交面积部分。

68.8 最近点对编号与多边形闭集查询

最近点对可返回原输入的两个不同下标，重复点距离为零。不足两点返回空值，端点重置为 $(-1,-1)$。内部排序下标，不改变输入序列。分治递归前保存中间横坐标，递归后各段按纵坐标排列，再归并和扫描窄带。局部距离用于剪枝，全局更新保存端点；同距最优解任选。坐标至 10^{12} 时距离平方可达 8×10^{24}，需用 int128；P7883 的坐标至 10^7 才允许在用法中转 long long。Library Checker closest_pair 的原编号输出见完整用法。

一般简单多边形直接保留输入边界调用 polygon_contains，不把凹形先变成凸包。返回 0/1/2 分别表示外部、边界、内部。先判在线段上，再用半开纵坐标区间累计绕数；边界反向不改变结果。凸包二分查询要求严格逆时针、无共线边内点、不重复首点，允许旋转起点；扇形内部对角线不等于多边形边界。

P4557 中两组点的凸包记为 A,B，第二组平移 d 后相交，当且仅当 $d\in A+(-B)$。先对第二组坐标取负，再求两凸包的 Minkowski 和，对原平移向量做闭集查询。不要把查询也取负；边界接触输出 1。各组点数及查询数至 10^5，结果坐标至 2×10^8；复杂度为 $O(n\log n+m\log m+q\log(n+m))$。这是比赛应用，不替代两个组件的正式非比赛模板题。

独立检查使用整数斜射线奇偶、凸包直接相交、全点对距离枚举；最近点对另通过 29 组官方数据的双模式本地 checker。本地接受记录与线上 AC、速度排名分开保留。

68.9 整数圆交点：精确判定与浮点构造

输入整数坐标绝对值、半径均不超过 10^9，半径可以为零。直线方向 $v=b-a$，令 $q=v\cdot v$、$c=v\times(a-o)$，判别式为 $D=r^2q-c^2$。$q=0$ 是退化直线；否则按 D 的符号精确分类。垂足 $h=o+\mathrm{perp}(v)c/q$，交点为 $h\pm v\sqrt{D}/q$。两圆则令 $v=b-a$、$q=v\cdot v$、$k=q+r^2-s^2$，用 $D=4r^2q-k^2$ 分类，交点为 $a+vk/(2q)\pm\mathrm{perp}(v)\sqrt{D}/(2q)$。$q=0$ 单独处理同心圆；相同正半径有无穷交点，相同零半径仅有一点。

这些整数中间值均在有符号 int128 范围内：$q\le 8\times10^{18}$，$|c|\le 8\times10^{18}$，$|k|\le 9\times10^{18}$；$c^2\le 6.4\times10^{37}$、$k^2\le 8.1\times10^{37}$，$4r^2q\le 3.2\times10^{37}$。差的两项非负，差值也安全。必须先提升到 int128 再做减法、乘法；扩大输入范围需重新核算。

精确分类不意味着坐标无舍入误差。AOJ CGL_7_D/E 的输入尺度仅为 10^4，本地按严格绝对 10^{-6} 检查；不能将此误差承诺扩展到 10^9。相切时核心只返回一个点，题目用法复制一次；按精确字典序排序浮点结果，不能用 eps 比较器。原浮点容差版仍用于明确接受近似分类的场景：例如圆心 $(-7717,-10000)$、半径 9758，直线经过 $(-10000,-367),(-5909,-108)$，整数判别式为 2168，两交点相距约 0.0227，旧容差却会归为相切，因此不能直接用于该题。

CGL_7_H 的多边形允许凹形，直接调用圆与多边形面积；CGL_7_I 包含同心、内含与相离圆，直接调用两圆交面积。本轮完整用法用 80 位精度参考核对，H 采用独立竖切片解析积分，并检查完整圆、半圆、四分之一圆锚点。局部验证、线上 AC 与速度排名分别记录。

68.10 公切线：切点次序也需要精确判定

CircleTangents 的 line.a、line.b 分别是第一、第二圆的切点。from_point 把给定点视为第一圆，因此 CGL_7_F 输出 line.b。该题整数范围为 10^3，点严格在圆外；非零的 d^2-r^2 至少为 1，故 $d-r\ge 1/(d+r)>2\times10^{-4}$，远大于默认相对容差所对应的距离。

CGL_7_G 则需按第一圆切点的横坐标、纵坐标输出。圆 $(0,0,3)$ 与 $(5,5,4)$ 有两个切点 $(1.8,-2.4),(1.8,2.4)$，浮点运算可能分别产生 1.7999999999999998 与 1.7999999999999985，直接排序会错排。IntegerTangents 精确决定输出次序，构造坐标后不再排序。

设圆心差为 $v=(x,y)$，$q=x^2+y^2$，外、内公切线分别取 $g=r-s$ 与 $g=r+s$，$D=q-g^2$；$D<0$ 无此类线，$D=0$ 一条，否则两条。单位法向为 $n=(vg\pm\mathrm{perp}(v)\sqrt{D})/q$。第一圆半径为正时，比较切点等价于比较法向的横、纵分量。每次比较化为 $A+B\sqrt{C}+E\sqrt{F}$ 的符号：先判断 $u=A+B\sqrt{C}$ 和 $w=E\sqrt{F}$，同号可直接决定；异号时比较 $u^2-w^2=(A^2+B^2C-E^2F)+2AB\sqrt{C}$，再乘 $\mathrm{sgn}(u)$。单根式也先判符号，再平方比较大小，避免丢失符号和伪等号。

输入坐标绝对值、半径至 10^4 时，$|A|\le 4\times10^8$，$|B|,|E|\le 2\times10^4$，$C,F\le 8\times10^8$。第二次比较的两项平方分别不超过 2.304×10^{35} 与 2.048×10^{35}，有符号 int128 可容纳；不能直接沿用圆交点函数的十亿范围。这是严格的根式比较器，没有对浮点横坐标作 eps 比较或取整分组。零半径、同心、相切与空结果沿用几何意义；法向仍可定义相切时的唯一切线，不要用两个重合切点计算方向。输入超范围或非整数时，应另行选择数值合同。

68.11 圆交点：容差分类与坐标误差

默认浮点接口按相对容差处理近切情况，不能从“距离误差小”推出“交点坐标误差小”。接近相切时，半弦长包含平方根，归并成一个点可能放大坐标误差。直线–圆与圆–圆函数现可显式传 `eps=0` 使用无容差分支，默认正容差分支不变；这不等于任意实数上的精确几何。实现及完整用法分别见第 994、996、995、998 页。

避免先开方再相减。 直线令 $v=b-a$、$q=v\cdot v$、$c=v\times(a-o)$，计算 $\Delta=r^2q-c^2$。若 $q=0$ 则直线退化；否则 Δ 的负、零、正分别对应无交点、相切和两个交点。垂足 $h=o+\mathrm{perp}(v)c/q$，偏移 $w=v\sqrt{\Delta}/q$，返回 $h\pm w$。两圆令 $v=b.o-a.o$、$q=v\cdot v$、$k=q+r^2-s^2$，计算 $\Delta=4r^2q-k^2$；同心情形先单独处理。此时 $h=a.o+vk/(2q)$、$w=\mathrm{perp}(v)\sqrt{\Delta}/(2q)$。这些公式没有复用整数接口，也没有把默认 eps 简单调小。

AOJ 整数输入域的额外保证。 CGL_7_D/E 给出的坐标为绝对值不超过 $B=10^4$ 的整数，半径为 $1..B$ 的整数。$q\le 8B^2$、$|c|\le 8B^2$、$|k|\le 9B^2$，上述每一步整数运算的绝对值均小于 $81B^4<2^{60}$。若 long double 为二进制且有效尾数至少 64 位，采用正常最近舍入，不启用 fast-math、危险重结合或改变精度的环境，这些中间整数和判别式符号均精确。53 位尾数不够：$9805^2\cdot 234901757-150276382^2=1$，但直接 53 位浮点计算可得到零。整数十亿坐标接口的范围不能挪给本结论；一般非整数输入也不继承它。

构造与排序。 设 $u=2^{-64}$。有交点时 $h-$ 圆心 与 w 的每个坐标绝对值均不超过 B。在算术正确舍入、`sqrtl` 误差至多一 ulp 的条件下，按除法、乘法、加法逐步估计，交点每坐标误差小于 $20uB<1.1\times10^{-14}$；固定 12 位小数再增加至多 5×10^{-13}，仍远小于题目 10^{-6}。这是一项带前提的误差分析，不把本地样本最大误差冒充全域证明。不同交点的非零横坐标间隔，直线情形至少 $2/q$，两圆情形至少 $1/q\ge1.25\times10^{-9}$，故上述误差不会颠倒顺序。相等横坐标来自方向的相应整数分量为零，计算结果也相等。直接按 (x,y) 字典序排序，不使用 epsilon 比较器；相切时按题面重复输出同一点。

范围之外。 `eps=0` 仅取消主动归并。其他实数和更大尺度上，判别式仍可能消去、上溢或下溢，交点构造仍可能病态。默认接口的近切反例继续保留，不宣称已获得通用稳健浮点圆交点。`RealGeometry` 兼容构造仍要求正 eps，因为它同时服务其他几何函数。

68.12 三维投影、旋转与平面求交的数值约定

三维向量见第 1053 页，直线与线段见第 1056 页，平面见第 1059 页。四份接口用法分别是第 1055、1058、1061、1062 页；目前没有该完整接口的正式模板题 AC。分类参考OI Wiki 三维计算几何基础。

`Line3(p,d)` 的第二参数是方向，表示 $p+td$；两个端点用 `Line3::through(a,b)` 构造。投影到无限直线时参数不截断，投影到线段时限制 $0 \le t \le 1$。零方向按点 p 处理距离和投影，不能作旋转轴。向量排序使用精确字典序，禁止以 epsilon 作为排序的相等条件。

对单位轴方向 u 和 $v = q - p$，右手旋转为

$$p + v\cos\theta + (u \times v)\sin\theta + u(u \cdot v)(1 - \cos\theta).$$

绕正 z 轴旋转 $\pi/2$ 时，正 x 轴转到正 y 轴；角度单位是弧度。`angle(a,b)` 返回非零向量的 $[0,\pi]$ 夹角；无向平面夹角另取 $\min(\theta, \pi - \theta)$。

平面保存点 p 与单位法向 n，方程为 $n\cdot(x-p)=0$。`equation(n,h)` 表示 $n\cdot x = h$，一般式 $Ax+By+Cz+D=0$ 应传 $h=-D$。法向必须非零，三点构造必须不共线。有符号距离是 $n\cdot(q-p)$，投影为 $q-n[n\cdot(q-p)]$。负法向翻转距离符号，不改变投影。

求交返回 1 表示单点或交线，0 表示无交，-1 表示整条线在面上或两平面重合；仅返回 1 时写输出参数。零方向线在面上时返回单点，不能返回整条线。两单位法向 n, m 非平行时，取 $d = \mathrm{unit}(n \times m)$、$v = d \times n$，两平面公共点可以取

$$q = p + v\frac{m \cdot (f.p - p)}{m \cdot v}, \qquad \text{交线为} q + td.$$

分母不能随意取绝对值，交线点不要求最靠近原点。

默认距离容差为 10^{-10}，角度正弦容差为 10^{-12}。线面按 $|n \cdot u|$，面面按 $|n \times m|$ 判近乎平行，再用代表点距离分类。这是一套数值规则，近乎平行的对象在精确意义上可能仍然相交；改变坐标尺度、代表点或容差可能改变阈值附近的分类。浮点求交不提供精确拓扑证书。需要精确共面谓词时使用第 1036 页的整数版本并遵守其坐标界。

68.12.1 原三维片段中需要避开的错误

kuangbin 原 188 行逐页核对后，保留完整夹具以记录以下缺陷：

- 平面角写成 $\arccos(n \cdot m)/(|n||m|)$，括号位置错误；法向 $(2,0,0)$ 与 $(0,2,0)$ 得到 $\pi/8$ 而不是 $\pi/2$。
- 交线分母取绝对值后丢失方向：$x = 1$ 与 $y = 2$ 得到的点不在第二平面上。
- 闭线段的点积条件写成等于零，漏掉中点；零线段距离还出现 0/0。
- epsilon 字典序不满足传递性，不能用于 `sort`；输出函数误用了 `scanf`。

新实现不复刻原稿固定 epsilon 混用不同量纲的行为。坐标必须有限，所有中间量须避免溢出和下溢；`hypot` 只改善长度计算，不能替整个算法保证任意尺度稳定。巨大平移与微小差值、近共线构造和近平行求交都可能放大误差，应先平移/缩放并按题意选容差。本机 GNU16 的 `long double` 尾数只有 53 位，不能假定它总比 `double` 更精确。

测试包含 4999 组非退化直线来源对照、8014 组完整接口数据，以有理数投影、四元数旋转和两平面关联条件作独立参考。普通、关闭断言、展开与抄写形式均运行，ASan/UBSan 通过；漏截断、旋转方向、法向归一化、交线位移符号四种错误变体被检出。这些是数值验证证据，不是对所有实数输入正确分类或线上资源通过的保证。

第八部分

杂项

第六十九章　离线算法

69.1　CDQ 分治

69.1.1　四维偏序：双层 CDQ

Dominance4D::count(p, strict=false) 返回 vector<int>，第 i 项为其他点 j 在四个坐标均 ≤p[i] 的数量；strict=true 改为四个坐标均严格小于。按原输入顺序返回，排除自身，重复点按不同输入编号计数；空输入返回空。坐标可为任意 signed64，输入不修改。n≤INT_MAX/2；每点答案 ≤n-1，总点对数须用 long long 汇总。

每点拆成更新与查询两个事件；x、y 同坐标时，弱模式更新先于查询，严格模式查询先于更新。外层 CDQ 固定 x 序，只抽取左半更新与右半查询；内层按 y 序分治，以 z 归并并用 Fenwick 统计 w 前缀。严格模式比较 z< 且查询 w 排名-1；弱模式 z≤、查询完整排名，最后减去自身一次。递归 CDQ，不模拟 DFS 栈。时间 O(n $\log^3$(n+1))，额外空间 O(n)，两层递归总调用深度 O(log(n+1))。坐标只比较，不做减一或取负，避免极值溢出；w 排名减一是压缩后的安全下标。

坐标压缩与 x 序事件：

```
struct Dominance4D
{
    static vector<int> count(const vector<array<long long, 4>> &p, bool strict = false)
    {
        assert(p.size() <= INT_MAX / 2);
        int n = p.size();
        vector<int> ans(n), bit(n + 1);
        vector<long long> coords;
        for (auto point : p) coords.push_back(point[3]);
        sort(coords.begin(), coords.end());
        coords.erase(unique(coords.begin(), coords.end()), coords.end());
        struct Event
        {
            int id, w;
            bool query;
        };
        vector<Event> events;
        for (int i = 0; i < n; i++)
        {
            int w = lower_bound(coords.begin(), coords.end(), p[i][3]) - coords.begin() + 1;
            events.push_back({i, w, false});
            events.push_back({i, w, true});
        }
        auto less = [&](const Event &a, const Event &b, int axis)
        {
            if (p[a.id][axis] != p[b.id][axis])
                return p[a.id][axis] < p[b.id][axis];
            return strict ? a.query > b.query : a.query < b.query;
        };
        sort(events.begin(), events.end(), [&](const Event &a, const Event &b)
        {
            return less(a, b, 0);
        });
```

Fenwick 与外层 CDQ（接上页同一函数）：

```
        auto add = [&](int x, int value)
        {
            for (; x <= n; x += x & -x) bit[x] += value;
        };
        auto sum = [&](int x)
        {
            int result = 0;
            for (; x; x -= x & -x) result += bit[x];
            return result;
        };
        auto outer = [&](auto &&self, int l, int r) -> void
        {
            if (r - l <= 1) return;
            int mid = l + (r - l) / 2;
            self(self, l, mid);
            self(self, mid, r);
            vector<Event> a;
            for (int i = l; i < mid; i++)
                if (!events[i].query) a.push_back(events[i]);
            for (int i = mid; i < r; i++)
                if (events[i].query) a.push_back(events[i]);
            sort(a.begin(), a.end(), [&](const Event &u, const Event &v)
            {
                return less(u, v, 1);
            });
            vector<Event> temp(a.size());
```

两次唯一分离 每个合法更新–查询对在 x 事件序中有先后顺序，恰好在外层递归的一处分到左右两半。只提取左更新和右查询后，再按 y 排序并递归，恰好在内层的一处分离。z 双指针与 w 前缀和检查余下两维，因此既不漏计也不重复计数。

相等坐标 x、y 相等时，严格模式将查询放在更新前面，阻止这一对进入贡献方向；弱模式反过来。z 的比较和 w 的前缀下标也必须一起切换。弱模式每个点的自身恰被算一次，最后减一；重复点仍按不同编号保留。

内层 CDQ、回滚与结果（接上页同一函数）：

```
            auto inner = [&](auto &&solve, int left, int right) -> void
            {
                if (right - left <= 1) return;
                int middle = left + (right - left) / 2;
                solve(solve, left, middle);
                solve(solve, middle, right);
                int i = left;
                for (int j = middle; j < right; j++)
                {
                    while (i < middle && (strict ? p[a[i].id][2] < p[a[j].id][2]
                                                 : p[a[i].id][2] <= p[a[j].id][2]))
                    {
                        if (!a[i].query) add(a[i].w, 1);
                        i++;
                    }
                    if (a[j].query) ans[a[j].id] += sum(a[j].w - strict);
                }
                for (int j = left; j < i; j++)
                    if (!a[j].query) add(a[j].w, -1);
                merge(a.begin() + left, a.begin() + middle,
                      a.begin() + middle, a.begin() + right, temp.begin() + left,
                      [&](const Event &u, const Event &v)
                      {
                          return p[u.id][2] < p[v.id][2];
                      });
                copy(temp.begin() + left, temp.begin() + right, a.begin() + left);
            };
            inner(inner, 0, a.size());
        };
        outer(outer, 0, events.size());
        if (!strict)
            for (int &value : ans) value--;
        return ans;
    }
};
```

模板题使用：QOJ21566 四维偏序

题意：四个坐标各为 1..n 的排列，统计四维全部严格小于的有向点对数；64 位汇总。

所需模板：Dominance4D（第 1112 页，69.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://qoj.ac/problem/21566

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<array<long long, 4>> p(n);
    for (auto &point : p)
        for (auto &x : point) cin >> x;
    auto count = Dominance4D::count(p, true);
    cout << accumulate(count.begin(), count.end(), 0LL) << '\n';
}
```

接口用法演示：API 四维逐点支配计数

演示约定：输入 n 和 strict(0/1)，再输入 n 个四维点；按原顺序逐行输出每点答案，排除自身。

所需模板：Dominance4D（第 1112 页，69.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://qoj.ac/problem/21566

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    bool strict;
    cin >> n >> strict;
    vector<array<long long, 4>> p(n);
    for (auto &point : p)
        for (auto &x : point) cin >> x;
    auto count = Dominance4D::count(p, strict);
    for (int value : count) cout << value << '\n';
}
```

69.1.2 静态三维偏序计数

代码依赖：Fenwick（第 412 页，33.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

count(points) 返回每个原输入位置被其他点逐坐标 ≤ 的次数，重复坐标仍是不同点，排除自身。输入 array<long long,3>，完整 signed64 坐标，空输入返回空；输入不变。内部按三维字典序合并重合点，压缩第三维，CDQ 归并与 Fenwick 回滚。O(n $\log^2$(n+1)) 时间、O(n) 空间、O(log(n+1)) 递归；n 须 ≤INT_MAX/2 且内存可分配，保证共享 Fenwick 的 int 步进不溢出。仅静态无权弱偏序，不含严格不等号、动态更新或 CDQ 递推。

```
struct Dominance3D
{
    static vector<int> count(const vector<array<long long, 3>> &p)
    {
        assert(p.size() <= INT_MAX / 2);
        int n = int(p.size());
        vector<int> order(n), group(n), w;
        iota(order.begin(), order.end(), 0);
        sort(order.begin(), order.end(), [&](int i, int j) { return p[i] < p[j]; });
        vector<array<long long, 3>> v;
        for (int i : order)
        {
            if (v.empty() || v.back() != p[i])
            {
                v.push_back(p[i]);
                w.push_back(0);
            }
            ++w.back();
            group[i] = int(v.size()) - 1;
        }
        int m = int(v.size());
        vector<long long> z;
        for (auto q : v) z.push_back(q[2]);
        sort(z.begin(), z.end());
        z.erase(unique(z.begin(), z.end()), z.end());
        struct Node
        {
            long long y;
            int z, w, id;
        };
        vector<Node> a(m), tmp(m);
        vector<int> cnt(m), ans(n);
        for (int i = 0; i < m; ++i)
            a[i] = {v[i][1], int(lower_bound(z.begin(), z.end(), v[i][2]) -
                                 z.begin()) + 1, w[i], i};
        Fenwick<int> bit(int(z.size()));
```

三维偏序的 CDQ 归并与原序返回（接上页同一 count 函数）：

```
        auto cdq = [&](auto &&self, int l, int r) -> void
        {
            if (r - l <= 1) return;
            int mid = l + (r - l) / 2;
            self(self, l, mid);
            self(self, mid, r);
            int i = l, k = l;
            for (int j = mid; j < r; ++j)
            {
                while (i < mid && a[i].y <= a[j].y)
                {
                    bit.add(a[i].z, a[i].w);
                    tmp[k++] = a[i++];
                }
                cnt[a[j].id] += bit.sum(a[j].z);
                tmp[k++] = a[j];
            }
            for (int t = l; t < i; ++t) bit.add(a[t].z, -a[t].w);
            while (i < mid) tmp[k++] = a[i++];
            copy(tmp.begin() + l, tmp.begin() + r, a.begin() + l);
        };
        cdq(cdq, 0, m);
        for (int i = 0; i < n; ++i)
            ans[i] = cnt[group[i]] + w[group[i]] - 1;
        return ans;
    }
};
```

模板题使用：Luogu P3810 【模板】三维偏序 / 陌上花开

题意：1≤n≤100000，1≤ 三个坐标 ≤k≤200000。对每个输入点统计其他点三个坐标均 ≤ 自身的次数，输出次数为 0..n−1 的点数直方图。重复位置分别计数且各自排除自己。k 只约束题面坐标，库内部压缩，无需外部初始化；空输入和 signed64 极值另属 API 扩展。

所需模板：Fenwick（第 412 页，33.3 节）、Dominance3D（第 1117 页，69.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3810

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    vector<array<long long, 3>> p(n);
    for (auto &[x, y, z] : p) cin >> x >> y >> z;
    vector<int> hist(n);
    for (int x : Dominance3D::count(p)) ++hist[x];
    for (int x : hist) cout << x << '\n';
}
```

69.1.3 三维最长链、计数与方案

Chain3D(points, mod, strict=true)，坐标 array<long long,3>，输入不变。

mod 为 1..INT_MAX，n≤INT_MAX/2 且内存可分配。

strict=true 要求每一步三个坐标都严格增加；false 为三坐标不减，并固定按 (x,y,z, 原输入编号) 递增的规范顺序，重合点可依编号串联，不把相同坐标的排列重复计数。length/count 为最长链长度/模方案数；空输入为 0 和 1%mod。len/ways/parent 按 0-based 原输入编号存每点结尾的最优状态与前驱，无前驱为-1；path() 返回一条最长链原编号。并列时逐步选编号较小的前驱/终点，不保证整条路径字典序最小；计数为 0 模 mod 不表示无方案。CDQ 左递归 → 转移 → 右递归，严格模式整组相同 x 不互转；y 扫描、z 压缩树状数组维护最大长度和同长计数。$O(n \log^2(n+1))$ 时间、O(n) 空间、O(log(n+1)) 递归；坐标只比较，支持完整 signed64。所有成员只读，重新赋值可重算。

```
struct Chain3D
{
    using Point = array<long long, 3>;

    struct State
    {
        int len = 0, ways = 0, end = -1;
    };

    vector<int> len, ways, parent;
    int length = 0, count = 0, last = -1;

    Chain3D(const vector<Point> &p, int mod, bool strict = true)
    {
        assert(mod > 0 && p.size() <= INT_MAX / 2);
        int n = p.size();
        count = 1 % mod;
        len.assign(n, 1);
        ways.assign(n, 1 % mod);
        parent.assign(n, -1);
        if (!n) return;
```

坐标顺序、分组与状态合并（接上页同一结构体）：

```
        vector<int> order(n), z(n), cut{0};
        vector<long long> values;
        iota(order.begin(), order.end(), 0);
        sort(order.begin(), order.end(), [&](int x, int y)
        {
            if (p[x] != p[y]) return p[x] < p[y];
            return x < y;
        });
        for (int i = 1; i < n; i++)
        {
            if (!strict || p[order[i]][0] != p[order[i - 1]][0]) cut.push_back(i);
        }
        cut.push_back(n);
        for (auto q : p) values.push_back(q[2]);
        sort(values.begin(), values.end());
        values.erase(unique(values.begin(), values.end()), values.end());
        for (int i = 0; i < n; i++)
        {
            z[i] = lower_bound(values.begin(), values.end(), p[i][2]) - values.begin() + 1;
        }
        vector<State> bit(values.size() + 1);
        auto join = [&](State x, State y)
        {
            if (x.len != y.len) return x.len > y.len ? x : y;
            x.ways = ((long long)x.ways + y.ways) % mod;
            x.end = min(x.end, y.end);
            return x;
        };
```

先转移再右递归（同一构造函数内）（接上页同一结构体）：

```
        auto cdq = [&](auto &&self, int l, int r) -> void
        {
            if (r - l <= 1) return;
            int mid = l + (r - l) / 2;
            self(self, l, mid);
            {
                vector<int> a(order.begin() + cut[l], order.begin() + cut[mid]);
                vector<int> b(order.begin() + cut[mid], order.begin() + cut[r]);
                auto by_y = [&](int x, int y)
                {
                    return p[x][1] < p[y][1];
                };
                sort(a.begin(), a.end(), by_y);
                sort(b.begin(), b.end(), by_y);
                int i = 0;
                for (int v : b)
                {
                    while (i < (int)a.size())
                    {
                        int u = a[i];
                        if (strict ? p[u][1] >= p[v][1] : p[u][1] > p[v][1]) break;
                        i++;
                        for (int t = z[u]; t < (int)bit.size(); t += t & -t)
                        {
                            bit[t] = join(bit[t], {len[u], ways[u], u});
                        }
                    }
                    State best;
                    for (int t = z[v] - strict; t > 0; t -= t & -t)
                    {
                        best = join(best, bit[t]);
                    }
                    if (!best.len) continue;
                    if (best.len + 1 > len[v])
                    {
                        len[v] = best.len + 1;
                        ways[v] = best.ways;
                        parent[v] = best.end;
                    }
                    else if (best.len + 1 == len[v])
                    {
                        ways[v] = ((long long)ways[v] + best.ways) % mod;
                        parent[v] = min(parent[v], best.end);
                    }
                }
                for (int j = 0; j < i; j++)
                {
                    for (int t = z[a[j]]; t < (int)bit.size(); t += t & -t) bit[t] = State{};
                }
            }
            self(self, mid, r);
        };
```

汇总答案与路径恢复（接上页同一结构体）：

```
        cdq(cdq, 0, cut.size() - 1);
        State best;
        for (int i = 0; i < n; i++) best = join(best, {len[i], ways[i], i});
        length = best.len;
        count = best.ways;
        last = best.end;
    }

    vector<int> path() const
    {
        vector<int> ans;
        for (int u = last; u != -1; u = parent[u]) ans.push_back(u);
        reverse(ans.begin(), ans.end());
        return ans;
    }
};
```

接口用法演示：三维最长链与原编号方案（接口演示）

演示约定：输入 n,mod,strict 及 n 个三维整数点。strict=1 为三维都严格增加；strict=0 为三维不减，重合点按原编号串联，不重复计算它们的排列。输出全局长度/模方案数、一条原编号路径，以及各原输入点的结尾长度/模方案数/前驱编号。空输入全局为 0 和 1%mod，路径为空；mod=1 时计数 0 也能恢复路径。

所需模板：Chain3D（第 1119 页，69.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/misc/cdq-divide/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, mod, strict;
    cin >> n >> mod >> strict;
    vector<Chain3D::Point> p(n);
    for (auto &v : p) cin >> v[0] >> v[1] >> v[2];
    Chain3D chain(p, mod, strict);
    cout << chain.length << ' ' << chain.count << '\n';
    for (int id : chain.path()) cout << id << ' ';
    cout << '\n';
    for (int i = 0; i < n; i++)
    {
        cout << chain.len[i] << ' ' << chain.ways[i] << ' ' << chain.parent[i] << '\n';
    }
}
```

应用题补充：HDU4742 Pinball Game 3D

题意：T≤3 组，每组 1≤n≤10^5 个互不重合的三维点，坐标 0..2^30。命中顺序要求三维均不减，按命中点集计数；输出最长长度及方案数模 2^30。调用 Chain3D 的弱偏序模式；无重合点使每个可行点集只有一个规范顺序。5 秒、32768K，当前最小抄写版已 AC。严格模式、重合点及其他模数仍属本地验证范围。

所需模板：Chain3D（第 1119 页，69.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://acm.hdu.edu.cn/showproblem.php?pid=4742

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        vector<Chain3D::Point> p(n);
        for (auto &v : p) cin >> v[0] >> v[1] >> v[2];
        Chain3D chain(p, 1 << 30, false);
        cout << chain.length << ' ' << chain.count << '\n';
    }
}
```

69.2 整体二分

69.2.1 整体二分：动态区间第 k 小

代码依赖：Fenwick（第 412 页，33.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

OverallKth(a) 读取 0-based 非空初始数组；后续位置用 1..n。set(pos,value) 登记单点赋值；query(l,r,k) 登记闭区间第 k 小并返回 0-based 查询 ID，1<=k<=r-l+1。run() 按查询 ID 返回 vector<long long>，答案对应登记查询时的状态，不是最后数组；可重复 run，也可继续登记后重新 run。支持完整 long long 值域，不做值的算术；同值修改跳过。依赖 Fenwick<int>，递归值域分治，保持时间顺序并撤销贡献。n<=INT_MAX/2，事件数 E=n+2U+Q<=INT_MAX（U 为实际变值次数），另须满足内存限制。V 为出现过的不同值数，时间 O(E log(E+1)+E log(n+1) log(V+1))，空间 O(n+E)。只读 current/events/queries，不外部修改。

```
struct OverallKth
{
    using ll = long long;

    struct Event
    {
        int l, r, k, id;
        ll value;
    };

    int n, queries = 0;
    vector<ll> current;
    vector<Event> events;

    OverallKth(const vector<ll> &a) : n(a.size()), current(a)
    {
        assert(n > 0);
        for (int i = 1; i <= n; i++)
        {
            events.push_back({i, 0, 1, -1, a[i - 1]});
        }
    }

    void set(int pos, ll value)
    {
        assert(1 <= pos && pos <= n);
        if (current[pos - 1] == value) return;
        events.push_back({pos, 0, -1, -1, current[pos - 1]});
        events.push_back({pos, 0, 1, -1, value});
        current[pos - 1] = value;
    }

    int query(int l, int r, int k)
    {
        assert(1 <= l && l <= r && r <= n);
        assert(1 <= k && k <= r - l + 1);
        int id = queries++;
        events.push_back({l, r, k, id, 0});
        return id;
    }
```

继续求解（接上页同一结构体）：

```
    vector<ll> run() const
    {
        vector<ll> answer(queries), values;
        if (queries == 0) return answer;
        auto a = events;
        for (auto e : a)
        {
            if (e.id < 0) values.push_back(e.value);
        }
        sort(values.begin(), values.end());
        values.erase(unique(values.begin(), values.end()), values.end());
        for (auto &e : a)
        {
            if (e.id < 0)
            {
                e.value = lower_bound(values.begin(), values.end(), e.value) - values.begin();
            }
        }
        vector<Event> buffer(a.size());
        vector<bool> left(a.size());
        Fenwick<int> bit(n);
```

继续求解（接上页同一结构体）：

```
        auto solve = [&](auto &&self, int lo, int hi, int begin, int end) -> void
        {
            if (begin == end) return;
            if (lo == hi)
            {
                for (int i = begin; i < end; i++)
                {
                    if (a[i].id >= 0) answer[a[i].id] = values[lo];
                }
                return;
            }
            int mid = lo + (hi - lo) / 2, count = 0;
            for (int i = begin; i < end; i++)
            {
                auto &e = a[i];
                if (e.id < 0)
                {
                    left[i] = e.value <= mid;
                    if (left[i]) bit.add(e.l, e.k);
                }
                else
                {
                    int smaller = bit.query(e.l, e.r);
                    left[i] = e.k <= smaller;
                    if (!left[i]) e.k -= smaller;
                }
                count += left[i];
            }
            int p = begin, q = begin + count;
            for (int i = begin; i < end; i++)
            {
                auto e = a[i];
                if (e.id < 0 && left[i]) bit.add(e.l, -e.k);
                if (left[i]) buffer[p++] = e;
                else buffer[q++] = e;
            }
            copy(buffer.begin() + begin, buffer.begin() + end, a.begin() + begin);
            self(self, lo, mid, begin, begin + count);
            self(self, mid + 1, hi, begin + count, end);
        };
        solve(solve, 0, int(values.size()) - 1, 0, a.size());
        return answer;
    }
};
```

应用题补充：P2617 Dynamic Rankings（整体二分）

题意：n,m<=100000，初值和修改值 0..1e9；Q l r k 查询闭区间第 k 小，C pos value 单点赋值。先按原时间顺序登记，最后统一求解输出；不继承在线 DynamicKth 的 AC。

所需模板：Fenwick（第 412 页，33.3 节）、OverallKth（第 1124 页，69.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2617

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    OverallKth solver(a);
    for (int i = 0; i < m; i++)
    {
        char op;
        cin >> op;
        if (op == 'Q')
        {
            int l, r, k;
            cin >> l >> r >> k;
            solver.query(l, r, k);
        }
        else
        {
            int pos;
            long long value;
            cin >> pos >> value;
            solver.set(pos, value);
        }
    }
    for (long long value : solver.run()) cout << value << '\n';
}
```

69.3 莫队算法

69.3.1 普通莫队算法

69.3.1.1 普通莫队：静态区间统计

```
struct Mo
{
    struct Query
    {
        int l, r, id;
    };
    int n;
    vector<Query> q;

    Mo(int n) : n(n)
    {
        assert(n >= 0);
    }
    int add(int l, int r)
    {
        assert(0 <= l && l <= r && r <= n);
        int id = q.size();
        q.push_back({l, r, id});
        return id;
    }
    template <class Add, class Del, class Ans>
    void run(Add add, Del del, Ans ans, int block = 0) const
    {
        if (q.empty()) return;
        if (block == 0) block = max(1, int(n / sqrt(double(q.size()))));
        assert(block > 0);
        auto ord = q;
        sort(ord.begin(), ord.end(), [&](const Query &a, const Query &b)
        {
            int x = a.l / block;
            int y = b.l / block;
            if (x != y) return x < y;
            return x & 1 ? a.r > b.r : a.r < b.r;
        });
        int l = 0;
        int r = 0;
        for (auto a : ord)
        {
            while (l > a.l) add(--l);
            while (r < a.r) add(r++);
            while (l < a.l) del(l++);
            while (r > a.r) del(--r);
            ans(a.id);
        }
        while (l < r) del(l++);
    }
};
```

Mo(n)，n≥0；add(l,r) 登记 0-based 半开区间并返回查询编号，允许空区间。run(add,del,ans,block=0) 中加删回调接收下标，答案回调接收原查询编号。开始时统计状态须为空，结束后删除剩余点恢复空状态；可重复 run，运行期间不能修改查询表。适用于加删任意位置语义相同的集合统计；逆序对等顺序敏感统计需另外区分左右端点。

查询按左端块号、右端蛇形排序；先扩张再收缩，保证删除位置仍在当前区间。回调 O(1) 时，正块长 B 的时间 O(q log(q+1)+qB+n^2/B+n)，额外空间 O(q)。默认 B=max(1,floor(n/sqrt(q)))，q=0 直接返回。n 和 q 须能用 int 表示。P2709 频次平方和须用 long long，最大 n^2=10^10。P2709 当前代码已 AC（30 个测试点，单点最大 69ms）；运行时间排名未核查。

模板题使用：P2709 莫队 / 小 B 的询问

题意：静态数组值域 1..k，询问闭区间内各值出现次数的平方和。n,m,k≤100000，输入 1-based 闭区间转成 0-based 半开区间；加删一个值由平方差 O(1) 更新，答案用 long long。

所需模板：Mo（第 1128 页，69.3.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2709

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k;
    cin >> n >> m >> k;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    Mo mo(n);
    for (int i = 0; i < m; i++)
    {
        int l, r;
        cin >> l >> r;
        mo.add(l - 1, r);
    }
    vector<int> cnt(k + 1);
    vector<long long> ans(m);
    long long sum = 0;
    auto add = [&](int i)
    {
        sum += 2LL * cnt[a[i]] + 1;
        cnt[a[i]]++;
    };
    auto del = [&](int i)
    {
        cnt[a[i]]--;
        sum -= 2LL * cnt[a[i]] + 1;
    };
    mo.run(add, del, [&](int id)
    {
        ans[id] = sum;
    });
    for (long long x : ans) cout << x << '\n';
}
```

应用题补充：P1494 小 Z 的袜子（分块莫队）

题意：n,m≤50000，颜色 1..n。维护同色无序对数，除以区间所有无序对数并约分；单点区间输出 0/1。输入 1-based 闭区间转 0-based 半开区间，答案按原询问顺序输出。

所需模板：Mo（第 1128 页，69.3.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1494

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    while (cin >> n >> m)
    {
        vector<int> a(n);
        for (int &x : a)
            cin >> x;
        Mo mo(n);
        for (int i = 0; i < m; i++)
        {
            int l, r;
            cin >> l >> r;
            mo.add(l - 1, r);
        }
        vector<int> cnt(n + 1);
        vector<pair<long long, long long>> ans(m);
        long long sum = 0;
        auto add = [&](int i)
        {
            sum += cnt[a[i]];
            cnt[a[i]]++;
        };
        auto del = [&](int i)
        {
            cnt[a[i]]--;
            sum -= cnt[a[i]];
        };
```

继续处理操作（接上页同一 main）：

```
        auto save = [&](int id, long long len)
        {
            long long den = len * (len - 1) / 2;
            if (sum == 0)
            {
                ans[id] = {0, 1};
                return;
            }
            long long d = gcd(sum, den);
            ans[id] = {sum / d, den / d};
        };
        mo.run(add, del, [&](int id)
        {
            save(id, mo.q[id].r - mo.q[id].l);
        });
        for (auto [num, den] : ans)
            cout << num << '/' << den << '\n';
    }
}
```

69.3.1.2 曼哈顿 MST 的递归 DFS 查询顺序

代码依赖：ManhattanMST（第 768 页，45.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

输入二维整数点，返回从原编号 0 开始的 DFS 先序排列；空输入返回空序列，重复点分别保留。可把静态区间 [l,r) 看作点 (l,r)，按返回编号移动莫队双指针。只负责排序，统计和答案由调用处维护。

复用 ManhattanMST 构造树，递归 DFS，每个点恰好一次。设 MST 权值为 T，由 DFS 往返及三角不等式，相邻访问点的 L1 距离之和 ≤2T。起始空区间到首个查询的移动另计；排序 O(q log(q+1)) 时间、O(q) 空间与最坏 O(q) 递归栈。不保证比普通分块更快，也不保证最短访问顺序。点坐标可为 signed64，距离界用 int128。查询数须 ≤INT_MAX；输入不变。P1494 当前实现已 100 分 AC，10 点单点最大 113ms、峰值 10.49MB；排名未核查。

```
vector<int> manhattan_order(const vector<ManhattanMST::Point> &p)
{
    assert(p.size() <= INT_MAX);
    int n = int(p.size());
    if (n == 0)
        return {};
    auto t = ManhattanMST::solve(p);
    vector<vector<int>> g(n);
    for (auto [u, v] : t.edges)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<int> ord;
    ord.reserve(n);
    auto dfs = [&](auto &&self, int u, int fa) -> void
    {
        ord.push_back(u);
        for (int v : g[u])
        {
            if (v != fa)
                self(self, v, u);
        }
    };
    dfs(dfs, 0, -1);
    return ord;
}
```

应用题补充：P1494 小 Z 的袜子（曼哈顿 DFS 排序）

题意：n,m≤50000，颜色 1..n。维护同色无序对数，除以区间所有无序对数并约分；单点区间输出 0/1。输入 1-based 闭区间转 0-based 半开区间，答案按原询问顺序输出。

所需模板：dsu（第 373 页，29.1.1 节）、ManhattanMST（第 768 页，45.1.2 节）、manhattan_order（第 1132 页，69.3.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1494

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    while (cin >> n >> m)
    {
        vector<int> a(n);
        for (int &x : a)
            cin >> x;
        vector<ManhattanMST::Point> q(m);
        for (auto &[l, r] : q)
        {
            cin >> l >> r;
            l--;
        }
        vector<int> cnt(n + 1);
        vector<pair<long long, long long>> ans(m);
        long long sum = 0;
        auto add = [&](int i)
        {
            sum += cnt[a[i]];
            cnt[a[i]]++;
        };
        auto del = [&](int i)
        {
            cnt[a[i]]--;
            sum -= cnt[a[i]];
        };
```

继续处理操作（接上页同一 main）：

```
        auto save = [&](int id, long long len)
        {
            long long den = len * (len - 1) / 2;
            if (sum == 0)
            {
                ans[id] = {0, 1};
                return;
            }
            long long d = gcd(sum, den);
            ans[id] = {sum / d, den / d};
        };
        int l = 0, r = 0;
        for (int id : manhattan_order(q))
        {
            auto [ql, qr] = q[id];
            while (l > ql)
                add(--l);
            while (r < qr)
                add(r++);
            while (l < ql)
                del(l++);
            while (r > qr)
                del(--r);
            save(id, r - l);
        }
        for (auto [num, den] : ans)
            cout << num << '/' << den << '\n';
    }
}
```

69.3.2 带修改莫队

69.3.2.1 带修改的莫队

ModifiedMo(n,m) 保存离线询问，n>0，m 为修改总数。add(l,r,t) 登记第 t 次修改完成后的 [l,r)，返回从 0 开始的询问编号；0≤l≤r≤n，0≤t≤m，询问数小于 INT_MAX。run(add,del,chg,ans,block=0) 排序并移动三个指针。add(p)/del(p) 按下标更新当前区间统计；chg(i,forward,l,r) 将编号 i 的修改前进或回退，并同步修正当前 [l,r) 内的贡献；ans(id) 记录结果。修改编号从 0 开始，前进第 i 次修改时原时间为 i，回退时原时间为 i+1。回调初始状态必须为原数组、空区间、时间 0；重复 run 前由调用方重置状态。状态不会自动恢复，原询问顺序不变。仅适用于可增删维护的区间统计，需区分左右端操作时另行拆分回调。默认块长约 n^(2/3)，也可显式指定正块长。n、询问数、修改数同阶且回调 O(1) 时总时间 O(n^(5/3))，调度器空间 O(q)。

```
struct ModifiedMo
{
    struct Query
    {
        int l, r, t, id;
    };

    int n, m;
    vector<Query> q;

    ModifiedMo(int n, int m) : n(n), m(m) { assert(n > 0 && m >= 0); }

    int add(int l, int r, int t)
    {
        assert(0 <= l && l <= r && r <= n);
        assert(0 <= t && t <= m);
        int id = q.size();
        q.push_back({l, r, t, id});
        return id;
    }
```

```
    template <class Add, class Del, class Chg, class Ans>
    void run(Add add, Del del, Chg chg, Ans ans, int block = 0) const
    {
        if (q.empty()) return;
        if (block == 0) block = max(1, int(pow(n, 2.0 / 3)));
        assert(block > 0);
        auto ord = q;
        auto cmp = [&](const Query &a, const Query &b)
        {
            int x = a.l / block, y = b.l / block;
            if (x != y) return x < y;
            int p = a.r / block, q = b.r / block;
            if (p != q) return (x & 1) ? p > q : p < q;
            return ((x ^ p) & 1) ? a.t > b.t : a.t < b.t;
        };
        sort(ord.begin(), ord.end(), cmp);
        int l = 0, r = 0, t = 0;
        for (auto q : ord)
        {
            while (t < q.t)
            {
                chg(t, true, l, r);
                t++;
            }
            while (t > q.t)
            {
                t--;
                chg(t, false, l, r);
            }
            while (l > q.l) add(--l);
            while (r < q.r) add(r++);
            while (l < q.l) del(l++);
            while (r > q.r) del(--r);
            ans(q.id);
        }
    }
};
```

模板题使用：Luogu P1903

题意：单点改色、区间不同颜色数，n,m≤133333，颜色 1..1000000。先读完操作，cur 仅用于记录每次修改前后的颜色，a 保持初始数组。题目闭区间 [L,R] 转为 [L-1,R)，询问时间是此前修改数。chg 必须先删旧贡献、改 a[p]、再加新贡献，回退使用 pre。按登记顺序输出。三维莫队，回调 O(1)，同阶规模时 O(n^(5/3))。

所需模板：ModifiedMo（第 1135 页，69.3.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1903

Luogu P1903：使用代码

```
int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    vector<int> a(n);
    for (int &v : a) scanf("%d", &v);
    vector<int> cur = a;
    vector<array<int, 3>> upd;
    vector<array<int, 3>> qry;
    for (int i = 0; i < m; i++)
    {
        char op;
        int x, y;
        scanf(" %c%d%d", &op, &x, &y);
        x--;
        if (op == 'Q')
            qry.push_back({x, y, int(upd.size())});
        else
        {
            upd.push_back({x, cur[x], y});
            cur[x] = y;
        }
    }
    ModifiedMo mo(n, upd.size());
    for (auto [l, r, t] : qry) mo.add(l, r, t);
    vector<int> cnt(1000001), res(qry.size());
    int tot = 0;
    auto add = [&](int p)
    {
        if (cnt[a[p]]++ == 0) tot++;
    };
    auto del = [&](int p)
    {
        if (--cnt[a[p]] == 0) tot--;
    };
    auto chg = [&](int t, bool fwd, int l, int r)
    {
        auto [p, pre, nxt] = upd[t];
        bool in = l <= p && p < r;
        if (in) del(p);
        a[p] = fwd ? nxt : pre;
        if (in) add(p);
    };
    auto ans = [&](int id)
    {
        res[id] = tot;
    };
    mo.run(add, del, chg, ans);
    for (int v : res) printf("%d\n", v);
}
```

69.3.3 树上莫队

69.3.3.1 树上莫队：点权与边权

代码依赖：HLD（第 659 页，42.3.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

先对非空连通树 HLD.build(root)，再 TreeMo(tree)；tree 须活到 run 结束且不能修改。点号 1..n，双欧拉序长 2n，n<=INT_MAX/2；DFS 递归。add(u,v,edge=false) 返回 0-based 查询 ID；默认点权，edge=true 把父边权记在子点并排除 LCA。run(add,del,ans,block=0) 的 add/del 参数是顶点，ans 参数是查询 ID；只适合与集合排列无关且可增删的统计。调用前统计为空，结束恢复空；不可在回调内修改树/查询，可重复运行。混用点边查询时回调须有一致的数据解释。默认块长 max(1,2n/sqrt(m))；自定 B>0。m>0 时调度 O(m log(m+1)+mB+n^2/B+n)，查询转换另需 O(m log n)，回调代价另计。空间 O(n+m)，复用 HLD；整数大小另须满足内存限制。

```
struct TreeMo
{
    struct Query
    {
        int l, r, extra, id;
    };

    const HLD &tree;
    vector<int> in, out, euler;
    vector<Query> q;

    TreeMo(const HLD &tree) : tree(tree), in(tree.n + 1), out(tree.n + 1)
    {
        assert(tree.n > 0 && tree.timer == tree.n);
        dfs(tree.rk[1], 0);
    }

    void dfs(int u, int p)
    {
        in[u] = euler.size();
        euler.push_back(u);
        for (int v : tree.g[u])
        {
            if (v == p) continue;
            dfs(v, u);
        }
        out[u] = euler.size();
        euler.push_back(u);
    }

    int add(int u, int v, bool edge = false)
    {
        assert(1 <= u && u <= tree.n && 1 <= v && v <= tree.n);
        if (in[u] > in[v]) swap(u, v);
        int w = tree.lca(u, v);
        int l = w == u ? in[u] : out[u];
        int extra = w == u ? 0 : w;
        if (edge) extra = w == u ? w : 0;
        int id = q.size();
        q.push_back({l, in[v] + 1, extra, id});
        return id;
    }
```

查询调度与状态恢复（接上页同一结构体）：

```
    template <class Add, class Del, class Ans>
    void run(Add add, Del del, Ans ans, int block = 0) const
    {
        if (q.empty()) return;
        int n = euler.size();
        if (block == 0) block = max(1, int(n / sqrt(double(q.size()))));
        assert(block > 0);
        auto ord = q;
        sort(ord.begin(), ord.end(), [&](const Query &a, const Query &b)
        {
            int x = a.l / block, y = b.l / block;
            if (x != y) return x < y;
            return (x & 1) ? a.r > b.r : a.r < b.r;
        });
        vector<bool> active(tree.n + 1);
        auto toggle = [&](int u)
        {
            if (active[u]) del(u);
            else add(u);
            active[u] = !active[u];
        };
        int l = 0, r = 0;
        for (auto a : ord)
        {
            while (l > a.l) toggle(euler[--l]);
            while (r < a.r) toggle(euler[r++]);
            while (l < a.l) toggle(euler[l++]);
            while (r > a.r) toggle(euler[--r]);
            if (a.extra) toggle(a.extra);
            ans(a.id);
            if (a.extra) toggle(a.extra);
        }
        for (int u = 1; u <= tree.n; u++)
        {
            if (active[u]) del(u);
        }
    }
};
```

应用题补充：SP10707 COT2 - Count on a tree II

题意：静态树路径不同点权数，n<=40000,m<=100000；离散化权值，双欧拉序维护出现次数，额外补 LCA。竞赛应用；边权语义另由核心对拍验证。同一驱动已通过用户模板题 U104310（n,m≤10^5，记录 302633483）；SPOJ 转交记录仍 Waiting。

所需模板：HLD（第 659 页，42.3.2 节）、TreeMo（第 1138 页，69.3.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/SP10707

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> a(n + 1), values;
    for (int u = 1; u <= n; u++)
    {
        cin >> a[u];
        values.push_back(a[u]);
    }
    sort(values.begin(), values.end());
    values.erase(unique(values.begin(), values.end()), values.end());
    vector<int> color(n + 1), count(values.size());
    for (int u = 1; u <= n; u++)
    {
        color[u] = lower_bound(values.begin(), values.end(), a[u]) - values.begin();
    }
    HLD tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        tree.add(u, v);
    }
    tree.build();
    TreeMo mo(tree);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        mo.add(u, v);
    }
```

继续处理操作（接上页同一 main）：

```
    vector<int> answer(m);
    int distinct = 0;
    auto add = [&](int u)
    {
        if (count[color[u]]++ == 0) distinct++;
    };
    auto del = [&](int u)
    {
        if (--count[color[u]] == 0) distinct--;
    };
    mo.run(add, del, [&](int id)
    {
        answer[id] = distinct;
    });
    for (int x : answer) cout << x << '\n';
}
```

69.3.4 回滚莫队

69.3.4.1 回滚莫队

RollbackMo(n)，n>=0；add(l,r) 登记 0-based 半开区间 [l,r)，允许空区间，返回 0-based 查询 ID。run(add, snapshot, rollback, ans, block=0)：add 接收位置，snapshot 返回当前快照，rollback 恢复该祖先快照，ans 接收查询 ID。开始时统计必须为空；结束恢复初始快照。每次 add 都须可撤销，包含长期右段；不保留失效快照、不修改查询。只支持与插入次序无关的区间统计。默认 B=max(1,n/sqrt(m))；m>0 时调度与 O(1) 增删总计 O(m log(m+1)+mB+n^2/B+n)，空间 O(m) 加回调状态/历史 O(n)。可重复 run，n=0 只允许空查询；无查询不调用回调。

```
struct RollbackMo
{
    struct Query
    {
        int l, r, id;
    };

    int n;
    vector<Query> q;

    RollbackMo(int n) : n(n)
    {
        assert(n >= 0);
    }

    int add(int l, int r)
    {
        assert(0 <= l && l <= r && r <= n);
        int id = q.size();
        q.push_back({l, r, id});
        return id;
    }

    template <class Add, class Snap, class Undo, class Ans>
    void run(Add add, Snap snapshot, Undo rollback, Ans ans, int block = 0) const
    {
        if (q.empty()) return;
        if (block == 0) block = max(1, int(n / sqrt(double(q.size()))));
        assert(block > 0);
        auto ord = q;
        sort(ord.begin(), ord.end(), [&](const Query &a, const Query &b)
        {
            int x = a.l / block, y = b.l / block;
            if (x != y) return x < y;
            return a.r < b.r;
        });
```

查询调度与状态恢复（接上页同一结构体）：

```
        auto empty = snapshot();
        int last = -1, r = 0;
        for (auto a : ord)
        {
            int b = a.l / block;
            int end = a.l + min(block - a.l % block, n - a.l);
            if (b != last)
            {
                rollback(empty);
                last = b;
                r = end;
            }
            if (a.r <= end)
            {
                auto saved = snapshot();
                for (int i = a.l; i < a.r; i++) add(i);
                ans(a.id);
                rollback(saved);
                continue;
            }
            while (r < a.r) add(r++);
            auto saved = snapshot();
            for (int i = end; i > a.l; ) add(--i);
            ans(a.id);
            rollback(saved);
        }
        rollback(empty);
    }
};
```

模板题使用：P5906 【模板】回滚莫队 & 不删除莫队

题意：n,m<=200000，值 1..2e9；对每个 1-based 闭区间输出相同值的最远下标差，无重复输出 0。离散化，保存各值最左/最右位置和全局答案，历史逐次恢复。

所需模板：RollbackMo（第 1142 页，69.3.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5906

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> a(n), values;
    for (int &x : a) cin >> x;
    values = a;
    sort(values.begin(), values.end());
    values.erase(unique(values.begin(), values.end()), values.end());
    for (int &x : a)
    {
        x = lower_bound(values.begin(), values.end(), x) - values.begin();
    }
    int m;
    cin >> m;
    RollbackMo mo(n);
    for (int i = 0; i < m; i++)
    {
        int l, r;
        cin >> l >> r;
        mo.add(l - 1, r);
    }
```

继续处理操作（接上页同一 main）：

```
    struct Change
    {
        int color, first, last, best;
    };
    vector<Change> history;
    vector<int> first(values.size(), n), last(values.size(), -1), answer(m);
    int best = 0;
    auto add = [&](int i)
    {
        int c = a[i];
        history.push_back({c, first[c], last[c], best});
        first[c] = min(first[c], i);
        last[c] = max(last[c], i);
        best = max(best, last[c] - first[c]);
    };
    auto snapshot = [&]()
    {
        return history.size();
    };
    auto rollback = [&](size_t saved)
    {
        while (history.size() > saved)
        {
            auto h = history.back();
            history.pop_back();
            first[h.color] = h.first;
            last[h.color] = h.last;
            best = h.best;
        }
    };
    mo.run(add, snapshot, rollback, [&](int id)
    {
        answer[id] = best;
    });
    for (int x : answer) cout << x << '\n';
}
```

69.3.5 莫队二次离线

69.3.5.1 莫队二次离线：异或位数配对

xor_hamming_pairs(a,queries,k,bits=14) 按查询输入顺序返回 long long 答案，统计 $l \le i < j < r$ 且 popcount(a[i] xor a[j])=k 的下标对。queries 为 0-based 半开区间，允许空区间、重复查询与空数组；$0 \le bits \le 20$，$0 \le a[i] < 2^{bits}$，n、m 和容器下标须可由 int 表示。k 超出 [0,bits] 直接返回全零；相等值来自不同下标才构成一对。莫队移动生成前缀事件，第二次离线按前缀扫描；k=0 显式扣掉自身。令 C=binom(bits,k)，时间 $O(2^{bits}+nC+m\log(m+1)+n\sqrt{m}+m)$，空间 $O(2^{bits}+n+m)$，m=0 直接返回。无需维护可逆的在线窗口，也不支持修改。

```
// Queries are 0-based half-open; count unordered pairs of distinct indices.
vector<long long> xor_hamming_pairs(const vector<int> &a,
                                    const vector<pair<int, int>> &queries,
                                    int k,
                                    int bits = 14)
{
    int n = a.size(), m = queries.size();
    vector<long long> answer(m);
    assert(0 <= bits && bits <= 20);
    int universe = 1 << bits;
    for (int x : a) assert(0 <= x && x < universe);
    for (auto [l, r] : queries) assert(0 <= l && l <= r && r <= n);
    if (!m || k < 0 || k > bits) return answer;
    vector<int> masks;
    for (int x = 0; x < universe; x++)
        if (popcount(unsigned(x)) == k) masks.push_back(x);
    vector<int> count(universe);
    vector<long long> prefix(n + 1);
    for (int i = 0; i < n; i++)
    {
        prefix[i + 1] = prefix[i] + count[a[i]];
        for (int mask : masks) count[a[i] ^ mask]++;
    }
    int block = max(1, int(n / sqrt(m)));
    vector<int> order(m);
    iota(order.begin(), order.end(), 0);
    sort(order.begin(),
         order.end(),
         [&](int x, int y)
         {
             int bx = queries[x].first / block, by = queries[y].first / block;
             if (bx != by) return bx < by;
             return bx & 1 ? queries[x].second > queries[y].second
                           : queries[x].second < queries[y].second;
         });

    struct Event
    {
        int id, l, r, sign;
    };

    vector<vector<Event>> events(n + 1);
```

莫队端点移动与前缀事件（接上页同一函数）：

```
    int l = 0, r = 0;
    for (int id : order)
    {
        auto [ql, qr] = queries[id];
        if (r < qr)
        {
            events[l].push_back({id, r, qr, -1});
            answer[id] += prefix[qr] - prefix[r];
            r = qr;
        }
        if (l > ql)
        {
            events[r].push_back({id, ql, l, 1});
            answer[id] -= prefix[l] - prefix[ql];
            l = ql;
        }
        if (r > qr)
        {
            events[l].push_back({id, qr, r, 1});
            answer[id] -= prefix[r] - prefix[qr];
            r = qr;
        }
        if (l < ql)
        {
            events[r].push_back({id, l, ql, -1});
            answer[id] += prefix[ql] - prefix[l];
            l = ql;
        }
    }
```

第二次离线与差分还原（接上页同一函数）：

```
    fill(count.begin(), count.end(), 0);
    for (int p = 0; p <= n; p++)
    {
        if (p)
            for (int mask : masks) count[a[p - 1] ^ mask]++;
        for (auto e : events[p])
            for (int i = e.l; i < e.r; i++)
            {
                int value = count[a[i]] - (k == 0 && i < p);
                answer[e.id] += 1LL * e.sign * value;
            }
    }
    long long total = 0;
    for (int id : order)
    {
        total += answer[id];
        answer[id] = total;
    }
    return answer;
}
```

模板题使用：Luogu P4887

题意：统计每个区间内异或值恰有 k 个 1 的无序下标对。输入 1-based 闭区间 [l,r]，只将 l 减一转为 0-based 半开区间；k 可以大于 14，此时全零。k=0 统计相等值的不同下标，不计自身。

所需模板：xor_hamming_pairs（第 1146 页，69.3.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4887

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k;
    cin >> n >> m >> k;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    vector<pair<int, int>> queries(m);
    for (auto &[l, r] : queries)
    {
        cin >> l >> r;
        l--;
    }
    for (long long x : xor_hamming_pairs(a, queries, k)) cout << x << '\n';
}
```

第九部分

数学速查与维护

第七十章　数学速查：条件先于公式

本附录保留历史评测证据；其中传统版 AC 为归档记录，当前只提供 vector 模板。

70.1　杜教筛：预筛与缓存都不可省略

整除反演建模（第 19.20.2 节，第 206 页）的两条卷积恒等式给出 `DuJiao` 的递推，见代码中的 `mertens` 与 `totient_sum`。把商相同的一段合并，系数为区间长度，递归参数严格变小；0 直接查表返回 0。超过预筛上限的结果在返回时写入缓存，后续相同参数直接复用。WIDA 收集的 `sumMu` 示例查询了 `fMu`，但未写入新结果；本库已有实现包含写回，不能照搬遗漏缓存的版本。

设最大查询为 N，预筛上限为 B。单次查询的常用上界为 $O(B+N/\sqrt{B})$，其中哈希访问按期望常数处理；取 B 约 $N^{2/3}$ 得到 $O(N^{2/3})$。多个不同查询可能继续增加缓存，不能把单次空间估计当成任意多询问的上界。保留缓存可复用，主动清空仅影响重算开销；预筛表与上限应视为只读。

接口范围为 $0 \le n \le 10^{11}$，返回精确整数；欧拉函数前缀和用 128 位。WIDA 的 `Z` 模数结果可在精确莫比乌斯前缀和返回后取模得到。负值先求余，仅在余数为负时加模数，避免无条件加模数的溢出风险。

两套实现以独立试除构建到 200000 的前缀参考，检查四种预筛上限、零输入、乱序与重复查询、缓存写回及清空重建。上限 10^{11} 使用 10^6 与 2×10^6 两种预筛规模核对双版本结果；这一项属于边界运行与一致性证据，不是独立答案对拍。普通与 ASan/UBSan 均通过，暂无该接口的在线 AC。

70.2　64 位判素与质因数分解

本库判素采用七个固定底数的 Miller–Rabin，覆盖无符号 64 位整数：

$$2,\ 325,\ 9375,\ 28178,\ 450775,\ 9780504,\ 1795265022.$$

将 $n-1=d2^s$、d 为奇数，检查 a^d 及后续平方链。小于 2 的输入不视为素数；底数模 n 为 0 时跳过该底数，不可误判为合数。模乘用无符号 128 位容纳完整的两个 64 位数之积。固定底数的适用范围依据公开 SPRP 记录，不外推到任意精度整数。

https://miller-rabin.appspot.com/

Pollard–Rho 用 $f(x)=x^2+c \pmod n$ 迭代，并求 $\gcd(|x-y|,n)$。只有 $1<d<n$ 的因子才返回；一次尝试超过轮数上限或得到 $d=n$，就换参数重试。其中模平方后的加法也在 128 位中执行，避免 n 接近 2^{64} 时加 c 溢出。该过程保证返回真因子，但随机尝试次数没有确定的最坏上界。递归分解两侧，直到判为素数，最后排序并保留重复因子。

调用 `factor(1)` 得到空数组；对 $n\ge2$，返回单元素说明 n 为素数。POJ 1811 所需的最小质因子可取排序结果首项；P4718 所需的最大质因子可取末项。不要对空数组取首尾，也不要绕过 `factor` 把素数传给内部 `rho`。

双版本对百万范围以独立埃氏筛核验判素，对 $1\ldots10000$ 以试除核验完整因子列表。另验证三个随机种子下的大质数平方、重复因子、强伪素数和 $2^{64}-1$；期望因子的素性均以试除独立验证。$2^{61}-1$ 使用完整分解 $n-1$ 的 Lucas 素性证书检查；模乘与模幂另以任意精度运算对拍。完整 POJ 1811 与 P4718 驱动分别验证 350 组输入，检查素数标签与最小、最大质因子。P4718 压力批次含 300 个接近 10^{18} 的平衡半素数，本机普通编译运行约 1.99、1.83 秒；机器差异与随机路径都会影响耗时，本地运行时间不直接代表线上性能。两套代码随后通过 P4718 全部 20 个测试点，最长单点显示约 2.11、2.12 秒，低于当前页面的 4 秒限制。该 AC 检查素数标签与最大质因子，范围为 $2\le n\le10^{18}$。完整因子重数、最小质因子、$n=1$ 与更大的无符号 64 位输入仍依据本地验证；普通与 ASan/UBSan 均通过，不构成随机运行时间保证。

来源：kuangbin 2.6；固定底数记录见上方链接。

70.3 单个数的欧拉函数

实现见第 150 页。对 $n \geq 1$，由容斥得到

$$\varphi(n) = n \prod_{p \mid n} \left(1 - \frac{1}{p}\right),$$

其中只枚举不同质因子，故质数幂 p^k 的贡献为 $p^{k-1}(p-1)$。每次更新按先除后乘的顺序：ans $\leftarrow$ (ans/p)$(p-1)$。当前尚未处理的质因子仍整除答案，而且答案只会减小，乘法结果不会超过原始 n。

无分解器参数的入口采用试除，循环上界用 $p \leq n/p$，剩下的 $n > 1$ 再处理一次。这能避免 p^2 溢出，但不消除 $O(\sqrt{n})$ 的耗时；不适合试除 64 位大素数。传入现有 `PollardRho` 对象时先分解，再在排序后的因子列表上去重更新；不重复构造分解器，后续调用沿用其随机状态。该入口没有确定的最坏运行时间。$\varphi(1) = 1$，两个入口均接受完整的正无符号 64 位输入；0 不在定义域内。

本地双版本验证包含 $1 \ldots 20000$ 的线性筛交叉检查、小范围直接计数互质整数，以及重复质因子、大质数平方、$2^{64} - 1$ 和已独立认证的大素数。普通与 ASan/UBSan 测试均通过；P4718 的 AC 只验证底层分解用途，不是此欧拉函数的新 AC。来源：kuangbin 2.7.1、2.7.3；WIDA 的单个数欧拉函数。

70.4 线性筛：三张表的约定

`LinearSieve` 同时构造最小质因子、欧拉函数和莫比乌斯函数。内层遇到第一个整除 i 的素数后停止，因此每个合数 j 只由 $p = \mathrm{lp}(j)$ 与 $i = j/p$ 生成一次，时间为 $O(n)$。若 $p \mid i$，则 $\varphi(ip) = p\varphi(i)$、$\mu(ip) = 0$；否则二者分别为 $(p-1)\varphi(i)$ 和 $-\mu(i)$。

判素使用 $x \geq 2$ 且 $\mathrm{lp}(x) = x$，不能漏掉前半条件。分解表内正整数时，反复取 $p = \mathrm{lp}(x)$、令 $x \leftarrow x/p$，直到 $x = 1$；重复因子会重复输出。0 不可分解，1 的质因子列表为空。上限要求 $0 \leq n <$ `INT_MAX`，避免构造时 $n + 1$ 溢出，并保证内存可用。筛乘积前先检查 $p \leq n/i$，不直接计算可能溢出的 ip。

双版本验证至 $n = 10^7$，包括小前缀、试除最小质因子、因子还原和逐项约数和证书：

$$\sum_{d \mid x} \varphi(d) = x, \qquad \sum_{d \mid x} \mu(d) = [x = 1].$$

这些恒等式在 $d = x$ 项系数为 1，可按 x 递增唯一确定两张表。普通与 ASan/UBSan 均通过；不据此宣称莫比乌斯反演的所有应用已验证。

来源：kuangbin 2.1.1、2.1.2、2.7.2、2.7.4、2.13.1；WIDA 的全部欧拉函数表。

WIDA 的“使用莫比乌斯反演求解欧拉函数”先存储全部约数，再使用

$$\varphi(n) = n - \sum_{d \mid n,\ d < n} \varphi(d).$$

这与 $\varphi * 1 = \mathrm{id}$ 等价；再卷积 μ 可得 $\varphi(n) = \sum_{d \mid n} \mu(d)(n/d)$。本库用线性筛提供同一张表，无需额外保存约数列表。百万范围另分别运行直接卷积和真约数扣减，逐项核对两套筛表；两种方法都不复用线性筛的递推分支。

70.5 区间筛：只保存目标区间

实现见第 152 页。设 $0 \leq L \leq R \leq$ `INT_MAX`，先筛出不超过 $\lfloor \sqrt{R} \rfloor$ 的素数，再对每个素数 p 从

$$\max\bigl(p^2, \lceil L/p \rceil p\bigr)$$

开始标记区间内的倍数。任何合数都有不超过自身平方根的素因子，所以不会遗漏；从 p^2 开始也不会误删 p 自身。输出跳过 0 和 1，其余未标记数恰为素数。底筛与区间标记共用同一个外层素数循环，但使用两张独立的表。

令 $W = R - L + 1$，空间为 $O(\sqrt{R} + W)$，包含返回的素数数组；时间为 $O((\sqrt{R} + W)\log\log(R+3))$。函数每次调用都重建底筛，适合题目中的少量区间查询。端点虽然是 `int`，向上取整的加法、p^2、标记变量和输出遍历变量均用 `long long`；否则 $R =$ `INT_MAX` 时最后一次递增也会溢出。区间长度仍受可用内存约束，不适合一次保存整个 32 位值域。

POJ 2689 与 UVa 10140 的驱动读到 EOF，扫描相邻素数差。最近、最远都只在严格更优时更新，距离并列保留最先出现的一对；少于两个素数时输出题目指定句子。两套实现以试除和 Miller–Rabin 判素进行独立对拍，并检查平方边界、单点区间及右端点为 `INT_MAX` 的百万长度区间；完整驱动验证 208 组输入，含官方样例。普通测试与 ASan/UBSan 均通过；尚无本库在线 AC。

来源：kuangbin 2.1.3；https://onlinejudge.org/external/101/10140.pdf。

70.6 矩形 GCD 计数：缩放后容斥

实现见第 205 页。由 $[g=1]=\sum_{d\mid g}\mu(d)$，前缀矩形互质数对数为

$$F(A,B)=\sum_{i=1}^{\min(A,B)}\mu(i)\left\lfloor\frac{A}{i}\right\rfloor\left\lfloor\frac{B}{i}\right\rfloor.$$

两个商同时不变的右端点取 $r=\min(A/\lfloor A/l\rfloor,B/\lfloor B/l\rfloor)$，用莫比乌斯前缀和合并 $[l,r]$。不能只按一个商分块。只需筛到 $\min(A,B)$，不要求较长的一边也在预处理范围内。$A=0$ 或 $B=0$ 时循环为空，返回 0；正整数条件不包含坐标轴。

矩形内 $\gcd(x,y)=k$ 等价于 x,y 均为 k 的倍数，且缩放后互质。因此答案为四个前缀矩形的容斥：

$$\begin{aligned}&F(\lfloor b/k\rfloor,\lfloor d/k\rfloor)-F(\lfloor(a-1)/k\rfloor,\lfloor d/k\rfloor)\\&-F(\lfloor b/k\rfloor,\lfloor(c-1)/k\rfloor)+F(\lfloor(a-1)/k\rfloor,\lfloor(c-1)/k\rfloor).\end{aligned}$$

这里必须先减 1 再除以 k；零商无需特殊补偿。输入端点为正 `int`，最终答案不超过矩形面积，能用有符号 64 位保存。分块累加使用 128 位，避免依赖正负中间项的额外界。

双版本以小矩形逐对求 gcd 为独立参考，另验证百万正方形的 $F(N,N)=2\sum_{i=1}^{N}\varphi(i)-1$，以及按短边剩余类直接计数的 `INT_MAX` 长条。P2522 双版本完整程序验证五万次查询，包括缩放、窄边和无解；普通与 ASan/UBSan 均通过。尚无本库在线 AC。

来源：kuangbin 2.13.2（BZOJ 2301）；https://www.luogu.com.cn/problem/P2522。

70.7 幂的约数和：不除以 $p-1$

实现见第 127 页。若 $a=\prod p_i^{e_i}$，则 a^b 的约数和是 $\prod_i(1+p_i+\cdots+p_i^{e_ib})$。几何级数的除法公式在模意义下要求分母可逆；合数模数或 $p\equiv1$ 时不能直接套用。

令 $G(a,n)$ 同时返回 $P=a^n$ 与 $S=\sum_{i=0}^{n-1}a^i$。先递归求一半项数 $h=\lfloor n/2\rfloor$ 的 (P,S)；偶数项的结果为 $(P^2,S(1+P))$，奇数项再补最后的 $a^{2h}=P^2$，得到 $(P^2a,S(1+P)+P^2)$，全过程取模。$n=0$ 的结果为 $(1,0)$，因此底数为 0 时，非空级数的和仍为 1。本接口参数是项数，不是最高指数；约数和必须传 e_ib+1。

把幂与和一起返回，可使每层只做常数次运算，总时间 $O(\log(n+1))$。长度允许无符号 128 位，避免 e_ib 在 64 位中溢出；模数及底数允许完整无符号 64 位。所有模乘与必要的加法在 128 位中执行，任意正模数均可用。底数 $a\geq1$，$b=0$ 或 $a=1$ 都返回 $1 \bmod m$；0 的约数和不在接口定义域内。

双版本以直接枚举约数、逐项几何求和和任意精度矩阵幂独立核验，包括模数 1、合数模数、完整 128 位项数，以及指数乘重数超过 64 位的案例。P1593 完整程序各验证 126 组输入，包含 $p\equiv0,1 \pmod{9901}$ 和最大输入；普通与 ASan/UBSan 均通过，在线 AC 待补。

来源：kuangbin 2.12；https://www.luogu.com.cn/problem/P1593。

70.8 任意数组的批量逆元

设 $s_i=\prod_{j<i}a_j \pmod m$，$s_0=1$。仅对总乘积 s_n 求一次逆元，随后从右向左处理。若当前 $t=s_{i+1}^{-1}$，则

$$a_i^{-1}=ts_i \pmod m,\qquad s_i^{-1}=ta_i \pmod m.$$

先保存原 a_i 再覆盖输出，避免误用已写入的逆元更新 t。总乘积与 m 互素当且仅当每一项均与 m 互素，因此一次扩展欧几里得也能判断整个批次是否可行；模数无需为素数。实现见第 165 页。

接口返回可选向量：无解是空的可选值，空输入是有效的空向量，两者不同。模数为 1 时按接口约定返回全零。调用者可以移动不再使用的输入数组，避免额外复制。P5431 的驱动程序计算 $\sum_{i=1}^{n} k^i/a_i$，注意权重从 k^1 开始。五百万项规模采用缓冲的字符读取，并移动输入以控制峰值内存。双版本已做乘法证书和非互素拒绝的穷举检查，以及 64 位极值、空数组、模数 1 的检查；完整驱动另用逐项逆元和几何级数进行对拍。P5431 双版本均通过全部 5 个测试点；动态版记录 297561197，传统版记录 297561804。最慢单点分别为 386、384 毫秒，峰值内存均为 76.80 MB。合数模数、不可逆输入和完整 64 位范围以上述本地测试验证。

该方法可处理任意可逆数组；连续整数的专用递推见下一节。

来源：https://oi-wiki.org/math/number-theory/inverse/。

70.9 连续整数的线性逆元表

素数模数 p 下，对 $1 < i < p$，写 $p = qi + r$，其中 $q = \lfloor p/i \rfloor$，$r = p \bmod i$。r 非零，且 $r < i$。将等式模 p 后乘以 $i^{-1}r^{-1}$，得到

$$i^{-1} \equiv -\lfloor p/i \rfloor \, (p \bmod i)^{-1} \pmod{p}.$$

从 $1^{-1} = 1$ 开始递推，右侧总是已算过的较小下标，故总时间为 $O(N)$。实现见第 167 页；第 0 项只作占位，允许 $N = 0$。$N \geq p$ 时区间包含无逆元的 p，不能套用该表。

递推还依赖 r 可逆，不能仅凭 i 与合数模数互素就照搬。例如 $i = 3, m = 8$ 时 $r = 2$ 不可逆。kuangbin 的“简洁写法”递归式也有这一限制，本书以一般扩展欧几里得接口代替其不成立的合数推广。若 $\gcd(a, m) = 1$，扩展欧几里得给出 $ax + my = 1$，归一化 x 即为逆元；若不互素，则整数方程无解。素数下也可用 a^{p-2} 求逆元；kuangbin 标为“利用欧拉函数”的代码实际使用这一费马形式。

P3811 双版本完整程序逐行输出逆元，已在本地对三百万行结果逐项检验 $ix_i \equiv 1$。P3811 双版本均通过全部 6 个测试点，单点最长分别为 221 ms 与 219 ms。单个逆元另用小模数穷举与费马乘法证书交叉验证；这部分不由 P3811 的 AC 覆盖。

来源：https://oi-wiki.org/math/number-theory/inverse/；WIDA 数论逆元节与 kuangbin 2.4。

70.10 线性同余：最小解与最小周期

实现见第 168 页，接口约定为 $ax + b \equiv 0 \pmod{m}$。先把 a 归一化到 $[0, m)$，令 $g = \gcd(a, m)$。有解当且仅当 $g \mid b$；若 $au + mv = g$，则一个特解为 $u(-b/g)$，所有解为 $x = x_0 + (m/g)t$。归一化到 $[0, m/g)$ 后得到最小非负解。除以 g 后 a/g 与 m/g 互素，因而 m/g 是最小正周期。当 $a = 0$ 且 $m \mid b$ 时所有整数均为解，返回 $(0, 1)$；否则无解。模数 $m = 1$ 始终返回 $(0, 1)$。

输入常数项支持 `LLONG_MIN`，内部用 `-i128(b)`，不能先执行 64 位的 `-b` 再转换。若题目给出 $ax \equiv c \pmod{m}$，可先将 c 归一化到 $[0, m)$ 再取负传入，避免 $c =$ `LLONG_MIN` 时在调用点溢出。本接口依赖的 exgcd 见第 19.7.3 节（第 159 页），不必重复抄写。

P1516 中相遇条件为 $(m - n)t + (x - y) \equiv 0 \pmod{L}$。传入速度差与起点差，第一返回值就是最早非负相遇时间；失败时输出 `Impossible`。不能擅自把负的速度差取绝对值，也不能把两者同余的初始位置强制判为需要跳跃。

小模数测试逐个枚举全部剩余类，检查解集、最小解和最小周期；大数用任意精度等式证书验证。完整驱动以直接模拟、交换青蛙和构造大周期最早相遇时间验证 242 组输入，包括等速和初始位置同余。两套代码均通过普通与 ASan/UBSan 测试，在线 AC 待补。

来源：WIDA 收集的 jiangly exgcd 文件内 `sol`；https://www.luogu.com.cn/problem/P1516。

70.11 扩展 CRT：逐式合并

kuangbin 2.5 的 `solve` 对应单次合并，`MLES` 对应下面的逐式循环。实现见第 169 页的 `crt_merge`，初始取 $r = 0, m = 1$，依次合并 $x \equiv b \pmod{n}$。空方程组保持 $(0, 1)$，表示任意整数；一旦无解即停止，不能忽略失败继

续合并。先把余数归一化。设 $g=\gcd(m,n)$，由扩展欧几里得得到 $mu+nv=g$。写 $x=r+mt$，原问题化为 $mt\equiv b-r \pmod n$。若 $g\nmid(b-r)$ 则无解，否则

$$q=n/g,\qquad t\equiv\frac{b-r}{g}u \pmod q,\qquad 0\le t<q.$$

新周期为 mq，新余数为 $(r+mt)\bmod(mq)$。所有可行 t 相差 q，故新余数加新周期的任意整数倍恰好是全部解；逐式合并时这一不变量保持成立。模数不必互素，模数 1 也可直接参与。

参考实现使用 `int` 保存合并状态，本库扩展为 64 位并显式报告周期越界；避免可能溢出的 `abs(a-a0)`。

代码要求正模数且最终周期不超过有符号 64 位上限；中间乘法与扩展欧几里得系数使用 128 位整数。无解返回假；有解而周期越界抛出异常。失败时旧周期不变，旧余数已归一化。

k 条方程的时间为 $O(k\log M)$，其中 M 是输入及中间模数的上界；递归欧几里得使用 $O(\log M)$ 栈空间。

P4777 的完整程序按 “模数、余数” 读入，题目保证有解且周期不超过 10^{18}。本地验证包含双版本小模数穷举、任意精度整数对拍、有符号极值、无解与越界，以及十万条同余式的完整程序测试。另穷举至多三个小模数方程组成的所有有序系统，检查空系统与后续矛盾。双版本均已通过 P4777 全部 15 个测试点；动态版记录 297521102，传统版记录 297521173。

kuangbin 的“解同余方程组”是模 7 的线性方程组消元应用，见后面的工厂工期一节。来源：https://oi-wiki.org/math/number-theory/crt/；WIDA 数论模板的 excrt 节。

70.12 Garner：总周期可以超过整数范围

设正模数 m_i 两两互素，$P=\prod_i m_i$。经典构造为 $x=\sum_i b_iP_i(P_i^{-1}\bmod m_i)\bmod P$，其中 $P_i=P/m_i$。模 m_i 时其他项都为零，本项为 b_i；两解之差被所有 m_i 整除，因而被 P 整除。但直接写此式需要存储整个 P，并防止乘法溢出。

Garner 改用混合进制

$$x=d_0+d_1m_0+d_2m_0m_1+\cdots,\qquad 0\le d_i<m_i.$$

在确定第 i 位前，对每个后续模数维护已确定部分的余数 v_j，以及当前位权 $c_j=\prod_{t<i}m_t \bmod m_j$。于是

$$d_i=(b_i-v_i)c_i^{-1}\bmod m_i.$$

把 c_jd_i 加到 v_j，再将 c_j 乘 m_i 即可进入下一位。互素性保证逆元存在；模数 1 的位固定为 0，可直接保留。位数范围保证重建的 x 恰在 $[0,P)$，因此就是最小非负解。实现见第 171 页。

若只需 $x \bmod T$，从高位向低位做 Horner 计算：每步将当前值乘对应模数，加上该位，再对 T 取模。T 无需与 m_i 互素，因为这里不对 T 求逆。这返回的是规范解 x 的余数；一般解 $x+kP$ 对 T 的余数可能不同。例如模 3 余 2、模 5 余 3，规范解是 8；对 7 取模为 1，但另一个解 23 的余数是 2。若用多模 NTT 恢复整数系数，仍须证明真实系数位于所选规范区间，不能在乘积不够大时直接把 CRT 的代表元当成原整数。

双版本本地验证使用任意精度整数的经典 CRT 求和作为独立对拍，检查混合进制的完整重建、负余数、有符号极值、非互素输入拒绝和 1000 个模数。P1495 完整程序按题目保证的 $P\le10^{18}$ 输出规范解，双版本均通过全部 11 个测试点；动态版记录 297521594，传统版记录 297521658。超出该题范围的能力以上述本地测试验证。来源：https://oi-wiki.org/math/number-theory/crt/。

70.13 素数模平方根

奇素数 p 下，对 $n\not\equiv0 \pmod p$，Euler 判别式 $n^{(p-1)/2}\equiv1 \pmod p$ 当且仅当存在平方根，否则为 -1。当 $p\equiv3 \pmod 4$ 且有解时，$r=n^{(p+1)/4}$ 为一根，另一根是 $p-r$。

Cipolla 选择 a 使 $w=a^2-n$ 是非平方，令 $\omega^2=w$。在扩域中取 $\alpha=a+\omega$，有 $\alpha^p=a-\omega$，故 $\alpha^{p+1}=n$。令 $\beta=\alpha^{(p+1)/2}$，则 $\beta^2=n$ 且 $\beta^{p-1}=n^{(p-1)/2}=1$，所以 β 落在原素数域中。实现见第 189 页；零和模数 2 单独处理。模数为合数时不能照搬此判别和扩域构造。

70.14 乘法阶与原根

对 $\gcd(a,n)=1$，阶是最小的正整数 d 使 $a^d\equiv 1 \pmod n$，且 $d\mid\varphi(n)$。从 $d=\varphi(n)$ 开始，对每个素因子 q，只要 $a^{d/q}\equiv 1$ 就继续除去 q，所得即为实际阶。对于不可逆的 a，接口返回 0，表示此处没有定义乘法阶。

原根要求阶达到 $\varphi(n)$。只需对其每个不同素因子 q 检查 $a^{\varphi(n)/q}\not\equiv 1$：若实际阶是真因子，商中必含某个这样的 q。必须先检查 $\gcd(a,n)=1$；例如模 9 的候选 3 虽通过上述非 1 判定，却不是原根。$n\ge 2$ 时，原根恰存在于 $2,4,p^k,2p^k$，其中 p 是奇素数。原根存在且取一个原根 g 后，g^t 的阶是 $\varphi(n)/\gcd(t,\varphi(n))$，所以枚举与 $\varphi(n)$ 互素的指数即可得到全部原根，共 $\varphi(\varphi(n))$ 个；这些数按大小输出时还需排序。

实现见第 190 页。本地验证穷举小模数的幂循环，与双版本的阶、最小值和完整集合逐项比较；另验证大素数、奇素数幂、无原根合数和有符号输入边界。P6091 的完整程序输出排序后第 $d,2d,\dots$ 个原根，零个输出时仍保留空行。双版本均通过全部 11 个测试点；动态版记录 297558740，传统版记录 297559082。求阶、最小原根的最小性及更大输入范围仍以上述本地证据验证。若 $d=\operatorname{ord}_n(a)$，则 $\operatorname{ord}_n(a^k)=d/\gcd(d,k)$。两个可逆元素乘积的阶整除各自阶的最小公倍数；两阶互素时，乘积的阶等于两阶之积。来源：https://oi-wiki.org/math/number-theory/primitive-root/；kuangbin 2.17。

70.15 Carmichael：最小公共指数

$\lambda(n)$ 是模 n 的所有可逆元素之阶的最小公倍数，约定 $\lambda(1)=1$。它整除 $\varphi(n)$，但可以严格更小，例如 $\lambda(8)=2$ 而 $\varphi(8)=4$。按互素的素数幂分解 $n=\prod_i p_i^{e_i}$ 后，由 CRT，对每个分量同时成立的幂等式也对 n 成立，因此

$$\lambda(n)=\operatorname{lcm}_i \lambda(p_i^{e_i}),\qquad \lambda(p^e)=\begin{cases}2^{e-2}, & p=2,\ e\ge 3,\\ p^{e-1}(p-1), & \text{其他情形.}\end{cases}$$

奇素数幂的单位群有原根，公共指数即其大小。模 2^e 的奇数可表示为 $\pm 5^k$，当 $e\ge 3$ 时 5 的阶为 2^{e-2}，这给出二幂情形的上界和达到上界的元素。实现见第 162 页，试除版本适用于有符号 32 位正模数。

对可逆底数可把指数对 $\lambda(n)$ 取模；非互素底数仍须另行处理。例如模 8 下 $2^2=4$，但 $2^{2\bmod\lambda(8)}=1$，直接降幂会错误。另有 $\lambda(\operatorname{lcm}(a,b))=\operatorname{lcm}(\lambda(a),\lambda(b))$，这也提供了独立的组合检查条件。

合数 n 是 Carmichael 数当且仅当 $\lambda(n)\mid(n-1)$。等价的 Korselt 判别是：n 无平方因子，且每个素因子 p 都满足 $(p-1)\mid(n-1)$。素数也满足前一整除式，判定时必须排除素数。这些合数对所有互素底数都通过 Fermat 判定，不能用单纯的 Fermat 检验代替素性判定。无平方因子再配合 CRT，也使所有整数 a 满足 $a^n\equiv a \pmod n$。

双版本以小模数的全部实际阶之最小公倍数对拍，另检验大二幂、奇素数幂、32 位边界及公共指数证书。

UVa10006 完整程序对全部合法输入进行本地 Korselt 对拍；在线 AC 待补。

来源：https://oi-wiki.org/math/number-theory/primitive-root/；

应用题：https://onlinejudge.org/external/100/10006.pdf。

70.16 Lucas：按素数进制逐位计算

在特征为素数 p 的域中，$(1+x)^p=1+x^p$。若 $n=\sum_i n_i p^i$，则

$$(1+x)^n=\prod_i (1+x^{p^i})^{n_i}.$$

每个因子选择的次数在 $0,\dots,n_i$ 之间，严格小于 p，因此位数表示唯一。比较 x^k 的系数即得 $\binom{n}{k}=\prod_i\binom{n_i}{k_i} \pmod p$；若某位 $k_i>n_i$，答案立即为零。仅预处理 $0!,\dots,(p-1)!$ 及其逆元，这些数均非零。$p!$ 为零，不能把阶乘表延长后继续用除法公式。实现见第 177 页。

预处理后每位只做常数次乘法，适合固定小素数模数下的多次查询。接口接收 n,k，不接收“n,m 表示选 n 个”的隐含约定；P3807 的驱动程序显式调用 $\operatorname{choose}(n+m,n)$。题目范围内求和安全，完整 64 位输入自行求和时仍要检查溢出。本地验证包含 Pascal 三角、精确大整数、小模数重建和最大无符号 64 位下标；P3807 双版本均已通过全部 25 个测试点；动态版记录 297556209，传统版记录 297556596。完整 64 位参数及额外边界以上述本地对拍验证。来源：https://oi-wiki.org/math/number-theory/lucas/。

70.17 合数模数组合数与插值

对素数幂 $q=p^e$，记 $v_p(n!)=\sum_{j\ge1}\lfloor n/p^j\rfloor$，$U_p(n)=n!/p^{v_p(n!)}$。设 $t=v_p(n!)-v_p(k!)-v_p((n-k)!)$。当 $t\ge e$ 时组合数模 q 为零；否则

$$\binom{n}{k}\equiv U_p(n)\,U_p(k)^{-1}\,U_p(n-k)^{-1}\,p^t \pmod q.$$

这里仅对与 p 互素的单位部分求逆。各素数幂上的结果通过 CRT 合并，对应第 178 页的实现。

在素数域中，n 个模 p 两两不同的横坐标确定唯一的次数小于 n 的多项式。定义 $w_i=y_i/\prod_{j\ne i}(x_i-x_j)$，则

$$f(k)=\sum_{i=0}^{n-1} w_i\prod_{j<i}(k-x_j)\prod_{j>i}(k-x_j).$$

该式不除以 $k-x_i$，查询点命中样本点时仍然成立。当 $x_i=i$ 时，分母为 $i!\,(n-1-i)!\,(-1)^{n-1-i}$，要求 $n\le p$。实现见第 350 页。对整数幂前缀和 $S_d(k)=\sum_{i=1}^{k} i^d$，当 $0\le d\le p-2$ 时，它在模 p 意义下可用次数至多 $d+1$ 的多项式表示；取 $0,1,\dots,d+1$ 的样本即可。不能只因某序列前几项能插值，就断言后续仍由该多项式给出。

70.18 阶乘表：组合数与排列数

当素数模数为 p 且 $0\le k\le n<p$ 时，阶乘均非零，因此

$$C(n,k)=n!\,(k!)^{-1}\,((n-k)!)^{-1},\qquad P(n,k)=n!\,((n-k)!)^{-1} \pmod p.$$

将表从旧上限扩展到 N，先正向补齐阶乘，只对 $N!$ 求一次逆元，再用 $(i-1)!^{-1}=i\,(i!)^{-1}$ 反向补齐新逆阶乘项即可。不需要每一项分别快速幂求逆。较小上限的初始化请求保留已有表。查询前显式扩表，之后查询为常数时间；内部两表重分配时旧元素引用不能继续使用。实现见第 276 页。

WIDA 的“组合数 (Lucas)”文件实际提供的是普通阶乘、逆阶乘、组合数和排列数，没有逐位 Lucas 递推。本书以此节覆盖该文件的数学功能，用显式扩表取代查询时自动扩表，适用范围仍须限制在模数以内。验证使用独立 Pascal 三角和下降阶乘，另检查百万项表及扩表前后的原查询结果。

70.19 组合计数

容斥：$|\bigcup A_i|=\sum_{\emptyset\ne S}(-1)^{|S|+1}|\bigcap_{i\in S}A_i|$。二项式反演：若 $b_n=\sum_{k=0}^{n}\binom{n}{k}a_k$，则 $a_n=\sum_{k=0}^{n}(-1)^{n-k}\binom{n}{k}b_k$。

$$\sum_k\binom{a}{k}\binom{b}{n-k}=\binom{a+b}{n},\qquad C_n=\binom{2n}{n}-\binom{2n}{n+1}.$$

差分形式的 Catalan 公式避免直接除以 $n+1$，在模数下尤其有用，但组合数本身仍须正确求模。斯特林数的定义、快速行列求解与反演统一见第 288 页。

球与盒	空盒允许	空盒禁止
不同球，不同盒	k^n	$k!S(n,k)$
相同球，不同盒	$\binom{n+k-1}{k-1}$	$\binom{n-1}{k-1}$
不同球，相同盒	$\sum_{j=0}^{k}S(n,j)$	$S(n,k)$

上表默认 $k\ge1$；$n=0$、$k=0$ 等边界应按原问题约定单独处理。

70.20 生成函数与递推

普通生成函数 $A(x)=\sum a_nx^n$ 的乘法对应普通卷积。指数生成函数 $A(x)=\sum a_nx^n/n!$ 的乘法对应 $c_n=\sum_k\binom{n}{k}a_kb_{n-k}$。形式幂级数求逆要求常数项可逆；$\log A$ 取 $A(0)=1$，$\exp A$ 取 $A(0)=0$。有限域中积分涉及 $1/i$，必须避免 i 被特征整除。

$$B_{\text{new}}=B(2-AB) \pmod{x^{2m}},\qquad (\log A)'=A'/A.$$

BM 从样本得到最短线性递推。要对未来项使用，必须另外证明序列确实具有不超过相应阶数的线性递推；少量数值吻合不是证明。矩阵转移、有限状态计数常可提供阶数上界。

70.21 线性代数与生成树

域上的线性方程组一致时，解集为一个特解加零空间，零空间维数为未知数个数减秩。浮点消元中的秩依赖误差阈值；模素数消元是精确域运算；合数模数下不可随意除以非零元。无向图 Laplacian 的对角元为度数，非对角元为负的边重数。删去同一行列后的行列式为生成树数。自环不参与生成树；重边需要累加。只有一个顶点时空余子式行列式约定为 1。有向图需要重新明确入度/出度 Laplacian 与根向内/向外树的方向，不能直接复用无向封装。

70.22 模线性方程组：工厂工期

POJ2947（同题 UVa1564）给出每位工人制造的产品类型和起止星期。设第 j 种产品需 x_j 天，第 i 条记录中出现 a_{ij} 次，以星期一为 0、星期日为 6，则

$$\sum_j a_{ij}x_j \equiv \text{finish}_i - \text{start}_i + 1 \pmod 7.$$

起止两天均计入；同一种产品重复出现必须累加系数。跨星期无需特判，模数类型负责归一化负数。每个余数在 $[3,9]$ 中恰有一个代表，因此模方程的解数就是原问题的解数。

直接使用第 361 页的模高斯消元，模数取 7。增广矩阵有 m 行、$n+1$ 列，最后一列是右端项。先判断 `consistent`：假时输出无解；为真且秩小于 n 时输出多解；否则读取特解，余数小于 3 的分量加 7。不能先以秩不足判断多解，因为零系数行仍可能有非零右端项。本题所有非零系数均可逆；合数模数不具备这一性质，不能直接套用此消元。

完整双版本程序位于 `verify/poj/2947.*.cpp`。验证包含所有起止星期组合、产品重复 7 次导致系数归零、矛盾的零行、独立枚举所有 3～9 天赋值，以及用可逆行变换构造的 300 阶稠密系统。该应用验证覆盖分类及唯一解输出，不代替零空间基等其他接口的验证。来源：`https://onlinejudge.org/external/15/1564.pdf`；kuangbin 2.10.2。

70.23 数值算法与几何

排序必须使用严格弱序；不能以“差小于 eps 则相等”定义通用排序比较器。叉积与行列式容易发生抵消；整数题优先精确整数判定，浮点构造单独处理误差。自适应积分的局部误差估计需要函数足够规则；奇点、间断与强振荡必须先分段处理。三分搜索要求单峰性，整数域最后枚举剩余区间；二分答案要求判定单调性。

来源索引 本章为重新组织的公式速查，扩展范围依据 `https://oi-wiki.org/math/`。各算法实现、模板题用法与验证进度以仓库逐项清单为准；本章公式不能代替对应的完整代码与验证。

70.24 线性递推：系数方向与外推条件

约定 $a_i = \sum_{j=1}^{k} c_{j-1}a_{i-j}$，初值是 $a_0, \ldots, a_{k-1}$。对应特征多项式为

$$F(x) = x^k - c_0x^{k-1} - c_1x^{k-2} - \cdots - c_{k-1}.$$

若 $x^n \bmod F(x) = \sum_{i=0}^{k-1} r_i x^i$，则 $a_n = \sum_{i=0}^{k-1} r_i a_i$。代码由高次向低次消去多项式项，第 j 个系数写入原次数减 j 的位置，避免倒置系数。例如 Fibonacci 初值为 $[0,1]$，递推系数为 $[1,1]$。

BM 在域上求有限前缀的最短递推。本书实现要求素数模数；一般的模合数环不能保证非零偏差存在逆元。仅凭若干样本无法证明真实序列以后一直遵循同一个规律。若事先知道序列满足阶数至多 d 的齐次线性递推，至少给出前 $2d$ 项，再用恢复出的系数求后续项。系数向量末尾的零不能擅自删除：它可能对应非平稳的初始段，改变阶数会改变递推开始适用的位置。

P5487 同时输出最短系数与远项，见第 267 页。第一行仅有 $c_0, \ldots, c_{k-1}$，没有阶数前缀；取恰好 k 个初值再求远项。全零前缀得到空系数时保留空首行，第二行输出 0。题目保证最短递推唯一；本程序用 BM 与特征多项式求余，时间 $O(nk + k^2 \log m)$，空间 $O(n + k)$。

小规模参考逐阶建立全部递推方程，用高斯消元验证最短性和唯一系数，用伴随矩阵幂求远项。最大 5000 阶使用独立构造 $a_n = \sum_{t=1}^{k} t^n$，特征多项式为 $\prod_{t=1}^{k}(x - t)$。前 $2k$ 项的 Hankel 矩阵是 VV^{T}，其中 $V_{i,t} = t^i$；不同非零底数给出可逆的 Vandermonde 矩阵，所以最短阶数确为 k。远项直接求 $\sum t^m$，无需调用被测递推算法。普通与 ASan/UBSan 均通过，本地时间与线上 AC、速度排名分开记录。

70.25 素数模高次剩余与单位根

实现见第 195 页；依赖本书离散对数、线性同余和快速幂。设 p 为素数，g 是原根，$k > 0$。$a = 0$ 时根只有零；否则写 $a = g^b$、$x = g^y$，方程转为

$$ky \equiv b \pmod{p-1}.$$

令 $d = \gcd(k, p-1)$。当 $d \nmid b$ 时无解；否则指数解为 $y = y_0 + i(p-1)/d$，$0 \le i < d$。因此用三个数存全部根：

$$\texttt{first} = g^{y_0}, \qquad \texttt{ratio} = g^{(p-1)/d}, \qquad \texttt{count} = d.$$

依次乘 ratio 即可枚举；不保证数值递增，也不保证 first 是最小根。ratio 是阶为 d 的单位根，因而枚举恰有 d 个不同的根。模 $p = 2$ 同样有效；指数同余的模数是 1，唯一非零根为 1。

接口支持 $p \le 10^{12}$，需要调用者提供已验证的素数与原根；k 支持完整 unsigned long long，先对 $p-1$ 取模后解同余。a 支持完整 signed long long，先正规化。返回空 optional 表示无解，返回 `{0,1,1}` 表示唯一零根。给定原根后的期望时间、空间为 $O(\sqrt{p})$；枚举全部根另外需要 $O(d)$ 时间。模乘须先扩展到 128 位，不可用 64 位乘积直接取模。

测试穷举小于 100 的每个素数、全部原根及小指数，用任意精度模幂逐个枚举候选根并比较整个集合；覆盖负数、64 位边界、模 2、无解与零根。另检查 998244352 个根的紧凑表示，以及素数 999999999989 上的植根测试；后一个素数与原根的条件由试除及乘法阶证书独立确认。普通与 ASan/UBSan 均通过，尚无专门在线 AC。

这是 OI Wiki 高次剩余 中离散对数转线性同余方法的素数模实现。素数幂及非互素被开方数见下一节；一般合数模数见后文；改良 Tonelli–Shanks 尚需另补，不能把本节状态解读为该页面全部覆盖。

70.26 素数幂模高次剩余与根的提升

实现见第 196 页。令 $M = p^e$。当 $a = 0$ 时，全部根是 $p^{\lceil e/k \rceil}$ 的倍数。否则令 $t = v_p(a) < e$，必须有 $k \mid t$，否则无解。取 $s = t/k$，$q = p^{e-t}$，写 $x = p^s y$，只需求单位方程

$$y^k \equiv a/p^t \pmod{q}.$$

每个单位根 y 对应原模数下的根

$$x = p^s y + \ell p^s q, \qquad 0 \le \ell < p^{t-s}.$$

这些提升保持互不相同，也解释了非互素被开方数可能产生大量根。

奇素数时，提供模 p^e 的原根，约化后仍是模 q 的原根，沿用离散对数转线性同余的方法。$q = 2$ 时单位根只有 1；$q \ge 4$ 且为二的幂时，单位表示为 $(-1)^z 5^y$。符号指数模 2，5 的指数模 $q/4$；$q = 4$ 时只有符号分支。分别解两个指数同余即可，不可假设二的高次幂存在原根。

返回结构体保存 first、两个 ratio、两个 count，以及 step、lifts。全部根由下式给出：

$$\big(\texttt{first} \cdot \texttt{ratio}[0]^i \cdot \texttt{ratio}[1]^j \bmod \texttt{step}\big) + \ell \cdot \texttt{step}.$$

其中 $0 \le i < \texttt{count}[0]$、$0 \le j < \texttt{count}[1]$、$0 \le \ell < \texttt{lifts}$。`get` 已处理模乘溢出，`size` 返回三者乘积；不保证根按数值排序。零根情形复用同一表示。

模数上限 10^{12}，k 支持完整无符号 64 位。只检查范围，不检查 p 的素性和奇素数原根的阶，调用者须保证。默认参数 $g = 0$ 仅用于 $p = 2$；奇素数不能漏传原根。时间和空间由 BSGS 决定，期望 $O(\sqrt{q})$；单次按索引取根使用快速幂，需 $O(\log M)$。

已穷举多个素数的全部不超过 256 的素数幂、所有被开方数与小指数，用任意精度模幂直接枚举所有根，比较整个集合。另测有符号与无符号 64 位边界、2^{39} 的非平凡离散对数、3^{20} 的非互素方程，以及数千亿根的紧凑表示。普通与 ASan/UBSan 均通过；尚无专门在线 AC。一般合数的 CRT 组合见下一节；改良 Tonelli–Shanks 仍未完成，OI Wiki 页面继续标记部分覆盖。

70.27 一般合数模高次剩余：CRT 组合

实现见第 200 页。调用者提供完整分解 $M = \prod_{t=0}^{F-1} m_t$，其中 $m_t = p_t^{e_t}$ 两两互素，并提供每个奇素数幂的原根。先分别求局部根族；任一局部无解，整体即无解。若局部根数为 c_t，则整体根数为 $\prod_t c_t$。只保存局部根族，不创建这个大小的数组。

对每个因子预处理

$$E_t = \frac{M}{m_t}\left(\left(\frac{M}{m_t}\right)^{-1} \bmod m_t\right) \bmod M.$$

对一组局部根 r_t，唯一对应的标准代表是

$$x = \sum_t r_t E_t \bmod M.$$

逆元存在是因为因子两两互素。`get(index)` 将索引按各个 c_t 转成混合进制数字，再把局部索引拆成两个乘法根族索引和加法提升索引，最后用上述公式组合。CRT 双射保证不重复、不遗漏，输出顺序不保证递增。

$M \le 10^{12}$，$k > 0$；空因子列表表示 $M = 1$，返回唯一代表 0。`mod` 保存模数，`total` 保存总解数，`parts` 和 `weight` 保存局部根族与 E_t。模数乘积检查上界，CRT 的乘积先扩展至 128 位。单次按索引查询为 $O(F \log M)$；求解的主要时间和空间为局部 BSGS，因子列表接口不包含整数分解与原根搜索的成本；自动准备重载见下一节。

已对 $1 \le M \le 180$、所有被开方数与小指数直接枚举所有根，与两套代码的完整输出集合比较，并反转因子顺序重新检查。另覆盖有符号与无符号 64 位边界、混合素数幂上的非互素方程和大量零根，以及十一种不同素因子组成模数时的全部 1024 个平方根。普通与 ASan/UBSan 均通过；尚无专门在线 AC。至此已有已知分解及原根时的 BSGS 求根路线，更快的改良 Tonelli–Shanks 等内容仍未覆盖，OI Wiki 条目保留部分完成状态。

70.28 自动准备模数分解与原根

实现见第 199 页。`root_factors(M,rho)` 把 $M \le 10^{12}$ 分解为互异素数的幂，返回合数求根需要的 (p, e, g) 列表；模数 1 返回空列表。二的幂不需要原根，固定填零。奇素数幂 $m = p^e$ 的乘法群是循环群，设 $\varphi = m/p \cdot (p-1)$。分解 φ 后，候选 g 为原根当且仅当

$$p \nmid g, \qquad g^{\varphi/q} \not\equiv 1 \pmod{m} \quad \text{对每个不同素因子} q \mid \varphi.$$

从 2 起枚举候选，返回最小原根；所有奇素数幂均有原根。这里检查的是模 p^e 的阶，不能仅取一个模 p 的原根而不验证提升。

重载 `CompositeRoots::solve(a,k,M,rho)` 自动调用此准备过程。

随机分解工作区类型为 `PollardRho`。如果需要在同一个模数下处理多个方程，先保存因子列表，调用原有列表接口，避免重复分解和原根搜索。准备成本包括 M 与各 φ 的因数分解，以及逐个候选的模幂检验；不能把这部分直接算作常数时间。

独立试除验证 $1 \ldots 500$ 的全部模数与多组接近 10^{12} 的边界值，并用任意精度模幂验证返回原根的阶。小模数进一步检查更小候选不能生成全部单位。直接传模数的两套求根接口，在小模数上与全部候选根的枚举结果比较。普通与 ASan/UBSan 均通过；尚无自动准备入口的在线 AC。

70.29 任意正模数行列式

实现见第 322 页。素数模高斯消元可以除以任意非零主元，合数模数没有这个性质。本节只使用倍加行和交换行，不求逆元。先把元素正规化到 $[0,m)$，对第 i 列的两个元素 a,b，用整数商 $q=\lfloor a/b\rfloor$ 执行 $R_i\leftarrow R_i-qR_j$，再交换 R_i,R_j。该列的新数对为 $(b,a \bmod b)$，所以循环按欧几里得算法终止。其余元素每步对 m 取模，只改变整数代表，不改变行列式模 m 的值。

倍加行不改变行列式；交换行改变符号。消去所有下三角元素后，乘上对角元素及累计符号即得结果。剩余部分的该列全零时直接返回零，不需要判断某个非零元素是否与模数互素。例如 $\mathrm{diag}(2,3)$ 在模 8 下的行列式为 6，不能因为第一个主元没有逆元就判定失败。

支持任意正 signed 64 位模数与完整 signed 64 位输入元素。正规化后单个乘积小于 2^{126}，行更新差值和结果乘积都放入有符号 128 位。空矩阵按约定返回 $1 \bmod m$；模 1 的所有返回值为零。时间上界 $O(n^3 \log m)$，矩阵副本占 $O(n^2)$ 空间。本节可以用于合数模的拉普拉斯余子式，但第 55.2 节（第 961 页）的 `MatrixTree<mod>` 接口仍使用素数模高斯，不能仅把其模板参数改成合数；合数模使用第 55.4 节（第 967 页）的独立接口。

测试用排列定义穷举所有元素属于 $\{-1,0,1\}$ 的三阶矩阵，并用任意精度 Bareiss 消元验证随机矩阵。覆盖不可逆主元、奇异矩阵、模 1、有符号 64 位边界、Fibonacci 长欧几里得过程，以及 300 阶对角矩阵和完全图余子式的 Cayley 公式。两套版本普通与 ASan/UBSan 均通过；尚无专门在线 AC。行变换依据可参考 OI Wiki 行列式。

70.30 参数化 NTT 与模数适配

实现见第 231 页。传入素数 p 和原根 g；长度 n 必须为二次幂且整除 $p-1$。代码自动计算 $L = 2^{v_2(p-1)}$ 作为长度上限，不代表分配这个长度的数组没有成本。两个非空数组的普通卷积需要补齐到不小于长度和减一的二次幂。模 998244353 的 $L = 2^{23}$，模 167772161 的 $L = 2^{25}$，两者均可用原根 3。不能只改模数而忽略原根和长度条件；接口只检查基本范围，不检查素性或原根的阶。

正变换按 g 的相应单位根计算，逆变换使用逆单位根并乘 n^{-1}。模数支持 int 范围内的素数，模整数的加减和乘积使用 64 位中间量。蝶形层以半长度递增，避免最后一层之后的有符号整数溢出。

独立参考用任意精度整数直接计算 DFT 和朴素卷积，同时验证正逆变换复原。已检查七组素数和原根，包括模 2、3、17、97，斯特林数的默认模数与洛谷题目模数，以及 2013265921、原根 31。这些测试中的素性及原根条件另用试除和乘法阶证书确认。大规模全一数组卷积逐项比较已知三角形系数，检查补齐长度的边界。两套实现普通与 ASan/UBSan 均通过。

70.31 Bostan–Mori 与快速递推单项

代码见第 268 页。对于 $a_i=\sum_{j=1}^{k} c_{j-1}a_{i-j}$，令 $Q(x)=1-\sum_{j=1}^{k} c_{j-1}x^j$，$P(x)=(Q(x)\sum_{i=0}^{k-1} a_i x^i) \bmod x^k$，则 $[x^n]P/Q=a_n$。注意系数按前一项、前两项的顺序给出，初值从 a_0 开始。

$$\frac{P(x)}{Q(x)}=\frac{P(x)Q(-x)}{Q(x)Q(-x)}=\frac{U_0(x^2)+xU_1(x^2)}{V(x^2)}.$$

分母的奇数次项抵消，因此只保留分子与 n 同奇偶的项，分母保留偶数项，再将 n 右移一位。最终 $n=0$ 时答案是 $P(0)/Q(0)$。实现不需要分子次数小于分母次数，空分子直接返回零；分母常数项必须非零。每轮系数长度不增加，初始最大长度 d 必须满足 $2d-1$ 不超过 NTT 容量。总时间 $O(d\log d\log(n+1))$，空间 $O(d)$；$n=0$ 直接取常数项。

独立验证包括按定义做形式幂级数除法、友矩阵快速幂，以及 $(1-x)^{-32000}$ 的二项式系数闭式；覆盖完整无符号 64 位下标、常数分母、不等于 1 的非零常数项、空分子、不真分式、零递推系数和多种 NTT 素数。P4723 驱动读取有符号系数并归一化；两套版本均获在线 AC：动态版与 传统版各 5 个测试点全部通过，最慢单点分别为 907 ms、905 ms。OJ 覆盖模 998244353、$n\le 10^9$、$k\le 32000$；其他模数、完整 64 位下标与通用有理函数接口的扩展范围由本地测试支持。OI Wiki 线性递推 中的连续区间取项与其他快速算法仍待补，不将单项算法视为整页覆盖。

70.32 集合变换与不相交子集卷积

代码见第 242 页与第 244 页。长度 $N = 2^m$ 确定全集的位数，数组第 s 项对应集合 s。OR 变换计算 $\widehat{a}_s = \sum_{t\subseteq s} a_t$，AND 变换计算所有超集的和；逆变换将逐位加法改成减法。XOR 变换的系数是 $(-1)^{\mathrm{popcount}(s\&t)}$，蝶形将 (u, v) 变为 $(u+v, u-v)$，逆变换每层再除以 2。因此 OR、AND 可用于任意模数，XOR 需要 2 可逆；本实现限制为奇数模数，使用 $(\bmod +1)/2$，不对合数模数使用费马小定理。三类卷积均是变换后逐项相乘，再逆变换。

$$c_s = \sum_{t\subseteq s} a_t b_{s\setminus t} \qquad \text{要求两个集合不相交。}$$

普通 OR 卷积允许重叠，不能直接用于上式。将输入按集合大小分层，$A_{i,s} = [|s| = i]a_s$，B 同理。每层 OR 变换后，对层数做卷积：$H_{k,s} = \sum_{i=0}^{k} \widehat{A}_{i,s}\widehat{B}_{k-i,s}$。对每个 k 逆 OR 变换，得到并集恰为 s、两集合大小之和为 k 的贡献。取 $k = |s|$，由 $|u| + |v| = |u \cup v| + |u \cap v|$ 得到交集为空。时间 $O(m^2 2^m)$，空间 $O(m2^m)$；本实现逐层复用结果缓冲区，不用第三组二维数组。$m = 20$ 时两组分层数组本身占 168 MiB，另计输入和线性缓冲。

验证独立枚举变换系数、所有下标对与子掩码，并检查复合模数与 `INT_MAX` 模数。大规模使用 $a_s = x^{|s|}$、$b_s = y^{|s|}$ 的逐位乘积恒等式：OR 为 $(x+y+xy)^{|s|}$，AND 为 $(xy)^{|s|}(1+x+y)^{m-|s|}$，XOR 为 $(x+y)^{|s|}(1+xy)^{m-|s|}$，不相交子集卷积为 $(x+y)^{|s|}$。P4717 两套版本各 10 点在线 AC：动态版和 传统版，最慢单点 84 ms、81 ms。

P6097 两套版本各 5 点在线 AC：动态版和 传统版，最大测试点显示 2.70 s、2.69 s，内存 188.83 MB、189.00 MB。两题 OJ 仅验证各自题面模数和长度范围，其他模数及长度 1 仍由本地独立测试覆盖。OI Wiki FWT 的其他运算及更广泛应用仍待覆盖。

70.33 Taylor 平移与等比点求值

代码见第 270 页和第 246 页。多项式使用升幂系数，输入长度 n 表示次数小于 n，允许末尾零。平移输出的第 i 项满足

$$[x^i]f(x+c)=\frac{1}{i!}\sum_{j=i}^{n-1}f_j j!\frac{c^{j-i}}{(j-i)!}.$$

倒序 $f_j j!$，与 $c^j/j!$ 卷积，再取第 $n-1-i$ 项并乘 $1/i!$。保留原数组长度；零多项式可以用空数组表示，返回空数组。使用 NTT 素数及原根，要求 $2n-1$ 不超过变换容量；此时所需阶乘非零。时间 $O(n\log n)$、空间 $O(n)$。这与给出 $f(0),f(1),\dots$ 后平移点值的任务不同，后者尚待补充。

Chirp Z 返回 $f(ar^i)$，$0\le i<m$。设 $T_j=j(j-1)/2$，由 $ij=T_{i+j}-T_i-T_j$ 得

$$f(ar^i)=r^{-T_i}\sum_{j=0}^{n-1}f_j a^j r^{-T_j}r^{T_{i+j}}.$$

将第一项数组倒序，与 r^{T_j} 卷积，抽取第 $n-1+i$ 项。三角数幂通过累乘生成，不计算可能溢出的 $j(j-1)$。令 L 为不小于 $n+m-1$ 的最小二次幂，直接做模 x^L-1 的循环卷积。普通乘积最高次数为 $2n+m-3$，而所需区间任意下标 $t\ge n-1$ 满足 $t+L\ge 2n+m-2$，因此不会收到高次项回卷，所取系数仍然精确。要求 $n+m-1$ 不超过 NTT 容量，总时间 $O((n+m)\log(n+m))$、空间 $O(n+m)$。

约定 $r^0=1$，故 $r=0$ 时第一项为 $f(a)$，余项均为 $f(0)$；$a=0$ 时所有点为零，$r=1$ 时所有点相同。这些情况单独计算，避免对零求逆。$m=0$ 返回空数组，空输入返回 m 个零。同一份算法支持小阶公比、重复点与有限域中多次绕回，输出保持原顺序。

独立验证采用 Pascal 二项式展开和 Horner 求值，涵盖 7 种 NTT 素数、零公比与单位公比、小素数完整容量边界。大规模平移用 $(x+b)^d$ 检查每个系数；等比点求值用几何级数闭式逐点检查。普通与 ASan/UBSan 均通过。Library Checker 的 Polynomial Taylor Shift 和 等比点求值 已准备 vector 完整驱动；本地检查各题 524288 的最大规模，尚无在线 AC。参考OI Wiki 平移及 Chirp Z，更广泛的点值平移和应用另行覆盖。

70.34 可持久化区间拼接的懒标记

代码见第 544 页。对象从全零版本开始，同一对象内版本共享坐标域。区间加仅复制被访问节点，不向旧子节点下传标记。节点的和满足 $\text{sum} = \text{left.sum} + \text{right.sum} + \text{lazy} \cdot \text{len}$。读取时把祖先累积增量作为参数传下去，因此查询完全只读。

拼接含义为将目标版本某个半开区间替换成源版本同一坐标区间。由于两棵树祖先的 lazy 可能不同，不能直接将源子树挂到目标祖先下面。递归参数 offset 保持“源祖先增量减目标祖先增量”；下降时更新为 $\text{offset} + \text{source.lazy} - \text{destination.lazy}$。当区间完整覆盖，复制源节点并将这个 offset 加到其 lazy 和 sum，此后挂到目标祖先下面时，逻辑值才与源版本相同。offset 为零时直接共享源节点；没有覆盖时保留目标节点。空区间、零增量与全区间拼接都可仅增加版本别名。

每一层至多有两个边界节点，其他分支直接返回或复制一个整段节点，所以单次修改、拼接均为 $O(\log n)$ 新节点，查询为 $O(\log n)$ 时间、零新节点。不支持不同坐标偏移的平移粘贴，也不回收仍被旧版本引用的节点。所有 long long 的和、差、懒标记与长度乘积都应在范围内；节点数与版本数须小于 `INT_MAX`。

验证用独立向量保存各分支版本，逐区间检查更新、拼接和历史版本；包含正负增量、不同祖先标记、空区间、全段共享，以及查询前后的节点数量检查。规模验证包含长度 200000 的 10000 次区间版本组合，并在长度 `INT_MAX` 的隐式坐标域验证端点操作与共享根。普通与 ASan/UBSan 均通过；区间加 1、后缀拼接和单点读取的 vector 应用已有 QOJ8240 AC，其余通用接口范围仍由本地验证覆盖。来源需求是2023 杭州站题解 第 24–26 页的 K 题，完整驱动及验证见下节。

回退时间与追加版本编号。 SP11470 To the moon 完整用法见第 549 页。模板 add 返回不断追加的版本编号，而题目的时间戳在 B 操作后可能减小，二者不能直接混用。用 ver[t] 保存时间戳 t 对应的模板版本；C 从 ver[t] 更新，再令 ver[t+1] 指向新版本并推进时间。再次经过同一时间戳时替换映射，不能写入原来版本的节点。Q、H 读取对应映射；B 只移动当前时间。初值无需逐点建树：把初始前缀和单独保留，树中仅存后续区间增量，每次查询将两部分相加。这样初始构造 $O(n)$，之后每次 C/Q/H 为 $O(\log n)$，B 为 $O(1)$；不调用 splice。本地以完整数组快照独立核验分支与历史，并检查正负大初值、反复回退、时间戳覆盖和最大规模部分区间更新。

70.35 杭州 2023 K：跨版本拼接应用

Card Game 完整用法见第 551 页，使用前节的可持久化区间模板。

将下标转为从 0 开始，设 $F_i(r)$ 为区间 $[i,r]$ 的剩余牌数，p 为 i 之后第一张与 a_i 相同的牌，若不存在则 $p=n$。当 $r<p$ 时，第一张牌始终保留，剩余牌的过程与从 $i+1$ 开始相同；当 $r\ge p$ 时，到达 p 会将此前区间全部清空。因此

$$F_i(r)=\begin{cases}1+F_{i+1}(r), & r<p,\\ F_{p+1}(r), & r\ge p.\end{cases}$$

每个版本的坐标是右端点 r，值是 $F_i(r)$。从 $i=n-1$ 向前处理：复制版本 $i+1$ 并在 $[i,p)$ 加 1，再把 $[p,n)$ 拼接自版本 $p+1$。$p=n$ 时省去后一步。询问原题的 $[l,r]$ 时，从版本 $l-1$ 读取坐标 $r-1$ 的单点值。端点先与上次答案异或解码，不能将输入查询预先离线排序。

$n,q\le 300000$，牌号位于 $[1,n]$。构建及查询时间分别为 $O(n\log n)$、$O(q\log n)$。驱动另计算各次操作可能访问的相交节点数作为分配上界，一次 reserve 足够容纳整个构建过程，避免复制扩容的临时峰值。该计数仅依据区间结构，不影响牌局答案，且同样为 $O(n\log n)$。

本地参照按题意逐张放牌，遇重复就实际删除栈中相应后缀。穷举长度不超过 7 的二元牌序列与全部区间，并打乱询问顺序、真实编码异或输入。另有随机牌局、$n=q=300000$ 的全异、全同、周期序列闭式检查，以及稀疏重复大数组的短区间逐张模拟。vector 驱动的历史 AC通过 41 个正式测试和 3 个额外测试，最慢 789 ms、452688 KB。当前展开程序与该快照仅格式不同，本轮未重新提交；传统版已退役。此应用仅调用区间加 1、后缀拼接、单点读取；一般有符号增量与区间求和仍由前节本地测试支持。

70.36 树路径交：投影与单点交

核心见第 652 页，P3398 完整用法见第 653 页。将 x 投影到树路径 a–b，投影是 $\mathrm{lca}(a,b)$、$\mathrm{lca}(a,x)$、$\mathrm{lca}(b,x)$ 中深度最大者。三条两点间路径在同一个分叉点相遇；按同一根计算 LCA 即得这个分叉点，与顶点编号无关。设 c,d 的投影是 p,q。若 $p\neq q$，路径 c–d 必经过 p–q，后者正是两路径的交集。若 $p=q$，两端可能在同一条旁支，不能直接认为有交；须检查 $d(c,p)+d(p,d)=d(c,d)$。通过时为单点交，否则为空。这里距离和 depth 都按边数，不能换成带权根距离；零权边会破坏上述距离等式的判别。

接口返回交集端点和顶点数；空交是 $\{0,0,0\}$，公共边数为 $\max(0,\mathrm{vertices}-1)$。P3398 仅输出是否相交，不验证具体端点。本地接口测试穷举至五点的全部标号树、每个根及四端点，独立构造两条路径的顶点集合求交，核验数量与返回端点间的整条路径。完整驱动另验 146 棵小树、100 棵随机换号树，以及十万点链和十万次查询；两种编译模式均通过。EulerLCA 预处理 $O(n\log n)$、每次路径交 $O(1)$；若换用 HLD 回调则为 $O(\log n)$。完整驱动以 C++20 O2 在P3398 记录 299972443通过全部十个测试点，最慢单点 134 ms、峰值 31.89 MB；该 AC 仅对应布尔相交判定，不涵盖端点、精确数量或带权距离。

70.37 非负权树的点集直径摘要

代码见第 638 页。一个非空点集只需保存任意一对直径端点 a,b，即可回答任意树上点 x 到该点集的最远距离：$\max(d(x,a),d(x,b))$。边权非负，包括零权；该结论不能推广到一般图或负权树。

证明可将点 u 投影到路径 a–b，记投影距 a 为 t，离路径的距离为 h，直径长为 D。由于 $d(a,u),d(b,u)\le D$，有 $h\le\min(t,D-t)$。再将查询点投影到同一路径，投影距 a 为 s，离路径的距离为 g。经两个投影点连接得到 $d(x,u)\le g+h+|s-t|\le g+\max(s,D-s)$，右侧恰为 $\max(d(x,a),d(x,b))$。因此插入新点只需比较到两个旧端点的距离；合并点集时，只插入另一个摘要的两个端点即可。并列直径可任选，不应假设端点编号固定或合并后的端点与合并顺序无关。

Euler LCA 现在同时保存根距离，但其原有 distance 接口仍返回边数。带权距离请显式调用 weighted_distance / Weighted_Distance；采用 $(D_u-D_z)+(D_v-D_z)$ 避免不必要的两倍根距离中间值。所有路径权和须能用 long long 表示。独立验证穷举小点集及合并，逐点检查最远距离与返回顶点；带权路径用独立树遍历参照，并覆盖全部根、零边权与 `LLONG_MAX` 单边。原有 Euler LCA 全小树回归，以及 50 万点、边权 10^9 的递归链也通过普通与 ASan/UBSan 检查。

离线添叶仍是固定树度量。 CF379F New Year Tree 的完整用法见第 640 页。先把全部添叶操作建成最终树，再用 HLD 求距离；只有旧树外接新叶，不会改变两个旧点之间的唯一简单路径。因此可以按出现顺序向摘要插点，尚未出现的点只帮助预处理距离，不提前进入点集。初始四点也必须插入；每次两个新点都插完，再输出 length。$q\le 500000$ 时最终有 $4+2q$ 个点，采用 HLD 的 $O(q)$ 空间、$O(q\log q)$ 总时间。递归 DFS 保留；本地放大测试栈，最大实例包含五十万层链。250 组随机合法添叶过程逐步重建当前树，以独立两次 BFS 求直径，另验满规模链；普通与 ASan/UBSan 均通过。这份用法仅覆盖 insert 和 length；任意点集合并、空集和最远点见接口独立测试。2026-09-29 洛谷提交入口返回 Codeforces RemoteJudge 暂不可用，故不登记此题线上 AC。

杭州 2023 F 中，点权互异。令 g 为全树点权的 mex，只需按点权 $0,1,\dots,g-1$ 建立前缀摘要。某个前缀全部位于以 x 为中心、半径 k 的范围内，当且仅当该摘要的最远距离不超过 k。二分最长满足条件的前缀长度即为答案；$g=0$ 时直接为零。题面树边带权，不能用边数距离。完整驱动在 `verify/qoj/8235.*.cpp`，本地使用实际权距与 mex 集合参照，$n=q=500000$ 的链和星形树保持递归 DFS。两套驱动各 50 个正式测试和 1 个额外测试全部在线 AC：动态版 666 ms、198076 KB，传统版 588 ms、198196 KB。OJ 验证题面正权边、前缀插入和最远距离判定；零权边、完整 64 位距离与任意摘要合并仍由独立本地验证支持。

70.38 函数图的入环链与到达查询

代码见第 799 页。三份完整用法分别见第 802 页（CSES1750 跳转）、第 803 页（CSES1160 最少步数）、第 803 页（P2921 不同访问点数）。题目点号均从 1 开始，构造时统一减 1；只有跳转返回的顶点需要加 1，最少步数的 0 和不可达的-1 应原样输出。P2921 只需 depth 加环长，不代表它测试了跳转或两点可达查询。本轮完整驱动穷举至四点的全部 288 个后继映射，独立沿后继走到首次重复，核验步数、跳转与访问数；另验随机映射和题目最大规模。CSES1750 的题面步数上限为 10^9，完整 unsigned64 范围另由本地测试覆盖，不能由该题 AC 推出。

每个点有且仅有一个后继，故每个弱连通分量由一个有向环和若干指向环的树组成。先不断移除入度为零的点，剩下的点恰在环上；沿后继枚举各环。反向处理移除序列时，后继的环编号、入环点和入环距离均已确定，可直接传给当前点。这里使用的是入度剥离队列，不是用显式栈模拟 DFS。

跳转 k 步时，若 k 小于入环距离，用普通倍增；否则先到入环点，减去入环步数，再对环长取模。因此即使 k 达到无符号 64 位上界，也只需 $O(\log n)$ 层倍增表。环上位置相加使用无符号宽整数，避免两个大下标的 int 中间值溢出。

判断从 u 到 v 的最少步数时：不同环编号必不可达。若 v 在入环树上，先检查 u 的深度至少为 v 的深度，再跳过深度差，看是否恰好到 v，从而排除同分量的不同分支。若 v 在环上，答案为 u 的入环步数加上沿后继方向的环上距离。不可达返回 -1，从自己到自己为零步。字段 position 只对环点有意义，不要将树点的 -1 当作环坐标。

独立参照逐步行走直至首次重复，同时记录每个点的第一次出现位置。穷举不超过 5 个点的全部后继映射，并检查跳转、最少步数、入环点与环长；另验证同对象重建、空图、50 万点长链连环、50 万个自环和完整无符号 64 位步数。普通与 ASan/UBSan 均通过。完整驱动对应CSES 1750、CSES 1160 和 P2921，均使用 vector 版驱动。前两题检查跳转与可达步数，后一题仅使用“入环步数 + 环长”，不能替代查询接口的验证。P2921 的当前程序已取得线上 AC；两道 CSES 查询题的线上评测仍待完成。

70.39 杭州 2023 H：随机事件顺序与函数图

Sugar Sweet II 的完整驱动为 `verify/qoj/8237.*.cpp`，复用函数图分解与逆阶乘表。事件 u 发生前，自己的数量仍为 a_u；目标 $v = b_u$ 只可能为 a_v 或 $a_v + w_v$。因此 $a_u < a_v$ 时必定发生，$a_u \ge a_v + w_v$ 时必定不发生。其余情况下，事件 u 成功当且仅当事件 v 更早执行且成功。

将两类确定事件的后继改为自己，另保存哪些是确定成功的终点。未修改的依赖环没有成功起点：最早执行的环上事件必失败，后续也无法形成成功链。自环同理，自己不可能严格少于自己。沿函数图到达确定失败终点或纯依赖环的节点，成功概率都是零。若到达确定成功终点，且入环距离为 d，沿途 $d+1$ 个事件必须按唯一的顺序出现。均匀随机全排列对这 $d+1$ 个事件诱导均匀相对顺序，因此成功概率为 $1/(d+1)!$。这些顺序约束有关联，不能将每条依赖当作独立的二分之一概率。期望为 $a_u + w_u/(d+1)!$，在模 10^9+7 下计算。

题面所有增量为正，$\sum n \le 500000$；阶乘表跨测试组只扩充不缩小，避免大量单点测试时反复求逆。当前驱动复用带倍增表的通用函数图，时间、空间为 $O(n \log n)$；概率传播本身只读取 entry 和 depth，不调用跳转与最少步数接口。

验证穷举小输入及事件全排列，逐步修改真实糖果数量，最后汇总精确总和并除以排列数。另外检查 50 万点的依赖链、纯环，以及 50 万组单点测试，普通与 ASan/UBSan 均通过。两套驱动各 31 个正式测试与 2 个额外测试全过：动态版 162 ms、84608 KB，传统版 158 ms、84612 KB。本应用不能验证函数图的 advance 或 steps 接口。

70.40 杭州 2023 G：开放时间的单位边最短路

驱动 `verify/qoj/8236.*.cpp` 直接枚举网格邻居，不建立四倍规模的边表。初始蛇身从头到尾编号 $0,\dots,k-1$，第 $i\ge 1$ 节所在点的最早进入时间为 $k-i$，头与其他空格为零。旧蛇尾允许在同一步进入，所以尾格的限制必须是 1。长度为 4 的满 2×2 网格样例距离为 $0,3,2,1$，平方和为 14。

每步只能删除或移动一个尾节，故这些时间限制必要。最短路可以取不重复顶点的路径；行走前把途中等待的总时间全部用于缩短蛇身，随后沿路径连续移动。进入初始蛇身第 i 节时，已删除的尾节数量至少为 $k-i$；新形成的蛇身不会与无重复顶点的路径相撞。因此这些限制也充分。等待超过 $k-1$ 没有必要，因为此时全部初始蛇身均已释放。

转移为 $d_v=\max(d_u+1,\mathrm{open}_v)$。按距离递增处理顶点时，第一个发现 v 的前驱就给出最小候选值，故不需要后续松弛。候选等于 d_u+1 的顶点进入普通 FIFO；其余进入对应开放时间的桶。每次取两者中距离较小的一项。桶指针只能扫描到下一个 FIFO 距离，否则可能越过之后才被发现的较早开放时间。令 $R=\max \mathrm{open}$，复杂度 $O(V+E+R)$，空间 $O(V+R)$。此性质依赖所有边权为 1；一般非负权仍需优先队列。

本地以独立堆 Dijkstra 对拍，并穷举 2×3 网格障碍和全部 744 种初始蛇形，对真实蛇身序列逐步执行移动与缩尾 BFS，核对每个点的最早到达时间。另验证正式样例、满长链和 3000×3000 空网格及隔断网格；普通与 ASan/UBSan 均通过。答案用 unsigned long long 累加距离平方，自然实现模 2^{64}。两套驱动在 QOJ8236 各通过 58 个正式与 3 个额外测试：动态版 324 ms、123080 KB，传统版 294 ms、122768 KB。线上证据对应单位网格、源点时间为零及初始蛇身开放时间；一般有向图与非零源点时间由本地对拍覆盖。

70.41 带修莫队：时间轴与区间状态

先离线读完操作，记录每个单点修改的下标、修改前值和修改后值。询问的时间 t 表示前 t 次修改已执行；输入中的询问本身不推进时间。同一位置的第二次修改，其旧值必须来自第一次修改后的数组，不能始终使用初始值。需要离散化时，应将全部初值与修改后值一同纳入。

调度器保持三个指针 (l,r,t)，回调统计恰好对应当前版本的半开区间 $[l,r)$。前进修改 i 时写入 after，回退时写入 before。若修改点在当前区间内，必须先删除旧贡献、改值、再加入新贡献；区间外只改数组。随后移动区间端点，最后按原询问编号写答案。扩展区间先于收缩，支持空区间和互不相交区间。整个 run 结束后状态停留在排序后的最后一条询问；再次调用前须重置原数组和统计量。

左端点分块、右端点分块，再按时间排序，相邻块采用蛇形顺序。时间关键字是真实修改次数，不能把询问编号当作时间维度。令块长为 B，询问数为 Q，修改数为 U，则常数时间回调下的移动量上界为

$$O\left(QB+\frac{n^2}{B}+U\min\left(Q,(\lfloor n/B\rfloor+1)^2\right)\right).$$

另有 $O(Q\log Q)$ 排序与 $O(Q)$ 调度空间。三者同阶时取 $B\approx n^{2/3}$，得到 $O(n^{5/3})$；规模悬殊时可手动调整块长。回调需要区分左右端增删时应拆分四个方向，当前简版适合不同数、频次矩等统计。

P1903 数颜色 驱动以颜色频次数组维护不同数，真实操作顺序建立修改前后值，随后交给调度器。本地快照测试逐项检查每个时间、每个区间及修改回退状态，另以顺序 set 查询核对完整驱动。最大规模验证 133333 个元素和操作，并覆盖仅修改、仅询问与重复修改同一位置。最终简短命名版本在 P1903 记录 297653743 通过全部 13 个测试，最慢单点 546 ms、峰值内存 14.65 MB。此记录使用默认块长与不同颜色数统计；自定义块长、空区间及任意登记时间由本地测试覆盖。最快提交榜的排名及总数尚未核验。

70.42 SCC 模板题与速度榜口径

QOJ906 来自 Library Checker，要求输出强连通分量的拓扑序。本库 Tarjan 编号按逆拓扑序产生，驱动将输入点号加一，再按分量编号从大到小输出，输出点号减一。CCF_NOI 的记录 2940638 通过全部 11 个测试，203 ms、58636 KB。验证覆盖 add、run、bel 和完整分量输出，不调用 dag。本地独立可达性参考核对分量等价关系及每条跨分量边方向，并测试 50 万点链、环和多分量。普通与 ASan/UBSan 均通过；本地测试线程使用 1 GiB 栈容纳 sanitizer 扩大的递归栈帧，提交代码保持普通 main 和递归 DFS。

2026-09-12 核对速度榜时，页面第 66 行处于 203 ms 的第 65–67 行并列区间。但该表合并同一用户的提交，不能将行号与满分提交数 578 配对。以 CCF_NOI 逐页读取页面链接的满分列表，共取得 143 条，其中 93 条严格更快、3 条同为 203 ms。可见范围内按耗时并列排名为 94 out of 143，位置区间为 94–96。统计页显示所有提交 1053、满分提交 578；与可见列表的计数差异原因尚未核实，因此全站排名仍为待查。记录清单与证据见 `docs/ranking-audits.json`，不得将该局部排名改写为全站排名。

70.43 并查集模板题的接口范围

P3367 输入点号为 1-based，驱动统一减一后调用 dsu。记录 297668102 通过全部 20 个测试，最慢单点 196 ms、峰值内存 2.27 MB。该输出验证 merge、same 及其调用的 find，不验证 size、groups；后两者已由独立标签参考和 50 万点长链测试核对。

70.44 ACL 的组合约定

固定参考 AtCoder Library 提交 `864245a00b00`，公开模块分为数据结构、图论、数学与字符串。学习短小的运算接口及明确的组合契约，不直接复制全部工程支持层。尤其注意：懒标记 composition(f,g) 表示先作用 g、再作用 f；SCC 分量方向与 2-SAT 的真假编码不能分别替换。本库已补齐 ModInt 的四种复合赋值，Fenwick 的模数区间和组合通过独立整数余数参考及 ASan/UBSan。新增运算只有本地验证，不能套用历史 OJ AC；取模和仍不适用于 kth。完整 12 模块、6 内部头文件的对照及缺项清单见 `docs/ACL-REVIEW.md`。

70.45 动态树路径乘积：虚子树与双向信息

附件“爱莲说”对应的量是

$$F(u)=\sum_{v\in V}\prod_{x\in \mathrm{path}(u,v)} w_x.$$

包括单点路径，所以一个孤立点的答案是其点权，而不是 1。固定根后，递推为 $F(u)=w_u(1+\sum_{v\ \text{为子节点}} F(v))$。使用第 609 页的 `TreePathProducts`，可以单点修改并动态选取查询根。它维护固定树，不包含连边、删边。

辅助树按实链顺序维护三项：整条链点权积 P、从链首出发的贡献 L、从链尾出发的贡献 R。每个点另记虚儿子贡献之和 G。对于左、右辅助子树，合并式为

$$P=P_l w P_r,\qquad L=L_l+P_l w(1+G+L_r),\qquad R=R_r+P_r w(1+G+R_l).$$

空辅助树取 $P=1, L=R=0$。翻转时必须同时交换左右儿子和 L,R；只翻儿子会破坏换根后的方向。

`access` 把旧右链变为虚儿子，将它的 L 加入 G；把新右链变为实儿子，从 G 减去它的 L。旧右链的父指针保留为路径父亲。修改点权前先将该点换为根，确保旧贡献已经沿访问路径正确移出虚子树项；然后修改权值并更新合并信息。换根结束时根处的 L 就是所求总和。整个过程只用加减乘，没有除法，零权、负权与合数模数同样适用。

接口使用 0-based 顶点；内部辅助树为 1-based，内部函数不可直接作为外部 API。附件输入参数 x 对应 $w=1-x$，这个转换在使用程序中显式完成。从源码可确定上述递推和输入协议，但原题外部地址尚未核验，因此只登记本地应用用法。建树 $O(n)$，修改和查询摊还 $O(\log n)$，总空间 $O(n)$；建树和辅助树下传保持递归，最坏深度 $O(n)$。本地以逐终点路径枚举、模 8、零权、20 万点链与星形及反复换根验证，不将普通 LCT 的线上记录视为这个变体的 AC。

70.46 隐式 Treap 位翻转与顺序反转

附件 FHQ Treap 中名为 `Reverse` 的操作是逐位异或 1；它没有交换左右儿子，不是区间顺序反转。

`flip_bits(l,r)` 明确表达前者，要求区间元素均为 0 或 1；原有 `reverse(l,r)` 表达后者。两者均使用 1-based 闭区间。例如 0011 的前三位取反得到 1101，而前三位顺序反转得到 1001。二进制序列的区间和就是 1 的个数，仍可直接调用 `query`。

取反可写作 $x \mapsto 1-x$。用符号标记与加法标记表示 $x \mapsto sx+b$，其中 $s \in \{1,-1\}$。取负时同时令 $s \leftarrow -s, b \leftarrow -b$，再加 1；下传时必须先符号、再加法。若把先后顺序颠倒，已有加法会变成错误的符号。顺序反转则只交换儿子并翻转其标记，它与逐元素的取负、加法可交换。结构体公开位翻转接口仅用于二进制值；所有标记和取负中间量仍须在 `long long` 范围内，不能对 `LLONG_MIN` 直接取负。

附件 `Build` 按中点建立树后给节点随机优先级，未修复优先级堆序；这不能直接继承随机 Treap 的平衡复杂度证明。本库逐次 `insert` 构建，期望 $O(n \log n)$，每次分裂合并均维持堆序；不能把这种构建成本写为线性。随机优先级下单次操作期望 $O(\log n)$，不保证针对已知种子的自适应对抗操作的最坏界。分裂、合并及中序输出保持递归。附件缺少完整牛客题号和主程序，因此新增使用例只作为本地接口示例。

70.47 替罪羊树：重复计数与墓碑清理

第 523 页使用三个不同的量：`size` 计元素副本总数，`nodes` 计物理节点数，`live` 计仍有副本的不同键数。重复值增加计数，删除最后一个副本后保留墓碑。查询排名只累计副本数，不能用物理节点数代替。

更新回溯时，若某孩子占节点数的比例超过 3/4，或者有效键不足物理节点的一半，就递归中序收集活节点，按中点重建。重建复用活节点编号，保留重复次数。重建后的中序和计数均不变，故集合与排名不变；所有新子树均满足平衡与有效比例。令 d 为当前不同键数，树高为 $O(\log(d+2))$，因为每层节点数缩小到至多 3/4，根的物理节点数至多为 $2d$。查询最坏对数，更新摊还对数；一次重建仍可能线性。删除清理后的墓碑不回收到节点池，空间按累计分配量计算，不按当前集合大小计算。

附件的 `Rank` 只有命中相等键才加一，未命中时可能少一；仅插入 1 后查询 2 的排名就是一个反例。本版统一定义为严格较小副本数加一，空树中的排名也为 1。严格前驱后继使用 `optional`，避免数值哨兵与合法键冲突；完整 `long long` 键无需做减一或加一。P3369 保证第 k 小及邻居存在，使用程序才能直接解引用结果。所有递归保持原样；本地排序向量对拍、重建不变量及 20 万键规模均已验证，线上待补。

70.48 莫队二次离线：前缀事件与自身扣除

对区间内满足 $H(i,j) = [\text{popcount}(a_i \text{ xor } a_j) = k]$ 的无序下标对，使用 `xor_hamming_pairs`。令 $t(i) = \sum_{j<i} H(i,j)$，$P(x) = \sum_{i<x} t(i)$，以及 $T(p,i) = \sum_{j<p} H(i,j) - [k=0 \wedge i<p]$。右端扩展时贡献为 $t(i) - T(L,i)$，左端扩展为 $T(R,i) - t(i)$；删除取相反数。整段端点移动的 t 和由 P 的区间差求出，剩余部分存成按前缀长度分桶的事件。第二遍顺扫只做插入，每次枚举异或后有 k 个置位的掩码更新计数；事件中的每个下标只读一次计数，并在 $k=0$ 时扣自身。最后按莫队顺序累加窗口答案差，再按原编号返回。

区间统一为 0-based 半开区间，先扩右、扩左，再缩右、缩左，保持窗口合法。允许空区间；没有查询时直接返回。位宽默认为 14，若 k 超出位宽则答案全零。P4887 的 $k < 2^{14}$ 并不代表 $k \le 14$。令 $C = \binom{b}{k}$，块长约为 $n/\sqrt{m}$，总时间 $O(2^b + nC + m\log(m+1) + n\sqrt{m} + m)$，空间 $O(2^b + n + m)$。端点移动不再乘枚举掩码数 C。答案与前缀和使用 long long；十万个相等数的配对数超过 int32。本地全枚举、随机逐对参考及十万规模的 $k = 0, 7$ 分支通过；线上验证待补。

70.49 线段树维护单调栈：双向扫描与接雨水

`MonotoneStackSeg` 维护区间加后的运行最大值贡献。每段的 F_0, F_1 分别为从左、从右扫描的最高值减原值之和。若外部高度 x 不低于整段最大值，贡献为 $x \cdot len - sum$。否则，将孩子按扫描顺序记为 A, B：若 $x \ge \max A$，递归 B；否则递归 A，而 B 的贡献已经是 $F_{parent} - F_A$。每次只递归一个孩子，因此单段带边界扫描为 $O(\log n)$。区间平

移不改变这些差值，懒标记只改最大值、原值和与标记本身。区间修改回溯及任意子区间扫描最坏 $O(\log^2(n+1))$，建树 $O(n)$。

对全局前缀和后缀最大值 L_i, R_i，有 $\max(L_i, R_i) = M$，其中 M 为全局最大值。因此

$$\sum_i \big(\min(L_i, R_i) - a_i\big) = F_0 + F_1 - \big(nM - \sum_i a_i\big).$$

根上即可 $O(1)$ 返回雨水体积，不必查询最大点位置。整体平移不改变体积，负高度也成立；带外部高度的 scan 则明确使用调用者的边界。所有乘法、和、标记及中间运算须在 long long 内，不能只检查最终答案。附件原题为 NERC 2023 G，使用例属于比赛应用，不计正式模板题覆盖；小规模有符号穷举、三十万次随机更新和二十万点规模本地验证通过，线上待补。

70.50 动态 K-D Tree：矩形和与重建边界

`KDTreeSum` 按深度交替坐标轴，以另一轴打破平局。重复坐标累加权值，其他点沿轴全序递归插入；孩子过重时收集编号并用 `nth_element` 重建，复用原节点。包围盒从本点初始化，仅合并非空孩子，不用有限正负无穷；重建直接返回根编号，不用坐标 -1 作为状态标志。权值为零时仍保留节点，负权不影响几何剪枝。

每个孩子大小至多为父节点的 $3/4$，所以树高为 $O(\log(d+1))$；矩形部分相交却可能递归两边，查询最坏仍为 $O(d)$。按平均线性选择计，一次大小 s 的重建需 $O(s\log(s+1))$，插入摊还 $O(\log^2(d+1))$；不是每次操作的最坏对数界。矩形使用闭边界，只比较坐标、不做坐标减法；各点绝对权值和及更新中间量须在 long long 内。

P4148 的在线输入需逐参数 XOR 上次答案，驱动预留二十万个节点以避免扩容峰值。本地默认 int 坐标节点为 56 字节，另有重建编号和运行环境开销；该分配量不能代替线上 20 MB 内存通过记录。独立 map 对拍、分割与包围盒审计、坐标极值及二十万规模已本地验证，线上待补。

70.51 CDQ 卷积递推：贡献只跨越一次

`cdq_convolution` 求

$$f_i = b_i + \sum_{j<i} f_j g_{i-j}, \qquad g_0 = 0.$$

在半开区间 $[l, r)$ 中，先递归求完左半 $[l, m)$，再将这段 f 与 $g_{0\ldots r-l-1}$ 卷积，把乘积的第 $i-l$ 项加入右半的 f_i，最后递归右半。任意 $j < i$ 的依赖只有一个最小分治节点将二者分到左右两边，所以每份贡献恰好加入一次。顺序不能改成先求两个孩子再卷积。

根层完整卷积结果长度为 $n + \lfloor n/2 \rfloor - 1$，须不超过 NTT 的最大二次幂容量；单项输入不做变换，空输入返回空。一般初始项 b_i 均可使用，P4721 只取 $b_0 = 1$。时间 $O(n\log^2(n+1))$，额外空间 $O(n)$，递归深度 $O(\log n)$。附件标题为“分治 FFT”，实现实际依赖 NTT；这里保留 CDQ 算法，不以形式幂级数求逆替代。独立二次递推、多素数域、二十万项闭式参考及完整主程序已本地验证，线上待补。

70.52 多项式带余除法：先反转再求逆

`PolynomialDivision::divide(a,b)` 返回规范化的商、余式，零多项式用空向量。去掉输入高次零后，除数必须非零；低次被除式直接返回零商和原被除式。设 $A = \deg a$、$B = \deg b$、$k = A - B + 1$，反转后余式从 x^k 起，故

$$\operatorname{rev}(q) = \operatorname{rev}(a)\ \operatorname{rev}(b)^{-1} \pmod{x^k}.$$

除数最高项非零，反转后可求逆。取前 k 项再反转得到商，最后用 $r = a - qb$ 得余式。P4512 要求定长输出，使用程序负责补零；Library Checker 输出去高次零后的长度，零项对应空行。输入各长度不超过 2^{22} 是当前完整卷积容量的充分界，时间 $O(N\log(N+1))$、空间 $O(N)$。独立长除法、五十万项构造与官方本地 checker 已双模式验证；线上待补。

70.53 级数开根：前导零和自由尾项

`FpsSqrt::sqrt(a,n)` 仅使用前 n 项，不足补零。全零返回全零；首个非零项次数 k 为奇数，或其系数非二次剩余，则返回无解。否则设 $a = x^k h$，从较小的非零模平方根开始，迭代

$$b_{\text{new}} = \frac{b + h/b}{2}, \qquad b_{\text{new}}^2 - h = \frac{(b^2 - h)^2}{4b^2}.$$

误差最低次数翻倍，故求出 h 的根前 $n-k$ 项后左移 $k/2$ 即可。所得根末尾还有 $k/2$ 个不受平方同余约束的系数，本接口统一取零。因此一般截断问题不应说成“只有两个解”。返回 optional；$n = 0$ 返回有值的空向量。$n \le 2^{22}$；时间 $O(n \log(n+1))$ 加一次标量模开根，额外空间 $O(n)$。P5205 的 $a_0 = 1$ 返回首项 1，一般前导零与无解另由 Library Checker 使用程序展示。二次平方参考、最低次数边界、五十万项闭式及官方本地 checker 已双模式验证；线上待补。

70.54 ln 与 exp：只处理所需前缀

对数要求 $a_0 = 1$，由 $(\log a)' = a'/a$ 求导、求逆、积分得到；指数要求 $a_0 = 0$，从 $b = 1$ 开始使用 $b \leftarrow b(1 + a - \log b)$ 倍增。仅求前 n 项时，先把对数输入截到 n 项，避免输入的无关尾项造成变换容量超限。积分需 $1, \dots, |a|$ 都可逆；固定素数域中要求 $|a| < 998244353$。当前 $n \le 2^{22}$ 是变换容量充分界，log/exp 为 $O(n \log(n+1))$ 时间、$O(n)$ 空间。测试使用独立三角系数递推，并包含超过 NTT 容量的长输入只求一项、非二次幂截断与五十万项闭式。

70.55 幂函数的大指数：保留三份指数信息

`FpsPower::power(a,exponent,n)` 读入十进制数字串，计算模 x^n 的非负整数次幂。$n = 0$ 返回空；零次幂统一返回单位多项式，包含 0^0。若 $k > 0$，设 $a = x^s c h$、$h(0) = 1$、$c \neq 0$；全零或 $sk \ge n$ 时结果全零，否则

$$a^k = x^{sk} c^k \exp(k \log h) \pmod{x^n}.$$

同时保留 $k \bmod p$、$k \bmod (p-1)$ 和 $\min(k, n)$：第一份用于对数系数，第二份用于 c^k，第三份用于真实零次幂及位移。当前 $n \le 2^{22} < p$，故 $h(x)^p = h(x^p) \equiv 1 \pmod{x^n}$，单位首项部分只依赖 $k \bmod p$。指数余数为零并不表示原指数为零。判定 $sk \ge n$ 先用 $k > (n-1)/s$ 比较，通过后才乘，避免指数乘法溢出。指数位数 L，时间 $O(L + n \log(n+1))$、额外空间 $O(n)$。P5245 的常数项为 1，P5273 与 Library Checker 允许一般首项；共用主程序必须将指数读作字符串。独立截断乘法、十万位指数、五十万项二项式及官方本地 checker 已双模式验证；线上待补。

70.56 集合变换：逆变换对模数的要求

OR 变换是子集和，单层为 $(u, v) \mapsto (u, u+v)$；AND 是超集和，单层为 $(u+v, v)$。它们只需加减，模数可以为偶数或合数。XOR 变换使用字符 $(-1)^{\text{popcount}(s \& i)}$；单层 $(u+v, u-v)$ 连做两次等于乘 2，故当前接口要求奇数模数，以 $(p+1)/2$ 为逆 2，不依赖费马小定理。长度为正的二次幂，长度 1 可用；三种卷积不能与不相交子集卷积混淆。独立字符定义式、下标对枚举、合数模数及最大规模完整驱动已双模式核验。

70.57 浮点 FFT：舍入前先约束最坏误差

复数变换采用正指数，逆变换使用共轭根并除以变换长度。长度为一时没有蝶形操作；位逆序从一开始，避免负移位。单位根按各自角度直接计算，避免长链旋转因子连乘。本节要求二进制至少 53 位有效精度、最近舍入、禁用 fast-math，且单位根复数绝对误差不超过 $8u$，其中 $u = 2^{-53}$。三角函数精度是平台前提，不由 C++ 类型名称保证；本机已逐个核验最大网格的全部半圆根。

设变换长度 $N=2^{\ell}$，两输入欧氏范数乘积为 S。Percival 定理 5.1 对三次 radix-2 FFT 卷积给出误差界

$$S\left((1+u)^{3\ell}(1+\sqrt{5}u)^{3\ell+1}(1+8u)^{3\ell}-1\right).$$

整数入口限制 $N\le 2^{22}$、$\max|a_i|\max|b_i|\sqrt{|a||b|}\le 2\cdot 10^{12}$，从而每项误差小于 0.166；使用 llroundl 同时处理正负系数。超过此限制不能只因浮点结果接近整数就宣称正确。

任意模卷积先把有符号输入归一化，再拆成三个 11 位块。结果长度最多 2^{19}。每路 $S\le 2047^2(2^{18}+1)$，误差小于 0.079。六次正变换复用，九次逆变换分别舍入后用整数模合并。此处以较大的常数换取明确的 53 位精度边界，不假定 long double 更宽。模数可为一或合数；空输入返回空，常数乘法合法。百万次数整数卷积、完整 int 输入、最大长度拆系数卷积已用闭式与精确整数参考核验；本地验证不等于在线 AC。

70.58 SPFA 费用流与残量方案

SPFA 备选每次在残量图上直接求最短路，保留负边、限流和继续增广，但不允许负环。每次调用先将全部顶点以零距离入队检查，包括源点不可达的负环；因为不连通的负环同样能降低总费用，不能忽略。检查时不改流量，发现负环抛异常。记录严格改进路径的边数，达到顶点数即拒绝；不要用松弛次数冒充路径边数。零费用环不发生严格改进，不会反复改写前驱。

沿最短残量路增广保持当前流量下的最小费用性。反边可以撤销已经送过的流，最终方案应通过原边编号查询 used，不能累加历史路径当作最终流量。flow 返回本次新增流量与费用，重复调用须同源汇；传零限流仍检查负环。最小费用流允许负边与允许负环是两种不同的契约，后者应使用专门算法。

容量为 long long，距离及费用为 int128；费用不能为 LLONG_MIN，因反边须取相反数。另要求总流不超过 long long，且 $\sum cap\,|cost|<2^{120}$。P3381 另保证答案不超过 $2^{31}-1$，其示例才可转换为 long long 输出。SPFA 版实际增广 A 次的复杂度为 $O((A+1)VE)$，不能假设 $A=O(V)$；通常优先势能 Dijkstra，本版保留为直接的备选写法。

两版通过 4000 组独立流量枚举及边界测试，完整驱动通过最大规模本地检查。验证细节见仓库 docs/FLOW-ATTACHMENT.md；本地通过不等于线上 AC 或速度排名。

70.59 2-SAT 编号方向与双连通定义

TwoSAT 的 add(x,a,y,b) 表示 $(x=a)\vee(y=b)$。强制值写 add(x,a,x,a)，蕴含 $(x=a)\Rightarrow(y=b)$ 写 add(x,!a,y,b)。本库使用 Kosaraju 拓扑编号，真文字所在分量编号更大时取真；附件 Tarjan 的分量编号为逆拓扑序，比较方向相反。不能更换 SCC 实现而不检查赋值的不等号。solve 可在追加子句后重算，仅成功时读取 ans；一般复杂度为 $O(n+m)$，不保证字典序最小。

点双采用连通且无割点的定义：桥两端也成块，孤立点单独成块，割点允许属于多个块。自环不改变顶点间连通性，不额外产生点双；原边号仍保留，且不标桥。每条逻辑无向边只调用一次 add，不能把两个方向分别当作两条边加入。共享 lowlink 只跳过父边编号，不能跳过所有指向父顶点的重边，否则会误判桥。附件点双单独跳过父顶点虽不改变点双阈值的真假，但它的 low 不可直接复用来判桥。

BiconnectedCore 的点号为 $1\ldots n$、边号从零开始。run 后 blocks 是点双顶点集，bridge 是原边桥标记，bel 是 $1\ldots cnt$ 的边双号；边双编号不承诺拓扑方向。不连通输入的圆方图和桥缩点结果均为森林。st 仅保存未归块顶点，不是模拟 DFS 的执行栈；每次 run 重新计算所有结果。

独立子集、删点、删边、真值表与最大规模驱动检查均有普通及 ASan/UBSan 记录。LC 点双 22 组、边双 21 组官方数据双模式本地 checker 接受，不等于线上 AC。百万变量矛盾链在 ASan 下需更大栈，本地以 1 GiB 线程栈运行原递归算法；提交仍用普通 main。细节见仓库 docs/GRAPH-ATTACHMENT.md。当前共享核心的边双输出已由 CCF_NOI 提交 QOJ999 获 AC（记录 3061643，87ms，42352kb）；本次在线结果不核验 blocks、cut、森林或补边方案。

SCC 的完整拓扑序输出见第 806 页，注意其编号方向与 TarjanSCC 相反。固定 LC scc 的 12 组官方数据由本地 checker 接受；枚举三点有向图、独立传递闭包以及 50 万点长链、环、分离二元环核对分组和跨组顺序。同一 Kosaraju 驱动已由 CCF_NOI 提交 QOJ906 获 AC（记录 3061625，226ms，76512kb）；dag 接口及其他未调用的捆绑算法不在本次覆盖范围，速度排名尚未核验。

已有点双分解时，removal_components 返回原图块数 before 和独立删点后的总块数 after。删除孤立点会使总数减一；非单点点双每包含该顶点一次，给删除后的所在连通分量多贡献一块。AOJ 割点用法见第 821 页，只判 after[u]>before；UVA10765 完整数量及排名见第 822 页，以负块数和原点号排序，别把输入的输出名额 m 当作边数，保留每组输出后的空行。独立逐点删除图并重新求连通块，检查数量及同分顺序。

连通无向图能强连通定向当且仅当无桥。必要性：桥不论朝哪边定向，都不能双向通过；充分性：DFS 树边向下，非树边向祖先，每个非根子树都有返回严格祖先的边，逐层可回到根，根又能沿树边到达所有顶点。第 820 页先判桥再定向，每条逻辑边仅 add 一次，输出保留原边编号顺序。完整驱动枚举至五点的 771 个连通简单图，以逐边删除判可行性，再检查每条输出边及正反可达性；10 万点、30 万边边界和重边、自环的额外契约测试均通过。上述测试均有普通及 ASan/UBSan 记录，算法保留递归，不是线上 AC 或速度排名。

圆方森林保留全部原点 $1\dots n$，第 i 个点双对应方点 $n+i+1$（i 从零开始）。简单路径三元组应用见第 833 页：原点权为 -1，方点权为块的大小。对于同一连通分量中的不同原点 s,f，圆方树路径经过的各块，恰好给出可能作为中间点的顶点并集。同一块内任意第三点可放在两个指定端点间的简单路径上；块间只通过割点连接。方点权先计块大小，路径内部割点的 -1 去重，两端的 -1 排除 s,f，故路径权和就是可选中间点数，而不是简单路径条数。

设所在树有 C 个原点，删去节点 u 后各支的原点数为 a_i。有序原点端点对经过 u 的次数为 $2\sum_{i<j}a_ia_j+2(C-1)[u\le n]$；后项计端点等于 u 的情形。sz 仅统计原点，父侧大小为 $C-sz[u]$，遍历子树时用前缀和累计两两乘积。乘以节点权后求和，使用 long long。每棵树分别求 C，孤立点贡献零；总时间空间 $O(n+m)$。

桥森林邻接项是（边双号，原桥号），不是原点端点；原点所属边双读 bel。第 812 页直接计度一叶子，第 824 页返回原点补边方案。连通且允许新增重边时，每个叶子至少需一个新增边端点，故至少 $\lceil L/2\rceil$ 条边。DFS 顺序收集叶子，奇数时补首叶，再前后半段配对；每条桥两侧的叶子区间都有配对跨过，故达到下界。仅一个边双时不加边。非连通森林不能直接套此公式或调用构造接口。P2860 只检查条数，方案端点及桥编号另以本地逐边删除和多重集检验；小图额外枚举全部新增边多重集验证最优性。P4630、P2860 都是比赛应用，不代替正式模板题覆盖或线上 AC。

70.60 回文半径与在线回文计数

Manacher 的 odd[i] 包含中心，最长奇回文为 $[i-odd[i]+1,i+odd[i]-1]$，长度 $2odd[i]-1$；even[i] 以 i 前间隙为中心，最长偶回文为 $[i-even[i],i+even[i]-1]$，长度 $2even[i]$。所有 odd/even 之和是回文的出现总次数，应以 long long 累加；这不是不同回文串数量。恢复最长子串时记录对应起点和长度即可。

附件插入分隔符后的半径有包含中心与不含中心两种约定，不能直接套到本库数组。哨兵版本还依赖字母表：小写字符串的写法不能原样推广到含标记或零字节的输入。本库显式检查边界，空串和任意字节均合法；按字节而非 Unicode 字素比较，int 索引要求 $n<2^{30}$。P3805 的 1100 万字符相同、交替和无长度至少二回文的三种闭式输入已通过双模式本地检查。

PAM 的 suffixes 是当前回文节点的非空回文后缀数量，等于 fail 节点的 suffixes 加一。P5496 必须先按上一个答案解码当前字符，再 add，输出 t[last].suffixes。hits 仅统计作为最长后缀的命中次数；完整次数应取 occurrences() 的返回值。该函数复制后再沿 fail 汇总，不改 hits，因此重复查询及查询后继续追加均安全。total 为逐位置 suffixes 的总和，最多 $n(n+1)/2$；distinct 才是不同非空回文串数。PAM 固定小写字母，内部前置 -1 与 $0\dots25$ 编码不同，不继承 Manacher 的任意字节契约。

独立逐子串逆序比较、PAM 全短串及两题最大规模驱动均在普通和 ASan/UBSan 下通过。Manacher 的 LC 全部 24 组官方数据双模式本地 checker 接受，不等于线上 AC。详细接口与记录见仓库 docs/PALINDROME-ATTACHMENT.md。

70.61 整数后缀数组的边界与 SAM 次数

SuffixArray 的整数入口要求 $0 \le a[i] < alphabet$；稀疏大整数先保序离散化。sa/rk/lcp 都从零编号，lcp[k] 比较 sa[k-1] 与 sa[k]，首项为零。附件第二份倍增模板额外要求 LCP 遇分隔符停止，不能把普通 LCP 不加处理地代入。令 remain[i] 为从 i 到下一个分隔符之前的普通字符数，则分段前缀长为

$$height[k] = \min\{lcp[k], remain[sa[k-1]], remain[sa[k]]\}.$$

分隔符位置 remain 为零，连续或重复分隔符同样适用；保留原 lcp 可避免两种语义混用。两个字符串拼接后，相同前缀的后缀在 SA 中连续，所以找最长公共子串只需检查异来源的相邻后缀。输出时转回各自原串的半开区间，不把分隔符当原字符。

SAM 状态 u 代表长度 $(len[link[u]], len[u]]$ 内具有同一 endpos 集合的子串。真实扩展状态的 occ 种子为一，克隆为零；附件存放插入 id 的 endpos 字段不等于完整出现次数。counts 按长度倒序传播，不能按创建编号倒序，因为克隆可能后创建。查询返回副本，不改种子，所以重复查询后继续 extend 安全。根计数为串长 n，不能解释成空串的 $n+1$ 个出现位置；非空子串只看非根状态。P3804 仅在 count[u]>1 时取 count[u]*len[u]，乘积用 long long，无重复子串则为零。若需要每个前缀对应的状态，调用者在每次 extend 后保存 last 即可。

独立后缀排序、逐子串 endpos 集合、分隔符截断与完整百万规模驱动双模式通过。LC 后缀数组 50 组、最长公共子串 28 组官方数据双模式本地 checker 接受，不等于线上 AC 或速度排名。细节见仓库 docs/SUFFIX-ATTACHMENT.md。

70.62 广义 SAM 点数与后缀语言

GeneralSAM 的状态按文档编号与结束位置组成的 endpos 集合等价。先 add 全部小写串，再 build；根为零，distinct 返回不同非空子串总数，a.size() 包含根。构造后不再 add，重复 build 安全；重建另开对象。重复串和空串在库内合法，P6139 则要求输入非空串。

多串 endpos 类不能直接称为后缀并集的最小 DFA。例如 ab、b 的三个非空子串 a、b、ab 具有不同 endpos 集合，含根共四点；仅识别后缀并集时，读完 b 和 ab 的状态可合并，含初态只需三点（省略拒绝汇点）。P6139 官方样例 aa、ab、bac、caa 要求不同子串数与点数均为十，符合 endpos 口径；把后缀 Trie 按右语言最小化则为七点，不能据题面中的“最小”二字输出七。完整用法直接输出 a.size()，不额外做 DFA 合并。

独立枚举文档位置集合与两种右语言计算核对上述区别。完整驱动的 600 组随机参考及排列、百万字符链和克隆密集串、两条各五十万字符的交替串、四十万文档合计百万字符，以及深十五完整二叉 Trie 均通过普通和 ASan/UBSan。这些是本地证据，尚未新增 P6139 线上 AC 或排名。构造与口径详见仓库 docs/GENERAL-SAM-NOTES.md。

70.63 排列约束与最大流单位边分类

NERC 2024 K（QOJ 10424）给两套排列与同一大小为 k 的集合所形成的部分子序列。已知元素切开未知段：段内候选来自原排列的对应区间，需求是未知位置数；已知元素单独成需求一的组。检查锚点严格递增与区间长度足够；首尾设虚拟锚点。保留零需求组，使每个元素在两侧都恰好属于一组。

源连左组、右组连汇，容量为需求；每个元素直接对应左右组间的一条单位边。两侧需求总和均为 k，所以流量达到 k 等价于所有组恰好选够，等价于原题存在合法集合。这替代附件的元素拆点路径，减少节点且保留逐元素对应关系。仅区间检查不足以保证两侧同时可行。

保存 add 返回的原边号，满流后调用 unit_flow_edges。返回的 forced 为真输出 Y，否则 possible 为真输出问号，否则输出 N。原始网络分层无环，流恰好对应所选集合；残量 SCC 用于判断能否沿环改变某条单位边，不是最小割边分类。网络规模 O(n)，整数流值至多 n，因此本题增广求流 $O(n^2)$，之后一次 SCC 为 O(n)。

完整驱动通过 8166 个小输入穷举、2000 个独立子集枚举参考与最大 n=2000 边界检查。另按官方资料包 117 个配方本地重建输入，经官方 validator、两份 accepted 参考实现及普通/ASan/UBSan 下的官方 checker 核对；生成器无用越界迭代的修订有单独记录。这是比赛应用与本地证据，不增加正式模板题覆盖，也不表示线上 AC。细节见仓库 docs/KNOWNS-UNKNOWNS.md。

70.64 点分治、启发式合并与虚树用法

P3806 在建完树后只调用一次 build，询问 count_exact 是否非零。计数对象是无序不同顶点对，零权路径允许产生零距离，但不能把点与自身配对。预处理 $O(n\log^2 n)$，每次查询 $O(n\log n)$；边修改后必须重建。

CF600E 输出子树所有最高频颜色编号之和，读 answer 而非 max_freq。并列颜色全部相加，十万种不同颜色的和超过 32 位；颜色压缩后仍须累加原编号。轻子树先算后清空，重子树保留；清空整份临时统计后再重置最大频率与颜色和。该程序为比赛应用，非比赛模板题仍待补。

P2495 每次显式加入根 1，再构造虚树。虚边和 vertices 都是原树编号，只清零本次 vertices 的 dp；逆序遍历虚边，孩子为关键点时取路径最小边权，否则取路径最小边权与孩子 dp 的较小值。使用 minimum，不能用路径总长 distance。只清除本次 keys 的关键标记；祖先与后代同时关键也允许。题目来自省选，本册将其作为应用用法保留；最大答案可达 24999900000，须用 long long。

三份完整驱动分别通过路径枚举、祖先直方图、删边子集枚举；最大规模长链仍用递归 DFS，本地通过扩大测试进程栈匹配竞赛环境。本地证据与历史线上 AC 快照分开保存。

70.65 压位小波矩阵与区间顺序统计

`WaveletMatrix` 先将值离散化，再从高位到低位稳定地把当前序列的零位元素放前面、一位元素放后面。同一路径下的原区间始终对应当前层的连续区间。记左右端点的一位前缀数为 a,b，则区间内零位数为 $z=r-l-b+a$。第 k 小若 $k<z$，下一层区间为 $[l-a,r-b)$；否则减去 z，进入 $[mid+a,mid+b)$，并设置答案的当前位。每层按 64 位存储，配一份整字前缀计数；末尾多留一个字，保证 $r=n$ 且 n 为 64 的倍数时仍可查询。掩码移位量用位置模 64，不会移位 64。

`kth(l,r,k)` 的区间和排名都为 0-based；P3834 的闭区间调用应为 `kth(l-1,r,k-1)`。第 k 大（k 从零开始）为 `kth(l,r,r-l-1-k)`；两者都要求 $0\le k<r-l$。`less(l,r,x)` 严格小于 x；`freq(l,r,x)` 直接沿 x 所在路径计数，不能用 $x+1$ 来规避等值查询，否则在最大有符号整数处溢出。空数组、空区间可计数，但不能求第 k 小；单一值域的层数为零，直接返回唯一值或区间长度。核心支持完整有符号 64 位值，三份正式题驱动按各题实际非负值范围验证。

该结构适合静态查询，不支持修改或历史版本。与持久化线段树的功能重叠不意味着来源中的每份划分树实现都已完成审计。WIDA 条目的压位查询功能已落实；kuangbin 划分树已另行实现为 DivisionTree，逐层保存值和前缀，空间为 $O(n\log(n+1))$。重复中位数按左半容量分配，详见 `docs/DIVISION-TREE.md`。独立子区间排序、直接计数、字边界和最大规模验证，以及两个 Library Checker 题目的官方本地 checker 报告见 `docs/WAVELET-MATRIX.md`。这些记录不是线上 AC，也不是最快提交榜排名。

70.66 模 2 压位消元与完整解空间

`GaussXor::solve(a,n)` 的输入为 m 行 $n+1$ 列的 01 字符串，末列是右端项。每一主元列消去其余所有行；主元列成为单位列后，把自由变量全设零即可读出特解。每次只把一个自由变量设一，主元变量取对应自由列的系数，得到一个零空间基。这些基在自由列上的投影构成单位阵，所以独立且覆盖全部齐次解。全部解为特解异或任意基子集，有解时解空间维数为 $n-rank$；LC 的输出第一行就是该维数，不能输出系数秩。

每行按 64 位打包。异或从当前列所在字开始：新主元行在先前列均为零，所以同字中较早的位也不会被改坏。增广位编号为 n，始终分配 $\lfloor n/64\rfloor+1$ 个字，n 为 64 倍数时右端项在独立尾字。空方程组有 n 个自由变量；零变量系统若有任何右端项为一则无解。无解和零变量的有解情况都可能返回空特解，必须查看 `consistent`。

消元是精确模 2 运算，没有浮点误差；输出完整基最坏需要 n^2 个字符。特解不保证一的个数最少，开关最优化需要另外枚举自由基组合或使用题目结构。kuangbin 2.10.1 的 POJ1681 开关应用另以独立首行追赶和物理按键子集完成本地验证，见后文最少开关用法。独立全解集枚举和 4096 阶官方本地 checker 证据见 `docs/GAUSS-XOR.md`，不等于线上 AC。

70.67 严格次小生成树与换边

SecondMST 用 Kruskal 选定一棵 MST，再用递归 DFS 和倍增维护树路径上两个不同的最大边权及其原边编号。每条非树非自环边换入后，非严格模式删路径最大边；若要求严格增大且换入边恰等于路径最大值，应删路径第二大的不同权边。没有这样的第二值则跳过，而不是把另一棵同权 MST 当作严格次小。路径权为 0, 1、换入权为 1 就是两种定义不同的最小反例。

返回的 tree 是一份 MST 边号集合；从中删除 out、加入 in 得到 next 的方案。同端点同权重边也使用不同编号，自环永远不进入树。无次小树时 next 为空；断图时 connected=false，不能把默认总权零当成答案。所有总权和差值用 int128，边号 -1 表示缺项，避免以最小有符号权当哨兵。普通 dsu 保持路径减半和 y 根挂 x 根，DFS 不改为显式栈。

交换配对说明单边换入即可覆盖次小答案：将另一棵树与 MST 的不同边作合法交换配对，MST 最优性保证各配对差值非负。若总权严格增大，其中至少有一个正差不超过总差；枚举所有环上的最优严格换边就能达到全局最小正差。P4180 为 BJWC 应用题，保证严格次小存在；其权值范围可安全转回 long long，但一般接口不可如此假设。6368 份生成树全子集对照、原边方案检查、十万点链和三十万边完整驱动双模式证据见 docs/SECOND-MST.md。

70.68 模 2 消元后的最少开关用法

POJ1681 的源体将每个方格作为一个变量，行号为目标格，系数为自身及四邻的按键，原白格右端项为一、黄格为零。解集是特解异或零空间基的任意子集，须枚举后取一的最少个数，不能只数任意特解的一。递归枚举某个基时先不选、再异或选择，返回后再次异或恢复；无解输出 inf。

令自由维数为 d，枚举代价为 $O(n^2 2^d)$，递归深度 d。本方格问题第一行按法一旦确定，后续行由消除上一行的白格唯一确定，所以 $d \le n$。该界只针对这个四邻棋盘，不能套用到任意图或任意异或方程组。622 份棋盘采用独立首行追赶参考，其中三阶以内全部棋盘再与物理按键子集穷举核对；验证到十五阶、无解与多测重置。原 POJ 题面当前不可达，依据固定 kuangbin 源体重建应用；本地通过不冒充原站 AC。详见 docs/SWITCH-MINIMUM.md。

70.69 闭合集、树形图与全局割的完整用法

P2762 把实验设为正权点、仪器设为负权点，实验到仪器连依赖边。依赖数量不固定，先用 getline 读一整行，再用 istringstream 分解；这样兼容题面的 CRLF，也不会把下一实验的赞助误读成仪器编号。maximum_closure 返回的点号中，实验为 $1..m$，仪器为 $m+1..m+n$；第二类输出时减去 m。空方案仍输出两个空行，再输出零收益。共享仪器只计费一次，赞助总额用 long long。输出方案可不唯一，不能只比较固定编号。

P4716 的点号和根都减一，调用 Arborescence::solve。有值时才解引用；无解输出 -1。每个非根点有入边不代表根能到达所有点，必须处理入边环。该接口只返回费用，不恢复边方案；核心用 optional 区分无解与合法的负费用 -1。P5632 先累加对称矩阵，再调用 StoerWagner::solve；side 是一侧的原顶点编号，仍为 0-based。核心要求至少两个点、非负权。单点不存在非平凡割，驱动另行约定输出零。两题的费用能转 long long 是题面范围保证，不能据此缩小核心的 128 位费用类型。

660 份实验计划、355 份最小树形图和 891 份全局割输入，分别以子集、入边组合、非平凡割枚举核验，另测最大规模及打印程序；普通与 ASan/UBSan 均通过。完整契约与证据见 docs/CUT-APPLICATIONS.md。P2762 为应用，P4716/P5632 为正式模板题；本地验证不等于线上 AC。

第七十一章　来源与覆盖维护

71.1　参考基线

- kuangbin, ACM 常用算法模板, 2018 年版本：https://kuangbin.github.io/2018/08/01/ACM-template/。
- WIDA XCPC：https://github.com/hh2048/XCPC，参考快照 484e9e0。
- OI Wiki 数学：https://oi-wiki.org/math/。

本项目重新实现的代码与公式速查不能冒用原作者的验证记录。完整覆盖审计应为每个来源条目记录实现文件、替代关系与验证证据，保留遗漏项直至完成。

71.2　后续覆盖项

尚需完成的范围以覆盖表为准，包括动态树、平衡树、一般图匹配、最小树形图、上下界流、高阶几何构造、半平面交、三维凸包、数值积分、复杂数论及多项式、搜索与动态规划等。不能用通用模块的存在推断其覆盖了所有原书例题。

算法索引

ac_shortest, 51
ACWeighted, 53
AdaptiveSimpson, 353
AffineSegTree, 428
AffineSequenceTreap, 484
AhoCorasick, 45
Arborescence, 785
AssignmentSpectrum, 934

ballot theorem, 286
Barrett32, 144
basis_intersection, 332
basis_sum_intersection, 334
batch_inverse, 165
batch_units, 163
BellmanFord, 750
Berlekamp–Massey, 339
Berlekamp-Massey, 1156
berlekamp_massey, 367
Bezout identity, 159
BiconnectedCore, 807
BigInteger, 115
Binomial, 276
BipartiteMatching, 922
block_cut_forest, 832
Blossom, 940
BostanMori, 268
BoundedCirculation, 909
BoundedMaxFlow, 911
bridge_augmentation, 823
bridge_component_forest, 811
Burnside, 307
BurnsideAverage, 301

Cactus, 827
Carmichael, 1155
carmichael, 162
CartesianTree, 538
Catalan, 286, 1156
Cayley formula, 954
Cayley–Hamilton, 339
cc_hash_table, 370
cdq_convolution, 229
CentroidDiameter, 704

CentroidNearest, 700
CentroidPairs, 687
CentroidSum, 695
Chain3D, 1119
chirp_z, 246
Cipolla, 1154
circle_coverage_area, 1026
circle_intersections, 996
circle_intersections_i64, 1004
circle_overlap_area, 999
circle_union_area, 1022
CirclePolygon, 1009
CircleTangents, 1011
closest_pair_i64, 1093
ClosestPair, 1096
ColorComponents, 675
ColorMax, 671
ComplexFFT, 221
CompositeBinomial, 181
CompositeRoots, 200
conditional expectation, 346
convex_contains_i64, 1074
convex_cut, 1030
convex_diameter2, 1083
convex_tangents_i64, 1067
convolution_fft, 225
convolution_i64, 234
convolution_mod, 236
convolution_mod_fft, 227
coprime pairs, 1152
CoprimePairs, 205
count_four_cycles, 834
CRT, 146
crt_merge, 169
cut_tree_values, 893

dag_path_determinant, 975
DagDominator, 981
DagLongest, 713
DC3, 69
DenseDijkstra, 739
det_prime, 319
determinant_exact, 324
determinant_mod, 322
digit_count, 15
Dijkstra, 722
Dinic, 871
DirectedEuler, 853

Dirichlet convolution, 206
DiscreteLog, 194
DivisionTree, 463
divisor sum, 1152
divisor_sum_power, 127
Dominance3D, 1117
Dominance4D, 1112
DominatorTree, 977
dsu, 373
Du Jiao sieve, 1150
DuJiao, 208
DynamicAC, 55
DynamicDiameter, 633
DynamicKth, 579
DynamicPathMax, 624
DynamicPeachTree, 613

EdgeCompression, 816
EGF, 277
EightPuzzle, 2
EnclosingCircle, 1098
enumerate_triangles, 836
Euler characteristic, 1065
Euler theorem, 146
Euler totient, 1151
euler_phi, 150
euler_power, 161
EulerLCA, 644
ExactCover, 5
exCRT, 1153
exkmp, 44
ExLucas, 178
exLucas, 1156
extended_gcd, 156

Fenwick, 412
Fenwick2D, 407
FibonacciPeriod, 281
floor moments, 220
floor quotient grouping, 203
floor sum, 219
floor_moments, 217
floor_sum, 215
flow_unique, 885
Floyd, 726
FpsFunctions, 261
FpsInverse, 248
FpsPower, 264

FpsSqrt, 257
FunctionalGraph, 799

Garner, 1154
garner, 171
GaussMod, 361
GaussReal, 364
GaussXor, 357
GCD 加权求和, 206
GcdSequenceTreap, 489
GeneralSAM, 84
generating functions, 1156
geometric series, 1152
gomory_hu, 891
gp_hash_table, 370
gp_map, 370
grid_cycle, 17
grid_cycles, 21

HLD, 659
HLPP, 865
Hull3D, 1038
Hull3DMetrics, 1048
Hungarian, 929

inclusion-exclusion, 1156
independent_set, 927
indicator variable, 346
integer partitions, 299
integer_hull, 1071
IntegerGeometry3D, 1036
IntegerHalfplanes, 1089
IntegerPlane, 1000
IntegerTangents, 1013
inverse_table, 167
Isap, 881

Johnson, 719

KDMin, 599
KDNearest, 588
KDRange, 593
KDTreeSum, 583
kmp_match, 39
Korselt criterion, 1155
Kruskal, 780
KruskalTree, 763
KShortestWalks, 760
KthResidue, 195
Kummer theorem, 175

Lagrange, 350
Lagrange interpolation, 1156
Lagrange inversion, 274
lattice polygon, 1065
lazy_segtree, 417
LazySeg, 427
LCT
    虚子树, 1175
LeftistHeap, 390
Legendre formula, 175
LexTwoSAT, 850
LiChao, 454
lifting the exponent, 173
LiftingLCA, 647
Line3, 1056
line_circle_i64, 1002
line_circle_intersections, 994
line_intersection_real, 992
line_projection, 991
linear congruence, 1153
linear sieve, 1151
linear_congruence, 168
linear_equation, 157
LinearSieve, 151
LinkCutTree, 604
LongChain, 654
Lowlink, 824
LTE, 173
Lucas, 177, 1155
Lucas theorem, 146
lyndon, 107

manacher, 99
Manhattan MST, 784
manhattan_order, 1132
ManhattanMST, 768
matching_edges, 926
Matrix Tree theorem, 1157
matrix_inverse, 312
matrix_inverse_mod2, 315
MatrixTree, 961
MatrixTreeExact, 970
MatrixTreeMod, 967
maximum_closure, 889
MaxPlusMatrix, 317
MergeSplay, 504
MergeSplitTree, 437
mex, 348

Miller–Rabin, 1150
Min25, 212
MinCostFlow, 894
mincut_edges, 887
MinCycle, 842
minimum spanning tree, 784
minimum_mean_cycle, 838
minimum_rectangle, 1086
minimum_rotation, 105
MinimumCover, 9
minkowski_sum, 1075
mint, 129
mixed_euler_orientation, 861
mixed_euler_trail, 864
Mo, 1128
Mobius inversion, 206
Mod64, 124
mod_inverse, 164
mod_sqrt, 189
ModifiedMo, 1135
ModInt, 133
ModMatrix, 310
monotone_dp_layer, 27
MonotoneHull, 25
MonotoneStackSeg, 432
Montgomery, 140
multiplicative order, 1155
MultipointEvaluation, 252

necklace_colorings, 305
NegativeCostFlow, 902
NttConvolution, 231
numerical stability, 1157

odd_cycle_vertices, 813
odd_induced_partition, 630
offline_rectangle_sum, 414
OfflineLCA, 641
OGF, 277
OnlineSAM, 91
order_match, 41
OrderedSplay, 498
OrderedTreap, 475
orient_edges, 819
ost, 473
OverallKth, 1124

p-adic valuation, 173
PalindromicTree, 102

Partitions, 297
path_intersection, 652
pentagonal number theorem, 299
permutation_cycles, 304
PersistentArray, 542
PersistentDistinct, 555
PersistentDSU, 567
PersistentKth, 540
PersistentOrderedTreap, 570
PersistentParityIndex, 562
PersistentRange, 544
PersistentWeightedIndex, 558
PersistentXorTrie, 576
pheap, 386
Pick theorem, 1065
Plane3, 1059
Pollard–Rho, 1150
PollardRho, 154
Polya, 307
polygon_area2, 1005
polygon_centroid, 1032
polygon_contains, 1008
polygon_union_area, 1016
polynomial_interpolation, 255
polynomial_shift, 270
PolynomialDivision, 249
PositionBasis, 328
PotentialDSU, 377
power_sum, 125
prefix_function, 37
prefix_lcs, 67
Prim, 776
Prime64, 149
prime_count, 210
PrimePowerRoots, 196
PrimitiveRoot, 190
Prüfer sequence, 954

q_binomial, 185
quadratic residue, 1154

rank, 1157
Rational, 110
RealPlane, 987
RealPolarLess, 989
RealSpace, 1053
rectangle_union_area, 1081
RectangleFenwick, 410

recurrence_nth, 266
reflection principle, 286
regular_matching, 918
release_bfs, 724
removal_components, 820
Richardson extrapolation, 356
RollbackDSU, 375
RollbackMo, 1142
root_factors, 199
rp, 471

SAIS, 57
sam_lcs, 82
SAMDocuments, 94
SAMLex, 79
ScapegoatTree, 523
SCC, 804
SecondMST, 771
segment_distance_real, 1063
SegmentBeats, 457
segmented sieve, 1151
segmented_primes, 152
SegmentLiChao, 448
segtree, 422
SequenceKth, 527
SequenceMaxTreap, 494
SequenceScapegoat, 532
SequenceSplay, 516
SequenceTreap, 479
SetConvolution, 242
ShiftSplay, 512
ShortestPathTree, 733
Simplex, 341
Simpson integration, 356
Spfa, 743
SpfaFlow, 898
Sprague-Grundy, 348
square_counts, 67
StaticMod64, 136
StaticRMQ, 399
StaticRMQ2D, 403
SteinerTree, 788
Stirling numbers, 288, 1156
stirling_first_column, 291
stirling_first_row, 289
stirling_second_column, 294
stirling_second_row, 293
StoerWagner, 915

StrictSecondShortest, 757
StringHash, 33
subset_convolution, 244
SubtreeColors, 680
SuffixArray, 61
SuffixAutomaton, 74
SuffixLCP, 64
SupportHull, 1077

tail sum, 346
TarjanSCC, 794
TimeConnectivity, 380
Treap
    位翻转, 1176
tree_probability, 964
TreeBlockDistinct, 392
TreeDiameter, 638
TreeIsomorphism, 710
TreeMarket, 691
TreeMo, 1138
TreePathKth, 552
TreePathProducts, 609
triangle_incircle, 1034
tutte, 956
TwoSAT, 847

UndirectedEuler, 858
unit_flow_edges, 877

VirtualTree, 683

WaveletMatrix, 466
WeightedBlossom, 943
WeightedMatching, 931
Widget Factory, 1157
word_chain, 856
wqs_independent_set, 30

xor_hamming_pairs, 1146
XorBasis, 326
XorPathSum, 618
XorWalk, 336

z_function, 43
ZkwFlow, 905

上升阶乘, 288
下降阶乘, 288
二元一次不定方程, 159
二项式反演, 279

互质数对, 207

动态树 DP, 1175
区间筛素数, 1151
升幂引理, 173
原根, 1155
复合逆, 274
容斥原理, 279
带修莫队, 1174
开放时间最短路, 1174
形式留数, 274

恰好计数, 279
扩展中国剩余定理, 1153
批量逆元, 1152
排列数, 1156
斯特林反演, 296
斯特林数, 288
无序数对, 207
替罪羊树, 1176
有限状态计数, 339
树上路径交, 1170
模高斯消元, 1157
混合进制, 1154
满射, 279

矩形 GCD 计数, 1152
筛法选型, 203
类欧几里德, 219
线性同余方程, 1153
线性逆元表, 1153
线性递推, 339, 1157
群作用, 307

计数建模, 277
贪吃蛇, 1174

阶乘表, 1156
隐式生成函数, 274